Programare cu premeditare

Cătălin Frâncu

Prefață

Aici voi spune ceva introductiv.

- Problemele sînt ordonate după dificultate.
- Pentru Codeforces și Kilonova aveți nevoie de un cont pentru a vedea sursele.
- Am inclus legături către versiunile online ale surselor acolo unde evaluarea este publică. Rapoartele de evaluare arată și timpul și memoria.

Cuprins

I Principii de programare **31**

1 Cod curat și cod spaghetti **33**

1.1 Exemplul 1 . . . 33
1.2 Exemplul 2 . . . 36
1.3 Exemplul 3 . . . 37
1.4 Exemplul 4 . . . 39
1.5 Cîteva principii pentru cod curat . . . 40
1.5.1 Formatare . . . 41
1.5.2 Limbajul de programare . . . 41
1.5.3 Funcții . . . 42
1.5.4 Programe . . . 43
1.5.5 Comentarii . . . 44
1.6 Concluzii . . . 45

2 Bune practici în programare **47**

2.1 Invarianți de buclă . . . 47
2.1.1 Căutarea maximului într-un vector . . . 48
2.1.2 Interclasarea a doi vectori ordonați . . . 48
2.1.3 Alte exemple . . . 49
2.1.4 Căutarea binară . . . 49
2.2 Programare prin contract . . . 51
2.2.1 Stivă . . . 51
2.2.2 Adăugarea într-o tabelă hash . . . 52
2.3 Depanare . . . 52
2.3.1 Găsirea unui contraexemplu mic . . . 53
2.3.2 Tipăriți, tipăriți, tipăriți . . . 55
2.3.3 Sfaturi practice . . . 55
2.3.4 Studiu de caz 1 . . . 56
2.3.5 Studiu de caz 2 . . . 59

II Structuri de date pe vectori **65**

3 Arbori de intervale **68**

3.1 Reprezentare . . . 68
3.1.1 Memoria necesară . . . 69
3.1.2 Reprezentări alternative . . . 70
3.2 Operații elementare . . . 70
3.2.1 Actualizarea punctuală . . . 70
3.2.2 Construcția în $\mathcal{O}(n \log n)$ . . . 71
3.2.3 Construcția în $\mathcal{O}(n)$ . . . 71
3.2.4 Calculul sumei pe interval . . . 71
3.2.5 Căutarea unei sume parțiale . . . 73
3.2.6 Căutarea într-un arbore de maxime . . . 73
3.2.7 Adaptarea la alte tipuri de operații . . . 74
3.2.8 Implementarea recursivă . . . 74
3.3 Probleme . . . 74
3.3.1 Problema Xenia and Bit Operations (Codeforces) . . . 74
3.3.2 Problema Distinct Characters Queries (Codeforces) . . . 75
3.3.3 Problema K-query (SPOJ) . . . 75
3.3.4 Problema Sereja and Brackets (Codeforces) . . . 76
3.3.5 Problema Copying Data (Codeforces) . . . 76
3.3.6 Problema PHF (FMI No Stress 2013) . . . 77
3.3.7 Problema Points (Codeforces) . . . 78
3.3.8 Problema Medwalk (Lot 2025) . . . 78

4 Arbori de intervale cu propagare *lazy* **81**

4.1 Operații pe interval . . . 81
4.2 Implementare recursivă (actualizări punctuale) . . . 85
4.3 Implementare recursivă (actualizări pe interval) . . . 86
4.4 Arta proiectării unui arbore de intervale . . . 87
4.5 Probleme . . . 88
4.5.1 Problema Polynomial Queries (CSES) . . . 88
4.5.2 Problema Nezzar and Binary String (Codeforces) . . . 89
4.5.3 Problema Simple (infO(1)Cup 2019) . . . 89
4.5.4 Problema Balama (Baraj ONI 2024) . . . 90
4.5.5 Problema A Simple Task (Codeforces) . . . 91
4.5.6 Problema Periodic RMQ Problem (Codeforces) . . . 92
4.5.7 Problema Circuit (Lot 2025) . . . 93
4.5.8 Problema Babel (Baraj ONI 2025) . . . 95

5 Ștergerea din structuri de date care nu admit ștergeri **97**

5.1 Probleme . . . 97

5.1.1 Problema Addition on Segments (Codeforces) . . . 97
5.1.2 Problema Extending a Set of Points (Codeforces) . . . 99

6 Arbori indexați binar 102
6.1 *Benchmarks* . . . 102
6.1.1 Varianta 1 (*point update, range query*) . . . 103
6.1.2 Varianta 2 (*range update, range query*) . . . 103
6.1.3 Concluzii . . . 103
6.2 Actualizări punctuale și interogări pe interval . . . 104
6.3 Reprezentare . . . 104
6.4 Operația de interogare (suma unui interval) . . . 104
6.5 Operația de actualizare (adăugare pe poziție) . . . 105
6.6 Construcția în $\mathcal{O}(n)$ . . . 106
6.7 Găsirea unei valori punctuale . . . 107
6.8 Căutarea binară a unei sume parțiale . . . 108
6.9 Alte operații decît adunarea . . . 109
6.10 Probleme . . . 110
6.10.1 Problema The Permutation Game Again (SPOJ) . . . 110
6.10.2 Problema Multiset (Codeforces) . . . 110
6.10.3 Problema Hanoi Factory (Codeforces) . . . 111
6.10.4 Problema Subsequences (Codeforces) . . . 111
6.10.5 Problema D-query (SPOJ) . . . 112
6.10.6 Problema Magic Board (CodeChef) . . . 113
6.10.7 Problema Little Artem and Time Machine (Codeforces) . . . 113
6.10.8 Problema Ball (Codeforces) . . . 114
6.11 Interogări punctuale și actualizări pe interval . . . 115
6.12 Interogări și actualizări pe interval . . . 116
6.13 Arbori indexați binar 2D . . . 118
6.13.1 Structura informației . . . 118
6.13.2 Calculul sumei dintr-un dreptunghi . . . 119
6.13.3 Actualizări punctuale . . . 120
6.13.4 Construcția în $\mathcal{O}(n^2)$ . . . 121

7 Structuri 2D offline 122
7.1 Precalcularea . . . 122
7.2 Procesarea efectivă . . . 123
7.3 Probleme . . . 123
7.3.1 Problema Festival (ONI 2025, clasele 11-12) . . . 123
7.3.2 Problema Medwalk, revizitată (Lot 2025) . . . 125

8 Descompunere în radical 128
8.1 Actualizări punctuale . . . 128

8.2 Actualizări pe interval . . . 129
8.3 Optimizări . . . 131
8.3.1 Evitați împărțirile! . . . 131
8.3.2 Alegerea mărimii blocurilor . . . 131
8.3.3 Alegerea mărimii blocurilor pentru operații inegale . . . 132
8.4 Probleme . . . 132
8.4.1 Problema Mexitate (ONI 2018 clasa a 9-a) . . . 132
8.4.2 Problema Give Away (SPOJ) . . . 133
8.4.3 Problema Holes (Codeforces) . . . 134
8.4.4 Problema Piezișă (Baraj ONI 2022) . . . 135
8.5 Descompunere după operații . . . 136
8.6 Probleme . . . 137
8.6.1 Problema Serega and Fun (Codeforces) . . . 137
8.6.2 Problema Rev (Lot 2008) . . . 138
8.7 Procesări diferite înainte și după $\sqrt{n}$ . . . 140
8.8 Probleme . . . 140
8.8.1 Problema Time to Raid Cowavans (Codeforces) . . . 140
8.9 Descompunere în valori distincte . . . 141
8.10 Probleme . . . 141
8.10.1 Problema Sandor (Baraj ONI 2025) . . . 141
8.10.2 Problema Puzzle-bila (Lot 2025) . . . 143

9 Algoritmul lui Mo 146
9.1 Algoritmul lui Mo fără actualizări . . . 146
9.1.1 Analiză de complexitate . . . 147
9.1.2 Optimizare de viteză . . . 147
9.2 Algoritmul lui Mo cu actualizări . . . 148
9.2.1 Mărimea blocului, ordinea operațiilor, complexitate . . . 148
9.3 Probleme . . . 148
9.3.1 Problema Powerful Array (Codeforces) . . . 148
9.3.2 Problema Most Frequent Value (SPOJ) . . . 149
9.3.3 Problema RangeMode (Infoarena Cup 2013) . . . 149
9.3.4 Problema Kiloforces (Concurs RoAlgo #8) . . . 150
9.3.5 Problema Machine Learning (Codeforces) . . . 151

III Structuri de date avansate 153

10 Policy-based data structures 155
10.1 Declararea unui set . . . 155
10.2 Folosirea . . . 156
10.3 Alte variante . . . 157

10.4 Statistici de ordine și augmentare . . . 159
10.5 Concluzii . . . 159
10.6 Probleme . . . 159

11 Skip lists **160**
11.1 Noțiuni de probabilitate . . . 160
11.2 Structura de date . . . 162
11.3 Analiza de complexitate . . . 164
11.4 Augmentarea pentru statistici de ordine . . . 164
11.5 Implementarea . . . 165
11.5.1 Alocarea turnului de pointeri . . . 165
11.5.2 Optimizarea generatorului de numere aleatorii . . . 165
11.5.3 Alegerea probabilității . . . 166
11.6 Benchmarks . . . 166

12 Treapuri **168**
12.1 Reprezentări . . . 169
12.2 Căutarea . . . 170
12.3 Inserarea . . . 170
12.3.1 Menținerea heapului prin rotații . . . 170
12.3.2 Menținerea heapului prin *split* . . . 172
12.4 Ștergerea . . . 173
12.4.1 Menținerea heapului prin rotații . . . 173
12.4.2 Menținerea heapului prin *merge* . . . 173
12.5 Augmentarea pentru statistici de ordine . . . 174
12.6 Treapuri implicite . . . 175
12.7 Operații pe interval . . . 177
12.8 Benchmarks . . . 177
12.9 Probleme . . . 177
12.9.1 Problema Cut and Paste (CSES) . . . 177
12.9.2 Problema Reversals and Sums (CSES) . . . 177
12.9.3 Problema T-Shirts (Codeforces) . . . 178
12.9.4 Problema Clasament (Lot 2016) . . . 180
12.9.5 Problema Roata Norocului (Lot 2024) . . . 181

IV Alte structuri de date **184**

13 Păduri de mulțimi disjuncte **185**
13.1 Probleme . . . 185
13.1.1 Problema Restructuring Company (Codeforces) . . . 185
13.1.2 Problema Trampoline (infO(1) Cup 2020) . . . 186

V Măiestrie pe biți **188**

14 Operații pe biți. Compactarea variabilelor **189**
14.1 Operații elementare . . . 189
14.1.1 Noțiuni de bază . . . 189
14.1.2 Măști . . . 189
14.1.3 Operații pe măști de biți . . . 190
14.2 Numărarea biților de 1 dintr-o valoare (popcount) . . . 190
14.2.1 Metoda naivă . . . 190
14.2.2 Metoda Kernighan . . . 191
14.2.3 Funcții built-in . . . 191
14.2.4 Tabel precalculat . . . 191
14.2.5 Tabel precalculat + conversie . . . 191
14.2.6 Calcul paralel . . . 191
14.3 Logaritmul în baza 2 . . . 192
14.4 Programare dinamică pe măști de biți . . . 193
14.5 Probleme . . . 193
14.5.1 Problema Fill the Bag (Codeforces) . . . 193
14.5.2 Problema Compatible Numbers (Codeforces) . . . 194
14.5.3 Problema Yet Another Substring Reverse (Codeforces) . . . 196
14.5.4 Problema Cases (Codeforces) . . . 198
14.5.5 Problema The Minimum Number of Variables (Codeforces) . . . 198
14.5.6 Problema Math Test (Codeforces) . . . 200
14.6 Compactarea variabilelor . . . 202
14.6.1 Bitset . . . 202
14.6.2 Variabile pe 2 biți . . . 204
14.7 Probleme . . . 204
14.7.1 Problema DNA Evolution (Codeforces) . . . 204
14.8 Iterarea prin submăștile unei măști . . . 206
14.9 Probleme . . . 207
14.9.1 Problema Crisp String (Codeforces) . . . 207

15 Tehnica *Meet in the middle* **209**
15.1 Definiția tehnicii . . . 209
15.2 Probleme . . . 209
15.2.1 Problema Meet in the Middle (CSES) . . . 209
15.2.2 Puzzle-urile de pe Infoarena . . . 210
15.2.3 Problema Anya and Cubes (Codeforces) . . . 212
15.2.4 Problema Prime Gift (Codeforces) . . . 213
15.2.5 Problema Lizard Era: Beginning (Codeforces) . . . 214
15.2.6 Problema Make Them Similar (Codeforces) . . . 215
15.2.7 Problema Light Switches (Codeforces) . . . 216

15.2.8 Problema Algebra Flash (Codeforces) . . . 217
15.2.9 Problema Helping Hiasat (Codeforces) . . . 219

VI Arbori **221**

16 Unelte și algoritmi esențiali **222**
16.1 Generatoare de arbori aleatorii . . . 222
16.2 Probleme . . . 223
16.2.1 Problema Subordinates (CSES) . . . 223
16.2.2 Problema Tree Matching (CSES) . . . 223
16.2.3 Problema Tree Diameter (CSES) . . . 223
16.2.4 Problema Tree Distances II (CSES) . . . 224
16.2.5 Problema White-Black Balanced Subtrees (Codeforces) . . . 224
16.2.6 Problema Blood Cousins (Codeforces) . . . 225

17 Liniarizarea arborilor **227**
17.1 Timpi de intrare și de ieșire din DFS . . . 227
17.2 Testul de strămoș . . . 228
17.3 Liniarizarea. Tipuri de liniarizare . . . 228
17.3.1 Liniarizarea DFS . . . 229
17.3.2 Liniarizarea Euler . . . 229
17.3.3 Liniarizarea Euler cu repetiție . . . 229
17.4 Probleme . . . 230
17.4.1 Problema Tree Queries (Codeforces) . . . 230
17.4.2 Problema Subtree Queries (CSES) . . . 231
17.4.3 Problema Path Queries (CSES) . . . 231
17.4.4 Problema New Year Tree (Codeforces) . . . 231
17.4.5 Problema Max Flow (USACO) . . . 232
17.4.6 Problema Distinct Colors (CSES) . . . 233
17.4.7 Problema Arb (Lot 2008) . . . 234
17.4.8 Problema Disconnect (Infoarena) . . . 235

18 Tehnica *small-to-large* **237**
18.1 Generalități . . . 237
18.2 Probleme . . . 237
18.2.1 Problema Fixed-Length Paths I (CSES) . . . 237
18.2.2 Problema Distinct Colors (CSES) (din nou) . . . 238
18.2.3 Problema Lomsat Gelral (Codeforces) . . . 239
18.2.4 Problema Tokens on a Tree (CodeChef) . . . 239
18.2.5 Problema Blood Cousins Return (Codeforces) . . . 240
18.3 DFS exclusiv . . . 241
18.4 Probleme . . . 243

18.4.1 Problema Tree and Queries (Codeforces) . . . 243
18.4.2 Problema Dominant Indices (Codeforces) . . . 244

19 Cel mai apropiat strămoș comun **246**
19.1 Sumar: RMQ (*range minimum query*) . . . 246
19.1.1 RMQ cu arbore de intervale . . . 247
19.1.2 RMQ cu arbore indexat binar . . . 247
19.1.3 RMQ cu descompunere în radical . . . 247
19.1.4 RMQ cu tabelă rară . . . 247
19.1.5 RMQ cu stivă ordonată . . . 248
19.2 LCA cu liniarizare . . . 248
19.3 LCA cu descompunere în radical . . . 249
19.4 LCA cu binary lifting ($\log n$ pointeri per nod) . . . 250
19.5 LCA cu binary lifting (2 pointeri per nod) . . . 250
19.6 LCA cu algoritmul lui Tarjan (offline) . . . 250
19.7 Benchmarks . . . 251
19.8 Probleme . . . 251
19.8.1 Problema Gold Transfer (Codeforces) . . . 251
19.8.2 Problema A and B and Lecture Rooms (Codeforces) . . . 252
19.8.3 Problema Company (Codeforces) . . . 253
19.8.4 Problema Duff in the Army (Codeforces) . . . 254

20 Algoritmul lui Mo pe arbore **255**
20.1 Limitele liniarizărilor . . . 255
20.2 Reducerea la algoritmul lui Mo . . . 255
20.3 Un exemplu . . . 256
20.4 Un caz particular . . . 256
20.5 Descrierea completă . . . 257
20.6 Structura de date necesară . . . 257
20.7 Probleme . . . 257
20.7.1 Problema Dating (Codeforces) . . . 257
20.7.2 Problema Notdivisible (IIOT 2022-2023 runda 3) . . . 259

21 Descompunere *heavy-light* **261**
21.1 Limitările structurilor anterioare . . . 261
21.2 Descompunerea . . . 262
21.3 Detalii de implementare . . . 263
21.4 Probleme . . . 264
21.4.1 Problema Heavy Path Decomposition (Infoarena) . . . 264
21.4.2 Problema Disruption (USACO) . . . 264
21.4.3 Problema Subway Lines (Codeforces) . . . 266
21.4.4 Problema Rafaela (Lot 2014) . . . 267

21.4.5 Problema Doi arbori (Lot 2024) . . . 270
21.4.6 Problema Query on a Tree VI (CodeChef) . . . 273
21.4.7 Problema Adă caii (Lot 2025) . . . 274

22 Descompunere în centroizi **276**
22.1 Definiție . . . 276
22.2 Proprietăți . . . 276
22.3 Găsirea unui centroid . . . 277
22.4 Descompunerea în centroizi . . . 277
22.5 Probleme . . . 279
22.5.1 Problema Finding A Centroid (CSES) . . . 279
22.5.2 Problema Mystery Tree (CodeChef) . . . 279
22.5.3 Problema Ciel the Commander (Codeforces) . . . 280
22.5.4 Problema Fixed-Length Paths I (CSES) (din nou) . . . 280
22.5.5 Problema Xenia and Tree (Codeforces) . . . 281
22.5.6 Problema Flareon (Lot 2017) . . . 282
22.5.7 Problema Digit Tree (Codeforces) . . . 284
22.5.8 Problema Cucuruz (Concurs RoAlgo # 7) . . . 286
22.5.9 Problema LaNaic (Lot 2024) . . . 287

VII Matematică **290**

23 Elemente de teoria numerelor **291**
23.1 Exponențiere binară . . . 291
23.1.1 Mica teoremă a lui Fermat . . . 291
23.1.2 Exponențiere în $\mathcal{O}(1)$ cu precalculare în $\mathcal{O}(\sqrt{n})$ . . . 292
23.1.3 Aplicație: Suma unei progresii geometrice . . . 293
23.2 Divizibilitate . . . 296
23.3 Principiul includerii și excluderii . . . 296
23.3.1 Implementare iterativă . . . 297
23.3.2 Implementare recursivă . . . 297
23.4 Indicatorul lui Euler . . . 297
23.4.1 Calculul lui φ . . . 298
23.4.2 Calculul lui φ de la 1 la n . . . 299
23.5 Teorema lui Euler . . . 299
23.6 Algorithmul lui Euclid extins . . . 300
23.6.1 Implementarea recursivă . . . 300
23.6.2 Implementarea iterativă . . . 301
23.7 Inverse modulare . . . 302
23.7.1 Prin exponențiere rapidă . . . 302
23.7.2 Prin algoritmul lui Euclid extins . . . 302

23.7.3 Benchmarks . . . 303
23.8 Funcția Möbius . . . 303
23.8.1 Relația cu principiul includerii și excluderii . . . 303
23.8.2 Definiția funcției Möbius . . . 303
23.8.3 Diagramă Euler-Venn . . . 304
23.8.4 Implementare . . . 305
23.9 Formula de inversiune Möbius . . . 307
23.9.1 Funcția Möbius este multiplicativă . . . 307
23.9.2 Suma funcției Möbius peste divizori este multiplicativă . . . 308
23.9.3 Valoarea sumei funcției Möbius peste divizori . . . 308
23.9.4 Formula de inversiune a lui Möbius . . . 309
23.9.5 Aplicații . . . 310
23.10 Probleme . . . 312
23.10.1 Problema Exponentiation (CSES) . . . 312
23.10.2 Problema Exponentiation II (CSES) . . . 312
23.10.3 Problema Eugene and Big Number (CodeChef) . . . 312
23.10.4 Problema Counting Divisors (CSES) . . . 312
23.10.5 Problema Common Divisors (CSES) . . . 312
23.10.6 Problema Sum of Divisors (CSES) . . . 313
23.10.7 Problema Prime Multiples (CSES) . . . 313
23.10.8 Problema List of Integers (Codeforces) . . . 313
23.10.9 Problema Small GCD (Codeforces) . . . 314
23.10.10 Problema Secvxor (Baraj ONI 2023) . . . 315
23.10.11 Problema Afacere (Lot 2009) . . . 316
23.10.12 Problema Playing with GCD (SPOJ) . . . 317
23.10.13 Problema Power Tower (Codeforces) . . . 318
23.10.14 Problema GCD Extreme (SPOJ) . . . 318
23.10.15 Problema Counting Coprime Pairs (CSES) . . . 319
23.10.16 Problema Coprime Sequences (Codeforces) . . . 319
23.10.17 Problema Coprime Triples (CodeChef) . . . 319
23.10.18 Problema Coprime Arrays (Codeforces) . . . 319
23.10.19 Problema Mocha and Stars (Codeforces) . . . 321
23.10.20 Problema Pastile (Baraj ONI 2021) . . . 322
23.10.21 Problema Beautiful Numbers (Codeforces) . . . 323

24 Teorema chinezească a resturilor **324**
24.1 Un exemplu tabelar . . . 324
24.2 Un exemplu cu doi moduli . . . 325
24.3 Un exemplu cu 3 moduli . . . 326
24.4 Cazul general cu n moduli . . . 326
24.5 Cazul cu moduli necoprimi . . . 327

24.5.1 Condiția de existență . . . 327
24.5.2 Metoda de construcție . . . 328
24.6 Probleme . . . 329
24.6.1 Problema Koxia and Number Theory (Codeforces) . . . 329
24.6.2 Problema Remainders Game (Codeforces) . . . 330
24.6.3 Problema Decipher the String (Codeforces) . . . 331
24.6.4 Problema Two Chandeliers (Codeforces) . . . 331
24.6.5 Problema GCD Guess (Codeforces) . . . 332

25 Elemente de bază în combinatorică 335
25.1 Formule pentru combinări . . . 335
25.2 Calculul combinărilor . . . 336
25.3 Permutări cu repetiție . . . 336
25.3.1 Aplicație . . . 337
25.4 Combinări cu repetiție . . . 337
25.4.1 Aplicații . . . 337
25.5 Numărarea permutărilor fără punct fix . . . 337
25.6 Numerele lui Catalan . . . 338
25.7 Lema lui Burnside . . . 339
25.8 Un puzzle . . . 339
25.9 Probleme . . . 340
25.9.1 Problema Binomial Coefficients (CSES) . . . 340
25.9.2 Problema Creating Strings II (CSES) . . . 340
25.9.3 Problema Distributing Apples (CSES) . . . 340
25.9.4 Problema Christmas Party (CSES) . . . 340
25.9.5 Problema Bracket Sequences I (CSES) . . . 341
25.9.6 Problema Bracket Sequences II (CSES) . . . 341
25.9.7 Problema Counting Necklaces (CSES) . . . 341
25.9.8 Problema Counting Grids (CSES) . . . 341
25.9.9 Problema Number of Permutations (Codeforces) . . . 341
25.9.10 Problema Counting Factorizations (Codeforces) . . . 341
25.9.11 Problema Monocarp and the Set (Codeforces) . . . 343
25.9.12 Problema Devu and Flowers (Codeforces) . . . 343
25.9.13 Problema Lengthening Sticks (Codeforces) . . . 344
25.9.14 Problema Shuffle (Codeforces) . . . 345
25.9.15 Problema Manhattan (Lot 2017) . . . 346

26 Rangul permutărilor și al combinărilor 349
26.1 Definiții . . . 349
26.2 Rangul permutărilor . . . 350
26.2.1 Următoarea permutare . . . 350
26.2.2 *Ranking* . . . 350

26.2.3 *Unranking* . . . 350
26.2.4 Implementare . . . 350
26.3 Rangul combinărilor . . . 351
26.3.1 Următoarea combinare . . . 351
26.3.2 *Ranking* lexicografic . . . 352
26.3.3 *Unranking* lexicografic . . . 352
26.3.4 *Ranking* și *unranking* colexicografic . . . 352
26.4 Probleme . . . 354
26.4.1 Problema Misha and Permutation Summation (Codeforces) . . . 354
26.4.2 Problema Inversion Sort (SPOJ) . . . 354
26.4.3 Problema Four Chips (SPOJ) . . . 355
26.4.4 Problema Long Permutation (Codeforces) . . . 355
26.4.5 Problema Arbperm2 (NerdArena) . . . 356
26.4.6 Problema Hipersimetrie (ONI 2019, clasele 11-12) . . . 357
26.4.7 Problema Trim (Baraj ONI 2023) . . . 361
26.4.8 Problema Bubblesort (Lot 2010) . . . 361
26.4.9 Problema Valentine's Day (infO(1) Cup 2023) . . . 363

27 Rezolvarea recurențelor liniare prin exponențiere rapidă 367
27.1 Șirul lui Fibonacci. Matrice generatoare . . . 367
27.2 Alte recurențe liniare . . . 368
27.3 Recurențe liniare cu termen constant . . . 369
27.4 Probleme . . . 369
27.4.1 Problema Al k-lea termen Fibonacci (Infoarena) . . . 369
27.4.2 Problema Iepuri (Infoarena) . . . 369
27.4.3 Problema Recurenta2 (Infoarena) . . . 370
27.4.4 Problema Graph Paths I (CSES) . . . 371
27.4.5 Problema Hprob (Infoarena) . . . 371
27.4.6 Problema Chef and Riffles (CodeChef) . . . 372
27.4.7 Problema Zid (ONI 2024, clasele 11-12) . . . 374
27.4.8 Problema Stairs and Lines (Codeforces) . . . 376
27.4.9 Problema Marfa (Lot 2014) . . . 378
27.4.10 Problema Options (IIOT 2017-2018 runda internațională) . . . 380

VIII Geometrie computațională 381

28 Elemente de bază 382
28.1 Principii de implementare . . . 382
28.2 Puncte și vectori . . . 382
28.2.1 Adunare, înmulțire cu o constantă . . . 383
28.2.2 Produsul scalar (engl. *dot product*) . . . 383

28.2.3 Produsul vectorial (engl. *cross product*) . . . 383
28.2.4 Unghiuri . . . 384
28.2.5 Rotații . . . 384
28.3 Drepte și segmente . . . 384
28.3.1 Test de paralelism . . . 385
28.3.2 Test de perpendicularitate . . . 385
28.3.3 Intersecția a două drepte . . . 386
28.3.4 Separarea planului . . . 386
28.3.5 Distanța punct-dreaptă . . . 386
28.3.6 Intersecția a două segmente . . . 386
28.4 Orientare; determinanți . . . 387
28.5 Arii . . . 388
28.5.1 Aria unui poligon oarecare prin metoda trapezelor . . . 388
28.5.2 Aria unui poligon oarecare prin metoda triunghiurilor . . . 389
28.6 Probleme . . . 389
28.6.1 Problema Cuiburi (Baraj ONI 2010) . . . 389
28.6.2 Problema Ace (OJI 2017, clasa a 9-a) . . . 389
28.6.3 Problema Baba Oarba (Lot 2016) . . . 390
28.6.4 Problema Arhitect (OJI 2023, clasa a 10-a) . . . 390
28.6.5 Problema Elicoptere (OJI 2016, clasele 11-12) . . . 391
28.6.6 Problema Arpa and an Exam About Geometry (Codeforces) . . . 391
28.6.7 Problema Part (Baraj ONI 2006) . . . 392
28.6.8 Problema Bear and Floodlight (Codeforces) . . . 393
28.6.9 Problema TrapEZZ (Moisil++ 2023, clasele 11-12) . . . 393
28.6.10 Problema Terenuri (Baraj ONI 2011) . . . 394
28.6.11 Problema Metin2 (Finala IIOT 2021-22) . . . 395

29 Algoritmi specifici **397**
29.1 Teorema lui Pick . . . 397
29.2 Înfășurătoarea convexă . . . 397
29.3 Algoritmi de baleiere . . . 398
29.4 Șublerul rotitor . . . 398
29.5 Probleme . . . 399
29.5.1 Problema Copaci (Infoarena) . . . 399
29.5.2 Problema Emptri (Lot 2015) . . . 399
29.5.3 Problema Înfășurătoare convexă (Infoarena) . . . 400
29.5.4 Problema Înfășurătoare convexă (NerdArena) . . . 400
29.5.5 Problema Testament (Lot 2011) . . . 400
29.5.6 Problema Magic (JBOI 2023) . . . 400
29.5.7 Problema Triangular Queries (CodeChef) . . . 402
29.5.8 Problema Nuke (Lot 2010) . . . 404

29.5.9 Problema Hill Walk (USACO Gold 2013) . . . 405
29.5.10 Problema Ydist (Lot 2014) . . . 405
29.5.11 Problema Fossil in the Ice (CodeChef) . . . 408
29.5.12 Problema Rubarba (Infoarena) . . . 409

IX Grafuri 410

30 Arbori parțiali minimi 411
30.1 Algoritmii lui Kruskal și Prim . . . 411
30.2 Proprietăți ale arborilor parțiali minimi . . . 412
30.3 Actualizarea APM . . . 412
30.4 Probleme . . . 413
30.4.1 Problema Arbore parțial de cost minim (Infoarena) . . . 413
30.4.2 Problema Autobuze3 (AGM 2015) . . . 413
30.4.3 Problema Rusuoaica (FMI No Stress 9 Warmup) . . . 414
30.4.4 Problema MinOr Tree (Codeforces) . . . 414
30.4.5 Problema 0-1 MST (Codeforces) . . . 415
30.4.6 Problema Minimum Spanning Tree for Each Edge (Codeforces) . . . 416
30.4.7 Problema Apm2 (Infoarena) . . . 417
30.4.8 Problema Edges in MST (Codeforces) . . . 418
30.4.9 Problema Envy (Codeforces) . . . 419
30.4.10 Problema Gift (Codeforces) . . . 419
30.4.11 Problema DFS Trees (Codeforces) . . . 421

31 Conectivitate 423
31.1 Sortarea topologică . . . 423
31.1.1 Algoritmul lui Kahn . . . 423
31.1.2 Algoritmul bazat pe DFS . . . 424
31.2 Componente tare conexe . . . 424
31.2.1 Terminologie . . . 424
31.2.2 Algoritmul lui Kosaraju . . . 425
31.2.3 Algoritmul lui Tarjan . . . 426
31.3 Componente biconexe, punți, puncte de articulație . . . 426
31.3.1 Definiții . . . 427
31.3.2 Puzzles . . . 427
31.3.3 Algoritmul Hopcroft-Tarjan . . . 427
31.4 Probleme . . . 429
31.4.1 Problema Course Schedule (CSES) . . . 429
31.4.2 Problema Book (Codeforces) . . . 429
31.4.3 Problema Componente tare conexe (Infoarena) . . . 430
31.4.4 Problema Mr. Kitayuta's Technology (Codeforces) . . . 430

31.4.5 Problema Ralph and Mushrooms (Codeforces) . . . 431
31.4.6 Problema Bertown Roads (Codeforces) . . . 431
31.4.7 Problema Tourist Reform (Codeforces) . . . 432
31.4.8 Problema We Need More Bosses (Codeforces) . . . 432
31.4.9 Problema Forbidden Cities (CSES) . . . 433
31.4.10 Problema Case of Computer Network (Codeforces) . . . 434
31.4.11 Problema Dog Trick Competition 2 (IIOT 2023-2024 runda 4) . . . 435
31.4.12 Problema Regate (ONI 2022, clasele 11-12) . . . 437

32 Distanțe **440**
32.1 Terminologie . . . 440
32.2 Algoritmul Bellman-Ford . . . 440
32.3 Caz particular: graf aciclic (*dag*) . . . 441
32.4 Caz particular: muchii ne-negative . . . 442
32.5 Caz particular: BFS . . . 442
32.6 Caz particular: 0-1 BFS . . . 443
32.7 Caz particular: costuri mici . . . 443
32.8 Probleme . . . 443
32.8.1 Problema Bellman-Ford (Infoarena) . . . 443
32.8.2 Problema Dijkstra (Infoarena) . . . 444
32.8.3 Problema Monsters (CSES) . . . 444
32.8.4 Problema Shortest Routes II (CSES) . . . 444
32.8.5 Problema High Score (CSES) . . . 444
32.8.6 Problema Flight Discount (CSES) . . . 445
32.8.7 Problema Three States (Codeforces) . . . 445
32.8.8 Problema Graph and Graph (Codeforces) . . . 445
32.8.9 Problema Termite (Lot 2014) . . . 446
32.8.10 Problema President and Roads (Codeforces) . . . 448
32.8.11 Problema Nastya and Unexpected Guest (Codeforces) . . . 449
32.8.12 Problema Minimum Path (Codeforces) . . . 450
32.8.13 Problema Shortest Path (Codeforces) . . . 450

33 Drumuri euleriene **452**
33.1 Algoritmi . . . 452
33.2 Probleme . . . 453
33.2.1 Problema Ciclu eulerian (Infoarena) . . . 453
33.2.2 Problema De Bruijn Sequence (CSES) . . . 454
33.2.3 Problema Tanya and Password (Codeforces) . . . 454

34 Fluxuri **455**
34.1 Descrierea problemei . . . 455
34.1.1 Exerciții teoretice . . . 456

34.2 Metoda Ford-Fulkerson . . . 456
34.2.1 Rețeaua reziduală . . . 456
34.2.2 Drumuri de creștere . . . 457
34.2.3 Metoda Ford-Fulkerson . . . 457
34.3 Algoritmul Edmonds-Karp . . . 457
34.3.1 Optimizare . . . 458
34.4 Fluxuri și tăieturi . . . 458
34.4.1 Aflarea tăieturii minime . . . 460
34.5 Algoritmul lui Dinic . . . 460
34.5.1 Demonstrație de corectitudine . . . 461
34.5.2 Găsirea unui blocaj de flux . . . 461
34.5.3 Calculul complexității . . . 461
34.6 Lectură suplimentară . . . 462
34.7 Probleme . . . 463
34.7.1 Problema Download Speed (CSES) . . . 463
34.7.2 Problema Police Chase (CSES) . . . 463
34.7.3 Problema Distinct Routes (CSES) . . . 463
34.7.4 Problema Soldier and Traveling (Codeforces) . . . 463
34.7.5 Problema Fox and Dinner (Codeforces) . . . 464
34.7.6 Problema Petya and Graph (Codeforces) . . . 465
34.7.7 Problema Asfalt (Lot 2011) . . . 467
34.7.8 Problema Echipe (Lot 2025) . . . 468

35 Cuplaje în grafuri bipartite 472
35.1 Definiții . . . 472
35.2 Modelarea ca problemă de flux . . . 472
35.3 Căi alternante . . . 473
35.4 Algoritmul Hopcroft-Karp . . . 473
35.4.1 Analiză de complexitate . . . 473
35.4.2 Detalii de implementare . . . 474
35.5 Algoritmul lui Kuhn . . . 474
35.5.1 Detalii de implementare . . . 474
35.6 Teorema lui Kőnig . . . 475
35.7 Probleme . . . 476
35.7.1 Problema School Dance (CSES) . . . 476
35.7.2 Problema Revoluție (ONI 2009, clasele 11-12) . . . 476
35.7.3 Problema Valuable Paper (Codeforces) . . . 476
35.7.4 Problema Teroriști (Baraj ONI 2010) . . . 477
35.7.5 Problema Brevity Is the Soul of Wit (Codeforces) . . . 478
35.7.6 Problema Access Levels (Codeforces) . . . 478
35.7.7 Problema Pswap (Baraj ONI 2021) . . . 479

36 Fluxuri de cost minim **483**
36.1 Descrierea problemei . . . 483
36.2 Rețeaua reziduală . . . 483
36.3 Mecanismul general . . . 484
36.4 Algoritmii Bellman-Ford sau SPFA . . . 485
36.5 Algoritmul lui Johnson . . . 485
36.6 Adaptarea algoritmului lui Johnson pentru FMCM . . . 486
36.7 Probleme . . . 487
36.7.1 Problema Parcel Delivery (CSES) . . . 487
36.7.2 Problema Task Assignment (CSES) . . . 487
36.7.3 Problema Match (IIOT 2019-2020 runda 3) . . . 488
36.7.4 Problema Hoață (Baraj ONI 2022) . . . 490
36.7.5 Problema Lifts (IATI Shumen 2022) . . . 493

X Algoritmi pe șiruri de caractere **497**

37 Căutări în șiruri de caractere. Algoritmul Rabin-Karp **498**
37.1 Terminologie . . . 498
37.2 Algoritmul naiv . . . 499
37.2.1 Toate caracterele din șablon sînt diferite . . . 499
37.2.2 Unul dintre șirurile S și P este generat aleatoriu . . . 500
37.2.3 m are valoare apropiată de n . . . 500
37.3 Funcții hash . . . 500
37.4 Căutarea cu funcții hash . . . 501
37.5 Funcții *rolling hash* . . . 501
37.6 Algoritmul Rabin-Karp . . . 502
37.7 Probleme . . . 504
37.7.1 Problema Radio (Lot 2010) . . . 504

38 Căutări cu automate finite. Algoritmul Knuth-Morris-Pratt **507**
38.1 Terminologie . . . 507
38.2 Automate finite . . . 507
38.2.1 Alte exemple de automate . . . 509
38.3 Automate pentru recunoașterea de subșiruri . . . 510
38.4 Algoritmul de căutare cu automate finite . . . 512
38.5 Algoritmul Knuth-Morris-Pratt . . . 513
38.5.1 Analiza complexității . . . 516
38.6 Aplicații . . . 516
38.7 Probleme . . . 516
38.7.1 Problema Bart (NerdArena) . . . 516
38.7.2 Problema Cifru (Baraj ONI 2006) . . . 517

38.7.3 Problema Anthem of Berland (Codeforces) . . . 517

39 Funcția Z **519**
39.1 Definiție . . . 519
39.2 Calcularea funcției Z . . . 519
39.3 Implementare . . . 521
39.4 Analiza complexității . . . 521
39.5 Aplicații . . . 522

40 Quicksort ternar **525**
40.1 Algoritmul . . . 525
40.2 Implementare . . . 525
40.3 Benchmarks . . . 527
40.4 Probleme . . . 527
40.4.1 Problema Cuvinte (ONI 2021, clasele 11-12) . . . 527
40.4.2 Problema Sabin (Baraj ONI 2015) . . . 529

41 Șiruri de sufixe **531**
41.1 Definiție . . . 531
41.2 Aplicații . . . 531
41.2.1 Aparițiile unui subșir P în S (*string matching*) . . . 532
41.2.2 Rotația minimă dpdv lexicografic . . . 532
41.3 Cele mai lungi prefixe comune . . . 532
41.4 Aplicații . . . 533
41.4.1 Cel mai lung subșir care apare de cel puțin două ori . . . 533
41.4.2 Cel mai lung subșir care apare de cel puțin k ori . . . 533
41.4.3 Cel mai scurt subșir care apare o singură dată . . . 533
41.4.4 Cel mai lung subșir comun între două șiruri S și T . . . 533
41.4.5 Numărul de subșiruri distincte . . . 534
41.4.6 Cel mai lung subșir palindrom . . . 534
41.5 Construcția șirului de sufixe . . . 534
41.5.1 Algoritmul în $\mathcal{O}(n^2 \log n)$ . . . 534
41.5.2 Algoritmul în $\mathcal{O}(n \log^2 n)$ . . . 534
41.5.3 Algoritmul în $\mathcal{O}(n \log n)$ . . . 537
41.6 Construcția vectorului LCP . . . 540
41.6.1 Implementare . . . 541
41.6.2 Analiza complexității . . . 541
41.7 Probleme . . . 542
41.7.1 Problema Carry Bit (IIOT 2023-2024 runda 3) . . . 542
41.7.2 Problema Periodic Words (IIOT 2023-2024 runda 1) . . . 543
41.7.3 Problema Phone Book (Finala IIOT 2022-2023) . . . 544

42 Arbori trie **545**
42.1 Implementare . . . 545
42.2 Probleme . . . 546
42.2.1 Problema Enigma (Lot 2011) . . . 546
42.2.2 Problema Maximum Xor Subarray (CSES) . . . 547
42.2.3 Problema Esoteric Bovines (IIOT 2023-2024 runda internațională) . . . 547
42.2.4 Problema Cipher (IIOT 2022-2023 runda internațională) . . . 550

XI Probleme diverse **552**

43 Probleme diverse **553**
43.1 Problema Liars (Baraj ONI 2025) . . . 553
43.1.1 Generalități . . . 553
43.1.2 Numărarea configurațiilor . . . 553
43.1.3 Găsirea unei configurații . . . 554
43.2 Problema Pavare infinită (Lot 2025) . . . 555
43.2.1 Găsirea conturului . . . 555
43.2.2 Verificarea criteriului . . . 555
43.2.3 Detalii de implementare . . . 556
43.3 Problema Triunghi (Lot 2024) . . . 556
43.3.1 Soluția prin înjumătățire . . . 556
43.3.2 Detalii de implementare . . . 557
43.3.3 Misterioasa soluție în $\mathcal{O}(n^2)$ . . . 557
43.4 Problema Subiectul I („Nervi”) (Lot 2024) . . . 557
43.4.1 Soluție în $\mathcal{O}(n^3)$ . . . 558
43.4.2 Soluție în $\mathcal{O}(n^2)$ . . . 558
43.5 Problema Subiectul III („Single crossing”) (Lot 2024) . . . 558
43.5.1 Compararea permutărilor . . . 559
43.5.2 Sortare . . . 559
43.5.3 Verificare . . . 559
43.6 Problema Gurger Brill (Lot 2024) . . . 560
43.6.1 Soluție în $\mathcal{O}(n^2)$ interogări . . . 560
43.6.2 Soluție în $\mathcal{O}(n \log n)$ interogări . . . 560
43.6.3 Soluție în $4\sqrt{n}$ interogări . . . 560
43.6.4 Soluție în $4 \log n$ interogări . . . 561
43.7 Problema Takara (Baraj ONI 2019) . . . 562
43.7.1 Formula de recurență . . . 562
43.7.2 Eliminarea funcției max . . . 562
43.7.3 Reducerea la $\mathcal{O}(n^2)$ . . . 563
43.8 Problema Tornade (Baraj ONI 2025) . . . 565
43.9 Problema As de Spade (Lot 2024) . . . 566

43.10 Problema Scurtcircuit (Lot 2025) . . . 568
43.10.1 Compresorul 3:2 . . . 568
43.10.2 Reducerea de la k numere la două numere . . . 569
43.10.3 Generarea și propagarea transportului . . . 569
43.10.4 Adunarea a două numere . . . 570
43.11 Problema Lungcircuit (Lot 2025) . . . 570
43.11.1 Soluția „tractor” . . . 571
43.11.2 Soluție cu liste . . . 572
43.12 Problema Cal-cule (Lot 2025) . . . 572
43.12.1 Enunțul . . . 572
43.12.2 O soluție în $\mathcal{O}(n \log n)$ . . . 573
43.12.3 Soluția optimă . . . 574
43.13 Problema 6 de Pentagrame (Lot 2024) . . . 576
43.13.1 Observații teoretice . . . 576
43.13.2 Structuri de date . . . 577

Postfață **578**

XII Anexă: Cod-sursă 581

1 Principii - Bune practici în programare **582**
1.1 Generator de teste . . . 582

2 Arbori de intervale **584**
2.1 Problema Xenia and Bit Operations (Codeforces) . . . 584
2.2 Problema Distinct Characters Queries (Codeforces) . . . 585
2.3 Problema K-query (SPOJ) . . . 588
2.4 Problema Sereja and Brackets (Codeforces) . . . 592
2.5 Problema Copying Data (Codeforces) . . . 595
2.6 Problema PHF (FMI No Stress 2013) . . . 597
2.7 Problema Points (Codeforces) . . . 599
2.8 Problema Medwalk (Lot 2025) . . . 602

3 Arbori de intervale cu propagare *lazy* **608**
3.1 Problema Polynomial Queries (CSES) . . . 608
3.2 Problema Nezzar and Binary String (Codeforces) . . . 614
3.3 Problema Simple (infO(1)Cup 2019) . . . 618
3.4 Problema Balama (Baraj ONI 2024) . . . 622
3.5 Problema A Simple Task (Codeforces) . . . 629
3.6 Problema Periodic RMQ Problem (Codeforces) . . . 637
3.7 Problema Circuit (Lot 2025) . . . 642
3.8 Problema Babel (Baraj ONI 2025) . . . 645

4 Ștergerea din structuri de date care nu admit ștergeri **654**
4.1 Problema Addition on Segments (Codeforces) 654
4.2 Problema Extending a Set of Points (Codeforces) 656

5 Arbori indexați binar **661**
5.1 Problema The Permutation Game Again (SPOJ) 661
5.2 Problema Multiset (Codeforces) 662
5.3 Problema Hanoi Factory (Codeforces) 664
5.4 Problema Subsequences (Codeforces) 666
5.5 Problema D-query (SPOJ) 668
5.6 Problema Magic Board (CodeChef) 671
5.7 Problema Little Artem and Time Machine (Codeforces) 674
5.8 Problema Ball (Codeforces) 676

6 Structuri 2D offline **679**
6.1 Problema Festival (ONI 2025, clasele 11-12) 679
6.2 Problema Medwalk revizitată (Lot 2025) 683

7 Descompunere în radical **690**
7.1 Problema Mexitate (ONI 2018 clasa a 9-a) 690
7.2 Problema Give Away (SPOJ) 696
7.3 Problema Holes (Codeforces) 701
7.4 Problema Piezișă (Baraj ONI 2022) 703
7.5 Problema Serega and Fun (Codeforces) 712
7.6 Problema Rev (Lot 2008) 723
7.7 Problema Time to Raid Cowavans (Codeforces) 727
7.8 Problema Sandor (Baraj ONI 2025) 728
7.9 Problema Puzzle-bila (Lot 2025) 732

8 Algoritmul lui Mo **737**
8.1 Problema Powerful Array (Codeforces) 737
8.2 Problema Most Frequent Value (SPOJ) 739
8.3 Problema RangeMode (Infoarena Cup 2013) 741
8.4 Problema Kiloforces (Concurs RoAlgo #8) 744
8.5 Problema Machine Learning (Codeforces) 747

9 Skip lists **751**
9.1 Implementare de *skip lists* augmentate 751

10 Treaps **755**
10.1 Implementare de treap augmentat 755
10.2 Problema Cut and Paste (CSES) 767
10.3 Problema Reversals and Sums (CSES) 770

10.4 Problema T-Shirts (Codeforces) . . . 773
10.5 Problema Clasament (Lot 2016) . . . 777
10.6 Problema Roata Norocului (Lot 2024) . . . 785

11 Păduri de mulțimi disjuncte **793**
11.1 Problema Restructuring Company (Codeforces) . . . 793
11.2 Problema Trampoline (infO(1) Cup 2020) . . . 794

12 Operații pe biți **803**
12.1 Problema Fill the Bag (Codeforces) . . . 803
12.2 Problema Compatible Numbers (Codeforces) . . . 804
12.3 Problema Yet Another Substring Reverse (Codeforces) . . . 807
12.4 Problema Cases (Codeforces) . . . 809
12.5 Problema The Minimum Number of Variables (Codeforces) . . . 811
12.6 Problema Math Test (Codeforces) . . . 813
12.7 Implementare de bitset . . . 816
12.8 Implementare de variabile pe doi biți . . . 820
12.9 Problema DNA Evolution (Codeforces) . . . 823
12.10 Problema Crisp String (Codeforces) . . . 833

13 Tehnica *Meet in the middle* **838**
13.1 Problema Meet in the Middle (CSES) . . . 838
13.2 Problema Anya and Cubes (Codeforces) . . . 842
13.3 Problema Prime Gift (Codeforces) . . . 848
13.4 Problema Lizard Era: Beginning (Codeforces) . . . 849
13.5 Problema Make Them Similar (Codeforces) . . . 852
13.6 Problema Light Switches (Codeforces) . . . 858
13.7 Problema Algebra Flash (Codeforces) . . . 861
13.8 Problema Helping Hiasat (Codeforces) . . . 864

14 Arbori - probleme esențiale **867**
14.1 Generator simplu de arbori aleatorii . . . 867
14.2 Generator avansat de arbori aleatorii . . . 868
14.3 Problema Subordinates (CSES) . . . 871
14.4 Problema Tree Matching (CSES) . . . 873
14.5 Problema Tree Diameter (CSES) . . . 875
14.6 Problema Tree Distances II (CSES) . . . 877
14.7 Problema White-Black Balanced Subtrees (Codeforces) . . . 879
14.8 Problema Blood Cousins (Codeforces) . . . 880

15 Arbori - liniarizare **884**
15.1 Problema Tree Queries (Codeforces) . . . 884
15.2 Problema Subtree Queries (CSES) . . . 886

15.3 Problema Path Queries (CSES) . . . 888
15.4 Problema New Year Tree (Codeforces) . . . 891
15.5 Problema Max Flow (USACO) . . . 895
15.6 Problema Distinct Colors (CSES) . . . 902
15.7 Problema Arb (Lot 2008) . . . 904
15.8 Problema Disconnect (Infoarena) . . . 912

16 Arbori - small-to-large **917**
16.1 Problema Fixed-Length Paths I (CSES) . . . 917
16.2 Problema Distinct Colors (CSES) (din nou) . . . 918
16.3 Problema Lomsat Gelral (Codeforces) . . . 920
16.4 Problema Tokens on a Tree (CodeChef) . . . 926
16.5 Problema Blood Cousins Return (Codeforces) . . . 934
16.6 Problema Tree and Queries (Codeforces) . . . 941
16.7 Problema Dominant Indices (Codeforces) . . . 947

17 Arbori - cel mai apropiat strămoș comun **953**
17.1 LCA cu descompunere în radical . . . 953
17.2 LCA cu binary lifting ($\log n$ pointeri per nod) . . . 955
17.3 LCA cu binary lifting (2 pointeri per nod) . . . 959
17.4 LCA cu algoritmul lui Tarjan (offline) . . . 965
17.5 Problema Gold Transfer (Codeforces) . . . 967
17.6 Problema A and B and Lecture Rooms (Codeforces) . . . 969
17.7 Problema Company (Codeforces) . . . 972
17.8 Problema Duff in the Army (Codeforces) . . . 976

18 Arbori - algoritmul lui Mo pe arbore **981**
18.1 Problema Dating (Codeforces) . . . 981
18.2 Problema Notdivisible (Codeforces) . . . 990

19 Arbori - descompunere *heavy-light* **997**
19.1 Problema Heavy Path Decomposition (Infoarena) . . . 997
19.2 Problema Disruption (USACO) . . . 1005
19.3 Problema Subway Lines (Codeforces) . . . 1015
19.4 Problema Rafaela (Lot 2014) . . . 1017
19.5 Problema Doi arbori (Lot 2025) . . . 1031
19.6 Problema Query on a Tree VI (CodeChef) . . . 1043
19.7 Problema Adă caii (Lot 2025) . . . 1049

20 Arbori - descompunere în centroizi **1057**
20.1 Problema Finding a Centroid (CSES) . . . 1057
20.2 Problema Mystery Tree (CodeChef) . . . 1060
20.3 Problema Ciel the Commander (Codeforces) . . . 1061

20.4 Problema Fixed-Length Paths I (CSES) (din nou) . . . 1065
20.5 Problema Xenia and Tree (Codeforces) . . . 1068
20.6 Problema Flareon (Lot 2017) . . . 1075
20.7 Problema Digit Tree (Codeforces) . . . 1078
20.8 Problema Cucuruz (Concurs RoAlgo #7) . . . 1087
20.9 Problema LaNaic (Lot 2024) . . . 1090

21 Elemente de teoria numerelor 1098
21.1 Suma unei progresii aritmetice . . . 1098
21.2 Problema Exponentiation (CSES) . . . 1101
21.3 Problema Exponentiation II (CSES) . . . 1101
21.4 Problema Eugene and Big Number (CodeChef) . . . 1102
21.5 Problema Counting Divisors (CSES) . . . 1103
21.6 Problema Common Divisors (CSES) . . . 1104
21.7 Problema Sum of Divisors (CSES) . . . 1105
21.8 Problema Prime Multiples (CSES) . . . 1106
21.9 Problema List of Integers (Codeforces) . . . 1108
21.10 Problema Small GCD (Codeforces) . . . 1111
21.11 Problema Secvxor (Baraj ONI 2023) . . . 1113
21.12 Problema Afacere (Lot 2009) . . . 1117
21.13 Problema Playing with GCD (SPOJ) . . . 1121
21.14 Problema Power Tower (Codeforces) . . . 1122
21.15 Problema GCD Extreme (SPOJ) . . . 1124
21.16 Problema Counting Coprime Pairs (CSES) . . . 1125
21.17 Problema Coprime Sequences (Codeforces) . . . 1129
21.18 Problema Coprime Triples (CodeChef) . . . 1131
21.19 Problema Coprime Arrays (Codeforces) . . . 1133
21.20 Problema Mocha and Stars (Codeforces) . . . 1135
21.21 Problema Pastile (Baraj ONI 2021) . . . 1137
21.22 Problema Beautiful Numbers (Codeforces) . . . 1139

22 Teorema chinezească a resturilor 1142
22.1 Problema Koxia and Number Theory (Codeforces) . . . 1142
22.2 Problema Remainders Game (Codeforces) . . . 1144
22.3 Problema Decipher the String (Codeforces) . . . 1144
22.4 Problema Two Chandeliers (Codeforces) . . . 1148
22.5 Problema GCD Guess (Codeforces) . . . 1151

23 Elemente de bază în combinatorică 1158
23.1 Problema Binomial Coefficients (CSES) . . . 1158
23.2 Problema Creating Strings II (CSES) . . . 1159
23.3 Problema Distributing Apples (CSES) . . . 1161

23.4 Problema Christmas Party (CSES) . . . 1162
23.5 Problema Bracket Sequences I (CSES) . . . 1164
23.6 Problema Bracket Sequences II (CSES) . . . 1165
23.7 Problema Counting Necklaces (CSES) . . . 1167
23.8 Problema Counting Grids (CSES) . . . 1168
23.9 Problema Number of Permutations (Codeforces) . . . 1169
23.10 Problema Counting Factorizations (Codeforces) . . . 1171
23.11 Problema Monocarp and the Set (Codeforces) . . . 1173
23.12 Problema Devu and Flowers (Codeforces) . . . 1175
23.13 Problema Lengthening Sticks (Codeforces) . . . 1178
23.14 Problema Shuffle (Codeforces) . . . 1179
23.15 Problema Manhattan (Lot 2017) . . . 1183

24 Combinatorică - rangul permutărilor și al combinărilor 1190
24.1 Rangul permutărilor . . . 1190
24.2 Rangul combinărilor . . . 1197
24.3 Problema Misha and Permutation Summation (Codeforces) . . . 1200
24.4 Problema Inversion Sort (SPOJ) . . . 1202
24.5 Problema Four Chips (SPOJ) . . . 1204
24.6 Problema Long Permutation (Codeforces) . . . 1208
24.7 Problema Arbperm2 (NerdArena) . . . 1210
24.8 Problema Hipersimetrie (ONI 2019, clasele 11-12) . . . 1212
24.9 Problema Trim (Baraj ONI 2023) . . . 1215
24.10 Problema Bubblesort (Lot 2010) . . . 1218
24.11 Problema Valentine's Day (infO(1) Cup 2023) . . . 1222

25 Rezolvarea recurențelor liniare prin exponențiere rapidă 1227
25.1 Problema Al k-lea termen Fibonacci (Infoarena) . . . 1227
25.2 Problema Iepuri (Infoarena) . . . 1228
25.3 Problema Recurenta2 (Infoarena) . . . 1230
25.4 Problema Graph Paths I (CSES) . . . 1232
25.5 Problema Hprob (Infoarena) . . . 1234
25.6 Problema Chef and Riffles (CodeChef) . . . 1236
25.7 Problema Zid (ONI 2024, clasele 11-12) . . . 1238
25.8 Problema Stairs and Lines (Codeforces) . . . 1239
25.9 Problema Marfa (Lot 2014) . . . 1241
25.10 Problema Options (IIOT 2017-2018 runda internațională) . . . 1245

26 Geometrie - Elemente de bază 1248
26.1 Problema Cuiburi (Baraj ONI 2010) . . . 1248
26.2 Problema Ace (OJI 2017, clasa a 9-a) . . . 1250
26.3 Problema Baba Oarba (Lot 2016) . . . 1252

26.4 Problema Arhitect (OJI 2023, clasa a 10-a) . . . 1254
26.5 Problema Elicoptere (OJI 2016, clasele 11-12) . . . 1257
26.6 Problema Arpa and an Exam About Geometry (Codeforces) . . . 1260
26.7 Problema Part (Baraj ONI 2006) . . . 1260
26.8 Problema Bear and Floodlight (Codeforces) . . . 1263
26.9 Problema TrapEZZ (Moisil++ 2023, clasele 11-12) . . . 1265
26.10 Problema Terenuri (Baraj ONI 2011) . . . 1268
26.11 Problema Metin2 (Finala IIOT 2021-22) . . . 1271

27 Geometrie - Algoritmi specifici 1275
27.1 Problema Copaci (Infoarena) . . . 1275
27.2 Problema Emptri (Lot 2015) . . . 1276
27.3 Problema Înfășurătoare convexă (Infoarena) . . . 1277
27.4 Problema Înfășurătoare convexă (NerdArena) . . . 1279
27.5 Problema Testament (Lot 2011) . . . 1281
27.6 Problema Magic (JBOI 2023) . . . 1283
27.7 Problema Triangular Queries (CodeChef) . . . 1285
27.8 Problema Nuke (Lot 2010) . . . 1288
27.9 Problema Hill Walk (USACO Gold 2013) . . . 1291
27.10 Problema Ydist (Lot 2014) . . . 1293
27.11 Problema Fossil in the Ice (CodeChef) . . . 1295
27.12 Problema Rubarba (Infoarena) . . . 1298

28 Grafuri - Arbori parțiali minimi 1302
28.1 Problema Arbore parțial de cost minim (Infoarena) . . . 1302
28.2 Problema Autobuze3 (AGM 2015) . . . 1306
28.3 Problema Rusuoaica (FMI No Stress 9 Warmup) . . . 1308
28.4 Problema MinOr Tree (Codeforces) . . . 1310
28.5 Problema 0-1 MST (Codeforces) . . . 1313
28.6 Problema Minimum Spanning Tree for Each Edge (Codeforces) . . . 1316
28.7 Problema Apm2 (Infoarena) . . . 1319
28.8 Problema Edges in MST (Codeforces) . . . 1321
28.9 Problema Envy (Codeforces) . . . 1325
28.10 Problema Gift (Codeforces) . . . 1328
28.11 Problema DFS Trees (Codeforces) . . . 1333

29 Grafuri - Conectivitate 1337
29.1 Problema Course Schedule (CSES) . . . 1337
29.2 Problema Book (Codeforces) . . . 1340
29.3 Problema Componente tare conexe (Infoarena) . . . 1345
29.4 Problema Mr. Kitayuta's Technology (Codeforces) . . . 1351
29.5 Problema Ralph and Mushrooms (Codeforces) . . . 1353

29.6 Problema Bertown Roads (Codeforces) . . . 1356
29.7 Problema Tourist Reform (Codeforces) . . . 1358
29.8 Problema We Need More Bosses (Codeforces) . . . 1360
29.9 Problema Forbidden Cities (CSES) . . . 1362
29.10 Problema Case of Computer Network (Codeforces) . . . 1364
29.11 Problema Dog Trick Competition 2 (IIOT 2023-2024 runda 4) . . . 1367
29.12 Problema Regate (ONI 2022, clasele 11-12) . . . 1370

30 Grafuri - Distanțe 1375
30.1 Problema Bellman-Ford (Infoarena) . . . 1375
30.2 Problema Dijkstra (Infoarena) . . . 1379
30.3 Problema Monsters (CSES) . . . 1387
30.4 Problema Shortest Routes II (CSES) . . . 1390
30.5 Problema High Score (CSES) . . . 1391
30.6 Problema Flight Discount (CSES) . . . 1393
30.7 Problema Three States (Codeforces) . . . 1395
30.8 Problema Graph and Graph (Codeforces) . . . 1398
30.9 Problema Termite (Lot 2014) . . . 1401
30.10 Problema President and Roads (Codeforces) . . . 1408
30.11 Problema Nastya and Unexpected Guest (Codeforces) . . . 1412
30.12 Problema Minimum Path (Codeforces) . . . 1415
30.13 Problema Shortest Path (Codeforces) . . . 1420

31 Grafuri - Drumuri euleriene 1426
31.1 Problema Ciclu Eulerian (Infoarena) . . . 1426
31.2 Problema De Bruijn Sequence (CSES) . . . 1430
31.3 Problema Tanya and Password (Codeforces) . . . 1432

32 Grafuri - Fluxuri 1436
32.1 Problema Download Speed (CSES) . . . 1436
32.2 Problema Police Chase (CSES) . . . 1447
32.3 Problema Distinct Routes (CSES) . . . 1450
32.4 Problema Soldier and Traveling (Codeforces) . . . 1453
32.5 Problema Fox and Dinner (Codeforces) . . . 1457
32.6 Problema Petya and Graph (Codeforces) . . . 1462
32.7 Problema Asfalt (Lot 2011) . . . 1465
32.8 Problema Echipe (Lot 2025) . . . 1469

33 Grafuri - Cuplaje 1477
33.1 Problema School Dance (CSES) . . . 1477
33.2 Problema Revoluție (ONI 2009, clasele 11-12) . . . 1481
33.3 Problema Valuable Paper (Codeforces) . . . 1483

33.4 Problema Teroriști (Baraj ONI 2010) . . . 1486
33.5 Problema Brevity Is the Soul of Wit (Codeforces) . . . 1490
33.6 Problema Access Levels (Codeforces) . . . 1493
33.7 Problema Pswap (Baraj ONI 2021) . . . 1496

34 Grafuri - Fluxuri de cost minim 1502
34.1 Problema Parcel Delivery (CSES) . . . 1502
34.2 Problema Task Assignment (CSES) . . . 1504
34.3 Problema Match (IIOT 2019-2020 runda 3) . . . 1507
34.4 Problema Hoață (Baraj ONI 2022) . . . 1511
34.5 Problema Lifts (IATI Shumen 2022) . . . 1518

35 Șiruri - Algoritmul Rabin-Karp 1524
35.1 Problema Radio (Lot 2010) . . . 1524

36 Șiruri - Algoritmul Knuth-Morris-Pratt 1527
36.1 Problema Bart (NerdArena) . . . 1527
36.2 Problema Cifru (Baraj ONI 2006) . . . 1528
36.3 Problema Anthem of Berland (Codeforces) . . . 1530

37 Șiruri - Quicksort ternar 1533
37.1 Problema Cuvinte (ONI 2021, clasele 11-12) . . . 1533
37.2 Problema Sabin (Baraj ONI 2015) . . . 1535

38 Șiruri de sufixe 1539
38.1 Problema Carry Bit (IIOT 2023-2024 runda 3) . . . 1539
38.2 Problema Periodic Words (IIOT 2023-2024 runda 1) . . . 1543
38.3 Problema Phone Book (Finala IIOT 2022-2023) . . . 1549

39 Arbori trie 1554
39.1 Problema Enigma (Lot 2011) . . . 1554
39.2 Problema Maximum Xor Subarray (CSES) . . . 1555
39.3 Problema Esoteric Bovines (IIOT 2023-2024 runda internațională) . . . 1557
39.4 Problema Cipher (IIOT 2022-2023 runda internațională) . . . 1564

40 Probleme diverse 1568
40.1 Problema Liars (Baraj ONI 2025) . . . 1568
40.2 Problema Pavare infinită (Lot 2025) . . . 1573
40.3 Problema Triunghi (Lot 2024) . . . 1578
40.4 Problema Subiectul I („Nervi”) (Lot 2024) . . . 1582
40.5 Problema Subiectul III („Single crossing”) (Lot 2024) . . . 1590
40.6 Problema Gurger Brill (Lot 2024) . . . 1596
40.7 Problema Takara (Baraj ONI 2019) . . . 1600

40.8 Problema Tornade (Baraj ONI 2025) 1603
40.9 Problema As de Spade (Lot 2024) 1608
40.10 Problema Scurtcircuit (Lot 2025) 1612
40.11 Problema Lungcircuit (Lot 2025) 1618
40.12 Problema Cal-cule (Lot 2025) 1625
40.13 Problema 6 de Pentagrame (Lot 2024) 1629

Partea I

Principii de programare

Programarea este o activitate profund socială. De cînd există software, oamenii au schimbat cod între ei și au învățat unii de la alții cum să scrie cod. Programele pot trece prin mîna mai multor programatori concomitent și succesiv, uneori continuînd să existe la zeci de ani după ce dezvoltatorii originali au părăsit echipa.

Prin contrast, programarea competitivă este o activitate mai singuratică. De cele mai multe ori programele sînt dezvoltate individual sau în echipe mici, sînt scurte și sînt efemere. Rostul lor primar dispare odată cu încheierea concursului. Acest stil duce la obiceiuri de programare foarte diferite și adesea nocive.

Este fantastic că jurizarea programelor este automatizată, rapidă și ferită de slăbiciunea umană. Aceasta ne diferențiază de concursurile altor materii. Dar, tocmai de aceea, profesorii tind să se declare mulțumiți cînd elevii iau 100 de puncte sau *Accepted* pe vreun site, fără să le citească niciodată codul-sursă.

În plus, una dintre părțile cele mai frumoase ale algoritmicii este că algoritmii sînt corecți în mod demonstrabil. Paradoxal, tocmai asta duce la cod mai urît, căci nici profesorii, nici elevii nu prea pun preț pe detaliile de implementare și pe lizibilitate.

Nu în ultimul rînd, la nivel de liceu există o ruptură între mediul academic și mediul industrial (la facultate legătura este cu mult mai strînsă). Majoritatea profesorilor de informatică de liceu nu au citit și nu au scris niciodată cod practic și nu simt nevoia să codeze clar.

Sper ca în această parte să vă conving că stilul de codare, precum și adoptarea unor bune practici de inginerie software considerate normale în industrie, vă pot ajuta și în programarea competitivă. Nu este obligatoriu să începeți cu această parte, dar sper să reveniți curînd la ea. Dacă ați ales să dedicați niște zeci sau sute de ore acestei cărți, înseamnă că vă doriți să creșteți ca programatori, iar atunci lăsați-mă și pe mine să decid cum se va întîmpla asta, nu alegeți doar ce credeți voi că este suficient. 🙂

Capitolul 1

Cod curat și cod spaghetti

Pentru învățarea programării competitive, Internetul este o binecuvîntare și un blestem, mai exact 10% binecuvîntare și 90% blestem. Acest lucru nu ar trebui să ne surprindă. Regăsim această proporție pe Internet în orice alt domeniu: știri, știință, informații medicale etc.

De aceea, pe de o parte este fantastic că Internetul abundă în informații despre programarea competitivă. Pe de altă parte, mulți elevi preiau din această supă culturală un stil de codare neîngrijit, care duce la programe încîlcite, care pot conține buguri nedetectabile. Este un mod gratuit și absolut evitabil de a rata un concurs.

Filozofia mea este că, la nivelul de ONI și Baraj ONI la care se plasează acest curs, ideile problemelor nu sînt principala dificultate. Desigur, la loturi și la concursuri internaționale nivelul este altul. Fiecare dintre noi s-a gîndit uneori 2-3 zile o problemă de lot și tot n-a știut să o rezolve. Dar la ONI și la Baraj ONI contează mai mult **capacitatea de a exprima clar, în cod, niște idei teoretice de dificultate medie**. Am constatat că majoritatea elevilor nu pierd puncte pentru că n-au știut un anume algoritm sau o anume structură de date, ci pentru că n-au reușit să le exprime corect în cod și s-au lovit de buguri sau de ineficiență (TLE).

Astfel ajungem la observația că două programe care rezolvă aceeași problemă pot diferi radical ca lizibilitate, robustețe, ușurință a implementării. Să vedem niște exemple concrete și să încercăm să deducem niște principii generale pentru a scrie cod curat.

1.1 Exemplul 1

Ce face următorul program?

```
#include <stdio.h>

int v[2][100001];
int n;

int main() {
  scanf("%d",&n);
```

```
  for (int i=0;i<n;i++)
    scanf("%d",&v[0][i]);
  for (int i=0;i<n;i++)
    scanf("%d",&v[1][i]);
  int x=-1;
  for(int i=0;i<n;i++){
    if(x==-1 && v[0][i]!=v[1][i]){
      if(v[0][i]<v[1][i]){
        x=0;
      }else{
        x=1;
      }
    }
    if(x==-1){
      printf("%d ",v[0][i]);
    }else{
      printf("%d ",v[x][i]);
    }
  }
  printf("\n");
  return 0;
}
```

Programul este relativ criptic. Observațiile-cheie sînt:

- Programul citește doi vectori în `v[0]` și `v[1]`.
- Variabila `x` are valoarea -1 cît timp vectorii sînt egali pe prefix.
- Cînd (și dacă) programul găsește prima diferență, `x` ia valoarea 0 sau 1 pentru a indica vectorul mai mic.
- Programul tipărește elementul curent din vectorul identificat ca fiind mai mic (sau din vectorul 0 cît timp prefixele sînt egale).

Cu alte cuvinte, programul compară doi vectori de aceeași lungime `n` și îl tipărește pe cel mai mic lexicografic. Deși funcționează, el este greoi de citit și de înțeles. Este ceea ce, colocvial, numim **cod spaghetti** (engl. *spaghetti code*). Iată un alt program care face același lucru:

```
#include <stdio.h>

const int MAX_SIZE = 100'000;

int v[MAX_SIZE + 1], w[MAX_SIZE + 1];
int n;

void readArrays() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
  }
```

```
  for (int i = 0; i < n; i++) {
    scanf("%d", &w[i]);
  }
}

int* getSmaller() {
  v[n] = 0;
  w[n] = 1; // santinele pentru egalitate perfectă

  int i = 0;
  while (v[i] == w[i]) {
    i++;
  }

  return (v[i] < w[i]) ? v : w;
}

void writeArray(int* v) {
  for (int i = 0; i < n; i++) {
    printf("%d ", v[i]);
  }
  printf("\n");
}

int main() {
  readArrays();
  int* smaller = getSmaller();
  writeArray(smaller);

  return 0;
}
```

De ce consider a doua variantă mai „curată”? De la nivel macro spre micro, iată cîteva motive:

- Funcția `main` listează, ca într-un cuprins, capitolele programului (funcțiile).
- Fiecare capitol are un nume clar, care ține loc de orice comentariu.
- Fiecare capitol face un singur lucru, cel promis în titlu.
- În particular, tipărirea nu mai poluează codul comparării, ci este separată de aceasta.
- Fiecare capitol este ușor de cuprins cu privirea.
- Dacă mă interesează un capitol anume, nu am nevoie să înțeleg ce se întîmplă în celelalte.
- Programul nu mai depinde de stocarea indexată a vectorilor în `v[0]` și `v[1]`.
- Programul elimină cazul particular (egalitatea perfectă) folosind o santinelă.
- Programul subliniază prezența santinelei izolînd dimensiunea maximă a intrării (100 000) de elementul suplimentar (+1).
- Astfel, compararea propriu-zisă devine banală: avansăm prin vector pînă la prima diferență.
- Spațierea în jurul punctuației și liniile goale între blocuri fac codul mai lizibil.

Singurul aspect la care a doua versiune pare că „pierde” este lungimea. Programul are 44 de linii

față de 29. Dar diferența între liniile de cod „real”, fără comentarii și linii goale, este de 34 de linii la 27. Ca fapt divers, puteți obține acest număr cu comanda Linux:

```
cpp -fpreprocessed -P program.cpp | wc -l
```

Oricum am măsura, diferența de 15 linii sau 7 este un preț justificat. Scurtimea programului nu trebuie să fie un scop, după cum în tipografie scopul nu este să înghesuim o carte pe un număr cît mai mic de pagini, ci să facem textul lizibil. De aici decurg alegerea unor fonturi lizibile și suficient de mari, spațierea între rînduri, marginile ample pe conturul paginii etc.

1.2 Exemplul 2

Iată încă un exemplu. Ce face această funcție?

```
int mysteryFunction(int* v, int n) {
  int ans = 0, i = 0, h;

  while ( i < n ) {
    ans++;
    h = v[i];
    i++;
    while ( i < n && h == v[i] ) {
      i++;
    }
  }

  return ans;
}
```

De data aceasta, codul este bine formatat, dar neclaritățile apar la nivel logic. Iată o altă versiune:

```
int countDistinct(int* v, int n) {
  int count = 1; // v[0] este distinct

  for (int i = 1; i < n; i++) {
    if (v[i] != v[i - 1]) {
      count++;
    }
  }

  return count;
}
```

Consider această versiune mai clară, întrucît există un singur nivel de imbricare (`for`), față de prima versiune (`while` în `while`). Cred că și conceptual această versiune este mai clară: un element contează la numărătoare dacă este diferit de anteriorul.

Cum facem un program mai clar? Nu există o rețetă perfectă. Bunăoară, am stat în dubiu dacă să înlocuiesc liniile 5-7 cu forma de mai jos. Aș putea fi convins de ambele abordări, dar am preferat-o pe prima pentru că o poate înțelege și o persoană care nu vorbește limba *leet*.

```
    count += (v[i] != v[i - 1]);
```

Un cuvînt despre afișare. Eu aș implementa-o astfel:

```
void writeAnswer(int answer) {
  printf("%d\n", answer);
}
```

Cînd scriu astfel de cod, și cînd vă cer și vouă să scrieți astfel, sînt conștient că lupt împotriva unor ani de pregătire într-un spirit minimalist. Dar, logic vorbind, tipărirea este un capitol separat, un bloc logic separat. Apelul la `printf()` nu are ce căuta în `main()`, care se preocupă de un nivel de abstracție mai înalt. De exemplu, secțiunea *Acknowledgements* (mulțumirile) dintr-o carte au poate una-două fraze. Dar nu le punem direct în cuprins, ci creăm o secțiune aparte pentru ele.

1.3 Exemplul 3

Codul următor face parte dintr-un program mai lung, care rezolvă problema Shuffle. Vă invit să urmărim doar variabila `onceAdd`. Ce putem spune despre scopul său?

```
onceAdd = i = j = count1 = ans = 0;
while (i < n) {
    while (j < n && (count1 < k || s[j] == '0')) {
        count1 += (s[j] == '1');
        ++j;
    }

    if (count1 == k && j == n && s[i] == '1') { // last subseq
        // cout << i << " " << j << "-LAST---" << numberWays1(count1 - 1, j - i - 1) << "\n";
        ans = (ans + numberWays(count1 - 1, j - i - 1)) % MOD;
        onceAdd = (numberWays(count1 - 1, j - i - 1) > onceAdd ? 1 : onceAdd);
    }

    if (count1 == k) {
        // cout << i << " " << j << "----";
        if (s[i] == '0') {
            // cout << numberWays1(count1 - 1, j - i - 1) << "\n";
            ans = (ans + numberWays(count1 - 1, j - i - 1)) % MOD;
            onceAdd = (numberWays(count1 - 1, j - i - 1) > onceAdd ? 1 : onceAdd);
        }
        else {
            // cout << numberWays1(count1, j - i - 1) << "\n";
            ans = (ans + numberWays(count1, j - i - 1)) % MOD;
```

```
            onceAdd = (numberWays(count1 - 1, j - i - 1) > onceAdd ? 1 : onceAdd);
            count1--;
        }
    }


    ++i;
}
if (onceAdd == 0)
    ans = 1;
```

Codul are diverse defecte, dar l-am reprodus integral tocmai ca să evidențiez că aceste defecte îngreunează înțelegerea:

- DRY (*Don't Repeat Yourself*);
- cod duplicat, dar cu mici diferențe (cititorul trebuie să citească toate instanțele cu maximum de atenție);
- cod vestigial care trebuia șters mai degrabă decît comentat;
- `while` în loc de `for` pentru variabila `i`.

Cu puțin efort înțelegem: `onceAdd` începe de la 0 și poate deveni 1 cînd o anumită condiție este îndeplinită. Dar era mai bine ca autorul să faciliteze această înțelegere. Dacă variabila este booleană, ea trebuie declarată ca atare și modificată cu operatori booleeni, nu cu un operator ternar alambicat.

Mai ales, observăm că, odată ce `onceAdd` devine 1, ea nu se mai modifică (ambele ramuri ale operatorului ternar `?:` îi atribuie valoarea 1), dar condiția operatorului continuă să fie evaluată de fiecare dată. Aceasta este o risipă.

În procesul de rescriere, autorul ar fi putut să scoată factor comun expresiile comune, care ar elimina repetiția (și recalculările care costă timp!). Am fi ajuns, cel puțin, la forma de mai jos, în care ciclul de viață al lui `onceAdd` este mult mai clar.

```
j = count1 = ans = 0;
bool onceAdd = false;
for (int i = 0; i < n; i++) {
    while (j < n && (count1 < k || s[j] == '0')) {
        count1 += (s[j] == '1');
        ++j;
    }

    if (count1 == k && j == n && s[i] == '1') { // last subseq
        int comb = numberWays(count1 - 1, j - i - 1);
        ans = (ans + comb) % MOD;
        onceAdd |= (comb > 0);
    }

    if (count1 == k) {
```

```
        if (s[i] == '0') {
            int comb = numberWays(count1 - 1, j - i - 1);
            ans = (ans + comb) % MOD;
            onceAdd |= (comb > 0);
        }
        else {
            int comb = numberWays(count1, j - i - 1);
            ans = (ans + comb) % MOD;
            comb = numberWays(count1 - 1, j - i - 1);
            onceAdd |= (comb > 0);
            count1--;
        }
    }
}
if (!onceAdd) {
    ans = 1;
}
```

1.4 Exemplul 4

Următoarea funcție testează intersecția a două segmente pe axa Ox.

```
struct Interval {
    int left, right;
};

bool intersect(pair<int, int> a, pair<int, int> b) {
    return ( ( a.left >= b.left && a.left < b.right ) ||
             ( a.right > b.left && a.right <= b.right ) ) ||
           ( ( b.left >= a.left && b.left < a.right ) ||
             ( b.right > a.left && b.right <= a.right ) );
    return 0;
}
```

Ce consider spaghetti la acest cod este:

1. Diavolul se ascunde în detalii. Funcția va returna `false` pentru $[1, 2] \cap [2, 3]$. Așadar, ea testează intersecția strictă (cel puțin două puncte). Funcția nu clarifică acest lucru.
2. Numărul de condiții „pare” nenecesar de mare. La o analiză atentă, el poate fi redus.
3. Funcția mapează tacit un `std::pair` peste un `struct Interval`. Nici nu mi-a fost clar de ce `left` și `right` se compilează! Iar dacă ulterior modificăm `struct`-ul, programul nu se va mai compila.

Aș clarifica (1) denumind funcția `strictIntersect()` sau altcumva. Pentru (2), putem reformula logica astfel: dorim ca ambele intervale să se deschidă (întîi `a` și apoi `b` sau invers, nu contează), apoi ambele să coexiste pe o distanță nedegenerată, apoi ambele intervale să se închidă. Iar pentru (3), merită să folosim tipul `Interval`, dacă tot l-am declarat.

Rezultă codul scurt și clar:

```
bool strictIntersect(Interval a, Interval b) {
    return (max(a.left, b.left) < min(a.right, b.right));
}
```

Sau, fără funcții externe:

```
bool strictIntersect(Interval a, Interval b) {
    return (a.left < b.right) && (b.left < a.right);
}
```

1.5 Cîteva principii pentru cod curat

Să încercăm să formulăm mai clar observațiile anterioare și cîteva în plus. Puteți citi cărți despre așa ceva. De exemplu, vă recomand cu căldură Clean Code de Robert C. Martin. Este o carte pe care o puteți citi în două-trei zile, plină de exemple și de anecdote haioase.

Iată un citat din această carte.

> It is not enough for code to work. Code that works is often badly broken. Programmers who satisfy themselves with merely working code are behaving unprofessionally. They may fear that they don't have time to improve the structure and design of their code, but I disagree. Nothing has a more profound and long-term degrading effect upon a development project than bad code. Bad schedules can be redone, bad requirements can be redefined. Bad team dynamics can be repaired. But bad code rots and ferments, becoming an inexorable weight that drags the team down. Time and time again I have seen teams grind to a crawl because, in their haste, they created a malignant morass of code that forever thereafter dominated their destiny.
>
> Robert C. Martin, *Clean Code*

Știu, acesta pare un avertisment pentru un viitor îndepărtat, după facultate, cînd vă veți angaja. **Dar există cod suficient de murdar încît chiar și autorul codului se încurcă în el după 30 de minute.** Haideți să facem următorul exercițiu. Din experiența voastră, de ce vi se întîmplă să pierdeți puncte?

1. Nu am știut algoritmul sau structura de date necesară.
2. Am scris cod prea lent (TLE).
3. Am consumat prea multă memorie (MLE).
4. Am avut un bug / răspuns greșit / *core dump*.
5. Nu am reușit să o fac să meargă nici măcar local, pe teste mici.
6. M-am încurcat în programare și nu am reușit să o duc la capăt.
7. M-am ambiționat pe soluția de 100 de puncte și am luat mai puțin decît dacă făceam un subtask.

8. M-am ambiționat pe problema asta, în loc să trec la altă problemă.

Capitolul de față este prima piatră înspre cucerirea motivelor 4-7 de mai sus. Suspectez că vă întîlniți cu ele relativ des.

1.5.1 Formatare

Cred că nu este cazul să discutăm despre indentare, spațiere etc. Vom citi (foarte) pe sărite ghidul de stil C++ al lui Google.

1.5.2 Limbajul de programare

Alegeți nume grăitoare

A denumi lucruri și ființe este un privilegiu și o plăcere a vieții.

Folosiți nume clare pentru variabile. Dacă aveți un vector `v` și un vector `w` unde `w[i]` reține poziția primului element mai mare decît `v[i]` aflat la stînga lui `i`, poate că ar fi mai bine ca `w` să se numească `leftGreater` sau `prevBigger` sau chiar și `left`. Orice este mai bine decît `w`.

Dacă aveți o funcție care copiază un obiect în alt obiect (șiruri, vectori etc.), nu le denumiți `a` și `b`. Numiți-le `src` și `dest`.

Desigur, nu sărim calul. Dacă ne denumim variabila `maximulDinVectorulCititLaIntrare`, probabil vom dăuna clarității. Dar, dacă ne fixăm claritatea ca scop, deciziile devin relativ logice.

Corolar: Evitați „maparea în memorie”

Dacă aveți 10 variabile `a`, `b`, ..., `j` într-un cod de 50 de linii, îi cereți cititorului (care, repet, puteți fi chiar voi peste 30 de minute!) să țină minte rostul fiecărei variabile. Practic, cititorul trebuie să mențină o tabelă hash în memorie.

Corolar: Glumiți cu măsură

A fi programator înseamnă a avea the hacker spirit. Joaca inteligentă impregnează tot ce facem.

La polul opus se află frustrarea, pe care o înțeleg și uneori o împărtășesc. O funcție care nu vrea să meargă te poate aduce la disperare. Ajungi să o peticești cu `if`-uri și steguleț, mereu mai e nevoie de o variabilă în plus, iar soluția pare aproape, dar îți scapă printre degete. Exasperat, denumești a 27-a variabilă `kkt`.

Ambele situații sînt OK, cîtă vreme numele nu dăunează clarității programului. Este preferabil ca valoarea maximă să se numească `max`, `maxVal` sau `vmax`, nu `ceva` sau `boss`.

Corolar: Nu păcăliți cititorul

La o temă anterioară, un elev și-a denumit o clasă `aibe`, dar ea implementa un arbore de intervale.

Creați un limbaj specific problemei

Limbajul de programare este o convenție. El ne îndrumă sintaxa, dar numai pînă la un punct. Numele de variabile și deciziile de design ne aparțin. Ele sînt modurile în care **noi extindem limbajul de programare**, în funcție de nevoile programului. Numim aceasta **limbaj specific domeniului** (engl. *domain-specific language*). Folosiți-l din plin!

Dacă vectorul este de puncte, denumiți-l `p` sau `pt` sau `points`.

Dacă problema este despre monștri care au o culoare și un număr de vieți, creați un `struct monster` cu cîmpurile `color` și `lives`. În niciun caz nu vă mulțumiți cu un `std::pair`! Vezi și structured binding.

Dacă într-o buclă alegeți maximul pe baza unui criteriu complex, atunci izolați compararea într-un operator sau într-o funcție de comparare. Exemplu: sortarea polară a unor puncte, cu departajare după distanța față de origine.

Dacă nu construiți un limbaj specific domeniului, dacă vă denumiți vectorii `v`, `v1`, `v2` și `v3`, este ca și cînd limbajul C ar cere cuvintele-cheie `sparrow` și `breathe` în loc de `for` și `while`. Ar fi un limbaj ilogic.

Aceasta se aplică și la nume, dar **și la nivel mai înalt**. Adaptați structurile de date și tipurile de date la nevoile problemei! Exemplu clasic: nu folosiți varianta recursivă de arbori de segmente pentru o problemă care face doar actualizări punctuale. Varianta iterativă este mai scurtă, mai rapidă, mai clară și nu induce așteptarea falsă că vor exista actualizări pe interval. (Vom detalia în lecția respectivă).

„Toate îmi sînt îngăduite, dar nu toate îmi sînt de folos”

Nu vă simțiți obligați să folosiți cele mai obscure posibilități ale unui limbaj doar fiindcă ele există. Dacă puteți programa curat și eficient fără *template*-uri odioase, moștenire multiplă etc., feriți-vă de ele.

1.5.3 Funcții

Funcțiile trebuie să fie scurte

Obișnuiam să zic că „funcțiile trebuie să încapă într-un ecran”. Dar, de cînd cu Ctrl-minus, toate funcțiile încap într-un ecran. Azi mă panichez cînd funcția mea depășește 10 linii și simt o nevoie imperioasă să o fragmentez.

Există contraargumente, dar o regulă bună este: dacă o funcție face două lucruri **sau** dacă operează la mai multe niveluri de abstracție, este momentul să o spargeți.

Funcțiile trebuie să facă un singur lucru

Cu cît înghesuiți mai mult în aceeași funcție, cu atît mai misterioasă va deveni ea pentru cititor. Cititorului îi va fi mai greu să găsească bucata de cod care îl interesează.

Desigur, un program cap-coadă are multe lucruri de făcut. Dacă algoritmul nostru constă din preprocesare + o parcurgere, atunci undeva trebuie să existe două apeluri de funcții, să zicem `preprocess()` și `dfs()`.

Funcțiile trebuie să opereze la un singur nivel de abstracție

Să considerăm o funcție care calculează cel mai lung subșir comun al două șiruri. Nu este treaba acelei funcții să adauge `'\n'` la rezultat în vederea tipăririi. Nu este nici treaba funcției `main()`. Adăugarea lui `'\n'` îi revine funcției de tipărire.

Similar, dacă funcția `main()` apelează 3-4 funcții de nivel înalt (citire, sortare, procesare, tipărire), nu este bine să inserăm, în `main()`, o buclă `for`, să spunem pentru preprocesarea vectorului după citire. Acel nivel de abstracție este mai jos și trebuie extras într-o funcție, să-i spunem `preprocess()`.

Nu exagerați cu numărul de argumente

Pot accepta că o funcție necesită 4-5-6 argumente. Dacă lucrurile se opresc aici, este OK. Dar dacă funcția apelează altă funcție care apelează altă funcție, și toate funcțiile declară aceleași argumente, acesta este o formă de repetiție.

Mai mult, redundanța indică o greșeală de organizare. Dacă acele variabile circulă mereu împreună, este un indiciu că ele sînt conectate logic. Ar trebui grupate într-un `struct`, pe care apoi îl putem trimite ca (unic) parametru.

Funcțiile nu trebuie să aibă efecte secundare

Dacă denumiți o funcție `sortArray()`, ea trebuie să facă doar atît. Nu are voie să modifice alte variabile în afară de vectorul dat, cum ar fi să numere elementele distincte. Nu are voie nici măcar să modifice vectorul altfel decît prin sortare (de exemplu, să elimine minimul și maximul).

La acest cerc ne preocupă și eficiența, deci nu propun să facem parcurgeri suplimentare ale datelor doar ca să putem separa logic pașii. De exemplu, la o problemă de tip preprocesare + interogări, nu propun să stocăm răspunsurile în memorie doar ca să le putem afișa la final. Este OK să le afișăm pe parcurs.

Dar putem, cel puțin, să alegem un nume corect care să descrie tot ce face funcția.

1.5.4 Programe

DRY (*Don't Repeat Yourself*)

Leneșul mai mult aleargă. În situații cum ar fi parcurgerea unei matrice în cele patru direcții, copierea și adaptarea codului pare mai rapidă, dar poate duce la erori. Cititorul (toată lumea în cor: *care puteți fi chiar voi după 30 de minute*) trebuie să se asigure că bucățile similare nu diferă în moduri subtile.

Scoateți factor comun bucățile reutilizabile și parametrizați-le!

Doar programare structurată

Teorema programelor structurate a arătat, încă din 1966, că tot ce dorim să calculăm putem calcula cu programe structurate. Iar programarea structurată pune mai puțină presiune pe creierul cititorului, căci fluxul programului poate deveni neinteligibil cu cîteva instrucțiuni `break` și `continue`.

Desigur, nu trebuie să fim dogmatici. De exemplu, este OK să testăm 3 precondiții la intrarea într-o funcție și să avem 3 instrucțiuni `return` premature, decît să scriem tot corpul funcției indentat cu 3 niveluri.

Iar dacă funcția are cel mult 10 linii, probabil și o formă nestructurată va fi inteligibilă (exemplu: căutarea liniară într-un vector întreruptă cu `return`). Dar aș prefera să respectați întîi la sînge regulile, înainte să învățați cînd este OK să le încălcați.

Un program este ca un articol de ziar

Suma principiilor anterioare este aceasta: un program începe cu un rezumat (funcția `main()`) care listează operațiile principale ale programului. Din rezumat se disting capitole și secțiuni cu amănunte (celelalte funcții). Este regretabil că limbajul C cere să scriem funcțiile înaintea folosirii lor. Programele C trebuie citite de jos în sus.

Similar, un articol de ziar (bine scris) începe cu una sau două fraze care expun rezumatul știrii, apoi paragrafele următoare expandează acel rezumat.

Acest principiu îmi amintește de principiul din UX numit progressive disclosure. Nu-i cereți cititorului să înțeleagă totul deodată, ci dați-i șansa să ajungă rapid și fără mult efort mintal la bucata de cod care îl interesează.

Exemplu funcțional minim

În cazul temelor, odată ce aveți un program funcțional, aș vrea să eliminați din program tot codul comentat, codul de debug, variabilele nefolosite etc. Aceasta mă ajută pe mine, cititorul, să mă concentrez pe codul funcțional.

Ați trimis vreodată un *bug report* pe GitHub sau o întrebare pe StackOverflow? Toată lumea acolo vă va cere un *minimum working example*, degrevat de contextul mai larg în care ați întîlnit bugul. Așa și aici.

1.5.5 Comentarii

Comentariile nu se pot substitui codului curat

În această privință mi-am schimbat opinia în timp. Pe vremuri aș fi spus că un `main()` de 50 de linii, cu comentarii pentru fiecare bloc major, este la fel de bun ca un `main()` la nivel înalt care apelează 5 funcții de cîte 10 linii. Astăzi nu mai sînt la fel de sigur. Decît un comentariu ca `// calculează minimul din fereastra glisantă` prefer o funcție ca `minimumOverSlidingWindow()`, care

elimină nevoia de orice comentarii. Cartea *Clean Code* explică de ce a doua variantă este preferabilă: în esență, pentru că codul se modifică în timp, iar comentariile ajung să nu mai reflecte codul pe care îl adnotează.

Să considerăm codul spaghetti din Exemplul 1. Îl putem adnota cu comentarii; dar acele comentarii, deși îi explică cititorului ce se întîmplă, continuă să-i ceară un *leap of faith*. Prin comparație, codul curat se autoexplică.

Apoi, cum putem modifica un cod care, deși este bine comentat la nivel de bloc, este neinteligibil din interior? El este ca o cutie neagră.

Fără comentarii inutile

De exemplu:

```
// Sortează vectorul
std::sort(x, x + n);
```

Scrieți comentarii pentru decizii de design

Deciziile de design nu decurg din cod, cum ar fi **de ce** ați preferat o structură de date și nu pe alta, **de ce** ați decis să răsturnați un vector, ce reprezintă informația stocată în matricea unei programări dinamice etc. Aceste decizii merită documentate. Să zicem:

```
// Stocăm matricea și cele trei rotații cu 90°, 180° și 270°.
int mat[4][MAX_N][MAX_N];
```

Scrieți comentarii la nivelul cititorului

Comentariile trebuie să-i clarifice unui potențial cititor ce face codul. La acest cerc, avem cu toții un nivel suficient ca să recunoaștem dintr-o privire șabloane ca afișarea sau citirea unui vector, backtracking, căutare liniară, căutare binară, bordarea unei matrice, parcurgerea unui arbore etc. Acestea nu necesită comentarii.

1.6 Concluzii

Vă recomand să adoptați aceste sfaturi în programele voastre **de acasă**. Desigur, la olimpiadă să nu cumva să continuați să rafinați programul odată ce ați luat 100 de puncte. 🤣 Dar, dacă aplicăm zilnic aceste sfaturi, ele ne vor intra în reflex și ne vor ajuta să scriem programe care să meargă din prima. Mi se întîmplă frecvent să scriu un program care trece toate testele de la prima rulare. Ajung să introduc intenționat un `+1` la afișarea răspunsului, ca să fac programul să pice teste, ca să mă conving că nu am greșit ceva la invocare.

Citirea unui program nu trebuie să semene cu rezolvarea unui mister sau cu spargerea unui cifru. La profesorii de matematică întîlnesc uneori această gîndire ezoterică: soluțiile cu adevărat

frumoase sînt criptice, tocmai pentru ca doar cei inițiați, cei cu chemare spre matematică, să le poate înțelege. Nu sînt de acord cu această atitudine nici la matematică și cu atît mai puțin la noi, la informatică. Un progamator bun își scrie programele astfel încît sensul fiecărei linii să decurgă imediat din organizarea programului și din numele alese.

Capitolul 2

Bune practici în programare

Un program scris, dar care nu merge, nu este „99% gata, am doar un bug”. Depanarea poate ocupa între 0% și 99% din timpul de implementare, în funcție de calitatea codului. Iar la final, tot ce rămîne este punctajul.

Bugurile nu sînt o pedeapsă divină. Sînt o parte normală din viața de programator. Ce contează este că avem mecanisme prin care (1) să le reducem pe cît se poate frecvența și (2) să le depistăm sistematic și rapid cînd apar.

Din experiența mea de predare, la fiecare temă cam 2-3 elevi nu reușesc să ducă problema la capăt. Aceasta este OK, și este parte din datoria mea ca instructor să-i înarmez pe elevi cu uneltele necesare. Dar și elevii trebuie să își facă partea lor de treabă! În capitolul anterior am văzut cum putem scrie surse lizibile. În capitolul prezent vom învăța să scriem surse fără buguri și să le depanăm pe cele care, în ciuda celor mai bune intenții, se strecoară în program.

2.1 Invarianți de buclă

> Un program nu face ce vrei tu să facă, un program face ce îi spui tu să facă.
>
> – folclor

Iată o unealtă utilă în general, dar mai ales în situațiile cînd nu sîntem 100% siguri că programul nostru este corect și face ce ne dorim.

Definiția 1 (Invariant de buclă)**.** Un invariant de buclă este o proprietate a unei bucle `while` sau `do-while` care este adevărată la începutul și la sfîrșitul fiecărei iterații.

Condiția specificată în `while` este ea însăși un invariant de buclă, în mod tautologic: dacă nu ar fi adevărată, programul nu ar intra în buclă! Dar ne interesează proprietăți care nu sînt imediat aparente din cod. Dacă facem o promisiune pentru fiecare iterație a buclei, atunci trebuie să onorăm promisiunea doar la iterația curentă, iar prin inducție bucla va funcționa corect.

Iată cîteva exemple.

2.1.1 Căutarea maximului într-un vector

(Sursa: Wikipedia)

Am scris codul cu `while`, nu cu `for`, ca să evidențiez incrementarea variabilei de ciclu.

```
int m = v[0], i = 1;

while (i < n) {
  // m este maximul dintre v[0], ..., v[i - 1]
  if (v[i] > m) {
    m = v[i];
  }
  // Aici m este maximul dintre v[0], ..., v[i].
  i++;
  // Iar aici m este din nou maximul dintre v[0], ..., v[i - 1].
}
```

2.1.2 Interclasarea a doi vectori ordonați

```
a[m] = b[n] = INFINIT; // santinele
int i = 0, j = 0, k = 0;

while ((i < m) || (j < n)) {
  // 1. i + j < m + n. Nu neapărat interesant...
  // 2. k = i + j. Idem...
  // 3. Elementele a[0...i-1] și b[0...j-1] au fost interclasate în c[0...k-1].
  // 4. c[k-1] ≤ a[i] și c[k-1] ≤ b[j]
  if (a[i] < b[j]) {
    c[k++] = a[i];
    i++;
  } else {
    c[k++] = b[j];
    j++;
  }
  // Aceleași 4 proprietăți sînt adevărate și aici.
}
```

Invarianții 1 și 2, deși neinteresanți, ne dau totuși o idee de optimizare. Putem reformula condiția buclei ca `while (k < m + n)`, ceea ce, dacă vectorul a se termină primul, va scuti cîteva comparații. Mai mult, bucla are număr cunoscut de pași $(m + n)$, deci o putem scrie ca:

```
for (int k = 0; k < m + n; k++) {
  if (a[i] < b[j]) {
    c[k] = a[i++];
  } else {
    c[k] = b[j++];
  }
```

```
}
```

2.1.3 Alte exemple

Mulți dintre invarianții de mai jos i-am învățat cu toții în primii ani de programare, fără să ne gîndim explicit la ei. Dar în situațiile mai complexe ajută să ne gîndim exact ce dorim de la buclă.

- Sortarea prin selecție sau prin inserție: la pasul i, elementele `v[0]...v[i - 1]` sînt sortate.
- Bubble sort: la pasul i, cele mai din dreapta (cel puțin) i elemente sînt sortate.
- Căutare liniară: la pasul i, elementul căutat nu există în `v[0]...v[i-1]`.
- Algoritmul lui Euclid: la pasul (x, y), $\gcd(x, y)$ este egal cu $\gcd(a, b)$, unde (a, b) este perechea originală.
- Stivă ordonată (problema vizibilității clădirilor): la orice moment, stiva conține valori crescătoare.
- Maximul într-o fereastră glisantă de lățime k: la orice moment, elementele interesante stocate în deque au valori descrescătoare.

2.1.4 Căutarea binară

Iată și un exemplu mai puțin banal. Căutarea binară este un algoritm pe care multă lume îl implementează greșit. Să pornim de la codul pentru căutarea binară într-un vector sortat crescător. O mare parte dintre elevi înfloresc codul cu condiții suplimentare, steguleț, `return` prematur. Un exemplu tipic de cod arată astfel:

```
st = 0;
dr = n - 1;
gasit = false;
while (st <= dr && !gasit) {
    mij = (st + dr) / 2;
    if (v[mij] == x) {
        gasit = true;
    } else if (x < v[mij]) {
        dr = mij - 1;
    } else {
        st = mij + 1;
    }
}
```

Dar iată și o versiune de cod fără umplutură:

```
int bin_search(int x) {
  int l = 0, r = n;
  while (r - l > 1) {
    int mid = (l + r) / 2;
```

```
    if (v[mid] > x) {
      r = mid;
    } else {
      l = mid;
    }
  }
  return l;
}
```

Ca să înțelegem cum procedează funcția, să considerăm datele:

```
n = 20
v = { 12, 15, 15, 17, 19, 19, 19, 19, 19, 19, 22, 30, 35, 35, 35, 38, 40, 43, 45, 49 }
```

În cazul fericit, x apare o singură dată în vector, iar `bin_search` îi returnează poziția. Dar ce returnează funcția în următoarele situații?

1. $x = 5$
2. $x = 100$
3. $x = 27$
4. $x = 19$

Cum se comportă programul la următoarele modificări?

5. Înlocuim `v[mid] > x` cu `v[mid] >= x`.

Cum adaptăm programul la următoarele cerințe?

6. Primim vectorul sortat descrescător.
7. Dorim să returnăm prima apariție în loc de ultima.

Putem urmări manual, pe hîrtie, valorile lui l și r pentru toate cazurile de mai sus. Dar ajută dacă formulăm un invariant de buclă, care este chiar esența algoritmului:

Valoarea x există în intervalul $[l, r)$ sau nu există în vector.

Acest invariant ne ajută să înțelegem cum variază l și r:

- r poate doar să scadă. Cînd r scade, aruncăm o bucată care nu îl conține pe x.
- l poate doar să crească. Cînd l crește, aruncăm o bucată care fie nu îl conține pe x, fie îl conține, dar mai există o copie a lui x pe poziția l.
- La orice moment, $l < r$. Rezultă că funcția returnează o valoare cuprinsă între 0 și $n - 1$, niciodată n.
- Așadar, funcția nu distinge cazurile $x = v[0]$ și $x < v[0]$, respectiv $x = v[n-1]$ și $x > v[n-1]$. Este nevoie de verificarea `x == v[l]`, fie la revenirea din funcție, fie (și mai bine) la finalul funcției.
- În afara acestor extreme, funcția returnează **cea mai mare poziție care conține un element mai mic sau egal cu val**.

Acum putem răspunde la întrebări:

1. $x = 5 \implies$ funcția returnează 0
2. $x = 100 \implies$ funcția returnează 19
3. $x = 27 \implies$ funcția returnează 11 (poziția lui 22, ultimul element mai mic sau egal cu 27)
4. $x = 19 \implies$ funcția returnează 9 (ultima poziție pe care apare 19)
5. Dacă înlocuim `>` cu `>=`, funcția va funcționa greșit: poate arunca porțiunea din vector care conține unica apariție a lui x.
6. Pentru a opera pe un vector sortat descrescător, este suficient să înlocuim `>` cu `<`.
7. Pentru a găsi prima apariție a lui x, trebuie să inițializăm intervalul cu $(-1, n-1]$ și să adaptăm prin oglindire codul care aruncă jumătate din interval:

```
int bin_search_first_ge(int x) {
  int l = -1, r = n - 1; // (l...r]
  while (r - l > 1) {
    int mid = (l + r) / 2;
    if (v[mid] < x) {
      l = mid;
    } else {
      r = mid;
    }
  }
  return r;
}
```

Putem opera cu ușurință și încredere aceste modificări, deoarece înțelegem ce ne dorim la fiecare iterație a buclei.

2.2 Programare prin contract

Programarea prin contract este tot o bună practică în programare. Autorul scrie specificații clare pentru funcțiile, interfețele, clasele etc. pe care le scrie. Aceste specificații se numesc *contracte*. Cei care folosesc codul vor înțelege mai clar cum să-l apeleze, ce parametri sînt acceptabili etc.

Bun, dar ce are asta a face cu cercul nostru? Ca și invarianții de buclă, contractele sînt utile cînd începeți să vă încurcați în propriul cod. Atunci merită să vă opriți și să puneți, la începutul fiecărei funcții sau bucle critice, fraze ca:

- Această funcție se așteaptă ca X.
- Această funcție promite că Y.

Iată două exemple simple.

2.2.1 Stivă

```
// Această funcție se așteaptă ca stiva să nu fie plină.
```

```
// Această funcție promite că stiva nu va fi goală.
// Această funcție promite că un apel pop() imediat următor îl va primi pe x.
// Nu e de interes pentru un apelant extern, dar această funcție promite
// că top va rămîne poziția primului element disponibil.
void push(int x) {
  s[top++] = x;
}

// Această funcție se așteaptă ca stiva să nu fie goală.
int pop() {
  return v[--top];
}
```

2.2.2 Adăugarea într-o tabelă hash

```
// Această funcție se așteaptă ca dicționarul să nu fie plin.
// Această funcție se așteaptă ca key != "".
// Această funcție promite că get(key) va returna value.
void add(string key, string value) {
  // ... implementare de hash map ...
}
```

Contractul nu descrie exact ce fac funcțiile, dar oferă niște garanții și explică că, pentru niște apeluri în afara specificațiilor, programul va da eroare sau comportamentul este nedefinit. Este la latitudinea voastră să fortificați codul cu `assert`-uri și cu mesaje de eroare informative sau să lăsați situația să degenereze spre rezultate greșite, buclă infinită sau *core dump*.

2.3 Depanare

Așa cum învățăm algoritmi și structuri de date, pentru ca mai tîrziu să-i putem reproduce rapid la cerere, tot așa putem învăța și tehnici de depanare, cu elemente universale care se aplică tuturor tipurilor de buguri.

De aceea, vă încurajez să nu lăsați neterminate sursele în care aveți buguri. Dimpotrivă, fiecare bug este o ocazie de a vă exersa aptitudinile de depanatori, ceea ce vă va face programatori mai rapizi și mai completi.

Abordarea completă pe care o recomand pentru depanare este:

1. Nu codați încîlcit! Astfel veți evita majoritatea bugurilor. Însușiți-vă recomandările din capitolul anterior. Separați codul în funcții scurte, care fac un singur lucru fără efecte laterale. Nu complicați logica buclelor (steguleț, `break`, `continue`). Dați nume rezonabile variabilelor. Nu scrieți cod pe șapte niveluri de imbricare. Practic, vă recomand o doză sănătoasă de neîncredere în sine. Nu aveți capacitatea infinită de a înțelege un program scris oricît de prost. Nimeni n-o are.

2. Nu vă panicați! Bugurile nu sînt o pedeapsă divină. Sînt o parte naturală a programării. Amintiți-vă că aveți toate uneltele necesare ca să le reparați.
3. Recitiți codul și explicați-vi-l în gînd. Asta poate scoate la iveală greșeli de tipar, dar și de algoritm.
4. Tipăriți informații, începînd cu blocurile de cod pe care le suspectați.
5. Asigurați-vă că înțelegeți fiecare variabilă și contractul ei. Exemplu: cînd o variabilă x calculată într-o buclă are la final valoarea greșită, înțelegeți care ar trebui să fie valorile ei parțiale? Și cea inițială ar trebui să fie 0 sau 1 sau cît?
6. Dacă bugul apare doar pe teste mari, verificați absolut toate dimensiunile vectorilor și toate operațiile care pot depăși `int`-ul. Eu chiar caut în sursă toate semnele `+`, `-` și `*` și mă asigur că trec pe `long long` acolo unde poate fi necesar.
7. Găsiți un test mic pe care programul greșește. Trebuie mic, nu aveți ce face cu un test mare. Uneori răspunsul este greu de calculat mintal, caz în care scrieți un brut. Pe care, apropo, trimiteți-l ca să aveți 20-30 de puncte în buzunar.
8. Odată izolat testul, bugul este ca și rezolvat. Tipăriți variabile cu mesaje informative de genul `printf("comb(%d, %d) = %d\n", n, k, result)`. Faceți un fel de înjumătățire. Probabil nu este util să verificați vectorul imediat după citire. Dar dacă verificați datele construite pînă undeva la jumătatea programului, veți înjumătăți intervalul suspect de cod.

2.3.1 Găsirea unui contraexemplu mic

Nu toate bugurile se lasă găsite doar prin recitirea programului. Uneori tot ce avem este informația *Wrong answer* pe un test mare și necunoscut. Cum progresăm? Avem nevoie de un test mic și cunoscut. Puteți să-l căutați și de mînă, dar asta poate dura. Dacă după 1-2 minute nu reușiți, este mai rapid să urmați calea de mai jos. În mod cert v-o recomand la teme, dar ea merită luată în calcul chiar și la concursuri.

Iată un studiu de caz pentru problema Kiloforces.

Mai întîi, vă trebuie **o sursă naivă**. Ea poate fi $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$, nu contează. Contează doar să aveți 100% încredere în corectitudinea ei. De regulă o putem obține în 5 minute luînd citirea și scrierea cu copy-paste din sursa eficientă și scriind restul brut. Pentru problema Kiloforces, odată ce știm că trebuie să sortăm valorile t pe interval, putem face exact asta, într-un vector separat.

Doi, vă trebuie **un generator de teste aleatorii**. Am anexat un generator pe care probabil voi îl puteți scrie în 10 minute. El este didactic și poate fi mult scurtat și adaptat pentru alte nevoi.

În sfîrșit, trei, **rulăm programele în buclă**. Rulăm sursa naivă și pe cea cu bug pe teste progresiv mai mari, cu un script Bash, pînă cînd răspunsurile diferă. Probabil și Windows are ceva echivalent; orice sistem de operare folosiți, vă recomand să vă familiarizați cu limbajul său pentru scripturi și să nu vă bazați niciodată doar pe IDE!

```
count=0
while true; do
```

```
  count=$((count+1))
  echo ==== Test $count
  ./gen > data.in
  ./naive < data.in > 1.out
  ./buggy < data.in > 2.out
  diff 1.out 2.out || break
done
```

Putem rula scriptul ca și ca one-liner (vedeți mai jos). Ce fac aceste comenzi? Pe scurt,

- `count` este o variabilă incrementată și afișată la fiecare repetiție, ca să știm cîte teste am făcut.
- Operatorii `<` și `>` redirecționează intrarea și respectiv ieșirea. Așadar, `gen.cpp` este făcut să tipărească la ieșirea standard, dar din Bash îi spunem să pună datele în `data.in`. Apoi, `naive` și `buggy` citesc din `data.in` și scriu în `1.out` și respectiv `2.out`.
- `diff` este o unealtă standard care compară două fișiere. Ea returnează codul 0 dacă fișierele sînt identice și nonzero dacă fișierele diferă.
- `|| break` spune „dacă comanda anterioară s-a terminat cu cod nonzero, ieși din buclă”.
- Altfel o putem opri cu combinația de taste Ctrl-C.

```
[cata@neil kiloforces]$ count=0; while true; do count=$((count+1)); echo ==== Test $count;
./gen > data.in; ./cata-mo-fenwick-v1 < data.in > 1.out; ./buggy < data.in > 2.out;
diff 1.out 2.out || break; done
==== Test 1
==== Test 2
==== Test 3
1c1
< 7
---
> 16
[cata@neil kiloforces]$
```

De aici, încercăm teste de diverse mărimi, progresiv mai mari, pînă găsim o diferență. Eu am început cu $n = 5$ și $q = 1$, care a mers perfect pentru 1 000 de teste. La $q = 2$ scriptul a găsit imediat diferența de mai sus. Acesta a fost un indiciu clar că problema este la tranziția între două interogări.

Altă discrepanță pe care am găsit-o cu teste mai mari a fost:

```
==== Test 77
419c419
< -2163867162
---
> 2131100134
```

Suma celor două numere este exact 2^{32}, deci bugul este la trecerea pe 64 de biți. Într-adevăr, programul în cauză nu folosea deloc `long long`!

2.3.2 Tipăriți, tipăriți, tipăriți

Dacă exemplul în sine nu evidențiază sursa bugului, atunci apelați la `printf`, la hîrtie și la creion! Eu nu știu altă cale mai bună. Trebuie să vă asigurați că toate datele intermediare ale programului corespund cu intenția voastră. Totul este *fair game*: am stabilit deja că undeva programul greșește, deci suspectați totul, chiar și citirea, chiar și sumele parțiale! Tipăriți informațiile modificate de fiecare funcție la finalul funcției.

Încercați să găsiți un format de tipărire prietenos, care să evidențieze intrările și ieșirile din funcții etc. De exemplu, la tipărirea unui arbore de intervale eu inserez `'\n'` după nodurile de pe fiecare nivel, ca să văd arborele stratificat.

Pentru probleme care cer afișarea la ieșirea standard, tipăriți mesajele de debug cu `cerr` sau `stderr`. Astfel evitați să uitați vreun mesaj de debug care să polueze datele de ieșire.

2.3.3 Sfaturi practice

Definiți contracte pe structurile de date

Recitirea atentă a codului poate ajuta la găsirea bugului. Putem prinde folosirea lui `i` în loc de `j`. Dar există o categorie de buguri care țin de nerespectarea contractului unei structuri de date. De aceea este important să definim un astfel de contract. Nu trebuie neapărat să îl notați într-un comentariu, dar trebuie să vi-l clarificați vouă înșivă. Altfel recitim degeaba codul, căci verificăm fără să știm ce și încercăm modificări la întîmplare.

Am întîlnit această situație în repetate rînduri la arbori de intervale cu propagare *lazy*, unde este esențial să înțelegem ce stochează fiecare nod și cum păstrăm coerența structurii după operații de tip *push* sau *pull*. Similar și pentru structura menținută în probleme cu algoritmul lui Mo și pentru orice alte structuri de date.

Programați modular

Este mult mai simplu să vă convingeți că o bucată de cod își respectă contractul dacă o puteți izola într-o funcție de 2-5-10-15 linii. Bugul cel mai ușor de reparat este bugul pe care îl evităm.

Nu includeți `<bits/stdc++.h>`

Depanarea poate presupune să compilăm de zeci de ori programul. Dacă includem headerul `<bits/stdc++.h>`, consecința este că importăm tot STL-ul și îl compilăm, deși utilizăm poate 1% din el. Am văzut ocazional timpi de compilare de 3-4 secunde, care se înjumătățesc dacă includem doar headerele dorite (`<vector>`, `<iostream>` etc.). În timp de concurs, aceste diferențe, însumate la 1-2-3 minute, contează.

În plus, `<bits/stdc++.h>` este nestandard. Codul va merge doar pe GCC. Puteți citi o analiză mai amplă pe Stack Overflow.

2.3.4 Studiu de caz 1

Iată sursa unui elev pentru problema Kiloforces. Sursa trece subtaskurile mici, apoi începe să dea erori de tip *Wrong answer* și *Time limit exceeded*. Citiți-o cu atenție și încercați să găsiți bugul (sau bugurile)! Pentru a înțelege rezolvarea, ajută să fiți familiari cu Algoritmul lui Mo.

```
#include <algorithm>
#include <iostream>
#include <set>

using namespace std;

const int BLOCK_SIZE = 300;
const int MAX_N = 1e5;
const int MAX_Q = 1e5;

struct query {
    int l, r;
    int pos;
};

int N, Q;

int p[MAX_N], t[MAX_N];
int prefix_sum[MAX_N];

query queries[MAX_Q];

int ans[MAX_Q];

void read() {
    cin >> N >> Q;
    for (int i = 0; i < N; ++i) {
        cin >> p[i];
    }
    for (int i = 0; i < N; ++i) {
        cin >> t[i];
    }
    for (int i = 0; i < Q; ++i) {
        cin >> queries[i].l >> queries[i].r;
        --queries[i].l;
        --queries[i].r;
        queries[i].pos = i;
    }
}

void build_prefix_sum() {
    prefix_sum[0] = p[0];
    for (int i = 1; i < N; ++i) {
```

```
        prefix_sum[i] = prefix_sum[i - 1] + p[i];
    }
}

void sort_queries() {
    sort(queries, queries + Q, [](query a, query b) {
        int block_a = a.l / BLOCK_SIZE;
        int block_b = b.l / BLOCK_SIZE;
        if (block_a != block_b) {
            return block_a < block_b;
        }
        else if (block_a & 1) {
            return a.r > b.r;
        }
        else {
            return a.r < b.r;
        }
    });
}

struct block_data {
    set<int> penalties;
    int points(int l, int r) {
        if (l) {
            return prefix_sum[r] - prefix_sum[l - 1];
        }
        return prefix_sum[r];
    }
    int penalty(int l, int r) {
        int val = 0;
        int count = r - l;
        for (auto x : penalties) {
            val += x * count;
            --count;
        }
        return val;
    }
    int score(int l, int r) {
        return points(l, r) - penalty(l, r);
    }
};

void mo() {
    block_data data;
    int left = queries[0].l;
    int right = queries[0].r;
    for (int i = left; i <= right; ++i) {
        data.penalties.insert(t[i]);
    }
    for (int i = 0; i < Q; ++i) {
```

```
        while (left < queries[i].l) {
            data.penalties.erase(t[left++]);
        }
        while (left > queries[i].l) {
            data.penalties.insert(t[--left]);
        }
        while (right < queries[i].r) {
            data.penalties.insert(t[++right]);
        }
        while (right > queries[i].r) {
            data.penalties.erase(t[--right]);
        }
        ans[queries[i].pos] = data.score(left, right);
    }
}

void write() {
    for (int i = 0; i < Q; ++i) {
        cout << ans[i] << '\n';
    }
}

int main() {
    read();
    build_prefix_sum();
    sort_queries();
    mo();
    write();
    return 0;
}
```

Voi ce ați observat? Iată mai întîi cîteva observații generale despre cod:

- Codul este bine scris și structurat. Asta întotdeauna ajută la depanare!
- Pare că folosește un `set` în locul unor structuri mai eficiente (problema se poate rezolva cu un AIB).
- Mai rău, funcția `penalty()` iterează prin set. Aceasta explică TLE-ul.
- Sursa trece primele două subtaskuri, deci probabil nu trebuie să verificăm codul elementar (sume parțiale etc.).

Sper că ați găsit și cele două buguri. Există două lucruri în neregulă în funcția `mo()`:

1. Există o asimetrie la mutarea indicilor, o vedeți? `left++`, `--left`, `++right`, `--right`. Judecînd după inițializare, la iterația curentă menținem în structură intervalul închis $[left, right]$. Atunci ultima cantitate ar trebui să fie `right--`: întîi excludem elementul $right$, apoi restrîngem indexul.
2. Le dau elevilor mei de nenumărate ori următorul sfat, dar mulți nu mă ascultă. Spuneți-mi Casandra. 😢 În algoritmul lui Mo, este prudent să extindeți intervalul în stînga și în

dreapta și abia apoi să-l contractați. Altfel, se întîmplă ciudățenii cînd intervalul devine negativ, fie și numai tranzitoriu. Exemplu: dacă trecem de la [10, 20] la [30, 40] și deplasăm întîi capătul stîng, vom ajunge tranzitoriu la intervalul negativ [30,20]. Apoi deplasăm capătul drept și ajungem la [30, 40], dar setul `penalties` va include toate penalizările de la 20 la 40! Problema survine deoarece adăugarea și ștergerea din `set` sînt asimetrice. Adăugarea garantează inserarea elementului, dar eliminarea nu garantează ștergerea lui (pentru că nu are noțiunea de frecvență negativă). Dacă facem trecerea [10, 20] → [10, 40] → [30, 40], intervalul este mereu bine definit.

2.3.5 Studiu de caz 2

Iată și o altă sursă, de data aceasta pentru problema Polynomial Queries. Sursa primește *Wrong answer* chiar și pe teste mici. Rezolvarea folosește arbori de intervale. Citiți-o cu atenție și vedeți dacă vreun bug vă sare în ochi!

```
#include <bits/stdc++.h>

using namespace std;

const int nmax = 2e5;

vector<int> v(nmax + 3);

struct AInt {
  struct node {
    int64_t sum = 0, start = 0, ratie = 0;
  };
  vector<node>aint;
  AInt() {
    aint.resize(4 * nmax + 3);
  }
  void build(int nod, int st, int dr) {
    if(dr < st)
      return;
    if(st == dr) {
      aint[nod].sum = v[st];
    } else {
      int mid = (st + dr) / 2;
      build(nod * 2, st, mid);
      build(nod * 2 + 1, mid + 1, dr);
      aint[nod].sum = aint[nod * 2].sum + aint[nod * 2 + 1].sum;
    }
  }
  void push(int nod, int st, int dr) {
    if(st > dr)
      return;
    if(st == dr) {
```

```
      if(aint[nod].start > 0)
        aint[nod].sum += aint[nod].start;
      aint[nod].start = aint[nod].ratie = 0;
      return;
    }
    int mid = (st + dr) / 2;
    aint[nod].sum += aint[nod].start * (dr - st + 1);
    aint[nod].sum += aint[nod].ratie * (dr - st + 1) * (dr - st) / 2;
    if(aint[nod].start > 0) {
      aint[nod * 2].start += aint[nod].start;
      aint[nod * 2 + 1].start += aint[nod].start + (mid + 1 - st);
    }
    aint[nod * 2].ratie += aint[nod].ratie;
    aint[nod * 2 + 1].ratie += aint[nod].ratie;
    aint[nod].start = aint[nod].ratie = 0;
  }
  void update(int nod, int st, int dr, int l, int r) {
    if(dr < st || l > dr || r < st)
      return;
    if(l <= st && dr <= r) {
      aint[nod].start += (st - l + 1);
      aint[nod].ratie++;
      push(nod, st, dr);
    } else {
      push(nod, st, dr);
      int mid = (st + dr) / 2;
      update(nod * 2, st, mid, l, r);
      update(nod * 2 + 1, mid + 1, dr, l, r);
      push(nod * 2, st, mid);
      push(nod * 2 + 1, mid + 1, dr);
      aint[nod].sum = aint[nod * 2].sum + aint[nod * 2 + 1].sum;
    }
  }
  int64_t query(int nod, int st, int dr, int l, int r) {
    if(dr < st || l > r || l > dr || r < st)
      return 0;
    push(nod, st, dr);
    if(st == l && dr == r) {
      return aint[nod].sum;
    } else {
      int mid = (st + dr) / 2;
      return query(nod * 2, st, mid, l, min(r, mid)) + query(nod * 2 + 1,
                   mid + 1, dr, max(mid + 1, l), r);
    }
  }
};

AInt myAint;
int cer, st, dr, x, n, q;
```

```
void read() {
  cin >> n >> q;
  for(int i = 1; i <= n; i++)
    cin >> v[i];
}

void solve() {
  myAint.build(1, 1, n);
  for(int i = 1; i <= q; i++) {
    cin >> cer >> st >> dr;
    if(cer == 1) {
      myAint.update(1, 1, n, st, dr);
    } else {
      cout << myAint.query(1, 1, n, st, dr) << "\n";
    }
  }
}

int main() {
  read();
  solve();
  return 0;
}
```

Calitatea codului nu este perfectă, dar asta este explicabil, căci conține multe încercări de depanare.

Codul face foarte multe operații `push`, ceea ce pe mine m-a trimis pe o pistă falsă. În realitate, dacă operația `push` este bine scrisă, ea este idempotentă: o putem apela de oricîte ori fără efecte secundare.

Următoarea bănuială pe care am avut-o a fost nerespectarea contractului în arborele de intervale? Care este acest contract? El rezultă cel mai bine din funcția update:

```
if(l <= st && dr <= r) {
  aint[nod].start += (st - l + 1);
  aint[nod].ratie++;
  push(nod, st, dr);
}
```

Codul adaugă progresia în cîmpurile `start` și `ratie`, dar nu și în cîmpul `sum`. Așadar, contractul pare să fie: suma reală din intervalul subîntins de un nod este cîmpul `sum` plus suma progresiei date de `start` și `ratie`.

Cu această presupunere, am verificat absolut toate operațiile și am găsit un prim bug în circa 15 minute. Probabil că autorul l-ar putea găsi mai repede, fiind familiar cu codul. La revenirea din update, propagarea în sus a rezultatului este:

```
aint[nod].sum = aint[nod * 2].sum + aint[nod * 2 + 1].sum;
```

Dar pe `nod` nu mai avem progresie, căci am făcut push. De aceea, `sum` trebuie să conțină chiar suma reală. Deci codul ar trebui să includă și progresiile de pe cei doi fii! Am scris o funcție separată pentru suma unei progresii, căci vor exista trei apeluri la ea (două aici și una în `push`). Astfel, codul devine:

```
int64_t sum_prog(int64_t start, int64_t ratie, int lung) {
  return start * lung + ratie * (lung - 1) * lung / 2;
}

...

aint[nod].sum =
  aint[nod * 2].sum +
  aint[nod * 2 + 1].sum +
  sum_prog(aint[nod * 2].start, aint[nod * 2].ratie, mid - st + 1) +
  sum_prog(aint[nod * 2 + 1].start, aint[nod * 2 + 1].ratie, dr - mid);
```

Principial, această funcție merita scrisă de la bun început, chiar dacă era folosită doar în `push`. Este o funcție scurtă, clară, implementarea unei formule matematice consacrate.

Acum eram destul de încrezător că implementarea respectă contractul și am comentat toate apelurile nenecesare la funcția `push`. Am ajuns la o nouă sursă, care încă avea un bug.

Am scris un generator rudimentar de date de test și am comparat în buclă această sursă cu soluția mea (corectă) la problemă. Am încercat cu $n = 5$ și $q = 2$ nu am găsit contraexemple. Așadar, pare că programul gestionează bine o singură actualizare. Dar pentru $n = 5$ și $q = 3$ am găsit rapid contraexemplul de mai jos, pentru care răspunsul corect este 9, dar programul elevului afișează 8:

```
5 3
1 1 1 1 1
1 1 5
1 1 2
2 2 3
```

De aici încolo putem efectiv să depanăm programul. Cheia sînt figurile pe hîrtie și multe mesaje tipărite. Am afișat arborele după construcție și după fiecare update:

```
void print() {
  for (int i = 1; i <= 12; i++) {
    cerr << "(" << aint[i].sum << " " << aint[i].start << " "
         << aint[i].ratie << ") ";
    if ((i & (i + 1)) == 0) {
      cerr << "| ";
```

```
    }
  }
  cerr << "\n";
}

...

myAint.update(1, 1, n, st, dr);
cerr << "După update:\n";
myAint.print();
```

Astfel am obținut codul de debug:

```
(5 0 0) | (3 0 0) (2 0 0) | (2 0 0) (1 0 0) (1 0 0) (1 0 0) | (1 0 0) (1 0 0)
(0 0 0) (0 0 0) (0 0 0)
După update:
(5 1 1) | (3 0 0) (2 0 0) | (2 0 0) (1 0 0) (1 0 0) (1 0 0) | (1 0 0) (1 0 0)
(0 0 0) (0 0 0) (0 0 0)
După update:
(23 0 0) | (12 0 0) (2 4 1) | (2 2 2) (1 3 1) (1 0 0) (1 0 0) | (1 0 0) (1 0 0)
(0 0 0) (0 0 0) (0 0 0)
8
```

Aici am greșit că nu am mai afișat și arborele după `query`, care ar fi făcut încă un `push` care ar fi evidențiat greșeala. Voi o vedeți mai jos?

```
După query:
(23 0 0) | (12 0 0) (2 4 1) | (8 0 0) (4 0 0) (1 0 0) (1 0 0) | (1 2 2) (4 0 0)
(0 0 0) (0 0 0) (0 0 0)
```

În loc de asta, am tipărit o mulțime de informații în funcția `push`, care m-au dus la aceeași observație. Iată ce am desenat între timp pe foaie.

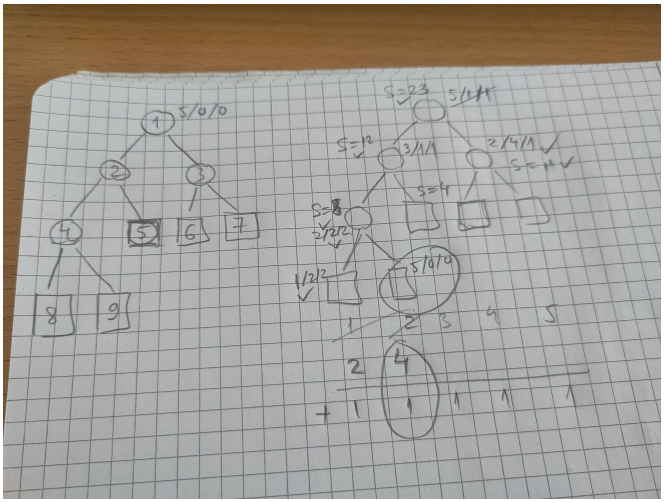


Figura 2.1: Depanarea unei surse la problema Polynomial Queries.

Frunza `(4 0 0)` ar fi trebuit să fie `(5 0 0)`. De aici, devine clar că trebuie să ne uităm la spargerea unei progresii între cei doi fii și găsim eroarea pe linia:

```
aint[nod * 2 + 1].start += aint[nod].start + (mid + 1 - st);
```

Codul corect este:

```
aint[nod * 2 + 1].start += aint[nod].start + (mid + 1 - st) * aint[nod].ratie;
```

Am testat cu generatorul pe teste cu $n = 50$ și $q = 30$ și nu am găsit erori în 10 000 de teste. Am încredere că această versiune este corectă.

Partea II

Structuri de date pe vectori

Următoarele capitole tratează structuri de date care pot procesa anumite operații pe vectori în timp mai bun decît $\mathcal{O}(N)$. Ocazional aceste structuri se aplică și matricilor.

Subiectele de ONI / baraj ONI / lot din anii trecuți abundă în probleme rezolvabile cu astfel de structuri:

- 3dist (baraj ONI 2022)
- 6 de Pentagrame (lot 2024)
- Babel (baraj ONI 2025)
- Balama (baraj ONI 2024)
- Bisortare (ONI 2021)
- Circuit (lot 2025)
- Emacs (baraj ONI 2021)
- Erinaceida (lot 2022)
- Guguștiuc (baraj ONI 2022)
- Împiedicat (baraj ONI 2023)
- Lupușor (ONI 2022)
- Medwalk (lot 2025)
- Perm (baraj ONI 2024)
- Piezișă (baraj ONI 2022)
- Subiectul III (lot 2024)
- Șirbun (baraj ONI 2023)
- Trapez (lot 2025)

Pare o idee bună să le învățăm și să le stăpînim bine. 🙂 Concret, vom studia trei structuri:

1. arbori de intervale;
2. arbori indexați binar;
3. descompunere în radical.

Vom exemplifica structurile și vom face benchmarks pe două probleme didactice. Apoi vom vedea, prin probleme, cum putem extinde aceleași structuri pentru nevoi mai complicate.

Varianta 1 (actualizări punctuale, interogări pe interval): Se dă un vector de N elemente întregi și Q operații de două tipuri:

1. $\langle 1, x, val \rangle$: Adaugă val pe poziția x a vectorului.
2. $\langle 2, x, y \rangle$: Calculează suma pozițiilor de la x la y inclusiv.

Varianta 2 (actualizări pe interval, interogări pe interval): Similar, dar operația 1 este pe interval:

1. $\langle 1, x, y, val \rangle$: Adaugă val pe pozițiile de la x la y inclusiv.
2. $\langle 2, x, y \rangle$: Calculează suma pozițiilor de la x la y inclusiv.

Vom menționa ocazional și **Varianta 3 (actualizări pe interval, interogări punctuale)**:

1. $\langle 1, x, y, val \rangle$: Adaugă val pe pozițiile de la x la y inclusiv.

2. $\langle 2, x \rangle$: Returnează valoarea poziției x.

Toate implementările mele sînt disponibile pe GitHub.

Capitolul 3

Arbori de intervale

Arborii de intervale[1] (AINT) sînt o structură foarte puternică și flexibilă. Ușurința implementării depinde de natura operațiilor pe care dorim să le admitem.

3.1 Reprezentare

Ca multe alte structuri (heap-uri, AIB, păduri disjuncte), arborii de intervale se reprezintă pe un simplu vector. Ei sînt arbori doar la nivel logic, în sensul că fiecare poziție din vector are o altă poziție drept părinte.

Pentru început, să presupunem că vectorul dat are $n = 2^k$ elemente. Atunci vectorul necesar S are $2n$ elemente, în care cele n elemente date sînt stocate începînd cu poziția n. Apoi,

- Cele $n/2$ elemente anterioare stochează valori agregate (sume, minime, xor etc.) pentru perechi de valori din vectorul dat.
- Cele $n/4$ elemente anterioare stochează valori agregate pentru grupe de 4 valori din vectorul dat.
- ...
- Elementul $S[1]$ stochează valoarea agregată a întregului vector.
- Valoarea $S[0]$ rămîne nefolosită.

Iată un exemplu pentru $n = 16$. Datele de la intrare se regăsesc pe pozițiile 16-31.

[1] Există o inversiune între nomenclatura internațională și cea românească. Internațional, structura pe care o învățăm astăzi se numește segment tree, iar interval tree este o structură diferită, care stochează colecții de intervale. Cîțiva ani am înotat împotriva curentului și am fost (posibil) singurul român care se referea la această structură ca „arbori de segmente". În acest curs am adoptat și eu denumirea încetățenită.

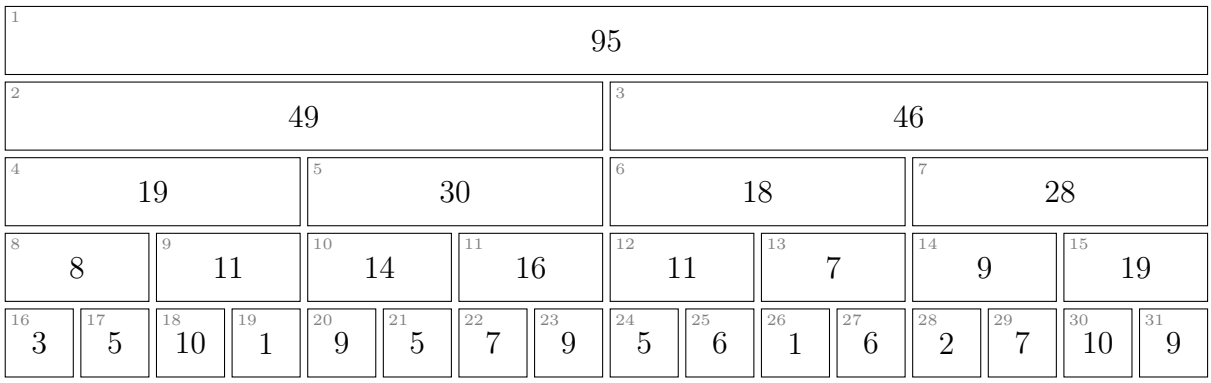


Figura 3.1: Un arbore de intervale cu 16 frunze și 15 noduri interne. Valorile din fiecare celulă reprezintă suma din frunzele subîntinse de acea celulă. Cu cifre mici este notat indicele fiecărei celule.

Facem cîteva observații preliminare:

- Fiii unui nod i sînt $2i$ și $2i + 1$.
- Părintele lui i este $\lfloor i/2 \rfloor$.
- Toți fiii stîngi au numere pare și toți fiii drepți au numere impare.

De exemplu, fiii lui 6 sînt 12 și 13, iar fiii acestora sînt respectiv 24-25 și 26-27. Aceasta corespunde cu intenția noastră ca 6 stocheze informații agregate (suma) despre nodurile 24-27.

După cum vom vedea în secțiunea următoare, arborii de intervale obțin timpi logaritmici pentru operații, deoarece numărul de niveluri este $\log n$.

3.1.1 Memoria necesară

În această formă, structura necesită $2n$ memorie pentru n elemente dacă n este putere a lui 2 sau foarte aproape. De exemplu, pentru $n = 1024$, sînt necesare 2048 de celule. Dar, dacă n depășește cu puțin o putere a lui 2, atunci el trebuie rotunjit în sus. Pentru $n = 1025$, baza arborelui necesită 2048 de celule, iar arborele în întregime necesită 4096 de celule. De aceea spunem că, în cel mai rău caz, arborele poate ajunge la $4n$ celule ocupate în cel mai rău caz.

În realitate, necesarul este doar de $3n$ cu puțină atenție la alocare. Pentru $n = 1025$, alocăm 2048 de celule pentru nivelurile superioare ale arborelui, dar putem aloca fix 1025 pentru bază (nu 2048). Totalul este circa $3n$.

Pentru a calcula următoare putere a lui 2, putem folosi bucla naivă:

```
int p = 1;
while (p < n) {
  p *= 2;
}
n = p;
```

Sau o buclă care folosește *bit hacks*:

```
while (n & (n - 1)) {
  n += n & -n;
}
```

Mai concis, putem folosi funcția `__builtin_clz(x)`, care ne spune cu cîte zerouri începe numărul `x`:

```
n = 1 << (32 - __builtin_clz(n - 1));
```

3.1.2 Reprezentări alternative

Există și reprezentări mai compacte, care ocupă exact $2n-1$ noduri, adică strictul necesar teoretic. Iată un exemplu pentru un vector cu 6 noduri.

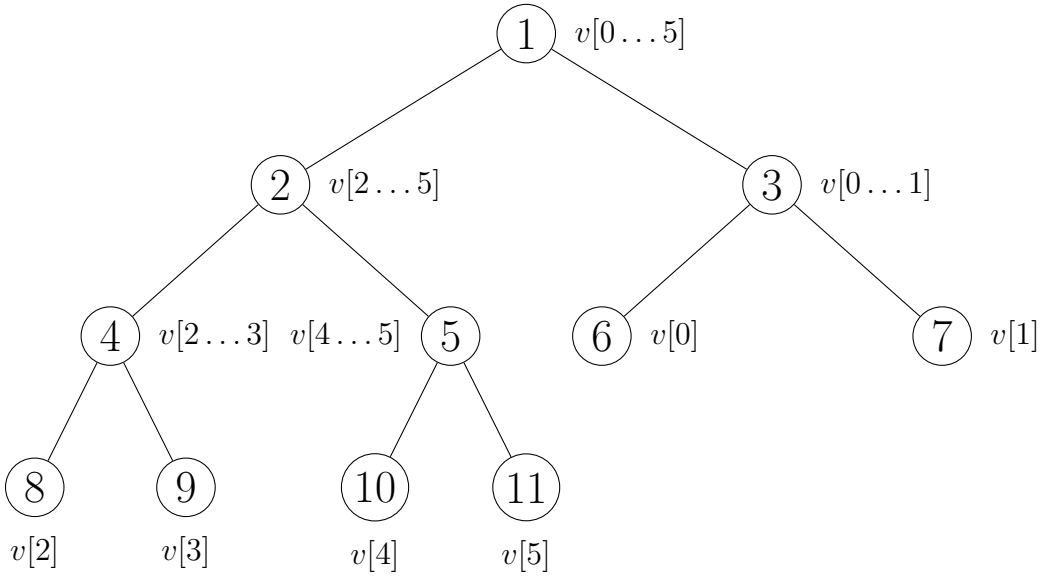


Figura 3.2: Reprezentarea arborilor de intervale cu exact $2n-1$ noduri.

Vedem că frunzele (adică vectorul dat, $v[0] \ldots v[5]$) se află pe pozițiile consecutive 6-11. În schimb, această reprezentare pare mai greu de vizualizat și încalcă o abstracție importantă: frunzele nu mai sînt la același nivel. Structura se pretează la operațiile de actualizare și interogare, dar nu sînt sigur că se pretează și la restul operațiilor pe care le discutăm în secțiunile următoare. De aceea prefer să folosesc și să predau structura rotunjită la 2^k noduri.

3.2 Operații elementare

3.2.1 Actualizarea punctuală

Nu uitați că poziția i din datele de intrare este stocată efectiv în $s[n+i]$. Apoi, cînd elementul aflat pe poziția i primește valoarea val, toate nodurile care acoperă poziția i trebuie recalculate:

```
void set(int pos, int val) {
  pos += n;
  s[pos] = val;
  for (pos /= 2; pos; pos /= 2) {
    s[pos] = s[2 * pos] + s[2 * pos + 1];
  }
}
```

Dacă nu ni se dă noua valoare absolută, ci variația δ față de valoarea anterioară, atunci codul este chiar mai simplu, căci toți strămoșii poziției se modifică tot cu δ:

```
void add(int pos, int delta) {
  for (pos += n; pos; pos /= 2) {
    s[pos] += delta;
  }
}
```

Apropo de *clean code*: Remarcați că am denumit funcțiile `set` și `add`, nu le-am denumit pe ambele `update`. Astfel am evidențiat diferența dintre ele.

3.2.2 Construcția în $\mathcal{O}(n \log n)$

O variantă de construcție este să invocăm funcția set de mai sus pentru fiecare valoare de la intrare. Complexitatea va fi $\mathcal{O}(n \log n)$.

3.2.3 Construcția în $\mathcal{O}(n)$

Putem reduce timpul de construcție dacă doar inserăm valorile frunzelor, fără a le propaga la strămoși. La final calculăm foarte simplu nodurile interne, în ordine descrescătoare.

```
void build() {
  for (int i = n - 1; i >= 1; i--) {
    s[i] = s[2 * i] + s[2 * i + 1];
  }
}
```

3.2.4 Calculul sumei pe interval

Să calculăm suma pe intervalul original [2, 12], care corespunde intervalului [18, 28] din reprezentarea internă. Ideea este să descompunem acest interval într-un număr logaritmic de segmente, mai exact [18,19], [20,23], [24,27] și [28,28]. Avantajul descompunerii este că avem deja calculate sumele acestor intervale, respectiv în nodurile 9, 5, 6 și 28.

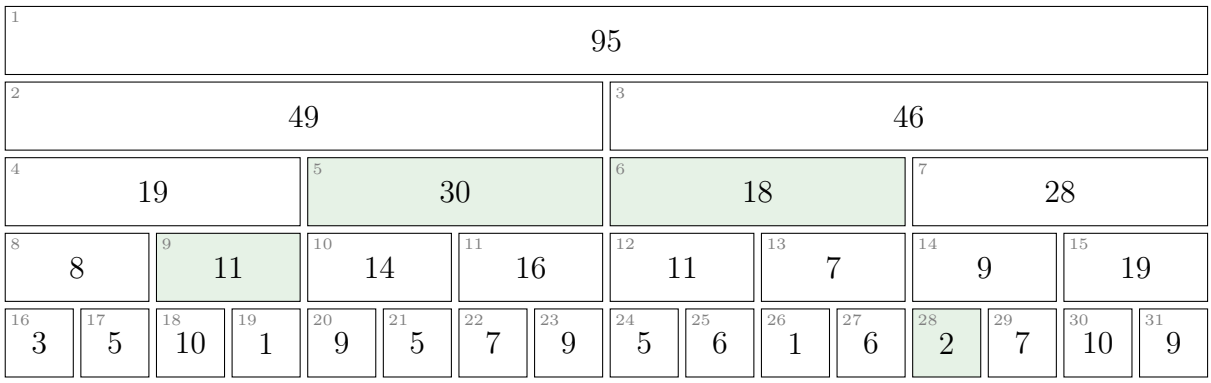


Figura 3.3: Suma intervalului [18, 28] este egală cu suma valorilor nodurilor 9, 5, 6 și 28.

Pornim cu doi pointeri l și r din capetele interogării date. Apoi procedăm astfel:

- Dacă l este fiu stîng, putem aștepta ca să includem un strămoș al său, care va include și alte poziții utile. În schimb, dacă l este fiu drept, trebuie să îl includem în sumă, căci orice strămoș al său va include și elemente inutile din stînga lui l. Apoi avansăm l spre dreapta.
- Printr-un raționament similar, dacă r este fiu stîng, includem valoarea sa în sumă și avansăm r spre stînga.
- Urcăm pe nivelul următor prin înjumătățirea lui l și r.
- Continuăm cît timp $l \leq r$.

Astfel, vom selecta cel mult două intervale de pe fiecare nivel al arborelui și vom restrînge corespunzător intervalul dat, pînă cînd îl reducem la zero. De aici rezultă complexitatea logaritmică.

```
long long query(int l, int r) { // [l, r] închis
  long long sum = 0;

  l += n;
  r += n;

  while (l <= r)  {
    if (l & 1) {
      sum += s[l++];
    }
    l >>= 1;

    if (!(r & 1)) {
      sum += s[r--];
    }
    r >>= 1;
  }

  return sum;
}
```

Clarificare: la ultimul nivel, dacă $l = r$, atunci $s[l]$ va fi selectat exact o dată, fie datorită lui l,

fie datorită lui r, după cum poziția este impară sau pară.

3.2.5 Căutarea unei sume parțiale

Ca și la AIB-uri, dacă toate valorile sînt pozitive are sens întrebarea: pe ce poziție suma parțială atinge valoarea P? Pentru simplitate, recomand să adăugați o santinelă de valoarea infinită pe poziția n. Aceasta garantează că suma parțială se atinge întotdeauna, iar dacă răspunsul este n, atunci de fapt suma parțială nu există în vectorul fără santinelă.

```
int search(int sum) {
  int pos = 1;

  while (pos < n) {
    pos *= 2;
    if (sum > s[pos]) {
      sum -= s[pos++];
    }
  }

  return pos - n;
}
```

3.2.6 Căutarea într-un arbore de maxime

Dat fiind un vector v cu n elemente, ni se cere să răspundem la interogări de tipul $\langle pos, val \rangle$ cu semnificația: găsiți cea mai mică poziție $i > pos$ pe care se află o valoare $v[i] > val$. În secțiunea următoare vom vedea problemele Points și Împiedicat care au această nevoie.

Pentru rezolvare, să construim peste acest vector un arbore de intervale de maxime. Fiecare nod stochează maximul dintre cei doi fii ai săi. Ca urmare, fiecare nod stochează maximul dintre frunzele pe care le subîntinde. Atunci soluția constă din doi pași:

- Mergi la dreapta și în sus, similar pointerului l din operația de sumă pe interval prezentată anterior. Oprește-te cînd ajungi la un nod cu o valoare $> val$. Știm că acest nod subîntinde cel puțin o frunză de valoare $> val$.
- Din acest nod, coboară în fiul care are la rîndul său o valoare $> val$. Dacă ambii fii au această proprietate, coboară în fiul stîng. Oprește-te cînd ajungi la o frunză.

Pentru a simplifica codul, putem adăuga o santinelă infinită la finalul vectorului, ca să ne asigurăm că problema are soluție.

```
int find_first_after(int pos, int val) {
  pos += n + 1;

  while (v[pos] <= val) {
    if (pos & 1) {
```

```
      pos++;
    } else {
      pos >>= 1;
    }
  }

  while (pos < n) {
    pos = (v[2 * pos] > val) ? (2 * pos) : (2 * pos + 1);
  }

  return pos - n;
}
```

Am inclus acest algoritm, deși este rar întîlnit în pratică, pentru a ilustra flexibilitatea uriașă a arborilor de intervale.

3.2.7 Adaptarea la alte tipuri de operații

Aceeași structură de date poate răspunde la multe alte feluri de actualizări și interogări. Nu detaliem aici, vom studia probleme. Ce este important este să ne dăm seama ce stocăm în fiecare nod și cum combină părintele informațiile din cei doi fii.

3.2.8 Implementarea recursivă

Există și o implementare recursivă, pe care nu o vom discuta acum (o menționez doar ca să o fac de rîs). O vom discuta în capitolul următor. Este păcat că mulți elevi învață și stăpînesc doar acea implementare, pe care o aplică și cînd nu este nevoie de ea, deși implementarea iterativă de mai sus este de 2-3 ori mai rapidă. Implementarea iterativă ar trebui să fie implementarea voastră de referință oricînd este suficientă.

Exemplu: din implementarea iterativă rezultă imediat că:

1. Complexitatea este $\mathcal{O}(\log n)$, întrucît l și r urcă exact un nivel la fiecare iterație.
2. De pe fiecare nivel selectăm cel mult două intervale.

Vă urez succes să demonstrați aceste lucruri în implementarea recursivă. 😈

3.3 Probleme

3.3.1 Problema Xenia and Bit Operations (Codeforces)

enunț • sursă

Problema este simplisimă. O includ doar ca exemplu de arbore care face operații diferite pe niveluri diferite.

3.3.2 Problema Distinct Characters Queries (Codeforces)

enunț • sursă

Există diverse abordări pentru această problemă. Una este să construim un AIB sau un AINT pentru fiecare caracter, cu memorie totală $\mathcal{O}(\Sigma n)$ (tradițional Σ denotă mărimea alfabetului). Fiecare structură reține pozițiile pe care apare un caracter. Modificările sînt simple: debifăm poziția în AIB-ul corespunzător vechiului caracter și o marcăm în AIB-ul noului caracter. Pentru interogări, verificăm pentru fiecare din cele 26 de caractere dacă suma pe intervalul dat este non-zero. Rezultă o complexitate de $\mathcal{O}(\Sigma q \log n)$.

Dar iată și o soluție mai elegantă, care reduce complexitatea la $\mathcal{O}(q \log n)$, folosind paralelismul nativ pe 32 de biți al procesorului. Vom folosi 26 de biți din fiecare întreg, cîte unul pentru fiecare caracter. Într-o frunză care stochează litera 'f' vom seta pe 1 doar cel de-al șaselea bit, așadar valoarea întreagă va fi `000...000100000`. Apoi, un nod intern va stoca OR-ul pe biți al frunzelor din intervalul acoperit. Acest gen de informație se numește **mască de biți** (engl. *bitmask*).

Ce semnifică acest OR pe biți? Fiecare dintre biți va fi 1 dacă și numai dacă litera corespunzătoare apare cel puțin o dată în intervalul acoperit. Să observăm că bitul 6 va fi 1 indiferent dacă intervalul conține un caracter 'f' sau multiple caractere 'f'. Rezultă că fiecare mască va avea atîția biți setați (biți 1) cîte caractere distincte există în interval.

Facem actualizări în acest arbore înlocuind masca din frunză și propagînd valoarea spre strămoși cu operația OR. Pentru a răspunde la interogări,

- colectăm cele $\mathcal{O}(\log n)$ măști care compun interogarea;
- le combinăm cu OR;
- numărăm biții din rezultat, de exemplu cu funcția `__builtin_popcount`.

3.3.3 Problema K-query (SPOJ)

enunț • surse

Problema fiind offline, este destul de natural să ordonăm interogările. Sper să vă obișnuiți și voi să luați în calcul această posibilitate.

Ordonarea după capătul stîng sau drept nu pare să ducă nicăieri. Exemplu: ordonăm interogările după capătul drept dr. Atunci, după ce adăugăm elementul $a[dr]$ la structura noastră (oricare ar fi ea), trebuie să răspundem la interogări de tipul: cîte numere $> k$ există începînd cu poziția st? Eu nu am reușit să găsesc o structură echilibrată care să răspundă la întrebări. Poate voi reușiți?

În schimb, ordonarea descrescătoare după valoare duce la o soluție relativ directă. Pentru o interogare (st, dr, k), marcăm (cu 1) într-o structură de date toate pozițiile elementelor mai mari decît k. Apoi numărăm valorile 1 din intervalul $[st, dr]$.

Pentru a găsi rapid toate elementele mai mari decît k (care nu au fost deja inserate în structură), rezultă că trebuie să sortăm și vectorul în ordine descrescătoare, reținînd și poziția originală a

fiecărei valori.

În fapt, putem implementa această soluție chiar și cu un AIB. Sursa este identică cu cea bază pe arbori de intervale cu excepția `struct`-ului. În acest caz, timpii de rulare sînt aproape egali, dar în general vă recomand să folosiți AIB unde se poate.

3.3.4 Problema Sereja and Brackets (Codeforces)

enunț • sursă

Iată și o problemă pentru a cărei rezolvare este mai puțin clar că ne ajunge un arbore de intervale. Vom construi un arbore în care nodurile stochează valori mai complexe care se combină după reguli speciale.

Să considerăm o subsecvență contiguă. Din ce constă ea? Dintr-un subșir (pe sărite) care este bine format, plus niște paranteze deschise neîmperecheate, plus niște paranteze închise neîmperecheate. De exemplu, în subșirul `))) ( () ( () ( () (` am evidențiat cu bold cele 6 caractere bine formate. Rămîn 4 paranteze deschise și 3 închise. Să notăm aceste cantități cu f (lungimea subșirului bine format), d (surplusul de paranteze deschise) și i (surplusul de paranteze închise).

Cum combinăm două subsecvențe adiacente (f_1, d_1, i_1) și (f_2, d_2, i_2)? Clar putem concatena porțiunile bine formate. Dar mai mult, putem prelua și $\min(d_1, i_2)$ perechi dintre surplusurile de paranteze deschise, respectiv închise. Șirul rezultat va fi bine format. Ne putem convinge de asta eliminînd porțiunile bine formate f_1 și f_2, ca și cînd ele nu ar exista. Dacă nu sînteți convinși, puteți apela la o definiție echivalentă pentru un șir de paranteze bine format: pentru orice prefix, diferența dintre numărul de paranteze deschise și închise este pozitivă.

Rezultă că intervalul concatenat va avea parametrii:

- $f = f_1 + f_2 + 2\min(d_1, i_2)$
- $d = d_1 + d_2 - \min(d_1, i_2)$
- $i = i_1 + i_2 - \min(d_1, i_2)$

Construcția arborelui se face ca de obicei, combinînd fiii doi cîte doi. La interogare este nevoie de puțină atenție pentru a colecta și combina intervalele în ordinea corectă (de la stînga la dreapta). Ne bazăm pe observația că operația de compunere nu este comutativă, dar este asociativă.

3.3.5 Problema Copying Data (Codeforces)

enunț • sursă

Aici întîlnim o formă complementară a arborilor de intervale: actualizări pe interval și interogări punctuale (*range update, point query*). Mecanismul necesar folosește o reprezentare puțin diferită. O problemă foarte similară este Range Update Queries (CSES).

(Cei dintre voi care stăpînesc arborii de intervale cu propagare *lazy* vor fi tentați să se repeadă la aceia: Pe fiecare nod ținem informația *lazy* că segmentul din b a fost suprascris cu un segment

din a începînd de la o poziție p (sau cu o deplasare $\pm p$, cum preferați). La actualizări, propagăm informația la fii după nevoie. La interogare, propagăm informația pînă în frunza cerută, pentru a afla de unde provine. Dar nu este nevoie de aceste complicații.)

Să pornim de la observația de bun simț: Dacă o copiere acoperă o poziție, atunci la descompunerea sa în intervale, unul dintre acele intervale va fi strămoș al poziției respective (*duh!*).

Ne vom folosi și de numerele de ordine ale interogărilor, care vor funcționa ca niște momente de timp între 1 și q. Acum, să construim un arbore de intervale care, pentru o operație de copiere (x, y, k):

- Descompune intervalul $[y, y + k - 1]$ prin metoda obișnuită.
- Notează pe fiecare interval momentul t și diferența $x - y$.

Dacă ulterior o altă copiere va acoperi unul dintre aceste intervale, vom nota acolo momentul t' și diferența $x' - y'$. Atunci ultimul moment (și, implicit, ultima proveniență) a suprascrierii unei poziții este dată de cel mai mare moment de timp **dintre toți strămoșii poziției**.

3.3.6 Problema PHF (FMI No Stress 2013)

enunț • sursă

Problema ne cere să simulăm un șir de meciuri de piatră-hîrtie-foarfecă de tip „cîștigătorul la masă" și să admitem actualizări punctuale pe acest șir. Deoarece nu ne permitem o simulare în $\mathcal{O}(n)$ pentru fiecare din cele q actualizări, vom căuta să accelerăm simularea la $\mathcal{O}(\log n)$.

Caracterul de pe fiecare poziție, să-i spunem X, este un meci între X și cîștigătorul meciului de pe poziția anterioară. Echivalent, X este o funcție definită pe mulțimea $\{P, H, F\}$ cu valori tot în $\{P, H, F\}$, unde $X(c)$ este chiar rezultatul unui meci între X și c. De exemplu, P este funcția:

$$\begin{cases} P(P) &= P \\ P(H) &= H \\ P(F) &= P \end{cases}$$

Atunci o înșiruire de caractere este o compunere de funcții. De exemplu, dintr-un șir de intrare de patru caractere, numite generic $XYZT$, îl tratăm pe X ca argument, iar rezultatul final este $T(Z(Y(X)))$ sau $(T \circ Z \circ Y)(X)$.

Orice funcție are nevoie de un argument. 🙂 De aceea, tratăm separat primul caracter, iar pe celelalte $n-1$ le punem într-o structură. (O altă abordare este să definim primul caracter ca pe o funcție care returnează acel caracter independent de intrarea fictivă). Această structură trebuie să mențină rezultatul compunerii caracterelor, cu modificări. Vom folosi un arbore de intervale unde informația dintr-un nod este funcția compusă a intervalului subîntins. Reprezentăm aceste funcții prin tabelul complet (trei valori). Tabelele frunzelor le definim manual, iar tabelul unui nod intern este compunerea tabelelor celor doi fii. Tabelul rădăcinii este ceea ce ne interesează:

compunerea pozițiilor $2 \dots n$ din șir, adică o funcție pe care o vom aplica primului caracter din șir.

Implementarea mea rotunjește numărul de noduri la o putere a lui 2. De aceea la dreapta vom avea și noduri vide, pe care le tratăm ca pe funcții identice ($X(c) = X$).

3.3.7 Problema Points (Codeforces)

enunț • sursă

Problema are rating de 2800 pentru că se compune din multe blocuri, dar niciunul nu este de speriat, căci sîntem deja versați în arbori de intervale. 😎 Aș zice că problema ar fi grea la un baraj ONI sau ușoară la lot.

Ca să putem construi un arbore de intervale, în primul rînd normalizăm coordonatele x. Păstrăm și o tabelă cu valorile originale, căci pe acelea trebuie să le afișăm.

Am putea reformula întrebarea pentru operația `find x y`: dintre toate punctele cu $x' > x$, există vreunul cu $y' > y$? Ne gîndim că am putea folosi un AINT de maxime, indexat după x, cu valori din y, cu interogarea: „Caută maximul pe intervalul $[x + 1, n)$ și spune-mi dacă este mai mare decît y”.

Dar astfel aflăm doar dacă există un punct. Ca să-l găsim, întrebarea corectă este: „dă-mi cea mai din stînga poziție după x pe care maximul depășește y”. Din fericire, putem face asta cu același AINT de maxime, așa cum am explicat în secțiunea de teorie:

1. Pornind de la prima poziție validă (în cazul nostru, $x + 1$), mergem în sus și spre dreapta, spre intervale tot mai mari, pînă cînd găsim o poziție de valoare $> y$. Ca să evităm cazurile particulare, adăugăm la finalul vectorului o santinelă de valoare infinită.
2. De la această poziție, coborîm în timp ce menținem în vizor valoarea $> y$. Dacă putem coborî în orice direcție, preferăm stînga.

Astfel putem gestiona operațiile de adăugare (cînd maximul pentru un x fixat poate doar să crească). Următoarea întrebare este cum gestionăm ștergerile. Cea mai directă soluție este să menținem cîte un `set` STL pentru fiecare coordonată x. Suma mărimilor acestor seturi nu va depăși n. Cu metoda `rbegin()` putem afla noul maxim după inserări și ștergeri.

Ultima întrebare, odată ce stabilim că răspunsul pentru `find x y` este la abscisa x', este: care dintre punctele cu această abscisă este răspunsul? Folosim același set și metoda `upper_bound()` pentru a afla cel mai mic y' strict mai mare decît y.

Complexitatea soluției este $\mathcal{O}(n \log n)$, atît pentru normalizarea inițială cît și pentru procesarea operațiilor. Fiecare operație necesită o căutare în set și o căutare sau actualizare în AINT.

3.3.8 Problema Medwalk (Lot 2025)

enunț • sursă

Problema admite și o soluție diferită, mult mai rapidă, bazată pe AIB-uri 2D, dar iată o soluție care folosește doar arbori de intervale.

Din enunț putem defini forma drumului: el va merge pe linia de sus a unor coloane, apoi va folosi ambele linii de pe o coloană c pentru a coborî, apoi va merge pe linia de jos a coloanelor rămase. Acum, să presupunem că avem un oracol care, pentru orice interogare, ne spune coloana c. Atunci vom muta restul coloanelor fie în stînga, fie în dreapta lui c, pentru a folosi valoarea de sus sau de jos, oricare este mai mică.

Cu alte cuvinte, mulțimea de valori de pe drumul care minimizează medianul constă din

- minimele de pe toate coloanele;
- minimul dintre maximele de pe coloane.

Răspunsul la fiecare interogare este elementul median al acestei mulțimi. Logica pentru a afla a k-a valoare este relativ simplă și implică trei valori: al k-lea minim, al $k-1$-lea minim și minimul maximelor. De aici înainte, putem abstractiza matricea ca doi vectori, unul cu minimele perechilor și altul cu maximele. De exemplu, cînd o coloană se modifică din (3, 6) în (3, 2), atunci minimul se modifică din 3 în 2, iar maximul din 6 în 3.

De aceea, avem nevoie de două structuri independente:

- O structură pentru maxime, care să admită actualizări punctuale și interogare de minim pe interval.
- O structură pentru minime, care să admită actualizări punctuale și interogări de al k-lea element pe interval.

Pentru prima structură, ochiul nostru de-acum experimentat ne spune că putem folosi un simplu AINT. Dar pentru a doua? Am găsit pe StackOverflow o idee bine explicată, pe care o reiau.

Vom folosi un arbore de intervale **pe valori**. Așadar, nu indexăm pozițiile conform cu pozițiile din vector, ci cu valorile existente în vector. Fiecare frunză din aint, corespunzătoare unei valori v, reține o colecție ordonată (un `set`, în esență) cu pozițiile pe care apare valoarea v. Fiecare nod intern reține reuniunea colecțiilor fiilor săi. Cu alte cuvinte, dacă un nod subîntinde valorile $[l, r]$, colecția sa va enumera toate pozițiile pe care apar valori între l și r.

Remarcăm că memoria necesară este $\mathcal{O}(n \log V_{max})$, deoarece aint-ul conține V_{max} valori, deci are înălțime $\log V_{max}$, iar fiecare poziție din vectorul original va fi enumerată în $\log V_{max}$ colecții.

Pentru actualizare, trebuie să ștergem poziția modificată din lista vechii valori minime și din listele tuturor strămoșilor. Apoi inserăm poziția în listele noii valori minime. De exemplu, dacă minimul coloanei 100 se modifică din 30 în 20, atunci de la poziția 30 din aint și din toți strămoșii eliminăm elementul 100 din colecție. Apoi la poziția 20 în aint și în toți strămoșii inserăm elementul 100.

Rămîne să descriem interogările. Pentru a afla al k-lea minim dintr-un interval de coloane $[l, r]$, pornim din rădăcina arborelui de intervale (luînd așadar în calcul toate valorile de la 0 la V_{max}). Consultăm fiul stîng (valorile $1 \ldots V_{max}/2$) și ne întrebăm: cîte apariții au aceste valori pe poziții din $[l, r]$? Putem răspunde la această întrebare printr-o diferență, reducînd întrebarea la forma:

cîte apariții au aceste valori pe pozițiile $0 \dots r$? Așadar, trebuie numărate elementele mai mici sau egale cu r din setul rădăcinii. Set-ul simplu din STL nu poate gestiona această întrebare, dar putem folosi un set extins din PB/DS. Nu detaliem acum, dar ne vom reîntîlni cu acest tip de date.

Dacă numărul de valori între 0 și $V_{max}/2$ care apar pe poziții între l și r este $\geq k$, atunci acolo se va afla și al k-lea element, deci coborîm în fiul stîng. Altfel coborîm în fiul drept.

Complexitatea algoritmului este $\mathcal{O}((n+q) \log n \log V_{max})$. De exemplu, fiecare interogare coboară $\log V_{max}$ niveluri, iar la fiecare nivel face o căutare într-un set de $\mathcal{O}(n)$ elemente în timp $\mathcal{O}(\log n)$.

Capitolul 4

Arbori de intervale cu propagare *lazy*

4.1 Operații pe interval

Să reluăm exemplul din capitolul trecut și să spunem acum că dorim să adăugăm 100 pe intervalul [2, 12], corespunzător nodurilor [12, 28] din arbore.

Ca să nu facem efort $\mathcal{O}(n)$, vom descompune intervalul ca mai înainte și vom nota informația „+100" în nodurile 9, 5, 6 și 28, cu semnificația că valoarea reală a tuturor frunzelor de sub aceste noduri a crescut cu 100.

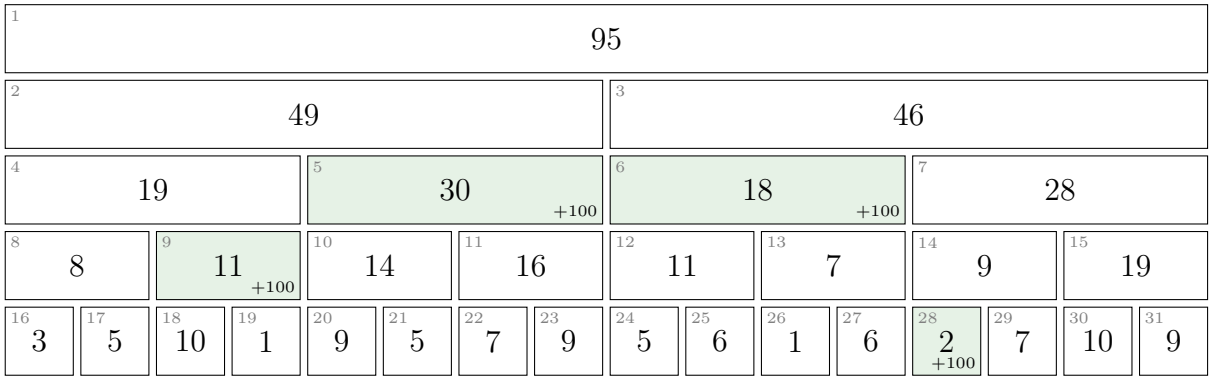


Figura 4.1: Pentru a adăuga 100 pe intervalul [18, 28], notăm valoarea *lazy* 100 pe nodurile 9, 5, 6 și 28.

Aceasta este o **informație *lazy***: o informație care stă într-un nod intern și care trebuie propagată tuturor frunzelor subîntinse de acel nod. Totuși, amînăm efortul acestei propagări pînă cînd el devine strict necesar; tocmai de aceea se numește **propagare *lazy***. (Mulți elevi denumesc întreaga structură „AINT cu *lazy*", dar asta este... lene.)

Evaluarea *lazy* este un concept des întîlnit:

- Memoizarea unor valori într-un vector / matrice, cu speranța că nu va fi nevoie să calculăm tabelul complet.
- Amînarea evaluării lui `y` în expresia booleană `x || y`, cu speranța că `x` va fi evaluat ca adevărat, iar `y` va deveni irelevant.

- Inițializarea unei componente costisitoare dintr-un program doar cînd devine necesară (o conexiune la baza de date, o zonă a hărții dintr-un joc).

Așadar, definim un al doilea vector numit `lazy` și executăm `lazy[x] += 100` pe pozițiile 9, 5, 6 și 28.

Motivul pentru care treaba se complică este următorul. Dacă acum primim o interogare de sumă pe intervalul [25,29]? Nu putem să însumăm, ca de obicei, pozițiile 25, 13 și 14, căci pierdem din vedere că unele dintre noduri au (cîte) +100. Sigur, putem lua asta în calcul, dar trebuie să clarificăm operațiile, altfel efortul poate deveni $\mathcal{O}(n)$.

În primul rînd, introducem două funcții noi (le puteți include în alte funcții, dar pentru claritate le puteți declara de sine stătătoare):

- `push()`, care propagă informația *lazy* de la un nod la fiii săi;
- `pull()`, care combină în părinte informația din cei doi fii după o actualizare.

```
void push(int node, int size) {
  s[node] += lazy[node] * size;
  lazy[2 * node] += lazy[node];
  lazy[2 * node + 1] += lazy[node];
  lazy[node] = 0;
}

void pull(int node, int size) {
  s[node] =
    s[2 * node] + lazy[2 * node] * size / 2 +
    s[2 * node + 1] + lazy[2 * node + 1] * size / 2;
}
```

Pentru problema dată (dar nu pentru toate problemele), codul are nevoie să știe numărul de frunze subîntinse (`size`).

Să presupunem acum că dorim să calculăm suma intervalului [2, 12] și că este posibil să avem niște sume *lazy* în multe alte noduri. Știm că codul descompune interogările în intervale mai scurte și nu urcă mai sus de acestea. Dacă există valori *lazy* mai sus (să zicem în rădăcină), codul nu va afla de ele. De aceea, în pregătirea interogării, trebuie să vizităm toți strămoșii intervalului și să propagăm în jos (*push*) informația *lazy*. Dar, dacă ne gîndim, lista completă a acestor strămoși constă doar din strămoșii capetelor de interval! Pentru intervalul [18,28], este nevoie să propagăm în jos informația *lazy* din strămoșii lui 18 (adică 1, 2, 4 și 9) și ai lui 28 (adică 1, 3, 7 și 14).

Dacă apelăm push din acești strămoși, de sus în jos, avem garanția că informația pe intervalele dorite este la zi. Vă rămîne vouă ca experiment de gîndire să demonstrați că, după operațiile *push*, nu va mai exista informație *lazy* în niciun strămoș al niciunei poziții din interogare.

Astfel obținem o funcție foarte similară cu cea din capitolul trecut

```
void push_path(int node, int size) {
```

```
  if (node) {
    push_path(node / 2, size * 2);
    push(node, size);
  }
}

long long query(int l, int r) {
  long long sum = 0;
  int size = 1;

  l += n;
  r += n;
  push_path(l / 2, 2); // pornim din părinte
  push_path(r / 2, 2);

  while (l <= r)  {
    if (l & 1) {
      sum += s[l] + lazy[l] * size;
      l++;
    }
    l >>= 1;

    if (!(r & 1)) {
      sum += s[r] + lazy[r] * size;
      r--;
    }
    r >>= 1;
    size <<= 1;
  }

  return sum;
}
```

Dacă viteza este crucială, putem scrie și o funcție `push_path` cu circa 10% mai rapidă, iterativă, folosind operații pe biți. Să considerăm nodul $22 = 10110_{(2)}$. Strămoșii lui sînt 1, 2, 5 și 11 care au respectiv reprezentările binare 1, 10, 101 și 1011, care sînt fix prefixele lui 10110! Deci îl vom deplasa pe 10110 la dreapta cu 4, 3, 2 și respectiv 1 bit pentru a-i obține strămoșii.

```
void push_path(int node) {
  int bits = 31 - __builtin_clz(n);
  for (int b = bits, size = n; b; b--, size >>= 1) {
    int x = node >> b;
    push(x, size);
  }
}

// Acum primul apel este chiar din frunză:
...
push_path(l);
```

```
push_path(r);
...
```

Actualizările sînt foarte similare. Apelăm `pull()` după terminarea actualizărilor, deoarece trebuie să lăsăm arborele într-o stare coerentă și trebuie să preluăm orice modificare de la fii.

```
void pull_path(int node) {
  for (int x = node / 2, size = 2; x; x /= 2, size <<= 1) {
    pull(x, size);
  }
}

void update(int l, int r, int delta) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;
  push_path(l / 2, 2);
  push_path(r / 2, 2);

  while (l <= r)  {
    if (l & 1) {
      lazy[l++] += delta;
    }
    l >>= 1;

    if (!(r & 1)) {
      lazy[r--] += delta;
    }
    r >>= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}
```

Iată și o altă implementare care combină bucla `while` principală cu funcția `pull_path`.

Notă: În această implementare, valoarea *lazy* se aplică întregului subarbore, inclusiv nodului însuși. În implementarea de pe CP Algorithms, valoarea *lazy* se aplică subarborelui fără nodul însuși. Oricare dintre formulări este acceptabilă, cîtă vreme o folosiți consecvent.

Notă: În practică, cîmpurile `lazy` și `s` merită încapsulate într-un `struct`. Datorită localității acceselor la memorie, diferența de viteză este notabilă (circa 25%). Aici le-am lăsat separate pentru concizie.

4.2 Implementare recursivă (actualizări punctuale)

Lecția trecută am spus că există și o implementare recursivă. Să o examinăm acum (mulți o știți deja).

```
void update(int node, int pl, int pr, int pos, int delta) {
  if (pr - pl == 1) {
    s[node] += delta;
  } else {
    int mid = (pl + pr) >> 1;
    if (pos < mid) {
      update(2 * node, pl, mid, pos, delta);
    } else {
      update(2 * node + 1, mid, pr, pos, delta);
    }
    s[node] = s[2 * node] + s[2 * node + 1];
  }
}
```

Metoda recursivă cară după ea 5 parametri:

- `node`: nodul curent din arbore (aka poziția în vector);
- `pl`, `pr`: intervalul din vectorul inițial acoperit de `node`. Eu am optat pentru implementarea cu `pl` inclusiv și `pr` exclusiv. Dacă preferați intervale închise, este OK.
- `pos`, `delta`: poziția de modificat și valoarea de adăugat/scăzut.

Vedem că funcția coboară recursiv în fiul stîng sau fiul drept, după caz. Un exemplu de apel ar fi:

```
update(1, 0, n, some_pos, some_val);
```

Mai interesant, iată și implementarea funcției de interogare (sumă pe interval):

```
long long query(int node, int pl, int pr, int l, int r) {
  if (l >= r) {
    return 0;
  } else if ((l == pl) && (r == pr)) {
    return s[node];
  } else {
    int mid = (pl + pr) >> 1;

    return
      query(node * 2, pl, mid, l, min(r, mid)) +
      query(node * 2 + 1, mid, pr, max(l, mid), r);
  }
}
```

Regăsim trei din aceiași parametri, `node`, `pl` și `pr`. În plus,

- `l`, `r`: Intervalul [închis, deschis) pe care dorim să calculăm suma.

Funcția se reapelează pe cei doi fii, restrîngînd corespunzător intervalul $[l, r)$. Iată o imagine care arată arborele de apeluri pentru calculul sumei pe intervalul [3, 10):

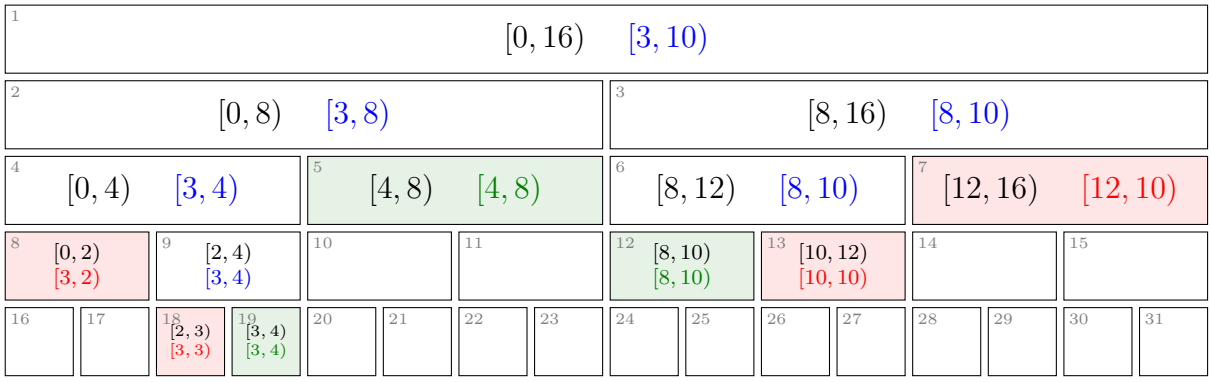


Figura 4.2: Arborele de apeluri în funcția de actualizare recursivă pe intervalul $[3, 10)$.

Am ilustrat cu albastru nodurile care au nevoie să-și apeleze descendenții, cu verde nodurile selectate integral, iar cu roșu nodurile eliminate.

Complexitatea rămîne $\mathcal{O}(\log n)$, deși funcția se reapelează pentru ambii fii. De ce?

Discutăm implementarea recursivă pentru că ea plutește prin supa culturală și vreau să puteți citi cod scris astfel. Dar ea este un exemplu de dopaj, de implementare repetată *mot à mot* indiferent de nevoile problemei. Implementarea recursivă este de 2-3 ori mai lentă decît cea iterativă pentru actualizări punctuale. Presupun că există două motive:

1. Implementarea recursivă cară după ea 5-6 parametri la fiecare apel, care trebuie copiați, puși/scoși de pe stivă etc. Implementarea iterativă folosește doar 3 variabile.
2. Implementarea recursivă este nevoită să pornească din rădăcină, să coboare pînă la frunze, apoi să revină din recursivitate. Implementarea iterativă se oprește imediat ce termină de descompus intervalul $[l, r]$.

La varianta cu propagare *lazy* diferența de timp aproape dispare, pentru că ambele implementări trebuie să urce pînă la rădăcină.

Nu vă năpustiți la implementarea recursivă dacă nu este nevoie. Rezistați tentației de a fi leneși, de a învăța o singură structură de date, pe care să o pictați indiferent de situație! Trebuie să aspirați la mai mult de atît, dacă este să vă meritați locul în lot.

4.3 Implementare recursivă (actualizări pe interval)

În această implementare, observăm cum:

- apelăm `push` înainte de reapelarea recursivă, pentru a-i garanta fiecărui nod că deasupra sa nu mai există informații lazy;

- apelăm `pull` după revenirea din recursivitate, ca să lăsăm arborele într-o stare coerentă.

```
long long query(int node, int pl, int pr, int l, int r) {
  if (l >= r) {
    return 0;
  } else if ((l == pl) && (r == pr)) {
    return s[node] + lazy[node] * (r - l);
  } else {
    push(node, pr - pl);
    int mid = (pl + pr) >> 1;
    return
      query(node * 2, pl, mid, l, min(r, mid)) +
      query(node * 2 + 1, mid, pr, max(l, mid), r);
  }
}

void update(int node, int pl, int pr, int l, int r, int delta) {
  if (l >= r) {
    return;
  } else if ((l == pl) && (r == pr)) {
    lazy[node] += delta;
  } else {

    push(node, pr - pl);
    int mid = (pl + pr) >> 1, child_size = (pr - pl) >> 1;
    update(2 * node, pl, mid, l, min(r, mid), delta);
    update(2 * node + 1, mid, pr, max(l, mid), r, delta);
    pull(node, child_size);

  }
}

void process_ops() {
  for (int i = 0; i < num_queries; i++) {
    if (q[i].t == OP_UPDATE) {
      update(1, 0, n, q[i].l - 1, q[i].r, q[i].val);
    } else {
      answer[num_answers++] = query(1, 0, n, q[i].l - 1, q[i].r);
    }
  }
}
```

4.4 Arta proiectării unui arbore de intervale

Atunci cînd primim o problemă și avem de proiectat o structură de date, cum știm dacă un arbore de intervale se potrivește scopului? În general, trebuie să urmărim trei lucruri.

În primul rînd, pentru a putea procesa rapid actualizările pe interval, dorim să facem efort $\mathcal{O}(1)$

în fiecare interval elementar din descompunere. De aceea, în general informația *lazy* stochează fix ce primim de la operațiile de *update*: o cantitate de adăugat sau de atribuit, un bit de semn, un bit care arată că intervalul curent trebuie inversat etc.

Este nevoie de atenție la compunerea actualizărilor. Dacă avem două cantități de adăugat pe același interval, cîmpul *lazy* va reține suma cantităților. Dacă avem două atribuiri succesive pe același interval, cîmpul *lazy* o va reține doar pe ultima.

În al doilea rînd, pentru a putea procesa rapid interogările pe interval, dorim să facem efort $\mathcal{O}(1)$ în fiecare interval din descompunere. De aceea, informația propriu-zisă din fiecare nod trebuie să includă valorile pe care le cer operațiile de *query*. Acestea sînt un punct de pornire, dar uneori nu sînt suficiente, ci este nevoie să menținem mai multe valori din care să le putem alege pe cele cerute.

În sfîrșit, în al treilea rînd trebuie să verificăm că avem tot ce ne trebuie pentru a compune două intervale alăturate la interogare, pentru a recalcula un părinte din cei doi fii ai săi (operația *pull*) și pentru a propaga informația *lazy* de la părinte la fii (operația *push*).

O regulă de aur este că, la începutul și la sfîrșitul fiecărei funcții, arborele trebuie să fie într-o stare **coerentă**. Aceasta înseamnă că trebuie să definim un **contract**, o promisiune despre care este structura logică a fiecărui nod. Vă recomand chiar să notați acest contract într-un comentariu de una-două fraze, la începutul codului pentru AINT. Apoi, scrieți codul astfel încît, la intrarea și la ieșirea din orice funcție, toate nodurile arborelui să respecte acel contract.

De exemplu, dacă informația *lazy* este o cantitate de adăugat pe tot subarborele, atunci operația *push* trebuie neapărat să se încheie prin a pune pe 0 valoarea *lazy* din părinte. În niciun caz nu trebuie să încheiem operația *push* lăsînd aceeași valoare *lazy* în părinte și în fii.

4.5 Probleme

4.5.1 Problema Polynomial Queries (CSES)

enunț • surse

Problema seamănă mult cu cea discutată la teorie, dar pe intervale nu mai adăugăm constante, ci progresii aritmetice. Așadar, pare natural să reținem exact această informație *lazy*: în fiecare nod reținem că în fiecare frunză acoperită de acel nod trebuie să adăugăm cîte un termen al unei progresii cu un anumit prim element și pasul (deocamdată) 1. De exemplu, dacă în figura 4.1 facem o actualizare pe intervalul [18, 28], atunci în nodul 5 notăm progresia cu primul termen 3 și pasul 1. Informația *lazy* este o pereche $\langle 3, 1 \rangle$.

Trebuie tratate atent diversele cazuri care iau naștere. Dacă două progresii acoperă același interval, vor lua naștere progresii cu pas mai mare decît 1. Să luăm un exemplu:

- Progresia cu primul termen 5 și pasul 3, așadar 5, 8, 11, 14, ...
- Progresia cu primul termen 2 și pasul 7, așadar 2, 9, 16, 23, ...

- După însumare dorim să avem termenii 7, 17, 27, 37, ...
- Rezultă că suma este și ea o progresie cu primul termen 7 și pasul 10. Cu alte cuvinte, informațiile *lazy* se pot compune ușor: $\langle 5, 3\rangle + \langle 2, 7\rangle = \langle 7, 10\rangle$.

La propagarea în jos a informației *lazy*, în cei doi fii vom adăuga progresii cu același pas. În fiul drept, primul termen trebuie calculat, dar este ușor. Dacă într-un nod care acoperă 16 elemente avem o progresie cu primul element 3 și pasul 5, atunci fiul drept va începe cu al nouălea termen al progresiei:

$$3 + 5 \cdot (16/2) = 43$$

La operațiile de adăugare, pe toate intervalele din descompunere vom aduna progresii cu pasul 1, dar primul element diferă pentru fiecare interval (la fel, nu este greu de calculat).

Contractul pe care l-am ales pentru implementarea iterativă este:

- `first` și `step` înseamnă că pe nodurile din intervalul acoperit trebuie adăugate valorile `first`, `first + step`, `first + 2 * step`, ...
- Valoarea `s` din fiecare nod **nu** include și suma progresiei dată de `<first, step>` din acel nod.

4.5.2 Problema Nezzar and Binary String (Codeforces)

enunț • sursă

Problema este relativ directă odată ce „ne prindem” că trebuie să procesăm operațiile în ordine inversă. Știm șirul final f și fie $[l, r]$ ultimul interval inspectat de Nanako. În momentul inspecției, șirul curent trebuia să fie identic cu f pe pozițiile $[1, l) \cup (r, n]$, căci pe acelea nu le putem modifica. Pe pozițiile $[l, r]$ trebuiau să fie doar biți 0 sau doar biți 1. Care dintre ele? Știm că la ultima modificare am modificat strict mai puțin de jumătate din biți. Să notăm cu z numărul de zerouri și cu u numărul de unu de pe pozițiile $[l, r]$ din f. Iau naștere trei cazuri:

1. Dacă $z > u$, înseamnă că la pasul anterior $[l, r]$ conținea doar 0.
2. Dacă $z < u$, înseamnă că la pasul anterior $[l, r]$ conținea doar 1.
3. Dacă $z = u$, problema nu are soluție, căci nu putem opera modificarea necesară.

Astfel, toate operațiile sînt forțate, mergînd înapoi în timp. Răspunsul este `YES` doar dacă putem procesa toate operațiile, iar la final ajungem la șirul s.

Rezultă că, pentru a efectua efectiv operațiile, avem nevoie de un arbore de segmente cu valori de 0 și 1 în frunze, cu funcții de sumă pe interval (pentru a stabili majoritatea) și de atribuire pe interval (pentru a face *undo* la o operație).

4.5.3 Problema Simple (infO(1)Cup 2019)

enunț • sursă

Să aplicăm regula menționată ca să proiectăm un arbore de intervale pentru această problemă.

- În cîmpul *lazy* vom stoca valorile primite la update, așadar cantitățile de adăugat pe tot subarborele.
- În cîmpurile propriu-zise vom stoca valorile necesare pentru interogări, așadar minimul par pe interval și maximul impar pe interval.
- Ce altceva ne mai trebuie ca să putem menține informația la actualizări? Să observăm că o cantitate *lazy* impară schimă paritatea valorilor pe întregul interval. Noul minim par este fostul minim impar, plus cantitatea *lazy*. De acea, vom introduce încă două cantități: maximul par și minimul impar.

Ca de obicei, este important ca la implementare să alegem dacă valoarea *lazy* este deja inclusă în nodul curent și mai trebuie aplicată doar la subarbore sau dacă ea trebuie aplicată inclusiv nodului curent. Eu am ales prima variantă. Ambele sînt bune, cîtă vreme codul respectă alegerea făcută.

Pe unele intervale nu vor exista valori pare sau impare. Dacă facem cazuri speciale pentru toate acele situații, vom avea undeva între 5 și 10 `if`-uri de presărat prin cod. O abordare mai simplă este să notăm în acele noduri $+\infty$ pentru a arăta că nu există minime și $-\infty$ pentru a arăta că nu există maxime. Apoi lăsăm aceste valori să se combine fără să mai tratăm cazuri particulare. La final, știm că orice valori definite vor fi între 1 și $2 \cdot 20^9 + 2 \cdot 20^5 \cdot 2 \cdot 20^9$, conform limitelor din enunț. Valorile din afara acestui interval le interpretăm ca fiind nedefinite.

În cod am încercat să separ funcțiile specifice nodului de funcțiile specifice arborelui.

4.5.4 Problema Balama (Baraj ONI 2024)

enunț • surse

Vă veți întîlni des cu probleme unde soluția devine simplă dacă analizăm informațiile în altă ordine. În cazul de față, în loc să luăm în calcul liniile (care sînt subsecvențe ordonate), să analizăm coloanele.

Care va fi răspunsul pe ultima coloană? Desigur, va fi maximul din vector. Mult mai interesantă este întrebarea: care va fi răspunsul pe penultima coloană? Va fi cel mai mare element care este vreodată (în cel puțin o fereastră) **al doilea maxim**.

Exemplu: fie maximul global 1 000 și fie al doilea maxim global 999. Dacă 999 stă foarte departe de 1 000, la distanță de cel puțin k, atunci 999 va fi maxim în toate ferestrele de lățime k care îl conțin. Cu alte cuvinte, 999 se va regăsi doar pe ultima coloană în matricea B (și va fi mascat de 1 000). Să spunem că următoarele valori din șir, în ordine descrescătoare, sînt 998, 997 și 996 și toate se află la distanță mare unele de altele. Niciunul dintre ele nu va apărea pe penultima coloană în B.

În schimb, să spunem că următoarea valoare ca mărime, 995, se află aproape (la distanță $< k$) de o valoare anterioară, cum ar fi 997. Atunci există o fereastră în care 995 și 997 coexistă, deci 995 va fi al doilea maxim din acea fereastră și va apărea pe penultima coloană în B. Cum alte valori mai mari nu au această proprietate, 995 este răspunsul pe penultima poziție a soluției.

(Amănunt esențial 😉: 995 nu poate fi și al treilea maxim. Dacă exista o fereastră de lățime k care îl cuprindea pe 995 și alte două valori anterioare, atunci înainte să ajungem la 995 una dintre acele două valori anterioare ar fi fost al doilea maxim).

Astfel, putem considera elemente în ordine descrescătoare și, pentru fiecare element x ne întrebăm: cîte elemente văzute anterior conține fiecare dintre ferestrele care îl conțin pe x (cel mult k la număr)? Dacă o astfel de fereastră are e elemente, și dacă a $e+1$-a valoare din soluție (numărînd de la dreapta) este încă necunoscută, atunci pune x pe poziția $e+1$ a soluției.

Exemplu: Dacă considerăm elementul 900 și constatăm că într-una din ferestrele care îl conțin pe 900 existau deja alte 5 valori, atunci în acea fereastră 900 este al 6-lea element. Dacă a 6-a poziție din soluție este încă neocupată, scriem 900 acolo, acesta fiind maximul posibil.

Implementarea sună fioros, dar nu este! În realitate avem nevoie de o structură cu două operații:

- Incrementează pozițiile de la st la dr, pentru a arăta că în ferestrele de la $[st, st+k-1]$ și pînă la $[dr, dr+k-1]$ avem cîte un element în plus.
- Află maximul de pe pozițiile de la st la dr.

Operația a doua ne este suficientă deoarece ferestrele nu vor ajunge brusc la 6 elemente. Vor apărea mai întîi ferestre cu 1, 2, 3, 4, 5 elemente. Cu alte cuvinte, soluția se completează de la dreapta spre stînga.

Vom implementa un AINT de maxime în care informația *lazy* din fiecare nod este valoarea de adăugat pe fiecare poziție din intervalul acoperit.

4.5.5 Problema A Simple Task (Codeforces)

enunț • surse

Problema cere să sortăm la cerere porțiuni dintr-un string, apoi să afișăm rezultatul.

Este nevoie să facem fiecare operație de sortare în $\mathcal{O}(\log n)$, deci vom folosi un arbore de segmente cu propagare *lazy*. Ce stocăm în fiecare nod?

Varianta 1

Într-o primă variantă, stocăm 26 de arbori de „biți”, cîte unul pentru fiecare literă a alfabetului. La sortare, colectăm numărul de apariții ale fiecărui caracter în parte. Apoi calculăm poziția secvenței (contigue) de 1-uri pe care o ocupă acel caracter în subsecvența sortată. Setăm acea porțiune pe 1, iar capetele stînga și dreapta le setăm pe 0. Așadar, informația *lazy* ar putea fi: întregul subarbore al acestui nod este 0 (respectiv 1).

De exemplu, dacă stabilim că un nod care acoperă intervalul [16,24) va conține șirul „abcdefgh”, atunci litera „d” va ocupa poziția 19. În AINT-ul pentru litera „d” vom seta un 1 la poziția 19 și 0 la pozițiile [16,19) și [20,24).

Varianta 2

Stocăm un singur arbore în care informația dintr-un nod este un vector f de 26 de elemente cu frecvențele fiecărei litere. Atunci informația *lazy* poate avea 3 valori:

1. Frecvențele din f sînt corecte, dar trebuie propagate la fii în ordine crescătoare.
2. Frecvențele din f sînt corecte, dar trebuie propagate la fii în ordine descrescătoare.
3. Frecvențele din f sînt corecte și au fost propagate la copii.

Atunci o sortare constă din

- Colectarea frecvențelor din intervalele acoperite.
- Redistribuirea frecvențelor în aceleași intervale, de la a la z sau de la z la a.

Operația `push` trebuie să împartă un vector de frecvențe în doi vectori, în care literele mai mici sînt repartizate fie în stînga (cînd `lazy == 1`), fie în dreapta (cînd `lazy == 2`).

Detalii de implementare

- Am parametrizat buclele de la a la z și de la z la a ca să nu duplic codul.
- Am separat logic codul pentru noduri (care știe să adune și să împartă vectori de frecvențe) de codul pentru AINT (care știe să colecteze și să distribuie recursiv aceste frecvențe).
- Am folosit o variabilă-acumulator locală struct-ului `segment_tree`, ca să nu-l trimit ca parametru peste tot.
- Am denumit funcțiile `collect` și `distribute` în loc de `query` și `update`, ca să folosesc un limbaj specific problemei.

Este nevoie de atenție la un caz particular. Cînd propagăm în sus informația la finalul unei sortări, nu trebuie să însumăm fiii în acei părinți care au informație *lazy*. Acei părinți au deja frecvențele bine calculate, în schimb fiii lor au informație învechită.

4.5.6 Problema Periodic RMQ Problem (Codeforces)

enunț • sursă

Principala dificultate a problemei este observația că, din cele $k \cdot n$ coordonate, cel mult $2q$ sînt de interes: capetele operațiilor. Deci putem proceda astfel:

1. Colectăm capetele operațiilor și intervalele dintre ele.
2. Tratăm capetele ca pe niște intervale de lățime 1.
3. Calculăm RMQ-ul pe fiecare interval.
4. Normalizăm de la 0 aceste intervale, și deci și coordonatele operațiilor.
5. Construim un arbore de segmente cu cel mult $4q$ puncte, cu operații de atribuire pe interval și minim pe interval.
6. Procesăm operațiile.

O clarificare: de ce este nevoie să colectăm separat capetele de operații? Deoarece acestea pot și începuturi, și sfîrșituri de interval. De exemplu, dacă avem operații pe [5, 10] și [10, 20], atunci

poziția 10 nu poate fi comasată nici cu [5,9], nici cu [11, 20], ci trebuie să stea separat.

Calculul RMQ-ului periodic este relativ facil. Intervalele de lungime cel puțin n acoperă complet vectorul b, deci pot returna minimul vectorului. Intervalele mai scurte de n acoperă fie o perioadă, fie două (un sufix și un prefix de perioadă). Le putem procesa cu o structură clasică (*sparse table*).

Normalizarea necesită atenție, de exemplu la perechi de coordonate consecutive. Într-o primă variantă, codul meu insera un interval fictiv, de RMQ infinit, între două coordonate ca 1 000 și 1 001. Dar aceasta ducea la răspunsuri greșite.

Normalizarea cu un simplu vector pare considerabil mai rapidă (sau poate este doar o diferență de evaluator). Eu am ales să colectez, pentru fiecare coordonată de interes c, valorile c și $c + 1$. Apoi în tabela rară am calculat RMQ pe intervale închise-deschise. Exemplu: pentru interogarea [10, 20] am colectat coordonatele 10, 11, 20 și 21 și am calculat RMQ pe intervalele [10, 11), [11, 20) și [20, 21). Reuniunea acestora este chiar intervalul dorit.

În sfîrșit, arborele de segmente în sine răspunde la atribuiri pe interval și interogări de minim pe interval. L-am implementat cu contractul: $m[x]$ este minimul dintre toate frunzele subîntinse de x. În plus, un cîmp boolean $dirty[x]$ este adevărat dacă și numai dacă $m[x]$ trebuie propagat la fii. Așadar, la atribuire pe interval setăm valoarea în cîmpul m și punem *dirty* pe true, iar la RMQ doar luăm minimul dintre toate valorile m întîlnite, indiferent de valoarea *dirty*.

4.5.7 Problema Circuit (Lot 2025)

enunț • sursă

Problemele de lot tind să aibă idei complicate. Este un motiv de bucurie cînd prindem una pe care o putem rezolva băbește!

Să studiem exemplul din enunț, $A = (5\ 3\ 2\ 4\ 1)$. Vom enumera toate subsecvențele și, pentru fiecare, vom nota în cîte subșiruri este fiecare dintre elemente maximul. (Desigur, o subsecvență de lungime L va avea $2^L - 1$ subșiruri.)

subsecvența	**valoarea 1**	**valoarea 2**	**valoarea 3**	**valoarea 4**	**valoarea 5**
5					1
5 3			1		2
5 3 2		1	2		4
5 3 2 4		1	2	4	8
5 3 2 4 1	1	2	4	8	16
3			1		
3 2		1	2		
3 2 4		1	2	4	
3 2 4 1	1	2	4	8	
2		1			
2 4		1		2	
2 4 1	1	2		4	
4				1	
4 1	1			2	
1	1				
total	5	12	18	33	31

Tabela 4.1: Subsecvențele șirului (5 3 2 4 1) și frecvențele maximelor.

De exemplu, în (3 2 4 1), valoarea 3 este maximă de 4 ori pentru că 3 este maxim în toate subșirurile care îl conțin pe 3, dar nu și pe 4.

Desigur, din tabel decurge răspunsul $5 \cdot 1 + 12 \cdot 2 + 18 \cdot 3 + 33 \cdot 4 + 31 \cdot 5 = 370$.

Acum, să înțelegem mai bine de unde vine totalul 33 corespunzător valorii 4. Să figurăm toate subsecvențele care îl conțin pe 4 și contribuțiile acestora la total. În plus, să marcăm cu `*` valorile mai mici decît 4, căci pe acelea avem libertatea să le includem sau nu în subșir.

```
  5   3   2   4   1
      *   *       *
[             4 ]
[             8     ]
    [         4 ]
    [         8     ]
        [     2 ]
        [     4     ]
            [ 1 ]
            [ 2     ]
     Total:  33
```

Observăm că, dacă extindem o secvență oarecare $[l, r]$ spre stînga, $[l-1, r]$, atunci contribuția ei se dublează cînd poziția $l-1$ conține o valoare mai mică decît valoarea curentă (4), altfel contribuția rămîne constantă. La fel și spre dreapta. De aceea, suma contribuțiilor este un produs de două paranteze. Contribuția din stînga este $1 + 2 + 4 + 4$, iar cea din dreapta este $1 + 2$. Produsul acestora este 33.

Dat fiind că trebuie să putem număra rapid valorile mai mici decît 4 (sau în general decît x) în stînga și în dreapta lui x, ne vine ideea să procesăm elementele în ordine crescătoare. Menținem un vector de lungime n de puteri ale lui 2. De la dreapta spre stînga, pornim cu 1 și dublăm valoarea elementului oricînd el este mai mic decît cel curent. Pentru vectorul de mai sus, la momentul procesării valorii 4, vectorul este 8 8 4 2 2. Acum, putem calcula paranteza stîngă (1+2+4+4) ca fiind suma pe prefix la poziția valorii 4 (adică 8+8+4+2) de împărțit la valoarea de pe acea poziție (2).

Pentru sufixe menținem o structură identică, fix aceeași clasă, doar oglindim pozițiile. Folosim un arbore de intervale cu operații de dublare pe prefix și sumă pe prefix.

4.5.8 Problema Babel (Baraj ONI 2025)

enunț • sursă

Soluția din editorial este extrem de elegantă, dar necesită un pic de inspirație divină. Iată soluția pe care am găsit-o eu, mai pământească.

Mai întîi, definim o recurență tipică pentru probleme cu turnurile din Hanoi. Pentru o configurație dată, fie $C(k, t)$ costul minim pentru a aduce discurile $0 \dots k$ pe tija t. Dacă discul k se află deja pe tija t, atunci $C(k, t) = C(k-1, t)$. Dacă discul k se află pe o altă tijă, t', atunci:

1. Mutăm discurile $0 \dots k-1$ pe cea de-a treia tijă, t''. Cost: $C(k-1, t'')$.
2. Mutăm discul k pe tija t. Cost: 1.
3. Mutăm discurile $0 \dots k-1$ de pe tija t'' pe t. Cost: $2^k - 1$.

Așadar, costul total este o sumă din 2^k pentru toate discurile k care nu se află pe tija așteptată.

Complicația apare pentru că aceste „așteptări" depind de așteptările anterioare. Concret, să definim trei vectori:

- $r[0 \dots n-1]$ indică tija pe care se află fiecare disc (am numerotat tijele cu 0, 1 și 2).
- $e[0 \dots n-1]$ indică tija pe care ne așteptăm să se afle fiecare disc ca să nu aibă costuri, și anume:
  - Dacă $r[i+1] = e[i+1]$, atunci $e[i] = e[i+1]$. În cuvinte, dacă $i+1$ se află unde ne dorim, iar i se află deasupra lui, nu trebuie să-l mutăm.
  - Altfel $e[i] = 3 - r[i+1] - e[i+1]$. În cuvinte, i trebuie pus pe a treia tijă.
- $f[i] = r[i] - e[i] \mod 3$.

Cu aceste notații, costul unei configurații este $\sum 2^i$ pentru discurile i pentru care $f[i] \neq 0$. Facem două precizări:

1. Vom duce toate discurile pe tija pe care se află deja discul $n-1$ (pe acela nu are sens să-l mutăm).
2. Dacă avem un interval oarecare de discuri $[x, y]$ pe aceeași tijă t, și dacă y ar fi trebuit să fie pe tija t', atunci $e[x \dots y]$ va avea forma $t'\ t''\ t'\ t''\ \dots t'$ (prima valoare depinde de paritatea lui $y - x$).

Acum să luăm un exemplu de configurație cu 12 tije și să calculăm vectorii e și f:

```
    0  1  2  3  4  5  6  7  8  9 10 11
r:  1  0  2  2  1  2  2  0  1  0  1  1
e:  1  2  2  2  0  1  0  0  2  1  1  1
f:  0  1  0  0  1  1  2  0  2  2  0  0
```

Acum să considerăm că mutăm discurile $[st, dr] = [6 - 10]$ pe tija 0 și să vedem ce se modifică:

```
    0  1  2  3  4  5  6  7  8  9 10 11
r:  1  0  2  2  1  2  0  0  0  0  0  1
e:  0  0  1  0  2  2  1  2  1  2  1  1
f:  1  0  1  2  2  0  2  1  2  1  2  1
```

Pe pozițiile $[dr + 1, n - 1]$, desigur, nu se modifică nimic.

Pe pozițiile $[st, dr]$, dacă discul dr ajunge unde ne așteptăm, atunci **toate** acele discuri ajung unde ne așteptăm, deci $f[st, dr]$ devine 0. Altfel, toate acele discuri ajung pe tijele greșite, iar f va avea una dintre formele 1 2 1 2 1... sau 2 1 2 1 2...

Pe pozițiile $[0, st-1]$, dacă $e[st-1]$ rămîne nemodificat, atunci mutarea nu afectează acele poziții, iar f rămîne nemodificat. Altfel, valorile $f[0 \dots st - 1]$ se modifică alternativ cu +1 -1 +1 -1… sau -1 +1 -1 +1… Nu am demonstrat formal această observație, dar intuitiv ea este adevărată deoarece putem calcula telescopic valoarea $f[i]$ în funcție de $r[i \dots n - 1]$ (reducînd e-urile).

Cu aceste observații, folosim trei arbori de intervale. Pentru vectorul r avem nevoie de un aint relativ simplu, cu atribuire pe interval și interogare punctuală.

Pentru vectorul f vom construi doi arbori, unul pentru pozițiile pare și unul pentru pozițiile impare. Atunci atribuirile alternante devin atribuiri de constantă (pe interval), iar modificările alternante cu ± 1 devin incrementări modulo 3 pe interval. Interogările de sumă devin sume de puteri ale lui 4, deoarece fiecare din cei doi arbori conține puteri ale lui 2 din 2 în 2.

Capitolul 5

Ștergerea din structuri de date care nu admit ștergeri

Există situații cînd avem nevoie de o structură de date S care să suporte adăugări și ștergeri. Știm să facem adăugarea într-o complexitate $\mathcal{O}(T(n))$, dar nu știm deloc să facem ștergerea. Să spunem, în schimb, că știm să facem operații *undo* (ștergerea elementelor în ordinea inversă adăugării lor). Această operație este adesea mai simplă. În această situație, putem folosi arborii de intervale ca să implementăm și ștergeri în general, dacă plătim un cost suplimentar de $\mathcal{O}(\log n)$. Așadar, complexitatea finală va fi $\mathcal{O}(T(n) \log n)$ per operație.

Iată un exemplu. Să spunem că dorim o structură care să admită adăugări, ștergeri și interogări de maxim. Am putea folosi un `set` din STL (pe care noi înșine nu-l putem implementa ușor). Dar, dacă mereu am avea de șters doar ultimul element adăugat, am putea implementa structura absolut elementar! Cum?

Folosim o singură stivă în care ținem perechi de valori: ultima valoare adăugată și maximul până la acel moment. Pentru a șterge un element, eliminăm un element din stivă, iar maximul din noul vârf al stivei este maximul elementelor rămase.

În supa culturală românească, tehnica se numește „AINT pe timp”. CP Algorithms are un articol foarte bun despre subiect.

5.1 Probleme

5.1.1 Problema Addition on Segments (Codeforces)

enunț • sursă

Simplificarea cerinței

Problema conține un mic *red herring* (pistă falsă). Superficial, ea întreabă: Dacă alegem orice mulțime de intervale, ce valori între 1 și n putem face să fie maxime? Dar o întrebare echivalentă este: Dacă alegem orice mulțime de intervale, ce valori între 1 și n putem obține?

Pentru a demonstra echivalența, să presupunem că obținem o valoare a pe o poziție oarecare, dar ea nu este maximă. Altundeva există o valoare $b > a$. Dar asta înseamnă că am selectat cel puțin un interval care îl acoperă pe b, dar nu și pe a. Eliminăm acel interval. Repetăm raționamentul pînă cînd a devine maxim.

Problema rucsacului

Cu această cerință, problema amintește de problema rucsacului 0/1. Să fixăm o poziție i. Considerăm toate valorile x pentru interogările $[l, r]$ care satisfac $l \leq i \leq r$. Rezolvăm problema rucsacului pentru acele valori și aflăm mulțimea de sume pe care le putem obține (limitate la valoarea n conform enunțului).

Rezolvarea pentru o poziție (rucsacul clasic) are complexitate de $\mathcal{O}(qn)$, așadar efortul total este $\mathcal{O}(qn^2)$, prea mare. Dar dacă am încerca să baleiem pozițiile de la 1 la n? Ca să avansăm de la poziția i la $i + 1$ și să actualizăm rucsacul, avem nevoie de o structură de date cu următoarele operații:

1. Adaugă un obiect în rucsac (pentru intervalele care încep la $i + 1$).
2. Scoate un obiect din rucsac (pentru intervalele care se termină la i).

Adăugarea este relativ cunoscută. O reiau aici pentru cei care nu au mai văzut-o. Dacă folosim al k-lea obiect de greutate x, putem obține (1) orice sume puteam obține cu primele $k-1$ obiecte și (2) aceleași sume mărite cu x.

```
for (int i = n; i >= x; i--) {
  posibil[i] |= posibil[i - x];
}
```

Dar ștergerea? Dată fiind o mulțime de sume posibile, care dintre ele rămîn posibile după ștergerea unui element? Putem încerca următoarea abordare. Să stocăm pe fiecare poziție nu doar 0/1 (putem sau nu obține o sumă), ci numărul de moduri în care putem obține această sumă. Atunci codul pentru adăugare este:

```
for (int i = n; i >= x; i--) {
  d[i] += d[i - x];
}
```

Am putea adapta acest cod și la ștergere, nu?

```
for (int i = n; i >= x; i--) {
  d[i] -= d[i - x];
}
```

... da și nu. Valorile din `d` pot fi enorme. Dacă rucsacul conține k obiecte, toate de valoare 1, atunci numărul moduri de a obține suma $k/2$ este $C_k^{k/2}$. Putem opera modulo o valoare M aleasă

la întîmplare și să sperăm că nu avem *false negatives*, adică nu raportăm 0 moduri de a obține o sumă cînd de fapt avem kM moduri cu $k \neq 0$. Dar haideți să nu trăim periculos!

Arborele de intervale

Să luăm fiecare interogare (l, r, x). Să descompunem intervalul $[l, r]$ în noduri după metoda cunoscută și să adăugăm valoarea x la o colecție (un simplu `vector`) în fiecare dintre aceste noduri. Atunci semnificația colecției este: orice valoare din această colecție este folosibilă pe orice poziție acoperită de nod. Cu alte cuvinte: mulțimea de interogări care acoperă o poziție (o frunză din arbore) este **reuniunea colecțiilor din strămoși**. Doar că nu stocăm interogările efective, căci ne interesează doar valorile x.

Să mai remarcăm și că suma mărimilor colecțiilor este $\mathcal{O}(q \log n)$, deoarece fiecare valoare x aparține la $\mathcal{O}(\log n)$ colecții.

Și acum cheia tehnicii: facem **o parcurgere DFS** a tuturor nodurilor. Dacă încă nu sînteți familiari cu parcurgerile arborilor, o definiție echivalentă este: vizităm recursiv toate nodurile arborelui. Menținem rucsacul calculat pentru toate valorile x de pe calea de la rădăcină la nodul curent. La intrarea într-un nod adăugăm la rucsac toate valorile din colecția C din nod, în $\mathcal{O}(n|C|)$, iar la revenire ștergem elementele colecției și „recalculăm rucsacul".

... Dar am ajuns din nou la nevoia ștergerilor, nu? Nu chiar. La reapelarea DFS-ului, **creăm o copie** a rucsacului curent, pe care o pasăm ca parametru nodurilor-fii. La revenirea din fii, aruncăm pur și simplu copia. (Sursa mea obține acest efect trimițînd rucsacul prin valoare, nu prin referință.) Cum spuneam, operațiile *undo* făcute în ordinea inversă adăugării tind să fie considerabil mai simple.

Valorile rucsacului într-o frunză f ne arată ce sume putem obține pe poziția f. Facem reuniunea valorilor rucsacului în toate frunzele și *voilà!*, problema este gata. Complexitatea este $\mathcal{O}(qn \log n)$. Cum spuneam, există $\mathcal{O}(q \log n)$ obiecte în toți vectorii din aint, iar adăugarea la rucsac a unui obiect necesită $\mathcal{O}(n)$.

Detaliu de implementare

Pentru a calcula rucsacul, ne este suficient un vector de booleeni. Adică un bitset. Iar, dacă citim codul, el aplică operația OR între rucsacul curent și cel shiftat la stînga cu x biți. Astfel obținem o constantă fantastică: calculul rucsacului nu iterează prin n booleeni, ci prin $n/64$ valori pe 64 de biți.

5.1.2 Problema Extending a Set of Points (Codeforces)

enunț • sursă

Această problemă este doar cu puțin mai complicată. Chiar vom avea nevoie de rollbacks („reveniri"?).

Cunoașterea soluției unei probleme poate ajuta la rezolvarea ei (una din legile lui Murphy). Dat fiind că sîntem la capitolul despre evitarea ștergerilor, să rezolvăm întîi problema doar pentru adunări.

Structura lui $E(S)$

Cum arată $E(S)$? Să ne imaginăm că S constă din două linii orizontale de ordonate y_1 și y_2, fiecare conținînd niște puncte. Dacă există două puncte cu același x, așadar (x, y_1) și (x, y_2), atunci toate punctele de pe oricare dintre linii pot fi dublate pe cealaltă linie. Așadar $E(S)$ va consta dintr-o mulțime de perechi de puncte, sau o rețea de $2k$ puncte, unde k este numărul de coordonate x distincte din S.

Extinzînd raționamentul la un set oarecare S, $E(S)$ va consta dintr-o colecție de rețele. Fiecare rețea va avea un set de $k \times l$ puncte, mai exact toate punctele dintr-un produs cartezian $\{x_1, x_2, \ldots, x_k\} \times \{y_1, y_2, \ldots, y_l\}$. Oricare două rețele din $E(S)$ nu vor avea niciun x comun și niciun y comun; altfel, cele două rețele s-ar uni prin extensii succesive. Mărimea lui $E(S)$ este suma mărimilor rețelelor (suma produselor $k \cdot l$).

Efectul adăugărilor

Cînd inserăm un punct iau naștere cîteva cazuri, după cum există sau nu rețele care să includă coordonatele x, respectiv y ale punctului. De exemplu, dacă există o rețea de dimensiuni $k \times l$ care include coordonata x și o altă rețea de dimensiuni $k' \times l'$ care include coordonata y, atunci punctul (x, y) cauzează unirea prin extensii succesive într-o rețea de dimensiuni $(k + k') \times (l + l')$.

Problema este, așadar, una relativ clasică de conectivitate, doar ceva mai complexă. O vom rezolva exact cu păduri de mulțimi disjuncte, dar modificate ca să admită și reveniri.

Procesarea ștergerilor

Știm că vom construi un arbore de intervale peste operații și că vom face un DFS pe acest arbore. Fiecare frunză corespunde unui moment în timp (un număr de operații) și în fiecare frunză vrem să calculăm $E(S)$ pe punctele active la acel moment.

Rezultă că trebuie să reprezentăm în AINT intervalele de timp pentru care fiecare punct este activ. De exemplu, dacă punctul (x, y) este adăugat la operația 10, șters la operația 20 și adăugat la operația 30, iar în total există 100 de operații (numerotate de la 0 la 99), atunci:

- Punctul este activ pe intervalul [10, 19].
- Punctul este activ pe intervalul [30, 99].

Cînd citim datele, putem folosi o tabelă hash ca să reținem ultima activare a unui punct. Cînd reîntîlnim același punct, îl marcăm ca activ pe intervalul determinat și îl ștergem din tabelă. Alternativ, putem folosi o simplă sortare în locul tabelei hash. Sortăm punctele după x, apoi după y, apoi după timp. Atunci toate activările și dezactivările unui punct apar în vectorul sortat pe poziții consecutive.

Detalii de implementare

Nu putem folosi compresia căii în DSF, deoarece ea poate face multe modificări, iar operațiile de revenire devin imposibile. În schimb, vom folosi *union by size* ca să obținem timp logaritmic pentru operațiile în DSF.

Este nevoie și să studiem cu atenție ce se modifică la o unificare, ca să nu salvăm mai mult decît este necesar. Eu am reușit să folosesc doar 6 variabile.

Nu în ultimul rînd, este important să gestionăm cît mai comod cazurile particulare la inserarea punctului (x, y):

- Nu există rețele cu coordonata x nici cu y.
- Există o rețea cu coordonata x.
- Există o rețea cu coordonata y.
- Există două rețele diferite cu coordonatele x și y.
- Există aceeași rețea care include și x, și y.

Inițial m-am chinuit să folosesc două DSF, una pentru x-i și una pentru y-i. Dar codul s-a complicat mult. În loc de asta, folosesc un singur DSF, pentru rețele. Inițializez rețele cu etichetele $0 \ldots MAX_COORD - 1$ pe axa Ox și $MAX_COORD \ldots 2 \cdot MAX_COORD - 1$ pe axa Oy. Pentru fiecare rețea rețin mărimile (numărul de x distincți și numărul de y distincți). Dacă inițializăm corect aceste mărimi, ca să respecte regula de compunere de mai sus, atunci cazurile particulare dispar.

Capitolul 6

Arbori indexați binar

Arborii indexați binar (AIB), numiți și arbori Fenwick, iar în engleză *binary indexed trees (BIT)*, servesc ca și arborii de intervale tot la rezolvarea în $\mathcal{O}(\log n)$ a unor operații pe vectori. Ei sînt mai puțin flexibili și universali decît arborii de intervale. Nu toate problemele rezolvabile cu AINT pot fi rezolvate și cu AIB. Dar acolo unde se potrivesc, AIB-urile sînt ușor de codat și sînt de 2-3 ori mai rapide decît arborii de intervale.

6.1 *Benchmarks*

Dacă se potrivesc mai multe structuri, contează pe care o alegem? Ca să alegem în cunoștință de cauză, iată niște măsurători de viteză (*benchmarks*). Le-am făcut în 2025 pe un procesor AMD Ryzen 7 4700U, la acea vreme comparabil cu evaluatoarele de la Kilonova și Codeforces.

Am măsurat timpii de rulare pentru diverse implementări ale problemei în ambele variante (actualizări punctuale sau pe interval).

- arbori indexați binar, $\mathcal{O}(\log n)$ per operație;
- arbori de segmente iterativi, $\mathcal{O}(\log n)$ per operație;
- arbori de segmente recursivi, $\mathcal{O}(\log n)$ per operație;
- descompunere în radical, $\mathcal{O}(\sqrt{n})$ și cel mult două împărțiri per operație;
- descompunere în radical, $\mathcal{O}(\sqrt{n})$ și $\mathcal{O}(\sqrt{n})$ împărțiri per operație.

Precizez că vom discuta descompunerea în radical abia în capitolul următor, dar pare un moment bun să privim aceste *benchmarks*.

Am ales limitele $n = q = 500.000$ pentru ambele variante ale problemei. Pentru unele programe contează cîte dintre operații sînt interogări și cîte sînt actualizări. Pentru aceste situații, am măsurat doi timpi, notați astfel:

- 250u/250q: există cîte 250.000 de operații din fiecare tip;
- 100u/400q: există 100.000 de actualizări și 400.000 de interogări.

Toate testele sînt pe `long long` și 90% dintre intervale au lungime peste $n/2$. Toți timpii măsoară

strict partea de procesare (excluzînd citirea și scrierea).

6.1.1 Varianta 1 (*point update, range query*)

structură	**timp**
arbore indexat binar	23 ms
arbore de intervale iterativ (250u/250q)	46 ms
arbore de intervale iterativ (100u/400q)	55 ms
arbore de intervale recursiv (250u/250q)	116 ms
arbore de intervale recursiv (100u/400q)	130 ms
descompunere în radical (250u/250q)	113 ms
descompunere în radical (100u/400q)	177 ms
descompunere în radical cu împărțiri (250u/250q)	763 ms
descompunere în radical cu împărțiri (100u/400q)	1.219 ms

Tabela 6.1: Timpii de rulare pentru sume pe interval și actualizări punctuale.

6.1.2 Varianta 2 (*range update, range query*)

structură	**timp**
arbore indexat binar (250u/250q)	66 ms
arbore indexat binar (100u/400q)	58 ms
arbore de intervale iterativ (250u/250q)	183 ms
arbore de intervale iterativ (100u/400q)	175 ms
arbore de intervale recursiv (250u/250q)	211 ms
arbore de intervale recursiv (100u/400q)	203 ms
descompunere în radical (250u/250q)	235 ms
descompunere în radical (100u/400q)	274 ms

Tabela 6.2: Timpii de rulare pentru sume pe interval și actualizări pe interval.

6.1.3 Concluzii

Reținem că:

- Arborii indexați binar sînt de departe cei mai rapizi.
- Pentru actualizări punctuale, arborii de intervale iterativi sînt de două ori mai rapizi decît cei recursivi.
- Descompunerea în radical ține binișor pasul cu arborii de segmente. Din experiență, aceasta este o particularitate a problemei alese. Pentru alte probleme diferența poate fi mai mare.
- Împărțirile îngreunează enorm descompunerea în radical.

6.2 Actualizări punctuale și interogări pe interval

6.3 Reprezentare

AIB-ul descompune informația în felul următor: Poziția k din vector stochează suma ferestrei de p elemente care se termină la poziția k, unde p este cea mai mare putere a lui 2 care îl divide pe k. De exemplu, pentru $k = 40$, $p = 8$. Așadar, pe poziția 40 AIB-ul va stoca suma celor 8 valori de pe pozițiile $[33 \dots 40]$.

Iată un exemplu care arată sus valorile pe care dorim să le reținem, iar jos valorile concrete pe care ajunge să le stocheze vectorul. Subliniez că AIB-ul nu folosește memorie suplimentară, ci doar stochează diferit informația în același vector. Suspectez că și de aici provine eficiența lui în raport cu arborii de intervale.

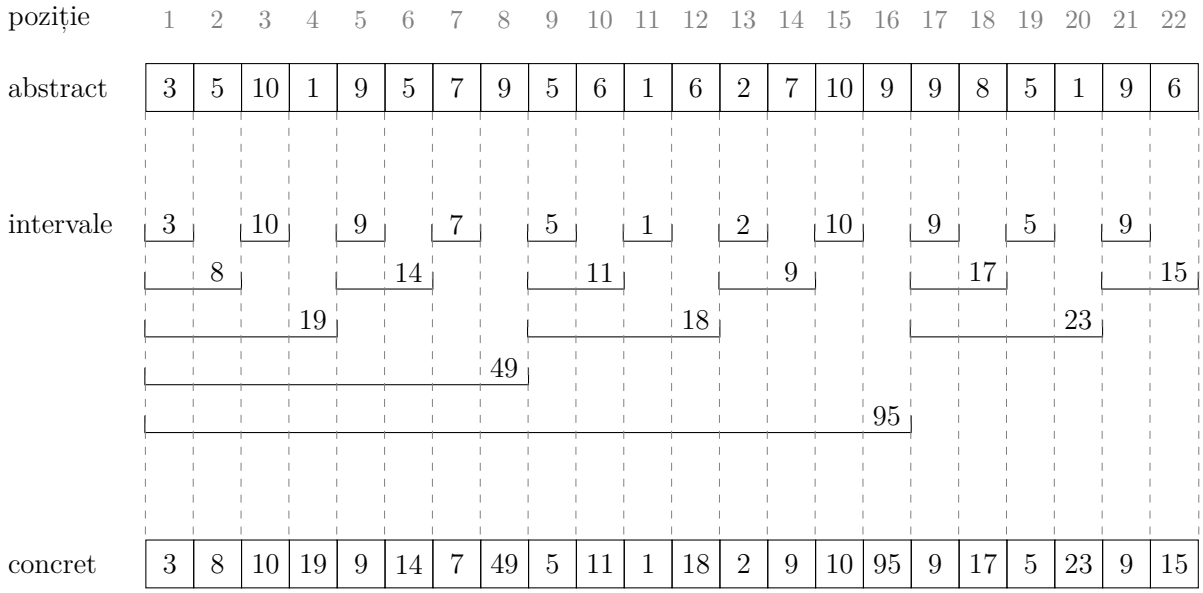


Figura 6.1: Un arbore indexat binar cu 22 de poziții. Vectorul de sus este cel abstract, iar vectorul de jos este cel stocat concret în memorie. Fiecare interval arată pozițiile a căror sumă o notăm în capătul din dreapta al intervalului.

6.4 Operația de interogare (suma unui interval)

AIB-urile tratează interogările pe un interval oarecare $[x, y]$ prin diferența a două interogări pe prefix, $[1, y]$ și $[1, x-1]$. Pentru a răspunde la o interogare pe prefix, de exemplu suma pe intervalul $[1, 21]$, descompunem acel prefix în intervale dintre cele stocate în AIB, respectiv $[1, 16]$, $[17, 20]$ și $[21, 21]$. Odată ce includem o poziție x și tot intervalul pe care îl acoperă ea, pentru a ajunge la următoarea poziție de însumat trebuie, prin definiție, să scădem cea mai mare putere a lui 2 care îl divide pe x. Rezultă codul:

```
struct fenwick_tree {
  int v[MAX_N + 1]; // indexare de la 1
```

```
  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int range_sum(int from, int to) {
    return prefix_sum(to) - prefix_sum(from - 1);
  }
};
```

Expresia `pos &= pos - 1` elimină cel mai din dreapta bit de 1 dintr-un număr; de exemplu, din $20 = 10100_{(2)}$ ea obține $16 = 10000_{(2)}$. Pe cazul general,

```
pos             = abc...xyz1000...000
pos - 1         = abc...xyz0111...111
pos & (pos - 1) = abc...xyz0000...000
```

De aici rezultă și complexitatea $\mathcal{O}(\log n)$, căci reprezentarea oricărei poziții în baza 2 are cel mult $\log n$ biți de 1.

6.5 Operația de actualizare (adăugare pe poziție)

La actualizarea pe o poziție, trebuie actualizate toate intervalele care conțin acea poziție. De exemplu, la actualizarea poziției 11 trebuie actualizate intervalele $[11, 11]$, $[9, 12]$ și $[1, 16]$, așadar trebuie recalculate pozițiile 11, 12 și 16 din AIB. Această parte pare magică: de ce pozițiile 13, 14 și 15 nu trebuie actualizate? Dar ne putem convinge că, cît ne îndepărtăm de poziția inițială (11), ne interesează doar pozițiile responsabile de intervale suficient de mari încît să acopere poziția 11.

```
void add(int pos, int val) {
  do {
    v[pos] += val;
    pos += pos & -pos;
  } while (pos <= n);
}
```

Pentru a înțelege expresia `pos & -pos`, să facem o scurtă digresiune. Numerele cu semn sînt reprezentate în calculator în complement față de 2. Aceasta înseamnă că, pentru a reprezenta un număr negativ,

- Reprezentăm întîi numărul pozitiv (valoarea absolută).

- Îi inversăm toți biții.
- Adăugăm 1.

De exemplu, pentru a îl reprezenta pe -20 procedăm astfel:

- Îl reprezentăm pe +20: `000...00010100`.
- Îi inversăm toți biții: `111...11101011`.
- Adăugăm 1: `111...11101100`.

Această reprezentare are două avantaje:

1. Putem folosi aceleași circuite logice pentru operații pe numere cu sau fără semn.
2. Reprezentările lui +0 și −0 sînt identice, `000...000`. În complement față de 1 există două reprezentări, `000...000` și `111...111`, ceea ce este straniu.

Revenind, observăm acum că `20 & -20` este `000...00000100`, adică formula `pos & -pos` izolează ultimul bit, adică mărimea intervalului subîntins de `pos`. Prin adăugarea acestei cantități la `pos`, obținem intervalul imediat următor care include poziția `pos`.

Dacă din orice motiv această expresie vă scapă din memorie, puteți inventa pe loc formule echivalente, de exemplu:

```
pos = (pos | (pos - 1)) + 1;
```

Complexitatea este tot $\mathcal{O}(\log n)$ deoarece la fiecare pas eliminăm cel puțin un bit 1 din reprezentarea binară a lui pos.

6.6 Construcția în $\mathcal{O}(n)$

Dat fiind un vector-sursă `src`, este tentant să construim arborele într-un al doilea vector apelînd de n ori rutina de adăugare:

```
void build(int* src) {
  for (int i = 1; i <= n; i++) {
    add(i, src[i]);
  }
}
```

Această metodă cere timp $\mathcal{O}(n \log n)$. Există însă o metodă care refolosește vectorul și rulează în $\mathcal{O}(n)$:

```
void build() {
  for (int i = 1; i <= n; i++) {
    int j = i + (i & -i);
    if (j <= n) {
      v[j] += v[i];
    }
```

```
  }
}
```

Explicație: fiecare element `v[i]` este propagat doar la următorul element `j` care include poziția `i`. Este treaba acelui segment să propage adaosul și mai departe. Pentru scenariul relativ comun în care construim AIB-ul, apoi facem n interogări și n actualizări, construcția în $\mathcal{O}(n)$ reduce costul de rulare cu circa 20%.

Paradoxal, tocmai fiindcă este atît de rapid, costul de funcționare al unui AIB este adesea înecat de costul altor operații (în special intrarea/ieșirea). De aceea optimizările sînt greu de observat. Dar *the hacker spirit* ne obligă să folosim oricum soluția inteligentă.

6.7 Găsirea unei valori punctuale

Pentru a găsi valoarea pe o singură poziție `k`, o putem calcula în $\mathcal{O}(\log n)$ ca pe `sum(k) - sum(k - 1)`. Sau, desigur, putem păstra o copie a vectorului real. Dar există și o implementare în $\mathcal{O}(1)$ amortizat.

Să considerăm poziția $k = 60$. Dacă o calculăm prin diferența sumelor parțiale, obținem

```
val(60) = sum(60) - sum(59) = (v[60]          + v[56] + v[48] + v[32]) -
                              (v[59] + v[58] + v[56] + v[48] + v[32])
                            = v[60] - (v[59] + v[58])
```

Se vede că, de la poziția 56 încolo, sumele de intervale se anulează în cele două paranteze. Nu este o coincidență. Fie:

```
k     = bbb...bbb10000
k - 1 = bbb...bbb01111
```

Unde `b` sînt niște biți oarecare, iar poziția k se termină într-un bit 1 urmat de cîțiva (posibil 0) biți de 0. Atunci, în calculul sumelor parțiale, pozițiile k și $k-1$ vor elimina biți de la coadă pînă cînd vor ajunge la strămoșul comun, care este

```
str   = bbb...bbb00000
```

De aceea, codul este:

```
int get_value_at(int pos) {
  int result = v[pos];
  int ancestor = pos & (pos - 1);
  pos--;
  while (pos != ancestor) {
    result -= v[pos];
    pos &= pos - 1;
  }
  return result;
```

```
}
```

Acest cod pare tot logaritmic. În realitate, jumătate din valorile din AIB (cele de pe poziții impare) stochează chiar valoarea în acel punct, deci bucla din `get_value_at()` va face 0 iterații. Un sfert din valorile din AIB vor face o iterație, o optime dintre ele vor face două iterații. În general, pentru o poziție k, funcția `get_value_at()` va face atîtea iterații cîte zerouri are la coadă reprezentarea binară a lui k. Media acestei valori este 1 (așadar constantă) dacă distribuția lui k este uniformă și aleatorie.

6.8 Căutarea binară a unei sume parțiale

Dacă vectorul (abstract) are doar valori non-negative, atunci sumele parțiale sînt nedescrescătoare și are sens întrebarea: Care este prima poziție pe care se atinge suma parțială S?

Putem face o căutare binară naivă: examinăm suma parțială la poziția $n/2$, apoi la una dintre pozițiile $n/4$ sau $3n/4$ după caz, etc. Dar fiecare dintre aceste interogări durează $\mathcal{O}(\log n)$, deci complexitatea totală a algoritmului va fi $\mathcal{O}(\log^2 n)$. Dar iată și o metodă în $\mathcal{O}(\log n)$, similară cu căutarea binară prin „metoda Mihai Pătrașcu” (ca să adoptăm nomenclatura din supa culturală olimpică).

Fie p cea mai mare putere a lui 2 cel mult egală cu n. Pentru exemplul inițial, $n = 22$, deci $p = 16$. Observația-cheie este că putem afla suma parțială pe pozițiile $[1 \ldots p]$ printr-o singură operație: ea este exact `v[p]`! După cum `v[p] < S` sau `v[p] > S`, ne îndreptăm atenția către pozițiile din stînga sau din dreapta lui p. Următoarea interogare o vom face la `v[p / 2]` sau la `v[3 * p / 2]`.

În practică, invariantul este: `pos` reprezintă cea mai mare poziție cunoscută pe care suma parțială **nu** atinge valoarea S. La final, funcția returnează `pos + 1`. De asemenea, ținem cont că nu întotdeauna putem avansa la dreapta (nu putem depăși valoarea n).

```
int bin_search(int sum) {
  int pos = 0;

  for (int interval = max_p2; interval; interval >>= 1) {
    if ((pos + interval <= n) && (v[pos + interval] < sum)) {
      sum -= v[pos + interval];
      pos += interval;
    }
  }

  return pos + 1;
}
```

Exemplu: pentru AIB-ul din figura 6.1 și suma parțială 72, algoritmul funcționează astfel:

- Verifică poziția 16. Suma este 95, prea mare.

- Verifică poziția 8. Suma este 49. Așadar, căutăm suma parțială 72 - 49 = 23 începînd dincolo de poziția 8.
- Verifică poziția 12. Suma este 18. Așadar, căutăm suma parțială 23 - 18 = 5 începînd dincolo de poziția 12.
- Verifică poziția 14. Suma este 9, prea mare.
- Verifică poziția 13. Suma este 2. Așadar, căutăm suma parțială 5 - 2 = 3 începînd dincolo de poziția 13.
- Răspunsul este poziția 14.

Aici, `max_p2` este cea mai mare putere a lui 2 care nu depășește n. O putem afla naiv, de exemplu astfel:

```
max_p2 = n;
while (max_p2 & (max_p2 - 1)) {
  max_p2 &= max_p2 - 1;
}
```

, sau într-o singură linie cu funcția `__builtin_clz(n)`, care returnează numărul de zerouri la stînga lui n:

```
max_p2 = 1 << (31 - __builtin_clz(n));
```

Corolar: putem folosi căutarea binară pentru a afla prima poziție cu o valoare nenulă într-un AIB. Aceasta este poziția pe care suma parțială atinge valoarea 1. Similar putem afla ultima poziție cu o valoare nenulă. Mai trebuie doar să menținem și suma elementelor din AIB, ceea ce cere două linii de cod.

Corolar: dacă AIB-ul ține valori de 0 și 1, putem folosi căutarea binară pentru a afla poziția celui de-al k-lea bit 1 (în engleză această valoare se numește *k-th order statistic*). Ea este fix poziția pe care suma parțială atinge valoarea k. Multe probleme de permutări, care necesită evidența elementelor văzute / nevăzute, se încadrează aici (exemplu: codificarea / decodificarea permutărilor).

6.9 Alte operații decît adunarea

Arborii Fenwick pot gestiona și alte operații, cîtă vreme ele sînt **inversabile**. Reamintesc că **suma** pe intervalul $[l \ldots r]$ se calculează ca **diferența** sumelor pe intervalele $[1 \ldots r]$ și $[1 \ldots l-1]$. Deci operația inversă (scăderea) trebuie să fie definită. Exemplu: operația xor, operația de înmulțire modulo un număr prim etc.

Deoarece operația *max* nu este inversabilă, AIB-urile nu suportă, pe cazul general, operațiile:

1. actualizare punctuală;
2. maxim pe interval.

Totuși, putem folosi AIB-uri pentru interogări de maxim dacă următoarele condiții sînt adevărate:

1. Toate interogările sînt pe prefix (capătul stînga este întotdeauna 1).
   - Sau toate interogările sînt pe sufix, caz în care putem reflecta toți indicii față de n.
2. Prin actualizare, valorile pot doar să crească.

Temă de gîndire: De ce este necesar ca toate valorile să crească? Ce poate să meargă prost dacă într-un AIB de maxime valorile pot să și scadă?

Raționamente similare putem aplica pentru operațiile *min*, *and* și *or*. Cum reformulăm condiția 2 pentru aceste operații?

Un exemplu mai extravagant este operația *or* pe `bitset`-uri, vezi problema Erinaceida.

6.10 Probleme

6.10.1 Problema The Permutation Game Again (SPOJ)

enunț • sursă

Problema ne cere să aflăm **rangul** unei permutări (engl. *rank*). Acesta este numărul de ordine al permutării în lista ordonată lexicografic a tuturor permutărilor mulțimii $\{1, 2, \ldots, n\}$.

Echivalent, trebuie să răspundem eficient la întrebarea: cîte permutări vin înaintea celei date în lista permutărilor?

Să considerăm un exemplu. Dacă primul element al permutării este 9, atunci toate permutările care încep cu $1, 2, \ldots, 8$ o vor preceda în listă. Există $8(n-1)!$ astfel de permutări.

Similar, dacă al doilea element este 3, atunci toate permutările care încep cu 91 sau 92 o vor preceda în listă. Există $2(n-2)!$ astfel de permutări.

Dar dacă al treilea element este 7? Acum trebuie să ținem cont de faptul că pe 3 l-am văzut deja. Trebuie să socotim permutările care încep cu $931, 932, 934, 935, 936$. Există $5(n-3)!$ astfel de permutări.

Cu alte cuvinte, trebuie să răspundem eficient la întrebarea: cîte elemente mai mici decît cel curent am văzut în prefixul dinaintea elementului curent? Putem gestiona această informație cu un AIB de 0 și 1. Cînd procesăm un element de valoare x, adunăm 1 pe poziția x în AIB. Astfel, suma parțială din AIB pe o poziție y ne va arăta cîte elemente mai mici decît y am procesat pînă în prezent.

Pentru un plus de eficiență, sursa nu reține întreaga permutare, ci doar citește cîte un element, îl ia în calcul la rang, îl bifează în AIB, apoi îl aruncă.

6.10.2 Problema Multiset (Codeforces)

enunț • sursă

„Aproape” putem rezolva problema cu un singur vector de frecvențe. Dar avem nevoie să găsim eficient al k-lea element ca să-l putem șterge. Un vector de frecvențe ne dă inserări în $\mathcal{O}(1)$, dar ștergeri în $\mathcal{O}(n)$.

De aceea, înlocuim vectorul cu un AIB în care pe poziția x notăm frecvența lui x în multiset. Astfel putem căuta al k-lea element reformulînd definiția: al k-lea element este poziția p pe care suma parțială atinge sau depășește valoarea k.

6.10.3 Problema Hanoi Factory (Codeforces)

enunț • sursă

Pare natural să sortăm inelele descrescător după diametrul exterior. Ce facem la egalitate? Toate inelele de același diametru exterior pot fi stivuite, caz în care îl vom prefera deasupra pe cel cu diametrul interior minim, ca să ne maximizăm șansele de a putea pune alt inel deasupra lui. Așadar, ca departajare, sortăm inelele descrescător după diametrul interior.

Acum orice turn valid va fi un subșir din șirul sortat, pe sărite, dar fără reordonare. Și atunci putem defini relativ ușor o recurență calculabilă în $\mathcal{O}(n^2)$. Fie H_i înălțimea maximă a unui turn care are în vîrf inelul i. Atunci inelul aflat imediat sub i, fie el j, respectă condițiile $j < i$ și $in_j < out_i$. Așadar,

$$H_i = h_i + \max_{j<i,\ in_j<out_i} H_j$$

Pentru a reduce complexitatea la $\mathcal{O}(n \log n)$, procesăm inelele de la stînga la dreapta. Atunci $j < i$ este întotdeauna respectată și trebuie doar să răspundem la întrebarea: dintre toate inelele cu $in_j <$ un x dat (unde $x = out_i$), care este valoarea maximă pentru H_j? Putem răspunde la întrebare cu un AIB de maxime, indexat după diametrele interioare, pe care îl interogăm despre maximul pe pozițiile $[1 \dots out_i - 1]$. După calcularea lui H_i, optimizăm maximul din AIB pe poziția in_i cu valoarea H_i.

Diametrele pot fi mari, dar le putem normaliza în intervalul $[1 \dots 2n]$.

Există și o soluție mai ingenioasă, cu o stivă ordonată, care nu face obiectul acestui capitol.

6.10.4 Problema Subsequences (Codeforces)

enunț • sursă

Problema pare abordabilă cu programare dinamică. Să căutăm întîi formula de recurență, care nu este dificilă. Fie $C_{l,i}$ numărul de subsecvențe crescătoare de lungime l terminate pe poziția i. Atunci:

$$C_{l,i} = \sum_{j<i,\ a_j<a_i} C_{l-1,j}$$

Implementarea în $\mathcal{O}(kn^2)$ este așadar directă. Cum procedăm să reducem calculul sumei de la $\mathcal{O}(n)$ la $\mathcal{O}(\log n)$? Observăm aici un mecanism pe care îl vom regăsi și la alte probleme. Pare că dorim o interogare bidimensională (suma valorilor $C_{l-1,j}$ pe poziții unde $j < i$ și simultan $a_j < a_i$). În realitate, însă, putem reduce interogarea la una unidimensională.

Să inserăm într-un AIB valorile $C_{l-1,1} \dots C_{l-1,i-1}$. Atunci condiția $j < i$ este automat satisfăcută și dorim suma valorilor pe pozițiile unde $a_j < a_i$. De aceea, vom indexa AIB-ul nu după j, ci după a_j. Cu alte cuvinte, vom scrie $C_{l-1,j}$ nu la poziția j, ci la poziția a_j. Desigur, după ce calculăm $C_{l,i}$ adăugăm și $C_{l-1,i}$ la AIB.

Ca fapt divers, puteam face reducerea la interogări unidimensionale pe dos: iterăm prin elemente în ordinea crescătoare a valorilor, astfel încît condiția $a_j < a_i$ să fie automat satisfăcută. Atunci AIB-ul ar fi fost indexat după pozițiile propriu-zise, deci $C_{l-1,j}$ ar fi stat chiar la poziția a_j.

Noua complexitate este $\mathcal{O}(kn \log n)$: pentru fiecare dintre cele $k \times n$ valori ale lui C facem o interogare și o actualizare în AIB. Ca optimizare de memorie, ne este suficientă o singură linie din matricea C.

6.10.5 Problema D-query (SPOJ)

enunț • sursă

La prima vedere AIB-urile ne sînt inutile aici, pentru că funcția „numărul de elemente distincte" nu poate fi calculată prin diferențe de intervale: dacă în intervalul $[1, 10]$ avem 5 elemente distincte, iar în $[1, 20]$ avem 8 elemente distincte, nu știm destule despre numărul de elemente distincte din $[11, 20]$.

Dar, ca la multe alte probleme, varianta offline (în care primim de la început toate interogările) este considerabil mai simplă decît varianta online (în care trebuie să răspundem la o interogare înainte de a o primi pe următoarea).

Pare natural să sortăm interogările și să le scanăm cumva, dar cum? Să fixăm o poziție r și să considerăm toate intervalele care se termină la r. Dacă un interval $[l, r]$ conține o valoare x, atunci o poate conține o dată sau de multiple ori, dar în mod sigur va include **cea mai din dreapta** apariție a lui x înainte de r. Și atunci, pentru o poziție fixată r, dorim să bifăm toate pozițiile $i \in [1, r]$ pentru care nu există o altă poziție $j \in [i + 1, r]$ cu $v[i] = v[j]$.

De exemplu, pentru $r = 8$ și prefixul (1 1 7 6 1 2 6 2), dorim să stocăm bifele (0 0 1 0 1 0 1 1), pentru a indica cele mai din dreapta apariții ale lui 1, 2, 6 și 7. Atunci răspunsul la interogarea $[5, 8]$ va fi tocmai numărul de bife (adică suma) din intervalul $[5, 8]$, pentru că astfel ne asigurăm că numărăm exact o apariție, ultima, a fiecărei valori din interval.

AIB-ul de bife este ușor de actualizat. Dacă, de exemplu, următorul element din vector este 7, trebuie să ștergem bifa de la ultima apariție a lui 7 (poziția 3) și să o aplicăm pe poziția 9. Avem nevoie de un vector cu poziția ultimei apariții a fiecărei valori (dacă există), ceea ce este fezabil deoarece valorile nu depășesc 1.000.000.

La final, reordonăm interogările conform ordinii inițiale și afișăm răspunsurile.

6.10.6 Problema Magic Board (CodeChef)

enunț • sursă

Pare că avem de-a face cu o matrice binară uriașă, dar secretul este să reținem separat informații despre linii și despre coloane. Observația-cheie este că operațiile `Set` modifică doar linii și coloane întregi.

Putem reformula o interogare de tipul `RowQuery i` astfel. Dacă ultima resetare a liniei i a fost la momentul de timp t (operația cu numărul t) și la valoarea 0, atunci cîte coloane au fost modificate din 0 în 1 după momentul t? Dacă au fost k coloane modificate, răspunsul la interogare este $n-k$.

Similar, dacă ultima resetare a liniei i a fost la momentul de timp t (operația cu numărul t) și la valoarea 1, și dacă ulterior k coloane au fost modificate din 1 în 0, atunci răspunsul la interogare este chiar k.

Astfel, trebuie să răspundem la întrebări de tipul: cîte modificări există la valoarea v la timpi $> t$? De aceea, vom ține două AIB-uri pe coloane, indexate după timp (adică după numărul operației), în care stocăm timpul ultimei resetări a fiecărei coloane în 0 și respectiv în 1.

De asemenea, cînd primim o interogare, trebuie să știm timpul ultimei modificări a acelei linii sau coloane, ceea ce putem stoca naiv: un vector de perechi (timp, valoare).

Informațiile pe linii și pe coloane sînt perfect simetrice. Vom studia codul ca să vedem cum putem elimina duplicarea codului. Asta doar dacă evitarea duplicării codului este importantă pentru noi. 😉

6.10.7 Problema Little Artem and Time Machine (Codeforces)

enunț • sursă

Este o problemă simplă, cu multe rezolvări. Dar **citiți atent enunțul**, care este neclar. Pentru intrarea

```
4
1 1 7
3 10 7
2 2 7
3 11 7
```

răspunsul trebuie să fie

```
1
0
```

Așadar, valoarea 7 a fost adăugată la timpul 1 și ștearsă la timpul 2. Dar la momentele 10 și 11 răspunsul este 1, respectiv 0, pentru că prima interogare vine înaintea ștergerii **în ordinea din**

fișier.

Problema se reduce destul de rapid la adăugări punctuale și sume pe prefix. Răspunsul la interogarea de pe poziția i despre timpul t și valoarea v este suma inserărilor (+1) și a ștergerilor (-1) operate pe valoarea v la timpi $t' < t$ și indici de interogare $i' < i$.

Observăm mai întîi că răspunsurile pentru fiecare valoare pot fi grupate și procesate independent. Am ales să fac asta sortînd interogările după valoare și procesînd grupuri de interogări referitoare la aceeași valoare. Pe Codeforces am tot văzut surse care se reped la `map`, dar... de ce am face asta?

Pentru o valoare fixată, regăsim problema clasică 2D: trebuie să numărăm valorile din dreptunghiul de timpi $[1, t-1]$ și indici $[1, i-1]$. Așadar, vom construi un AIB indexat după timp în care adăugăm și ștergem valoarea la diverse momente de timp. Procesăm operațiile în ordinea din fișier și răspunsul la o operație i va fi suma pe prefixul de lungime i. Este nevoie să normalizăm timpii în prealabil.

Pentru a pregăti AIB-ul pentru o nouă valoare, trebuie să-l golim după procesarea operațiilor pentru a valoare. Putem face aceasta repetînd operațiile cu semn schimbat (-1 pentru adăugări, +1 pentru ștergeri).

Pentru a scurta codul, putem întoarce pe dos logica structurii: sortăm operațiile după timpii t și indexăm AIB-ul după indicii operațiilor, i. Deoarece acești indici sînt deja între 1 și n, scăpăm de normalizare! Paradoxal, această implementare este mai lentă, deși face o sortare mai puțin. 🥲

6.10.8 Problema Ball (Codeforces)

enunț • sursă

Să abstractizăm problema: date fiind n puncte în spațiu, cîte dintre ele sînt dominate de un alt punct? Spunem că un punct Q domină un punct P dacă $x_Q > x_P$, $y_Q > y_P$ și $z_Q > z_P$.

Ideea generală a rezolvării este să construim o structură de date în care să includem punctele unul cîte unul. Înainte de a include fiecare punct P, ne întrebăm: printre punctele deja existente în structură, este vreunul care îl domină pe P? Remarcăm că ne promitem că orice punct va fi adăugat la această structură înaintea tuturor punctelor pe care le domină.

Ca și la alte probleme, vom scăpa de constrîngerile pe una dintre dimensiuni – să zicem z – sortînd punctele descrescător după z. Apoi vom ignora coordonata z pentru tot restul problemei. Intenția este ca, orice formă va lua structura de date pe care o proiectăm, să avem garanția cînd evaluăm un punct P că punctele existente în structură îl domină pe P pe axa Oz. Această condiție este **aproape** suficientă pentru a ne respecta promisiunea sus-menționată.

Dar există un detaliu, înainte de a uita complet de coordonatele z. Dacă mai multe puncte au coordonata z egală, atunci niciunul nu îl domină pe celălalt conform definiției. Totuși, dacă îl inserăm pe $Q = (5, 5, 10)$ în structură ca $(5, 5)$, apoi evaluăm punctul $P = (3, 3, 10)$, riscăm să

credem că Q îl domină pe P. Rezultă că trebuie să mai facem o promisiune: dacă un punct Q din structură domină punctul curent P pe x și pe y, el îl domină și pe z.

O primă soluție procesează punctele în grupe cu același z. Pentru fiecare grupă, întîi evaluăm punctele (decidem dacă sînt dominate sau nu) și abia apoi le adăugăm la structură. Această ordine este corectă întrucît nicio pereche dintre aceste puncte nu se domină între ele. Iată o sursă cu această abordare.

A doua soluție, din sursa anexată, observă că este suficient să sortăm punctele descrescător după z, apoi crescător după x. Nu mai este necesară gruparea punctelor. Această sortare garantează respectarea celor două promisiuni:

1. Dacă Q îl domină pe P, atunci în particular $z_Q > z_P$, deci Q este deja în structură.
2. Dacă un punct Q din structură îl domină pe P pe x și pe y, atunci nu poate avea același z, căci l-am fi procesat pe P înaintea lui Q. Deci cu siguranță $z_Q > z_P$.

Mulțumiri lui Victor D. pentru această soluție elegantă.

Acum putem ataca problema bidimensională. Pentru punctul P ne întrebăm: există vreun punct văzut anterior care să aibă și x-ul și y-ul mai mare? Așadar, trebuie să aflăm dacă există vreun punct în cadranul de la (x_P, y_P) la (∞, ∞). Iar coordonatele sînt prea mari pentru un AIB 2D.

Aici intervine ultimul artificiu. Dorim să reducem problema la o interogare unidimensională pe sufix: dintre toate punctele văzute anterior, considerîndu-le doar pe cele cu $x > x_P$, există vreunul cu $y > y_P$? Putem reformula această întrebare ca pe una de maxim: Dă-mi maximul lui y pe domeniul $[x_P, \infty)$. Dacă acest maxim este $> y_P$, atunci există un punct care îl domină pe p, altfel nu.

După cum am mai spus, dacă vrem să construim un AIB care să răspundă la interogări pentru $x > x_P$, rezultă că vom indexa AIB-ul după coordonatele x. Două detalii de implementare.

1. Trebuie să normalizăm coordonatele x la intervalul $[1, n]$. Nu ne interesează valorile exacte, ci doar relațiile între ele.
2. AIB-ul de maxime știe să calculeze doar maxime pe prefix, nu pe sufix. Dar putem să reflectăm toate valorile x normalizate față de n pentru a transforma interogările pe sufix în interogări pe prefix.

Observație tangențială: Întotdeauna estimați mărimea fișierului de intrare! În acest caz, avem 1,5 milioane de numere pe 9 cifre plus spații, deci circa 15 MB. Timpul de rulare este efectiv dominat de citire. Sursa inclusă a rulat în 1.100 ms. O a doua sursă, cu citire rapidă, a rulat în 180 ms.

6.11 Interogări punctuale și actualizări pe interval

Putem privi actualizările pe interval ca pe un fel de „Șmen al lui Mars online”. În șmenul lui Mars prelucrăm operațiile „adaugă x pe pozițiile $[l \ldots r]$” adăugînd x pe poziția l și $-x$ pe poziția $r+1$. Deosebirea este că în Șmenul lui Mars interogările vin doar la sfîrșit, după toate actualizările, pe

cînd în varianta online interogările pot veni și pe parcurs.

AIB-ul poate fi adaptat foarte ușor la această nevoie:

1. Actualizare: adaugă x pe poziția l și $-x$ pe poziția $r+1$.
2. Interogare pe poziția k: calculează suma prefixului $[1 \dots k]$.

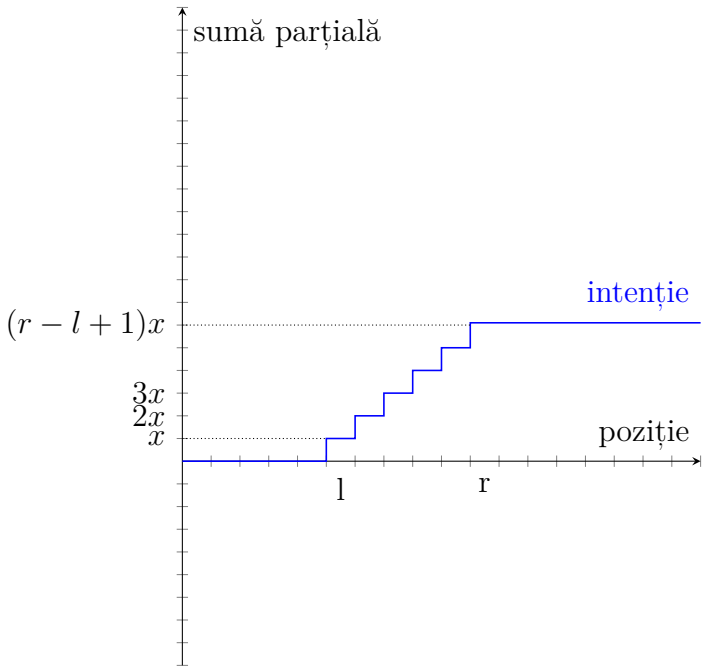
Aceasta funcționează deoarece, la interogarea pe poziția k,

1. Dacă $k < l$, atunci suma prefixului $[1 \dots k]$ nu va include pozițiile l și $r+1$.
2. Dacă $k > r$, atunci suma prefixului $[1 \dots k]$ va include pozițiile l și $r+1$, care se vor anula reciproc. Dorim aceasta, deoarece $k \notin [l, r]$, deci variația intervalului $[l, r]$ nu trebuie să afecteze poziția k.
3. Dacă $l \leq k \leq r$, atunci suma prefixului $[1 \dots k]$ va include doar poziția l, nu și poziția $r+1$, deci poziția k va fi afectată de variația pe intervalul $[l, r]$.

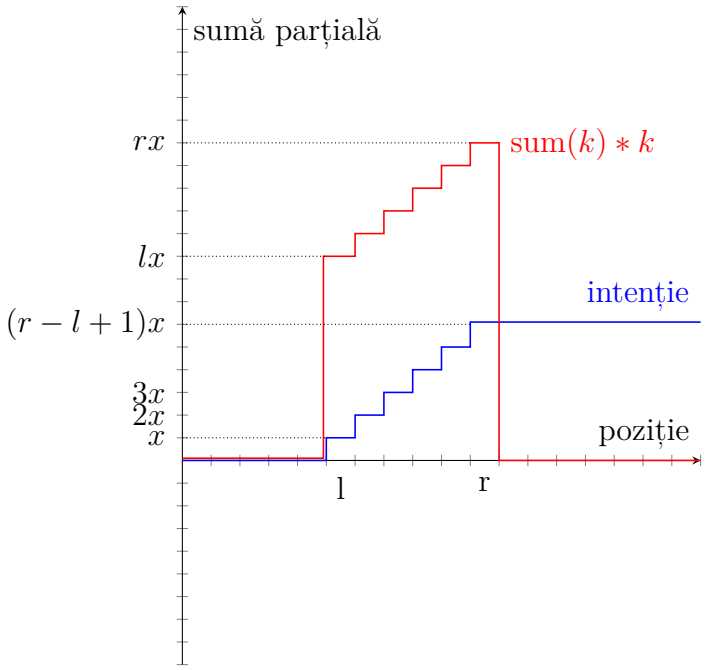
6.12 Interogări și actualizări pe interval

Să rezolvăm și această versiune, doar de amorul artei. Nu cred că discuția de mai jos se aplică la altceva decît la sume (la xor-uri, de pildă).

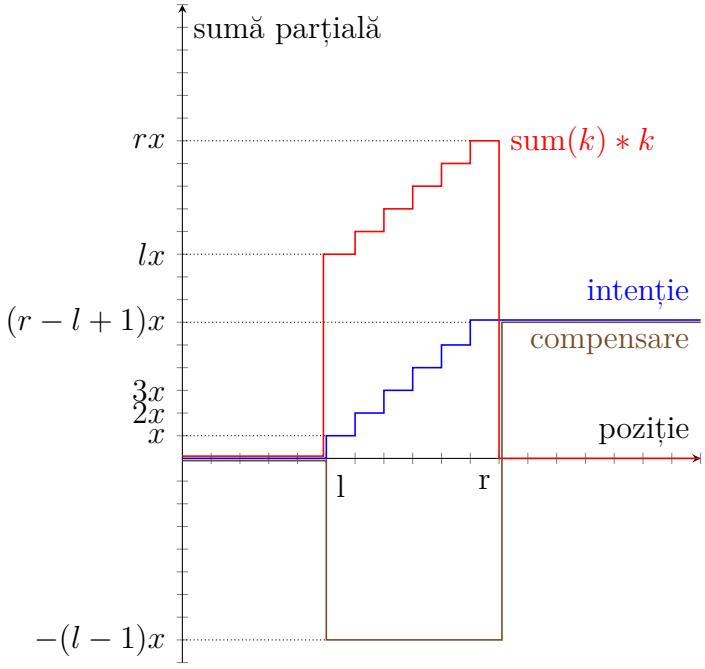
Avînd în vedere că AIB-urile se preocupă în special de sume parțiale, să examinăm grafic sumele parțiale după operația de adăugare a lui x pe intervalul $[l, r]$. Pe pozițiile $[l \dots r]$ ia naștere o funcție scară: dorim ca sumele parțiale să crească cu x, $2x$, ..., $(r-l+1)x$.



Intuitiv, deoarece suma parțială crește cu poziția, sîntem tentați să returnăm, la poziția k, valoarea $\text{sum}(k) \times k$. Pentru ca după r suma parțială să se oprească din crescut, adăugăm x la poziția l și scădem x la poziția $r+1$, similar cu șmenul lui Mars. Doar că atunci funcția $\text{sum}(k) \times k$ are graficul:



Într-adevăr, observăm că suma parțială la stînga lui l este 0, iar la dreapta lui r este $x - x = 0$. Totuși, pare că cele două grafice nu au nicio legătură! Dar nu este chiar așa. Observăm că diferența între cele două funcții arată relativ simplu:



Pentru a corecta al doilea grafic ca să arate ca primul, trebuie să menținem un al doilea AIB în care:

- să scădem pe pozițiile $l, \ldots, r$ valoarea $(l-1)x$;
- să adăugăm pe pozițiile $r+1, \ldots, n$ valoarea $(r-l+1)x$.

În termeni de sume parțiale, trebuie:

- să scădem pe poziția l valoarea $(l-1)x$;
- să adăugăm pe poziția $r+1$ valoarea rx.

Astfel ia naștere codul care, dacă nu-l înțelegem, poate părea foarte ezoteric:

```
struct fenwick_tree_2 {
  fenwick_tree v, w;

  void from_array(long long* src, int n) {
    v.n = n;
    w.from_array(src, n);
  }

  void range_add(int l, int r, long long val) {
    v.add(l, val);
    v.add(r + 1, -val);
    w.add(l, -val * (l - 1));
    w.add(r + 1, val * r);
  }

  long long prefix_sum(int pos) {
    return v.prefix_sum(pos) * pos + w.prefix_sum(pos);
  }

  long long range_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
  }
};
```

6.13 Arbori indexați binar 2D

Putem extinde arborii indexați binar la matrice, cu costuri $\mathcal{O}(\log^2 n)$ pentru operații. Să considerăm următoarele operații (*point update, rectangle query*):

- `1 row col val`: Adaugă val la coordonatele (row, col)
- `2 row1 col1 row2 col2`: Calculează suma dreptunghiului (row_1, col_1) - (row_2, col_2) inclusiv.

6.13.1 Structura informației

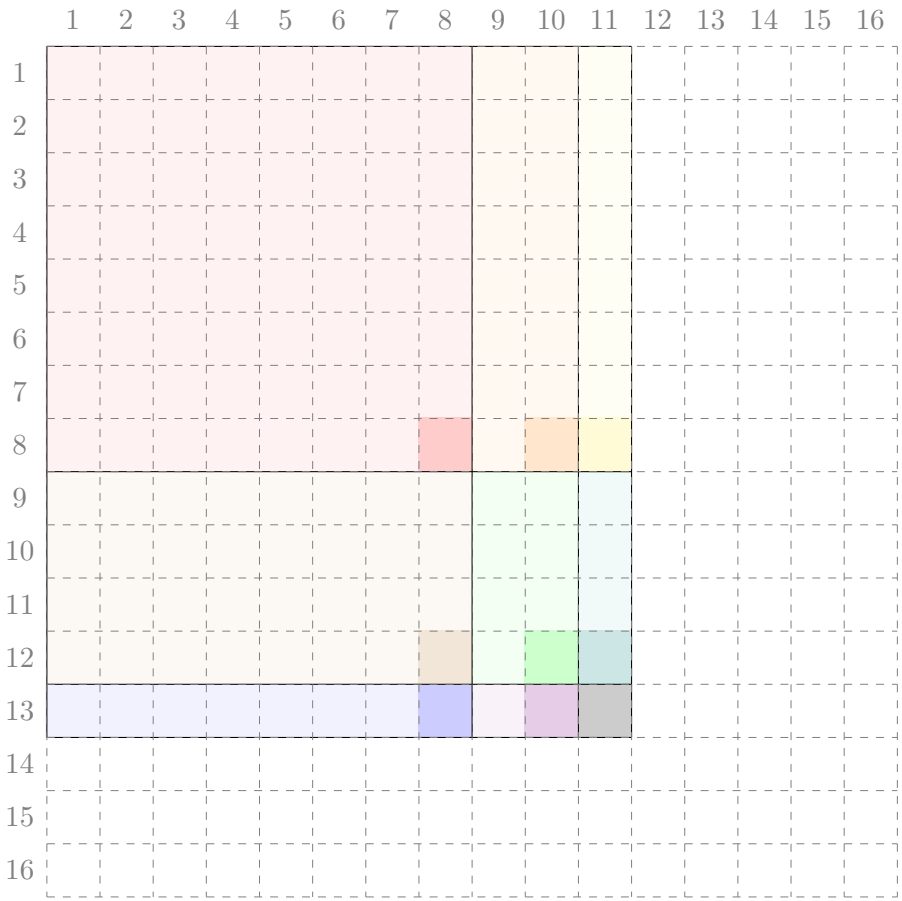
Așa cum arborele 1D modifică structura informației din vector, arborele 2D modifică structura informației din matrice. El stochează anumite sume parțiale. Mai exact, arborele stochează la poziția (r, c) suma elementelor din submatricea (originală) de dimensiuni $p \times q$ cu colțul dreapta-jos la coordonatele (r, c), unde

- p este cea mai mare putere a lui 2 care îl divide pe r
- q este cea mai mare putere a lui 2 care îl divide pe c.

De exemplu, la poziția 40, 60 arborele stochează suma elementelor din matricea de dimensiuni 8×4 cuprinsă între liniile 33 și 40 și coloanele 57 și 60.

6.13.2 Calculul sumei dintr-un dreptunghi

Putem folosi această structură pentru a calcula suma dreptunghiurilor de forma $(1, 1) - (r, c)$. Iată o figură pentru $r = 13$ și $c = 11$.



Dreptunghiul de dimensiune 13×11 se compune din 3×3 dreptunghiuri cu laturile puteri ale lui 2. Trebuie să însumăm valorile din colțurile jos-dreapta ale acestor dreptunghiuri (desenate mai întunecat). Codul este:

```
int prefix_sum(int row, int col) {
  int s = 0;

  for (int r = row; r; r &= r - 1) {
    for (int c = col; c; c &= c - 1) {
      s += mat[r][c];
    }
  }

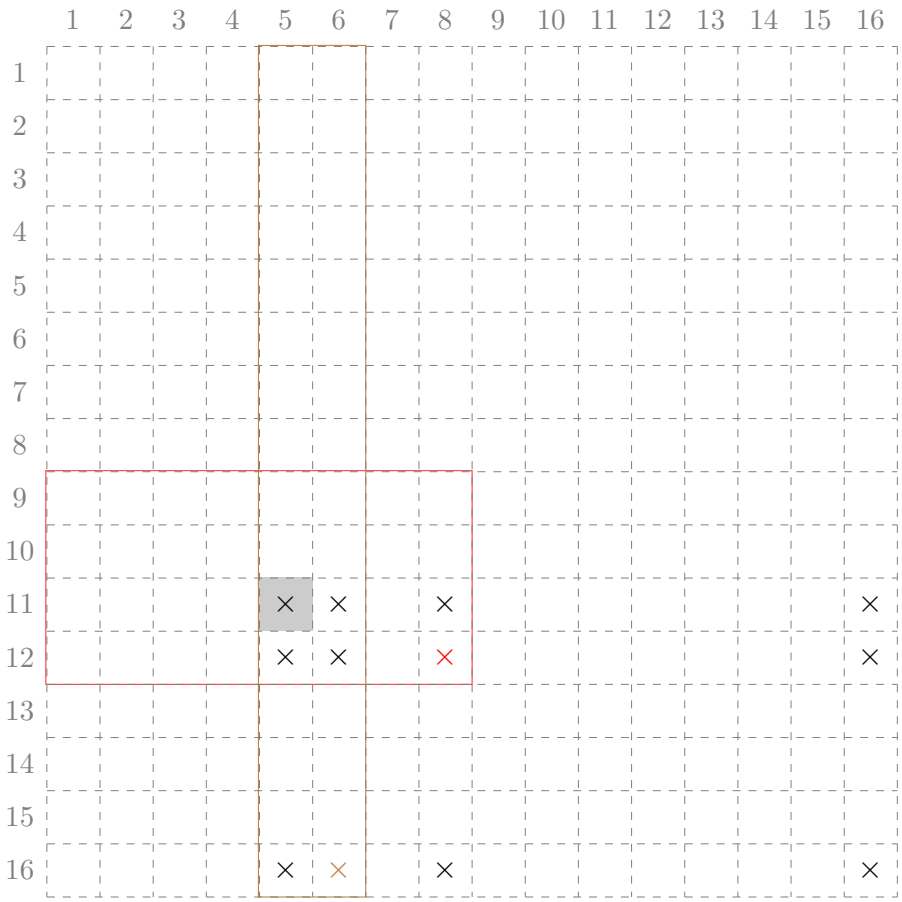
  return s;
}
```

Desigur, putem calcula prin includeri și excluderi suma unui dreptunghi arbitrar:

```
int rectangle_sum(int row1, int col1, int row2, int col2) {
  return
    + prefix_sum(row2, col2)
    - prefix_sum(row2, col1 - 1)
    - prefix_sum(row1 - 1, col2)
    + prefix_sum(row1 - 1, col1 - 1);
}
```

6.13.3 Actualizări punctuale

Ca și la varianta 1D, cînd modificăm valoarea de la coordonatele (r, c) trebuie să modificăm corespunzător toate dreptunghiurile care includ acele coordonate. Iată un exemplu pentru linia 11, coloana 5:



Se observă că trebuie actualizate liniile 11, 12 și 16, adică exact cele care ar include poziția 11 într-un AIB unidimensional. Similar, trebuie actualizate coloanele 5, 6, 8 și 16, adică exact cele care ar include poziția 5. Am evidențiat cu roșu și cu auriu două dintre aceste coordonate, $(12, 8)$ și $(16, 6)$, și dreptunghiurile aferente lor, pentru a evidenția că ele includ celula (11, 5).

Codul pentru actualizare este:

```
void add(int row, int col, int val) {
  for (int r = row; r <= n; r += r & -r) {
    for (int c = col; c <= n; c += c & -c) {
      mat[r][c] += val;
    }
  }
}
```

6.13.4 Construcția în $\mathcal{O}(n^2)$

Și acest arbore poate fi construit in-place, refolosind matricea dată la intrare și în timp $\mathcal{O}(1)$ per element.

În varianta unidimensională, trebuia să propagăm valoarea de la poziția x la poziția $x + (x \,\&\, -x)$. În varianta bidimensională am vrea să procedăm astfel:

- Fie $r' = r + (r \,\&\, -r)$ și $c' = c + (c \,\&\, -c)$.
- Propagăm valoarea de la poziția (r, c) la poziția (r, c'). De acolo, ea se va propaga automat la coloanele următoare.
- Propagăm valoarea de la poziția (r, c) la poziția (r', c). De acolo, ea se va propaga automat la liniile următoare, iar pe fiecare linie pe coloanele următoare.

Dar apare o problemă: valoarea de la (r, c) se va propaga la (r', c') de două ori, pe căi diferite! Din fericire, este suficient să o scădem o dată. Iată codul:

```
void build() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      int next_r = r + (r & -r);
      int next_c = c + (c & -c);

      if (next_r <= n) {
        mat[next_r][c] += mat[r][c];
      }

      if (next_c <= n) {
        mat[r][next_c] += mat[r][c];
      }

      if ((next_r <= n) && (next_c <= n)) {
        mat[next_r][next_c] -= mat[r][c];
      }
    }
  }
}
```

Capitolul 7

Structuri 2D offline

Acest minicapitol prezintă o clasă de structuri de date care pot procesa operații pe perechi de puncte cu valori într-un interval $[1, n]$ unde $n \sim 10^5$. Structurile obțin timp $\mathcal{O}(n \log^2 n)$ și folosesc memorie $\mathcal{O}(n \log n)$. Aceste structuri se aplică doar în cazul în care datele sînt cunoscute în avans (*offline*). Ele se bazează pe structurile deja studiate (arbori de intervale și arbori indexați binar). Ca să nu tot menționăm „AINT sau AIB" peste tot, ne vom referi doar la AINT-uri.

7.1 Precalcularea

Structurile funcționează astfel. Pe una dintre axe, să spunem pe Ox, construim un arbore de intervale A. Așa cum ne așteptăm, într-un nod oarecare (frunză sau nod intern), acest arbore va stoca informații despre toate punctele ale căror x se încadrează în intervalul subîntins de nod.

Pe Oy nu putem aloca în fiecare dintre nodurile lui A cîte un AINT de mărime n, căci nu ne permitem memorie $\mathcal{O}(n^2)$. În loc de aceasta, facem o trecere prin datele de intrare și colectăm, pentru fiecare nod din A, toate coordonatele y distincte ale punctelor pe care le acoperă nodul. Apoi alocăm un AINT secundar în acest nod, doar de mărimea necesară. În paralel, ținem în nod și lista sortată a coordonatelor y. Arborele secundar este construit peste aceste coordonate.

Suma mărimilor arborilor secundari este $\mathcal{O}(n \log n)$, pentru că fiecare punct va fi acoperit de $\log n$ noduri din A, deci va contribui cu $+1$ la mărimea arborilor secundari din acele noduri.

Rezumînd, precalcularea:

- Parcurge toate punctele de la intrare.
- Pentru fiecare punct $P(x, y)$, parcurge toți strămoșii poziției x în A.
- În fiecare nod strămoș, notează coordonata y.
- La final, sortează lista de coordonate y din fiecare nod și inițializează cîte un AINT secundar în fiecare nod.

Care este complexitatea precalculării? O putem calcula și cu metode elementare. Dar mai instructiv este să apelăm la The Master Theorem. Fie $T(k)$ efortul necesar pentru a preprocesa k puncte. Presupunînd o distribuire relativ uniformă a punctelor pe x, obținem

$$T(k) = \mathcal{O}(k \log k) + 2T(k/2)$$

Regăsim aici cazul 2 al *The Master Theorem*, deoarece $c = c_{crit} = \log_2 2 = 1$. Așadar,

$$T(n) = \mathcal{O}(n \log^2 n)$$

7.2 Procesarea efectivă

Abia apoi procesăm efectiv punctele și facem operațiile specifice problemei. Să considerăm operația de actualizare într-un punct. De exemplu, problema imediat următoare, Festival, atribuie fiecărui punct cîte o valoare. Pentru a actualiza valoarea unui punct $P(x, y)$,

- Vizităm toți strămoșii poziției x în A.
- În fiecare nod strămoș, căutăm binar poziția lui y în lista de y-i a nodului.
- La acea poziție actualizăm AINT-ul secundar conform nevoilor problemei.

Complexitatea acestei operații este $\mathcal{O}(\log^2 n)$ pentru că trebuie să actualizăm $\mathcal{O}(\log n)$ structuri secundare și fiecare actualizare necesită $\mathcal{O}(\log n)$.

Pentru operația de interogare 2D într-un dreptunghi $(x_1, y_1) - (x_2, y_2)$,

- Descompunem $[x_1, x_2]$ în intervale ca în orice AINT.
- În fiecare interval căutăm binar în lista de y-i cea mai mică coordonată $y_{lo} \geq y_1$ și cea mai mare coordonată $y_{hi} \leq y_2$.
- Căutăm rezultatul dorit în arborele secundar pe intervalul $[y_{lo}, y_{hi}]$.
- Agregăm aceste rezultate pentru a obține răspunsul la operație.

Și această operație are tot complexitatea $\mathcal{O}(\log^2 n)$ din același motiv ca mai sus.

7.3 Probleme

7.3.1 Problema Festival (ONI 2025, clasele 11-12)

enunț • sursă

Începem cu precizarea că există și soluții mai eficiente la această problemă (exemplu), bazate pe tehnica Divide et Impera CDQ. Dar iată și soluția de manual cu structuri 2D offline.

Formula de recurență

Este relativ evident că festivalurile formează un DAG. Nodurile corespund festivalurilor, iar între un festival A și un festival B există o muchie dacă și numai dacă Andrei poate ajunge de la A la B. Fiind un DAG, graful are o sortare topologică, de exemplu dacă sortăm festivalurile crescător

după t. De aici rezultă și o formulă implementabilă în $\mathcal{O}(n^2)$. Fie D_i satisfacția maximă pentru a vizita un șir de festivaluri care se încheie la festivalul i. Atunci:

$$D_i = S_i + \max_j \{D_j \mid \text{Andrei poate ajunge de la } j \text{ la } i\}$$

Putem figura grafic un festival (x, t) ca pe un punct $(x, y = t)$. Atunci zona din plan din care Andrei poate ajunge la un festival are forma din figura 7.1:

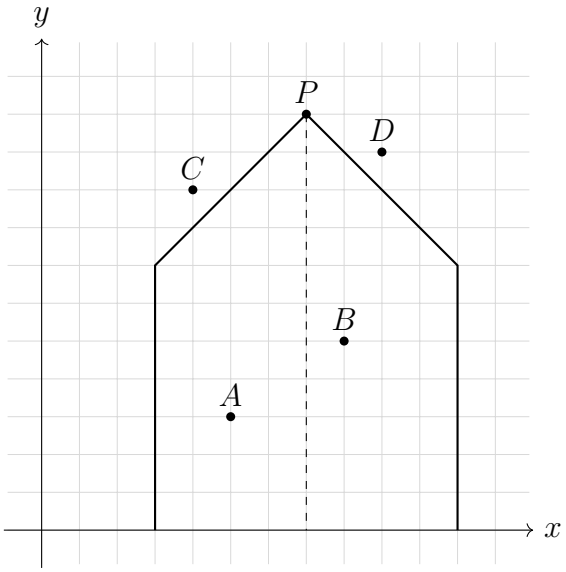


Figura 7.1: Zona de acoperire a festivalului P pentru $d = 4$. Punctul P poate fi vizitat după punctele A sau B, dar nu și după C sau D.

Astfel reducem problema la una de căutări și actualizări într-o structură de date. Pentru fiecare festival i în ordine crescătoare a lui y, calculăm valoarea festivalului ca S_i plus valoarea maximă din zona de acoperire. Apoi notăm această valoare la coordonatele festivalului.

Reducerea la structuri de date 2D

Operațiile pe structuri de date diagonale pot fi complicate. De aceea, să încercăm să reducem zona de acoperire la dreptunghiuri.

Dacă împărțim zona mergînd pe linia punctată care coboară din P, obținem două trapeze. Să analizăm trapezul din dreapta. Nu putem extinde pur și simplu acest trapez la o zonă dreptunghiulară, deoarece am include și punctul D, de la care Andrei nu poate ajunge la P. În schimb, să procedăm astfel. Vom înlocui fiecare punct (x, y) cu punctul $(x, y + x)$. Cu alte cuvinte, pe fiecare coloană i translatăm punctele în sus cu i pași. Astfel, zona trapezoidală devine dreptunghică. Desigur, vom introduce în zona de acoperire și niște puncte cu y negativ, dar aceasta nu denaturează rezultatele, întrucît nu există festivaluri cu timp negativ.

Întrebarea „care este satisfacția maximă în trapezul drept al lui $P(x, y)$?” devine o interogare de maxim în dreptunghiul $(x, -\infty) - (x + d, y + x)$.

Pentru trapezul stîng, dorim ca punctele din stînga să urce mai mult decît cele din dreapta, deci vom inversa translația, înlocuind fiecare punct (x, y) cu punctul $(x, y - x)$. Remarcăm că aceasta poate duce la coordonate y negative, dar vom vedea odată ce definitivăm structura de date că valorile negative nu ne deranjează. Vom folosi două structuri de date, cîte una pentru fiecare tip de translație.

Întrebarea „care este satisfacția maximă în trapezul stîng al lui $P(x, y)$?" devine o interogare de maxim în dreptunghiul $(x - d, -\infty) - (x, y - x)$.

Desigur, D_i este maximul valorilor din cele două trapeze. După calculare, trebuie să notăm valoarea D_i obținută la $(x, y + x)$ în structura dreaptă și la $(x, y - x)$ în structura stîngă.

Structura necesară

Acum știm ce operații trebuie să admită structura dorită:

1. Notarea unei valori într-un punct.
2. Găsirea valorii maxime dintr-un dreptunghi care se extinde în jos pînă la $-\infty$.

Coordonatele respectă $0 \leq x \leq 10^9$ și $-2 \cdot 10^9 \leq y \leq 2 \cdot 10^9$. Ca să putem folosi structuri 2D offline, trebuie să normalizăm valorile x. Este nevoie de atenție la interogări, căci valorile $x - d$ și $x + d$ nu înseamnă „$\pm d$ poziții în structură", ci înseamnă „cea mai mică poziție pe care se găsește o valoare cel puțin egală cu $x - d$" și respectiv „cea mai mare poziție pe care se găsește o valoare cel mult egală cu $x + d$.

Coordonatele y nu trebuie normalizate.

Deoarece avem nevoie de maxim pe interval, vom folosi un arbore de intervale ca structură primară. În structurile secundare avem nevoie de maxim pe prefix, în condițiile în care valorile pot doar să crească, deci putem folosi și AIB-uri. Ca detaliu de implementare, putem folosi AIB-uri indexate de la 0 sau, dacă dorim să le folosim pe cele mai naturale, indexate de la 1, trebuie să nu uităm să adăugăm cîte o santinelă cu $y = -\infty$ în fiecare AIB.

De aici încolo, implementarea este tipică.

7.3.2 Problema Medwalk, revizitată (Lot 2025)

enunț • sursă

Am discutat această problemă și în capitolul de arbori de intervale. Am găsit o soluție bazată pe un AINT de seturi, construit peste valorile din matrice. Să vedem acum una de 5 ori mai rapidă, probabil cea pe care a dorit-o comisia.

Primele observații se mențin. Decuplăm (conceptual) matricea în doi vectori, unul cu minimele și unul cu maximele de pe fiecare coloană. Pentru a minimiza medianul (scorul unui interval $[l, r]$), căutăm un drum minim lexicografic. Acesta va consta din minimele de pe intervalul $[l, r]$ și din minimul maximelor. Medianul acestei mulțimi va fi una din trei valori posibile (logica deciziei este simplă):

- fie minimul maximelor;
- fie medianul minimelor;
- fie elementul anterior medianului minimelor.

Pentru cazul (1), minimul maximelor îl menținem într-un aint simplu, cu actualizări punctuale și cu interogări de minim pe interval. Pentru cazurile (2) și (3) trebuie să răspundem la interogări de al k-lea element pe interval. Aici introducem următoarea soluție, constînd dintr-un AIB 2D offline construit peste minimele din matrice.

Dorim să căutăm binar al k-lea element. Bunăoară, prima dată ne întrebăm: este el mai mare decît $V_{max}/2$? Dacă da, îl căutăm între $V_{max}/2$ și V_{max}. Dacă nu, îl căutăm între 1 și $V_{max}/2$. În general, pentru plaja de valori curentă, $[x, y]$, vom calcula mijlocul $m = (x+y)/2$ și îi vom adresa structurii de date întrebarea: cîte valori între x și m apar la intrare pe poziții între l și r? Dacă răspunsul este $\geq k$, atunci al k-lea element are valoarea între x și m. Altfel, el are valoarea între m și y.

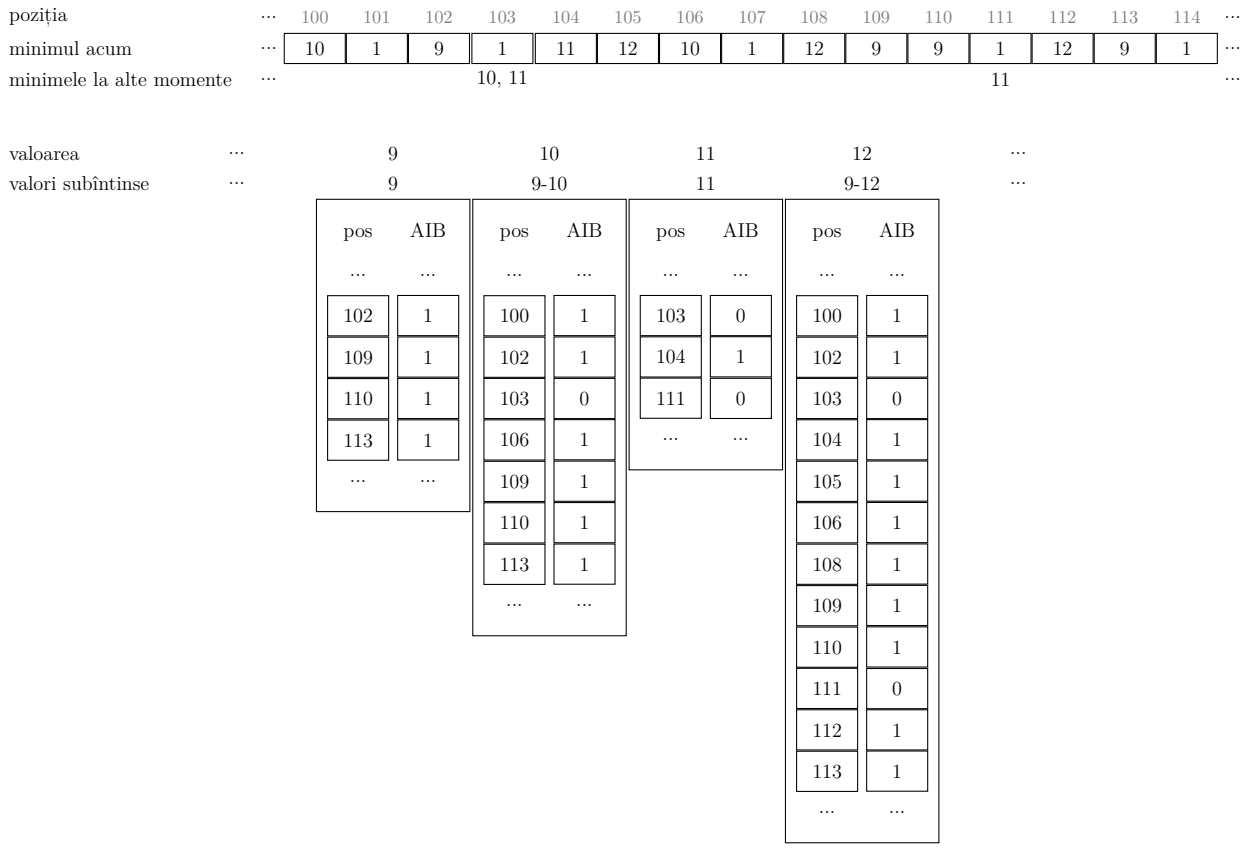
Să încercăm o structură naivă: o matrice binară uriașă A de dimensiuni $n \times V_{max}$, unde $A_{p,v}$ reține 1 dacă valoarea v apare în matrice la poziția p, 0 altfel. Atunci răspunsul la întrebarea „cîte valori între v_1 și v_2 apar pe poziții între l și r?" este suma dreptunghiului $[l, r] \times [v_1, v_2]$. Sună bine, dar memoria nu ne permite această matrice.

De aceea, vom comprima fiecare coloană într-un AIB pornind de la următoarea observație. Coloana v va reține 1, oricînd pe durata întregului program, doar pe liniile p cu proprietatea că poziția p în vectorul de minime va avea, cel puțin după o operație, valoarea v. De aceea, în primă fază simulăm operațiile și colectăm toate minimele posibile pe toate pozițiile, care vor fi cel mult $n + q$. (De aici provine partea de *offline* din nume.) Pe fiecare coloană v stocăm:

- O listă a pozițiilor posibile pentru valori 1, ordonate crescător.
- Un AIB de 0/1 construit peste această listă.

De exemplu, dacă valoarea 9 este măcar o dată minimă pe pozițiile 102, 109, 110, 113, atunci pe coloana 9 stocăm vectorul $[102, 109, 110, 113]$ și un AIB de 0/1 cu 4 poziții disponibile.

Dar facem mai mult decît atît, căci v-am promis un AIB 2D. 🙂 Vectorul de coloane este el însuși un AIB. Coloana v nu reține doar pozițiile aparițiilor lui v, ci ale oricărei valori din $(v - p, v]$, unde p este cea mai mare putere a lui 2 mai mică sau egală cu v. În particular, cele 4 valori 102, 109, 110, 113 exemplificate mai sus nu apar doar în lista coloanei 9, ci și în listele coloanelor $10, 12, 16, 32, 64, \ldots$.



Să observăm că fiecare poziție apare în lista unui număr logaritmic de coloane. De aceea, suma lungimilor tuturor listelor (și a tuturor AIB-urilor) este $\mathcal{O}(n \log V_{max})$.

Pe această structură putem răspunde în timp $\mathcal{O}(n \log n \log V_{max})$ la interogări de al k-lea element.

Capitolul 8

Descompunere în radical

Pe lîngă faptul că oferă complexitate optimă pentru unele probleme, metoda oferă și un substitut bun pentru algoritmi în $\mathcal{O}(n \log n)$ și în special $\mathcal{O}(n \log^2 n)$, în cazul în care nu găsim ideea. Descompunerea în radical este, în experiența mea, mult mai ușor de implementat.

8.1 Actualizări punctuale

Revenim la problema inițială: actualizări punctuale, sume pe interval. Împărțim vectorul în blocuri de lungime $k = \sqrt{n}$. Așadar, vor exista $\lceil n/k \rceil$ blocuri (engl. *blocks* sau *buckets*). Pentru fiecare bloc, menținem o informație suplimentară: suma elementelor din acel bloc.

Memorie suplimentară: $\mathcal{O}(\sqrt{n})$.

```
int v[MAX_N];
int b[MAX_BUCKETS];
int bs, nb;

// Se poate implementa și cu împărțiri pentru concizie (sînt doar n).
void init_buckets() {
  nb = sqrt(n + 1);
  bs = n / nb + 1;

  for (int i = 0; i < nb; i++) {
    int bucket_start = i * bs;
    for (int j = 0; j < bs; j++) {
      b[i] += v[j + bucket_start];
    }
  }
}

int array_sum(int* v, int l, int r) {
  int sum = 0;
  while (l < r) {
    sum += v[l++];
```

```
  }
  return sum;
}

int fragment_sum(int l, int r) {
  return array_sum(v, l, r);
}

int bucket_sum(int l, int r) {
  return array_sum(b, l, r);
}

int range_sum(int l, int r) { // [l, r)
  int bl = l / bs, br = r / bs;
  if (bl == br) {
    return fragment_sum(l, r);
  } else {
    return
      // capete
      fragment_sum(l, (bl + 1) * bs) +
      fragment_sum(br * bs, r) +
      // blocuri complete
      bucket_sum(bl + 1, br);
  }
}

void point_add(int pos, int val) {
  v[pos] += val;
  b[pos / bs] += val;
}
```

Vă recomand să lucrați pe intervale închise la stînga, deschise la dreapta, ca să simplificați aritmetica.

Încheiem secțiunea cu observația că funcția `range_sum` are complexitatea $\mathcal{O}(k + n/k)$: Ea iterează naiv prin blocurile acoperite parțial, așadar $\mathcal{O}(k)$, și iterează rapid prin blocurile acoperite complet, așadar $\mathcal{O}(n/k)$. Alegem $k = \sqrt{n}$ ca să minimizăm acea sumă, dar vom vedea în unele exemple că pot lua naștere și alte sume care duc la valori diferite pentru k.

8.2 Actualizări pe interval

Folosim o informație suplimentară care amintește de informația propagată *lazy* din arborii de segmente. Pentru problema dată (sume pe interval), am denumit această informație `bdelta`. Ea are semnificația: `bdelta[j]` este o valoare care trebuie adăugată la fiecare element din blocul `j`. Așadar, valoarea reală a unui element `i` din blocul `j` este `v[i] + bdelta[j]`.

```
int fragment_sum(int l, int r, int bucket) {
```

```
  return
    (r - l) * bdelta[bucket] +
    array_sum(v, l, r);
}

int bucket_sum(int l, int r) {
  return
    array_sum(bsum, l, r) +
    bs * array_sum(bdelta, l, r);
}

int range_sum(int l, int r) {
  int bl = l / bs, br = r / bs;
  if (bl == br) {
    return fragment_sum(l, r, bl);
  } else {
    return
      // capete
      fragment_sum(l, (bl + 1) * bs, bl) +
      fragment_sum(br * bs, r, br) +
      // blocuri complete
      bucket_sum(bl + 1, br);
  }
}

void array_add(long long* v, int l, int r, int val) {
  while (l < r) {
    v[l++] += val;
  }
}

void fragment_add(int l, int r, int bucket, int val) {
  bsum[bucket] += (r - l) * val;
  array_add(v, l, r, val);
}

void range_add(int l, int r, int val) {
  int bl = l / bs, br = r / bs;
  if (bl == br) {
    fragment_add(l, r, bl, val);
  } else {
    // capete
    fragment_add(l, (bl + 1) * bs, bl, val);
    fragment_add(br * bs, r, br, val);

    // blocuri complete
    array_add(bdelta, bl + 1, br, val);
  }
}
```

Paranteză: conceptual, aint-urile fac același lucru cu descompunerea în radical, deci multe concepte se translatează între cele două, cum ar fi propagarea *lazy*. Marea inovație a arborilor de intervale este că garantează descompunerea în $\mathcal{O}(\log n)$ blocuri.

8.3 Optimizări

8.3.1 Evitați împărțirile!

Codul anterior face doar două împărțiri per operație. În general, aveți nevoie strict de o împărțire pentru fiecare indice primit ca parametru. Evitați stilul de mai jos, care face $\mathcal{O}(\sqrt{n})$ împărțiri (sau operații modulo) per operație. El poate fi **de cîteva ori mai lent** (vedeți *benchmark*-urile).

```
int bl = l / bs, br = r / bs;
int sum = 0;

// stînga
do {
  sum += v[l++];
} while (l % bs);

// dreapta
while (r % bs) {
  sum += v[--r];
}

// blocuri complete
for (int i = bl + 1; i < br; i++) {
  sum += b[i];
}

return sum;
```

8.3.2 Alegerea mărimii blocurilor

Am auzit că mărimea blocului merită declarată constantă, deoarece compilatorul va optimiza împărțirile. Am experimentat, dar diferențele nu sînt semnificative. Am încercat chiar să declar mărimea o putere a lui 2, pentru ca împărțirile să fie ieftine. Codul rezultat a fost mai lent! Cred că motivul este că se dezechilibrează acea funcție $k + n/k$ pe care descompunerea în radical o optimizează.

Diferența de viteză este vizibilă cînd codul face multe împărțiri. Cînd codul face $\mathcal{O}(1)$ împărțiri per operație, nu mai contează.

Are sens să declarați mărimea constantă dacă vi se pare codul mai simplu (evitați cîteva calcule la inițializare). În acest caz, vă recomand să puneți constanta mică (3-4) cînd lucrați local și să-i

dați valoarea reală cînd trimiteți sursa. Altfel, dacă testați local pe teste mici și mărimea blocului 300, nu veți testa decît codul cu interogări și actualizări într-un singur bloc.

8.3.3 Alegerea mărimii blocurilor pentru operații inegale

Dacă programul vostru depășește timpul, merită să variați mărimea blocurilor în jurul lui $\sqrt{n}$ ca să vedeți dacă se schimbă ceva. Fie b numărul de blocuri și fie l lungimea unui bloc. Operațiile pot depinde doar de una dintre aceste variabile sau pot depinde de ambele, dar în mod inegal. Exemplu: dacă constanta pentru cele două capete de segment (de lungime medie $l/2$) este mult mai mare decît constanta pentru blocurile întregi, merită redus l-ul puțin.

Iată un exemplu mai interesant. Să spunem că nu știm să rezolvăm corect una dintre operații și găsim doar o implementare în $\mathcal{O}(b+l^2)$. Pare catastrofic: dacă alegem $b = l = \sqrt{n}$, avem de fapt o soluție în $\mathcal{O}(n)$. 🤔

Dar se poate asimptotic mai bine. Să alegem $b = n^{2/3}$, caz în care $l = n/b = n^{1/3}$. Dacă $n = 100.000$, atunci $b = 2.174$ și $l = 46$. O soluție în $\mathcal{O}(\sqrt{n})$ per operație ar face circa 600 de operații, pe cînd a noastră va face circa 4.300 de operații. Desigur, diferența este notabilă, dar, cu o implementare eficientă, avem șanse să „ținem aproape".

Pauză de matematică: mai exact, noi vrem să găsim minimul funcției

$$f(x) = x^2 + \frac{n}{x}$$

Funcția își atinge minimul cînd derivata este zero, iar derivata este

$$f'(x) = 2x - \frac{n}{x^2} = \frac{2x^3 - n}{x^2}$$

Rezultă că l minim este $\sqrt[3]{n/2} \approx 37$, iar funcția va face sub 4.100 de operații.

8.4 Probleme

8.4.1 Problema Mexitate (ONI 2018 clasa a 9-a)

enunț • surse

Această problemă nu mi se pare nici în ruptul capului de clasa a 9-a. Posibil de baraj juniori.

Am notat cu m numărul de linii deoarece m vine înaintea lui n în alfabet, după cum și k vine înaintea lui l în alfabet.

Să luăm în calcul soluția naivă: calculăm mex-ul matricei din colțul stînga-sus, apoi translatăm în diverse feluri matricea pentru a recalcula incremental mex-urile celorlalte ferestre. De exemplu, putem urma un traseu șerpuit: translatăm matricea la dreapta pînă la capăt, apoi o dată în jos, apoi în stînga pînă la capăt etc.

La fiecare translație, menținem într-o structură S informații despre frecvența elementelor din matrice. Ștergem din structură elementele care ies din fereastră și le inserăm pe cele care intră în fereastră. Atunci complexitatea translatărilor va fi $\mathcal{O}(mnk + ml)$. Ca să minimizăm efortul, rotim sau transpunem matricea cînd $k > l$. Astfel, $k \leq l$ și complexitatea translatărilor va fi $\mathcal{O}(mn\sqrt{mn})$.

Odată ce am adus raționamentul pînă aici, eu am și trimis o sursă, cu structura S implementată naiv ca vector de frecvențe. Astfel m-am asigurat că restul programului funcționează, căci el are suficient de multă logică (matrice de dimensiuni arbitrare, transpoziții, șerpuire...). Deoarece am gîndit totul modular, am programat structura `frequency_tracker` să expună funcțiile `add`, `remove` și `mex`. Pentru versiunea optimizată, doar am rescris acele funcții.

Pentru punctaj maxim, ce ne dorim de la structura S? Dorim să facem $\mathcal{O}(mn\sqrt{mn})$ inserări și ștergeri și $\mathcal{O}(mn)$ calcule de mex. Rezultă că ne permitem $\mathcal{O}(\sqrt{mn})$ per apel de mex, dar avem nevoie de $\mathcal{O}(1)$ pe inserare și ștergere.

Atunci putem folosi descompunerea în radical. Stocăm vectorul naiv de frecvențe. În plus, în fiecare bloc stocăm numărul de valori nenule. Inițial această valoare este 0. Ea crește cu 1 ori de cîte ori frecvența unui element din bloc crește de la 0 la 1 și, invers, scade cu 1 ori de cîte ori frecvența unui element din bloc scade de la 1 la 0. Funcția mex caută din bloc în bloc pînă cînd găsește un bloc cu cel puțin o valoare nulă, apoi caută naiv prin acel bloc.

8.4.2 Problema Give Away (SPOJ)

enunț • surse

Problema are limită de timp mare (1-2 secunde, iar conform paginii *status*, chiar peste 6 secunde). Acesta este un indiciu bun că o soluție în $O((n+q)\sqrt{n})$ este arhisuficientă. Doar că pare mai rău de atît! Să presupunem că ținem pe fiecare bloc o structură S (nu știm încă ce). Atunci:

- Sigur putem procesa actualizările în $O(\sqrt{n})$, chiar și naiv la o adică.
- La căutare, pare simplu să procesăm naiv blocurile acoperite parțial, în $O(\sqrt{n})$.
- Dar ce facem cu blocurile acoperite complet? Dacă folosim orice structură echilibrată, introducem un factor logaritmic în plus pentru căutarea lui c.

Rezultă o complexitate de $\mathcal{O}(q\sqrt{n}\log n)$. Dar în 6 secunde, este acceptabil.

Mărimea blocurilor

Dacă am vrea să optimizăm suma $k + n/k$, am alege $k = \sqrt{n} \approx 707$. Dar noi dorim să optimizăm suma $k + n/k \log k$. Ochiometric log-ul va fi undeva între 7 și 12, deci este important să-l mărim pe k. Atunci $\log k$ va crește lent, dar n/k va scădea rapid. Din cîteva încercări pe hîrtie aflăm că valoarea optimă pentru k este 2.000 sau 3.000. Într-adevăr, timpii de execuție pe acolo ating optimul.

Detalii de implementare

Așadar, dorim o structură S care să admită inserări, ștergeri, și numărarea elementelor mai mari sau egale cu o valoare dată. Eu am izolat această structură într-un `struct` și am scris o primă implementare naivă (S este doar un vector). Cu doar 5 linii de cod în plus, sursa naivă mă ajută să verific corectitudinea restului programului.

A doua încercare a fost cu structuri de date din STL, care trece în 3 secunde. Dar este nevoie să memorați papagalicește două noțiuni (le vom relua în capitolele viitoare):

1. `set`-ul simplu nu este suficient, căci el poate să caute valoarea c, dar nu și să numere elementele mai mari sau egale cu c. Este nevoie de structuri cu statistici de ordine (PBDS).
2. Blocurile pot conține valori egale, deci ne trebuie un multiset. Multisetul PBDS este o încropeală, iar ștergerea trebuie rescrisă.

Dar a treia încercare este elementară și trece în 1.3 secunde: pe fiecare bloc ținem vectorul original (ca să știm ce element înlocuim) și o copie sortată. Putem căuta binar în acea copie, iar la inserare/ștergere o actualizăm prin deplasări naive.

8.4.3 Problema Holes (Codeforces)

enunț • sursă

Îmi place această problemă pentru că este nestandard. Nu facem efectiv împărțirea în blocuri și nu stocăm informații agregate pe fiecare bloc. În loc de aceasta, fiecare poziție *pos* reține informații ca să își accelereze trecerea prin blocul său:

- care este ultima destinație din același bloc pe care o vizitează o bilă pornind de la *pos*;
- cîte salturi face bila pînă la acea destinație.

Atunci operațiile sînt:

- La interogare, urmărim traseul bilei din bloc în bloc, în timp $\mathcal{O}(\sqrt{n})$.
- La actualizare, trebuie să recalculăm poziția modificată și toate pozițiile din stînga ei, din același bloc, tot în $\mathcal{O}(\sqrt{n})$.

Sursa mea se apropie de limita de timp (700 ms din 1000 ms). Am încercat următoarea optimizare care cred că ajută. Am ales o mărime mai mare pentru blocuri, 1.000 în loc de cea teoretică ($\sqrt{100.000} \approx 316$). Astfel accelerăm interogările, care traversează mai puține blocuri, în defavoarea actualizărilor, care au de recalculat mai multe poziții. Dar actualizările operează foarte local, pe 1.000 de poziții vecine și vor beneficia de cache. În schimb, interogările sar de colo-colo prin vector.

Remarc și că cele mai rapide soluții ajung la 122 ms. Ele folosesc *link-cut trees* pentru a obține $\mathcal{O}(n \log n)$. Putem percepe structura ca fiind arborescentă: părintele unei poziții este poziția unde sare bila. Atunci actualizările schimbă structura arborelui, ancorînd poziția modificată de un alt părinte. De aici (cred) decurge soluția cu *link-cut trees*.

8.4.4 Problema Piezișă (Baraj ONI 2022)

enunț • surse

Un element comun tuturor soluțiilor este: renumerotăm vectorul de la 1. Acum, dacă xorul pe $[l, r]$ este 0, atunci xorurile pe $[0, l-1]$ și $[0, r]$ sînt egale. Așadar, calculăm xorurile parțiale și le normalizăm, ca să fie indexabile. Acum răspunsul la orice interogare $[l, r]$ este o pereche (x, y) cu $0 \leq x < l$, $r \leq y \leq n$ și $v[x] = v[y]$.

Brute force de 100p

Menționez pentru completitudine că următorul *brute force* optimizat ia 100p: Răspundem la interogări online, imediat ce le primim. Fie o interogare $[l, r]$. Căutăm binar ultima apariție a lui $v[r]$ pe o poziție anterioară lui l (pentru aceasta, colectăm în prealabil listele de poziții pentru fiecare valoare distinctă din v). Fie această poziție q. Atunci avem o soluție de mărime r–q, posibil ∞ dacă elementul $v[r]$ nu apare înainte de poziția l. Repetăm aceeași întrebare la pozițiile $r+1, r+2, \ldots$. Menținem minimul răspunsurilor la aceste căutări, fie el m. Ne oprim la poziția $l+m$ (sau, desigur, la n), deoarece dincolo de ea am putea primi doar răspunsuri mai mari decît optimul curent. Afișăm răspunsul m.

Metoda 1

Fără sortarea interogărilor nu am găsit nicio soluție. Așadar, să sortăm interogările după capătul stîng și să răspundem la ele baleind l de la 1 la n. Acum, pentru o interogare $[l, r]$, dorim ca pentru fiecare poziție $r' \geq r$ să aflăm dacă valoarea $v[r']$ există pe vreo poziție $l' \leq l$. Aceasta este mult prea scump, deci dorim cumva să răspundem la întrebări pentru mai multe poziții simultan.

Să descompunem vectorul în bucăți de mărime $\sqrt{n}$. Pentru o interogare $[l, r]$, fie blocurile celor două capete bl și br. Atunci răspunsul poate avea capătul stîng fie în blocul bl (în stînga lui l), fie într-un bloc anterior. Similar pentru capătul drept. Cazul care ne încurcă este cel în care ambele capete sînt în blocurile bl, respectiv br. Vrem să evităm să comparăm naiv aceste $\mathcal{O}(\sqrt{n} \times \sqrt{n}) = \mathcal{O}(n)$ posibilități.

Astfel ne vine ideea să calculăm o singură informație pentru toate pozițiile din stînga lui l. Fie $left[x]$ ultima poziție (înaintea lui l) pe care apare valoarea x. Evident, putem menține vectorul $left$ în $\mathcal{O}(1)$ pe măsură ce l avansează.

Capătul drept al interogării poate varia oricum, și aici intervine descompunerea în radical. Fie $best[b]$ cea mai mică soluție cunoscută pînă în prezent între orice valoare din blocul b și orice valoare din stînga lui l. Putem menține vectorul $best$ în $\mathcal{O}(1)$ per bloc pe măsură ce l avansează (așadar efort $\mathcal{O}(n\sqrt{n})$ efort global). Pentru aceasta, trebuie să cunoaștem, pentru elementul în curs de procesare $v[l]$, cea mai din stînga apariție a sa în fiecare bloc următor. Putem precalcula această informație la început, într-o manieră similară cu colectarea listelor de apariții a fiecărei valori. Vezi vectorul `ptr` și funcția `preprocess_values`.

Cu aceste informații, răspunsul pentru interogarea curentă $[l, r]$ este minimul dintre:

- Valorile *best* ale blocurilor mai mari decît br.
- Diferențele $r' - left[v[r']]$ pentru elementele din blocul br începînd cu poziția r.

Codul este lung, cam urît și cam lent, dar ia 100p.

Metoda 2

Citind surse mai rapide decît a mea, am găsit-o pe aceasta, care folosește o metodă mai simplă. Principala diferență este că sortează interogările diferit: crescător după blocul lui l, iar la egalitate descrescător după r. Această sortare amintește de algoritmul lui Mo, pe care îl vom discuta în curînd.

Vectorul $left$ are aceeași definiție ca mai înainte, dar include doar elementele dinaintea blocului bl.

La procesarea unui bloc, avem avantajul că toate r-urile scad. De aceea, și la dreapta putem ține un vector $right$ similar cu $left$: $right[x]$ indică prima apariție a lui x pe o poziție mai mare decît r-ul curent. Atenție, vectorul $right$ trebuie golit și recalculat pentru fiecare bloc bl. Acest efort este $\mathcal{O}(n\sqrt{n})$, deci acceptabil.

Pe măsură ce r scade, menținem și valoarea minimă m a oricărei perechi $right[x] - left[x]$. Deoarece r doar scade, intervalele doar scad, deci m poate doar să scadă.

Acum, pentru $[l, r]$, răspunsul poate fi:

- chiar m, dacă soluția are capătul stîng înaintea blocului bl;
- altfel, minimul dintre $right[x] - x$ pentru toate valorile x din blocul bl, din stînga lui l.

Implementarea este conceptual mai simplă și de peste două ori mai rapidă!

8.5 Descompunere după operații

Iată acum o tehnică înrudită. Proiectăm o structură relativ naivă pentru procesarea operațiilor. Prin „naiv” înțelegem, de exemplu, inserarea într-un vector prin deplasarea elementelor, fără a folosi structuri de date echilibrate.

Facem aceste operații naive cîtă vreme ele se încadrează în $\mathcal{O}(\sqrt{n})$ sau $\mathcal{O}(\sqrt{q})$ per operație.

Periodic, trebuie să intervenim pentru ca structura naivă să nu degenereze în timp și să nu ajungă la $\mathcal{O}(n)$ sau $\mathcal{O}(q)$. De aceea, aproximativ o dată la $\sqrt{q}$ operații iterăm prin structură și o „consolidăm” (ce înseamnă asta depinde de la problemă la problemă). Această consolidare poate dura $\mathcal{O}(n)$, pentru ca efortul total al consolidărilor să fie $\mathcal{O}(n\sqrt{q})$. Această limită de timp face consolidările să fie facile, în general.

Să studiem două probleme concrete.

8.6 Probleme

8.6.1 Problema Serega and Fun (Codeforces)

enunț • surse

Trebuie să procesăm eficient operațiile:

1. Rotește circular la dreapta, cu o poziție, un interval $[l, r]$.
2. Raportează frecvența unei valori k într-un interval $[l, r]$.

Iată întîi soluția „clasică", cu descompunere în blocuri de mărime egală. Soluția relativ directă. În fiecare bloc putem menține:

1. O listă a elementelor.
2. Informații despre frecvență (`map` sau vector simplu).

Atunci, la rotire, trebuie să:

1. Rotim naiv blocurile acoperite parțial.
2. Rotim eficient blocurile acoperite complet. La rotire, adăugăm la începutul listei elementul care ne parvine de la blocul anterior și trimitem ultimul element din listă în blocul următor.

La interogare, trebuie să:

1. Consultăm element cu element blocurile acoperite parțial.
2. Consultăm vectorii de frecvență ai blocurilor acoperite complet.

Discuție despre implementarea cu structuri STL

Putem rezolva problema și cu structuri ca `deque`, `map` sau `unordered_map`, dar este nevoie de o implementare atentă ca să nu depășim timpul și memoria.

În particular, mare atenție la operatorul `[]`! Este tentant să îl folosim ca să aflăm frecvența unui element. Dacă elementul nu există în `map`, operatorul va returna 0, ceea ce este corect. Dar **operatorul inserează elementul** dacă nu exista deja.

Ce înseamnă asta? În mod normal, suma mărimilor tabelelor hash din toate blocurile va fi n. Dar, dacă pentru fiecare din cele q interogări noi căutăm o valoare inexistentă în fiecare dintre cele $\sqrt{n}$ blocuri, și dacă toate acele valori sînt inserate, suma mărimilor tabelelor va ajunge la $q\sqrt{n}$. Necesarul de memorie va crește enorm. Este obligatoriu să folosim iteratori pentru căutarea sau decrementarea frecvențelor, pentru ca mărimea tabelelor să rămînă $\mathcal{O}(\sqrt{n})$.

Detalii de implementare

Prime sursă stochează lista de elemente din fiecare bloc într-un `deque` din STL. A doua sursă folosește un simplu buffer circular: un vector de mărime `BUCKET_SIZE` împreună cu un pointer la primul element din listă. Atunci putem face rotirea completă în $\mathcal{O}(1)$ mutînd pointerul de start

cu un element spre stînga. Pe poziția noului element de start scriem valoarea provenită din blocul anterior.

O altă diferență este că prima sursă stochează frecvențele din fiecare bloc într-o tabelă hash, pe cînd a doua folosește un vector de frecvențe, căci elementele au valori de cel mult n. Necesarul de memorie crește la $\mathcal{O}(n \times numBuckets)$. Din acest motiv, merită să experimentăm cu mai puține blocuri de dimensiune mai mare. Într-adevăr, pentru blocuri de mărime $2\sqrt{n} \approx 640$, timpul scade la jumătate față de $\sqrt{n}$.

Mai remarcăm că frecvența într-un singur bloc încape pe tipul `short`, nu este nevoie de `int`. Doar cu această modificare am redus timpul de rulare cu 20%. Am experimentat și cu `unsigned char`, dar atunci frecvența maximă stocabilă ar fi 256, deci mărimea maximă a unui bloc ar trebui să fie 256, iar timpul se înrăutățește.

Soluție cu descompunere după operații

Iată acum și rezolvarea în care lăsăm blocurile să fluctueze ca lungime. La rotire, extragem efectiv elementul de pe poziția r din blocul său și îl inserăm la indicele corect în blocul poziției l. Blocurile dintre l și r rămîn nemodificate. Ca fapt divers, complexitatea rotirii depinde doar de mărimea blocului, nu și de numărul de blocuri.

Cu timpul, lungimile blocurilor pot degenera, ceea ce poate încetini operațiile: dacă un bloc devine foarte mare, atunci inserarea, ștergerea și numărarea de elemente din acel bloc vor fi lente. De aceea, periodic refacem structura blocurilor:

1. Colectăm toate blocurile într-un vector. Acesta este chiar conținutul real al vectorului.
2. Redistribuim elementele în blocuri de mărime egală.

Putem face redistribuirea fie periodic (la fiecare $\mathcal{O}(\sqrt{q})$ operații), fie la nevoie, cînd în momentul inserării detectăm că unul dintre blocuri a atins un anumit prag, să zicem dublu față de lungimea inițială. În ambele cazuri, complexitatea rămîne $\mathcal{O}(q\sqrt{n})$.

Această implementare este relativ elementară și se mișcă de circa două ori mai repede decît precedenta!

8.6.2 Problema Rev (Lot 2008)

enunț • sursă

Descompunerea în radical după poziții nu pare să ducă nicăieri. Un interval răsturnat generează modificări majore în blocurile acoperite. În schimb, descompunerea în radical după operații duce la o soluție în $\mathcal{O}((q+n)\sqrt{q})$. Această descompunere ne permite să tratăm naiv operațiile, căci efortul nu crește dincolo de $\mathcal{O}(\sqrt{q})$.

Concret, inițial vom reprezenta starea vectorului printr-un singur „bloc”:

$$[0, n-1]$$

Semnificația este că trebuie să iterăm prin pozițiile $0, 1, \ldots, n-1$ (am indexat pozițiile de la zero). Dacă acum inversăm intervalul $[13, 28]$, atunci pentru a descrie starea curentă a vectorului avem nevoie de trei blocuri:

$$[0, 12], [28, 13], [29, n-1]$$

Blocul din mijloc arată că elementele trebuie parcurse de la indicele 28 descrescător către indicele 13. În cazul general în care descrierea vectorului constă din mai multe blocuri și dorim să inversăm intervalul $[l, r]$, urmăm pașii:

1. Găsește blocurile de care aparțin l și r.
2. Dacă l nu cade pe prima poziție a blocului său, sparge blocul în două.
3. Dacă r nu cade pe ultima poziție a blocului său, sparge blocul în două.
   - Astfel ne asigurăm că intervalul $[l, r]$ acoperă numai blocuri întregi.
4. Răstoarnă secvența de blocuri dintre blocurile lui l și r.
5. Inversează capetele fiecărui bloc din această secvență.

Iată un exemplu mai complicat. Fie $n = 40$ și lista:

$$[20-25], [39-36], [0-15], [35-26], [19-16]$$

(Blocurile au lungimi 6, 4, 16, 10, 4.) Să inversăm intervalul $[6, 30]$. Aflăm că poziția #6 este în blocul $[39-36]$ pe poziția 0 (indice 39), iar poziția #30 este în blocul $[35, 26]$ pe poziția 4 (indice 31). Poziția #6 este așadar la început de bloc. Poziția #30 nu este la sfîrșit de bloc, așa că spargem blocul $[35-26]$ și obținem lista:

$$[20-25], [39-36], [0-15], [35-31], [30-26], [19-16]$$

Acum inversăm blocurile $[39-36]$, $[0-15]$, $[35-31]$ și le schimbăm direcția, obținînd:

$$[20-25], [31-35], [15-0], [36-39], [30-26], [19-16]$$

Pentru a găsi valoarea de pe o poziție, parcurgem naiv lista de blocuri pînă ajungem la cel care include poziția dorită. De exemplu, poziția #20 se regăsește în blocul $[15, 0]$ pe poziția 9, așadar la indicele 6 în vectorul original.

Lungimea listei de blocuri poate crește cu 2 la fiecare operație. Ca să nu ajungem la timpi $\mathcal{O}(q)$ per operație, la fiecare (circa) $\sqrt{q}$ operații **defragmentăm** lista. Pentru aceasta, iterăm prin lista de blocuri și vărsăm într-un vector elementele de la indicii întîlniți. Astfel reconstruim vectorul real. Apoi resetăm lista de blocuri la același unic bloc, $[0, n-1]$.

8.7 Procesări diferite înainte și după $\sqrt{n}$

Următoarele două secțiuni teoretice prezintă două tehnici care ating timp $\mathcal{O}(n\sqrt{n})$ din motive matematice. Tehnicile nu duc la descompunere în radical „convențională”, cu blocuri, dar nu știu unde altundeva să le încadrez.

Prima tehnică pornește de la observațiile:

1. Între 1 și $\sqrt{n}$ există $\sqrt{n}$ valori[1].
2. Valorile de forma $\lfloor n/k \rfloor$, cu $k > \sqrt{n}$, iau doar $\mathcal{O}(\sqrt{n})$ valori distincte.

8.8 Probleme

8.8.1 Problema Time to Raid Cowavans (Codeforces)

enunț • sursă

În foarte multe cuvinte, problema ne dă un vector cu n elemente și q interogări $\langle prim, pas \rangle$. Răspunsul fiecărei interogări este suma valorilor de pe pozițiile care formează progresia cu primul termen $prim$ și pasul pas.

Desigur, codul naiv care însumează progresiile este lent:

```
long long naive_prog_sum(int first, int step) {
  long long sum = 0;
  for (int i = first; i <= n; i += step) {
    sum += w[i];
  }
  return sum;
}
```

Dar... nu chiar atît de lent! Dacă pasul este cel puțin $\sqrt{n}$, progresia va avea cel mult $\sqrt{n}$ termeni. Rămîne să tratăm progresiile cu pasul mic (așadar $1, 2, \ldots, \sqrt{n}$). Ne permitem să facem o preprocesare în $\mathcal{O}(n)$ pentru fiecare din acești pași. Pentru un pas pas, preprocesăm efectiv $prep[i]$ = răspunsul la progresia cu primul termen i și pasul pas. Apoi putem răspunde la interogările pentru acel pas în $\mathcal{O}(1)$.

```
void preprocess(int step) {
  for (int i = n; (i > n - step) && (i >= 1); i--) {
    prep[i] = w[i];
  }
  for (int i = n - step; i >= 1; i--) {
    prep[i] = w[i] + prep[i + step];
  }
}
```

[1] From the Department of Redundancy Department.

Complexitatea totală este:

- $\mathcal{O}(q\sqrt{n})$ pentru progresiile cu pas mare (calculate naiv);
- $\mathcal{O}(n\sqrt{n}+q)$ pentru preprocesarea tuturor răspunsurilor pentru pași mici.

Alte probleme similare:

- Train Maintenance (Codeforces);
- Căsuța (Lot juniori 2025) – atenție, la momentul scrierii acestei secțiuni problema nu are checker.

8.9 Descompunere în valori distincte

Această tehnică pornește de la observația: dacă suma unor numere naturale este n, atunci numerele au $\mathcal{O}(\sqrt{n})$ valori distincte. Demonstrația decurge din inversa sumei Gauss.

8.10 Probleme

8.10.1 Problema Sandor (Baraj ONI 2025)

enunț • sursă

Observăm că algoritmul lui Sandor este un algoritm *greedy* pentru problema rucsacului. Să facem trei experimente de gîndire despre natura acestui algoritm.

În primul rînd, dacă eliminăm un obiect pe care algoritmul oricum nu l-ar selecta (pentru că nu încape), atunci vom obține aceeași sumă ca și cînd nu am elimina nimic. Practic, obiectul nu există pentru algoritm.

Mai interesant, putem generaliza prima observație. Dacă există k obiecte de aceeași greutate și, la acel pas în algoritm, în rucsac încap mai puțin de k din aceste obiecte, atunci pe oricare dintre ele încercăm să-l eliminăm vom obține aceeași greutate ca și cînd nu am elimina nimic. De ce? Dacă, de exemplu, există 5 obiecte identice și doar 3 încap în rucsac, atunci dacă îl eliminăm pe primul algoritmul îl va adăuga pe al 4-lea.

În sfîrșit, deoarece în rucsac punem obiecte cu greutatea totală cel mult g, rezultă că vom pune $\mathcal{O}(\sqrt{g})$ greutăți distincte.

De aceea, merită să stocăm mai degrabă un vector de serii (în sursă le-am numit *runs*) de elemente egale, $\langle val, cnt\rangle$, decît valorile originale. Peste acest vector precalculăm niște *jump pointers*, adică răspunsurile la întrebări de tipul „Cu ce serie să continui algoritmul dacă rucsacul mai are capacitate rămasă c?” Cu această reprezentare, evaluarea algoritmului lui Sandor este elementară și necesită timp $\mathcal{O}(\sqrt{g})$. Mai mult, putem parametriza algoritmul ca să-l putem rula începînd cu oricare dintre serii, nu neapărat cu prima.

Cerința 1

De aici rezultă un algoritm relativ naiv pentru cerința 1. El necesită $\mathcal{O}(\sqrt{g}^2)$, adică $\mathcal{O}(g)$.

Mergînd de la stînga la dreapta prin vector, considerăm fiecare serie $\langle val, cnt \rangle$. Dacă în prezent în rucsac încap mai puțin de *cnt* obiecte, atunci conform observației din preambul oricum am elimina un element din serie obținem tot soluția inițială (avem *cnt* astfel de moduri).

Dacă încap toate obiectele, atunci are sens să ne punem întrebarea „dar dacă eliminăm unul, ce obținem?". Deci punem în rucsac doar $cnt - 1$ obiecte și simulăm algoritmul lui Sandor (naiv) pentru restul vectorului. Apoi revenim la problema originală, punem în rucsac toate cele *cnt* obiecte și continuăm.

Așadar, facem $\mathcal{O}(\sqrt{g})$ simulări ale algoritmului, pentru o complexitate totală de $\mathcal{O}(g)$. Este important să nu luăm în calcul valorile care nu încap în rucsac, deoarece complexitatea ar crește la $\mathcal{O}(n\sqrt{g})$. Acele valori le sărim folosind *jump pointers*. Doar le contorizăm, prin diferența între n și numărul de valori pe care le-am luat în calcul. Fiecare valoare sărită ne dă un mod de a obține greutatea algoritmului lui Sandor fără eliminări.

Vom parametriza și cerința 1 pentru a o putea rula începînd cu orice serie și cu orice capacitate reziduală a rucsacului, nu neapărat cu prima serie și cu capacitatea g.

Cerința 2

Dacă reușim să facem același gen de trecere prin vector și pentru cerința 2, și să delegăm subprobleme la algoritmul naiv (fără eliminări) și la cerința 1 (o eliminare), atunci vom obține o complexitate de $\mathcal{O}(g\sqrt{g})$. Pentru aceasta, trebuie să analizăm cazurile posibile.

În primul rînd, cîtă vreme din seria curentă nici măcar un obiect nu încape un rucsac, trecem la seria următoare și contorizăm numărul de obiecte ignorate. Fie acest contor *mult*. Să spunem că avem capacitatea $c = 100$ și am ignorat serii de greutate 200, 150 și 130, totalizînd $mult = 10$ obiecte. Atunci avem $C^2_{mult} = 45$ de moduri de a elimina două din aceste obiecte. Rulăm algoritmul lui Sandor naiv și vedem ce sumă obținem. Adăugăm 45 la răspunsul pentru acea sumă.

Dacă măcar un exemplar încape în rucsac, atunci putem încerca să eliminăm un obiect din seriile anterioare și unul din seria curentă. Deci apelăm cerința 1 începînd cu poziția curentă, și îi transmitem că fiecare soluție găsită trebuie socotită de *mult* ori, deoarece există *mult* moduri de a elimina un obiect dinaintea seriei curente.

Dacă seria curentă include cel puțin două obiecte, putem încerca să eliminăm două obiecte din seria curentă, punînd $cnt-2$ în rucsac. Desigur, există C^2_{cnt} moduri de a face asta, iar pentru restul vectorului rulăm Sandor varianta de bază. Îi trimitem algoritmului parametrizat multiplicatorul C^2_{cnt} pentru soluția pe care o va găsi.

Similar, putem elimina doar un obiect din grupa curentă, punînd $cnt - 1$ în rucsac, ceea ce putem face în *cnt* moduri distincte. Iar pentru seriile următoare apelăm cerința 1, spunîndu-i că soluțiile

găsite trebuie numărate de cîte *cnt* ori.

În sfîrșit, putem să punem toate obiectele în rucsac și să reconsiderăm cerința 2 de la seria următoare.

8.10.2 Problema Puzzle-bila (Lot 2025)

enunț • sursă

Formularea programării dinamice

Vom denumi **fereastră** o serie de celule libere consecutive.

Pare firesc să ne dorim să calculăm valori de următorul tip. Fie $C_{r,c}$ costul pentru a aduce bila pe rîndul r, coloana c. E destul de clar că $C_{r,c}$ va depinde doar de valori din C de pe rîndul $r-1$. Dacă $C_{r,c}$ va depinde de $C_{r-1,c'}$, atunci va trebui să calculăm și costul de a aduce pe rîndul r o fereastră pe intervalul $[c', c]$.

Așadar, să definim și $D_{r,c,l}$ ca fiind costul de a aduce pe rîndul r o fereastră de lățime (cel puțin) l terminată la coloana c. Acum putem defini recurența pentru C:

$$C_{r,c} = \min_{c'=1}^{c}(C_{r-1,c'} + D_{r,c,c-c'+1})$$

Recurența modelează ideea că mergem pe rîndul $r-1$ pînă la coloana c', apoi coborîm pe rîndul r unde am pregătit o fereastră care ne duce pînă la coloana c.

Reducerea complexității

O soluție în $\mathcal{O}(nm^2)$ procedează astfel: pentru fiecare celulă (r, c) considerăm fiecare fereastră de pe rîndul r. Dacă fereastra are lățime l și trebuie deplasată cu p celule pentru a o alinia la dreapta cu coloana c, atunci putem optimiza $C_{r,c}$ cu cantitatea:

$$p + \text{rmq}(C_{r-1,c-l+1}, \ldots, C_{r-1,c})$$

, unde desigur rmq este funcția de minim pe interval. Doar că există $\mathcal{O}(m)$ ferestre pe fiecare rînd, deci complexitatea este prea mare. Aici intervine descompunerea în valori distincte! Există doar $\mathcal{O}(\sqrt{m})$ lățimi distincte de ferestre pe fiecare rînd. De aceea putem itera doar prin aceste lungimi. Pentru o lungime fixată l, nu are sens să testăm decît cele mai apropiate ferestre din stînga, respectiv din dreapta coloanei curente c. De aici rezultă complexitatea totală $\mathcal{O}(nm\sqrt{m})$.

Implementarea nu este deloc simplă datorită următoarei situații pe care o enunțăm, dar nu o detaliem (ea este descrisă cu exemple în cea de-a doua sursă). Cînd aducem o fereastră din stînga coloanei c, decizia este simplă: trebuie să o deplasăm pînă cînd capătul ei drept atinge coloana c. Dar, cînd aducem o fereastră din dreapta coloanei c, avem libertatea să o deplasăm fie pînă cînd capătul ei stîng atinge coloana c, fie mai mult de atît. Depinde ce coloană $c' < c$ ne interesează

să „prindem” în recurență. Poate există o valoare c' destul de mică (costul deplasării pînă acolo este mare), dar unde $C_{r-1,c'}$ este foarte mic și justifică costul deplasării. Ia naștere un al doilea tabel RMQ construit nu peste valorile $C_{r-1,c}$, ci peste $C_{r-1,c} - c$.

Cîteva cuvinte despre *Arpa's trick*

Eu sînt mereu în căutare de noi structuri pentru RMQ. 😂 Îmi displace tabela rară deoarece folosește $\mathcal{O}(n \log n)$ memorie și evit să mă reped la ea. Iată o structură interesantă, numită *Arpa's trick*. Aparent, și alte națiii au șmenurile lor... CP Algorithms are o lecție foarte bună.

Algoritmul răspunde la interogări în ordine crescătoare după capătul drept. Deci putem întîi să sortăm interogările sau să le distribuim în liste după capătul drept. Apoi folosim o pădure de mulțimi disjuncte, pe care o instanțiem pe măsură ce baleiem poziția capătului drept.

Cînd ajungem la o poziție nouă p unde se află valoarea x, în primul rînd instanțiem o nouă rădăcină în DSF: părintele lui p este p. În plus, p devine părinte și pentru toate rădăcinile anterioare p' care aveau valori $x' > x$. Procedăm astfel deoarece, pentru orice interogări viitoare, valoarea x' nu va fi niciodată RMQ, deoarece x va face și ea parte din orice interogare care îl conține pe x'. Pentru implementarea acestei părți folosim o stivă ordonată.

La interogarea $\text{rmq}[p', p]$, unde p este poziția curentă, răspunsul este rădăcina arborelui lui p'. Într-adevăr, dorim cea mai mică valoare cu care a fost p' unit la orice moment.

Am rezolvat problema Puzzle-bila folosind *Arpa's trick*. Implementarea este dificilă și ineficientă, deoarece colectarea în avans și ordonarea interogărilor de care vom avea nevoie îngreunează codul (ca să folosesc un eufemism). Dar codul pentru *Arpa's trick* în sine este scurt și clar. El funcționează în $\mathcal{O}(\log^* n)$ amortizat cu memorie $\mathcal{O}(n)$.

```
struct arpa_s_trick {
  int val[MAX_N];
  int parent[MAX_N];
  int n;

  // O stivă cu pozițiile care încă nu au un element mai mic la dreapta.
  int st[MAX_N], ss;

  void reset() {
    ss = 0;
    n = 0;
  }

  void append(int x) {
    while (ss && (val[st[ss - 1]] >= x)) {
      parent[st[--ss]] = n;
    }
    st[ss++] = n;
    parent[n] = n;
    val[n++] = x;
```

```
  }

  int find(int p) {
    return (parent[p] == p)
      ? p
      : (parent[p] = find(parent[p]));
  }

  int rmq(int p) {
    return val[find(p)];
  }
};
```

Capitolul 9

Algoritmul lui Mo

Algoritmul lui Mo atinge tot complexitatea de $\mathcal{O}((q+n)\sqrt{n})$, ca și metoda descompunerii în radical, dar printr-o metodă destul de diferită. El duce adesea la cod mai simplu, dar necesită ca interogările să fie date în avans (*offline*).

9.1 Algoritmul lui Mo fără actualizări

Algoritmul lui Mo ordonează interogările și le procesează în această ordine:

1. După blocul capătului stîng.
2. La egalitate, după capătul drept.

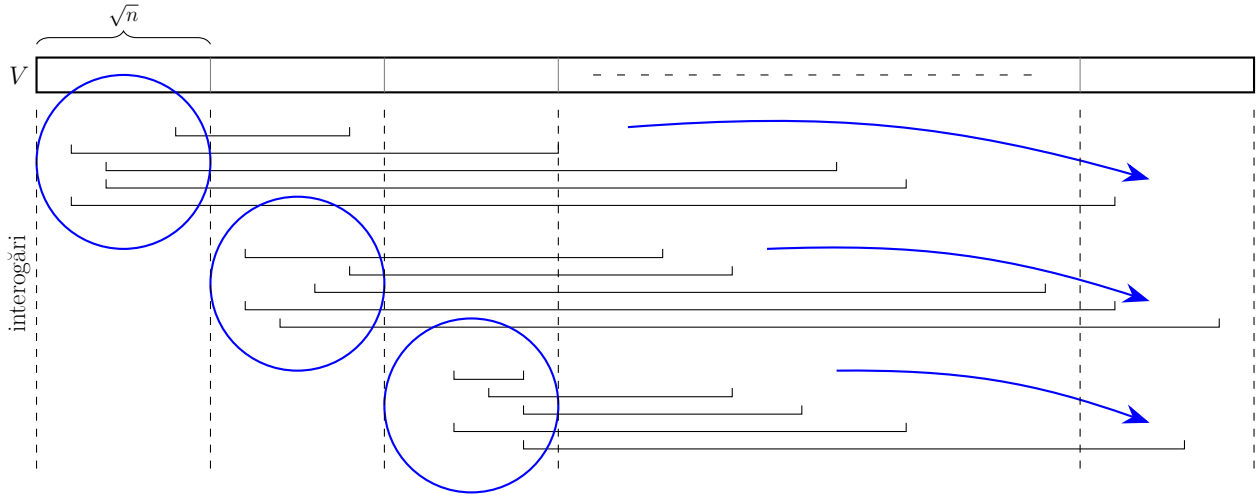


Figura 9.1: Ordonarea interogărilor în algoritmul lui Mo.

Algoritmul lui Mo ține minte informații despre interogarea curentă (răspunsul, dar posibil și alte informații). Pentru a trece la următoarea interogare, algoritmul:

1. Extinde intervalul curent cu cîte un pas, pînă cînd ajunge să includă și următoarea interogare.

2. Restrînge intervalul curent cu cîte un pas, pînă cînd ajunge să coincidă cu următoarea interogare.

Atenție! Pașii trebuie executați în această ordine. Altfel puteți ajunge la intervale negative, din care încercați să eliminați elemente care nu au fost adăugate, cu consecințe neprevăzute.

Vom relua problema D-query (SPOJ) și vom examina implementarea algoritmului lui Mo. Logica este considerabil mai simplă decît la implementarea cu AIB, în $\mathcal{O}(n \log n)$. Surprinzător, timpul de rulare este identic!

9.1.1 Analiză de complexitate

La fiecare interogare, trebuie să deplasăm capetele intervalului curent ca să se suprapună cu interogarea. Care este efortul total?

- Capătul stîng se poate deplasa cu cel mult $\sqrt{n}$ la fiecare interogare, plus n pași pentru toate trecerile între blocuri. Efortul total este de cel mult $q\sqrt{n} + n$ pași.
- Capătul drept se deplasează cu cel mult n pași la dreapta pentru toate interogările dintr-un bloc, apoi cu cel mult n pași înapoi la stînga la trecerea între blocuri. Există $\sqrt{n}$ blocuri, deci efortul total este de cel mult $2n\sqrt{n}$ pași.

Să spunem că includerea sau excluderea unei poziții durează $\mathcal{O}(f(n))$. Pentru problema Dquery efortul este $\mathcal{O}(1)$, dar pentru alte probleme costul poate diferi. Atunci complexitatea totală a algoritmului lui Mo este:

$$\mathcal{O}((q+n)\sqrt{n}f(n))$$

9.1.2 Optimizare de viteză

Nu ne costă aproape nimic (doar un `if` în plus la sortare) ca, în cadrul unui bloc, să ordonăm interogările astfel:

- crescător după capătul drept în blocurile impare;
- descrescător după capătul drept în blocurile impare.

Atunci nu mai plătim costul deplasării spre stînga „în gol” a capătului drept al intervalului curent. Iată o sursă cu această optimizare la problema Dquery. Timpii de rulare nu diferă, dar în general optimizarea poate ajuta.

De amorul artei, strict pentru problema Dquery, am normalizat valorile din vectorul dat. Pot fi cel mult 30.000 de valori distincte, ceea ce înseamnă că vectorul de frecvențe (pe `short`) poate ocupa doar 60 kB, în loc de 2 MB. Din nou, timpii nu diferă.

Aș preciza, tot pentru Dquery, că soluțiile țin foarte bine pasul cu soluția cu AIB (70-75 ms în loc de 60-65).

9.2 Algoritmul lui Mo cu actualizări

Algoritmul lui Mo se pretează și la probleme care au actualizări, cu o complexitate de $\mathcal{O}((q+n)n^{2/3})$. El este în continuare offline, avînd nevoie să citească și să sorteze interogările (și, posibil, să normalizeze valorile).

Esența este să introducem o nouă dimensiune pentru operații, în afară de capetele de interval l și r: timpul t, adică indicele operației (între 1 și q). Structura de date curentă stochează informații despre un interval $[L, R]$ la un moment de timp T, adică despre vectorul cu toate actualizările cu indici mai mici sau egali cu T. Pentru a răspunde la interogarea $[l, r]$ la momentul t, trebuie să:

1. Extindem intervalul curent pînă îl include pe $[l, r]$ (ca și pînă acum).
2. Restrîngem intervalul curent pînă devine egal cu $[l, r]$ (ca și pînă acum).
3. „Avansăm" timpul dacă $T < t$, procesînd actualizările din intervalul $(T, t]$.
4. „Dăm înapoi" timpul dacă $t < T$, inversînd actualizările din intervalul $[t, T)$.

Dacă actualizările efectuate sau inversate se întîmplă în intervalul curent $[l, r]$, actualizăm și structura de date, altfel nu.

9.2.1 Mărimea blocului, ordinea operațiilor, complexitate

Să alegem B blocuri de mărime S. Acum, să sortăm operațiile după blocul lui l, apoi după blocul lui r, apoi după t. Atunci:

Poziția lui l se schimbă cu $\mathcal{O}(S)$ la fiecare operație, plus $\mathcal{O}(n)$ la toate trecerile între blocuri, așadar cu $O(n + qS)$ în total.

Poziția lui r se schimbă cu $\mathcal{O}(S)$ la fiecare operație. În plus, pentru fiecare bloc al lui l (deci de B ori), r va face o trecere prin cele n poziții cu un efort total de $\mathcal{O}(qS + nB)$.

Pentru fiecare pereche de blocuri, t poate trece prin toate cele q operații, cu un efort total de $\mathcal{O}(qB^2)$.

Așadar, complexitatea algoritmului este $\mathcal{O}(qS + nB + qB^2)$. De aceea dorim ca $B^2 \approx S$ și alegem $B \approx n^{1/3}$ și $S \approx n^{2/3}$, pentru o complexitate totală de $\mathcal{O}((q+n)n^{2/3})$.

9.3 Probleme

9.3.1 Problema Powerful Array (Codeforces)

enunț • sursă

Problema este o aplicație directă a algoritmului lui Mo. Am copiat sursa de la D-query și am adaptat-o. Atenție doar la lucrul pe 64 de biți.

9.3.2 Problema Most Frequent Value (SPOJ)

enunț • sursă

Problema este doar puțin mai complicată decît precedentele. Odată ce ne vine ideea că am putea-o rezolva cu algoritmul lui Mo, întrebarea este: ce ne trebuie ca să menținem cea mai mare frecvență?

Răspunsul, desigur, este: menținem toate frecvențele. Dar nu este suficient. Cînd frecvența lui x crește, să spunem de la 7 la 8, atunci frecvența maximă:

- se păstrează dacă era deja 8 sau mai mare;
- crește la 8 dacă era 7 (notă: nu putea fi mai mică de 7, căci x avea frecvența 7).

Pînă aici, toate bune. Dar cînd frecvența lui x scade, să spunem de la 8 la 7? Atunci frecvența maximă:

- se păstrează dacă era mai mare decît 8 (un alt element y avea frecvență maximă);
- se păstrează dacă era 8 și mai există un alt element y cu frecvență 8;
- altfel scade la 7.

Cazul al doilea este critic. Cum aflăm dacă alt element are frecvența egală cu frecvența elementului care iese din interval? Răspunsul este: menținem numărul de elemente care au fiecare frecvență. Un fel de frecvență a frecvențelor, dacă vreți. 🙂 Cînd frecvența lui x scade de la 8 la 7, decrementăm numărul de elemente cu frecvență 8 și îl incrementăm pe cel cu frecvență 7. Dacă frecvența maximă era 8, dar acum nu mai există elemente cu frecvență 8, atunci ne aflăm în cazul al treilea.

9.3.3 Problema RangeMode (Infoarena Cup 2013)

enunț • sursă

Problema seamănă cu precedenta, dar nu cere frecvența maximă, ci elementul minim care atinge această valoare. Nu mai putem folosi fix aceeași abordare din cauza ștergerilor. Dacă există mai multe elemente cu frecvența maximă, și dacă cel mai mic dintre ele dispare, care este următorul minim?

Soluția este simplă: nu facem ștergeri! 😅 Glumesc, iar soluția nu este deloc evidentă. Ea împrumută sortarea interogărilor din algoritmul lui Mo. Atunci o interogare $[l, r]$ constă din:

1. „Partea stîngă”: porțiunea de la l pînă la sfîrșitul blocului curent, care poate varia cu $\mathcal{O}(\sqrt{n})$ elemente între interogări.
2. „Partea dreaptă”: porțiunea de la începutul blocului următor pînă la r, care poate doar să crească între interogări.

Desigur, mai există și cazul particular în care l și r se află în același bloc.

Încercăm să aflăm răspunsul la fiecare interogare aditiv, compunînd partea dreaptă cu partea stîngă. Structura de date pe care o proiectăm, S, menține pentru partea dreaptă un vector de

frecvențe simplu, în $\mathcal{O}(n)$ pentru toate interogările din bloc. Cînd trecem la blocul următor de interogări, golim acest vector. Așadar, efortul total pentru partea dreaptă este $\mathcal{O}(n\sqrt{n})$.

Ce facem cu partea stîngă? Dacă adăugăm elemente în S, va trebui să le ștergem la final. Am vrea să clonăm S pentru fiecare interogare, dar acea operație este scumpă. Iată o soluție struțo-cămilă, dar care funcționează:

1. După ce terminăm de adăugat elementele din partea dreaptă, salvăm din structura S doar răspunsul (elementul cu frecvența maximă).
2. Continuăm să adăugăm în S elementele din partea stîngă și să recalculăm răspunsul.
3. La final, notăm răspunsul. Acesta este răspunsul la interogare.
4. Eliminăm din S elementele din partea stîngă, fără să mai recalculăm răspunsul.
5. Restaurăm răspunsul salvat la pasul (1).

Soluția merge pentru că face ștergeri, dar evită componenta pe care nu știm s-o rezolvăm, a recalculării răspunsului la ștergere.

9.3.4 Problema Kiloforces (Concurs RoAlgo #8)

enunț • sursă

Problema are o parte teoretică rezonabil de simplă. Dacă rezolvăm problemele $(p_1, t_1), (p_2, t_2), \ldots, (p_k, t_k)$ în această ordine, atunci scorul se obține astfel:

- Adunăm punctajele $p_1 + p_2 + \cdots + p_k$.
- Scădem penalizarea t_1 de $k-1$ ori, deoarece ea afectează problemele de la 2 la k.
- Scădem penalizarea t_2 de $k-2$ ori.
- …
- Scădem penalizarea t_{k-1} o dată.

Rezultă că punctajele pozitive sînt aceleași, indiferent de ordinea de rezolvare. Pentru a minimiza penalizările, trebuie să rezolvăm problemele în ordinea crescătoare a penalizării. Deci am redus problema la a menține vectorul t ordonat și a-i calcula suma $t_1 \cdot (k-1) + t_2 \cdot (k-2) + \cdots + t_{k-1} \cdot 1$.

Restul ideii (cum implementăm asta) pare greu, dar noi știm de la bun început că vom folosi algoritmul lui Mo. Atunci implementarea devine rezonabilă. Trebuie să menținem pe intervalul curent răspunsul (și orice este necesar pentru aflarea răspunsului). Și trebuie să actualizăm răspunsul cînd adăugăm și cînd ștergem un element (o penalizare).

Ca să proiectăm structura de date, să vedem concret cum arată o adăugare. Logica pentru ștergere este similară. Să spunem că în intervalul curent avem următoarele penalizări, fiecare cu multiplicatorul său:

```
Penalizări:         2   5  10  13  17  21
Multiplicatori:    x5  x4  x3  x2  x1  x0
```

Și acum să spunem că extindem intervalul și adăugăm penalizarea 8. Deci dorim să ajungem în starea:

```
Penalizări:          2   5   8  10  13  17  21
Multiplicatori:    x6  x5  x4  x3  x2  x1  x0
```

Ce s-a schimbat? Am cîștigat la penalizări:

- $2 \times 1 + 5 \times 1$, așadar $2 + 5$, întrucît toate penalizările mai mici decît 8 au căpătat $+1$ la multiplicator.
- 8×4, noul element adăugat, care are multiplicatorul 4 întrucît există 4 penalizări mai mari decît 8.
- Penalizările 10, 13, 17, 21 nu și-au schimbat multiplicatorii.

Cu alte cuvinte, ne trebuie o structură care să ofere:

- Suma valorilor pe prefix.
- Numărul valorilor pe sufix (care se transformă ușor în prefix, căci cunoaștem mărimea intervalului curent, care coincide cu numărul de penalizări din structură).

Putem folosi un AIB de 0/1 pentru numărul valorilor și un alt AIB pentru sumă în care la poziția penalizării notăm fix valoarea penalizării. Pentru a îmbunătăți localitatea cache-ului, eu am ales să încapsulez cele două cantități într-un struct.

Subliniez că în sursă am încapsulat fiecare structură și i-am definit un scop clar. Asta a făcut implementarea rapidă (ceea ce pentru mine este o realizare 👴). Mi-au trebuit cam 30 de minute pentru codare și 10 pentru depanare.

9.3.5 Problema Machine Learning (Codeforces)

enunț • sursă

Problema cere să admitem 100.000 operații de două tipuri pe un vector cu 100.000 de elemente:

1. Pentru un interval $[l, r]$ tipărește mex-ul frecvențelor acelui interval. Așadar, tipărește valoarea minimă $f > 0$ pentru care nu există un element de frecvență f în intervalul $[l, r]$.
2. Modifică punctual vectorul, $a[i] = x$.

Implementarea mea este școlărească și medie ca eficiență. Iată ce am învățat din ea.

Pare mai natural să stocăm separat actualizările și interogările.

Ca să pot face *undo* la actualizări, cel mai simplu mi s-a părut să stochez și vechea valoare în fiecare operație de actualizare. Putem obține vechile valori în mod elementar, scriind actualizările într-un vector pe măsură ce le facem (avem nevoie de o copie a vectorului original, pe care o distrugem).

Ca să descriu momentul curent, am ales să mențin un indice în vectorul de actualizări, care pointează la prima operație încă neaplicată. Ca să „derulez” sistemul la momentul de timp t, avansez spre dreapta în vectorul de actualizări dacă timpul operației neaplicate este mai mic decît t, respectiv avansez spre stînga dacă timpul ultimei operații aplicate este mai mare decît t. Ca fapt divers, timpii nu pot fi egali, căci t este timpul unei interogări, care nu va apărea în vectorul de actualizări.

Mi-a fost mai simplu să adaug santinele la timpii 0 și $q+1$, ca să nu verific condiții suplimentare de ieșire din vector.

Strict referitor la problema Machine Learning: funcția mex pare greu de menținut incremental. Dacă o frecvență f încetează să mai aibă elemente, iar f este mai mic decît mex-ul curent, atunci f devine noul mex. Aceasta este partea simplă. Dar dacă apare un nou element de frecvența mex-ului, cum recalculăm noul mex? Riscăm să introducem un factor suplimentar la complexitate.

Soluția este de fapt brutală. Pot exista cel mult $\sqrt{n}$ frecvențe diferite (deoarece suma frecvențelor este n, așa cum am arătat în capitolul de descompunere în radical). De aceea, putem calcula funcția mex naiv pentru fiecare interogare cu o complexitate totală de $\mathcal{O}(q\sqrt{n})$, care nu este dominantă.

O ultimă observație specifică problemei: ne interesează doar frecvențele valorilor, nu ordinea lor relativă. De aceea, am ales o variantă mai simplă pentru codul de normalizare.

Partea III

Structuri de date avansate

În această parte vom studia structuri de date care, în anumite situații, sînt mai puternice decît arborii de intervale, arborii indexați binar sau structurile de date echilibrate din STL (`set` sau `map`). Aceste structuri sînt:

- *policy-based data structures* (PBDS);
- *skip lists*;
- treapuri.

Ne dorim structuri care stochează chei (ca un `set`) sau perechi cheie-valoare (ca un `map`) și care pot răspunde unor nevoi ca:

1. `kth_element(k)`: care este a k-a cheie din structură, în ordine crescătoare?
2. `order_of(x)`: a cîta cheie este/ar fi x, în ordine crescătoare?
3. Menținerea de chei implicite.
4. Suport pentru chei mari (neindexabile).
5. Suport pentru operații complexe, cum ar fi răsturnarea unei porțiuni din structură.

Operațiile 1 și 2 sînt cunoscute colectiv ca **statistici de ordine** (engl. *order statistics*). Pentru a le implementa, vom introduce conceptul de **structuri de date augmentate**. În esență, aceste structuri de date rețin, pentru fiecare nod sau pointer, și numărul de noduri acoperite sau subîntinse de acel nod sau pointer.

Cheile implicite se referă la posibilitatea de a menține numerele de ordine ale tuturor elementelor. Ne dorim o structură asemănătoare unui vector (atribuire / citire pe poziție) sau unui arbore de intervale (sume / maxime pe interval), dar care să admită și inserări și ștergeri. Putem vedea aceasta ca pe o structură în care fiecare element are o cheie egală chiar cu indicele elementului. Doar că nu putem stoca **explicit** acești indici, deoarece la o inserare sau ștergere trebuie să renumerotăm $\mathcal{O}(n)$ indici. De aceea vom stoca **implicit** cheile într-o manieră care să ne permită să le calculăm eficient la nevoie.

Iată un rezumat al posibilităților oferite de cîteva structuri deja studiate și de cele pe care urmează să le studiem.

operația	**AIB, AINT**	**set/map STL**	**PBDS**	**treap**	**skip list**
inserare	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
ștergere	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
căutarea unei chei	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
minim / maxim	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`kth_element(k)`	$\mathcal{O}(\log n)$	✗	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`order_of(x)`	$\mathcal{O}(\log n)$	✗	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
chei implicite	✗	✗	✗	✓	✓
chei mari	✗	✓	✓	✓	✓

Tabela 9.1: Structuri de date și posibilitățile lor.

În experiența mea, treapurile sînt cele mai rapide, seturile PBDS sînt doar cu puțin mai lente, iar *skip lists* cam de 1,5-2 ori mai lente. Pe de altă parte, consider că *skip lists* sînt mai ușor de implementat decît treapurile. *YMMV*.

Capitolul 10

Policy-based data structures

Acest capitol este doar un ciot: exemple de folosire și puțin context.

Policy-based data structures sînt o colecție de structuri de date din STL cu mai multă flexibilitate decît containerele standard `set` și `map` din STL. Declararea lor este un pic alambicată, tocmai datorită flexibilității de a ne alege componentele din care vrem să asamblăm structura.

10.1 Declararea unui set

Iată cum declarăm un set PBDS cu suport pentru statistici de ordine.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > my_set;

my_set s;
```

Să elucidăm această incantație magică. Ea capătă sens dacă ne amintim că și containerul `set` din STL este implementat de regulă ca arbore roșu-negru.

- `__gnu_pbds::tree`: Declarăm un tip de arbore echilibrat.
- `int`: Cheile sînt întregi.
- `__gnu_pbds::null_type`: Nu dorim perechi cheie-valoare (acela ar fi un `map`), deci specificăm că dorim valori nule.
- `std::less<int>`: Dorim comparatorul „<” pentru a ordona crescător setul.

- `__gnu_pbds::rb_tree_tag`: Dorim arbori roșu-negru, nu splay sau alte variante de arbori echilibrați.
- `__gnu_pbds::tree_order_statistics_node_update`: Dorim menținerea statisticilor de ordine la actualizarea nodurilor.

Precizez că mulți elevi preferă să condenseze acest cod incluzînd complet tot namespace-ul `__gnu_pbds`:

```
using namespace __gnu_pbds;
```

Totuși, acest obicei trebuie evitat. Dacă includem alandala toate definițiile din acel namespace, fie că ne trebuie, fie că nu, vom polua spațiul global de nume. În orice proiect ingineresc cu mai multă substanță decît un program de olimpiadă, aceasta este o sursă de conflicte și bătăi de cap. Puteți citi o lungă listă de contraargumente și de alternative mai sănătoase aici.

10.2 Folosirea

Seturile PBDS respectă specificațiile seturilor din STL, deci putem apela metode ca `insert`, `erase` etc. în plus, putem folosi metodele:

- `find_by_order(k)`: Returnează un `my_set::iterator` care pointează la al k-lea element din set sau `s.end()` dacă al k-lea element nu există.
- `order_of_key(x)`: Returnează numărul de ordine pe care îl are x în set sau pe care l-ar avea dacă l-aș insera acum.

Iată un exemplu complet.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <stdio.h>

typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > my_set;

int main() {
  my_set s;

  s.insert(2);
  s.insert(4);
  s.insert(6);
  s.insert(8);
```

```
  assert(s.find_by_order(-1) == s.end());
  printf("%d\n", *s.find_by_order(0));  // 2
  printf("%d\n", *s.find_by_order(1));  // 4
  printf("%d\n", *s.find_by_order(2));  // 6
  printf("%d\n", *s.find_by_order(3));  // 8
  assert(s.find_by_order(4) == s.end());

  printf("%lu\n", s.order_of_key(-1));  // 0
  printf("%lu\n", s.order_of_key(2));   // 0
  printf("%lu\n", s.order_of_key(4));   // 1
  printf("%lu\n", s.order_of_key(5));   // 2
  printf("%lu\n", s.order_of_key(6));   // 2
  printf("%lu\n", s.order_of_key(10));  // 4

  return 0;
}
```

10.3 Alte variante

Dacă dorim un `map`, trebuie doar să specificăm tipul de date al valorilor asociate. În acest caz, căutările, cum ar fi `find_by_order(k)`, returnează un pointer la o pereche (cheie, valoare), exact ca și `std::map`.

Iată un exemplu care mapează numere la caractere.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <stdio.h>

typedef __gnu_pbds::tree<
  int,
  char,
  std::less<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > my_map;

int main() {
  my_map s;

  s.insert({2, 'P'});
  s.insert({4, 'Q'});
  s.insert({6, 'R'});
  s.insert({8, 'S'});

  assert(s.find_by_order(-1) == s.end());
  printf("%c\n", s.find_by_order(0)->second);  // P
  printf("%c\n", s.find_by_order(1)->second);  // Q
```

```
  printf("%c\n", s.find_by_order(2)->second);  // R
  printf("%c\n", s.find_by_order(3)->second);  // S
  assert(s.find_by_order(4) == s.end());

  return 0;
}
```

Dacă dorim un multiset (un set care să admită valori repetate), la declarare trebuie să specificăm comparatorul `std::less_equal<int>`. Ștergerea dintr-un multiset necesită atenție suplimentară. Nu putem șterge valori, ci trebuie să aflăm poziția valorii, să obținem un iterator la acea poziție, apoi să ștergem iteratorul.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <stdio.h>

typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less_equal<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > multiset;

int main() {
  multiset s;

  s.insert(1);
  s.insert(2);
  s.insert(2);
  s.insert(2);
  s.insert(3);

  int rank = s.order_of_key(2);
  multiset::iterator it = s.find_by_order(rank);
  s.erase(it);

  for (int x: s) {
    printf("%d\n", x); // 1 2 2 3
  }

  return 0;
}
```

10.4 Statistici de ordine și augmentare

Cum funcționează statisticile de ordine în PBDS? Nu putem ști sigur fără să citim codul sursă, dar iată o presupune informată, pe care o vom implementa și noi însine în capitolele următoare.

Un set PBDS este implementat ca un arbore de căutare echilibrat (roșu-negru). Acesta garantează o adîncime $\mathcal{O}(\log n)$. Cînd căutăm cheia x, dorim nu doar să returnăm un iterator (un pointer) la ea, ci și să îi raportăm numărul de ordine. Voi cum ați rezolva acest deziderat?

Soluția este ca în fiecare nod u al arborelui de căutare să stocăm, pe lîngă informația obișnuită (cheia, pointeri la fii, la părinte etc.) și mărimea subarborelui lui u. Tocmai de aceea numim acest arbore o **structură de date augmentată**. Containerele stl standard (`set` și `map`) nu au fost proiectate cu suport pentru agumentare, de aceea ele nu pot calcula statistici de ordine.

Cum aflăm numărul de ordine al lui x? Împărțim problema recursiv în trei cazuri simple. Fie $key(u)$ cheia din nodul u și fie L mărimea subarborelui stîng al lui u.

1. Dacă $x = key(u)$, atunci numărul de ordine al lui x este L.
2. Dacă $x < key(u)$, atunci căutăm recursiv numărul de ordine al lui x în subarborele stîng.
3. Dacă $x > key(u)$, atunci căutăm recursiv numărul de ordine al lui x în subarborele drept, iar la rezultat adunăm $L + 1$ pentru subarborele stîng și pentru rădăcină.

La fel de simplu tratăm și problema inversă, a găsirii cheii cu numărul de ordine k.

1. Dacă $k = L$, atunci returnăm $key(u)$.
2. Dacă $k < L$, atunci căutăm recursiv cheia numărul k în subarborele stîng.
3. Dacă $k > L$, atunci căutăm recursiv cheia numărul $k - L - 1$ în subarborele drept.

10.5 Concluzii

Ca programatori de competiție, este bine să cunoașteți aceste structuri de date, care ocazional vă conduc la o implementare ușoară. Ele vă vor ajuta și în postura inversă, de compunători de probleme, prin care vă doresc să treceți cel puțin o dată în viață. Veți inventa uneori probleme frumoase, care necesită inteligență și tenacitate pentru o implementare cu structuri de date proprii, dar care se rezolvă în 10 linii cu PBDS. Cu tot regretul, acele probleme nu sînt practice pentru un concurs.

În orice caz, vă recomand să nu vă bazați doar pe PBDS, să nu fiți doar utilizatori de STL!

10.6 Probleme

Nu am pregătit probleme specifice pentru PBDS, dar ele sînt presărate prin restul cărții: Give Away, Medwalk, Tree and Queries.

Capitolul 11

Skip lists

Skip lists sînt o structură de date originală, atipică. Pentru cei ca noi, educați exclusiv cu structuri de date deterministe, prima întîlnire cu *skip lists* poate fi fascinantă, căci ele sînt o structură de date probabilistică.

Vom studia cîteva noțiuni matematice necesare, apoi vom analiza structura de date și codul. *Skip lists* nu sînt la fel de eficiente ca *policy based data structures* sau ca treapurile, dar consider că implementarea lor este relativ facilă.

11.1 Noțiuni de probabilitate

Această secțiune ar putea ușor să ocupe cîteva ore, dar ne vom concentra doar pe noțiunile strict necesare.

O **variabilă aleatorie** este o cantitate care depinde de un fenomen aleatoriu. Exemple din viața reală: aruncarea unei monede; extragerea de cărți dintr-un pachet de cărți de joc; aruncarea unui zar.

Putem clasifica toate rezultatele posibile ale unui fenomen aleatoriu și putem atribui probabilități (valori reale între 0 și 1) acestor rezultate. Acest tabel de probabilități se numește **distribuția** variabilei aleatorii. Suma probabilităților din tabel trebuie să fie 1.

Există multe tipuri de distribuții, dintre care menționăm 3.

Distribuția uniformă. Dacă extragem o carte la întîmplare dintr-un pachet (cu cărțile numerotate de la 1 la 52), probabilitatea de a vedea orice carte este 1/52. În acest scenariu, variabila aleatorie X este numărul cărții extrase, iar distribuția este $P(X = k) = 1/52$ pentru $1 \leq k \leq 52$.

Distribuția binomială. Să calculăm probabilitatea ca, din 10 aruncări cu zarul, să obținem fix de 3 ori valoarea 6. Putem alege oricum cele 3 aruncări favorabile din totalul de 10. Odată fixate pozițiile, avem probabilitatea de 1/6 ca fiecare dintre acele aruncări să producă un 6 și probabilitatea de 5/6 ca fiecare dintre celelalte aruncări să nu producă un 6. Rezultă formula:

$$P(X = 3) = C_{10}^3 \left(\frac{1}{6}\right)^3 \left(\frac{5}{6}\right)^7$$

Distribuția se numește binomială pentru că, dacă generalizăm formula la n aruncări și $0 \leq k \leq n$ aruncări favorabile, și dacă presupunem că șansa de succes la o aruncare este p, formula generală urmează exact binomul lui Newton pentru $[p + (1 - p)]^n$:

$$P(X = k) = C_n^k p^k (1 - p)^{n-k}$$

Distribuția geometrică, de interes pentru acest capitol. Ne interesează să calculăm numărul de aruncări cu zarul necesare pînă cînd obținem un 6. Dacă notăm cu X această variabilă aleatorie, atunci:

- Cu probabilitatea de $\frac{1}{6}$, vom obține un 6 din prima aruncare.
- Cu probabilitatea de $\frac{5}{6} \cdot \frac{1}{6}$, vom obține un 6 din a doua aruncare.
- Cu probabilitatea de $\frac{5}{6} \cdot \frac{5}{6} \cdot \frac{1}{6}$, vom obține un 6 din a treia aruncare.

Generalizînd,

$$P(X = k) = \left(\frac{5}{6}\right)^{k-1} \cdot \frac{1}{6}$$

Distribuția geometrică este infinită. Pentru orice k, putem avea ghinionul să nu obținem un 6 din primele k aruncări. Ea se numește geometrică deoarece probabilitățile diverselor valori formează o progresie geometrică.

Valoarea așteptată a unei variabile aleatorii, notată cu $E[X]$, este media valorilor posibile ponderată cu probabilitățile acestora.

Pentru **distribuția uniformă**, valoarea așteptată este pur și simplu media valorilor. Pentru un pachet cu 52 de cărți, valoarea așteptată a unei cărți este 26,5.

Pentru **distribuția binomială** cu n repetiții și probabilitatea p de succes, valoarea așteptată este np. Această formulă nu este ușor deductibilă din formula combinatorică, dar poate fi demonstrată mult mai ușor. Cele n experimente sînt independente (o aruncare unui zar nu este influențată de precedentele). De aceea, dintre cele n experimente ne așteptăm ca o fracție de p să aibă succes (să aruncăm un 6).

Pentru **distribuția geometrică** cu probabilitatea p de succes la fiecare experiment, numărul așteptat de experimente pînă la primul succes este $1/p$. Pentru o monedă, valoarea așteptată este 2, pentru un zar este 6.

Putem demonstra această afirmație și algebric. Pornim de la definiția

$$E[X] = \sum_{k \geq 1} k \cdot (1-p)^{k-1} \cdot p$$

și formăm un triunghi de termeni:

$$\begin{aligned}
E[X] = p & \\
&+ (1-p) \cdot p && + (1-p) \cdot p && \\
&+ (1-p)^2 \cdot p && + (1-p)^2 \cdot p && + (1-p)^2 \cdot p && \\
&+ (1-p)^3 \cdot p && + (1-p)^3 \cdot p && + (1-p)^3 \cdot p && + (1-p)^3 \cdot p \\
&+ \dots &&&&&&
\end{aligned}$$

Pe fiecare coloană calculăm suma unei progresii geometrice, apoi calculăm suma sumelor. Este un proces laborios. Iată însă o metodă mult mai elegantă. Tocmai pentru că aruncările cu zarul nu au memorie, iau naștere două cazuri:

- Cu probabilitatea p experimentul reușește din prima încercare.
- Cu probabilitatea $1-p$ am irosit prima încercare și reluăm experimentul **cu aceeași valoare așteptată**.

Rezultă recurența:

$$E[X] = p \cdot 1 + (1-p) \cdot (1 + E[x])$$

Pe care o rezolvăm trivial și obținem

$$E[X] = 1/p$$

Acest ultim rezultat ne interesează pentru *skip lists*, pentru că analiza lor de complexitate se reduce exact la acest rezultat.

11.2 Structura de date

Vom defini o structură de date care seamănă cu o listă simplu înlănțuită, dar care are diverse niveluri de scurtături, pointeri care avansează mai multe elemente deodată.

Să considerăm întîi cazul unei colecții statice de numere (fără modificări) în care dorim doar să căutăm valori. Atunci am putea precalcula următoarea structură:

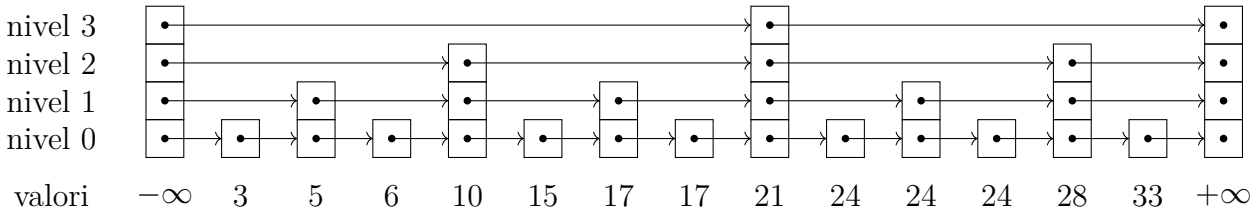


Figura 11.1: O listă pe niveluri, utilă pentru date statice.

Observăm că pe nivelul 0, toate elementele sînt înlănțuite. Pe nivelul 1, pointerii avansează din două în două elemente. În general, pe nivelul k, pointerii avansează cîte 2^k elemente, posibil cu excepția ultimului pointer. Pentru simplificarea operațiilor, am inclus santinele la începutul și la sfîrșitul listelor, corespunzătoare valorilor $\pm\infty$. Structura are $\lceil \log(n-1) \rceil$ niveluri, astfel încît pe ultimul nivel să existe un singur pointer dintr-un nod diferit de santinele.

Într-o astfel de structură, căutările funcționează garantat (deterministic) în timp $\mathcal{O}(\log n)$. Pornind de pe cel mai înalt nivel, din santinela stîngă:

- Urmăm pointerul dacă valoarea căutată este cel puțin egală cu valoarea din destinația pointerului (21).
- Dacă valoarea căutată este mai mică decît 21, nu urmăm pointerul.
- În orice situație, coborîm un nivel și repetăm procedura.

În acest algoritm, ne folosim de observația că între orice două turnuri de pointeri de înălțime h există exact un pointer de înălțime $h-1$.

Limitarea, desigur, este că dorim ca structura să admită și inserări și ștergeri. Dacă toate inserările au loc, să zicem între valorile 10 și 15, aceasta poate denatura structura de pointeri și ne poate face să petrecem foarte mult timp pe un nivel anume, iar timpul de căutare se degradează la $\mathcal{O}(n)$.

De aceea, renunțăm la controlul determinist asupra structurii de pointeri și adoptăm o abordare probabilistică. La inserarea unui element, îi vom genera un turn de pointeri cu înălțime între 1 și $\log n$, aleasă dintr-o distribuție geometrică de probabilitate cu $p = 1/2$. Cu alte cuvinte, cu probabilitatea 1/2 turnul va avea un singur pointer, pe nivelul 0. Cu probabilitatea 1/4 turnul va avea doi pointeri etc. Pentru siguranță, impunem o limită superioară de $\lceil \log n \rceil$ niveluri. Va rezulta o structură mai puțin uniformă, dar la fel de puternică din punct de vedere asimptotic. Figura 11.2 arată un exemplu pentru același set de date.

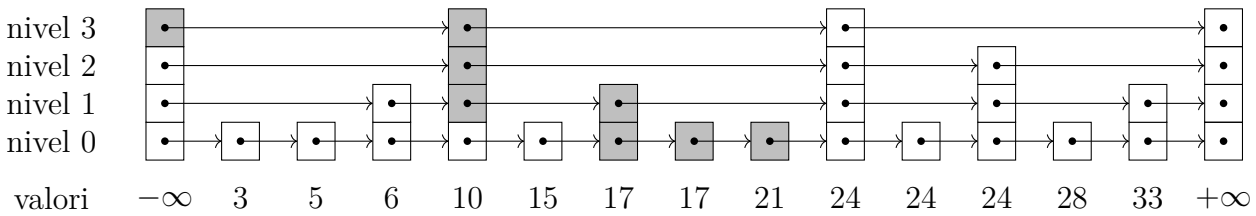


Figura 11.2: O listă pe niveluri probabilistică. Pointerii colorați cu gri sînt cei consultați pe parcursul căutării valorii 21.

Algoritmul de căutare funcționează ca mai înainte, doar că între două turnuri de pointeri de înălțime h pot exista zero sau mai multe turnuri de înălțime $h-1$. De exemplu, pentru căutarea valorii 21, algoritmul:

- Pornește de pe nivelul 3, valoarea $-\infty$.
- Avansează la valoarea 10.
- Coboară pe nivelul 2, valoarea 10, întrucît pointerul de pe nivelul 3 ar duce la valoarea 24, prea mare.
- Coboară pe nivelul 1, valoarea 10.
- Avansează la valoarea 17.
- Coboară pe nivelul 0, valoarea 17.
- Avansează la valoarea 17.
- Avansează la valoarea 21.

11.3 Analiza de complexitate

Pe măsură ce inserăm și ștergem valori, structura pointerilor se va modifica. Dar numărul așteptat de pointeri de înălțime $h-1$ dintre doi pointeri de înălțime h va fi 1. Raționamentul pornește de la valoarea așteptată a distribuției geometrice. Presupunînd că ne interesează doar turnurile de înălțimi $h-1$ și h, cîte turnuri ne așteptăm să vedem pînă cînd întîlnim următorul turn de înălțime h? Răspunsul este $1/p = 2$, cu alte cuvinte ne așteptăm să existe, în medie, un turn de înălțime $h-1$.

Aceasta înseamnă că ne așteptăm ca, la fiecare pointer traversat, algoritmul să parcurgă circa jumătate din distanța rămasă pînă la valoarea căutată. În plus, algoritmul va coborî $\log n$ niveluri pe verticală. Rezultă o complexitate de $\mathcal{O}(\log n)$.

11.4 Augmentarea pentru statistici de ordine

Pentru a putea răspunde la statistici de ordine, este suficient ca fiecare pointer să își rețină și distanța, aducă numărul de elemente pe care le sare. Trebuie să actualizăm aceste valori la inserarea și la ștergerea de elemente.

La inserarea unui element, dacă îi generăm un turn de h pointeri, acei pointeri se vor interpune între pointerii anterior și următor de pe același nivel. Așadar, distanțele pointerilor nou creați vor prelua o parte din distanțele pointerilor pe care îi întrerup. În plus, pointerii care trec pe deasupra turnului creat capătă +1 la distanță. (TODO: figură).

La ștergerea unui element se întîmplă fenomenul opus: pointerii a căror destinație era elementul șters cîștigă la distanțele lor distanțele pointerilor care porneau din elementul șters. Pointerii care treceau pe deasupra elementului șters pierd 1 din distanță.

11.2

11.5 Implementarea

Anexa include o implementare de referință și exemple de folosire pentru *skip lists* augmentate pentru statistici de ordine. Am inclus doar varianta pe care o consider canonică (cu turnuri de pointeri prealocate ca `int[]`). Există multe decizii de implementare, pe care le discut mai jos. Puteți regăsi toate implementările într-un repo pe GitHub.

11.5.1 Alocarea turnului de pointeri

Avînd în vedere că fiecare nod are un turn de pointeri de înălțime variabilă și calculată în momentul inserării, este tentant să folosim un `std::vector`. Dar acele implementări tind să fie cele mai lente, de 2-3 ori mai lente decît treapurile și seturile PBDS, după cum o arată secțiunea de *benchmarks*.

O altă abordare este să prealocăm vectori (*arrays*) de înălțime fixă și egală cu $\log n$. Codul va fi mai rapid, dar necesarul de memorie crește. Pentru o structură cu 500 000 de elemente, toate nodurile au cîte 21 de pointeri și 21 de distanțe, precum și alte informații, adică 176 de octeți per nod. Totalul necesar este de 88 MB.

O a treia implementare alocă turnurile de pointeri într-un buffer comun. Un turn nou își ia din acel buffer doar spațiul strict necesar pentru înălțimea aleasă. Aceasta reduce memoria la 22 MB. Este o diferență semnificativă. Un dezavantaj al acestei implementări este fragmentarea memoriei, ceea ce ne pune probleme dacă dorim să recuperăm memoria eliberată prin ștergeri. Dar, dacă bufferul este suficient de mare, putem să omitem recuperarea.

Am inclus în anexă implementarea cu *arrays*, pentru concizie și pentru că este cea mai rapidă. Dacă memoria este o problemă, implementarea cu buffer global diferă doar cu cîteva linii la alocarea unui nod.

11.5.2 Optimizarea generatorului de numere aleatorii

Cum generăm înălțimea turnului? Nu putem folosi `rand()` (sau generatorul vostru favorit de numere aleatorii), căci `rand()` selectează uniform, or noi dorim o distribuție geometrică. Dar vom apela `rand() % 2` în mod repetat pînă cînd experimentul reușește. Această metodă, deși naivă, este suficient de bună, căci știm că numărul de apeluri mediu la `rand()` va fi 2.

```
int get_height() {
  int h = 1;
  while ((rand() & 1) && (h < MAX_LEVELS)) {
    h++;
  }
  return h;
}
```

Pentru concizia codului, putem și să apelăm `rand()` o singură dată și să numărăm biții de 0 cei mai puțin semnificativi. Acest artificiu funcționează pentru că răspunsul va fi 0 cu probabilitatea

1/2, 1 cu probabilitatea 1/4 etc. Dar trebuie să setăm explicit unul dintre biții returnați de `rand()` ca să ne asigurăm că răspunsul nu va depăși înălțimea maximă.

```
int get_height() {
  int r = rand() | (1 << (MAX_LEVELS - 2));
  return 1 + __builtin_ctz(r);
}
```

Am mai încercat și să apelez `rand()` o singură dată, apoi să folosesc biții returnați pentru următoarele 30 de experimente. Din nou, economia este neglijabilă, tocmai pentru că numărul de apeluri la `rand()` este $\mathcal{O}(n)$, pe cînd complexitatea totală a operațiilor pe *skip lists* este $\mathcal{O}(n \log n)$.

11.5.3 Alegerea probabilității

Din curiozitate, am făcut experimente și cu alte probabilități decît 0,5, respectiv cu 0,375 și cu 0,25. Cînd p scade, înălțimea turnurilor scade, dar numărul de pași pe orizontală crește. Invers, cînd p este prea mare vom petrece prea mult timp mergînd pe verticală. Viteza de execuție scade pe măsură ce ne abatem de la $p = 0,5$.

11.6 Benchmarks

Tabelul 11.1 enumeră timpii obținuți cu diversele variante de *skip lists* augmentate cu statistici de ordine. Am testat toate combinațiile de alocare a turnurilor și de generare a numerelor aleatorii. Am rulat 100 de faze pe fiecare structură, iar o fază operează pe $n = 500\,000$ de numere și face 6 operații pentru fiecare număr (inserări, ștergeri, căutări și statistici de ordine). Așadar, numărul total este de 300 000 000 de operații.

	array fix de pointeri	**array variabil**	**vector**
`rand()` simplu	157 s	186 s	265 s
`rand()` cu `__builtin_ctz`	157 s	187 s	262 s
`rand()` pe biți, $p = 0,5$	157 s	186 s	264 s
`rand()` pe biți, $p = 0,375$	175 s	203 s	299 s
`rand()` pe biți, $p = 0,25$	202 s	228 s	360 s

Tabela 11.1: Timpii obținuți pentru 300 de milioane de operații pe *skip lists* augmentate. Pentru comparație, un `set` PBDS obține 117 s, iar un treap 90 s.

Dar putem trage cîteva concluzii.

1. *Skip lists* sînt mai ineficiente decît structurile din STL și decît treaps.
2. Implementările cu `std::vector` sînt semnificativ mai lente.
3. Abaterea de la $p = 0,5$ dăunează vitezei.

Gînduri de final? *Skip lists* pot fi un adaos util la arsenalul vostru de structuri de date. Ele sînt ceva mai lente decît treapurile și nu pot rezolva un spectru de probleme atît de larg. În schimb,

sînt mai ușor de implementat decît treapurile – deși opinia voastră poate diferi.

Capitolul 12

Treapuri

Treapurile sînt, ca și *skip lists*, o structură de date probabilistică. Treapurile sînt arbori binari de căutare care adaugă o componentă aleatorie pentru echilibrare.

Un treap (engl. *tree + heap*) este un arbore binar în care fiecare nod conține două valori numerice: o **cheie** și o **prioritate**. După chei, arborele este un arbore binar de căutare: fiecare nod are cheia mai mare sau egală cu toate cheile din subarborele stîng și mai mică sau egală cu toate cheile din subarborele drept. După priorități, arborele este un max-heap: fiecare nod are prioritatea mai mare sau egală cu toate prioritățile din subarbore. În particular, rădăcina are prioritate maximă.

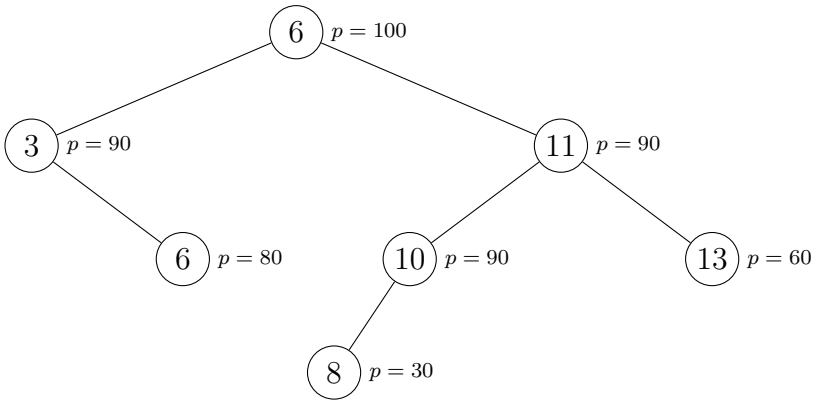


Figura 12.1: Un exemplu de treap. Cheile sînt scrise în noduri, iar prioritățile în afara nodurilor.

Treapurile se aseamănă cu *arborii cartezieni* în privința proprietății de heap.

O proprietate definitorie este că treapurile primesc la intrare doar cheile. Prioritățile sînt atribuite aleatoriu, intern, în momentul inserării nodului, sînt imutabile pe durata vieții nodului și servesc doar pentru echilibrarea probabilistică. Ca să înțelegem de ce echilibrarea funcționează, să considerăm treapul oricînd pe parcursul vieții sale, după un număr arbitrar de inserări și ștergeri. Prioritatea maximă poate apărea pe oricare dintre chei, dar valoarea așteptată a poziției priorității maxime este pe cheia mediană. Așadar, ne așteptăm ca rădăcina să împartă celelalte $n - 1$ noduri în două grupe de mărimi relativ egale. Același argument se aplică recursiv tuturor

subarborilor. Desigur, ocazional se poate întîmpla ca prioritatea maximă să apară pe nodul cu cheie maximă dintr-un subarbore, ceea ce înseamnă că singura organizare corectă este ca acel nod să nu aibă fiu drept, iar toate celelalte noduri să cadă în subarborele stîng al nodului. Dar acest eveniment va fi excepția, nu regula. Ne așteptăm ca înălțimea treapului să fie logaritmică.

Un mod echivalent (și ușor testabil) de a ne gîndi la echilibrare este: care este lungimea medie a unui drum pînă la o frunză? Să pornim cu un arbore de $n = 100\,000$ de noduri. Generăm un număr aleatoriu k între 1 și n cu semnificația că prioritatea maximă apare pe cea de-a k-a cheie. Deci reducem problema la un subarbore cu $k - 1$ noduri. Dacă sîntem norocoși și $k \ll n$, am redus problema la un subarbore foarte mic. Dacă sîntem ghinioniști și $k \approx n$, reducem problema cu foarte puțin. Dar nu putem fi mereu ghinioniști. Numărul de iterații pînă cînd ajungem la $n = 0$ va fi logaritmic.

12.1 Reprezentări

Articolul de pe CP Algorithms folosește pointeri și alocare dinamică:

```
struct treap {
  int key, pri;
  treap *l, *r;

  treap(int key) {
    this->key = key;
    pri = rand();
    l = r = NULL;
  }

  ...
};
```

Ca întotdeauna, vă recomand să prealocați static un vector de elemente și să înlocuiți pointerii cu indici în acest vector. Viteza crește cam cu 50%.

```
struct node {
  int key, pri;
  int l, r;
};

struct treap {
  node v[N + 1];
  int n, root;

  void init() {
    n = 1; // Marchează nodul 0 ca folosit, întrucît 0 înseamnă NULL.
    root = 0;
  }
```

```
  ...
}
```

Cîmpul `root` pointează la rădăcină și poate fi 0 dacă treapul este vid.

Anexa conține implementări complete pentru aceste reprezentări. Reprezentarea cu alocare statică este la rîndul ei diferențiată în funcție de metodele folosite pentru operații, despre care vom vorbi îndată.

12.2 Căutarea

Căutarea este 100% identică cu căutarea într-un arbore binar de căutare. Treapul inovează doar partea de echilibrare.

```
int search(int key) {
  int t = root;
  while (t && (key != v[t].key)) {
    t = (key < v[t].key) ? v[t].l : v[t].r;
  }
  return t;
}

bool contains(int key) {
  return search(key) != 0;
}
```

12.3 Inserarea

Cînd inserăm un nod, dorim să păstrăm proprietatea de arbore binar de căutare. Așadar, vom restructura cîțiva pointeri ca să facem loc pentru noul nod. Dar aceasta poate încălca proprietatea de max-heap a priorităților.

Pentru menținerea heapului există două metode, iar de aici implementarea diverge. Menținerea prin rotații este conceptual mai simplă. O metodă mult mai puternică și ceva mai rapidă este menținerea prin spargeri și unificări (engl. *split* și *merge*), folosite respectiv pentru inserări și ștergeri.

12.3.1 Menținerea heapului prin rotații

Istoric vorbind, toți arborii echilibrați au la bază conceptul de rotații, pe care le folosesc pentru echilibrare. Cînd un fiu este mult mai adînc decît celălalt, inversăm niște pointeri astfel încît fiul să urce în locul rădăcinii, păstrînd ordonarea de arbore binar de căutare. Ca lectură suplimentară, puteți citi despre rotații în:

- AVL (inventați în 1962!);
- Splay Tree (optimizați pentru accesări frecvente);
- Red-Black Tree (foarte folosiți în practică, dar greu de implementat).

Pentru treapuri, rotațiile servesc doar la refacerea heapului. Facem inserarea ca în orice arbore binar de căutare: coborîm din rădăcină urmînd pointerul stîng sau drept după cum dictează cheia. Noul nod devine fiu al uneia dintre frunze. Apoi, vizităm frunza și toți strămoșii ei și verificăm respectarea priorităților. De exemplu, dacă fiul stîng f are prioritate mai mare decît a tatălui t, atunci rotim subarborele lui t spre dreapta.

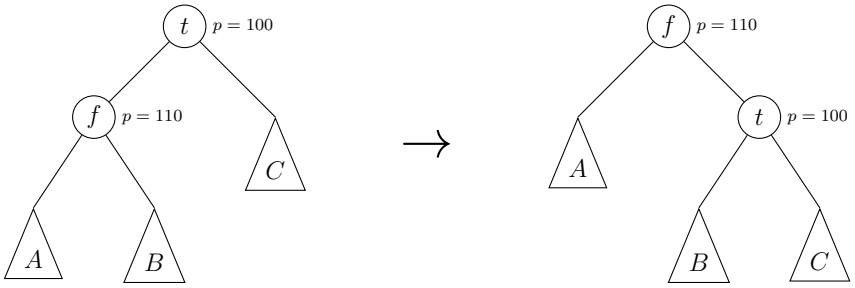


Figura 12.2: Rotația spre dreapta a unui subarbore. Rotația spre stînga funcționează simetric.

Observăm că, atît înainte cît și după rotație, ordinea cheilor este $A \leq f \leq B \leq t \leq C$. Așadar, rotația menține proprietatea de arbore binar de căutare.

Implementarea recursivă a acestui cod (în varianta cu noduri alocate static) este:

```
// Inserează cheia key în subarborele lui t și scrie în t noua rădăcină.
void insert(int& t, int key) {
  if (!t) {
    // Am ajuns la o frunză. Alocă un nou nod și returnează indicele lui.
    v[n] = { .key = key, .pri = rand(), .l = 0, .r = 0 };
    t = n++;
  } else if (key < v[t].key) {
    // Inserează în stînga. Contăm pe fiul stîng să se reorganizeze intern.
    // Rotește spre dreapta dacă t și fiul stîng au prioritățile inversate.
    insert(v[t].l, key);
    if (v[v[t].l].pri > v[t].pri) {
      t = rotate_right(t);
    }
  } else {
    insert(v[t].r, key);
    if (v[v[t].r].pri > v[t].pri) {
      t = rotate_left(t);
    }
  }
}
```

12.3.2 Menținerea heapului prin *split*

Cu această metodă, inserarea funcționează diferit. Fie u nodul de inserat, cu cheia k_u și prioritatea p_u. Pornim din rădăcină și coborîm tot mai jos, urmînd pointerul stîng sau drept după cum dictează cheia k_u, dar ne oprim atunci cînd întîlnim un nod v de prioritate $p_v < p_u$ (ceea ce înseamnă că toate nodurile din subarborele lui v au priorități mai mici decît p_u). Acum procedăm astfel:

- u îl dezlocuiește pe v din treap, luîndu-i locul.
- Spargem (*split*) subarborele lui v în două treapuri, unul cu chei $k \leq k_u$ și unul cu chei $k > k_u$.
- Aceste două treapuri devin fiul stîng și respectiv fiul drept al lui u.

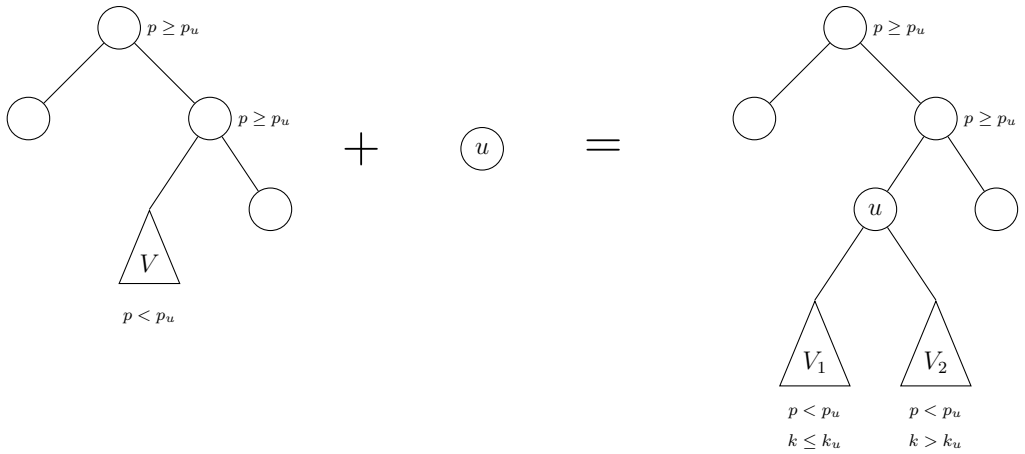


Figura 12.3: Inserarea unui nod prin *split*.

Putem proiecta recursiv operația `split(int t, int k, int &l, int &r)`. Ea primește la intrare un treap și o cheie k și trebuie să returneze rădăcinile celor două treapuri rezultate (care pot fi și nule). Am denumit aceste valori de returnat l și r (de la *left* și *right*). Începem prin a examina rădăcina t.

1. Dacă $k_t \leq k$, atunci t va face parte din l, unde va fi tot rădăcină. Cu alte cuvinte, l este t. De asemenea, fiul stîng al lui t rămîne nealterat. În continuare, rămîne să spargem fiul drept al lui t, tot după cheia k. Treapul cu valori mici devine fiul drept al lui t. Treapul cu valori mari devine r.
2. Cazul $k_t > k$ este simetric.

```
void split(int t, int key, int& l, int& r) {
  if (!t) {
    l = r = 0;
  } else if (v[t].key <= key) {
    split(v[t].r, key, v[t].r, r);
    l = t;
  } else {
    split(v[t].l, key, l, v[t].l);
```

```
    r = t;
  }
}

void insert(int& t, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, v[elem].key, v[elem].l, v[elem].r);
    t = elem;
  } else {
    insert(v[t].key <= v[elem].key ? v[t].r : v[t].l, elem);
  }
}

void insert(int key) {
  v[n] = { .key = key, .pri = rand(), .l = 0, .r = 0 };
  insert(root, n++);
}
```

12.4 Ștergerea

Și la ștergere detaliile operației diferă în funcție de metoda aleasă. Nu mai includem și codul, ca să nu duplicăm porțiuni mari din anexă, care conține codul complet.

12.4.1 Menținerea heapului prin rotații

Dacă cheia k pe care dorim să o ștergem apare într-o frunză (sau dacă nu apare deloc în treap), atunci ștergerea este banală. Dar dacă cheia apare într-un nod intern u? Atunci nu putem să ștergem pur și simplu nodul, căci se pune problema cum unificăm cei doi fii ai lui u, acum rămași orfani.

În schimb, putem să reducem problema la una cunoscută. Facem o rotație în nodul u. Direcția rotației o alegem astfel încît locul lui u să fie luat de fiul cu prioritatea mai mare. Aceasta, cum știm păstrează ordonarea cheilor. În schimb, nodul u a coborît un nivel! Repetăm procedeul pînă cînd u devine frunză, apoi îl ștergem.

12.4.2 Menținerea heapului prin *merge*

De fapt, problema enunțată în secțiunea anterioară este o operație standard, perechea lui *split*, numită *merge*. Logica este similară cu *split*, dar în sens invers.

Să proiectăm recursiv operația `merge(int& t, l, r)`. Ea primește la intrare două treapuri (l și r) și returnează în t rădăcina treapului unificat. Iau naștere două cazuri:

1. Dacă prioritatea rădăcinii lui l este mai mare, atunci ea va deveni rădăcina treapului unificat.

De asemenea, fiul stîng al lui l rămîne nealterat. Rămîne să unificăm recursiv fiul drept al lui l cu r, iar rezultatul să-l „agățăm” ca fiu drept al lui l.

2. Cazul în care prioritatea rădăcinii lui r este mai mare este simetric.

Așadar, pentru ștergerea nodului u trebuie doar să unificăm cei doi fii și să „agățăm” rezultatul de părintele lui u.

12.5 Augmentarea pentru statistici de ordine

După discuțiile purtate în capitolele despre seturi PBDS și despre *skip lists*, cred că vă este ușor să deduceți cum augmentăm treapurile pentru statistici de ordine. Este suficient să păstrăm pe fiecare nod o valoare suplimentară: numărul de noduri din subarbore (inclusiv nodul însuși).

Acest număr trebuie recalculat oricînd reorganizăm arborele. Pentru funcții implementare recursiv, putem scrie o metodă simplă, `update_cnt`, pe care să o apelăm la revenirea din recursivitate. De exemplu, operația *split* pentru treapuri augmentate devine:

```
void update_cnt(int t) {
  if (t) {
    v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
  }
}

void split(int t, int key, int& l, int& r) {
  if (!t) {
    l = r = 0;
  } else if (v[t].key <= key) {
    split(v[t].r, key, v[t].r, r);
    l = t;
    update_cnt(t);
  } else {
    split(v[t].l, key, l, v[t].l);
    r = t;
    update_cnt(t);
  }
}
```

Cu această informație suplimentară, codul funcției `order_of()` decurge ușor. Trebuie să însumăm nodurile cu chei mai mici decît cheia dată:

```
int order_of(int key) {
  int result = 0;
  int t = root;
  while (t) {
    if (key > v[t].key) {
      result += 1 + v[v[t].l].cnt;
      t = v[t].r;
```

```
    } else {
      t = v[t].l;
    }
  }

  return result;
}
```

Codul funcției `kth_element()` este de asemenea elementar. Trebuie să nagivăm prin arbore lăsînd în stînga cele k noduri cu cheile cele mai mici:

```
int kth_element(int k) {
  int t = root;
  while (k != v[v[t].l].cnt) {
    if (k < v[v[t].l].cnt) {
      t = v[t].l;
    } else {
      k = k - 1 - v[v[t].l].cnt;
      t = v[t].r;
    }
  }
  return v[t].key;
}
```

12.6 Treapuri implicite

Pînă acum, toate discuțiile despre *skip lists* și despre treapuri ne-au făcut să creștem ca algoritmiști și ca ingineri software. Dar tot ce oferă aceste structuri putem obține și cu un set PBDS „la cheie”. În secțiunea aceasta și în următoarea vom studia, în sfîrșit, două tipuri de treapuri care depășesc puterile unui set PBDS.

Să considerăm următorul deziderat: ne dorim un vector / listă / colecție indexată care să suporte următoarele operații în $\mathcal{O}(\log n)$:

- Returnează al k-lea element.
- Modifică al k-lea element.
- Inserează un element pe poziția a k-a.
- Șterge al k-lea element.

După cum am spus în introducerea părții, această structură se numește **treap cu chei implicite** sau **treap implicit**. Nu vom mai stoca chei ordonate crescător ca pînă acum, ci valorile colecției în ordinea în care apar ele. Dorim ca la o parcurgere în inordine să obținem exact elementele colecției. Cu alte cuvinte, dorim ca al k-lea nod în inordine să fie chiar al k-lea element din colecție. De exemplu, dacă fiul stîng al rădăcinii este un subarbore cu k noduri, atunci rădăcina trebuie să conțină valoarea $v[k]$.

Oricare dintre implementările de treap se pretează la această adaptare. Exemplificăm implementarea cu *split* și *merge*. Precizăm că am omis verificările de integritate, pentru a nu lungi codul. De exemplu, nu verificăm că poziția inserării este cuprinsă între 0 și n. Includem mai jos doar bucățile de cod care diferă substanțial față de implementările anterioare. Anexa include o implementare completă.

Funcțiile `int get(int pos)` și `void set(int pos, int val)` sînt practic o reimplementare a funcției `kth_element(int k)`. Găsim întîi nodul în cauză, apoi fie îi returnăm valoarea, fie i-o modificăm.

Funcția `split(int pos, ...)` are o nouă misiune: să spargă treapul dat în două treapuri, din care primul să conțină k noduri, iar al doilea restul. Implementarea este foarte similară cu spargerea după valoare.

Funcția `insert()` procedează astfel: coboară în arbore cît timp prioritatea nodului de inserat este mai mică decît a nodului curent. Ea coboară pe stînga sau pe dreapta după cum poziția de inserat este mai mică sau egală, sau mai mare decît, mărimea subarborelui stîng. Cînd coboară în fiul drept, funcția contabilizează numărul de elemente lăsate în urmă în fiul stîng. Cînd găsește nodul unde trebuie inserată noua valoare (confom priorității), sparge treapul în alte două treapuri, care devin fiii nodului inserat. Fiul stîng va conține atîtea elemente cîte mai are (încă) de lăsat în urmă nodul inserat.

```
void insert(int& t, int pos, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, pos, v[elem].l, v[elem].r);
    t = elem;
  } else if (pos <= v[v[t].l].cnt) {
    insert(v[t].l, pos, elem);
  } else {
    insert(v[t].r, pos - v[v[t].l].cnt - 1, elem);
  }
  update_cnt(t);
}

void insert(int pos, int val) {
  v[n] = { .val = val, .pri = rand(), .cnt = 1, .l = 0, .r = 0 };
  insert(root, pos, n++);
}
```

Funcția `merge()` este nemodificată. Funcția `erase(int pos)` procedează similar cu funcția `get(pos)`: coboară pînă la poziția dorită, apoi apelează `merge()` pe fiii poziției.

Una peste alta, codul complet pentru structura de treap implicit are circa 110 linii spațiate.

12.7 Operații pe interval

Treapurile admit operații pe interval, în același fel în care arborii de intervale admit operații pe interval. În acest caz, avem nevoie de implementarea cu *split* și *merge*, deoarece trebuie să putem izola un treap care conține doar cheile din intervalul dorit.

Introducem noțiunea de informație cu propagare *lazy*, de exemplu o cantitate de adunat la fiecare nod din subarbore. Avantajul față de arborii de intervale este că nodurile pot subîntinde intervale de lungimi arbitrare, cu valori mari, și pot fi reorganizate la nevoie.

Pentru un exemplu de implementare, vedeți problema Reversals and Sums (CSES).

12.8 Benchmarks

Tabelul 12.1 listează timpii obținuți cu diversele implementări de treapuri augmentate cu statistici de ordine. Programul folosește același schelet de testare ca și tabelul 11.1, așadar face circa 300 000 000 de operații într-o structură cu $n = 500\,000$ de numere. Pe aceleași date, un set PBDS rulează în 117 ms.

varianta	**timpul**
treap cu *split* și *merge*	90 ms
treap cu *split* și *merge*, alocat dinamic	135 ms
treap cu rotații	97 ms

Tabela 12.1: Timpii obținuți pentru 300 de milioane de operații pe treapuri.

Toate uneltele pentru benchmark și implementările de treap sînt disponibile și într-un repo pe GitHub.

12.9 Probleme

12.9.1 Problema Cut and Paste (CSES)

enunț • sursă

Problema este o aplicație directă a treapurilor implicite. Creăm un treap cu o literă în fiecare nod. Pentru o operație (a, b), este suficient să spargem treapul în trei porțiuni, respectiv $X = [1, a-1]$, $Y = [a, b]$ și $Z = [b+1, n]$. Apoi alipim aceste porțiuni în ordinea XZY.

Remarcăm că această problemă se mulează perfect peste implementarea cu *split* și *merge*, dar nu și peste cea cu rotații.

12.9.2 Problema Reversals and Sums (CSES)

enunț • sursă

Această problemă este un exemplu tipic de treap cu informații cu propagare *lazy*. Merită să luăm în calcul posibilitatea de a folosi un simplu arbore de intervale, dar din păcate acela nu se pretează. Dacă descompunem un interval $[a, b]$ și notăm în fiecare nod din descompunere informația *lazy* booleană „acest interval trebuie răsturnat", vom greși. Intervalele nu trebuie doar răsturnate, ci și mutate la stînga sau la dreapta, căci vor ajunge să ocupe alte poziții în listă.

Să pornim de la un treap implicit, căci conceptual memorăm valori dintr-un vector. Acest treap vine cu cîmpurile obișnuite în fiecare nod: valoare, prioritate și contor (mărimea subarborelui). În plus, vom adăuga încă două cantități:

1. O valoare întreagă `subtree_sum` care reține suma valorilor din subarbore. Vom menține în timp real această sumă, ca și pe contor, într-o funcție `update_node` pe care o apelăm ori de cîte ori structura arborelui se modifică. Știm deja să facem acest lucru pentru treapuri implicite.
2. O valoare booleană `flip` care indică că întregul subarbore trebuie răsturnat.

Să vedem acum cum implementăm cele două operații.

Pentru `range_sum(a, b)`, putem să spargem treapul ca și în problema anterioară în $X = [1, a-1]$, $Y = [a, b]$ și $Z = [b+1, n]$. Răspunsul va fi valoarea `subtree_sum` din Y. Apoi alipim la loc fragmentele în ordinea inițială.

Pentru `range_reverse(a, b)`, procedăm la fel, dar pe rădăcina lui Y setăm `flip` la valoarea 1.

Ca și la arborii de intervale, este important să propagăm informația *lazy* la fii în momentele-cheie. Am indicat aceste momente prin comentarii în cod. Am denumit funcția tot `push()` ca și la arborii de intervale.

12.9.3 Problema T-Shirts (Codeforces)

enunț • sursă

Chiar dacă găsim problema în vreo colecție de linkuri pe Internet și știm că se rezolvă cu treapuri, folosirea acestora nu este evidentă.

Să începem cu partea simplă. Pare natural să ordonăm oferta de tricouri așa cum le evaluează clienții: descrescător după calitate, apoi crescător după preț. De aici încolo, calitatea nu ne mai interesează. De asemenea, dacă ne ajută putem ordona bugetele clienților, să spunem crescător.

Am încercat să construiesc un treap implicit cu costurile tricourilor, iar în acest treap, dat fiind un buget b, să navighez logaritmic spre ultimul tricou cumpărat. Dar problema este că prețurile nu sînt ordonate, iar clienții nu cumpără tricouri dintr-un interval compact, ci pe sărite. Ne-am dori să aflăm rapid cîte tricouri cumpără un client, și cu ce preț total, din fiul stîng al nodului curent (ca să știm cu ce buget rămas coborîm în fiul drept). Dar pare imposibil. Pentru prețuri ca $1, 1\,000\,000, 1, 1\,000\,000, 1, 1\,000\,000$ nu putem evalua ușor situația.

Problema nu include un editorial, dar am găsit ideea rezolvării în acest comentariu. Ținem **un treap de bugete**, inițializat cu bugetele inițiale ale clienților, ordonate. Apoi evaluăm tricourile

în ordinea de mai sus. Întrebarea este cum actualizăm rapid clienții care mai au bani să cumpere tricoul i. Dorim să adăugăm $+1$ la numărul de tricouri și $-c_i$ la bugetul rămas pentru toți acești clienți.

Pentru tricoul curent i, de cost c_i, să apelăm `split(c[i])`. Am numit treapurile rezultate T_N (de la *no buy* / nu cumpăra) și respectiv T_B (de la *buy* / cumpără). Pe T_B notăm informația *lazy* că toți clienții au $+1$ tricou și $-c_i$ la buget. Dar acum trebuie să refacem treapul ordonat după noile bugete. Cum să procedăm? Operația `merge()` funcționează doar cînd T_N și T_B acoperă intervale disjuncte de chei (orice cheie din T_N este mai mică sau egală cu orice cheie din T_B). Dar, după ce toți clienții din T_B au plătit c_i, cheile pot fi intercalate.

Soluția este să spargem din nou T_B în valori $< c_i$ și $\geq c_i$ (pe noile bugete). Să numim aceste treapuri T_P (de la *poor* / sărac) și T_R (de la *rich* / bogat). În cuvinte,

- T_N sînt clienți cu bugete din $[0, c_i)$, care nu au cumpărat tricoul i.
- T_P sînt clienți cu bugete din $[0, c_i)$ după ce au cumpărat tricoul i.
- T_R sînt clienți cu bugete din $[c_i, \infty)$ după ce au cumpărat tricoul i.

Acum unificăm T_N, T_P și T_R astfel:

- Inserăm naiv valorile din T_P în T_N. Obținem un treap cu toate valorile în $[0, c)$.
- Unificăm acest treap cu T_R apelînd `merge()`, căci acum cheile sînt separate de c.

Pasul inserării naive îl rezolvăm cu un DFS prin T_P. Nu este nevoie să implementăm ștergeri din T_P, căci îl dezmembrăm complet. Este suficient să setăm pe `NULL` pointerii fiecărui nod din T_P pentru a-l transforma într-o frunză.

Toate operațiile sînt $\mathcal{O}(n \log n)$, cu excepția inserărilor naive. Care este efortul total pentru aceste operații? Să considerăm ce se întîmplă într-o inserare naivă. Avem un cost c și un buget $b \in [c, 2c)$ (dacă am fi avut $b \geq 2c$, atunci nodul ar fi stat în T_R, nu în T_P). Dar atunci să observăm că noul buget, $b - c$, scade sub jumătate din bugetul original b:

$$b < 2c \implies \frac{b}{2} < c \implies b - c < \frac{b}{2}$$

În cuvinte, de cîte ori este inserat naiv în T_N, bugetul clientului scade la jumătate sau chiar mai jos. Deci fiecare client poate fi inserat naiv de cel mult $\log VAL_MAX$ ori. Complexitatea inserărilor naive, și a întregului algoritm, este $\mathcal{O}(n \log n \log VAL_MAX)$.

Detaliu de implementare

În cod veți observa că funcțiile `split()` și `insert()` folosesc operatorii $<$ și respectiv $\leq$ pentru a compara bugetele. Această decizie este importantă, iar acest comentariu de pe Codeforces explică foarte bine motivul. Pe măsură ce procesăm tricouri, iar bugetele clienților încep să scadă, vom avea tot mai multe bugete egale. În acest moment trebuie să fim atenți cum gestionăm cheile egale într-un treap. Altfel riscăm să dezechilibrăm treapul și să ajungem la adîncime $\mathcal{O}(n)$.

Să ne imaginăm o operație *split(x)* pe un treap cu toate cheile egale cu x. Să spunem că algoritmul alege să mute cheile egale cu x în dreapta, deci după spargere treapul stîng va rămîne gol.

Acum să ne imaginăm operația *insert(x)*, din nou într-un treap cu toate cheile egale cu x. După cum știm, *insert* începe prin a coborî de la rădăcină pînă cînd prioritatea noului nod depășește prioritatea subarborelui curent. Toate cheile fiind egale, noi decidem dacă lăsăm funcția să coboare în nodul stîng sau în cel drept. Cînd prioritatea noului nod este maximă, spargem subarborele în două treapuri care devin fiii nodului inserat.

Aici intervine dezechilibrarea. Dacă lăsăm funcția *insert* să coboare în fiul drept, iar dacă *split* transferă toate nodurile existente tot în fiul drept, putem obține în timp un treap de adîncime $\mathcal{O}(n)$. Este important ca *insert* să prefere direcția opusă lui *split*.

12.9.4 Problema Clasament (Lot 2016)

enunț • surse

Cred că problema „se cere” rezolvată cu hashing. Nu știu cum altfel putem compara și clasifica m permutări de lungime n. Dacă acceptăm asta, atunci cumva trebuie să menținem eficient codul hash al permutării. Un algoritm de hashing eficient și ușor de implementat este să privim permutarea ca pe un număr de n cifre într-o bază B și să calculăm acel număr modulo un m ales.

Soluție cu treap

Fiindcă sîntem la capitolul de treapuri, să ținem într-un treap ID-urile elevilor (de la 0 la $n-1$), ordonate după scor și timp. Atunci putem calcula recursiv codul hash al unui treap. Dacă treapul stochează ID-urile 5, 8 și 2 în subarborele stîng, 4 în rădăcină și 3 și 7 în subarborele drept, și dacă luăm $B = 10$ pentru simplitate, atunci dorim ca codul hash să fie 582437, pe care îl putem sparge ca:

$$582437 = 582 \cdot 10^3 + 4 \cdot 10^2 + 37$$

Generalizînd,

- Fie L codul hash al subarborelui fiului stîng.
- Fie R codul hash al subarborelui fiului drept.
- Fie S_R mărimea (numărul de noduri) al subarborelui fiului drept.
- Fie i ID-ul din rădăcina subarborelui.
- Atunci codul hash este $(L \cdot B^{S_R+1} + i \cdot B^{S_R} + R) \bmod M$

Este important să alegem o bază B primă. Altfel putem ajunge la coliziuni (permutări diferite cu coduri hash identice).

Acum putem definitiva structura de date. În fiecare nod avem nevoie să stocăm codul hash și mărimea subarborelui. Putem recalcula aceste valori în $\mathcal{O}(1)$ cînd reorganizăm treapul, fie că alegem implementarea cu *split* și *merge*, fie cu rotații.

Ca detaliu de implementare, cînd un scor se modifică trebuie să ștergem ID-ul elevului respectiv din treap și să-l reinserăm conform noului scor. Ca să nu risipim memorie, cu puțină atenție putem refolosi chiar nodul șters.

Soluție cu descompunere în radical

Putem să stocăm permutarea curentă și în blocuri de mărime $\mathcal{O}(\sqrt{n})$. Această descompunere amintește de problema Serega and Fun de la capitolul de descompunere în radical. Acolo aveam nevoie să facem rotații pe subsecvențe, dar operația din problema de față este similară.

Nu insistăm ca blocurile să aibă fix lungime $\sqrt{n}$, ci le lăsăm să se dezechilibreze prin ștergeri și inserări. Dacă un bloc ajunge la o lungime-limită, să spunem $2\sqrt{n}$, atunci reechilibrăm structura: colectăm toate blocurile într-un vector, apoi le redistribuim. În practică, testele problemei nu duc la reechilibrări.

Pe fiecare bloc menținem codul hash al fragmentului de permutare din acel bloc. La o modificare de scor, recalculăm în $\mathcal{O}(\sqrt{n})$ codul hash al celor (cel mult) două blocuri afectate. După fiecare modificare, recalculăm codul hash al întregii permutări combinînd codurile celor $\mathcal{O}(\sqrt{n})$ fragmente după aceeași regulă ca mai sus.

Cele două soluții au timpi practic egali, ceea ce arată că treapurile au constantă cam proastă.

12.9.5 Problema Roata Norocului (Lot 2024)

enunț • sursă

Costul comprimării unui arbore

Să considerăm cazul unui arbore-stea cu 5 noduri, cu centrul de valoare 8 și vecinii de valori 6, 10, 20 și 30. Să analizăm două strategii. În prima, unim 8 cu vecinii mai scumpi, apoi cu cel mai ieftin.

- Unește 10-8, cost 2.
- Unește 20-8, cost 12.
- Unește 30-8, cost 22.
- Unește 8-6, cost 2.
- Cost total 38.

În a doua strategie, unim 8 cu 6 cîndva mai devreme:

- Unește 10-8, cost 2.
- Unește 8-6, cost 2.
- Unește 20-6, cost 14.
- Unește 30-6, cost 24.
- Cost total 42.

Pare că rentează să unim fiecare nod cu vecinii săi mai scumpi decît el înainte de a-l uni cu unul mai mic. Pentru a ne verifica intuiția, să privim coloana stîngă în listele de mai sus. Toate valorile apar exact o dată, cu excepția minimului (6). Este normal, căci un nod apare în stînga în momentul cînd este eliminat. Așadar, suma coloanei stîngi este constantă. Atunci, pentru a minimiza costul, trebuie să maximizăm coloana dreaptă. De aceea vom amîna să eliminăm un nod pînă cînd devine inevitabil.

(Ca fapt divers, problema în forma inițială se referea la un singur arbore, fără actualizări. În acel caz, o strategie suficientă este: la fiecare pas, comprimăm muchia care maximizează minimul dintre cele două noduri.)

Rezultă că fiecare nod va contribui la coloana dreaptă de atîtea ori cîți vecini mai scumpi decît el are. La egalitate putem departaja oricum, cu condiția să nu facem dublă contabilitate, ci exact unul dintre noduri să-l contorizeze pe celălalt. Deci formula pentru cost este:

$$cost(u) = \sum_{x \text{ în subarbore}} V_x - \min_{x \text{ în subarbore}} V_x - \sum_{x \text{ în subarbore}} V_x \cdot B_x$$

Am notat cu B_x numărul de vecini ai lui x mai scumpi decît x. În funcție de implementare, poate fi nevoie să scădem 1 din B_x însuși, pentru a-l exclude pe părintele lui x. Implementarea de mai jos evită acest caz.

Operațiile necesare

Oricînd dorim să proiectăm o structură de date, pornim de la întrebarea: ce vrem să aflăm?

Primii doi termeni ai formulei, adică suma și minimul din subarbore, cu actualizări, sînt partea ușoară a problemei. Vom face o liniarizare DFS, peste care construim un arbore de intervale de sume și de minime, cu actualizări punctuale și cu interogări pe interval. Dacă notăm cu $t_i[u]$ și $t_o[u]$ timpii de intrare și de ieșire din DFS, atunci aflăm primii doi termeni ai formulei făcînd interogări pe intervalul $[t_i[u], t_o[u]]$.

Pentru ultimul termen al formulei, eu am reorganizat informația astfel. Fiecare muchie (x, y) are o **contribuție**, desigur egală cu $\min(V_x, V_y)$. Dacă însumăm contribuțiile tuturor muchiilor din subarbore, ajungem exact la suma dorită: valoarea V_x luată de atîtea ori de cîte ori x este minim pe o muchie.

Fiecare nod u este responsabil să stocheze **suma contribuțiilor pe muchiile de la u la fiii lui u**. Să numim această cantitate **contribuția unui nod**. Putem stoca contribuțiile nodurilor tot în arborele de intervale. Atunci contribuția totală a unui subarbore (acel ultim termen al formulei) este tot o simplă sumă pe interval.

Interogări

Răspunul la o interogare este așadar cumulul a trei valori într-un arbore de intervare elementar: suma valorilor, minus valoarea minimă, minus suma contribuțiilor.

Actualizări

Desigur, actualizarea valorii unui nod în arborele de intervale este elementară. Dar cum actualizăm contribuția unui nod? Să vedem ce contribuții se schimbă pe muchii la modificarea valorii unui nod u din V_u în V'_u.

1. Suma contribuțiilor dintre u și fiii lui u, stocată în u.
2. Contribuția dintre u și părintele său, stocată în părinte (dacă există).

Pentru (1), să presupunem că valoarea crește, deci $V_u < V'_u$. Cazul în care valoarea scade se calculează la fel, doar cu semn schimbat.

- Pentru fiii x cu $V_x < V_u$ contribuția nu se schimbă: contribuția pe acele muchii continuă să fie V_x.
- Pentru fiii x cu $V_u \leq V_x < V'_u$ contribuția era V_u și acum devine V_x. Să spunem că există k astfel de fii. Pentru a actualiza contribuția nodului u, vom scădea $k \cdot V_u$ și vom adăuga suma valorilor acestor fiii.
- Pentru fiii x cu $V'_u < V_x$ contribuția era V_u și acum devine V'_u. Să spunem că există l astfel de fii. Pentru a actualiza contribuția nodului u, vom aduna $l \cdot (V'_u - V_u)$.

Așadar, avem nevoie de o structură care să ne dea rapid numărul de elemente dintr-un interval de valori dat și suma valorilor acestora.

Pentru (2), putem calcula naiv contribuția, fiind vorba de o singură muchie. Apoi este nevoie să-i spunem părintelui: în lista ta de valori se află o valoare V_u. Schimbă acea valoarea în V'_u.

Din păcate, nu cunosc nicio structură de date pentru aceste nevoie mai simplă decît un treap. Pe lîngă augmentarea nodurilor din treap cu numărul de elemente din subarbore, avem nevoie și de augmentarea cu suma elementelor. Întrucît fiecare nod își construiește un treap cu fiii, suma mărimilor treapurilor este $n - 1$.

Partea IV

Alte structuri de date

Capitolul 13

Păduri de mulțimi disjuncte

Acest capitol nu conține teorie, deocamdată.

Fac o precizare legată de nomenclatură. Nu îmi plac denumirile larg răspîndite:

- *union-find*, pentru că nu îmi place să identific structura de date prin operații. Este ca și cum am denumi arborii de intervale *query-update*.
- *DSU* (*disjoint set union*), din nou pentru că include numele unei operații.

În aceste note voi folosi denumirea DSF (engl. *disjoint-set forest*), care consideră că arată mult mai clar ce reprezintă structura: o colecție arbori disjuncți care partiționează o colecție de obiecte.

Urmează cîteva probleme de care m-am izbit de-a lungul anilor și care folosesc DSF în moduri mai puțin evidente. Vă recomand și probleme din alte capitole care folosesc DSF:

- Extending a Set of Points
- Minimum Spanning Tree for Each Edge
- Regate
- 0-1 MST

13.1 Probleme

13.1.1 Problema Restructuring Company (Codeforces)

enunț • sursă

Chiar de la citirea enunțului recunoaștem problema ca pe una „servită” de păduri de mulțimi disjuncte. Singura noutate este că operațiile de tipul 2 trebuie procesate eficient, nu în $\mathcal{O}(y-x)$.

Este suficient să folosim un vector în plus, `end[]`, în care $end[i]$ indică că intervalul $[i, end[i]]$ este cunoscut ca fiind reunit. Aceasta garantează că operațiile de reuniune de interval necesită un efort total de $\mathcal{O}(n \log^* n)$. De exemplu, odată ce am unificat pozițiile $i, i+1 ii+2$ nu vom mai consulta niciodată poziția $i+1$ (decît, desigur, dacă primim o operație care începe de acolo).

13.1.2 Problema Trampoline (infO(1) Cup 2020)

enunț • surse

Soluția cu binary lifting

Problema se petrece într-o matrice uriașă ($10^9 \times 10^9$). Multe din aceste probleme se rezolvă cu baleiere (engl. *scan line*).

Editorialul propune următoarea abordare (prima care mi-a venit și mie, este drept). Să facem următoarele observații teoretice referitoare la o cale, $(r_1, c_1) - (r_2, c_2)$:

1. Dacă avem de coborît, întotdeauna merită să coborîm pe prima trambulină disponibilă. Nu avem niciun motiv să așteptăm și să coborîm mai tîrziu, pe o trambulină mai la dreapta. Tot acolo ajungem.
2. Calea este fezabilă doar dacă există trambuline pe rîndurile $r_1, r_1 + 1, \ldots, r_2 - 1$, pe coloane nedescrescătoare.
3. Calea este fezabilă dacă ultima trambulină ne lasă pe rîndul r_2 pe o coloană cel mult egală cu c_2.

Atunci, putem sorta trambulinele descrescător după coloană. Similar, sortăm interogările descrescător după c_1. Apoi le procesăm interclasat, de la dreapta la stînga.

Cînd întîlnim o trambulină la (r, c), notăm cîte niveluri poate ea să coboare. Răspunsul este 1 + cîte niveluri poate să coboare ultima trambulină întîlnită pe rîndul $r + 1$, sau doar 1 dacă nu există o trambulină pe rîndul $r + 1$. Practic, trambulina curentă își notează indicele trambulinei de pe rîndul $r + 1$. Putem vedea acești indici ca pe niște pointeri la părinți, ceea ce duce la o structură arborescentă printre trambuline.

Cînd întîlnim o interogare $(r_1, c_1) - (r_2, c_2)$, tratăm separat cazurile speciale cum ar fi $r_1 > r_2$ sau $r_1 = r_2$. În rest, vrem să știm dacă trambulina curentă de pe rîndul r_1 poate să coboare $r_2 - r_1$ rînduri. Mai ales, vrem să știm *la ce coloană* ne lasă ultima trambulină, cea care ne coboară de la nivelul $r_2 - 1$ la r_2: ne lasă la o coloană cel mult egală cu c_2?

Așadar, pentru fiecare trambulină, vrem să aflăm indicele trambulinei care este al $r_2 - r_1$-lea strămoș. Putem afla această informație cu *binary lifting*, ca la multe alte probleme de arbori.

Soluția cu *disjoint sets*

Să mai facem o observație teoretică:

4. Dacă, în timp ce procesăm două interogări, ajungem cu ele în același punct, traiectoria lor viitoare va fi identică.

Prin asta vrem să spunem că ambele căi vor prefera, la fiecare moment, prima trambulină posibilă. Pentru completitudine, menționăm că fiecare dintre căi se va opri din folosit trambuline după ce ajunge pe rîndul-destinație. Vom clarifica acest aspect. Dar important este că, grupînd interogă-

rile, le putem lăsa pe toate să coboare simultan printr-o trambulină, oricîte ar fi. Recunoaștem unificările specifice DSF.

Concret, normalizăm numerele rîndurilor (păstrăm originalele). Facem o baleiere pe coloane, de la stînga la dreapta și de sus în jos. Pentru coloana curentă c, vom menține pentru fiecare rînd r mulțimea de interogări aflate în prezent la (r, c). Procesăm trei tipuri de evenimente:

1. O interogare nouă începe la (r, c). Adăugăm indicele acelei interogări la mulțimea de pe rîndul r. *Exemplu*: Dacă pe rîndul r aveam interogările 2, 10, 13, unite într-o mulțime disjunctă cu reprezentatul 10, și dacă la (r, c) începe interogarea cu indicele 8, unificăm mulțimea lui 10 cu mulțimea lui 8 (care îl conținea doar pe 8).
2. O trambulină se află la (r, c). Dacă rîndurile r și $r+1$ aveau coordonate consecutive înainte de normalizare, toate interogările de la r coboară la $r+1$. Concret, unificăm mulțimile celor două rînduri, iar noua coordonată pentru toate interogările va fi $r + 1$. Rîndul r rămîne fără interogări (deocamdată).
3. O interogare se termină la (r, c). Verificăm pe ce rînd se află mulțimea disjunctă care conține acea interogare. Răspunsul este *Yes* dacă și numai dacă rîndul este cel puțin egal cu r.

O clarificare pentru 3. Noi lăsăm interogările să coboare oricît de jos. Cînd dăm peste finalul interogării, ne întrebăm: a reușit interogarea să coboare **cel puțin** pînă la linia r înainte să ajungă la coloana c? Dacă da, atunci desigur am fi putut ajunge și **exact** la (r, c) dacă omiteam niște trambuline. Astfel evităm să separăm mulțimi odată unificate.

Ordonarea evenimentelor este adesea importantă, dacă avem mai multe la aceleași coordonate. De exemplu, poate începe o interogare nouă care apoi folosește imediat trambulina de la aceleași coordonate.

Un caz particular: algoritmul nu tratează cazurile în care $r_1 > r_2$ sau $c_1 > c_2$. Acestea trebuie filtrate de la intrare.

Codul fiecărei funcții este elementar. Nu folosim decît DSF și sortări, fără arbori sau *binary lifting*.

Partea V

Măiestrie pe biți

Tehnici pentru calcule compacte și eficiente

Capitolul 14

Operații pe biți. Compactarea variabilelor

14.1 Operații elementare

Multe dintre structurile de date pe care le folosim au legătură cu puterile lui 2: arborii indexați binar, arborii de segmente, heapurile, *bitsets*... De aceea, este important să fiți familiari cu o listă de operații utile.

Această listă este inspirată din articolele Bit Twiddling Hacks și Bit manipulation.

14.1.1 Noțiuni de bază

- C are suport pentru constante hexazecimale (`0xdeadbeef`) și binare (`0b1100101101`).
- De asemenea, puteți tipări valori în bazele 8 și 16 cu printf sau folosind STL.
- O cifră hexa are 4 biți în baza 2. Deci `0xce = 0b1100'1110`.
- Uneori valorile hexa ajută la claritate. `0xff'ffff` arată că avem 24 de biți 1, poate mai clar decît `((1 << 24) - 1)`. Dar depinde de gusturi.

14.1.2 Măști

Conceptual, o mască pe n biți este un număr binar care descrie o submulțime a unei mulțimi cu n elemente, indexate de la 0 la $n-1$. Bitul k are valoarea 1 dacă și numai dacă submulțimea descrisă include elementul k. Măștile se pretează la submulțimi mici (32 sau 64 de biți). Dincolo de aceasta avem nevoie de vectori de măști (cum ar fi `bitset` din STL).

Exemple din lumea reală:

- Multe constante predefinite în limbajele de programare ocupă un singur bit (sînt puteri ale lui 2), iar programatorul le poate combina cu OR pe biți. Vezi codurile de eroare din PHP. Programatorul poate decide să afișeze sau să ascundă anumite tipuri de eroare. De exemplu, apelînd funcția `error_reporting(E_ALL & ~E_NOTICE & ~E_DEPRECATED)`, programatorul

arată că vrea să vadă toate erorile în afară de cele minore (`E_NOTICE`) și cele despre perimare (`E_DEPRECATED`).

- În multe programe, utilizatorii au permisiuni, care sînt definite ca puteri ale lui 2, iar permisiunile unui utilizator sînt o mască (o submulțime din mulțimea totală).
- Programele de șah moderne descriu tabla ca pe o colecție de măști pe 64 de biți: una pentru pozițiile pionilor albi, una pentru pozițiile pionilor negri etc. Este foarte convenabil că tabla de șah are fix 64 de pătrate.

Măștile au anumite avantaje, cum ar fi:

1. Este foarte ușor să facem operații de intersecție, reuniune, diferență folosind operatorii pe biți `&`, `|`, `^` și `~`.
2. Valoarea numerică a măștii creează o bijecție naturală între mulțimea $\{0, 1, \ldots, 2^n - 1\}$ și mulțimea submulțimilor unei mulțimi. Deci putem stoca eficient informații despre toate submulțimile într-un vector de 2^n elemente indexate după ordinea lexicografică a submulțimii.

14.1.3 Operații pe măști de biți

operația	**efectul**
`x & 31` sau `x & 0x1f`	Restul împărțirii lui x la 32.
`x & 1`	Test de (im)paritate.
`x & (1 << k)`	Test dacă al k-lea bit este 1. Atenție, nu returnează 0/1, ci $0/2^k$.
`(x >> k) & 1`	Test dacă al k-lea bit este 1. Returnează 0/1.
`x |= (1 << k)`	Setează bitul k pe 1.
`x &= ~(1 << k)`	Setează bitul k pe 0.
`x ^= (1 << k)`	Neagă bitul k.
`x & (x-1)`	Elimină ultimul bit 1 din n. Util și ca test dacă n este putere a lui 2.

Tabela 14.1: Operații pe măști de biți.

14.2 Numărarea biților de 1 dintr-o valoare (popcount)

De exemplu, pentru 187, care în binar este `10111011`, dorim răspunsul 6. Programul complet implementează metodele de mai jos și măsoară timpii de rulare.

14.2.1 Metoda naivă

Shiftăm în mod repetat numărul la dreapta și numărăm biții de 1.

```
pop = 0;
while (x) {
  pop += x & 1;
  x >>= 1;
```

```
}
```

14.2.2 Metoda Kernighan

Eliminăm și contorizăm cîte un bit de 1.

14.2.3 Funcții built-in

Folosim funcțiile `__builtin_popcount` (pentru `int`) și `__builtin_popcountll` (pentru `long long`) sau `std::popcount` din STL. Atenție, aceasta din urmă apare doar în standardul C++20.

Notă: Funcțiile built-in pot deveni mult mai rapide pe arhitecturile care admit instrucțiunea nativă `popcnt`. Dacă știți sigur că mașina de evaluare admite această instrucțiune, îi puteți cere compilatorului să genereze codul adecvat cu comanda:

```
#pragma GCC target("popcnt")
```

Atenție, dacă CPU-ul evaluatorului nu admite această instrucțiune, rezultatul este nedefinit.

14.2.4 Tabel precalculat

Construim un tabel de 256 de valori care precalculează rezultatul pentru un octet. Extragem octeții numărului prin shiftare.

14.2.5 Tabel precalculat + conversie

Construim același tabel de 256 de valori. Extragem octeții numărului prin conversia la `char*`.

14.2.6 Calcul paralel

Iată o metodă bazată pe calcul paralel, care face `log(sizeof(n))` operații și nu folosește condiții. Pentru exemplificare, să considerăm numărul pe 16 biți $\overline{abcdefghijklmnop}$. Dorim să facem următoarele 4 operații:

1. Să înlocuim fiecare grupă de doi biți cu suma acelor biți. Cu alte cuvinte, dorim ca în locul grupei $\overline{ab}$ să scriem 00 dacă $a = b = 0$, 01 dacă $a = 0$ și $b = 1$ sau $a = 1$ și $b = 0$, respectiv 10 dacă $a = b = 1$.
2. Să înlocuim fiecare grupă de 4 biți cu suma celor două grupe de 2 biți din care se compune.
3. Să înlocuim fiecare grupă de 8 biți cu suma celor două grupe de 4 biți din care se compune.
4. Să înlocuim întregul număr (16 biți) cu suma celor două grupe de 8 biți din care se compune.

Astfel, la finalul celor 4 operații, în locul numărului original vom avea exact numărul de biți din număr. Să începem cu operația 1. Secretul este să procesăm toate cele 8 grupe de cîte doi biți simultan, adică în paralel. Pentru aceasta, shiftăm biții impari ($\overline{a0c0e0g0i0k0m0o0}$) la dreapta și

îi adunăm cu biții pari ($\overline{0b0d0f0h0j0l0n0p}$). Observăm că, dacă procedăm astfel, grupele de doi biți nu se influențează între ele, deoarece nu există transport dincolo de acești doi biți.

În binar, masca care selectează biții pari este `0b0101010101010101`, care în hexazecimal se scrie ca `0x5555`. Pe 32 de biți obținem instrucțiunea:

```
x = ((x >> 1) & 0x55555555) + (x & 0x55555555);
```

Pentru operația 2, dorim să adunăm fiecare grupă de doi biți cu număr de ordine impar cu grupa de doi biți din dreapta ei. Deci vom shifta numărul cu doi biți, vom reține doar grupele pare folosind masca binară `0b0011001100110011`, respectiv `0x3333`, și le vom aduna (în paralel) cu aceleași grupe din numărul original:

```
x = ((x >> 2) & 0x33333333) + (x & 0x33333333);
```

Procedînd la fel pentru grupe de 4, 8 16 și eventual 32 de biți, obținem codul anexat.

Rezultatele pentru 100 000 000 de operații pe 32 de biți sînt:

metoda	**viteza (milioane de op. / secundă)**
naiv	72
Kernighan	69
`std::popcount` / `__builtin_popcount`	570
`__builtin_popcount` cu `pragma`	2 350
tabel precalculat	990
tabel precalculat + `char*`	900
calcul paralel	2 560

Tabela 14.2: Implementări de *popcount* și vitezele lor.

14.3 Logaritmul în baza 2

De exemplu, pentru pentru 187, care în binar este `10111011`, dorim răspunsul 7 (indicele celui mai din stînga bit 1). Codul implementează (aproape) aceleași metode. De data aceasta, diferențele sînt surprinzător de mici, căci este greu de evitat branching-ul datorat condițiilor.

metoda	viteza (milioane de op. / secundă)
naiv	820
tabel precalculat 256 bytes + două `if`-uri	1 694
tabel precalculat 256 bytes + `while`	768
tabel precalculat 64 KiB	1 472
`__builtin_clz` / `std::countl_zero`	785
`__builtin_clz` cu `pragma`	2 350
calcul paralel	861
calcul paralel + `popcount`	1 209
înmulțire de Bruijn	719

Tabela 14.3: Implementări de logaritm în baza 2 și vitezele lor.

Notă: Ca și în cazul operației *popcount*, multe dintre implementări devin considerabil mai rapide dacă folosim instrucțiunea nativă `lzcnt`, care numără zerourile din stînga ale unui întreg. Dacă procesorul admite instrucțiunea, i-o putem cere compilatorului cu comanda:

```
#pragma GCC target("lzcnt")
```

Cu această optimizare, instrucțiunea `__builtin_clz` devine cea mai rapidă metodă.

14.4 Programare dinamică pe măști de biți

Există o clasă de probleme în care n (dimensiunea intrării) este mică, pe la 20-25. În aceste probleme avem de calculat informații despre fiecare mască pe n biți, de exemplu informații despre fiecare subșir al unui vector cu n elemente sau despre fiecare submulțime a unei mulțimi cu n obiecte. Tehnica de rezolvare pornește de la două observații generale:

1. Este eficient să stocăm aceste informații într-un vector cu 2^n poziții, unde poziția i reține informația corespunzătoare măștii i (care la rîndul ei corespunde unui subșir sau unei submulțimi).
2. Informațiile despre o mască cu k biți de 1 decurg din măștile cu $k-1$ biți de 1. Cu alte cuvinte, informațiile despre o submulțime cu k obiecte decurg din cele k submulțimi cu $k-1$ obiecte.

Numim această tehnică **programare dinamică pe măști de biți**. Rezolvările acestor probleme tind să aibă complexitatea $\mathcal{O}(n \cdot 2^n)$ sau $\mathcal{O}(n^2 \cdot 2^n)$.

În rest, nu este foarte mult de spus teoretic. Vom regăsi multe exemple în problemele aferente.

14.5 Probleme

14.5.1 Problema Fill the Bag (Codeforces)

enunț • sursă

Dacă suma cutiilor este mai mică decît capacitatea sacului, atunci desigur nu există soluție. Invers, dacă suma cutiilor este mai mare sau egală cu capacitatea sacului, atunci există soluție. În cel mai rău caz, putem tăia totul pînă la mărime 1.

Iată o soluție greedy corectă. Să examinăm cutiile de la mari la mici. Fie k mărimea curentă. Dacă mai avem k spațiu în sac, sigur merită să luăm cutia. Dacă nu o luăm, înseamnă că vom lua o mulțime de cutii mai mici a căror sumă este mai mare sau egală cu k. Dar putem demonstra că acea mulțime include o submulțime de sumă exact k. Este preferabil să luăm acum cutia de mărime exact k cu zero tăieturi.

Dacă nu mai avem k spațiu în sac, atunci ne punem întrebarea dacă merită să tăiem cutia, fie ca să luăm acum $k/2$, fie ca să continuăm să o tăiem pe viitor în cutii și mai mici. Desigur, tăieturile costă, deci am prefera să nu facem asta, ci să luăm mai tîrziu niște cutii mai mici cu volumul total k. De aceea, vom ține permanent evidența cantității pe care ne permitem să o refuzăm. Inițial, această cantitate este $R = \sum a_i - n$. Pe măsură ce refuzăm cutii, R scade cu volumul cutiilor refuzate. Ne permitem să refuzăm cutia curentă dacă $R \geq k$. Dacă nu ne permitem, incrementăm numărul de tăieturi, decrementăm numărul de cutii de mărime k și îl creștem cu 2 pe cel de cutii de mărime $k/2$.

Ca implementare, cel mai simplu este să menținem un singur vector de frecvență, indexat după logaritmul cutiilor. De exemplu, la indicele 3 vom ține numărul de cutii de mărime 8.

Editorialul pare să ofere o soluție inversă, de la mic la mare.

14.5.2 Problema Compatible Numbers (Codeforces)

enunț • surse

Să definim operatorul $a \subseteq b$ cu semnificația că reprezentarea binară a lui a conține 1 doar pe poziții unde și b conține 1.

Ideea de bază este că, dacă y este compatibil cu x, atunci y este compatibil și cu orice număr $z \subseteq x$. De aceea, calculăm următoarele informații. Pentru orice mască x, definim `fit[x]` ca fiind fie un număr z de la intrare astfel încît $z \subseteq x$, fie 0 dacă nu există un astfel de z.

Calculăm vectorul `fit` extinzînd măștile cîte un bit la un moment dat. Inițial, `fit[a[i]] = a[i]` pentru fiecare număr de la intrare. Apoi, pentru un z oarecare, dacă `fit[z]` nu este încă definit, încercăm să eliminăm pe rînd fiecare bit din masca z. Dacă masca x obținută are `fit[x]` definit, atribuim `fit[z] = fit[x]`.

Dorim să evaluăm masca z după ce am evaluat toate măștile care au cu un bit de 1 mai puțin. Ordinea naturală, crescătoare, oferă această proprietate, deci codul este simplu.

Odată calculat vectorul `fit`, trecem prin vectorul a și, pentru fiecare element a_i, definim $b = \overline{a_i}$ și tipărim `fit[b]` sau `-1`, după caz.

Optimizare de memorie

De amorul artei, am reușit să nu țin un întreg pentru fiecare mască, ci doar un bit. Necesarul de memorie a scăzut de la 20 MB la 4,3 MB. Ideea este să reținem doar informația „există (sau nu) un număr conținut în această mască". Dacă răspunsul este „da", atunci la afișare este nevoie să aflăm **care** este acel număr. Îl putem recupera bit cu bit: pornind de la masca completă, eliminăm cîte un bit, cîte un bit pînă cînd nu se mai poate. Atunci numărul rămas trebuie să fie printre cele de la intrare.

Este interesant că timpul de rulare a rămas practic neschimbat, deși constanta teoretică a crescut: complexitatea afișării nu mai este $\mathcal{O}(n)$, ci $O(p \cdot n)$. Vedem din nou beneficiile consumului redus de memorie. Bufferul de lucru are doar 2^{22} biți, adică 512 KiB de memorie.

Optimizare de viteză

Implementarea naturală propagă informația din `fit` iterînd după mască și, pentru fiecare mască, prin toți biții săi:

```
void compute_fits() {
  for (int mask = 1; mask < MAX_VALUE; mask++) {
    for (int bit = 0; bit < MAX_BITS; bit++) {
      if ((mask & (1 << bit)) && !fit[mask]) {
        fit[mask] = fit[mask ^ (1 << bit)];
      }
    }
  }
}
```

Am fost șocat să descopăr că este de două ori mai rapid să iterăm întîi după bit și apoi după mască:

```
void compute_fits() {
  for (int bit = 0; bit < MAX_BITS; bit++) {
    for (int mask = 1; mask < MAX_VALUE; mask++) {
      if ((mask & (1 << bit)) && !fit[mask]) {
        fit[mask] = fit[mask ^ (1 << bit)];
      }
    }
  }
}
```

Cred că știu explicația. În prima variantă, calculul unei măști accesează și măști relativ îndepărtate ca valoare, ceea ce „rupe" cache-ul. În a doua variantă, cel puțin pentru biții mici (pînă la 12), masca curentă accesează măști la distanță de cel mult 2^{12} octeți, deci din aceeași pagină de memorie sau una vecină.

14.5.3 Problema Yet Another Substring Reverse (Codeforces)

enunț • sursă

Observații teoretice

Primele observații sînt relativ elementare. În primul rînd, orice două subsecvențe pot fi făcute vecine printr-o operație bine aleasă. Așadar, vom căuta două subsecvențe care:

1. Să fie disjuncte (să ocupe poziții diferite în șir).
2. Să conțină caractere distincte.
3. Să aibă suma lungimilor maximă.

De fapt, condiția (1) nu este necesară. Dacă subsecvențele nu sînt disjuncte, nu vor avea nici caractere distincte.

Detalii de implementare

Pentru implementare vom folosi măști de biți. Unele concepte sînt simple și vă vor intra în reflex (dacă nu v-au intrat deja). Oricînd parametrii au ordinul 20-30, sau orice pînă la 64, puteți folosi măști de biți pentru a calcula, compara, reuni, intersecta mulțimi de valori din acel domeniu.

Așadar, să reprezentăm orice subsecvență de caractere distincte printr-o mască de $\Sigma = 20$ de biți corespunzători literelor *a...t*, unde bitul 0 corespunde literei a, iar bitul 19 literei t. Fiecare bit are valoarea 1 dacă și numai dacă litera corespunzătoare apare în subsecvență. De exemplu, subsecvența *cbtf* corespunde măștii `1000 0000 0000 0010 0110` (nu uitați că bitul 0 este cel mai din dreapta).

Un concept des întîlnit este calcularea tuturor măștilor posibile. În acest caz, există cel mult $2^{20} \approx 1\,000\,000$ de măști posibile. Putem construi un vector de `bool` (sau chiar un `bitset`) care indică ce măști putem obține, oriunde în șirul dat. Construcția este simplă și durează $\mathcal{O}(\Sigma \cdot n) \approx 20\,000\,000$.

```
void build_obtainable_masks() {
  for (int i = 0; i < len; i++) {
    int mask = 0;
    int j = i;
    while ((j < len) && !(mask & (1 << s[j]))) {
      // Extinde subsecvența (i,j-1) cu poziția j.
      // Setează bitul corespunzător lui s[j].
      mask |= 1 << s[j++];
      obtainable[mask] = true;
    }
  }
}
```

Acum putem reformula cerința astfel. Căutăm două măști cu proprietățile:

1. Ambele pot fi obținute (sînt marcate în vectorul sus-menționat).
2. Sînt disjuncte (and-ul pe biți este 0).
3. Numărul total de biți (cardinalul, numit și *populație*) este maxim.

Dacă iterăm prin măști, pentru fiecare mască A care poate fi obținută putem calcula complementul ei $B = \bar{A}$. Cu alte cuvinte, B indică ce litere mai avem voie să folosim pentru a le lipi de cele din A. Pe masca B dorim să o întrebăm: dintre toate submulțimile tale care pot fi obținute, care este populația maximă?

Acesta este un exemplu tipic de programare dinamică pe măști. Există două posibilități:

1. Dacă masca B poate fi obținută, chiar B este răspunsul: dintre toate submulțimile lui B, B are cardinalul maxim (*duh*).
2. Altfel, încercăm pe rînd să modificăm fiecare bit 1 din B la valoarea 0 și vedem care dintre măștile astfel obținute ne dă un răspuns maxim.

Dacă vă ajută, nu uitați că o mască este o submulțime a mulțimii complete (care în cazul nostru este mulțimea $\{a, b, \ldots, t\}$). Reformulată în termeni de mulțimi, recurența de mai sus este: submulțimea maximă a lui B care poate fi obținută este fie B însăși, fie submulțimea maximă a oricărei submulțimi obținute prin eliminarea unui singur element din B.

```
void build_submask_populations() {
  for (int mask = 1; mask < (1 << SIGMA); mask++) {
    if (obtainable[mask]) {
      submask_pop[mask] = __builtin_popcount(mask);
    } else {
      for (int bit = 0; bit < SIGMA; bit++) {
        if (mask & (1 << bit)) {
          int s = submask_pop[mask ^ (1 << bit)];
          improve(submask_pop[mask], s);
        }
      }
    }
  }
}
```

Codul de mai sus iterează prin toți cei 20 de biți și îi procesează pe cei care au valoarea 1 în masca `mask`. Cu puțin efort am putea itera doar prin biții de 1, cu metoda Kernighan. Dar intuiesc că nu ar fi mai rapid. Numărul de biți evaluați ar scădea la jumătate (căci numărul mediu de biți 1 este 20/2), dar constanta ar crește.

La final, ultimul pas este să găsim o mască m care maximizează suma `popcount(m) + submask_pop[~m]`. Cu alte cuvinte, pentru o mască m fixată cel mai bun rezultat este dat de toți biții măștii m și de cît mai mulți biți ai unei măști care reprezintă o subsecvență din șir, dar care este disjunctă cu m.

14.5.4 Problema Cases (Codeforces)

enunț • sursă

Putem reformula problema astfel: trebuie să găsim o submulțime C a alfabetului („cazurile") astfel încît orice subșir de k caractere consecutive să conțină cel puțin un caracter din C. Dacă există mai multe astfel de submulțimi C, trebuie să o găsim pe cea de cardinal minim.

Ne interesează mai mult ce caractere apar în fiecare fereastră, nu și frecvența lor. De aceea, fiecărei ferestre îi vom asocia o mască. Atunci o mască C este viabilă dacă intersecția ei cu masca fiecărei ferestre este nenulă.

Este mai simplu să operăm pe negativ: dacă o mască este disjunctă cu oricare dintre măștile ferestrelor, acea mască nu poate fi soluție. De aici rezultă un algoritm în trei pași:

Măștile cunoscute a fi rele

Fie M masca unei ferestre de k caractere. Atunci complementul său, $\overline{M}$, este rău, adică nu poate fi niciodată soluție la problemă. De ce? Prin definiție $\overline{M}$ nu va crea niciun terminator de cuvînt în fereastra curentă. De exemplu, dacă $k = 4$ și fereastra curentă este *bebe*, atunci masca M are biții pentru b și e setați, iar complementul său ($\overline{M} = acdfghijk...$) este sigur rău, pentru că nu inserează pauze în textul *bebe*.

De asemenea, trebuie obligatoriu ca ultima literă din șir să fie terminator, deci complementul măștii care conține doar ultimul caracter este rău.

Propagarea măștilor rele

Dacă o mască este rea, cu atît mai mult orice submască a ei va fi rea. Pentru propagare, aplicăm metoda cunoscută. Odată ce depistăm o mască rea, nu iterăm prin toate submăștile ei, ci doar prin cele obținute prin eliminarea cîte unui bit. Le marcăm pe acestea ca rele și va fi treaba lor să-și marcheze submăștile ca rele.

Exemplu: Dacă masca `0011101` este rea, vom marca drept rele măștile `0001101`, `0010101`, `0011001` și `0011100`.

Găsirea soluției

Acest pas este cel mai simplu. Iterăm prin toate măștile care nu sînt marcate ca rele și o selectăm pe cea cu populație minimă. Putem folosi, de exemplu, unul dintre built-in-urile `std::popcount` sau `__builtin_popcount`.

14.5.5 Problema The Minimum Number of Variables (Codeforces)

enunț • sursă

Și aceasta este o problemă relativ standard de programare dinamică pe măști de biți.

Reprezentarea datelor

Trebuie să vedem unde intervin cele 2^n stări așteptate. Este destul de natural să privim lucrurile astfel. Să reindexăm vectorul a de la 0. Să construim acum un vector boolean $v[0 \dots 2^n - 1]$ care reține cîte o informație pentru fiecare mască pe n biți. Pentru o mască m, fie b indicele celui mai semnificativ bit 1 (unde $0 \leq b < n$). Atunci desemnăm $v[m]$ ca fiind `true` dacă și numai dacă putem procesa variabilele $a_0, \dots a_b$ astfel încît pe cele care corespund la biți de 1 să le stocăm în prezent în variabile. De exemplu, pentru masca $77 = 1001101_{(2)}$, $v[77]$ va fi `true` dacă putem procesa valorile $a_0 \dots a_6$ astfel încît la final să avem patru variabile cu valorile a_0, a_2, a_3 și a_6.

Subliniem că, prin convenție, o mască unde bitul cel mai semnificativ (MSB) este b se referă întotdeauna la starea sistemului după ce am procesat elementele $a_0, \dots, a_b$. Aceasta deoarece ultima variabilă este în mod necesar stocată (deci bitul aferent este 1).

Putem inițializa sistemul cu $v[1] =$ `true`, care semnifică că am procesat primul element și l-am salvat într-o variabilă.

Programarea dinamică

Conform enunțului, pentru a procesa elementul a_b trebuie:

- Să fi procesat primele b elemente.
- Să avem între variabilele curente două (sau aceeași luată de două ori) a căror sumă să fie a_b.
- Să punem valoarea a_b într-o variabilă nouă sau să suprascriem una dintre variabilele curente.

Putem implementa aceste condiții cu o programare dinamică de tip înainte. Din informațiile despre măștile pe exact b biți le deducem pe cele pe exact $b + 1$ biți. Concret, dacă masca m pe b biți poate fi obținută, și dacă există două elemente indicate în masca m a căror sumă este a_b, atunci putem obține:

- Masca $m' = m \mid 2^b$. Ea are cu un bit în plus față de m, semnificînd că folosim o variabilă nouă.
- Colecția de măști obținute din masca m' prin ștergerea oricărui bit 1 în afară de bitul b. Ele au la fel de mulți biți ca m, semnificînd că suprascriem variabila al cărei bit dispare.

Complexitatea este $\mathcal{O}(n \cdot 2^n)$ datorită generării colecției descrise mai sus.

Verificarea sumelor

Așadar, înainte de a completa măștile de variabile pentru bitul b, trebuie să ne întrebăm care dintre măștile anterioare (pe b biți) sînt cu adevărat utile, în sensul că includ două elemente a căror sumă este a_b. De exemplu, pentru vectorul $a = (1, 2, 3, 100, 200)$, putem procesa primele 3 elemente în diverse moduri, obținînd diverse măști pe 3 biți. Dar toate sînt inutile, căci nicio combinație din primele 3 valori nu ne va da suma 100.

Vom folosi un al doilea bitset de măști pentru acest scop, calculabil tot în $\mathcal{O}(n \cdot 2^n)$. Vom calcula

în aceeași iterație toate măștile în care MSB este bitul $b-1$. Le vom procesa în ordine crescătoare, astfel încît la procesarea unei măști să avem garanția că toate submăștile ei au fost procesate. Pentru masca m, fie k bitul cel mai puțin semnificativ (LSB). Avem trei cazuri:

1. Dacă $m \oplus 2^k$ oferă suma a_b, atunci și m oferă suma a_b.
2. Altfel, iterăm prin toți biții 1 ai lui m. Dacă pentru orice bit i avem $a_i + a_k = a_b$, atunci evident masca m oferă suma a_b.
3. Altfel masca m nu oferă suma a_b și nu o vom folosi la programarea dinamică înainte.

Soluția

Numărul minim de variabile necesar este populația minimă a oricărei măști fezabile pe exact n biți (cu bitul $n-1$ setat).

14.5.6 Problema Math Test (Codeforces)

enunț • surse

Soluția teoretică

Ideea problemei mi s-a părut grea. Știind că $n \leq 10$ și că ne permitem să iterăm prin toate submulțimile de elevi, de ce am face-o?

Putem varia **calculul valorii absolute** pentru fiecare elev. Într-o mască de elevi, un bit 1 pe poziția i va însemna că dorim să calculăm modulul ca $r_i - x_i$. Cu alte cuvinte dorim ca $r_i \geq x_i$. Viceversa, un bit de 0 înseamnă că dorim ca $r_i \leq x_i$. Iau naștere două tabere de elevi, cărora dorim să le maximizăm, respectiv să le minimizăm scorurile. Le vom numi „elevi max" și „elevi min".

Odată fixată masca, vom căuta să dăm punctaje problemelor care să crească scorurile elevilor max și să evite să le crească pe ale elevilor min. Dar cum? Dacă o întrebare are rezolvitori din ambele tabere, cum să o punctăm? Mai ales, cum ne asigurăm că scorurile elevilor max vor fi deasupra așteptărilor, iar ale elevilor min vor fi sub așteptări?

Observația-cheie este: **nu trebuie neapărat să respectăm inegalitățile** $r_i \geq x_i$ **și** $r_i \leq x_i$. Dacă un sistem de punctare nu respectă unele dintre cele n inegalități, este OK. Tot ce pierdem este că obținem o surpriză totală neoptimă. De exemplu, dacă noi insistăm să calculăm $|5-7|$ ca $-7+5=-2$, vom obține un total incorect. Dar va exista un alt sistem care respectă inegalitățile și va obține o surpriză mai bună.

Așadar, putem proceda astfel pentru fiecare mască:

1. Inițializează valoarea surprizei cu $-x_i$ pentru elevii max și $+x_i$ pentru elevii min.
2. Acum, pentru orice întrebare dorim să adunăm punctajul său pentru elevii max care au rezolvat-o și să-l scădem pentru elevii min care au rezolvat-o.
3. Surpriza totală va fi suma valorilor de la punctele (1) și (2).

4. Pentru fiecare întrebare, calculează-i scorul ca diferența între numerele de elevi max, respectiv min care au rezolvat-o.
5. Dacă punctăm cu P puncte o întrebare de scor S, ea va contribui cu $P \times S$ puncte la surpriza totală.
6. Rezultă că putem sorta cele m întrebări crescător după scor și să le atribuim punctajele $1, 2, \ldots, m$ în această ordine.

Detalii de implementare

Merită să citim de la bun început datele ca măști de biți. Fiecare întrebare are o mască de n biți cu rezolvitorii.

```
for (int stud = 0; stud < n; stud++) {
  for (int q = 0; q < m; q++) {
    int bit = getchar() - '0';
    solvers[q] |= bit << stud;
  }
  getchar();
}
```

Scorul unei întrebări este diferența populațiilor de 1 și de 0:

```
int get_score(int solvers, int stud_mask) {
  // solvers = masca de elevi care au răspuns corect la întrebarea curentă
  // stud_mask = masca curentă de elevi max
  int good = __builtin_popcount(solvers & stud_mask);
  int bad = __builtin_popcount(solvers & ~stud_mask);
  return good - bad;
}
```

Se vede că scorul unei întrebări poate varia între $-n$ și $+n$. De aceea, în prima sursă chiar colectez întrebările într-un vector de liste indexat după scor. Astfel putem sorta întrebările în $\mathcal{O}(m+n)$.

A doua sursă nu mai colectează întrebările, ci doar distribuția scorurilor într-un simplu vector indexat de la $-n$ la n. Din acest vector putem deduce valoarea surprizei totale. De exemplu, dacă $n = 10$ și vectorul este $(5, 3, 3, \ldots)$, atunci:

- Primele 5 întrebări au punctajele 1, 2, 3, 4, 5 și scor -10. Contribuția acestora la surpriza totală este $15 \cdot (-10) = -150$.
- Următoarele 3 întrebări au punctajele 6, 7, 8 și scor -9. Contribuția acestora este $21 \cdot (-9) = -189$.
- Următoarele 3 întrebări au punctajele 9, 10, 11 și scor -8. Contribuția acestora este $30 \cdot (-8) = -240$.
- etc.

Așadar, putem afla valoarea surprizei pentru fiecare mască cu niște sume Gauss. Apoi alegem

masca M care maximizează surpriza. Mai trebuie doar să reconstituim punctajele întrebărilor cunoscînd masca M și vectorul de scoruri aferent.

Pentru aceasta, este suficient să calculăm sume parțiale peste vectorul de scoruri: $P = (5, 8, 11, \dots)$. Să observăm că aceste valori sînt exact punctajele cele mai mari pentru o întrebare cu acel scor (notate cu bold mai sus). De exemplu, 11 este ultimul punctaj pentru întrebările de scor -8. Acum parcurgem întrebările în ordine și le calculăm scorurile. Ori de cîte ori vedem o întrebare de scor -8, îi vom atribui punctajul din vectorul P, pe care apoi îl vom decrementa. Astfel, cele 3 întrebări de scor -8 (căci știm că sînt exact 3) vor primi punctajele 11, 10 și 9.

Observație de încheiere: Există doar $2^n = 1\,024$ de măști de rezolvitori pentru cele m întrebări. La prima vedere, pare un cîștig să înlocuim vectorul de lungime 10 000 cu unul de lungime 1 024 (frecvențele măștilor de rezolvitori). Din păcate, problema poate avea multe teste mici și atunci $\mathcal{O}(t \cdot 2^n)$ poate fi mai scump decît $\mathcal{O}(\sum m)$.

14.6 Compactarea variabilelor

Cînd o variabilă are nevoie doar de un bit, putem „înghesui” 8 variabile pe un octet. Dacă avem nevoie de un milion de booleeni, îi putem compacta în 128 KiB de memorie. Vom cheltui puțin la compactare și decompactare, dar vom economisi memorie. Există multiple argumente pentru compactare.

1. Consumul de memorie scade. În loc de 2 GB putem folosi doar 256 MB. Ocazional putem găsi o soluție la care comisia nu s-a gîndit.
2. Cu cît ne încadrăm mai bine în cache, cu atît viteza crește.
3. Dacă avem de făcut operații pe vectori booleeni (and, or, xor, numărarea biților), le putem procesa în grupe de cîte 32 sau 64.

Desigur, putem folosi un `bitset` STL. Dar este simplu să ne implementăm propria structură. Iar STL-ul nu ne mai ajută la variabile pe 2 biți sau pe 4 biți, care pot fi și ele compactate, sau la probleme ca ERA, unde avem nevoie de funcții pe care `std::bitset` nu le oferă.

14.6.1 Bitset

Conceptual, dorim să înlocuim

- atribuirea `v[pos] = val` cu o funcție `void set(int pos, bool val);`
- citirea `v[pos]` cu o funcție `bool get(int pos)`.

Vom face împărțirea astfel. Elementele 0-31 stau în primul *word* (`unsigned int`). Elementele 32-63 stau în al doilea *word*. Concret, elementul `pos` va sta în *word*-ul `pos / 32`, altfel spus `pos >> 5`. Dintre cei 32 de biți ai *word*-ului, elementul `pos` va sta pe bitul `pos % 32`, altfel spus `pos & 31`. Ca fapt divers, compilatoarele moderne fac deja aceste transformări pentru voi. Puteți folosi notația `pos % 32` dacă vi se pare mai lizibilă.

Așadar, funcția de citire a unui bit va fi un *one-liner* simplu:

```
bool get(int pos) {
  return (v[pos >> 5] >> (pos & 31)) & 1;
}
```

La atribuire, discuția devine un pic mai nuanțată. Unele operații cum ar fi setarea bitului pe 1 sau pe 0 sînt relativ directe:

```
void set(int pos) {
  v[pos >> 5] |= 1 << (pos & 31);
}

void clear(int pos) {
  v[pos >> 5] &= ~(1 << (pos & 31));
}
```

Dar dacă dorim o funcție generică, una care să ia valoarea bitului ca parametru? Este important să nu testăm condiții (să nu facem *branching*). Acest cod este de 3-4 ori mai lent decît un vector naiv de `bool`:

```
void set_branching(int pos, bool val) {
  unsigned& c = v[pos >> 5];
  unsigned mask = 1 << (pos & 31);

  if (val) {
    c |= mask;
  } else {
    c &= ~mask;
  }
}
```

Dar putem face și atribuirea fără *branching*. Prima jumătate a instrucțiunii șterge vechea valoare a bitului, iar a doua jumătate atribuie valoarea nouă.

```
void set(int pos, bool val) {
  unsigned& c = v[pos >> 5];
  int shift = pos & 31;
  c = (c & ~(1 << shift)) | (val << shift);
}
```

Anexa conține implementarea completă. Am inclus și implementări pe 8 biți, care folosesc un vector de $n/8$ `unsigned char`. Iată niște *benchmark*-uri pentru un vector de 10^6 elemente cu 10^9 citiri și 10^9 scrieri.

metoda	viteza (milioane de op. / secundă)
naiv	1 700
vector compactat, 32 de biți	1 350
vector compactat, 8 biți	1 200
vector compactat, cu *branching*, 8 sau 32 de biți	450
`std::bitset`	500

Tabela 14.4: Implementări de bitset și vitezele lor.

Aplicație: Operații booleene rapide

Atît versiunea proprie, cît și cea STL, ne oferă operații logice *en gros*, cîte 32 deodată. TODO: Discuție despre problema rucsacului.

14.6.2 Variabile pe 2 biți

Toată discuția despre *bitsets* se pretează și la variabile pe doi biți. Consumul de memorie scade de 4 ori, întrucît folosim exact doi biți în locul fiecărui octet. Anexa conține implementarea completă. Includ aici doar rutinele principale:

```
struct packed_array {
  unsigned v[ARRAY_SIZE / 16];

  void set(int pos, int val) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    v[word] = (v[word] & ~(3 << offset)) | (val << offset);
  }

  int get(int pos) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    return (v[word] >> offset) & 3;
  }
};
```

metoda	viteza (milioane de op. / secundă)
naiv	1 700
vector compactat, 32 de biți	950

Tabela 14.5: Metode de compactare pe doi biți și vitezele lor.

14.7 Probleme

14.7.1 Problema DNA Evolution (Codeforces)

enunț • surse

Iată o problemă aleasă ușor forțat pentru a ilustra compactarea variabilelor. Am căutat „DNA” pe codeforces, întrucît alfabetul ADN are doar 4 litere. 🤓

Soluție în $\mathcal{O}(n \log n)$

Pentru fiecare literă L, pentru fiecare lungime a unei infecții, $1 \leq e \leq 10$, și pentru fiecare rest h modulo e, reținem cîte un AIB în care notăm 1 pe pozițiile din s congruente cu h modulo e care conțin o literă L. Să presupunem că îl denumim `aib[L][e][h]`.

Actualizările sînt relativ ușor de făcut. La modificarea poziției x, trecem de pe 1 pe 0 valorile din toate AIB-urile corespunzătoare vechii litere și resturilor $x \mod m$ pentru fiecare $1 \leq m \leq 10$. Apoi adăugăm 1 în AIB-urile noii litere și acelorași resturi.

Interogările pornesc de la observația că, dacă litera L apare pe poziția h într-o infecție de lungime e, atunci ne interesează de cîte ori apare ea pe pozițiile $l + h, l + 2h$ etc. Trebuie doar să consultăm AIB-ul corespunzător acestui scop, `aib[L][e][(l + h) % e]`, cerîndu-i suma pe intervalul $[l, r]$. Procedăm la fel pentru fiecare poziție $0 \leq h < m$.

Din ce am observat pe Codeforces, aceste soluții tind să folosească între 150 și 300 MB de memorie. Ordinul de mărime este corect: 55 de AIB-uri pentru 4 litere, fiecare conținînd 100 000 de întregi, necesită $55 \times 4 \times 100\,000 \times 4 = 88$ MB de memorie. Restul este posibil să fie risipă.

Soluție în $\mathcal{O}(n\sqrt{n})$

Iată și o soluție în $\mathcal{O}(n\sqrt{n})$ care consumă doar 64 kB de memorie. Din această cauză, ea operează 100% din time în cache-ul L1 și obține timpi comparabili cu soluția anterioară. Nu am ales întîmplător acest deziderat de 64 kB. Evaluatorul Codeforces folosește un procesor i5-3470, care are un cache L1 de 64 kB per core. Este o cifră comună pentru procesoarele moderne.

Folosim descompunerea în radical, deoarece ea tinde să folosească puțină memorie suplimentară. Pe fiecare bloc stocăm date similare cu metoda anterioară: numărul de litere de fiecare tip pentru fiecare lungime de *query* și fiecare rest modulo acea lungime.

Să facem niște calcule.

- Șirul s necesită 2 biți per literă, așadar 25 000 de octeți.
- Un bloc are nevoie de $1 + 2 + \cdots + 10 = 55$ de contoare pentru fiecare dintre cele 4 litere. Contoarele încap pe `short`. Rezultă un necesar de 440 de octeți per bloc (eu am 444, căci reține și poziția de început a blocului).
- Rezultă că ne permitem circa 90 de blocuri, deoarece $25\,000 + 90 \cdot 444 = 64\,960 < 65\,536$.

Prima variantă a codului este directă.

Varianta a doua compactează șirul s. Ea este puțin mai lentă.

Varianta a treia elimină majoritatea operațiilor de împărțire. Ea este de circa 3 ori mai rapidă decît celelalte.

Într-o a patra variantă (pe care nu o mai includ), am crescut numărul de blocuri pînă la 317 (valoarea teoretică, $\sqrt{100\,000}$). Codul a devenit mai lent.

Concluzii

Această problemă nu este un exemplu perfect pentru compactarea variabilelor. În schimb, ea este un exemplu instructiv că reducerea de 3 000 de ori a memoriei (de la 200 MB la 64 kB) compensează cu succes un algoritm asimptotic mai slab. Să nu uităm că $\log 100\,000 \approx 16{,}6$, pe cînd $\sqrt{100\,000} \approx 316$. În plus, AIB-urile au constantă bună. Deci **teoretic descompunerea logaritmică ar trebui să fie de 19 ori mai lentă**.

14.8 Iterarea prin submăștile unei măști

Ocazional avem nevoie ca, pentru o mască binară m, să iterăm prin toate numerele binare care conțin biți 1 doar pe poziții pe care m conține biți 1. De exemplu, pentru masca `0110010`, dorim să iterăm prin măștile

```
0000000
0000010
0010000
0010010
0100000
0100010
0110000
0110010 = M
```

Surprinzător, putem face acest lucru cu o buclă elementară:

```
for (unsigned s = m; s; s = (s - 1) & m) {
  // s este o submască a lui m
}
```

Remarcăm că bucla nu tratează masca `s = 0`. Dacă o doriți și pe aceea, faceți un caz particular în afara buclei sau scrieți un cod puțin mai urît pentru a include 0 în buclă, de exemplu:

```
int s = m + 1;
do {
  s = (s - 1) & m;
  // procesează masca s
} while (s);
```

De ce funcționează acest cod? În cazul simplu cînd m este contiguă, să zicem `1111111`, observăm că `(s - 1) & m = s - 1`, deci `s` iterează pur și simplu descrescător de la `1111111` la `0000000`. Dacă m conține și zerouri intercalate, atunci s va itera descrescător prin combinațiile de 1, ignorînd zerourile. Exemplu:

```
m           = 110111110011000
s           = 010110000000000
s - 1       = 010101111111111
(s - 1) & m = 010101110011000
```

14.9 Probleme

14.9.1 Problema Crisp String (Codeforces)

enunț • sursă

Problema ne împinge spre cîteva observații de bază.

1. Putem folosi măști de litere, de exemplu pentru a caracteriza un șir după niște eliminări. Prin convenție, am ales 1 pentru literele eliminate și 0 pentru literele prezente.
2. Masca șirului inițial este 0 (nu am eliminat nimic).
3. Ordinea eliminării contează.
4. Există $2^{17} \approx 130\,000$ măști, deci ne permitem undeva la 1 000 de operații elementare per mască.

Să presupunem că reușim să clasificăm măștile în *bune* (care corespund unui subșir *crisp*) și *rele* (celelalte). Dacă calculăm frecvențele literelor, atunci din orice mască putem afla lungimea șirului corespunzător în $\mathcal{O}(p)$. Problema este 99% rezolvată, dar nu este suficient să luăm masca bună corespunzătoare șirului minim. Pot exista măști bune, dar la care nu putem ajunge prin eliminări pornind de la masca 0. În particular, masca $2^p - 1$, corespunzătoare eliminării tuturor caracterelor, este garantat bună (nimeni nu mai este vecin cu nimeni). Dar poate să nu fie accesibilă.

Totuși, completarea este simplă. Evaluăm toate măștile bune. Dacă o mască bună m nu este accesibilă, adică nu există nicio altă mască bună m' din care s-o obținem pe m înlocuind un 0 cu 1, atunci o reclasificăm pe m ca rea. Motivul este că nu există niciun drum de la 0 la m. Este important să facem evaluarea măștilor într-o ordine care să garanteze că toate submăștile au fost deja evaluate. Ordinea numerelor naturale este suficientă (deci putem itera prin măști cu un simplu `for`).

Acum sîntem pregătiți să atacăm miezul problemei: clasificarea măștilor în bune și rele. Ce înseamnă că o mască este rea? Înseamnă că ea are doi biți x și y, posibil egali, cu proprietățile:

- Au valoarea 0 (așadar sînt încă prezenți în șir).
- Literele corespunzătoare, x și y, sînt incompatibile.
- Undeva în șirul original există un interval mărginit de literele x și y, care nu mai conține alte litere x și y, și în care toate literele intermediare au bitul corespunzător 1. Cu alte cuvinte, undeva am făcut o serie de eliminări care au alipit x de y.

Cînd depistăm o astfel de situație, atunci nu doar masca sus-menționată este rea, ci și toate supramăștile ei: în afară de biții x, y și cei corespunzători literelor intermediare, ceilalți biți pot

avea orice valoare (0 sau 1).

Următoarea soluție s-ar putea încadra în timp, foarte la limită. Construim măști ternare pe 17 biți, unde 0 = literă încă activă, 1 = literă eliminată, 2 = literă pentru care încă nu am luat o decizie. În acest vector marcăm măștile rele, unde biților liberi le asociem cifra ternară 2. Apoi parcurgem recursiv aceste măști, înlocuind valorile de 2 în toate modurile cu 0 sau 1. În final, acele valori care folosesc, în baza 3, doar cifrele 0 și 1 corespund măștilor binare și descriu măștile rele. Complexitatea ar fi $\mathcal{O}(p \cdot 3^p) \approx 2{,}2 \cdot 10^9$, la limită.

Dar iată și o soluție mai naturală. Iterăm prin toate perechile de caractere (x, y). Pentru fiecare pereche, în $\mathcal{O}(n)$ colectăm toate aparițiile acestor caractere în șir și marcăm explicit ca rele măștile dintre aparații. După ce le colectăm pe toate, iterăm prin toate cele 2^{p-2} măști care au biții x și y 0 (sau 2^{p-1}, dacă $x = y$). Extindem mulțimea de măști rele astfel: dacă o mască considerată bună poate proveni dintr-o mască rea cu un bit în plus, masca devine și ea rea.

Este important să judecăm fiecare pereche (x, y) separat de alte perechi. De aceea, după ce găsim toate măștile rele, copiem informația într-un vector global și golim vectorul curent. Facem copierea cu OR, întrucît o mască este rea dacă este rea din cauza oricărei perechi de litere.

Ca optimizare, pentru iterarea doar prin 2^{p-2} biți am folosit codul de iterare prin submăștile unei măști. Codul este considerabil mai rapid decît dacă am itera prin toate cele 2^p măști și ne-am întreba „au biții x și y valoarea 0?”

Complexitatea pentru colectarea intervalelor este $\mathcal{O}(p^2 \cdot n)$. Ea poate fi redusă la $\mathcal{O}(p \cdot n)$, dar reducerea nu face obiectul acestui curs.

Capitolul 15

Tehnica *Meet in the middle*

15.1 Definiția tehnicii

Teoretic nu este foarte mult de spus despre tehnica *meet in the middle*. Tehnica se aplică la probleme cu soluții exponențiale, unde ne ajută să reducem complexitatea de la $\mathcal{O}(p^n)$ la $\mathcal{O}(p^{n/2})$.

Abordarea cu *meet in the middle* sparge problema dată în două subprobleme care sînt tot exponențiale. De aceea, soluțiile seamănă cu soluțiile forță brută. Metoda generală este:

1. Căutați soluția în $\mathcal{O}(p^n)$.
2. Spargeți problema în două: doi vectori de cîte $n/2$ elemente etc.
3. Aplicați soluția naivă fiecărei jumătăți.
4. Găsiți un mod de a combina cele două jumătăți.
    - La acest moment este posibil să descoperim că trebuie să extindem structura jumătății stîngi, ca să stocăm mai multe informații.

Dacă vedeți vreo problemă cu limita $30 \leq n \leq 40$, probabil se rezolvă cu această tehnică. 🙂

Acest capitol este foarte ingineresc. Majoritatea problemelor duc la subprobleme interesante de implementare și de optimizare. Multe dintre implementări pot fi simplificate cu măști de biți.

Resurse utile:

- Eticheta [meet-in-the-middle] pe Codeforces;
- F1!, colecția de articole SEPI pentru olimpici (capitolul despre MITM).

15.2 Probleme

15.2.1 Problema Meet in the Middle (CSES)

enunț • surse

Rezolvarea este esența tehnicii *meet in the middle*: împărțim cele 40 de elemente în două grupe de 20. Pentru fiecare dintre grupe avem destul timp și memorie ca să calculăm sumele celor 2^{20}

submulțimi și să le ordonăm, obținînd doi vectori. Apoi căutăm, cu tehnica celor doi pointeri, numărul de perechi de indici care dau suma x.

La iterarea cu cei doi pointeri trebuie să tratăm atent grupele de elemente egale.

Desigur, pentru o mulțime cu $n/2$ elemente este banal să producem lista ordonată a sumelor celor $2^{n/2}$ submulțimi în $\mathcal{O}(n \cdot 2^{n/2})$. Putem obține lista fie cu un bitmask, fie prin parcurgere recursivă. Ambele metode presupun o sortare la final, care pentru un vector de $2^{n/2}$ elemente are complexitate $\mathcal{O}(2^{n/2} \log 2^{n/2}) = \mathcal{O}(n \cdot 2^{n/2})$.

Dar există și o soluție mai puțin cunoscută în $\mathcal{O}(2^{n/2})$. Pe evaluatorul CSES, ea este de 5 ori mai rapidă! Vom proceda astfel pentru fiecare dintre jumătăți. Considerăm cele $n/2$ elemente unul cîte unul. Să spunem că am obținut lista sortată L_k a sumelor submulțimilor primelor k elemente. Pentru a-l include și pe cel de-al $k+1$-lea element, fie el x, observăm că L_{k+1} conține:

- toate valorile din L_k, pentru că avem opțiunea să nu-l folosim pe x;
- toate valorile din L_k crescute cu x, dacă alegem să îl folosim pe x.

Așadar, este suficient să îl stocăm pe L_k într-un vector și să-l interclasăm cu $L_k + x$. Această operație necesită 2^{k+1} comparații, deci efortul însumat pentru toate k-urile este $\mathcal{O}2^{n/2}$.

15.2.2 Puzzle-urile de pe Infoarena

Cosmin Negruțeri a scris pentru Infoarena un articol teoretic despre *meet in the middle*. Merită să petrecem cîteva minute cu aceste puzzle-uri, întrucît sînt pe alt tipic decît restul problemelor din acest capitol.

Friend of a friend

Acesta este un puzzle de încălzire. Indexăm cumva listele prietenilor lui A și B și vedem dacă au vreun element în comun. În practică, indexăm doar lista de prieteni a lui A (de exemplu într-o tabelă hash). Apoi, pentru fiecare prieten al lui B verificăm dacă apare în tabelă.

De ce astfel? Presupunînd că o persoană are în medie k prieteni, dacă chiar am colecta lista prietenilor prietenilor lui A am obține $\mathcal{O}(k^2)$ elemente. Algoritmul de mai sus rulează în $\mathcal{O}(k)$.

Six degrees of separation

Similar. Pentru fiecare dintre cei doi utilizatori, colectăm cunoștințele aflate la distanță cel mult 3. Apoi le intersectăm.

Equal partitions

Cum ar funcționa soluția în $\mathcal{O}(2^n)$? De multe ori ea ne conduce spre soluția cu *meet in the middle*. Iterăm prin toate cele 2^n măști și le dăm semnificația: biții 0 corespund elementelor din prima mulțime, iar biții 1 elementelor din a doua mulțime. Dacă o mască corespunde unor mulțimi cu sume egale, aceea este soluția.

Acum, să presupunem că iterăm prin cele $2^{n/2}$ măști pentru primele $n/2$ elemente. Vom stoca „ceva” pentru fiecare mască (vom vedea ce). Acum iterăm prin celelalte $2^{n/2}$ măști. Cum spuneam mai sus, acestea corespund unei partiționări a elementelor. Pentru o mască m din dreapta fixată, să spunem că sumele celor două submulțimi sînt X și Y. Înseamnă că dorim să întrebăm jumătatea stîngă: există vreo partiționare care obține diferența $|X - Y|$?

Rezultă că exact asta vom stoca în jumătatea stîngă! Calculăm, pentru fiecare mască, sumele submulțimilor aferente, A și B, și adăugăm valoarea $|A - B|$ într-o tabelă hash.

Minimum vertex cover

Aici este mai mult de povestit și este un intermezzo interesant. Vom demonstra întîi o egalitate utilă:

Dacă enumerăm toate cele 2^n măști pe n biți și pentru fiecare submască enumerăm toate sumăștile, efortul este $\mathcal{O}(3^n)$ (deși intuiția ne spune că ar fi $\mathcal{O}(4^n)$).

Iată o demonstrație frumoasă bazată pe combinatorică. Iată și o demonstrație bazată pe bijecții. Să considerăm masca 10011 pe 5 biți. Așadar, am ales ca din cei 5 biți să setăm doi pe 0. Acum îi vom enumera submăștile, de la 00000 la 11111. Așadar pot apărea noi zerouri... dar acestea nu sînt la fel ca primele. 🙂 Concret, să construim o bijecție între submăștile submăștilor pe n biți și mulțimea numerelor în baza 3 pe n biți:

- Cifra 1 semnifică că am ales bitul 1 în submască, apoi și în submasca submăștii.
- Cifra 0 semnifică că am ales bitul 0 în submască.
- Cifra 2 semnifică că am ales bitul 1 în submască, apoi 0 în submasca submăștii.

De exemplu, 20012 se referă la submasca 00010 obținută ca submască a submăștii 10011.

De aici rezultă un algoritm relativ clasic pentru problema MVC.

Square fence

Ce ar face soluția în $\mathcal{O}(4^n)$? Ar număra în baza 4 pe n cifre. Pentru o mască (în baza 4) fixată, cifrele 0, 1, 2 și 3 dictează partiționarea scîndurilor. Vedem dacă vreo partiție duce la sume egale.

Să facem asta pentru primele $n/2$ scînduri. Obținem diverși cvadrupleți (a, b, c, d), pe care îi punem într-o tabelă hash. Noi știm suma dorită S pentru fiecare submulțime (este 1/4 din suma lungimilor celor n scînduri). Acum iterăm prin celelalte $n/2$ scînduri și, pentru fiecare cvadruplet (a', b', c', d') găsit, căutăm în tabela hash cvadrupletul $(S - a', S - b', S - c', S - d')$.

8 puzzle

Cu indiciul că ne sînt suficiente 31 de mutări, soluția dorită este probabil: generăm cel mult 15 mutări din poziția de start și le hash-uim. Apoi generăm cel mult 16 mutări din poziția finală (regulile de mutare sînt aceleași indiferent de direcție) și căutăm pozițiile rezultate în tabela hash.

Dar probabil putem și să facem un BFS simplu, căci spațiul de poziții este doar 9!.

4 reversals

Facem două răsturnări pe șirul S, în toate modurile posibile, și punem rezultatele într-o tabelă hash. Apoi facem două răsturnări pe șirul T, în toate modurile posibile, și căutăm rezultatul în tabela hash.

15.2.3 Problema Anya and Cubes (Codeforces)

enunț • surse

O soluție naivă ar genera toate combinațiile legale de (cel mult) trei posibilități pentru fiecare element:

- nu îl includem;
- îl includem cu valoarea sa proprie;
- îl includem cu valoarea factorial.

Rezultă o complexitate de $\mathcal{O}(3^n)$, prea mare. M-am gîndit foarte sumar la soluții cu programare dinamică, de exemplu definind $D(s, i, j)$ ca numărul de moduri de a obține suma s cu primele i obiecte folosind j abțibilduri. Nu cred că o putem face să meargă, pentru că trebuie să gestioneze prea multe sume distincte.

Astfel ajungem la ideea de a sparge obiectele în 13 în stînga și 12 în dreapta și de a face generarea naivă pe aceste jumătăți. Vom obține doi vectori cu lungimea totală $3^{13}+3^{12} \approx 2 \cdot 10^6$. Elementele acestor vectori sînt perechi $\langle x, f \rangle$: suma x și numărul de abțibilduri necesare f. Pentru fiecare element din stînga trebuie să numărăm perechile corespondente din dreapta, de sumă exact $S - x$ obținută cu cel mult $k - f$ abțibilduri.

De aici soluțiile diverg. Un exemplu cu tabele hash ar putea fi:

- Grupează toate sumele obținute în stînga. Atașează fiecărei sume x un vector v în care $v[i]$ reține în cîte moduri am obținut suma x cu i abțibilduri.
- Fă sume parțiale peste acești vectori, astfel va $v[i]$ să rețină în cîte moduri putem obține suma x cu cel mult i abțibilduri.
- Pune toate sumele și vectorii lor într-o tabelă hash indexată după suma x.
- Pentru fiecare pereche $\langle x, f \rangle$ din dreapta, caută în tabela hash suma $S - x$, poziția $k - f$ din vector.
- Răspunsul este suma acestor valori din tabela hash.

Ca în multe alte ocazii, putem înlocui n căutări într-o tabelă hash cu sortarea + algoritmul cu doi pointeri. Sortăm vectorii celor două jumătăți. Apoi iterăm un pointer de la stînga la dreapta prin primul vector și de la dreapta la stînga prin al doilea vector, căutînd elemente de sumă egală cu S.

Ce este nou este cum procedăm cînd găsim acele elemente. În fapt, vom avea un grup de elemente egale cu x în stînga și un grup de elemente egale cu $S - x$ în dreapta. Acum avem o subproblemă. Trebuie să numărăm perechile cu suma abțibildurilor cel mult k. Din fericire, putem rezolva

subproblema cu un nou algoritm two pointers, care de data aceasta iterează doar prin cele două grupuri!

Optimizare

Am reușit să reduc timpul de rulare cu 30% cu următoarea observație. Perechile <sumă, abțibilduri> ocupă 16 octeți (8 + 1, dar rotunjit la 16). Dar nu este complicat să compactăm cele două valori în doar 64 de biți. Sumele sînt cel mult 10^{16}, adică ocupă doar 54 de biți, iar abțibildurile sînt cel mult 25, deci ocupă 5 biți. Le vom compacta într-un `long long` unde suma ocupă biții 5-58, iar abțibildurile biții 0-4. La sortare, comparăm natural aceste variabile, ceea ce duce exact la sortarea dorită: întîi după sumă, apoi după numărul de steguleț e. Astfel folosim jumătate din memorie. Nu știu de ce Codeforces raportează 20 MB, cînd eu în mod evident aloc doar 16 MB + mărunțișuri. Codeforces minte! 🥲

Deși codul face niște operații în plus pentru conversia între structura comprimată și cele două cîmpuri, totuși el este semnificativ mai rapid. Este un exemplu (al cîtelea?) că economia de memorie duce la cod mai rapid.

15.2.4 Problema Prime Gift (Codeforces)

enunț • sursă

Cred că acest gen de problemă are nevoie de puțin cod auxiliar, căci este greu să avem intuiția despre dimensiunile problemei și ale subproblemelor.

Indiciul că se face cu *meet in the middle* ar trebui să ne pună pe drumul cel bun. Ca nomenclatură, aceste numere se numesc numere urîte. Vom genera numerele urîte cu primii f factori și, separat, numerele urîte cu ultimii $n - f$ factori. Le vom pune în doi vectori, să zicem A și B, pe care îi vom sorta. Astfel, orice număr urît (pentru problema inițială) va fi un produs de două numere urîte, cîte unul din fiecare vector.

Nu prea știm cum să generăm direct al k-lea număr urît (aceasta era o întrebare clasică din folclor cînd eram eu licean!). Dar putem proceda invers: putem număra numerele urîte din intervalul $[1, z]$. Atunci putem căuta binar cel mai mic z pentru care există exact k numere urîte în $[1, z]$.

Pasul următor: pentru a număra numerele urîte din $[1, z]$, putem itera prin vectorul A. Pentru fiecare $x \in A$, căutăm binar valoarea maximă din $y \in B$ pentru care $xy \leq z$. Dacă y apare pe poziția p, atunci x poate forma p numere urîte.

De fapt, știm că n căutări binare pot fi întotdeauna înlocuite cu un algoritm cu doi pointeri. Așadar, pentru a afla numărul de numere urîte din $[1, z]$ complexitatea este $\mathcal{O}(|A| + |B|)$. Complexitatea totală a algoritmului de căutare este $\mathcal{O}((|A| + |B|) \log VAL_MAX)$, iar generarea și sortarea vectorilor este $\mathcal{O}(|A| \log |A| + |B| \log |B|)$ presupunînd că reușim să generăm vectorii în $\mathcal{O}(1)$ per element.

Dar aici intervine flerul... sau lipsa lui, pe care o suplinim prin cod auxiliar. 🤣 Cît de mari pot

fi vectorii? Milioane? Zeci de milioane? Și cum facem să-i echilibrăm ca să minimizăm $|A| + |B|$? Am scris un program care genera naiv numerele. Este destul de clar că:

1. Cu cît permitem mai multe numere prime, cu atît mai multe numere urîte vom avea.
2. Cu cît numerele prime sînt mai mici, cu atît mai multe numere vom avea.

Așadar, cel mai prost caz va fi $n = 16$ și $p = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53)$. Pe acesta putem face măsurători. Împărțirea în $16 = 8+8$ nu este fericită. Ea generează 7 000 000 de numere urîte cu primele 8 numere prime și doar 70 000 cu celelalte 8. Împărțirea cea mai fericită este $16 = 6 + 10$, care generează 1 000 000 de numere urîte cu primele 6 numere prime și 500 000 cu celelalte 10.

15.2.5 Problema Lizard Era: Beginning (Codeforces)

enunț • sursă

Enunțul problemei este ușor încîlcit, dar rezolvarea teoretică este relativ clară. Există $3^{25} \approx 8{,}4 \cdot 10^{11}$ moduri de a selecta aventurierii, căci pentru fiecare misiune exact unul din 3 va rămîne acasă. Așadar, pentru MITM vom genera două seturi de $3^{13} \approx 1{,}6 \cdot 10^6$ submulțimi și respectiv 3^{12} submulțimi.

Pentru fiecare submulțime vom calcula atitudinile lui L, M și W. Apoi, trebuie să găsim toate perechile de cîte un element din cele două seturi care **se însumează** la valori egale. Cum formulăm asta ca pe o problemă de căutare? Reținem tripletele ca pe primul termen și diferențele. De exemplu, vom stoca (100, 110, 120) ca pe (100, +10, +20) dacă apare în tabela stîngă. Dacă apare în tabela dreaptă, schimbăm semnele diferențelor și stocăm tripletul ca (100, -10, -20).

Cum folosim aceste informații? Fie tripletul (100, 110, 120) din partea stîngă și (200, 190, 180) din partea dreaptă. Observăm că

- Tripletele se însumează la (300, 300, 300), deci reprezintă o soluție validă.
- Tripletele se reprezintă ca (100, +10, +20), respectiv (200, +10, +20).

Rezultă că, atunci cînd căutăm fiecare triplet din partea dreaptă în structura de date din partea stîngă, cheia căutării este perechea de diferențe, în exemplul de mai sus (+10, +20). Atitudinea lui L (100, respectiv 200) va fi valoarea asociată cheii (+10, +20) în cele două structuri. Trebuie să ținem minte că pot exista mai multe soluții pentru același triplet și trebuie să o folosim pe cea mai mare.

Putem implementa structura părții stîngi ca pe o tabelă hash (`unordered_map`). Dar implementarea mea pare să se miște cu mult mai rapid. Ea toarnă toate tripletele în același vector și le sortează după trei criterii:

- Crescător după cheie (vectorul de diferențe).
- La egalitate, crescător după jumătate (cele din stînga înaintea celor din dreapta).
- La egalitate, pe cele din jumătatea stîngă le sortează crescător după L, iar pe cele din jumătatea dreaptă descrescător după L.

Atunci două perechi complementare vor apărea consecutiv în sortare. Soluția este dominată (cam 75% din timp) de sortarea a două milioane de triplete. Am încercat o optimizare: să generez și să sortez doar vectorul mic (de 3^{12} elemente), iar celorlalte 3^{13} elemente să le caut perechea binar în primul vector. Dar este considerabil mai lentă.

Pentru reconstituirea soluției, mi s-a părut cel mai direct să reținmasca în baza trei pentru fiecare triplet, deși aceasta consumă memorie suplimentară.

15.2.6 Problema Make Them Similar (Codeforces)

enunț • surse

Soluția naivă este: pentru fiecare din cele 2^{30} măști, generăm vectorul b și vedem dacă toate elementele sale sînt similare. De aici decurge soluția *meet in the middle*. Iterăm prin toate măștile pe 15 biți. Pentru fiecare mască și pentru fiecare element al lui b, notăm numărul de biți (*popcount*) din jumătatea stîngă a elementului xor-ată cu masca. Obținem așadar 2^{15} vectori a cîte $n \leq 100$ elemente cu valori între 0 și 15. Procedăm similar pentru cei 15 biți cel mai puțin semnificativi. Timpul și memoria necesare sînt:

$$2 \cdot 2^{15} \cdot 100 \approx 6\,500\,000$$

Pentru fiecare vector mai reținem și masca cu care l-am obținut. Acum trebuie să găsim doi vectori, cîte unul din fiecare jumătate, care însumați să dea un vector cu toate valorile egale. Dacă îi găsim, atunci răspunsul la problemă constă din concatenarea celor două măști.

Cum găsim cei doi vectori $P = (p_0, p_1, \ldots, p_{n-1})$ și $Q = (q_0, q_1, \ldots, q_{n-1})$? Procedăm ca la problema Lizard Era: Beginning. Nu ne pasă **care** este suma vectorilor, doar ca ea să fie **aceeași** pe toate pozițiile. De aceea, putem stoca doar diferențele față de primul termen, așadar

$$P' = (p_1 - p_0, p_2 - p_0, \ldots p_{n-1} - p_0)$$

și similar pentru Q'. Atunci trebuie să găsim doi vectori P' și Q' cu valori egale în modul, dar cu toate semnele schimbate. Într-adevăr, existența acestor doi vectori ar însemna că, pe orice poziție i,

$$p_i - p_0 = -(q_i - q_0)$$
$$p_i + q_i = p_0 + q_0$$

și deci toate sumele sînt egale. Și mai bine, putem nega toate valorile din Q', caz în care trebuie să găsim doi vectori perfect egali, cîte unul din fiecare jumătate.

De aici implementarea se ramifică. Putem pune jumătatea stîngă într-o tabelă hash de vectori,

în care apoi căutăm fiecare element din jumătatea dreaptă. Sursele mele procedează diferit. Ele pun ambele jumătăți în aceeași structură de date (un vector de `struct`-uri). Pentru fiecare vector de $n-1$ elemente reținem, suplimentar, și jumătatea din care provine. Apoi sortăm această structură după criteriul: `struct`-ul a vine înaintea `struct`-ului b dacă vectorul a este mai mic lexicografic decît vectorul b. Atunci, dacă există doi vectori egali, din jumătăți diferite, ei vor fi în mod necesar consecutivi. Putem compara naiv fiecare pereche de doi vectori consecutivi, în timp $\mathcal{O}(2^{16} \cdot n)$.

Pentru sortare, este important să evităm comparația naivă de vectori, care reîncepe compararea de la elementul 0. Sortarea ternară (*three-way radix quicksort*) se pretează mult mai bine.

Optimizare de memorie

A doua versiunea de sursă face o optimizare uriașă de memorie (de la 7 MB la 300 KB). Nu este nevoie să stocăm vectorii de diferențe de lungime $n-1$. Putem recalcula la cerere orice element cu cost $\mathcal{O}(1)$. Este nevoie să stocăm doar masca cu care am construit vectorul și jumătatea stîngă/dreaptă din care provine. Din nou, codul care consumă mai puțină memorie este mai rapid, deși face conversii suplimentare. Cine-ar fi crezut? 😉

15.2.7 Problema Light Switches (Codeforces)

enunț • sursă

Pornim de la observația elementară că două apăsări ale aceluiași buton se anulează reciproc. De aceea, vom apăsa fiecare buton cel mult o dată. Deci putem reformula problema astfel: pentru fiecare zi, trebuie să găsim un subset minim de butoane care, pornind din starea „zero” (toate becurile stinse) să ducă becurile exact în starea în care le lasă eroul în acea zi.

Putem gîndi totul în termeni de `bitset`-uri. Fie $x_1, \ldots, x_s$ bitseturile celor s butoane, unde $x_i[j] = 1$ dacă și numai dacă butonul i afectează becul j. Fie $y_1, \ldots, y_d$ bitseturile celor d zile. Atunci soluția pentru ziua i va fi un subset minimal al lui x al cărui xor să fie y_i.

Soluția în $\mathcal{O}(n \cdot d \cdot 2^s)$ ar fi: calculează xor-ul pentru fiecare submulțime a lui x, apoi pentru fiecare zi caută bitsetul zilei în mulțimea de bitseturi generate. Fiecare comparație este liniară, în $\mathcal{O}(n)$. Desigur, această soluție este ineficientă, dar ne dă ideea pentru *meet in the middle*:

1. Calculăm 2^l bitseturi pe care le putem obține doar cu primele l butoane. Le punem într-o tabelă hash (vom avea nevoie să căutăm prin ele). Pentru fiecare bitset reținem și cu cîte butoane l-am obținut. Dacă mai multe combinații de butoane duc la același bitset, reținem numărul minim de apăsări.
2. Calculăm 2^r bitseturi pe care le putem obține doar cu ultimele $r = n-l$ butoane. Le stocăm într-un vector simplu. Din nou, reținem numărul de apăsări.
3. Acum răspundem la interogări. Pentru bitsetul zilei curente, y, iterăm prin vectorul de 2^r elemente. Pentru fiecare element x din acest vector, căutăm valoarea $x \oplus y$ în tabela hash. Dacă îl găsim, însumăm numărul de apăsări necesare pentru $x \oplus y$, respectiv pentru

x. Minimul acestor sume este răspunsul pentru ziua curentă.

Complexitatea este $\mathcal{O}(2^l + d \cdot 2^r)$. Așadar, cele două jumătăți nu sînt neapărat egale. 🙂 Această situație îmi amintește de descompunerea în radical, unde blocurile pot devia considerabil de la $\sqrt{n}$ elemente.

Optimizări

Merită să deviem puțin de la subiectul zilei pentru a enumera multe idei de optimizare. Unele nu au contat, dar cele care au contat au redus timpul de 4 ori. Alte surse (neinteligibile...) reduc timpul de încă 8-9 ori!

1. Memoria este și ea un factor. Împărțirea 30=18+12 consumă de 4 ori mai puțină memorie decît împărțirea 30=20+10 și este doar cu 10% mai lentă.
2. Implementarea naivă folosește chei de tip `bitset`. Deci STL trebuie să știe să calculeze coduri hash pentru așa ceva. Oare cît de bine se descurcă? În versiunea 2 mi-am implementat propria funcție `hash_code` pentru bitseturi, exprimată ca $(r^{p_1} \oplus r^{p_2} \oplus \cdots \oplus r^{p_k}) \mod M$, unde p_i sînt pozițiile biților. Am ales $r = 37$, iar M este un număr prim pe 64 de biți. Probabilitatea unei coliziuni este infimă, căci folosim doar 2^{30} de coduri hash dintr-un spațiu de 2^{64}.
3. De la un punct mi-am implementat propriul bitset.
4. Am precalculat puterile r^i, ca să nu mai fac operații modulo în calculul codurilor hash.
5. La procesarea unei zile, cînd iterăm prin vectorul drept și observăm că deja folosim mai multe butoane decît soluția optimă găsită pînă la acel moment, nu mai are sens să facem și căutarea în tabela hash. Oricum nu putem obține o soluție mai bună. Putem chiar să sortăm bitseturile din vectorul drept după numărul de butoane apăsate.
6. Marele avantaj al funcției hash descrise mai sus este că, pentru două bitseturi A și B, avem $h(A \oplus B) = h(A) \oplus h(B)$. Asta înseamnă că putem menține incremental codul hash, odată la citirea bitsetului și ulterior la operațiile xor între bitseturi.
7. Odată ajunși aici, ne dăm seama că nu mai avem nevoie de bitseturi propriu-zise! Un bitset este doar un cod hash pe 64 de biți.

15.2.8 Problema Algebra Flash (Codeforces)

enunț • sursă

Problema are o soluție naivă în $\mathcal{O}(2^m \cdot n)$: generăm toate combinațiile de culori și, pentru fiecare combinație, verificăm dacă putem parcurge vectorul. Deci ideea de *meet in the middle* este să generăm, separat, combinații de cîte $m/2$ culori.

Dar chiar și așa, $\mathcal{O}(2^{m/2} \cdot n)$ va depăși timpul! Trebuie să scăpăm cumva de factorul de n. Observăm că o pereche de culori consecutive $\langle x, y \rangle$ înseamnă că trebuie să cumpărăm măcar una dintre culorile x și y. Numim această pereche o *relație* între x și y. Astfel ia naștere o matrice de relații de $m \times m = 1\,600$, mult mai puțin decît $n = 300\,000$. Matricea este chiar simetrică și are doar $m \times (m+1)/2 = 780$ de valori utile.

Am reprezentat matricea ca pe un vector de măști de 40 de biți, ceea ce paralelizează unele operații. De exemplu, dacă avem o mască de culori deja cumpărate a și masca b de relații a unei alte culori x, putem verifica ușor dacă x trebuie cumpărată obligatoriu: $b \wedge \neg a$. Astfel testăm dacă x are relații care nu apar în a. Dar cred că, și dacă nu sînteți versați în folosirea măștilor pe biți, puteți lua punctaj maxim. Unele operații ar fi de $m/2$ ori mai scumpe, dar s-ar încadra în timp.

La citirea datelor, am inclus explicit relațiile $\langle c_1, c_1 \rangle$ și $\langle c_n, c_n \rangle$. Este cel mai simplu mod de a forța cumpărarea culorilor pentru prima și ultima platformă. Tehnic, puteam să elimin liniile c_1 și c_n din calcule și să reduc problema la $m - 1$ sau $m - 2$ culori, dar ar fi necesitat cod în plus.

Acum, să procesăm cele două jumătăți ca pînă acum. Intenția generală este:

1. Generează toate combinațiile din jumătatea stîngă L, adică cu culorile din intervalul $[m/2, m)$.
2. Generează toate combinațiile din jumătatea dreaptă R, adică cu culorile din intervalul $[0, m/2)$.
3. Pentru fiecare combinație din L, găsește cea mai ieftină pereche din R astfel încît cele două să satisfacă:
   - toate relațiile între două culori din L;
   - toate relațiile între o culoare din L și una din R;
   - toate relațiile între două culori din R.

Să considerăm întîi L. Vom opera pe măști compuse cu biții $[m/2, m)$. Dar nu toate măștile ne interesează, ci doar cele care satisfac relațiile din L. În această privință, jumătatea R nu ne poate ajuta ulterior. Putem să generăm (recursiv sau iterativ) fiecare mască, iar la final să verificăm dacă ea satisface toate relațiile din L. Dar este mai eficient, și deloc greu, să verificăm validitatea pe parcursul recursivității. Observația este că, pentru orice pereche de biți/culori $\langle j, k \rangle$ cu $j < k$, dacă nu am cumpărat bitul j la vremea sa, trebuie să îl cumpărăm pe k. Așadar: dacă bitul curent, k, are vreo relație cu orice bit 0 din intervalul $[m/2, k]$, atunci bitul k poate avea doar valoarea 1. Altfel el poate avea valorile 0 sau 1, ca de obicei în recursivitate. Codul ar putea arăta astfel:

```
void rec(int k, int total_cost, u64 mask) {
  ...
  u64 unsat = rel[k] & bit_range(k, mid) & ~mask;
  if (!unsat) {
    // Sîntem liberi să alegem valoarea 0.
    rec(k + 1, total_cost, mask);
  }
  // Întotdeauna sîntem liberi să alegem valoarea 1.
  rec(k + 1, total_cost + cost[k], mask | (1ull << k));
}
```

Tot pe parcursul recursivității din L putem gestiona și relațiile dintre L și R. Cînd alegem valoarea 1 pentru un bit din L, aceasta nu spune nimic despre biții din R cu care el are relații. Aceia pot

fi 0 sau 1. În schimb, cînd alegem valoarea 0 pentru un bit din L, sîntem forțați să cumpărăm toți biții din R care au o relație cu el. Așadar, ori de cîte ori completăm o mască validă din L, vom fi completat și o mască din R, o submulțime pe care sîntem obligați să o cumpărăm.

Acum, fie $S \subseteq R$ masca cu restul biților din R, cei pe care (încă) avem libertatea să-i cumpărăm sau nu. Ce relații mai avem de satisfăcut? Doar cele între biți din S! Desigur, vrem să facem asta cu cost minim. Astfel ajungem, în sfîrșit, la întrebarea pe care vrem să i-o putem adresa fiecărei măști $S \subseteq R$: care este submulțimea cea mai ieftină a lui S pe care trebuie să o cumpărăm pentru a satisface toate relațiile dintre doi biți din S? Sursa mea numește acesta *costul redus* al măștii S și îl calculează tot într-o singură parcurgere.

Fie k bitul cel mai semnificativ (cu valoare 1) din S. Facem următoarele observații:

1. Costul redus minim se obține luînd minimul din două variante: fie cumpărăm bitul k, fie nu-l cumpărăm.
2. Dacă îl cumpărăm, îl plătim și reducem problema la aflarea costului redus al restului măștii.
3. Dacă nu îl cumpărăm, sîntem obligați să cumpărăm toți ceilalți biți din S cu care k are relații. Eliminăm toți acești biți și bitul k însuși din mască și reducem problema la aflarea costului redus al măștii rămase.

Sursa anexată calculează pasul (3) în $\mathcal{O}(1)$ per bit cumpărat. A doua variantă precalculează costul tuturor măștilor și obține timp $\mathcal{O}(1)$, dar merge la fel de repede.

15.2.9 Problema Helping Hiasat (Codeforces)

enunț • sursă

Problema seamănă considerabil cu Algebra Flash.

Problema Helping Hiasat seamănă bine cu cele discutate în clasă.

Mai întîi, să abstractizăm enunțul. Dacă Hiasat își schimbă username-ul, apoi este vizitat de k prieteni, îl poate mulțumi doar pe (cel mult) unul dintre ei. O primă idee care mi-a venit mie de aici este: pentru fiecare grup de vizite consecutive construim o mască X. Orice submască a lui X care are cel puțin doi biți setați este nefezabilă (prietenii corespunzători biților 1 nu pot fi mulțumiți simultan).

Dar putem simplifica această observație. Orice pereche de biți din X se exclud reciproc. Soluția poate include cel mult unul dintre ei. Așadar, putem construi o matrice booleană de $m \times m$ cu perechile de biți / prieteni care se exclud reciproc. Am ales să stochez această matrice ca pe m variabile `long long`, căci este util ca fiecare bit să aibă o mască a biților cu care se exclude.

Problema se reduce la găsirea unei mulțimi independente maxime.

Soluția în $\mathcal{O}(2^m)$ este acum directă. Parcurgem recursiv toți biții. Avem întotdeauna opțiunea să nu alegem bitul k. Avem și opțiunea să îl alegem doar dacă el nu se exclude cu niciun bit din masca biților aleși pînă acum.

Pentru *meet in the middle*, ne închipuim că am rezolvat subproblema pentru jumătatea stîngă și am obținut cel mult 2^{20} măști fezabile. Cînd rezolvăm jumătatea dreaptă, ori de cîte ori obținem o mască fezabilă acolo, Y, vrem să ne întrebăm: **Care este populația maximă a unei măști fezabile, X, din stînga care nu intră în conflict cu Y?**

Aici este singurul pas specific problemei. Trebuie să ne asigurăm că niciun bit din X nu se exclude cu niciun bit din Y-ul curent. De aceea, pe măsură ce compunem (recursiv, ca de obicei) masca Y vom construi și „masca excluziunilor": masca tuturor biților din stînga care devin interziși. Cînd adăugăm un bit k la masca dreaptă Y, masca excluziunilor lui Y crește (cu OR) cu masca excluziunilor lui k.

Ultima subproblemă de rezolvat este: am completat o mască Y, îi știm masca excluziunilor și, prin negație, obținem o mască Z cu biții din stînga care ne sînt încă permiși. Acum dorim să obținem rapid submasca fezabilă a lui Z de populație maximă. Pe aceasta o precalculăm cu programare dinamică pe biți, imediat după recursivitatea stîngă și înainte de a da drumul la cea dreaptă.

Partea VI

Arbori

Capitolul 16

Unelte și algoritmi esențiali

Acest curs presupune deja cunoscute reprezentarea arborilor și parcurgerile DFS și BFS. De aceea, în acest capitol vom prezenta doar o unealtă pentru depanare (generatoarele de arbori) și vom rezolva cîteva probleme de bază.

16.1 Generatoare de arbori aleatorii

Generarea unor arbori aleatorii este și interesantă din punct de vedere teoretic, dar vă poate ajuta și la depanarea problemelor pe arbori.

Dacă nu ne interesează forma arborelui, generatoarele de arbori sînt la fel de ușor de scris ca generatoarele de vectori + interogări. Iată un generator de bază care generează un arbore cu n noduri și k interogări de tip pereche de noduri. L-am folosit la problema Max Flow (USACO).

În schimb, dacă dorim să testăm viteza unei soluții, sau dacă compunem o problemă și dorim date de test greu de fentat, avem nevoie ca:

- Arborele să aibă un lanț foarte lung.
- Arborele să aibă un nod cu foarte mulți vecini.
- Aceste structuri să nu poată fi decelate din datele de intrare (de exemplu, lanțul să nu conțină fix nodurile $1, 2, \ldots, n/2$.
- Generarea să dureze $\mathcal{O}(n)$.

Iată un generator avansat care răspunde acestor nevoie și poate genera, la cerere, și interogări sau actualizări. El poate fi apelat în trei moduri și poate fi ușor adaptat la altele:

1. Doar structura arborelui, fără valori în noduri, fără operații.
2. Valori inițiale în fiecare nod, actualizări în noduri, interogări în noduri.
3. Fără valori în noduri, interogări pe căi (perechi de noduri).

El acceptă trei parametri pentru definirea structurii:

1. n: numărul de noduri;
2. c: lungimea minimă garantată a cel puțin unui lanț;

3. d: gradul minim garantat al cel puțin unui nod.

Generatorul funcționează astfel:

- Generează o permutare aleatorie a nodurilor.
- Unește primele c noduri în lanț.
- Unește următoarele $n - d - c$ noduri de noduri dinaintea lor, alese aleatoriu.
- Unește ultimele d noduri de un nod dintre primele $n - d$, ales aleatoriu.
- După ce a generat cele $n - 1$ muchii, le amestecă, astfel încît primele $c - 1$ muchii tipărite să nu formeze un lanț etc.

16.2 Probleme

16.2.1 Problema Subordinates (CSES)

enunț • surse

Cerința este clasică: aflarea mărimii subarborelui fiecărui nod. Am inclus două implementări: cu vectori STL și cu liste înlănțuite scrise de la zero. Implementarea cu liste este de două ori mai rapidă.

16.2.2 Problema Tree Matching (CSES)

enunț • sursă

Problema se rezolvă cu un singur DFS. Ea este un bun exemplu de raționament recursiv pe arbore. Ca în multe alte situații, putem trata nodul curent în relație cu fiul său (dacă nodul și fiul sînt necuplați, cuplează-i) sau în raport cu părintele (dacă nodul și părintele sînt necuplați, cuplează-i).

16.2.3 Problema Tree Diameter (CSES)

enunț • surse

Există două soluții relativ diferite. Una face două parcurgeri DFS (puteți citi aici o demonstrație de corectitudine):

- Facem un DFS pornind din orice nod x. Fie a nodul cel mai depărtat de x.
- Facem un al doilea DFS pornind din a. Fie b nodul cel mai depărtat de a.
- $a \rightsquigarrow b$ este unul dintre diametrele arborelui.

Soluția cu o singură parcurgere este cu 20% mai rapidă (ambele neoptimizate). Implementăm recursiv observația că diametrul constă din două lanțuri descendente care pornesc dintr-un strămoș comun u. (Este ușor de tratat și cazul cînd diametrul este doar un lanț pornind din rădăcină, considerînd atunci al doilea lanț ca fiind rădăcina însăși, o cale de lungime zero.) Atunci fiecare nod are două sarcini:

- Să actualizeze un maxim global cu suma maximă a căilor raportate de oricare doi fii distincți. La fiecare cale, u adaugă 1 (muchia care pleacă din u însuși).
- Să raporteze la părinte distanța maximă de la u pînă la orice frunză din subarbore.

Ambele soluții trebuie să fie scurte (5-6 linii în plus față de șablonul de declarații, citire, DFS).

Discuție secundară: pentru claritate, `diam` nu trebuia să fie variabilă globală, ci DFS-ul trebuia să-l returneze, ca maxim dintre valorile raportate de fii. Pentru viteză, însă, și pentru economisirea memoriei pe stivă, cred că putem face concesia să-l declarăm pe `diam` global. Cititorii mai pedanți decît mine 😅 pot încapsula `diam` și parcurgerea DFS într-o clasă.

16.2.4 Problema Tree Distances II (CSES)

enunț • sursă

Pentru un nod fixat (să zicem 1), putem calcula răspunsul cu un singur DFS. Apoi, vom introduce un concept întîlnit ocazional: recalcularea unei valori la schimbarea rădăcinii. Să spunem că suma cerută pentru nodul u este S_u și dorim să o calculăm pe S_v pentru nodul v, vecin cu u. Să spunem că, raportat la muchia $u-v$, de partea lui u avem n_u noduri, iar de partea lui v avem $n_v = n - n_u$ noduri. Atunci, ca să trecem din S_u în S_v, observăm că:

- pentru n_v noduri distanțele scad cu 1;
- pentru n_u noduri distanțele cresc cu 1;

Rezultă tranziția simplă $S_v = S_u - n_v + n_u = S_u + n - 2n_v$.

Dacă facem toate tranzițiile dinspre rădăcina inițială (1) spre fii, atunci n_v va fi întotdeauna mărimea subarborelui lui v.

16.2.5 Problema White-Black Balanced Subtrees (Codeforces)

enunț • sursă

Includ această problemă pentru a discuta o altă parcurgere decît DFS, în cazul în care arborele este dat printr-un vector de părinți. Am ales o problemă simplă. Pentru una cu mai multă substanță, vedeți Arbsumpow (baraj ONI 2021).

Avantajul acestei reprezentări este că simplitatea și memoria redusă: $n-1$ întregi în loc de $2(n-1)$.

Desigur, puteți converti formatul dat la cel obișnuit (liste de vecini). Dar uneori vectorul de părinți este suficient. În multe probleme trebuie să calculăm pentru fiecare nod o valoare care depinde, recurent, de valorile fiilor. Atunci putem folosi codul:

```
for (int u = 1; u <= n; u++) {
  scanf("%d", &p[u]);
  num_children[p[u]]++;
}
```

```
for (int u = 1; u <= n; u++) {
  int s = u;
  while (s && !num_children[s]) {
    process(s);  // raportează la părinte ce trebuie raportat
    s = p[s];
    num_children[s]--;
  }
}
```

Rolul funcției `process` este să raporteze la părinte informațiile dintr-un nod. Pe parcurs, valoarea `num_children[u]` arată cîți fii ai lui `u` încă nu și-au raportat informațiile. Astfel, cînd îi va veni rîndul părintelui să fie procesat, el va fi acumulat informațiile de la toți fiii.

Ordinea nodurilor este *bottom-up*. Programul încearcă vizitarea fiecărui nod de cel mult două ori: fie cînd îi vine rîndul în ordine numerică, fie în momentul în care ultimul său fiu este vizitat.

16.2.6 Problema Blood Cousins (Codeforces)

enunț • sursă

Soluția propusă în editorial, în $\mathcal{O}(q \log n)$, procedează astfel.

- Măsoară timpii DFS și adîncimea fiecărui nod.
- Pentru a afla eficient al p-lea strămoș, construiește în fiecare nod lista strămoșilor de gradele $1, 2, \ldots, 2^k$.
- Colectează nodurile arborelui, în ordinea DFS, în liste separate pentru fiecare adîncime. Așadar $L[0]$ va conține doar rădăcina, $L[1]$ va conține fiii rădăcinii etc.
- Pentru a afla numărul de descendenți la distanță p ai unui nod u care are timpii DFS $t_i[u]$ și $t_o[u]$, căutăm binar în lista $L[d[u] + p]$ primul și ultimul nod cu timpi DFS între $t_i[u]$ și $t_o[u]$.

Iată și o soluție în timp liniar. Mi se pare mai simplă, căci folosește doar liste și două parcurgeri DFS.

- Distribuim interogările (v, p) în liste după nodul v.
- Rulăm un DFS în care menținem o stivă explicită de noduri în curs de vizitare. Concret, la intrarea în nodul u la adîncime $d[u]$ notăm $st[d[u]] = u$.
- La intrarea în nodul v, procesăm toate interogările (v, p). Al p-lea strămoș al lui v este $st[d[v] - p]$ (dacă există). Înlocuim fiecare p cu acest strămoș.

Astfel, după un prim DFS, toate interogările au luat forma: cîți descendenți are nodul v la adîncime p? Vom scădea 1 pentru a răspunde la cerința problemei. Putem răspunde la aceste întrebări cu un al doilea DFS.

- Redistribuim interogările (v, p) în liste după noul nod v.

- Rulăm un DFS în care menținem numărul de noduri vizitate pe fiecare nivel. Concret, la intrarea în nodul u la adîncime $d[u]$ îl incrementăm pe $st[d[u]]$.
- Acum răspunsul pentru o interogare (v, p) este diferența între $st[d[v] + p]$ la intrarea și la ieșirea din nodul v. Iterăm prin toate interogările referitoare la nodul v la intrarea și la ieșirea din recursivitate.

Capitolul 17

Liniarizarea arborilor

Să considerăm un arbore cu n noduri și cu valori în noduri. Liniarizarea acestui arbore este o parcurgere DFS care calculează informații suplimentare. Putem folosi aceste informații ca să gestionăm operații pe arbore, cum ar fi:

- Modificarea valorii unui nod sau a unei muchii.
- Modificarea valorilor pe calea de la un nod la rădăcină.
- Modificarea valorilor în tot subarborele unui nod.
- Interogări despre valoarea unui nod.
- Interogări despre valorile pe calea de la un nod spre rădăcină.
- Interogări despre valorile din subarborele unui nod.

17.1 Timpi de intrare și de ieșire din DFS

Să considerăm următoarea parcurgere DFS. Ea este elementară, cu o singură noutate: menține un contor global pe care îl incrementează la intrarea și la ieșirea dintr-un nod. (Din motive inginerești, ca să evidențiez că variabila `time` este folosită doar în funcția `dfs`, nu am declarat-o globală, ci statică în funcție. Efectul este același.)

```
void dfs(int u, int parent) {
  static int time = 0;

  time_in[u] = ++time;
  for (int v: adj[u]) {
    if (v != parent) {
      dfs(v, u);
    }
  }
  time_out[u] = ++time;
}
```

Dacă vreți, `time` este un metronom care bate la fiecare parcurgere a unei muchii, fie în jos (la

intrarea într-un nod), fie în sus (la ieșirea dintr-un nod). Toate valorile din vectorii `time_in` și `time_out` vor fi distincte și cuprinse între 1 și $2n$. Iată un exemplu.

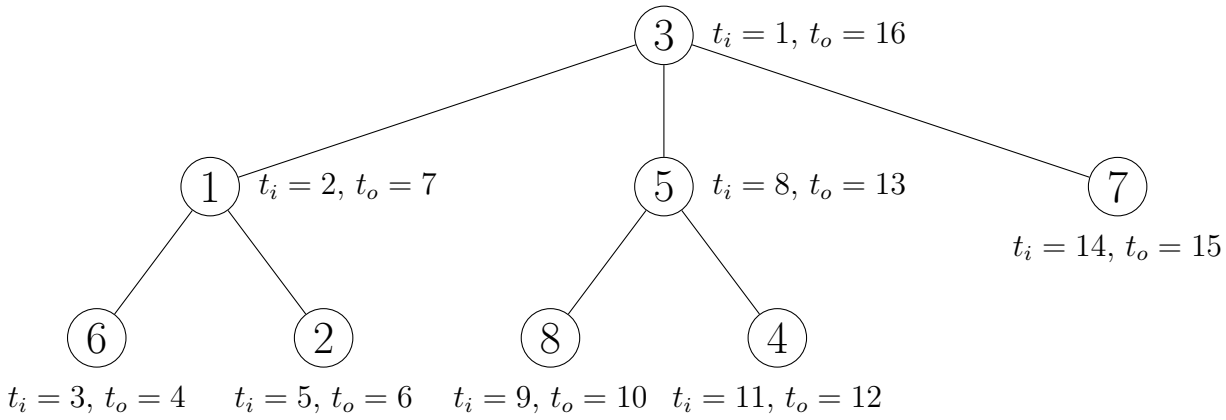


Figura 17.1: Timpii de intrare și de ieșire din noduri, varianta 1.

Într-o altă variantă, incrementăm timpul numai la intrarea în nod, nu și la ieșire. Atunci în vectorii t_i și t_o vom avea valori între 1 și n. Iată aceste valori pentru același arbore:

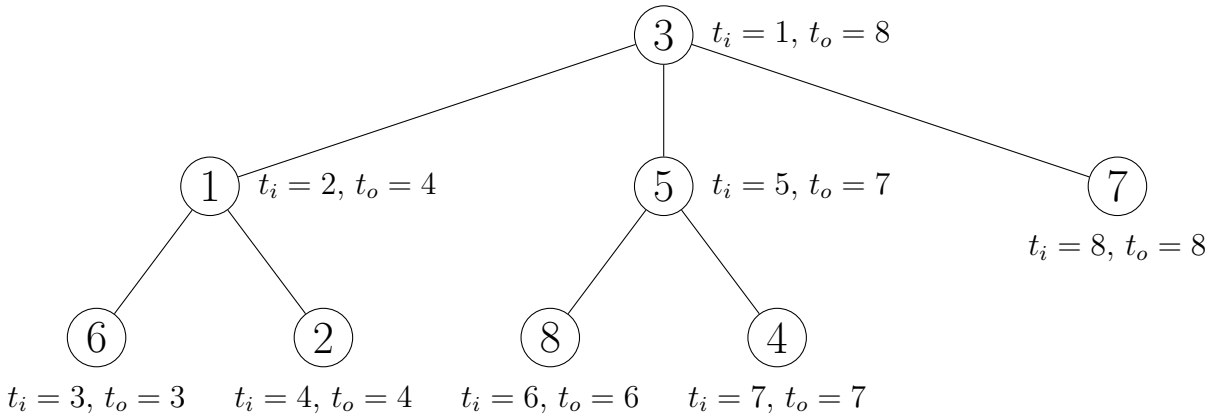


Figura 17.2: Timpii de intrare și de ieșire din noduri, varianta 2.

17.2 Testul de strămoș

O aplicație directă a timpilor de intrare și de ieșire este că putem decide în $\mathcal{O}(1)$ dacă un nod u este strămoș al altui nod v. Este necesar ca $t_i[u] < t_i[v] < t_o[v] < t_o[u]$. Inegalitatea poate fi strictă ($<$) sau permisivă ($\leq$) în funcție de varianta aleasă mai sus și dacă dorim ca u să fie considerat propriul său strămoș sau nu.

17.3 Liniarizarea. Tipuri de liniarizare

Liniarizarea unui arbore este un vector. Obținem acest vector printr-o parcurgere DFS în care emitem, la anumite momente, numărul nodului curent. Acest instrument puternic ne permite

să rezolvăm probleme pe arbori folosind structuri familiare pe vectori: AIB, arbori de segmente, căutări binare, algoritmul lui Mo...

Există trei tipuri de liniarizări, foarte asemănătoare și la fel de ușor de obținut. Fiecare este utilă în alte situații.

17.3.1 Liniarizarea DFS

În această liniarizare, emitem nodul curent (adică îl adăugăm la vectorul rezultat) cînd intrăm în nod. Pentru arborele din Figura 17.2, vectorul este:

poziție	1	2	3	4	5	6	7	8
nod	3	1	6	2	5	8	4	7

Tabela 17.1: Liniarizarea de tip DFS.

Cu alte cuvinte, această liniarizare constă din nodurile arborelui în ordinea în care le descoperă DFS-ul. Tocmai de aceea, fiecare nod u apare în vector la poziția $t_i[u]$ (în varianta 2, din figura 17.2). De altfel, pentru acest tip de liniarizare în practică nu vom construi efectiv vectorul, ci doar vectorii t_i și t_o.

O observație importantă pentru toate liniarizările este că subarborele oricărui nod corespunde unui interval contiguu din vector. De exemplu, nodurile 5, 8 și 4 apar pe poziții consecutive. În liniarizarea DFS, subarborele oricărui nod u acoperă pozițiile dintre $t_i[u]$ și $t_o[u]$ inclusiv. De exemplu, subarborele nodului 5 ocupă pozițiile de la 5 la 7. Pentru a exploata această structură, în probleme de arbori cu valori în noduri vom stoca într-un vector valoarea fiecărui nod u pe poziția $t_i[u]$. Atunci, folosind structurile cunoscute pe vector, putem face actualizări și interogări pe subarbori întregi.

17.3.2 Liniarizarea Euler

În această liniarizare, emitem nodul curent de două ori: la intrare și la ieșire. Pentru arborele din Figura 17.1, vectorul este:

poziție	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
nod	3	1	6	6	2	2	1	5	8	8	4	4	5	7	7	3

Tabela 17.2: Liniarizarea de tip Euler.

Observăm că pozițiile nodului u în vector sînt $t_i[u]$ și $t_o[u]$. Vom studia probleme care arată cum folosim această liniarizare pentru a admite actualizări și interogări pe calea de la rădăcină la un nod oarecare u.

17.3.3 Liniarizarea Euler cu repetiție

Pe Internet nu am găsit o distincție clară între liniarizarea anterioară și aceasta, așa că am inventat un nume, sper că acceptabil. În această liniarizare, emitem nodul curent la intrare și la revenirea

din fiecare fiu. În particular, emitem frunzele o singură dată. Pentru arborele din Figura 17.1, vectorul este:

poziție	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
nod	3	1	6	1	2	1	3	5	8	5	4	5	3	7	3

Tabela 17.3: Liniarizarea de tip Euler cu repetiție.

Lungimea acestei liniarizări este întotdeauna $2n-1$. Emitem fiecare nod la intrare, ceea ce ocupă n poziții. În plus, emitem fiecare nod de atîtea ori cîți fii are. Dar suma numărului de fii pentru toate nodurile este $n-1$, deoarece fiecare nod este fiul cuiva, cu excepția rădăcinii.

Folosim această liniarizare cu preponderență pentru interogări de LCA (engl. *lowest common ancestor*), pe care le vom studia în capitolul 19.

17.4 Probleme

17.4.1 Problema Tree Queries (Codeforces)

enunț • sursă

Probabil am putea implementa următoarea soluție. Pentru fiecare interogare, pornim din cel mai de jos dintre noduri, mergînd pînă la rădăcină, și vedem dacă întîlnim toate celelalte noduri sau părinții acestora. Dar această soluție necesită $\mathcal{O}(n)$ per interogare pentru un arbore degenerat. Probabil ea poate fi adusă la $\mathcal{O}(\sqrt{n})$ sau $\mathcal{O}(\log n)$ prin tehnici mai complicate.

În loc de acesta, să procedăm astfel. Fie u cel mai de jos nod dintr-o interogare. Atunci toate celelalte noduri **sau** părinții lor trebuie să se afle pe calea de la u la rădăcină. Cu alte cuvinte, să fie strămoși ai lui u. Mai mult, decizia se simplifică astfel: dacă un nod v este pe calea de la u la rădăcină, atunci și părintele lui v se va afla pe calea de la u la rădăcină. Deci este suficient să testăm doar părinții și să răspundem la întrebarea: Sînt părinții tuturor nodurilor dintr-o interogare strămoși ai celui mai de jos nod? (Complicația cu noduri și părinți este mai mult un *red herring*, o diversiune care poate păcăli pe cineva.)

Vom folosi testul de strămoș studiat anterior: v este strămoș al lui u dacă și numai dacă $t_i[v] < t_i[u] < t_o[u] < t_o[v]$.

Detalii de implementare

Am preferat să declar variabila `time` statică, în interiorul funcției `euler_tour`, ca să evidențiez că nu este folosită altundeva.

Nodurile dintr-o interogare nu trebuie stocate, sortate etc. Nu avem nevoie de adîncimea nodurilor. Pur și simplu îl reținem pe cel mai de jos găsit pînă atunci, fie el l. Cînd citim un nod nou, u, îi aflăm părintele p. Pot lua naștere trei cazuri:

- Dacă p este strămoș al lui l, atunci l nu se modifică, iar calea care conține toate nodurile de pînă acum rămîne calea de la l la rădăcină.
- Dacă l este strămoș al lui p, atunci p îl înlocuiește pe l, iar calea care conține toate nodurile de pînă acum este calea de la p la rădăcină.
- Altfel nu există nicio cale care să le conțină pe l și pe p, deci răspunsul la interogarea curentă este `NO`.

17.4.2 Problema Subtree Queries (CSES)

enunț • sursă

Această problemă este o aplicație directă a liniarizării. Facem o liniarizare de tip DFS pentru a calcula doar vectorul t_i (timpul de intrare în fiecare nod). Apoi construim un vector indexat după timp, în care notăm valoarea fiecărui nod u la poziția $t_i[u]$. Peste acest vector construim un simplu arbore Fenwick sau orice structură preferați care să ofere actualizare punctuală + sumă pe interval.

O altă variantă clasică a acestei probleme ar fi ca operațiile de actualizare să fie pe subarbore: atribuie tuturor nodurilor din subarborele lui u o valoare x, incrementează toate nodurile cu o cantitate x etc. Desigur, aceste operații s-ar traduce în operații de actualizare pe interval, pentru care putem folosi un arbore de intervale cu propagare *lazy*.

17.4.3 Problema Path Queries (CSES)

enunț • sursă

Această problemă ne arată utilitatea liniarizării Euler. Să considerăm momentul în care DFS-ul intră în nodul u, adică $t_i[u]$. El corespunde prefixului din liniarizare de la poziția 1 pînă la poziția $t_i[u]$. Observăm că:

1. Nodurile complet vizitate înainte de intrarea în u (numite și *noduri negre*) apar de cîte două ori.
2. Nodurile în curs de vizitare (calea de la u la rădăcină, numite și *noduri gri*) apar cîte o dată.
3. Nodurile încă nevizitate (numite și *noduri albe*) nu apar niciodată.

Putem exprima punctul (2) și în termeni de strămoși, discutați anterior: strămoșii lui u sînt singurele noduri ale căror intervale cuprind intervalul lui u.

De aici rezultă cum putem folosi paritatea aparițiilor ca să notăm informații de pe calea spre rădăcină. Notăm valoarea fiecărui nod v cu semnul + la poziția $t_i[v]$ și cu semnul - la poziția $t_o[v]$. Astfel, suma prefixului $[1, t_i[u]]$ va fi exact suma valorilor pe calea de la u la rădăcină.

17.4.4 Problema New Year Tree (Codeforces)

enunț • sursă

Problema este relativ directă, doar cu puțin mai complicată decît precedentele. După liniarizare, avem nevoie de o structură de date care să admită operațiile:

1. `range_set(l, r, val)`: scrie valoarea val pe tot intervalul $[l, r]$
2. `range_count_distinct(l, r)`: returnează numărul de valori distincte din intervalul $[l, r]$.

Problema numărării de elemente distincte cu actualizări nu este trivială, dar varianta de față are un avantaj: valorile sînt între 1 și 60, deci putem folosi măști de biți. Rezultă că avem nevoie de un arbore de intervale cu propagare *lazy*, cu operațiile:

1. atribuiri pe interval;
2. OR pe biți pe interval.

17.4.5 Problema Max Flow (USACO)

enunț • surse

Metoda 1: Vectori de diferențe pe arbore

O primă soluție aduce cu vectori de diferențe („șmenul lui Mars"), dar în varianta arborescentă. Pentru fiecare nod dorim să calculăm numărul de căi care trec prin acel nod. Atunci, pentru fiecare cale (u, v), dorim să incrementăm valorile tuturor nodurilor de pe cale. Fie w cel mai de jos strămoș comun (LCA) al perechii (u, v). Atunci dorim, echivalent,

- Să incrementăm toate valorile de la u la rădăcină.
- Să incrementăm toate valorile de la v la rădăcină.
- Să scădem cu 1 valoarea lui w (care a fost incrementat de două ori).
- Să scădem cu 2 valorile de la părintele lui w la rădăcină.

Putem face asta adunînd ± 1 în nodurile respective, apoi propagînd în sus acele valori. Implementarea este lungă întrucît trebuie să calculeze LCA și am ales să fac asta cu algoritmul lui Tarjan, cel mai eficient în situația offline (vom detalia în viitorul apropiat).

Metoda 2: AIB peste liniarizare

La momentul vizitării nodului u, putem împărți arborele în 3 zone disjuncte:

1. Zona 1 este porțiunea vizitată înainte de u.
2. Zona 2 este subarborele lui u.
3. Zona 3 este porțiunea care va fi vizitată după revenirea din u.

Atunci, căile care trec printr-un nod u sînt de trei feluri:

1. Cele care încep în zona 1 și se termină în zona 2.
2. Cele care încep în u, indiferent dacă se termină în zona 2 sau în zona 3.
3. Cele care pornesc dintr-un descendent al lui u și trec prin u, fie că se termină în alt descendent al lui u, fie în zona 3.

Pentru a afla aceste informații pentru fiecare nod, facem întîi o liniarizare de tip DFS imediat ce citim arborele. Reamintim că subarborele fiecărui nod ocupă un interval contiguu în această liniarizare.

Abia acum citim restul datelor (căile). Pentru fiecare nod u colectăm lista de căi care încep din u și lista de căi care se termină în u. Mai exact, pentru fiecare cale (u, v), cunoscînd liniarizarea, ne asigurăm că $t_i[u] < t_i[v]$ (le interschimbăm la nevoie). Apoi adăugăm v la lista de căi care pornesc din u și u la lista de căi care se termină în v.

Acum putem rula o a doua parcurgere DFS în care ținem evidența căilor active. La intrarea într-un nod marcăm ca active căile care pornesc din u. La ieșire, marcăm ca inactive căile care se termină în u (adică le ștergem din evidență).

Care este structura necesară pentru această evidență? De exemplu, pentru întrebarea (1) de mai sus avem nevoie să știm ce căi active se termină în zona 2. Putem folosi un arbore Fenwick peste liniarizare în care notăm $+1$ la momentul nodului de sfîrșit al căilor active. Atunci răspunsul la (1) este suma pe $[t_i[u], t_o[u]]$.

Pentru (2) răspunsul este pur și simplu lungimea listei de căi care încep în u.

Pentru (3), la revenirea dintr-un fiu v avem nevoie să știm ce căi active au început în subarborele lui v. De aceea, vom folosi un al doilea arbore Fenwick în care notăm $+1$ la momentul nodului de început al căilor active.

Această metodă nu mai necesită calculul LCA.

Metoda 3 (cred): *Small to large*

Cred că fiecare nod își poate menține colecția de căi care trec prin el. Un părinte va unifica aceste colecții, eliminîndu-le pe cele care apar de două ori (pentru care părintele este LCA). Dacă folosim cea mai mare colecție dintre cele ale fiilor, timpul rezultat va fi $\mathcal{O}(n + q \log q)$.

17.4.6 Problema Distinct Colors (CSES)

enunț • sursă

După liniarizare și după normalizarea culorilor, problema se reduce destul de direct la numărarea culorilor distincte dintr-un interval. O variantă este cu algoritmul lui Mo, în $\mathcal{O}(n\sqrt{n})$. O altă variantă posibilă este cu tehnica *small to large* (vom reveni cînd studiem tehnica).

Soluția mai eficientă, în $\mathcal{O}(n \log n)$, este cea studiată în capitolele anterioare (problema D-query). Procesăm interogările în ordinea crescătoare a capătului drept. Pe parcursul procesării, într-un AIB ținem valori de 1 pentru poziția celei mai din dreapta apariții a fiecărei culori.

În practică implementarea se simplifică mult, căci interogările pe liniarizare au o structură foarte specială. Dacă răspundem la interogări la revenirea din nod, atunci în mod evident ele vor fi ordonate după capătul drept, deoarece capătul drept este dat de timpul curent din DFS, care poate doar să crească! De aceea,

1. Nu mai este nevoie să colectăm și să sortăm interogările. Le putem procesa chiar în DFS.
2. Putem renunța complet la stocarea timpilor de intrare în nod.
3. Nu (prea) mai are sens să normalizăm culorile în prealabil. Putem extinde AIB-ul să se ocupe de asta.

De aceea vă încurajez mereu să adaptați cunoștințele voastre la particularitățile problemei. Per ansamblu, veți cîștiga timp și eficiență.

17.4.7 Problema Arb (Lot 2008)

enunț • surse

Problema este de lot, dar rezolvabilă cu liniarizare și chiar și fără, doar cu tehnici de bază pe arbori. 2008 erau vremuri mai simple...

Soluția cu liniarizare

Noi știm deja să construim un AIB peste liniarizarea DFS pentru a face interogări pe subarbore. Dar problema ne cere sume peste vîrful subarborelui, la o adîncime dată. Acel vîrf nu mai ocupă neapărat un interval contiguu în liniarizare. Cum ne adaptăm?

Ne reîntîlnim cu o tehnică pe care sper că v-ați amintit-o. Vom adăuga valorile în AIB într-o anumită ordine astfel încît, pentru orice interogare, să existe un moment cînd intervalul subîntins de subarbore să conțină doar valorile necesare pentru interogare, nu și pe cele de mai jos în subarbore.

Care ar fi această ordine? Ea pare să aibă legătură cu adîncimea nodurilor. Concret, facem întîi o liniarizare DFS în care doar notăm timpii de intrare și de ieșire din noduri (așadar, intervalele subîntinse). Apoi, începem să inserăm valorile din noduri în AIB în ordinea adîncimii nodurilor. Există două metode pentru a face asta, ambele rezonabil de ușoare:

1. Fie pe parcursul liniarizării colectăm și nodurile în liste după adîncimea lor.
2. Fie facem un BFS clasic, cu coadă.

Să considerăm o interogare (u, h) referitoare la subarborele lui u la adîncime cel mult h. Dacă adîncimea lui u este $d[u]$, atunci momentul optim pentru a răspunde la această interogare este după ce am inserat în AIB toate nodurile pînă la adîncimea $d[u] + h$ inclusiv. În acel moment, o interogare în AIB pe intervalul $[t_i[u], t_o[u]]$ va aduna contribuția 0 pentru nodurile aflate la adîncime mai mare decît $d[u] + h$.

Soluția fără liniarizare

Dacă vreți, soluția anterioară are restricții pe două dimensiuni. Ea cere suma nodurilor (1) cu timpi între $t_i[u]$ și $t_o[u]$ și (2) cu adîncimi între $d[u]$ și $d[u] + h$. Așa cum ne amintim de la lecția despre AIB-uri, **am procesat nodurile în ordinea adîncimii și am indexat AIB-ul după timp**.

Dar tot la lecția de AIB-uri spuneam că putem trata invers dimensiunile. Concret, la problema Little Artem and Time Machine inversarea dimensiunilor ne scăpa de nevoia unei normalizări. Așadar, **vom indexa AIB-ul după adîncime și vom procesa nodurile în ordinea timpului**. Mai exact, vom parcurge nodurile cu un banal DFS, iar cînd adăugăm valoarea unui nod în AIB (chiar la intrarea DFS-ului în acel nod), o adăugăm la poziția dată de adîncimea nodului.

Atunci, cînd putem răspunde la interogarea (u, h)? Răspunsul la interogare constă din suma modificărilor din AIB pe pozițiile de la $d[u]$ la $d[u] + h$, dar numai între intrarea și ieșirea din nodul u. Desigur, putem calcula această cantitate ca pe o diferență: calculăm suma din AIB la intrarea în nod, la ieșirea din nod, apoi le scădem între ele.

Implementarea este foarte comparabilă ca viteză cu prima, dar mai scurtă.

O soluție în $\mathcal{O}(n + q)$

De la unii dintre elevi (1, 2 etc.) am învățat o soluție în $\mathcal{O}(n + q)$! Ea combină ideile celor două soluții anterioare. Răspunsul pentru interogarea (u, h) este suma subarborelui lui u minus suma sumelor subarborilor tuturor descendenților lui u la adîncime $d[u] + h + 1$. Menținem un vector în care, pe fiecare nivel, însumăm sumele subarborilor ale căror rădăcini sînt pe acel nivel. Consultăm acel vector la intrarea și la ieșirea din nodul u, pentru fiecare interogare despre nodul u.

Timpul de rulare nu este semnificativ mai mic. Fișierele de intrare sînt respectabile (7-8 MB), ceea ce mai atenuează timpii de rulare, iar AIB-urile au constantă bună. Dar asta nu ne știrbește plăcerea de a reduce complexitatea asimptotică.

17.4.8 Problema Disconnect (Infoarena)

enunț • sursă

Precizez că testele par incorect alese. Soluțiile cele mai rapide traversează naiv arborele. Ne facem că n-am observat. 😀

O rezolvare directă este similară cu problema Max Flow. Marcăm muchiile șterse cu 1. Apoi, două noduri sînt conectate dacă suma pe calea dintre ele este 0. Pentru a calcula suma pe o cale, calculăm sumele de la fiecare nod pînă la rădăcină, minus dublul sumei de la LCA la rădăcină.

Iată și o altă rezolvare, care evită nevoia de a calcula LCA. Alegem o rădăcină oarecare (1). Cînd ștergem o muchie, marcăm nodul de adîncime mai mare ca șters. Acum definim un concept clasic: pentru orice nod u, fie $LMA(u)$ (engl. *lowest marked ancestor*) cel mai de jos nod marcat pe calea de la u la rădăcină, inclusiv.

Pentru a răspunde la o interogare de conectivitate (u, v), considerăm 4 cazuri:

1. $LMA(u)$ și $LMA(v)$ sînt nedefinite. Atunci, desigur, calea este liberă și u și v sînt conectate.
2. Exact unul dintre $LMA(u)$ și $LMA(v)$ este definit. Atunci nodul marcat există undeva mai jos de $LCA(u, v)$, așadar el deconectează u de v.

3. $LMA(u)$ și $LMA(v)$ există și sînt egale. Atunci nodul marcat există undeva mai sus de $LCA(u, v)$, deci nu există alte obstacole. u și v sînt conectate.
4. $LMA(u)$ și $LMA(v)$ există și sînt diferite. Atunci fiecare nod are propriul său blocaj mai jos de $LCA(u, v)$, deci nodurile sînt deconectate.

Condiția este mai ușor de testat decît pare. Dacă lăsăm $LMA(u) = 0$ pentru valori nedefinite, atunci trebuie doar să testăm dacă $LMA(u) = LMA(v)$.

Cum menținem informația de LMA? Este suficient un singur arbore de segmente, fără propagare *lazy*, construit peste liniarizare. La marcarea unui nod u ca șters, marcăm tot intervalul subîntins de u chiar cu valoarea u. La interogarea $LMA(v)$, pornim din frunza din AINT corespunzătoare nodului v și raportăm prima valoare întîlnită (sau 0 dacă nu întîlnim valori scrise).

Mai există un singur aspect important. Un nod din AINT poate fi acoperit de mai multe noduri din arbore. Dacă mai multe actualizări (ștergeri) acoperă același nod, cînd suprascriem valori și cînd nu? Este important că nodurile respective din arbore sînt toate în relația strămoș-descendent. De aceea, cele mai de jos au prioritate. Echivalent, cele care subîntind un interval mai mic în AINT au prioritate. AINT-ul procedează exact astfel: acordă valorilor scrise priorități egale cu lungimea intervalului scris.

Capitolul 18

Tehnica *small-to-large*

18.1 Generalități

Există o clasă de probleme pe arbori care, implementate naiv, au complexitate $\mathcal{O}(n^2)$. În aceste probleme, informația pentru un nod are mărime $\mathcal{O}(n)$ și trebuie compusă din informațiile fiilor. Exemplu: fiecare nod u dorește să-și cunoască frecvențele adîncimilor nodurilor din subarbore, raportate la u. Cu alte cuvinte, u dorește să știe cîți fii, nepoți, strănepoți etc. are. Pentru a calcula acest vector, u trebuie:

1. să însumeze vectorii primiți de la fii;
2. să translateze cu 1 toate valorile (fiul fiului lui u este nepotul lui u);
3. să se adauge pe sine însuși la distanță 0.

Pentru un arbore degenerat (un lanț de lungime n), o implementare naivă va copia și modifica succesiv vectori de lungime $1, 2, 3, \ldots, n$, așadar $\mathcal{O}(n^2)$ în total.

Tehnica *small-to-large* spune: este OK să transferăm O(mărimea fiului) informații de la fiecare fiu la părinte, **cîtă vreme ne asigurăm că transferăm din structura mai mică în cea mai mare**. Astfel, fiecare informație va fi transferată într-o structură de (cel puțin) două ori mai mare, deci numărul de transferuri este limitat la $\log n$. Complexitatea totală este $\mathcal{O}(n \log n)$ sau, în unele cazuri, chiar $\mathcal{O}(n)$.

Pentru exemplul de mai sus, am putea stoca vectorii de frecvență ca liste, astfel încît părintele să se poată adăuga la începutul listei. Aceasta rezolvă cerințele (2) și (3) în $\mathcal{O}(1)$. Pentru cerința (1), este suficient ca părintele să compare acumulatorul (lista cu frecvențele fiilor văzuți pînă acum) cu lista primită de fiul curent și să o adune pe cea mai scurtă la cea mai lungă.

18.2 Probleme

18.2.1 Problema Fixed-Length Paths I (CSES)

enunț • sursă

Putem rezolva problema exact cu ideile expuse teoretic. Fiecare nod raportează la părintele său o listă cu numărul de noduri aflate la distanțe 0 (el însuși), 1 (fiii), 2 (nepoții) etc. Pentru a-și calcula lista, fiecare nod:

1. Însumează listele primite de la toți fiii.
2. Deplasează lista rezultată cu o poziție (fiul fiului meu este nepotul meu).
3. Se adaugă pe sine însuși la distanță 0.

Pentru implementare, putem folosi un deque, care oferă inserarea la început în $\mathcal{O}(1)$. Pentru însumarea listelor, adăugăm mereu lista mai scurtă la cea mai lungă. În plus, fiecare nod u răspunde la întrebarea: pentru cîte căi de lungime k sînt eu LCA (nodul cel mai de sus de pe cale)? Apoi adaugă această valoare la un contor global. Pentru a calcula răspunsul, nodul u confruntă lista primită de la fiecare fiu v cu listele obținute și însumate anterior: dacă fiul v raportează a noduri la o distanță l și fiii anteriori raportaseră b noduri la o distanță $k-2-l$, adăugăm $a \cdot b$ la contorul global.

Care este complexitatea? Pare că ar fi $\mathcal{O}(n \log n)$, după cum spuneam: cînd copiem un nod din lista L_1 în lista L_2, la final L_2 va stoca informații (frecvențe) despre cel puțin dublul numărului de noduri din L_1. Deci fiecare nod poate fi copiat de cel mult $\log n$ ori.

Dar putem da o limită și mai strînsă. Pentru această problemă soluția este, de fapt, $\mathcal{O}(n)$! Programul nu operează cu noduri, ci doar cu distanțe. Cînd frecvența unei distanțe d în L_1 este 2 sau mai mare, nu mai pierdem timp individual ca să copiem acele noduri din L_1 în L_2, ci le „copiem" pe toate simultan, însumînd niște frecvențe. Practic, fiecare nod este copiat o singură dată. Vă puteți convinge de asta adăugînd contoare globale în interiorul buclelor critice și verificînd că ele nu depășesc valoarea totală n.

Notă: pentru a interschimba două liste, folosiți întotdeauna metoda `swap()`, care doar schimbă pointeri între ei. Niciodată nu folosiți operatorul `=`, care face copieri de elemente și are complexitate $\mathcal{O}$(lungime)!

18.2.2 Problema Distinct Colors (CSES) (din nou)

enunț • sursă

Să reluăm această problemă și să o rezolvăm cu tehnica *small-to-large*. O variantă ar fi ca fiecare nod să țină o listă sortată de culori. Atunci părintele trebuie să interclaseze listele fiilor. Dar nu putem face interclasarea în O(lista mai scurtă), ci doar în O(suma lungimilor). Aceasta duce la o soluție în $\mathcal{O}(n^2)$. Exemplu: $n = 1.000.000$, iar rădăcina are 1.000 de fii, fiecare cu cîte 1.000 de noduri în subarbore. Toate culorile sînt distincte. Atunci costul total al interclasărilor în rădăcină ar fi:

$$2.000 + 3.000 + \cdots + 1.000.000$$

Ca să combinăm doi fii în O(fiul mai mic), putem menține culorile într-o tabelă hash (`unordered_set`).

Atunci este ușor să „vărsăm” tabela mai mică în cea mai mare.

Două observații de implementare:

1. Această variantă consumă mai mult spațiu pe stivă, căci are variabile locale mai mari. Pe calculatorul meu local, testele mari chiar umplu stiva!
2. Această variantă este mai lentă decît cea cu AIB (nicio surpriză).

18.2.3 Problema Lomsat Gelral (Codeforces)

enunț • surse

Folosim tehnica *small-to-large*, așa cum se poate deduce și din numele problemei 🦊. Este important ca fiecare fiu să stocheze $\mathcal{O}$(subarbore) informații, nu $\mathcal{O}(n)$. Deci vom folosi o tabelă hash de culori cu frecvențele lor.

Ca observație interesantă, este mult mai eficient să calculați și să returnați din DFS tabela hash și celelalte informații, decît să le stocați în fiecare nod. Iată și o astfel de implementare, considerabil mai lentă.

Există și o soluție cu algoritmul lui Mo, puțin mai lentă, dar care consumă mai puțină memorie. Este necesară atenție la detalii. Ce informații stochează intervalul curent? Vă recomand să izolați acele structuri de date într-un `struct` sau o clasă separată.

Problemă similară: Christmas Balls (IIOT 2021/22 runda 2).

18.2.4 Problema Tokens on a Tree (CodeChef)

enunț • surse

O problemă echivalentă este Short Code (inspirată din viața reală, avînd în vedere cum își denumesc unii elevi variabilele).

Să demonstrăm teoretic ce avem de făcut. Iată diverse observații.

Observația 1. O monedă A poate „sări” aparent peste altă monedă B: practic, monedele fiind identice, o urcăm pe B pînă la destinația dorită, apoi pe A în locul lui B.

Observația 2. În orice soluție, monedele se vor afla la vîrful arborelui. Niciodată nu vom avea o monedă sub un nod gol, căci am putea face o mutare în plus.

Observația 3. Dacă rădăcina conține inițial o monedă, atunci acea monedă nu pleacă nicăieri și nicio altă monedă nu îi va lua locul. Deci putem rezolva problema independent pentru subarbori.

Observația 4. Dacă rădăcina nu conține inițial o monedă, atunci dintr-unul dintre subarborii fiilor o monedă va urca în rădăcină. Restul fiilor vor fi rezolvați independent, conform Observației 3. Moneda care va urca în rădăcină este cea de adîncime maximă după ce rezolvăm fiii, ca să facem cît mai mulți pași în plus. Urcarea este aparentă, conform Observației 1.

Așadar, implementarea trebuie să combine în mod eficient fiii unui nod, apoi să găsească cea mai de jos monedă și să o urce în părinte. Am găsit două variante de implementare.

Implementare cu *small-to-large*

Fiecare nod u calculează un vector/listă f unde $f[i]$ pentru $i \geq 0$ este numărul de monede la distanță i de u. Fiecare părinte însumează listele fiilor așa cum știm, iar algoritmul este $\mathcal{O}(n)$, căci nodurile odată comasate își pierd identitatea. În plus, nodul u se adaugă pe sine însuși la începutul listei și, dacă este loc, simulează urcarea unei monede transferînd o unitate de pe ultima poziție din f pe prima.

Detalii de implementare. Implementarea cu `std::list` este cu vreo 25 de linii mai scurtă decît cea cu liste proprii, dar este de două ori mai lentă și consumă dublul memoriei. Implementarea cu `std::deque` este nefolosibilă ca timp și ca memorie (spre 1 GB). Probabil, clasa `deque` are costuri fixe, iar multe deque-uri mici sînt foarte scumpe.

Implementare cu liniarizare + RMQ

Construim o liniarizare DFS și notăm, în nodurile care conțin monede, adîncimea acelor noduri. În nodurile care nu conțin monede nu notăm nimic. Pentru a găsi cea mai de jos monedă din subarborele unui nod u, găsim maximul pe intervalul subîntins de u. Pentru a muta moneda, scriem 0 în locul acelui maxim (moneda dispare) și scriem adîncimea lui u la poziția nodului u (moneda apare acolo). Apoi creștem numărul total de mutări cu diferența dintre cele două adîncimi. Avem nevoie de RMQ cu actualizare, deci putem folosi varianta pe arbori de segmente. Rezultă o complexitate de $\mathcal{O}(n \log n)$. Codul este de 2-3 ori mai lent și consumă mai multă memorie decît implementarea cu *small-to-large*.

18.2.5 Problema Blood Cousins Return (Codeforces)

enunț • surse

Toate soluțiile încep prin a transforma numele în numere. Cea mai la îndemînă implementare folosește un `unordered_map`.

Implementarea cu *small-to-large*

O soluție destul de brutală cu *small-to-large* este: fiecare nod u ține un vector de set-uri, unde setul de pe poziția i reține numerele distincte regăsite la adîncime i (raportat la adîncimea lui u) în subarborele lui u. Cînd combinăm doi vectori, combinăm două cîte două seturile reprezentînd aceeași adîncime. Trebuie să avem grijă să folosim *small-to-large* în două locuri:

1. Copiem vectorul mai mic în cel mai mare.
2. Pentru fiecare pereche de seturi, îl copiem pe cel mai mic în cel mai mare.

Note de implementare:

1. `unsorted_set` este mai lent decît `set`, deși algoritmic vorbind ne-ar trebui prima, care oferă timp constant. Dar tabelele hash (adică `unsorted_set`) au o mărime minimă dată de vectorul pe care se bazează.
2. Reprezentarea listelor de seturi cu `deque` este de două ori mai lentă decît cu `vector`.

Implementarea este rezonabil de scurtă, dar nu prea eficientă.

Implementarea cu tehnici de bază

Iată și o abordare care necesită doar parcurgeri, liniarizare și căutare binară. Soluția este greu de codat, dar este mai rapidă.

1. Calculăm ordinea BFS. Atunci răspunsul la orice interogare se va traduce în numărarea elementelor distincte de pe un interval contiguu din BFS. Știm să facem asta folosind un simplu AIB și ordonînd interogările după capătul drept (vezi problema D-query).
2. Pentru fiecare interogare, rămîne să aflăm primul și ultimul nod de la o anumită adîncime din subarborele lui u, ca să știm pe ce interval din BFS facem numărarea. Putem face asta cu două parcurgeri DFS și o stivă. La intrarea în fiecare nod, notăm valoarea sa pe stivă în dreptul adîncimii sale. La revenirea din nod, lăsăm stiva intactă. Atunci, la revenirea într-un nod u aflat la adîncimea d, putem ști care este **ultimul** său descendent de la adîncimea d': fie nu există, fie este nodul de la poziția d' de pe stivă. Pentru a afla **primul** descendent de la adîncimea dorită, facem același DFS, dar iterînd prin fiii nodurilor în ordine inversă.

Sună rezonabil din vorbe, dar prima implementare a fost migăloasă.

O soluție considerabil mai scurtă (cea inclusă în anexă) procedează astfel:

1. Calculăm aceeași ordine BFS.
2. Înlocuim fiecare interogare $\langle u, d\rangle$ cu interogarea $\langle depth[u]+d, t_{in}[u], t_{out}[u]\rangle$. Cu alte cuvinte, fiecare interogare interoghează un interval aflat la o anumită adîncime și cu noduri între timpii dați. Sortăm și procesăm interogările cu un AIB ca de obicei.

O a treia implementare este: colectăm nodurile separat pe fiecare nivel, în ordinea timpilor de vizitare (care este chiar ordinea DFS). Acum intervalul de interes pentru o interogare referitoare la nodul u este pe un nivel cunoscut și este delimitat de timpii de intrare și de ieșire din u.

Este greu să construim cîte un AIB pe fiecare nivel, dar putem folosi un `set` cu aceeași informație: pozițiile ultimelor apariții ale fiecărui element distinct. Iată o implementare curată a fostului olimpic Alex Nuță.

18.3 DFS exclusiv

Am învățat această tehnică din sursa unui legendary grandmaster. 🤩 Ulterior am aflat că ea se mai numește și Sack sau DSU pe arbori. Pot fi de acord cu prima denumire, dar nu și cu a doua, în primul rînd pentru că DSU este un nume greșit pentru DSF (engl. *disjoint set forest*) și în al doilea rînd pentru că structura face adăugări și ștergeri care n-au nicio legătură cu implementarea

canonică de mulțimi disjuncte. Prefer să îi spun **DFS exclusiv**, care descrie mult mai bine cum funcționează tehnica.

Implementarea din acel tutorial mi se pare criptică. Sper că o pot clarifica.

Să ne gîndim la clasa de probleme care fac interogări pe subarbore: frecvențe, sume, numere distincte etc. De exemplu, problema următoare, Tree and Queries, face interogări pe un arbore cu valori în fiecare nod. Interogările au forma (v, k) = numărul de valori distincte care apar de cel puțin k ori în subarborele lui v. Multe din aceste probleme se pot rezolva cu tehnica *small-to-large* deja studiată.

Iată și o rezolvare cu un DFS „pe steroizi". În loc să calculăm cîte o structură în fiecare nod, vom menține o singură structură globală, S. DFS-ul va adăuga și va șterge din S informații despre noduri la diverse momente. El garantează că, pentru orice nod u, va exista un moment în DFS cînd S va conține informații exact despre subarborele lui u. La acel moment vom putea răspunde la interogările despre u.

Desigur, nu putem face un DFS simplu, căci atunci la intrarea într-un nod S va fi deja populată cu date din afara subarborelui său, ceea ce va da răspunsuri incorecte la orice interogări despre u. În loc de aceasta, începem prin a calcula pentru fiecare nod u care este fiul său cu subarborele cel mai mare. Notăm această informație cu $h[u]$ (de la *heavy*). Acum, $DFS(u)$ procedează astfel:

1. Apelează recursiv toți fiii cu excepția lui $h[u]$.
2. La revenirea din fiecare fiu v, elimină recursiv din S toate nodurile din subarborele lui v.
3. Apelează recursiv $DFS(h[u])$. De data aceasta, lasă informațiile în S.
4. Adaugă la loc subarborii eliminați la pasul (2).
5. Adaugă în S nodul u însuși.
6. Răspunde la interogări despre subarborele lui u.

Se nasc două întrebări. Prima: de ce funcționează corect algoritmul? Mai exact, de ce la pasul (6) vectorii conțin doar informații despre subarborele lui u? Să considerăm un alt nod w, din afara subarborelui lui u. Există trei posibilități.

1. w este strămoș al lui u („nod gri"). Atunci, în mod garantat, w nu apare în structura de date la momentul procesării lui u, deoarece w este adăugat în structură doar la finalul recursivității.
2. w este un „nod negru" într-un subarbore deja vizitat. Fie a strămoșul comun al lui u și v și fie a_w și a_u fiii lui a care pornesc către w, respectiv către u. Deoarece sîntem în procesul de vizitare a lui a_u, înseamnă că a_w și tot subarborele său (inclusiv w) au fost temporar șterși din structură.
3. w este un „nod alb", undeva într-un subarbore încă nevizitat. Atunci w încă nu a fost descoperit de DFS.

A doua întrebare: care este complexitatea algoritmului? Dacă nu am trata special nodul $h[u]$, atunci complexitatea ar fi pătratică. Fiecare nod este eliminat la pasul (2) și readăugat la pasul (5) pentru fiecare strămoș al său.

Dacă tratăm special fiii *heavy*, să analizăm numărul de eliminări și readăugări. Ori de cîte ori un strămoș w cauzează eliminarea unui descendent u, înseamnă că w **nu** este fiu *heavy* al părintelui său. Echivalent, părintele lui w este cel puțin de două ori mai mare decît w. Așadar, eliminarea nodului u poate fi cauzată de cel mult $\log n$ dintre strămoșii săi. Complexitatea parcurgerii este $\mathcal{O}(n \log n)$.

Cum fiecare eliminare și adăugare necesită o operație în AIB, rezultă că complexitatea întregului algoritm este $\mathcal{O}(n \log^2 n)$.

Vom citi codul care rezolvă problema următoare.

18.4 Probleme

18.4.1 Problema Tree and Queries (Codeforces)

enunț • surse

Implementare brută

Să vedem mai întîi abordarea cu *small-to-large* clasic. Proiectăm o structură de date care:

1. Să mențină frecvențele culorilor din subarborele curent.
2. Să raporteze numărul de frecvențe cel puțin egale cu o valoare dată.

Pentru (2) am dori un vector de frecvențe ale frecvențelor: $g[x]$ stochează numărul de culori care au frecvența x. Pe acest vector, răspunsul la o interogare (u, x) este suma pe sufixul $g[x \dots n]$. Dar implementarea este alunecoasă, căci dorim ca structura să ocupe spațiu $\mathcal{O}$(subarbore), nu $\mathcal{O}(n)$. Deci orice AIB sau arbore de intervale construit peste g trebuie extins dinamic pe măsură ce urcăm în arbore.

În schimb, putem folosi un multiset: un set ordonat al tuturor frecvențelor (nenule). Ca să putem răspunde la întrebarea „cîte frecvențe mai mari sau egale cu x există?”, avem nevoie de PBDS (*policy-based data structure*), o colecție extinsă de structuri de date din STL. Ne-am mai întîlnit cu seturi PBDS la problema Give Away, în capitolul de descompunere în radical.

Soluția este relativ directă, dar ca să o puteți scrie în timp de concurs trebuie să memorați două lucruri:

1. Incantația magică necesară pentru a declara o structură de date PBDS.
2. Codul necesar pentru ștergerea dintr-un multiset. Cînd o frecvență se modifică, noi dorim să ștergem din multiset o singură apariție a vechii frecvențe, dar un simplu apel la `erase` le-ar șterge pe toate.

Implementare cu DFS exclusiv

Vom menține aceleași informații, dar global:

- un vector de frecvențe;
- un AIB peste vectorul de frecvențe ale frecvențelor.

Notă de implementare: putem stoca listele de adiacență și listele de interogări ca `vector` sau ca `list`, căci nu avem nevoie de acces aleatoriu. Implementarea cu `list` este considerabil mai lentă și consumă mai multă memorie.

18.4.2 Problema Dominant Indices (Codeforces)

enunț • surse

Problema nu este mult mai complicată decît precedenta. Doar enunțul este ușor greoi.

Soluție cu *small-to-large* clasic

Să definim întîi un termen pentru a simplifica discuția. Numim **lățimea** la adîncime d a subarborelui unui nod u ca fiind numărul de descendenți ai lui u aflați la adîncime d față de u. Cu această definiție, problema ne cere să tipărim, pentru fiecare nod u, adîncimea la care lățimea subarborelui lui u este maximă. La egalitate, vom prefera adîncimea minimă (așadar, mai aproape de rădăcina subarborelui, u).

Vom construi, în fiecare nod u, un vector h în care $h[i]$ reține lățimea la adîncime i a subarborelui. L-am denumit astfel întrucît, în fapt, h este o histogramă a distanței de la u la nodurile din subarbore. Simultan vom menține și indicele dominant din acest vector.

Vom calcula recursiv structura de date combinînd structurile fiilor. Pentru a reduce complexitatea de la $\mathcal{O}(n^2)$ vom „vărsa" mereu vectorul mai mic în cel mai mare.

Rămîne întrebarea cum menținem sau cum găsim indicele dominant. Dar să observăm că toate operațiile sînt de adăugare. Niciodată nu vom avea nevoie să scădem valori din vectorii h. De aceea, structura poate menține indicele maximului, pe care îl recalculăm la fiecare operație de adăugare.

Soluție cu *small-to-large* exclusiv

Reluăm același șablon, care este mare, dar care se particularizează în mod banal pentru această problemă. Vom menține o singură structură globală în care inserăm și ștergem subarbori. Algoritmul garantează că, pentru orice nod u, la sfîrșitul apelului recursiv în nodul u, structura globală va conține exclusiv informații despre subarborele lui u.

Calculăm înălțimile tuturor nodurilor. Cînd inserăm sau cînd ștergem un nod, incrementăm sau decrementăm frecvența nodurilor pe nivelul corespunzător adîncimii lui u. Atunci răspunsul pentru u este pur și simplu poziția frecvenței maxime în structura globală (poziția minimă, dacă există mai multe), minus înălțimea nodului u.

Pentru a ține evidența maximului, putem folosi o structură logaritmică, să spunem un arbore de intervale. Dar putem afla răspunsul și în timp constant, astfel:

- Cînd inserăm un nod, iar lățimea nivelului său crește, actualizăm în mod elementar maximul.
- Cînd ștergem noduri nu actualizăm maximul, căci nu știm cum (vedeți ce simplu este? 🙂).
- În schimb, înainte să demarăm parcurgerea recursivă a unui fiu care nu este *heavy*, salvăm poziția maximului.
- După ce parcurgem acel fiu, imediat îl și ștergem. Apoi restaurăm poziția maximului din valoarea salvată.

Complexitatea este $\mathcal{O}(n \log n)$ datorită algoritmului *small-to-large* exclusiv.

Sursa anexată folosește vectori STL pentru listele de adiacență. Am scris și o implementare cu liste proprii, care este puțin mai lentă. Cred că explicația este următoarea: pentru un arbore atît de mare, faptul că listele sînt împrăștiate prin memorie dăunează cache-ului. În schimb, `stl::vector` își ține elementele grupate și are *cache locality* bun.

Capitolul 19

Cel mai apropiat strămoș comun

Dat fiind un arbore cu rădăcină, pentru orice două noduri u și v definim **cel mai apropiat strămoș comun** ca fiind nodul de adîncime maximă (sau nodul „cel mai jos") care îi are ca descendenți atît pe u, cît și pe v. Considerăm că un nod este propriul său descendent, ceea ce înseamnă că cel mai apropiat strămoș comun al lui u și v poate fi chiar unul dintre aceste noduri, dacă este strămoș al celuilalt.

În literatura română am întîlnit și denumirea „cel mai mic strămoș comun", doar că această denumire mi se pare improprie. Nu comparăm nodurile ca mărime, ci ca adîncime. Pentru abreviere, o vom folosi pe cea din limba engleză (LCA - *lowest common ancestor*).

Formal, dat fiind un arbore cu n noduri, dorim să răspundem la q întrebări de forma:

- $LCA(u, v)$: găsește nodul de adîncime maximă care este strămoș atît al lui u, cît și al lui v.

CP Algorithms inventariază multe dintre metodele disponibile.

O aplicație directă a LCA-ului este aflarea distanțelor între noduri. Notăm cu $d[u]$ adîncimea unui nod u. Atunci

$$dist(u, v) = (d[u] - d[LCA]) + (d[v] - d[LCA]) = d[u] + d[v] - 2 \cdot d[LCA]$$

19.1 Sumar: RMQ (*range minimum query*)

RMQ este o problemă clasică pe vectori. Dat fiind un vector V cu n elemente, trebuie să răspundem la q întrebări de forma $\langle l, r \rangle$ cu semnificația: să se găsească minimul valorilor $V[l \dots r]$.

Problema are diverse variațiuni. Ni se poate cere *valoarea* minimă sau *poziția* valorii minime. Vectorul poate fi static sau poate avea actualizări.

Acest curs nu are un capitol dedicat pentru RMQ, dar o vom trata superficial aici întrucît ea se leagă de algoritmii pentru LCA. Iată un sumar al metodelor folosite în programarea competitivă.

Precizări:

metodă	**preprocesare**	**interogări**	**memorie extra**
Arbore de intervale	$\mathcal{O}(n)$	$\mathcal{O}(q \log n)$	$\mathcal{O}(n)$
Arbore Fenwick (AIB)	$\mathcal{O}(n)$	$\mathcal{O}(q \log n)$	0
Descompunere în radical	$\mathcal{O}(n)$	$\mathcal{O}(q\sqrt{n})$	$\mathcal{O}(\sqrt{n})$
Tabelă rară (*sparse table*)	$\mathcal{O}(n \log n)$	$\mathcal{O}(q)$	$\mathcal{O}(n \log n)$
Stivă ordonată	$\mathcal{O}(q \log q)$	$\mathcal{O}(n + q \log n)$	$\mathcal{O}(n)$

Tabela 19.1: Metode pentru problema RMQ.

- Tabela rară și stiva ordonată nu permit actualizări.
- Arborele Fenwick permite doar interogări pe prefix și doar actualizări descrescătoare.

Să trecem în revistă, doar pentru completitudine, structurile de date.

19.1.1 RMQ cu arbore de intervale

Construim un arbore de intervale în care nodul părinte reține minimul fiilor săi. Admite actualizări punctuale sau, în varianta cu propagare *lazy*, și actualizări pe interval. Am tot studiat aceste structuri de date, nu mai detaliem aici.

19.1.2 RMQ cu arbore indexat binar

Construim un AIB peste vector. Admite doar interogări pe prefix și doar actualizări descrescătoare.

19.1.3 RMQ cu descompunere în radical

Pentru fiecare bloc reținem minimul. Asimptotic operațiile sînt mai lente, dar aș zice că actualizarea pe interval este mai simplu de codat decît la arborii de intervale.

19.1.4 RMQ cu tabelă rară

Elevii se dau în vînt după această metodă. Codul este aproximativ:

```
void compute_rmq(int* v, int n) {
  // r[p][i] = minimul pe intervalul v[i]...v[i + 2^p - 1]
  for (int i = 0; i < n; i++) {
    r[0][i] = v[i];
  }
  for (int p = 1; (1 << p) <= n; p++) {
    for (int i = 0; i + (1 << p) <= n; i++) {
      r[p][i] = min(r[p - 1][i], r[p - 1][i + (1 << (p - 1))]);
    }
  }
}
```

```
int rmq(int left, int right) { // inclusiv
  int p = 31 - __builtin_clz(right - left + 1); // log_2 din lungime
  return min(r[p][left], r[p][right - (1 << p) + 1];
}
```

Totuși, consumul de memorie al metodei *sparse table* este enorm. În afară de cazul în care avem multe interogări, timpul de $\mathcal{O}(1)$ per interogare nu justifică risipa de memorie. Exemplu: Cu *sparse table*, pentru $n = 200\,000$ vom folosi $200\,000 \times 18$ întregi, adică 14,4 MB. Pentru arborele de intervale, presupunînd că îl completăm pînă la 256K elemente, vom folosi 512K întregi, adică 2 MB. Nu am rulat benchmark-uri, dar mă aștept să existe o diferență de viteză din cauza cache-ului.

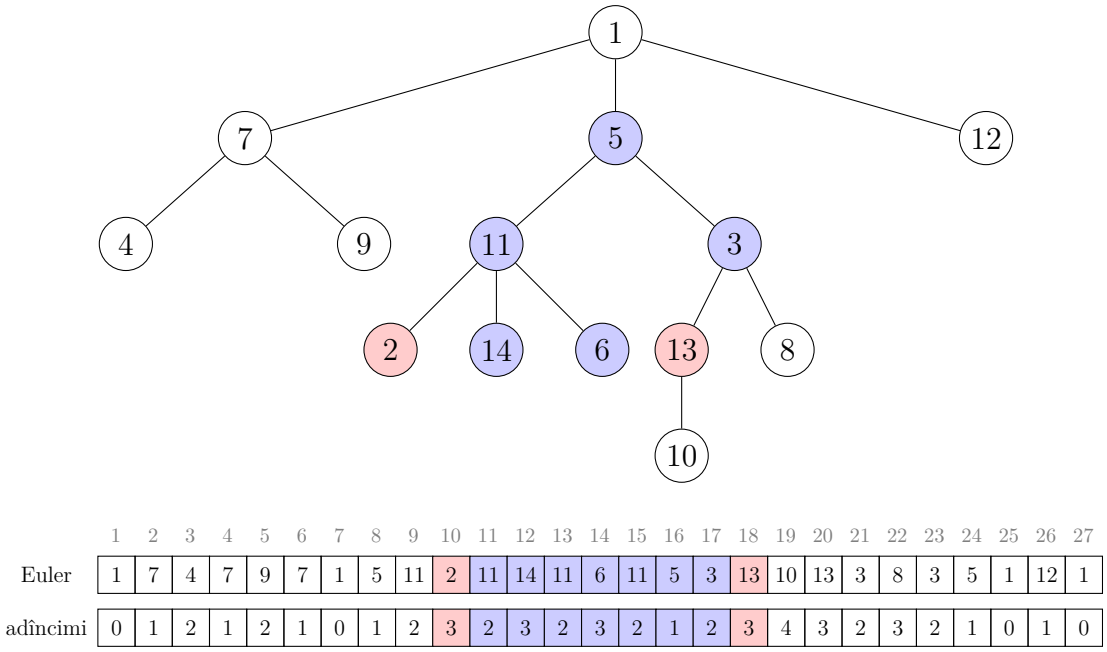
19.1.5 RMQ cu stivă ordonată

Includ și această metodă ad-hoc. Ordonăm interogările după capătul drept. Parcurgem vectorul de la stînga la dreapta și menținem o stivă crescătoare de minime parțiale. La poziția r putem răspunde la toate interogările $[l, r]$. Răspunsul este valoarea minimă din stivă aflată pe o poziție mai mare sau egală cu l. Așadar este suficientă o căutare binară.

Exemplu: fie vectorul $V = (6, 3, \mathbf{2}, 10, 8, \mathbf{6}, 9, 15, \mathbf{9}, \mathbf{20}, \dots)$. Cînd ajungem la elementul 20, stiva constă din valorile trecute cu aldin. Cînd răspundem la interogările cu capătul r pe elementul 20, răspunsul va fi una dintre valorile din stivă, în funcție de poziția capătului l.

19.2 LCA cu liniarizare

Facem o liniarizare Euler cu repetiție. În vector notăm nodurile, dar ne interesează de fapt adîncimea acelor noduri. Astfel, putem reformula interogările LCA ca „găsește nodul de adîncime minimă dintre ultima apariție a lui u și prima apariție a lui v”. Așadar, toate metodele de RMQ pe vector se aplică și aici.



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Euler	1	7	4	7	9	7	1	5	11	2	11	14	11	6	11	5	3	13	10	13	3	8	3	5	1	12	1
adîncimi	0	1	2	1	2	1	0	1	2	3	2	3	2	3	2	1	2	3	4	3	2	3	2	1	0	1	0

Figura 19.1: Aflarea LCA printr-o liniarizare Euler cu repetiție. LCA(2,13) se reduce la o interogare de minim pe intervalul [10,18]. Adîncimea minimă este 1, corespunzătoare nodului 5.

În continuare, vom mai discuta patru metode specifice arborilor, respectiv:

metodă	**preprocesare**	**interogări**	**memorie extra**
Descompunere în radical	$\mathcal{O}(n)$	$\mathcal{O}(q\sqrt{n})$	$\mathcal{O}(n)$
Binary lifting ($\log n$ pointeri)	$\mathcal{O}(n \log n)$	$\mathcal{O}(q \log n)$	$\mathcal{O}(n \log n)$
Binary lifting (2 pointeri)	$\mathcal{O}(n)$	$\mathcal{O}(q \log n)$	$\mathcal{O}(n)$
Tarjan offline	$\mathcal{O}(q + n \log^* n)$	—	$\mathcal{O}(n)$

Tabela 19.2: Metode pentru aflarea LCA.

19.3 LCA cu descompunere în radical

Următoarele trei metode folosesc noțiunea de *jump pointers*: pointeri la strămoși care ne ajută să accelerăm urcarea din u și din v în căutarea LCA-ului. Pentru descompunerea în radical,

- Fiecare nod ține un pointer la părinte și unul la strămoșul aflat cu $\sqrt{n}$ noduri mai sus.
- Construcția evidentă durează $\mathcal{O}(n\sqrt{n})$: din fiecare nod, urcăm $\sqrt{n}$ niveluri. Ea poate fi redusă la $\mathcal{O}(n)$ dacă menținem stiva DFS pe durata parcurgerii.
- Fiecare interogare durează $\mathcal{O}(\sqrt{n})$.
- Variantă: fiecare nod stochează un pointer la cel mai de jos strămoș de adîncime multiplu de $\sqrt{n}$.

implementare

19.4 LCA cu binary lifting ($\log n$ pointeri per nod)

- Fiecare nod ține pointeri la strămoșii aflați mai sus cu 1, 2, 4, 8... niveluri.
- Construcția durează $\mathcal{O}(n \log n)$, similară algoritmului de RMQ.
- Necesită $\mathcal{O}(n \log n)$ memorie.
- Fiecare interogare durează $\mathcal{O}(\log n)$.
  - Deci nu prea mai merită. Pe vector plăteam memoria $\mathcal{O}(n \log n)$ pentru că interogările durau $\mathcal{O}(1)$.

Detaliu de implementare

Metodele cu *jump pointers* pot fi implementate în două feluri:

1. Aducem nodurile la aceeași adîncime. Apoi urcăm în paralel cu ambele pînă la strămoșul comun.
2. Îl urcăm pe u cît timp nu devine strămoș al lui v. La final, u este LCA-ul.

A doua implementare (cu test de strămoș) este mai scurtă și puțin mai rapidă.

implementări

19.5 LCA cu binary lifting (2 pointeri per nod)

- Un articol bun pe Codeforces (imaginile sînt foarte utile).
- Fiecare nod x ține un pointer la părinte și un pointer numit jump, construit după regula:
- Fie y părintele lui x, fie $z = jump[y]$ și fie $t = jump[z]$.
- Dacă distanțele între y-z și z-t sînt egale, atunci $jump[x] = t$.
- Altfel $jump[x] = y$.
- Iau naștere niște pointeri cu o structură curioasă. Cei pasionați pot citi despre secvență pe OEIS, secvențele A082850 și A182105.
- Folosim această informație pentru a găsi LCA în $\mathcal{O}(\log n)$.
- Orice structură de pointeri „aproximativ" logaritmică funcționează aici. Am făcut și un experiment cu formula LSB (exemplu: un nod la adîncime $20 = 10100_2$ va pointa patru nivele mai sus).

implementări

19.6 LCA cu algoritmul lui Tarjan (offline)

- Funcționează cînd interogările sînt date în avans.
- Distribuie interogările după noduri. Interogarea (u, v) este distribuită în listele lui u și v.
- Răspunde la interogări într-un singur DFS (!).
- Principiu de bază: grupăm nodurile deja vizitate („negre") și pe cele în curs de vizitare („gri") în mulțimi disjuncte (cu *union-find*).

- Fiecare mulțime va conține exact un nod gri, plus toți descendenții săi, cu excepția nodului gri în care se află acum DFS-ul. La terminarea unui nod gri, el este unit cu părintele său.
- Putem răspunde la o interogare (u, v) în momentul în care v este negru, iar pe u tocmai îl vizităm. Răspunsul (LCA) este nodul gri din mulțimea lui v.

implementare

19.7 Benchmarks

Am făcut aceste teste pe un arbore cu 200.000 de noduri, cu un lanț de cel puțin 150.000 de noduri. Fișierele de intrare au 5,2 MB.

- Tarjan: 160 ms
- *binary lifting*, doi pointeri (cu test de strămoș sau nu): 190 ms
- *binary lifting*, doi pointeri (metoda LSB): 250 ms
- *binary lifting*, $\log n$ pointeri, cu test de strămoș: 290 ms
- *binary lifting*, $\log n$ pointeri: 320 ms
- descompunere în radical: 900 ms

Concluzii: cînd interogările sînt date în avans, Tarjan cîștigă. În rest, metodele cu *binary lifting* cu doar doi pointeri per nod sînt mai rapide și consumă mai puțină memorie decît metoda cu $\log n$ pointeri per nod.

19.8 Probleme

19.8.1 Problema Gold Transfer (Codeforces)

enunț • sursă

Atenție mare la garanția din enunț: $c_i > c_{p_i}$. Cu alte cuvinte, pentru orice operație de cumpărare trebuie să pornim din rădăcină spre nodul u și să cumpărăm orice cantități disponibile, pînă satisfacem cererea.

Cu timpul, nodurile de la rădăcină se vor goli, deci pare o idee bună să găsim rapid cel mai de jos strămoș care încă are aur disponibil. Dacă reușim să-l găsim în $\mathcal{O}(f(n))$ (intenția problemei fiind $f(n) = \log n$), atunci complexitatea globală va fi $\mathcal{O}(q \cdot f(n) + n)$. De ce? Din acel strămoș putem parcurge lanțul în jos pas cu pas, cumpărînd tot aurul disponibil, pînă satisfacem cererea. Fiecare nod poate fi golit cel mult o dată, iar ultimul nod din fiecare interogare poate păstra niște aur. De aceea, efortul total pentru parcurgerea lanțurilor este $\mathcal{O}(n + q)$.

Detalii de implementare

Arborele este dinamic, deci nu putem construi o liniarizare.

Am ales să accelerez urcarea în arbore cu *binary lifting* cu doi pointeri, dar orice altă metodă este acceptabilă.

Odată ce golim un nod, avem nevoie să coborîm în fiul său care duce spre nodul original. Am ales să fac acest lucru cu o stivă, dar soluția este cam lentă (2x față de altele).

Întrucît nu avem de ce să parcurgem toți fiii unui nod, nu avem nevoie de liste de adiacență, ci doar de pointeri la părinte.

19.8.2 Problema A and B and Lecture Rooms (Codeforces)

enunț • sursă

Problema cere să răspundem la m întrebări de forma: Date fiind nodurile u și v (posibil egale), cîte noduri din arbore se află la distanță egală de u și de v?

Pentru soluția teoretică, următoarea vizualizare este utilă. Să „atîrnăm" arborele de nodurile u și v, ca pe o ghirlandă bine întinsă. Atunci calea cea mai scurtă $u - v$ va fi orizontală, iar de lanțurile de pe cale vor atîrna restul subarborilor.

Dacă distanța $u - v$ este impară, atunci răspunsul este 0. Dacă distanța este pară, atunci fie w nodul de la jumătatea distanței. Nodurile aflate la distanță egală de u și de v vor fi w și orice nod din subarborii care atîrnă din w.

În practică vom alege o rădăcină și vom precalcula informații pentru LCA. Apoi, eu am redus problema la următoarele cazuri (poate se poate și mai simplu):

1. Dacă distanța $u - v$ este impară, răspunsul este 0.
2. Dacă $u = v$, răspunsul este n.
3. Dacă u și v au adîncimi egale, atunci răspunsul este: mărimea subarborelui lui $LCA(u, v)$ minus mărimile subarborilor fiilor lui $LCA(u, v)$ care pornesc către u, respectiv către v.
4. Altfel, să presupunem că u are adîncime mai mare decît v. Atunci nodul w este undeva mai jos de LCA, mergînd către u. Răspunsul este: mărimea subarborelui lui w minus mărimea subarborelui fiului lui w care pornește către u.

Detalii de implementare

Punctul (4) presupune să calculăm al k-lea strămoș. De exemplu, dacă adîncimile sînt $d[u] = 50$, $d[v] = 20$ și $d[LCA] = 10$, atunci distanța $u - v$ este $40 + 10 = 50$, jumătatea distanței este 25, iar nodul w este al 25-lea strămoș al lui u.

De aceea, metoda lui Tarjan nu ne prea ajută, ci avem nevoie de o metodă cu *jump pointers*.

La punctele (3) și (4), pentru a afla „fiul lui w care pornește către u", putem refolosi informațiile pentru al k-lea strămoș. Dacă notăm cu k diferența pe înălțime între w și u, atunci răspunsul este al $k - 1$-lea strămoș al lui u.

La arbori, multe informații sînt redundante și putem stoca doar o submulțime. Exemple:

- Mărimea subarborelui lui u nu trebuie stocată implicit dacă cunoaștem timpii DFS, ci poate fi calculată ca $t_o[u] - t_i[u] + 1$.
- Paritatea distanței poate fi calculată fără a calcula distanța efectivă. Calculăm doar suma adîncimilor.
- Al k-lea strămoș al unui nod poate fi găsit ca strămoșul de la adîncimea $d[u] - k$ al unui nod, dacă cunoaștem adîncimile nodurilor.

19.8.3 Problema Company (Codeforces)

enunț • sursă

Problema poate fi rezumată astfel: dintre toate nodurile cu numere de la l la r, cum putem elimina un nod astfel încît LCA-ul celor rămase să aibă o adîncime cît mai mare, raportată la rădăcină?

Întrebarea teoretică la care trebuie să răspundem este: care este LCA-ul unei submulțimi de noduri? Observăm că nu este nevoie să iterăm prin toate, ci este suficient să calculăm LCA-ul între primul și ultimul nod în ordinea descoperirii lor în DFS. De aceea, ca să încercăm să coborîm LCA-ul, trebuie să eliminăm unul dintre aceste noduri. Altfel LCA-ul submulțimii după eliminarea unui nod va rămîne nemodificat față de LCA-ul original.

Să presupunem că găsim aceste noduri, fie ele x și y. Încercăm să îl eliminăm pe x și să calculăm LCA-ul submulțimii $[l, x-1] \cup [x+1, r]$. Apoi îl eliminăm pe y și calculăm LCA-ul submulțimii $[l, y-1] \cup [y+1, r]$. Afișăm varianta care duce la un LCA de adîncime maximă.

Detalii de implementare

Ca să găsim timpii DFS minim/maxim ai nodurilor din intervalul $[l, r]$, putem construi un vector $t[]$ unde $t[u]$ este timpul vizitării nodului u. Astfel reducem subproblema la una de RMQ. Pentru a deduce, din acești timpi, nodul efectiv (primul nod vizitat dintre toate din $[l, r]$) este suficient un vector $inv[]$ unde $inv[i]$ este nodul vizitat la momentul i. Practic t și inv sînt permutări inverse.

Pentru intervalele la care ajungem după eliminarea lui x și y (respectiv $[l, x-1]$, $[x+1, r]$ și celelalte) avem nevoie să aflăm LCA-ul, deci avem nevoie tot de RMQ ca să aflăm primul și ultimul nod în ordinea DFS. Dar am făcut următoarea observație care reduce mult numărul de interogări.

Fie a, b, c, d primul, al doilea, penultimul și respectiv ultimul nod din intervalul $[l, r]$, în ordinea DFS. Atunci iau naștere două cazuri:

- Dacă îl eliminăm pe a, LCA-ul nodurilor rămase este $LCA(b, d)$.
- Dacă îl eliminăm pe d, LCA-ul nodurilor rămase este $LCA(a, c)$.

De aceea, am proiectat o structură de date care să returneze, pentru o interogare $[l, r]$, primele două minime și primele două maxime din intervalul $[l, r]$. Practic structura returnează exact cele patru valori a, b, c, d.

Putem folosi RMQ cu sparse table, cu atenție la detalii la calculul celui de-al doilea maxim/minim (intervalele de lungime putere a lui 2 pot fi suprapuse). Dar nu-mi place ideea de a consuma memorie $\mathcal{O}(n \log n)$ fără rost. 😐 De aceea, am implementat structura ca pe un arbore de intervale. Operația critică este combinarea a două tupluri (a', b', c', d') și (a'', b'', c'', d''). Implementarea este relativ ușoară:

- Dacă $a' < a''$ atunci primul minim este a', iar al doilea minim este $\min(b', a'')$.
- Dacă $a' \geq a''$ atunci primul minim este a'', iar al doilea minim este $\min(b'', a')$.
- Similar pentru maxime.

Sursa mea este printre cele mai rapide, cu excepția celor care parsează intrarea. Suspectez că ajută mult (1) economia de memorie și (2) calculul simultan al minimelor și al maximelor, în aceeași structură de date.

19.8.4 Problema Duff in the Army (Codeforces)

enunț • sursă • coloană sonoră 🙂

Rezumat: Se dă un arbore cu n noduri și m locuitori. Locuitorul i locuiește în orașul c_i. Răspundeți la q interogări de forma (u, v, a) cu semnificația: tipăriți primii a locuitori, ordonați după ID, care locuiesc pe calea (u, v). Notă: $a \leq 10$.

Problema se duce tot în direcția aflării LCA. Dacă $LCA(u, v) = w$, și dacă aflăm populațiile pe căile (u, w) și (v, w), putem să le interclasăm în $\mathcal{O}(a)$ și să păstrăm cele mai mici a valori.

Cum aflăm populația pe o cale? Nu văd o soluție în $\mathcal{O}(a + \log n)$, dar $\mathcal{O}(a \log n)$ este facilă. Orice algoritm de LCA ne oferă această complexitate dacă precalculăm, împreună cu pointerii în sus, și populația acoperită de acei pointeri. De exemplu, cu metoda *binary lifting*, fiecare din pointerii peste 1, 2, 4, 8, ... niveluri reține și primii a membri ai populației de pe acea cale. Populația pe calea de lungime 16 se obține interclasînd cele două populații pe căile de lungime 8 și păstrînd cel mult 10 minime.

Atenție la metoda folosită! *Binary lifting* cu memorie $\mathcal{O}(n \log n)$ va necesita $100\,000 \times 18 \times 10$ întregi, adică 72 MB în plus. Este fezabil, dar este de evitat. Implementarea mea este printre cele mai rapide și nu face nimic deosebit, doar folosește *binary lifting* cu doi pointeri. Economia de memorie înseamnă economie de timp.

Ne întîlnim, din nou, cu o situație care cere să particularizați structurile de date, nu doar să le pictați. Un alt exemplu de astfel de problemă este: să se preproceseze un arbore astfel încît să putem răspunde eficient la interogări de forma $(u, v) =$ maximul pe calea $u - v$. Și aici putem face fiecare *jump pointer* să rețină maximul pe calea subîntinsă. Problema are și soluții mai eficiente, dar aceasta este relativ elementară.

Capitolul 20

Algoritmul lui Mo pe arbore

20.1 Limitele liniarizărilor

Ca o scurtă recapitulare, liniarizările ne ajută:

- pentru interogări pe subarbore: liniarizarea DFS atribuie fiecărui subarbore un interval compact din vector;
- pentru interogări pe calea de la un nod la rădăcină: liniarizarea Euler + vectorii de diferențe atribuie fiecărei căi un prefix din vector.

Uneori știm să procesăm și interogări pe căi arbitrare, vezi problema Max Flow. Dar algoritmii funcționează exprimînd calea (u, v) ca pe o combinație de căi (u, r), (v, r) și (l, r), unde r este rădăcina arborelui, iar $l = LCA(u, v)$. Așadar, avem nevoie ca funcțiile pe fiecare cale să fie **inversabile** (sume, xor-uri). În probleme de minim/maxim, valori distincte etc., ce știm pînă acum este insuficient.

20.2 Reducerea la algoritmul lui Mo

Putem folosi algoritmul lui Mo pentru a răspunde la interogări pe căi în unele cazuri neinversabile. Găsiți un tutorial destul de bun pe Codeforces, pe care îl vom relua aici. În esență,

1. Construim o liniarizare de tip Euler (două apariții pentru fiecare nod).
2. Transformăm interogările pe căi (u, v) în interogări pe intervale în liniarizare.
3. Sortăm interogările și le aflăm răspunsurile cu algoritmul lui Mo.

Desigur, misterul este la pasul 2. De aceea, să considerăm...

20.3 Un exemplu

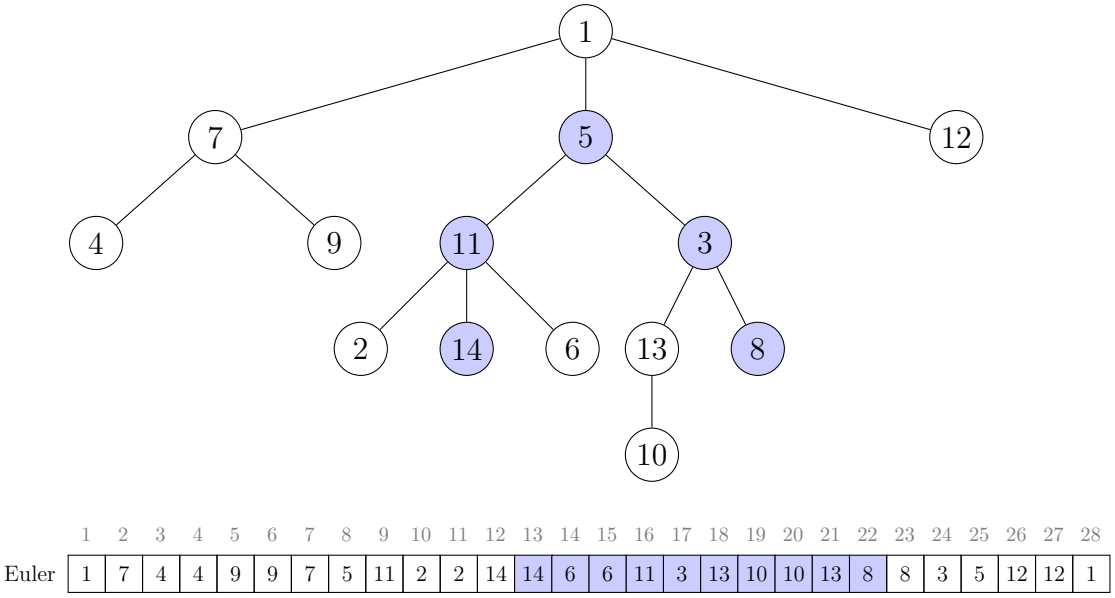


Figura 20.1: Un arbore cu liniarizarea Euler și intervalul corespunzător interogării pe calea $(14, 8)$.

Să considerăm interogarea $(14, 8)$, privitoare la nodurile 14, 11, 5, 3, 8. În liniarizarea Euler, ne interesează intervalul dintre timpii 13 și 22. Am ales acest interval deoarece el se întinde de la **ultima** apariție a lui 14 pînă la **prima** apariție a lui 8. Să facem niște observații privitoare la acest interval.

1. Nodurile 14, 11, 3 și 8 apar exact o dată.
2. Nodul $5 = LCA(14, 8)$ nu apare.
3. Nodurile 6, 13 și 10 apar de două ori.
4. Celelalte noduri (1, 2 etc.) nu apar.

Ne putem convinge ușor că aceste observații sînt generale. Trebuie doar să urmărim evoluția DFS-ului. De exemplu, observația (1): pornim de la momentul cînd DFS-ul părăsește nodul 14, deci va părăsi în curînd și nodul 11. Apoi explorează nodurile 3 și 8, dar nu apucă să le părăsească în intervalul ales. De aceea, toate aceste noduri apar exact o dată.

20.4 Un caz particular

Mai trebuie să tratăm și cazul cînd, pentru o interogare (u, v), avem $LCA(u, v) = u$, cu alte cuvinte u este strămoș al lui v.

Clarificare: mereu vom ordona perechea (u, v) în ordinea descoperirii în DFS. De aceea LCA-ul poate fi doar u, niciodată v. Aceasta deoarece un nod este descoperit înaintea tuturor descendenților săi.

Pentru interogarea (5,14), privitoare la nodurile 5, 11, și 14, vom considera intervalul de timp [8,

12], cuprins între primele apariții ale lui 5 și 14. Se schimbă doar observația (2): LCA-ul apare și el exact o dată.

20.5 Descrierea completă

Pentru o interogare (u, v) cu $t_i[u] < t_i[v]$:

1. Dacă u este strămoș al lui v, atunci considerăm intervalul din liniarizare $[t_i[u], t_i[v]]$. Nodurile de pe cale sînt cele care apar exact o dată în acest interval.
2. Dacă u **nu** este strămoș al lui v, atunci considerăm intervalul $[t_o[u], t_i[v]]$. Nodurile de pe cale sînt cele care apar exact o dată, plus nodul $LCA(u, v)$.

20.6 Structura de date necesară

Acum putem specifica ultimul amănunt: ce anume stochează structura de date pentru intervalul curent? Desigur, depinde de problemă, dar un mecanism este general.

Structura trebuie să țină minte ce noduri apar în ea exact o dată. De exemplu, pentru subsecvența (5 11 2 2 14), structura trebuie să știe că nodurile 5, 11 și 14 apar exact o dată. Dacă extindem spre dreapta secvența și încorporăm încă un 14, informația despre nodul 14 trebuie **ștearsă** din structură, căci 14 nu mai este pe cale.

Pe măsură ce extindem și contractăm intervalul cu algoritmul lui Mo, la anumite momente structura va reține date pentru intervale care nu corespund unor căi. Dar asta este OK cîtă vreme, în momentul unei interogări, structura este coerentă.

20.7 Probleme

20.7.1 Problema Dating (Codeforces)

enunț • surse

Dacă doriți probleme suplimentare, puteți încerca:

- ușoară: Count on a Tree II (SPOJ)
- grea: So Close Yet So Far (CodeChef)

Problema se încadrează destul de clar în situația sus-menționată. Interogările sînt pe cale și nu sînt ușor de combinat din bucăți. Dacă notăm cu $l = LCA(u, v)$, nu este evident cum am putea calcula, apoi însuma, valorile pe căile (u, l) și (l, v).

În schimb, dacă ne vine ideea să încercăm algoritmul lui Mo pe arbore, soluția este facilă. Structura curentă menține

- un boolean pentru fiecare nod, ca să știm ce noduri se află în structură;
- frecvența numerelor favorite, separat pentru fete și pentru băieți.

Includerea / excluderea unui nod este ușoară. Pe parcurs, menținem în permanență răspunsul (numărul de perechi pe care le putem forma).

Anexa include tot codul, dar esența este:

```
struct mo_tracker {
  bool on[MAX_NODES + 1];  // nodurile care apar exact o dată în interval
  int f[MAX_NODES + 1][2]; // frecvența numerelor favorite pentru fiecare gen
  int l, r;
  unsigned num_pairs;

  void init() {
    l = 1;
    r = 0;
  }

  void toggle(int pos) {
    int u = euler[pos];
    on[u] = !on[u];
    int sign = on[u] ? +1 : -1;
    f[nd[u].fav][nd[u].gender] += sign;
    num_pairs += sign * f[nd[u].fav][!nd[u].gender];
  }

  unsigned query(int target_l, int target_r, int extra_fav, bool extra_gender) {
    while (l > target_l) {
      toggle(--l);
    }
    while (r < target_r) {
      toggle(++r);
    }
    while (l < target_l) {
      toggle(l++);
    }
    while (r > target_r) {
      toggle(r--);
    }

    if (extra_fav) {
      return num_pairs + f[extra_fav][!extra_gender];
    } else {
      return num_pairs;
    }
  }
};
```

Comparația implementărilor

Implementarea pe care o consideram mai eficientă este cea cu Tarjan pentru LCA și cu liste proprii. Dar ea este doar cu 10% mai rapidă decît implementarea mai scurtă, cu metoda celor doi pointeri pentru LCA și cu vectori STL. Fie Codeforces este foarte neconsecvent, fie eu nu mai înțeleg lumea. 🥲

20.7.2 Problema Notdivisible (IIOT 2022-2023 runda 3)

enunț • surse

Problema ne cere să răspundem la interogări pe căi. Cred că abordarea doar cu liniarizare și interogări pe calea pînă la rădăcină nu duce nicăieri. În schimb, abordarea cu algoritmul lui Mo pe arbore este promițătoare.

Implementarea algoritmului în sine este relativ *boilerplate*: liniarizare de tip Euler, precalcularea informațiilor pentru interogări de LCA și algoritmul lui Mo propriu-zis. Acum, să vedem ce menținem în structura pentru intervalul curent.

Avem nevoie să includem/excludem o valoare de pe cale, preferabil în timp $\mathcal{O}(1)$. Dar ce înseamnă să „includem/excludem" o valoare? Aici este partea matematică a problemei. Desigur, n-am vrea să bifăm/debifăm toți multiplii acelei valori.

Ideea necesară este: ar putea fi vreodată răspunsul 15? Cu alte cuvinte, putem atribui niște valori pe o cale astfel încît $2, 3, \ldots, 14$ să nu fie răspunsul, dar 15 să fie? Este destul de clar că nu. Pentru ca 3 să nu fie răspunsul, este necesar ca una dintre valorile de pe cale să fie chiar 3, căci 3 nu are alți divizori. Dar valoarea 3 îl va exclude și pe 15 dintre răspunsurile posibile. De aceea, răspunsul la orice interogare va fi un număr prim.

Mai mult decît atît, dacă o cale conține o valoare compusă, cum ar fi 15, o putem ignora pur și simplu. Decizia dacă răspunsul pe acea cale va fi 2 sau 3 sau 5 etc. nu este influențată de numărul 15, deoarece niciun număr prim nu se divide cu 15.

Și atunci problema se reduce (1) la a menține evidența numerelor prime din intervalul curent, cu frecvențele lor și (2) la a găsi primul număr prim care nu apare în evidență. De aici rezultă și că avem nevoie doar de primele $n+1$ numere prime și le putem ignora pe cele mai mari. În cazul cel mai defavorabil, calea va conține toate cele n noduri cu numere prime distincte, deci al $n+1$-lea va fi răspunsul.

Vă reamintesc că am prezentat în capitolul de descompunere în radical problema Mexitate, din care am învățat o metodă să includem/excludem valori în $\mathcal{O}(1)$ și să determinăm mex-ul în $\mathcal{O}(\sqrt{n})$.

Moment de inginerie software

Haideți să zăbovim puțin asupra implementării. Ea este bine modularizată, delimitînd următoarele niveluri de abstracție:

1. Un generator de numere prime (ciur). Nu are nevoie să știe nimic despre lumea de afară, ci doar de un parametru n ca să știe cîte numere să genereze. Apoi expune metodele:
   - `get(k)`: returnează al k-lea număr prim;
   - `find(p)`: returnează numărul de ordine al lui p sau `NONE` dacă p nu este prim.
2. Un gestionar de mex-uri pe intervalul $[0, n)$. Nu are nevoie să știe nimic despre lumea de afară. În particular, nu știe că indicii pe care îi gestionează sînt indici în lista de numere prime. Sînt etaje complet separate. Expune metodele:
   - `add(x)`;
   - `remove(x)`;
   - `get_mex()`.
3. Un arbore. Se bazează pe generatorul de numere prime. Încă de la citire, înlocuiește numerele prime cu numerele lor de ordine, iar numerele compuse cu `NONE`. Expune metodele:
   - `read()`: citire;
   - `flatten()`: construiește liniarizarea Euler;
   - `make_query(u, v)`: convertește un query pe arbore într-un query pe liniarizare.
   - Nu expune metodele `is_ancestor(u, v)` și `lca(u, v)`. Nu este treaba restului lumii cum funcționează metoda `make_query()`.
4. Un gestionar pentru Mo. Se bazează pe gestionarul de mex. Nu știe nimic despre arbore. Expune metodele:
   - `init()`;
   - `query(l, r, extra_val)`.
5. Programul principal, scurt ($\sim$ 35 de linii), **singurul** care are viziunea de ansamblu. El orchestrează:
   - generarea numerelor prime;
   - citirea arborelui;
   - liniarizarea arborelui;
   - citirea și convertirea interogărilor;
   - ordonarea interogărilor conform algoritmului lui Mo;
   - procesarea interogărilor;
   - afișarea răspunsului.

Nu am pretenția că aceasta este singura implementare corectă. Putem coda același lucru și în C, fără încapsulare, dar atunci o implementare elegantă ar separa altcumva etajele logice. Ce este important este să nu lăsați aceste etaje să se încîlcească. Dacă variabilele și codul acestor etaje se amestecă, complexitatea programului crește pătratic.

Capitolul 21

Descompunere *heavy-light*

Descompunerea *heavy-light* (engl. *heavy-light decomposition* sau HLD, numită și *heavy path decomposition*) este încă o tehnică de liniarizare a arborilor, utilă pentru interogări pe căi.

HLD costă un factor suplimentar de $\mathcal{O}(\log n)$, așadar genul de întrebări la care răspundem în $\mathcal{O}(\log n)$ pe vector ne vor costa $\mathcal{O}(\log^2 n)$ pe arbore.

Înainte de a studia HLD, subliniez că este o unealtă puternică, dar greu de codat și lentă. Înainte de a vă repezi la ea, întrebați-vă dacă nu există o soluție mai simplă, adaptată nevoilor problemei. HLD intră doar în materia de lot (nu de baraj).

21.1 Limitările structurilor anterioare

Să pornim de la următoarea cerință. Se dă un arbore cu n noduri. Fiecare nod are o valoare. Trebuie să procesăm q operații de două tipuri:

1. `update(v, x)`: Valoarea nodului v devine x.
2. `max(u, v)`: Găsește valoarea maximă de pe lanțul $u - v$.

Să trecem în revistă uneltele pe care le-am studiat pînă acum.

- Un simplu DFS nu pare că poate propaga suficiente informații.
- Liniarizarea DFS se pretează la interogări pe subarbore, nu pe cale.
- Liniarizarea Euler se pretează la interogări pe calea de la un nod u la rădăcină. Pentru interogări de maxime, nu putem descompune căi arbitrare în diferențe de căi pînă la rădăcină.
- Tehnica *small-to large* nu ne ajută aici.
- LCA + *binary lifting* rezolvă doar problema fără actualizări. Fiecare pointer notează și maximul nodurilor peste care trece. Dar la actualizare, dacă avem un nod cu $\mathcal{O}(n)$ fii, vom fi nevoiți să actualizăm informația din $\mathcal{O}(n)$ pointeri.
- Similar și pentru orice tentativă de descompunere în radical.
- Algoritmul lui Mo pe arbore funcționează, dar lent. Structura pentru intervalul curent

trebuie să admită inserarea și ștergerea de valori și interogarea de maxim. Complexitatea va fi $\mathcal{O}(q\sqrt{n}\log n)$.

21.2 Descompunerea

Știm deja că liniarizarea DFS garantează că orice subarbore corespunde unui interval compact. Dar dorim mai mult de atît. Să examinăm Figura 21.1

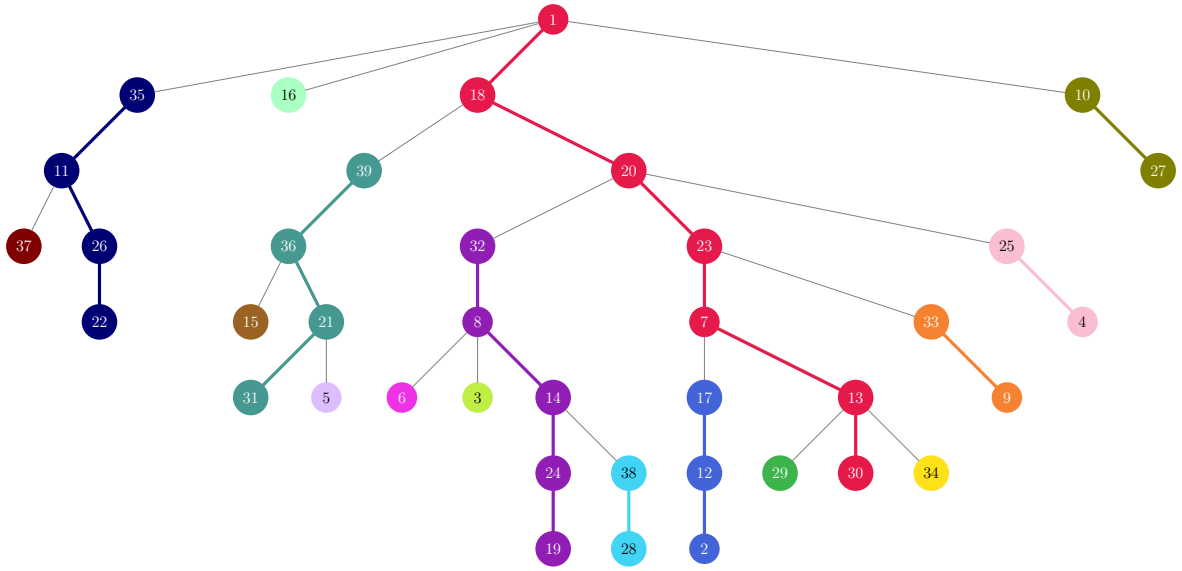


Figura 21.1: Un arbore în care muchia grea a fiecărui nod este îngroșată și colorată. Fiecare culoare indică un lanț de muchii grele consecutive.

Pentru fiecare nod intern u din arbore, identificăm fiul v cu subarborele maxim (cu cele mai multe noduri). Numim nodul v **fiu greu** (engl. *heavy*) al lui u, iar muchia (u, v) **muchie grea**. Ceilalți fii ai lui u și celelalte muchii care coboară din u se numesc **fii ușori** (engl. *light*), respectiv **muchii ușoare**. De exemplu, fiul greu al rădăcinii 1 este 18. Figura 1 indică muchiile ușoare cu linii subțiri, iar muchiile grele cu linii groase și colorate. Dacă un nod are mai mulți fii cu același număr maxim de noduri în subarbore, îl putem alege pe oricare ca greu.

Observăm că, dacă pornim din orice nod, putem urma muchia grea pînă la o frunză. Astfel iau naștere **lanțuri grele**. Am colorat muchiile grele din același lanț folosind aceeași culoare.

Mai facem un pas esențial: reorganizăm lista de adiacență a fiecărui nod ca să mutăm fiul greu primul (vom vedea cum implementăm asta în practică). Este important că această modificare nu schimbă răspunsurile la problemele tipice de arbori. Ordinea fiilor nu contează. Pentru arborele din Figura 21.1, versiunea reorganizată apare în Figura 21.2.

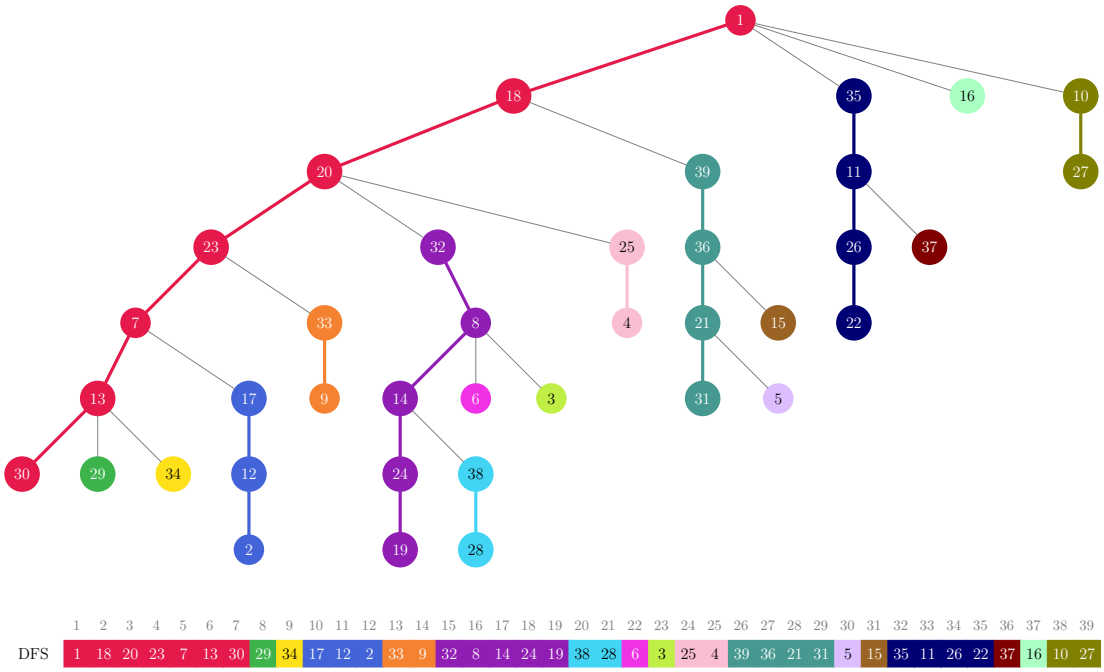


Figura 21.2: Același arbore în care, în fiecare listă de adiacență, am mutat fiul greu primul. Lanțurile corespund la intervale contigue în liniarizarea DFS.

Despre acest arbore putem da două garanții foarte puternice.

1. Lanțurile formate din muchii grele corespund la intervale contigue în liniarizare. Într-adevăr, fiecare fiu greu este primul nod pe care îl vizitează părintele său. De exemplu, la intrarea în nodul 32, următoarele noduri vizitate vor fi 8, 14, 24 și 19.
2. Calea de la orice nod la rădăcină vizitează, total sau parțial, cel mult $\log n$ lanțuri. Echivalent, calea conține cel mult $\log n$ muchii ușoare. Demonstrația este identică cu cea de la metoda small to large: cînd urcăm dintr-un fiu v în părintele u aflat pe alt lanț, prin definiție traversăm o muchie ușoară. Deci v este fiu ușor al lui u, ceea ce înseamnă că v are un frate greu h. Subarborele lui h este cel puțin la fel de mare ca al lui v, deci subarborele lui u este cel puțin dublu față de subarborele lui v.

Proprietatea (1) ne spune că putem trata lanțurile ca pe niște vectori. În particular, putem construi peste ele structuri de date ca arbori Fenwick sau arbori de intervale, care ne dau informații despre porțiuni de lanțuri în $\mathcal{O}(\log n)$. Proprietatea (2) ne spune că orice cale $u - v$ vizitează $\mathcal{O}(\log n)$ lanțuri. Avem, așadar, o metodă generală de a răspunde la interogări pe căi în $\mathcal{O}(\log^2 n)$.

21.3 Detalii de implementare

Facem întîi un DFS pentru calcularea mărimii subarborilor și a fiilor grei. Nu modificăm listele de adiacență, ci doar calculăm un cîmp `heavy` pe fiecare nod. Apoi facem un al doilea DFS, care liniarizează arborele, apelînd întîi fiul greu, apoi pe ceilalți.

Nu construim cîte o structură (AIB, AINT etc.) pe fiecare lanț! Ca la orice liniarizare, construim

o singură structură peste toate cele n noduri. Este datoria programului să nu acceseze intervale care „încalecă” mai multe lanțuri.

Ce structură folosim depinde de natura problemei. Dacă problema cere informații pe căi de la noduri la rădăcină, atunci căile vizitează doar prefixe (capetele de sus) ale lanțurilor. Deci s-ar putea ca un AIB să fie suficient. Dacă problema cere informații între orice două noduri, atunci căile pot vizita și porțiuni din mijlocul lanțurilor. Deci probabil avem nevoie de arbori de segmente.

Nu este necesar să implementăm LCA. În loc de asta, fiecare nod menține un pointer la capătul superior al lanțului. Astfel urcăm „naiv” din lanț în lanț, căci știm deja că numărul de lanțuri este logaritmic.

21.4 Probleme

21.4.1 Problema Heavy Path Decomposition (Infoarena)

enunț • surse

Aceasta este exact problema studiată la teorie. Vom citi codul.

Prima versiune este cea „canonică”. A doua versiune face o optimizare minoră care elimină nevoia calculării înălțimii nodurilor. Ea pornește de la următoarea observație. Dorim ca, odată ce u ajunge pe lanțul comun, să se oprească din urcat și să-l aștepte și pe v (pe acel lanț comun vom face ultima interogare). Dar, dacă u a ajuns pe lanțul comun, iar v încă nu, atunci în mod necesar $t_{in}[u] < t_{in}[v]$, căci toate nodurile de pe acel lanț sînt vizitate primele în DFS, fiind noduri heavy. Așadar, este suficient să comparăm timpii DFS ai lui u și v.

Subliniem că în funcția `query` parcurgem mereu lanțul al cărui capăt de sus are adîncime mai mare. De ce nu comparăm pur și simplu înălțimile lui u și v? Dați un contraexemplu!

O problemă echivalentă este Path Queries II (CSES).

21.4.2 Problema Disruption (USACO)

enunț • surse

Vom discuta trei soluții, de amorul artei.

Soluția cu HLD

Să considerăm o cale de înlocuire (u, v). Căror muchii le poate ea suplini dispariția? Doar celor de pe calea $u - v$ din arbore. Așadar, o soluție teoretică este: pentru fiecare cale de înlocuire (u, v) de cost c, notifică toate muchiile de pe calea $u - v$ că au la dispoziție o înlocuire de cost c. După procesarea tuturor căilor, pentru fiecare muchie tipărește costul minim despre care a fost notificată, sau -1 dacă muchia nu a primit notificări.

Concret, ce înseamnă aceste notificări? Fie $l = LCA(u, v)$. Atunci vrem să marcăm costul c pe căile $u - l$ și $v - l$. De aici ne poate veni ideea descompunerii *heavy-light*, care garantează că o cale de înlocuire va genera cel mult $\mathcal{O}(\log n)$ intervale de actualizat.

Ce structură folosim? Pe fiecare lanț avem nevoie de operațiile:

1. `range_set(l, r, c)`: pe fiecare poziție $l \leq i \leq r$, atribuie $v[i] = \min(v[i], c)$.
2. `query(i)`: returnează $v[i]$.

Desigur, `range_set` va descompune $[l, r]$ în niște intervale și va scrie valoarea c pe acele intervale. Cum procedăm după aceasta? Nu vă repeziți la implementarea cu propagare *lazy*! Interogările sînt punctuale, nu pe interval, deci este suficient să vizităm toți strămoșii unui nod (din aint) și să returnăm minimul valorilor găsite. Mai mult, toate interogările vor veni după toate actualizările. Deci putem face o singură propagare top-down la sfîrșitul actualizărilor. Atunci în fiecare frunză din aint vom avea exact răspunsul dorit.

Mai sînt două mici goluri de umplut:

1. Arborele de intervale trebuie inițializat cu ∞.
2. Conceptual, problema cere informații despre muchii. Dar, datorită DFS-ului, fiecare muchie unește un fiu de părintele său. Vom stoca informațiile în fiu.

Soluția cu *small-to-large*

Soluția oficială pornește de la altă observație teoretică. Orice muchie (u, v) poate fi înlocuită de o cale de înlocuire (x, y) cu proprietatea că x și y se află de părți diferite ale muchiei (u, v). Formulată în termeni de DFS, condiția este: muchia de la orice nod u la părintele său poate fi înlocuită de orice cale de înlocuire (x, y) care are **exact** un capăt în subarborele lui u. Să denumim o astfel de cale de înlocuire **viabilă pentru** u.

Așadar, dorim ca fiecare nod u să-și calculeze mulțimea de căi viabile și să o raporteze pe cea de cost minim. Atunci mulțimea unui nod u provine din:

1. reuniunea mulțimilor fiilor,
2. plus căile care au un capăt chiar în u,
3. minus căile care apar de două ori în (1) și (2), deoarece o cale care începe și se termină în subarborele lui u nu este viabilă pentru u.

Putem folosi tehnica *small-to-large* pentru a garanta că fiecare element este transferat între mulțimi de cel mult $\log m$ ori. Mulțimile însăși trebuie să poată raporta valoarea minimă, deci vor avea un cost logaritmic (de exemplu, cu `std::set`). Complexitatea totală este $\mathcal{O}(n \log m + m \log^2 m)$.

Puteți găsi o implementare foarte concisă (dar cam lentă) aici. Ea stochează în seturi perechi $(cost, id)$: costul unei căi servește la găsirea costului minim, iar ID-ul servește la găsirea duplicatelor, pentru eliminarea lor. Eu am ales o modalitate diferită: am sortat căile de înlocuire după cost, astfel încît ID-urile mai mici să corespundă la costuri mai mici.

Iată încă un detaliu de folosire a `set`-urilor. O implementare naivă va insera sau șterge căi cu următorul cod:

```
if (s.count(id)) {
  s.erase(id);
} else {
  s.insert(id);
}
```

Totuși, acest cod face două căutări per operație: una pentru găsirea elementului și a doua pentru inserare sau ștergere, după caz. Dar se poate și cu o singură căutare! Funcția insert, dacă eșuează, returnează un iterator la elementul care a cauzat eșecul.

```
std::pair<set::iterator, bool> p = s.insert(id);
if (!p.second) {
  // Inserarea a eșuat, iar p.first pointează chiar la elementul-duplicat.
  s.erase(p.first);
}
```

Soluția cu DFS exclusiv

Dacă vă amintiți, la problema Tree and Queries am discutat un DFS special care folosește o singură structură de date globală. DFS-ul garantează că fiecare nod u va avea acces, la un moment dat în timp, la structura de date care conține informații doar despre subarborele lui u.

Putem folosi acest algoritm și aici. Avem nevoie să includem/excludem noduri din structură, ceea ce presupune să includem/excludem căile care au un capăt în acele noduri. Așadar, avem nevoie de o structură de date cu operațiile:

1. Activează/dezactivează o poziție.
2. Returnează poziția minimă activă (sau o valoare specială dacă nu există poziții active).

Un AIB este suficient, cu căutare binară pentru (2). Ce complexitate are această soluție?

Benchmarks

Testele sînt mici, deci nu prea concludente, dar iată rezultatele:

- Cu descompunere *heavy-light*: 30 ms, 7 MB.
- Cu *small-to-large*: 74 ms, 18 MB.
- Cu *small-to-large* exclusiv: 55 ms, 12 MB.

21.4.3 Problema Subway Lines (Codeforces)

enunț • sursă

Problema este o aplicație directă a descompunerii *heavy-light*. Pentru a afla numărul de noduri comune între căile (a, b) și (c, d),

- Adăugăm 1 în toate nodurile de pe calea (a, b).
- Aflăm suma pe calea (c, d). Acesta este și răspunsul la interogare.
- Scădem 1 din toate nodurile de pe calea (a, b), pentru a face curățenie înaintea următoarei interogări.

Primele mele implementări foloseau structuri de date cu adăugări pe interval și sume pe interval: Un arbore de intervale cu propagare *lazy*, respectiv un AIB cu adăugare pe interval. Dar iată și o soluție doar cu HLD, fără alte structuri.[1]

- Vizităm calea (a, b). Aceasta vizitează porțiuni din niște lanțuri. Pe acele lanțuri notăm intervalul vizitat.
- Vizităm calea (c, d). Pe fiecare dintre lanțurile vizitate, intersectăm porțiunea vizitată de (c, d) cu porțiunea vizitată de (a, b), dacă există. Răspunsul la interogare este suma acestor intersecții.
- Revizităm calea (a, b) și ștergem intervalele notate pentru a face curățenie.

În practică, am folosit un contor global de operații ca să evit pasul de curățenie. La vizitarea căii (a, b) scriem contorul pe lanțurile vizitate, iar la vizitarea căii (c, d) comparăm contorul curent cu cel scris pe lanțurile vizitate.

Pentru a scurta codul, am unificat funcțiile pentru căile (a, b) și (c, d).

21.4.4 Problema Rafaela (Lot 2014)

enunț • surse

Ca observație preliminară, interogările pot fi reformulate astfel: dacă aș înrădăcina arborele în nodul u, care este populația maximă a unui subarbore al rădăcinii?

În practică vom fixa rădăcina în nodul 1. Fie $S(u)$ populația subarborelui nodului u. Iau naștere două cazuri:

1. Răspunsul la interogare este $S(1) - S(u)$ dacă populația maximă sosește pe muchia de la u la părinte. Desigur, $S(1)$ este populația întregului arbore, care este trivial de întreținut.
2. Răspunsul la interogare este $\max S(v)$, unde v este un fiu al lui u.

Soluție doar cu arbore de intervale

Pare natural să încercăm să menținem $S(u)$ pentru toate nodurile. Cînd populația lui u se modifică cu Δ, toți strămoșii lui u cîștigă Δ populație, deci parcurgem toate lanțurile de la u la rădăcină și adăugăm Δ pe prefixele corespunzătoare ale acestor lanțuri.

[1] Am învățat această rezolvare de la elevii mei. Pare că nu mi-am urmat propriul avertisment de a nu mă repezi la structuri de date. De aceea s-a inventat proverbul „să faci ce zice popa, să nu faci ce face popa".

Cum stabilim maximul pentru cazul (2) de mai sus? Este nevoie de puțin spirit de observație. Dacă insistăm să interogăm doar fiii lui u, aceștia sînt dispersați prin liniarizare. Dar este suficient să interogăm **tot subarborele** lui u, fără u însuși. Nu avem cum să greșim, căci, dacă maximul s-ar afla într-un nepot sau strănepot al lui u, cu atît mai mult fiul din care provine acel nepot sau strănepot ar avea populație și mai mare. Așadar, aflăm subarborele maxim cu o interogare pe intervalul $[t_{in}[u] + 1, t_{out}[u]]$.

Implementarea necesită doar un arbore de intervale cu adăugare pe interval și maxim pe interval.

Soluție cu arbore de intervale + caz separat pentru fiul greu

Nu-mi place să dau doar soluții de idee. Pentru aceea există matematica. 😁 Iată și o soluție mai muncitorească.

Din nou, la actualizarea unui nod facem operații de adăugare pe interval pe toate lanțurile. Acum diferențiem cazul (2) în două subcazuri: Subarborele cu populație maximă îl are fiul greu sau unul dintre fiii ușori. Ca să evităm confuzia, reamintesc că fiul greu este ales după numărul de noduri, care nu are neapărat și populația maximă.

Fiul greu îl putem interoga în $\mathcal{O}(\log n)$. Putem chiar să folosim doar un arbore Fenwick (AIB) cu actualizare pe interval și interogare punctuală.

Nu ne permitem să interogăm toți fiii ușori, dar este suficient să-l aflăm eficient pe cel mai populat. Cel mai „ciobănește", fiecare nod poate menține un `std::multiset` cu mărimile subarborilor ușori (așadar nu și mărimea subarborelui greu). Atunci pentru a răspunde la interogări comparăm trei valori:

1. Părintele (populația totală minus populația nodului însuși).
2. Fiul greu.
3. Fiul ușor (maximul din `multiset`).

Ce implică actualizările în această structură? Partea frumoasă este că **nu** trebuie să actualizăm seturile nodurilor de pe fiecare lanț vizitat, căci acele lanțuri constau, prin definiție, din muchii grele. Trebuie actualizate doar nodurile-părinte ale fiecărui lanț, căci doar acele muchii sînt ușoare.

De amorul artei, putem folosi și o structură mai elegantă decît seturile STL. O parcurgere BFS este suficientă, căci în acea parcurgere fiii fiecărui nod devin vecini. Așadar, ne este suficient un arbore de intervale peste parcurgerea BFS, cu actualizare punctuală și interogare de maxim pe interval. Iată o implementare.

Am observat că testele nu pedepsesc iterarea naivă prin fiii ușori. Am trimis și o ultimă sursă care ia 100p astfel.

Soluție cu descompunere în radical

Includ și această soluție pentru familiarizarea cu astfel de alternative la HLD. Ea obține 70 de puncte.

Procesăm operațiile în blocuri de mărime circa $\sqrt{q}$. La începutul fiecărui bloc facem un DFS ca să calculăm $S(u)$ pentru toate nodurile. Acum, pentru o interogare în nodul u, am dori să luăm maximul dintre valorile pentru fiii lui u ai căror arbori nu au suferit modificări (valori pe care le știm deja) și valorile fiilor modificați, aduse la zi. Similar, exteriorul subarborelui lui u poate să fi suferit modificări.

Dacă avem 10 interogări într-un singur nod u, pe permitem să consultăm toți fiii lui u **o singură dată**. Asta face, de exemplu, DFS-ul, al cărui efort total este $\mathcal{O}(n)$. Nu ne permitem să iterăm prin toți fiii lui u pentru fiecare interogare, căci ajungem la $\mathcal{O}(q \cdot n)$, de exemplu pentru un arbore-stea cu toate interogările în centrul arborelui.

Observațiile-cheie sînt că există interogări despre cel mult $\sqrt{q}$ noduri, iar pentru fiecare nod u cel mult $\sqrt{q}$ dintre subarborii lui u au modificări.

Restul nu este trivial, dar sînt doar detalii de implementare care urmăresc să obțină:

1. Efort $\mathcal{O}(n)$ la începutul blocului.
2. Vizitarea fiilor nemodificați ai lui u o singură dată per bloc $\rightarrow$ efort total $\mathcal{O}(n)$ per bloc.
3. Vizitarea fiilor modificați o dată per interogare $\rightarrow$ efort $\mathcal{O}(\sqrt{q})$ per interogare.

Dacă facem asta, obținem complexitatea totală $\mathcal{O}((n+q)\sqrt{q})$.

Distribuim actualizările din bloc în noduri. Colectăm actualizările din bloc și le sortăm după timpul DFS. Este important să avem în vedere că nu toate actualizările se aplică tuturor interogărilor. Depinde de ordinea lor cronologică dinaintea sortării. Din punct de vedere al unui nod u, actualizările vor fi:

- actualizări în afara subarborelui lui u;
- actualizări în u însuși;
- actualizări în primul fiu al lui u;
- ...
- actualizări în ultimul fiu al lui u;
- actualizări în afara subarborelui lui u.

Astfel putem afla trei cantități pentru fiecare interogare

1. modificările din afara subarborelui (spre părinte);
2. modificările pe cel mai populat fiu nemodificat;
3. modificările pe fiecare fiu modificat.

Calculăm aceste informații într-o singură trecere prin fiii lui u. Categoriile (1) și (2) cer efort $\mathcal{O}(1)$, iar categoria (3) cere efort $\mathcal{O}(1)$ per interogare, căci trebuie să decidem dacă modificarea se aplică fiecărei interogări sau nu.

21.4.5 Problema Doi arbori (Lot 2024)

enunț • surse

Facem întîi o observație teoretică. Drumul cel mai scurt de la G_u la H_v este calea $u - v$ plus un ocol dus-întors de la unul dintre nodurile de pe calea $u - v$ la o frunză activă. Așadar problema se reduce la două subprobleme:

1. (simplă) Aflarea distanțelor între noduri.
2. (grea) Aflarea distanței minime de la orice drum de pe calea $u - v$ la o frunză activă.

Facem și o altă observație: nodul 1 poate fi și el frunză (în sensul că poate avea grad 1). Dacă înrădăcinați arborele în nodul 1 ca toată lumea, aceasta poate fi o sursă de buguri. Pare totuși că testele nu acoperă acest caz.

Soluția cu HLD

Să înrădăcinăm arborele în 1 și să construim HLD. Acum fie o interogare (u, v) și fie $w = LCA(u, v)$. Prin definiție, orice cale în arbore întîi urcă, apoi coboară (și urcușul și coborîșul pot avea lungime 0). De aceea, cînd căutăm calea spre cea mai apropiată frunză activă de orice nod de pe calea $u - v$ este suficient să tratăm cazurile:

1. Pornim dintr-un nod de pe cale și coborîm pînă la frunză.
2. Pornim din w, urcăm și apoi coborîm.

Vom stoca pe lanțuri informații care să ne permită să tratăm aceste două cazuri.

Cazul I: Drumul spre frunză coboară

Să considerăm unul dintre lanțurile vizitate pe calea $(u - w)$. Din acest lanț vom interoga o porțiune $[x, y]$. x este fie capătul de sus al lanțului, fie w. y este fie u, fie nodul în care se ancorează lanțul vizitat anterior.

Dacă drumul spre o frunză pornește chiar din y, atunci ne-ar ajuta să aflăm rapid adîncimea minimă a unei frunze active din subarborele lui y. Știm să facem asta cu un arbore de intervale S_1 cu actualizări punctuale și interogări de minim. La activarea unei frunze, notăm chiar adîncimea frunzei pe poziția corespunzătoare în liniarizare. La dezactivarea frunzei, notăm ∞. Putem afla adîncimea minimă a unei frunze din subarborele lui y cu o interogare în S_1 pe intervalul subîntins de y. Distanța pînă la frunza f este diferența de adîncimi dintre f și y.

Dacă drumul spre frunză pornește de undeva dintr-un nod-ancoră $a \in [x, y)$, atunci distanța pînă la frunză este diferența de adîncimi dintre frunză și a. Desigur, nu vrem să interogăm fiecare nod din $[x, y)$, dar facem următoarea observație. O frunză de adîncime 30 ancorată într-un nod de adîncime 20 are aceeași distanță (10) ca și o frunză de adîncime 31 ancorată într-un nod de adîncime 21. De aceea, menținem un al doilea arbore de intervale S_2 care stochează pentru un nod u **diferența** dintre adîncimea oricărei frunze din subarborele lui u și adîncimea lui u.

Cazul II: Drumul spre frunză urcă din w

Vom urca pe lanțuri pînă la rădăcină. Din nou, fie $[x, y]$ porțiunea din lanțul curent pe care o interogăm.

Dacă drumul spre frunză pornește chiar din y, atunci ne interesează chiar adîncimea frunzei, iar din lanțul $w - y -$ frunză aflăm imediat distanța de la calea $u - v$ la frunză. Arborele de intervale S_1 este util și aici.

Dacă drumul spre frunza f pornește dintr-un nod-ancoră $a \in [x, y)$, atunci distanța totală de la calea $u - v$ la f este (notînd cu $d(u)$ adîncimea nodului u):

$$[d(w) - d(a)] + [d(f) - d(a)] = d(w) + [d(f) - 2 \cdot d(a)]$$

De aceea, avem nevoie de un al treilea arbore de intervale, S_3, care stochează pentru un nod u diferența dintre adîncimea oricărei frunze din subarborele lui u și **dublul** adîncimii lui u.

Detaliu (esențial)

La schimbarea stării unei frunze f vom recalcula în arborii de intervale valorile următoarelor noduri:

- frunza f;
- nodul în care se ancorează lanțul frunzei;
- nodul în care se ancorează lanțul anterior;
- etc.

Remarcăm, în particular, că frunza f nu notifică alți strămoși de pe propriul ei lanț. Fie x un astfel de strămoș. Se poate întîmpla următorul scenariu:

- Frunza f este activată.
- O altă frunză f' din subarborele lui x este activată.
- f' declanșează actualizarea lui x, care va include și informații despre f.
- Frunza f este deactivată.
- x nu mai află niciodată despre dezactivarea lui f.

Soluția este ca, la propagarea în sus a unei modificări, să interogăm în S_1 **doar subarborele light al unui nod**. Ce modificare este necesară ca să putem identifica intervalul corespunzător din liniarizare?

Soluția cu descompunere în radical

Menționez foarte pe scurt și o soluție în $\mathcal{O}((n+q)\log q)$. Ea nu trece testele mari. Dar, în general, soluțiile cu descompunere în radical sînt o alternativă viabilă și posibil mai ușor de înțeles. Iar matematic $\sqrt{n}$ nu este departe de $\log^2 n$ pe plaja de valori pentru n din problemele obișnuite.

Precalculăm liniarizarea Euler cu repetiție a arborelui, peste care construim tabela rară de RMQ. Astfel putem răspunde la interogări de LCA în $\mathcal{O}(1)$ (avem nevoie de $\mathcal{O}(1)$ deoarece vom face $\mathcal{O}(q\sqrt{q})$ astfel de interogări).

Precalculăm *binary lifting*, preferabil varianta cu doar doi pointeri. Astfel vom putea calcula în $\mathcal{O}(\log n)$ o anumită informație pe căi.

Procesăm interogările în blocuri de mărime $\sqrt{q}$. Clasificăm frunzele active în două categorii:

1. Frunze safe: frunze active pe durata întregului bloc.
2. Frunze unsafe: frunze active pe porțiuni din bloc.

La începutul fiecărui bloc, facem următoarea preprocesare în $\mathcal{O}(n)$:

1. Clasifică frunzele active în *safe* și *unsafe*.
2. Calculează distanța minimă de la fiecare nod la o frunză *safe*. Este suficient un BFS multi-sursă.
3. Pentru fiecare pointer de salt, calculează distanța minimă de la orice nod acoperit de salt la o frunză *safe*. Este suficient un DFS.

Efortul total pentru preprocesări este $\mathcal{O}(n\sqrt{q})$. Procesăm operațiile din bloc și ținem evidența mulțimii de frunze *unsafe*. Pentru o interogare (u, v), răspunsul va fi minimul dintre

1. Distanța minimă de la cale la o frunză *safe*, calculată în $\mathcal{O}(\log n)$ cu *jump pointers*.
2. Pentru fiecare frunză *unsafe* f, distanța $d(u, f) + d(f, v)$.

Deoarece fiecare dintre cele q interogări poate consulta $\mathcal{O}(\sqrt{q})$ frunze *unsafe*, efortul total este $\mathcal{O}(q \log q)$ pentru interogări.

Momentul de inginerie: măsurarea timpului

Pentru soluția cu descompunere în radical, nu am reușit să reduc timpul de rulare sub 4-5 secunde pe testele mari. Nevenindu-mi să cred că poate fi nevoie de atît de mult timp, am măsurat numărul de apeluri la diverse funcții și timpul petrecut în ele. Las aici codul pe care l-am folosit, în caz că îl ajută pe inginerul din voi.

```
#include <sys/time.h>

long long t0, t_total, cnt;

void start_clock() {
  timeval tv;
  gettimeofday(&tv, NULL);
  t0 = 1'000'000LL * tv.tv_sec + tv.tv_usec;
}

void stop_clock() {
  timeval tv;

  gettimeofday(&tv, NULL);
  long long t = 1'000'000LL * tv.tv_sec + tv.tv_usec;
  t_total += t - t0;
}
```

```
int preprocess_block(int start) {
  cnt++;
  start_clock();   // <---------------
  int end = classify_leaves(start);
  bfs_from_safe_leaves();
  jdist_dfs(1);
  stop_clock();    // <---------------

  return end;
}

int main() {
  ...
  fprintf(stderr, "Time: %0.6lf cnt: %lld\n", t_total * 0.000001, cnt);
}
```

21.4.6 Problema Query on a Tree VI (CodeChef)

enunț • sursă

Problema are un enunț simplu și elegant, dar este foarte laborioasă.

Am „trișat" un pic la rezolvare, căci știam că este problemă de HLD și mi-am pus întrebarea: HLD ne ajută să facem operații pe căi, deci cum aș folosi-o la o problemă despre subarbori? Așa am ajuns la o soluție parțială.

Definim **domeniul unui nod** u ca fiind mulțimea de noduri din subarborele lui u, de aceeași culoare cu u, și legate de u prin noduri de aceeași culoare. Fie $S(u)$ = mărimea domeniului lui u. Atunci, ca să răspundem la o interogare despre u, urcăm din u cît timp se poate, mergînd pe noduri de aceeași culoare. Fie w ultimul nod atins. Răspunsul este $S(w)$.

Dacă folosim HLD, îl putem găsi pe w dacă menținem pe fiecare lanț un AIB de culori (0/1) cu suport pentru căutare binară.

Treburile se complică la actualizare. Cînd un nod u își schimbă culoarea, schimbările necesare sînt:

1. Toți strămoșii consecutivi care au vechea culoare a lui u pierd $S(u)$ din domeniu.
2. Recalculăm $S(u)$.
3. Toți strămoșii consecutivi care au noua culoare a lui u cîștigă $S(u)$ la domeniu.

Operațiile (1) și (3) sînt operații de adăugare / scădere pe interval. Dar nu am reușit să implementez eficient operația (2). Astfel ajungem la soluția oficială. Menținem pentru fiecare nod **două valori**:

1. $S_B(u)$ = mărimea domeniului lui u dacă u ar fi negru.
2. $S_W(u)$ = mărimea domeniului lui u dacă u ar fi alb.

Actualizarea acestor valori presupune niște cazuri particulare. De exemplu, dacă un nod u devine alb, atunci toți strămoșii săi negri, **dar și primul strămoș alb**, pierd $S_B(u)$ din $S_B(w)$.

Temă de gîndire: „Aint pe timp"

Am implementat și o altă soluție bazată pe metoda TODO:referință-internă ștergerii dintr-o structură care nu admite (ușor) ștergeri, cu un arbore de intervale indexat după timp. Problema fiind una de conectivitate online, ideea mi s-a părut bună. Dar am făcut o greșeală copilărească: aici nu activăm și dezactivăm muchii, ci noduri. Deci o operație poate cauza $\mathcal{O}(n)$ operații în pădurea de mulțimi disjuncte. Sursa mea ia TLE pe teste adversariale.

Nu știu dacă putem reduce complexitatea. Voi ce credeți?

21.4.7 Problema Adă caii (Lot 2025)

enunț • sursă

Să pornim de la o descompunere *heavy-light* tradițională. Nu ne batem prea mult capul cu operația de interogare. Aceea este punctuală, deci probabil că orice structură vom construi peste lanțuri vom putea să aflăm o valoare punctuală. În schimb, să analizăm migrația spre un nod u. Observăm că:

1. Pe un lanț L aflat între u și rădăcină, caii migrează către nodul în care calea către u se desprinde din L.
2. Pe alte lanțuri, caii migrează în sus.
3. Caii pot sări de pe un lanț pe altul.

De exemplu, în figura 21.2, migrația către nodul 28 (albastru-deschis) înseamnă că:

1. Pe lanțul roșu 1-30, caii migrează către nodul 20.
2. Caii din nodul 20 coboară pe lanțul violet în nodul 32.
3. Pe lanțul violet 32-19 caii migrează către nodul 14.
4. Caii din nodul 14 coboară pe lanțul albastru deschis în nodul 38.
5. Pe alte lanțuri, caii migrează în sus.
6. Caii aflați în nodurile din vîrful altor lanțuri migrează pe lanțul-părinte.

Așadar, pe fiecare lanț dorim să stocăm o structură de date care implementează rezonabil de eficient operațiile:

1. Migrează toate valorile spre stînga (spre rădăcină).
2. Migrează toate valorile spre o poziție din structură.
3. Citește / modifică o poziție.

Operațiile (1) și (3) sînt ușor de implementat în $\mathcal{O}(1)$ cu un buffer circular pe fiecare lanț. Am ales să stochez aceste buffere într-un singur vector global, separat de nodurile arborelui.

În schimb, operația (2) pare a fi $\mathcal{O}(\text{lungime_lanț})$. De aceea, vom pune o limită de $\mathcal{O}(\sqrt{n})$ pe lungimea oricărui lanț. Dacă un lanț ar fi, în mod natural, mai lung de atît, după primele $\mathcal{O}(\sqrt{n})$

noduri vom forța începerea unui lanț nou. Aceasta va cauza apariția mai multor lanțuri, dar nu multe.

Cu această modificare, și pentru o optimizare posibil valoroasă, vom implementa (2) copiind naiv jumătatea mai scurtă a vectorului și shiftînd-o pe cea mai lungă conform bufferului circular.

În plus, ținem evidența lanțurilor nevide, deoarece doar pe acelea avem de implementat operația (1). Pot exista $\mathcal{O}(n)$ lanțuri, de exemplu într-un arbore stea, dar pare imposibil să menținem populații de cai pe $\mathcal{O}(n)$ lanțuri. Cu fiecare operație de migrare, unele populații de cai se vor ciocni și se vor însuma.

Sînt 99% sigur că acest algoritm este eficient, dar demonstrația este alunecoasă. Intuiția mea îmi spune astfel. Temerea noastră este că poate exista o migrație (sau mai multe) care să necesite efort $\mathcal{O}(n)$. Aceasta s-ar putea întîmpla dacă există $\mathcal{O}(\sqrt{n})$ lanțuri pe calea de la u la rădăcină (datorită limitei de lungime), iar pe fiecare lanț este nevoie să facem operația (2) și să copiem naiv $\mathcal{O}(\sqrt{n})$ elemente.

Dar, dacă drumul de la u la rădăcină include vreun lanț complet, pe acela nu vom face efort $\mathcal{O}(\text{lungime})$, ci doar $\mathcal{O}(1)$, deoarece migrația în acel lanț va fi doar în jos, spre u! De exemplu, în figura 21.2, pentru $u = 30$ elementele de pe lanțul roșu migrează doar în jos.

De aceea, un caz adversarial ar fi o cale de la u la rădăcină care, ori de cîte ori schimbă lanțul, se „înțeapă" undeva la jumătatea noului lanț. Or acest lucru este imposibil datorită modului în care funcționează decompunerea *heavy-light*. Din punct de vedere al nodului de înțepare, lanțul ar prefera să continue pe calea spre u, dacă aceasta este destul de lungă. Așadar, dacă am avea $\mathcal{O}(\sqrt{n})$ lanțuri, toate de lungime $\mathcal{O}(\sqrt{n})$, numărul total de noduri de pe acea cale ar deveni $\mathcal{O}(n)$ și majoritatea lanțurilor s-ar înțepa în capătul de jos al lanțului anterior.

Capitolul 22

Descompunere în centroizi

Încheiem studiul arborilor cu tehnica descompunerii în centroizi. Ea ne permite să rezolvăm probleme rezistente la alte abordări.

Ca și descompunerea *heavy-light*, descompunerea în centroizi este o unealtă avansată. Este bine să o știți, căci uneori nu găsim o soluție mai elementară. Dar problemele de ONI și Baraj ONI se pot rezolva și cu tehnici mai simple.

22.1 Definiție

Fiind dat un arbore cu n noduri, un **centroid** este un nod al arborelui cu proprietatea că, dacă îl eliminăm, atunci toți arborii rezultați au cel mult $\lfloor n/2 \rfloor$ noduri.

Centroidul este un centru de masă, definit în funcție de greutatea subarborilor (ca număr de noduri). A nu se confunda cu **centrul**, care este definit în funcție de distanțe (el minimizează distanța maximă pînă la alt nod).

22.2 Proprietăți

Orice arbore are un singur centroid sau doi centroizi adiacenți. Pentru demonstrație, să considerăm figura următoare.

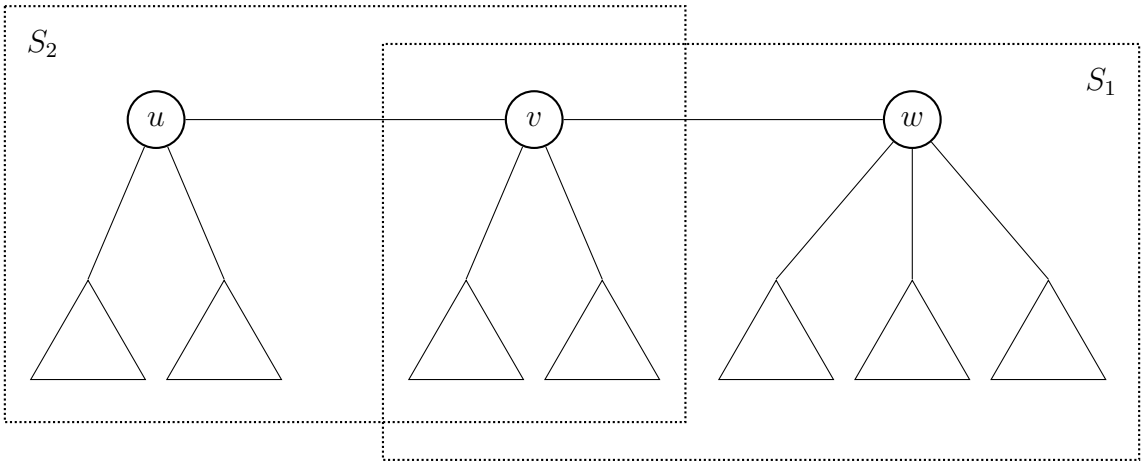


Figura 22.1: Un arbore ipotetic cu doi centroizi aflați la distanță 2.

Să presupunem că u și w ar fi centroizi aflați la distanță 2. Din punctul de vedere al lui u, subarborele S_1 are cel mult $\lfloor n/2 \rfloor$ noduri. Din punctul de vedere al lui w, subarborele S_2 are, de asemenea, cel mult $\lfloor n/2 \rfloor$ noduri. Rezultă că

$$|S_1| + |S_2| \leq n$$

Dar acest lucru este imposibil, căci în mod evident S_1 și S_2 acoperă întreg arborele, iar nodul v (și orice eventuali subarbori ai săi) sînt acoperiți de două ori. Deci $|S_1| + |S_2| \geq n + 1$.

Așadar, dacă există mai mulți centroizi, atunci ei sînt adiacenți. Dar într-un arbore nu putem amplasa mai mult două noduri astfel încît oricare două să fie adiacente. Deci există cel mult doi centroizi.

22.3 Găsirea unui centroid

După cum vom vedea în problema Finding a Centroid, algoritmul de găsire a unui centroid este simplu. Facem un DFS pentru calculul mărimilor subarborilor. Apoi, pornind din rădăcină, căutăm succesiv fii cu mai mult de $n/2$ noduri și coborîm în ei pînă cînd nu mai găsim niciunul. Complexitatea este $\mathcal{O}(n)$.

22.4 Descompunerea în centroizi

Descompunerea în centroizi repetă de mai multe ori următoarea procedură:

1. Găsește un centroid pentru subarborele curent.
2. Colectează informații din subarborele curent care ajută la rezolvarea problemei. Probabil folosește unul sau mai multe DFS-uri pornind din centroid.
3. Elimină centroidul.
4. Reapelează recursiv algoritmul pentru subarborii disjuncți care iau naștere.

5. Cazul de bază este un subarbore cu un singur nod. Nodul este centroid.

Iată un exemplu în care descompunerea în centroizi necesită cinci niveluri de adîncime.

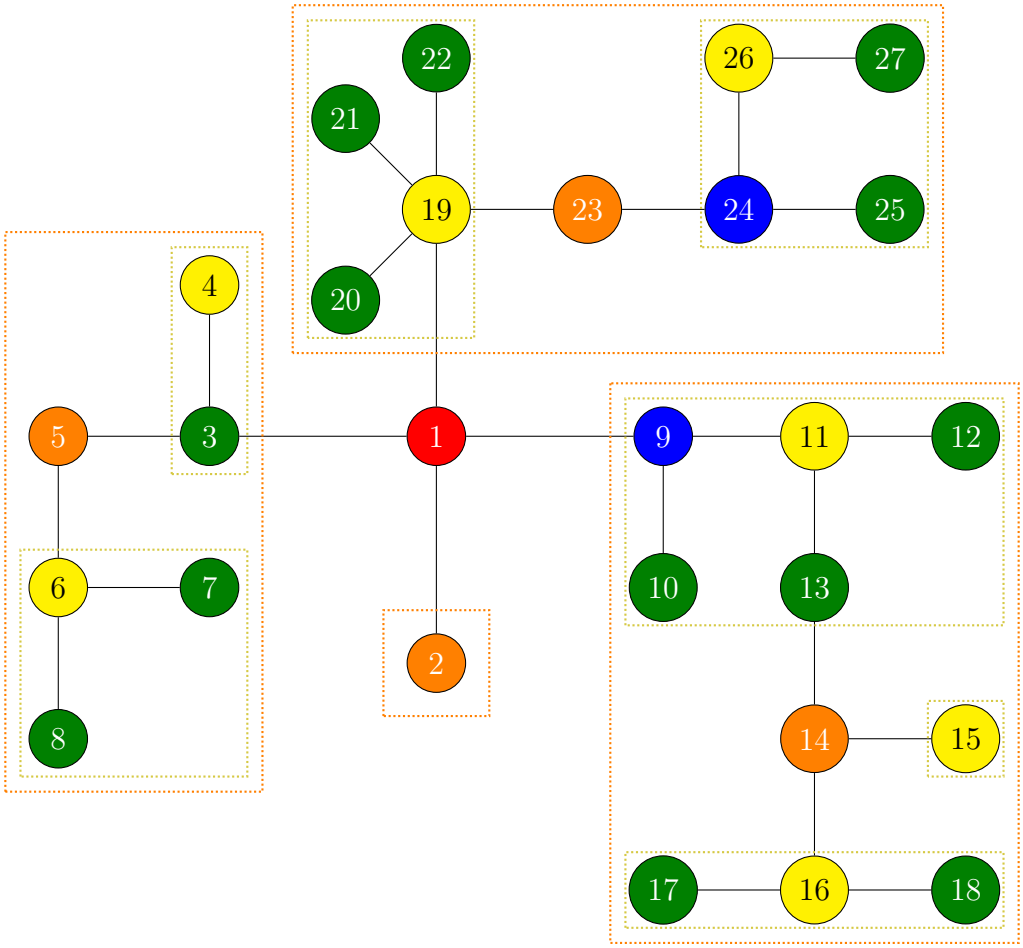


Figura 22.2: Un arbore descompus în centroizi. Nodul 1 (roșu) este centroid de nivelul 1. Pentru subarborii rezultați prin eliminarea lui 1, nodurile portocalii sînt centroizi de nivelul 2. Nodurile galbene, verzi și albastre sînt centroizi de nivelurile 3, 4 și respectiv 5.

Această abordare are avantajul că fiecare nod face parte din cel mult $\lceil \log n \rceil$ descompuneri. De exemplu, nodul 8 face parte din:

- Arborele inițial (cu centroidul 1).
- Arborele cu centroidul 5 (chenar portocaliu).
- Arborele cu centroidul 6 (chenar galben).
- Arborele constînd doar din nodul 8, cu centroidul 8.

De aceea, o implementare care face efort $\mathcal{O}(\text{mărime_subarbore})$ în fiecare subarbore, cum ar fi de exemplu un DFS, va ajunge la o complexitate totală de $\mathcal{O}(n \log n)$.

Atenție! Efortul trebuie să depindă de mărimea subarborelui, **nu** de n. Fiecare nod va deveni centroid la un nivel mai mare sau mai mic de fărîmițare. Așadar, dacă orice bucată din cod face efort $\mathcal{O}(n)$ per nod, atunci complexitatea totală va deveni $\mathcal{O}(n^2)$. Uneori această condiție

necesită un efort conștient, de exemplu la dimensionarea, inițializarea și ștergerea structurilor de date pentru subarborele curent.

Dacă efortul într-un subarbore este mai mare decît liniar, atunci descompunerea în centroizi adaugă un factor logaritmic. De exemplu, dacă efortul într-un subarbore de mărime k este $\mathcal{O}(k \log k)$, atunci complexitatea totală devine $\mathcal{O}(n \log^2 n)$. Pentru detalii, vedeți Master Theorem, cazul 2.

Descompunerea în centroizi ne spune „este OK să apelezi un DFS din fiecare nod". Desigur, ce face acel DFS depinde de la problemă la problemă. Dar ocazional putem scrie algoritmi relativ naivi (plus șablonul de cod pentru descompunere) și totuși să obținem complexitate $\mathcal{O}(n \log n)$.

22.5 Probleme

22.5.1 Problema Finding A Centroid (CSES)

enunț • surse

Nu este foarte mult de spus, problema este directă. Reținem șablonul util pentru toate problemele de descompunere în centroizi:

1. DFS-ul inițial dintr-un nod oarecare, cu aflarea mărimii subarborilor.
2. Caută un fiu greu al nodului curent.
3. Dacă există un fiu greu, coboară în el și mergi la pasul anterior.
4. Ultimul nod găsit este centroidul.

Există două implementări. Cea recursivă (la coadă) combină pașii (2) și (3) într-o singură funcție. Odată identificat fiul greu, reapelăm căutarea centroidului din acel fiu. Implementarea iterativă separă pașii (2) și (3) și rezolvă pasul (3) cu un `while`.

22.5.2 Problema Mystery Tree (CodeChef)

enunț • sursă

Iată o problemă simpatică, interactivă. Se dă un arbore cu $n \leq 200\,000$ de noduri, cu valori în noduri. Structura arborelui este cunoscută, dar valorile nu. Trebuie să găsim un maxim local: un nod cu valoarea mai mare sau egală cu valorile tuturor vecinilor săi. Putem pune cel mult 20 de întrebări de forma „Este u un maxim local?". Interactorul ne răspunde cu -1 dacă am găsit un maxim local sau, în caz contrar, cu un nod v cu valoare strict mai mare decît a lui u.

Cum am rezolva problema pe un vector?

Pentru generalizarea pe arbore, trebuie să punem fiecare întrebare despre un nod care reduce problema la jumătate sau mai puțin. Acesta este exact centroidul subproblemei curente.

Ca implementare, aici putem vedea cum ștergem un nod. Nu modificăm listele de adiacență, ci doar îl marcăm ca fiind șters.

Din păcate, la această problemă nu putem trimite surse. Dar am încredere în corectitudinea soluției mele!

22.5.3 Problema Ciel the Commander (Codeforces)

enunț • surse

Soluția cu descompunere în centroizi

Problema se reduce absolut elementar la decompunerea în centroizi. În centroidul întregului arbore scriem A, în centroizii de nivel 2 scriem B etc. Alfabetul este suficient deoarece $\lceil \log_2 100\,000 \rceil = 17$.

Soluție cu un singur DFS și măști de biți

Putem folosi și următoarea abordare *greedy*. În frunze scriem Z, căci nu avem niciun motiv să nu facem asta. În părinții frunzelor putem scrie Y. Complicațiile apar cînd un nod are un fiu Y și un fiu Z. Ce facem în cazul general?

Să introducem noțiunea de **vizibilitate**. Spunem că o literă X este **vizibilă** dintr-un nod u dacă există o apariție a lui X undeva în subarborele lui u care să nu fie **mascată** de o literă mai mică decît X pe calea pînă la u.

Ideea centrală este că o literă mai mică maschează toate literele mai mari din subarbore, care nu mai necesită alte acțiuni.

Atunci am putea pune în u cea mai mare literă care satisface două condiții:

1. Trebuie să fie mai mică decît orice literă care este vizibilă din doi fii diferiți ai lui u, deoarece trebuie să separăm cele două apariții.
2. Trebuie să fie diferită de orice literă vizibilă din u.

Odată ce ia această decizie, u returnează la părinte mulțimea de litere vizibile. Putem folosi măști de biți și operații de aflare a LSB pentru a obține o soluție în $\mathcal{O}(n)$.

22.5.4 Problema Fixed-Length Paths I (CSES) (din nou)

enunț • sursă

Ne reîntîlnim cu această problemă, pe care am rezolvat-o cu tehnica *small-to-large*. În lipsa acelei idei, descompunerea în centroizi ne poate da o idee mai directă.

O cale de lungime k (și orice cale în general) fie trece prin centroidul arborelui, fie este conținută complet într-unul dintre subarbori. Deci putem însuma răspunsurile întrebărilor, pentru fiecare centroid, „Cîte căi de lungime k trec prin tine?”.

Iar răspunsul la această întrebare este comparativ mai simplu decît tehnica *small-to-large*. Facem un DFS din centroid. Fiecare fiu al centroidului raportează distribuția pe adîncimi a nodurilor

din subarborele său. Părintele (centroidul) combină aceste informații. Putem face asta în diverse feluri, dar esența este: un nod aflat la adîncime d formează căi de lungime k cu toate nodurile aflate la adîncime $k - d$ în fiii centroidului vizitați anterior.

Această implementare este de peste 2 ori mai lentă decît implementarea cu *small-to-large*.

Întrebare despre cod: care este rostul variabilei `max_depth`?

22.5.5 Problema Xenia and Tree (Codeforces)

enunț • surse

Soluția cu descompunere în radical

Menționăm sumar și această soluție, ca să fiți mereu alerți la posibilitățile de rezolvare. Este o soluție lunguță, dar nu grea. Soluția seamănă mult cu cea de la Doi arbori și este mai simplă decît aceea.

Procesăm operațiile în blocuri de circa $\sqrt{q}$. La începutul unui bloc facem o parcurgere BFS din toate nodurile roșii. Acesta ne va oferi pentru fiecare nod u o cantitate $d[u]$ care reprezintă distanța minimă de la u pînă la orice nod roșu.

Acum, răspunsul la o interogare din acel bloc va fi minimul dintre $d[u]$ și distanța de la u pînă la orice nod colorat în roșu cîndva în blocul curent. Dar în blocul curent colorăm în roșu cel mult $\sqrt{q}$ noduri, deci putem răspunde la interogări calculînd cel mult $\sqrt{q}$ distanțe.

Pentru a obține timp $\mathcal{O}(\sqrt{q})$ per interogare, este necesar să putem calcula distanțe în $\mathcal{O}(1)$, ceea ce se reduce la a calcula LCA în $\mathcal{O}(1)$. Vom folosi structura cu care ne-am mai întîlnit: o parcurgere Euler peste care construim tabela rară de RMQ.

Soluția se încadrează lejer în timp (sub 1s pe limită de timp de 5s).

Soluția cu descompunere în centroizi

Și acum, la subiectul zilei! Calea de la un nod u la orice nod roșu (sau, în general, orice alt nod) va fi cuprinsă complet într-un subarbore centroid: în cel mai rău caz subarborele centroid al nodului 1 (care este întregul arbore), dar posibil într-un subarbore mai mic.

Prin natura ei, descompunerea în centroizi garantează că orice nod u face parte din cel mult $\log n$ subarbori centroizi sau, echivalent, are cel mult $\log n$ strămoși centroizi. Și atunci, la o interogare, am putea să-i verificăm pe toți acești strămoși. Așadar, pentru interogarea u și pentru fiecare v strămoș centroid al lui v, ne întrebăm:

1. Care este distanța de la u la v?
2. Care este distanța de la v pînă la orice nod roșu din subarborele centroid al lui v?

Partea frumoasă este că putem accesa aceste distanțe în $\mathcal{O}(1)$ fără *binary lifting*, LCA, RMQ sau alte structuri suplimentare. Reamintiți-vă că **ne permitem cîte un DFS din fiecare**

centroid. Într-un astfel de DFS dintr-un centroid de nivel k vom calcula distanțele de la fiecare nod din subarborele centroidului pînă la centroid. Facem această preprocesare înainte de a procesa interogările.

Memoria necesară pentru aceste informații este $\mathcal{O}(n \log n)$.

Ce implică actualizările? La colorarea unui nod u, toți strămoșii centroizi ai lui u trebuie notificați că în subarborele lor a apărut un nod nod roșu. Fiecare strămoș v își reține distanța pînă la cel mai apropiat nod roșu din subarborele său și și-o minimizează cu distanța $u - v$ (pe care u o cunoaște din DFS-urile de preprocesare).

Complexitatea provine din:

1. Descompunerea în centroizi, care știm că este $\mathcal{O}(n \log n)$.
2. Procesarea celor q operații. Colorările și interogările presupun modificarea sau consultarea unui vector de $\log n$ elemente.

Rezultă o complexitate totală de $\mathcal{O}((n + q) \log n)$. Cum este și de așteptat, sursa este de peste două ori mai rapidă decît cea bazată pe descompunere în radical.

22.5.6 Problema Flareon (Lot 2017)

enunț • sursă

Să presupunem că ne-a venit deja ideea să folosim descompunerea în centroizi. 😈 Să considerăm o flacără care pornește dintr-un nod u. Fie $c_1, c_2, \ldots, c_k$ strămoșii centroizi ai nodului u, unde c_1 este rădăcina arborelui (nodul 1 în majoritatea implementărilor), iar c_k este chiar nodul u. Vom contabiliza separat contribuția flăcării pe căi care trec prin c_1, pe căi care trec prin c_2 fără să ajungă la c_1 etc.

Deci vom încerca o soluție în doi pași. Fie r rădăcina (centroidul) subarborelui curent.

1. Colectăm în r lista de flăcări care provin din subarborele lui r. Pentru fiecare flacără ne interesează puterea cu care ajunge în rădăcină.
2. Propagăm această listă de flăcări în tot subarborele. Nu avem voie să propagăm o flacără în subarborele din care provine.

Notînd cu s mărimea subarborelui curent, este important ca ambii pași să funcționeze în $\mathcal{O}(s)$. În particular, nu putem colecta flăcările într-o listă, ci trebuie să le compactăm cumva într-o structură de mărime cel mult s.

Reprezentarea flăcărilor

Ideea compactării este următoarea. Dacă dintr-un nod pornesc 3 flăcări de mărime 2, 5 și 10, le putem unifica într-o singură flacără de mărime 17, a cărei putere scade cu 3 la fiecare muchie parcursă. Așadar, reținem doar suma puterilor și numărul de flăcări. La fiecare deplasare, suma puterilor scade cu numărul de flăcări.

Aceasta funcționează... o vreme. În primul nod avem putere 17, în următorul 14 (= 1 + 4 + 9), iar în următorul 11 (= 0 + 3 + 8). Dar aici flacăra mică se stinge, iar puterea trebuie să continue să scadă doar cu 2. Următoarea putere este 9 (= 2 + 7).

Totuși, sîntem pe calea bună. Putem grupa flăcările după putere. Să spunem că în rădăcina r am acumulat $f[p]$ flăcări de putere p, unde $p \geq 1$. Atunci cînd propagăm aceste flăcări în subarbore, la un nod de la adîncimea d trebuie însumate doar flăcările cu $p \geq d$. Cînd reapelăm DFS-ul pentru un fiu, ne aflăm la adîncime $d+1$, deci le scădem din numărul total de flăcări pe cele $f[d]$ care s-au stins.

Remarcăm că puterile pot fi mari (problema nu specifică, dar ele se apropie de 10^9). Nu putem stoca un vector atît de mare, dar nici nu este nevoie. Față de abordarea inițială (doar cu numărul de flăcări și numărul puterilor), ne interesează și frecvențele **doar pentru flăcările care se vor stinge cîndva**. Flăcările care au o putere mai mare sau egală cu s nu se vor stinge în acest subarbore.

Recapitulînd, informațiile necesare în rădăcină sînt (1) numărul de flăcări, (2) suma puterilor acestora și (3) distribuția pe frecvențe a flăcărilor de putere mai mică decît s.

Evitarea propagării în același fiu

Cum spuneam, o flacără nu trebuie propagată din r înspre același fiu din care flacăra a ajuns în r. Văd două abordări aici.

Una este să stocăm informații despre fiecare fiu în parte: ce flăcări ajung acolo și cu ce puteri rămase? Cînd propagăm flăcările printr-un fiu u, calculăm în fiecare nod diferența dintre informațiile din r și u.

Dintr-o sursă de pe Kilonova am învățat și o altă abordare elegantă.

1. Inițial contabilizăm toate flăcările cu semnul „+”.
2. Înainte de propagarea flăcărilor într-un fiu, facem un DFS ca sa contabilizăm flăcările din acel fiu cu semnul „-”.
3. După terminarea propagării, apelăm același DFS ca să contabilizăm flăcările din acel fiu, de data aceasta cu semnul „+”.

Din păcate, incluzînd și DFS-ul pentru găsirea centroidului, ajungem la 5 DFS-uri din fiecare centroid. Este o constantă măricică.

Idee de implementare: ștergerea nodurilor

Tocmai fiindcă codul include multiple DFS-uri, mi-am dat seama că „tîrăsc” după mine peste tot condiția `!nd[u].dead` prin tot codul. Astfel mi-a venit ideea: nu este mai scurt/rapid/clar să ștergem efectiv nodul?

Mai scurt nu este, căci la ștergerea lui u trebuie consultate toate listele de adiacență ale vecinilor lui u și șterse aparițiile lui u din acele liste. Deci codul se lungește cu 10-15 linii.

Mai rapid... discutabil. A doua sursă a mea este mai rapidă cu 14%, dar poate fi și un dram de noroc acolo. Eliminarea unui nod face totuși un efort proporțional cu suma gradelor vecinilor. Paradoxal, am optimizat codul ca să fac eliminarea în $\mathcal{O}(1)$ per muchie ștearsă și a devenit mai lent.

Dar aș argumenta că codul devine mai clar. Efectiv, arborele devine o pădure prin fărîmițare, iar subarborii nu au cunoștință unul de celălalt. Listele de adiacență nu mai sînt poluate de gunoaie.

Rămîne la alegerea voastră ce variantă folosiți.

22.5.7 Problema Digit Tree (Codeforces)

enunț • surse

Problema amintește puțin de Fixed Length Paths I, doar că nu trebuie să numărăm căile de o anumită lungime, ci pe cele care, citite ca număr zecimal, sînt divizibile cu M.

Din nou, vom arăta cum să rezolvăm problema pentru o rădăcină fixată, iar descompunerea în centroizi va face restul pentru noi.

Căi în sus și în jos

Cum procesăm o cale care trece prin rădăcină? Pare natural să o spargem în:

- calea care urcă pînă la rădăcină, care dă restul r_1 modulo M;
- calea care coboară din rădăcină, de lungime d muchii, care dă restul r_2 modulo M.

Întreaga cale va avea atunci restul $r_1 \cdot 10^d + r_2 \pmod M$. Dacă dorim ca acest rest să fie 0, atunci putem scrie

$$r_1 = (-r_2) \cdot 10^{-d}$$

Deci, pentru fiecare nod u, calculăm restul r_2 al căii de la rădăcină la u, apoi căutăm numărul de căi de la alte noduri v la rădăcină care dau restul r_1 dorit. Atenție, u și v trebuie să se găsească **în fii diferiți ai rădăcinii**. De asemenea, perechile (u, v) sînt ordonate. Cel mai corect pare să facem două DFS-uri separate, întîi pentru căile care urcă, apoi pentru cele care coboară.

Recapitulînd, pentru fiecare centroid:

1. Facem o parcurgere DFS. Calculăm resturile pe care le putem obține pe căi care urcă pînă în rădăcină și frecvențele acestor resturi.
2. Facem o a doua parcurgere DFS. Menținem lungimea căii curente și restul modulo M. În fiecare nod, calculăm restul-pereche necesar pentru a obține restul total 0. Aflăm frecvența acelui rest-pereche, ignorînd frecvența din fiul rădăcinii în care se află DFS-ul curent.

Mai trebuie tratate și două cazuri particulare, relativ simple:

1. Căi care doar urcă. După primul DFS, creștem răspunsul cu frecvența restului 0, indiferent din ce fiu.
2. Căi care doar coboară. În al doilea DFS, ori de cîte ori restul căii curente este 0, incrementăm răspunsul.

`map` și `unordered_map`

Concret, ce structură de date stocăm? Prima mea încercare a fost cu doar două tabele. Dat fiind un rest r pe o cale care urcă din u în rădăcină, într-un fiu v al rădăcinii, am incrementat două valori:

1. $t[r]$, unde r este un `map` de la întreg (restul) la întreg (frecvența restului). Așadar, t menține frecvențele resturilor indiferent din ce fiu provin.
2. $c[\langle r, v\rangle]$, unde c este un map de la (întreg,întreg) (restul și fiul rădăcinii pe care am pornit) la întreg (frecvența).

Atunci, în al doilea DFS, pornind pe un fiu al rădăcinii v și aflîndu-ne în nodul curent u, calculăm restul căii care coboară, r_2. Apoi calculăm restul corespunzător necesar r_1. În sfîrșit, aflăm numărul de apariții utile ale lui r_1 cu expresia

$$t[r_1] - c[\langle r_1, v\rangle]$$

Procedăm astfel deoarece calea nu poate să urce și să coboare tot prin v.

Am avut surpriza (deși nu mai este chiar surpriză) că `unordered_map` este mult mai lent decît `map`. O explicație poate fi că pentru fiecare centroid (așadar, de n ori) instanțiem două `unordered_maps`, ceea ce este lent. Cu `map` am obținut un timp mediu pe CF (1500-1600 ms). Complexitatea teoretică a crescut la $\mathcal{O}(n \log^2 n)$.

`map` cu eliminare și reinserare

Pentru un cod mai lent, dar considerabil mai scurt, folosim același artificiu ca la problema Flareon:

1. Menținem un singur map de frecvențe, t.
2. În acesta stocăm, ca mai sus, frecvențele resturilor pe căi care urcă.
3. La al doilea DFS, pe căile care coboară, iterăm prin fiii rădăcinii.
4. Înainte de a lansa DFS-ul dintr-un fiu, eliminăm căile care urcă prin acel fiu.
5. După revenirea din fiu, adăugăm la loc căile care urcă prin acel fiu.

Astfel evităm să combinăm căile care urcă și coboară prin acel fiu. Esența codului este:

```
void count_pairs_through(int u) {
  // ...

  for (edge e: nd[u].adj) {
    if (!nd[e.v].dead) {
```

```
      head_dfs(e.v, u, 1, e.digit, +1);
    }
  }

  for (edge e: nd[u].adj) {
    if (!nd[e.v].dead) {
      head_dfs(e.v, u, 1, e.digit, -1);
      tail_dfs(e.v, u, 1, e.digit);
      head_dfs(e.v, u, 1, e.digit, +1);
    }
  }

  // ...
}
```

Vectori + sortare + căutare binară

De amorul artei, am încercat și următoarea abordare. Să observăm că întîi facem toate inserările în structura noastră de date, apoi toate interogările de frecvență. De aceea, am înlocuit `map`-urile cu doi vectori:

1. În locul lui t, un vector de perechi <rest, frecvență>.
2. În locul lui c, un vector de tripleți <rest, fiu, frecvență>.

După primul DFS, am sortat tripleții și am comasat duplicatele (însumînd frecvențele). Pentru a căuta informații, am folosit căutarea binară.

Timpul de rulare s-a înjumătățit. De reținut!

22.5.8 Problema Cucuruz (Concurs RoAlgo # 7)

enunț • sursă

O primă dificultate este să ne decidem în ce mod să agregăm datele pentru a le calcula.

Fie $C(u, v)$ suma valorilor de pe calea $u - v$. Facem întîi observația că fiecare pereche de noduri (u, v) aflate la distanță cel mult p trebuie luată în calcul la răspuns. Într-adevăr,

- fie $u < v$ și atunci $C(u, v)$ contribuie la $Fss(v)$,
- fie $u > v$ și atunci $C(u, v)$ contribuie la $Fss(u)$.

S-ar putea să existe soluții bazate pe tehnica *small-to-large* care pornesc de la această observație. Dar iată soluția bazată pe descompunere în centroizi. Ea amintește (pînă la un punct) de problema Fixed-Length Paths I.

Procesarea din fiecare centroid

Cînd procesăm un centroid u dorim să contabilizăm contribuția tuturor căilor de lungime $l \leq p$ care trec prin acel centroid (dar nu și prin alt centroid de nivel mai înalt). Restul căilor sînt

complet cuprinse în subarborele unuia dintre fii și vor fi găsite la un moment viitor.

Ca să contabilizăm toate căile, vom considera că am procesat primii fii ai lui u și că avem informațiile despre ei într-o structură (vedem imediat ce informații și ce structură). Acum procesăm și fiul următor, să-i spunem v. Dorim să modelăm faptul că orice cale din fiii anteriori de lungime $d \leq p$ poate fi combinată cu orice căi din subarborele lui v de lungime cel mult $p - d$. Ca să fim eficienți, trebuie să încercăm să ținem totaluri pe nivel. Așadar, pentru fiecare adîncime d între 0 și p vom ține $left[d]$ = suma costurilor $C(u, x)$ pentru toate căile (u, x), din fiii procesați anterior, terminate în noduri x care se află la adîncimea d (raportat la nodul u). Similar vom ține $right[d]$ pentru căile de lungime d din fiul curent.

Să studiem regula de combinare. Dacă am avea explicit patru căi în stînga cu sumele a, b, c, d și trei căi în dreapta cu sumele e, f, g, atunci am obține prin combinare 12 căi cu sumele $a + e, a + f, a + g, b + e, \ldots, d + g$. Observăm că fiecare dintre sumele a, b, c, d trebuie adunată de 3 ori și fiecare dintre sumele e, f, g trebuie adunată de 4 ori. De aici rezultă că:

1. Nu este nevoie să ținem sume individuale, ci doar sumele în stînga $(a + b + c + d)$ și în dreapta $(e + f + g)$.
2. Avem nevoie să ținem minte nu doar **sumele** costurilor căilor pe fiecare nivel, ci și **numărul** de căi pe fiecare nivel, fie ele $cnt_left[d]$ și $cnt_right[d]$.

După ce calculăm contribuția la răspuns pentru aceste combinații între fiul v și fiii anteriori, vom încorpora informația din $right$ în $left$. În principiu, facem o simplă adunare pe fiecare nivel, cu completarea că la $right[d]$ trebuie să mai adăugăm și $cnt_right[d] \cdot V[u]$, deoarece la toate acele căi trebuie să adăugăm și nodul u cu valoarea sa.

Back to basics

Cum spuneam, căile de adîncime $d \leq p$ din fiii anteriori trebuie combinate cu căile din subarborele fiului curent de lungime cel mult $p-d$, adică de adîncimi $0, 1, \ldots, p-d$. Este tentant să ne gîndim la sume pe interval, iar de aici la AIB-uri.

Dar nu uitați de uneltele elementare! În primul rînd, vom parcurge complet fiul curent. Abia apoi vom face interogări de sumă pe prefix. Rezultă că ne sînt suficiente sumele parțiale peste vectorul $right$! Astfel eliminăm un factor de $\mathcal{O}(\log n)$ și obținem o soluție în $\mathcal{O}(n \log n)$.

22.5.9 Problema LaNaic (Lot 2024)

enunț • sursă

Soluție în $\mathcal{O}(n^2)$

Menționăm o soluție ineficientă, dar ușor de codat și care face o observație teoretică importantă. Lanțul optim trebuie să aibă un capăt undeva. Deci pornim cîte un DFS din fiecare nod ca să aflăm cît de departe putem ajunge pornind din acel nod și făcînd cel mult m modificări. Pentru nodul curent, reținem:

- Numărul de modificări necesare pentru a putea ajunge la el. Dacă depășim m modificări, oprim DFS-ul.
- Lungimea sufixului de muchii de aceeași culoare din stiva DFS curentă. Oricînd această lungime depășește k, incrementăm numărul de modificări necesare și resetăm lungimea.

Observăm că numărul de modificări necesare pentru a sparge un lanț monocrom de lungime l este întotdeauna $\lfloor \frac{l}{k+1} \rfloor$.

Soluție în $\mathcal{O}(n \log n)$

Să considerăm o descompunere în centroizi a arborelui. Pentru centroidul curent x, ne punem întrebarea: care este lanțul k-alternant maxim care trece prin x? După cum știm, dacă rezolvăm acest pas în $\mathcal{O}(S(x))$, unde $S(x)$ este mărimea subarborelui lui x, atunci complexitatea globală va fi $\mathcal{O}(n \log n)$.

Construim o pereche de vectori C_0 și C_1, unde $C_i(j)$ este **costul minim** necesar pentru a construi un lanț k-alternant care pornește din x pe o muchie de valoare i, are lungime (adîncime) j și se termină undeva în subarborele lui x. Indicii j iau valori între 0 și $H(x)$, unde $H(x)$ este înălțimea în muchii a subarborelui lui x.

Definim **costul** al unui astfel de lanț ca fiind o pereche: numărul minim de modificări necesare, iar, în caz de egalitate, lungimea minimă a prefixului de muchii de valoare i cu care începe lanțul. Raționamentul din spatele acestei definiții este că lanțul optim prin x va consta din două lanțuri (sau, în cazul degenerat, un singur lanț) care pornesc spre doi fii diferiți. Dacă cele două lanțuri pornesc pe prefixe de aceeași culoare, și dacă suma lungimilor acestor prefixe depășește k, este necesară o modificare suplimentară, altfel nu. Nu avem niciun motiv să preferăm un lanț care face mai multe modificări, dar care are prefix mai scurt. Putem amîna acele modificări pînă cînd și dacă devin necesare. Mai remarcăm și că vectorii C sînt monotoni: costul crește cu adîncimea.

Pentru fiecare fiu y putem calcula un astfel de vector, C_y. Există unul singur, deoarece muchia (x, y) are fie valoarea 0, fie 1. Este necesară o parcurgere DFS. O implementare rezonabilă este: lansăm o primă parcurgere pe muchii de aceeași valoare cu muchia (x, y), al cărei scop este să determine lungimea prefixului de aceeași valoare. Din ea comutăm la o a doua parcurgere DFS ori de cîte ori întîlnim prima muchie de valoare opusă.

Pe măsură ce fiii lui x își încheie parcurgerile, x are de făcut două operații:

1. Să caute lanțul maxim între fiul curent y și oricare din fiii anteriori.
2. Să adauge informația din C_y la C_0 sau C_1.

Deoarece vectorii sînt monotoni, putem face pasul (1) în $\mathcal{O}(H_{max} + H(y))$, unde H_{max} este adîncimea maximă a oricărui fiu anterior. Folosim tehnica *two pointers* pentru a căuta suma maximă, dar nu mai mult de M, a două elemente din doi vectori sortați. Pasul (2) este trivial, constînd doar din compararea celor doi vectori poziție cu poziție.

Pentru complexitate optimă, este important să procesăm fiii lui x în ordinea crescătoare a adîn-

cimii. Altfel H_{max} poate crește rapid, de exemplu dacă explorăm fiul cel mai adînc primul. După sortarea fiilor, efortul total pentru x va fi $\mathcal{O}(S(x))$. Sursa anexată sortează fiii naiv, cu `std::sort`. Am încercat și să fac sortarea în timp liniar, distribuind fiii după adîncimi, dar diferența de timp este minoră.

Partea VII

Matematică

Capitolul 23

Elemente de teoria numerelor

23.1 Exponențiere binară

Consider acest subiect ca fiind de bază. Dacă aveți nelămuriri, căutați resurse pe Internet, de exemplu pe CP Algorithms. Reiau aici doar codul:

```
const int MOD = 1'000'000'007;

int bin_exp(int b, int e) {
  int result = 1;

  while (e) {
    if (e & 1) {
      result = (long long)result * b % MOD;
    }
    b = (long long)b * b % MOD;
    e >>= 1;
  }

  return result;
}
```

23.1.1 Mica teoremă a lui Fermat

Pentru orice număr prim p și orice a cu $0 \leq a < p$,

$$a^{p-1} \equiv 1 \pmod{p}$$

De aici rezultă și că putem reduce exponentul modulo $p-1$:

$$a^e \equiv a^{e \bmod p-1} \pmod{p}$$

Vom folosi această teoremă în secțiunile următoare pentru a calcula inverse modulare.

23.1.2 Exponențiere în $\mathcal{O}(1)$ cu precalculare în $\mathcal{O}(\sqrt{n})$

Dacă exponențierea este operația critică (adică dacă avem foarte des nevoie de ea), o putem accelera la $\mathcal{O}(1)$ printr-o descompunere în radical. Am avut această nevoie la problema Ashima (Lot juniori 2023).

```
const int MOD = 1'000'000'007;
const int BITS = 15;
const int ROOT = 1 << BITS;

// pow2: 2^0, 2^1, 2^2, ..., 2^32767
// pow2_large: 2^0, 2^32768, 2^65536, 2^98304, ... 2^(32768*32767)
int pow2[ROOT];
int pow2_large[ROOT];

void precompute_powers_of_2() {
  pow2[0] = 1;
  for (int i = 1; i < ROOT; i++) {
    pow2[i] = pow2[i - 1] * 2 % MOD;
  }

  pow2_large[0] = 1;
  pow2_large[1] = (2 * pow2[ROOT - 1]) % MOD;
  for (int i = 2; i < ROOT; i++) {
    pow2_large[i] = (long long)pow2_large[i - 1] * pow2_large[1] % MOD;
  }
}

int power_of_2(long long e) {
  e %= (MOD - 1); // Mica teoremă a lui Fermat.

  long long small = pow2_large[e >> BITS];
  long long large = pow2[e & (ROOT - 1)];
  int result = small * large % MOD;

  return result;
}
```

Codul calculează în $\mathcal{O}(1)$ valori 2^e pentru $e < 2^{30}$. Metoda este să îl scriem pe $e = 2^{15} \cdot f + g$. Atunci:

$$2^e = 2^{2^{15} \cdot f + g} = (2^{2^{15}})^f \cdot 2^g$$

În prealabil, codul precalculează 2^i și $(2^{2^{15}})^i$ pentru $0 \leq i < 2^{15}$, ceea ce conduce la timp constant.

23.1.3 Aplicație: Suma unei progresii geometrice

Date fiind a, n și modulul M, să calculăm $S_n = 1 + a + a^2 + \cdots + a^{n-1} \mod M$.

Știm deja că putem rescrie:

$$S_n = \frac{a^n - 1}{a - 1}$$

și deci putem răspunde la întrebare printr-o exponențiere rapidă și un invers modular (vom vorbi în curînd despre inverse modulare). Dar dacă modulul nu este prim, atunci inversul modular poate fi nedefinit. De aceea, să căutăm o soluție care apelează doar la înmulțiri.

Implementare recursivă v1

Dacă n este par, putem scrie recurența:

$$\begin{aligned} S_n &= (1 + a + a^2 + \cdots + a^{\frac{n}{2}-1}) + (a^{\frac{n}{2}} + \cdots + a^{n-1}) \\ &= (1 + a^{\frac{n}{2}})(1 + a + a^2 + \cdots + a^{\frac{n}{2}-1}) \\ &= (1 + a^{\frac{n}{2}})S_{\frac{n}{2}} \end{aligned}$$

Dacă n este impar, putem reduce problema la una cu n par:

$$\begin{aligned} S_n &= 1 + a(1 + a + a^2 + \cdots + a^{n-2}) \\ &= 1 + a \cdot S_{n-1} \end{aligned}$$

De aici rezultă imediat codul:

```
int s(int a, int n) {
  if (n == 1) {
    return 1;
  }

  if (n & 1) {
    int rec = s(a, n - 1);
    return (1 + (long long) a * rec) % MOD;
  } else {
    long long mult = 1 + bin_exp(a, n / 2);
    return mult * s(a, n / 2) % MOD;
  }
}
```

Problema cu acest cod este că are complexitatea $\mathcal{O}(\log^2 n)$, căci face $\mathcal{O}(\log n)$ exponențieri rapide.

Implementare recursivă v2

Putem scăpa de exponențieri dacă îi cerem funcției reapelate să ne returneze și $a^{\frac{n}{2}}$. În acest caz, vă rog să evitați să returnați două valori din funcție (cum ar fi să pasați două argumente prin referință). Implementarea curată încapsulează cele două valori de returnat într-un `struct`.

```
struct prog {
  long long term; // stochează a^n pentru un n oarecare
  long long sum;  // stochează 1 + a + ... + a^{n-1} pentru același n
};

prog s(int a, int n) {
  if (n == 1) {
    return { a, 1 };
  }

  if (n & 1) {
    prog p = s(a, n - 1);
    return {
      p.term * a % MOD,
      (1 + a * p.sum) % MOD,
    };
  } else {
    prog p = s(a, n / 2);
    return {
      p.term * p.term % MOD,
      (1 + p.term) * p.sum % MOD,
    };
  }
}
```

Implementare iterativă

De amorul artei, să căutăm și o soluție iterativă, similară cu cea pentru exponențierea rapidă. În limbaj natural, exponențierea rapidă enumeră puterile $a, a^2, a^4, a^8 \ldots$ și le înmulțește pe cele care compun reprezentarea binară a lui n. De exemplu, $a^{42} = a^2 \cdot a^8 \cdot a^{32}$.

Să luăm un exemplu mare, ca să punem în evidență recurența: $n = 364 = 101101100_{(2)}$. Vă recomand și vouă această abordare: nu vă alegeți doar exemple cu $n = 3$. Durează mai mult să scrii un exemplu mai mare, dar soluția iese imediat în evidență.

$$
\begin{aligned}
S_{364} &= (1 + a^{182}) \cdot S_{182} \\
S_{182} &= (1 + a^{91}) \cdot S_{91} \\
S_{91} &= 1 + a[(1 + a^{45}) \cdot S_{45}] \\
S_{45} &= 1 + a \cdot [(1 + a^{22}) \cdot S_{22}] \\
S_{22} &= (1 + a^{11}) \cdot S_{11} \\
S_{11} &= 1 + a \cdot [(1 + a^{5}) \cdot S_{5}] \\
S_{5} &= 1 + a \cdot [(1 + a^{2}) \cdot S_{2}] \\
S_{2} &= (1 + a^{1}) \cdot S_{1} \\
S_{1} &= 1
\end{aligned}
$$

Dacă privim lista de indici calculați (1, 2, 5, 11, 22, 45, 91, 182, 364), observăm că acestea sînt exact prefixele lui 364 în baza 2. Deci trebuie să construim termenii lui S de la mic la mare, iterînd prin biții lui S de la stînga la dreapta (invers decît la exponențierea binară). Nu strică să specificăm un contract despre ce stochează iterația curentă, așa cum se vede în codul de mai jos.

```
int s(int a, int n) {
  int num_bits = 32 - __builtin_clz(n);
  long long sum = 0;
  long long ak = 1;

  for (int i = 0; i < num_bits; i++) {
    // Contract: fie k numărul format din primii i biți ai lui n.
    // Atunci la această linie ak = a^k și sum = 1 + a + ... + a^{k-1}.
    // Din vechiul ak (deci a^{k/2}) calculăm noul sum.
    sum = (1 + ak) * sum % MOD;
    ak = ak * ak % MOD;

    int bit = (n >> (num_bits - i - 1)) & 1;
    if (bit) {
      sum = (1 + a * sum) % MOD;
      ak = ak * a % MOD;
    }
  }

  return sum;
}
```

Pentru claritatea contractului, codul este puțin nenatural: valorile mici ale lui i corespund la biți din stînga. Îl putem scurta dacă inversăm direcția lui i:

```
int s(int a, int n) {
```

```
  int num_bits = 32 - __builtin_clz(n);
  long long sum = 0;
  long long ak = 1;

  for (int i = num_bits - 1; i >= 0; i--) {
    sum = (1 + ak) * sum % MOD;
    ak = ak * ak % MOD;

    if ((n >> i) & 1) {
      sum = (1 + a * sum) % MOD;
      ak = ak * a % MOD;
    }
  }

  return sum;
}
```

Anexa conține implementarea completă, cu testare.

23.2 Divizibilitate

Această secțiune este doar un ciot ca să reamintim următoarea complexitate. Suma armonică $n + \frac{n}{2} + \frac{n}{3} + ... + \frac{n}{n}$ este $\mathcal{O}(n \log n)$. Vom vedea această egalitate folosită astfel: pentru toate valorile k de la 1 la n, putem să iterăm prin toți multiplii de k, iar complexitatea va fi $\mathcal{O}(n \log n)$.

23.3 Principiul includerii și excluderii

Acest principiu nu are legătură directă cu teoria numerelor, dar survine adesea în probleme de divizibilitate. De aceea, pare un moment bun să îl abordăm. La nivelul acestui curs, principiul este cunoscut, dar îl reiau foarte pe scurt. Ocazional îl voi denumi prescurtat pinex (engl. *PIE* sau *inclusion-exclusion principle*).

Din teoria mulțimilor știm că

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Iar pentru trei mulțimi,

$$|A \cup B \cup C| = (|A| + |B| + |C|) - (|A \cap B| + |B \cap C| + |C \cap A|) + |A \cap B \cap C|$$

Pentru mai mult de trei mulțimi, desigur, regula generală este că luăm cu semnul + intersecțiile de număr impar de mulțimi și cu semnul − intersecțiile de număr impar de mulțimi.

Vom lua ca exemplu problema Prime Multiples (CSES), pe care o regăsiți și în secțiunea de

probleme. Subliniem că numerele de la intrare sînt prime și distincte. De asemenea, numărul de numere între 1 și n divizibile cu un x dat este de $\lfloor n/x \rfloor$. Atunci găsim soluția astfel:

1. Adunăm numerele divizibile cu oricare număr prim.
2. Scădem numerele divizibile cu orice pereche de numere prime (adică cu produsul perechii).
3. Adunăm numerele divizibile cu orice triplet de numere prime.
4. etc.

Vedem adesea două implementări pentru acest principiu.

23.3.1 Implementare iterativă

Implementarea iterativă numără binar de la 1 la $2^k - 1$. Fiecare dintre aceste valori corespunde unei combinații de numere prime. Valoarea este o mască în care biții 1, și doar aceia, indică numerele prime care participă în combinație. Pentru numerele prime $11, 7, 5, 3, 2$, masca $10110_{(2)}$ se referă la combinația $11 \times 5 \times 3$.

Această implementare are complexitatea $\mathcal{O}(2^k \cdot k)$, deoarece fiecare mască trebuie descompusă pentru a-i afla biții de 1.

23.3.2 Implementare recursivă

Implementarea recursivă construiește masca bit cu bit, cîte unul la fiecare nivel al recursivității. În fapt, nici nu mai transmitem ca parametru masca, ci direct produsul numerelor prime selectate pînă acum. La fiecare nivel putem să selectăm sau nu numărul curent, deci facem două apeluri recursive.

Avem nevoie și de numărul de numere selectate pînă acum. Dar de fapt, paritatea acestui număr ne ajunge. Pentru extra concizie, putem înmulți produsul curent cu -1 ori de cîte ori îl înmulțim cu un număr prim.

Această implementare are complexitate $\mathcal{O}(2^k)$, semnificativ mai mică față de cea recursivă (pe CSES, 0,02 s față de 0,13 s).

23.4 Indicatorul lui Euler

În lumea programării competitive, multe probleme de calcul modular se referă la moduli primi. Aceasta se întîmplă din motivul, destul de tangențial, că majoritatea problemelor cer răspunsul modulo $10^9 + 7$ sau modulo altă valoare primă și de aceea avem nevoie să calculăm puteri, inverse, combinări etc. modulo acea valoare.

Dar vom da, acolo unde putem, și generalizări pentru moduli neprimi. Vom lua fără demonstrație unele dintre afirmații (știu că asta vă întristează peste măsură).

Definiția 2 (Indicatorul lui Euler)**.** Indicatorul lui Euler (engl. Euler's totient function) al unui număr n este numărul de numere coprime cu n cuprinse între 1 și n. Îl notăm cu $\varphi(n)$.

O formulă de calcul este:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

unde p iterează prin divizorii primi ai lui n. În limbaj natural, pentru fiecare divizor prim p, eliminăm o fracție de $1/p$ din n, căci fiecare al p-lea număr se divide cu p și deci nu este coprim cu n. Exemple:

- Pentru orice număr prim, $\varphi(n) = n - 1$, căci toate numerele de la 1 la $n - 1$ sînt coprime cu n.
- Pentru orice produs de numere prime distincte, $\varphi(pq) = (p-1)(q-1)$. Vezi tabelul următor.

~~0~~	1	2	3	4	~~5~~	6
~~7~~	8	9	~~10~~	11	12	13
~~14~~	~~15~~	16	17	18	19	~~20~~
~~21~~	22	23	24	~~25~~	26	27
~~28~~	29	~~30~~	31	32	33	34

Tabela 23.1: O demonstrație că $\varphi(5{\cdot}7) = 4{\cdot}6 = 24$. Numerele care **nu** sînt coprime cu 35 sînt tăiate. Din cele 35 de numere, cele de pe prima coloană se divid cu 7. Pe fiecare dintre celelalte coloane există exact un număr divizibil cu 5 (din cauză că 5 și 7 sînt coprime). În general, dintr-un tabel de pq valori vom tăia $p + q - 1$ valori și vom reține $(p - 1)(q - 1)$.

- Pentru orice produs de puteri de numere prime,

$$\varphi(n = p^a \cdot q^b) = n \cdot \frac{p-1}{p} \frac{q-1}{q}$$

Generalizînd la oricîți factori primi, rezultă a doua formulă din pagina Wikipedia. Mai exact, dacă $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$, atunci

$$\varphi(n) = p_1^{k_1-1}(p_1 - 1)p_2^{k_2-1}(p_2 - 1) \dots p_r^{k_r-1}(p_r - 1)$$

23.4.1 Calculul lui φ

Calculul în $\mathcal{O}(\sqrt{n})$ este absolut elementar. El ia fracția $(p - 1)/p$ pentru prima apariție a unui factor prim p, apoi elimină din n toate aparițiile lui p.

```
int euler_totient(int n) {
  int phi = n;
  for (int p = 2; p * p <= n; p++) {
    if (n % p == 0) {
      phi = phi / p * (p - 1);
      while (n % p == 0) {
        n /= p;
```

```
      }
    }
  }
  if (n > 1) {
    phi = phi / n * (n - 1);
  }
  return phi;
}
```

23.4.2 Calculul lui φ de la 1 la n

Putem folosi o metodă similară cu Ciurul lui Eratostene pentru a calcula toate valorile lui φ de la 1 la n:

```
void compute_all_phi() {
  for (int i = 0; i <= MAX_N; i++) {
    phi[i] = i;
  }

  for (int i = 2; i <= MAX_N; i++) {
    if (phi[i] == i) {
      for (int j = i; j <= MAX_N; j += i) {
        phi[j] -= phi[j] / i;
      }
    }
  }
}
```

Inițial, considerăm că fiecare număr este coprim cu toate numerele (inclusiv cu el însuși). Cînd considerăm un număr `i` pentru care încă avem egalitatea `phi[i] == i`, atunci `i` este prim. Doar pentru numerele prime, codul iterează prin toți multiplii și îi înmulțește cu `(i - 1) / i` (în practică, codul face o scădere pentru a evita o înmulțire). Rezultatul final este cel dorit: fiecare număr este înmulțit cu $\frac{p-1}{p}$ pentru fiecare factor prim p al său.

Complexitatea este $O(n \log \log n)$, deoarece avem de-a face cu suma inverselor numerelor prime.

23.5 Teorema lui Euler

Teorema lui Euler este o generalizare a Micii teoreme a lui Fermat. Dacă a și n sînt întregi coprimi, atunci

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

Recunoaștem cazul particular al Micii teoreme a lui Fermat, cînd p era prim, iar $\varphi(p) = p - 1$. Rezultă că, pentru a calcula $a^b \mod n$, putem întîi să calculăm $\varphi(n)$ și să reducem exponentul b

modulo $\varphi(n)$.

23.6 Algorithmul lui Euclid extins

O cărămidă de bază în teoria numerelor este calcularea inverselor modulare. Majoritatea elevilor le calculează folosind exponențierea rapidă, și vom învăța și noi cum. Dar este bine să știți și algoritmul lui Euclid, deoarece acesta este mai rapid și funcționează indiferent dacă modulul este prim sau compus.

Algoritmul lui Euclid simplu primește două numere a și b și returnează $d = \text{cmmdc}(a, b)$. Algoritmul extins găsește, pe lîngă d, și două valori x și y pentru care

$$ax + by = d$$

Exemplu: $\text{cmmdc}(14, 35) = 7$. Algoritmul lui Euclid va găsi și valorile $x = -2$ și $y = 1$ cu care

$$14 \cdot (-2) + 35 \cdot 1 = 7$$

Pentru cei pasionați de matematică, Identitatea lui Bézout demonstrează că aceste valori există. Valorile nu sînt unice. Alte valori posibile sînt $14 \cdot 3 + 35 \cdot (-1) = 7$ sau $14 \cdot 8 + 35 \cdot (-3) = 7$.

23.6.1 Implementarea recursivă

Ca și algoritmul lui Euclid simplu, și cel extins reduce problema (a, b) la problema (b, r). Fie $a = qb + r$. Odată ce am găsit x' și y' care satisfac $bx' + ry' = d$, cum calculăm x și y? Nu este greu; doar îl înlocuim pe r cu $a - qb$:

$$\begin{aligned} d &= bx' + ry' \\ &= bx' + (a - qb)y' \\ &= a(y') + b(x' - qy') \end{aligned}$$

, de unde rezultă coeficienții lui a și b, respectiv y' și $x' - qy'$.

```
void extended_euclid_recursive(int a, int b, int& d, int& x, int& y) {
  if (!b) {
    d = a;
    x = 1;
    y = 0;
  } else {
    int xp, yp;
    extended_euclid_recursive(b, a % b, d, xp, yp);
```

```
    x = yp;
    y = xp - (a / b) * yp;
  }
}
```

23.6.2 Implementarea iterativă

Varianta iterativă este mai alunecoasă și nu este cu mult mai rapidă. O prezentăm doar ca experiment de gîndire. Fie A și B valorile inițiale. Implementarea distruge valorile a și b, înlocuindu-le după cum știm cu b și $a \mod b$. Concomitent, ea menține patru valori x, y, x', y' cu invariantul că, la orice moment:

$$\begin{cases} Ax + By &= a \\ Ax' + By' &= b \end{cases} \tag{23.1}$$

Inițial, $A = a$ și $B = b$, deci $x = 1$, $y = 0$, $x' = 0$ și $y' = 1$. Știm că la finalul algoritmului $a = d$ și $b = 0$, deci valorile pe care trebuie să le returneze algoritmul sînt valorile finale pentru x și y. Pentru menținerea invariantului, știm că dorim să înlocuim perechea (a, b) cu (b, r). Cu ce trebuie să înlocuim sistemul de ecuații? Să scriem:

$$\begin{aligned} a - qb &= r \\ (Ax + By) - q(Ax' + By') &= r \\ A(x - qx') + B(y - qy') &= r \end{aligned}$$

Așadar, noul sistem de ecuații este:

$$\begin{cases} Ax' + By' &= b \\ A(x - qx') + B(y - qy') &= r \end{cases} \tag{23.2}$$

Și rezultă înlocuirile $(x, x') \to (x', x - qx')$ și $(y, y') \to (y', y - qy')$.

```
void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}
```

Ca și algoritmul simplu, cel extins se oprește cînd $b = 0$, iar atunci c.m.m.d.c.-ul este a (ultimul rest obținut).

23.7 Inverse modulare

Dorim să calculăm inversul lui a modulo un număr M, adică să găsim un număr b astfel încît

$$a \cdot b \equiv 1 \pmod{M}$$

Reamintesc că inversul există dacă și numai dacă a și M sînt coprime. Altfel, dacă $\text{cmmdc}(a, M) = d > 1$, orice „invers" b am propune vom observa că $ab \pmod{M} > 1$, căci se divide cu d.

23.7.1 Prin exponențiere rapidă

Din teorema lui Fermat rezultă imediat că

$$a^{M-2} \equiv a^{-1} \pmod{M}$$

Așadar, putem calcula inversul a^{-1} prin exponențiere rapidă. Acest mecanism funcționează doar cînd modulul M este prim. Cînd modulul este compus este nevoie să calculăm indicatorul lui Euler și să folosim formula:

$$a^{\varphi(M)-1} \equiv a^{-1} \pmod{M}$$

Trebuie să facem acest lucru chiar și dacă a este coprim cu M. De exemplu, inversul lui $a = 5$ modulo $M = 14$ este 3, căci $5 \cdot 3 = 15 \equiv 1 \pmod{14}$. În schimb, $5^{14-2} = 5^{12} \equiv 1 \pmod{14}$, incorect. Este nevoie să calculăm $\varphi(14) = 6$ și $5^5 = 3125 \equiv 3 \pmod{14}$.

23.7.2 Prin algoritmul lui Euclid extins

Din algoritmul lui Euclid decurge o metodă imediată de calcul al inverselor. Inversul lui a modulo n este valoarea b pentru care

$$ab \equiv 1 \pmod{M} \implies ab + kM = 1$$

Apelînd algoritmul lui Euclid extins între a și M îl aflăm pe b (și, ca produs secundar, pe k). Observăm din nou că a are invers dacă și numai dacă este coprim cu M. Dacă $\text{cmmdc}(a, M) = d > 1$, atunci combinația liniară $ab + kM$ va fi întotdeauna multiplu de d, niciodată 1.

Atenție: algoritmul poate returna un invers negativ. Dacă este așa, adăugați-i M pentru a-l aduce în intervalul $[1, M)$.

23.7.3 Benchmarks

Am măsurat vitezele calculării de inverse cu cele trei metode. Diferențele sînt notabile, dar nu cruciale. Rețineți algoritmul lui Euclid extins pentru situații de TLE.

```
[cata@neil number-theory]$ ./inverse-benchmark
Fermat: 10518 ms pentru 100000000 inverse (9.5 M inverse/sec)
Euclid recursiv: 7549 ms pentru 100000000 inverse (13.2 M inverse/sec)
Euclid iterativ: 6831 ms pentru 100000000 inverse (14.6 M inverse/sec)
```

23.8 Funcția Möbius

23.8.1 Relația cu principiul includerii și excluderii

Să pornim de la problema Counting Coprime Pairs (CSES), pe care o regăsiți și în secțiunea de probleme. Soluția bazată pe pinex este următoarea. Examinăm succesiv numerele și, pentru fiecare număr x, ne propunem să aflăm cu cîte numere examinate anterior este el coprim. Suma acestor răspunsuri este răspunsul la problemă.

Pentru a afla răspunsul pentru x, întîi îl factorizăm pe x (construim ciurul ca să factorizăm rapid toate numerele). Apoi, de exemplu, dacă $x = 14$, el va fi coprim:

- cu toate numerele anterioare;
- minus cele anterioare divizibile cu 2;
- minus cele anterioare divizibile cu 7;
- plus cele anterioare divizibile cu 14.

Implementarea cu pinex este anexată. Remarc în treacăt artificiul cu care numărăm multiplii unui număr și îi incrementăm într-o singură trecere recursivă.

23.8.2 Definiția funcției Möbius

Haideți acum, în loc să contorizăm contribuția fiecărui număr, să contorizăm contribuția fiecărui divizor. Ar fi bine ca totalul general să fie același. 🙂 Să considerăm divizorul 14. Ori de cîte ori examinăm un multiplu de 14 (cum ar fi 14, 28 sau 70), în cursul pinex vom procesa și divizorul 14. El va contribui întotdeauna cu semnul +, căci are un număr par de factori primi distincți. La primul multiplu de 14 el va adăuga 0 (căci nu există anterior alți multipli de 14). La al doilea va adăuga 1, apoi 2, apoi 3 etc. Așadar, contribuția unui divizor d va fi o sumă Gauss luată cu un semn anume:

$$\mu(d) \times \frac{M(d)(M(d)-1)}{2}$$

Cu $M(d)$ am notat numărul de multipli de d printre cele n numere date, iar μ este funcția Möbius, care ne va spune cu ce semn trebuie să luăm contribuția. Pentru a-l calcula pe $M(d)$ pentru

toți divizorii d, ne reamintim de suma armonică. Așadar, ne permitem să calculăm frecvențele numerelor de la intrare, apoi să însumăm frecvențele tuturor multiplilor lui d, pentru fiecare d.

Acum putem să iterăm prin toți divizorii de la 1 la MAX_VAL și să însumăm contribuția lor. Dar trebuie să aflăm rapid pe care îi luăm cu $+$, pe care cu $-$ și pe care îi omitem, căci conțin factori repetați (cum ar fi $18 = 2 \cdot 3 \cdot 3$).

Am ajuns în sfîrșit la Funcția Möbius, notată cu $\mu(x)$. Ea definește exact cantitatea care ne interesează în această problemă, respectiv:

- $+1$ dacă x este liber de pătrate și are un număr par de factori primi.
- -1 dacă x este liber de pătrate și are un număr impar de factori primi.
- 0 dacă x nu este liber de pătrate (adică se divide cu un pătrat perfect).

23.8.3 Diagramă Euler-Venn

Simt nevoia să explic de ce ignorăm numerele libere de pătrate. Un prim argument este „așa procedează și pinex". Într-adevăr, cînd factorizăm numărul curent noi colectăm doar factorii primi distincți, indiferent de exponent. Pentru numărul $72 = 2^3 \cdot 3^2$, pinex va lua în calcul doar divizorii 1, 2, 3 și 6.

Iată și o altă demonstrație (*kill it twice* 🙂). Pentru un divizor ca $24 = 2^3 \cdot 3$, este suficient să facem o diagramă cu factorii primi 2 și 3. Ne interesează zona cu c.m.m.d.c = 1.

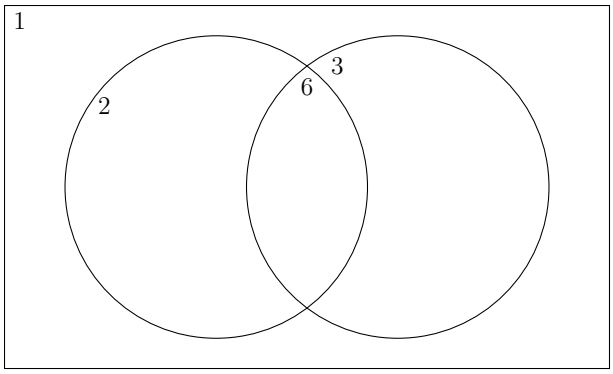


Figura 23.1: Mulțimile pentru coprimalitatea cu 24.

Dacă ținem morțiș, putem și să adăugăm mulțimi pentru numerele divizibile cu 4, 8, 12 și 24, dar muncim degeaba. Scopul nostru este să măsurăm cardinalul mulțimii „1" cu minimum de efort, iar submulțimile numerelor divizibile cu 2, 3 și 6 ne sînt suficiente.

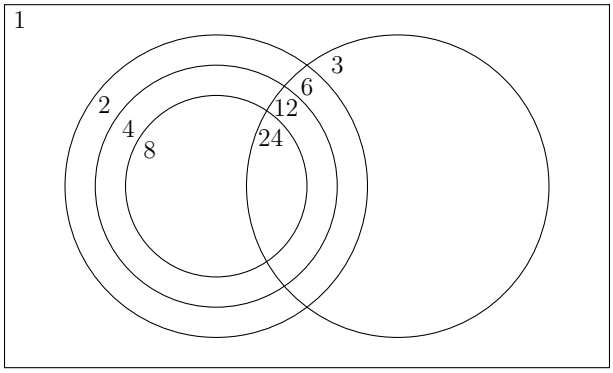


Figura 23.2: O colecție redundantă de mulțimi.

23.8.4 Implementare

Vom studia trei metode de calcul, bazate pe trei idei diferite, toate interesante.

metoda	**timp pentru** $n = 10\,000\,000$
$\mathcal{O}(n \log \log n)$, ciur	180 ms
$\mathcal{O}(n \log \log n)$, ciur + cel mai mic factor	250 ms
$\mathcal{O}(n \log n)$	640 ms (1 000 ms fără optimizare)

Tabela 23.2: Operații pe măști de biți.

Metoda 1

Metoda 1 calculează, în primă fază, în $u[i]$ produsul divizorilor primi ai lui i, mai mici sau egali cu $\sqrt{n}$, cu semnul + dacă există un număr par de divizori primi sau - în caz contrar. Concomitent, atribuie $u[i] = 0$ pentru toate valorile i care se divid cu un pătrat perfect. De exemplu $u[6] = 6$, $u[42] = -42$, $u[18] = 0$.

Ce valori nenule putem întîlni la final?

- Fie $u[i] = \pm i$, caz în care $\mu[i]$ este exact semnul lui $u[i]$,
- fie $u[i] \neq \pm i$, caz în care i mai are un factor prim, mai mare decît $\sqrt{n}$, deci $\mu[i]$ are semn opus lui $u[i]$.

```
// Învățat de la https://mathoverflow.net/a/99545
void compute_mobius(int n) {
  for (int i = 1; i <= n; i++) {
    u[i] = 1;
  }

  for (int i = 2; i * i <= n; i++) {
    if (u[i] == 1) {
      for (int j = i; j <= n; j += i) {
        u[j] *= -i;
      }
      for (int j = i * i; j <= n; j += i * i) {
```

```
        u[j] = 0;
      }
    }
  }

  for (int i = 2; i <= n; i++) {
    if (u[i] == i) {
      u[i] = 1;
    } else if (u[i] == -i) {
      u[i] = -1;
    } else if (u[i] < 0) {
      u[i] = 1;
    } else if (u[i] > 0) {
      u[i] = -1;
    }
  }
}
```

Complexitatea este $\mathcal{O}(n \log \log n)$. Ne reîntîlnim cu suma inverselor numerelor prime.

Metoda 2

Metoda 2 este cea din sursa anexată. Ea folosește un ciur cu cel mai mic factor și tratează școlărește cele trei cazuri de mai sus.

```
// Adaptat după https://www.geeksforgeeks.org/program-for-mobius-function-set-2/
void compute_smallest_factors(int n) {
  factor[1] = 1;
  for (int i = 2; i <= n; i++) {
    if (!factor[i])  {
      for (int j = i; j <= n; j += i) {
        if (!factor[j]) {
          factor[j] = i;
        }
      }
    }
  }
}

void compute_mobius(int n) {
  u[1] = 1;
  for (int i = 2; i <= n; i++) {
    if (factor[i] == factor[i / factor[i]]) {
      u[i] = 0;
    } else {
      u[i] = -u[i / factor[i]];
    }
  }
```

```
}
```

Metoda 3

Pe aceasta am învățat-o de la Mircea Rebengiuc! Ea este considerabil mai lentă pentru n foarte mare. Dar pentru valori practice ($n \leq 1\,000\,000$), diferențele sînt neglijabile.

```
void compute_mobius(int n) {
  u[1] = 1;
  for (int i = 1; i <= n; i++) {
    if (u[i]) { // optional speed optimization
      for (int j = 2 * i; j <= n; j += i) {
        u[j] -= u[i];
      }
    }
  }
}
```

Cum funcționează? Putem da și o demonstrație combinatorică. De exemplu, pentru $i = 30$, valorile care vor contribui la $u[30]$ vor fi $u[2]$, $u[3]$ și $u[5]$ cu semnul + și $u[1]$, $u[6]$, $u[10]$ și $u[15]$ cu semnul -. Rezultat: -1. Presupunînd că i are f factori primi, și considerînd toate submulțimile acestor factori, vom obține suma

$$u[i] = \sum_{k=0}^{f-1} C_f^k \cdot (-1)^k$$

Această sumă coincide cu dezvoltarea binomială a lui $(1-1)^f$, cu excepția ultimului termen. Ea va avea valoarea ± 1 în funcție de paritatea lui f.

O explicație mult mai elegantă pornește de la egalitatea $\sum_{d|n} \mu(d) = 0$ (pentru $n > 1$), pe care o vom studia în secțiunea despre formula de inversiune Möbius.

23.9 Formula de inversiune Möbius

Această secțiune este mai mult teoretică.

23.9.1 Funcția Möbius este multiplicativă

Să demonstrăm întîi că funcția Möbius este multiplicativă, adică $\mu(1) = 1$ și $\mu(mn) = \mu(m)\mu(n)$ pentru orice valori m și n **coprime** (acest aspect este important).

Voi urma această demonstrație. Dacă fie m fie n **nu** sînt libere de pătrate, atunci nici mn nu va fi liber de pătrate. Atunci fie $\mu(m)$, fie $\mu(n)$ vor fi 0, iar $\mu(mn)$ va fi tot 0. Deci se verifică egalitatea $\mu(mn) = \mu(m)\mu(n)$.

Altfel, m și n sînt libere de pătrate și coprime. Atunci m este un produs de k factori primi distincți, iar n este un produs de **alți** l factori primi distincți. Rezultă că mn este un produs de $k+l$ factori primi distincți (și liber de pătrate). Deci avem pe de o parte că $\mu(mn) = (-1)^{k+l}$, iar $\mu(m)\mu(n) = (-1)^k(-1)^l$, deci din nou se verifică egalitatea $\mu(mn) = \mu(m)\mu(n)$.

23.9.2 Suma funcției Möbius peste divizori este multiplicativă

Mai facem un pas către inversiunea Möbius și introducem funcția

$$f(n) \triangleq \sum_{d|n} \mu(d)$$

Să demonstrăm că și această funcție este multiplicativă. Clar, $f(1) = 1$ deoarece 1 are un singur divizor și $\mu(1) = 1$. Acum, fie două numere coprime m și n. Orice divizor d al lui mn poate fi scris ca produs dintre un divizor d_1 al lui m și unul d_2 al lui n. Mai mult, deoarece m și n sînt coprime, și divizorii d_1 și d_2 vor fi coprimi, așadar $\mu(d_1d_2) = \mu(d_1)\mu(d_2)$. Punînd totul cap la cap obținem:

$$\begin{aligned}
f(mn) &= \sum_{d|mn} \mu(d) \\
&= \sum_{d_1|m,\ d_2|n} \mu(d_1d_2) \\
&= \sum_{d_1|m,\ d_2|n} \mu(d_1)\mu(d_2) \\
&= \sum_{d_1|m} \sum_{d_2|n} \mu(d_1)\mu(d_2) \\
&= \Big(\sum_{d_1|m} \mu(d_1)\Big) \cdot \Big(\sum_{d_2|n} \mu(d_2)\Big) \\
&= f(m)f(n)
\end{aligned}$$

23.9.3 Valoarea sumei funcției Möbius peste divizori

Să demonstrăm egalitatea:

$$f(n) \triangleq \sum_{d|n} \mu(d) = \begin{cases} 1 & \text{dacă } n = 1 \\ 0 & \text{dacă } n > 1 \end{cases}$$

Desigur, dacă $n = 1$, singurul divizor este 1, iar $\mu(1) = 1$.

Demonstrație cu combinatorică

Dacă n nu este liber de pătrate, atunci el va avea și divizori neliberi de pătrate. Aceia vor avea $\mu(d) = 0$ și îi putem ignora la sumă. Așadar, în $f(n)$ putem ține cont doar de divizorii lui n liberi de pătrate, adică putem lua în calcul doar factorii primi ai lui n la puterea 1. De exemplu, pentru $n = 84 = 2^2 \cdot 3 \cdot 7$ putem calcula efectiv suma pentru $n' = 2 \cdot 3 \cdot 7 = 42$.

Dacă n este liber de pătrate și constă din r factori primi, atunci pentru fiecare $k = 0, 1, \ldots, r$ vor exista C_r^k moduri de a forma divizori cu k dintre factorii primi, iar toți acei divizori vor avea $\mu = (-1)^k$. De aceea,

$$f(n) = \sum_{k=0}^{r} C_r^k \cdot (-1)^k = (1 + (-1))^r = 0$$

Demonstrație pornind de la multiplicativitatea lui f

Dacă n are forma $n = p^k$ pentru un număr prim p și $k \geq 1$, atunci divizorii lui n sînt $1, p, p^2, \ldots, p^k$. Știm că $\mu(1) = 1$, $\mu(p) = -1$, iar pentru exponenți $e \geq 2$, $\mu(p^e) = 0$ pentru că acei divizori nu sînt liberi de pătrate. Deci $f(n) = 1 + (-1) + 0 + \cdots + 0 = 0$.

În cazul general, fie $n = p_1^{k_1} \cdot p_2^{k_2} \cdot \cdots \cdot p_r^{k_r}$. Atunci:

$$\begin{aligned} f(n) &= f(p_1^{k_1} \cdot p_2^{k_2} \cdot \cdots \cdot p_r^{k_r}) \\ &= f(p_1^{k_1}) \cdot f(p_2^{k_2}) \cdot \cdots \cdot f(p_r^{k_r}) \\ &= 0 \cdot 0 \cdot \cdots \cdot 0 \\ &= 0 \end{aligned}$$

Corolar: Calculul funcției Möbius

Acum înțelegem metoda 3 de calcul al funcției Möbius. Pornind de la funcția f, ea definește

$$\mu(n) = -\sum_{d|n,\ d \neq n} \mu(d)$$

23.9.4 Formula de inversiune a lui Möbius

Acum avem tot ce ne trebuie ca să demonstrăm Formula de inversiune a lui Möbius. O reluăm și aici. Dacă f și g sînt două funcții care satisfac relația

$$g(n) = \sum_{d|n} f(d) \text{ pentru } n \geq 1$$

atunci

$$f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) \text{ pentru } n \geq 1$$

Vom urma demonstrația aceasta și primul răspuns. Pornim de la formula lui f în care facem schimbarea de notație $d \to n/d$:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = \sum_{d|n} \mu\left(\frac{n}{d}\right) g(d)$$

Aici îl înlocuim pe g conform ipotezei date:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = \sum_{d|n} \left(\mu\left(\frac{n}{d}\right) \sum_{d'|d} f(d') \right)$$

Exprimăm această sumă de perechi sub altă formă:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = \sum_{d'|d|n} f(d') \mu\left(\frac{n}{d}\right)$$

Așadar, ne pregătim să-l trecem pe d' în exterior. Acum, pentru un d' fixat, ce valori poate lua n/d? Deoarece $d'|d$, rezultă că $(n/d)|(n/d')$. Facem notația $m = n/d$ și ajungem la:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = \sum_{d'|n} f(d') \sum_{m|\frac{n}{d'}} \mu(m)$$

În partea dreaptă recunoaștem suma funcției Möbius peste divizorii lui n/d'. Iau naștere două cazuri.

1. Cînd $d' = n$, suma din dreapta iterează prin divizorii lui 1. Știm că valoarea acelei sume este 1.
2. Cînd $d' < n$, atunci $n/d' > 1$, deci suma din dreapta este 0.

Rezultă, în sfîrșit, că:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = f(n) \cdot 1 + \sum_{d'|n,\ d<n} f(d') \cdot 0 = f(n)$$

23.9.5 Aplicații

Spre șocul meu, nu am găsit aplicații directe ale acestei formule. Invariabil am descoperit că toate problemele pe care le rezolvă inversiunea Möbius pot fi rezolvate și direct cu funcția Möbius sau chiar mai elementar. Iată totuși cîteva exemple.

Numărul de divizori

Dacă alegem $f(n) = 1$, atunci $g(n) = \sum_{d|n} f(d)$ este numărul de divizori ai lui n. Prin inversiunea Möbius obținem că:

$$\sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) = 1$$

Exemplu: Pentru $n = 10$, divizorii săi sînt 1, 2, 5 și 10, care au respectiv 1, 2, 2 și 4 divizori. Obținem că

$$\mu(1)g(10) + \mu(2)g(5) + \mu(5)g(2) + \mu(10)g(1) = 1 \cdot 4 - 1 \cdot 2 - 1 \cdot 2 + 1 \cdot 1 = 1$$

Putem folosi inversiunea Möbius ca să calculăm $g(k) =$ numărul de divizori ai lui k pentru fiecare valoare $1 \leq k \leq n$ dacă separăm unul dintre termenii sumei:

$$g(n) = \mu(1) \cdot g\left(\frac{n}{1}\right) = 1 - \sum_{d|n,\ d \neq 1} \mu(d) g\left(\frac{n}{d}\right)$$

Putem implementa această formulă: fiecare divizor contribuie la numărul de divizori ai fiecărui multiplu al său. Dar... noi am făcut deja fix acest lucru în problema Counting Divisors.

Suma divizorilor

Similar, dacă alegem $f(n) = n$, atunci $g(n) = \sum_{d|n} f(d)$ este suma divizorilor lui n, notată cu $\sigma(n)$, despre care am vorbit la problema Sum of Divisors. Prin inversiunea Möbius obținem că:

$$\sum_{d|n} \mu(d) \sigma\left(\frac{n}{d}\right) = n$$

Ca și în exemplul anterior, putem calcula $\sigma(k)$ pentru toate valorile $1 \leq k \leq n$, dar știm deja să facem acest lucru fără să folosim inversiunea Möbius.

Numărul de perechi coprime

Vom urma Exemplul 1 din acest articol de pe Codeforces. Dat fiind n, trebuie să numărăm perechile coprime $(x, y) \in [1, n] \times [1, n]$. Cerința seamănă cu problema Playing with GCD, cu trei diferențe:

1. Trebuie să calculăm perechile coprime în loc de necoprime.
2. Trebuie să includem și perechile neordonate, adică $x > y$.
3. Trebuie să aflăm răspunsul doar pentru un n dat, nu pentru toate $1 \leq k \leq n$.

Soluția la Playing with GCD era o sumă de valori φ. Inversiunea Möbius duce la o soluție diferită, care însă calculează răspunsul pentru un singur n.

23.10 Probleme

Lista include și probleme educaționale, clasice.

23.10.1 Problema Exponentiation (CSES)

enunț • sursă

Includ această problemă doar ca să trecem în revistă algoritmul de exponențiere rapidă.

23.10.2 Problema Exponentiation II (CSES)

enunț • sursă

Includ această problemă doar ca să trecem în revistă mica teoremă a lui Fermat.

Facem o remarcă importantă: modulul este prim. Dacă modulul nu ar fi prim, soluția ar putea fi incorectă. Exemplu: $2^{7^3} \equiv 8 \pmod{10}$, după cum ne putem convinge cu orice calculator sau observînd periodicitatea modulo 4 a ultimei cifre a lui 2^k. În schimb, dacă rezolvăm problema ca mai sus, obținem $7^3 \equiv 1 \pmod{9}$ și $2^1 \equiv 2 \pmod{10}$, incorect. Discrepanța izvorăște din faptul că $2^9 \neq 1 \pmod{10}$.

23.10.3 Problema Eugene and Big Number (CodeChef)

enunț • sursă

Problema este o aplicație directă a sumei unei progresii geometrice.

23.10.4 Problema Counting Divisors (CSES)

enunț • sursă

Putem rezolva problema și cu ciurul lui Eratostene cu păstrarea unui divizor pentru fiecare număr. Dar iată o soluție alternativă, care calculează toți divizorii tuturor numerelor din $[1, 10^6]$. Soluția are două `for`-uri, este elegantă și rapidă.

Regăsim aici observația că suma armonică $n + \frac{n}{2} + \frac{n}{3} + ... + \frac{n}{n}$ este $\mathcal{O}(n \log n)$.

23.10.5 Problema Common Divisors (CSES)

enunț • sursă

La prima vedere, iterarea prin toți divizorii posibili d și numărarea tuturor multiplilor posibili nu are cum să se încadreze în timp. Suprimați-vă acest instinct! 🙂 Regăsim și aici suma armonică și complexitatea totală $\mathcal{O}(n \log n)$.

Cînd valorile sînt mici, nu uitați de posibilitatea vectorilor de frecvență.

23.10.6 Problema Sum of Divisors (CSES)

enunț • sursă

Multe probleme de sumă se rezolvă agregînd răspunsurile în altă ordine. Haideți să iterăm după valoarea divizorului de adunat. Divizorul k intră în suma divizorilor lui $k, 2k, 3k, \ldots$, așadar trebuie adunat de $\lfloor n/k \rfloor$ ori. Contribuția lui k la răspunsul final este $\lfloor n/k \rfloor \cdot k$. De aici obținem o soluție în $\mathcal{O}(n)$, prea lentă:

$$S = \sum_{k=1}^{n} \lfloor n/k \rfloor \cdot k$$

Aici regăsim o aplicație a descompunerii în radical unde procedăm diferit înainte și după $\sqrt{n}$. Pînă la $k = \sqrt{n}$, calculăm suma ca mai sus cu efort $\mathcal{O}(\sqrt{n})$.

Între $\sqrt{n}$ și n, există doar $\sqrt{n}$ valori distincte pentru $\lfloor n/k \rfloor$. De exemplu, pentru $n = 100$ observăm că toți divizorii între 21 și 25 trebuie adunați de 4 ori și toți divizorii între 26 și 33 trebuie adunați de 3 ori. De aceea, este mai eficient să punem întrebarea: care divizori trebuie contorizați de t ori (unde $1 \leq t \leq \sqrt{n}$)? Pentru a afla răspunsul, fie așadar k unul dintre divizorii care trebuie contorizați de t ori. Pornim de la definiție:

$$\begin{aligned}
&\frac{n}{k} - 1 < \lfloor \frac{n}{k} \rfloor \leq \frac{n}{k} \implies \\
&\frac{n}{k} - 1 < t \leq \frac{n}{k} \implies \\
&n - k < tk \leq n \implies \\
&\frac{n}{t+1} < k \leq \frac{n}{t}
\end{aligned}$$

Așadar, contribuția la răspunsul final a divizorilor care trebuie contorizați de t ori este

$$t \cdot \left(\left[\left\lfloor \frac{n}{t+1} \right\rfloor + 1 \right] + \cdots + \left\lfloor \frac{n}{t} \right\rfloor \right)$$

23.10.7 Problema Prime Multiples (CSES)

enunț • surse

Aceasta este o problemă tipică rezolvabilă cu principiul includerii și excluderii. Vezi secțiunea teoretică pentru discuție.

23.10.8 Problema List of Integers (Codeforces)

enunț • surse

Rezolvarea implică să factorizăm multe numere de cel mult 1 000 000, deci am început cu ciurul lui

Eratostene în care pentru fiecare număr calculez cel mai mic divizor prim și cîtul după împărțirea prin acest divizor de cîte ori se poate. De exemplu, pentru $x = 225$ divizorul este 3, iar cîtul este $225/3^2 = 25$.

Nu cred că există o formulă directă pentru aflarea lui $L(x, p)$, dar putem folosi căutarea binară. Fie $C(n, p)$ numărul de numere între 1 și n coprime cu p. Aflarea lui $C(n, p)$ pentru un n dat este exact problema anterioară, Prime Multiples, dar negată. Deci știm să o rezolvăm cu pinex. Numărul maxim de factori primi distincți ai lui n este 7.

Apoi, $L(x, p)$ este cea mai mică valoare a lui n pentru care $C(n, p)$ atinge valoarea $C(x, p) + k$. Îl putem căuta binar pe n. (Întrebare: cum alegem o valoare inițială acoperitoare pentru extrema dreaptă?)

Am anexat două implementări, una care îl factorizează explicit pe p și face pinex pe mulțimea divizorilor și alta care consumă cel mai mic divizor al lui p, direct din p.

23.10.9 Problema Small GCD (Codeforces)

enunț • sursă

Dorim să calculăm suma funcției f aplicate la cei C_n^3 tripleți dintr-un vector de numere. Putem încerca diverse lucruri, dar ar fi bine ca, la un moment dat, să ne vină ideea (clasică) să agregăm datele altfel: să grupăm tripleții după valoarea f.

Așadar, fie $T(x)$ numărul de tripleți pentru care funcția f are valoarea x. Atunci răspunsul la problemă va fi:

$$\sum_{x=1}^{MAX_VAL} x \cdot T(x)$$

Fiind la lecția de pinex, să calculăm $T'(x)$ ca numărul de tripleți pentru care funcția f este **multiplu** de x (așadar $x, 2x, 3x, \dots$), apoi să calculăm $T(x)$ ca:

$$T(x) = T'(x) - T(2x) - T(3x) - \dots$$

Vom calcula aceste valori descrescător de la $x = MAX_VAL$ la 1, astfel încît valorile necesare pentru $T(x)$ să fie deja calculate. Vedem că efortul necesar pentru $T(x)$ este $\mathcal{O}(MAX_VAL/x)$, ceea ce înseamnă că putem obține complexitatea $\mathcal{O}(MAX_VAL \log MAX_VAL)$.

Cum procedăm pentru un x fixat? Pentru a-l afla pe $T'(x)$ vrem să numărăm tripletele (a, b, c) unde:

- a și b sînt multipli de x;
- $a \leq b \leq c$

Aici cred că merg diverse abordări: să iterăm după a, b sau c din enunț. Următoarea soluție

funcționează. Pentru un x fixat, iterăm prin toate valorile lui b multiplu de x. Pentru un b fixat, putem afla numărul de tripleți cu acel b și de valoare $f(a, b, c) = x$. Pentru aceasta, definim trei cantități, pe care le putem menține relativ ușor pe baza vectorului de frecvențe și care corespund candidaților posibili pentru a, b și c:

- l = numărul de multipli ai lui x strict mai mici decît b (acestea corespund oarecum a-urilor);
- f_b = frecvența lui b;
- r = numărul de elemente strict mai mari decît b (acestea corespund oarecum c-urilor).

Există o mică complicație: trebuie să generăm separat patru cazuri posibile:

- $a < b < c$: numărul acestor tripleți este $l \cdot f_b \cdot r$.
- $a = b < c$: numărul acestor tripleți este $C_{f_b}^2 \cdot r$.
- $a < b = c$: numărul acestor tripleți este $l \cdot C_{f_b}^2$.
- $a = b = c$: numărul acestor tripleți este $C_{f_b}^3$.

23.10.10 Problema Secvxor (Baraj ONI 2023)

enunț • surse

Problema este un pic artificială, ca toate problemele cu xor. Totuși, dacă încercăm exemplul cu hîrtie și creion, ne vom lămuri ce este acel xor al xor-urilor tuturor subsecvențelor bune. În definitiv, răspunsul va fi un xor al unor elemente din vector, luate o singură dată. Aceasta deoarece, tipic pentru xor, dacă luăm un element de două ori, cele două apariții se anulează.

Și atunci, ce elemente participă la xor-ul final? Acelea care apar **într-un număr impar de secvențe bune**. Deci putem reformula problema astfel: pentru fiecare element din vector, vrem să aflăm cîte secvențe bune îl includ.

Deoarece nu ne interesează numărul, ci doar paritatea lui, în implementare putem menține această contabilitate boolean, modulo 2. Optimizarea nu este doar teoretică, ci și practică! Pentru că nu ne mai pasă dacă adunăm sau scădem valori, nu mai este nevoie să transmitem variabila de semn (± 1) în funcția recursivă. Astfel am redus timpul de la 192 ms la 120 ms!

De aici, problema devine una de optimizare. Colegul vostru Horia are o rezolvare foarte interesantă, care nici nu stochează elementele vectorului, ci le procesează chiar la citire. Nu înțeleg toate detaliile, dar el răspunde la întrebarea: pentru elementul curent, care este contribuția la răspuns a tuturor secvențelor bune care se termină aici?

Abordarea mea este mai elementară. Să presupunem că știm numărul de secvențe bune care conțin poziția $k - 1$. Cum îl putem deduce pe cel pentru poziția k? Trebuie să operăm două modificări:

- Să scădem secvențele bune care s-au terminat la poziția $k - 1$.
- Să adăugăm secvențele bune care încep la poziția k.

Vom precalcula răspunsul pentru (2) într-un vector `right[k]` = numărul de secvențe bune care încep la poziția k. Cum facem asta? Trebuie să știm cîte elemente la dreapta poziției k au un

factor prim comun cu `v[k]`. De aceea, vom calcula vectorul `right` de la dreapta la stînga. Vom menține pentru fiecare valoare x informația `mult[x]` = numărul de elemente văzute pînă acum care sînt multipli de x. La fiecare pas, vom aplica pinex pe divizorii lui `v[k]`, apoi vom include acei divizori în vectorul `mult`.

Odată precalculată această informație, vom rula același algoritm de la stînga la dreapta și vom calcula un vector `left` (pe acesta nu îl mai stocăm explicit). Acum avem ce ne trebuie ca să menținem răspunsul la întrebarea originală: Cîte secvențe bune acoperă poziția k?

Optimizare

Sursa lui Horia m-a ambiționat și pe mine să caut o rezolvare care să apeleze doar n pinex-uri, nu $2n$. Am reușit astfel. Parcurgem vectorul doar de la stînga la dreapta și construim doar informația despre multipli la stînga lui k. Pentru a o putea accesa, în același timp, și pe cea din dreapta, facem o precalculare în timp $\mathcal{O}(MAX_VAL \log MAX_VAL)$ care calculează, global pentru tot vectorul, numărul de multipli ai fiecărei valori.

Atunci putem afla numărul de multipli de x la dreapta poziției curente k prin diferența între numărul global și numărul cunoscut la stînga.

23.10.11 Problema Afacere (Lot 2009)

enunț • surse

Problema miroase de la o poștă a problemă rezolvabilă cu principiul includerii și excluderii. Există două indicii:

1. $n \leq 20$
2. Ni se cere numărul de șiruri compatibile cu **cel puțin un** *wildcard*.

Așadar, vom genera submulțimi de *wildcards*. Pentru fiecare submulțime, contribuția ei la total este numărul de șiruri compatibile cu toate *wildcard*-urile din submulțime, iar semnul contribuției este + sau cu - în funcție de cardinalul submulțimii.

Algoritmul de bază

Cum calculăm contribuția unei submulțimi? Dacă pe cel puțin o coloană (poziție în șiruri) găsim două sau mai multe litere distincte, atunci desigur niciun șir nu se va potrivi cu toate simultan, deci contribuția este zero. Altfel, pe fiecare coloană numărul de posibilități va fi 1 dacă există vreo literă (sîntem obligați să luăm acea literă) sau Σ dacă există doar caractere `?` (putem lua orice literă). Prin Σ ne referim la valoarea 26 + extinderea curentă. Așadar, contribuția submulțimii va fi Σ^q, unde q este numărul de coloane care conțin doar `?`.

De aici rezultă un algoritm în $\mathcal{O}(nr \cdot 2^n \cdot l)$, prea lent. Pentru fiecare dintre cele nr extensii rulăm cîte un pinex recursiv. La fiecare apel recursiv avem nevoie de $\mathcal{O}(l)$ ca să combinăm *wildcard*-ul curent cu submulțimea construită pînă acum.

Eliminarea factorului nr

Putem să rulăm un singur algoritm pinex în locul celor nr, astfel. Am spus că toate contribuțiile vor fi de tipul Σ^q. În loc să le calculăm imediat, hai să le contabilizăm într-un vector de frecvență, `coef`, indexat după q. Dacă, de exemplu, dorim să adunăm la soluție Σ^5, vom executa `coef[5]++`.

În fapt, noi calculăm coeficienții unui polinom în variabila Σ, de gradul l.

La finalul pinexului, vom evalua acest polinom pentru fiecare Σ dintre cele nr.

Complexitatea acestei soluții este $\mathcal{O}(2^n \cdot l + l \cdot nr)$.

O implementare mai eficientă

Putem chiar și să eliminăm factorul $\mathcal{O}(l)$ de pe fiecare nivel din pinex, astfel. Pe parcursul recursivității, noi trebuie să răspundem la următoarele două întrebări despre *wildcard*-ul curent:

1. Este el compatibil cu submulțimea existentă?
2. Cîte semne de întrebare are reuniunea dintre *wildcard* și submulțimea existentă? (De aici aflăm care este contribuția ei la răspuns.)

Dacă o submulțime este incorectă, înseamnă că trebuie să existe două *wildcards* în această submulțime care conțin litere diferite pe o poziție. Putem precalcula naiv această informație în $\mathcal{O}(n^2 \cdot l)$, căci valorile sînt minuscule. Rezultă, pentru fiecare *wildcard*, o mască pe 20 de biți care reține cu ce alte *wildcards* este el compatibil.

O submulțime are `?` pe o poziție k dacă toate elementele sale au `?` pe acea poziție. De aceea, vom precalcula pentru fiecare *wildcard* o mască pe l biți cu 1 pentru `?` și 0 pentru litere (avem nevoie de `__int128`). Atunci masca unei submulțimi de *wildcards* este AND-ul pe biți al măștilor elementelor, iar numărul de `?` din masca submulțimii este populația acelei măști.

Complexitatea totală este $\mathcal{O}(n^2 \cdot l + 2^l + l \cdot nr)$.

23.10.12 Problema Playing with GCD (SPOJ)

enunț • sursă

Această problemă este „servită” pentru algoritmul de calcul simultan al funcției Euler pentru toate numerele de la 1 la n.

Să ne imaginăm o matrice A de $n \times n$ elemente în care doar jumătatea de sub diagonală este definită. Definim $A_{i,j}$ ca fiind 1 dacă cmmdc$(i, j) > 1$ și 0 altfel. Atunci prin definiție suma de pe rîndul i este numărul de numere care **nu** sînt coprime cu i, adică $i - \varphi(i)$. Răspunsul pentru un n dat este:

$$\sum_{i=1}^{n} (i - \varphi(i))$$

23.10.13 Problema Power Tower (Codeforces)

enunț • sursă

Vom folosi Teorema lui Euler. Facem deocamdată abstracție de condiția de coprimalitate. Să considerăm o expresie ca $a^{b^{c^d}} \mod m$. O putem descompune astfel:

- Dorim să calculăm $a^{b^{c^d}} \mod m$
- Putem reduce exponentul lui a modulo $\varphi(m)$.
- Deci trebuie să calculăm $b^{c^d} \mod \varphi(m)$.
- Putem reduce exponentul lui b modulo $\varphi(\varphi(m))$.
- Deci trebuie să calculăm $c^d \mod \varphi(\varphi(m))$.
- Putem reduce exponentul lui c modulo $\varphi(\varphi(\varphi(m)))$.
- Deci trebuie să calculăm $d \mod \varphi(\varphi(\varphi(m)))$.

La prima vedere, o abordare în $\mathcal{O}(n)$ este ineficientă, dar să observăm că funcția compusă $\varphi(\varphi(\ldots\varphi(m)))$ scade rapid. Dacă m este prim, iar $\varphi(m) = m - 1$, atunci $m - 1$ va fi par, și deci nu va fi coprim cu niciun alt număr par, deci $\varphi(m-1) < \frac{m-1}{2}$. Cu alte cuvinte, prin compunere repetată φ se înjumătățește la fiecare două iterații, deci va deveni 1 după $2 \log m$ iterații. Indiferent cît de lung este intervalul $[l, r]$, ne putem opri cînd modulul devine 1.

Partea frumoasă a problemei este că, pentru orice interogare $[l, r]$, ultimul rezultat trebuie calculat modulo m, penultimul modulo $\varphi(m)$ etc. Deci putem precalcula lista modulilor o singură dată, la început.

Partea grea a problemei este că posibil va trebui să calculăm $a^b \mod c$ în cazul în care a și c nu sînt coprime, deci aplicarea Teoremei lui Euler și reducerea lui b modulo $\varphi(c)$ nu funcționează. Am înțeles acest lucru după ce rezolvasem restul problemei; altfel nu aș fi selectat problema pentru curs. 😊 Am găsit o explicație rezonabilă pe Stack Overflow, care arată că putem lua exponentul modulo $\varphi(c)$, dar trebuie să adăugăm încă c. Ideea este că valorile $a^1, a^2, a^3, \ldots$ ciclează după o vreme, dar au nevoie de un număr de pînă la $\varphi(c)$ iterații ca să intre în ciclu. Așadar,

$$a^b \equiv a^{\varphi(c)+b \mod \varphi(c)} \pmod{c}$$

23.10.14 Problema GCD Extreme (SPOJ)

enunț • sursă

Să considerăm matricea naivă A unde $A_{i,j} = \text{cmmdc}(i, j)$, definită doar pentru triunghiul de deasupra diagonalei ($i < j$). Atunci răspunsul la problemă pentru un n dat va fi suma coloanelor $1 \ldots n$. Să calculăm suma pe fiecare coloană, fără a construi explicit matricea. La final vom face sume parțiale peste sumele coloanelor, pentru a putea oferi răspunsurile în timp constant.

Ca la multe alte probleme, în loc să iterăm pe linii sau coloane, să iterăm prin cmmdc-uri. Fie un cmmdc g. Pe ce coloane apare el și de cîte ori? Desigur, prin definiție g divide numărul liniei și pe al coloanei, deci va apărea pe coloanele de forma kg cu $k \geq 1$. De cîte ori apare g pe coloana

kg? În mod sigur, pe linii de forma lg, dar nu pe toate. Este necesar ca cmmdc$(k, l) = 1$, căci altfel cmmdc$(kg, lg) > g$ și numărul scris la coloana kg, linia lg ar fi diferit de g.

Cu alte cuvinte, dorim să numărăm cîte valori l între 1 și k sînt coprime cu k. Aceasta este exact definiția lui $\varphi(k)$. Punînd cap la cap observațiile, contribuția fiecărui divizor comun g pe fiecare coloană kg este $g \cdot \varphi(k)$. Datorită sumei armonice, complexitatea este $\mathcal{O}(n \log n)$.

23.10.15 Problema Counting Coprime Pairs (CSES)

enunț • surse

Am folosit problema în secțiunea de teorie. O includ și aici pentru completitudine.

23.10.16 Problema Coprime Sequences (Codeforces)

enunț • sursă

Și aici recunoaștem principiul includerii și excluderii. Să considerăm că valorile din vector ar fi doar 1, 2, 3 și 6. Notăm cu $M(d)$ numărul de multipli de d din șir. Atunci răspunsul este

$$(2^n - 1) - (2^{M(2)} - 1) - (2^{M(3)} - 1) + (2^{M(6)} - 1)$$

Într-adevăr, din toate cele $2^n - 1$ submulțimi le scădem pe cele care au divizorul comun 2. Similar pentru 3, dar am scăzut de două ori submulțimile care au divizorul comun 6.

Pe cazul general, codul pinex ar itera prin toți divizorii de la 1 la MAX_VAL. I-ar ignora pe cei liberi de pătrate, iar pentru restul ar lua contribuția cu semnul + sau − conform parității numărului de factori primi. De aceea putem regîndi problema, iterînd după divizor ca și la problema anterioară. Rezultă algoritmul:

1. Numărăm în $\mathcal{O}(n \log n)$ multiplii fiecărui divizor.
2. Calculăm funcția Möbius μ.
3. Contribuția fiecărui divizor d este $\mu(d) \cdot (2^{M(d)} - 1)$.

23.10.17 Problema Coprime Triples (CodeChef)

enunț • sursă

More of the same. 🙂 Problema aceasta are rating 2484 pe CodeChef, dar odată ce ne vine ideea să folosim funcția Möbius, soluția este imediată. Din toți tripleții posibili, îi scădem pe cei cu cmmdc 2, îi scădem pe cei cu cmmdc 7, dar îi adunăm la loc pe cei cu cmmdc 14 etc. Așadar, contribuția fiecărui divizor d este $\mu(d) \cdot C^3_{M(d)}$.

23.10.18 Problema Coprime Arrays (Codeforces)

enunț • sursă

Rezolvarea pentru un singur i

Problema pornește de la o observație elementară. Numărul de valori între 1 și i care se divid cu d este $\lfloor i/d \rfloor$. De aceea, fie $f(i,d)$ numărul de vectori de lungime n, cu valori între 1 și i și al căror cmmdc este multiplu de d. Atunci:

$$f(i,d) = \lfloor i/d \rfloor^n$$

Putem precalcula valorile $1^n, 2^n, \ldots, k^n$ ca să le avem la îndemînă.

Fie $C(i)$ răspunsul pentru un i fixat. Putem folosi principiul includerii și excluderii ca să îl aflăm pe $C(i)$:

$$C(i) = f(i,1) - f(i,2) - f(i,3) - f(i,5) + f(i,6) - f(i,7) + f(i,10) - f(i,11) - f(i,13) + f(i,14) + \ldots$$

Ne interesează valoarea lui f doar pentru numere i libere de pătrate. Este același raționament cu diagrame Euler-Venn, doar că acum mulțimea lui d nu se mai referă la multiplii de d, ci la vectorii care au un GCD multiplu de d. Rescriem ușor formula de mai sus folosind funcția Möbius:

$$C(i) = \sum_{d=1}^{k} \mu(d) f(i,d)$$

Rezultă un algoritm în $\mathcal{O}(k)$ pentru un i fixat. Global, este nevoie să calculăm funcția Möbius în $\mathcal{O}(k \log\log k)$ și puterile x^n în $\mathcal{O}(k \log n)$.

Rezolvarea completă

Ca să nu facem efort total $\mathcal{O}(k^2)$, să încercăm să calculăm doar diferențele la trecerea de la $i-1$ la i. Care dintre valorile $f(i,d)$ se vor schimba? Reamintim că $f(i,d) = \lfloor i/d \rfloor^n$, deci valoarea f va crește cu 1 atunci cînd $d|i$, iar pentru alte valori ale lui d va rămîne constantă.

O primă abordare este deci: pentru i-ul curent, iterăm prin toți divizorii săi. Pentru fiecare divizor d scădem din $C(i-1)$ valoarea $(i/d-1)^n$ și adăugăm valoarea $(i/d)^n$. La final, valoarea obținută este $C(i)$. S-ar putea să se încadreze în timp, dar nu am încercat.

O abordare mai eficientă pornește de la conceptul des întîlnit: să agregăm datele după divizorul d, nu după i. Să construim vectorul de diferențe $\Delta(i) = C(i) - C(i-1)$. Atunci răspunsurile $C(i)$ vor fi sume parțiale peste vectorul Δ și le putem afla la final în $\mathcal{O}(k)$.

Divizorul d va contribui la diferențele $\Delta(d), \Delta(2d), \Delta(3d)$.... Regăsim din nou suma armonică, iar efortul total necesar pentru a construi vectorul Δ este $\mathcal{O}(k \log k)$.

Pentru un mic bonus de viteză (10%), am redus numărul de operații modulo prin stocarea vectorului de diferențe pe `long long`.

23.10.19 Problema Mocha and Stars (Codeforces)

enunț • sursă

Problema are o parte de teoria numerelor și o parte de programare dinamică. Ca dificultate, o consider o problemă de nivel de baraj ONI.

Teoria numerelor

Putem să pictăm direct formula cu funcția Möbius sau, pentru claritate, putem să pornim de la Pinex (principiul includerii și excluderii), ca în multe alte probleme. Inițial presupunem că toate șirurile au cmmdc-ul 1. Doar că astfel le-am numărat și pe cele care au cmmdc-ul 2 sau un multiplu, pe care le scădem. Similar și pentru 3. Dar astfel am scăzut de două ori șirurile care au cmmdc-ul 6, pe care le adunăm la loc etc. De aceea, fie $P(d)$ numărul de șiruri al căror cmmdc este d sau un multiplu de d. Atunci trebuie să calculăm formula, de-acum familiară:

$$P(1) - P(2) - P(3) - P(5) + P(6) - \dots$$

Pe aceasta o putem rescrie folosind funcția Möbius, μ:

$$\sum_{d=1}^{m} \mu(d) \cdot P(d)$$

Programare dinamică

Acum, pentru un d fixat, să calculăm $P(d)$. Așadar, să numărăm șirurile $a_1, \dots, a_n$ care întrunesc condițiile:

1. Toate valorile sînt multipli de d.
2. $a_i \in [l_i, r_i]$ pentru $1 \leq i \leq n$.
3. Suma valorilor este cel mult m.

Putem să împărțim toate cantitățile prin d. Notăm $l'_i = \lceil l_i/d \rceil$, $r'_i = \lfloor r_i/d \rfloor$ și $m' = \lfloor m/d \rfloor$ și reducem condițiile la două:

1. $a_i \in [l'_i, r'_i]$ pentru $1 \leq i \leq n$
2. Suma valorilor este cel mult m'.

Această subproblemă este o numărare de partiții cu o soluție destul de directă în $\mathcal{O}(n \cdot m'^2)$. Fie $C(k, s)$ numărul de moduri de a fixa primele k valori astfel încît să obținem exact suma s. Atunci fiecare valoare posibilă pentru a_{k+1}, adică $x \in [l'_{k+1}, r'_{k+1}]$, adaugă valoarea $C(k, s)$ la $C(k+1, s+x)$. Cu alte cuvinte, adăugăm valoarea celulei $C(k, s)$ la tot intervalul $C(k+1, s+l'_{k+1} \dots s+r'_{k+1})$.

Putem să reducem complexitatea programării dinamice la $\mathcal{O}(m'n)$ dacă facem adăugările pe interval în $\mathcal{O}(1)$. Cum? Cu un vector de diferențe (aka șmenul lui Mars). Cînd propagăm celula $C(k, s)$, o adunăm cu $+$ la $C(k+1, s+l_{k+1})$ și cu $-$ la $C(k+1, s+r'_{k+1}+1)$. Facem aceasta

pentru toate sumele s de pe linia k. Apoi calculăm sume parțiale pe linia $C(k+1)$ ca să aflăm valorile reale.

Să nu uităm însă că $m' = \frac{m}{d}$ și că noi trebuie să rulăm programarea dinamică pentru $d = 1, 2, \ldots, m$. Așadar, complexitatea totală este

$$\mathcal{O}(\frac{m}{1} \cdot n + \frac{m}{2} \cdot n + \frac{m}{3} \cdot n + \ldots) = \mathcal{O}(mn \log m)$$

O notă despre optimizare: `if`-ul de mai jos a redus timpul de rulare cu 20%. Am uitat complet acest aspect! De asemenea, mă mir că compilatorul execută `count_partitions(d)` chiar și atunci cînd știe că produsul va fi 0.

```
for (int d = 1; d <= m; d++) {
  if (mu[d]) {
    answer += (long long)mu[d] * count_partitions(d);
  }
}
```

23.10.20 Problema Pastile (Baraj ONI 2021)

enunț • sursă

Problema este complexă, dar o piesă de bază este funcția Möbius. Din enunț rezultă destul de direct că există $\prod_{i=1}^{n} p_i$ vectori, dintre care trebuie să îi numărăm pe cei cu cmmdc $= 1$. Mulțumită funcției Möbius, știm că trebuie să numărăm, pentru fiecare k, vectorii care au cmmdc-ul k sau multiplu de k. Editorialul notează această cantitate cu cnt_k. Atunci răspunsul va fi

$$\sum_{k=1}^{V_{min}} \mu(k) \cdot cnt_k$$

Să vedem acum cum aflăm valorile cnt_k. Formula naivă este:

$$cnt_k = \prod_{i=1}^{n} \left\lfloor \frac{p_i}{k} \right\rfloor$$

De exemplu, pentru $k = 7$ și vectorul $(10, 15, 17, 20, 30, 32)$ obținem:

$$cnt_7 = 1 \cdot 2 \cdot 2 \cdot 2 \cdot 4 \cdot 4 = 1 \cdot 2^3 \cdot 4^2$$

Desigur, nu dorim să facem efort $\mathcal{O}(n)$, dar ne reîntîlnim (din nou) cu suma armonică. Ne permitem să facem efort $\mathcal{O}(V_{max}/k)$ pentru divizorul k pentru a obține complexitatea $\mathcal{O}(V_{max} \log V_{max})$. Și atunci putem calcula expresia de mai sus iterînd prin valori, nu prin vector. Ne întrebăm: cîte

valori dau cîtul 2 la împărțirea prin 7? Răspuns: trei valori, anume 15, 17 și 20. Și cîte valori dau cîtul 4 la împărțirea prin 7? Răspuns: două valori, anume 30 și 32.

Așadar, ultima cărămidă necesară este: dat fiind vectorul original, valoarea lui k și un cît dorit, m, cîte valori din vector dau cîtul m la împărțirea prin k? Răspunsul, desigur, este numărul de elemente cu valori între mk și $(m+1)k-1$. Putem afla răspunsurile în $\mathcal{O}(1)$ dacă precalculăm sume parțiale peste vectorul de frecvență.

23.10.21 Problema Beautiful Numbers (Codeforces)

enunț • sursă

Problema este elementară. O folosim ca să ilustrăm inversele modulare.

Toate numerele frumoase de lungime n pot fi grupate în $n+1$ grupe:

- cele care conțin n cifre a și 0 cifre b;
- cele care conțin $n-1$ cifre a și o cifră b;
- ...
- cele care conțin 0 cifre a și n cifre b.

Toate numerele din grupa cu x cifre a și $n-x$ cifre b au suma cifrelor $xa+(n-x)b$ și vor fi excelente dacă și numai dacă $xa+(n-x)b$ este un număr frumos. Știm și că grupa conține $C(n,x)$ numere. Așadar, trebuie pur și simplu să însumăm combinările $C(n,x)$ pentru acele valori x pentru care suma cifrelor este frumoasă.

Întrucît n este mare, vom folosi formula

$$C(n,k) = n! \cdot [k! \cdot (n-k)!]^{-1}$$

Există diverse moduri de a face acest lucru. Sursa mea calculează inversele din formula de mai sus. Dar putem și să precalculăm toate factorialele inverse.

Capitolul 24

Teorema chinezească a resturilor

Teorema chinezească a resturilor (engl. *The Chinese remainder theorem* sau *CRT*) nu este strict necesară pentru olimpiade, dar ne ajută mult să ne familiarizăm cu algebra modulară

Teorema 1 (Teorema chinezească a resturilor)**.** Fie n numere întregi $m_1, m_2, \ldots, m_n$, astfel încît oricare două dintre ele sînt coprime. În plus, fie n numere întregi $a_1, a_2, \ldots, a_n$ astfel încît $0 \leq a_i < m_i, \forall i$. Atunci există un unic x cu $0 \leq x < \prod_i m_i$ astfel încît $x \equiv a_i \pmod{m_i}, \forall i$.

Dacă sînteți familiari cu teoria numerelor, puteți citi direct:

- pagina Wikipedia;
- o pagină care tratează și cazul în care modulii nu sînt coprimi;
- o pagină cu rezultate mai avansate (dincolo de scopul acestui capitol);
- pagina CP Algorithms pentru cod.

Eu voi încerca să demonstrez lucrurile mai băbește și să arăt de unde scoatem unele formule (*hint*: nu din inspirația divină).

24.1 Un exemplu tabelar

Iată tabelul complet pentru $m_1 = 4, m_2 = 7$.

	$a_2 = 0$	$a_2 = 1$	$a_2 = 2$	$a_2 = 3$	$a_2 = 4$	$a_2 = 5$	$a_2 = 6$
$a_1 = 0$	0	8	16	24	4	12	20
$a_1 = 1$	21	1	9	17	25	5	13
$a_1 = 2$	14	22	2	10	18	26	6
$a_1 = 3$	7	15	23	3	11	19	27

Tabela 24.1: Toate valorile posibile pentru a_1 și a_2 cînd $m_1 = 4$ și $m_2 = 7$.

Pornind de la acest tabel putem demonstra existența și unicitatea unei valori între 0 și 27 pentru fiecare pereche (a_1, a_2). Într-adevăr, toate valorile între 0 și 27 își au locul undeva în tabel. Vom

demonstra întîi că x-ul pentru fiecare celulă este unic. Să presupunem că două valori x_1 și x_2 ar sta în aceeași celulă. Atunci am avea că

$$\begin{cases} 4 \mid (x_1 - x_2) \\ 7 \mid (x_1 - x_2) \end{cases}$$

De aici deducem că

$$28 \mid (x_1 - x_2)$$

Singura explicație este că $x_1 = x_2$. Din unicitatea lui x rezultă și existența lui x. Avem de distribuit 28 de numere în 28 de celule de capacitate 1, deci rezultă că toate celulele sînt ocupate. Acest argument se generalizează la oricîți moduli coprimi.

Acum să vedem cum îl putem afla algoritmic pe x.

24.2 Un exemplu cu doi moduli

Să rezolvăm sistemul de ecuații

$$\begin{cases} x & \equiv 3 \pmod 7 \\ x & \equiv 4 \pmod{11} \end{cases}$$

Cu puțin chin putem găsi răspunsul 59; de exemplu, putem enumera valorile $11k + 4$, respectiv 4, 15, 26, 37, 48, 59. Ne oprim pentru că $59 \equiv 3 \pmod 7$.

Iată și o soluție constructivă. Dorim să scriem $x = 3P + 4Q$ și să găsim niște valori P și Q astfel încît

- Modulo 7, Q să dispară, iar P să fie 1 (astfel rămînem cu 3).
- Modulo 11, P să dispară, iar Q să fie 1 (astfel rămînem cu 4).

Dacă vreți, am redus problema originală la două probleme similare, dar speciale: căutăm $P \equiv 1 \pmod 7$ și $P \equiv 0 \pmod{11}$, respectiv $Q \equiv 1 \pmod{11}$ și $Q \equiv 0 \pmod 7$.

Calculăm inversul fiecărui modul modulo celălalt (putem face asta, pentru că modulii sînt coprimi).

- $8 \cdot 7 = 56 \equiv 1 \pmod{11}$, deci inversul lui 7 modulo 11 este 8.
- $2 \cdot 11 = 22 \equiv 1 \pmod 7$, deci inversul lui 11 modulo 7 este 2.

Acum scriem $P = 2 \cdot 11$, tocmai pentru că el dispare modulo 11 și este 1 modulo 7. Similar, scriem $Q = 8 \cdot 7$, pentru că el dispare modulo 7 și este 1 modulo 11. Rezultă

$$x = 3 \cdot 2 \cdot 11 + 4 \cdot 8 \cdot 7 = 290$$

Este destul de clar că putem folosi orice valoare congruentă cu x modulo 77, în particular 290 mod 77 = 59. Pentru cazul general, vezi Wikipedia.

24.3 Un exemplu cu 3 moduli

Să rezolvăm sistemul de ecuații

$$\begin{cases} x & \equiv 2 \pmod 5 \\ x & \equiv 6 \pmod 7 \\ x & \equiv 4 \pmod{11} \end{cases}$$

Ca mai sus, căutăm $x = 2P + 6Q + 4R$ unde:

- Modulo 5, Q și R să dispară, iar P să fie 1;
- Modulo 7, P și R să dispară, iar Q să fie 1;
- Modulo 11, P și Q să dispară, iar R să fie 1.

Rezultă că P este multiplu de 7 și de 11 (deci de 77). Inversul lui 77 modulo 5 este 3. Deci fixăm $P = 7 \cdot 11 \cdot 3 = 231$. De amorul artei, am putea alege și valoarea $2 \cdot 77 = 154$, iar inversul acesteia ar fi 4. Deci am putea fixa și $P = 2 \cdot 7 \cdot 11 \cdot 4 = 616$. Dar nu am avea nimic de cîștigat.

Similar, inversul lui $5 \cdot 11$ modulo 7 este 6, deci $Q = 5 \cdot 11 \cdot 6$. În sfîrșit, inversul lui $5 \cdot 7$ modulo 11 este 6, deci $R = 5 \cdot 7 \cdot 6$. Punînd totul cap la cap, obținem:

$$x = 2 \cdot (7 \cdot 11 \cdot 3) + 6 \cdot (5 \cdot 11 \cdot 6) + 4 \cdot (5 \cdot 7 \cdot 6) = 3282$$

Ca mai sus, putem opera modulo $5 \cdot 7 \cdot 11 = 385$, așadar putem alege $x = 202$.

24.4 Cazul general cu n moduli

Putem generaliza formula de mai sus observînd că x este o sumă de termeni, cîte unul pentru fiecare rest și modul. Termenul pentru a_i și m_i este un produs dintre a_i, produsul tuturor modulilor în afară de m_i și inversul acestui produs față de m_i. În limbaj matematic,

$$M \stackrel{\text{def}}{=} \prod_{i=1}^{n} m_i$$

$$M_i \stackrel{\text{def}}{=} \frac{M}{M_i}$$

$$b_i = M_i^{-1} \mod m_i$$

$$x = \sum_{i=1}^{n} a_i \cdot M_i \cdot b_i \mod M$$

Codul necesar este elementar:

```
long long crt() {
  long long M = 1;
  for (int i = 0; i < n; i++) {
    M *= m[i];
  }

  long long x = 0;
  for (int i = 0; i < n; i++) {
    long long others = M / m[i];
    long long b = inverse(others, m[i]);
    x = (x + r[i] * others % M * b) % M;
  }

  return x;
}
```

24.5 Cazul cu moduli necoprimi

Metoda de mai sus nu funcționează dacă modulii sînt necoprimi, deoarece poate eșua cînd încearcă să inverseze modulii unii față de alții (exemplu: 6 nu are invers față de 8). Studiem altă metodă mai generală, care desigur poate fi aplicată și în cazul anterior. Rețineți-o pe oricare vă place!

24.5.1 Condiția de existență

Mai întîi, condiția de existență a soluției. Să rezolvăm sistemul de ecuații

$$\begin{cases} x & \equiv 10 \pmod{28} \\ x & \equiv 11 \pmod{35} \end{cases}$$

Putem să aplicăm CRT în sens invers ca să desfacem modulii în valori coprime. Din fiecare ecuație de mai sus deducem două:

$$\begin{cases} x & \equiv 2 \pmod 4 \\ x & \equiv 3 \pmod 7 \\ x & \equiv 1 \pmod 5 \\ x & \equiv 4 \pmod 7 \end{cases}$$

Observăm imediat că sistemul nu are soluții datorită ecuațiilor (2) și (4). Descompunerea ne arată și cum putem repara contradicția. Pentru fiecare doi moduli necoprimi m_1 și m_2, condiția necesară pentru ca sistemul să aibă ecuații este

$$\text{cmmdc}(m_1, m_2) \mid (a_1 - a_2)$$

De exemplu, sistemul de mai jos are soluții pentru că $7 \mid (25 - 11)$:

$$\begin{cases} x & \equiv 25 \pmod{28} \\ x & \equiv 11 \pmod{35} \end{cases}$$

Dacă există, soluția este unică modulo $\text{cmmmc}(m_1, m_2, \ldots, m_n)$ (în cazul de mai sus, 140).

24.5.2 Metoda de construcție

Metoda funcționează pentru două ecuații. Dacă avem trei sau mai multe, le procesăm pe primele două și obținem o nouă congruență modulo $\text{cmmmc}(m_1, m_2)$. La aceasta adăugăm a treia ecuație ș.a.m.d. Așadar, fie sistemul:

$$\begin{cases} x & \equiv a \pmod m \\ x & \equiv b \pmod n \end{cases}$$

Să scriem naiv ce înseamnă prima congruență. Există un k astfel încît

$$x = km + a$$

Inserăm în a doua congruență:

$$\begin{aligned} km + a \equiv b \pmod n &\implies \\ km \equiv (b - a) \pmod n &\implies \\ \text{Există un } l \text{ astfel încît } \quad km &= (b - a) + ln \end{aligned}$$

Ne-ar plăcea să spunem pur și simplu $k = (b - a) \cdot [m^{-1} \pmod n]$, dar inversul poate fi nedefinit. În schimb, fie $g = \text{cmmdc}(m, n)$. Să observăm în ultima ecuație că toți termenii se divid cu g,

după cum rezultă din condiția de existență. Atunci scriem:

$$\begin{aligned} k\frac{m}{g} &= \frac{b-a}{g} + l\frac{n}{g} \\ k\frac{m}{g} &\equiv \frac{b-a}{g} \pmod{\frac{n}{g}} \\ k &\equiv \frac{b-a}{g}\left(\frac{m}{g}\right)^{-1} \pmod{\frac{n}{g}} \end{aligned}$$

De data aceasta inversul este definit, căci m/g și n/g sînt coprime și pozitive.

24.6 Probleme

24.6.1 Problema Koxia and Number Theory (Codeforces)

enunț • sursă

O primă observație: dacă există două valori egale, $a_i = a_j$, atunci cmmdc$(a_i+x, a_j+x) = a_i+x \geq 2$, deci problema nu are soluție. Deci o primă condiție este ca toate valorile să fie distincte. Nu știu cum să includ acest caz în restul codului, așa că l-am testat explicit.

Ca atîtea alte probleme, și aceasta devine ușoară dacă procesăm datele în altă ordine decît cea pe care o implică enunțul. În loc să tratăm perechile una cîte una, să tratăm divizorii unul cîte unul. Ce înseamnă că cmmdc-ul oricărei perechi este 1? Înseamnă că orice număr prim divide cel mult una dintre valorile $a_i + x$. Să alegem un număr prim la întîmplare: 2. 🙂 Ce ne trebuie ca să existe un x astfel încît 2 să dividă cel mult una dintre valorile $a_i + x$? Trebuie ca

1. fie cel mult o valoare a_i să fie pară (caz în care vom alege un x par);
2. fie cel mult o valoare a_i să fie impară (caz în care vom alege un x impar).

Dacă există două valori pare și două valori impare în vectorul a, atunci problema nu are soluție. Similar și pentru 3 și pentru alți divizori. Să considerăm un divizor prim p. Valorile a_i se vor încadra în p clase de resturi modulo p (similar și valorile $a_i + x$). Deci trebuie ca

1. fie cel mult o valoare a_i să fie congruentă cu 0 modulo p (caz în care vom alege un x de forma kp);
2. fie cel mult o valoare a_i să fie congruentă cu 1 modulo p (caz în care vom alege un x de forma $kp - 1$);
3. fie cel mult o valoare a_i să fie congruentă cu 2 modulo p (caz în care vom alege un x de forma $kp - 2$);
4. ...

Deci trebuie să iterăm prin numerele prime și să verificăm, pentru fiecare număr prim p, dacă clasificarea valorilor a_i în clase de resturi modulo p lasă cel puțin un rest care apare cel mult o dată.

Mai rămîne o întrebare: cîte numere prime trebuie să verificăm? Toate pînă la MAX_VAL? Răspunsul este nu: sînt suficiente primele $n/2$ numere prime. Dincolo de asta, este clar că va exista un rest care apare cel mult o dată (principiul cutiei): împărțim cele n numere date în mai mult de $n/2$ clase de resturi.

Odată ce știm ce rest dorim modulo fiecare dintre aceste numere prime, Teorema chinezească a resturilor ne spune că există un x care obține toate aceste resturi simultan. Problema este un exemplu de aplicare a CRT care nu necesită și implementarea ei.

24.6.2 Problema Remainders Game (Codeforces)

enunț • sursă

Putem rezolva problema doar cu cunoștințe de divizibilitate. Fie $k = 100 = 2^2 \cdot 5^2$. Dacă $x \bmod 100$ este unic determinat, atunci putem afla în mod banal $x \bmod 4$ și $x \bmod 25$. Contrapozitiva (implicația în sens invers) este: dacă nu putem afla $x \bmod 4$ și $x \bmod 25$, atunci nici $x \bmod 100$ nu este unic determinat.

Ca să-l aflăm pe $x \bmod 4$, trebuie ca un c_i să fie divizibil cu 2^2 (sau mai mult). De exemplu, dacă $c_i = 24 = 2^3 \cdot 3$ pentru un i oarecare, din restul modulo 24 aflăm imediat restul modulo 4. Dacă, în schimb, niciun c_i nu se divide cu 4, ci doar cu 2 (sau nici măcar cu 2), nu avem destule informații ca să cunoaștem restul lui x modulo 4.

Desigur, acest argument se aplică pentru orice alt factor prim p din descompunerea lui k. Deci o primă soluție este: factorizează-l pe k, apoi pentru fiecare factor prim p^e verifică dacă vreun c_i se divide cu p^e. Există $\mathcal{O}(\log k)$ factori primi, deci algoritmul rulează în $\mathcal{O}(\sqrt{k} + n \log k)$.

Din această implementare am învățat o idee mai scurtă, mai elegantă și care folosește $\mathcal{O}(1)$ memorie. Reformulăm algoritmul anterior astfel: pentru fiecare factor prim p^e, asigură-te că maximul exponenților lui p în vectorul c este cel puțin e. Cu alte cuvinte, asigură-te că

$$p^e \mid \text{cmmmc}(c_1, c_2, \ldots, c_n)$$

Doar că cmmmc-ul poate depăși reprezentarea. Dar facem observația că nu ne interesează **cît de mare** este exponentul maxim, ci doar dacă **atinge** valoarea e. Pentru fiecare c_i putem limita exponentul lui p ca să nu depășească e, iar aici ne ajută funcția cmmdc:

$$p^e \mid \text{cmmmc}(\text{cmmdc}(c_1, p^e), \ldots, \text{cmmdc}(c_n, p^e))$$

Iar observația finală este că putem procesa toți factorii primi simultan. Efectiv, **cmmmc și cmmdc sînt operații de maxim și respectiv de minim pe exponenți**, pentru toți factorii primi independent și simultan:

$$k \mid \text{cmmmc}(\text{cmmdc}(c_1, k), \ldots, \text{cmmdc}(c_n, k))$$

Toată această demonstrație decurge direct și din Teorema chinezească a resturilor: $x \mod k$ este unic determinat dacă și numai dacă k este un divizor al celui mai mic multiplu comun, $[c_1, c_2, \ldots, c_n]$.

24.6.3 Problema Decipher the String (Codeforces)

enunț • surse

Să observăm că, pînă la lungimea 26, putem trimite șirul `ab...z` sau un prefix al său și vedem ce permutare primim. Obținem răspunsul aplicînd permutarea inversă șirului citit de la intrare. Dincolo de lungimea 26, această abordare nu mai funcționează, deoarece trebuie să introducem litere duplicat în interogări și nu știm cum le mapează permutarea.

Dat fiind că $26^3 > 10\,000$, vom compune cele trei interogări astfel:

1. `ab...zab...zab...z...` (perioadă 26).
2. `a*26 b*26 ... z*26 a*26 ...` (fiecare caracter se repetă de 26 de ori).
3. Fiecare caracter se repetă de $26^2 = 676$ de ori.

Astfel, pe fiecare din cele n poziții vom avea o combinație unică de litere. Concret, dacă interpretăm literele ca pe cifrele $0 \ldots 25$ în baza 26, atunci cifrele de pe coloana c vor fi exact reprezentarea lui c în baza 26 (pe trei cifre).

Acum interpretăm răspunsul astfel. Pe fiecare coloană d luăm cele trei cifre și formăm un număr c cu ele. Atunci permutarea secretă mapează coloana c la coloana d.

Există însă și o soluție frumoasă, chiar dacă nenecesară, bazată pe Teorema chinezească a resturilor. Alegem trei numere coprime între ele al căror produs să depășească 10 000. Eu am ales 18, 23 și 25, dar puteau fi și 23, 25 și 26 sau orice altceva. Emitem următoarele trei interogări:

1. `abcdefghijklmnopqr` (18 litere) repetat pînă la lungime n;
2. `abcdefghijklmnopqrstuvw` (23 de litere) repetat pînă la lungime n;
3. `abcdefghijklmnopqrstuvwxy` (25 de litere) repetat pînă la lungime n.

Astfel, pe fiecare coloană c cele trei litere ne indică resturile lui c modulo 18, 23 și 25. Acum interpretăm răspunsul astfel. Pe fiecare coloană d luăm cele trei cifre/resturi. Conform CRT, există un rest unic al coloanei c modulo $18 \cdot 23 \cdot 25 = 10\,350$. Cu alte cuvinte, c este unic determinat, iar CRT ne arată și un mecanism de calculare.

24.6.4 Problema Two Chandeliers (Codeforces)

enunț • sursă

Problema este o aplicație relativ directă a CRT pentru doi moduli, posibil necoprimi.

Să considerăm întîi cazul cmmdc$(n, m) = 1$. Atunci știm că cele două șiruri se vor repeta identic după o perioadă $P = nm$. Dacă o valoare v apare în ambele șiruri, ea se va suprapune fix o dată în fiecare perioadă. De ce? Dacă $a_i = b_j = v$, atunci poziția suprapunerii, x, întrunește condițiile:

$$\begin{cases} x & \equiv i \pmod{n} \\ x & \equiv j \pmod{m} \end{cases}$$

Așadar, conform CRT, $0 \leq x < P$ există, este unic și îl putem calcula. Similar putem afla toate zilele dintr-o perioadă în care se suprapun culori identice. Numărul de zile cu culori diferite din fiecare P zile este $P-S$, unde S este numărul de culori care apar în ambele șiruri. Aflăm numărul de perioade complete $C = k/(P-S)$ și reducem k-ul modulo $P-S$. Fie k' restul. Pentru a afla poziția lui k' în ultima perioadă, notăm într-un vector toate pozițiile suprapunerilor tuturor culorilor din S, îl sortăm și căutăm liniar în el. De exemplu, dacă zilele cu suprapuneri sînt 3, 8, 12, 20, atunci a 11-a zi de enervare este 14: 1, 2, 4, 5, 6, 7, 9, 10, 11, 13, 14.

În cazul general, m și n nu sînt coprime, ceea ce schimbă cîteva lucruri:

- Perioada P se generalizează la cmmmc(n, m).
- Trebuie folosită metoda CRT care funcționează și pe moduli necoprimi.
- Două poziții $a_i = b_j = v$ nu se mai suprapun garantat, ci doar dacă cmmdc$(n, m) \mid (a_i - b_j)$ (așa cum știm de la CRT).

Partea specifică problemei (căutarea celui de-al k-lea element lipsă într-un vector sortat) cere și ea puțină atenție. Cred că implementarea se simplifică dacă în loc de enervarea k folosim „răbdarea” $k-1$, adică numărul de zile în care Vasya nu se enervează. Atunci răspundem la întrebarea „cîte zile rezistă Vasya fără să se enerveze?” și adăugăm 1.

24.6.5 Problema GCD Guess (Codeforces)

enunț • surse

Soluție cu 30 de întrebări

O primă abordare îl compune pe x bit cu bit. De exemplu, putem afla paritatea lui x astfel: punem întrebarea $(1, 3)$. Dacă cmmdc$(x+1, x+3)$ este par, atunci x este în mod necesar impar și viceversa.

Acum să presupunem că am aflat deja biții cei mai nesemnificativi ai lui x, să spunem $10110_2 = 22$. Așadar, $x \equiv 22 \pmod{32}$. Ce ar putea fi bitul 5? Fie 0, caz în care $x \equiv 22 \pmod{64}$, fie 1, caz în care $x \equiv 54 \pmod{64}$. Să construim o interogare care diferențiază posibilitățile! Am vrea să alegem a și b astfel încît

$$\text{cmmdc}(a+22, b+22) \neq \text{cmmdc}(a+54, b+54)$$

Cu puțin efort, găsim de exemplu $a = 32 - 22 = 10$ și $b = 3 \cdot 32 - 22 = 74$. Așadar, punem întrebarea $(10, 74)$ și putem primi două răspunsuri:

Dacă $x = 64k + 22$, răspunsul este cmmdc$(64k + 32, 64k + 96)$, deci cmmdc-ul va fi divizibil cu 32, dar nu cu 64.

Dacă $x = 64k + 54$, răspunsul este cmmdc$(64k + 64, 64k + 128)$, deci cmmdc-ul va fi divizibil cu 64.

Soluție cu 23 sau 22 de întrebări

Soluția bazată pe Teorema chinezească a resturilor pornește de la următoarea observație. Putem afla restul lui x modulo p punînd $p - 1$ întrebări:

- $(1, p + 1)$: Dacă cmmdc-ul este multiplu de p, atunci x are forma $kp - 1$.
- $(2, p + 2)$: Dacă cmmdc-ul este multiplu de p, atunci x are forma $kp - 2$.
- ...
- $(p - 1, 2p - 1)$: Dacă cmmdc-ul este multiplu de p, atunci x are forma $kp - (p - 1) = kp + 1$.
- Altfel, x este multiplu de p.

Mai mult, calculul pentru doi moduli coprimi $p_1 < p_2$ durează doar $p_2 - 1$ întrebări. De exemplu, pentru a afla resturile lui x modulo 17 și 23 punem întrebările:

- $(1, 17 \cdot 23 + 1)$
- $(2, 17 \cdot 23 + 2)$
- ...
- $(22, 17 \cdot 23 + 22)$

Dacă pentru 5 răspunsul este multiplu de 17, iar pentru 8 răspunsul este multiplu de 23, atunci știm că $x \equiv -5 \equiv 12 \pmod{17}$ și $x \equiv -8 \equiv 15 \pmod{23}$. Generalizînd, vom căuta un set de moduli coprimi astfel încît:

- Produsul modulilor să depășească 1 miliard.
- Maximul modulilor să fie cît mai mic (acesta dictează numărul de întrebări, căci numărul necesar este modulul maxim minus unu).

Maximul minim pe care îl putem găsi este 23, iar un exemplu de moduli este

$$2^4 \cdot 3^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 = 1\,070\,845\,776$$

Implementarea mea pune 23 de întrebări, deoarece nu am mai tratat separat cazul în care din 22 de întrebări nu găsim restul. 😊

Soluție cu 12 întrebări (!!)

Am învățat următoarea abordare remarcabilă de la Rareș Neculau. El alege modulii astfel:

$$2^{11} \cdot 3^6 \cdot 5^2 \cdot 7^2 = 1\,828\,915\,200$$

Ca și în secțiunea anterioară, putem trata modulii simultan (de exemplu, punînd inițial întrebări modulo $2 \cdot 3 \cdot 5 \cdot 7$). Mai mult decît atît, putem afla restul modulo 49 din doar 12 întrebări:

- Punem întîi 6 întrebări pentru a afla dacă x are forma $7k + 0, \ldots 7k + 6$. Să spunem că aflăm că $x = 7k + 5$.
- Apoi punem încă 6 întrebări pentru a afla dacă x are forma $49k + 5, 49k + 12, \ldots, 49k + 47$.

Deci din restul modulo p^e algoritmul poate decide restul modulo p^{e+1} în $p - 1$ întrebări. De aceea el poate decide toate lucrurile următoare simultan:

- resturile modulo $2, 2^2, 2^3, \ldots, 2^{11}$ (necesită 11 întrebări);
- resturile modulo $3, 3^2, \ldots, 3^6$ (necesită $2 \cdot 6 = 12$ întrebări);
- resturile modulo 5 și 25 (necesită $2 \cdot 4 = 8$ întrebări);
- resturile modulo 7 și 49 (necesită $2 \cdot 6 = 12$ întrebări).

De exemplu, la întrebarea a 7-a vom ști ce resturi vrem să încercăm modulo 2^7, 3^4, 5^2 și 7^2. Este posibil ca modulo unele dintre aceste numere să fi aflat deja restul. Facem produsul celorlalți moduli și, cu CRT, aflăm restul global care ne interesează, pe care îl trimitem în întrebare.

Din resturile modulo $2^{11}, 3^6, 5^2$ și 7^2 aflăm restul modulo 1 828 915 200. Cu restricția de 12 întrebări, problema s-ar încadra cu brio la lot, posibil chiar la un concurs internațional.

Capitolul 25

Elemente de bază în combinatorică

25.1 Formule pentru combinări

Încercați să memorați formule de combinatorică. Oricare dintre ele vă poate servi la un moment dat.

Coeficientul binomial:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{25.1}$$

O formulă echivalentă:

$$\binom{n}{k} = \frac{n \cdot (n-1) \cdot \cdots \cdot (n-k+1)}{k!} \tag{25.2}$$

O exprimare recurentă, care duce la triunghiul lui Pascal:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \tag{25.3}$$

O listă de proprietăți, preluate din articolul foarte bun de pe CP Algorithms:

$$\binom{n}{k} = \binom{n}{n-k} \tag{25.4}$$

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1} \tag{25.5}$$

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n \tag{25.6}$$

$$\sum_{m=0}^{n} \binom{m}{k} = \binom{n+1}{k+1} \tag{25.7}$$

$$\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m} \tag{25.8}$$

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n} \tag{25.9}$$

$$\sum_{k=0}^{n} k\binom{n}{k} = n \cdot 2^{n-1} \tag{25.10}$$

25.2 Calculul combinărilor

Dacă putem stoca $\mathcal{O}(n^2)$ numere în memorie, putem calcula tabelul de combinări cu triunghiul lui Pascal.

Dacă avem de calculat doar o combinare ocazională și dacă rezultatul încape pe `long long`, putem folosi formula 25.2. Nici măcar nu este nevoie să calculăm inverse, căci nu facem împărțiri! Calculăm rezultatul de la dreapta spre stînga:

$$(n-k+1)/1 \cdot (n-k+2)/2 \cdot (n-k+3)/3...$$

Ne bazăm pe observația că, atunci cînd vine vremea să împărțim prin x, vom fi înmulțit deja x termeni consecutivi, dintre care unul se divide cu x.

Pentru valori mari, de regulă problema ne va cere să operăm modulo un număr prim. În această situație, avem nevoie să calculăm factorialele pînă la $n!$ și inversele lor modulare. Apoi, putem calcula în $\mathcal{O}(1)$ orice C_n^k.

Ca optimizare, putem calcula toate inversele factorialelor în $\mathcal{O}(n)$, apelînd o singură dată funcția de calcul al inversei. Secretul este să pornim de la n spre 0:

```
inv_fact[MAX_N] = inverse(fact[MAX_N]);
for (int i = MAX_N - 1; i >= 0; i--) {
  inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
}
```

25.3 Permutări cu repetiție

O mulțime de n elemente distincte are, desigur, $n!$ permutări. Cînd elementele nu sînt distincte (mulțimea este un multiset), atunci numărul de permutări este

$$\frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!}$$

Unde $n_1, n_2, \dots, n_k$ sînt multiplicitățile elementelor din set și $n_1+n_2+\dots+n_k = n$. Vezi Wikipedia pentru detalii.

25.3.1 Aplicație

Dacă desfacem parantezele polinomului $(a + b + c + d)^{10}$ și simplificăm, care este coeficientul monomului $a^2bc^4d^3$?

25.4 Combinări cu repetiție

Problema clasică de combinări cu repetiție este: în cîte moduri putem așeza n obiecte (identice) în k cutii?

Metoda Stars and bars reduce problema la una de combinări obișnuite, iar soluția este C_{n+k-1}^{n}. Ea procedează astfel:

- Reprezintă fiecare obiect printr-o stea. Vor exista n stele.
- Separă fiecare două cutii printr-o bară. Vor exista $k - 1$ bare.
- Orice mod de a amesteca stelele și barele corespunde în mod unic unei soluții. De exemplu, reprezentarea `**|***||*` înseamnă că sînt 4 cutii, care conțin respectiv 2, 3, 0, 1 obiecte.

O altă interpretare este: încărcăm un robot cu cele n obiecte și îl lăsăm să viziteze cele k cutii. Stelele și barele reprezintă instrucțiuni în programul robotului. O stea înseamnă „lasă un obiect în cutia curentă”, iar o bară înseamnă „avansează la cutia următoare”.

25.4.1 Aplicații

- (Variantă) În cîte moduri putem așeza n obiecte în k cutii astfel încît fiecare cutie să conțină **cel puțin un obiect**?
- În cîte moduri îl putem scrie pe n ca sumă de k termeni numere naturale? Ordinea termenilor contează.
- (Variantă) În cîte moduri îl putem scrie pe n ca sumă de k termeni numere naturale **nenule**?
- Cîte soluții în numere naturale are ecuația $a + b + c + d = 10$?
- Cîte soluții în numere naturale are inecuația $a + b + c + d \leq 10$?
- Cîți termeni are polinomul $(a + b + c + d)^{10}$ după ce desfacem parantezele și simplificăm?

25.5 Numărarea permutărilor fără punct fix

Am întîlnit inițial problema ca puzzle de logică: n persoane vin la o petrecere și își pun pălăriile în cuier. La plecare, ei se întreabă: cîte moduri există de a redistribui pălăriile, cîte una fiecărei

persoane, astfel încît nimeni să nu plece acasă cu pălăria proprie? Vezi pagina Wikipedia pentru (muuuulte) informații.

Valoarea teoretică este numărul întreg cel mai apropiat de $\frac{n!}{e}$.

Formula de recurență este $D_n = (n-1) \cdot (D_{n-1} + D_{n-2})$. Vom urmări demonstrația.

Există și o formulă de calcul cu principiul includerii și excluderii. Vom urmări demonstrația. Implementarea clasică, cu inverse factoriale, merge, dar există și una mai simplă.

25.6 Numerele lui Catalan

Al n-lea număr al lui Catalan, C_n, indică numărul de șiruri de n paranteze deschise și n paranteze închise care sînt parantezate corect. Pagina Wikipedia listează numeroase probleme echivalente. Reținem în special problema: cîte căi există de la (0,0) la (n, n), mergînd doar la dreapta și în sus, care nu trec deasupra diagonalei?

Pe baza acestei formulări vom urmări demonstrația a 2-a (una dintre multele). Rezultă formula

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

Demonstrația ne ajută să rezolvăm problema generalizată: cîte parantezări există care să înceapă obligatoriu cu $k \leq n$ paranteze deschise? Demonstrația este identică cu cea pentru problema originală:

- Pornind de la coordonatele $(k, 0)$ mai avem de făcut $n-k$ pași la dreapta și n în sus.
- Există o bijecție între căile rele și căile răsturnate.
- Căile răsturnate ajung la $(n-1, n+1)$, deci orice cale răsturnată mai are de făcut $n-k-1$ pași la dreapta și $n+1$ în sus.

De aceea, numărul total de căi pornind de la $(k, 0)$ este C^n_{2n-k}, iar numărul de căi rele este C^{n+1}_{2n-k}. Atunci numărul de căi bune este

$$\begin{aligned}
&\binom{2n-k}{n} - \binom{2n-k}{n+1} \\
&= \frac{(2n-k)!}{n!(n-k)!} - \frac{(2n-k)!}{(n+1)!(n-k-1)!} \\
&= \left[\frac{1}{n-k} - \frac{1}{n+1}\right] \cdot \frac{(2n-k)!}{n!(n-k-1)!} \\
&= \frac{k+1}{(n-k)(n+1)} \cdot \frac{(2n-k)!}{n!(n-k-1)!} \\
&= \frac{k+1}{n+1} \cdot \frac{1}{n-k} \cdot \frac{(2n-k)!}{n!(n-k-1)!} \\
&= \frac{k+1}{n+1} \cdot \binom{2n-k}{n}
\end{aligned}$$

Așadar,

$$C_n^{(k)} = \frac{k+1}{n+1}\binom{2n-k}{n}$$

.

25.7 Lema lui Burnside

Din secțiunea următoare, ultimele două probleme de pe CSES sînt rezolvabile cu această lemă. Din păcate, demonstrația este formulată doar în termeni de grupuri și nu am investit timpul necesar ca să o înțeleg. Dar principiul este următorul. Lema ne ajută să numărăm obiecte-unicat dintr-o colecție, în condițiile în care unele transformări (rotații, simetrii etc.) transformă un obiect într-altul. Procedăm astfel:

- Fie t numărul de transformări posibile.
- Fie C_r numărul de obiecte care rămîn nemodificate în urma unei transformări de tipul r.
- Atunci numărul de obiecte-unicat din colecție este $\frac{1}{t}\sum_{r=0}^{t-1} C_r$.

Pentru problema Counting Necklaces, definim transformarea r drept rotirea colierului cu r poziții. Atunci

- $C_0 = m^n$, deoarece putem asambla m^n coliere, iar dacă nu modificăm nimic... toate colierele rămîn nemodificate. 🤓
- $C_1 = m$, deoarece dacă un colier nu se modifică prin rotirea cu o poziție, atunci mărgeaua 1 are aceeași culoare cu mărgeaua 2, mărgeaua 2 are aceeași culoare cu mărgeaua 3 etc. Deci toate mărgelele au aceeași culoare.
- $C_2 = m$ dacă n este impar. De exemplu, pentru $n = 7$, mărgeaua 1 va urma pozițiile 3, 5, 7, 2, 4, 6, 1. Din nou, este necesar ca toate mărgelele să aibă aceeași culoare.
- $C_2 = m^2$ dacă n este par. Putem alege culorile primelor două mărgele, iar ele vor dicta culorile restului de mărgele.
- În general, $C_r = m^{\gcd(r,n)}$.

Pentru problema Counting Grids, raționamentul este foarte similar, dar numărul de rotații este fixat (4), iar numărul de libertăți pentru o transformare aleasă trebuie calculat. Pentru $r = 0$, numărul de posibilități este 2^{n^2} etc.

Suspectez că unele dintre probleme pot fi rezolvate și cu principiul includerii și excluderii.

25.8 Un puzzle

Iată și un puzzle care se rezolvă cu una dintre tehnicile de mai sus. Nu spunem care. 🙂

Soluția pe care o știu eu este inductivă. Fie Ana și Zaharia primul și respectiv ultimul pasager. Pentru $n = 2$, desigur probabilitatea ca Zaharia să-și găsească locul liber este $1/2$. Acum să presupunem că am rezolvat problema pentru $1, 2, \ldots, n$ și să o rezolvăm pentru $n + 1$. Ana poate alege 3 tipuri de loc: (a) propriul său loc, caz în care Zaharia are 100% șanse să-și ocupe locul; (b) locul lui Zaharia, caz în care Zaharia are 0 șanse să-și ocupe locul; (c) locul unui alt pasager, Xavier, caz în care putem reduce problema la cel mult n pasageri. Vor urca (posibil) un număr de pasageri diferiți de Xavier, care își găsesc locurile libere. Apoi va urca și Xavier.

În acest moment regăsim exact problema originală! Avem $m < n$ locuri libere, dintre care unul este al Anei, și locul lui Xavier care este ocupat. Efectiv, Xavier joacă în acest moment rolul Anei. Dacă Xavier ocupă locul Anei, ciclul se închide, iar Zaharia își va ocupa locul, similar cum în problema inițială, dacă Ana își ocupă propriul loc, ciclul se închide.

Media ponderată a celor trei probabilități este $1/2$.

25.9 Probleme

Primele 8 probleme sînt educaționale, clasice.

25.9.1 Problema Binomial Coefficients (CSES)

enunț • sursă

Este o problemă clasică de calcul al combinărilor. Pentru a calcula $C_n^k = \frac{n!}{k!(n-k)!}$ avem nevoie de inversele factorialelor. Repet că le putem calcula pe toate n în $\mathcal{O}(n)$.

25.9.2 Problema Creating Strings II (CSES)

enunț • sursă

Este o problemă clasică de permutări cu repetiție. Ca optimizare, cum rezolvăm problema fără memorie $\mathcal{O}(n)$, ci doar $\mathcal{O}(\Sigma)$?

25.9.3 Problema Distributing Apples (CSES)

enunț • sursă

Este o problemă clasică de combinări cu repetiție (*Stars and bars*). Atenție la implicația tacită: copiii sînt distincți (contează dacă Gigel primește un măr și Ionel două sau invers), dar merele sînt identice (nu contează care sînt cele două mere primite de Ionel).

25.9.4 Problema Christmas Party (CSES)

enunț • surse

Este o problemă clasică de numărare a permutărilor fără puncte fixe. Includ trei surse, două cu principiul includerii și excluderii și una cu formula de recurență.

25.9.5 Problema Bracket Sequences I (CSES)

enunț • sursă

Este o problemă elementară de calcul al numărului lui Catalan.

25.9.6 Problema Bracket Sequences II (CSES)

enunț • sursă

Probleme cere să calculăm numărul de parantezări cu prefix impus.

25.9.7 Problema Counting Necklaces (CSES)

enunț • sursă

Este o aplicație directă a Lemei lui Burnside.

25.9.8 Problema Counting Grids (CSES)

enunț • sursă

Și acestă problemă este o aplicație directă a Lemei lui Burnside.

25.9.9 Problema Number of Permutations (Codeforces)

enunț • sursă

Problema este directă, rezolvabilă cu principiul includerii și excluderii. Din toate cele $n!$ permutări,

- le scădem pe cele sortate după a;
- le scădem pe cele sortate după b;
- le adunăm pe cele sortate simultan după a și după b.

Sursa include și un exemplu de inginerie care vă poate ajuta. Ca să evităm duplicarea (sau triplicarea) codului, putem trimite funcții ca parametru.

25.9.10 Problema Counting Factorizations (Codeforces)

enunț • sursă

Soluția urmează editorialul, care este bine scris.

Să remarcăm că, dacă $f(m) = \{a_1, a_2, \ldots, a_{2n}\}$, atunci n dintre cele $2n$ numere citite la intrare trebuie să fie bazele, numere prime și distincte. Altfel problema nu are soluție. Sursa nu tratează special acest caz, dar el merita remarcat. Deoarece n dintre numere trebuie să fie prime, să citim cele n numere și să le clasificăm în prime și compuse (pe 1 îl tratăm ca număr compus). Putem folosi testul naiv de primalitate, fiind vorba despre doar 2 022 de numere mici.

Mai departe, să remarcăm că exponenții nu trebuie neapărat să fie distincți. De aceea, cele n numere rămase după alegerea bazelor pot fi permutate, dar cu eliminarea repetițiilor. De aceea, nu ne interesează numerele însele, ci frecvențele acestora. Să spunem că avem p numere prime distincte cu frecvențele $g_1, g_2, \ldots, g_p$ și c numere compuse distincte cu frecvențele $h_1, h_2, \ldots, h_c$.

Orice mod de a construi un m înseamnă că n dintre frecvențele numerelor prime vor scădea cu 1 (acelea sînt bazele alese). Să notăm frecvențele rămase cu $g'_1, g'_2, \ldots, g'_p$, unde $g'_i = g_i$ sau $g'_i = g_i - 1$. Celelalte numere rămase reprezintă cei n exponenți, iar frecvențele lor sînt $g'_1, g'_2, \ldots, g'_p, h_1, h_2, \ldots h_c$ (unele dintre aceste valori pot fi 0). Numărul de moduri de a permuta acești exponenți pentru a obține m-uri diferite este:

$$\frac{n!}{g'_1!g'_2!\ldots g'_p!h_1!h_2!\ldots h_c!}$$

Acum trebuie să însumăm această cantitate peste toate modalitățile de atribuire a valorilor g'_i. Scoatem factor comun expresia

$$T = \frac{n!}{h_1!h_2!\ldots h_c!}$$

și rămîne să însumăm rezultatele posibile pentru

$$\frac{1}{g'_1!g'_2!\ldots g'_p!}$$

Aici ne permitem să aplicăm o programare dinamică de complexitate $\mathcal{O}(n^2)$. Fie $d(i, j)$ suma termenilor de forma

$$\frac{1}{g'_1!g'_2!\ldots g'_i!}$$

unde exact j dintre factorii g' sînt **diferiți** de factorul g corespunzător (adică am ales j numere prime dintre primele i frecvențe). Atunci pe poziția i putem să alegem valoarea $g'_i = g_i$ sau valoarea $g'_i = g_i - 1$. Rezultă recurența:

$$d(i, j) = \frac{1}{g_i!} \cdot d(i-1, j) + \frac{1}{(g_i - 1)!} \cdot d(i-1, j-1)$$

Aici, primul termen semnifică că **nu** am folosit un număr prim din frecvența g_i, deci dintre primele $i - 1$ numere prime încă avem de ales j, iar al doilea termen semnifică că am folosit un număr prim din frecvența g_i, deci pînă la $i - 1$ mai avem de ales $j - 1$ numere prime. Răspunsul la problemă va fi $T \cdot d(p, n)$.

Putem implementa programarea dinamică folosind un singur vector.

25.9.11 Problema Monocarp and the Set (Codeforces)

enunț • sursă

Problema necesită o mică observație, dar sper să aveți deja acest reflex format. Este mult mai simplu să considerăm operațiile de la sfîrșit spre început. Iau naștere trei cazuri:

- Dacă ultimul caracter este `>`, atunci ultimul număr inserat este n. Reducem problema la una identică de mărime $n-1$.
- Similar, dacă ultimul caracter este `<`, atunci ultimul număr inserat este 1.
- Dacă ultimul caracter este `?`, atunci avem $n-2$ variante pentru ultimul număr inserat. Problema se reduce tot la una identică de mărime $n-1$, dar nu direct, ci printr-o renumerotare. De exemplu, dacă am alege să punem `3` pe ultima poziție, atunci mulțimea numerelor anterioare ar fi $\{1,2,4,5,6\}$, pe care însă o putem trata ca și pe $\{1,2,3,4,5\}$ fără a schimba natura problemei.

De aceea, este suficient să menținem produsul valorilor p pentru pozițiile p care conțin caracterul `?`. Aritmetica ține cu noi dacă indexăm șirul natural, de la 0. Cu alte cuvinte, poziția 0 de la intrare corespunde celui de-al doilea element din permutare. Apoi, ori de cîte ori poziția *pos* capătă valoarea `?`, înmulțim acest produs cu *pos*. Ori de cîte ori poziția *pos* capătă valoarea `<` sau `>` înmulțim produsul cu inversul lui *pos*.

Este nevoie de un caz special cînd `s[1] = '?'`, deoarece 0 nu are invers. În acel caz, răspunsul este oricum 0, deoarece după primul număr inserat, al doilea va fi obligatoriu fie minim (`<`), fie maxim (`>`).

25.9.12 Problema Devu and Flowers (Codeforces)

enunț • surse

Problema este o formă de *stars and bars*. În loc să punem s obiecte în n cutii, trebuie să le luăm, dar matematica este aceeași. În plus, cutiile au capacități. Spunem că o distribuție *saturează* o cutie de capacitate f_i dacă ia strict mai mult de f_i obiecte din cutie.

Limita $n \leq 20$ este un indiciu puternic că soluția va fi exponențială în numărul de cutii. De aici ajungem la următoarea soluție cu principiul includerii și excluderii:

- Numărăm toate cele C_{s+n-1}^{n-1} distribuții, care saturează 0 sau mai multe cutii.
- Scădem toate distribuțiile care saturează cel puțin o cutie.
- Adăugăm toate distribuțiile care saturează cel puțin două cutii.
- etc.

Odată ce fixăm mulțimea de cutii de saturat, cum anume le saturăm? Răspunsul este: plasăm $f_i + 1$ obiecte în fiecare cutie saturată i. Apoi distribuim restul de obiecte cu metoda *stars and bars* simplă, ceea ce, eventual, va adăuga și alte obiecte în cutiile saturate. Astfel ne asigurăm că generăm toate distribuțiile cu care am putea satura respectivele cutii.

Detalii de implementare

Toate combinările de calculat for avea forma C_x^{n-1}, unde n este constant, iar x este de ordinul a 10^{14}. De aceea, nu putem folosi formula naturală

$$\frac{x!}{(x-n+1)! \cdot (n-1)!}$$

întrucît nu putem calcula factoriale atît de mari. În schimb, putem folosi formula

$$\frac{x \cdot (x-1) \cdot \cdots \cdot (x-n+2)}{(n-1)!}$$

Ca optimizare, putem precalcula numitorul la începutul problemei. Astfel ia naștere prima sursă.

Putem chiar să definim constante pentru numitorul $1/(n-1)!$ pentru cele 20 de valori posibile pentru n. Sursa se scurtează, căci putem renunța la codul pentru calcularea inverselor. Totuși, sursa pierde din claritate.

Pentru elevii care se dau în vînt după programe scurte, indiferent de claritate, am scris și o ultimă sursă aliniată la standardele de lizibilitate ale lui Codeforces!

25.9.13 Problema Lengthening Sticks (Codeforces)

enunț • sursă

Soluție directă

Există o soluție bazată pe numărare directă, dar are multe cazuri particulare. Fie x, y și z cantitățile adăugate la a, b și respectiv c. Iterăm prin toate valorile lui z de la 0 la l și scriem inegalitățile în x și y care rezultă din inegalitatea triunghiului. De exemplu:

$$(a+x) < (b+y) + (c+z) \implies x - y < b + c - a + z$$

Astfel deducem limite superioare și inferioare pentru $x+y$ și $x-y$. Acestea delimitează un dreptunghi în planul xOy rotit cu 45°. Aria acestui dreptunghi este ușor de aflat. Din păcate, dreptunghiul mai trebuie decupat cu condițiile suplimentare $x \geq 0$ și $y \geq 0$, ceea ce duce la figuri complicate.

Soluție prin excludere

O altă abordare este să numărăm toate modurile de a distribui cel mult l unități, apoi să excludem tripletele ilegale.

Știm că putem distribui **exact** l unități în C_{l+2}^{l} moduri (este o problemă clasică de *stars and bars*). Rezultă că numărul de moduri de a distribui cel mult l unități este

$$C_{l+2}^{l} + C_{(l-1)+2}^{l-1} + \cdots + C_{2}^{0}$$

Această sumă este egală cu C_{l+3}^{3} (vezi equația 25.8). Dacă preferați, putem introduce un al patrulea băț fictiv care să preia surplusul din l nefolosit cu a, b și c. Ajungem la aceeași formulă. Ea poate fi calculată și cu un `for` în $\mathcal{O}(l)$.

Acum să trecem la excludere. Din nou, fie x, y și z cantitățile adăugate la a, b și respectiv c. Presupunem că dorim ca $a+x \geq b+y+c+z$ (vom proceda similar pentru $b+y$ și pentru $c+z$). Obținem inegalitățile:

$$\begin{cases} y+z & \leq l-x \\ y+z & \leq a+x-b-c \end{cases}$$

Fie $h = \min(l-x, a+x-b-c)$ limita superioară pentru $y+z$. Dacă h este negativ, nu există combinații valide pentru x și y. Dacă $h \geq 0$, atunci numărul de combinații valide este

$$0+1+\cdots+(h+1) = \frac{(h+1)(h+2)}{2}$$

25.9.14 Problema Shuffle (Codeforces)

enunț • surse

Ca la orice problemă de numărare, trebuie să ne asigurăm de două lucruri:

1. Că numărăm toate posibilitățile.
2. Că nu numărăm nicio posibilitate de mai multe ori.

Facem și observația universală că șirul dat trebuie să aibă cel puțin k caractere `1`.

Soluția în $\mathcal{O}(n^2)$

Este tentant să încercăm următoarea abordare: pentru fiecare subsecvență $W = [l, r]$ adăugăm la răspuns cantitatea C_{r-l+1}^{x}, unde x este numărul de caractere `1` din W. Dar este incorect, căci vom număra și unele permutări care lasă nemodificate porțiuni din W. Din această cauză, două subsecvențe parțial suprapuse, W și W', vor număra de două ori permutările care modifică doar zona comună $W \cap W'$.

Să contabilizăm subsecvențele într-un mod care elimină duplicatele. Fie $[l, r]$ o subsecvență amestecată astfel încît l și r sînt prima, respectiv ultima poziție modificată. Secvența merită luată în calcul dacă întrunește trei condiții:

1. Are lungime cel puțin 2. O secvență de lungime 1 nu poate fi amestecată. 🤔
2. Conține cel mult k cifre 1. Mai multe nu avem voie să selectăm, dar putem selecta mai puține (ne prefacem că amestecăm o secvență care include $[l, r]$, dar lasă nemodificate prefixul și

sufixul).

3. Secvența conține cifrele necesare pentru a nega pozițiile l și r. Dacă $s[l] = s[r] = 1$, secvența trebuie să conțină și două zerouri. Similar dacă $s[l] = s[r] = 0$. Iar dacă $s[l] \neq s[r]$, atunci nu mai punem alte condiții, căci întotdeauna putem interschimba $s[l]$ cu $s[r]$.

Odată fixate capetele, adăugăm la răspuns o cantitate de forma C_{x+y}^{x}.

Soluția în $\mathcal{O}(n)$

Iată un alt mod de a privi condiția (2). Cînd alegem o subsecvență care conține **exact** k de 1 și o permutăm, unele dintre permutări vor lăsa nemodificate un prefix și un sufix al subsecvenței. În acele situații este ca și cînd am permuta o subsecvență cu **cel mult** k de 1.

Pornind de la această observație, putem optimiza soluția de mai sus astfel. Să fixăm doar capătul stîng, l. Atunci capătul drept r poate fi oriunde începînd de la $l + 1$ pînă la cea de-a k-a apariție a unui 1, plus toate zerourile următoare. Apoi, impunem o singură condiție:

1. Undeva în $[l + 1, r]$ există un caracter diferit de $s[l]$. Altfel nu ne putem asigura că prima modificare este pe poziția l.

Odată fixat capătul stîng, orice amestecare distinctă a caracterelor din $[l, r]$, cîtă vreme garantează că îl modifică pe $s[l]$, este o soluție distinctă și numărată o singură dată. Dacă am grupa acum aceste amestecări după poziția ultimului caracter modificat, r, am regăsi subsecvențele $[l, r]$ din soluția anterioară.

25.9.15 Problema Manhattan (Lot 2017)

enunț • surse

Această problemă ne forțează să trecem prin cîteva formule de combinatorică. Iată soluția, așa cum am conceput-o eu.

Algoritmul naiv ar face efort $\mathcal{O}(MAX_COORD^4)$ pentru a procesa fiecare pereche de puncte. Este nevoie de îmbunătățiri succesive pentru a inventaria drumurile în grupe mai mari.

Oglindiri pentru reducerea numărului de cazuri

Enunțul garantează că există o dreaptă (orizontală sau verticală) care separă zonele. Cu alte cuvinte, zona B se află undeva „în jurul" zonei A. Am vrut să reduc numărul de cazuri la unul singur: zona B se află strict în dreapta zonei A. De aceea am aplicat două transformări, la nevoie:

1. Dacă separația este pe verticală (zona B este deasupra sau dedesubtul zonei A), atunci oglindim zonele față de diagonală, inversînd x-ii cu y-ii. Astfel ne asigurăm că există un x care separă zonele.
2. Dacă A este în dreapta lui B, interschimbăm A cu B.
3. Pentru simplitate, am translatat totul pentru aduce latura stîngă a zonei A pe axa Oy.

Observăm că toate liniile orizontale din zona A ocupă același interval pe axa Ox, pe care îl vom denumi $[0, w]$ (de la engl. *width*). Similar, toate liniile orizontale din zona B ocupă același interval pe axa Ox, pe care îl vom denumi $[x_1, x_2]$.

Drumuri de la un punct la o linie

Fie $PL(d)$ numărul de drumuri de la un punct de pe latura stîngă a zonei A (așadar $x = 0$) la o linie din zona B, astfel încît distanța pe verticală de la punct la linie să fie d. Pentru simplitate, vom considera punctul ca fiind originea $(0, 0)$ și linia ca fiind $(x_1, d) \to (x_2, d)$. Desigur, numărul de drumuri de la $(0, 0)$ la un punct oarecare (x, d) este C^d_{x+d}. Deci avem:

$$PL(d) = \sum_{x=x_1}^{x_2} C^d_{x+d} = C^{d+1}_{x_2+d+1} - C^{d+1}_{x_1+d}$$

Am folosit formula pentru sume de combinări cu k fixat:

$$\sum_{m=0}^{n} C^k_m = C^{k+1}_{n+1}$$

Drumuri de la o linie la o linie

Mai departe, fie $LL(d)$ numărul de drumuri de la orice punct de pe o linie din zona A la orice punct de pe o linie din zona B, astfel încît distanța pe verticală dintre linii să fie d. Pentru simplitate, vom considera liniile $(0, 0) \to (w, 0)$ și $(x_1, d) \to (x_2, d)$.

Observăm că, pe măsură ce punctul din zona A avansează la $(1, 0)$, apoi la $(2, 0)$ etc., distanțele pînă la zona B scad la $[x_1 - 1, x_2 - 1]$, apoi la $[x_1 - 2, x_2 - 2]$ etc. Obținem sumele:

$$LL(d) = \sum_{x=0}^{w} (C^{d+1}_{x_2+d+1-x} - C^{d+1}_{x_1+d-x}) = (C^{d+2}_{x_2+d+2} - C^{d+2}_{x_2+d+1-w}) - (C^{d+2}_{x_1+d+1} - C^{d+2}_{x_1+d-w})$$

Acest rezultat exprimă drumurile între orice două linii aflate la distanță d pe verticală. Remarcăm că valorile w, x_1 și x_2 sînt constante pentru orice pereche de linii din cele două zone. Mai remarcăm și că $LL(d) = LL(-d)$: nu contează dacă drumurile merg d pași în sus sau în jos.

De aici, există (cel puțin) doi algoritmi.

Varianta 1: Agregarea pînă la zone întregi

Drumuri de la o linie la o zonă Fie acum $LZ(d_1, d_2)$ numărul de drumuri de la linia $(0, 0) \to (w, 0)$ la zona $(x_1, d_1) \to (x_2, d_2)$, unde $0 \le d_1 \le d_2$. Desigur, trebuie doar să însumăm $LL(d)$ de la d_1 la d_2. Pentru a putea calcula sumele de combinări cu aceeași formulă ca mai sus, îl vom rescrie pe $LL(d)$ folosind egalitatea $C^k_n = C^{n-k}_n$.

$$LZ(d_1, d_2) = \sum_{d=d_1}^{d_2} LL(d) = \sum_{d=d_1}^{d_2} [(C_{x_2+d+2}^{d+2} - C_{x_2+d+1-w}^{d+2}) - (C_{x_1+d+1}^{d+2} - C_{x_1+d-w}^{d+2})] = \sum_{d=d_1}^{d_2} [(C_{x_2+d+2}^{x_2} - C_{x_2+d+1-w}^{x_2-1-w}) - ($$

Rezultă o formulă cu 8 termeni, pe care o omitem pentru economie de spațiu și timp. Ea se regăsește în cod.

Drumuri între cele două zone Acest ultim pas se complică deoarece zonele nu sînt neapărat separate pe verticală. De aceea, nu putem pur și simplu să însumăm formula din secțiunea anterioară pentru toate coordonatele y din zona A. În loc de aceasta, ne vom mulțumi cu o soluție în $O(MAX_COORD)$.

Iterăm prin fiecare coordonată y din zona A. Dacă această coordonată nu intersectează zona B, atunci apelăm exact formula de mai sus, înlocuind însă limitele verticale d_1 și d_2 cu $y_1 - y$ și $y_2 - y$. Dacă însă coordonata y intersectează zona B, atunci împărțim drumurile care pornesc de pe linia y în două categorii:

1. drumuri care urcă înspre zona $(x_1, y) \to (x_2, y_2)$;
2. drumuri care coboară înspre zona $(x_1, y - 1) \to (x_2, y_1)$.

Cazul (1) îl tratăm ca și pînă acum, iar cazul (2) îl oglindim față de y, obținînd dreptunghiul $(x_1, y + 1) - (x_2, 2y - y_1)$.

Varianta 2: Agregarea pînă la linii, apoi contabilitate

Ne folosim doar de $LL(d)$. Iterăm prin toate valorile posibile pentru deplasarea d. Pentru o deplasare d aleasă, adunăm la rezultat cantitatea $LL(d)$ înmulțită cu numărul de perechi de linii, una din A și una din B, între care distanța pe verticală este d. Să notăm această cantitate cu $P(d)$.

Cît este $P(d)$? Dat fiind că zona B ocupă liniile pe verticală de la y_1 la y_2, o putem pune în corespondență cu liniile din zona A cu coordonate între $y_1 - d$ și $y_2 - d$ (astfel ca deplasarea pe verticală să fie de d pași). Așadar, trebuie doar să intersectăm intervalul zonei A, să-l numim (y_1^A, y_2^A), cu intervalul de interes $(y_2 - d, y_1 - d)$. Această operație este elementară.

Capitolul 26

Rangul permutărilor și al combinărilor

Programa de clasa a 10-a conține și *Determinarea numărului de ordine pentru elementele combinatoriale*. Ceea ce poate însemna orice din vreo 10 subiecte diferite. 🙂

Vom discuta despre permutări și combinări. Aceleași principii se aplică și la alte elemente combinatoriale (aranjamente, submulțimi etc.).

26.1 Definiții

Rangul unui obiect este numărul lui de ordine în lista ordonată a tuturor obiectelor de acel tip. Exemple:

- Rangul permutării (3 1 2) este 4 (pornind de la 0). Cele 4 permutări anterioare, în ordine, sînt (1 2 3), (1 3 2), (2 1 3) și (2 3 1).
- Rangul combinării de 5 luate cîte 3 (1 4 5) este 5. Combinările anterioare sînt (1 2 3), (1 2 4), (1 2 5), (1 3 4), (1 3 5).

Vom folosi termenii englezești *ranking* pentru aflarea rangului unui obiect dat și *unranking* pentru aflarea obiectului dîndu-i-se rangul. Ordinea lexicografică nu este unica posibilă. De exemplu, problema Arbperm2 specifică altă ordine, iar în problemele Inversion Sort și Four Chips sîntem liberi să ne alegem orice ordine, singurul scop fiind să mapăm permutări/combinări la întregi ca să ținem evidența obiectelor deja întîlnite.

Următorul obiect al unui obiect dat este obiectul de rang cu 1 mai mare. Exemple:

- Permutarea care urmează după (3 1 2) este (3 2 1).
- Combinarea de 5 luate cîte 3 care urmează după (1 4 5) este (2 3 4).

Aveți deja toate uneltele ca să calculați ad-hoc toate aceste informații, dar este bine să ne familiarizăm cu unele subtilități.

26.2 Rangul permutărilor

26.2.1 Următoarea permutare

Fie permutările

$$P = (2\ 5\ 4\ 8\ 7\ 6\ 3\ 1\ 0)$$
$$next(P) = (2\ 5\ 6\ 0\ 1\ 3\ 4\ 7\ 8)$$

Procedura pentru a trece de la P la $next(P)$ este simplă:

1. Găsește cel mai lung sufix descrescător, (8 7 6 3 1 0).
2. Schimbă elementul anterior sufixului, 4, cu cel mai mic element din sufix mai mare decît acesta (6).
3. Răstoarnă sufixul.

Complexitatea amortizată pentru mai multe apeluri este $\mathcal{O}(1)$ (de ce?). Cred că funcția din STL `std::next_permutation()` aplică fix acest algoritm. Ea garantează că face cel mult $n/2$ interschimbări (de ce?).

26.2.2 *Ranking*

Deoarece $21! > 2^{64}$, calculul complet este de interes doar pentru permutări de cel mult 20 de elemente (problema Inversion Sort). În rest, trebuie să operați cu ranguri fără a le calcula efectiv.

Calculați manual rangul pe exemplul de mai sus (103679)! Vom deduce că întrebarea esențială este: pentru o poziție dată k, cîte valori mai mici decît $P(k)$ se află pe pozițiile anterioare?

26.2.3 *Unranking*

Întrebarea de bază la *unranking* este cea inversă: care este valoarea potrivită de așezat la poziția curentă? Ea va fi a k-a valoare cu proprietatea că numărul de permutări care încep cu primele $k-1$ valori însumate nu depășește rangul dorit, dar dacă includem și a k-a valoare depășim rangul.

Pentru ordinea naturală (lexicografică), algoritmul se reduce la o împărțire la $(n-k)!$ și la întrebarea: care este a k-a cea mai mică valoare încă nescrisă la stînga poziției curente?

Calculați permutarea din rangul de mai sus!

26.2.4 Implementare

Putem răspunde la aceste întrebări în timp total $\mathcal{O}(n \log n)$ folosind un AIB sau alte structuri cu interogare în timp logaritmic. Atenție însă, pentru n mic implementarea naivă (pătratică) va fi

mai rapidă. În schimb, putem cîștiga timp folosind măști de biți.

Pentru *unranking*, trebuie să răspundem rapid la întrebarea: care este al k-lea bit zero dintr-o mască de biți? O variantă este să precalculăm acest răspuns pentru toate măștile. Dar pentru $n = 20$, aceasta va necesita memorie $2^n \cdot n = 20$ MB, ceea ce nu este ideal. Există și o operație primitivă, PDEP, dar ea nu este implementată pe toate arhitecturile și poate funcționa chiar mai lent decît căutarea naivă.

Vom citi implementările acestor algoritmi. Iată și un *benchmark* pentru $n = 20$:

```
[cata@neil combinatorics]$ ./permutation-rank
inițializare: 942 ms pentru 10000000 operații (10 Mops/sec)
--
rank naiv: 890 ms pentru 10000000 operații (11 Mops/sec)
rank cu AIB: 1984 ms pentru 10000000 operații (5 Mops/sec)
rank cu popcount: 180 ms pentru 10000000 operații (55 Mops/sec)
--
unrank naiv: 3424 ms pentru 10000000 operații (2 Mops/sec)
unrank cu AIB: 4016 ms pentru 10000000 operații (2 Mops/sec)
unrank cu popcount și tabel: 1195 ms pentru 10000000 operații (8 Mops/sec)
unrank cu popcount și pdep: 5739 ms pentru 10000000 operații (1 Mops/sec)
```

26.3 Rangul combinărilor

În cazul combinărilor, discuțiile sînt mai nuanțate. Problemele de rang au valori mici pentru k, dar nu neapărat și pentru n. Bunăoară, pentru $k = 3$, valoarea maximă pentru n astfel încît $C_n^k < 2^{64}$ (adică rangul să încapă pe `unsigned long long`) este circa 2.000.000, pe cînd pentru $k = 20$, valoarea maximă pentru n este 86.

26.3.1 Următoarea combinare

Să analizăm un exemplu. Fie combinarea de 20 luate cîte 6 (numerotate de la 0):

$$C = (4\ 7\ 12\ 17\ 18\ 19)$$
$$next(C) = (4\ 7\ 13\ 14\ 15\ 16)$$

Rezultă că algoritmul este:

1. Găsește ultima poziție i pentru care $C[i] - i < n - k$.
2. Dacă acea poziție nu există, atunci C este ultima lexicografic.
3. Altfel, incrementează $C[i]$ și pe toate pozițiile $j > i$ atribuie $C[j] = C[j-1] + 1$.

Și acest algoritm are complexitate amortizată $\mathcal{O}(1)$ (de ce?).

26.3.2 *Ranking* lexicografic

Să considerăm combinarea de 4 elemente dintr-un set de 20 (numerotate începînd de la 0):

$$C = (4\ 7\ 13\ 16)$$

Dacă dorim să îi calculăm rangul lexicografic, atunci facem următoarele observații:

- Există $C_{19}^3 = 969$ combinări care încep cu 0, deoarece dacă îl alegem pe 0 mai rămîn 19 alte elemente din care trebuie să alegem 3.
- Există $C_{18}^3 = 816$ combinări care încep cu 1.
- Există $C_{17}^3 = 680$ combinări care încep cu 2.
- Există $C_{16}^3 = 560$ combinări care încep cu 3.
- Toate combinările de mai sus sînt mai mici lexicografic decît P.
- Există $C_{14}^2 = 91$ combinări care încep cu 4 5, deoarece mai rămîn valorile 6-19 din care trebuie să alegem două.
- Există $C_{13}^2 = 78$ combinări care încep cu 4 6.
- Există $C_{11}^1 = 11$ combinări care încep cu 4 7 8.
- ...
- Există $C_7^1 = 7$ combinări care încep cu 4 7 12.
- Există $C_5^0 = 1$ combinări care încep cu 4 7 13 14.
- Există $C_4^0 = 1$ combinări care încep cu 4 7 13 15.

Rangul lui (4 7 13 16) este suma combinărilor de mai sus, adică 3241.

Algoritmul funcționează în $\mathcal{O}(n)$. El poate fi îmbunătățit pînă la $\mathcal{O}(k)$ cu formula

$$\sum_{m=0}^{n} C_m^k = C_{n+1}^{k+1}$$

26.3.3 *Unranking* lexicografic

Calculați combinarea din rangul de mai sus! Algoritmul naiv funcționează în $\mathcal{O}(n+k)$ urmînd exact pașii de mai sus. De exemplu, va scădea succesiv valorile 919, 816, 680 și 560 din rangul dat și va constata că nu mai poate scădea încă $C_{15}^3 = 455$, deci primul element trebuie să fie 4.

Putem reduce complexitatea la $\mathcal{O}(k+\log n)$ folosind sume parțiale și căutări binare. Nu sînt sigur dacă există vreun algoritm în $\mathcal{O}(k)$.

26.3.4 *Ranking* și *unranking* colexicografic

O altă ordine notabilă este ordinea colexicografică. În acestă ordine, combinările sînt ordonate de la dreapta către stînga. Ne ajută mai mult să considerăm că elementele au valori între 0 și $n-1$, iar pozițiile între 0 și $k-1$, ca în tabelul următor:

rang	combinare	proveniență
0	(0 1 2)	$C_0^1 + C_1^2 + C_2^3$
1	(0 1 3)	$C_0^1 + C_1^2 + C_3^3$
2	(0 2 3)	$C_0^1 + C_2^2 + C_3^3$
3	(1 2 3)	$C_1^1 + C_2^2 + C_3^3$
4	(0 1 4)	$C_0^1 + C_1^2 + C_4^3$
5	(0 2 4)	$C_0^1 + C_2^2 + C_4^3$
6	(1 2 4)	$C_1^1 + C_2^2 + C_4^3$
7	(0 3 4)	$C_0^1 + C_3^2 + C_4^3$
8	(1 3 4)	$C_1^1 + C_3^2 + C_4^3$
9	(2 3 4)	$C_2^1 + C_3^2 + C_4^3$
10	(0 1 5)	$C_0^1 + C_1^2 + C_5^3$
11	(0 2 5)	$C_0^1 + C_2^2 + C_5^3$
12	(1 2 5)	$C_1^1 + C_2^2 + C_5^3$
13	(0 3 5)	$C_0^1 + C_3^2 + C_5^3$
14	(1 3 5)	$C_1^1 + C_3^2 + C_5^3$
15	(2 3 5)	$C_2^1 + C_3^2 + C_5^3$
16	(0 4 5)	$C_0^1 + C_4^2 + C_5^3$
17	(1 4 5)	$C_1^1 + C_4^2 + C_5^3$
18	(2 4 5)	$C_2^1 + C_4^2 + C_5^3$
19	(3 4 5)	$C_3^1 + C_4^2 + C_5^3$

Tabela 26.1: Combinări de $n = 6$ luate cîte $k = 3$, ordonate colexicografic

Calculul rangului unei combinări este facil în $\mathcal{O}(k)$. Să luăm exemplul combinării $X = (2\ 3\ 5)$ și să procedăm de la dreapta la stînga.

- Toate combinările terminate cu 2, 3 sau 4 vin înaintea lui X. Numărul lor este exact numărul de combinări care folosesc doar valorile 0, 1, 2, 3 sau 4, deci C_5^3.
- Similar, combinările terminate cu $(a\ 5)$, unde $a < 3$, vin înaintea lui X. Numărul lor este C_3^2.
- În sfîrșit, combinările terminate cu $(b\ 3\ 5)$, unde $b < 2$, vin înaintea lui X. Numărul lor este C_2^1. Astfel obținem rangul lui X, $10 + 3 + 2 = 15$.

```
u64 rank_colex(int* c) {
  u64 result = 0;
  for (int i = 0; i < K; i++) {
    result += choose[c[i]][i + 1];
  }
  return result;
}
```

Și partea de *unranking* este simplă în $\mathcal{O}(n)$. Să pornim cu rangul 5. Vom completa combinarea X de la dreapta la stînga.

- Ar putea ultima valoare să fie 5? Știm că există $C_5^3 = 10$ combinări care folosesc doar valorile 0-4 și toate ar trebui să vină înaintea lui X. Dar $10 > 5$, deci ultima valoare **nu**

poate fi 5.

- Ar putea ultima valoare să fie 4? Știm că există $C_4^3 = 4$ combinări care folosesc doar valorile 0-3 și toate ar trebui să vină înaintea lui X. Deoarece $4 \leq 5$, ultima valoare este 4. Scădem 4 din rang și dorim să aflăm permutarea de rang 1 dintre cele terminate cu 4.
- Ar putea penultima valoare să fie 3? Știm că există $C_3^2 = 3$ moduri de a lua două din valorile 0-2 și toate ar trebui să vină înaintea lui X. Deoarece $3 > 1$, penultima valoare **nu** poate fi 3.
- Ar putea penultima valoare să fie 2? Știm că există $C_2^2 = 1$ mod de a lua două din valorile 0-1 și el ar trebui să vină înaintea lui X. Deoarece $1 \leq 1$, penultima valoare este 2. Scădem 1 din rang și dorim să aflăm permutarea de rang 0 dintre cele terminate cu 2 4.
- Similar raționăm că prima valoare nu poate fi 1, dar va fi 0. Aflăm că $X = (0\ 2\ 4)$.

```
void unrank_colex(u64 rank, int* dest) {
  int n = N - 1;
  for (int k = K; k; k--) {
    while (choose[n][k] > rank) {
      n--;
    }
    rank -= choose[n][k];
    dest[k - 1] = n;
  }
}
```

Implementarea completă este anexată.

26.4 Probleme

26.4.1 Problema Misha and Permutation Summation (Codeforces)

enunț • sursă

Este o problemă directă de calcul al rangului unei permutări (*ranking*) și de aflare a permutării cu un rang dat (*unranking*). Singura neplăcere este că nu putem stoca rangul pe un tip de date scalar. Îl vom stoca pe un vector în baza factorial (adică poziția i poate lua valori între 0 și i).

26.4.2 Problema Inversion Sort (SPOJ)

enunț • sursă

Problema este una de BFS. Trebuie să aflăm distanțele (în număr de operații) de la *abcdefghij* la toate celelalte permutări ale șirului.

Este nevoie de o observație ca să rezolvăm cazul cînd permutarea de pornire este diferită de *abcdefghij*. Putem rescrie permutarea de pornire, redenumind literele ca să ajungem la *abcdefghij*, și aplicăm aceeași redenumire și permutării-destinație. Atunci soluția trebuie să fie identică, doar

cu alte nume pentru litere.

Cum tratăm BFS-ul? O variantă este să stocăm un `map` de la șiruri la distanțe. Dar cred că această metodă va fi lentă. Este mai eficient să mapăm fiecare permutare la rangul său. Astfel ne ajunge un vector de $10! \approx 3\,600\,000$ octeți pentru distanțe.

Noi nu știm să facem mutări pe ranguri, ci doar pe permutări. De aceea, avem două variante:

1. Ținem coada BFS doar de ranguri. La scoaterea din coadă, apelăm *unrank* ca să recuperăm permutarea, pe care facem mutări.
2. Ținem coada BFS de perechi (permutare, rang). Va fi probabil mai rapid și nu mai este nevoie să scriem funcția *unrank*.

Dat fiind că n este fixat și mic, putem calcula rangul în $\mathcal{O}(1)$ per poziție cu măști de biți.

Am încercat și 3 optimizări:

1. v2 calculează mai eficient mutările, făcînd o singură transpoziție per mutare. Efectul este mic, semn că apelurile la *rank* sînt predominante.
2. v3 calculează *popcount* cu tabel predefinit. Timpul de rulare scade sub jumătate. Aceasta este varianta pe care am inclus-o în anexă.
3. v4 operează pe cifre în loc de litere. Timpul de rulare scade cam cu 5%.

26.4.3 Problema Four Chips (SPOJ)

enunț • sursă

Problema este similară cu Invesort, în sensul că necesită un BFS de la starea inițială. Ca și acolo, putem evita partea de *unranking*.

Este nevoie de ceva mai multă logistică, în special la generarea mutărilor. Există cel mult 20 de mutări posibile din fiecare poziție: 4 piese × (2 deplasări + 3 salturi). Ca să nu includ acel cod în bucla principală a BFS-ului, am decis să parametrizez generatorul de mutări, ca să accepte un parametru între 0 și 19.

Putem discuta despre diverse idei de optimizare a generatorului de mutări. De exemplu, putem simplifica testul dacă o celulă este goală (funcția `empty()`) dacă stocăm și un bitset de 70 de biți pentru ocuparea tablei.

26.4.4 Problema Long Permutation (Codeforces)

enunț • sursă

Problema este relativ clasică de *ranking*. Este nevoie doar de următoarea observație teoretică. Pot exista cel mult $2 \cdot 10^5$ operații de avansare a rangului cu cel mult 10^5 fiecare, deci rangul maxim la care putem ajunge este $2 \cdot 10^{10}$.

De cîte elemente este nevoie ca să atingem acest rang? Există $14! \approx 8,7 \cdot 10^{10}$ moduri de a permuta ultimele 14 elemente, deci primele $n - 14$ elemente nu se modifică niciodată. De aceea,

spargem permutarea în:

1. Primele $n-14$ elemente (nu mai puțin de 0), care vor avea întotdeauna valorile $1, 2, \ldots, n-14$.
2. Restul, pe care chiar le vom permuta. Pentru simplitate, le putem renumerota de la 0, iar la operațiile de sumă parțială ne vom aminti să adunăm la fiecare cîte $n-14$.

Cele două operații se reduc la:

1. Menține rangul curent al permutării (suma x-urilor de pînă acum). La fiecare nou x, generăm permutarea de 14 elemente.
2. Descompune intervalul dat într-unul inclus în prefixul imutabil și unul inclus în permutarea generată.

Pentru claritate, sursa încapsulează separat permutarea „clasică” și permutarea „din bucăți”.

26.4.5 Problema Arbperm2 (NerdArena)

enunț • sursă

Remarcăm că definiția rangului este semnificativ diferită, dar rangul este încă ușor de calculat. Teoretic problema constă tot din *ranking* – adaugă k – *unranking*. Dar rangul depășește rapid `long long`.

Rezolvarea greșită

Să socotim rangurile de la dreapta spre stînga. Atunci vrem să calculăm permutarea cu rang cu k mai mic.

Începem să scoatem elemente din permutare și să calculăm rangul. Elementul n va contribui cu $0 \le x < n$, în funcție de poziția sa. Elementul $n-1$ va contribui cu $y \cdot n$, unde $0 \le y < n-1$. De exemplu, elementul 3 poate contribui cu 0 (pentru cea mai din dreapta grupă de 4 permutări) sau cu 4 pentru a doua cea mai din dreapta grupă de 4. Elementul $n-2$ va contribui cu $z \cdot n(n-1)$, unde $0 \le z < n-2$. De exemplu, elementul 2 va contribui cu 0 pentru cele 12 permutări din dreapta și cu 12 pentru cele 12 permutări din stînga.

Cînd ajungem la un rang cel puțin egal cu k, ne oprim din eliminat. Scădem k din rang și reinserăm elementele în ordinea impusă de rangul rămas. Practic, urcăm în arbore pînă cînd nodul curent subîntinde cel puțin k frunze la dreapta.

Nici măcar nu este nevoie de structuri de date pe vectori. Putem face operațiile de eliminare/inserare în vector în $\mathcal{O}(n)$, căci sînt puține. Această sursă ia 100p... dar de fapt greșește. Doar că testele din problema originală (Arbperm) nu expun cazul special care trebuie tratat. De aceea am creat problema Arbperm2, cu ultimul test modificat.

Unde este greșeala?

Greșeala apare pe testul:

```
100000 1
2 1 3 4 5 6 7 8 ... 99999 100000
```

Răspunsul este:

```
100000 99999 ... 8 7 6 5 4 3 1 2
```

Exemplul arată că pot exista $\mathcal{O}(n)$ elemente care migrează și pe care soluția de mai sus trebuie să le elimine și să le reinsereze. Mai rău, trebuie să urcăm în arbore pînă cînd subîntindem 100!/2 noduri, ceea ce depășește reprezentarea.

Rezolvarea corectă

Facem următoarea observație. Valoarea n migrează circular prin pozițiile $0, 1, \ldots, n-1$, începînd de la poziția pe care o are la intrare. Deci îi putem afla poziția finală. Pentru exemplul 2 din enunț, valoarea 4 începe de pe poziția 2 și ciclează prin pozițiile 3, 0, 1, 2, 3, 0, 1.

Mult mai interesantă este valoarea $n-1$. Acesta migrează cu o poziție spre dreapta ori de cîte ori valoarea n ajunge pe prima poziție. De asemenea, pozițiile acestei valori trebuie calculate între 0 și $n-2$, ignorînd aparițiile lui n.

Pe cazul general al unei valori v, aflată pe poziția inițială $a[v]$:

- Poziția finală $b[v]$ este dată de $a[v]$ și numărul de avansuri modulo v.
- $a[v]$ și $b[v]$ țin cont doar de elementele mai mici decît v.
- Numărul de avansuri al valorii $v-1$ este dat de numărul de treceri peste 0 ale lui v.

Putem găsi pozițiile inițiale $a[v]$ ale fiecărei valori cu un AIB. Apoi calculăm în mod banal pozițiile finale $b[v]$. Din acestea putem recupera permutarea finală. Procesăm valorile v de la mare la mic. Valoarea n stă fix pe poziția $b[n]$. Apoi, fiecare valoare x stă pe a $b[x]$-a poziție încă liberă din permutarea finală.

26.4.6 Problema Hipersimetrie (ONI 2019, clasele 11-12)

enunț • sursă

În acest capitol ne-am concentrat pe rangul permutărilor și al combinărilor, dar iată și o problemă de *unranking* pe matrice / pe șiruri binare. Este genul meu de problemă: 😁 un experiment de gîndire interesant care duce la un program clar și relativ scurt.

O imagine

Aceasta este figura pe care am folosit-o la rezolvarea problemei, redesenată pentru claritate, dar altfel nemodificată. Mi se pare că pe baza acestei figuri, codul se scrie singur (Hans Niemann dă din cap aprobator). Vă recomand și vouă ca, dacă ideea problemei nu este clară, să alegeți exemple mai mari decît $n = 3$ sau $n = 4$. Investiți timp într-un exemplu mare, căci el va simplifica înțelegerea algoritmului.

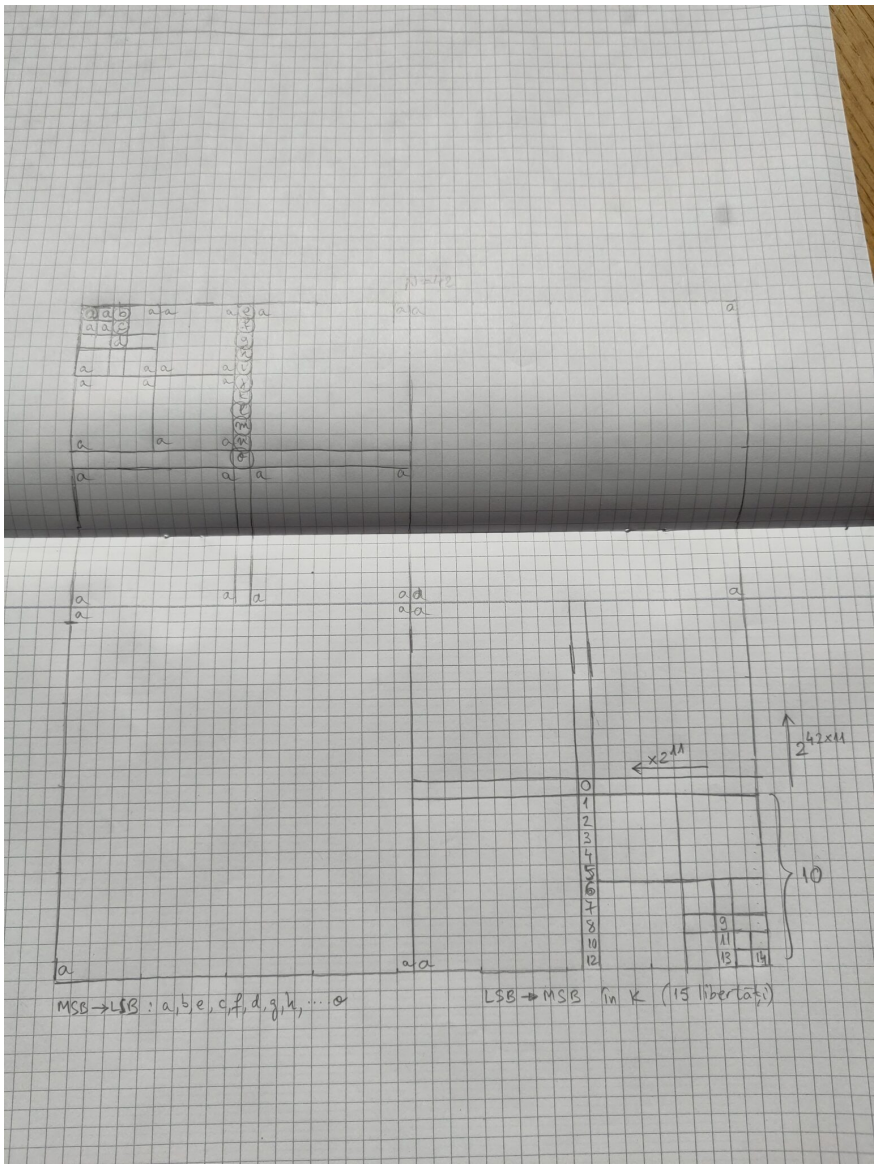


Figura 26.1: O imagine pentru $n = 42$.

Apropo de acest efort, menționez un citat favorit al meu:

> People said I did the impossible, but that's wrong: I merely did something so boring that nobody else had been willing to do it.
>
> Jacob Kaplan-Moss, creatorul Django

Este trist că o olimpiadă se poate numi „de informatică" în timp ce concurenților nu le este permis accesul în sală cu foi de matematică la discreție. Mai degrabă aș concura fără o mînă decît fără hîrtie de matematică.

Și acum, să vedem ce învățăm din figură.

Algoritmul teoretic

Dacă desenăm cîteva simboluri și urmărim unde se propagă ele prin hipersimetrie, observăm că:

- O matrice de dimensiune pară este formată din patru sferturi identice.

- O matrice impară este formată din patru sferturi identice și o cruce cu patru brațe identice.

Începem să ne dăm seama că sîntem liberi să alegem niște biți, iar restul vor fi impuși. De aici rezultă că numărul de matrice hipersimetrice va fi o putere a lui 2, acesta fiind și motivul pentru care îl primim pe k în baza 2. Fie $L(n)$ numărul de libertăți al unei matrici, adică numărul de biți independenți pe care îi putem alege. Atunci:

- $L(2k) = L(k)$ deoarece, după ce completăm un sfert, celelalte trei sînt impuse.
- $L(2k+1) = L(k) + k + 1$ deoarece, după ce completăm un sfert, mai putem da valori independente unia dintre brațele crucii (inclusiv centrul).

Astfel am ales pentru figura de mai sus $n = 42$, care ilustrează ambele cazuri. Există 15 libertăți, pe care le-am denumit alfabetic, și deci există 2^{15} matrice hipersimetrice binare. Șirul de la intrare va avea cel mult 15 biți. A k-a matrice (*0-based*) este cea în care scriem biții lui k în locul literelor, apoi îi propagăm prin hipersimetrie. Deoarece în enunț k pornește de la 1, trebuie să îl decrementăm înainte de a începe.

Problema ne cere să tratăm matricea rezultată ca pe un număr cu n^2 biți, citit de la stînga la dreapta și de sus în jos.

Detalii de implementare

Am preferat să operez în colțul din dreapta-jos al matricei (acolo unde sînt biții mai puțin semnificativi). Astfel restul matricei poate fi completat prin înmulțiri, nu prin împărțiri. Am reprezentat și acolo cele 15 libertăți, de data aceasta cu valori de la 0 la 14 corespunzătoare biților lui k (unde 0 = cel mai puțin semnificativ bit).

Se vede că biții sînt distribuiți pe $\mathcal{O}(\log n)$ coloane, unde fiecare coloană este cel mult jumătate din precedenta. Mai vedem că vom avea nevoie de coloane de biți, dar prioritatea lor crește pe linie, deci coloanele **nu** constau din subsecvențe contigue din k de la intrare, ci pe sărite.

Nu în ultimul rînd, suma înălțimilor coloanelor poate fi de sute de milioane. Cum $k \leq 1\,000\,000$, rezultă că o mare parte dintre biți vor fi implicit 0. Este important ca programul să nu-i proceseze decît pe cei de la intrare. Putem face asta dacă îl parcurgem pe k de la LSB spre MSB, iar libertățile de la centru spre periferie. Ne oprim cînd epuizăm biții lui k, deoarece restul ar fi zerouri, care nu afectează valoarea matricei.

Așadar, am izolat într-o clasă numită `bit_stream` codul care

- primește dimensiunea n;
- iterează prin biții lui k și îi distribuie pe coloanele potrivite;
- la cerere, returnează una cîte una coloanele în ordinea descrescătoare a înălțimii (astfel le va cere codul principal).

Recurența

Ne dorim ca funcția recursivă `compute_matrix(int s)` să genereze matricea de $s \times s$ din colțul dreapta-jos al matricei totale. Funcția returnează valoarea matricei generate. Atenție: matricea nu va ocupa $s \times s$ biți compacți, ci fiecare s biți vor ocupa porțiunea cel mai puțin semnificativă dintr-un grup de n biți. De exemplu, o matrice de 3×3 va ocupa 9 biți dintre primii $3n$, și anume biții $0, 1, 2, n, n+1, n+2, 2n, 2n+1, 2n+2$.

Procedăm astfel deoarece ne va fi mult mai ușor să translatăm un colț în celelalte trei colțuri. O matrice pară de mărime s trebuie copiată astfel:

- La stînga cu s coloane, prin înmulțirea valorii sale cu 2^s.
- În sus cu s linii, prin înmulțirea valorii sale cu 2^{sn}.
- În sus-stînga cu s linii și s coloane, prin înmulțirea cu 2^{sn+s}.

Așadar, dacă notăm cu V_s valoarea matricei de latură s, atunci

$$V_{2s} = V_s \cdot (1 + 2^s) \cdot (1 + 2^{sn})$$

Același raționament se aplică și pentru matricele impare, dar pentru acestea trebuie să inserăm și crucea centrală. Este acceptabil să facem efort $\mathcal{O}(1)$ per element, căci efortul total nu va depăși $\mathcal{O}(k)$ (din nou, dacă avem grijă să nu cerem decît primii k biți). Iar cu efort $\mathcal{O}(1)$ per element, completarea crucii este ușoară:

- Centrul trebuie amplasat într-un singur loc.
- Celelalte elemente trebuie amplasate de cîte patru ori.

Amplasarea unui bit la linia i și coloana j (numărînd din colțul de jos-dreapta) înseamnă pur și simplu creșterea valorii matricei cu 2^{ni+j}.

Puterile lui 2

O ultimă componentă necesară este posibilitatea de a calcula puteri mari ale lui 2. Din secțiunea de mai sus vedem că exponenții pot ajunge la n^2, adică 10^{18}. Știm să facem asta cu exponențiere binară, dar se poate și în $\mathcal{O}(1)$ (vezi capitolul TODO). Reiau aici ideea pe scurt.

În primul rînd, deoarece rezultatul este modulo $M = 1\,000\,000\,007$, putem aplica mica teoremă a lui Fermat, conform căreia $2^e \equiv 2^{e \bmod (M-1)} \pmod{M}$. Deci am redus exponenții la valori mai abordabile.

Apoi aplicăm descompunerea în radical. Calculăm doi vectori:

- $2^0, 2^1, 2^2, \ldots, 2^{32767}$ și
- $2^0, 2^{1 \cdot 32768}, 2^{2 \cdot 32768}, \ldots, 2^{32767 \cdot 32768}$

Astfel putem scrie $2^e = 2^{(e/32768) \cdot 32768} \cdot 2^{e \bmod 32768}$, pe care îl putem calcula în $\mathcal{O}(1)$ operații.

26.4.7 Problema Trim (Baraj ONI 2023)

enunț • sursă

Nu am citit editorialul, dar este posibil ca la această problemă Comisia să fi scăpat din vedere o soluție. Cel puțin pe Kilonova, la o limită de 0,9 secunde, există multe soluții în sub 0,1 secunde.

Pare natural să sortăm interogările, apoi să începem să emitem șirul de biți. Doar că nu îl vom emite unul cîte unul, ci în tranșe mai mari. La fiecare tranșă emisă, calculăm răspunsurile pentru interogările aflate în acea tranșă.

Fie $n = 13$. Atunci vom emite, în această ordine, numerele cu 1, 2, 3, ..., 13 biți de 1. Să considerăm numerele cu 7 biți de 1. Cel de-al doilea criteriu este valoarea numărului. La rîndul ei, valoarea numărului este dată în primul rînd de lungimea numărului: un număr cu 7 biți de 1, de lungime totală 10, va fi cuprins între 513 și 1023 și va fi întotdeauna mai mic decît un număr cu 7 biți de 1, de lungime totală 11, care va fi cuprins între 1025 și 2047.

Abordarea mea denumește **tranșă** o astfel de combinație de (număr de biți 1) × (lățimea numerelor). Algoritmul emite o tranșă de lățime w biți în $\mathcal{O}(1 + w \cdot \text{nr_interogări})$. Deoarece numărul de tranșe este $\mathcal{O}(n^2)$, va rezulta o complexitate totală de $\mathcal{O}(n^2 + qn)$.

Să considerăm tranșa de numere cu 7 biți 1 de lățime 10. Cum arată această tranșă?

1. Toate numerele au forma 1*bbbbbbbb*1, unde exact 5 dintre cei 8 biți centrali au valoarea 1.
2. Fiecare număr are o multiplicitate: el apare de 4 ori consecutiv, deoarece operăm pe numere care aveau inițial 13 biți. Numărul de 10 biți poate fi completat cu 3 zerouri la stînga sau la dreapta în 4 moduri distincte.
3. Există C_8^5 grupe de cîte 4 numere, ordonate conform ordinii lexicografice a combinărilor.
4. Lungimea totală a tranșei este $C_8^5 \cdot 4 \cdot 10$ biți.

Piesa finală din puzzle este să tipărim al p-lea bit din tranșă. Calculînd cîtul împărțirii lui p la 40 știm în zona cărei combinări se află el, iar calculînd restul împărțirii lui p la 10 știm al cîtelea bit din combinare îl dorim. Aceasta este o operație clasică de *combination unranking*, implementată în funcția `find_bit()`.

26.4.8 Problema Bubblesort (Lot 2010)

enunț • sursă

Aceasta este clar o problemă de unranking. Dintre toate permutările care au o anumită proprietate, trebuie să o tipărim pe a k-a. Dar detaliile sînt relativ dificile.

Detalierea ordinului unei permutări

Mai întîi, cum putem exprima ordinul unei permutări într-un mod util?

Să încercăm să calculăm numărul de treceri pe care îl face algoritmul bubblesort. Un mod vizual de a explica acest algoritm este: el „ia în mînă” un element x pe care îl mută spre dreapta, poziție

cu poziție, pînă cînd întîlnește un alt element $y > x$. Acolo îl depune pe x, îl ia pe y în mînă și continuă astfel pînă la poziția n.

De aici putem face următoarea observație. Dacă un element $v[i]$ are g elemente mai mari decît el la stînga lui, atunci **exact unul** dintre aceste g elemente va migra la dreapta lui i la fiecare trecere făcută de bubblesort. Într-adevăr,

1. Algoritmul va veni dinspre stînga cu cel mai mare element găsit pînă la poziția $i-1$ inclusiv, pe care îl va interschimba cu $v[i]$.
2. Pe $v[i]$ nu îl vor sări două sau mai multe elemente, deoarece pozițiile interschimbărilor cresc.
3. $v[i]$ însuși nu poate migra spre dreapta cît timp are elemente mai mari decît el la stînga sa.

Avînd în vedere că bubblesort mai face o ultimă trecere pentru a se asigura că vectorul este sortat, rezultă că numărul necesar de treceri, adică ordinul permutării, este

$$m = 1 + \max_{i=1}^{n} g[i]$$

unde $g[i]$ este numărul de elemente mai mari decît $v[i]$ aflate la stînga lui $v[i]$.

Numărarea permutărilor de un ordin dat

Cîte permutări de ordin m există? Dorim ca $\max_i g[i] = m - 1$. Din experiență, numărarea unor vectori în care maximul are **fix** o valoare cerută este complicată. În loc de aceasta, să numărăm permutările în care maximul are **cel mult** o valoare dată, ceea ce este mai simplu. Așadar, vom număra permutările de ordin cel mult m și, separat, pe cele de ordin $m - 1$. Prin diferență le putem număra pe cele de ordin exact m. Atunci valorile $g[i]$ trebuie să ia valori între 1 și un maxim dat ($m - 1$ sau $m - 2$).

Să judecăm această proprietate nu după poziții, ci după valori (pentru ordinul m). Valoarea 1 trebuie să apară undeva pe pozițiile $1 \dots m$. Dacă ar apărea pe poziția $m + 1$ sau mai mare, atunci ar avea cel puțin m valori mai mari decît ea la stînga, incorect. Spunem că valoarea 1 are m **libertăți**.

Valoarea 2 trebuie să apară pe pozițiile $1 \dots m + 1$. Astfel ne asigurăm că ea are cel mult m valori la stînga, dintre care cel mult $m - 1$ sînt mai mari ca valoare (cealaltă este valoarea 1). Similar, valoarea 3 trebuie să apară pe pozițiile $1 \dots m + 2$ etc. Ultima valoare cu m libertăți este $n - m + 1$, care trebuie să ocupe una dintre pozițiile $1 \dots n$. Restul valorilor pot fi așezate oriunde pe pozițiile rămase.

Numărul de permutări de ordin m este produsul acestor libertăți, adică $m^{n-m+1} \cdot (m-1)!$.

Calculul permutării de un rang dat

Acum procedăm așa cum știm: încercăm să așezăm pe prima poziție valoarea 1, apoi 2, apoi 3, și pentru fiecare valoare numărăm permutările de ordin m care încep cu ea. Cînd suma acestor numere devine cel puțin egală cu k, ultima valoare încercată este soluția pe poziția 1. Procedăm

similar pe pozițiile $2, 3, \ldots, n$.

Deci ajungem la întrebarea: cîte permutări de ordin exact m încep cu un prefix dat? Așa cum mă cunoașteți, 🙂 am încercat băbește un exemplu mai mare, ca să înțeleg cum evoluează libertățile elementelor.

Am căutat manual datele pentru permutarea `4 2 5 1 3 7 6`, care sînt $n = 7$, $m = 4$ și $k = 590$. Următorul tabel arată căutarea primei valori din soluție.

```
Permutări de ordin          ≤ 4           |           ≤ 3         | exact 4
Libertăți pentru: 1 2 3 4 5 6 7           | 1 2 3 4 5 6 7         |
Prefix []          4×4×4×4×3×2×1 = 1536   | 3×3×3×3×3×2×1 = 486   | 1050
                                                                    (ca fapt divers)

Prefix [1]         1×4×4×4×3×2×1 =  384   | 1×3×3×3×3×2×1 = 162   |  222
Prefix [2]         3×1×4×4×3×2×1 =  288   | 2×1×3×3×3×2×1 = 108   |  180
Prefix [3]         3×3×1×4×3×2×1 =  216   | 2×2×1×3×3×2×1 =  72   |  144
Prefix [4]         3×3×3×1×3×2×1 =  162   | 2×2×2×1×3×2×1 =  48   |  114
```

Să explicăm linia pentru prefixul [1]. Valoarea 1 nu mai are decît o libertate, deoarece i-am impus să stea pe poziția 1. Dar valoarea 2 continuă să aibă 4 libertăți (pozițiile 2-5) etc. Pe linia pentru prefixul [2] observăm că 1 își recapătă 3 din cele 4 libertăți: el poate sta pe pozițiile 2-4, căci poziția 1 este ocupată. Valoarea 2 nu mai are decît o libertate, iar de la valoarea 3 încolo situația este normală etc.

Ne oprim la valoarea 4, întrucît $222+180+144+114 > 590$. Rezultă că am identificat $222+180+144 = 546$ permutări care încep cu valorile 1, 2 și 3 și trebuie să o găsim pe a $590 - 546 = 44$-a care începe cu 4.

Includ aici și calculele pentru poziția a doua. Restul este similar.

```
Prefix [4 1]       1×3×3×1×3×2×1 =   54   | 1×2×2×1×3×2×1 =  24   |   30
Prefix [4 2]       2×1×3×1×3×2×1 =   36   | 1×1×2×1×3×2×1 =  12   |   24
```

Aici ne oprim deoarece $30 + 24 > 44$. Rezultă că al doilea element din soluție este 2 și trebuie să găsim a $44 - 30 = 14$-a permutare care începe cu 4 2. *Rinse and repeat.*

Programul cere și numere mari, ceea ce poate nu este un lucru rău, este o ocazie să le exersați. Sursa devine de 10 ori mai rapidă dacă folosim o bază mare, să spunem 10.000, în loc de 10 pentru numerele mari.

26.4.9 Problema Valentine's Day (infO(1) Cup 2023)

enunț • sursă

Problema este relativ pe față una de *unranking*, dar optimizările sînt interesante.

Formularea ca problemă de *unranking*

Să abordăm problema clasic. Pornim cu toate valorile în mînă, urmînd să le depunem în $p[1], p[2]$ etc. Ne întrebăm: după ce completăm un prefix, ce valoare alegem dintre cele din mînă? Din nou, soluția clasică este să iterăm printre valorile din mînă în ordine crescătoare. Pentru fiecare valoare x ne punem întrebarea: dacă depunem valoarea x, putem completa restul permutării în cel puțin k moduri? Dacă da, chiar extindem prefixul cu x. Altfel, scădem numărul de moduri din k și încercăm următoarea valoare din mînă.

Așadar, am redus problema de unranking la o cărămidă mai simplă: cîte permutări valide există cu setul de valori din mînă și setul de constrîngeri din sufixul rămas? Dacă n-am avea constrîngeri, atunci răspunsul ar fi pur și simplu $h!$, unde h este numărul de valori pe care le ținem în mînă.

Pasul specific problemei

Partea aceasta am rezolvat-o pur și simplu rulînd exemplul 4 pe hîrtie.

Soluția teoretică (fără să ne preocupăm de eficiență) este să procesăm valorile din mînă în ordine crescătoare. Valoarea 1 o putem depune peste orice constrîngere egală cu 1. Valoarea 2 o putem depune peste orice constrîngere mai mică sau egală cu 2, minus unu căci una dintre acele poziții a fost ocupată de valoarea 1.

În general, numărul de posibilități pentru valoarea x este $C[x] = A[x] - B[x]$, unde $A[x]$ este numărul de constrîngeri mai mici sau egale cu x, iar $B[X]$ este numărul de valori din mînă mai mici decît x. Numărul de permutări posibile este $P = \prod_x C[x]$.

Desigur, dorim să calculăm P mai repede decît $\mathcal{O}(n)$. Să vedem cum îl putem calcula incremental pe măsură ce depunem valori din mînă. Să pornim de la exemplul 4 din enunț și să spunem că am așezat primele 6 valori (6 12 1 2 13 4) peste primele 6 constrîngeri (6 12 1 2 13 3). Să spunem că la acest moment avem calculat vectorul C și produsul aferent, P.

```
constr = 1  1  1  4  5  6  6  6  7  7  9 11 13 18
mînă   = 3  5  7  8  9 10 11 14 15 16 17 18 19 20
C      = 3  4  8  7  7  6  6  6  5  4  3  3  2  1,
P = 365.783.040, k = 331.301.270
```

Exemplu: valoarea 8 poate fi așezată peste primele 10 constrîngeri, dar 3 dintre acestea vor fi fost ocupate de valorile 3, 5 și 7. Deci $C[8] = 7$.

Următoarea constrîngere pe care trebuie să o acoperim este $a[7] = 13$. Deci încercăm pe rînd valorile din mînă începînd cu 14. Dacă eliminăm constrîngerea 13 și valoarea 14, obținem vectorul:

```
constr = 1  1  1  4  5  6  6  6  7  7  9 11 -- 18
mînă   = 3  5  7  8  9 10 11 -- 15 16 17 18 19 20
C      = 3  4  8  7  7  6  6 --  5  4  3  3  2  1,
P = 60.963.840
```

Observăm că restul valorilor din C nu s-au modificat. Într-adevăr, valorile mai mici decît 14 nu și-au schimbat posibilitățile de amplasare. Valorile mai mari pierd o variantă de amplasare pentru că constrîngerea 13 a dispărut, dar cîștigă una pentru că și valoarea 14 (care ar revendica o poziție) a dispărut. Deci putem simula așezarea primei valori din mînă în $\mathcal{O}(1)$.

Cum tratăm actualizarea? Întrucît $P < k$, trebuie să încercăm și valoarea 15. În vectorii inițiali ștergem constrîngerea 13 și valoarea 15:

```
constr = 1  1  1  4  5  6  6  6  7  7  9 11 -- 18
mînă   = 3  5  7  8  9 10 11 14 -- 16 17 18 19 20
C      = 3  4  8  7  7  6  6  5 --  4  3  3  2  1,
P = 60.963.840
```

Observăm ceva în plus: $C[14] = 5$. El a scăzut cu 1 deoarece constrîngerea 13 a dispărut. Celelalte valori nu se modifică, pentru același raționament ca și mai înainte. Deducem regula generală:

- La depunerea unei valori x care satisface constrîngerea $y \leq x$, $C[z]$ scade cu 1 pentru valorile z cu $y \leq z < x$. În rest, valorile din C nu se modifică.

Așadar, putem actualiza eficient C și P pe măsură ce căutăm valorile de amplasat.

Calculul produsului

Există un mare „dar”. Produsul P al valorilor din C poate depăși reprezentarea. Din fericire, ne interesează doar să știm dacă el este mai mic decît k (caz în care îl scădem din k și încercăm să depunem următoarea valoare) sau mai mare sau egal cu k (caz în care depunem valoarea testată curentă).

O abordare este să menținem efectiv C ca `std::multiset`, cu excepția valorilor de 1, care sînt elemente neutre la produs. Cînd avem nevoie de produs, îl calculăm naiv. Dacă depășește k, returnăm $+\infty$ (sau chiar k). Calculul este $\mathcal{O}(\log k)$, căci în cel mult $\log k$ factori vom depăși produsul k. Putem chiar să testăm mărimea multisetului și să returnăm ∞ dacă el conține mai mult de $\log k$ factori.

Optimizarea produsului

Ca discuție secundară, iată o reprezentare fără STL pentru mulțimea de factori. Ea obține $\mathcal{O}(1)$ pentru inserarea și ștergerea unui factor, dar calculul produsului rămîne $\mathcal{O}(\log k)$. Cîștigul ca timp este cam de 15

Ce dorim de fapt? Dorim să știm factorii și frecvențele lor. Asta putem face cu un vector. Dar dorim și să putem itera prin factori ca să-i înmulțim. Pentru asta, stocăm trei vectori:

- Frecvențele factorilor.
- O listă cu factorii distincți.
- Poziția fiecărui factor în listă (dacă există).

Dacă codăm îngrijit, singurele modificări apar doar în reprezentarea internă a mulțimii de factori.

Reprezentarea mîinii

Ca principiu de inginerie, nu este bine să trîntiți prima structură care vă vine în minte. Corect este să porniți de la operațiile pe care trebuie să le suporte structura. Pentru reprezentarea mîinii, dorim:

1. Să o inițializăm cu toate valorile $1, 2, \ldots n$.
2. Să găsim cea mai mică valoare mai mare sau egală cu constrîngerea curentă.
3. Să iterăm prin valorile următoare, în ordine crescătoare.
4. Să ștergem valoarea aleasă în final.

Sigur, ne putem repezi la `std::set`, care oferă timp $\mathcal{O}(\log n)$. Dar, cu puțin efort, putem folosi un *disjoint set forest*, care oferă timp $\mathcal{O}(\log^* n)$ și reduce timpul de rulare de circa 4 ori.

Observația-cheie este că, la depunerea și la ștergerea unei valori x, trebuie să unificăm x cu $x+1$. Din nou, modificările afectează strict la structura modificată.

Un sfat despre abordarea problemelor

Dacă obțineți 100p cu structuri prefabricate și cod scurt, *kudos*! Dar, dacă primiți TLE, este vital să aveți ochiul format ca să știți ce puteți optimiza. Acest proces de gîndire este relativ general: Oare există o altă structură care rezolvă aceleași operații asimptotic mai rapid?

Odată ce găsiți structura mai eficientă, vă va fi mult mai ușor să o codați dacă separați structura de codul care o apelează. Ideal, chiar și interfața (funcțiile pe care structura le expune) va rămîne nemodificată. Altfel, dacă amestecați reprezentarea internă a structurii cu codul care apelează structura, orice adaptare devine mai grea.

Capitolul 27

Rezolvarea recurențelor liniare prin exponențiere rapidă

Să pornim de la codul binecunoscut pentru calculul lui a^n în $\mathcal{O}(\log n)$:

```
int mod_pow(int a, int n) {
  long long result = 1;

  while (n) {
    if (n & 1) {
      result = result * a % MOD;
    }
    a = (long long)a * a % MOD;
    n >>= 1;
  }

  return result;
}
```

Algoritmul funcționează rapid pentru că accelerează succesiuni lungi de înmulțiri simple, înlocuindu-le cu ridicări la pătrat. Putem folosi aceeași rețetă pentru a ridica la putere orice obiecte care pot fi înmulțite și a căror înmulțire este asociativă, în particular matricele pătrate și permutările. Trebuie doar ca, în loc de o valoare întreagă, `a` să fie o matrice, iar în loc de operatorul `*` pentru înmulțire de întregi să implementăm înmulțirea de matrice.

Vom vedea cum exponențierea de matrice ne poate ajuta în probleme de programare dinamică în care trebuie să împingem limita pentru n pînă la valori foarte mari.

27.1 Șirul lui Fibonacci. Matrice generatoare

Să considerăm un exemplu celebru: șirul lui Fibonacci, definit ca

$$\begin{cases} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \quad \text{pentru } n \geq 2 \end{cases}$$

Dacă n este foarte mare, de ordinul a 10^{18}, atunci calculul liniar nu ne mai ajută. Vom încerca să exprimăm recurența cu următoarele elemente:

- Un vector-coloană, V_n, pentru pasul curent al formulei. Acesta trebuie să includă cel puțin termenul de calculat, F_n.
- Un vector-coloană, V_{n-1}, pentru pasul anterior al formulei. Acesta trebuie să includă toți termenii de care depine F_n, în cazul nostru F_{n-1} și F_{n-2}, și posibil și alte cantități.
- O matrice A, numită **matrice generatoare**, cu proprietatea că

$$V_n = A \cdot V_{n-1}$$

Pentru șirul lui Fibonaci, obținem relația:

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}$$

Remarcăm că matricea este constantă (independentă de n). De aceea, putem scrie:

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

Iar pe A^{n-1} îl putem calcula cu exponențiere rapidă.

Arta pentru acest gen de probleme este tocmai să găsim forma vectorului-coloană și matricea generatoare. Observăm că vectorul-coloană din dreapta trebuie să conțină toate cantitățile necesare pentru calculul lui F_n, ceea ce dictează și structura vectorului din stînga (cei doi sînt identici, doar indicii diferă cu 1).

Să vedem două exemple mai complicate.

27.2 Alte recurențe liniare

Să considerăm un exemplu în care recurența implică 4 termeni anteriori. Vom folosi o matrice de 4×4. Termenii F_0, F_1, F_2 și F_3 sînt dați, iar formula de recurență este:

$$F_n = aF_{n-1} + bF_{n-2} + cF_{n-3} + dF_{n-4}$$

Atunci recurența exprimată matricial este:

$$\begin{pmatrix} F_n \\ F_{n-1} \\ F_{n-2} \\ F_{n-3} \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \\ F_{n-3} \\ F_{n-4} \end{pmatrix}$$

27.3 Recurențe liniare cu termen constant

Să considerăm acum recurența:

$$F_n = aF_{n-1} + bF_{n-2} + cF_{n-3} + d$$

Secretul aici este să nu încercăm să construim vectori-coloană doar de 3 elemente. Dat fiind că există și un termen liber, trebuie să punem și o valoare scalară pe vectorii-coloană. Avem două variante:

$$\begin{pmatrix} F_n \\ F_{n-1} \\ F_{n-2} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \\ F_{n-3} \\ 1 \end{pmatrix}$$

sau

$$\begin{pmatrix} F_n \\ F_{n-1} \\ F_{n-2} \\ d \end{pmatrix} = \begin{pmatrix} a & b & c & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \\ F_{n-3} \\ d \end{pmatrix}$$

27.4 Probleme

27.4.1 Problema Al k-lea termen Fibonacci (Infoarena)

enunț • sursă

Aceasta este exact problema educațională discutată la începutul capitolului. Am scris manual cele patru valori din produsul a două matrice de 2×2.

27.4.2 Problema Iepuri (Infoarena)

enunț • sursă

Și această problemă necesită o recurență relativ simplă. Matricea fiind de 3×3, am scris înmulțirea cu 3 `for`-uri.

27.4.3 Problema Recurenta2 (Infoarena)

enunț • sursă

Recurența începe să se complice! Să procedăm în etape.

Să presupunem, pentru început, că problema ne-ar cere doar să aflăm X_n. Acestă recurență știm să o scriem deja:

$$\begin{pmatrix} X_n \\ X_{n-1} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_{n-1} \\ X_{n-2} \\ 1 \end{pmatrix}$$

Acum să atacăm problema sumelor. Să presupunem că problema ne-ar cere să aflăm suma simplă $S_n = \sum_{k=1}^{n} X_k$. Reținem că matricea de recurență trebuie să fie **constantă**. Nu putem adăuga valori din șirul X acolo, ci doar constante exprimabile în funcție de a, b și c. De asemenea, matricea și vectorii trebuie să fie **finiți**. Nu putem include toți termenii de la X_1 la X_n doar fiindcă S_n îi cere. Rezultă că trebuie să „cărăm după noi" și suma S_n și să o exprimăm recurent:

$$S_n = S_{n-1} + X_n = S_{n-1} + (aX_{n-1} + bX_{n-2} + c)$$

Observăm că am expandat valoarea lui X_n. Aceasta deoarece în vectorul-coloană din dreapta (V_{n-1}) noi voi avea acces doar la cantități de rangul $n-1$ sau mai mic. Acum nu este greu să construim matricea de recurență:

$$\begin{pmatrix} X_n \\ X_{n-1} \\ S_n \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & 0 & c \\ 1 & 0 & 0 & 0 \\ a & b & 1 & c \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_{n-1} \\ X_{n-2} \\ S_{n-1} \\ 1 \end{pmatrix}$$

Acum putem ataca și cerința originală a problemei: $S_n = \sum_{k=1}^{n} k \cdot X_k$. Din nou, nu avem voie să punem în matricea de recurență cantități variabile, cum ar fi k. Să vedem ce adăugăm la V_{n-1} și cum obținem o formă închisă, minimă și suficientă pentru V_n.

Știm că $S_n = S_{n-1} + nX_n$. Dar noi trebuie să exprimăm termenii de rang n în funcție de ranguri mai mici, deci îl vom scrie pe nX_n ca:

$$\begin{aligned} nX_n &= n \cdot (aX_{n-1} + bX_{n-2} + c) \\ &= a(n-1)X_{n-1} + aX_{n-1} + b(n-2)X_{n-2} + 2bX_{n-2} + c(n-1) + c \end{aligned}$$

De aici rezultă V_{n-1} trebuie să includă în plus cantitățile $(n-1)$, $(n-1)X_{n-1}$ și $(n-2)X_{n-2}$, deci V_n trebuie să includă cantitățile n, nX_n și $(n-1)X_{n-1}$. Recurența completă este un pic (doar un

pic) fioroasă. Ea cere doar meticulozitate, dar nu este dificil de dedus.

$$\begin{pmatrix} X_n \\ X_{n-1} \\ nX_n \\ (n-1)X_{n-1} \\ S_n \\ n \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & 0 & 0 & 0 & 0 & c \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ a & 2b & a & b & 0 & c & c \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ a & 2b & a & b & 1 & c & c \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_{n-1} \\ X_{n-2} \\ (n-1)X_{n-1} \\ (n-2)X_{n-2} \\ S_{n-1} \\ n-1 \\ 1 \end{pmatrix}$$

Codul trebuie scris cu atenție și eventual comentat pe alocuri. Altfel devine *a malignant morass of code* (citat din marii clasici).

27.4.4 Problema Graph Paths I (CSES)

enunț • sursă

Această problemă amintește de algoritmul Floyd-Warshall:

```
for (int k = 0; k < n; k++) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
    }
  }
}
```

Cu costuri pe muchii și cu funcția min, algoritmul calculează distanțele minime. La pasul k, $A_{i,j}$ reține distanța minimă între nodurile i și j care trec prin nodurile intermediare $1 \dots k$.

Pentru problema de față, să inițializăm $A_{i,j}$ cu numărul de arce orientate (i, j) (0 sau 1). Atunci cine este A^2? Prin definiție,

$$A^2_{i,j} = \sum_{k=1}^{n} A_{i,k} \cdot A_{k,j}$$

Dar acesta este exact numărul de drumuri de lungime 2 care pornesc din i, trec prin orice alt nod k și se termină în j! Printr-un raționament similar, A^k reține numerele de căi de lungime k între orice două noduri. Răspunsul la problemă este $A^k_{1,n}$.

27.4.5 Problema Hprob (Infoarena)

enunț • sursă

Această problemă necesită cîteva cunoștințe de probabilități.

În esență, trebuie să construim un graf cu $4! = 24$ de stări (noduri). O stare reprezintă o ordine a obiectelor, iar o muchie reprezintă o rearanjare făcută de unul dintre clienți, cu una dintre probabilitățile p_1, p_2, p_3 sau p_4 (sau 0 dacă permutarea nu este una dintre cele 4 posibile). Dacă reprezentăm graful folosind o matrice de adiacență A, atunci pe fiecare linie vom regăsi cîte o dată probabilitățile p_1, p_2, p_3 și p_4, precum și 20 de valori 0.

Am putea hardcoda matricea A imediat după citirea datelor, dar procesul este laborios și riscant. Mai sigur este să o construim programatic. De aceea, vom avea nevoie să facem *ranking* și *unranking* la permutările de 4 elemente. Această parte o putem hardcoda, fiind suficient de simplă.

Acum, să analizăm ce semnifică produsul de matrice:

$$A^2_{i,j} = \sum_{k=1}^{n} A_{i,k} \cdot A_{k,j}$$

El arată exact probabilitatea de a ajunge din starea i în starea j după doi clienți, trecînd prin orice stare intermediară k. Adunarea este exact operația dorită, căci evenimentele sînt independente: primul client face o tranziție și numai una, nu poate urma mai multe muchii. De aceea, probabilitățile de forma $A_{i,k} \cdot A_{k,j}$ se adună.

Așadar, răspunsul la problemă este $A^n_{0,0}$.

Pentru o problemă similară, vezi Graph Paths II (CSES). Ea necesită tot un fel de înmulțire de matrice, dar cu funcția *min*.

27.4.6 Problema Chef and Riffles (CodeChef)

enunț • sursă

Uneltele pe care le-am discutat pentru înmulțirea rapidă a matricelor se aplică și pentru permutări. Problema este directă: ne dă o permutare P și ne cere să o aplicăm de k ori permutării identice.

Tot ce trebuie să știm este că compunerea de permutări este asociativă (atenție, nu și comutativă!). Iată un exemplu școlăresc pentru $n = 10$. Îl includ doar ca să nu uitați că unele noțiuni vi le puteți demonstra singuri în cîteva minute. Șansa să obținem o egalitate dacă compunerea de permutări nu ar fi asociativă este practic nulă.

$$P = (2\ 6\ 3\ 9\ 8\ 5\ 1\ 4\ 10\ 7)$$
$$Q = (3\ 7\ 6\ 2\ 10\ 1\ 9\ 4\ 5\ 8)$$
$$R = (6\ 7\ 9\ 1\ 5\ 4\ 3\ 8\ 10\ 2)$$
$$PQ = (7\ 1\ 6\ 5\ 4\ 10\ 3\ 2\ 8\ 9)$$
$$QR = (9\ 3\ 4\ 7\ 2\ 6\ 10\ 1\ 5\ 8)$$
$$(PQ)R = (3\ 6\ 4\ 5\ 1\ 2\ 9\ 7\ 8\ 10)$$
$$P(QR) = (3\ 6\ 4\ 5\ 1\ 2\ 9\ 7\ 8\ 10)$$

Putem demonstra și formal asociativitatea. Compunerea de permutări este definită ca

$$(P \circ Q)(i) = Q(P(i))$$

Așadar, avem că:

$$((P \circ Q) \circ R)(i) = R((P \circ Q)(i)) = R(Q(P(i)))$$
$$(P \circ (Q \circ R))(i) = (Q \circ R)(P(i)) = R(Q(P(i)))$$

Odată demonstrată asociativitatea, putem calcula răspunsul ca P^k prin exponențiere rapidă, cu complexitatea $\mathcal{O}(n \log k)$.

Nu face obiectul acestui capitol, dar, pentru completitudine, enunțăm și o soluție în $\mathcal{O}(n)$, bazată pe descompunerea în cicluri. Să considerăm următorul tabel cu primele 31 de puteri ale unei permutări (dacă nu am spus-o suficient, îmi plac exemplele mari, căci din ele putem face deducții mult mai pertinente decît din exemplele mici).

```
P^1  = [  5  1  8  9 11 10  4  2  6  7  3] = (1 5 11 3 8 2)(4 9 6 10 7)
P^2  = [ 11  5  2  6  3  7  9  1 10  4  8] = (1 11 8)(2 5 3)(4 6 7 9 10)
P^3  = [  3 11  1 10  8  4  6  5  7  9  2] = (1 3)(2 11)(4 10 9 7 6)(5 8)
P^4  = [  8  3  5  7  2  9 10 11  4  6  1] = (1 8 11)(2 3 5)(4 7 10 6 9)
P^5  = [  2  8 11  4  1  6  7  3  9 10  5] = (1 2 8 3 11 5)(4)(6)(7)(9)(10)
P^6  = [  1  2  3  9  5 10  4  8  6  7 11] = (1)(2)(3)(4 9 6 10 7)(5)(8)(11)
P^7  = [  5  1  8  6 11  7  9  2 10  4  3] = (1 5 11 3 8 2)(4 6 7 9 10)
P^8  = [ 11  5  2 10  3  4  6  1  7  9  8] = (1 11 8)(2 5 3)(4 10 9 7 6)
P^9  = [  3 11  1  7  8  9 10  5  4  6  2] = (1 3)(2 11)(4 7 10 6 9)(5 8)
P^10 = [  8  3  5  4  2  6  7 11  9 10  1] = (1 8 11)(2 3 5)(4)(6)(7)(9)(10)
P^11 = [  2  8 11  9  1 10  4  3  6  7  5] = (1 2 8 3 11 5)(4 9 6 10 7)
P^12 = [  1  2  3  6  5  7  9  8 10  4 11] = (1)(2)(3)(4 6 7 9 10)(5)(8)(11)
```

```
P^13 = [  5  1  8 10 11  4  6  2  7  9  3] = (1 5 11 3 8 2)(4 10 9 7 6)
P^14 = [ 11  5  2  7  3  9 10  1  4  6  8] = (1 11 8)(2 5 3)(4 7 10 6 9)
P^15 = [  3 11  1  4  8  6  7  5  9 10  2] = (1 3)(2 11)(4)(5 8)(6)(7)(9)(10)
P^16 = [  8  3  5  9  2 10  4 11  6  7  1] = (1 8 11)(2 3 5)(4 9 6 10 7)
P^17 = [  2  8 11  6  1  7  9  3 10  4  5] = (1 2 8 3 11 5)(4 6 7 9 10)
P^18 = [  1  2  3 10  5  4  6  8  7  9 11] = (1)(2)(3)(4 10 9 7 6)(5)(8)(11)
P^19 = [  5  1  8  7 11  9 10  2  4  6  3] = (1 5 11 3 8 2)(4 7 10 6 9)
P^20 = [ 11  5  2  4  3  6  7  1  9 10  8] = (1 11 8)(2 5 3)(4)(6)(7)(9)(10)
P^21 = [  3 11  1  9  8 10  4  5  6  7  2] = (1 3)(2 11)(4 9 6 10 7)(5 8)
P^22 = [  8  3  5  6  2  7  9 11 10  4  1] = (1 8 11)(2 3 5)(4 6 7 9 10)
P^23 = [  2  8 11 10  1  4  6  3  7  9  5] = (1 2 8 3 11 5)(4 10 9 7 6)
P^24 = [  1  2  3  7  5  9 10  8  4  6 11] = (1)(2)(3)(4 7 10 6 9)(5)(8)(11)
P^25 = [  5  1  8  4 11  6  7  2  9 10  3] = (1 5 11 3 8 2)(4)(6)(7)(9)(10)
P^26 = [ 11  5  2  9  3 10  4  1  6  7  8] = (1 11 8)(2 5 3)(4 9 6 10 7)
P^27 = [  3 11  1  6  8  7  9  5 10  4  2] = (1 3)(2 11)(4 6 7 9 10)(5 8)
P^28 = [  8  3  5 10  2  4  6 11  7  9  1] = (1 8 11)(2 3 5)(4 10 9 7 6)
P^29 = [  2  8 11  7  1  9 10  3  4  6  5] = (1 2 8 3 11 5)(4 7 10 6 9)
P^30 = [  1  2  3  4  5  6  7  8  9 10 11] = (1)(2)(3)(4)(5)(6)(7)(8)(9)(10)(11)
P^31 = [  5  1  8  9 11 10  4  2  6  7  3] = (1 5 11 3 8 2)(4 9 6 10 7)
```

Observăm că puterea a k-a a permutării inițiale parcurge fiecare ciclu din k în k. De exemplu,

- Puterea a 2-a sparge ciclul de lungime 6 în două cicluri de lungime 3.
- Puterea a 3-a îl sparge în 3 cicluri de lungime 2.
- Puterile 6, 12, 18 etc. pun fiecare element de pe acel ciclu pe poziția sa proprie (p[i] = i).
- Puterea 30 este permutarea identică. Acest lucru nu este întîmplător, căci cmmmc$(6, 5) = 30$. Spunem că ordinul permutării p este 30.
- Puterea 31 revine la permutarea inițială.

27.4.7 Problema Zid (ONI 2024, clasele 11-12)

enunț • sursă

Pare natural să colectăm întîi informații despre turnuri (ziduri de lățime 1), apoi despre ziduri de lățimi arbitrare.

Informații despre turnuri

Fie $T_{h,e}$ numărul de turnuri de înălțime h în care suma pieselor pare este e[1]. Deducem formula de recurență în funcție de piesa din vîrful turnului, de înălțime l:

$$T_{h,e} = \sum_{l=1,3,5,\ldots} T_{h-l,e} + \sum_{l=2,4,6,\ldots} T_{h-l,e-l}$$

[1]Pentru această problemă am ales inițiale de la cuvintele englezești h = *height*, w = *width*, e = *even*, l = *last*.

Codul de mai jos decurge ușor, doar că el este $\mathcal{O}(n^3)$, prea lent:

```
void compute_width_1_tower() {
  t[0][0] = 1;
  for (int h = 1; h <= n; h++) {
    for (int even = 0; even <= h; even += 2) {
      long long sum = 0;
      for (int last = 1; last <= h - even; last += 2) {
        sum += t[h - last][even];
      }
      for (int last = 2; last <= even; last += 2) {
        sum += t[h - last][even - last];
      }
      t[h][even] = sum % MOD;
    }
  }
}
```

Putem să reducem timpul de calcul al lui T la $\mathcal{O}(1)$ per element. Să observăm că prima sumă din formula de recurență este o sumă din 2 în 2 pe coloana e, iar a doua sumă este o sumă din 2 în 2 pe diagonale. Putem menține aceste sume pe măsură ce baleiem matricea T, așa cum procedează sursa finală. Astfel construim matricea T în $\mathcal{O}(n^2)$.

De acum încolo folosim doar informații despre linia T_n, căci zidurile se compun doar din turnuri de înălțime exact n.

Informații despre ziduri

Acum fie $D_{w,e}$ numărul de ziduri de lățime w (și înălțime n) în care suma pieselor pare este e. Formula de recurență pornește de la suma pieselor pare din ultimul turn:

$$D_{w,e} = \sum_{l=0,2,4,\ldots} D_{w-1,e-l}$$

Și această formulă este ușor de implementat, dar complexitatea este $\mathcal{O}(mnk)$, deoarece avem $m \cdot k$ valori, iar l poate fi cel mult $n/2$. Putem reduce substanțial memoria dacă observăm că matricele T și D folosesc doar liniile pare (e este întotdeauna par). Așadar, redefinim $T_{n,e}$ ca numărul de turnuri de înălțime n în care suma pieselor pare este $2e$. Similar, redefinim $D_{w,e}$ ca numărul de ziduri de lățime w în care suma pieselor pare este $2e$.

Chiar și așa, încă este nevoie de o îmbunătățire asimptotică.

Găsirea unei matrici de tranziție

Să ne gîndim la D_w ca la un vector-coloană constînd din $D_{w,0}, D_{w,1}, \ldots, D_{w,k/2}$. Atunci putem exprima relația dintre D_w și D_{w-1} prin înmulțirea cu o matrice.

$$\begin{pmatrix} D_{w,0} \\ D_{w,1} \\ D_{w,2} \\ \dots \\ D_{w,k/2-1} \\ D_{w,k/2} \end{pmatrix} = \begin{pmatrix} T_{n,0} & 0 & 0 & \dots & 0 & 0 \\ T_{n,1} & T_{n,0} & 0 & \dots & 0 & 0 \\ T_{n,2} & T_{n,1} & T_{n,0} & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ T_{n,k/2-1} & T_{n,k/2-2} & T_{n,k/2-3} & \dots & T_{n,0} & 0 \\ T_{n,k/2} & T_{n,k/2-1} & T_{n,k/2-2} & \dots & T_{n,1} & T_{n,0} \end{pmatrix} \begin{pmatrix} D_{w-1,0} \\ D_{w-1,1} \\ D_{w-1,2} \\ \dots \\ D_{w-1,k/2-1} \\ D_{w-1,k/2} \end{pmatrix}$$

Fie A matricea din mijloc. Remarcăm că A este independentă de w. Rezultă că putem începe cu $D_0 = (1, 0, 0, \dots, 0)$ și putem calcula

$$D_w = A^m \cdot D_0$$

Apoi, răspunsul care ne trebuie este $D_{m,k/2}$, care datorită înmulțirii cu D_0 (care are 1 doar pe prima poziție) va coincide cu $A^m_{k/2,0}$ (colțul din stînga-jos al lui A^m).

Îl putem calcula pe A^m prin exponențiere rapidă, dar algoritmul va fi prea lent. Vom face $\mathcal{O}(\log m)$ înmulțiri de matrice, iar fiecare înmulțire este $\mathcal{O}(k^3)$.

Accelerarea înmulțirii de matrice

Am învățat ultima parte a algoritmului citind sursa lui Rareș Neculau.

Putem face cîteva deducții facile despre A^i prin inducție:

1. A^i este 0 deasupra diagonalei principale.
2. A^i este simetrică față de diagonala secundară.
3. În particular, prima coloană și ultima linie sînt egale (dar răsturnate).
4. Elementele de pe prima coloană și de pe ultima linie depind doar de valorile lui T_n și de prima coloană și de ultima linie ale lui A^{i-1}.

De aceea, este suficient să calculăm ultima linie (și implicit prima coloană) din A^i, ceea ce durează doar $O(k^2)$ per iterație. Complexitatea totală este $\mathcal{O}(n^2 + k^2 \log m)$.

Pentru o economie enormă de memorie, putem reține pe durata construcției doar ultima linie din T și din sumele parțiale pe coloane și pe diagonale. Astfel, sursa finală nu folosește deloc matrice, ceea ce reduce memoria necesară de la 120 MB la sub 1 MB.

27.4.8 Problema Stairs and Lines (Codeforces)

enunț • sursă

Problema este dură, dar apare în capitolul despre exponențieri de matrice... Cu această informație, ea devine abordabilă.

Problema aduce a programare dinamică, deci am vrea să găsim o recurență, probabil pe coloane. Ne folosim de faptul că înălțimea h este mică, ceea ce ne dă de înțeles că ne permitem o soluție

exponențială în h. Ce-ar fi să iterăm prin toate măștile pe h biți? Aceasta ar însemna toate configurațiile de pereți pe o coloană verticală. Pentru două măști date, l pe o coloană și r pe coloana următoare, putem ușor să numărăm configurațiile valide de pereți orizontali. O configurație este invalidă dacă, la orice înălțime i, măștile l și r au valoarea 1 (pereți verticali), iar configurația are pereți orizontali la înălțimile $i-1$ și i. Aceasta necesită doar trei `for`-uri și niște `and`-uri pe biți, în esență:

```
void make_transition_matrix(int h) {
  for (int left = 0; left < SIZE; left++) {
    for (int right = 0; right < SIZE; right++) {
      t[left][right] = 0;
      // Forțează pereți orizontali la înălțimile 0 și h, deci biții 0 și h
      // sînt setați.
      for (int horiz = 1 + (1 << h); horiz < (1 << (h + 1)); horiz += 2) {
        if ((left & right & horiz & (horiz >> 1)) == 0) {
          t[left][right]++;
        }
      }
    }
  }
}
```

Așadar, cu efort $\mathcal{O}((2^h)^3)$, deci circa 2 000 000 de operații elementare, putem precalcula o matrice de tranziție, G, care ne arată în cîte moduri putem trece de la o mască l de pereți verticali la o mască r.

De aici obținem o soluție ineficientă. Să definim $D(c, m)$ ca numărul de moduri de a completa primele c coloane, rămînînd cu o mască m de pereți verticali. Se observă că masca m „șterge" istoria: coloanele $c+1$ și următoarele nu mai depind de nimic din trecut, doar de m. De aceea, programarea dinamică funcționează, iar formula de recurență pentru $D(c, m)$ este:

$$D(c, m) = \sum_{a=0}^{2^h-1} G[a][m] \cdot D(c-1, a)$$

Problema este că complexitatea totală este $\mathcal{O}(\sum w_i \cdot 2^{2h})$, adică circa 11 miliarde de operații. Ca să o reducem, observăm ca și în problemele anterioare că matricea G este independentă de numărul coloanei curente. Dacă îl percepem pe $D(c)$ ca pe un vector-coloană cu 2^h poziții, atunci putem scrie matricial formula de recurență:

$$\begin{pmatrix} D(c,0) \\ D(c,1) \\ \dots \\ D(c,m) \\ \dots \\ D(c,2^h-1) \end{pmatrix} = \begin{pmatrix} G[0][0] & G[1][0] & \dots & G[2^h-1][0] \\ G[0][1] & G[1][1] & \dots & G[2^h-1][1] \\ \dots & \dots & \dots & \dots \\ G[0][m] & G[1][m] & \dots & G[2^h-1][m] \\ \dots & \dots & \dots & \dots \\ G[0][2^h-1] & G[1][2^h-1] & \dots & G[2^h-1][2^h-1] \end{pmatrix} \begin{pmatrix} D(c-1,0) \\ D(c-1,1) \\ \dots \\ D(c-1,2^h-1) \end{pmatrix}$$

sau, echivalent,

$$D(c) = G^T \cdot D(c-1)$$

Putem construi matricea G gata transpusă, dar parcă este mai natural să gîndim vectorii $D(c)$ ca pe vectori-linie și să scriem formula matricială:

$$D(c) = D(c-1) \cdot G$$

În orice caz, acum putem procesa separat fiecare dintre înălțimile $0 \leq i \leq h$. Pentru lățimea w_i corespunzătoare vom avea:

$$D(w_i) = D(0) \cdot G^{w_i}$$

Iar vectorul de intrare $D(0)$ este vectorul de ieșire al etapei anterioare, de înălțime $i-1$. Alternativ, putem privi problema astfel. Pe cele $h = 7$ trepte ale scării avem h matrice de tranziție diferite, $G_1 \dots G_h$, fiecare de ridicat la puterea $w_1 \dots w_h$. Produsul acestor puteri de matrice ne arată numărul de moduri de a trece de la o mască la alta după $w_1 + w_2 + \dots w_h$ coloane. Din această matrice finală ne interesează elementul de pe linia și coloana $2^h - 1$, întrucît pe prima și ultima coloană a scării toți pereții verticali sînt marcați (conturul).

Matricea G are dimensiune 2^h, deci o înmulțire de matrice va costa 8^h. Vom face $\log w_i$ înmulțiri pentru fiecare înălțime i, ceea ce duce la complexitatea totală $\mathcal{O}(8^h \sum_i \log w_i)$.

Ca optimizare observăm că, pentru înălțimi $i < h$, matricea G este goală, cu excepția colțului de jos-dreapta. Putem sări porțiuni mari din înmulțirea matricelor, deoarece cubul dimensiunii scade rapid cu înălțimea. În practică, programul devine de 5 ori mai rapid.

27.4.9 Problema Marfa (Lot 2014)

enunț • sursă

Să găsim întîi orice formulă de recurență, fără să ne preocupăm de valoarea mare a lui z. Ce ne trebuie ca să definim complet o stare (o zi), ca să putem apoi determina tranziția între zile?

- Numărul zilei. De aici, modulo n, putem afla comanda în acea zi.
- Numărul de zile anterioare consecutive în care jumătatea stîngă a fost ocupată.
- Similar pentru jumătatea dreaptă.

Așadar, fie $C[i][s][d]$ numărul de moduri de a transporta marfă timp de i zile, astfel încît după a i-a zi cele două jumătăți să fi fost folosite timp de s, respectiv d zile consecutive. Vom folosi programarea dinamică de tip înainte pentru a construi $C[i+1]$ (pare mai natural). Fie t cererea de transport în ziua $i+1$.

- Dacă $t = 0$, atunci din $C[i][s][d]$ putem ajunge doar în $C[i+1][0][0]$: nu avem nimic de transportat, deci contoarele celor două jumătăți se resetează.
- Dacă $t = 1$, atunci din $C[i][s][d]$ putem ajunge fie în $C[i+1][s+1][0]$, fie în $C[i+1][0][d+1]$. Contorul pentru jumătatea nefolosită se resetează. În ambele cazuri, înainte de a face propagarea trebuie să verificăm dacă $s < k-1$, respectiv $d < k-1$.
- Dacă $t = 2$, atunci din $C[i][s][d]$ putem ajunge doar în $C[i+1][s+1][d+1]$ și doar dacă $s < k-1$ și $d < k-1$.

Starea inițială este $C[0] = 0$ pe tot stratul 0 în afară de $C[0][0][0] = 1$. Starea finală este suma valorilor de pe stratul $C[z]$ (deoarece putem termina cu orice grad de folosire a camionului). Această soluție merge, dar $\mathcal{O}(zk^2)$ este prea lent.

Desigur, ne dorim să-l înlocuim pe $\mathcal{O}(z)$ cu $\mathcal{O}(\log z)$. Dar recurența de mai sus variază de la o zi la alta, pentru că depinde de cererea zilnică t. Din această cauză nu putem calcula matricea de tranziție. Ce-i de făcut?

Observăm că și n (durata unei săptămîni) este mic. Asta înseamnă că putem calcula naiv matricea de tranziție **pentru o săptămînă întreagă**. Această matrice va fi identică pentru orice săptămînă. Odată ce o calculăm, o putem ridica la puterea $\lfloor \frac{z}{n} \rfloor$. Apoi, mai avem nevoie de o a doua matrice pentru ultimele $z \mod n$ zile. Am putea calcula naiv aceste ultime zile, dar ar fi păcat, căci am duplica efort. Facem deja acest efort în scopul calculării matricei după n zile.

Trebuie să clarificăm ce reprezintă această matrice. $T_{i,j}$ este modul de a trece din starea i după 0 zile în starea j după n zile. Dar, de fapt, ce este o stare? O stare este o pereche de valori (s, d). Iar s și d iau valori între 0 și $k-1$. Așadar, matricea T are dimensiuni $k^2 \times k^2$. De exemplu, o reprezentare naturală este să numerotăm cele k^2 valori pentru s și d. Astfel, dacă

$$i = i_s \cdot k + i_d$$

și

$$j = j_s \cdot k + j_d$$

, atunci $T_{i,j}$ înseamnă, conceptual, numărul de moduri de a începe săptămîna cu i_s folosiri ale jumătății stîngi și i_d folosiri ale jumătății drepte și de a o încheia cu j_s, respectiv j_d folosiri.

Recapitulînd, algoritmul este:

- Calculăm matricea T pentru o săptămînă.
- Calculăm matricea R pentru primele $z \mod n$ zile dintr-o săptămînă.
- Calculăm matricea totală $A = T^{\lfloor z/n \rfloor} \cdot R$.
- Răspunsul la problemă este suma elementelor de pe linia 0 a matricei A, întrucît începem din starea 0 ($s = d = 0$) și putem sfîrși în orice stare.

27.4.10 Problema Options (IIOT 2017-2018 runda internațională)

enunț • sursă

Problema are multe elemente comune cu problema Marfa. Dacă ați înțeles-o pe aceea, puteți reutiliza multe idei.

Pornim de la recurența relativ simplă (la valoarea noastră...). Fie $w[r][c]$ numărul de drumuri de la $(0, 0)$ la (r, c) (judecăm 0-based). Condițiile inițiale sînt $w[0][0] = 1$, $w[*][0] = 0$ în rest . Formula de recurență este:

$$w[r][c] = \begin{cases} w[r-1][c-1] + w[r][c-1] + w[r+1][c-1] & \text{dacă celula } (r, c) \text{ este uscat} \\ 0 & \text{altfel} \end{cases}$$

(cu verificări suplimentare pentru ca $r \pm 1$ să nu iasă de pe hartă). Recurența funcționează, dar calculele necesită efort $\mathcal{O}(LC)$, prea mare.

Observăm că nu putem scrie o matrice generatoare pentru toate coloanele, căci regulile de tranziție depind de natura terenului (uscat / apă). În schimb, putem scrie o matrice generatoare **pentru o întreagă perioadă**. În esență, matricea G ar avea semnificația: G_{r_1,r_2} este numărul de drumuri de la celula $(r_1, 0)$ la celula (r_2, k). Atunci G^p ar avea semnificația: $G^p_{r_1,r_2}$ este numărul de drumuri de la celula $(r_1, 0)$ la celula (r_2, pk).

Subliniem că nu mai este suficient un vector. Nu putem număra doar drumurile de la $(0, 0)$ la (r_2, k), deoarece în al doilea și următoarele blocuri de k coloane putem începe de pe orice rînd.

Dacă notăm cu $b = \lfloor \frac{c}{k} \rfloor$ numărul de blocuri complete și cu $r = c \mod k$ numărul de coloane din ultimul bloc, atunci cu matricea G^b ajungem pînă la coloana kb. Putem să rezolvăm separat ultimele r coloane, dar ar fi păcat. În procesul de construire a matricei G, noi vom fi construit la un moment dat matricea de tranziție pentru primele r coloane. Trebuie doar să-i facem o copie la momentul potrivit.

Este nevoie de atenție la detalii. Bunăoară, mie mi-a fost mai ușor să vizualizez tranzițiile astfel. Pentru blocul de coloane $[0, k)$, G_{r_1,r_2} este numărul de drumuri care încep pe linia r_1, **în stînga blocului** (deci pe coloana -1) și se termină pe linia r_2, coloana $k - 1$.

Partea VIII

Geometrie computațională

Capitolul 28

Elemente de bază

Acest capitol se axează pe formule pentru

- puncte;
- drepte;
- teste de orientare;
- arii;
- intersecții.

Formulele de trigonometrie de la orele de matematică vă vor fi de ajutor!

28.1 Principii de implementare

Evitați împărțirile! Aceasta vă scapă de diverse erori de precizie și cazuri-limită (împărțiri prin zero). Ca bonus, codul devine mai rapid.

Luați în calcul cazurile particulare: unghiuri de 90°, drepte paralele, puncte coliniare etc. De multe ori, programul le tratează fără cod suplimentar.

Preferați comparațiile în locul valorilor exacte. Un exemplu frecvent: ne pasă mai mult dacă C se află pe dreapta AB, la stînga sau la dreapta ei. Ne pasă mai puțin care este valoarea exactă a unghiului ABC.

Puteți găsi multe tutoriale teoretice utile, de exemplu pe CP Algorithms sau pe UCF.

28.2 Puncte și vectori

Vom folosi interschimbabil următoarele trei noțiuni:

- Punctul $P = (x_P, y_P)$.
- Vectorul $\vec{p}$ de la origine la P.
- Matricea de 2×1 $\begin{pmatrix} x_P \\ y_P \end{pmatrix}$.

Notăm cu $|\vec{p}|$ sau pur și simplu cu p lungimea vectorului.

28.2.1 Adunare, înmulțire cu o constantă

Vectorii pot fi adunați între ei și scalați cu o constantă, ceea ce se traduce prin adunarea și înmulțirea cu o constantă a matricelor.

28.2.2 Produsul scalar (engl. *dot product*)

Se numește așa pentru că rezultatul este o valoare scalară. Are două definiții echivalente (demonstrați!):

- $\vec{p} \cdot \vec{q} = x_p x_q + y_p y_q$
- $\vec{p} \cdot \vec{q} = p \cdot q \cdot \cos\theta$, unde θ este unghiul dintre vectori.

Motivație (de exemplu): deplasarea unei greutăți pe orizontală cu o forță oblică.

Produsul scalar este comutativ și distributiv.

Aplicație: **Proiecții**. Lungimea proiecției vectorului p pe direcția vectorului q este

$$\text{proj}_{\vec{q}}\, \vec{p} = \frac{\vec{p} \cdot \vec{q}}{|\vec{q}|}$$

Aplicație: **Test de perpendicularitate**. Doi vectori sînt perpendiculari dacă produsul lor scalar este 0. Exemplu: pentru punctele $P = (5, 2)$ și $Q = (-4, 10)$, unghiul POQ este drept, iar $-4 \cdot 5 + 2 \cdot 10 = 0$. Mai mult, produsul este pozitiv / negativ după cum θ este mai mic sau mai mare de 90°.

Aplicație: **Comparații de unghiuri**. Vom avea ocazional nevoie să sortăm puncte radial, după unghiul pe care îl fac față de un reper comun. Nu ne interesează unghiurile în sine, ci să le comparăm între ele. Se aplică pe orice domeniu unde funcția cosinus este monotonă (de exemplu 0-180°).

28.2.3 Produsul vectorial (engl. *cross product*)

Se numește așa pentru că rezultatul este o valoare vectorială. Vectorul este perpendicular pe planul determinat de p și q, cu sensul dat de regula șurubului. Mărimea vectorului are două definiții echivalente (din nou, demonstrați!):

- $|\vec{p} \times \vec{q}| = x_p y_q - y_p x_q$
- $|\vec{p} \times \vec{q}| = p \cdot q \cdot \sin\theta$.

Motivație: mărimea produsului vectorial este aria paralelogramului descris de $\vec{p}$ și $\vec{q}$. Alternativ, este dublul ariei triunghiului. Faptul că produsul este orientat înseamnă că aria poate fi pozitivă sau negativă: produsul vectorial „iese din" sau „intră în" planul tablei după cum unghiul θ este pozitiv (p se rotește peste q în sensul trigonometric) sau negativ.

Aplicație: **Coliniaritate**. Dacă trei puncte A, B, C sînt coliniare atunci produsul vectorial $\vec{AB} \times \vec{AC} = 0$.

Aplicație: **Aria unui poligon** prin două metode: (1) ca sumă de trapeze și (2) ca sumă de triunghiuri cu vîrful în origine (vezi secțiunea Arii pentru detalii).

28.2.4 Unghiuri

Putem afla concret unghiul dintre doi vectori cu funcțiile `arcsin`, `arccos`, `atan` sau `atan2`. Atenție la cazurile particulare și erorile de precizie! Funcția `atan2` are meritul că primește la intrare valorile pe componente, `y` și `x`.

Funcțiile trigonometrice inverse sînt lente! Dacă problema permite, înlocuiți-le cu comparații de unghiuri, unde produsele de mai sus sînt suficiente.

28.2.5 Rotații

Pentru a roti punctul (x, y) în jurul originii cu θ radiani, putem aplica o matrice de rotație:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x\cos\theta & -y\sin\theta \\ x\sin\theta + y\cos\theta \end{pmatrix}$$

Putem calcula manual un exemplu, să zicem rotația punctului $(\frac{\sqrt{3}}{2}, \frac{1}{2})$ cu 30°.

Pentru a face rotația relativ la un alt punct, (x_0, y_0), putem face întîi o translație cu $(-x_0, -y_0)$, apoi rotația, apoi translația la loc cu $(+x_0, +y_0)$.

28.3 Drepte și segmente

Există multe forme echivalente pentru ecuația unei drepte în plan. Majoritatea rezultă imediat dintr-un desen (cu asemănare de triunghiuri).

1. $ax + by + c = 0$ – forma generală. Nu are cazuri particulare. Poate descrie cu ușurință drepte verticale ($a = 1$ și $b = 0$) sau orizontale ($a = 0$ și $b = 1$). Doar că nu întotdeauna decurge ușor din datele de la intrare.
2. $\frac{x-x_1}{x_2-x_1} = \frac{y-y_1}{y_2-y_1}$ – dreapta care trece prin punctele (x_1, y_1) și (x_2, y_2). De regulă aceasta decurge din datele de intrare (puncte). Este nedefinită pentru drepte verticale și orizontale (căci $x_1 = x_2$ sau $y_1 = y_2$), dar putem ușor preveni asta înmulțind mezii și extremii.
3. $y - y_1 = m(x - x_1)$ — dreapta care trece prin punctul (x_1, y_1) și are pantă m. Panta este tangenta unghiului format de dreaptă cu axa Ox. Este nedefinită pentru drepte verticale, căci unghiul este de 90° și panta este infinită.
4. $y = mx + y_0$ – dreapta care trece prin punctul $(0, y_0)$și are pantă m (engl. slope + y-intercept).

5. $y_0x + x_0y - x_0y_0 = 0$ – dreapta care taie axele Ox și Oy în punctele $(x_0, 0)$ și $(0, y_0)$ (engl. x-intercept + y-intercept).

28.3.1 Test de paralelism

Trecînd de la forma (1) la forma (3) aflăm că, în forma generală, panta este $-\frac{a}{b}$. Două drepte $ax+by+c=0$ și $dx+ey+f=0$ sînt paralele dacă și numai dacă au aceeași pantă, deci $-\frac{a}{b} = -\frac{d}{e}$ sau, echivalent, $ae = bd$.

28.3.2 Test de perpendicularitate

Două drepte sînt perpendiculare dacă și numai dacă produsul pantelor este egal cu -1:

$$-\frac{a}{b} \cdot -\frac{d}{e} = -1 \iff ad + be = 0$$

Demonstrație: în figura de mai jos, date fiind pantele m_1 și m_2 (unde, atenție, m_2 este negativă), construim punctele Q și R pe cele două drepte și scriem teorema lui Pitagora în triunghiul dreptunghic PQR:

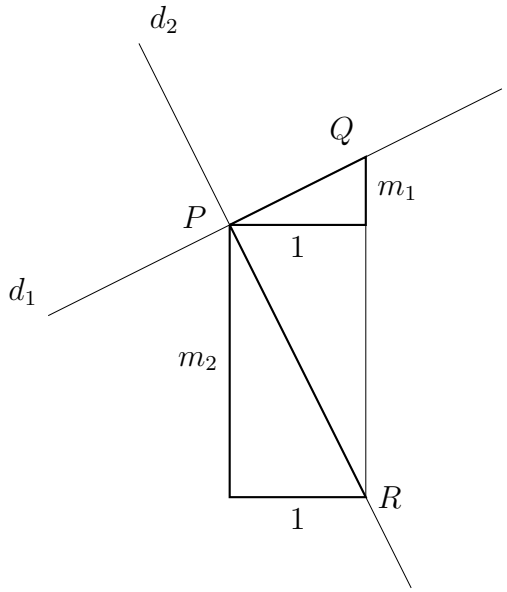


Figura 28.1: Dreptele d_1 și d_2 au respectiv pantele m_1 și m_2.

$$PQ^2 + PR^2 = QR^2$$
$$1^2 + m_1^2 + 1^2 + m_2^2 = (m_1 - m_2)^2$$
$$m_1 m_2 = -1$$

28.3.3 Intersecția a două drepte

Se rezolvă prin rezolvarea sistemului de două ecuații cu două necunoscute:

$$\begin{cases} ax + by + c &= 0 \\ dx + ey + f &= 0 \end{cases}$$

Cazuri particulare: dacă $a/d = b/e$ (sau, echivalent, $ae = bd$), atunci dreptele sînt paralele și nu există soluții. Dacă raportul este egal și cu c/f, atunci dreptele sînt confundate și există o infinitate de soluții.

28.3.4 Separarea planului

Valoarea $ax + by + c$ va fi 0 pentru toate punctele de pe dreaptă și doar pentru acelea. Dar, mai mult decît atît, valoarea va fi în mod consecvent pozitivă sau negativă pentru punctele de o parte și de cealaltă a dreptei. Așadar, două puncte (x_1, y_1) și (x_2, y_2) se află de aceeași parte a dreptei dacă $ax_1 + by_1 + c$ și $ax_2 + by_2 + c$ au același semn.

28.3.5 Distanța punct-dreaptă

Dacă, în plus, ecuația dreptei are **forma canonică**, adică $a^2 + b^2 = 1$, atunci înlocuind un punct în ecuația dreptei nu obținem doar un număr pozitiv/negativ oarecare, ci distanța de la punct la dreaptă.

Putem aduce ecuația dreptei la forma canonică împărțind a, b și c prin $\sqrt{a^2 + b^2}$.

28.3.6 Intersecția a două segmente

Date fiind segmentele AB și CD, dacă avem nevoie de intersecția lor, putem să calculăm intersecția dreptelor și să vedem dacă ea se află în interiorul ambelor segmente. Dacă, în schimb, vrem doar să știm dacă segmentele se intersectează, putem folosi separarea planului, care nu necesită împărțiri:

- Se află A și B de părți diferite ale dreptei CD?
- Se află C și D de părți diferite ale dreptei AB?

Discuție: ce lipsește din implementarea de mai jos? Are erori de precizie? Erori de împărțire prin zero? Cazuri particulare degenerate?

```
typedef struct {
  int x, y;
} point;

int sgn(long long x) {
  return (x > 0) - (x < 0);
}
```

```
int det(point a, point b, point c) {
  return sgn((long long)(b.x - a.x) * (c.y - a.y) -
             (long long)(c.x - a.x) * (b.y - a.y));
}

bool segments_intersect(point a, point b, point c, point d) {
  return
    (det(a, b, c) != det(a, b, d)) &&
    (det(c, d, a) != det(c, d, b));
}
```

28.4 Orientare; determinanți

Am ajuns deja la observația că dreapta separă planul în valori pozitive, negative și nule. Atunci cînd ecuația dreptei are o orientare, de exemplu cînd este specificată prin două puncte P și Q care îi dau un sens, putem căpăta informații ca stînga / dreapta și sens trigonometric / sens orar.

Concret, dîndu-se trei puncte A, B, și C, sînt ele (a) date în ordine trigonometrică sau (b) date în ordine orară sau (c) coliniare? Echivalent, vectorul $\vec{BC}$ cotește, față de vectorul $\vec{AB}$, (a) la stînga sau (b) la dreapta sau (c) nu cotește?

Abordarea directă este să calculăm ecuația dreptei AB sub forma $ax + by + c = 0$ și să înlocuim coordonatele punctului C în această ecuație. Mai simplu, lăsăm ecuația dreptei AB în forma determinată de două puncte. Înlocuind punctul generic (x, y) cu (x_C, y_C) obținem:

$$\frac{x_C - x_A}{x_B - x_A} = \frac{y_C - y_A}{y_B - y_A}$$
$$(x_C - x_A)(y_B - y_A) - (y_C - y_A)(x_B - x_A) = 0$$

Putem scrie partea stîngă ca pe un determinat:

$$\begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix} \gtrless 0$$

Valoarea determinantului este:

- pozitivă dacă A, B și C sînt în sens trigonometric;
- negativă dacă A, B și C sînt în sens antitrigonometric;
- 0 dacă A, B și C sînt coliniare.

Aplicație: **test de incluziune a punctului în triunghi** sau în orice poligon convex. Testăm determinanții triunghiurilor formate de punct cu fiecare latură a poligonului, P_iP_{i+1}. Dacă punctul

este strict în poligon, vom obține peste tot același semn. Dacă punctul este pe o latură sau într-un vîrf, vom obține și una sau două valori 0. Dacă punctul este exterior poligonului, vom obține atît valori pozitive, cît și negative. Dacă, în plus, știm ordinea în care sînt specificate vîrfurile poligonului, atunci știm exact ce semn așteptăm, ceea ce scurtează puțin implementarea.

28.5 Arii

De fapt, determinantul de mai sus ne spune mult mai mult decît un simplu semn. Valoarea lui, luată în modul, este dublul ariei triunghiului ABC:

$$\mathcal{A}_{ABC} = \frac{1}{2}\begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix}$$

28.5.1 Aria unui poligon oarecare prin metoda trapezelor

Pentru fiecare latură a poligonului, $P_{i-1}P_i$, să considerăm trapezul dreptunghic determinat de P_{i-1}, P_i și proiecțiile lor pe Ox.

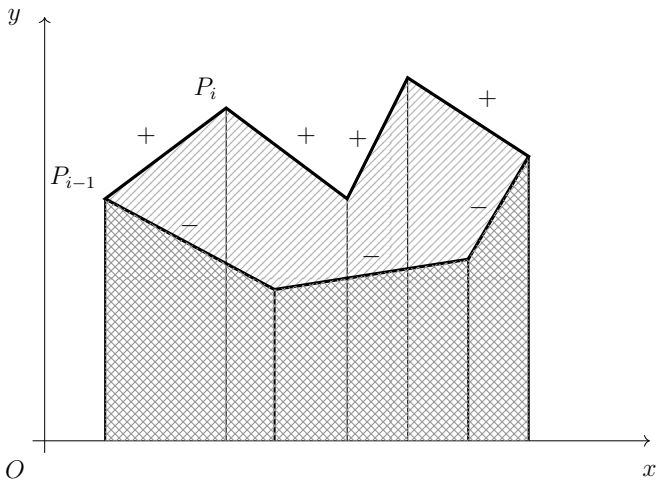


Figura 28.2: Aria heptagonului cu contur gros se obține prin însumarea ariilor cu semn ale celor 7 trapeze.

Acest trapez are aria:

$$\mathcal{A} = (x_i - x_{i-1}) \cdot \frac{y_i + y_{i-1}}{2}$$

Însumăm toate aceste arii și luăm rezultatul în modul. Frumusețea constă în faptul că trapezele pentru care $x_i > x_i - 1$ vor contribui cu arii pozitive, iar celelalte cu arii „negative”. Diferența este exact aria poligonului.

28.5.2 Aria unui poligon oarecare prin metoda triunghiurilor

O metodă similară, dar cu o formulă puțin mai simplă, consideră pentru latura $P_{i-1}P_i$ triunghiul $OP_{i-1}P_i$. Prin fix același raționament deducem că modulul sumei ariilor triunghiurilor este aria poligonului. Iar, deoarece unul dintre puncte este $(0, 0)$, aria se calculează foarte simplu:

$$\mathcal{A}_{OP_{i-1}P_i} = \frac{1}{2}(x_i y_{i-1} - x_{i-1} y_i)$$

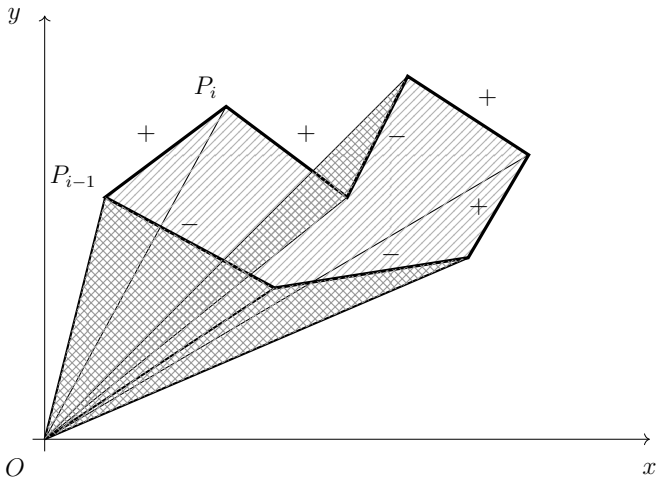


Figura 28.3: Aria heptagonului cu contur gros se obține prin însumarea ariilor cu semn ale celor 7 triunghiuri.

28.6 Probleme

28.6.1 Problema Cuiburi (Baraj ONI 2010)

enunț • sursă

2010 erau vremuri mai simple. 😅 Cărămida de bază a problemei este să testăm incluziunea între două figuri geometrice, care pot fi dreptunghiuri sau cercuri. Deci există un `if` cu patru cazuri, pe care le vom discuta.

Problema cere un fel de drum de lungime maximă într-un graf în care figurile sînt noduri, iar muchiile indică incluziunea. Dar putem observa că graful este aproape aciclic. Poate exista un ciclu doar între figuri identice. Rezultă că putem căuta o sortare topologică astfel încît toate muchiile să meargă doar spre dreapta. Putem sorta după arie sau după perimetru (eu am ales aria).

După sortare, definim $l[i]$ = lungimea maximă a unei cuibăreli terminate cu cuibul i și calculăm vectorul l în complexitate totală $\mathcal{O}(n^2)$.

28.6.2 Problema Ace (OJI 2017, clasa a 9-a)

enunț • sursă

Nu este foarte mult de povestit. Examinăm fiecare rază pornind de la (n, m). Pentru o rază cu progresia (dr, dc), iterăm prin toate punctele și menținem maximul. Prin „maxim" înțelegem acul care ne obligă să „ridicăm" privirea cel mai mult de la orizontală, adică acul al cărui vîrf formează un unghi maxim cu orizontala și cu punctul (n, m).

Am ales să citesc matricea invers ca să pot porni razele de la $(0, 0)$.

28.6.3 Problema Baba Oarba (Lot 2016)

enunț • sursă

Va fi nevoie să facem D pași pentru a ajunge la Zeratul, deci problema ne permite 50 de apeluri ca să ne așezăm pe direcția potrivită. Este oare suficient?

O idee destul de directă este să folosim căutarea binară pentru a înjumătăți sectorul de cerc în care se poate afla Zeratul relativ la Tassadar. Să detaliem!

La o distanță de cel mult 100.000 de metri ne este permisă o abatere de cel mult 1 metru. Ce înseamnă asta ca unghiuri? Cît de precisă trebuie să fie direcția aleasă? Pentru unghiuri foarte mici, știm că $\sin \alpha \approx \alpha$. Așadar, trebuie să calculăm direcția cu o precizie de $1/100.000$. Inițial, direcția poate fi oriunde între 0 și 2π, deci trebuie să reducem intervalul de circa 6,3 milioane de ori. Rezultă că ne trebuie 23 de iterații și avem dreptul să facem cam două interogări per iterație.

Astfel, am redus problema la întrebarea: știind sectorul curent în care se află Zeratul, cum îl înjumătățim punînd doar două întrebări? (Cum înjumătățim sectorul, nu pe Zeratul.)

O soluție imperfectă, dar suficient de bună, calculează bisectoarea sectorului curent și face un pas la stînga, **perpendicular pe bisectoare**. Zeratul este în stînga sau în dreapta bisectoarei după cum Tassadar s-a apropiat sau s-a depărtat de el. Apoi Tassadar face pasul înapoi.

Aș mai preciza și că putem testa problema offline, ca să nu consumăm *submissions* aiurea. Este nevoie să ne scriem propriul babaoarba.h, dar nu este o treabă prea grea.

28.6.4 Problema Arhitect (OJI 2023, clasa a 10-a)

enunț • surse

Problema este elementară. Trebuie să grupăm niște segmente după pante, avînd grijă să numărăm pantele m împreună cu pantele $-1/m$ pentru orice m. Ca să nu operez cu pante infinite, am ales să rotesc toate segmentele pentru a le aduce în cadranul I, mai exact, în intervalul [0°, 90°). Putem face asta cu cîteva `if`-uri, dar mi s-a părut mai simplu să fac efectiv cel mult trei rotații:

```
while ((dx <= 0) || (dy < 0)) {
  int tmp = dy;
  dy = -dx;
  dx = tmp;
}
```

De aici, implementarea se ramifică:

1. Putem opera chiar pe perechi de numere. Pentru a testa egalitatea a două pante `a` și `b`, testăm egalitatea produselor `a.dx * b.dy` și `a.dy * b.dx`.
2. Putem opera pe `double`, căci `dx` este nenul și putem face împărțirea. Nu am întîmpinat erori de precizie, deși în teorie ele sînt posibile.
3. Pentru numărarea duplicatelor, putem să colectăm pantele și să le sortăm sau să le stocăm într-un map.

28.6.5 Problema Elicoptere (OJI 2016, clasele 11-12)

enunț • sursă

Problema cere:

1. Să stabilim ce perechi de triunghiuri pot fi unite prin segmente orizontale sau verticale de lungime cel mult k.
2. Să construim graful indus în care triunghiurile sînt noduri, iar segmentele sînt muchii.
3. Să calculăm arborele parțial minim al a cestui graf și să raportăm diverse informații.

Mai departe, trebuie doar să evităm complicațiile la punctul (1). Cu $n \leq 100$ eficiența este complet irelevantă. Mai important este să programăm îngrijit.

Sper să fie evident că distanța minimă se obține (și) printr-un segment care conține un vîrf al unui triunghi. O demonstrație poate fi: să considerăm că distanța minimă ia naștere undeva între două laturi AB și respectiv CD, dar fără a conține niciun vîrf. Atunci:

- fie $AB \nparallel CD$, caz în care putem translata segmentul de lungime „minimă” pentru a îl scurta,
- fie $AB \parallel CD$, caz în care putem translata segmentul pînă atinge un vîrf, păstrîndu-i lungimea.

Codul meu evidențiază subprobleme succesiv mai mici. Cărămida de bază este aflarea distanței orizontale de la un punct la un segment. Nu am reușit să evit cazul particular în care segmentul este și el orizontal. Pentru tratarea cazului vertical, am preferat să reflect punctele față de diagonala principală ((x, y) devine (y, x)), apoi să rulez tot cazul cu segment orizontal.

28.6.6 Problema Arpa and an Exam About Geometry (Codeforces)

enunț • sursă

Și această problemă este relativ elementară. Pentru a putea roti A peste B, trebuie să alegem un centru P astfel încît $PA = PB$. Așadar, P trebuie să fie pe mediatoarea segmentului AB. Similar, P trebuie să fie pe mediatoarea segmentului BC. În concluzie, P este centrul cercului circumscris triunghiului ΔABC. Din fericire, nu trebuie să aflăm acest centru, ci doar să știm dacă el există. Este suficient să verificăm că A, B și C nu sînt coliniare.

Pentru a putea roti **simultan** A peste B și B peste C, să ne închipuim că rotim cercul centrat în P. Este nevoie ca $\angle APB = \angle BPC$, ceea ce se poate exprima mai simplu ca $AB = BC$.

28.6.7 Problema Part (Baraj ONI 2006)

enunț • sursă

Ideile de geometrie necesare sînt facile. Trebuie doar să evităm să testăm explicit condiția „piesele acoperă toată porțiunea desenată", care sună infernal. În loc de asta, putem verifica următoarele condiții:

1. Suma ariilor triunghiurilor este egală cu aria tablei.
2. Toate cele $3n$ puncte sînt pe tablă.
3. Oricare două triunghiuri nu au intersecții nedegenerate.

Condiția (1) este banală. Aplicăm pur și simplu determinantul pentru calculul ariei triunghiului. Ca să rămînem pe tipuri de date întregi, calculăm dublul tuturor ariilor.

Condiția (2) se reduce la un test de includere a unui punct P într-un triunghi ABC. Un mod de a implementa asta este să verificăm că ariile triunghiurilor PAB, PBC și PCA au același semn (sau 0). Dar triunghiul ABC nu este oarecare, ci este chiar tabla. Știm deja că P este în primul cadran și trebuie să aflăm doar dacă este sub segmentul $(0, b) - (a, 0)$. De aceea, putem reduce mult testul. Eu am ales să calculez, pentru coordonata x dată, care este y-ul maxim admisibil. Astfel am ajuns la formula:

$$ay + bx \leq ab$$

Ajungem astfel la condiția (3), cea mai dificil de testat. Prima mea idee a fost: două triunghiuri se intersectează dacă și numai dacă unul are un vîrf strict în interiorul celuilalt. Dar această idee greșește pe cazuri ca Steaua lui David sau două triunghiuri confundate. Am ajuns la următoarea idee corectă, posibil prea complicată:

- Dacă oricare din cele 3×3 perechi de laturi se intersectează, atunci triunghiurile se intersectează. Atenție, vorbim de intersecții nedegenerate: segmentele nu au aceeași dreaptă-suport, iar intersecția nu este fix la capătul niciunui segment.
- Dacă unul dintre triunghiuri este complet inclus în celălalt (inclusiv pe latură), atunci triunghiurile se intersectează.
- Altfel triunghiurile nu se intersectează.

Ca detalii de implementare:

- Încă de la citire ne putem asigura că toate triunghiurile sînt date în ordine trigonometrică. Aceasta simplifică diverse teste.
- Toate operațiile sînt pe întregi.
- Metodele sînt scurte și încapsulate în `struct`-urile `point`, `segment` și `triangle`.

- Unde există ambiguități, am adăugat un scurt comentariu, în special privind tratarea cazurilor-limită.

28.6.8 Problema Bear and Floodlight (Codeforces)

enunț • sursă

Problema are două componente, ambele ușoare (la valoarea noastră...).

Componenta de geometrie

Cărămida de bază este: presupunînd că am reușit, folosind o submulțime de reflectoare, să ducem ursul în punctul $(x, 0)$, pînă unde îl putem duce folosind un nou reflector aflat la x_c, y_c și de unghi a?

Metoda 1: Aflăm, de exemplu cu funcția `atan2`, unghiul format de vectorul $(x_c, y_c) - (x, 0)$ cu orizontala. Adăugăm a la acest unghi. Obținem ecuația unei drepte care trece prin (x_c, y_c) și are un unghi determinat. În ecuația acestei drepte înlocuim y cu 0 și aflăm noul x.

Metoda 2: Aplicăm o matrice de rotație punctului $(x, 0)$ raportat la (x_c, y_c). Astfel obținem un punct (x', y'). Din punctele coliniare (x_c, y_c),(x', y') și $(x_{nou}, 0)$ aflăm prin asemănare x_{nou}. Mi se pare o metodă mai simplă și este și mai rapidă, întrucît putem precalcula sinusul și cosinusul unghiului de rotație a. În rest, facem doar înmulțiri. În metoda 1 nu avem această posibilitate, căci mereu aplicăm arctangenta unui segment diferit.

Este nevoie să tratăm cazul cînd reflectorul ajunge să bată orizontal sau chiar în sus, caz în care noul x poate fi nedefinit sau poate „fugi" înapoi în stînga. Dar este suficient doar un `if`: dacă $y' \geq y_c$, înseamnă că am rotit reflectorul într-o poziție care acoperă semidreapta orizontală de la (x_c, y_c) spre dreapta.

Componenta de programare dinamică

Dat fiind că n este mic, ne permitem să scriem o programare dinamică pe măști de biți. Pentru o mască m reținem x-ul maxim la care putem ajunge folosind doar reflectoarele cu bitul setat în mască, în orice ordine.

Cum calculăm acest maxim? Dintre reflectoarele din m, unul trebuie să fie ultimul. Deci, rînd pe rînd, eliminăm cîte un bit, luăm x-ul maxim oferit de masca rămasă și îl extindem cu reflectorul al cărui bit l-am eliminat. Astfel obținem complexitatea totală $\mathcal{O}(n \cdot 2^n)$.

28.6.9 Problema TrapEZZ (Moisil++ 2023, clasele 11-12)

enunț • sursă

Soluțiile relativ evidente sînt $\mathcal{O}(n^4)$, care iterează prin toate mulțimile de 4 puncte, și $\mathcal{O}(n^3 \log n)$ sau chiar $\mathcal{O}(n^3)$, care fixează 3 puncte și îl caută pe al 4-lea.

Pentru o soluție în $\mathcal{O}(n^2 \log n)$, să vedem ce putem face pentru perechi de puncte. Dacă fixăm o bază și ne întrebăm „unde ar putea fi cealaltă bază?”, nu (cred că) ajungem nicăieri, pentru că bazele trebuie să fie bine aliniate pentru ca trapezul să fie isoscel. Însă așa ne poate veni ideea să folosim mediatoarele pe baze: două segmente reprezintă bazele unui trapez isoscel dacă au aceeași mediatoare.

Restul sînt detalii de implementare. Decît să grupez segmentele într-un `map` după ecuația mediatoarei, am preferat să le sortez după mediatoare, apoi să le procesez în blocuri cu aceeași mediatoare, ceea ce s-a dovedit a fi mai rapid.

Este nevoie de atenție și la condițiile (3) și (4): trebuie să eliminăm trapezele degenerate și dreptunghiurile.

28.6.10 Problema Terenuri (Baraj ONI 2011)

enunț • sursă

Problema definește exact diagramele Voronoi, dar rezolvarea nu se bazează pe ele (din fericire). Vom studia natura diagramelor Voronoi și vom trage concluzia că un țăran este liber dacă și numai dacă el se află pe înconjurătoarea convexă a setului curent de puncte (inclusiv pe o latură). Deocamdată nu avem nevoie să cunoaștem algoritmi pentru înfășurătoarea convexă. Îi vom studia în lecția viitoare.

Așadar, problema se reduce la menține incremental înfășurătoarea convexă, pe măsură ce setul de puncte crește. Înfășurătoarea poate doar să crească, deci țăranii care sînt liberi pot deveni dominați, niciodată invers. Întrebarea-cheie este: cum procesăm un nou punct? Dacă el se află în interiorul înfășurătoarei, atunci țăranul aferent este dominat de la bun început și îl putem ignora. Dacă punctul se află pe înfășurătoare sau în afara ei, trebuie să îl adăugăm la înfășurătoare și eventual să scoatem punctele care formează unghiuri concave.

Acest lucru poate fi codat scurt și elegant, cu un singur `map<double, point>`. Alegem un centru arbitrar, de exemplu medianul primelor două puncte de la intrare. Raportat la centru, menținem punctele de pe înfășurătoare sortate polar. Am folosit funcția `atan2` pentru calculul unghiului, căci datele sînt deja numere reale. Restul implementării cere doar o navigare îngrijită printre vecinii, în `map`, ai punctului nou inserat.

Merită să avem o discuție aici despre evitarea redundanței în `std::map`. Dorim să inserăm punctul și să obținem un iterator la el, pentru a elimina unghiurile reflexe. Cel mai scurt cod ar arăta așa:

```
auto it = hull.find(angle);

if (it == hull.end()) {
hull[angle] = p;
it = hull.find(angle);
} else {
```

```
  it->second = p;
}
```

Doar că acest cod face căutarea de trei ori! De două ori cu funcția `find()` și a treia oară cu operatorul `[]`. Putem reduce aceasta la doar două căutări, căci funcția `map::insert` returnează un iterator la elementul inserat, exact ce ne trebuie:

```
iter it = h.find(angle);

if (it == h.end()) {
  it = h.insert({angle, p}).first;
} else {
  it->second = p;
}
```

Sursa completă anexată arată cum putem face o singură căutare, ceea ce mai cîștigă 10% din timp. Secretul este o combinație de două lucruri.

- Funcția `lower_bound()` returnează un iterator la primul element mai mare sau egal cu cheia căutată, adică exact la poziția inserării.
- Funcția `insert()` acceptă iteratorul de mai sus ca parametru și face inserarea în $\mathcal{O}(1)$.

28.6.11 Problema Metin2 (Finala IIOT 2021-22)

enunț • sursă

Ideea de rezolvare nu este excesiv de grea. Totul este să nu cădem în capcana de a menține intersecția poligonului, care pare o abordare nerezonabil de dificilă.

Mai rezonabil este să colectăm toate cele $4n$ laturi ca pe niște drepte **orientate**. Apoi le sortăm după pantă. Acum știm că poligonul-intersecție trebuie să se afle **în stînga tuturor dreptelor**. O primă observație este că, dacă există mai multe drepte paralele (cu aceeași pantă), este suficient să o păstrăm pe cea mai din stînga. Dacă intersecția o va respecta pe aceasta, le va respecta automat și pe restul.

Acum să considerăm că am iterat printr-o parte din drepte și am construit o parte din poligon, $ABC \ldots OPQ$. Să procesăm următoarea dreaptă în ordinea pantei, d. Dacă ea continuă „natural” poligonul, în sens trigonometric, atunci Q se va afla la stînga lui d, iar poligonul se continuă cu o porțiune din d. Dacă, în schimb, Q se află la dreapta lui d, atunci prin definiție Q nu are cum să facă parte din intersecție.

(TODO: figură)

Rezultă că putem ține o stivă de drepte presupuse a forma laturile intersecției. Pentru ușurință, ținem și o stivă de intersecții între perechi de drepte consecutive. Acestea formează vîrfurile poligonului. Fiecare dreaptă procesată elimină din stivă vîrfurile aflate în dreapta ei, precum și

dreptele aferente acelor vîrfuri. Apoi se inserează pe ea însăși în stivă, împreună cu punctul de intersecție între ea și ultima dreaptă din stivă.

La final, rezultă o linie frîntă în formă de „6", care se poate autointersecta și poate conține multe segmente redundante dincolo de poligonul propriu-zis. Acestea trebuie eliminate, de la ambele capete, folosind același test de orientare (stînga = OK, dreapta = elimină).

Detaliu de implementare)

Cînd sortăm dreptele după pantă, la egalitate (drepte paralele) dorim să le sortăm de la stînga la dreapta. Cum definim „stînga" și „dreapta"? Dreapta d_1 este în stînga dreptei d_2 dacă, înlocuind un punct (orice punct) de pe dreapta d_1 în ecuația dreptei d_2 obținem o valoare pozitivă.

Cel mai simplu mi-a fost să rețin, pentru fiecare dreaptă, un punct cunoscut a fi pe dreaptă (unul dintre cele două puncte date la intrare). Există și o metodă de a afla un punct de pe dreapta d_1 în momentul testului, ceea ce reduce memoria, dar codul devine mai lung și mai greoi.

Capitolul 29

Algoritmi specifici

29.1 Teorema lui Pick

Teorema lui Pick (Wikipedia, CP Algorithms) definește o relație între aria unui poligon cu vîrfuri de coordonate întregi (numit și poligon laticeal) și numărul de puncte laticeale din interiorul său și de pe conturul său. Dacă poligonul are I puncte laticeale interioare și B puncte pe contur (engl. *boundary*), atunci aria sa A este:

$$A = I + \frac{B}{2} - 1$$

Demonstrația nu este grea, dar este laborioasă. Ea demonstrează corectitudinea formulei de la figuri simple spre complexe:

1. Dreptunghiuri cu laturile paralele cu axele.
2. Triunghiuri dreptunghice cu laturile paralele cu axele.
3. Triunghiuri oarecare, prin completarea lor, în exterior, cu triunghiuri dreptunghice pentru a forma un dreptunghi.
4. Poligoane oarecare, prin triangulare.

29.2 Înfășurătoarea convexă

Definiție: **Înfășurătoarea convexă** a unei mulțimi de n puncte este cel mai mic poligon convex care include mulțimea, pe contur sau în interior.

Vom descrie trei algoritmi clasici:

1. Graham scan, de complexitate $\mathcal{O}(n \log n)$.
2. Lanțuri monotone, tot de complexitate $\mathcal{O}(n \log n)$.
3. Împachetarea cadoului (numit și „marșul lui Jarvis"), de complexitate $\mathcal{O}(n \cdot h)$.

Complexitatea $\mathcal{O}(n \log n)$ este optimă. Dacă ar exista un algoritm asimptotic mai bun, l-am

putea folosi ca să sortăm numere mai rapid decît $\mathcal{O}(n \log n)$. Fie $v[1...n]$ vectorul de sortat. Pentru fiecare element $v[i]$ creăm un punct în plan $(v[i], v[i]^2)$. Calculăm înfășurătoarea convexă a mulțimii de puncte. Cum punctele sînt pe o parabolă, toate vor fi pe înfășurătoarea convexă, care va enumera aceste puncte în ordine... adică va sorta vectorul v.

Ca la multe alte implementări de geometrie computațională, dificultatea constă în evitarea cazurilor particulare.

29.3 Algoritmi de baleiere

Un **algoritm de baleiere** (engl. *sweep line algorithm*) este o tehnică prin care iterăm printr-o colecție de obiecte (puncte, segmente, dreptunghiuri etc.) prin translatarea în plan a unei drepte imaginare, de exemplu prin translatarea orizontală a unei drepte verticale. Menținem o structură de date relevantă pentru poziția curentă a dreptei. Examinăm momentele în care dreapta atinge o poziție notabilă: coordonata unui punct, extremitatea stîngă sau dreaptă a unui dreptunghi etc. Aceste momente se numesc **evenimente**. Pentru fiecare eveniment actualizăm structura de date, ceea ce de regulă ne permite să actualizăm și răspunsul la problemă.

Folosim această tehnică pentru a obține timpi de rulare asimptotic mai buni decît cu o soluție naivă. Wikipedia oferă puțin context istoric și un exemplu vizual pentru calculul diagramelor Voronoi.

Unele dintre problemele de baleiere necesită efectiv geometrie computațională, folosind formule pentru intersecții, orientări etc. Altele sînt rezolvabile și fără, doar cu menținerea unor structuri de date peste numere întregi.

Vom discuta teoretic două probleme clasice:

1. Dîndu-se o colecție de n segmente, găsiți două care se intersectează sau raportați că nu există nicio intersecție. Puteți găsi explicații în detaliu pe CP Algorithms și o prezentare cu mai multă grafică aici. Pentru o problemă înrudită, vedeți Lights (CodeChef).
2. Calculați aria / perimetrul reuniunii a n dreptunghiuri cu laturile paralele cu axele. Pentru o formulare apropiată, vedeți Vika and Segments (Codeforces).

29.4 Șublerul rotitor

Această metodă (engl. *rotating calipers*) amintește un pic de tehnica celor doi pointeri. Ea reduce la $\mathcal{O}(n)$ timpul de rezolvare pentru probleme pe care le-am putea rezolva în $\mathcal{O}(n \log n)$ prin n căutări binare. Ea rezolvă multe probleme clasice ca:

- Diametrul unui poligon (distanța maximă între două puncte din poligon).
- Lățimea unui poligon (distanța minimă între două drepte paralele care cuprind poligonul).
- Distanța minimă/maximă dintre două poligoane.
- Dreptunghiul de arie/perimetru minim care cuprinde un poligon.

- ... și multe altele, enumerate *pe Wikipedia*.

Există cîteva noțiuni comune acestor probleme:

1. O **dreaptă-suport** este o dreaptă care trece printr-un vîrf al poligonului fără a trece prin interiorul acestuia.
2. O pereche de **puncte antipodale** constă din puncte care admit drepte-suport paralele.
3. Metoda șublerului rotitor se bazează pe enumerarea tuturor perechilor de puncte antipodale.
4. Trecerea de la o pereche la următoarea se poate face în $\mathcal{O}(1)$.

29.5 Probleme

29.5.1 Problema Copaci (Infoarena)

enunț • sursă

Soluția aplică direct Teorema lui Pick.

29.5.2 Problema Emptri (Lot 2015)

enunț • sursă

Teorema lui Pick ne pune pe direcția corectă. Dorim ca $I = 0$ și $B = 3$, de unde rezultă $A = \frac{1}{2}$. Restul este contabilitate. Fie cele două puncte (a, b) și (c, d) și fie $c \leq a$. Aria triunghiului va fi $\frac{1}{2}|ad - bc|$, calculabilă prin orice metodă preferați. Așadar, dorim ca $ad - bc = \pm 1$. Mai mult, trebuie ca $b \leq a$ și $\gcd(a, b) = 1$, căci altfel segmentul $(0, 0) - (a, b)$ va conține puncte în plus.

Încercînd să găsesc o soluție în $\mathcal{O}(n^2)$, m-am împiedicat de cea în $\mathcal{O}(n \log \log n)$. Să fixăm valorile a și b. Cîte variante există pentru c și d? Observăm că formula $ad - bc = 1$, tocmai pentru că a și b sînt coprime, „seamănă” cu Identitatea lui Bézout și/sau algoritmul lui Euclid extins, din care știm că soluția este unică modulo a. Similar, va exista o soluție unică și pentru $ac - bd = -1$.

De exemplu, pentru punctul $(a, b) = (5, 2)$ obținem ecuațiile $5d - 2c = \pm 1$, care ne dă punctele $(c, d) = (2, 1)$ și respectiv $(c, d) = (3, 1)$.

Dacă pentru fiecare pereche de numere coprime (a, b) există fix două soluții, rezultă că trebuie doar să numărăm aceste perechi. Pentru aceasta ne folosim de funcția Euler, căci prin definiție $\varphi(a)$ este exact numărul de valori b coprime cu a. Răspunsul final este suma valorilor $\varphi(a)$, dublată conform raționamentului de mai sus, și $+1$ pentru triunghiul special $(0, 0) - (1, 0) - (1, 1)$:

$$R = 1 + 2 \sum_{a=2}^{n} \varphi(a)$$

Soluția oficială (descărcabilă de pe Infoarena) pomenește despre secvențe Farey. Nu știu ce sînt acelea, misiunea mea se încheie aici. 🙂

29.5.3 Problema Înfășurătoare convexă (Infoarena)

enunț • sursă

Aceasta este o problemă educațională. Enunțul este permisiv: el promite că nu vor exista puncte coliniare pe înfășurătoare. Funcția `graham_scan()` implementează strict algoritmul de stivă ordonată, în 10 linii de cod. Orice detalii dorim, le putem obține din sortarea atentă a punctelor. Apoi, algoritmul

- nu are cazuri particulare;
- nu procedează diferit pe jumătatea de sus și pe cea de jos;
- nu tratează special niciun punct în afară de primul.

29.5.4 Problema Înfășurătoare convexă (NerdArena)

enunț • sursă

Problema este aproape identică cu precedenta. Enunțul este mai strict și ne cere să nu includem puncte pe laturile înfășurătorii, ci doar în colțuri. Este nevoie de un `if` în plus la sortare (la unghi polar egal, luăm în calcul și distanța față de `p[0]`). De asemenea, trebuie să folosim operatorul corect la scoaterea din stivă, ca să excludem punctele de pe laturile înfășurătorii.

29.5.5 Problema Testament (Lot 2011)

enunț • sursă

Ideea rezolvării problemei este relativ directă. Trebuie să separăm cumva cele mn puncte în m mulțimi ale căror poligoane înfășurătoare să fie disjuncte. Am ales să ordonez punctele după x, iar la egalitate după y. Apoi luăm tranșe de cîte n puncte și le facem înfășurătoarea convexă.

Datorită acestei ordonări, primul și ultimul punct din fiecare tranșă vor fi garantat pe înfășurătoare (nu pot face unghiuri de 180° cu vecinii).

Pentru implementare, am ales varianta cu lanțuri monotone, deoarece punctele sînt deja sortate după x. Un punct eliminat din stivă este automat adăugat la lista punctelor interioare.

29.5.6 Problema Magic (JBOI 2023)

enunț • sursă

Problema este încadrată la o tehnică numită *Convex Hull Trick*. Puteți citi mai multe despre CHT pe Codeforces (de exemplu). Mărturisesc că nu înțeleg diferența față de algoritmul normal de înfășurătoare convexă. Este fix aceeași abordare cu o stivă ordonată. În plus, mă nemulțumesc termenii ca „trick” și „șmen”, pentru că ei induc noțiunea falsă că algoritmica este o colecție de trucuri și șmenuri.

Reducerea la geometrie computațională este destul de clară. Vrăjitorului i îi corespunde punctul în plan (x_i, e_i). Pentru fiecare vrăjitor i, trebuie să aflăm un alt vrăjitor j pentru care panta

$(x_j, e_j) - (x_i, 0)$ este maximă. Vom rezolva întîi problema la stînga ($j < i$). Pentru a rezolva problema la dreapta, vom oglindi toate coordonatele x și vom reapela aceeași rutină, ca să nu duplicăm cod. La final, vom oglindi la loc coordonatele x ca să tipărim răspunsurile în ordinea corectă.

Să ne imaginăm că vrăjitorul i locuiește în vîrful unui stîlp de înălțime e_i la coordonata x_i. Stîlpii maschează parțial sau complet alți stîlpi aflați dincolo de ei. Pentru a alege un mentor la stînga, vrăjitorul i trebuie să coboare la baza stîlpului și să „privească în sus" căutînd stîlpul al cărui vîrf face unghi maxim cu orizontala.

Fie P punctul vrăjitorului curent. Cînd comparăm doi mentori posibili, A și B, unde $x_A < x_B < x_P$, P îl va prefera pe A dacă B se află sub segmentul AP; altfel P îl va prefera pe B. Echivalent, vrăjitorul A va fi preferabil tuturor vrăjitorilor de sub linia AP.

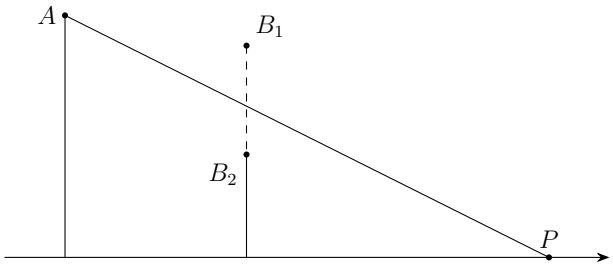


Figura 29.1: Privind din punctul P, stîlpul B_1 maschează stîlpul A complet, dar stîlpul B_2 nu. De aceea, vrăjitorul P l-ar prefera pe B_1 față de A ca mentor, dar l-ar prefera pe A față de B_2.

Să considerăm acum un exemplu mai complex. În figura 29.2, fiecare vrăjitor de la A la F este mentor pentru vecinul său din dreapta. Remarcăm că arcul $ABCDEF$ este convex. Pentru claritate, am împărțit zona de deasupra arcului în triunghiuri.

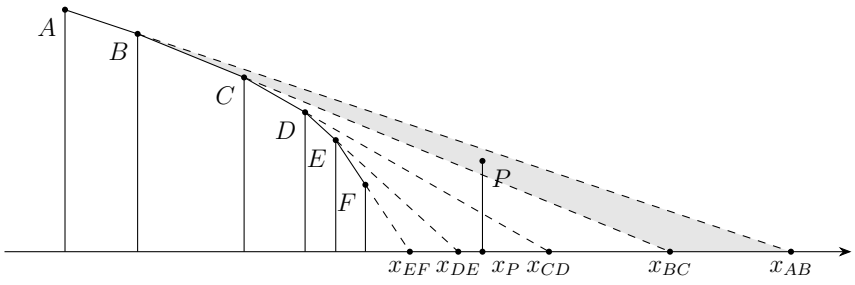


Figura 29.2: Mentorul vrăjitorului P este D deoarece x_P se află în triunghiul lui D. Mai mult, punctul P se află în triunghiul lui B, deci vrăjitorii $C - F$ nu vor mai fi mentori niciodată.

Fiecare vrăjitor va fi mentor pentru punctele din triunghiul corespunzător. De exemplu, triunghiul lui B este colorat cu gri. Ce avem de făcut cînd procesăm următorul vrăjitor, P? Mai întîi, stînd la $(x_P, 0)$, „privim în sus" către cel mai de sus vrăjitor vizibil. În figură, acela este vrăjitorul D,

deci îl alegem pe D ca mentor la stînga pentru P. Dar putem face mai mult decît atît. Deoarece $x_P > x_{DE}$, vrăjitorul P este în afara triunghiurilor lui E și F. **Indiferent cît de înalt este stîlpul lui** P, vrăjitorii E și F nu vor mai fi niciodată mentori pentru coordonate $x > x_P$.

Mai mult, punctul $P(x_P, e_P)$ se află în triunghiul vrăjitorului B. Aceasta ne spune că nici vrăjitorii C și D nu vor mai fi vreodată mentori pentru valori mai mari ale lui x, deoarece stîlpul lui P separă punctul C de porțiunea din triungi de coordonate $x > x_P$ (și similar pentru D). Pentru acele coordonate, doar A, B și P mai pot fi mentori.

În practică, vom menține o stivă de vrăjitori care mai au vreo șansă să fie mentori pe viitor. Cînd procesăm vrăjitorul curent P, ștergem din vîrful stivei toți vrăjitorii care devin invizibili. Există un mod mai simplu de a decide dacă un vrăjitor trebuie șters, unul care folosește doar testul de orientare stînga-dreapta. Examinăm ultimii doi vrăjitori de pe stivă, X și Y. Dacă $\angle XYP$ este reflex (mai mare decît 180°, atunci îl ștergem pe Y. Acest proces este echivalent cu raționamentul bazat pe triunghiuri din figura 29.1.

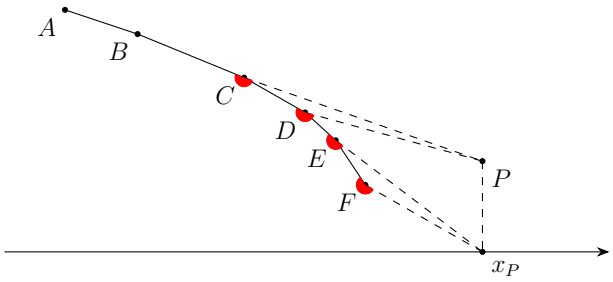


Figura 29.3: Punctul $(x_P, 0)$ șterge vrăjitorii F și E din stivă. Apoi, punctul P șterge vrăjitorii D și C din stivă. Unghiurile marcate cu roșu sînt reflexe.

Deoarece fiecare vrăjitor poate fi șters din stivă o singură dată, complexitatea amortizată este $\mathcal{O}(n)$.

29.5.7 Problema Triangular Queries (CodeChef)

enunț • sursă

Problema nu este de geometrie, dar ilustrează bine conceptul de „eveniment”.

Ideea de baleiere ia naștere din observația că ne este mai simplu să ținem evidența punctelor dacă avem garanția că toate sînt de aceeași parte a liniei de baleiere. Putem alege multe direcții de baleiere; eu am ales direcția NE $\rightarrow$ SV. Așadar, considerăm o dreaptă orientată de la NV la SE, care va avea ecuația $x + y = a$, și o translatăm de la $a = +\infty$ la $a = -\infty$.

Să considerăm o interogare, reprezentată în figura de mai jos prin triunghiul E. Iau naștere șase zone marcate cu literele $A \ldots F$:

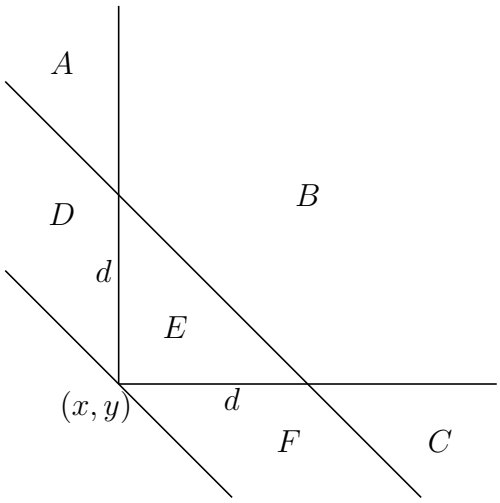


Figura 29.4: Poziția liniei de baleiere la două evenimente cauzate de interogarea triunghiulară E: baza și vîrful triunghiului.

Calculăm numărul de puncte din E prin diferență:

$$\begin{aligned} E &= (B + E) - B \\ &= [(A + B + C + D + E + F) - (A + D) - (C + F)] - [(A + B + C) - (A) - (C)] \end{aligned}$$

Toate cele 6 cantități din paranteze sînt ușor disponibile, dar la două momente diferite în timp. Cînd procesăm diagonala de sumă $x + y + d$ (cea de sus), atunci:

- $A + B + C$ este pur și simplu numărul de puncte văzute pînă atunci.
- A este numărul de puncte de coordonată x mai mică decît x-ul interogării. Putem menține ușor această informație într-un AIB.
- C este numărul de puncte de coordonată y mai mică decît y-ul interogării. De asemenea, putem menține această informație într-un AIB.

Similar putem afla celelalte cantități cînd procesăm diagonala de sumă $x+y$ (cea de jos). Rezultă că iau naștere trei tipuri de evenimente pe măsură ce linia de baleiere avansează:

1. Întîlnim un punct nou.
2. Întîlnim începutul unei interogări (ipotenuza triunghiului).
3. Întîlnim sfîrșitul unei interogări (vîrful dreptunghic al triunghiului).

O necesitate tipică pentru multe probleme este să clarificăm în ce ordine procesăm mai multe evenimente care survin la aceeași coordonată. În cazul nostru,

- Un punct nou trebuie procesat **după** ce procesăm orice bază care îl conține. Altfel îl vom scădea la contorizarea punctelor din triunghi, ceea ce este incorect.
- Un punct nou trebuie procesat **înainte** să procesăm vîrful dreptunghic al oricărui triunghi care se suprapune peste punct. De data aceasta vrem să contorizăm punctul.
- Între baza unui triunghi și vîrful altuia care coexistă pe aceeași linie de baleiere nu există

niciun conflict; le putem procesa în orice ordine.

Pentru claritate, vă îndemn să definiți și să colectați explicit evenimentele. La procesarea unui eveniment, în funcție de tipul său, apelați o funcție bine denumită. Atunci scrierea programului devine ușoară.

29.5.8 Problema Nuke (Lot 2010)

enunț • sursă

Enunțul spune, în două locuri diferite, că cercurile nu se suprapun. De ce este asta important? Pentru că fiecare punct poate contribui cu +1 la cel mult un cerc. Cu puțin fler, ajungem la concluzia că este mult mai simplu să examinăm punctele și să punem pentru fiecare întrebarea „este acest punct inclus într-un cerc, iar dacă da, în care?". Regăsim principiul des întîlnit că procesarea datelor într-o anumită ordine bine aleasă poate ușura mult rezolvarea unei probleme.

De aici pînă la baleiere mai este un singur pas. Sortăm punctele și cercurile după o coordonată (eu am ales x). Astfel, cînd procesăm punctul P ne interesează doar evidența cercurilor C active: cele care au $x_C - r \leq x_P$ și $x_C + r \geq x_P$. Așadar, evenimentele pe care le vom crea sînt:

- începutul unui cerc (la coordonata $x_C - r$ pentru fiecare cerc);
- sfîrșitul unui cerc (la coordonata $x_C + r$ pentru fiecare cerc);
- punct (la coordonata x_P pentru fiecare punct).

Să ne imaginăm linia de baleiere cînd trece printr-un punct P. Ea este verticală, trecînd prin x_P. Punctul, desigur, este un punct pe linie, la coordonata y_P. Dar ce sînt cercurile? Ce „vede" linia de baleiere? Vede coarde verticale (segmente). În mod notabil, **segmentele nu se intersectează**. De aici rezultă că putem stoca centrele cercurilor active într-o simplă structură ordonată (`set`, `map` etc.) Eu am ales un `map` deoarece, pentru un y dat, doresc să pot afla și care este cercul cu centrul la acel y.

Merită clarificat că nu insistăm să cunoaștem situația exactă a segmentelor (între ce coordonate y se întinde fiecare cerc). Ne este suficient să știm că ordinea centrelor nu se schimbă niciodată, pentru că dacă s-ar schimba, aceasta ar presupune că două cercuri s-au intersectat. De aceea, intervalul dintre două centre poate aparține fie unuia dintre cercuri, fie niciunuia, dar nu poate aparține altor cercuri.

Acum, cînd primim un punct P, îl căutăm pe y_P în această structură. El va fi încadrat de (cel mult) două centre de cercuri, unul deasupra și unul dedesubt. Acestea sînt singurele cercuri-candidat care l-ar putea conține pe P deoarece, cum am spus, cercurile ocupă segmente disjuncte pe linia de baleiere. Deci testăm acești doi candidați. Singura formulă de geometrie necesară este apartenența unui punct într-un cerc.

Ca de obicei, atenție la sortarea evenimentelor cu x egal! Ordinea este: deschide cerc, procesează punct, închide cerc, pentru a contoriza corect un punct care se află la extrema stîngă / dreaptă a unui cerc.

Sursa mea este didactică, cu rutine separate pentru citire, crearea evenimentelor, sortarea evenimentelor, baleiere și tipărire. Baleierea denumește clar evenimentele și apelează rutine separate pentru fiecare tip de eveniment. Desigur, codul poate fi (mult) condensat, dar mi se pare mai util să vedeți scheletul, care este reutilizabil la alte probleme de baleiere.

29.5.9 Problema Hill Walk (USACO Gold 2013)

enunț • sursă

Vom face o baleiere de la stînga la dreapta, în care evenimentele sînt cele $2n$ capete de interval, ordonate după x. La poziția curentă a liniei de baleiere dorim să avem evidența intervalelor active, ordonate de jos în sus. Atunci ne trebuie o structură de date care să admită operațiile:

1. Adaugă un interval (cînd întîlnim un eveniment de tip capăt stîng).
2. Șterge un interval (cînd întîlnim un eveniment de tip capăt drept).
3. Dă-mi intervalul anterior celui tocmai șters (pentru a actualiza poziția lui Bessie cînd intervalul ei curent se termină).

Implementarea este destul de alunecoasă. Intuitiv, am vrea să sortăm intervalele de la „jos" la „sus". Deci operatorul $a < b$ ar trebui să returneze `true` dacă și numai dacă a este poziționat „dedesubtul lui" b. Dacă două intervale acoperă un x comun (cu alte cuvinte, dacă proiecțiile lor pe axa Ox se suprapun), atunci noțiunea de „dedesubt" este bine definită și poate fi determinată folosind testul standard de orientare trigonometrică.

Dacă două intervale sînt disjuncte pe x, atunci noțiunea de „dedesubt" nu este bine definită. În special, nu putem încerca o sortare a tuturor celor n intervale simultan, căci relația de ordine este parțială, nu totală. Este important să comparăm doar intervale care coexistă la o coordonată x.

Putem folosi un `set` și operatorul `<` redefinit ca test de orientare.

29.5.10 Problema Ydist (Lot 2014)

enunț • sursă

Problema ne dă n puncte în plan și q raze (semidrepte) pornind din origine, toate aflate în cadranul I. Pentru fiecare rază, trebuie să determinăm distanța minimă pe verticală pînă la un punct aflat deasupra razei.

Problema este una de baleiere; ne gîndim la asta pentru că este mult mai simplu să răspundem la o întrebare (constînd dintr-o rază) cînd pînă acum am procesat doar puncte aflate deasupra razei. Din același motiv, vom face baleierea în ordine invers polară (de la 90° spre 0°).

Conceptul de baleiere

Așadar, avem de procesat două tipuri de evenimente: (1) apare un punct nou și (2) trebuie să răspundem la o interogare. Să considerăm un moment al baleierii, ilustrat în figura 29.5. Fie R

o interogare și fie P răspunsul la interogare. Ce putem spune despre restul planului? Mai exact, unde se pot afla puncte, unde nu se pot afla, și pe care le putem șterge?

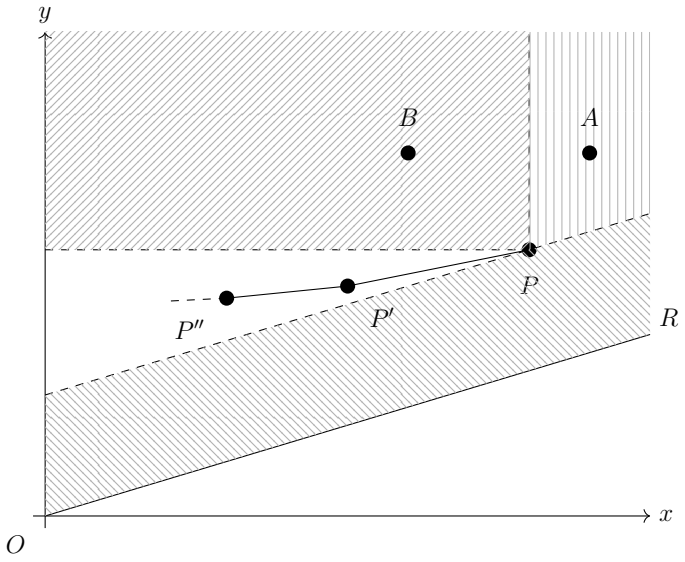


Figura 29.5

1. Nu se pot afla puncte în zona hașurată de sub P, situată între R și paralela la R prin P. Dacă ar exista vreun punct în acea zonă, el ar fi mai aproape de R decît P, deci el ar fi răspunsul.
2. Dacă există puncte în zona hașurată la nord-est de P, ele pot fi șterse, așa cum este cazul lui A. Punctul A este mai departe de R decît P. Mai mult, pe măsură ce raza de baleiere coboară, ea se va îndepărta de A mai repede decît de P. Cu alte cuvinte, niciodată în viitor A nu va mai putea fi răspunsul la vreo interogare.
3. Dacă există puncte în zona hașurată la nord-vest de P, ele pot fi șterse, așa cum este cazul lui B. Punctul B este, în prezent, mai departe de R decît P. Pe măsură ce raza de baleiere coboară, B recuperează din diferență. Dar la ce moment va egala el situația? La ce moment se vor afla B și P la distanțe egale de raza de baleiere? Doar atunci cînd raza va fi paralelă cu BP... adică niciodată. Așadar, nici B nu va mai fi vreodată răspunsul la o interogare.

Ordonarea după x și y

Astfel ne vine ideea să menținem un set de *puncte-candidat*: puncte care ar mai putea fi vreodată răspunsul la o interogare viitoare. Aceste puncte candidat se pot afla doar în triunghiul de la vest de P. Să privim din nou Figura 1. Fie P' cel mai din dreapta punct-candidat, cu excepția lui P. Punctului P' i se aplică aceeași regulă ca și lui P: punctele la nord-vest de el pot fi șterse. Așadar, următorul punct-candidat, P'', se va afla la sud-vest de P'.

Am demonstrat astfel că punctele-candidat sînt ordonate după x și după y.

Dacă ați avut în acest moment intuiția că putem menține punctele-candidat folosind o simplă stivă ordonată, sînteți pe drumul cel bun. Dar mai avem de demonstrat două lucruri.

Concavitatea

Prima întrebare este: cînd primim o interogare, care dintre punctele-candidat este răspunsul? Trebuie să le evaluăm pe toate? Poate exista o situație ca cea din figura 29.6, în care punctele ba se apropie, ba se depărtează de raza de baleiere?

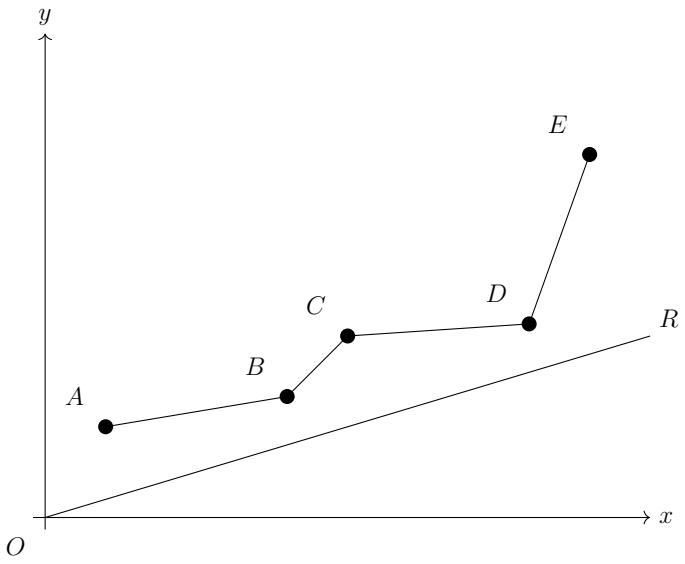


Figura 29.6

Răspunsul este „nu”. Curba punctelor-candidat este concavă (are forma literei U). (Fapt divers: Aici am pierdut circa 30 de minute încercînd să demonstrez că curba ar fi convexă...) Să urmărim figura 29.7, în care vedem 3 puncte-candidat A, B și C într-o formă convexă. Să încercăm să obținem o contradicție.

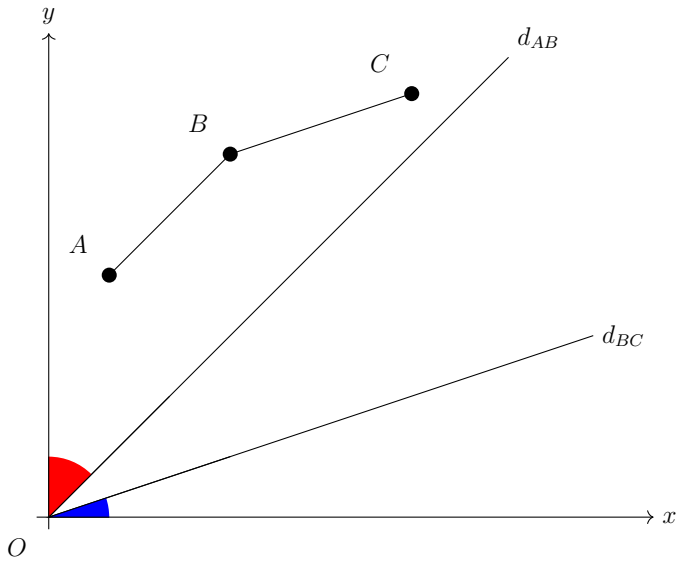


Figura 29.7

Există două raze de interes, $d_{AB} \parallel AB$ și $d_{BC} \parallel BC$. Să observăm că B este preferabil lui A (adică mai aproape de rază) doar pentru razele de deasupra lui d_{AB} (unghiul roșu). Similar, B

este preferabil lui C doar pentru razele de sub d_{BC} (unghiul albastru). Intersecția celor două unghiuri este vidă, așa că nu există niciun moment cînd B să le fie preferabil și lui A, și lui C. El poate fi șters. Astfel am demonstrat că lista punctelor-candidat este convexă.

De aici deducem că distanța la o rază, cînd iterăm prin toate punctele-candidat, este bitonă: ea scade pornind de la extreme spre centru. De aceea, putem aplica algoritmul clasic de eliminare din stiva ordonată: cînd primim o interogare, eliminăm ultimul punct din vîrful stivei cît timp penultimul punct are o distanță mai mică pînă la interogare.

Ordonarea polară

Ultima observație necesară privește inserarea punctelor. Cînd linia de baleiere întîlnește un nou punct P, unde este locul lui în lista de candidați? Poate fi undeva la mijloc, cauzînd o inserare în $\mathcal{O}(n)$?

Răspunsul este „nu”. Punctul P va avea cel mai mic unghi polar de pînă acum (căci baleierea este polară), iar lista punctelor-candidat este ordonată polar. Ca să ne convingem, să urmărim figura 29.8.

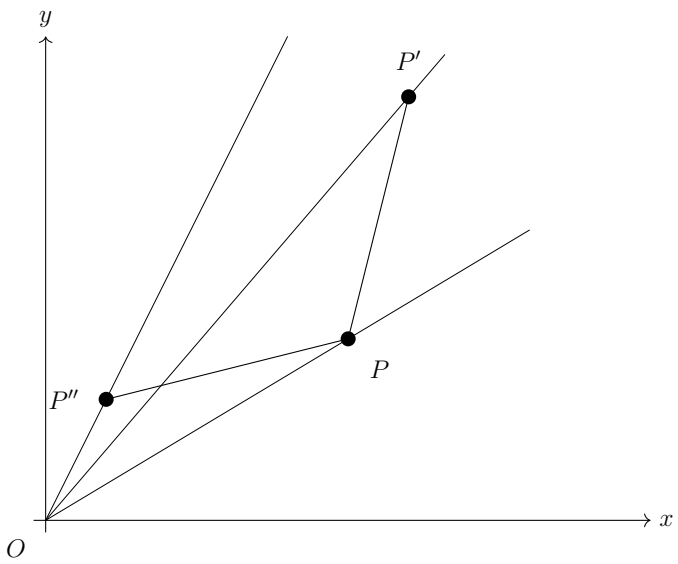


Figura 29.8

Punctul P este cel mai recent descoperit de raza de baleiere. Să presupunem că locul său, conform sortării după x sau y, ar fi între punctele anterioare P' și P''. Dar acum să observăm că, de fapt, puncul P' se află exact în situația punctului A din Figura 1. El nu va mai fi niciodată răspunsul la o interogare și îl putem șterge.

Aceasta este o veste bună. Punctul nou venit P se va așeza întotdeauna în vîrful unui stive. În prealabil, el elimină din stivă punctele care încalcă ordonarea polară / după x / după y.

29.5.11 Problema Fossil in the Ice (CodeChef)

enunț • sursă

Problema ne cere să găsim diametrul unui poligon.

Să fixăm un vîrf i al poligonului. Cum am putea găsi vîrful j cel mai depărtat de i? Remarcăm că, dacă pornim din i și parcurgem complet poligonul, distanța de i este bitonă: întîi crește, atinge maximul în vîrful j dorit, apoi scade. Deci am putea face o căutare binară.

Așadar, o primă soluție ar fi să facem n căutări binare, cîte una pornind din fiecare vîrf i. Dar, dacă avem reflexul de la two pointers bine format, știm că j avansează pe măsură ce i avansează. Mai exact, odată ce găsim punctul cel mai depărtat de $P[0]$ în $\mathcal{O}(n)$, putem găsi restul punctelor cele mai depărtate de $P[1], P[2], \ldots$ în $\mathcal{O}(1)$ amortizat.

29.5.12 Problema Rubarba (Infoarena)

enunț • sursă

Aceasta este exact problema dreptunghiului de arie minimă care cuprinde o mulțime de puncte.

Începem prin a calcula înfășurătoarea convexă. Acum, facem două observații:

1. Fiecare latură a dreptunghiului va conține cel puțin un vîrf. Demonstrația este banală: dacă o latură nu ar conține niciun vîrf, am putea restrînge dreptunghiul din acea direcție.
2. Există o latură a dreptunghiului care conține o latură a poligonului. Această demonstrație este netrivială, dar articolul original demonstrează că, în caz contrar, putem roti dreptunghiul pînă cînd se sprijină pe o latură a poligonului, iar aria dreptunghiului va scădea.

De aici, raționamentul seamănă cu cel anterior. Considerăm pe rînd fiecare latură-suport, între două vîrfuri consecutive i și $i+1$ (iar la final, circular, între $n-1$ și 0). Raportat la direcția acelei laturi, aflăm punctul cel mai depărtat. Am putea afla acel punct printr-o căutare binară, căci distanța de la vîrfuri la latura-suport este bitonă cînd parcurgem poligonul. Distanța maximă găsită ne dă înălțimea dreptunghiului. Prin procedee diferite, dar asemănătoare, putem căuta binar punctul cel mai din stînga și cel mai din dreapta relativ la latura-suport.

De aici, ne dăm seama că putem aplica metoda șublerului rotitor ca să trecem de la o latură-suport la următoarea în $\mathcal{O}(1)$ amortizat. Implementarea mea răspunde elegant (zisei eu cu modestie) la întrebările aparent complexe „Care este punctul cel mai din stînga/dreapta relativ la latura-suport?” și „Care este aria unui dreptunghi rotit?”.

Codul meu are un bug de eficiență, chiar în funcția `rotating_calipers()`. Îl vedeți?

calculat anterior sau nu.
Punctul r evoluează corect, avansînd de la poziția anterioară, dar punctele a și l sînt recalculate complet la fiecare iterație. Din fericire, testele nu pedepsesc această greșeală. Această versiune repară bugul eliminînd reinițializările lui a și l. Este nevoie de un `if` ca să decidem dacă l a fost

Partea IX

Grafuri

Capitolul 30

Arbori parțiali minimi

Definiție: Dat fiind un graf neorientat conex $G = (V, E)$ cu n noduri și m muchii, unde fiecare muchie are asociat un cost, un **arbore parțial** (engl. *spanning tree*) este o colecție de $n - 1$ muchii care conectează toate cele n noduri. Un **arbore parțial minim** (APM, engl. *minimum spanning tree, MST*) este un arbore parțial care minimizează suma costurilor celor $n - 1$ muchii.

30.1 Algoritmii lui Kruskal și Prim

Nu vom relua aici algoritmii, pe care sper că îi cunoașteți deja de la clasă. Ambii algoritmi sînt relativ intuitivi. Totuși, nu strică să ne familiarizăm și cu demonstrațiile. Ele sînt experimente de gîndire (ușoare) care ne ajută în probleme mai puțin directe.

Iată două demonstrații foarte elegante. Ele sînt inductive: arătăm că, la orice moment, colecția de muchii T pe care o mențin algoritmii este inclusă într-un APM. APM-ul ne este necunoscut, dar el există, iar algoritmii arată cum să extindem T cu o muchie astfel încît noua colecție T' să fie și ea inclusă într-un APM.

Vom introduce noțiunea de **tăietură** (engl. *cut*): O partiție a mulțimii de noduri V în două submulțimi X și Y. Spunem că o muchie **traversează tăietura** dacă are un capăt în X și unul în Y. Spunem că o tăietură **respectă** o mulțime de muchii A dacă nicio muchie din A nu traversează tăietura.

Cu această terminologie, vom arăta principiul comun al celor doi algoritmi. Nu pare, dar există unul! Ambii algoritmi construiesc un set de muchii A. La fiecare pas, ei definesc o tăietură care respectă mulțimea A, apoi caută și adaugă muchia minimă care traversează tăietura. Concret:

1. Algoritmul lui Prim definește tăietura ca fiind X = mulțimea de noduri conectate, Y = restul nodurilor. De aceea menține, pentru fiecare nod din Y, distanța minimă pînă la un nod din X.
2. Algoritmul lui Kruskal nu definește explicit tăietura. Orice tăietură este satisfăcătoare, atîta timp cît ea nu taie componentele conexe existente (pentru orice componentă conexă existentă, toate nodurile sînt de aceeași parte a tăieturii). De aceea algoritmul interzice

muchii între două noduri din aceeași componentă: acele muchii **nu** traversează tăietura.

Cormen (Teorema 21.1) demonstrează următoarea teoremă: Dacă mulțimea curentă A este inclusă într-un APM, și dacă alegem orice tăietură care respectă A, atunci muchia minimă e care traversează acea tăietură poate fi adăugată la A. Muchia e nu închide cicluri în A, iar $A \cup \{e\}$ continuă să fie inclusă într-un APM.

30.2 Proprietăți ale arborilor parțiali minimi

Pagina Wikipedia listează două proprietăți interesante și relativ evidente: proprietatea ciclului și proprietatea tăieturii.

Proprietatea ciclului spune că este necesar ca orice muchie din afara APM să fie cea mai scumpă de pe ciclul pe care îl închide în APM. Editorialul pentru problema Minimum Spanning Tree for Each Edge împinge aceasta cu un pas mai departe, spunînd că condiția este și suficientă: dacă alegem un arbore și dacă toate muchiile din afara acestui arbore au proprietatea ciclului, atunci arborele ales este APM. Editorialul numește asta „criteriul lui Tarjan". Demonstrația este constructivă: putem elimina din graf toate muchiile care încalcă proprietatea ciclului. Ceea ce rămîne este, în mod necesar, un APM.

Toți arborii parțiali minimi ai unui graf au aceeași distribuție de costuri ale muchiilor. Intuitiv, acest lucru se întîmplă deoarece algoritmul lui Kruskal va grupa, la sortare, muchiile de costuri egale.

30.3 Actualizarea APM

Această secțiune este interesantă teoretic, dar nu am întîlnit-o niciodată în probleme de olimpiadă.

Cum actualizăm un APM, T, cînd costul unei muchii se modifică?

- Dacă muchia este în T, iar costul ei scade, T rămîne APM.
- Dacă muchia nu este în T, iar costul ei crește, T rămîne APM.
- Dacă muchia nu este în T, iar costul ei scade, adăugăm muchia la T, ceea ce cauzează închiderea exact a unui ciclu, apoi eliminăm muchia maximă de pe acel ciclu.
  - Demonstrație: pornind de la algoritmul de bază (cu tăieturi). Fie $e = (u, v)$ muchia al cărei cost scade. Fie C ciclul închis de e în T. Fie $f = (x, y)$ muchia de cost maxim din C. Dacă $e = f$, am terminat. Altfel, $c(e) < c(f)$. Să ștergem muchia f din T. Rezultă o partiționare a grafului în două. Muchiile e și f (și posibil altele) sînt muchii care traversează tăietura. Muchia e este muchia minimă care traversează tăietura, căci, dacă ar exista o muchie mai mică g cu $c(g) < c(e) < c(f)$, atunci g ar fi făcut parte din T în locul lui f. Conform algoritmului, rezultă că $T - f + e$ este noul APM.
  - Același lucru îl facem și la adăugarea unei muchii: adăugarea este echivalentă cu scăderea costului de la ∞ la ceva finit.

- Dacă muchia este în T, iar costul ei crește, eliminăm muchia din T și o înlocuim cu cea mai ieftină muchie care reconectează T.
  - Demonstrația este identică: ia naștere o tăietură și selectăm cea mai ieftină muchie care traversează tăietura. Nu știu cît de eficient putem face acest lucru.

Cum actualizăm un APM, T, la adăugarea unui nod z? Nu putem pur și simplu să privim $(V, \{z\})$ ca pe o tăietură, deoarece z poate aduce cu el muchii foarte ușoare, ceea ce face ca T **să nu mai fie APM** pentru mulțimea sa de noduri. Putem folosi o parcurgere DFS specifică problemei, în $\mathcal{O}(n)$, sau putem rula algoritmii lui Kruskal și Prim pe mulțimea formată din T și muchiile nou adăugate de z. De ce?

30.4 Probleme

30.4.1 Problema Arbore parțial de cost minim (Infoarena)

enunț • surse

Problema este educațională. Vom citi sursele și vom explica algoritmii.

Algoritmul lui Kruskal rulează în $\mathcal{O}(m \log m)$ datorită sortării. Puteam face sortarea în $\mathcal{O}(m + c)$ dacă distribuiam muchiile în liste după cost, dar am decis să nu încarc programul.

Algoritmul lui Prim rulează în $O(m \log n)$. Într-adevăr, fiecare muchie evaluată poate duce la o îmbunătățire a distanței fiului, ceea ce necesită o inserare în coada de priorități. Remarcăm că, în această implementare, un nod u poate avea multiple copii în coadă, dacă distanța lui este optimizată de mai multe ori înainte ca u să fie selectat ca minim și adăugat la arbore.

Putem insista ca coada să conțină noduri distincte, dar nu este banal.

- CP Algorithms folosește un set. Cînd constatăm că distanța unui nod scade, îi cunoaștem vechea distanță d, deci putem căuta și șterge vechea pereche (u, d) din set.
- Putem folosi și un heap, dar trebuie să ni-l implementăm singuri. Fiecare nod menține și un pointer (un indice întreg) la poziția sa în heap. După modificarea distanței, acel nod din heap poate să urce sau să coboare, după caz.

30.4.2 Problema Autobuze3 (AGM 2015)

enunț • sursă

Și aceasta este o problemă relativ directă. Costul minim este dat de arborele parțial minim. Pentru trasarea drumurilor este necesar să parcurgem arborele din frunze spre rădăcină. Pentru a limita la 25 numărul de transferuri ale șoferilor, trebuie ca, atunci cînd un autobuz sosește din nodul fiu în nodul părinte, să-l comparăm cu autobuzul existent în nodul părinte și să transferăm autobuzul mai gol în cel mai plin (*small to large*).

Detalii de implementare

Am stat puțin pe gînduri dacă să folosesc algoritmul lui Kruskal sau pe al lui Prim. Voi ce părere aveți? Realitatea este că Kruskal pare mai greu de greșit și este mai scurt cu cîteva linii. În cazul de față, **poate** merita folosit Prim, care oferă pentru fiecare nod u pointerul la părinte (adică la nodul care l-a adăugat pe u la arbore). Sînt destul de sigur că acel cod ar cîștiga la memorie, căci nu mai este necesar DFS-ul. Dar în rest... pare mai greu de scris, iar Prim este ceva mai lent.

Pentru a reduce numărul de transferuri, meritau făcute două iterații în DFS: una pentru identificarea fiului heavy și una pentru transferul celorlalți fii, o singură dată. Dar, așa cum am învățat în capitolul 18, este mai ușor de codat, și suficient de eficient, să facem unificarea la revenirea din fiecare fiu. Încă rămîne valabil că fiecare șofer va fi transferat de cel mult $\log n$ ori.

Pentru reutilizarea `vector`-ilor între teste, îi ștergem cu `clear()`. Putem economisi niște memorie (de la 36 MB la 30 MB) dacă apelăm și `shrink_to_fit()` după `clear()`. Altfel, `clear()` nu garantează recuperarea memoriei.

30.4.3 Problema Rusuoaica (FMI No Stress 9 Warmup)

enunț • sursă

(Sper că știți cum se scrie corect *rusoaică*. Probabil scrierea greșită este doar o glumă între colegi.)

Problema este un experiment de gîndire pornind de la Kruskal. Putem opri algoritmul la costul A, cu semnificația că vindem toate muchiile mai scumpe decît A. Apoi, dacă rămînem cu k componente conexe, le putem interconecta cu $k-1$ muchii de cost A. Pentru simplitate, încă de la citire renunțăm la muchiile de cost mai mare decît A.

Ca detaliu de implementare, am ales să las structura de mulțimi disjuncte să țină și evidența numărului de arbori din pădure.

30.4.4 Problema MinOr Tree (Codeforces)

enunț • sursă

Problema este mai mult de idee; implementarea este elementară.

Putem încerca să sortăm crescător muchiile, în ideea să lăsăm algoritmul lui Kruskal să conecteze graful cu muchii ale căror biți sînt cît mai nesemnificativi. Dar garanția de optimalitate nu se mai aplică la OR la fel ca la sume. Odată ce Kruskal decide că este obligatoriu să folosească muchii cu bitul cel mai semnificativ (să spunem) 6, s-ar putea să fie optim să ștergem unele dintre muchiile anterioare, ca să evităm să folosim bitul 5.

În loc de asta, vom construi soluția bit cu bit. Să spunem că masca OR a tuturor costurilor ocupă 20 de biți.

- Încercăm să vedem dacă putem să conectăm graful doar cu muchii care **nu** folosesc bitul 19. Dacă da, ștergem acele muchii.

- Încercăm să vedem dacă putem să conectăm graful doar cu muchii care folosesc sau nu bitul 19, conform răspunsului anterior, dar **nu** folosesc bitul 18. Dacă da, ștergem acele muchii.
- Etc.

Ca optimizare, dacă un bit nu apare deloc în niciuna dintre muchii, putem omite complet acea iterație.

Ideea de APM ne trimite, de fapt, pe o pistă falsă. Problema este una de conexitate. De aceea nici nu trebuie să sortăm muchiile.

30.4.5 Problema 0-1 MST (Codeforces)

enunț • surse

Această problemă simpatică și un pic anapoda ne cere să găsim APM-ul într-un graf complet cu n noduri, în care toate muchiile au cost 0 în afară de m dintre ele, care au cost 1. Provocarea, desigur, este să facem asta în $\mathcal{O}(m+n)$ sau pe-acolo, fără a depinde de numărul real de muchii, care este $\mathcal{O}(n^2)$.

Problema este ușoară și are multiple soluții. Iată două dintre ele.

Soluția cu Kruskal

Putem adapta algoritmul lui Kruskal. Procesăm doar muchiile de 0 (cele care **nu apar** la intrare). Doar că trebuie să facem asta în $\mathcal{O}(m)$ (numărul de muchii care **apar** la intrare). Dacă rămînem cu k componente, atunci costul arborelui este $k-1$.

Pornim de la observația cu care ne-am mai întîlnit: nu contează ce muchii selectăm, ci cîte putem selecta fără a închide cicluri. Procesăm nodurile de la 1 la n și construim structura de mulțimi disjuncte ca de obicei. Ne permitem să petrecem în fiecare nod u timp $\mathcal{O}(\deg_1(u))$, unde $\deg_1(u)$ este numărul de muchii de cost 1 care pornesc din u (cele date la intrare).

Menținem pădurea de mulțimi disjuncte ca în algoritmul lui Kruskal, dar în plus reținem și mărimea fiecărei mulțimi. Pentru nodul curent u contorizăm cîte muchii de cost 1 se duc către fiecare mulțime disjunctă. Numim o mulțime **saturată** dacă în ea intră un număr de muchii de cost 1 egal cu mărimea ei. Dacă o mulțime nu este saturată, atunci există și muchii de cost 0 către ea (nu uitați că graful este complet). Deci, conform algoritmului lui Kruskal, unim mulțimea lui u cu toate mulțimile nesaturate.

Surprinzător, ne permitem să testăm fiecare mulțime din structură. Efortul pare să fie $\mathcal{O}(n)$ per nod, $\mathcal{O}(n^2)$ în total, dar realitatea este mai bună. Pentru un nod u, există cel mult $\deg_1(u)$ mulțimi saturate, deci efortul de a considera toate mulțimile saturate este $\mathcal{O}(m)$. Cît despre mulțimile nesaturate, acelea sînt imediat unificate și dispar din lista de componente, deci efortul global necesar pentru a le procesa este $\mathcal{O}(n \log^* n)$.

Soluția cu BFS

Există și o soluție frumoasă care folosește doar parcurgeri și conexitate, ca la numărarea componentelor conexe: din fiecare nod nevizitat pornim o parcurgere folosind doar muchiile de cost 0. Trebuie să avem grijă ca vizitarea fiecărui nod să coste $\mathcal{O}(\deg_1(u))$. Să denumim lista de muchii de cost 1 care pleacă din noul u **lista de nonadiacență** a lui u. Implementarea mea stochează listele de nonadiacență în cîte un `unordered_set` pentru fiecare nod u.

Apoi, menținem o listă L de noduri nevizitate. Cînd scoatem din coada BFS un nod u consultăm lista L și inserăm în coada BFS toate nodurile care nu apar în lista de nonadiacență a lui u (așadar, cele aflate la distanță 0 de u).

Poate părea că efortul este $\mathcal{O}(n^2)$, căci pentru fiecare nod vom consulta lista L care are $\mathcal{O}(n)$ noduri. Dar, în realitate, există două scenarii pentru fiecare element v din L:

1. v este în lista de adiacență a lui u, și deci rămîne în lista L. Dar v nu se va afla în această situație decît de $\deg(v)$ ori, deci efortul global este $\mathcal{O}(m)$.
2. v nu este în lista de adiacență a lui u. Atunci îl scoatem pe v din L și îl inserăm în coada BFS. Fiecare element poate fi eliminat doar o dată din L, deci efortul global este $\mathcal{O}(n)$.

De aceea, această soluție obține complexitatea $\mathcal{O}(m+n)$.

De amorul artei, am scris și o sursă optimizată care (1) folosește o coadă proprie și (2) folosește un singur vector în loc de `unordered_set` pentru a testa existența muchiilor.

30.4.6 Problema Minimum Spanning Tree for Each Edge (Codeforces)

enunț • sursă

O problemă practic identică este VS-Movies Group (finala IIOT 2020-2021).

Ideea teoretică

Iată o soluție care dă răspunsul corect, dar este prea lentă. Pentru a forța APM-ul să includă o muchie (u, v), setăm costul acelei muchii la 0. Alternativ, preinițializăm structura de mulțimi disjuncte (în algoritmul lui Kruskal) respectiv mulțimea de noduri incluse (în algoritmul lui Prim) ca să includem muchia (u, v). Dar nu ne permitem să calculăm un APM pentru fiecare muchie.

Soluția eficientă procedează astfel. Calculăm APM-ul o singură dată. Apoi, pentru fiecare muchie (u, v), aflăm un APM care include muchia:

1. Adăugăm muchia (u, v) la APM. Aceasta creează exact un ciclu.
2. Eliminăm ciclul ștergînd cea mai grea muchie de pe calea (u, v) din APM.

Să demonstrăm că această abordare este corectă. Fie $\mathcal{A}$ APM-ul nostru de cost C, fie c costul muchiei (u, v) și fie c_1 costul celei mai scumpe muchii de pe ciclul închis de (u, v). Atunci vom da răspunsul $C + c - c_1$.

Cînd ar putea fi acest răspuns greșit? Să presupunem că există un al doilea APM $\mathcal{A}'$, cu același cost C dar cu o structură foarte diferită, în care cea mai scumpă muchie de pe ciclul închis de (u, v) are un cost $c_2 > c_1$. Atunci am avea că $C + c - c_2 < C + c - c_1$. Să vedem unde este contradicția.

Observăm că muchia c_2 nu poate face parte din $\mathcal{A}$. Într-adevăr, dacă ar face parte, ea s-ar afla pe ciclul închis de (u, v) și am fi selectat-o pe ea în locul lui c_1. Dar atunci muchia c_2 închide un ciclu în $\mathcal{A}$. De aici apare contradicția. Din $\mathcal{A}$ știm că c_2 este cea mai scumpă muchie de pe acel ciclu (altfel am putea optimiza APM-ul $\mathcal{A}$). Dar, din proprietatea ciclului, știm că muchia c_2 nu poate face parte din niciun APM, deci nici din $\mathcal{A}'$.

Implementare cu LCA și *binary lifting*

Așadar, trebuie să răspundem la m interogări de maxime pe căi. Știm deja abordarea clasică, în $\mathcal{O}(\log n)$ per interogare: alegem o rădăcină pentru arbore, să zicem nodul 1, și precalculăm informații de LCA cu *binary lifting*. Fiecare pointer reține și valoarea maximă a unei muchii pe care o traversează.

Implementare cu mulțimi disjuncte modificate

Soluția mea împrumută o altă idee, pe care o vom detalia. Ea folosește o structură de date S care este „un fel de” *disjoint set forest* de complexitate $\mathcal{O}(\log n)$ per operație.

Concret, procesăm muchiile în ordine crescătoare a costului. Începem să unificăm componentele conexe conform algoritmului lui Kruskal. Dar nu folosim compresia căii, căci ne interesează ca S să păstreze o informație esențială, care s-ar pierde prin reorganizare. Pentru orice pereche de noduri deja conectate, u și v, dorim ca S să faciliteze găsirea muchiei celei mai scumpe de pe calea $u - v$.

Cînd procesăm muchia (u, v) de cost c, dacă u și v provin din componente diferite, găsim rădăcinile u' și v' ca de obicei. Apoi legăm u' de v' și notăm costul c. Aceasta garantează că, pentru a ajunge (în S) de la orice nod din componenta lui u' la orice nod din componenta lui v', trebuie să trecem prin muchia (u', v').

Dacă componenta lui u' este mai mare, legăm v' la u', altfel invers. Aceasta garantează că numărul de muchii de pe orice cale este logaritmic.

Cum folosim, concret, structura S ca să aflăm răspunsul pentru fiecare muchie? Dacă u și v provin din componente diferite, știm din algoritmul lui Kruskal că (u, v) face parte din APM, deci răspunsul pentru ea va fi costul APM-ului, odată ce îl determinăm.

Dacă, în schimb, procesăm o muchie (u, v) și determinăm că nodurile sînt deja conectate, atunci urcăm cu u și v prin S pînă cînd se întîlnesc, apoi returnăm muchia maximă întîlnită pe drum.

30.4.7 Problema Apm2 (Infoarena)

enunț • sursă

Problema este foarte similară cu cea anterioară (Minimum Spanning Tree for Each Edge). Dar merită reluată observația teoretică.

Intuiția ne spune să calculăm un APM. Apoi, pentru o interogare (u, v), răspunsul este $c - 1$, unde c este costul maxim de pe calea (u, v) din APM. Justificarea este că dorim să punem un cost cît mai mare pe (u, v), dar suficient de mic încît, la refacerea APM-ului, muchia (u, v) să rămînă în APM.

Cum demonstrăm asta? Nu putem demonstra doar pe APM-ul particular pe care l-a construit programul nostru, ci pe orice APM. Să studiem mai atent algoritmul lui Kruskal. Motivul pentru care pe o cale (u, v) există o muchie de cost maxim c este că, după procesarea muchiilor de cost $\leq c - 1$, componentele lui u și v nu erau încă conectate.

De aceea, dacă pentru interogarea (u, v) alegem costul $c - 1$, atunci algoritmul lui Kruskal ar alege garantat muchia. Detaliu la care vom reveni: algoritmul ar alege muchia indiferent unde ar fi ea sortată între alte muchii de cost $c - 1$.

Dacă punem costul muchiei $(u, v) = c$, atunci în mod evident algoritmul poate omite muchia (u, v), căci poate uni componentele folosind cealaltă muchie de cost c.

Implementarea mea folosește același pseudo-*disjoint set forest* cu timp logaritmic. Dar, ca și mai înainte, soluția cu LCA funcționează și ea perfect.

30.4.8 Problema Edges in MST (Codeforces)

enunț • sursă

Problema duce un pas mai departe experimentele de gîndire despre APM. Să considerăm că algoritmul lui Kruskal a procesat toate muchiile de cost mai mic decît c și a obținut niște componente conexe. Subliniem că partiționarea în componente nu depinde de sortarea pe care algoritmul lui Kruskal s-a întîmplat să o aleagă pentru muchii de costuri egale. Exemplu: dintr-un triunghi de muchii de cost 4, Kruskal va alege două. Cele trei noduri vor fi conectate indiferent de muchiile alese.

Atunci să considerăm graful componentelor, în care fiecare nod corespunde unei componente conexe, și să considerăm „pachetul" de muchii de cost c.

- O muchie va apărea în **toate APM-urile** dacă este punte.
- Altfel, o muchie va apărea în **unele APM-uri** dacă unește două componente distincte.
- Altfel, o muchie unește două noduri din aceeași componentă și nu va apărea în **niciun APM.**

Vom defini noțiunea de **punte**, iar deocamdată vom lua fără demonstrație codul de găsire a punților. Vom reveni la el în capitolul 31.

Implementarea constă din algoritmul lui Kruskal + un DFS pentru fiecare bloc de muchii egale. Atenție la două detalii de implementare:

- Fiecare DFS pe un pachet de k muchii trebuie să fie $\mathcal{O}(k)$. Orice `for` neatent, în $\mathcal{O}(n)$, poate duce complexitatea totală la $\mathcal{O}(n^2)$ dacă toate muchiile au costuri distincte.
- Deși graful inițial este simplu, în graful componentelor pot exista muchii paralele (între perechi de noduri din aceleași componente). Este nevoie de puțină atenție în algoritmul lui Tarjan de găsire a punților. Este acceptabil (și necesar) să luăm în calcul toate muchiile care duc înapoi la părinte, doar nu fix pe cea pe care am venit. Sursa mea folosește indicele original al muchiei. CP Algorithms oferă o soluție mai generală.

30.4.9 Problema Envy (Codeforces)

enunț • sursă

Să pornim (din nou...) de la algoritmul lui Kruskal. Ne reamintim că, după procesarea muchiilor de cost $\leq c$, structura componentelor conexe va fi identică. Pot varia doar conexiunile interne din fiecare componentă. Așadar, putem procesa muchiile din fiecare interogare grupate după greutăți.

Cînd va avea o interogare răspunsul „nu”? Cînd unul dintre aceste grupuri include muchii care închid cicluri. De exemplu, dacă o interogare include (printre altele) trei muchii de greutate 10, iar la acel moment în algoritmul lui Kruskal muchiile unesc componentele $C_1 - C_2$, $C_2 - C_3$ și $C_3 - C_1$, interogarea are răspunsul „nu”.

Cum testăm existența acestor cicluri? Editorialul propune, pentru fiecare interogare și pentru fiecare grup de muchii de același cost c, cîte un DFS doar pe acele muchii **la momentul potrivit** în algoritmul lui Kruskal, adică înainte de a procesa muchiile de cost c.

Soluția mea folosește un *disjoint set forest* cu suport pentru *undo*: procesăm muchiile de cost c din interogare, verificăm dacă vreuna dintre ele închide un ciclu, apoi restaurăm pădurea la starea anterioară. Abia la finalul interogărilor pentru toate greutățile c le încorporăm (permanent) în pădure.

Aveam așteptări mari de la implementare, dar este cam lentă. 🥲

30.4.10 Problema Gift (Codeforces)

enunț • surse

Problema are „undeva” o componentă de APM, dar trebuie să construim în jurul ei. Iată două abordări.

Soluție bazată pe Kruskal repetat

O variantă ar fi să sortăm muchiile după valorile g. Apoi, în bucla exterioară a algoritmului, încercăm toate prefixele acestui vector. Pentru prefixul pînă la poziția i inclusiv, înseamnă că sîntem dispuși să cheltuim g_i aur și să renunțăm la muchiile de la $i+1$ la m. Cu primele i muchii încercăm să formăm un arbore parțial minim. Dintre toate variantele posibile, ne interesează cea

care minimizează costul maxim în argint. Așadar, în bucla interioară ordonăm muchiile din prefix după valorile s și rulăm algoritmul lui Kruskal.

Pentru implementare, eu am sortat muchiile după g și, separat, am ținut un vector de indirectare (o permutare) după valorile s. Pentru Kruskal, parcurgeam toate cele m muchii, dar le procesam doar pe cele care respectau limita maximă pentru g. Acest algoritm este $\mathcal{O}(m^2 \log^* m)$, prea lent. Dar l-am îmbunătățit succesiv astfel:

1. Am creat separat o listă de muchii „admisibile", la care adăugam muchii pe măsură ce limita pentru g creștea. Am menținut această listă sortată, inserînd naiv muchiile admisibile. Avantajul este că Kruskal nu mai iterează de fiecare dată prin toate cele m muchii.
2. Dacă algoritmul lui Kruskal examinează doar primele k muchii admisibile, am trunchiat lista la poziția k. Niciodată algoritmul lui Kruskal nu va mai avea nevoie de restul de muchii. Ne interesează doar muchiile admisibile viitoare, în măsura în care ele sînt ieftine.
3. Cel mai important, am aruncat din lista de muchii admisibile chiar și muchiile ieftine, dar pe care Kruskal le-a refuzat din cauză că închid cicluri. Nu greșim dacă le aruncăm. De exemplu, dacă din 5 muchii de cost 10 Kruskal alege doar 3, el va continua să le prefere pe aceleași și în iterațiile viitoare. Se poate ca o muchie mai ieftină, să zicem de cost 8, să facă inutile unele dintre acele muchii de cost 10. Dar în niciun caz nu va reactiva o muchie deja refuzată.

Aceasta înseamnă că lista de muchii admisibile va avea, la sfîrșitul fiecărei iterații, lungime cel mult $n - 1$! Acum să analizăm complexitatea:

- Inserarea naivă în această listă a tuturor celor m muchii costă $\mathcal{O}(mn)$.
- Rularea algoritmului lui Kruskal de m ori costă $\mathcal{O}(mn \log^* n)$.
- Sortarea inițială a muchiilor costă $\mathcal{O}(m \log m)$.

Așadar, complexitatea totală este $\mathcal{O}(mn \log^* n + m \log m)$.

Soluție bazată pe actualizarea APM

Să reluăm ideea că, pe măsură ce sîntem dispuși să plătim mai mult aur, lista muchiilor disponibile crește. Cu aceste muchii noi urmărim să construim un APM care minimizează costul în argint.

Soluția anterioară rulează complet de la zero algoritmul lui Kruskal pe măsură ce lista de muchii admisibile crește. Dar avem și opțiunea să construim un prim APM, pe care apoi să îl actualizăm la adăugarea fiecărei muchii. Știm să facem asta în teorie: la adăugarea muchiei (u, v), căutăm cea mai scumpă muchie e de pe calea $u \rightsquigarrow v$ din arbore. Dacă costul muchiei (u, v) este mai mic decît costul muchiei e, ștergem muchia e și adăugăm muchia (u, v).

Cum implementăm asta? Dacă încercăm o soluție logaritmică în n, dăm de greu (și nu sînt sigur că se poate). Dar să nu uităm că n este mic! Ne permitem o soluție în $\mathcal{O}(n)$, care este facilă folosind o parcurgere DFS.

În implementarea mea, am dorit să evit codul pentru construcția inițială a APM-ului. De aceea, am pornit cu un „APM" inițial vid, care știe să raporteze din cîte componente conexe este format

(inițial n). Luăm în calcul la răspunsul final doar valorile pentru g care duc la APM-uri conexe. Pentru a putea raporta costul maxim în argint, APM-ul menține un multiset al valorilor s de pe toate muchiile.

Complexitatea acestui algoritm este dominată de sortarea muchiilor și de cele m DFS-uri, așadar $\mathcal{O}(mn + m \log m)$.

30.4.11 Problema DFS Trees (Codeforces)

enunț • sursă

Aceasta este mai puțin o problemă de APM și mai mult un experiment de gîndire despre DFS.

Teoria

Să definim doi termeni specifici DFS-ului în grafuri neorientate. Cînd ajungem într-un nod u,

- **Muchiile de arbore** sînt acele muchii (u, v) pe care DFS-ul se reapelează din v.
- **Muchiile înapoi** sînt acele muchii (u, v) pe care DFS-ul nu se reapelează, căci v a fost vizitat anterior.

Acum, să observăm că APM-ul este unic, costurile fiind distincte. Îl putem calcula cu algoritmul lui Kruskal (este foarte convenabil, căci primim deja muchiile în ordinea crescătoare a costului). Apoi, întrebarea pentru fiecare nod de pornire $1 \leq x \leq n$ este dacă DFS-ul va folosi ca muchii de arbore exact cele $n - 1$ muchii din APM.

Să considerăm întrebarea echivalentă: cum știm dacă DFS-ul din x va evita cele $m - n + 1$ muchii care nu fac parte din APM? Să considerăm o astfel de muchie, $e = (u, v)$. Dacă e nu face parte din APM, înseamnă că închide un ciclu (u, v) în APM. Acum (pentru a suta oară în acest curs), să schimbăm modul de agregare a datelor. În loc să fixăm x și să luăm în calcul toate muchiile, să fixăm muchia e și să luăm în calcul toate nodurile de pornire x. Marcăm ca „rău” orice nod de pornire care vizitează muchia e ca muchie de arbore. După ce luăm în calcul toate muchiile, răspunsul va fi `0` pentru nodurile care au fost marcate ca „rele” cel puțin o dată, `1` pentru restul.

Am ajuns astfel la întrebarea: pentru o muchie $e = (u, v) \notin$ APM, ce noduri de pornire x vizitează e ca muchie de arbore? Fie C ciclul format de e cu lanțul $u - v$ din APM. Fie y primul nod descoperit de DFS dintre toate cele din C. Dacă $y = u$, atunci DFS-ul nu va pleca pe muchia e, care este scumpă, ci va pleca (la un moment dat) spre v. Tot lanțul $u - v$, inclusiv v, va deveni subarbore al lui u în DFS.

Aceasta este observația crucială despre DFS: odată ce marchează u ca vizitat și pornește pe lanțul care duce spre v, DFS-ul nu va mai reveni în u pînă nu termină de explorat toate nodurile noi, care includ tot lanțul $u - v$. Cîndva în acest proces DFS-ul va descoperi nodul v și muchia e și va constata că este muchie înapoi. Nu este garantat că DFS-ul va explora lanțul în ordinea din APM. Pot exista alte muchii și lanțuri care să încîlcească traseul. Dar este garantat că va explora tot lanțul și va ajunge la v.

Același argument se aplică și pentru $y = v$. Dacă în schimb y este alt nod de pe lanț, atunci se aplică argumentul invers. Fie f și g muchiile către cele două noduri vecine cu y în C. DFS-ul va porni pe una dintre ele, să zicem pe f. Tot ciclul va deveni subarbore al lui y prin f, iar muchia g va fi muchie înapoi. Aceasta este o greșeală, căci g este muchie din APM și DFS-ul o omite.

Implementarea

Așadar, pentru fiecare muchie $e = (u, v)$ din afara APM-ului, nodurile care o vor traversa în DFS, și care trebuie marcate ca „rele", sînt cele din care, pornind, descoperim ciclul C printr-un nod diferit de u și de v. Sună monstruos de implementat! 😨

În realitate, este suficient să considerăm doar APM-ul. Dacă „atîrnăm" acest APM ca pe o frînghie de nodurile u și v, atunci trebuie să marcăm ca „rele" toate nodurile dintre u și v și toți subarborii acelor noduri. În practică arborele este înrădăcinat, deci iau naștere două cazuri:

1. Dacă u este strămoș al lui v, atunci marcăm ca rău tot subarborele fiului w al lui u care pornește către v, cu excepția subarborelui lui v. Similar dacă v este strămoș al lui u.
2. Dacă u și v nu sînt în relația strămoș-descendent, atunci marcăm ca rău tot arborele cu excepția subarborilor lui u și v.

Putem face aceste operații cu vectori de diferențe pe arbore:

1. Pentru cazul (1), notăm +1 în w și -1 în v.
2. Pentru cazul (2), notăm +1 în rădăcină, -1 în u și -1 în v.
3. Apoi propagăm valorile în jos (fiecare nod calculează suma valorilor de la el la rădăcină).

La final, nodurile „rele" sînt cele care au valori pozitive, nodurile bune sînt cele care au valori 0.

Capitolul 31

Conectivitate

31.1 Sortarea topologică

Sortarea topologică a unui graf orientat înseamnă să-i enumerăm cele n noduri într-o ordine astfel încît toate muchiile să meargă doar de la stînga spre dreapta. Un exemplu intuitiv este graful de dependințe între noțiunile învățate la școală: geometria 2D este necesară pentru geometria 3D etc. Wikipedia enumeră multe alte exemple.

Un graf orientat admite o sortare topologică dacă și numai dacă este aciclic. Un astfel de graf se numește dag (engl. ***d**irected **a**cyclic **g**raph*). Condiția este necesară deoarece, dacă graful conține un ciclu, atunci orice ordine în care enumerăm nodurile ciclului în sortare va face ca muchia dintre ultimul nod și primul să meargă spre stînga. Condiția este și suficientă, iar demonstrația este constructivă.

Problema educațională Course Schedule ne cere exact să sortăm topologic un graf orientat sau să raportăm dacă el conține un ciclu.

Există doi algoritmi clasici, ambii în $\mathcal{O}(m + n)$. Ambii necesită cod similar de lung, cu viteze și consumuri de memorie similare. Este bine să fiți familiari cu ambii, pentru că unele probleme se mulează mai bine peste unul dintre algoritmi.

31.1.1 Algoritmul lui Kahn

Acest algoritm este practic un BFS. Inițializăm o coadă cu nodurile cu grad la intrare 0. Acestea pot fi enumerate primele în sortarea topologică, în orice ordine. Explorăm graful, procesînd în mod repetat primul nod din coadă. Procesarea înseamnă să tipărim nodul și să incrementăm cîte un contor pe fiecare succesor al nodului, cu semnificația că acelui succesor i-am satisfăcut o cerință. Cînd contorul unui nod devine egal cu gradul său la intrare, îl introducem și pe el în coadă.

Graful conține un ciclu dacă coada rămîne goală înainte să explorăm toate nodurile.

31.1.2 Algoritmul bazat pe DFS

Pornim un DFS dintr-un nod oarecare. La fiecare nivel recursiv al DFS-ului, la revenirea din descendenți adăugăm nodul curent la ordinea topologică. Dacă după încheierea DFS-ului mai există noduri nedescoperite, alegem la întîmplare unul dintre acestea și pornim un nou DFS. De remarcat că algoritmul produce o sortare topologică **inversă**.

Puzzle: De ce nu este acceptabil să procedăm invers: să adăugăm nodul la ordinea topologică la intrarea în DFS, ca să obținem o ordine topologică normală?

Graful conține un ciclu dacă DFS-ul încearcă, la orice moment, să viziteze un nod gri (un nod din stiva DFS).

31.2 Componente tare conexe

Într-un graf orientat $G = (V, E)$, o **componentă tare conexă** (abr. *CTC*, engl. *strongly connected component*, abr. *SCC*) este un subgraf maximal în care pentru orice pereche de noduri u și v, v este accesibil din u și invers. Notăm acest lucru cu $u \rightsquigarrow v$ și $v \rightsquigarrow u$.

Vom folosi această figură din *Introduction to Algorithms*.

Problema educațională Ctc ne cere exact să tipărim componentele tare conexe ale unui graf orientat.

31.2.1 Terminologie

Graful transpus se notează cu $G^T = (V, E^T)$ și se obține inversînd sensul tuturor muchiilor din E. Remarcăm că G^T are aceleași componente tare conexe ca și G: dacă $u \rightsquigarrow v$ și $v \rightsquigarrow u$ în G, parcurgînd căile în sens invers obținem că $v \rightsquigarrow u$ și $u \rightsquigarrow v$ în G^T.

Graful componentelor se notează cu G^{SCC} și este definit astfel:

- Contractăm fiecare componentă tare conexă, cu nodurile și cu muchiile ei, într-un singur nod.
- Unificăm toate muchiile dintre două componente într-o singură muchie.

Graful componentelor este un dag. Demonstrația este prin reducere la absurd: dacă în G^{SCC} ar exista un ciclu prin două noduri u și v, atunci în graful original ar exista căi între orice două noduri din componentele tare conexe corespunzătoare lui u și v, deci cele două componente ar fi una și aceeași.

Generalizăm **timpii de descoperire și de părăsire** a unui nod ($t_{in}[u]$ și $t_{out}[u]$) la submulțimi de noduri. Pentru $U \subseteq V$, definim:

- $t_{in}[U] = \min_{u \in U} t_{in}[u]$
- $t_{out}[U] = \max_{u \in U} t_{out}[u]$

Vom studia doi algoritmi, comparabili ca viteză. Ca și în cazul sortării topologice, vă încurajez să fiți familiari cu ambii.

31.2.2 Algoritmul lui Kosaraju

Acest algoritm face două parcurgeri DFS pentru a găsi componentele tare conexe:

1. Face o parcurgere DFS în G și reține $t_{out}[u]$ pentru fiecare nod u.
2. Calculează G^T.
3. Face o parcurgere DFS în G^T, considerînd nodurile în ordinea descrescătoare a lui t_{out}.
4. Fiecare submulțime găsită de DFS-ul de la pasul 3 este o componentă tare conexă.

De ce funcționează acest algoritm? Să studiem relația între valorile lui t_{out} (în graful original) și graful componentelor. Fie două componente C și C' între care există o muchie (u, v) cu $u \in C$ și $v \in C'$. Atunci putem demonstra că $t_{out}[C] > t_{out}[C']$. Într-adevăr, există două cazuri:

1. Dacă C este descoperită prima, fie $x \in C$ primul nod vizitat. Toate nodurile din C și din C' devin descendenți ai lui x în arborele DFS, deci nodul x va fi ultimul părăsit. Rezultă că $t_{out}[C] = t_{out}[x] > t_{out}[C']$.
2. Dacă C' este descoperită prima, fie $y \in C'$ primul nod vizitat. Toate nodurile din C' devin descendenți ai lui y în arborele DFS, dar nu și cele din C. Componenta C' va fi vizitată complet, apoi DFS-ul va reveni la bucla principală și la un moment viitor va vizita și componenta C. Rezultă că $t_{out}[C] > t_{out}[C']$.

În graful transpus G^T avem relația inversă: pentru orice muchie $(u, v) \in E^T$ cu $u \in C$ și $v \in C'$ vom avea $t_{out}[C] < t_{out}[C']$. **Atenție**: valorile t_{out} sînt cele calculate în graful original.

De aici rezultă corectitudinea algoritmului. Pasul 3 începe cu nodul u care maximizează $t_{out}[u]$. Acel DFS va explora o componentă C (în G^T) din care nu vor exista muchii spre alte componente. Cu alte cuvinte, DFS-ul nu se va revărsa în nicio altă componentă, ci va tipări strict nodurile din C. Apoi va alege alt nod de pornire dintr-o altă componentă C' care nu avea muchii spre alte componente (decît cel mult spre C, care a fost deja tipărită). Prin inducție, la orice moment cînd bucla principală va apela algoritmul DFS recursiv, algoritmul va tipări exact o componentă tare conexă.

Nu întîmplător, acest algoritm seamănă (un pic) cu o sortare topologică. Într-adevăr, ordinea în care componentele sînt tipărite este o sortare topologică a grafului G^{SCC} sau, alternativ, o sortare invers topologică a grafului G^{SCC^T}.

- Adevărat sau fals? Putem ca, la pasul 3, să parcurgem tot graful original, dar în ordinea crescătoare a valorilor t_{out}? Dați o demonstrație sau un contraexemplu. Indiciu: probabil nu folosim de 50 de ani un algoritm mai complicat decît trebuie. 🙂

Iată și un detaliu de implementare. Multe programe de pe Internet (CP Algorithms, Topcoder, GeeksforGeeks) țin în memorie două grafuri cu cod de genul:

```
while (m--) {
```

```
  cin >> a >> b;
  g[a].adj.push_back(b);
  g[b].rev_adj.push_back(a);
}
```

Această abordare folosește $\mathcal{O}(M + N)$ memorie suplimentară (N vectori cu un total de M elemente). Putem inversa listele de adiacență în $\mathcal{O}(M)$, atunci cînd devine necesar. Folosim doar $\mathcal{O}(N)$ memorie suplimentară pentru capetele de listă:

```
for (int u = 1; u <= n; u++) {
  for (int v: g[u].adj) {
    g[v].rev_adj.push_back(u);
  }
  g[u].adj.resize(0);
}
```

31.2.3 Algoritmul lui Tarjan

Acest algoritm face o singură parcurgere DFS. Pentru fiecare nod u stocăm două valori:

- $d[u]$ este adîncimea nodului
- $l[u]$ (*lowpoint*) este adîncimea minimă la care poate urca orice muchie înapoi din subarborele lui u.

Observația crucială este că, dacă niciun nod din subarborele lui u nu poate urca mai sus decît u, adică dacă $l[u] = d[u]$, atunci

- u este punctul de intrare într-o CTC;
- muchia de la părintele din DFS al lui u la u este o muchie între două CTC din G^{SCC}.

Pentru enumerarea efectivă a nodurilor din fiecare componentă, punem nodurile într-o stivă pe măsură ce le descoperim, dar **nu le scoatem din stivă** la revenirea din DFS. În schimb, cînd determinăm că un nod u este punctul de intrare într-o CTC, atunci scoatem toate nodurile din stivă, pînă la u inclusiv. Ele formează o CTC și le putem procesa după cum cere problema (de exemplu, le putem eticheta cu aceeași etichetă numerică).

Acest algoritm generează componentele într-o ordine invers topologică a grafului G^{SCC}.

31.3 Componente biconexe, punți, puncte de articulație

Ne îndreptăm acum atenția către probleme similare de conexitate în grafuri neorientate. Vom urmări exemplul propus în Cormen, fig. 20.10.

31.3.1 Definiții

- Un **punct de articulație** este un nod prin a cărui eliminare numărul de componente conexe crește.
- O **punte** este o muchie prin a cărei eliminare numărul de componente conexe crește.
- Un graf este **biconex** dacă, prin eliminarea oricărui nod, graful rămîne conex. Echivalent, un graf biconex este un graf fără puncte de articulație.
- O **componentă biconexă** este un subgraf biconex maximal.

Ultima definiție de mai sus este de pe Wikipedia. Cormen propune o definiție alternativă: o componentă biconexă este un set maximal de muchii astfel încît pentru orice pereche de muchii există un ciclu simplu care trece prin ambele. Diferența apare în cazul punților: o punte și nodurile de la capetele ei formează o componentă biconexă conform definiției de mai sus, dar nu și conform definiției din Cormen.

Conform Wikipedia, un punct de articulație poate face parte din mai multe componente biconexe. De aceea, Wikipedia definește componentele biconexe referitor la muchii, nu la vîrfuri. Mai exact, componentele biconexe sînt o partiție a mulțimii de muchii E.

Nu trebuie să vă împiedicați de aceste diferențe. Cîtă vreme știți ce doriți să codați, vă va fi ușor.

În problemele de astăzi vom întîlni și conceptul de **graf k-conex pe muchii** ca fiind un graf neorientat care rămîne conex dacă eliminăm mai puțin de k muchii. De exemplu, un graf 1-conex este pur și simplu un graf conex, iar un graf 2-conex este un graf fără punți.

31.3.2 Puzzles

- Adevărat sau fals? O muchie (u, v) aflată între două puncte de articulație este punte.
- Adevărat sau fals? Capetele oricărei punți sînt puncte de articulație.
- Adevărat sau fals? Un punct de articulație are cel puțin o muchie adiacentă care este punte.

31.3.3 Algoritmul Hopcroft-Tarjan

Este remarcabil că putem calcula toate aceste informații printr-o singură parcurgere DFS. Ca și la aflarea componentelor tare conexe, pentru fiecare nod u stocăm valorile $d[u]$ și $l[u]$. Atunci:

- Rădăcina parcurgerii este punct de articulație dacă și numai dacă are doi sau mai mulți fii.
- Un nod u diferit de rădăcină este punct de articulație dacă și numai dacă are cel puțin un fiu v pentru care $l[v] \geq d[u]$. Cu alte cuvinte, un fiu care nu poate accede la restul grafului dincolo de u.
- O muchie (u, v) vizitată din u spre v este punte dacă $l[v] > d[u]$ sau, echivalent, dacă $l[v] = d[v]$. Cu alte cuvinte, v nu poate ajunge la u decît prin muchia (u, v).

Ca și la aflarea CTC, pentru partiționarea muchiilor în componente biconexe, le vom pune într-o stivă cînd le descoperim. La revenirea din recursivitate, dacă nodul curent u este punct de

articulație, el îi datorează asta unui fiu v. Scoatem din stivă toate muchiile pînă la (u, v) inclusiv; ele formează o componentă biconexă.

Iată mai jos codul. El calculează toate proprietățile. Dacă problema nu le cere, desigur, codul se scurtează. Bunăoară, stiva nu este necesară decît pentru componentele biconexe.

```
// Stiva de la (u, v) pînă la vîrf formează o CBC.
void print_bcc(int u, int v) {
  printf("CBC:");
  do {
    --ss;
    printf(" %d-%d", st[ss].u, st[ss].v);
  } while (st[ss].u != u || st[ss].v != v);
  printf("\n");
}

void dfs(int u, int parent) {
  bool is_ap = false;
  int num_children = 0;
  nd[u].d = nd[u].low = ++time;

  for (int v: nd[u].adj) {
    if (!nd[v].d) {

      // Vizitează fiul și vezi cît poate urca.
      num_children++;
      st[ss++] = { u, v };
      dfs(v, u);
      nd[u].low = min(nd[u].low, nd[v].low);

      // Dacă v nu poate ajunge la u (sau mai sus), atunci (u,v) este punte.
      // În funcție de definiție, punțile pot sau nu să fie CBC de sine stătătoare.
      if (nd[v].low > nd[u].d) {
        printf("punte: %d-%d\n", u, v);
      }

      // Dacă v nu poate ajunge mai sus de u, atunci (1) o CBC începe la v și
      // (2) fie u este rădăcina, fie este punct de articulație.
      if (nd[v].low >= nd[u].d) {
        is_ap = true;
        print_bcc(u, v);
      }

    } else if (v != parent) {

      // Muchie înapoi -- asigură-te că nu o traversăm mergînd înainte.
      if (nd[v].d < nd[u].d) {
        nd[u].low = min(nd[u].low, nd[v].d);
        st[ss++] = { u, v };
```

```
      }

    }
  }

  // Rădăcina este punct de articulație dacă are copii multipli.
  // Alte noduri sînt puncte de articulație dacă au fost marcați de vreunul dintre copii.
  if ((!parent && (num_children > 1)) || (parent && is_ap)) {
    printf("PA: %d\n", u);
  }
}
```

Întrebări despre cod:

- De ce nu este nevoie să mai apelăm `print_bcc()` o dată la final?
- De ce tipărim componenta cînd aflăm că u este punct de articulație și nu cînd aflăm că (u, v) este punte?
- De ce optimizăm $l[u]$ o dată cu $l[v]$ și o dată cu $d[v]$?
- De ce ne asigurăm că $d[v] < d[u]$ pentru muchiile „înapoi”?

31.4 Probleme

31.4.1 Problema Course Schedule (CSES)

enunț • surse

Problema este educațională de sortare topologică. Includ surse cu cei doi algoritmi studiați.

31.4.2 Problema Book (Codeforces)

enunț • surse

Problema se duce destul de direct spre sortarea topologică. Observăm că, dacă vrem să învățăm un capitol v după un capitol u, atunci le putem învăța în aceeași trecere dacă $u < v$, altfel este nevoie de o trecere în plus. Problema se poate formaliza ca un graf orientat unde costul unei muchii (u, v) este 0 dacă $u < v$ sau 1 dacă $u > v$. Astfel, trebuie:

- Să tipărim -1 dacă graful nu poate fi sortat topologic (adică dacă conține vreun ciclu).
- Altfel, să tipărim costul maxim al unei căi.

Problema se rezolvă cu sortare topologică și o valoare pentru fiecare nod u, care arată numărul de treceri necesar pentru a învăța capitolul u. Valoarea lui u se calculează în funcție de valorile succesorilor.

Pentru completitudine, am implementat sortarea topologică și cu DFS, și cu BFS.

31.4.3 Problema Componente tare conexe (Infoarena)

enunț • surse

Această problemă este educațională, de componente tare conexe.

Pentru algoritmul lui Kosaraju, subliniez că nu este nevoie de calculul explicit al timpilor t_{out}. Ce ne interesează este ca în a doua parcurgere să procesăm nodurile descrescător după t_{out}. Așadar, este suficient ca, în prima parcurgere, să colectăm fiecare nod într-un vector chiar înainte de a părăsi DFS-ul (adică în locul unde am incrementa acel contor global `time`).

De amorul artei, includ și o sursă pentru algoritmul lui Kosaraju cu structuri de date proprii. Ea transpune graful fără memorie suplimentară, este de peste două ori mai rapidă și consumă de 4 ori mai puțină memorie decît implementarea cu STL.

31.4.4 Problema Mr. Kitayuta's Technology (Codeforces)

enunț • sursă

Problema este mai mult de idee, fiind greu de încadrat la „ceva": este o problemă *core* de grafuri. O putem rezuma astfel: dîndu-se n noduri izolate și m perechi ordonate de noduri, să se afle numărul minim de arce care trebuie trasate astfel încît pentru fiecare pereche (u, v) să existe o cale de la u la v.

Prin analogie cu componentele tare conexe, o **componentă slab conexă** (CSC) este un subgraf maximal astfel încît orice nod v să fie accesibil din orice nod u dacă ignorăm sensurile arcelor.

Fie G graful cerințelor date la intrare. Observăm că putem rezolva problema individual pe fiecare CSC din G, căci nu avem de ce să tragem vreo muchie între două CSC diferite. Acum, să considerăm o componentă cu k noduri. Iau naștere două cazuri.

1. Dacă componenta **nu conține** cicluri orientate, atunci ea este un dag. Fiind un dag, ea admite o sortare topologică S. Putem satisface toate cerințele componentei dacă unim cele k noduri cu $k-1$ muchii, două cîte două în ordinea din S. Putem demonstra că acest rezultat ($k-1$ muchii) este și optim. Să presupunem că ar exista o soluție care satisface toate cerințele componentei cu mai puțin de $k-1$ muchii. Aceste muchii nu ar putea conecta (slab) toate cele k noduri, deci există o tăietură a nodurilor componentei $A \cup B$ astfel încît nicio muchie a soluției să nu traverseze tăietura. Pe de altă parte, există în mod sigur o cerință care traversează tăietura, întrucît componenta este slab conexă. Acea cerință ar rămîne nesatisfăcută.
2. Dacă componenta **conține** un ciclu orientat, C, atunci putem oferi o soluție cu k muchii: conectăm toate nodurile într-un ciclu, formînd o componentă tare conexă. Din nou, putem demonstra că această soluție este optimă. Să presupunem că ar exista o soluție care satisface toate cerințele componentei cu doar $k-1$ muchii. Prin același argument de mai sus, pentru a păstra conexitatea, cele $k-1$ muchii trebuie să formeze un arbore (ignorînd orientarea). Dar atunci ele nu vor putea satisface toate cerințele de pe ciclul C.

31.4.5 Problema Ralph and Mushrooms (Codeforces)

enunț • sursă

Observăm că, în cadrul unei CTC, putem parcurge toate muchiile de o infinitate de ori, deci vom aduna profitul maxim de pe fiecare muchie. O subproblemă interesantă este: ce profit putem obține pe o muchie cu x ciuperci? Această funcție este OEIS A060432. Am încercat să găsesc o formulă în $\mathcal{O}(1)$ fără radicali, dar nu am reușit.

Odată ce părăsim o CTC, putem folosi muchia de plecare o singură dată, căci prin definiție nu mai putem reveni pe ea. Apoi putem consuma integral noua componentă etc. Rezultă că putem construi G^{SCC}, care este un dag, în care valoarea fiecărui nod este profitul maxim al muchiilor din toate nodurile din componenta asociată, iar valoarea fiecărei muchii este numărul de ciuperci din graful inițial. În acest nod trebuie să aflăm profitul maxim.

Aceasta este o problemă clasică de sortare topologică + programare dinamică. Vezi de exemplu:

- CSES 1680 Longest Flight Route (cea mai lungă cale într-un dag)
- CSES 1681 Game Routes (numărul de căi într-un dag)
- CSES 1686 Coin Collector (foarte asemănătoare cu problema curentă).

Soluția teoretică este să parcurgem componentele în ordine invers topologică, folosind orice algoritm de CTC. Fie $w[C]$ numărul total de ciuperci pe care le putem culege fără a părăsi componenta C. Atunci profitul maxim plecînd dintr-o componentă C este

$$profit[C] = w[C] + \max_{(u,v)}\{mushrooms[(u,v)] + profit[D]\}$$

unde D este o altă componentă tare conexă, iar (u,v) este o muchie cu $u \in C$ și $v \in D$.

Putem implementa explicit acest algoritm, construind G^{SCC}. De amorul artei, putem însă rezolva problema cu o singură parcurgere DFS! Sursa anexată oferă o astfel de implementare.

În majoritatea problemelor întîlnite, nu este strict necesar să construiți graful condensat, deși sînt de acord că implementarea cu o singură parcurgere poate fi alunecoasă.

31.4.6 Problema Bertown Roads (Codeforces)

enunț • sursă

Problema cere să orientăm muchiile unui graf neorientat conex astfel încît să obținem un graf tare conex. O problemă identică este CSES 2177. O observație relativ simplă este că, dacă graful neorientat are punți, atunci problema nu are soluție. Dacă orientăm muchia (u,v) ca $u \to v$, în graful rezultat nu va exista nicio cale de la v la u.

Dacă graful neorientat nu are punți, problema are întotdeauna soluție. Interesant este că implementarea este incredibil de simplă. Facem o parcurgere DFS și orientăm muchiile în sensul în

care le vizitează parcurgerea. Dintr-un nod u, pentru un vecin v, orientăm muchia ca $u \to v$ indiferent dacă este muchie de arbore sau muchie înapoi.

Trebuie doar să avem grijă să nu parcurgem aceeași muchie de două ori. Mai exact, muchia u, v va apărea în reprezentarea uzuală de două ori:

- nodul v în lista de adiacență a lui u;
- nodul u în lista de adiacență a lui v.

Dacă stăm în nodul u, iar nodul v este deja vizitat, atunci:

- Dacă $d[v] < d[u]$, atunci v este strămoș al lui u, muchia (u, v) este muchie înapoi și o vom orienta în sensul $u \to v$.
- Dacă $d[v] > d[u]$, atunci v este descendent al lui u, muchia (u, v) a fost deja vizitată și trebuie ca de această dată să o ignorăm.

31.4.7 Problema Tourist Reform (Codeforces)

enunț • sursă

Problema este din aceeași sferă cu precedenta. Trebuie să orientăm un graf neorientat pentru a maximiza accesibilitatea minimă, adică să maximizăm numărul minim de noduri accesibile din orice punct de pornire.

Știm de la problemele anterioare că, pentru o CTC, ne putem plimba la infinit printre nodurile ei. Interesant este cum să orientăm punțile din graful original. Fie X componenta 2-conexă (așadar, componenta mărginită de punți) din graful original cu număr maxim de noduri. Presupunem că X este unică, dar raționamentul este similar și dacă nu este. X va deveni o CTC în graful orientat. Rezultă că răspunsul este cel puțin $|X|$: putem orienta toate punțile înspre X, astfel că orice nod din X va putea vizita exact $|X|$ noduri, iar orice nod din afara lui X va putea vizita mai mult de $|X|$ noduri.

Acest răspuns este și optim. Să presupunem că orientăm orice punte astfel încît să se îndepărteze de X. Atunci ea delimitează din graful orientat o zonă în care toate CTC au mai puțin de $|X|$ noduri. Întrucît graful G^{SCC} este un dag, va exista în el o componentă Y din care nu iese nicio muchie. Pornind din acea componentă vom putea ajunge doar la $|Y| < |X|$ noduri.

Și această implementare necesită o singură parcurgere Tarjan. Mai mult, nu avem nevoie de stivă. Ne pasă doar **cîte** noduri are o CTC în clipa în care o identificăm, iar pentru aceasta este suficient să ținem un contor, nu toată stiva.

31.4.8 Problema We Need More Bosses (Codeforces)

enunț • sursă

Problema ne cere să calculăm diametrul maxim (ca număr de muchii) în arborele componentelor 2-conexe asociat unui arbore neorientat.

Așa cum am mai văzut, putem proceda incremental:

1. Calculăm punțile.
2. Condensăm fiecare componentă 2-conexă într-un nod și obținem arborele.
3. Calculăm diametrul arborelui.

Dar putem rezolva și această problemă cu un singur DFS. Aici este util să cunoașteți algoritmul de aflare a diametrului unui arbore cu un singur DFS, nu cu două. Pentru un nod u, diametrul **subarborelui** lui u poate fi de două feluri:

1. Fie trece prin u, caz în care u are nevoie să cunoască suma celor mai lungi două drumuri de la sine la frunze, care pornesc prin doi fii diferiți.
2. Fie nu trece prin u, caz în care u are nevoie să cunoască maximul diametrelor din subarborii fiilor.

Peste această recurență rămîne doar să adăugăm algoritmul lui Tarjan pentru găsirea punților. Dacă fiecare nod u își calculează distanța maximă, în număr de punți, pînă la orice frunză, atunci valoarea pentru u va proveni din maximul valorilor fiilor lui u. Nodul u adaugă 1 la valoarea returnată de un fiu v dacă muchia (u, v) este punte, altfel o preia ca atare.

31.4.9 Problema Forbidden Cities (CSES)

enunț • sursă

Întrebarea teoretică este, pentru fiecare triplet $\langle a, b, c\rangle$: Dacă elimin nodul c, mai rămîn a și b conectate? Intuiția ne împinge spre puncte de articulație. Răspunsul este: nu, dacă c este punct de articulație și dacă a și b cad în componente biconexe diferite raportat la c.

O variantă ar fi, așadar:

- Să construim graful condensat al componentelor biconexe, care este un arbore.
- Să construim LCA pe acest arbore.
- Să verificăm, pentru fiecare triplet $\langle a, b, c\rangle$ în care c este punct de articulație, dacă c se află pe calea a, b.

Mie îmi sună monstruos. 😨 Am preferat să investesc puțin timp într-o soluție... cu o singură parcurgere! Ideea de bază este: dacă un nod c observă că are un fiu d care nu poate urca mai sus decît c, atunci nodul c este punct de articulație, iar subarborele lui d nu poate comunica cu restul grafului decît prin c. Dacă există vreun triplet $\langle a, b, c\rangle$ astfel încît exact unul dintre a și b se află în subarborele lui d, atunci răspunsul pentru $\langle a, b, c\rangle$ este „nu”.

Pentru implementare, am extins algoritmul lui Tarjan pentru găsirea punctelor de articulație cu următorul sistem de notificări. Observați că ocazional schimb numele nodului. Fac aceasta ca să clarific perspectiva din care se întîmplă lucrurile, dar rutina DFS este una singură, relativ scurtă.

- La intrarea în nodul c, activăm o bifă prin care nodul spune „vreau să fiu notificat ori de cîte ori este vizitat vreun nod a sau b din tripletele pentru care eu sînt centrul”.

- La ieșirea din nodul c, dezactivăm această bifă. În particular, dacă a și b se regăsesc în afara subarborelui lui c, atunci c nu blochează calea $a \rightsquigarrow b$.
- Tot la ieșirea dintr-un nod, să-i zicem de această dată a, trecem prin toate tripletele în care a este extremitate și notificăm c-urile acestor triplete, dacă ele sînt în prezent abonate la notificări.
- Revenind în nodul c, după fiecare fiu d, și dacă c constată că este punct critic pentru d, atunci c trece prin lista de triplete care l-au notificat. Dacă pentru orice triplet $\langle a, b, c\rangle$ exact unul dintre a și b este descendent al lui d, atunci marcăm tripletul cu răspunsul „nu”.
- Indiferent dacă c este sau nu punct critic pentru d, c își golește lista de notificări după fiecare fiu. Este important ca, dacă a și b sînt accesibile prin doi fii diferiți, să le tratăm separat.

31.4.10 Problema Case of Computer Network (Codeforces)

enunț • sursă

Dacă am vedea această problemă „de la zero”, probabil nu am avea șanse să o rezolvăm. Dar, după cele anterioare, ea devine abordabilă. Din nou, trebuie să orientăm muchiile unui graf, inițial neorientat, asigurîndu-ne că în graful orientat există q căi impuse, de forma (u, v). Problema este rezonabilă în sensul că ne cere doar răspunsul da/nu, nu și orientarea efectivă.

Ca și mai înainte, putem transforma o componentă 2-conexă într-o componentă tare conexă, în care putem călători între orice două noduri. Mai rămîne să orientăm punțile. Să observăm că, dacă există o punte și două mesaje care au nevoie să traverseze acea punte în sensuri diferite, atunci problema nu are soluție.

Un mod de a implementa această observație este să construim arborele componentelor 2-conexe, în care muchiile sînt punți din graful original. Înrădăcinăm acest arbore în nodul 1. Raportat la un nod u și la muchia sa de intrare e, iau naștere patru tipuri de mesaje:

1. Mesaje care încep și se termină în subarborele lui u inclusiv.
2. Mesaje care încep și se termină în afara subarborelui lui u.
3. Mesaje care încep în subarborele lui u și se termină în afara lui.
4. Mesaje care încep în afara subarborelui lui u și se termină în el.

Mesajele de tipurile 1 sau 2 nu limitează în niciun fel orientarea lui e. Dacă avem doar mesaje de tipul 3, orientăm e dinspre u înspre rădăcină. Dacă avem doar mesaje de tipul 4, orientăm e dinspre rădăcină înspre u. Iar dacă avem mesaje de ambele tipuri (3 și 4), atunci problema nu are soluție.

Rezultă că putem ține evidența mesajelor de tip 3 și 4 în postordine. În plus, avem nevoie de informații LCA ca să putem șterge mesajele din evidență cînd ajungem la strămoșul lor comun. Din nou, putem implementa asta cu cărămizile cunoscute: graful componentelor, LCA cu $\log n$ pointeri per nod 😉 etc.

Dar și aici ne putem descurca cu o singură parcurgere. Dacă tot facem un DFS, ar fi păcat să

nu îl folosim și pentru a afla LCA-urile mesajelor. Deci combinăm algoritmul lui Tarjan pentru aflarea punților cu algoritmul lui Tarjan pentru LCA. 🤣

Un aspect care poate părea straniu este că noi operăm pe un graf, pe cînd algoritmul de LCA funcționează pe arbori. Dar ne putem adapta, cu condiția să lăsăm toate deciziile în seama nodurilor-șef ale componentelor 2-conexe (cele aflate imediat sub o punte). Celelalte noduri dintr-o componentă nu fac decît să propage contoarele către șef. Tot fiindcă operăm pe un graf, este posibil să ajungem într-un descendent de mai multe ori. Este important ca, la propagarea contoarelor spre părinte, să le ștergem din fiu, ca să nu avem surpriza că le-am propagat de mai multe ori.

31.4.11 Problema Dog Trick Competition 2 (IIOT 2023-2024 runda 4)

enunț • sursă

În esență, problema ne dă un graf orientat și un șir T de noduri și ne cere să vizităm, în ordine, un prefix cît mai lung al lui T, în ordinea $T_1, T_2, \ldots$, eventual folosind și alte noduri intermediare.

Faptul că sîntem la lecția de tare conexitate ar trebui să fie un indiciu. 🙂 Să observăm că putem rămîne oricît este nevoie în CTC-ul nodului T_1, vizitînd cît de mult se poate din T. Fie T_a primul nod nevizitat astfel. Dacă există un drum de la componenta lui T_1 la componenta lui T_a, putem continua traseul. Vizităm în continuare nodurile din T aflate în componenta lui T_a. Fie T_b primul nod nevizitat. Dacă există un drum de la componenta lui T_a la T_b, putem continua traseul etc. Remarcăm că, odată ce părăsim o CTC, nu mai putem reveni în ea. Dacă am putea, am închide un ciclu, ceea ce încalcă definiția CTC.

Implementare cu construcția grafului condensat

De aici, implementarea se ramifică. Am citit sursele din statistici și am observat, de regulă, această abordare:

- Algoritmul lui Kosaraju pentru determinarea CTC.
- Construcția explicită a grafului condensat.
- Pentru fiecare pereche (T_i, T_{i+1}) din componente diferite, BFS din T_i pe graful condensat pentru a determina dacă există o cale către T_{i+1}.
- Ordonarea crescătoare a vecinilor fiecărui nod, ceea ce permite răspunsuri cu căutare binară la întrebări de tipul „există muchia (T_i, T_{i+1})”?

Soluția este $\mathcal{O}(m + n)$ în ciuda BFS-urilor repetate, căci mergem mereu înainte prin dag-ul componentelor tare conexe. De aceea, BFS-urile au mereu loc pe porțiuni din dag încă neexplorate în BFS-urile anterioare.

Ca fapt divers, această sursă sortează topologic componentele, apoi decide dacă există un drum între două componente comparînd pur și simplu poziția componentelor în sortare. Această abordare este insuficientă. Problema răspunde 3, în loc de 2, pe testul

```
2 3
2 1
2
1 3
2 3
```

Implementare cu un singur DFS

Implementarea mea extinde algoritmul lui Tarjan pentru aflarea CTC. Știm că acesta emite componentele în ordinea invers topologică. Noi am prefera exact opusul, ca să pornim din componenta lui T_1 și să mergem înainte prin componente. Dar ne putem descurca și cu ordinea inversă.

Să definim **intervalul unei componente** C ca fiind intervalul de indici din T pe care îl vom vizita pe durata șederii în componenta C (dacă vom ajunge în ea, ceea ce deocamdată nu știm). Acesta este cel mai din stînga interval contiguu format exclusiv din noduri din C. Pot exista mai multe astfel de intervale în T, dar îl alegem pe cel mai din stînga deoarece, odată ce părăsim componenta C, nu mai putem reveni la ea pentru a vizita și alte intervale.

Pentru a determina intervalul componentei, încă de la citirea datelor calculăm, pentru fiecare nod u, indicele primei sale apariții în T (sau $+\infty$ dacă u nu apare deloc în T). Odată ce explorăm componenta C și îi cunoaștem nodurile, începutul intervalului lui C este minimul primelor apariții în T ale oricărui nod din C. Aflăm sfîrșitul intervalului extinzîndu-l naiv spre dreapta, pas cu pas, cît timp întîlnim noduri din C. Efortul total pentru aceste extinderi este $\mathcal{O}(n)$.

Acum, algoritmul lui Tarjan, în momentul în care termină de explorat componenta C, va fi întîlnit și muchii către alte componente (succesoare în dag-ul componentelor). Deci putem pune întrebarea: dacă ajungem în această componentă, cu ce altă componentă continuăm? *Obligatoriu* cu una care are acces la componenta care extinde spre dreapta intervalul lui C. De aceea, dintre toate componentele succesoare, păstrăm intervalul minim. Concret, fie componenta curentă C cu intervalul I_C și fie I_D intervalul minim al unei componente succesoare D. Iau naștere trei cazuri:

- Dacă I_D îl precede pe I_C, reținem doar I_D. Realitatea este că vom sări complet componenta C și doar o vom traversa ca să ajungem la D, unde putem vizita noduri din T.
- Dacă I_D urmează imediat după I_C, vom reține $I_C \cup I_D$. Vom vizita nodurile din C, apoi călătorim în D și vizităm și acele noduri.
- Dacă I_D urmează după I_C, dar la o distanță nenulă, reținem doar I_C. Realitatea este că ne vom opri după I_C, căci nu putem ajunge la următorul nod din T.

Cu această recurență, este suficient să facem un singur DFS din T_1. La final, ultima componentă explorată îl va conține pe T_1, iar intervalul său va avea forma $[1, X]$, semnificînd că putem vizita primele X noduri din T.

Pentru a răspunde la întrebări de tipul „există muchia (u, v)?”, eu am folosit pur și simplu un `unordered_set` de `long long` și am mapat fiecare pereche (u, v) la valoarea $u \cdot n + v$.

31.4.12 Problema Regate (ONI 2022, clasele 11-12)

enunț • sursă

Aceasta este o problemă de punți în grafuri neorientate, pe steroizi. Să considerăm arborele componentelor biconexe, G^{BCC}, care sînt conectate între ele prin punți. Ce putem spune despre un nod u? El va prefera să plătească, pentru fiecare nod v, costul minim al unei muchii de pe calea dintre componentele lui u și v din G^{BCC}, sau costul c_u dacă nu există punți mai ieftine decît c_u pe acea cale.

De fapt, să considerăm două noduri u și u' din aceeași componentă biconexă. Dacă calculăm punțile de la u și de la u' către toate celelalte noduri, desigur vom găsi aceeași colecție. Ce diferă între u și u' sînt pragurile c_u și $c_{u'}$ peste care nodurile preferă să renunțe la punți. Dacă toate costurile nodurilor ar fi mai mari decît toate costurile punților, am putea judeca problema doar pe punți.

Algoritmul 1: Eliminarea costurilor din noduri

O tehnică cu care ne întîlnim ocazional simplifică grafurile care au costuri atît în noduri, cît și pe muchii. Tehnica este **dublarea nodurilor** și detaliile depind de la caz la caz. Pentru problema Regate, să creăm pentru fiecare nod u o dublură, u', legată doar de u cu muchia (u, u') de cost r_u. Toate muchiile astfel create vor fi punți. Rezultă că de acum încolo putem judeca algoritmul doar pe punți. Răspunsul pentru un nod u este suma costurilor pentru legarea dublurii u' de toate nodurile originale, folosind cea mai ieftină punte. La final, din sumă vom scădea r_u deoarece u' nu trebuie legat și de u.

Mai departe, o idee destul de naturală este să procesăm punțile în ordinea costurilor. Ordonarea crescătoare nu pare să ducă nicăieri și iată de ce. Fie e puntea de cost minim, fie acesta c_e. Atunci observăm că toate nodurile aflate de o parte a muchiei e vor plăti cîte c_e pentru a se conecta cu fiecare nod aflat de cealaltă parte a muchiei e și viceversa. Apoi putem șterge muchia e și putem trata separat cei doi subarbori. Aceasta este o abordare cu **conectivitate decrementală**, de obicei grea.

Dar ea ne duce la ideea corectă: să inversăm operațiile, întrucît **conectivitatea incrementală** este ușor de gestionat cu structuri de mulțimi disjuncte! Să presupunem acum că e este puntea de cost maxim c_e și că ea unește componentele biconexe C_1 și C_2 Atunci nodurile din C_1 trebuie să se conecteze cu cele din C_2 cu costul unitar c_e, și viceversa, căci altă punte nu au la dispoziție.

Mai interesant, de acum încolo nodurile din C_1 și din C_2 vor avea o soartă comună: următoarea punte mai ieftină care unește C_1 **sau** C_2 cu o componentă C_3 va fi folosită de nodurile din ambele componente. La acel moment nodurile din C_1 și C_2 vor plăti interconectarea cu C_3 și invers. Aici intervine deci operația de unificare a mulțimilor disjuncte.

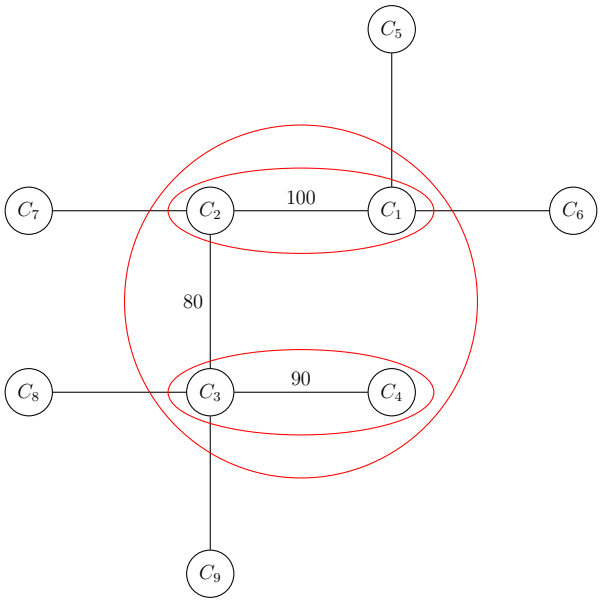


Figura 31.1: Unificarea componentelor biconexe prin punți.

În figura de mai sus, cea mai scumpă punte are costul 100 și unește C_1 cu C_2. Să notăm cu $|X|$ numărul de noduri din componenta X, exceptînd dublurile. Atunci C_1 își va nota costul $100|C_2|$, iar C_2 își va nota costul $100|C_1|$. Apoi vom unifica cele două componente. Similar vom unifica C_3 cu C_4 cu puntea de cost 90. Să denumim componentele rezultate după unificări C_{12} și C_{34}.

Cînd procesăm muchia de cost 80, fiecare nod din C_{12} își va nota costul $80|C_{34}|$ și viceversa. De aici deducem că dorim două lucruri suplimentare de la structura de mulțimi disjuncte.

Stocarea mărimilor în structura de mulțimi disjuncte

În primul rînd, cînd unificăm două componente trebuie să le cunoaștem mărimile. Această parte este simplă. Fiecare rădăcină din structura de mulțimi disjuncte stochează mărimea mulțimii sale. Inițial aceste mărimi sînt 1 pentru nodurile originale și 0 pentru dubluri. Dacă printr-o unificare v devine fiu al lui u, mărimea lui u crește cu $|v|$. Informația despre mărime este garantat corectă doar în rădăcini, dar asta este suficient.

Stocarea costurilor în structura de mulțimi disjuncte

În al doilea rînd, avem nevoie să memorăm costuri distincte pentru oricare două componente C_i și C_j pînă la unificarea lor, apoi costuri comune după unificare. Putem satisface această nevoie particularizînd structura de mulțimi disjuncte astfel:

1. Fiecare nod din structură (așadar, fiecare componentă biconexă) reține costul plătit de fiecare nod din componentă.
2. Costul total plătit de nodurile dintr-o componentă este dat de suma costurilor de pe calea de la nodul din structură pînă la rădăcina mulțimii.
3. Costurile trebuie ajustate atît la compresia căii, cît și la unificarea a două mulțimi, pentru a continua să respecte regula (2). Figura de mai jos ilustrează modificările necesare la compresia căii.

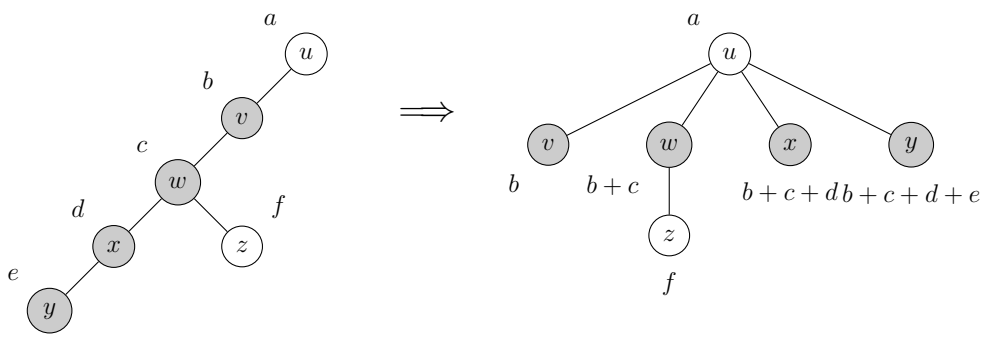


Figura 31.2: Compresia căii $u - y$. Cînd legăm nodurile w, x și y (și v) direct de rădăcina u, le recalculăm costurile astfel încît sumele pe căi să rămînă neschimbate, respectiv $a + b$ (pe calea de la u la v), $a + b + c$ (pe carea de la u la w) etc. Nodurile din afara căii nu necesită modificări. Costul pe calea de la u la z rămîne $a + b + c + f$.

Algoritmul 2: Interclasarea nodurilor și a punților

Iată și o soluție ceva mai scurtă. Nu mai dublăm nodurile. În loc de aceasta, procesăm în ordine descrescătoare punțile și nodurile, interclasînd valorile.

Să considerăm nodul u de cost r_u. Înaintea lui am procesat toate punțile mai scumpe decît r_u și am unificat corespunzător diverse componente biconexe. Să ne imaginăm așadar nodul u într-un subarbore al lui G^{BCC}, și anume subarborele care conține componenta lui u și componentele accesibile prin punți mai scumpe decît r_u. Dacă acest subarbore conține t noduri, atunci u ar prefera să plătească $r_u \cdot (t-1)$ pentru a se interconecta cu celelalte noduri, deoarece punțile sînt prea scumpe.

Răspunsul pentru u va consta din acest cost, plus costurile interconectărilor viitoare, cu punți mai ieftine decît r_u și care vor fi preferabile. Problema este că noi nu știm (încă) acele costuri. Vom ști la final costul total pentru componenta lui u, dar acel cost va include și interconectările trecute, cu punți mai scumpe decît r_u.

Dar acest neajuns îl putem rezolva ușor: măsurăm costul componentei lui u în clipa în care procesăm nodul u, apoi din nou la final. Prin diferență, aflăm costul interconectării componentei lui u cu punți mai ieftine decît r_u.

Sursa anexată implementează acest algoritm.

Capitolul 32

Distanțe

Pentru demonstrații și pentru o ordonare mai bună a noțiunilor, vă recomand cu căldură cartea Introduction to Algorithms.

32.1 Terminologie

În acest capitol vom discuta în special problema distanțelor minime de la un nod (sursă) la toate celelalte noduri dintr-un graf orientat cu costuri pe muchii. În engleză problema se numește *single source shortest paths (SSSP)*.

Considerăm aceste distanțe ca fiind ∞ pentru noduri inaccesibile din sursă. De asemenea, avînd în vedere că costurile pot fi și negative, considerăm distanțele ca fiind $-\infty$ pentru nodurile care pot fi accesate din sursă pe o cale care include un ciclu de cost negativ. Într-adevăr, întotdeauna putem obține un cost mai mic decît cel calculat, urmînd ciclul încă o dată.

În rest, distanțele sînt finite, iar căile de distanță minimă sînt aciclice (de ce?). Mai mult, orice prefix al unei căi minime este tot o cale minimă (de ce?). De aceea, totalitatea căilor minime are forma unui arbore. Pentru a reprezenta toate cele n căi minime ne este suficient un vector de părinți.

Toți algoritmii se bazează pe **estimarea pesimistă** a distanței: inițializăm distanțele tuturor nodurilor cu ∞, apoi le îmbunătățim succesiv pînă ajung la optimul teoretic. Vom nota cu $d[u]$ estimarea curentă pentru distanța de la s la u. Tehnica de bază în toți algoritmii este **relaxarea** unei muchii. Fie o muchie (u, v) și fie $d[u]$ și $d[v]$ estimările curente ale distanțelor. Relaxarea înseamnă că, dacă $d[u] + c(u, v) < d[v]$, atunci atribuim $d[v] \leftarrow d[u] + c(u, v)$.

32.2 Algoritmul Bellman-Ford

Vom începe cu acest algoritm general, deși în practica olimpiadelor sîntem mult mai familiari cu cazurile lui particulare (BFS, Lee, Dijkstra). Dar algoritmul Bellman-Ford funcționează și pe costuri negative.

Esența algoritmului este:

Algoritmul 1 Algoritmul Bellman-Ford

```
inițializează d[u] ← ∞ pentru fiecare u
d[s] ← 0
pentru i = 1, ..., n − 1 execută
    pentru e ∈ E execută
        relaxează muchia e
```

Dacă nu există cicluri negative, atunci algoritmul Bellman-Ford calculează distanțele corecte. Vom demonstra acest lucru prin inducție după lungimea căilor. După a k-a trecere prin muchii, algoritmul va fi calculat distanțele minime pentru toate nodurile pe căi de cel mult k muchii. Deoarece drumurile optime sînt aciclice, ele conțin cel mult $n-1$ segmente, după $n-1$ iterații vom fi aflat distanțele minime.

Dacă graful conține cicluri negative, le putem depista după a $n-1$ iterație: verificăm dacă există muchii care încă mai pot fi relaxate. De ce merge asta? Într-un graf fără cicluri negative, căile optime au lungime de cel mult $n-1$ muchii. Deci după a $n-1$ repetiție nu mai avem ce optimiza. Dacă există un ciclu negativ, atunci îl putem parcurge la infinit, relaxînd încă o muchie și încă una.

CP Algorithms oferă multe detalii practice de implementare:

- Terminarea prematură cînd o iterație nu mai aduce îmbunătățiri.
- Găsirea existenței unui ciclu negativ cu minimum de cod în plus.
- Găsirea efectivă a nodurilor unui ciclu negativ.

Complexitatea algoritmului este, desigur, $\mathcal{O}(mn)$.

O variantă mai rapidă în practică este algoritmul SPFA, descris tot pe CP Algorithms. Vom citi problema educațională Bellman-Ford și sursele aferente. Pentru unele teste ale acestei probleme, SPFA este de 10 ori mai rapid decît Bellman-Ford!

32.3 Caz particular: graf aciclic (*dag*)

Vom studia un exemplu și vom arăta de ce următorul algoritm în $\mathcal{O}(m+n)$ funcționează:

Algoritmul 2 Calculul distanțelor minime într-un dag

```
sortează graful topologic
pentru fiecare nod u în ordine topologică execută
    pentru fiecare muchie e = (u, v) execută
        relaxează muchia e
```

32.4 Caz particular: muchii ne-negative

Vom discuta sumar despre algoritmul lui Dijkstra, pe care îl presupunem cunoscut de la clasă. El este destul de asemănător cu algoritmul lui Prim pentru determinarea unui arbore parțial minim: pornește cu o mulțime formată doar din nodul-sursă și adaugă cîte un nod la această mulțime.

Algoritmul 3 Algoritmul lui Dijkstra

```
1: inițializează d[u] ← ∞ pentru fiecare u
2: d[s] ← 0
3: S = ∅
4: pentru i = 1, ..., n − 1 execută
5:     alege nodul u cu d[u] minim din V \ S
6:     pentru fiecare muchie e = (u, v) execută
7:         relaxează muchia e
```

În practică, folosim o coadă de priorități pentru alegerea minimului. Vom avea aceeași discuție ca și la algoritmului lui Prim: avînd în vedere că distanța unui nod din coadă se poate îmbunătăți (poate scădea), avem nevoie de o structură care să ne ofere operația `decrease-key` în $\mathcal{O}(1)$. Heapurile Fibonacci și alte cîteva heapuri oferă această complexitate. Cu acestea, putem implementa algoritmul lui Dijkstra în $\mathcal{O}(m + n \log n)$: vom avea cîte n inserări și eliminări din heap și m relaxări de muchii, fiecare în $\mathcal{O}(1)$.

Dacă folosim heap-uri normale sau `priority_queue`, atunci complexitatea va fi $\mathcal{O}((m + n) \log n)$. Iau naștere următoarele variante de implementare:

- Implementarea 1: Cu heap scris de mînă, cu suport pentru `decrease-key`. Avantajul este că putem ține o listă suplimentară de pointeri, cu care putem găsi fiecare nod în heap în $\mathcal{O}(1)$.
- Implementarea 2: Cu heap scris de mînă, fără suport pentru `decrease-key`. Cînd distanța unui nod scade, îl reinserăm în heap. Heapul poate avea mărime $\mathcal{O}(m)$ (de ce?).
- Implementarea 3: Cu `priority_queue`. Considerabil mai scurtă, dar și mai lentă.

Articolul de pe CP Algorithms oferă o discuție interesantă despre implementarea cu `priority_queue` sau cu `set`. Ideea de bază este că `set` ne permite să căutăm noduri și să le modificăm distanțele, pe cînd `priority_queue` nu. Deci implementarea cu `set` menține coada de mărime $\mathcal{O}(n)$. Totuși, cea cu `priority_queue` este mai rapidă în practică.

Discuție: cum se comportă algoritmul lui Dijkstra pe grafuri cu cicluri negative? Dar pe grafuri în care nu există cicluri negative, dar există muchii negative?

32.5 Caz particular: BFS

Putem vedea BFS (și Lee) ca pe un caz particular de distanțe în grafuri, în care toate muchiile au cost 1. Atunci putem înlocui coada de priorități cu o coadă normală, căci la orice moment

distanțele din coadă vor avea ordinea $k, \ldots, k, k+1, \ldots, k+1$. Complexitatea este, așa cum știam, $\mathcal{O}(m+n)$.

32.6 Caz particular: 0-1 BFS

Dacă muchiile au cost 0 sau 1, putem în continuare să folosim o simplă coadă, mai exact un deque. La relaxarea unei muchii (u, v) de cost 1, vom adăuga nodul v la sfîrșitul cozii. În schimb, la relaxarea unei muchii (u, v) de cost 0, vom adăuga nodul v la începutul cozii. Practic, v va fi următorul nod evaluat.

Articolul de pe CP Algorithms conține o implementare minimală.

32.7 Caz particular: costuri mici

Dacă toate costurile sînt întregi între 1 și k (sau între 0 și k, refolosind ideea din secțiunea anterioară), atunci algoritmul lui Dial (Robert B. Dial) ne oferă o soluție în $\mathcal{O}(m+kn)$. Distanța maximă în graf va fi $(n-1)k$, deci putem înlocui coada de priorități cu un vector V de $(n-1)k$ liste înlănțuite, unde $V[x]$ conține nodurile u a căror estimare curentă este $d[u] = x$. Inițial, $V[0]$ va conține doar elementul s, iar restul listelor vor fi vide. Procesăm listele de la stînga la dreapta și, pentru fiecare nod, îi relaxăm muchiile, mutîndu-i vecinii dintr-o listă într-alta dacă distanța lor se schimbă.

Două detalii de implementare:

- Pentru a face relaxarea în $\mathcal{O}(1)$, trebuie să putem găsi rapid un nod și să-l mutăm din lista sa, $V[d[u]]$, în noua listă corespunzătoare noii distanțe. Putem folosi un simplu vector de pointeri.
- Putem reduce memoria de la $\mathcal{O}(kn)$ la $\mathcal{O}(k+n)$ astfel. Cînd procesăm nodurile de la distanța x, vom face inserări și ștergeri doar din listele de pe pozițiile $x \ldots x+k$ (de ce?). Deci putem stoca din vectorul V doar fereastra curentă de lățime $k+1$, folosind un buffer circular sau o listă înlănțuită.

32.8 Probleme

Primele 6 probleme sînt educaționale, de pe Infoarena și de pe CSES.

32.8.1 Problema Bellman-Ford (Infoarena)

enunț • surse

Vom vedea două surse, una cu algoritmul Bellman-Ford clasic și una cu SPFA.

O problemă similară este Cycle Finding (CSES). Pentru găsirea ciclului, notăm și părintele fiecărui nod (adică nodul care îi cauzează relaxarea). Pornim din orice nod care încă este relaxat după a

n-a iterație și mergem din părinte în părinte pînă revenim la primul nod.

32.8.2 Problema Dijkstra (Infoarena)

enunț • surse

Există multe variante de implementare, dintre care am încercat trei:

1. Cu heap propriu, cu suport pentru `decrease-key`. Avantajul este că putem ține o listă suplimentară de pointeri, cu care putem găsi fiecare nod în heap în $\mathcal{O}(1)$.
2. Cu heap propriu, fără suport pentru `decrease-key`. Cînd distanța unui nod scade, îl reinserăm în heap. Heapul poate avea mărime $\mathcal{O}(m)$ (de ce?).
3. Cu `priority_queue` din STL. Considerabil mai scurtă, dar ceva mai lentă.

O problemă identică este Shortest Routes I (CSES).

32.8.3 Problema Monsters (CSES)

enunț • sursă

Problema se rezolvă direct cu algoritmul lui Lee. Singura (mică) dificultate este cum să găsim celulele în care eroul poate ajunge înaintea monștrilor. Soluția este să rulăm întîi un BFS multisursă din pozițiile monștrilor ca să aflăm, pentru fiecare celulă, timpul minim pînă cînd un monstru poate ajunge acolo. Apoi rulăm un BFS din poziția eroului, permițîndu-i să meargă doar pe acele celule în care poate ajunge mai repede decît monștrii.

32.8.4 Problema Shortest Routes II (CSES)

enunț • sursă

Problema se rezolvă direct cu algoritmul Floyd-Warshall. Atenție la muchii multiple între aceleași noduri.

32.8.5 Problema High Score (CSES)

enunț • sursă

Problema este rezolvabilă cu algoritmul Bellman-Ford, cu o mică adăugire. Negăm toate costurile muchiilor și căutăm costul minim de la 1 la n. Acum, dacă există vreun ciclu negativ, răspunsul încă s-ar putea să fie finit. Este posibil să existe doar cicluri negative accesibile din 1, dar din care nu există nicio cale către n.

Cum depistăm această situație? O primă abordare este să colectăm nodurile care încă mai sînt relaxate la ultima iterație (a n-a). Apoi facem un DFS pe graful transpus, pornind din n, și vedem dacă ajungem la unul dintre nodurile sus-menționate.

O variantă cu peste două linii mai scurtă 🙂 este să calculăm accesibilitatea pe parcursul relaxării. Cînd relaxăm muchia (u, v) și dacă nodul v este accesibil din n, atunci marcăm și nodul u ca accesibil din n.

32.8.6 Problema Flight Discount (CSES)

enunț • sursă

Problema este de Dijkstra cu o mică observație suplimentară. Putem menține pentru fiecare nod două valori: costul minim pentru a ajunge în acel nod folosind sau nefolosind reducerea. Rezultă cîteva întrebări despre implementare, în particular: ce ținem în *priority queue*?

Ghidul USACO menționează și o altă soluție elegantă. Pentru fiecare nod u calculăm distanța minimă de la nodul 1 la u și distanța minimă de la u la n. Pentru a doua categorie rulăm algoritmul lui Dijkstra din nodul n pe graful transpus. Apoi, pentru fiecare muchie $e = (u, v)$ ne întrebăm: care ar fi costul dacă am folosi reducerea pe această muchie? Răspunsul este suma distanțelor de la 1 la u, de la v la n și jumătate din costul muchiei e.

32.8.7 Problema Three States (Codeforces)

enunț • sursă

Problema este un pic migăloasă, dar nu foarte grea. Este un exemplu tipic de 0-1 BFS: într-un stat sau între două state ne deplasăm cu costul 0, iar pe o stradă cu costul 1. Dar ne putem încurca în gestiunea muchiilor. De exemplu, traseul stat → stradă → stat nu are costul 2, ci costul 1. În fapt, ca și la algoritmul lui Lee, considerăm costul plătit în clipa în care intrăm în nod.

Pornim pe rînd cîte un Lee din fiecare stat. Nu contează din ce pătrat anume, căci distanțele în interiorul țării vor fi 0, iar cele în afara țării nu depind de punctul de pornire.

Apoi, căutăm celula din matrice care minimizează suma distanțelor pînă la cele trei puncte de pornire. Acel punct poate fi:

- O celulă dintr-un stat, caz în care strada are formă liniară.
- O celulă de tip stradă, caz în care rețeaua de străzi va avea formă de stea. În acest caz trebuie să scădem 2 din suma distanțelor, deoarece am socotit această celulă de trei ori, dar o folosim o singură dată.

Ce trebuie să reținem din această problemă: BFS-ul se descurcă cu o coadă ordonată, iar Dijkstra are nevoie de un heap (*priority queue*). La 0-1 BFS încă ne este suficientă o coadă, pentru că o putem menține ordonată cîtă vreme inserăm la capătul potrivit.

32.8.8 Problema Graph and Graph (Codeforces)

enunț • sursă

Estimez că această problemă este la nivel de OJI clasele 11-12. Să pornim de la o observație teoretică relativ evidentă. Cînd este costul total finit? Atunci cînd există un moment de timp începînd de la care nu mai plătim nimic. Cu alte cuvinte, secvența de noduri vizitate în ambele grafuri este $u - v - w - x - \ldots$. În realitate, însă, odată ce găsim muchia comună $u - v$, putem călători pe ea, înainte și înapoi, la infinit.

Așadar, dorim să aflăm (1) dacă există o muchie $u - v$ comună ambelor grafuri, astfel încît să putem ajunge simultan în u, și (2) care este costul minim ca să ajungem simultan în u.

Implementarea creează un graf indus în care nodurile (stările) sînt perechi (u_1, u_2) cu semnificația că putem ajunge simultan în u_1 în primul graf și în u_2 în al doilea graf. Nodul de pornire este (s_1, s_2). Din starea (u_1, u_2) putem ajunge în orice stare (v_1, v_2) pentru care există muchii (u_1, v_1) în primul graf și (u_2, v_2) în cel de-al doilea. Costul tranziției este $|v_1 - v_2|$.

Pe acest graf rulăm algoritmul lui Dijkstra. Intenția este să găsim orice stare cu (v, v) în care putem ajunge dintr-o altă stare (u, u).

O particularitate pe care am întîlnit-o în sursele multor elevi este precalcularea nodurilor finale: acele noduri u care au un vecin comun v în ambele grafuri. Acest lucru nu este necesar: putem depista finalul în Dijkstra, în momentul în care expandăm o stare (u, u) și întîlnim același vecin (v, v).

32.8.9 Problema Termite (Lot 2014)

enunț • surse

Problema necesită o idee, dar sper că v-ați întîlnit cu ea suficient de des încît să o recunoașteți.

Observăm că, în timp, graful se fragmentează. Interogările cer, în esență, să stabilim momentul cînd două noduri se deconectează. Această problemă se numește *conectivitate decrementală* și este grea. Sper că ați recunoscut cu toții șablonul: putem să răsturnăm ordinea operațiilor, să pornim cu un graf vid și să reconectăm nodurile.

Faptul că din graf dispar noduri, nu muchii, cere doar o adaptare minoră. Două noduri unite printr-o muchie devin reconectate cînd ambele reapar, deci la minimul momentelor în care cele două noduri reapar. De asemenea, componenta t a fiecărei interogări (a, b, t) este doar o diversiune. Realmente ne interesează momentul cînd a și b se deconectează, urmînd ca din acel moment să scădem valoarea t.

În ce ordine dispar/reapar nodurile? Aceasta aflăm cu algoritmul lui Dijkstra, care este (aș zice) partea mai simplă a problemei. Așadar, scheletul algoritmului este:

1. Rulează algoritmul lui Dijkstra.
2. Inițializează mulțimea de structuri disjuncte (DSF),
3. Parcurge nodurile în ordinea descrescătoare a distanțelor.
4. Pentru fiecare nod u, pentru fiecare vecin v al lui u, dacă nodul v a reapărut înaintea lui u, atunci conectează în DSF nodurile u și v.

Partea mai dificilă a problemei este cum să răspundem la interogări. Iată două soluții, ambele în timp logaritmic per interogare.

Soluție cu *small-to-large*

Fiecare componentă u din DSF menține și o listă L_u a ID-urilor interogărilor care au **exact** un capăt în acea componentă. Pentru inițializare, vom distribui interogarea (a, b, t) în L_a și L_b. Așadar, suma lungimilor listelor din toate componentele este $2q$.

Cînd unificăm componentele nodurilor u și v, dorim să unificăm L_u cu L_v, mutînd lista mai mică în lista mai mare pentru a obține un număr de $\mathcal{O}(q \log q)$ mutări. Dar există o excepție. Dacă o interogare apare în ambele liste, înseamnă că muchia (u, v) este prima a cărei dispariție deconectează nodul u de v. Așadar, notăm momentul adăugării muchiei (u, v) ca răspuns la interogare, apoi ștergem interogarea din ambele liste.

Cum implementăm această structură? O variantă este să stocăm listele de interogări ca `std::set`. Dar atunci timpii pentru căutare / inserare / ștergere sînt $\mathcal{O}(\log q)$, iar complexitatea totală a unificărilor va fi $\mathcal{O}(q \log^2 q)$, probabil prea lentă pentru limita de timp.

Dar putem să eliminăm un factor logaritmic cu o structură elementară. Să stocăm listele de interogări ca pe niște simple liste înlănțuite. Desigur, știm să copiem elementele din L_u în L_v. Dar trebuie să răspundem la două întrebări pentru celelalte operații.

1. Cum facem căutările? Cînd considerăm o interogare $h = (a, b, t)$ din L_u, cum știm dacă ea există și în L_v? Soluția este să vedem, în primă fază, care dintre a și b fac parte din componenta lui u. Să spunem că este a. Atunci verificăm din ce componentă face parte nodul b. Dacă face parte din componenta v, atunci în mod evident va face apărea și în L_v.
2. Cum facem ștergerile? Cînd iterăm prin L_u și depistăm o interogare h care există și în L_v, cum îi găsim poziția ca să o ștergem? Soluția este să facem ștergerile lazy. Pe moment îl lăsăm pe h în L_v (dar cel puțin nu adăugăm și a doua copie, care provine din L_u). În compensație, de cîte ori mutăm un element h între două liste de interogări, ne asigurăm că nu avem deja răspunsul pentru h. Dacă îl avem, aruncăm valoarea h în loc să o mutăm.

Soluția cu mulțimi disjuncte modificate

Iată o structură cu care am rezolvat și problema Minimum Spanning Tree for Each Edge de la lecția de arbori parțiali. Menținem pentru fiecare componentă din DSF mărimea sa. Nu mai facem compresia căii, dar facem unificarea după mărime. Aceasta înseamnă că drumul de la orice nod pînă la rădăcina componentei nu va depăși $\log n$ noduri.

Definim *momentul* unei muchii (u, v) drept minimul distanțelor de la surse la u sau la v. Acesta este chiar momentul cînd muchia (u, v) dispare din graf ca urmare a dispariției unuia dintre noduri.

Cînd unificăm componentele lui u și v în DFS, identificăm ca de obicei rădăcinile componentelor, u' și v'. Să presupunem că componenta lui u este mai mică. Atunci unim u' cu v' ca de obicei și

notăm în u' momentul muchiei (u, v). Să notăm această cantitate cu $t[u']$. Ce semnifică $t[u']$? El este momentul de timp la care orice nod din componenta lui u se deconectează de orice nod din componenta lui v.

Putem folosi această informație ca să răspundem la interogări. Pentru o interogare (a, b), căutăm cel mai mic moment de pe calea $a \to b$ din DSF. Acela este momentul cînd a și b se deconectează. Ca detaliu de implementare, nu este nevoie să calculăm informații pentru LCA. Pur și simplu, în mod repetat urcăm un nivel cu acel nod dintre a și b care are mărimea cea mai mică (în număr de noduri).

32.8.10 Problema President and Roads (Codeforces)

enunț • sursă

Observația necesară pentru rezolvarea problemei este că avem nevoie să apelăm Dijkstra de două ori. Dacă apelăm Dijkstra o singură dată, din sursă, nu avem suficiente informații. Putem discuta următoarele reguli ca să vedem dacă ne ajută la ceva:

- „O muchie (u, v) este pe o cale minimă / pe orice cale minimă dacă $d[u] + c(u, v) = d[v]$."
- „O muchie (u, v) **nu** este pe nicio cale minimă dacă $d[u] + c(u, v) > d[v]$."

Așadar, apelăm Dijkstra din sursă și Dijkstra din destinație în graful transpus. Fie $d_T[u]$ distanța minimă de la destinație la u în graful transpus. O muchie (u, v) poate fi folosită pentru calea minimă $s - t$ dacă și numai dacă

$$d[u] + c(u, v) + d_T[v] = d[t]$$

De ce? Fie P_1 calea $s \rightsquigarrow u$ care ne oferă distanța $d[u]$ și fie P_2 calea în graful transpus $t \rightsquigarrow v$ care ne oferă distanța $d_T[v]$. Fie P_2^R calea P_2 răsturnată (revenind în graful direct). Fie $P = P_1 \cup \{(u, v)\} \cup P_2^R = s \rightsquigarrow u - v \rightsquigarrow t$. Iau naștere trei cazuri:

- Nu se poate ca $d[u] + c(u, v) + d_T[v] < d[t]$, căci ar însemna că distanța pe calea P este mai mică decît $d[t]$.
- Dacă $d[u] + c(u, v) + d_T[v] = d[t]$, atunci în mod evident P este o cale minimă.
- Dacă $d[u] + c(u, v) + d_T[v] > d[t]$, să presupunem prin absurd că există un drum minim care folosește muchia (u, v). Atunci costul acelui drum ar fi, în mod necesar, $d[t]$. Rezultă că acel drum minim fie ajunge de la s la u pe un cost strict mai mic decît $d[u]$, fie ajunge de la v la t pe un cost strict mai mic decît $d_T[v]$, căci altfel am ajunge fix la inegalitatea inițială. Așadar, una dintre distanțele calculate de Dijkstra nu este optimă, contradicție.

Dar cum știm dacă o muchie apare pe **orice** drum minim? Intuitiv, ne duce gîndul la punți. Există trei variante:

- Să verificăm, cu algoritmi de tare conexitate, dacă (u, v) este punte.
- Soluția din editorial: Să luăm în calcul doar muchiile care apar cel puțin pe o cale minimă.

Pentru aceste muchii, să considerăm toate momentele de timp între 0 și $d[t]$. Dacă, la orice moment k, există o singură muchie (u, v) cu $d[u] \leq k \leq d[v]$, atunci toate drumurile minime trec prin această muchie. Sînt cîteva cazuri particulare de tratat.

- Soluția cu hashing (kudos andrei_C1). Fie $w[u]$ și $w_T[u]$ numărul de moduri de a ajunge în u pe drumuri minime mergînd înainte din s, respectiv înapoi din t. Putem calcula aceste valori cu programare dinamică. Atunci o muchie apare pe orice drum minim dacă și numai dacă

$$w[u] \cdot w_T[v] = w[t]$$

Demonstrația este relativ directă: știm sigur că avem $w[u]$ drumuri minime pînă la u și, independent, $w_T[v]$ drumuri minime de la v la t. Dacă produsul acestora este chiar $w[t]$, atunci nu mai există niciun alt drum minim, separat de muchia (u, v). Din păcate, teoretic nu putem stoca aceste valori, căci ele pot fi foarte mari (demonstrați!). Dar putem opera modulo un număr prim ales de noi, iar aproximarea noastră va fi greu de pedepsit.

Paradoxal, întrebarea „putem ieftini o muchie ca să apară pe orice cale minimă $s - t$?" este mult mai simplă. Din ecuația $d[u] + c(u, v) + d_T[v] < d[t]$ calculăm $c(u, v)$ pentru a face muchia atractivă, iar singura limită este că nu-i putem reduce costul sub 1.

32.8.11 Problema Nastya and Unexpected Guest (Codeforces)

enunț • sursă

Problema se rezolvă relativ direct cu algoritmul lui Dial. Singura inspirație necesară este că trebuie să lucrăm în clase de resturi modulo g. Odată ce am ajuns pe o insulă x cu un rest y, nu vom mai avea nevoie niciodată să revenim pe acea insulă cu același rest. Ia naștere un graf cu $g \cdot m$ noduri în care trebuie să calculăm distanța minimă de la nodul (insula 1, restul 0) pînă la orice rest de pe insula m.

Dintr-o pereche x, y putem trece pe insula $x + 1$ dacă $x < m$ și dacă $d_{x+1} - d_x \leq g - y$, pentru că trebuie să apucăm să ajungem pe insula $x + 1$ înainte ca semaforul roșu să ne întrerupă. Noul rest este $y + d_{x+1} - d_x \bmod g$. Un raționament similar facem și pentru trecerea pe insula $x - 1$.

Deoarece la niciun moment nu putem călători mai departe de $g \leq 1000$, rezultă că putem folosi algoritmul lui Dial cu 1001 liste înlănțuite.

Mărturisesc că nu mă simt mulțumit de implementarea mea. Ea este rezonabil de rapidă, dar consumă multă memorie (117 MB), deoarece folosește celule distincte ori de cîte ori un element este adăugat la structură și nu le refolosește pe cele eliberate. S-ar putea ca STL-ul să conducă la memorie redusă.

Editorialul vorbește despre o soluție bazată pe 0-1 BFS, pe care eu însă nu o înțeleg.

32.8.12 Problema Minimum Path (Codeforces)

enunț • surse

Ideea problemei este grea, sau cel puțin prea grea pentru mine. 😇 Iată un indiciu: trebuie rezolvat un caz **mai general** decît cel în care eliminăm fix maximul și dublăm fix minimul.

Dacă am calcula costul unei căi pe care am eliminat orice muchie și am dublat orice muchie (posibil aceeași)? Atunci reducem problema la un Dijkstra cu 4 stări: în fiecare nod vom avea patru minime, pentru situațiile în care ajungem în nod:

- pe o cale nemodificată;
- pe o cale cu o muchie eliminată;
- pe o cale cu o muchie dublată;
- pe o cale cu ambele operații efectuate.

Acest caz general se reduce de fapt la enunțul problemei. Într-adevăr, algoritmul va genera și căi care elimină / adaugă alte muchii decît maximul / minimul, dar acelea nu vor fi optime.

Am încercat trei implementări, cu rezultate interesante. Prima sursă este cu heap propriu. A doua folosește `priority_queue`, fără alte modificări, și este cu 30% mai lentă. În schimb, memoria a scăzut cu 35%, căci pentru heapul propriu mi-a fost greu să estimez necesarul de memorie.

A treia sursă (neinclusă în anexă) schimbă ușor detaliile inserării în coadă. Mai exact, sursa a doua setează noua distanță a unui nod v în starea sv în momentul în care îl inserează în coadă:

```
if (new_dist < nd[v].dist[sv]) {
  nd[v].dist[sv] = new_dist;
  pq.push({new_dist, v, sv});
}
```

Sursa a treia setează noua distanță în momentul în care scoate nodul din coadă:

```
pq_elt top = pq.top();
pq.pop();
int u = top.u;
int su = top.state;

if (nd[u].dist[su] == INFINITY) {
  nd[u].dist[su] = top.dist;
  relax_all(u, su);
}
```

Memoria necesară s-a dublat și chiar mai rău! Deci atenție la formularea exactă.

32.8.13 Problema Shortest Path (Codeforces)

enunț • surse

Ideea teoretică este să reținem și ultimul pas prin care am ajuns într-un nod. Așadar, definim stări de tipul $\langle u, v \rangle$ cu semnificația „ne aflăm în nodul v și am ajuns aici din nodul u”. Există m stări, deoarece nu are sens să definim o stare $\langle u, v \rangle$ dacă nu există muchia (u, v) pe care să călătorim.

Între aceste stări putem face tranziții. Din $\langle u, v \rangle$ putem ajunge în $\langle v, w \rangle$ dacă tripletul (u, v, w) nu este interzis.

Stările și tranzițiile definesc, de fapt, un graf extins. Pe acest graf rulăm algoritmul BFS. La final, reconstituim soluția, de exemplu recursiv.

Implementare minimalistă

Prima sursă este orientată spre brevitate, dar nu este prea eficientă. Menținem un `unordered_set` global de triplete ca să aflăm în $\mathcal{O}(1)$ dacă o tranziție este permisă. Menținem distanțele calculate în BFS chiar într-o matrice de $n \times n$. Cînd explorăm o stare $\langle u, v \rangle$ și descoperim o nouă stare $\langle v, w \rangle$, în aceasta din urmă notăm informația că nodul anterior a fost u. Astfel putem reconstitui soluția.

Sursa folosește $\mathcal{O}(n^2)$ memorie, dar se poate mai bine.

Implementare eficientă

A doua sursă stochează distanțele chiar pe muchii pentru a renunța la matricea de $n \times n$. Concret, citim și sortăm toate muchiile și toate tripletele. Apoi:

- Lista de adiacență a nodului u este indicele primei muchii care se referă la u. Astfel aflăm vecinii în ordine crescătoare.
- Lista de continuări interzise pentru muchia (u, v) este indicele primului triplet care are primele două cîmpuri u și v. Astfel aflăm continuările interzise în ordine crescătoare.

La explorarea unei muchii (u, v) accesibilă la distanța d, interclasăm lista de succesori w ai lui v cu lista de continuări interzise. Punem distanța $d + 1$ doar pe muchiile care nu duc spre un nod interzis.

Astfel obținem memorie $\mathcal{O}(m+n+k)$, adică 5 MB în loc de 39 MB. Timpul scade de la 1 secundă la 0,4 secunde. Am chiar și o sursă mai veche, care folosește doar 2,2 MB, dar nu mai știu exact ce am făcut. 🥲

Capitolul 33

Drumuri euleriene

Într-un graf orientat sau neorientat, un **drum eulerian** (engl. *Eulerian path*) este un drum care vizitează fiecare muchie exact o dată. Un **ciclu eulerian** este un drum eulerian care este și ciclu.

Un graf neorientat admite un ciclu eulerian dacă și numai dacă este conex și toate nodurile au grad par. Similar, un graf orientat admite un ciclu eulerian dacă și numai dacă este tare conex și toate nodurile au gradul la intrare egal cu gradul la ieșire.

Un graf neorientat admite un drum eulerian dacă este conex și are zero sau două noduri de grad impar.

Nu întîmplător folosim aceeași denumire ca la parcurgerea Euler din liniarizarea arborilor. Dacă ne imaginăm că fiecare muchie din acel arbore apare de două ori (o dată la intrarea în DFS și o dată la revenire), atunci liniarizarea produce fix un circuit eulerian! (Corolar, ca fapt divers: dacă schimbăm rădăcina arborelui, atunci noua liniarizare Euler se obține din cea veche, prin rotirea vectorului).

33.1 Algoritmi

Pe cînd eram eu elev, lumea folosea un algoritm intuitiv, dar mai greoi de implementat. Presupunem că am verificat precondițiile și știm că există un ciclu eulerian.

1. Pornește dintr-un nod u arbitrar ales.
2. Parcurge muchii la întîmplare pînă cînd revii în u.
3. Adaugă ciclul obținut la o colecție.
4. Repetă pașii 1-3 cît timp mai sînt muchii de parcurs.
5. Înnădește ciclurile obținute.

Articolul de pe CP Algorithms prezintă un algoritm considerabil mai simplu, bazat pe o parcurgere DFS recursivă:

Algoritmul 4 Găsirea unui ciclu eulerian, algoritm recursiv

```
1: procedure CicluEulerian(u)
2:     pentru fiecare muchie (u, v) execută
3:         șterge muchia (u, v) din graf
4:         CicluEulerian(v)
5:     adaugă nodul u la soluție
```

Prima mea sursă de la problema Ciclu eulerian implementează exact acest algoritm recursiv. Cîteva mențiuni:

- O implementare corectă va trata fără cazuri particulare buclele și muchiile multiple.
- Cînd reprezentăm grafuri neorientate, orice muchie (u, v) va apărea de două ori, în listele de adiacență ale lui u și v. Este necesar să o ștergem din ambele liste (sau să o marcăm ca ștearsă).
- Acest algoritm generează soluția **în ordinea inversă** față de ordinea explorării. Detaliul devine important în grafuri orientate.
- Ca orice program recursiv, și acesta poate umple stiva. Spre deosebire de DFS-urile cu care sîntem obișnuiți, care ating adîncime cel mult $\mathcal{O}(n)$, acesta poate atinge adîncimea $\mathcal{O}(m)$.

Putem implementa acest algoritm și iterativ, simulînd pur și simplu stiva de apeluri a DFS-ului:

Algoritmul 5 Găsirea unui ciclu eulerian, algoritm iterativ

```
1: procedure CicluEulerian
2:     pune nodul de start pe stivă
3:     cît timp stiva nu este vidă execută
4:         fie u nodul din vîrful stivei
5:         dacă u mai are vreun vecin v atunci
6:             șterge muchia (u, v) din graf
7:             pune nodul v pe stivă
8:         altfel
9:             adaugă nodul u la soluție
10:            scoate nodul u de pe stivă
```

33.2 Probleme

33.2.1 Problema Ciclu eulerian (Infoarena)

enunț • surse

Aceasta este o problemă clasică de găsire a unui ciclu eulerian neorientat. Enunțul nu garantează că graful este conex, dar toate testele par să conțină grafuri conexe. Dacă conexitatea trebuie garantată, cred că cel mai simplu mod este să rulăm algoritmul în mod normal și să verificăm că el a generat un ciclu de lungime m.

O problemă similară este Mail Delivery (CSES).

33.2.2 Problema De Bruijn Sequence (CSES)

enunț • surse

Observația-cheie este că, după un șir de n biți, îi refolosim pe ultimii $n-1$ și mai adăugăm un bit nou pentru a genera un nou șir de n biți. De aceea, putem traduce problema într-un graf astfel:

- Există 2^{n-1} noduri corespunzînd la 2^{n-1} șiruri binare.
- În fiecare nod u intră două arce (orientate) dinspre șirurile al căror ultimi $n-2$ biți corespund cu primii $n-2$ biți ai lui u.
- Invers, din fiecare nod u ies două arce către șirurile al căror primi $n-2$ biți corespund cu ultimii $n-2$ biți ai lui u.
- Pe fiecare muchie notăm bitul care trebuie adăugat la nodul-sursă pentru a obține nodul-destinație.
- Intuitiv, nodul u semnifică că ultimii $n-1$ biți tipăriți au fost reprezentarea binară a lui u.
- Un ciclu eulerian în acest graf ar tipări valorile întîlnite pe muchii. Aceste valori coincid (prin definiție) cu ultimul bit al nodului-destinație, deci nu este nevoie să le stocăm explicit.

Articolul de pe Wikipedia include și o imagine pentru $n = 4$.

Odată făcută reducerea, codul este remarcabil de scurt. Nu este nevoie să reținem listele de adiacență. Vecinii unui nod decurg din numărul de ordine al nodului. De asemenea, numărul de vecini vizitați / rămași pentru un nod poate fi reprezentat ca un singur număr între 0 și 2, ceea ce ne indică automat și următoarea muchie vizitabilă din acel nod.

Și această problemă poate fi implementată recursiv sau iterativ.

33.2.3 Problema Tanya and Password (Codeforces)

enunț • sursă

Problema necesită o implementare atentă, dar reducerea ei la găsirea unui drum eulerian este fățișă. Nu cred că problema ar trebui să aibă rating de 2 500. Ca și la secvențele de Bruijn, nodurile au lungime cu 1 mai mică decît șirurile date. Așadar, pentru a arăta că tripletele *abc* și *bcd* se potrivesc, vom crea stările *ab*, *bc*, *cd* cu muchii între ele. Rezultă că există cel mult $62^2 = 3\,844$ noduri. În practică am lucrat în baza 64 pentru a accelera împărțirile.

De data aceasta graful este orientat, deci faptul că algoritmul emite un drum eulerian în ordine inversă devine relevant. Pentru a recupera drumul corect, am ales să inversez toate muchiile. Am stocat lista de adiacență ca pe un vector de 64 de întregi. De exemplu, pentru nodul *ab*, valoarea `adj['c']` este egală cu numărul de triplete *abc* de la intrare.

În rest, cea mai complicată parte a programului este găsirea nodului de start. 🙂

Capitolul 34

Fluxuri

Voi încerca să nu încarc capitolele de fluxuri și cuplaje cu prea multă teorie, dar uneori nu putem scăpa. 🙂 Vom demonstra unele teoreme acolo unde ne ajută să înțelegem fenomenul.

Vom împrumuta imagini și probleme din monumentala carte Introduction to Algorithms de Cormen, Leiserson, Rivest și Stein, ediția a 4-a. Ca întotdeauna, vă încurajez să cumpărați o carte dacă vă este utilă, chiar dacă o puteți găsi și pe Internet sau pe torente. Vă recomand de asemenea să citiți și materialele de pe CP Algorithms, Flows and related problems și următoarele.

34.1 Descrierea problemei

Definiția 3 (Rețea de flux)**.** O rețea de flux este un graf orientat $G = (V, E)$ în care fiecare muchie (u, v) are o capacitate $c(u, v) \geq 0$. Există două noduri speciale în această rețea: o sursă s și o destinație t.

Exemple din viața reală:

- rețeaua de apă, de gaz sau de electricitate a unui oraș;
- o rețea de calculatoare care schimbă pachete între ele;
- o mină și o rețea de străzi care transportă minereul la fabrica de procesare.

În toate aceste exemple, capacitatea este un debit, respectiv cantitatea maximă de material / resurse / date care pot trece prin muchie într-o unitate de timp. De asemenea, în aceste rețele se aplică prima lege lui Kirchhoff: suma debitelor într-un nod este 0, cu excepția nodurilor s și t.

Definiția 4 (Flux)**.** Un flux de la s la t printr-o rețea este o funcție $f : E \to \mathbb{R}$ care satisface două proprietăți:

1. Respectarea capacităților: pentru orice muchie (u, v),

$$0 \leq f(u, v) \leq c(u, v)$$

2. Conservarea fluxului: pentru orice nod $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$$

Definiția 5 (Valoarea fluxului)**.** Valoarea unui flux f este valoarea netă a fluxului care părăsește sursa s:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

Cu aceste definiții, problema fluxului maxim este: să se găsească un flux de valoare maximă într-o rețea de flux dată.

34.1.1 Exerciții teoretice

1. Cum tratăm cazul rețelelor cu surse și destinații multiple?
2. Cormen, exercițiul 24.1-6: dat fiind un graf $G = (V, E)$ și două noduri s și t, să se găsească două drumuri de la s la t astfel încît orice muchie $e \in E$ să apară pe cel mult un drum.
3. Cormen, exercițiul 24.1-7: cum tratăm cazul rețelelor care au capacități atît pe muchii, cît și în noduri? Exemplu din viața reală: routerele.

34.2 Metoda Ford-Fulkerson

34.2.1 Rețeaua reziduală

Neformal, rețeaua reziduală indusă de un flux f într-o rețea de flux ne arată, pe fiecare muchie, (1) cîte unități mai putem pompa (posibil 0) și (2) cîte unități mai putem scoate (posibil 0). Vom vedea exemple unde este util să reducem fluxul pe unele muchii pentru a-l putea mări pe altele, obținînd un nou flux de valoare mai mare.

Formal,

Definiția 6 (Rețea reziduală, capacități reziduale)**.** Rețeaua reziduală indusă de un flux f într-o rețea G se notează cu $G_f = (V, E_f)$ și are capacitățile reziduale c_f cu valorile:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{dacă } (u, v) \in E \\ f(v, u) & \text{dacă } (v, u) \in E \\ 0 & \text{altfel} \end{cases}$$

Remarcăm că muchiile reziduale (mulțimea E_f) se regăsesc printre muchiile originale sau inversele acestora.

Vom urmări figura 24.4 din Cormen pentru un exemplu.

34.2.2 Drumuri de creștere

Definiția 7 (Drum de creștere)**.** Dată fiind o rețea de flux G și un flux f în această rețea, un drum de creștere p este un flux în G_f.

Cu alte cuvinte, un drum de creștere **augmentează** fluxul existent, adică găsește un mod de a mai pompa niște unități de flux. Din nou, figura 24.4 din Cormen arată un drum de creștere. Cîte unități mai putem pompa? Răspunsul este minimul dintre capacitățile reziduale pe acel drum. Numim aceasta **capacitatea reziduală** a drumului de creștere și o notăm cu $c_f(p)$.

34.2.3 Metoda Ford-Fulkerson

Esența metodei este, pur și simplu: Cît timp există un drum de creștere p în rețeaua reziduală, augmentează fluxul cu $c_f(p)$ și recalculează rețeaua reziduală.

O denumim „metodă" și nu „algoritm" deoarece ea nu specifică detaliile găsirii drumului de creștere.

Vom citi o primă implementare, pentru problema Download Speed. Implementarea folosește DFS pentru a găsi drumuri de creștere. Cîteva detalii:

- Numărul de noduri în probleme de flux poate fi suficient de mic pentru a permite matrice de adiacență. Folosiți-le dacă preferați.
- Pentru o muchie (u, v) avem nevoie să găsim ușor muchia inversă (v, u), deoarece capacitatea reziduală pe o muchie scade cînd crește pe cealaltă. Eu am folosit faptul că muchiile ocupă celule adiacente în liste de adiacență. Dacă indicele uneia este pos, în mod garantat indicele celeilalte va fi `pos ^ 1`.
- Pe muchii stocăm doar capacitatea rămasă. Cum putem deduce din această informație fluxul efectiv pe fiecare muchie, fără a stoca informații suplimentare? Acest amănunt va deveni important la problemele de tăietură minimă.

Implementată astfel, metoda Ford-Fulkerson poate necesita $\mathcal{O}(|f|)$ iterații, fiecare în $\mathcal{O}(E)$, așa cum o arată figura 24.7 din Cormen. Mai mult, pentru valori iraționale (teoretice), este posibil ca algoritmul cu DFS să nu termine niciodată! De exemplu, la problema Maxflow de pe Infoarena testele 8, 9 și 10 durează foarte mult dacă le implementăm în $\mathcal{O}(n)$ per augmentare (pe testul 8 programul meu a durat 5 minute). Similar, testul 20 din problema Download Speed durează foarte mult.

34.3 Algoritmul Edmonds-Karp

Algoritmul Edmonds-Karp doar înlocuiește parcurgerea DFS cu una BFS, la fiecare căutare a unui drum de creștere. Prin aceasta, reușim să obținem o limită a numărului de iterații independentă de $|f|$, respectiv $\mathcal{O}(mn)$.

Definiția 8 (Muchie critică)**.** Muchia critică este muchia de capacitate minimă de pe un drum de creștere (cea care determină cantitatea pompată).

Schița demonstrației de complexitate a algoritmului Edmonds-Karp este:

1. Demonstrăm că distanțele minime $d(s, v)$ de la sursă la fiecare nod v nu scad niciodată pe măsură ce graful rezidual evoluează.
2. Esența demonstrației: Fie v nodul cel mai apropiat de s dintre toate cele ale căror distanțe au scăzut. Cum ar putea scădea distanța lui v de la o iterație f la următoarea iterație f'? Doar dacă în $G_{f'}$ a apărut o nouă muchie (u, v) care nu exista în G_f. Asta înseamnă că iterația f' a pompat flux în sensul $v \to u$. Dar algoritmul pompează flux doar în direcția BFS-ului. Așadar, v era strămoș al lui u în iterația f, dar a devenit succesor al lui u în iterația f', aceasta în condițiile în care $d(s, u)$ nu a scăzut. Așadar, distanța $d(s, v)$ de fapt a crescut, contradicție.
3. Demonstrăm că, pentru orice muchie (u, v), între două momente succesive cînd muchia devine critică, $d(s, u)$ crește cu cel puțin 2.
4. Distanțele nu pot fi niciodată mai mult de n.
5. Rezultă că o muchie nu poate fi critică de mai mult de $n/2$ ori, iar toate muchiile în total pot fi critice de $mn/2$ ori.

Așadar, algoritmul Edmonds-Karp va face cel mult $mn/2$ BFS-uri, avînd o complexitate totală de $\mathcal{O}(m^2n)$.

Implementarea este suficientă pentru problema Download Speed.

34.3.1 Optimizare

Infoarena menționează o optimizare care îmbunătățește timpii pentru problema Maxflow (dar nu și pentru problema Download Speed, unde chiar merge mai încet pe multe teste). Algoritmul calculează un arbore de părinți BFS cu efort $\mathcal{O}(m)$. Apoi încearcă să obțină beneficiul maxim din acest efort, căutînd mai multe drumuri de creștere. Nu ne oprim după ce găsim un drum de creștere prin predecesorul lui t în arborele BFS, ci căutăm drumuri de creștere pornind de la t prin **toți** predecesorii lui t. Unele dintre aceste căi vor avea valoarea 0, căci vor fi deja saturate. Totuși, algoritmul poate găsi mai multe drumuri de creștere.

Am inclus și o sursă cu această optimizare la problema Download Speed.

34.4 Fluxuri și tăieturi

Să considerăm acum problema Police Chase. Ea ne cere să eliminăm cît mai puține muchii dintr-un graf (neorientat, dar soluția este identică și în grafuri orientate) astfel încît să deconectăm un nod s de un nod t. Să vedem care este legătura acestei cerințe cu subiectul fluxurilor.

Am mai vorbit despre tăieturi în lecția despre arbori parțiali minimi. Vă reamintesc că o tăietură este o partiționare a grafului în două mulțimi de noduri disjuncte. Cînd vorbim despre fluxuri, ne interesează acele tăieturi care separă sursa s de destinația t. Convențional, notăm cele două mulțimi cu S și T.

Definiția 9 (Fluxul net)**.** Fluxul net printr-o tăietură (S, T) este

$$\sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

Teorema 2. Pentru un flux dat f, valoarea fluxului net prin orice tăietură $S - T$ este $|f|$.

O demonstrație facilă este inductivă. Pentru $S = \{s\}$ și $T = V - \{s\}$, în mod evident fluxul este $|f|$. Acum, să presupunem că printr-o tăietură $S - T$ avem fluxul net $|f|$. Să „tragem" de frontieră pentru a muta exact un nod u din T în S, deci fie $S' = S \cup \{u\}$ și $T' = T - \{u\}$. Ce s-a modificat? Vom vedea că am pierdut și am cîștigat exact muchii care intră și ies din u, iar suma contribuțiilor lor (datorită conservării fluxului) este 0. Deci valoarea fluxului net prin tăieturile $S - T$ și $S' - T'$ este aceeași.

Definiția 10 (Capacitatea unei tăieturi)**.** Capacitatea unei tăieturi (S, T) este

$$\sum_{u \in S} \sum_{v \in T} c(u, v)$$

Este relativ clar că prin nicio tăietură nu putem transmite mai mult flux decît capacitatea tăieturii (pe unde s-ar duce?). Așadar, fluxul maxim este mai mic sau egal cu capacitatea oricărei tăieturi. Alternativ, fluxul maxim este mai mic sau egal cu capacitatea tăieturii minime. Următoarea teoremă arată că, de fapt, cele două valori sînt chiar egale.

Teorema 3 (Max-flow min-cut)**.** Următoarele trei condiții sînt echivalente:

1. f este un flux maxim în G.
2. Nu există niciun drum de creștere în rețeaua reziduală G_f.
3. Există o tăietură (S, T) în G pentru care $c(S, T) = |f|$.

Demonstrația este circulară. Le enunțăm pe cele simple primele:

(1) $\implies$ (2): Dacă ar exista un drum de creștere, l-am putea folosi ca să creștem fluxul, deci f nu ar fi maxim.

(3) $\implies$ (1): Știm deja că orice flux este cel mult egal cu orice tăietură. Deci, dacă găsim un flux egal cu o tăietură, atunci fluxul este maxim.

(2) $\implies$ (3): Esența demonstrației este alegerea tăieturii. Nu întîmplător, fie S mulțimea nodurilor care încă sînt accesibile din s în G_f și fie T mulțimea nodurilor inaccesibile. Desigur, $t \in T$ căci altfel t ar fi accesibil din S și ar contrazice (2). Fie acum orice pereche de noduri $u \in S$ și $v \in T$. Iau naștere trei cazuri posibile:

1. Dacă $(u, v) \in E$, atunci obligatoriu muchia este saturată, altfel am mai putea pompa flux pe ea, deci $(u, v) \in G_f$, deci $v \in S$, contradicție. Așadar, $f(u, v) = c(u, v)$.
2. Dacă $(v, u) \in E$, atunci obligatoriu muchia este nefolosită, altfel am mai putea scoate flux pe ea, deci $c_f(u, v) > 0$, deci $v \in S$, contradicție. Așadar, $f(u, v) = 0$.

3. Altfel nu există muchii între u și v și automat $f(u,v) = f(v,u) = 0$.

Atunci fluxul net prin tăietura (S,T) este:

$$|f| = f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u) = \sum_{u \in S} \sum_{v \in T} c(u,v) - \sum_{u \in S} \sum_{v \in T} 0 = c(S,T)$$

34.4.1 Aflarea tăieturii minime

Implementarea (care rezolvă problema Mincut pe NerdArena) cere doar 1% efort suplimentar după găsirea fluxului maxim. Ultimul BFS, cel care constată că destinația t nu mai este accesibilă, lasă în cîmpul parent valori pozitive pentru nodurile accesibile, dar valoarea specială NONE pentru noduri inaccesibile. Trebuie doar să tipărim muchiile care unesc noduri accesibile cu noduri inaccesibile.

Există totuși un detaliu semnificativ. Să considerăm figura 24.6 din Cormen. Fluxul maxim este 23. În ultima rețea reziduală, tăietura este $S = \{s, v_1, v_2, v_4\}$ și $T = \{v_3, t\}$. Dar suma muchiilor care traversează tăietura este $12 + 9 + 7 + 4 = 32$. De unde apare discrepanța? Este important să ne amintim că pe muchia v_3, v_2 nu circulă flux de valoare 9, ci avem o capacitate de 9, nefolosită! Prin contrast, pe muchiile $12 + 7 + 4$ chiar circulă flux. Cu alte cuvinte, este vital să adunăm doar fluxul efectiv (muchii originale saturate), nu capacitățile reziduale pe care le stocăm de regulă în noduri.

34.5 Algoritmul lui Dinic

Algoritmul lui Dinic (Yefim Dinitz, 1970) este anterior algoritmului Edmonds-Karp (1972), dar a devenit popular mai tîrziu decît acesta din urmă. Acest lucru se întîmpla des în timpul Războiului rece, din cauza izolării dintre URSS și Occident. Vezi algoritmul criptografic RSA pentru un caz similar.

Ca resurse suplimentare, vă recomand CP Algorithms și Wikipedia, de unde vom folosi imaginile.

Algoritmii lui Dinic și Edmonds-Karp au în comun ideea drumurilor celor mai scurte (găsite cu BFS). În plus, algoritmul lui Dinic introduce trei noțiuni pentru a atinge o complexitate mai bună, respectiv $\mathcal{O}(n^2 m)$ față de $\mathcal{O}(nm^2)$ cu Edmonds-Karp.

Definiția 11 (Muchie admisibilă)**.** O muchie admisibilă într-o rețea de flux este o muchie care se îndepărtează de sursă. Cu alte cuvinte, este o muchie pe care distanța nodurilor crește cu 1 sau, echivalent, o muchie care face parte dintr-un drum minim de la s.

Definiția 12 (Rețea pe niveluri)**.** Rețeaua pe niveluri (engl. *level graph*, *layered network*) a unei rețele reziduale G_f este un subgraf al lui G_f în care păstrăm doar muchiile admisibile.

Definiția 13 (Blocaj de flux)**.** Un blocaj de flux (engl. *blocking flow*) este un flux care deconectează s de t în rețeaua pe niveluri[1].

[1]Clarificare: Vă reamintesc că un singur drum de creștere nu deconectează obligatoriu s de t. După cum arată

Algoritmul lui Dinic este, pur și simplu:

Algoritmul 6 Algoritmul lui Dinic

```
1: cît timp d[t] < ∞ execută
2:     calculează rețeaua pe niveluri, G_L
3:     caută un blocaj de flux
4:     augmentează fluxul total cu acel blocaj
5:     refă rețeaua reziduală
```

În practică, noi procedăm mai simplu. Nu construim explicit rețeaua pe niveluri, ci menținem aceeași rețea reziduală ca în toți algoritmii anteriori. Calculăm distanțele BFS exact ca în algoritmul Edmonds-Karp. Există două diferențe. Unu, în vreme ce Edmonds-Karp pompează flux doar pe muchiile de arbore BFS, Dinic pompează pe orice muchie pe care distanța BFS crește cu 1. Doi, Dinic caută mai multe drumuri de creștere, nu doar unul.

34.5.1 Demonstrație de corectitudine

Dacă nu mai există drumuri de la s la t în rețeaua pe niveluri G_L, atunci nu mai există drumuri nici în rețeaua reziduală G_f. De ce? Să presupunem prin absurd că ar mai exista un drum p în G_f. Atunci există unul sau mai multe noduri în p unde distanța rămîne constantă sau scade față de nodul anterior. Dar, pentru fiecare astfel de nod, putem construi un alt prefix unde distanțele chiar cresc cu 1 la fiecare pas. Acel drum ar fi în G_L, ceea ce contrazice presupunerea. Rezultă că nu mai există drumuri în G_f, deci fluxul găsit pînă la acel moment este maxim.

34.5.2 Găsirea unui blocaj de flux

Linia 2 a algoritmului de mai sus o implementăm cu un BFS, ca și în cazul lui Edmonds-Karp. BFS-ul doar calculează distanțele, nimic altceva. Liniile 3-5 le implementăm cu un DFS care amintește un pic de implementarea naivă de Ford-Fulkerson (cea în $\mathcal{O}(m \cdot |f|)$. Repetăm acest DFS cît timp mai putem pompa flux de la s la t.

Am inclus o implementare completă la problema Download Speed.

34.5.3 Calculul complexității

Vom calcula complexitatea unei faze (liniile 2-5), apoi vom calcula numărul maxim de faze necesare.

Complexitatea unei faze

BFS-ul durează, în mod evident, $\mathcal{O}(m)$. Ce putem spune despre DFS? La prima vedere, fiecare rulare a DFS-ului durează $\mathcal{O}(m)$ și saturează cel puțin o muchie. Aceasta ar duce la o complexitate a fiecărei faze de $\mathcal{O}(m^2)$. Dar putem calcula o limită mai strînsă folosind analiza amortizată.

și optimizarea pentru Edmonds-Karp propusă de Infoarena, pentru un BFS calculat pot exista mai multe drumuri de creștere disjuncte. Un blocaj de flux este o colecție maximală de drumuri de creștere.

- Clasificăm parcurgerile de muchii în DFS ca **avansuri** (reapelare DFS), **augmentări** (revenire de pe o cale pe care am pompat flux) și **retrageri** (revenire de pe o cale pe care nu am pompat flux).
- Avansurile sînt amortizate de augmentări sau de retrageri, deci nu trebuie contorizate.
- La o retragere, distrugem acea muchie, în sensul că nu o mai revizităm, căci știm că nu duce nicăieri. Așadar, există cel mult m retrageri.
- Fiecare cale găsită saturează cel puțin o muchie. Căile au lungime cel mult n, deci pentru fiecare n augmentări, o muchie devine saturată și dispare din graf. Așadar, pot exista cel mult mn augmentări.
- Rezultă că complexitatea găsirii unui blocaj, care dictează complexitatea unei faze, este $\mathcal{O}(mn)$.

La implementare, pentru fiecare nod, reținem un pointer în lista de adiacență care este global (se păstrează între DFS-urile aceleiași faze). La vizitarea unui nod, păstrăm pointerul dacă pompăm flux pe muchie (căci poate mai folosim acea muchie), dar îl avansăm dacă **nu** pompăm flux pe muchie.

Numărul de faze

Știm de la algoritmul Edmonds-Karp că distanțele nodurilor față de sursa s nu scad. În fapt, la algoritmul lui Dinic putem demonstra că distanța de la s la t crește cu cel puțin 1 la fiecare fază. Demonstrația aceasta este mai completă decît cea de pe CP Algorithms și procedează astfel:

1. Să considerăm rețeaua reziduală în două faze consecutive, G_f și $G_{f'}$.
2. Poate apărea o muchie nouă (u, v) doar dacă în G_f am pompat flux de la v spre u. Iar, dat fiind că Dinic pompează flux doar pe muchii admisibile (în sensul crescător al distanțelor), rezultă că muchia (v, u) era admisibilă, iar (u, v) nu era admisibilă în G_f. Generalizînd, orice muchii nou apărute în $G_{f'}$ nu erau admisibile în G_f.
3. Dacă mai există vreun drum de la s la t în $G_{f'}$, înseamnă că undeva pe acel drum există o muchie nou apărută, deci care era inadmisibilă în G_f.
4. Cu alte cuvinte, acest drum „pierde” un timp față de drumul anterior, cel din G_f, mergînd pe o muchie care nu avansează în graful BFS.
5. Pentru a recupera acest timp și a păstra distanța $s - t$ în $G_{f'}$ egală cu cea din G_f, ar trebui ca drumul să crească distanța cu 2 altundeva.
6. Dar aceasta este imposibil, căci distanța BFS pe orice muchie crește cu cel mult 1.

Rezultă că pot exista cel mult n faze, deci complexitatea totală este $\mathcal{O}(mn^2)$.

34.6 Lectură suplimentară

Direcții de explorat, pentru cei interesați:

- Fluxul maxim de cost minim

- Algoritmi de flux de tip push-relabel, dacă vă este dor de Tarjan. Sînt o clasă de algoritmi cu o abordare diferită de cea studiată (drumuri de creștere).

34.7 Probleme

34.7.1 Problema Download Speed (CSES)

enunț • surse

Problema este educațională și necesită aflarea fluxului maxim.

34.7.2 Problema Police Chase (CSES)

enunț • sursă

Problema este educațională și necesită aflarea tăieturii minime. Străzile au capacitate 1 și sînt bidirecționale. Pentru implementare, este suficient să creăm două muchii de capacitate 1, astfel încît fluxul să poată fi 1 în orice sens (și 1 rezidual în sensul opus). Nu este nevoie să creăm două perechi de muchii.

O problemă 99% identică este Mincut (NerdArena). Diferența minoră este că muchiile sînt unidirecționale și pot avea capacități naturale. Iată o sursă online.

34.7.3 Problema Distinct Routes (CSES)

enunț • sursă

Ideea nu este grea. Privim fiecare teleportor ca pe o muchie de capacitate 1. În acest fel, fiecare unitate de flux pe care o putem pompa va merge pe o cale diferită. La final, colectăm și eliminăm rutele una cîte una.

Întrebare: Am putea folosi algoritmul Ford Fulkerson naiv, cu DFS? Avantajul ar fi că putem colecta rutele imediat ce le găsim. Nu este nicio problemă, căci oricum ieșirea problemei este proporțională cu mărimea fluxului. Nu-i așa?

Nu, deoarece o iterație de flux (fie cu Ford-Fulkerson, fie cu Edmonds-Karp) poate anula unități de flux pompate în iterațiile anterioare. Atunci am folosi un teleportor în sensul greșit.

34.7.4 Problema Soldier and Traveling (Codeforces)

enunț • sursă

De regulă problemele de flux nu sînt formulate ca atare, ci trebuie să le reduceți la calculul unui flux. De asemenea, un mecanism întîlnit frecvent este dublarea nodurilor. Pentru problema de față,

- Construim nodurile $x_1, x_2, \ldots, x_n$ și $y_1, y_2, \ldots, y_n$, așezate vizual pe două coloane (x-ii în stînga, y-ii în dreapta). Conceptual, x este configurația trupelor înainte de mutare, iar y după mutare.
- Modelăm muchiile grafului original: pentru fiecare muchie (u, v) trasăm muchiile (x_u, y_v) și (x_v, y_u), de capacitate infinită. Astfel arătăm că oricîți soldați se pot muta între orașe pe muchiile existente.
- Trasăm muchii între fiecare pereche (x_i, y_i), de asemenea de capacitate infinită. Astfel arătăm că oricîți soldați pot rămîne pe loc în orașul i.
- Adăugăm o sursă s și o unim cu fiecare x_i printr-o muchie de capacitate a_i. Astfel arătăm că numărul total de soldați din orașul i este a_i și pentru fiecare dintre ei trebuie să decidem o acțiune. Acțiunea se traduce prin pomparea unei unități de flux, fie în y_i fie într-un oraș adiacent.
- Similar, adăugăm o destinație t și unim fiecare y_i cu aceasta printr-o muchie de capacitate b_i.

Cu această reprezentare, problema are soluție dacă și numai dacă există un flux de capacitate $\sum a_i$ și în plus $\sum a_i = \sum b_i$. Descrierea traseelor unităților de flux nu este unică. Evaluatorul acceptă orice soluție.

34.7.5 Problema Fox and Dinner (Codeforces)

enunț • sursă

Cum modelăm această problemă ca pe una de flux?

Să observăm că orice ciclu va alterna între numere pare și numere impare. Altfel, dacă alăturăm două numere pare sau două numere impare, suma va fi pară, deci nu va putea fi primă. Așadar, o primă condiție este ca n să fie par, iar numerele să fie jumătate pare și jumătate impare.

Ne vine astfel ideea să trasăm muchii între fiecare pereche de numere de sumă primă. Aceasta împarte natural graful în două mulțimi sau coloane: numerele pare la stînga, numerele impare la dreapta. Acum orice muchie va fi orientată de la stînga la dreapta. Să acordăm tuturor muchiilor capacitate 1.

Acum, dorim să producem cicluri de lungime cel puțin 4, căci enunțul interzice ciclurile de lugime 2. Rezultă că fiecare nod trebuie conectat cu exact două noduri din tabăra opusă (vecinii săi de la masă). De aceea, să adăugăm o sursă pe care o unim prin muchii de capacitate 2 cu toate numerele pare. Similar, să adăugăm o destinație pe care o unim prin muchii de capacitate 2 cu toate numerele impare.

Rețeaua este acum gata și trebuie să vedem dacă putem pompa n unități de flux prin ea.

Algoritmul Edmonds-Karp simplu (fără încercarea de optimizare) pare să fie suficient. Este posibil ca și metoda Ford-Fulkerson să fie suficientă, căci fluxul maxim este 100.

La final, trebuie să colectăm ciclurile rezultate. Remarcăm că de la stînga la dreapta mergem pe

muchii saturate (capacitate reziduală 0), iar de la dreapta la stînga revenim pe muchii reziduale (capacitate reziduală 1). Astfel, putem să iterăm prin toate valorile pare și, pentru fiecare valoare încă neexplorată, să urmăm muchiile pînă închidem un ciclu. Deoarece pentru orice unitate de flux la intrare există și una la ieșire, acest algoritm nu se va bloca niciodată, ci va găsi toate ciclurile.

34.7.6 Problema Petya and Graph (Codeforces)

enunț • sursă

Problema este relativ clasică. Haideți să prezentăm o formă mai generală.

Există m proiecte și n instrumente. Fiecare proiect are o listă de instrumente necesare ca să putem termina proiectul. Pentru fiecare proiect terminat i primim un venit p_i și fiecare instrument i are un cost c_i. Dacă cumpărăm un instrument, îl putem folosi la oricîte proiecte. Trebuie să găsim o mulțime de proiecte de terminat și instrumente de cumpărat ca să maximizăm profitul.

Trecerea de la problema dată la aceasta este simplă: muchiile sînt proiecte, iar nodurile sînt instrumente. Într-adevăr,

- Muchiile contribuie cu + la profitul total.
- Nodurile contribuie cu − la profitul total.
- Putem selecta o muchie doar dacă îi selectăm ambele noduri.
- Odată plătit costul unui nod, putem folosi nodul la oricîte muchii.
- Problema dată este mai particulară, deoarece o muchie depinde de exact două noduri, pe cînd un proiect poate depinde de oricîte instrumente.

Vom folosi terminologia cu proiecte și instrumente pentru a nu crea confuzie între graful dat la intrare și rețeaua de flux indusă.

Acum, reducerea la rețele de flux poate fi simplă, dar demonstrația de corectitudine nu. Putem încerca diverse variante de modelare pînă cînd ajungem la aceasta:

- Pe o coloană creăm cîte un nod A_i pentru fiecare proiect i.
- Pe o altă coloană creăm cîte un nod B_j pentru fiecare instrument j.
- În plus, folosim o sursă s și o destinație t.
- Pentru fiecare proiect i creăm o muchie de la s la A_i de capacitate p_i.
- Pentru fiecare instrument j creăm o muchie de la B_j la t de capacitate c_j.
- De la fiecare proiect la fiecare instrument de care depinde creăm o muchie de capacitate ∞.

De ce așa? Ce ar însemna un flux maxim în această problemă? Să facem următoarele observații.

Observație. O unitate de flux pompată prin această rețea „seamănă" cu un dolar încasat pe un proiect, care se duce prin rețeaua de flux și plătește un dolar pentru un instrument de care proiectul depinde.

Observație. Dacă saturăm o muchie (s, A_i), atunci „pare" că nu rentează să terminăm proiectul i, întrucît toate veniturile sale (p_i) le-am cheltuit pe instrumente.

Observație. Dacă după găsirea fluxului maxim nu saturăm o muchie (s, A_i), atunci înseamnă că nu mai putem pompa nicio unitate pe ea. De ce? Pentru că toate căile cu care se continuă muchia sînt saturate. De la proiect la instrumentele de care el depinde capacitățile sînt infinite, prin construcție. Înseamnă că muchiile saturate sînt de la toate aceste instrumente la t. Cu alte cuvinte, proiectul și-a cumpărat instrumentele, iar capacitatea reziduală (pozitivă) pe muchia (s, A_i) este profitul, deci proiectul merită terminat.

Observație. Dacă o muchie (B_j, t) este saturată, înseamnă că unul sau mai multe proiecte au găsit cu cale să satureze muchia, pentru a rămîne cu profit rezidual conform observației 3. Deci instrumentul j merită cumpărat. Invers, dacă muchia (B_j, t) rămîne nesaturată, atunci nu mai există niciun mod de a pompa vreo unitate de flux de la s către vreun proiect care folosește instrumentul j. Toate aceste proiecte sînt saturate, deci conform Observației 3 sînt neprofitabile. Instrumentul j nu merită cumpărat.

Pe mine m-a ajutat o schimbare de perspectivă ca să formalizez aceste observații intuitive. Să considerăm că am încasat (în avans) banii pentru toate proiectele. Acum trebuie să cumpărăm o submulțime a instrumentelor. Desigur, trebuie să plătim costul acestei submulțimi. Mai mult, pentru orice proiect i care depinde de un instrument care nu este în submulțime, trebuie să returnăm venitul p_i. Urmărim să minimizăm suma dintre instrumentele cumpărate și profiturile returnate.

TODO: figură

De aici mai avem doar un pas, dar fluxul maxim nu ne oferă imaginea corectă. Mult mai de ajutor ne este tăietura minimă! Vom folosi algoritmul de flux maxim doar ca să aflăm valoarea tăieturii minime (știm că ele sînt egale).

Noi dorim să evităm scenariul în care păstrăm un proiect i, dar refuzăm să cumpărăm un instrument necesar j. Să modelăm acest lucru ca pe calea $s \to A_i \to B_j \to t$ în rețeaua de flux. Dorim să nu existe nicio astfel de cale. Hai să găsim o colecție de muchii pe care să le eliminăm ca să ne îndeplinim dorința!

Nu putem tăia muchii dintre A_i și B_j. Asta ar simboliza că ne prefacem că proiectul i nu depinde de instrumentul j. 🙂 Dar putem tăia muchii de tipul (s, A_i) sau (B_j, t). O muchie (s, A_i) tăiată înseamnă că renunțăm la proiectul i, iar o muchie (B_j, t) tăiată înseamnă că cumpărăm instrumentul j.

Prin definiție, cînd spunem „vrem să nu existe nicio cale $s \to A_i \to B_j \to t$”, spunem că vrem să deconectăm s de t. Și vrem asta cu cost minim. Am ajuns exact la definiția tăieturii minime!

Așadar, profitul maxim pe care îl putem obține este suma veniturilor proiectelor, minus costul tăieturii minime pe care sîntem obligați să îl plătim (instrumente cumpărate + proiecte la care am renunțat). Iar costul tăieturii minime este egal cu costul fluxului maxim.

34.7.7 Problema Asfalt (Lot 2011)

enunț • sursă

După formalizare, înțelegem că problema ne cere să identificăm un set minim de muchii care să acopere toate drumurile minime de la S la D. Problema mi-a amintit de problemele President and Roads și Edges in MST, care cereau clasificarea muchiilor în drumuri minime, respectiv în arborii parțiali minimi.

Dacă problema încă pare greu de „apucat", să ne gîndim la cazuri particulare. Dacă există o punte între S și D, atunci în mod necesar ea este parte din toate drumurile minime, deci setul constă doar din ea. Dar nu este atît de simplu. Poate exista o muchie e care nu este factual punte (face parte dintr-un ciclu), dar restul muchiilor de pe ciclu sînt atît de scumpe, încît practic toate drumurile minime trec prin e. Și atunci soluția constă doar din e.

Așa ajungem la ideea de clasificare a muchiilor în trei categorii (în practică, doar două):

1. Muchii care nu fac parte din niciun drum minim. Acestea nu vor face parte niciodată din soluție, căci scumpirea lor nu afectează drumurile minime.
2. Muchiile care fac parte din orice drum minim. Oricare dintre acestea poate constitui, de una singură, o soluție.
3. Muchiile care fac parte din unele drumuri minime. Acestea trebuie combinate cu alte muchii pentru a găsi o soluție de cardinalitate cît mai mică.

De aici ne vine ideea tăieturii. Aruncăm muchiile din categoria 1 și le păstrăm doar pe cele din categoriile 2 și 3. Vă mai amintiți cum le depistăm? Facem două măsurători de distanțe cu algoritmul Dijkstra, odată pornind din S și o dată din D. Apoi, o muchie (u, v) este pe cel puțin un drum minim dacă și numai dacă

$$d(S, u) + c(u, v) + d(v, D) = d(S, D)$$

Pe acest graf nu ne mai interesează distanțele. Ne interesează să găsim un număr minim de muchii care să deconecteze S de D. Este exact problema tăieturii minime.

Un detaliu de implementare: Am ales să refolosesc muchiile grafului inițial. Cîmpul `c`, inițial folosit pentru distanțe în algoritmul lui Dijkstra, stochează capacități în algoritmul Edmonds-Karp. Muchiile care fac parte din rețeaua de flux capătă capacitate 1 doar în sensul indicat de egalitatea de mai sus. Muchiile care nu fac parte din rețeaua de flux capătă capacitate 0 în ambele sensuri.

După cum am vorbit anterior, tăietura minimă constă doar din fluxul pe muchiile directe, nu și pe cele reziduale. Spre deosebire de problema Mincut, unde încă de la citirea grafului știam că muchiile pare sînt directe, iar cele impare sînt reziduale (chiar noi le atribuiam astfel), aici ele pot surveni oricum, după cum se nimeresc distanțele. Așadar, în rutina `convert_to_flow_network()` am decis să setez explicit un cîmp boolean forward pe muchiile directe.

34.7.8 Problema Echipe (Lot 2025)

enunț • sursă

Iată o problemă extrem de dură. În concurs, doar Mihai Voicu a obținut 100 de puncte. Următoarele punctaje au fost 45, 23, 14 și restul sub 9. Deci nu ne punem problema să o înțelegem și să o codăm repede, ci doar să găsim o soluție în ritmul nostru (a mea are 300 de linii efective și mi-a consumat cam două zile).

Găsirea numărului de echipe

Compatibilitatea oamenilor în echipe ține doar de atributele lor, nu de risc. De aceea, numărul maxim de echipe, să-i spunem E, este constant și îl putem calcula o singură dată. La schimbarea scorului, reorganizăm echipele dar nu adăugăm și nu ștergem echipe.

Sper că de acum începeți să intuiți reducerea la flux: mapăm oamenii la rolurile pe care le pot îndeplini. Deoarece ar fi mult prea scump să construim un graf cu $\mathcal{O}(n)$ noduri, vom colecta doar frecvențele diferitelor categorii (combinații de atribute), numerotate de la 1 la 7 conform scrierii binare a atributelor. Apoi construim un graf cu:

- O mulțime A cu 7 noduri corespunzătoare celor 7 categorii.
- O mulțime B cu 4 noduri corespunzătoare celor 4 roluri (C, I, M, I/M).
- Un nod sursă s și un nod destinație t.
- Muchii de capacitate ∞ (sau n) de la fiecare categorie din A la rolurile compatibile din B.
- Muchii de la s la A, capacitatea fiecărei muchii fiind egală cu numărul de oameni din acea categorie.
- Pentru început, muchii de capacitate 0 de la B la t.

Pe acest graf dorim să găsim un flux maxim, adică să cuplăm cît mai multe persoane cu cele 4 roluri. Dar atenție! Fluxul pe cele 4 muchii de la B la t trebuie să fie egal, deoarece trebuie să avem la fel de multe persoane în fiecare rol. O implementare normală de flux nu garantează acest lucru. De aceea, procedăm astfel, în mod repetat:

- Crește cu cîte 1 capacitatea pe toate muchiile de la B la t.
- Refă fluxul și vezi dacă el a crescut cu 4.

Ne oprim cînd nu mai putem crește fluxul cu 4.

Care este complexitatea acestui prim pas? Deoarece graful are 13 noduri, putem considera parcurgerile acestui graf ca $\mathcal{O}(1)$. Folosim cea mai simplă metodă (Ford-Fulkerson cu DFS), care va face cel mult $n/4$ iterații. Complexitatea este $\mathcal{O}(n)$.

Ca implementare, am dat nume celor 13 noduri și am hardcodat matricea de adiacență. Am încapsulat tot codul referitor la flux într-o clasă `network_flow`, care ocupă circa 700 de octeți, suficient de mică încît să o putem copia în întregime. Restul programului folosește fluxul ca primitivă (operație de bază).

Găsirea riscului minim pentru riscurile inițiale

Aici orice abordare bazată pe flux este sortită eșecului. Toate riscurile pot diferi, deci nu mai putem grupa nodurile, iar algoritmii practici de flux cedează mult înainte de $n = 150\,000$.

Să ne reamintim, în schimb, că ne permitem să facem $\mathcal{O}(n)$ pompări de flux (DFS-uri) în graful de 13 noduri. Atunci putem încerca următoarea abordare greedy. Considerăm oamenii în ordine descrescătoare a riscului. Cînd considerăm persoana X din categoria A_X, încercăm să renunțăm la ea. Cu alte cuvinte, ne întrebăm: Dacă X n-ar exista, am mai putea găsi un flux cu E echipe? Dacă da, renunțăm la X. Altfel îl includem pe X în echipele inițiale.

Iată o schiță de demonstrație de corectitudine, prin reducere la absurd. Fie soluția optimă S, care îl include pe X. Să presupunem că algoritmul greșește, că renunță la X și obține o soluție neoptimă S' de risc mai mare decît S. În S' există o persoană X' care îndeplinește rolul lui X. Atunci putem să pornim de la S și să-l schimbăm pe X cu X' pentru a obține o configurație de risc și mai mic. Acest lucru este imposibil întrucît am presupus că S este optim.[2]

Acum să vedem cum implementăm algoritmul. Nu dorim să recalculăm fluxul de la 0 pentru fiecare dintre cei n oameni, pentru că am obține complexitatea $\mathcal{O}(n^2)$. Trebuie să adaptăm în permanență fluxul existent. Iau naștere două cazuri:

1. Dacă fluxul existent **nu** saturează muchia (s, A_X), atunci îl putem pur și simplu sări pe X deoarece există suficiente alte persoane din categoria A_X cu care să formăm echipe. De asemenea, reducem capacitatea muchiei cu 1. Astfel reflectăm faptul că nu putem sări oricîte persoane din categoria A_X, ci la un moment dat muchia va deveni saturată.
2. Dacă fluxul existent saturează muchia (s, A_X), atunci identificăm traseul unei unități de flux $s \to A_x \to B_y \to t$ și o eliminăm din graf. Nu creștem capacitatea pe muchia (s, A_x), deoarece dorim ca fluxul să ne dea o altă persoană, nu tot pe X. Apoi încercăm să pompăm o unitate de flux. Dacă putem, înseamnă că noul flux a reușit să reorganizeze persoanele pentru a obține $4E$ echipe, iar pe X îl putem sări.

Actualizările

Să ne gîndim la cele 4 scenarii care pot apărea cînd riscul lui X se modifică.

1. Dacă X era parte din echipe, iar riscul îi scade, atunci el va rămîne în echipe. Ar fi absurd să îl înlocuim acum – cu atît mai mult am fi avut motiv să îl înlocuim înainte de scăderea riscului.
2. Dacă X nu era parte din echipe, iar riscul îi crește, atunci el va rămîne în afara echipelor.
3. Dacă X era parte din echipe, iar riscul îi crește, atunci ar putea fi profitabil să îl înlocuim.
4. Dacă X nu era parte din echipe, iar riscul îi scade, atunci ar putea fi profitabil să îl aducem în echipe în locul altcuiva.

[2] Am spus „schiță” deoarece, de fapt, este posibil ca X' să fie deja în S, dacă diferența între S și S' presupune o reorganizare mai mare. În acel caz trebuie să căutăm persoana X'' care îl înlocuiește pe X' în S'. Astfel poate lua naștere un lanț de persoane, dar invariabil vom ajunge la o persoană care este disponibilă.

Putem demonstra că în scenariile (3) și (4) este necesară cel mult o înlocuire. Esența raționamentului este următoarea: Dacă prin modificarea riscului lui X devin necesare $k > 1$ înlocuiri pentru a reface soluția optimă, atunci puteam face $k - 1$ dintre aceste înlocuiri și înainte ca riscul lui X să se modifice și am fi obținut o soluție mai bună la acel moment.

Dar cum implementăm asta? Sînt doi pași, unul care ia în calcul doar categoria lui X, A_X, și altul care ia în calcul toate categoriile.

Așadar, la pasul 1 analizăm cîte persoane din categoria A_X are soluția (concret, care este fluxul pe muchia (s, A_X)). Fie k acest număr. Desigur că în soluție noi vom folosi persoanele cu cele mai mici k riscuri din A_x. Să menținem această informație în două multiseturi: riscurile persoanelor din soluție și riscurile persoanelor din afara soluției. (Clarificare: stocăm cîte două multiseturi pentru fiecare dintre cele 7 categorii).

Acum, cînd riscul lui X se schimbă, îl căutăm în multisetul său și îi actualizăm valoarea. Dacă în urma acestei actualizări multiseturile se încalecă, adică cea mai riscantă persoană din soluție depășește cea mai puțin riscantă persoană din afara soluției, le interschimbăm și recalculăm riscul total.

La pasul 2 ținem cont de reorganizări mai mari care pot deveni necesare, afectînd distribuția pe categorii. De exemplu, cînd riscul lui X crește, ideal am vrea să eliminăm cea mai scumpă persoană din clasa A_X și să-i spunem rețelei de flux „refă fluxul”. Din păcate, rețeaua de flux habar nu are despre riscuri. Este foarte posibil să ne dea înapoi fix persoana din clasa A_X sau o altă persoană de risc mare.

În loc de aceasta, vom încerca rînd pe rînd fiecare altă categorie A_Y. Concret,

- Dacă riscul lui X a scăzut, vom încerca pe rînd să ștergem o persoană din fiecare (altă) categorie A_Y și să inserăm o persoană din categoria A_X.
- Dacă riscul lui X a crescut, vom încerca pe rînd să inserăm o persoană din fiecare (altă) categorie A_Y și să ștergem o persoană din categoria A_X.

Pentru a ghida fluxul să ne dea categoria dorită, vom descrește cu 1 capacitatea categoriei eliminate și vom crește cu 1 capacitatea categoriei adăugate. Iată un mic fragment din cod, cel care încearcă să înlocuiască o persoană din categoria `del` cu o persoană din categoria `ins`.

```
// Încearcă să înlocuiască ultimul element inclus din del cu primul element
// exclus din ins. Operează pe graful gc, pe care îl inițializează cu o copie
// a lui g.
//
// Returnează noul risc, sau INF dacă înlocuirea este imposibilă sau neoptimă.
long long try_swap_pair(int del, int ins, network_flow* gc, long long best_so_far) {
  // Categoriile trebuie să fie diferite și trebuie să existe persoane de
  // șters și de adăugat.
  if (del == ins ||
      tc.risks_in[del].empty() ||
      tc.risks_out[ins].empty()) {
```

```
    return INF;
  }

  // Noul risc pe care l-am obține prin schimbare trebuie să fie o optimizare.
  int del_risk = *tc.risks_in[del].rbegin();
  int ins_risk = *tc.risks_out[ins].begin();
  long long new_risk = tc.total_in_risk - del_risk + ins_risk;

  if (new_risk >= best_so_far) {
    return INF; // Este o schimbare în rău.
  }

  // Acum are sens să încercăm să refacem fluxul.
  *gc = g;
  gc->remove_unit(del);
  gc->increase_source_capacity(ins);
  gc->ford_fulkerson();

  return (gc->flow == 4 * num_teams) ? new_risk : INF;
}
```

Complexitatea actualizărilor este $O(q \log n)$ datorită operațiilor pe multiseturi.

Capitolul 35

Cuplaje în grafuri bipartite

35.1 Definiții

Definiția 14 (Graf bipartit)**.** Un graf bipartit este un graf a cărui mulțime de noduri constă din două submulțimi, $V = L \cup R$, astfel încît toate muchiile duc din L în R.

Definiția 15 (Cuplaj într-un graf bipartit)**.** Un cuplaj într-un graf bipartit este o colecție de muchii astfel încît orice vîrf să apară în cel mult una dintre muchii.

Definiția 16 (Cuplaj maximal)**.** Un cuplaj maximal este un cuplaj care nu mai poate fi extins cu nicio altă muchie din E.

Definiția 17 (Cuplaj maxim)**.** Un cuplaj maxim este un cuplaj de cardinalitate maximă.

35.2 Modelarea ca problemă de flux

Similar problemelor din capitolul de fluxuri, putem modela cuplajul maxim ca problemă de flux dacă extindem graful cu următoarele elemente:

1. Un nod sursă, s.
2. Muchii de la s la fiecare nod din L, de capacitate 1.
3. Un nod destinație, t.
4. Muchii de la fiecare nod din R la t, de capacitate 1.
5. Muchiile originale dintre L și R pot avea orice capacitate între 1 și ∞ (nu contează, căci oricum primesc doar 1 din s).

Pe acest graf calculăm fluxul maxim de la s la t. Fiecare unitate de flux va urma o cale compusă din exact trei muchii. Muchia din centru va traversa din L în R și o vom socoti parte din cuplaj. Cuplajul astfel obținut este maxim.

Explicația intuitivă este că forțăm fiecare nod din L să fie cuplat cu cel mult un nod din R și viceversa. Ele nu pot fi cuplate cu mai multe noduri, întrucît primesc cel mult o unitate de flux din s, respectiv transportă cel mult o unitate de flux înspre t.

35.3 Căi alternante

Iată și cîțiva termeni specifici cuplajelor. Îi veți regăsi în articole. Sînt doar adaptări ale denumirilor specifice fluxurilor. Fie un cuplaj C (așadar o colecție de muchii disjuncte pe vîrfuri).

Definiția 18 (Nod cuplat)**.** Un nod se numește cuplat dacă are o muchie adiacentă din C. Altfel se numește necuplat.

Definiția 19 (Cale alternantă)**.** O cale alternantă este o cale care traversează alternativ muchii din C și din $E - C$.

Definiția 20 (Drum de creștere)**.** Un drum de creștere este o cale alternantă ale cărei primă și ultimă muchie fac parte din $E - C$.

Observăm că drumurile de creștere au un număr impar de muchii. Ele încep și se termină cu noduri necuplate din aceeași mulțime (L sau R).

Pentru un drum de creștere P, putem obține un nou cuplaj luînd diferența simetrică $C' = C \oplus P$ (diferența simetrică constă din toate elementele care sînt în C sau în P, dar nu în ambele). Cu alte cuvinte, pentru a obține C' eliminăm din C toate muchiile de pe cale și le adăugăm pe celelalte. Mulțimea C' este și ea un cuplaj (se verifică ușor) și conține cu o muchie în plus.

Prin fix același raționament observăm că, dacă P nu este o singură cale, ci o colecție de k căi disjuncte pe vîrfuri, atunci $C \oplus P$ este un cuplaj de cardinalitate $|C| + k$.

35.4 Algoritmul Hopcroft-Karp

Algoritmul lui Dinic, adaptat la grafuri bipartite, funcționează în $\mathcal{O}(m\sqrt{n})$ și poartă numele Hopcroft-Karp. Vedeți ce simple sînt fluxurile și cuplajele? 🙂

Algoritmul Hopcroft-Karp este formulat în termenii secțiunii anterioare:

1. Pornește cu $C = \emptyset$.
2. Cît timp se poate, găsește o colecție maximală P de căi alternante în sensul crescător al distanțelor și adaugă colecția la C.

La pasul 2 recunoaștem, de fapt, noțiunea de **blocaj de flux** din algoritmul lui Dinic. Singura diferență este că rețeaua pe niveluri are o formă particulară în grafurile bipartite. Ea constă din căi alternante.

35.4.1 Analiză de complexitate

La prima vedere, algoritmul Hopcroft-Karp are complexitatea $\mathcal{O}(mn)$: cuplajul maxim poate avea valoarea $\min(|L|, |R|) = \mathcal{O}(n)$, deci pot exista $\mathcal{O}(n)$ faze. Într-o fază, găsirea unei colecții de căi alternante durează $\mathcal{O}(m + n)$ deoarece DFS-ul nu va vizita de două ori aceeași muchie.

De fapt, putem da o limită mult mai puternică. Dacă facem căutarea căilor alternante cu BFS în rețeaua pe niveluri, atunci numărul de faze poate fi cel mult $2\sqrt{n}$. Demonstrația este mai simplă

decît pare, căci refolosim ce știm de la algoritmul lui Dinic:

1. Distanța de la s la t crește cu cel puțin 1 la fiecare fază, deci după $\sqrt{n}$ faze orice cale viitoare de la s la t va avea distanță mai mare decît $\sqrt{n}$.
2. După $\sqrt{n}$ faze, ce se mai poate întîmpla pînă la găsirea cuplajului maxim? Mai putem găsi un număr de drumuri de creștere, disjuncte pe vîrfuri, de lungime cel puțin $\sqrt{n}$. De aceea, există cel mult $\sqrt{n}$ astfel de drumuri, deci algoritmul va dura cel mult încă $\sqrt{n}$ faze.

35.4.2 Detalii de implementare

Implementarea este particularizată pentru grafuri bipartite și diferă mult de cea a algoritmului lui Dinic:

1. Nu mai extinde graful cu s și t.
2. Nu mai dublează muchiile ca la grafurile neorientate.
3. Nu mai construiește muchii reziduale.
4. Așadar, ține efectiv m muchii grupate în liste de adiacență doar pentru nodurile din stînga.
5. Nodurile drepte au ca unică informație un cîmp `pair`: nodul stîng cu care sînt cuplate printr-o muchie sau 0 pentru nodurile necuplate.
6. Nodurile stîngi au și ele cîmpul `pair` (nodul drept cu care sînt cuplate), plus lista de adiacență, plus un cîmp de distanță pentru BFS.

Aceste informații sînt suficiente pentru a rula algoritmii BFS și DFS specifici lui Dinic / Hopcroft-Karp. Problema School Dance conține o implementare de referință.

(Menționez cu tristă veselie că unul dintre primele rezultate la căutarea *[Hopcroft-Karp implementation]* conține un link la cod necitibil, pe care autorul îl consideră *easy to follow*. Dacă nu vă veți aminti nimic altceva din acest cerc, amintiți-vă doar să vă feriți de acest stil de codare.)

35.5 Algoritmul lui Kuhn

Algoritmul lui Kuhn este mai slab din punct de vedere al complexității: $\mathcal{O}(mn)$. Totuși, el este foarte ușor de implementat, așa că îl menționăm:

Algoritmul 7 Algoritmul lui Kuhn

```
inițializează un cuplaj vid
pentru fiecare nod u din mulțimea stîngă execută
    caută cu DFS un drum de creștere care pornește din u
```

35.5.1 Detalii de implementare

Problema School Dance conține o implementare completă. Algoritmul lui Kuhn stochează, pe lîngă lista de adiacență, doar două informații:

1. Pentru fiecare nod drept, perechea sa din stînga (sau NIL dacă nodul nu este cuplat).

2. Pentru fiecare nod stîng, un bit de vizitare.

În esență, algoritmul lui Kuhn implementează metoda Ford Fulkerson cu DFS. Observăm că, în bucla `for`, nu este necesar să omitem explicit nodurile stîngi deja cuplate. Ele nu vor putea fi cuplate a doua oară, deoarece au cîmpul de vizitare deja marcat, deci DFS-ul le va ocoli.

35.6 Teorema lui Kőnig

Trebuie să acoperim, măcar în treacăt, și acest subiect, pentru că se leagă de cele discutate pînă acum. Nu vom include decît o schiță de demonstrație și nu vom face aplicații, dar sper să vă ajute să recunoașteți conceptul dacă vă veți întîlni cu el.

Definiția 21 (Acoperire cu vîrfuri)**.** O acoperire cu vîrfuri a unui graf (engl. *vertex cover*) este un set de vîrfuri C cu proprietatea că orice muchie $e \in E$ are cel puțin un capăt în C.

Definiția 22 (Acoperire minimă cu vîrfuri)**.** O acoperire minimă cu vîrfuri (engl. *MVC, minimum vertex cover*) este o acoperire cu număr minim de noduri.

În grafuri generale, găsirea unei acoperiri minime este o problemă NP-hard. În grafuri bipartite, însă, putem găsi o soluție folosind Teorema lui Kőnig:

Teorema 4 (Teorema lui Kőnig)**.** În orice graf bipartit, mărimea cuplajului maxim este egală cu mărimea acoperirii minime cu vîrfuri.

Dacă vedeți o similitudine cu teorema flux maxim - tăietură minimă, aveți dreptate! Vom urmări demonstrația de pe Wikipedia, care pornește de la reducerea cuplajului maxim la problema fluxului maxim. Vom împrumuta imaginea din această demonstrație și vom detalia pașii demonstrației.

- Se demonstrează ușor că orice acoperire este mai mare sau egală cu orice cuplaj. Orice cuplaj de mărime k constă din k muchii distincte pe noduri. Pentru a „vedea” cele k muchii, orice acoperire trebuie să selecteze cel puțin k noduri.
- De aceea, dacă putem da un exemplu de cuplaj **egal** cu o acoperire, atunci cuplajul este maxim, iar acoperirea este minimă.
- Dat fiind un cuplaj maxim, vom deriva o acoperire minimă, trecînd prin intermediul tăieturii minime.
- Ca și la flux, alegem tăietura $S - T$ în care S constă din nodurile încă accesibile din s.
- Definim capacități infinite pe muchiile $A - B$, ceea ce știm că nu dăunează (nodurile din A primesc oricum doar o unitate de flux din s).
- Astfel este evident că nu există muchii din A_S în B_T: dacă ar exista, atunci tăietura $S - T$ ar avea capacitate infinită, or noi știm că ea este egală cu cuplajul maxim, care este desigur finit.
- (Un alt argument este că noi știm, din lecția de fluxuri, că orice muchii din S în T sînt saturate, ceea ce este imposibil cînd capacitatea este infinită.)
- Graful este orientat, deci nu există nici muchii de la B_S la A_T.

- Și atunci care este capacitatea tăieturii? Este capacitatea muchiilor care traversează tăietura, adică (s, u) unde $u \in A_T$ sau (v, t) unde $v \in B_S$. Rezultă că și valoarea fluxului maxim este $|A_T| + |B_S|$.
- Apoi alegem submulțimea $A_T \cup B_S$ și demonstrăm că ea acoperă toate muchiile grafului bipartit. Într-adevăr, singura combinație de submulțimi neacoperite este (u, v) cu $u \in A_S$ și $v \in B_T$, între care am demonstrat mai sus că nu există muchii.
- Fiind egală cu cuplajul maxim, acoperirea $A_T \cup B_S$ este minimă.

35.7 Probleme

35.7.1 Problema School Dance (CSES)

enunț • surse

Problema este educațională și necesită aflarea cuplajului maxim.

35.7.2 Problema Revoluție (ONI 2009, clasele 11-12)

enunț • sursă

Dacă aveți destulă experiență cu cuplajele în grafuri bipartite, s-ar putea să găsiți imediat reducerea problemei Revoluție. Trebuie să găsim un cuplaj de la linii la coloane cu condiția că putem cupla linia i cu coloana j numai dacă $A[i][j] = 1$.

Putem ajunge la aceeași concluzie și analizînd problema de la zero. Două valori de 1 pe aceeași coloană vor rămîne întotdeauna pe aceeași coloană:

- La interschimbarea a două linii, valorile își păstrează coloana.
- La interschimbarea a două coloane, valorile vor migra împreună pe noua coloană.

Aceasta înseamnă că, oricîte interschimbări am face, putem folosi exact un 1 de pe fiecare linie și de pe fiecare coloană. Deci, pentru a putea aduce n valori de 1 pe diagonală, trebuie în primă fază să găsim n valori de 1 pe linii și pe coloane diferite. Echivalent, pentru fiecare linie trebuie să selectăm o celulă care conține un 1, astfel încît toate celulele selectate să fie pe coloane diferite. Aceasta este exact problema cuplajului.

Odată găsit cuplajul, este nevoie de un minim de atenție ca să generăm operațiile corecte. Este suficient să iterăm fie prin linii, fie prin coloane, completînd diagonala principală de sus în jos. Pe măsură ce emitem operații de interschimbare, trebuie să actualizăm cuplajul, altfel operațiile viitoare vor fi incorecte.

35.7.3 Problema Valuable Paper (Codeforces)

enunț • sursă

Problema ne dă un graf cu valori pe muchii (nu capacități) și ne întreabă care este cea mai mică

valoare d pentru care, selectînd doar muchiile cu valori $\leq d$, putem găsi un cuplaj maxim. Este nevoie să ne vină ideea căutării binare: căutăm binar cea mai mică valoare suficientă. Pentru o greutate fixată, d, ne întrebăm: există un cuplaj maxim care folosește doar muchii de greutate $\leq d$?

Vă reamintesc (pe scurt): pentru ca problema să fie rezolvabilă cu căutare binară, trebuie ca o soluție pentru d să garanteze existența soluțiilor pentru orice $d' > d$. Cu alte cuvinte, șirul boolean

$$E_d = 1 \text{ dacă și numai dacă există un cuplaj maxim cu muchii de valoare } \leq d$$

trebuie să aibă forma $0, 0, 0, \ldots, 0, 1, \ldots, 1, 1, 1$. Nu uitați să verificați acest lucru! Dacă problema nu are această proprietate, atunci nu putem căuta binar soluția optimă.

Detalii de implementare

Este nevoie de Hopcroft-Karp, căci n este mare.

Căutarea binară trebuie să trateze situația în care nu există cuplaj nici măcar folosind toate muchiile. Și în aceste condiții, încă putem să scriem codul standard, cu un singur `while` și un singur `if`.

Putem căuta binar doar în cele m muchii date. Nu este nevoie să căutăm în tot spațiul de greutăți. Diferența este semnificativă (10^5 față de 10^9, deci raportul logaritmilor este 5:9).

Putem să sortăm muchiile după greutate și să adăugăm o santinelă infinită la sfîrșitul fiecărei liste de adiacență. Atunci fiecare iterație de BFS și DFS va funcționa în $\mathcal{O}(m_d)$ (mulțimea muchiilor de greutate $\leq d$), nu în $\mathcal{O}(m)$.

Puteți ține multe valori pe `short`, ceea ce ajută cache-ul.

35.7.4 Problema Teroriști (Baraj ONI 2010)

enunț • sursă

Sumarul problemei este: Se dau m cartonașe cu două fețe care au pe fiecare față cîte o literă. Se mai dă un mesaj de $n \leq m$ litere. Să se compună mesajul folosind cartonașele.

Reducerea la cuplaj este relativ directă, dar este insuficientă: creăm $m+n$ noduri pentru cartonașe și pentru litere. Trasăm muchii de la fiecare cartonaș (a, b) la toate instanțele literelor a și b din mesaj. Calculăm cuplajul maxim. Dar algoritmul Hopcroft-Karp va fi, probabil, prea lent.

Putem, în schimb, să grupăm toate cartonașele identice și să le calculăm frecvența. Similar, calculăm frecvența fiecărei litere din mesaj. Astfel obținem cel mult $26 \times 27/2 = 351$ noduri în stînga și 26 în dreapta, pe care calculăm fluxul maxim.

Implementarea este interesantă, căci ne cere și să enumerăm muchiile fluxului.

35.7.5 Problema Brevity Is the Soul of Wit (Codeforces)

enunț • sursă

Problema ne dă la intrare $n \leq 200$ șiruri între 1 și 10 caractere și ne cere să înlocuim fiecare șir cu o prescurtare a sa obținută astfel:

- Prescurtarea trebuie să constituie un subșir (eventual pe sărite) al șirului original.
- Prescurtarea trebuie să aibă între una și patru litere.
- Cele n prescurtări trebuie să fie distincte.

Fiind la capitolul de fluxuri și cuplaje, sînt sigur că v-a venit tuturor ideea să creăm un graf bipartit: în stînga cele n șiruri inițiale, iar în dreapta cele $26^4 + 26^3 + 26^2 + 26^1 \approx 500\ 000$ de prescurtări posibile. Pe acest graf căutăm un cuplaj de mărime n.

Dacă graful are 500 000 de noduri, cum se face că algoritmii lui Kuhn sau Hopcroft-Karp sînt suficienți? Răspunsul este că acești algoritmi depind asimptotic de **numărul de noduri din stînga**, nu de cel total. Numărul de noduri din dreapta influențează totuși memoria, căci avem nevoie să reținem pentru fiecare nod din dreapta dacă este cuplat și cu cine.

Pentru simplificarea reprezentării, eu am codificat prescurtările în baza 27, unde 0 înseamnă „cifră nefolosită", iar 1-26 înseamnă `'a'...'z'`. Pentru descifrarea soluției,

- luăm indicele nodului din dreapta cuplat cu fiecare nod din stînga;
- extragem naiv cifrele acelui nod, de la cea mai puțin semnificativă;
- răsturnăm șirul obținut pentru a-l afișa în ordinea corectă.

35.7.6 Problema Access Levels (Codeforces)

enunț • sursă

Problema este dură, dar indiciul că se reduce la flux sau cuplaj ne ajută mult.

Să începem cu un exemplu simplu (exemplele din enunț ne pun deja pe direcția bună). Fie documentele A, B, C, D și mulțimile de utilizatori care au acces la ele U_A, U_B, U_C, U_D. Dacă $U_D \subset U_C \subset U_B \subset U_A$, atunci putem pune toate documentele într-un singur grup, astfel:

- Inițial toți utilizatorii au nivel 1. Nivelele documentelor din grup vor începe de la 2, astfel că implicit niciun utilizator nu poate vedea niciun document din grup.
- Acordăm nivelul 2 documentului A și utilizatorilor din U_A.
- Acordăm nivelul 3 documentului B și utilizatorilor din U_B.
- Acordăm nivelul 4 documentului C și utilizatorilor din U_C.
- Acordăm nivelul 5 documentului D și utilizatorilor din U_D.

Documentele ale căror mulțimi de utilizatori nu se includ una într-alta nu pot face parte din același grup. Să presupunem prin absurd că punem două astfel de documente, A și B, în același grup G și să presupunem fără a restrînge generalitatea că $l[A] \geq l[B]$. Atunci toți utilizatorii din U_A trebuie să aibă un nivel de acces în G de cel puțin $l[A]$ pentru a putea vedea documentul A.

Prin ipoteză, $\exists x \in U_A - U_B$, iar acel x va avea un nivel $l[x] \geq l[A] \geq l[B]$, suficient ca să vadă documentul B, deși n-ar trebui.

Rezultă că putem modela problema într-un graf de documente în care există o muchie de la A la B dacă și numai dacă $U_A \subseteq U_B$. Dacă $U_A = U_B$, atunci tragem muchia de la documentul cu indice mai mic spre documentul cu indice mai mare. Astfel ne asigurăm că graful este aciclic.

În acest graf, orice cale reprezintă un grup de documente care pot sta în același grup. Căile reprezintă grupuri distincte, așadar orice două căi alegem trebuie să fie disjuncte pe vîrfuri. Căutăm să alegem un set de căi de cardinal minim care să acopere toate vîrfurile. Cum facem asta? Nu pare să fie nici o problemă de fluxuri, nici una de cuplaje.

Reducerea este cunoscută. Avem șanse să o reinventăm independent dacă ne amintim de principiul cu care ne-am întîlnit la temă: dublarea vîrfurilor. Creăm un graf bipartit cu documentele dublate pe două coloane L și R, iar dacă documentul i este inclus în documentul j, atunci trasăm muchie de la L_i la R_j. În acest graf bipartit, o incluziune de documente $U_D \subset U_C \subset U_B \subset U_A$ se traduce în cuplajul $(L_A, R_B), (L_B, R_C), (L_C, R_D)$. Căile pot avea lungime 0 dacă un document este singurul membru al unui grup (nu include și nu este inclus în alte documente).

Pentru a găsi un număr cît mai mic de căi, să observăm că fiecare cale are un sfîrșit, adică un document care nu mai este inclus în alte documente, adică un nod L_i care nu este cuplat cu niciun nod din R. Deci pentru a minimiza numărul de căi trebuie să minimizăm numărul de sfîrșituri de cale, deci să minimizăm numărul de noduri necuplate... adică să găsim un cuplaj maxim.

Restul sînt detalii de gestiune a documentelor și a utilizatorilor în implementare. Sursa mea este inspirată din tutorial, dar nădăjduiesc că este ceva mai clară.

35.7.7 Problema Pswap (Baraj ONI 2021)

enunț • sursă

Această problemă are două componente, ambele interesante: (1) construcția grafului și (2) rezolvarea problemei de grafuri. Graful indus este cel în care fiecărei permutări îi corespunde un nod, iar între oricare două permutări similare există o muchie.

Construcția grafului

Ca să trasăm muchiile grafului, dorim ca pentru fiecare pereche de permutări să verificăm dacă ele sînt la o transpoziție distanță. O condiție necesară și suficientă este ca permutările să difere pe exact două poziții. De aceea, toate abordările pe care le cunosc funcționează astfel:

1. Află poziția primei diferențe, a.
2. Află poziția ultimei diferențe, b.
3. Verifică dacă permutările coincid pe porțiunea $[a+1, b-1]$.

Pentru pasul (3) nu cunosc altă soluție decît cu hashing. Am ales metoda Rabin-Karp cu baza $B = 5\,113$ și modulul 2^{32}. Așadar, reprezentăm fiecare prefix al fiecărei permutări ca pe un număr

în baza B, iar prin diferențe și înmulțiri putem calcula codul hash al oricărei subsecvențe.

Pentru pașii (1) și (2) am încercat trei abordări. Le enumăr pe toate, ca fapt divers. Fie perechea curentă de permutări $p[i]$ și $p[j]$.

1. Cea mai lentă: Caută binar prima și ultima diferență. Verifică dacă $p[i]$ și $p[j]$ coincid pe porțiunea dintre aceste poziții.
2. Mai rapidă: Caută binar doar prima diferență. Dacă $p[i][k] = x$ și $p[j][k] = y$, atunci fie k' poziția lui y în $p[i]$ (avem nevoie să stocăm și permutările inverse ca să îl aflăm ușor pe k'). Verifică dacă $p[i]$ și $p[j]$ coincid pe intervalele $[k+1, k'-1]$ și $[k'+1, m]$.
3. Cea mai rapidă: Identifică prima diferență cu sortare ternară, apoi procedează ca mai sus.

Sortarea ternară este un algoritm util pentru sortarea șirurilor de caractere, dar el se aplică și aici. Vom citi codul, dar esența este:

Algoritmul 8 Sortarea ternară a unei colecții de permutări.

```
1:  procedure SortareTernară(l, r, i)   ▷ permutările p[l...r) coincid până la poziția i − 1
2:      alege un pivot z dintre p[k][i], l ≤ k < r
3:      partiționează permutările p[l...r) în trei grupe:
4:          p[l...a) în care p[k][i] < z
5:          p[a...b) în care p[k][i] = z
6:          p[b...r) în care p[k][i] > z
7:      pentru fiecare p[x] și p[y] din grupe diferite execută
8:          diff[x][y] ← i                          ▷ poziția primei diferențe
9:      SortareTernară(l, a, i)
10:     SortareTernară(a, b, i + 1)
11:     SortareTernară(b, r, i)
```

Căutarea binară a primei / ultimei diferențe este operația critică, de complexitate $\mathcal{O}(n^2 \log m)$. De aceea înlocuirea ei cu sortarea ternară este considerabil mai rapidă.

Calculul parității permutărilor

O mică subproblemă este să împărțim nodurile grafului în două mulțimi conform cu paritatea (semnul) permutărilor (vom vedea în secțiunea următoare de ce). Există două abordări.

Varianta simplă este să evaluăm fiecare pereche de permutări ca mai sus. Cînd trasăm o muchie, știm automat că cele două permutări au parități diferite, deoarece **transpozițiile schimbă semnul permutărilor**. La final, facem un DFS pe graful indus în care colorăm nodurile cu două culori.

Varianta un pic mai lungă, dar mai eficientă, este să precalculăm paritățile, după care nu mai este nevoie să testăm similaritatea decît între perechi de permutări cu semne opuse. Semnul unei permutări este definit ca -1^{inv}, unde inv este numărul de inversiuni. Știm cum să calculăm acest număr în $\mathcal{O}(n \log n)$ cu mergesort sau cu un AIB. Dar putem folosi și o definiție echivalentă a semnului: -1^{transp}, unde $transp$ este numărul de transpoziții. Iar pe acesta îl putem calcula în

$\mathcal{O}(n)$. Iterăm prin ciclurile permutării; fiecare ciclu de lungime pară presupune un număr impar de transpoziții și invers.

De exemplu, permutarea (3 6 4 5 7 8 1 9 2) este formată din ciclurile (1 3 4 5 7) și (2 6 8 9). Primul ciclu necesită 4 transpoziții, iar al doilea 3 transpoziții. Așadar, permutarea este impară.

Algoritmul pe graful indus

Odată construit graful, problema cere să găsim setul independent maxim. Complementar, dacă am trasa muchii între permutările nesimilare, atunci problema ar cere să găsim clica maximă. Ambele probleme sînt NP-hard pe grafuri generale.

Aici este momentul în care intuiția și o doză de matematică ne sînt de ajutor. Dacă orice muchie există doar între o permutare pară și una impară, înseamnă că graful este bipartit! (Eu am ajuns la această concluzie pe o cale ocolită, observînd pe multe exemple că toate ciclurile au lungime pară).

Pe un graf bipartit, gîndul ne zboară automat spre cuplaje. *Dacă măcăne ca o rață...* Dar care este legătura între setul independent maxim și cuplajul bipartit? Mai avem nevoie să punem o cărămidă (mică) peste Teorema lui Kőnig.

În orice graf, **orice set independent are drept complement o acoperire cu vîrfuri**. Într-adevăr, prin definiție un set independent S are proprietatea că, la capetele oricărei muchii, se află cel mult un nod din set. Atunci complementul $V \setminus S$ are proprietatea că, la capetele oricărei muchii, se află cel puțin un nod din set. Aceasta este exact definiția acoperirii.

De aceea, răspunsul la problemă este n minus mărimea acoperirii minime a grafului, despre care Teorema lui Kőnig ne spune că este egală cu mărimea cuplajului maxim.

Algoritmul lui Kuhn este suficient pentru punctaj maxim. De fapt, timpul soluției este dominat de citirea datelor (chiar și rapidă!) și de construirea grafului.

Demonstrație directă: Pentru completitudine, iată și o demonstrație care nu apelează la Teorema lui Kőnig.

Pasul 1. Fie K un cuplaj maxim. Setul independent maxim, S, poate include cel mult un nod de pe fiecare muchie din cuplaj. Deci el este obligat să omită cel puțin K noduri. Așadar, $S \leq N-K$.

Pasul 2. Să presupunem că $S < N - K$. Atunci S renunță la mai mult decît un nod de pe fiecare muchie din cuplaj. Există două cazuri și ambele duc la concluzia că, de fapt, cuplajul K poate fi crescut.

- Există un nod u dintre cele $N - 2K$ necuplate care nu este în S. Ne întrebăm: de ce nu? În mod obligatoriu pentru că are $k \geq 1$ vecini $v_1, v_2, \ldots, v_k$ care sînt în S. Dacă oricare v_i ar fi și el necuplat, am putea extinde cuplajul cu muchia (u, v_i). Deci fiecare v_i este cuplat cu cîte un nod $w_i \notin S$. Ce ne împiedică să le eliminăm pe v_i din S și să le adăugăm pe u și pe w_i, ceea ce ar crește setul cu 1? Trebuie să fie adevărat că setul w_i are alți vecini, cel puțin unul, $x_1, x_2, \ldots, x_l \in S$. Repetăm procesul pînă cînd ajungem înapoi în mulțimea de

seturi necuplate. Rezultă o cale de lungime impară, un drum de creștere care augmentează cuplajul cu o muchie.

- Există o muchie dintre cele K cuplate care nu are niciun capăt în S. Demonstrația este similară.

Din (1) și (2) rezultă că $S = N - K$.

Observație

Consider că Pswap nu avea ce căuta la acest nivel de pregătire și cred că este bine că fluxurile nu mai fac parte din programa curentă decît la nivel de lot. Implementarea este „un simplu" cuplaj pe grafuri bipartite, dar ajungerea la această concluzie necesită noțiuni teoretice grele: fie teorema Min-Cut Max-Flow și teorema lui Kőnig, fie o înțelegere foarte solidă a fluxurilor.

Cred că aceste probleme, în care algoritmii trebuie tociți fără demonstrație, duc la elevi rețetari, care pictează o implementare fără să înțeleagă de ce ea rezolvă problema. Această abordare este greșită. Olimpiadele vin și trec. Ce urmează după aceasta? Pe lume mai sînt și rovere marțiene și acceleratoare de particule și brațe robotice chirurgicale... nu putem să intuim software pentru toate.

Mă bucur că în anii recenți s-au rărit problemele care merg la adîncimi exagerate pe această direcție oarecum arbitrară a fluxurilor și cuplajelor în grafuri. Dacă am permite aceasta, de ce nu și programare liniară? Algoritmi de aproximare? Criptografie? Ecuații diofantice? Geometrie computațională 3-D?

Capitolul 36

Fluxuri de cost minim

36.1 Descrierea problemei

În acest capitol, vom extinde noțiunea de rețea de flux învățată anterior.

Definiția 23 (Rețea de flux cu costuri)**.** O rețea de flux cu costuri este un graf orientat $G = (V, E)$ în care fiecare muchie (u, v) are o capacitate $c(u, v) \geq 0$ și un cost unitar $p(u, v)$. Există două noduri speciale în această rețea: o sursă s și o destinație t.

Definiția 24 (Costul unui flux)**.** Dată fiind o rețea de flux cu costuri și un flux $f : E \to \mathbb{R}$ în această rețea, costul fluxului este $\sum_{(u,v)\in E} f(u, v) \cdot p(u, v)$.

Subliniem că costul pe fiecare muchie este per unitate de flux transportată pe muchia respectivă. Vom presupune că graful nu conține cicluri pe care suma costurilor este negativă. Existența acestor cicluri nu exclude neapărat o soluție, dar ne complică mult raționamentele.

Problema fluxului maxim este: dată fiind o rețea de flux cu costuri, dintre toate fluxurile maxime în această rețea să se găsească fluxul de cost minim.

În practică, algoritmii pe care îi vom studia rezolvă o generalizare a problemei: dată fiind o valoare dorită k a fluxului, vom găsi cel mai ieftin flux de valoare k. Cu alte cuvinte, vom găsi cel mai ieftin mod de a pompa k unități de flux de la s la t.

36.2 Rețeaua reziduală

Pentru fluxuri fără costuri, știm că pentru fiecare muchie $(u, v) \in E$ de capacitate c rețeaua reziduală G_f conține și o a doua muchie, (v, u), inițial de capacitate 0. Dacă ulterior pompăm x unități de flux pe muchia (u, v), capacitatea muchiei scade cu x, iar capacitatea muchiei (v, u) crește cu x.

La rețelele cu costuri procedăm similar. Pentru muchia $(u, v) \in E$ de capacitate c și cost p creăm și muchia reziduală (v, u) de capacitate (inițial) 0 și cost $-p$. Semnificația acestei reprezentări

este că, dacă vreodată pompăm flux în sensul (u, v) și ulterior ne răzgîndim și pompăm flux pe muchia reziduală (v, u), ne vom recupera costul plătit pentru acel flux.

36.3 Mecanismul general

Vom descrie doi algoritmi care funcționează pe același principiu general. Vom pompa mai întîi o unitate de flux pe drumul de la s la t care minimizează costul total plătit pentru acea unitate (pe toate muchiile prin care ea trece). Apoi vom pompa încă o unitate, apoi încă una etc. Pe măsură ce începem să saturăm muchii, vom fi nevoiți să căutăm căi $s - t$ de costuri mai mari. Cînd nu mai poate pompa flux (sau după ce a pompat cele k unități pe care ni le-am propus), algoritmul se termină.

Mai întîi să vedem cum găsim cel mai ieftin mod de a pompa o unitate de flux. Care este costul total al propagării acestei unități? Este suma minimă a costurilor pe oricare dintre căile de la s la t. Cu alte cuvinte, este **drumul minim** dacă privim rețeaua reziduală G_f ca pe un graf cu greutăți pe muchii egale cu costurile $p(u, v)$ și dacă luăm în calcul doar muchiile de capacitate pozitivă. De aceea algoritmii de flux maxim de cost minim se reduc, în esență, la căutarea repetată a unor drumuri de creștere, dar nu cu DFS sau cu BFS ca în grafurile fără costuri, ci cu algoritmi specifici drumurilor de cost minim (Dijkstra, Bellman-Ford etc.).

În restul acestui capitol, oricînd vom discuta despre drumuri minime ne referim la suma costurilor muchiilor de pe acele drumuri. Similar, cînd vom discuta despre cicluri negative ne referim la cicluri pe care suma costurilor este negativă.

Desigur, în practică, odată ce găsim o cale minimă P, nu este obligatoriu să pompăm o singură unitate pe ea. Ca și la algoritmii de flux fără costuri, putem pompa atîtea unități încît să saturăm cel puțin o muchie (fără a depăși numărul de unități rămase de pompat).

Demonstrație de corectitudine

Este intuitiv că această abordare *greedy* este corectă din punct de vedere teoretic. Iată o schiță a demonstrației.

Observație. Dacă graful inițial G nu conține cicluri negative, atunci la niciun moment pe parcursul algoritmului rețeaua reziduală G_f nu conține cicluri negative.

Demonstrație. Să presupunem prin absurd că există o primă iterație în urma căreia apare un ciclu de cost negativ. În această iterație, cu un algoritm oarecare am găsit o cale minimă P pe care am pompat flux, ceea ce a creat niște muchii reziduale. Cum ar putea apărea un ciclu negativ? Algoritmul trebuie să fi pompat flux pe o muchie (u, v) de cost pozitiv $p > 0$, ceea ce a creat muchia (v, u) de cost negativ $-p$, care a închis un ciclu negativ C. Acum să considerăm două căi de la s la t.

- Calea P, găsită de algoritm, cu traseul $s \rightsquigarrow u \to v \rightsquigarrow t$.
- Calea P' cu traseul $s \rightsquigarrow u \overset{C}{\rightsquigarrow} v \rightsquigarrow t$.

Cu alte cuvinte, în loc să meargă de la u direct la v, ca în P, P' ocolește folosind restul ciclului C. Pe acest segment, în loc să plătească costul $p > 0$, P' plătește costul $|C| - p < 0$. Așadar $|P'| < |P|$, ceea ce contrazice presupunerea că P este o cale minimă. □

Teorema 5 (Corectitudinea algoritmului *greedy*)**.** Dacă rețeaua de flux G nu conține cicluri negative, atunci algoritmul *greedy* găsește un flux maxim de cost minim.

Demonstrație. Demonstrația este inductivă. La început, fluxul vid are costul minim, 0. Acum, să presupunem că algoritmul *greedy* obține costul minim pentru primele $1, 2, \ldots k-1$ unități de flux, dar obține un cost neoptim pentru cea de-a k-a unitate pompată. Atunci există undeva două căi: calea P de cost minim pe care a ales-o algoritmul și o cale P' (cu $|P'| > |P|$) care ar obține un flux de cost mai mic. Dacă combinăm P cu muchiile reziduale create de P' obținem un ciclu care trece prin s și prin t și care are cost negativ. Acest lucru este imposibil conform observației anterioare. □

36.4 Algoritmii Bellman-Ford sau SPFA

O primă soluție este așadar: în mod repetat, caută o cale minimă (pe costuri) de la s la t folosind algoritmii Bellman-Ford sau SPFA.

Complexitatea acestui algoritm este $\mathcal{O}(Fmn)$, unde F este valoarea fluxului găsit, deoarece algoritmii Bellman-Ford și SPFA au complexitatea $\mathcal{O}(mn)$ și îi apelăm de $\mathcal{O}(F)$ ori. Problema Parcel Delivery include o implementare de referință.

36.5 Algoritmul lui Johnson

Prin definiție, rețelele de flux cu costuri conțin și muchii de cost negativ. Totuși, ne-am dori ca în locul algoritmilor în $\mathcal{O}(mn)$ să putem folosi algoritmul lui Dijkstra, pe care noi îl implementăm în $\mathcal{O}(m \log n)$. Putem face asta dacă folosim o adaptare a algoritmului lui Johnson, care modifică graful încît el să nu mai conțină muchii de cost negativ.

Vom citi mai întîi algoritmul lui Johnson și vom introduce noțiunea de **potențiale pe noduri**. Vom urmări figurile de pe Wikipedia și vom face următoarele observații teoretice (care apar și pe Wikipedia, dar într-o manieră care mie mi se pare încîlcită).

Observație. În graful echilibrat toate muchiile sînt **ne**negative.

Demonstrație. Fie o muchie (u, v) de greutate $w(u, v)$ (cu notațiile de pe Wikipedia) unde nodurile u și v au primit potențialele $h(u)$ și $h(v)$. De aici rezultă imediat că $h(u) + w(u, v) \geq h(v)$, căci altfel am putea optimiza $h(v)$, deci algoritmul de aflare a căilor minime ar fi greșit. Aceasta înseamnă că valoarea echilibrată este $w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$. □

Observație. Greutatea echilibrată a oricărei căi P de la s la t este $\sum_{e \in P} w(e) + h(s) - h(t)$.

Demonstraţie. Suma greutăților după echilibrare este telescopică. Dacă $P = (s, u_1, u_2, \ldots, u_k, t)$ atunci suma este:

$$\begin{aligned} w'(P) &= w(s, u_1) + h(s) - h(u_1) \\ &\quad + w(u_1, u_2) + h(u_1) - h(u_2) \\ &\quad + \ldots \\ &\quad + w(u_k, t) + h(u_k) - h(t) \\ &= \sum_{e \in P} w(e) + h(s) - h(t) \end{aligned}$$

□

Observație. Greutatea echilibrată a muchiilor de pe arborele căilor minime din graful original este 0.

Demonstraţie. Dacă o muchie (u, v) este pe o cale minimă de la s la un nod oarecare x în graful original, atunci ea respectă $h(u) + w(u, v) = h(v)$, adică $w(u, v) + h(u) - h(v) = 0$. □

Observație. Căile minime de la s la t în graful original sînt căi minime și în graful echilibrat.

Demonstraţie. Echilibrarea adaugă aceeași cantitate, $h(s) - h(t) = -h(t)$, la toate căile $s - t$. O cale care ocolea și era neoptimă în graful original va fi neoptimă și în graful echilibrat, iar o cale minimă în graful original va fi minimă și în graful echilibrat (și va avea costul echilibrat 0). □

Observație. Prin definiție, $h(t)$ este distanța minimă de la s la t în graful original. În practică, vom folosi această observație ca să accesăm mai rapid costul unității de flux pompate, fără să o mai calculăm prin însumarea pe muchii.

36.6 Adaptarea algoritmului lui Johnson pentru FMCM

Algoritmul lui Johnson este gîndit pentru aflarea drumului minim între oricare două noduri. Pe grafuri rare, el este asimptotic mai bun decît algoritmul Floyd-Warshall ($\mathcal{O}(n^3)$). Algoritmul lui Johnson necesită $\mathcal{O}(mn)$ pentru calculul potențialelor cu Bellman-Ford, apoi n apeluri de Dijkstra, așadar:

$$\mathcal{O}(mn) + n\mathcal{O}(m + n \log n) = \mathcal{O}(mn + n^2 \log n)$$

Chiar și în implementarea noastră ineficientă de Dijkstra, algoritmul încă este mai eficient decît Floyd-Warshall pe grafuri foarte rare, unde $m = \mathcal{O}(n)$:

$$\mathcal{O}(mn) + n\mathcal{O}(m \log n) = \mathcal{O}(mn \log n)$$

Să observăm totuși că intenția originală a algoritmului lui Johnson era să îl urmăm cu n apeluri la algoritmul lui Dijkstra, **pe un graf static**. Noi dorim altceva: să facem apeluri repetate la Dijkstra din nodul s, pe un graf care se modifică.

La primul apel, Dijkstra primește într-adevăr un graf în care potențialele h sînt corecte, iar costurile echilibrate sînt nenegative. Dar cînd pompăm flux pe calea găsită, vom elimina din graf muchiile saturate și, posibil, vom introduce muchii reziduale. Aceasta face ca potențialele calculate anterior să devină incorecte! Ce-i de făcut? Nu putem rula din nou algoritmul lui Bellman-Ford, căci tocmai asta încercăm să evităm.

Acest articol pe Codeforces explică (spre final) soluția. Putem să folosim distanțele calculate de Dijkstra ca potențiale pentru cea de-a doua iterație. Folosim aceste potențiale ca să echilibrăm (din nou) costurile pe muchii, în plus față de echilibrarea deja făcută. La prima vedere, apare următoarea problemă. Noi calculăm distanțele cu Dijkstra, dar ulterior pompăm flux pe calea minimă, ceea ce modifică graful. Mai putem folosi aceste distanțe? Articolul explică foarte frumos că da. Trebuie să analizăm cele două tipuri de modificări structurale pe care pomparea le aduce în graf.

1. Muchiile saturate dispar din graf. Acestea nu au cum să contrazică funcția de potențial, căci ce este de fapt această funcție? Este o restricție, o condiție impusă fiecărei muchii. O muchie dispărută înseamnă o restricție mai puțin.
2. Pot apărea muchii reziduale în graf. Dar aceste muchii sînt pe o cale minimă $s - t$ și deci respectă $p(u, v) + h(u) - h(v) = 0 \Leftrightarrow -p(u, v) + h(v) - h(u) = 0$, deci și muchia (v, u) nou creată va respecta potențialele.

36.7 Probleme

36.7.1 Problema Parcel Delivery (CSES)

enunț • sursă

Problema este educațională și necesită aflarea fluxului maxim de cost minim. Implementarea este clasică, cu algoritmul SPFA cunoscut.

36.7.2 Problema Task Assignment (CSES)

enunț • sursă

Și această problemă este educațională și necesită aflarea fluxului maxim de cost minim, dar într-un graf bipartit. Am construit explicit rețeaua de flux, dar problema poate fi rezolvată și cu un algoritm de cuplaj (algoritmul ungar).

Detalii de implementare:

- Nu am construit listele de adiacență. În loc de aceasta, funcția `relax_neighbors(int u)` relaxează vecinii potriviți în funcție de tipul nodului u (sursă, persoană, treabă, destinație).
- Am stocat matricea de adiacență. Risipa nu este mare, dat fiind că există $\mathcal{O}(n^2)$ muchii.

36.7.3 Problema Match (IIOT 2019-2020 runda 3)

enunț • sursă

Modelarea ca problemă de flux

De-acum sîntem familiari cu acest gen de probleme, deci sper că v-a venit următoarea idee de rețea de flux:

- Un nod sursă, s.
- O coloană cu m noduri pentru cele m meciuri.
- O coloană cu n noduri pentru cei n jucători.
- Un nod destinație, t.
- Muchii de la s la fiecare meci, de capacitate 1. Acestea garantează că fiecare meci va avea exact un cîștigător: cel înspre care se duce fluxul.
- Muchii de la fiecare meci la cei doi jucători participanți, de capacitate 1.
- Muchii de la jucători la t. Vom detalia mai jos parametrii, dar important este că valorile fluxului pe aceste muchii indică scorurile jucătorilor.

Să notăm scorul maxim minim cu S_M și scorul minim maxim cu S_m.

Faza I: Aflarea lui S_M

Să îl aflăm întîi pe S_M. Pentru un candidat x considerat, vom pune capacitatea x pe muchiile jucător-destinație, pentru a nu-i permite niciunui jucător să acumuleze mai mult de x puncte. Dacă problema admite soluție, fluxul maxim va avea valoarea m, deci $S_M \leq x$. Altfel înseamnă că am fost prea optimiști și nu putem repartiza toate meciurile, deci $S_M > x$.

Pentru a-l găsi pe S_M avem diverse abordări, cum ar fi căutarea binară. S_M poate avea valoarea $m/2$, de exemplu dacă toate cele m meciuri sînt între jucătorii 1 și 2. Chiar și cu un algoritm eficient de flux, să spunem Dinic ($\mathcal{O}(mn^2)$), căutarea binară necesită $\mathcal{O}(mn^2 \log n)$.

Iată și o abordare incrementală, care îl crește cu S_M cu cîte o unitate, dar procedează eficient, similar problemei Echipe. Atribuim întîi capacitatea 0 muchiilor jucător-destinație. Apoi creștem treptat aceste capacități, cu cîte o unitate (simultan pe toate muchiile) și actualizăm fluxul, pornind de la cel deja existent. Folosim algoritmul (relativ) naiv Ford-Fulkerson. Acest algoritm face $\mathcal{O}(m)$ repetiții care constau din:

- Creșterea capacităților pe n muchii: $\mathcal{O}(n)$.
- Cel mult un DFS eșuat, care constată că nu mai poate pompa flux și încheie repetiția: $\mathcal{O}(m+n)$.

- m DFS-uri cu succes **în total în toate repetițiile**, care cresc fluxul cu cîte o unitate.

Complexitatea totală a fazei I este $\mathcal{O}(m(m+n))$.

Faza II: Aflarea lui S_m

Precizez că există și o soluție fără costuri pe muchii, bazată pe circulații. Odată ce îl aflăm pe S_M, ne întrebăm succesiv: există vreo soluție în care **fluxul minim** pe oricare dintre muchiile jucător-destinație să fie S_M? Dacă nu, atunci poate $S_M - 1$, $S_M - 2$ etc.? Problemele de flux cu limite inferioare ale capacităților se reduc la probleme de flux „normale", doar cu capacități superioare. Dar reducerea nu este ușoară și depășește scopul prezentului capitol.

Să revenim la subiect. Cum modelăm aflarea lui S_m ca pe o problemă de flux? Pentru un candidat x, dacă $S_m \geq x$, ar trebui ca algoritmul să poată pompa cel puțin x unități de flux pe fiecare muchie jucător-destinație. De aceea, să îl stimulăm să facă acest lucru cu costuri! Știm deja că toate muchiile jucător-destinație trebuie să aibă capacitatea S_M. Vom sparge aceste muchii în cîte două:

- Una de capacitate x și cost 0.
- Una de capacitate $S_M - x$ și cost 1.

Care este rostul acestei dublări? Algoritmul va fi motivat să trimită cît mai mult flux pe muchiile de cost 0. Cel mai ieftin, dacă se poate, este să le satureze pe toate, ceea ce simbolizează că fiecare jucător a cîștigat cel puțin x meciuri. Costul acestui flux este $m - nx$, deoarece restul unităților dincolo de primele nx se vor duce pe muchii de cost 1.[1]

Din nou, putem să îl căutăm binar pe S_m, dar iată un algoritm incremental mai eficient, mai simplu de programat (doar DFS-uri, fără Dijkstra sau Bellman-Ford) și, îndrăznesc să susțin, mai elegant. Dacă răspunsul pentru x a fost afirmativ, atunci dorim:

1. Să pregătim rețeaua pentru $x+1$.
2. Să recalculăm costul minim al fluxului (valoarea fluxului este întotdeauna egală cu m).
3. Să comparăm noul cost cu $m - n(x+1)$.

Actualizarea rețelei. Vom actualiza rețeaua pentru $x+1$, dar **nu de la zero**. Ce se schimbă cînd trecem de la x la $x+1$? Capacitățile muchiilor jucător-destinație de cost 0 cresc cu 1, iar ale celor de cost 1 scad cu 1. Putem face acest lucru în $\mathcal{O}(n)$ cu un simplu `for`. Mai mult, dacă muchia de cost 1 are flux pe ea, putem muta o unitate din acest flux pe muchia de cost 0, ceea ce reduce costul fluxului cu 1. Dacă muchia de cost 1 nu are flux pe ea, atunci trebuie să căutăm alt mod de a profita de creșterea capacității muchiei de cost 0.

Recalcularea costului. Prin actualizarea rețelei am păstrat valoarea fluxului, adică m; nu mai putem căuta drumuri de creștere așa cum ne-am obișnuit la algoritmii de flux. Întrebarea este dacă, prin scăderea costului unor muchii de la 1 la 0, putem obține un flux de cost mai mic. Mai

[1] La fel de bine putem și să folosim costurile -1 și respectiv 0. Este alegerea voastră dacă judecați în termeni de „pedeapsă (cost pozitiv) pentru prea mult flux pentru această muchie" sau „premiu (cost negativ) pentru primele x unități de flux pe această muchie".

ales, dorim să obținem fluxul de cost minim incremental, din cel existent. Să ne gîndim de unde ar putea proveni reducerea costului. Toate muchiile au costul 0, cu excepția celor de cost 1 dintre jucători și destinație. Deci trebuie să redirecționăm o unitate de flux de cost 1 și să căutăm un alt drum spre t printr-o muchie de cost 0.

Din perspectiva rețelei reziduale, noi căutăm **un ciclu**! El trebuie să pornească din t pe o muchie de cost -1 (așadar perechea reziduală a unei unități de flux de cost 1) și să revină în t folosind doar muchii de cost 0. Adăugat la fluxul curent, acest ciclu menține fluxul total, dar reduce costul cu 1.

Ultima întrebare este cum găsim aceste cicluri de cost -1. Răspunsul este simplu: un DFS din t este suficient, care pornește pe fiecare muchie de cost -1 și are voie să continue doar pe muchii de cost 0 (și, desigur, cu capacitate reziduală pozitivă). Nu este nevoie de algoritmii lui Dijkstra sau Bellman-Ford.

Să calculăm și complexitatea fazei II. Spre deosebire de faza I, unde S_M putea fi $m/2$, să observăm că $S_m \leq \lfloor m/n \rfloor$, conform principiului lui Dirichlet. Așadar, pot exista cel mult m/n repetiții care constau din:

- Modificarea capacităților pe $2n$ muchii: $\mathcal{O}(n)$.
- Cel mult n DFS-uri cu succes care redirecționează cîte o unitate de flux (nu pot fi mai multe, deoarece la pasul anterior am crescut capacitatea muchiilor de cost 0 cu n în total): $\mathcal{O}(n(m+n))$.
- Cel mult cîte un DFS eșuat pornind din t spre fiecare jucător (mai exact, DFS-ul care constată că nu mai poate redirecționa o unitate de flux): $\mathcal{O}(n(m+n))$.

Complexitatea fazei II este $\mathcal{O}(m/n \cdot n(m+n)) = \mathcal{O}(m(m+n))$.

Așadar, complexitatea totală a algoritmului este remarcabil de joasă. Cu limite mai mari (să sicem $m = 5\,000$ și $n = 1\,000$), cred că problema are potențial de lot.

Detaliu de implementare

Este preferabil să creăm de la bun început un singur graf. În faza I nu folosim costurile. Pentru aflarea lui S_M creștem capacitățile muchiilor jucător-destinație de cost 1. Apoi începem să le descreștem, crescînd capacitățile muchiilor de cost 0. Astfel, graful nu trebuie resetat niciodată, ci informația curge natural din faza I în faza II.

36.7.4 Problema Hoață (Baraj ONI 2022)

enunț • surse

Modelarea ca problemă de flux

Flerul de a putea modela o problemă în termeni de rețele de flux vine cu experiența. De regulă, enunțul trebuie să menționeze niște restricții despre cîte evenimente simultane de un anumit tip

pot coexista. Acelea devin capacități pe muchiile rețelei de flux. Așa cum în problema Match un jucător nu putea cîștiga simultan mai mult de x partide, în problema de față din camera i în camera $i+1$ nu pot trece mai mult de x_i hoți cu aceeași greutate.

Dacă problema ar fi una de flux, atunci camerele ar fi noduri, iar alarmele ar fi muchii între camere. Și atunci ce reprezintă cei x_i hoți care trec prin această alarmă? Fiecare dintre acești hoți este o unitate de flux! Iar traseul fiecărui hoț ar trebui să reprezinte ce obiecte fură hoțul din fiecare cameră.

Următorul pas este să modelăm faptul că, desigur, toți hoții ajung pînă la urmă în camera $i+1$, dar x_i este limita impusă numărului de hoți cu aceeași greutate. De fapt, vor exista $G+1$ noduri pentru fiecare cameră. Conceptual, ia naștere o matrice A de $n \times (G+1)$ noduri unde $A_{i,j}$ (cu $1 \leq i \leq n$ și $0 \leq j \leq G$) semnifică starea unui hoț aflat în camera i cu greutatea j în rucsac. Din celula (i,j) nu pot trece în celula $(i+1,j)$ mai mult de x_i hoți.

De aceea, comportamentul unui hoț prin muzeu va corespunde unei căi care pornește din celula $s=(1,0)$ (prima cameră cu rucsacul gol) și face în mod repetat una din două acțiuni posibile pînă cînd iese din muzeu:

1. Avansează în camera următoare, deci trece din (i,j) în $(i+1,j)$.
2. Fură un obiect, deci trece din (i,j) în $(i,j+g_i)$.

În acest model, ce reprezintă valorile obiectelor? Ele sînt profituri, așadar costuri negative. Acțiunea 2 de mai sus vine cu costul $-v_i$.

Să observăm că din ultima cameră (n), hoții trebuie să părăsească muzeul indiferent ce greutate au în rucsac. Așadar, mai adăugăm un nod t și legăm de el toate nodurile de pe ultima linie a matricei, prin muchii de capacitate x_n.

Punînd toate elementele cap la cap, obținem graful:

- Există $n \cdot (G+1)+1$ noduri, dintre care primele $n \cdot (G+1)$ noduri corespund conceptual unei matrice cu n linii și $G+1$ coloane.
- Sursa s primul nod, iar destinația t este ultimul nod.
- Din fiecare nod (i,j) cu $1 \leq i \leq n$ și $0 \leq j \leq G-g_i$ există o muchie la dreapta, către nodul $(i,j+g_i)$ de capacitate k (sau infinit) și cost $-v_i$.
- Din fiecare nod (i,j) cu $1 \leq i < n$ și $0 \leq j \leq G$ există o muchie în jos, către nodul $(i+1,j)$, de capacitate x_i și cost 0.
- Din fiecare nod (n,j) cu $0 \leq j \leq G$ există o muchie către nodul t de capacitate x_n și cost 0.

Pe acest graf căutăm un flux de valoare k și cost minim. Fiecare unitate de flux reprezintă traseul unui hoț. De exemplu, primul hoț va căuta în mod natural să aleagă obiecte care maximizează raportul v_i/g_i (practic, va rezolva problema rucsacului). Următoarele unități de flux pot alege variante mai puțin profitabile și pot reface traseele hoților anteriori prin muchii reziduale.

Soluția cu SPFA

Să observăm că graful poate avea $n_G \approx 90\,000$ de noduri. Din fiecare nod pornesc cel mult două muchii, plus perechile lor reziduale, așadar graful poate avea $m_G \approx 360\,000$ de muchii. Dorim să pompăm $k \leq 50$ de unități de flux prin acest graf. Limitele par prea mari pentru algoritmul SPFA, care rulează în $\mathcal{O}(kn_Gm_G)$. Dar am implementat oricum acest algoritm pentru că este considerabil mai simplu decît Johnson + Dijkstra.

Spre surprinderea mea, programul rula în circa 2,2 secunde, aproape de limita de 2 secunde (nu știu dacă pe evaluatorul din concurs aș fi fost la fel de aproape). Am înlocuit coada STL cu o coadă proprie, iar timpul a scăzut cu 20-21% (de reținut!). Scăderea a fost similară pe toate testele. Astfel am reușit să obțin 100p.

Conform editorialului, totuși, intenția autorilor a fost o implementare cu algoritmul lui Johnson.

Soluția cu algoritmii lui Johnson și Dijkstra

Implementarea este relativ de manual, dar am cîteva observații interesante.

Pentru calculul inițial al potențialelor nu este nevoie să folosim algoritmul Bellman-Ford. Este drept că există muchii de cost negativ, dar graful este aciclic. Este suficient să relaxăm muchiile în ordinea sortării topologice, care coincide foarte convenabil cu ordinea crescătoare a nodurilor. Aceasta face funcția `init_potentials`.

După fiecare recalculare a potențialelor, am ales să echilibrez graful (în funcția `reweight_edges`) modificînd chiar costurile muchiilor. Puteam și să lăsăm costurile neatinse și să calculăm altundeva contribuțiile de la reechilibrare, dar nu este necesar.

Coada de priorități pentru Dijkstra reține perechi (distanță, nod). Din obișnuință, am dorit să evit tipul `std::pair`, așa că mi-am declarat propriul `struct pq_element` care dădea nume clare cîmpurilor. Apoi am declarat o coadă de priorități de acest tip:

```
struct pq_elt {
  int u, d;

  friend bool operator<(pq_elt a, pq_elt b) {
    return (a.d > b.d);
  }
};

std::priority_queue<pq_elt> pq;
```

Un șoc pentru mine, și o lecție foarte interesantă, a fost că acest cod mergea cam cu 33% mai lent decît versiunea în care foloseam o coadă de priorități de tipul `std::pair`! Am depanat fenomenul și am constatat că, pe testul cel mai mare, implementarea mea trecea circa 20 de milioane de elemente prin coadă, pe cînd implementarea cu `std::pair` trecea doar 14 milioane.

Vă dați seama de unde provine problema? Explicația este în codul comparatorului. Dacă există

noduri cu distanță egală (și probabil există multe), comparatorul meu nu are nicio preferință și lasă nodurile într-o ordine arbitrară. Comparatorul din STL oferă o ordine totală și, la egalitate, preferă nodul cu indice minim.

Printr-o coincidență, în problema de față această departajare este vitală. Dacă două noduri $u < v$ au aceeași distanță, iar noi îl procesăm întîi pe v, este foarte posibil ca ulterior, cînd îl procesăm pe u, distanța lui v să se modifice și să fie nevoie să refacem calculele.

Cînd am inclus și această departajare în comparatorul meu, timpii de rulare au devenit identici. În cele din urmă am renunțat complet la `struct`-ul propriu, căci am găsit o formulare lizibilă cu `std::pair`, care nu folosește numele criptice `first` și `second`.

36.7.5 Problema Lifts (IATI Shumen 2022)

enunț • sursă

Modelarea ca problemă de flux

Observăm că fiecare comandă trebuie procesată de un singur lift. Dacă am crea o rețea de flux, ar avea sens să reprezentăm comenzile prin cîte un nod și să legăm aceste noduri de o sursă s și de o destinație t prin muchii de capacitate 1. În acest graf, o unitate de flux ar corespunde traseului unui lift. Acum, să punem la punct detaliile.

Dorim ca un lift să poată procesa mai multe comenzi. Cum facem asta? Este nevoie să putem face tranziții între orice pereche de comenzi (i, j) cu $i < j$. Costul acestei tranziții este $|q_i - p_j|$. Iată graful corespunzător exemplului din enunț.

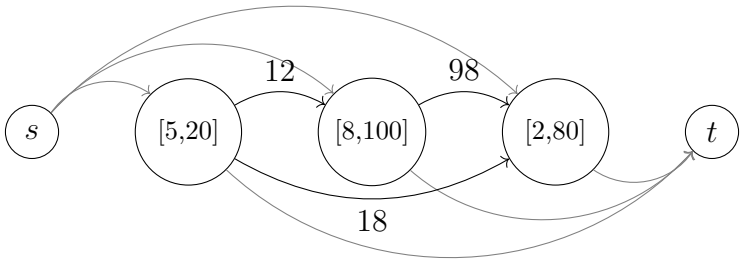


Figura 36.1: O încercare de graf pentru problema Lifts. Toate capacitățile sînt 1 și toate costurile sînt 0, cu excepția celor figurate pe muchii.

Cele două lifturi ar putea folosi traseele $s \to [5, 20] \to [8, 100] \to t$ și respectiv $s \to [2, 80] \to t$, de cost total 12. Dar există o problemă. Un flux și mai ieftin ar fi $s \to [5, 20] \to t$ și $s \to [2, 80] \to t$, de cost 0. Fiecare lift procesează o singură comandă, iar unele comenzi rămîn neprocesate! Cum convingem lifturile să proceseze colectiv toate comenzile? Similar problemei Match, vom înlocui fiecare nod cu o pereche de noduri unite printr-o muchie de cost $-\infty$. Aceasta va face comenzile

atractive, căci va acoperi orice cost de deplasare între două comenzi. La final, trebuie să nu uităm să adăugăm la costul găsit cantitatea $n \cdot \infty$ (desigur în cod vom înlocui ∞ cu o valoare finită).

În această reprezentare, deplasările între două comenzi sînt muchii de la un nod drept i la un nod stîng j cu $i < j$. Rezultă structura:

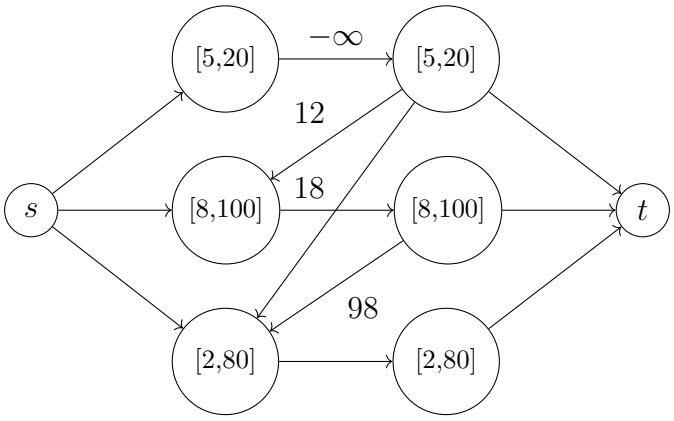


Figura 36.2: O nouă încercare de graf pentru problema Lifts.

Reducerea numărului de muchii la $\mathcal{O}(n\sqrt{n})$

Mai există un obstacol. Numărul de muchii din graf, inclusiv cele reziduale, este $2 \cdot (3n + C_n^2) \approx 100\,000\,000$, mult prea mare pentru a se încadra în timp și în memoria de numai 64 MB. Trebuie să reducem masiv numărul de muchii, dar cum?

O primă idee este cu descompunerea în radical. Să împărțim nodurile în $B \approx \sqrt{n}$ blocuri de cîte B noduri fiecare. Dorim ca un nod din dreapta, i, să conțină $\mathcal{O}(\sqrt{n})$ muchii, astfel:

- $\mathcal{O}(1)$ muchii către fiecare nod stîng j din același bloc cu i (ca și pînă acum).
- $\mathcal{O}(1)$ muchii către fiecare bloc de noduri stîngi aflate mai jos decît blocul lui i.

Pentru aceasta, avem nevoie să inserăm $2n$ noduri în plus. Pentru fiecare bloc de noduri stîngi procedăm astfel. Creăm două grupe de cîte B noduri, pe care le înlănțuim în două liste. Aceste liste le legăm cu muchii de cost 0 într-o corespondență 1:1 cu nodurile stîngi din bloc, dar nu în ordinea naturală, ci în ordinea primului etaj al comenzilor respective. Cu alte cuvinte, dacă comenzile din bloc încep la etajele 30, 10, 50, 20, 60 și 40, vom lega nodurile din prima listă respectiv la etajele 10, 20, 30, 40, 50 și 60, iar nodurile din a doua listă respectiv la etajele 60, 50, 40, 30, 20, 10. Muchia între oricare două noduri consecutive are un cost egal cu diferența de etaje (10 în acest exemplu).

De ce procedăm astfel? Să considerăm o comandă dintr-un nod drept r, aflat undeva mai sus decît blocul curent. Să spunem că ultimul etaj al comenzii este 27. Atunci este suficient să trasăm două muchii de la comandă la cele două liste înlănțuite:

- O muchie leagă comanda de nodul corespunzător etajului 30 din lista crescătoare. Muchia are cost 3.
- Cealaltă muchie leagă comanda de nodul corespunzător etajului 20 din lista decrescătoare. Muchia are cost 7.

Acum vedem că putem suplini lipsa muchiilor dintre r și orice nod stîng din blocul curent. Dacă o unitate de flux ajunge în r și dorește să continue de la nodul de nivel 50, atunci ea:

- va intra în lista crescătoare la nivelul 30 cu costul 3;
- va parcurge două muchii din listă, fiecare de cost 10;
- va părăsi lista pe muchia care duce la nivelul 50.

Astfel vedem că unitatea de flux a combinat o comandă terminată la nivelul 27 cu o comandă ulterioară care începe la nivelul 50 și a plătit costul 23, așa cum ne dorim.

Am introdus această reprezentare ca pe un pas intermediar, dar ea tot este insuficientă. Care este numărul de muchii din această reprezentare? Fiecare nod drept are:

- În medie, $\sqrt{n}/2$ muchii către nodurile stîngi din același bloc.
- În medie, $2 \cdot \sqrt{n}/2$ muchii către cele două liste înlănțuite din blocurile următoare.

Așadar, fiecare nod are în medie 150 de muchii, sau 300 incluzînd și perechile reziduale. Totalul este de circa 3 000 000 de muchii, tot prea mare.

Reducerea numărului de muchii la $\mathcal{O}(n \log n)$

Odată ce am înțeles principiul, putem face același salt pe care îl facem de la descompunerea în radical la arbori de intervale: nu mai folosim blocuri de mărime $\sqrt{n}$, ci blocuri de mărime 1, 2, 4, 8 etc. Acest comentariu rezumă ideea descompunerii.

Să considerăm, pentru ușurință, că n este o putere a lui 2, de exemplu $n = 32$. Să numerotăm comenzile (și nodurile) de la 0 la 31. Atunci, de exemplu, nodul drept 5 trebuie legat la nodurile stîngi $[6, 31]$. Descompunem acest interval în $[6, 7]$, $[8, 15]$ și $[16, 31]$, toate de lungimi puteri ale lui 2 și toate cu capetele stîngi aliniate cu un multiplu al lungimii.

Acum vom crea, pentru anumite intervale de lungime putere a lui 2, liste înlănțuite ca mai sus, care iterează prin nodurile din interval în ordine crescătoare și descrescătoare. Putem obține o implementare relativ concisă cu divide et impera (exemplu). Eu am încercat o implementare iterativă.

Cu această descompunere, numărul total de muchii este circa 780 000, care se încadrează în timp și în memorie. Problema Legacy (Codeforces) necesită o descompunere similară pentru algoritmul lui Dijkstra.

Calculul potențialelor

Algoritmul Bellman-Ford este prea lent pentru calculul inițial al distanțelor și al potențialelor. Mai este necesară o observație ingenioasă. Graful pe care l-am creat este aciclic, deci este suficient să îl sortăm topologic, de exemplu cu BFS, și să relaxăm muchiile în această ordine.

Spre tristețea mea, din această cauză am pierdut circa 30 de minute depanînd următoarea optimizare, care nu merge. Din punct de vedere al corectitudinii, este suficient să creăm cîte o singură listă pentru fiecare bloc. Pentru exemplul de mai sus, cu etajele 30, 10, 50, 20, 60 și 40, putem

crea o singură listă dublu înlănțuită care se leagă, în ordine de etajele 10, 20, 30, 40, 50 și 60. Pentru fiecare nod drept superior, ambii pointeri intră în această listă pe poziții vecine. Un nod drept terminat la etajul 27 va avea pointeri către nodurile 20 și 30.

Am neglijat însă faptul că fiecare pereche de elemente din aceste liste formează un ciclu. 🥲. De aceea, sortarea topologică eșua.

Partea X

Algoritmi pe șiruri de caractere

Capitolul 37

Căutări în șiruri de caractere. Algoritmul Rabin-Karp

Căutarea unui subșir într-un șir înseamnă găsirea tuturor pozițiilor la care subșirul apare în șir. Să introducem cîteva definiții și notații pentru restul acestei părți.

37.1 Terminologie

Definiția 25 (Șir de caractere)**.** Un șir de caractere este un vector cu elemente de tip caracter.

Notație. Vom nota cu Σ alfabetul din care fac parte elementele șirului.

În multe probleme de informatică, $\Sigma = \{a, b, \ldots, z\}$, dar ocazional alfabetul poate consta din 0 și 1, din litere mari etc.

Notație. Vom nota cu $|S|$ lungimea șirului S.

Vom opera pe șiruri C (`char*`), astfel că un șir de n caractere își stochează informația pe pozițiile $0 \ldots n-1$ și conține terminatorul `'\0'` pe poziția n. Codul poate fi ușor adaptat și pentru clasa C++ `string`.

Definiția 26 (Aparițiii, deplasări)**.** Spunem că un subșir P apare la poziția k într-un șir S dacă $|P| + k \leq |S|$ și $P[i] = S[i + k]$ pentru $0 \leq i < |P|$. Valoarea k se numește deplasare validă. Pozițiile la care P nu apare în S se numesc deplasări invalide.

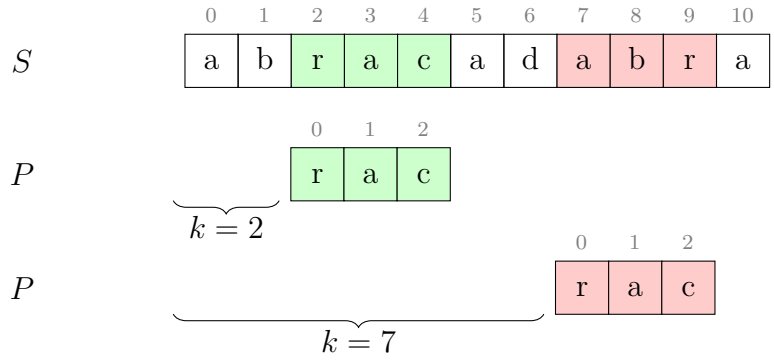


Figura 37.1: Pentru șablonul $P =$ `rac` și șirul $S =$ `abracadabra`, $k = 2$ este o deplasare validă, iar $k = 7$ este o deplasare invalidă.

Cu aceste definiții și notații, putem reformula căutarea subșirului P în șirul S ca găsirea tuturor deplasărilor valide ale lui P în S. Subșirul de căutat se mai numește și **șablon** (engl. *pattern*). Vom nota $|P| = m$ și $|S| = n$.

37.2 Algoritmul naiv

Următorul algoritm evaluează naiv fiecare deplasare și le tipărește pe cele valide. Pentru o deplasare `dep` fixată, algoritmul caută liniar prima nepotrivire.

```
int n = strlen(s);
int m = strlen(p);

for (int dep = 0; dep <= n - m; dep++) {
  int i = 0;
  while ((i < m) && (p[i] == s[i + dep])) {
    i++;
  }
  if (i == m) {
    printf("Deplasare validă: %d\n", dep);
  }
}
```

Algoritmul este ineficient, având complexitate $\mathcal{O}(m(n-m))$. Putem expune această ineficiență cu valori ca $S =$ `aaaaaaaaaa` și $P =$ `aaaaab`. Pentru completitudine, menționăm totuși că există situații în care algoritmul naiv se comportă optim. Iată trei dintre ele.

37.2.1 Toate caracterele din șablon sînt diferite

Să presupunem că pentru o deplasare k găsim o primă nepotrivire între $P[i]$ și $S[i+k]$. În particular, aceasta înseamnă că $P[1 \ldots i-1] = S[1+k \ldots i-1+k]$. Așadar, în toată acea regiune din S apar caractere din P, dar **nu** $P[0]$. Evaluarea acelor deplasări va eșua încă de la prima

comparație. Următoarea deplasare care are o șansă să fie validă va fi $k + i$.

Din acest motiv, complexitatea este $\mathcal{O}(n)$.

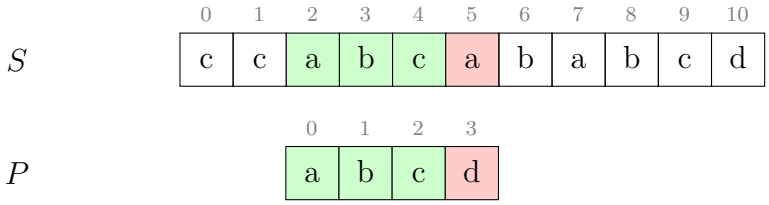


Figura 37.2: Pentru deplasarea $k = 2$, prima nepotrivire apare la poziția $i = 3$. Caracterul a nu apare pe pozițiile 3 și 4. Următoarea deplasare care merită verificată este $k = 5$.

37.2.2 Unul dintre șirurile S și P este generat aleatoriu

În această situație, fiecare comparație între un caracter din S și unul din P va eșua cu probabilitatea

$$\frac{|\Sigma| - 1}{|\Sigma|}$$

, de unde cu puțină teorie a probabilităților aflăm că numărul așteptat de comparații pentru o deplasare dată va fi

$$\frac{|\Sigma|}{|\Sigma| - 1}$$

Această valoare este 2 pentru alfabete binare și tinde la 1 pe măsură ce mărimea alfabetului crește. Și în acest caz, complexitatea este $\mathcal{O}(n)$.

37.2.3 m are valoare apropiată de n

Reamintim că complexitatea nu este $\mathcal{O}(mn)$, ci $\mathcal{O}(m(n - m))$.

37.3 Funcții hash

Dacă ați descărcat vreodată un fișier mare (de exemplu, o imagine de stick bootabil), poate ați observat că alături de el găsim o **sumă de control** (engl. *checksum*), de regulă sub forma unui număr hexazecimal lung calculat cu un algoritm ca SHA256. Care este rostul acestui *checksum*? În esență, este un mod de a ne asigura că am descărcat fișierul fără erori. Rulăm local același algoritm și ne asigurăm că obținem același rezultat ca pe site.

Un algoritm de *checksum* este un tip particular de **funcție hash** (diferențele depășesc scopul acestei culegeri). O funcție hash transformă niște date de o lungime arbitrară în alte date (numite

valori hash) de lungime fixă. Indiferent cît de mare este fișierul descărcat sau lungimea șirului citit, funcția hash returnează valori pe un număr fix de biți, de exemplu 256.

De la o funcție hash bună dorim următoarele proprietăți:

1. Să fie ușor de calculat.
2. Să traducă valori de intrare aproape identice în valori hash substanțial diferite.
3. Să genereze toate valorile hash posibile cu aceeași probabilitate pentru valori de intrare aleatorii; cu alte cuvinte, să distribuie bine valorile de intrare.

Este important să observăm că funcțiile hash **nu sînt injective**. Nici nu ar avea cum, căci spațiul valorilor de intrare este infinit, pe cînd spațiul valorilor hash este finit. Ce implică aceasta? Dacă două valori de intrare X și Y corespund la valori hash diferite, atunci știm sigur că $X \neq Y$. În schimb, dacă valorile hash sînt egale, nu avem nicio garanție că $X = Y$. De exemplu, pentru funcția hash $H(x) = x \bmod 10$, valorile de intrare 27 și 57 corespund aceleiași valori hash, 7. Această situație se numește **coliziune**.

37.4 Căutarea cu funcții hash

Din secțiunea anterioară decurge o metodă de căutare cu funcții hash:

```
int n = strlen(s);
int m = strlen(p);
int hp = hash(p, m);

for (int dep = 0; dep <= n - m; dep++) {
  if ((hash(s + dep, m) == hp) &&
      compar_naiv(p, s + dep, m)) {
    printf("Deplasare validă: %d\n", dep);
  }
}
```

Pentru concizie, am exclus codul funcțiilor `hash` și `compar_naiv`. Observăm că acest algoritm are tot complexitate $\mathcal{O}(m(n-m))$, din două motive:

1. Funcția hash, dacă îi permitem să examineze complet subșirul curent, are complexitate $\mathcal{O}(m)$. Vom arăta în secțiunea următoare cum putem reduce complexitatea la $\mathcal{O}(1)$.
2. Pentru date de intrare ca $S = \texttt{aaaaaaaaaa}$ și $P = \texttt{aaaaaa}$, toate valorile hash vor fi egale și algoritmul va degenera în $n - m$ comparări naive.

37.5 Funcții *rolling hash*

Să considerăm alfabetul $\Sigma = \{0, 1, \ldots, 9\}$. Să considerăm, pentru simplitate, că $m \leq 9$. Atunci putem defini funcția hash a unui subșir de lungime m ca fiind valoarea numerică a acelui șir. De exemplu, $H(\texttt{"2943"}) = 2\,943$.

Acum, fie $P = \texttt{9435}$ și $S = \texttt{1112943511}$. Atunci vom calcula $H(P) = 9\,435$ și $H(S[3...6]) = 2\,943$. Cînd trecem de la deplasarea 3 la deplasarea 4, putem calcula în doar $\mathcal{O}(1)$ noua valoare hash, $H(S[4...7])$. Procedăm astfel:

- Din 2 943 scădem prima cifră (adică 2) înmulțită cu 10^{m-1} (adică 1 000). Diferența este 943. În termeni de șiruri, am eliminat primul caracter din stînga.
- Pe 943 îl înmulțim cu 10. Obținem 9 430.
- Adăugăm următoarea cifră din S (5). Obținem 9 435. În termeni de șiruri, am adăugat la dreapta următorul caracter.

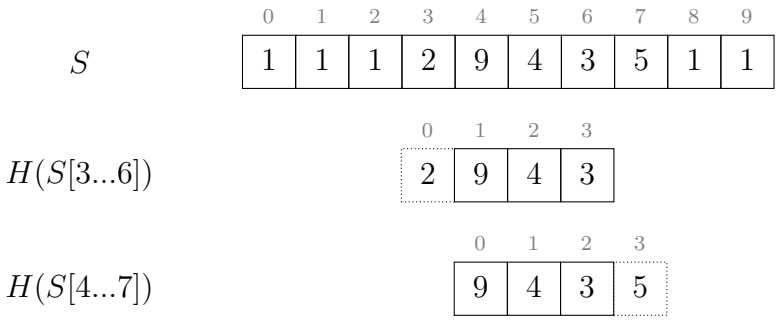


Figura 37.3: Pentru deplasarea $k = 3$, valoarea hash este 2 943.
Pentru $k = 4$, valoarea hash este $(2\,943 - 2\,000) \times 10 + 5 = 9\,435$.

Avantajul este că am redus la $\mathcal{O}(n)$ efortul total pentru a calcula valorile hash ale tuturor subșirurilor lui S de lungime m. Acest tip de funcție se numește *rolling hash* pentru că *se rostogolește* de la o poziție la următoarea.

În cazul general, valorile hash vor avea mărimi de ordinul $|\Sigma|^m$ și vor fi prea mari pentru a fi reprezentate în tipuri de date simple. Soluția este să operăm modulo o valoare q. Astfel, formula generală pentru a trece de la o valoare hash la următoarea este:

$$H(S[k+1...k+m]) = \left\{ \left[H(S[k...k+m-1]) - S[k] \cdot |\Sigma|^{m-1} \right] \cdot |\Sigma| + S[k+m] \right\} \bmod q$$

În concluzie, funcția traduce $|\Sigma|^m$ șiruri posibile în doar q valori hash. Așadar, pot apărea coliziuni. De aceea, ori de cîte ori găsim în S valoarea $H(P)$ este necesar să facem și o comparare naivă. Complexitatea teoretică a algoritmului rămîne $\mathcal{O}(m(n-m))$.

37.6 Algoritmul Rabin-Karp

Punînd cap la cap toate elementele de mai sus, rezultă codul:

```cpp
#include <stdio.h>
#include <string.h>

const int MAX_N = 1'000'000;
const int SIGMA = 26;
const int MOD = 1'000'000'007;

char s[MAX_N + 1], p[MAX_N + 1];
long long q; // sigma^{m-1}

void precompute_q(int exp) {
  q = 1;
  while (exp--) {
    q = q * SIGMA % MOD;
  }
}

long long init_hash(char* s, int len) {
  long long result = 0;
  for (int i = 0; i < len; i++) {
    result = (result * SIGMA + (s[i] - 'a')) % MOD;
  }
  return result;
}

long long avans_hash(long long old, char* s, int len) {
  old += MOD - (q * (s[0] - 'a')) % MOD;
  return (old * SIGMA + (s[len] - 'a')) % MOD;
}

bool compar_naiv(char* a, char* b, int len) {
  int i = 0;
  while ((i < len) && (a[i] == b[i])) {
    i++;
  }

  return (i == len);
}

int main() {
  scanf("%s %s", s, p);
  int n = strlen(s);
  int m = strlen(p);
  precompute_q(m - 1);

  long long hp = init_hash(p, m);
  long long hs = init_hash(s, m);

  for (int dep = 0; dep <= n - m; dep++) {
```

```
    if ((hs == hp) &&
        compar_naiv(p, s + dep, m)) {
      printf("Deplasare validă: %d\n", dep);
    }
    hs = avans_hash(hs, s + dep, m);
  }

  return 0;
}
```

Notăm următoarele considerente practice.

- Valoarea $|\Sigma|^{m-1}$ trebuie precalculată.
- Pentru a evita depășirile, este necesar ca $|\Sigma| \cdot q$ să încapă în tipul de date ales (de exemplu `int` sau `long long`).

Ca încercare de optimizare, putem folosi un modul putere a lui 2 pentru viteză mai mare. Putem chiar să operăm modulo 2^{32} sau modulo 2^{64} și să renunțăm complet la operațiile de împărțire (depășirile vor face automat acest lucru pentru noi). Totuși, este posibil ca datele de test să fie alese adversarial și să genereze multe coliziuni pentru aceste module arhicunoscute. În funcție de natura problemei, poate fi preferabil un modul prim ales la întîmplare.

Putem renunța la comparările naive dacă avem speranțe reale că nu vor exista coliziuni. Vom considera orice apariție a lui $H(P)$ ca fiind o deplasare validă. Atunci algoritmul devine $\mathcal{O}(m+n)$, dar este important să alegem un modul suficient de mare încît probabilitatea coliziunilor să fie infimă.

Dacă chiar și cu un modul mare întîlnim coliziuni, putem calcula două funcții hash modulo două valori diferite, $H_1(P)$ și $H_2(P)$. Șansa să întîlnim o coliziune modulo ambele valori simultan scade pătratic.

37.7 Probleme

37.7.1 Problema Radio (Lot 2010)

enunț • sursă

Cred că problema admite multe soluții. Iat-o pe cea pe care am găsit-o eu.

Observăm că procentul de diferențe nu depășește 10% (cînd $k = 50$ și $l = 500$). Din păcate, noi nu știm să calculăm hash-uri aproximative, care să genereze valori hash similare pentru date de intrare similare. Trebuie să ne rezumăm la porțiuni identice.

Pentru această problemă, vom redefini deplasarea validă ca fiind o deplasare în s unde cuvîntul curent se potrivește cu cel mult k diferențe. Dacă există o deplasare validă pentru un cuvînt p, ce putem spune despre potrivirile **continue și exacte**? Să observăm că cele (cel mult) k diferențe

segmentează celelalte $l-k$ caractere, care se potrivesc exact, în cel mult $k+1$ fragmente. Conform principiului lui Dirichlet, va exista un fragment potrivit exact de lungime

$$c \stackrel{\text{def}}{=} \left\lceil \frac{l-k}{k+1} \right\rceil$$

Pentru $k = 50$ și $l = 500$ obținem $c = 9$, iar pentru alte combinații c va fi chiar mai mare. Deoarece $26^9 \approx 5 \cdot 10^{12}$, iar s este generat aleatoriu, ne așteptăm să existe cel mult o apariție a unui astfel de fragment în s. Doar pentru deplasarea acestei apariții ne punem întrebarea dacă există cel mult k diferențe. Dar cum? Tot pentru că s este aleatoriu, răspunsul este: naiv, cu o buclă `for`. Pentru majoritatea deplasărilor, comparația naivă de șiruri va eșua rapid. La fiecare poziție avem o șansă de 25/26 să întîlnim un caracter diferit, deci numărul așteptat de comparații pînă cînd găsim $k+1$ diferențe va fi $(k+1) \cdot 26/25$.

Și atunci algoritmul este:

Algoritmul 9 Algoritmul în ansamblu pentru problema Radio

```
1: calculează valorile hash ale fiecărei ferestre de mărime c din s
2: pentru fiecare cuvînt p execută
3:     pentru fiecare fereastră w de mărime c din p execută
4:         pentru fiecare apariție a lui w în s execută
5:             calculează deplasarea d a lui p în s
6:             compară naiv p cu subșirul s[d ... d + l − 1]
```

Merită să discutăm și implementarea și cîteva optimizări notabile. În primul rînd, cum memorăm poziția apariției unei valori hash în s? Am operat modulo 2^{64} ca să elimin conflictele. Limita de memorie este de 64 MB, ceea ce din păcate exclude o tabelă hash de $n = 10^6$ valori pe 64 de biți. Prima mea idee a fost să pun valorile hash într-un vector, să le sortez, apoi să caut binar valoarea hash dorită. Această implementare are valoarea $\mathcal{O}(n \log n + ml(\log n + k))$, care provine din:

1. Generarea și sortarea celor $n - c + 1$ valori hash din s.
2. Pentru fiecare șir și fiecare deplasare, căutarea binară a unei valori hash ($\mathcal{O}(\log n)$) și apoi compararea naivă ($\mathcal{O}(k)$).

Programul obține punctaj maxim, dar în aproape 500 ms. Dar prima sursă din top obține 125 ms. Aceasta m-a ambiționat! 🙂

O primă optimizare a fost să elimin sortarea. Timpul necesar pentru a sorta un milion de elemente nu este neglijabil. În loc de aceasta, am redus modulul pînă la un număr prim în jurul lui 10^6. Ne așteptăm ca șirul aleatoriu s să distribuie bine cele $\approx 10^6$ valori hash într-un spațiu de $\approx 10^6$ posibilități, dar natural pot apărea conflicte. În practică, numărul maxim de duplicate este 9 indiferent de modulul prim pe care l-am ales. De aceea, am înlocuit tabela hash cu o matrice de $9 \cdot 10^6$ elemente, plus un vector pentru numărul de apariții al fiecărei valori hash, așadar circa 40 MB de memorie. Astfel am redus timpul semnificativ, la 333 ms.

O a doua optimizare majoră pornește de la următoarea observație: șirul s este aleatoriu, dar cuvintele p nu! 😈 Voi cum ați construi date de test adverse? Iată ideea mea: Alegem drept cuvinte subșiruri de mărime l din s, pe care apoi le modificăm în $k+1$ locuri, undeva spre finalul cuvîntului. Grupăm aceste modificări, lăsînd doar ocazional cîte un caracter potrivit exact. Așadar, pentru un cuvînte de mărime $l = 500$ vom avea un fragment inițial de mărime peste 400 care se potrivește exact.

Ce decurge de aici? Programul nostru alege $c = 9$, deci va încerca toate cele circa 390 de fragmente în această fereastră de mărime 400. Și pentru fiecare dintre ele va face aproape 500 de comparații pînă să ajungă la cea de-a 51-a diferență. Complexitatea este $\mathcal{O}(l^2)$ per cuvînt, $\mathcal{O}(ml^2)$ în ansamblu. De fapt, presupunerea noastră că comparația naivă va eșua rapid, în $\mathcal{O}(k)$, este corectă doar pentru deplasări aleatorii, nu și pentru date adverse.

Care este soluția? Să observăm că toate aceste fragmente de lungime c duc la comparații naive **la aceeași deplasare în** s. Așadar, putem reduce mult timpul dacă memorăm deplasările din s la care am făcut deja comparația naivă. Putem face asta cu o tabelă hash sau cu un simplu vector. Aceasta reduce timpul de rulare la circa 160 ms.

Ca optimizare minoră (încă circa 10 ms), putem alege modulul 2^{20}, care accelerează împărțirile. Care este efortul total pentru Rabin-Karp? Avem $n + ml \approx 2,2M$ de caractere și fiecare caracter intră și iese din fereastra de mărime c. Intrarea necesită o operație modulo, iar ieșirea una sau două în funcție de implementare. Deci avem de făcut 4,4 sau 6,6 milioane de împărțiri. Nu este neglijabil!

Capitolul 38

Căutări cu automate finite. Algoritmul Knuth-Morris-Pratt

Algoritmul KMP este denumit astfel după inițialele inventatorilor săi, Donald Knuth, James Morris și Vaughan Pratt. Este un algoritm remarcabil de scurt și de elegant, dar fără înțelegerea bazelor pe care este construit el poate părea mistic, revelația unui profet pe un munte. Pentru a destrăma acest mister, vom face un periplu prin lumea automatelor finite.

38.1 Terminologie

Introducem încă cîteva notații suplimentare față de Secțiunea 37.1.

Notație. Vom nota cu S_k prefixul de lungime k al șirului S.

Notație. Vom nota cu $A \sqsubseteq B$ faptul că A este prefix al lui B, așadar că $|A| \leq |B|$ și că $A[i] = B[i]$ pentru $0 \leq i < |A|$. Dacă în plus $|A| < |B|$, vom folosi notația strictă $A \sqsubset B$.

Notație. Vom nota cu $A \sqsupseteq B$ faptul că A este sufix al lui B, așadar că $|A| \leq |B|$ și că $A[i] = B[|B| - |A| + i]$ pentru $0 \leq i < |A|$. Dacă în plus $|A| < |B|$, vom folosi notația strictă $A \sqsupset B$.

38.2 Automate finite

Colocvial, un **automat finit determinist** (AFD) este un sistem teoretic care procesează caracter cu caracter un șir dat la intrare, de lungime arbitrară, dar finită. Automatul acceptă unele șiruri și le respinge pe restul. Mulțimea șirurilor pe care automatul la acceptă se numește **limbajul recunoscut** de automat.

Să studiem următorul exemplu.

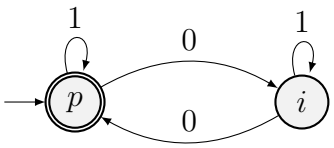


Figura 38.1: Un exemplu de automat finit determinist.

Distingem următoarele elemente:

- Automatul are două **stări** numite p și i. După fiecare caracter citit, automatul se va afla într-una din aceste stări.
- Starea p are o săgeată venind din exterior. Ea se numește **stare inițială**. În această stare se află automatul înainte de citirea primului caracter.
- Tot starea p are contur dublu. Aceasta o desemnează drept **stare de acceptare**. Pot exista oricîte stări de acceptare, inclusiv zero.
- Din fiecare stare pornește exact o săgeată pentru fiecare simbol din alfabet, în acest caz $\{0, 1\}$. Săgețile arată în ce stare trece automatul la citirea respectivului simbol. Aceste stări pot fi rezumate în **funcția** (sau **matricea**) **de tranziție**:

simbolul / starea	`0`	`1`
p	i	p
i	p	i

Tabela 38.1: Matricea de tranziție pentru automatul din Figura 38.1.

Pentru șirul de intrare `11010111`, automatul va porni din starea p și va trece succesiv prin stările p, p, i, i, p, p, p, p. Întrucît starea finală este una de acceptare, automatul va accepta șirul `11010111`.

Întrebarea este: Ce face acest automat în cazul general? Observația esențială este că el rămîne în aceeași stare pe caractere `1` și alternează între cele două stări pe caractere `0`. Rezultă că automatul se va afla în starea de acceptare (p) ori de cîte ori va fi văzut un număr par de caractere `0`. Formal, spunem că automatul recunoaște limbajul

$$L = \{s \mid s \in \{0,1\}^* \text{ și } s \text{ conține un număr par de zerouri}\}$$

De regulă avem de rezolvat problema inversă: dîndu-se un limbaj, dorim să construim un AFD care să-l recunoască. În acest caz, strategia este să înțelegem ce anume este reprezentativ pentru porțiunea din șir văzută pînă la caracterul curent și să definim stările automatului ca să captureze acea informație. Pentru automatul din Figura 38.1, informația reprezentativă este paritatea numărului de zerouri. De aceea am și denumit stările p (automatul a citit un număr par de zerouri, inițial 0) și i (automatul a citit un număr impar de zerouri).

38.2.1 Alte exemple de automate

Ca să ne familiarizăm cu procesul de gîndire prin care concepem un AFD pentru un limbaj dat, vom da cîteva exemple. Încercați să descoperiți voi înșivă ce limbaje recunosc aceste automate, înainte de a citi descrierile!

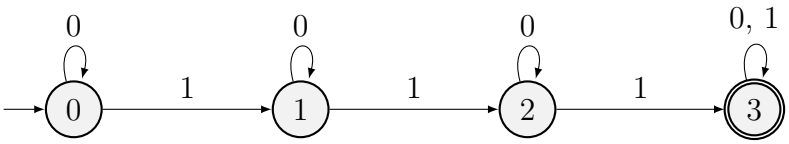


Figura 38.2

Automatul din Figura 38.2 ține evidența numărului de caractere `1` întîlnite: 0, 1, 2 sau 3 și peste. Într-adevăr, stările 0, 1, 2 și 3 au tocmai acest scop. Automatul acceptă șirurile care conțin cel puțin 3 caractere `1`.

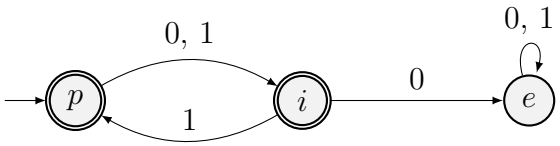


Figura 38.3

Automatul din Figura 38.3 acceptă șirurile care conțin `1` pe toate pozițiile pare. Pentru acest limbaj trebuie să ținem evidența a două informații: paritatea numărului de caractere citite (indicată de stările p și i) și dacă am văzut, pe orice poziție pară anterioară, un caracter `0`. În situația din urmă, automatul trece într-o stare „de eroare", e, în care el rămîne tot restul șirului și din care nu mai revine niciodată într-o stare de acceptare. Astfel, automatul va respinge toate șirurile în care vede un `0` pe o poziție pară și le va accepta pe restul.

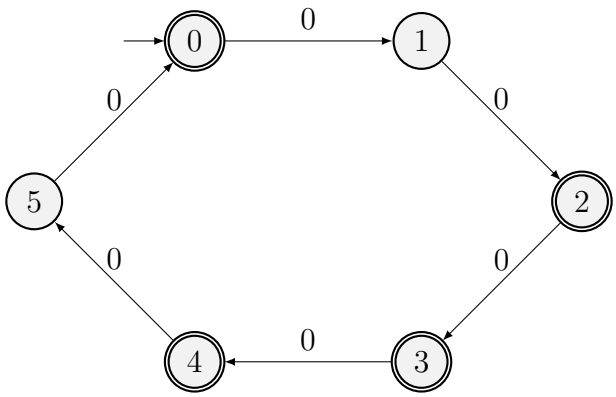


Figura 38.4

Automatul din Figura 38.4 operează pe un alfabet unar: șirurile conțin doar caracterul `0`. Observăm că el acceptă doar șiruri care constau din n simboluri, unde $n = 6k$ sau $n = 6k + 2$ sau $n = 6k + 3$ sau $n = 6k + 4$. Cu alte cuvinte, șiruri a căror lungime este multiplu de 2 sau multiplu de 3.

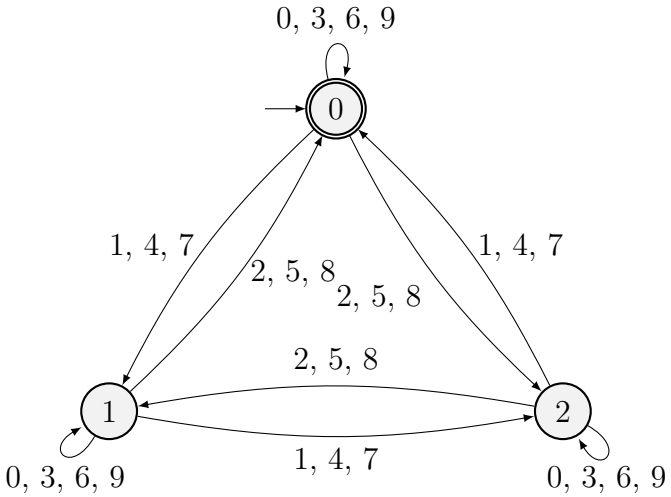


Figura 38.5

Automatul din Figura 38.5 operează pe alfabetul cifrelor zecimale. Dacă studiem cum sînt grupate cifrele, vom observa că automatul își schimbă starea în funcție de restul modulo trei al ultimei cifre citite. De exemplu, o cifră 1, 4 sau 7 va cauza trecerea în următoarea stare (ciclic). Astfel observăm că starea k arată că suma cifrelor citite pînă în prezent este congruentă cu k modulo 3. În fapt, automatul acceptă numere naturale divizibile cu 3.

38.3 Automate pentru recunoașterea de subșiruri

Să revenim la problema căutării unui subșir într-un șir. De exemplu, pe alfabetul binar `{a, b}`, să scriem un automat care ne ajută să găsim, în șirul dat la intrare, toate aparițiile șablonului `aab`. Metoda este să scriem un automat care recunoaște toate șirurile **terminate** în `aab`. Atunci automatul se va afla într-o stare de acceptare imediat după fiecare apariție a șablonului `aab`.

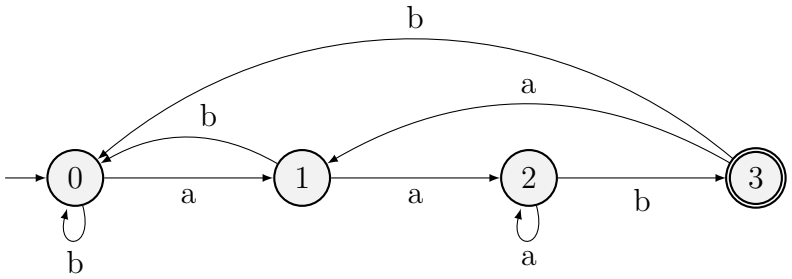


Figura 38.6: Un automat care recunoaște aparițiile subșirului `aab`.

Observăm că spre dreapta avem exact trei săgeți corespunzătoare caracterelor șablonului: `a`, `a`, `b`. Restul săgeților rămîn în aceeași stare sau merg spre stînga. Intuitiv, cele patru stări semnifică:

- 0: Ultimele caractere văzute nu sînt interesante.
- 1: Ultimul caracter văzut este `a`.
- 2: Ultimele două caractere văzute sînt `aa`.

- 3: Ultimele trei caractere văzute sînt aab.

În cazul general, starea k înseamnă că ultimele k simboluri citite de la intrare coincid cu prefixul de lungime k al șablonului. De aceea, este relativ clar că avem nevoie de trei săgeți care consumă caracterele a, a, b și merg exact cu o stare spre dreapta fiecare. Mai puțin evident este unde trebuie să ducă săgețile care merg spre stînga.

Un exemplu relevat este săgeata care ciclează în starea 2 pentru simbolul a. Într-adevăr, dacă ultimele două caractere văzute sînt aa și mai vedem încă unul, rezultă că ultimele trei caractere văzute sînt aaa. Dintre acestea, ultimele două încă se potrivesc cu definiția stării 2, deci automatul rămîne în acea stare. De exemplu, pentru șirul de intrare aaaab, automatul va trece succesiv prin stările $0, 1, 2, 2, 2, 3$ și va accepta șirul. Pentru același motiv, automatul trece din starea 3 în starea 1 pe simbolul a: putem folosi acel simbol ca eventual început al unei noi apariții a lui aab.

Să încheiem această secțiune cu un exemplu mai complex: un automat care recunoaște subșirul abacaba în alfabetul literelor mici.

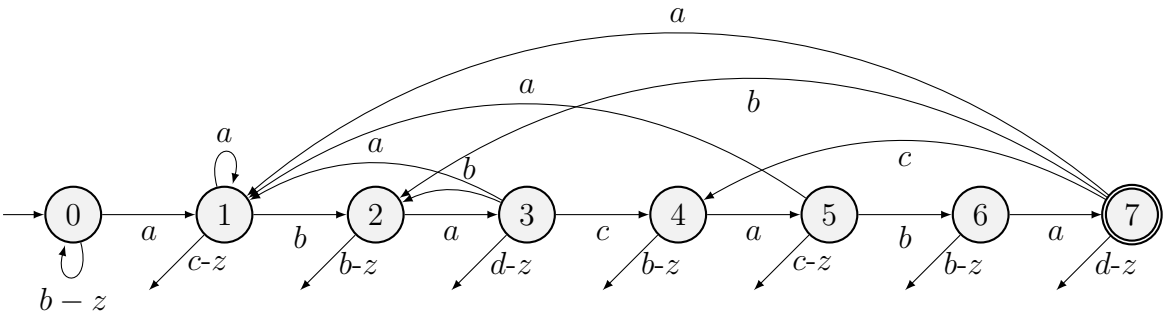


Figura 38.7: Un automat care recunoaște aparițiile subșirului abacaba. Săgețile de sub fiecare nod duc înapoi în starea 0.

Și în Figura 38.7 recunoaștem săgețile spre dreapta care consumă cîte un caracter din șablon. Detaliem și traiectoriile a două săgeți spre stînga:

- În starea 3, ultimele caractere văzute sînt aba. Săgeata din starea 3 în starea 2 pe simbolul b înseamnă că ultimele patru caractere văzute sînt abab, dintre care putem refolosi sufixul ab, cum ar fi de exemplu pentru șirul de intrare ababacaba.
- În starea 7, ultimele caractere văzute sînt abacaba. Săgeata din starea 7 în starea 4 pe simbolul c înseamnă că ultimele opt caractere văzute sînt abacabac, dintre care putem refolosi sufixul abac, cum ar fi de exemplu pentru șirul de intrare abacabacaba.

Dacă doriți să învățați mai multe despre automate finite și, în general, despre teoria limbajelor formale, vă recomand formidabila carte Introduction to the Theory of Computation de Michael Sipser.

38.4 Algoritmul de căutare cu automate finite

Acum, să generalizăm exemplele din secțiunea anterioară. Dorim să construim un automat care să recunoască un șablon arbitrar P primit la intrare. Putem deja spune destul de multe lucruri despre acest automat:

- El va avea $m+1$ stări numerotate de la 0 la m (reamintim că $m \stackrel{\text{not}}{=} |P|$).
- Starea inițială este 0.
- Unica stare de acceptare este m.
- Rămîne să definim matricea de tranziție, notată cu $\delta(q, c)$, pentru fiecare stare q și fiecare caracter c.

Dacă reușim să construim matricea de tranziție, algoritmul va rula în mod banal în $\mathcal{O}(n)$: tot ce are de făcut este să consume un caracter și să treacă în noua stare! Ori de cîte ori ajunge în starea m, algoritmul raportează o apariție a lui P.

Așadar, cum definim $\delta(q, c)$? Din Figurile 38.6 și 38.7 deducem un principiu general, a cărui demonstrație formală o omitem. Dacă automatul se află în starea q, înseamnă că ultimele q caractere citite coincid cu prefixul P_q. Dacă presupunem că următorul caracter întîlnit este c, atunci ultimele $q+1$ caractere citite sînt $P_q c$. Dintre acestea, încercăm să refolosim cît mai multe dintre ultimele, ca să rămînem într-o stare k cît mai mare.

Formal, cînd căutăm un șablon P într-un șir S, matricea de tranziție este:

$$\delta(q, c) = \max\{k \mid P_k \sqsupseteq P_q c\}$$

De aici decurge programul de mai jos, care construiește matricea în mod naiv, în $\mathcal{O}(|\Sigma| \cdot m^3)$. Complexitatea totală este $\mathcal{O}(|\Sigma| \cdot m^3 + n)$.

```
#include <stdio.h>
#include <string.h>

const int MAX_N = 1'000'000;
const int SIGMA = 26;

char s[MAX_N + 1], p[MAX_N + 1];
int d[MAX_N + 1][SIGMA];
int n, m;

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool naive_compare(char* a, char* b, int len) {
  int i = 0;
  while ((i < len) && (a[i] == b[i])) {
    i++;
```

```
  }

  return (i == len);
}

// Returnează true dacă și numai dacă P_k este sufix al lui P_{q}c.
bool is_suffix(int k, int q, char c) {
  return
    (k == 0) ||
    ((p[k - 1] == c) && naive_compare(p, p + (q - k + 1), k - 1));
}

void build_delta() {
  for (int q = 0; q <= m; q++) {
    for (int c = 0; c < SIGMA; c++) {
      int k = min(q + 1, m); // Nu putem depăși starea m.
      while (!is_suffix(k, q, c + 'a')) {
        k--;
      }
      d[q][c] = k;
    }
  }
}

int main() {
  scanf("%s %s", s, p);
  n = strlen(s);
  m = strlen(p);

  build_delta();

  int q = 0;
  for (int i = 0; i < n; i++) {
    q = d[q][s[i] - 'a'];
    if (q == m) {
      printf("Deplasare validă: %d\n", i - m + 1);
    }
  }

  return 0;
}
```

38.5 Algoritmul Knuth-Morris-Pratt

Înarmați cu aceste cunoștințe, putem analiza algoritmul KMP. El îmbunătățește major două aspecte ale algoritmului de căutare cu automate:

1. Reduce memoria necesară de la $\mathcal{O}(|\Sigma| \cdot m)$ la $\mathcal{O}(m)$.

2. Reduce timpul de calcul al informațiilor necesare de la $\mathcal{O}(|\Sigma| \cdot m^3)$ la $\mathcal{O}(m)$.

Algoritmul calculează o **funcție de prefix** π, un vector cu m elemente, dependent doar de P, dar nu de S nici de mărimea alfabetului. Funcția de tranziție δ răspundea la întrebarea (în limbaj natural): „Sînt în starea q și văd la intrare caracterul c. În ce stare să trec?" Funcția π răspunde la o întrebare similară: „Sînt în starea q. Presupunînd că următorul caracter de la intrare nu se potrivește, în ce stare să trec?"

Pentru a înțelege de ce această informație este semnificativă, să studiem comportamentul algoritmului cu automate finite pentru datele din figura 38.8.

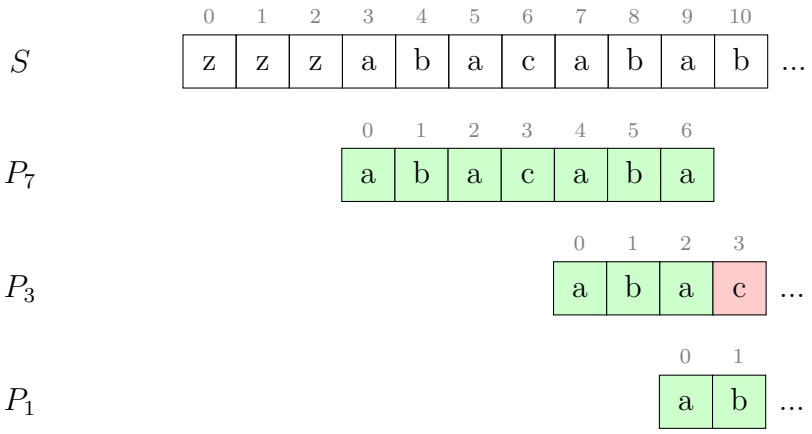


Figura 38.8: La poziția 10, nu putem extinde prefixul P_3 cu caracterul b, ci numai prefixul P_1.

Automatul va găsi o deplasare validă și va trece în starea $|P| = 7$ după ce consumă caracterele $S[0 \ldots 9]$. Apoi va consuma următorul caracter (b) și va trece din starea 7 în starea $\delta(7, \mathsf{b}) = 2$, deoarece poate reutiliza ultimele două caractere, ab. În fapt, automatul precalculează cel mai lung sufix al lui P_7 care este și sufix al lui P_7 și care poate fi extins cu caracterul b.

Algoritmul KMP are o abordare ușor diferită: el evaluează **toate** prefixele lui P_7 care sînt și sufixe ale lui P_7, respectiv $P_3 = \mathsf{aba}$ și $P_1 = \mathsf{a}$. Algoritmul încearcă să treacă în starea 3, dar constată că P_3 nu poate fi extins cu caracterul b (căci $\mathsf{b} \neq P[3] = \mathsf{c}$). Apoi, algoritmul încearcă să treacă în starea 1 și constată că P_1 poate fi extins. Așadar, algoritmul trece în starea 1, apoi consumă caracterul de la intrare (b) și urcă în starea 2.

Rezultă că funcția de prefix π aferentă unui șablon P trebuie să calculeze, pentru fiecare stare q, toate prefixele lui P_q care sînt și sufixe al lui P_q. În fapt, lucrurile sînt mai simple. Este suficient ca $\pi[q]$ să stocheze doar cel mai lung dintre aceste prefixe, din care apoi le putem deduce pe următoarele. De exemplu, prefixele lui $P_7 = \mathsf{abacaba}$ care sînt și sufixe sînt $P_3 = \mathsf{aba}$ și $P_1 = \mathsf{a}$, dar observăm că, la rîndul său, P_1 este prefix și sufix al lui P_3. De aceea, lista completă a prefixelor lui P_q care sînt și sufixe va fi $\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \ldots$

Formal, definiția lui π este:

$$\pi(q) = \max\{k \mid k < q \text{ și } P_k \sqsupset P_q\}$$

Cum construim vectorul π? În mod foarte elegant, P trebuie să răspundă la aceeași întrebare despre el însuși ca și despre vectorul S: dacă mă aflu în starea q, care este cel mai lung prefix al lui P care este și sufix al lui P_q?

Redăm implementarea algoritmului. Precizăm că indicii în `pi[]` sînt decalați cu 1, deoarece vectorul este indexat de la zero. De exemplu, informația „pentru șirul abacaba, cel mai lung prefix care este și sufix este aba" se notează la poziția 6, așadar `pi[6] = 3`.

```
#include <stdio.h>
#include <string.h>

const int MAX_N = 1'000'000;

char s[MAX_N + 1], p[MAX_N + 1];
int pi[MAX_N + 1];
int n, m;

void build_pi() {
  pi[0] = 0;
  int k = 0;
  for (int q = 1; q < m; q++) {
    while ((k > 0) && (p[k] != p[q])) {
      k = pi[k - 1];
    }
    if (p[k] == p[q]) {
      k++;
    }
    pi[q] = k;
  }
}

void kmp() {
  int q = 0; // starea curentă
  for (int i = 0; i < n; i++) {
    while ((q > 0) && (p[q] != s[i])) {
      q = pi[q - 1];
    }
    if (p[q] == s[i]) {
      q++;
    }
    if (q == m) {
      printf("Deplasare validă: %d\n", i - m + 1);
    }
  }
}
```

```
int main() {
  scanf("%s %s", s, p);
  n = strlen(s);
  m = strlen(p);

  build_pi();
  kmp();

  return 0;
}
```

38.5.1 Analiza complexității

Aparent, construcția lui π poate dura $\mathcal{O}(m^2)$, întrucît fiecare buclă `while` poate face m iterații. Totuși, algoritmul KMP, ca și automatul finit, avansează spre dreapta cel mult cu o stare la fiecare caracter citit. De aceea, numărul de incrementări ale lui k în funcția `build_pi()` este de cel mult $m-1$, iar numărul de scăderi nu poate fi mai mare. Funcția are complexitatea amortizată $\mathcal{O}(m)$. Prin același raționament, funcția `kmp()` are complexitatea amortizată $\mathcal{O}(n)$, iar complexitatea totală a algoritmului KMP este $\mathcal{O}(m+n)$.

38.6 Aplicații

Iată două aplicații foarte directe. Secțiunea de probleme conține aplicații mai complexe.

Test de rotație

Date fiind două șiruri S și T, determinați dacă T este rotația lui S.

Soluție: Concatenăm S cu el însuși. În șirul $S+S$ îl căutăm pe T.

Completarea unui palindrom

Dat fiind un șir S, adăugați cît mai puține caractere la finalul lui pentru a îl face palindrom.

Soluție: Ne interesează cel mai lung sufix al lui S care este palindrom. Prefixul dinaintea acestui palindrom trebuie răsturnat și adăugat după S. De aceea, Construim șirul $S^R+\#+S$. Pe acest șir calculăm funcția π. Prin definiție, $\pi[n-1]$ ne arată cel mai lung sufix al lui S care este prefix al lui S^R, adică exact cel mai lung sufix palindromic al lui S.

38.7 Probleme

38.7.1 Problema Bart (NerdArena)

enunț • sursă

Problema ne cere să aflăm cel mai mic prefix al unui șir S pe care, dacă îl repetăm, obținem șirul S. Ultima repetiție poate fi incompletă.

Fie răspunsul k. Atunci $S[0 \dots k-1] = S[k \dots 2k-1] = \dots$. Dar aceasta înseamnă că $S[0 \dots n-k-1] = S[k \dots n-1]$. Cu alte cuvinte, sufixul de lungime $n-k$ este și prefix! Răspunsul este pur și simplu $n - \pi[n-1]$.

38.7.2 Problema Cifru (Baraj ONI 2006)

enunț • sursă

În această problemă recunoaștem testul clasic dacă două șiruri S și T se pot obține unul dintr-altul prin rotații. Dar cum gestionăm permutarea?

Vom modifica algoritmul KMP ca să țină cont și de permutări. Să considerăm șablonul din exemplu, $P = (2\ 1\ 3\ 3\ 2\ 1)$. La poziția i dorim să aflăm lungimea celui mai lung prefix care *este* și sufix. Dar aceasta depinde de sensul cuvîntului „este”[1]. Nu ne interesează ca prefixul să fie **egal** cu sufixul, ci să existe o permutare care mapează sufixul în prefix. De exemplu, $\pi[4] = 2$ pentru că prefixul $(2\ 1)$ se potrivește cu sufixul $(3\ 2)$.

Cum construim funcția π astfel modificată? Să pornim de la $\pi[2] = 2$ (deoarece $(2\ 1)$ se potrivește cu $(1\ 3)$). Am putea extinde cu succes perechea de subșiruri cu o poziție dacă $(2\ 1\ 3)$ s-ar potrivi cu $(1\ 3\ 3)$. Dar nu este cazul. În primul subșir, 3 este o valoare nou întîlnită, pe cînd în al doilea valoarea 3 a apărut și pe poziția 1. De aceea nu există nicio permutare care să mapeze prefixul peste sufix. Așadar, trebuie să înlocuim testul de egalitate de caractere din algoritmul KMP normal cu un test de asemănare între caractere:

- Dacă $P[k]$ este la prima apariție (deci $P[k] \neq P[0], P[1], ..., P[k-1]$), trebuie ca și $P[i] \neq P[i-k], P[i-k+1], ..., P[i-1]$.
- Altfel, dacă $P[k] = P[k']$ pentru un $k' < k$, trebuie ca și $P[i] = P[i-(k-k')]$.

Putem verifica aceste (in)egalități dacă stocăm, pentru fiecare poziție k, poziția apariției anterioare a lui $P[k]$ în P.

Același raționament îl facem și în a doua trecere, în care îl căutăm pe P în $S+S$. Cînd evaluăm $S[i]$ și sîntem în starea k, vom confrunta aparițiile anterioare ale lui $S[i]$ în S cu aparițiile lui $P[k]$ în P.

38.7.3 Problema Anthem of Berland (Codeforces)

enunț • sursă

Problema ne poate duce rapid cu gîndul la elementele de bază:

- KMP și funcția prefix, deoarece avem un șablon ale cărui apariții se pot suprapune.
- Programare dinamică.

[1] Exact ca în cazul lui Bill Clinton.

Să punem la punct detaliile. Voi folosi simbolul P (*pattern*) în loc de T, din obișnuință. Ne putem convinge rapid că o abordare greedy nu funcționează. Iată un exemplu similar cu cel din enunț: $S =$ `????abcab`, $P =$ `abcab`. Atunci este important să completăm șirul ca $S =$ `?abcabcab`, începînd cu a doua poziție, pentru a obține două potriviri.

Ce parametri ne trebuie ca să definim bine recurența? Din exemplul de mai sus observăm că nu este suficient să definim $i =$ numărul de litere consumate din S ($0 \leq i \leq |S|$). Trebuie să putem spune „după prima literă, vrem să fim în starea 0, și abia de la a doua începînd să trecem în stările 1, 2 etc." Deci mai definim și $j =$ starea în care ne aflăm ($0 \leq j \leq |P|$). Fie $d_{i,j}$ numărul maxim de suprapuneri pe care îl putem obține pînă la aceste coordonate sau -1 dacă nu putem ajunge la aceste coordonate.

Eu am ales să formulez programarea dinamică înainte, astfel: din $d_{i,j}$ trecem în $d_{i+1,k+1}$, unde:

- k este orice tranziție posibilă din starea j;
- $S[i] = P[k]$ sau $S[i] =$ `?`;
- Cînd spunem „trecem în", vrem să spunem că $d_{i+1,k+1}$ primește maximul din ce avea deja și $d_{i,j}$, eventual plus încă o unitate dacă $k + 1 = |P|$ (deoarece am completat încă o apariție).

De exemplu, pentru șirul $P =$ `abacaba`, valoarea $d_{20}7$ arată cîte potriviri putem obține în primele 20 de caractere, astfel încît după al 20-lea (deci $S[19]$) să fim în starea 7 (deci să terminăm o potrivire). De aici putem trece:

- în $d_{21,4}$ dacă următorul caracter este `c` sau `?`, deoarece din `abacabac` putem refolosi 4 litere;
- în $d_{21,2}$ dacă următorul caracter este `b` sau `?`, deoarece din `abacabab` putem refolosi 2 litere;
- în $d_{21,1}$ dacă următorul caracter este `a` sau `?`, deoarece din `abacabaa` putem refolosi o literă;
- în $d_{21,0}$, altfel.

Soluție în $\mathcal{O}(SP^2)$

Prima iterație a codului meu proceda chiar astfel. Pentru fiecare i și fiecare j, pornea cu $k = j$ și repeta $k = \pi[k-1]$ pînă cînd ajungea la 0. Dar acest cod este prea lent, deoarece ajunge la $\mathcal{O}(SP^2)$ atunci cînd funcția π scade lent, de exemplu pentru un șablon ca $P =$ `aaa …aaa`.

Soluție în $\mathcal{O}(SP\Sigma)$

În loc de aceasta, putem să calculăm tranzițiile din starea $d_{i,j}$ în alte stări conform următorului caracter, care fie este impus (literă), fie poate lua Σ valori (`?`). Practic, trebuie să răspundem la întrebări de tipul: „Dacă ne aflăm în starea j, iar următorul caracter de la intrare este $S[i]$, în ce stare ajungem?". Aceste informații reprezintă exact matricea de tranziție despre care am vorbit în capitolul despre automate! Cu ajutorul funcției de prefix π, putem calcula această matrice în $\mathcal{O}(P^2)$ (o sursă mai veche) sau chiar $\mathcal{O}(P\Sigma)$.

Capitolul 39

Funcția Z

În acest minicapitol vom introduce încă o funcție utilă în algoritmii pe șiruri de caractere. Vă recomand cu căldură articolul de pe CP Algorithms, care este bine scris și din care acest capitol se inspiră.

39.1 Definiție

Dat fiind un șir S de n caractere, funcția Z asociată este

$$Z(i) = \max\{k \mid S[i \dots i+k) = S_k\} \quad \text{pentru} \quad 1 \leq i < n$$

În cuvinte, $Z(i)$ este numărul maxim k de caractere începînd cu poziția i care se potrivesc cu începutul lui S. Prin convenție, $Z(0) = 0$. Figura 39.1 arată un exemplu.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S	a	b	a	b	a	c	d	a	b	a	b	a	b	a	c	d
Z	0	0	3	0	1	0	0	5	0	7	0	3	0	1	0	0

Figura 39.1: Funcția Z. La poziția 2, $Z(2) = 3$, ceea ce arată că $S[2 \dots 5) = S[0 \dots 3)$. Similar, $Z(7) = 5$ arată că $S[7 \dots 12) = S[0 \dots 5)$.

39.2 Calcularea funcției Z

Desigur, putem calcula funcția Z naiv, în $\mathcal{O}(n^2)$:

```
for (int i = 1; i < n; i++) {
  while ((i + z[i] < n) && (s[z[i]] == s[i + z[i]])) {
    z[i]++;
  }
```

```
}
```

Dar există și un algoritm mai bun, în $\mathcal{O}(n)$. El procedează astfel. În primul rînd, să observăm că la orice poziție j intervalul $[j, j + Z(j))$ indică exact suprapunerea maximă cu un prefix al lui S. Acum, să considerăm că sîntem la pasul i și dorim să calculăm $Z(i)$. Dintre toate intervalele $[j, j + Z(j))$ calculate anterior (cu $j < i$), îl vom reține mereu pe cel cu capătul drept maxim, pe care îl vom denumi $[l, r)$. Cu alte cuvinte, intervalul $[l, r)$ este cel mai din dreapta interval de pînă acum despre care știm că este prefix al lui S.

Poziția r din șir are o semnificație aparte. De la r spre dreapta, algoritmul încă nu a examinat niciun caracter și nu știe nimic despre S. Deci r este ca o frontieră pe care treptat o împingem spre dreapta.

Acum să ne gîndim în ce situație se poate afla i raportat la $[l, r)$. Este clar că $i > l$, deoarece $[l, r)$ corespunde unei poziții j deja evaluate. Deci rămîn două cazuri.

Dacă $i \geq r$, sîntem dincolo de frontieră, în teritoriu necunoscut. Îl vom calcula pe $Z(i)$ naiv, poziție cu poziție. Dacă $Z(i) > 0$ sau dacă $i > r$, aceasta va conduce și la actualizarea intervalului $[l, r)$, deoarece capătul drept va fi $i + Z(i) > r$.

Iar dacă $i < r$, vom refolosi informații din valorile Z anterioare, astfel. Știm că $S[l \ldots r) = S[0 \ldots r - l)$. Asta înseamnă că $S[i \ldots r) = S[i - l \ldots r - l)$. Așadar, orice se poate spune despre subșirul lui S care începe la poziția i știm deja de la poziția $i - l$. De aceea, **în principiu**, $Z(i) = Z(i - l)$. Dar sînt necesare două modificări:

1. Din cauza frontierei r, noi nu cunoaștem mai mult de $r - i$ caractere la dreapta. Deci vom lua $Z(i) = \min(Z(i - l), r - i)$.
2. Se prea poate ca $S[i \ldots r)$ să continue să se potrivească cu prefixul lui S și dincolo de frontiera r. Deci vom face o extindere naivă a lui $Z(i)$.

Figura 39.2 ilustrează situația a doua.

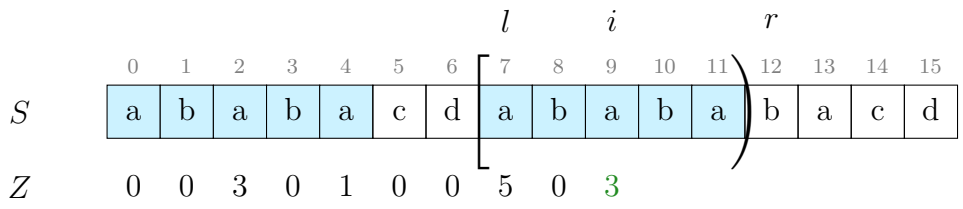


Figura 39.2: La poziția $i = 9$, intervalul curent este $[l, r) = [7, 12)$, ceea ce arată că $S[7 \ldots 12) = S[0 \ldots 5)$. De aceea, inițializăm $Z(9)$ cu $Z(9 - 7) = Z(2) = 3$. Ulterior, prin comparații naive, îl creștem pe $Z(9)$ pînă la valoarea 7. Noul interval va fi $[l, r) = [9, 16)$.

39.3 Implementare

```
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
  if (i < r) {
    z[i] = std::min(r - i, z[i - l]);
  }
  while (s[z[i]] == s[i + z[i]]) {
    z[i]++;
  }
  if (i + z[i] > r) {
    l = i;
    r = i + z[i];
  }
}
```

Două observații:

- Dacă `i >= r`, atunci valoarea inițială a lui `z[i]` rămîne cea preinițializată, adică 0.
- Comparația naivă din `while` va eșua cel mai tîrziu la sfîrșitul șirului, cînd vom compara `s[z[i]]` cu terminatorul de șir, `'\0'`. Dacă implementați algoritmul pe `std::string`, trebuie să adăugați condiția suplimentară `i < n`.

39.4 Analiza complexității

Ca și în cazul algoritmului KMP, și calculul funcției Z pare $\mathcal{O}(n^2)$ datorită buclelor imbricate `for` și `while`. Dar putem da o limită mai bună. La pasul i:

- Dacă $i \geq r$, atunci comparațiile naive din bucla `while` au ca efect creșterea lui r, cu excepția cazului în care chiar prima comparație eșuează. Aceasta deoarece sîntem dincolo de frontieră, deci intervalul $[i, i + Z(i))$ va fi noul interval ales.
- Dacă $i < r$ și am inițializat $Z(i)$ chiar cu $r - i$ (ca în cazul $i = 9$ din figura 39.2), atunci din nou fiecare iterație a buclei `while` cu excepția primeia va duce la creșterea lui r, pentru că $i + Z(i) = r$, deci capătul drept este deja la frontieră.
- Dacă $i < r$ și am inițializat $Z(i)$ cu o valoare mai mică decît $r - i$ (ca în cazul $i = 8$ din aceeași figură), atunci prima comparație din bucla `while` va eșua. Dacă n-ar fi fost așa, atunci aceeași comparație ar fi avut succes și la poziția $i - l$ și deci $Z(i - l)$ ar fi fost mai mare. De exemplu, presupunînd prin absurd că am determina că $Z(8) = 1$, aceasta ar însemna pe de o parte că $S[8] = S[0] =$ `a`. Pe de altă parte, noi știm că $S[8] = S[1]$ deoarece fereastra $[7, 12)$ este o copie a ferestrei $[0, 5)$. Atunci $S[1] = S[0]$ și ar fi trebuit ca și $Z[1]$ să fie egal cu 1.

Cum valoarea lui r nu poate depăși n, rezultă că comparația din bucla `while` are cel mult n eșecuri și cel mult n potriviri. Algoritmul rulează în $\mathcal{O}(n)$.

39.5 Aplicații

Să reluăm aplicațiile și problemele din capitolul anterior. Nu vom mai include programe complete, întrucît ele sînt foarte similare cu versiunile cu KMP.

Căutarea unui subșir într-un șir

Vom rezolva aceeași problemă ca și în capitolele despre Rabin-Karp și KMP: Date fiind un șir S de lungime n și un șablon P de lungime m, vom găsi toate aparițiile lui P în S. Pentru testare, puteți rezolva problema String Matching (CSES).

O primă variantă tratează separat șirurile S și P, similar algoritmului KMP. Construim întîi funcția Z peste șablonul P. Apoi parcurgem șirul S și aflăm la fiecare poziție numărul de caractere care se potrivesc cu un prefix al lui P. Calculele făcute în S nu mai afectează vectorul Z deja calculat.

Ca și în algoritmul KMP, cele două porțiuni de cod seamănă. Esența este:

```
void compute_z() {
  int l = 0, r = 0;
  for (int i = 1; i < m; i++) {
    if (i < r) {
      z[i] = std::min(r - i, z[i - l]);
    }
    while (p[z[i]] == p[i + z[i]]) {
      z[i]++;
    }
    if (i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
}

int count_occurrences() {
  int result = 0;
  int l = 0, r = 0, d;

  for (int i = 0; i < n; i++) {
    if (i < r) {
      d = std::min(r - i, z[i - l]);
    } else {
      d = 0;
    }
    while ((i + d < n) && (p[d] == s[i + d])) {
      d++;
    }
    if (i + d > r) {
```

```
      l = i;
      r = i + d;
    }

    if (d == m) {
      result++;
    }
  }

  return result;
}
```

O implementare mai concisă calculează funcția Z peste șirurile concatenate $P + S$. Odată ce atingem poziția m (deci intrăm în șirul S), orice poziție i unde $Z(i) \geq m$ indică o deplasare validă. Pentru claritate, putem să inserăm un simbol din afara alfabetului în concatenare: $P\#S$. Dar acest lucru nu este strict necesar.

```
int l = 0, r = 0;
int occurrences = 0;

for (int i = 1; i < m + n; i++) {
  if (i < r) {
    z[i] = std::min(r - i, z[i - l]);
  }
  while (p[z[i]] == p[i + z[i]]) {
    z[i]++;
  }
  if (i + z[i] > r) {
    l = i;
    r = i + z[i];
  }

  if ((i >= m) && (z[i] >= m)) {
    occurrences++;
  }
}
```

Test de rotație

Pentru a determina dacă două șiruri S și T se pot obține unul dintr-altul printr-o rotație, este suficient să îl căutăm pe T în $S + S$, folosind codul de mai sus.

Completarea unui palindrom

Dat fiind un șir S, adăugați cît mai puține caractere la finalul lui pentru a îl face palindrom.

Ca și în varianta cu KMP, ne interesează cel mai lung sufix al lui S care este palindrom. Pentru

aceasta, construim șirul $S^R + \# + S$. Pe acest șir de lungime $2n+1$ căutăm cea mai mică poziție k pentru care $k + Z(k) = 2n + 1$, deoarece sufixul care începe la această poziție este cel mai lung sufix palindromic al lui S.

Problema Bart (NerdArena)

Reluăm cerința: trebuie să aflăm cel mai mic prefix al unui șir S pe care, dacă îl repetăm, obținem șirul S. Ultima repetiție poate fi incompletă.

Logica este similară cu cea bazată pe KMP. Fie răspunsul k. Atunci $S[0 \dots k-1] = S[k \dots 2k-1] = \dots$. Dar aceasta înseamnă că $S[0 \dots n-k-1] = S[k \dots n-1]$. Prin definiție, aceasta înseamnă că $Z(k) = n - k$. Așadar, căutăm prima poziție k unde $k + Z(k) = n$.

Problema Cifru (Baraj ONI 2006)

Soluția este similară cu cea bazată pe KMP. În loc să testăm ciclicitatea cu KMP, o vom testa cu funcția Z. De asemenea, trebuie să redefinim calculul funcției Z ca să nu testeze egalitatea perfectă, ci egalitatea prin prisma unei permutări.

Nu mai detaliem codul, dar ilustrăm valoarea funcției Z astfel modificate pentru exemplul de la intrare, respectiv $P =$ `213321` și $S =$ `131322`. Am inserat un caracter `#` doar pentru claritate.

$P + \# + S + S$	2	1	3	3	2	1	#	1	3	1	3	2	2	1	3	1	3	2	2
Z	0	2	1	3	3	2	3	2	2	**6**	2	1	3	2	2	4	2	1	1

Tabela 39.1: Funcția Z pentru exemplul din problema Cifru. Valoarea maximă este 6 deoarece subșirul (1 3 2 2 1 3) corespunde cu prefixul (2 1 3 3 2 1) dacă aplicăm permutarea (2 3 1).

Capitolul 40

Quicksort ternar

În acest scurt capitol vom studia un algoritm simplu și util, dar cvasinecunoscut. Ca întotdeauna, pot exista alți algoritmi mai scurți sau mai rapizi, însă fiecare armă pe care ne-o adăugăm la arsenal este binevenită.

40.1 Algoritmul

Dacă dorim să sortăm naiv n șiruri de cîte m caractere, atunci sortarea naivă (de exemplu `std::sort`) va avea o complexitate de $\mathcal{O}(mn \log n)$, întrucît compararea a două stringuri durează $\mathcal{O}(m)$. Această sortare este agnostică. Ori de cîte ori compară două șiruri, ea repornește de la primul caracter. Dar, pe măsură ce intervalele de sortat se fragmentează, șirurile vor începe să aibă prefixe comune, posibil lungi. Ar fi frumos să evităm recompararea lor.

Algoritmul de quicksort ternar (engl. Multi-key quicksort) face exact acest lucru. Algoritmul alege un caracter pivot, p și face o partiționare similară cu quicksort, dar în trei categorii:

1. Stringuri care au pe prima poziție un caracter mai mic decît p.
2. Stringuri care au pe prima poziție caracterul p.
3. Stringuri care au pe prima poziție un caracter mai mare decît p.

Algoritmul se reapelează pentru aceste trei categorii, cu o observație esențială: știm că șirurile din categoria (2) au primul caracter identic, deci le vom reexamina începînd cu al doilea caracter. La fel procedăm și pentru caracterele următoare.

40.2 Implementare

```
char c[N][MAX_LEN + 1]; // conținutul șirurilor
char* s[N];             // ordinea șirurilor

// Sortează șirurile s[left...right), care sînt egale pînă la poziția pos-1.
// Modifică s[], dar nu copiază șirurile în sine, căci ar fi costisitor.
void tsort(int left, int right, int pos) {
```

```
  if (right - left <= 1) {
    return;
  }

  int i = left, lt = left, gt = right;
  char pivot = s[left][pos];

  // Invariant de buclă:
  // * s[left ... lt)     al pos-lea caracter este < pivot
  // * s[lt ... i)        al pos-lea caracter este == pivot
  // * s[i]               șirul în curs de evaluare
  // * s(i ... gt)        șiruri încă neevaluate
  // * s[gt ... right)    al pos-lea caracter este > pivot
  while (i < gt) {
    if (s[i][pos] < pivot) {
      swap(s[i++], s[lt++]);
    } else if (s[i][pos] > pivot)  {
      swap(s[i], s[--gt]);
    } else {
      i++;
    }
  }

  tsort(left, lt, pos);
  if (s[lt][pos] != '\0') {
    tsort(lt, gt, pos + 1);
  }
  tsort(gt, right, pos);
}

int main() {
  ...
  for (int i = 0; i < N; i++) {
    s[i] = (char*)c[i];
  }
  tsort(0, N, 0);
  ...
}
```

Remarcăm că algoritmul nu mută niciodată *conținutul* șirurilor, ci doar interschimbă *pointerii* la șiruri. Eu am ales să fac asta ținînd în `s` pointeri la șirurile efective. Dacă preferați, puteți gestiona ordinea folosind indici de linie într-o matrice de caractere sau alte tehnici.

Complexitatea teoretică este egală cu a algoritmului quicksort obișnuit, dar operația de bază este compararea de caractere individuale, nu de șiruri.

40.3 Benchmarks

Am făcut benchmark-uri cu alte două metode: stringuri STL sortate cu STL și stringuri C sortate cu STL. Setul de date conține 1.000.000 de șiruri aleatoare de lungime între 20 și 50. Rezultatele au fost:

metodă	timp
`std::sort` pe `string` C++	445 ms
`std::sort` pe `char*`	321 ms
sortare ternară pe `char*`	125 ms

Rezultatele variază, dar nu drastic, în funcție de distribuția literelor din șiruri. Sortarea ternară tratează bine cazurile degenerate (în special situațiile cu prefixe egale lungi).

Mai important, acest algoritm ne permite să intervenim **în timpul sortării** pentru a folosi valorile parțiale, după cum vom vedea în problemele următoare.

40.4 Probleme

40.4.1 Problema Cuvinte (ONI 2021, clasele 11-12)

enunț • sursă

Observație. Dacă setul inițial conține două cuvinte P și S unde $P \sqsubseteq S$ îl putem ignora pe S.

Demonstrație. Dacă un cuvînt generat, C, era compatibil cu setul inițial, el va fi compatibil și cu setul fără S (doar am eliminat o restricție). Dacă era incompatibil din cauza lui P sau a lui S, va fi incompatibil și cu setul fără S. Există trei cazuri posibile și în toate trei continuă să existe o incompatibilitate între C și P:

1. $C \sqsubseteq P \sqsubseteq S$
2. $P \sqsubseteq C \sqsubseteq S$
3. $P \sqsubseteq S \sqsubseteq C$

□

Vom elimina toate cuvintele din set care au prefixe tot în set și vom denumi ceea ce rămîne **setul redus**.

Mai departe, mie mi-a sărit în ochi exemplul 2 din enunț. El este suficient de mic încît să-l putem analiza de mînă, iar răspunsul este suficient de mare încît să ne confirme orice intuiție avem despre soluție. O primă intuiție este că răspunsul este un produs de factori, dintre care unii vor fi mici. Așadar, să folosim din plin uneltele pe care ni le oferă GNU/Linux! 🙂

```
[cata@neil cuvinte]$ factor 925829353
925829353: 31 89 335567
[cata@neil cuvinte]$ echo '925829353 + 1000000007' | bc | factor
```

```
1925829360: 2 2 2 2 3 3 5 7 7 13 13 17 19
```

Setul redus este (*aaaab*, *aab*, *bb*). Are sens să adăugăm cuvinte în ordinea crescătoare a lungimii, deoarece un cuvînt odată ales va interzice adăugarea altor cuvinte cărora el le este prefix.

Lungime 2. Ce cuvinte putem adăuga? Din cele $3 \times 3 = 9$ posibile, scădem prefixele de lungime 2 existente în setul de intrare, deci *aa* și *bb*. Rămîn 7 posibilități. Să alegem una și să o denumim simbolic *pp*. Nu contează dacă *pp* este *ac* sau *ca* sau altceva. Ce contează este că orice continuare a lui *pp* cu alte litere devine ilegală.

Lungime 3. Din $3^3 = 27$ de posibilități, 8 sînt ilegale:

- *bb* și *pp* urmate de orice caracter. Vom nota continuările cu caracterul ?, așadar *bb*? și *pp*?.
- Prefixele de lungime 3 din setul de intrare, deci *aaa* și *aab*. Rămîn 19. Să generăm un nou cuvînt simbolic, *qqq*. Facem remarca încurajatoare că 7 și 19 se află printre factorii răspunsului așteptat.

Lungime 4. Din $3^4 = 81$ de posibilități, 25 sînt ilegale. Să detaliem puțin ca să obținem regula generală:

- Existau deja 6 cuvinte de lungime 3: *bb*? și *pp*?.
- La acestea adăugăm cuvintele de la intrare de lungime fix 3: *aab*.
- La acestea adăugăm cuvintele de lungime 3 generate: *qqq*.
- Oricare dintre acestea **nu** pot fi extinse cu niciun caracter, deci $8 \times 3 = 24$ de cuvinte ilegale.
- În plus, mai există un singur prefix de lungime 4 în șirul de intrare: *aaaa*.

Posibilitățile legale sînt $81 - 25 = 56$, dar nu contează, căci nu avem de generat cuvinte de lungime 4. Acum ne putem aventura să formulăm regula generală.

- Operăm doar pe setul redus (eliminăm toate cuvintele care au prefixe în set).
- Fie Σ mărimea alfabetului.
- Fie l lungimea curentă a cuvintelor pe care le studiem.
- Fie f_l numărul de cuvinte de lungime l din setul redus și fie g_l numărul de cuvinte de lungime l pe care trebuie să le generăm noi.
- Fie C_l numărul de cuvinte de lungime l pe care nu avem voie să le mai extindem.
- Atunci $C_l = (C_{l-1} + f_{l-1} + g_{l-1}) \cdot \Sigma$. Aceasta deoarece nici cuvintele deja interzise anterior, nici cele din setul redus, nici cele generate de noi de lungime exact $l - 1$ nu mai pot fi continuate în mod legal cu niciun caracter.
- În plus, doar la pasul curent (fără repercusiuni la lungimile viitoare), nu avem să generăm cuvinte de lungime l care apar ca prefixe în setul redus. De exemplu, la $l = 4$ nu avem voie să generăm cuvîntul *aaaa*, dar la $l = 5$ vom avea voie să generăm cuvintele *aaaaa* sau *aaaac*.
- Din toate posibilitățile rămase, alegem atîtea cuvinte cîte ne cer datele de intrare. Ordinea contează.

Să ne continuăm analiza exemplului.

Lungime 5. Din $3^5 = 243$ de posibilități, 73 sînt ilegale: cele 24 anterioare $\times$ 3, plus singurul prefix de lungime 5 (*aaaab*). Din cele $243 - 73 = 170$ legale trebuie să alegem două, iar ordinea contează, deci 170×169 de posibilități. Să le denumim *rrrrr* și *sssss*.

Lungime 6. Din $3^6 = 729$ de posibilități, 225 sînt ilegale:

- Cele 72 anterioare de lungime 5 (pentru completitudine, ele sînt *bb*???, *pp*???, *aab*??, *qqq*??).
- Cele de la intrare de lungime fix 5 (*aaaab*).
- Cele de lungime 5 generate (*rrrrr*, *sssss*).
- Toate urmate de orice caracter, deci $75 \times 3 = 225$.
- Nu există prefixe de lungime 6 între datele de intrare.

Rămîn $729 - 225 = 504$ moduri de a alege cuvîntul de lungime 6. Rezultatul este $7 \times 19 \times 170 \times 169 \times 504 \mod 1\,000\,000\,007 = 925\,829\,353$.

Detalii de implementare

Rămîne să răspundem eficient la trei clase de întrebări:

1. Ce șiruri de la intrare sînt prefixe ale altora? Restul formează setul redus.
2. Cîte șiruri de lungime exact l există în setul redus?
3. Cîte prefixe distincte de lungime l există în setul redus?

Editorialul recomandă un trie. Nu e rău, dar putem adapta quicksortul ternar pentru această problemă. Să considerăm iterația curentă, unde sortăm intervalul $[l, r)$ la poziția *pos*.

Prima adaptare: dacă oricare dintre șiruri, fie el $s[i]$ cu $i \in [l, r)$, se termină la poziția *pos*, atunci notăm faptul că există un șir de lungime *pos* (întrebarea 2 de mai sus). Mai mult, **nu mai reapelăm deloc** sortarea pentru subintervale ale lui $[l, r)$. Șirul $s[i]$ este prefix pentru toate celelalte, care deci nu fac parte din setul redus (întrebarea 1 de mai sus).

A doua adaptare: dacă niciun șir din $[l, r)$ nu se termină la poziția pos, atunci știm că după partiționare zona din mijloc (șirurile care pe poziția *pos* sînt egale cu pivotul) definește un prefix distinct de lungime $pos + 1$, la care nu vom ajunge la niciun alt moment pe parcursul sortării (întrebarea 3 de mai sus). Despre celelalte două zone încă nu știm nimic. Este posibil ca ele să fie vide.

Programul include și o optimizare de viteză. Cînd recursivitatea rămîne cu un singur șir (deci $r = l+1$), ea se va apela pînă la finalul șirului ca să constate că, la fiecare pas *pos*, există un prefix distinct de lungime *pos*. Această recursivitate poate avea adîncime 300 000 pe teste patologice, ceea ce o face lentă și consumatoare de memorie. Am înlocuit acea recursivitate cu o buclă `for`.

40.4.2 Problema Sabin (Baraj ONI 2015)

enunț • sursă

Soluția oficială recomandă folosirea unui trie sau calcularea unor hash-uri pe prefixele fiecărui string. Iată și o soluție care folosește doar sortarea.

Mai întîi, să simplificăm determinarea similitudinii între două seturi de k șiruri a cîte p caractere. Varianta naivă compară efectiv perechi de șiruri. Iată o altă variantă care are tot complexitate $\mathcal{O}(kp)$, dar reduce problema la una cunoscută: vom interclasa cele k șiruri. De exemplu, din (`abc`, `def`, `ghi`) obținem `adgbehcfi`. Cu această organizare, două colecții de cărți au similitudinea x dacă șirurile interclasate au lungimea prefixului comun de lungime $kx \leq l < k(x+1)$.

Atunci putem reformula problema astfel: pentru fiecare interogare dorim să aflăm numerele de șiruri cu care ea are prefixe comune de lungimi cel puțin kx, respectiv cel puțin $k(x+1)$. Răspunsul la interogare este diferența acestor două numere.

De aceea, vom genera explicit toate cele q interogări, concatenînd k șiruri din cele m. Apoi le vom sorta **împreună cu cele n șiruri originale**. Mărimea totală a datelor de sortat va fi $(q+n)kp$, adică 18 MB. Pentru sortarea ternară, acesta este un fleac. 💪 Iată șirurile sortate pentru exemplul din enunț.

șir	**tip**	**lungimi de prefix dorite**	**răspuns**
`atbrczds`	șir		
`atbrxxxx`	interogare	[2, 4)	0
`atfrffxs`	interogare	[2, 4)	1
`ftarsffs`	interogare	[2, 4)	1
`ftxxxxxx`	șir		
`gfeafsaf`	interogare	[6, 8)	2
`gfeafsax`	șir		
`gfeafsdf`	șir		

Să observăm că interogarea `gfeafsaf` are prefixe comune de lungime 7, respectiv 6 cu cele două șiruri care o urmează. În general, să considerăm momentul sortării cînd procesăm șirurile $[st \ldots dr)$, care sînt cunoscute ca fiind egale pe primele l poziții. Dintre aceste șiruri, unele sînt interogări, iar altele sînt șiruri originale. Pentru fiecare interogare ne întrebăm: nu cumva lungimea l este exact lungimea kx? Dacă da, atunci notăm pe interogare numărul de șiruri din $[st, dr)$. Similar și pentru $k(x+1)$.

Mai este o singură subtilitate care cere atenție. Este posibil ca o interogare să fie evaluată la aceeași lungime kx de mai multe ori, pînă cînd la un moment dat ea va ajunge, în urma partiționării, în zona de mijloc și va avansa la lungimea $kx+1$. Este important să nu-i atribuim un răspuns decît prima oară.

Capitolul 41

Șiruri de sufixe

41.1 Definiție

Definiția 27 (Șir de sufixe)**.** Șirul de sufixe al șirului S de lungime n este lista celor n sufixe ale lui S ordonată alfabetic.

În practică nu stocăm efectiv sufixele, deoarece am avea nevoie de $\mathcal{O}(n^2)$ memorie. În loc de asta, stocăm doar indicii de început ai sufixelor.

Exemplu. Pentru $S =$ `abracadabra`, șirul de sufixe A este:

poziție	valoare	sufix
0	10	`a`
1	7	`abra`
2	0	`abracadabra`
3	3	`acadabra`
4	5	`adabra`
5	8	`bra`
6	1	`bracadabra`
7	4	`cadabra`
8	6	`dabra`
9	9	`ra`
10	2	`racadabra`

Tabela 41.1: Șirul de sufixe al șirului `abracadabra`.

41.2 Aplicații

Ca să apreciem utilitatea șirurilor de sufixe, înainte de a discuta construcția, iată cîteva aplicații care decurg imediat din șirul de sufixe:

41.2.1 Aparițiile unui subșir P în S (*string matching*)

Facem două căutări binare în A pentru a găsi prima și ultima apariție a unui sufix care începe cu P. Putem face asta în mod naiv în $\mathcal{O}(|P| \log n)$, cu o căutare binară în care la fiecare înjumătățire în comparăm pe P cu candidatul curent. O soluție mai bună obține $\mathcal{O}(|P|+\log n)$ folosind vectorul LCP, discutat mai jos.

Putem generaliza căutarea la o familie de șiruri, $S_1, S_2, \ldots, S_k$. Construim șirul de sufixe A al concatenării $S_1 \# S_2 \# \ldots \# S_k$, unde $\#$ este orice caracter din afara alfabetului. Aparițiile lui P în orice șir vor fi consecutive în A.

41.2.2 Rotația minimă dpdv lexicografic

Exemplu. Pentru șirul `banana`, rotația minimă este `abanan`. Vezi problema Glass Beads.

Construim șirul de sufixe al șirului SS și luăm primul sufix din A care începe pe o poziție mai mică decît n. O variantă mai simplă este să construim șirul de sufixe **circulare** ale lui S, după cum vom învăța în algoritmii de construcție (toate cele n sufixe au lungime exact n).

41.3 Cele mai lungi prefixe comune

Definiția 28 (Vectorul LCP)**.** Vectorul LCP (engl. *longest common prefix*) reține lungimea prefixului comun între fiecare două sufixe consecutive din șirul de sufixe.

Pentru șirul $S =$ `abracadabra`, vectorul este

A	LCP	sufix
10		`a`
7	1	`abra`
0	4	`abracadabra`
3	1	`acadabra`
5	1	`adabra`
8	0	`bra`
1	3	`bracadabra`
4	0	`cadabra`
6	0	`dabra`
9	0	`ra`
2	2	`racadabra`

Tabela 41.2: Șirul LCP al șirului `abracadabra`.

Peste vectorul LCP putem construi orice structură de tip RMQ (tabelă rară, arbore de intervale etc.) pentru a putea afla lungimea prefixului comun între orice două sufixe.

41.4 Aplicații

Toate aceste aplicații adaugă relativ puțin cod peste construcția propriu-zisă a vectorilor A și LCP. Pentru că nu am dorit să poluez secțiunea de probleme cu prea mult cod redundant, le discutăm doar teoretic.

41.4.1 Cel mai lung subșir care apare de cel puțin două ori

Exemplu. Pentru șirul `hocuspocus`, cel mai lung subșir repetat este `ocus`. Vezi problema GATTACA.

Lungimea acestui subșir este pur și simplu maximul din vectorul LCP. Pozițiile de start ale subșirului sînt cele două poziții din A care generează acel maxim.

41.4.2 Cel mai lung subșir care apare de cel puțin k ori

Exemplu. Pentru șirul `scoobydoobydoo` și $k = 3$, răspunsul este `oo`. Vezi problema Substr.

Extindem raționamentul anterior. Ce înseamnă că un șir de lungime l apare de k ori în șir? Înseamnă că există $k - 1$ valori consecutive în vectorul LCP, toate mai mari sau egale cu l. Așadar, putem folosi tehnica minimului în fereastră glisantă pentru a căuta minimul în fiecare subsecvență de lungime $k - 1$ a vectorului LCP. Răspunsul este maximul acestor minime.

41.4.3 Cel mai scurt subșir care apare o singură dată

Exemplu. Pentru șirul `abababba`, cel mai scurt subșir unic este `bb`. Vezi problema Subunic.

Considerăm, pe rînd, fiecare poziție i din șirul de sufixe. Pentru a diferenția sufixul care începe la $A[i]$ de alte subșiruri, trebuie să depășim prefixele comune pe care el le are cu subșirurile de la pozițiile $A[i-1]$ și $A[i+1]$. Așadar, trebuie să luăm $1 + \max(LCP[i], LCP[i+1])$ caractere. Este necesar să ne asigurăm că acel număr de caractere este disponibil (că nu depășim sfîrșitul șirului S). Răspunsul este poziția i care minimizează cantitatea de mai sus.

41.4.4 Cel mai lung subșir comun între două șiruri S și T

Exemplu. Pentru $S =$ `abacaba` și $T =$ `bbacba`, cel mai lung subșir comun este bac. Vezi problema DNA Sequencing.

Construim șirul de sufixe și vectorul LCP ale șirului $S\#T$. Acum căutăm valoarea maximă în LCP între două sufixe (consecutive) dintre care unul provine din S și unul din T, adică valorile corespunzătoare din A sînt una mai mică decît $|S|$ și una mai mare decît $|S|$.

Putem generaliza acest algoritm și la aflarea celui mai lung subșir comun într-o familie de șiruri, $S_1, S_2, \ldots, S_k$. Vezi problema Life Forms.

41.4.5 Numărul de subșiruri distincte

Exemplu. Pentru $S =$ `ababa` există 9 subșiruri distincte: `a`, `ab`, `aba`, `abab`, `ababa`, `b`, `ba`, `bab`, `baba`. Vezi problemele Distinct Substrings și Ghicit.

Ne punem întrebarea cîte subșiruri distincte încep la o poziție dată. Dar iterăm prin poziții în ordinea dată de șirul de sufixe. Să presupunem că la poziția $A[i]$ avem $LCP[i] = l$. Atunci primele l subșiruri le-am contabilizat deja cînd am procesat poziția $A[i-1]$. Toate celelalte, pînă la sfîrșitul șirului, sînt subșiruri nou întîlnite. Așadar, răspunsul este

$$\frac{n(n+1)}{2} - \sum_i LCP[i]$$

41.4.6 Cel mai lung subșir palindrom

Exemplu. Pentru șirul `anaaremere`, cel mai lung subșir palindrom este `remer`.

Construim șirul de sufixe și vectorul LCP al șirului $S\#S^R$, unde am notat cu S^R șirul S răsturnat. Pentru exemplul dat obținem șirul `anaaremere#eremeraana`. Căutăm și returnăm valoarea maximă din LCP care ia naștere între două sufixe din S și respectiv din S^R.

Pentru exemplul dat, sufixele `remeraana` și `remere#eremeraana` vor fi consecutive în A și vor avea o valoare LCP de 5.

41.5 Construcția șirului de sufixe

Algoritmii sînt relativ clari în teorie, dar codul este surprinzător de complex.

Există și algoritmi de construcție în $\mathcal{O}(n)$ (vezi Wikipedia), dar nu sînt implementabili în timp de concurs.

41.5.1 Algoritmul în $\mathcal{O}(n^2 \log n)$

Cel mai simplu este să sortăm pur și simplu sufixele ca pe orice alte șiruri. Acest algoritm naiv funcționează în $\mathcal{O}(n^2 \log n)$, deoarece sortarea presupune $\mathcal{O}(n \log n)$ comparații, iar compararea șirurilor durează $\mathcal{O}(n)$. Sortarea cu quicksort ternar, discutată anterior, se comportă bine pe șiruri aleatorii, dar este foarte lentă pe cazuri degenerate (multe caractere egale), deoarece suma lungimilor sufixelor este $\mathcal{O}(n^2)$.

41.5.2 Algoritmul în $\mathcal{O}(n \log^2 n)$

Pentru acest algoritm și următorul, conceptul de bază este **dublarea prefixului**:

1. Sortăm caracterele individuale (cu sortare prin numărare).
2. Sortăm toate subșirurile de două caractere.
3. Sortăm toate subșirurile de patru caractere.

4. …

Infoarena menționează acest algoritm care face un efort de $\mathcal{O}(n \log n)$ pentru fiecare dublare a prefixului, așadar $\mathcal{O}(n \log^2)$ în total.

Ne propunem să calculăm **permutarea inversă** șirului de sufixe, așadar să aflăm pentru fiecare poziție i al câtelea sufix este $S[i \dots n-1]$ în lista ordonată. Pentru lungimea $l = 2^0 = 1$, să inițializăm un vector cu ordinea fiecărei litere în alfabet. Aceasta este, într-adevăr, o sortare corectă a subșirurilor de lungime 1.

k	$l = 2^k$	a	b	r	a	c	a	d	a	b	r	a	$
0	1	0	1	17	0	2	0	3	0	1	17	0	$-\infty$

Cum facem trecerea la $l = 2$? Observația-cheie este că un subșir de lungime 2^{k+1} este o pereche de subșiruri de lungime 2^k, iar toate subșirurile de lungime 2^k au fost deja ordonate la pasul anterior. Așadar, algoritmul va sorta vectorul de perechi $(0, 1)$, $(1, 17)$, $(17, 0)$ etc. Similar procedează și la pașii următori. Iată tabelul complet (sînt necesare trei iterații după cea inițială):

k	$l = 2^k$	a	b	r	a	c	a	d	a	b	r	a	$
0	1	0	1	17	0	2	0	3	0	1	17	0	$-\infty$
1	2	2	5	8	3	6	4	7	2	5	8	1	0
2	4	2	6	10	3	7	4	8	2	5	9	1	0
3	8	3	7	11	4	8	5	9	2	6	10	1	0

Tabela 41.3: Permutările obținute de algoritmul polilogaritmic. La final, ordinea lui `acadabra$` este 4, indicînd că sufixul este mai mare decît `$`, `a$`, `abra$` și `abracadabra$`.

Putem opri prematur algoritmul dacă el depistează n clase distincte (adică a terminat de diferențiat toate sufixele). La final, rămîne doar să inversăm permutarea obținută.

Algoritmul este mai simplu decît următorul și se comportă acceptabil în practică (pe calculatorul meu, 66 ms pentru $n = 200\,000$, 460 ms pentru $n = 1\,000\,000$).

Implementare

Acesta este un fragment din implementarea completă cu generare de șiruri și *benchmarking*. Codul primește un șir `s` de `n` caractere și construiește vectorul `sa[]`.

```
struct sort_suffix_array {
  const char* s;
  int n, len;

  int sa[N + 1];   // Șirul de sufixe. sa[i] = al i-lea sufix în ordine lexicografică
  int ord[N + 1];  // ord[i] = sa^{-1}. ord[i] = ordinea lui s[i...n-1], cu duplicate
  int cl[N + 1];   // cl[i] = clasa lui sa[i], cu duplicate
```

```
  void init(char* s, int n) {
    this->s = s;
    this->n = n;
  }

  bool cmp(int i, int j) {
    if (ord[i] != ord[j]) {
      return (ord[i] < ord[j]);
    }

    i += len;
    j += len;

    return (i < n && j < n)
      ? (ord[i] < ord[j])
      : (i > j);
  }

  void build() {
    for (int i = 0; i < n; i++) {
      sa[i] = i;
      ord[i] = s[i];
      cl[i] = 0;
    }

    len = 1;
    while (cl[n - 1] != n - 1) { // ne oprim când ajungem la n clase
      std::sort(sa, sa + n, [this](int i, int j) {
        return cmp(i, j);
      });
      for (int i = 1; i < n; i++) {
        cl[i] = cl[i - 1] + cmp(sa[i - 1], sa[i]);
      }
      for (int i = 1; i < n; i++) {
        ord[sa[i]] = cl[i];
      }
      len *= 2;
    }
  }

  void verify() {
    for (int i = 0; i < n - 1; i++) {
      assert(strcmp(s + sa[i], s + sa[i + 1]) <= 0);
    }
  }
};
```

41.5.3 Algoritmul în $\mathcal{O}(n \log n)$

Avînd în vedere că sortăm n perechi de valori cel mult n, putem sorta subșirurile folosind radix sort. Rezultatul este un algoritm mult mai rapid decît precedentul: 6 ms pentru $n = 200\,000$, 33 ms pentru $n = 1\,000\,000$. Dar implementarea este alunecoasă. Iar acest algoritm, dacă îl implementăm, trebuie să îl implementăm eficient. Altfel el se comportă comparabil sau chiar mai rău decît precedentul! De exemplu, implementarea de pe USACO rulează cam în 800 ms, dublu față de algoritmul în $\mathcal{O}(n \log^2 n)$.

Facem următoarele observații despre implementare:

1. Adăugăm la finalul șirului un caracter fictiv, notat cu `$`. În practică el este chiar terminatorul de șir, `\0`.
2. Algoritmul operează circular pentru a evita întrebarea „există sau nu poziția cu l mai la dreapta?”.
3. Algoritmul folosește 3 vectori. Vectorul p este întotdeauna o permutare și indică o ordine parțială a sufixelor, corectă pînă la lungimea pasului curent. La final, p va fi șirul de sufixe.
4. Vectorul c reține, pe fiecare poziție, clasa subșirului care începe la acea poziție. Un subșir mai mic va fi într-o clasă mai mică. Clasele sînt contigue și încep de la 0.
5. Vectorul *cnt* reține sume parțiale de frecvențe, întîi peste valorile ASCII, apoi peste clasele calculate în c la pasul anterior. În radix sort, *cnt* semnifică indicele ultimei poziții unde vom distribui elementele dintr-o clasă.

Pasul inițial

La primul pas (caractere invididuale), vom avea:

	`$`	`a`	`b`	`c`	`d`	`r`
frecvențe	1	5	2	1	1	2
cnt	1	6	8	9	10	12

Tabela 41.4: Vectorul *cnt* pentru caractere individuale. Sumele parțiale arată că în prima versiune a lui p indicii caracterelor `a` vor fi [1, 6), indicii caracterelor `b` vor fi [6, 8) etc.

Din *cnt* calculăm vectorul p (un fel de „versiunea 1 a șirului de sufixe”), care enumeră subșirurile de lungime 1 în ordine crescătoare.

	0	1	2	3	4	5	6	7	8	9	10	11
p	11	10	7	5	3	0	8	1	4	6	9	2

Tabela 41.5: Vectorul p pentru caractere individuale. El semnifică că primul prefix este $S[11] =$ `$`, al doilea este $S[10] =$ `a` etc.

În sfîrșit, din p calculăm vectorul c, clasele de echivalență ale subșirurilor de lungime 1:

	a	b	r	a	c	a	d	a	b	r	a	$
c	1	2	5	1	3	1	4	1	2	5	1	0

Codul care realizează aceste lucruri este mai scurt decît descrierea sa în cuvinte:

```
void build_single_letters() {
  memset(cnt, 0, sizeof(int) * 256);
  for (int i = 0; i < n; i++) {
    cnt[(int)s[i]]++;
  }
  for (int i = 1; i <= 'z'; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p[--cnt[(int)s[i]]] = i;
  }

  c[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
  }
  num_classes = 1 + c[p[n - 1]];
}
```

Dublarea prefixului

Să considerăm că avem vectorii p și c pentru o lungime l și dorim să-i actualizăm pentru lungimea $2l$. Îl vom modifica pe p doar pe subsecvențele unde valorile c corespunzătoare sînt egale. Vom rearanja, la nevoie, acele zone din p.

Reamintim imaginea de ansamblu. Vectorul p ne garantează că, pentru $i < j$, avem $s[p[i] \dots p[i]+l) \leq s[p[j] \dots p[j]+l)$. Acum dorim să facem un radix sort: La egalitate, dorim să resortăm doi indici i și j conform cu subșirurile $s[p[i]+l \dots p[i]+2l)$ și $s[p[j]+l \dots p[j]+2l)$.

Să considerăm cele cinci litere `a` din `abracadabra$`. Dorim să sortăm perechile `ab`, `ac`, `ad`, `ab`, `a$`. Cu alte cuvinte, dorim să re-emitem acei indici (10, 7, 5, 3, 0), în aceeași zonă din p (pozițiile [1, 6)), dar ținînd cont de caracterul care urmează după fiecare `a`. Aceasta este partea crucială și destul de dificilă a codului. Să urmărim întîi exemplul pas cu pas, după care codul va deveni abordabil.

Ce ar face radix sort? Ar sorta întîi perechile după membrul drept, apoi, stabil, după membrul stîng. Dar noi avem deja perechile sortate după membrul drept, de la pasul anterior! Trebuie doar să ne amintim să nu iterăm prin subșiruri în ordinea dată de p, ci într-o ordine decalată cu l poziții. Acesta este un cîștig major de eficiență, căci practic noi facem doar o singură trecere din cele două pe care le face teoretic radix sort.

De aceea, construim un vector auxiliar p_2 unde $p_2[i]$ stochează al i-lea cel mai mic sufix dacă ne uităm cu o poziție mai la dreapta (sau, la pasul l, cu l poziții circular la dreapta). Astfel, vectorul p_2 ne permite să iterăm în ordine lexicografică prin jumătățile drepte ale perechilor de ordonat.

	0	1	2	3	4	5	6	7	8	9	10	11
p pentru $l = 1$	11	10	7	5	3	0	8	1	4	6	9	2
p_2 pentru $l = 1$	10	9	6	4	2	11	7	0	3	5	8	1
p pentru $l = 2$	11	10	7	0	3	5	8	1	4	6	9	2

Observăm că definim întîi $p_2[i] = p[i] - 1$ (circular modulo n). Acum parcurgem vectorul p_2 **de la dreapta la stînga**. Procesăm întîi $p_2[11] = 1$, care indică că, dintre toate literele `b`, cea de la poziția 1 prefațează cel mai mare subșir de două litere (întrucît este urmată de un `r`, care știm că este cea mai mare literă). Știm din cnt că `b`-urile ocupă intervalul $[6, 8)$, deci notăm $p[7] = 1$.

Urmează $p_2[10] = 8$, care indică că `b`-ul de la poziția 8 este urmat de cea mai mare literă rămasă ($S[9] =$ `r`). Continuăm să ocupăm intervalul `b`-urilor: $p[6] = 8$.

Urmează $p_2[9] = 5$, care indică că `a`-ul de la poziția 5 este urmat de cea mai mare literă rămasă ($S[6] =$ `d`). Începem să emitem pozițiile `a`-urilor, care ocupă intervalul $[1, 6)$: $p[5] = 5$. Astfel parcurgem tot vectorul p_2 și îl completăm pe p.

Acum rămîne doar să recalculăm vectorul de clase, c. Acest pas este similar cu pasul inițial: parcurgem (noul) p în ordine și incrementăm numărul clasei ori de cîte ori întîlnim două perechi distincte. De exemplu, perechile (`a`, `b`) și (`a`, `c`) au clasele (în vechiul c) (1, 2) și respectiv (1, 3), deci le vom atribui clase noi diferite.

Deoarece nu-l putem modifica pe c în timp ce îl consultăm, vom crea un nou vector, c_2, pe care apoi îl copiem peste c. Obținem vectorul:

	a	b	r	a	c	a	d	a	b	r	a	$
c	2	5	8	3	6	4	7	2	5	8	1	0

Implementarea este:

```
void build_prefix_len(int len) {
  memset(cnt, 0, sizeof(int) * num_classes);
  for (int i = 0; i < n; i++) {
    cnt[c[i]]++;
  }
  for (int i = 1; i < num_classes; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p2[i] = circ(p[i] - len + n, n);
  }
  for (int i = n - 1; i >= 0; i--) {
```

```
    p[--cnt[c[p2[i]]]] = p2[i];
  }

  c2[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    bool diff =  (c[p[i]] != c[p[i - 1]]) ||
      (c[circ(p[i] + len, n)] != c[circ(p[i - 1] + len, n)]);
    c2[p[i]] = c2[p[i - 1]] + diff;
  }
  num_classes = 1 + c2[p[n - 1]];
  memcpy(c, c2, sizeof(int) * n);
}
```

În sfîrșit, *driver*-ul care invocă cele două metode de mai sus este elementar:

```
void build() {
  build_single_letters();

  int len = 1;
  while (num_classes < n) {
    build_prefix_len(len);
    len *= 2;
  }
}
```

41.6 Construcția vectorului LCP

O metodă brutală ar putea fi: cînd calculăm vectorul de sufixe, reținem și valorile parțiale pentru vectorul c într-o matrice cu $\log n$ linii. Atunci, pentru a afla cel mai lung prefix între `bra` și `bracadabra`, putem folosi căutarea binară. Observăm că la lungimea $l = 8$ cele două șiruri au clase diferite. La fel și la lungime 4. La lungime 2 au aceeași clasă, la fel și la lungime 3. Deci $LCP(\texttt{bra}, \texttt{bracadabra}) = 3$. Rezultă o complexitate totală de $\mathcal{O}(n \log n)$.

Dar iată o metodă mai scurtă și mai elegantă, în timp $\mathcal{O}(n)$ și fără memorie suplimentară. Ea parcurge sufixele într-o ordine foarte particulară pentru a putea refolosi informații.

Reamintim că întrebarea „care este cel mai lung prefix comun?” are sens și pentru două sufixe care nu sînt consecutive în șirul de sufixe, iar răspunsul este dat de un minim pe interval în vectorul LCP.

Acum, să presupunem că am calculat deja $LCP(\texttt{ra}, \texttt{racadabra}) = 2$. Atunci, dacă eliminăm caracterul inițial `r` din ambele sufixe obținem, în mod trivial, că `a` și `acadabra` au un prefix comun de lungime 1. Informația nu ne ajută direct, întrucît `a` și `acadabra` nu sînt consecutive în șirul de sufixe. Între ele mai există `abra` și `abracadabra`. Dar aflăm de aici ceva important: $LCP(\texttt{abracadabra}, \texttt{acadabra}) \geq 1$.

Formal, fie A șirul de sufixe (care este o permutare) și fie A^{-1} permutarea inversă ($A^{-1}[i]$ arată

al cîtelea sufix este $S[i...n-1]$ în ordine lexicografică). Atunci, pentru orice $i > 0$,

$$LCP[A^{-1}[i+1]] \geq LCP[A^{-1}[i]] - 1$$

41.6.1 Implementare

Programul consideră subșirurile în ordinea descrescătoare a lungimii, începînd cu întregul S.

```
void build_lcp() {
  // Inversează-l pe a în a1.
  for (int i = 0; i <= n; i++) {
    a1[a[i]] = i;
  }
  // Acum a1[i] ne spune al cîtelea sufix, în ordine lexicografică, este s[i...n].

  lcp[0] = 0; // Tehnic, nedefinit
  int l = 0;
  for (int i = 0; i <= n; i++) {
    int k = a1[i];
    if (k) {
      // Știm că LCP(a[k - 1], a[k]) >= l. Creștem naiv cît putem.
      int j = a[k - 1];
      while (s[i + l] == s[j + l]) {
        l++;
      }
      lcp[k] = l;

      // Pregătim l-ul pentru următorul k.
      if (l) {
        l--;
      }
    }
  }
}
```

41.6.2 Analiza complexității

Deoarece avem o buclă `while` într-o buclă `for`, la prima vedere algoritmul are complexitatea $\mathcal{O}(n^2)$. Cu toate acestea, să observăm că l este o lungime care nu poate depăși n (ea servește la compararea de indici din S). Mai mult, l scade cu cel mult o unitate la fiecare iterație a buclei `for`. Rezultă că suma creșterilor lui l din bucla `while` nu poate depăși n. Așadar, construcția șirului LCP durează $\mathcal{O}(n)$.

41.7 Probleme

41.7.1 Problema Carry Bit (IIOT 2023-2024 runda 3)

enunț • sursă

Observații preliminare

Să considerăm două numere binare de lungime l, $p[0 \dots l-1]$ și $q[0 \dots l-1]$ (unde $p[0]$ și $q[0]$ sînt biții cei mai semnificativi). Cînd există depășire la adunare?

- Dacă $p[0] = q[0] = 1$, atunci în mod evident există depășire.
- Dacă $p[0] = q[0] = 0$, atunci nu există depășire. Ambele numere sînt mai mici decît 2^{l-1}, deci suma lor este mai mică decît 2^l.
- Dacă $p[0] \neq q[0]$, atunci există depășire dacă și numai dacă există depășire la adunarea $p[1 \dots n-1] + q[1 \dots n-1]$.

Atunci, pentru problema dată, pentru interogarea $\langle x, y, l \rangle$ trebuie să aflăm prima poziție la care șirurile $a[x \dots]$ și $b[y \dots]$ coincid. Există depășire dacă și numai dacă (1) poziția este la distanță cel mult l de pozițiile x, respectiv y, și (2) valorile care coincid sînt egale cu 1.

În practică, este mai simplu să negăm unul dintre șiruri, să spunem b, deoarece problema găsirii primei **diferențe** între două subșiruri este mai abordabilă. Notăm acest șir cu $\bar{b}$.

Soluția cu hashing

Menționez că există și o soluție mai simplă, bazată pe hashing (exemplu). Interpretăm șirurile ca pe numere într-o bază arbitrară, dar fixă. Nu baza 2, deoarece este foarte probabil că există teste contra ei. Pentru fiecare poziție i calculăm valoarea hash a prefixului $a[0 \dots i]$. Atunci prin diferență de prefixe putem afla valoarea hash a oricărui subșir al lui a, și similar pentru $\bar{b}$.

Dată fiind o interogare, căutăm binar prima lungime w la care valorile hash pentru $a[x \dots x+w-1]$ și $\bar{b}[y \dots y+w-1]$ diferă. Apoi considerăm simplu cazurile descrise mai sus.

Soluția cu șiruri de sufixe

Dacă concatenăm cele două șiruri (separate de un delimitator ca `\0`), atunci putem reformula interogările astfel: date fiind două poziții x și y, care este lungimea maximă a două subșiruri egale care încep de la acele poziții? Această problemă este aparent clasică și se numește *longest common extension* (LCE).

Dacă nu ne vine ideea cu hashing, nu strică să avem și arma universală a șirurilor de sufixe. Construim cele trei informații necesare:

1. Șirul de sufixe.
2. Vectorul LCP.
3. O structură pentru RMQ. Eu am folosit un AINT.

Acum, date fiind pozițiile de start x și y, aflăm cu RMQ lungimea w a celui mai lung prefix comun al sufixelor care încep la pozițiile x și y. Iau naștere aceleași trei cazuri ca mai sus:

1. Dacă $w \geq l$, atunci a și $\bar{b}$ coincid pe toată lungimea l. Echivalent, a și b diferă pe toată lungimea l, deci suma lor este exact 2^{l-1} și nu există depășire.
2. Altfel, dacă $a[x+w] = b[y+w] = 0$, atunci nu există depășire.
3. Altfel $a[x+w] = b[y+w] = 1$ și există depășire.

41.7.2 Problema Periodic Words (IIOT 2023-2024 runda 1)

enunț • surse

Problema pornește de la următoarea observație. Pentru o lungime de perioadă aleasă, putem reduce testul de periodicitate la o singură comparație de sufixe. Sper că v-ați amintit acest truc de la problema Bart.

De exemplu, pentru a afla dacă $S[20 \ldots 79]$, de lungime 60, are perioadă 5, vrem să aflăm dacă subșirurile $S[20 \ldots 24], S[25 \ldots 29], \ldots S[74 \ldots 79]$ sînt egale. Dar este suficient să testăm dacă $S[20 \ldots 74] = S[25 \ldots 79]$.

Astfel, ambele soluții de mai jos calculează, pentru fiecare interogare $[l, r]$ și fiecare divizor d al lui $r - l + 1$, dacă $S[l \ldots r-d] = S[l+d \ldots r]$. Ca optimizare, este suficient să testăm doar perioadele care corespund unui număr de repetiții prim. De exemplu, dacă un subșir de lungime 60 poate fi scris ca 20 de repetiții cu perioadă 3, el poate fi scris și ca 2 repetiții cu perioadă 30 sau 5 repetiții cu perioadă 12.

Soluția cu hashing

O primă soluție calculează hash-urile prefixelor într-o bază B și modulo o valoare M. Din acestea putem calcula hash-ul oricărui subșir.

Întrucît există circa $5 \cdot 10^9$ subșiruri, paradoxul zilei de naștere ne spune că trebuie să operăm modulo o valoare cel puțin $25 \cdot 10^{18}$ și preferabil chiar mai mare. Deci, la prima vedere, trebuie să alegem două module pentru a elimina coliziunile. De fapt, situația este mai simplă. Noi nu comparăm decît subșiruri **de aceeași lungime**, deci avem cel mult 10^6 subșiruri în aceeași clasă. Coliziunile între subșiruri de lungimi diferite nu ne deranjează. Dacă operăm modulo $2^{64} \approx 1,8 \cdot 10^{19}$, probabilitatea unei coliziuni este infimă.

Soluția cu șiruri de sufixe

Calculăm șirul de sufixe al lui S, precum și vectorul LCP și informații de RMQ peste acesta. Acum, date fiind interogarea $[l, r]$ și divizorul d, verificăm dacă sufixele care încep la pozițiile l și $l + d$ au un prefix comun de lungime cel puțin $r - l + 1 - d$.

41.7.3 Problema Phone Book (Finala IIOT 2022-2023)

enunț • sursă

Această problemă este relativ „servită” pentru șiruri de sufixe.

Cînd luăm în calcul un subșir al cuvîntului i și ne întrebăm dacă este specific acelui cuvînt, trebuie să-l putem căuta rapid în toate celelalte cuvinte. De aceea, să construim șirul de sufixe al concatenării tuturor cuvintelor: $S = s_1\texttt{\textbackslash 0}\ s_2\texttt{\textbackslash 0}\ \ldots\texttt{\textbackslash 0}\ s_n\ \texttt{\textbackslash 0}$.

Acum, să considerăm o poziție de start în interiorul cuvîntului i. Ne întrebăm: ca să obținem un subșir specific cuvîntului i, cîte caractere ar trebui să selectăm? Să spunem că el are poziția k în vectorul de sufixe. **În principiu**, vrem să diferențiem subșirul de precedentul și de următorul din vectorul de sufixe. Deci trebuie să selectăm $1 + \max(LCP[k], LCP[k+1])$ caractere.

Situația se complică un pic deoarece nu ne deranjează dacă acel sufix mai apare altundeva **în cuvîntul** i. Deci trebuie să căutăm înainte și înapoi prin vectorul LCP și să găsim cele mai apropiate sufixe care nu provin din cuvîntul i. Nu putem face naiv (liniar) această căutare, deoarece ea duce la timp pătratic dacă există puține cuvinte lungi. Dar putem procesa grupe de sufixe consecutive care provin din același cuvînt.

Concret, pentru fiecare poziție din șirul de sufixe mai reținem două informații: (1) din ce cuvînt provine acel sufix și (2) care este lungimea sufixului (pînă la următorul delimitator `\0`). Construim vectorul LCP și informații de RMQ peste LCP.

Acum căutăm grupe maximale de sufixe consecutive (lexicografic) care provin din același cuvînt. Dacă sufixele de la poziția a la poziția b toate provin din același cuvînt i, atunci pentru toate acestea sufixele relevante, precedent și următor, se află la pozițiile $a-1$ și $b+1$. De aceea, pentru fiecare poziție $a \leq k \leq b$, procedăm astfel:

1. Calculăm $r_1 = RMQ(a-1, k)$.
2. Calculăm $r_2 = RMQ(k, b+1)$.
3. Calculăm $r = 1 + \max(r_1, r_2)$. Acesta este numărul de caractere necesare pentru a diferenția sufixul al k-lea, care provine din cuvîntul i, de orice alte subșiruri din orice alte cuvinte în afară de i.
4. Dacă sufixul curent are cel puțin r caractere (pînă la sfîrșitul cuvîntului), atunci minimizăm răspunsul pentru cuvîntul i cu valoarea r.
5. Altfel, sufixul curent nu are destule caractere pentru a-l putea diferenția de celelalte cuvinte.

Optimizare

Putem omite cu totul structura de date pentru RMQ. Să observăm, de fapt, că toate interogările de minim sînt grupate. Pentru fiecare poziție $a \leq k \leq b$ ne interesează minimul pe intervalele $[a-1, k]$ și $[k, b+1]$. Putem calcula aceste minime prin două treceri, una înapoi și una înainte, prin intervalul $[a, b]$.

Capitolul 42

Arbori trie

Arborii trie[1] sînt structuri arborescente care stochează colecții de șiruri de caractere. După cum vom vedea în probleme, trie-urile sînt folosite adesea și pentru stocarea numerelor reprezentate (conceptual) ca șiruri de biți.

Secțiunea de teorie a acestui capitol este momentan incompletă. Vom urmări imaginile din articolul de pe Wikipedia.

42.1 Implementare

O implementare elementară este:

```
struct trie {
  struct node {
    int child[SIGMA];
    int cnt;
  };

  node nd[MAX_CHARS + 1];
  int size = 1;

  void insert(char* s) {
    int u = 1;
    for (int i = 0; s[i]; i++) {
      int c = s[i] - 'a';
      if (!nd[u].child[c]) {
        nd[u].child[c] = ++size;
      }
      u = nd[u].child[c];
      nd[u].cnt++;
    }
  }
```

[1] Pronunția acestui cuvînt este neclară, conform Wikipedia. Eu sînt adeptul pronunțării ca *try*, pentru a-l deosebi de *tree*.

```
};
```

Deciziile mele de design sînt:

1. Fiecare nod are un vector static cu 26 de pointeri la fii, cîte unul pentru fiecare literă a alfabetului. Această alocare este comodă, dar risipitoare. În problema Cipher am implementat și versiunea cu liste de fii de lungime variabilă.
2. Am prealocat static nodurile. În cazul cel mai defavorabil, numărul de noduri poate fi egal cu suma lungimilor șirurilor. Cînd inserăm un șir și avem nevoie să creăm un fiu nou, folosim următorul nod din vectorul prealocat.
3. Fiecare nod reține și numărul de șiruri care se continuă în subarbore. Această informație variază de la problemă la problemă.
4. Rădăcina arborelui este nodul 1, iar nodul 0 este nefolosit. Astfel, oricînd o poziție din vectorul `child` este 0, ea indică un pointer nul.

42.2 Probleme

42.2.1 Problema Enigma (Lot 2011)

enunț • sursă

Problema amintește de probleme clasice de descompunere în cuvinte: dat fiind un șir S și un dicționar D, să se numere descompunerile lui S în cuvinte din D (sau să se găsească o astfel de descompunere). Acest tip de probleme se pot rezolva cu programare dinamică înainte.

La poziția i, fie $w[i]$ numărul de moduri de a descompune prefixul $S[0...i]$. Acum, pentru fiecare cuvînt $C \in D$, dacă C apare în S începînd de la poziția $i+1$, atunci obținem $w[i]$ moduri de a descompune primele $i+|C|$ caractere, deci adăugăm $w[i]$ la $w[i+|C|]$.

Problema Enigma aduce variația că putem folosi orice prefix al cuvintelor din dicționar. Să remarcăm și că produsul $MS \leq 450\,000$, acceptabil. Deci putem refolosi aceeași metodă ca mai sus, doar că nu adăugăm $w[i]$ la o singură poziție viitoare, ci la **toate** pozițiile $i+1, i+2, \ldots, j$ cît timp subșirul $S[i \ldots j]$ se potrivește cu prefixul cel puțin unui cuvînt dintre cele M.

Trie-urile se potrivesc ca o mănușă cu această nevoie. Inserăm cele M cuvinte într-un trie. Apoi, pentru fiecare poziție i din S, pornind din rădăcina trie-ului, coborîm cît timp putem continua șirul S. Dacă la poziția $j > i$ există cnt continuări are lui $S[i+1 \ldots j]$, atunci la $w[j]$ adăugăm $w[i] \cdot cnt$ deoarece fiecare descompunere pînă la poziția i și fiecare șir dintre cele cnt formează o combinație unică.

Algoritmul 10 Algoritmul în ansamblu pentru problema Enigma

```
inserează cele M cuvinte într-un trie
pentru i = 0, 1, ..., |S| − 1 execută
    j ← i + 1
    u ← rădăcina trie-ului
    cît timp nodul u are cnt > 0 șiruri în fiul de pe litera S[j] execută
        w[j] ← w[j] + w[i] · cnt
        u ← fiul de pe litera S[j]
        j ← j + 1
```

42.2.2 Problema Maximum Xor Subarray (CSES)

enunț • sursă

Iată și o problemă care folosește un trie pentru stocarea unei colecții de numere pe k biți. Fiecare număr $x = \overline{b_{k-1}b_{k-2}\dots b_1 b_{0(2)}}$ corespunde unei căi de k noduri. Fiecare nod din trie are exact doi fii, ceea ce elimină risipa întîlnită în cazul literelor. Din păcate, majoritatea acestor probleme sînt probleme cu xor-uri (sau exclusiv), care mie mi se par artificiale. Să rezolvăm totuși două astfel de probleme, pentru cultura generală.

Problema Maximum Xor Subarray folosește un artificiu arhicunoscut: xor-ul unei secvențe $[l, r]$ este egal cu xor-ul prefixelor $[0, l-1]$ și $[0, r]$. De aceea, în loc să examinăm elementele șirului, vom examina xor-urile parțiale. Pentru fiecare valoare x, ne întrebăm: cu care altă valoare văzută anterior face x xor-ul maxim?

Vom construi răspunsul bit cu bit. Dacă există valori anterioare care încep cu $\neg\overline{b_{k-1}}$, atunci restrîngem căutarea la mulțimea acestor valori, iar răspunsul va începe cu bitul 1. Altfel răspunsul începe cu bitul 0. Similar calculăm și ceilalți biți ai răspunsului.

Din nou, trie-urile sînt structura potrivită deoarece, urmînd căi de la rădăină la frunze, putem să aflăm incremental informații despre prefixele binare ale valorilor văzute anterior.

42.2.3 Problema Esoteric Bovines (IIOT 2023-2024 runda internațională)

enunț • sursă

Problema este o generalizare a precedentei. Dați fiind doi vectori A și B cu cîte n elemente fiecare, trebuie să găsim a k-a cea mai mică valoare xor între un element din A și unul din B.

Soluție cu căutare binară

Să inserăm toate numerele din A într-un trie augmentat, în fiecare nod, cu numărul de numere care se continuă în subarborele nodului. Atunci pentru fiecare număr $b \in B$ putem să aflăm eficient diverse informații despre xor-urile pe care b le formează cu numere din trie. În particular,

putem să răspundem la întrebarea: cîte dintre cele n xor-uri sînt mai mici decît o valoare aleasă, v?

De aici deducem următorul algoritm bazat pe căutarea binară a răspunsului. Pentru un candidat x, ne întrebăm: cîte perechi $(A[i], B[j])$ au proprietatea $A[i] \oplus B[j] < x$? Restrîngem căutarea binară în stînga sau în dreapta lui x după cum numărul de perechi este mai mare sau mai mic decît k.

Complexitatea acestui algoritm este $\mathcal{O}(nl^2)$ unde $l \leq 60$ este mărimea valorilor. Această complexitate provine din

1. Căutarea binară a răspunsului în spațiul de valori $[0, 2^{60})$.
2. Iterarea prin vectorul B.
3. Pentru fiecare element $b \in B$, coborîrea prin trie.

Implementarea obține doar 40 de puncte. Este nevoie să reducem complexitatea.

Soluție cu parcurgere în lățime

Cel mai bun candidat pe care să-l eliminăm din complexitate este căutarea binară. Putem să aflăm răspunsul într-o singură trecere prin trie?

Haideți să începem cu pași mici: să aflăm primul bit al răspunsului. Cîte perechi $(A[i], B[j])$ încep cu 0? Să introducem notația $C(A, l-1, 0)$ pentru numărul de valori din A care au bitul $l-1$ egal cu 0. Atunci numărul de perechi este:

$$p = C(A, l-1, 0) \cdot C(B, l-1, 0) + C(A, l-1, 1) \cdot C(B, l-1, 1)$$

Desigur, primul bit al răspunsului va fi 0 sau 1 după cum $p \geq k$ sau $p < k$. Dar întrebarea este cum continuăm de aici. Să spunem că primul bit este 0. Practic, trebuie să partiționăm mulțimea A în numere care încep cu 0 sau cu 1. Fie aceste mulțimi A_0 și A_1. Similar o partiționăm pe B în B_0 și B_1. Ca să aflăm al doilea bit al răspunsului, trebuie să confruntăm, în paralel A_0 cu B_0 și A_1 cu B_1, ca să vedem dacă avem mai multe sau mai puține de k perechi care au al doilea bit 0.

Pare că menținerea aceste evidențe se fărîmițează într-o colecție netractabilă de submulțimi. Dar de fapt situația este sub control! Ca și în soluția anterioară, să inserăm toate valorile din A într-un trie. Acum, să adăugăm toate valorile din B în rădăcina acestui trie. Pe măsură ce ținem evidența perechilor cu xor-ul dorit, vom împinge valorile din B în jos, către frunzele trie-ului. Vom face aceasta în l iterații, cîte una pentru fiecare nivel al arborelui.

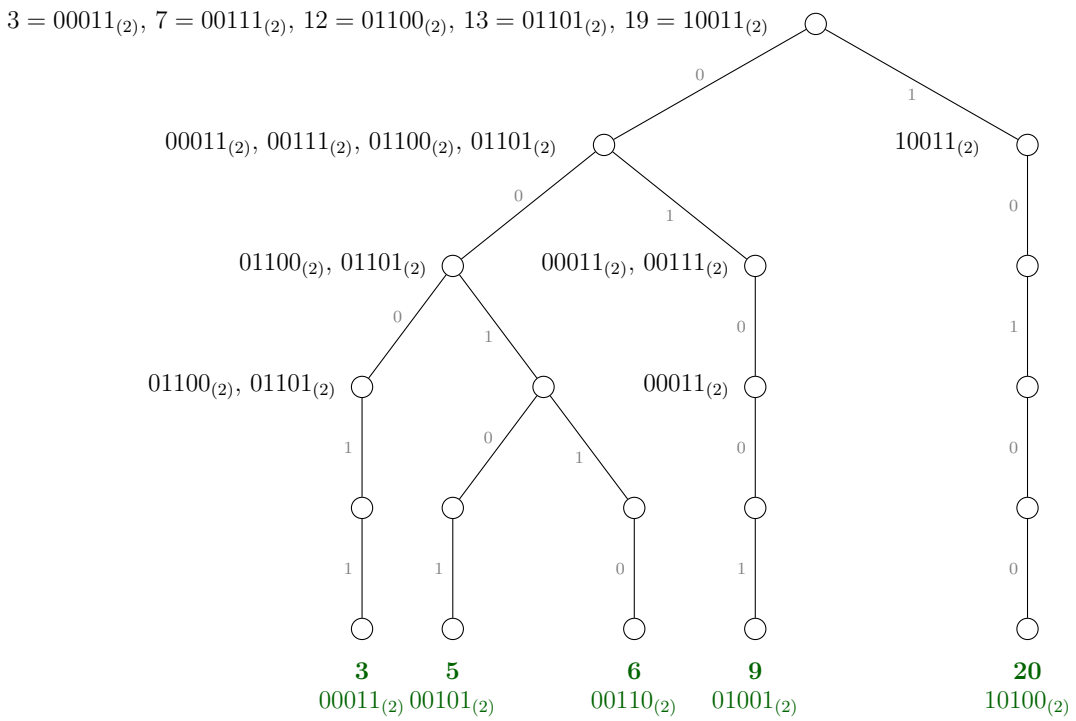


Figura 42.1: Trie-ul construit pentru exemplul 2 din enunț, în care dorim să obținem al $k = 10$-lea cel mai mic xor. În rădăcină obținem $4 \cdot 4 + 1 \cdot 1 = 17$ perechi cu xor-ul 0, deci primul bit al răspunsului este 0. Pe nivelul 1 (corespunzător celui de-al doilea bit din stînga) obținem $2 \cdot 3 + 2 \cdot 1 + 1 \cdot 1 = 9$ perechi cu xor-ul 0, deci al doilea bit al rezultatului este 1.

Pe primul nivel (rădăcina), vom număra perechile al căror xor începe cu 0, ca mai sus. Dacă aflăm că primul bit al răspunsului este 0, vom împinge valorile din B_0 în subarborele stîng, iar pe cele din B_1 în subarborele drept, iar dacă primul bit al răspunsului este 1 vom proceda invers.

Acum să considerăm un nivel oarecare, pe care dorim să aflăm bitul i al răspunsului. Cele n valori din B sînt distribuite între nodurile de pe nivelul i din trie. Pentru fiecare valoare $b \in B$, aflată într-un nod u, vom interoga fiii lui u pentru afla cu cîte valori din A obține b xor-ul 0, respectiv 1, pe poziția i. După ce însumăm răspunsurile pentru toate valorile din B, vom ști dacă dorim ca bitul i al răspunsului să fie 0 sau 1. În funcție de aceasta, facem o a doua trecere prin valorile din B ca să le mutăm mai jos cu un nivel.

Această soluție are complexitatea $\mathcal{O}(nl)$ și obține punctaj maxim, dar consumă peste 100 MB de memorie. Există $200\,000 \cdot 60 = 12\,000\,000$ de noduri și fiecare nod reține trei întregi: numărul de șiruri din acel nod și pointeri la fii.

Soluție optimizată

Din această soluție am învățat un algoritm care reduce enorm memoria, ceea ce înjumătățește și timpul de rulare. Să observăm că noi am construit oarecum „pe pilot automat”, din reflex, acest trie, dar el nu este necesar. Putem să pornim cu un trie format doar din rădăcină, în care se află toate cele $2n$ valori din A și B. Apoi împingem valorile în jos, spre noduri pe care le creăm

spontan, la nevoie.

În esență, soluția anterioară împingea în mod repetat fiecare valoare $b \in B$ într-un nod u, cu semnificația: pentru a obține prefixul dorit din răspuns, valoarea b mai poate forma perechi doar cu numerele din subarborele lui u. Să remarcăm că toate numerele din acel subarbore au un prefix comun pînă la nivelul curent.

Aceasta înseamnă că, dacă sortăm imediat după citire valorile din A, atunci fiecare valoare din B reține în permanență un interval continuu din A. Mai mult, toate valorile din B care au un prefix comun vor reține același interval din A. Cu alte cuvinte, la nivelul curent trebuie doar să reținem **o listă de perechi de intervale** de forma $\big([l_A, r_A), [l_B, r_B)\big)$. Xor-ul oricărei perechi $(A[i], B[j])$ este egal pe prefix cu răspunsul.

Inițial în rădăcină urmărim toate cele n^2 perechi, deci lista de perechi de intervale conține doar perechea $\big([0, n), [0, n)\big)$. Acum fie x poziția primului număr din A care are primul bit 1 și fie y poziția primului număr din B care are primul bit 1. Ca să aflăm cîte perechi dau un xor cu primul bit 0, folosim formula:

$$p = x \cdot y + (n - x) \cdot (n - y)$$

De aici iau naștere două cazuri:

1. Dacă $p \geq k$, atunci perechile pentru nivelul următor sînt $\big([0, x), [0, y)\big)$ și $\big([x, n), [y, n)\big)$.
2. Dacă $p < k$, atunci perechile pentru nivelul următor sînt $\big([0, x), [y, n)\big)$ și $\big([x, n), [0, y)\big)$.

Pe orice nivel vom avea cel mult n astfel de perechi, deoarece fiecare element din A (și din B) face parte din cel mult o pereche (dacă la orice moment un element din A rămîne fără perechi în B, sau invers, atunci invervalul corespunzător devine vid și putem să aruncăm perechea de intervale din listă).

De aceea, complexitatea este tot $\mathcal{O}(nl)$, dar memoria este $\mathcal{O}(n)$ în loc de $\mathcal{O}(nl)$ cît ocupă un trie. În practică, programul folosește doar 7 MB.

42.2.4 Problema Cipher (IIOT 2022-2023 runda internațională)

enunț • sursă

Să ignorăm pentru moment cerința ca $|s|$ să se dividă cu $|t|$. Atunci putem afla criptarea minimă foarte direct, folosind un trie, astfel. Să luăm ca exemplu $s[0] =$ `f`. Cea mai bună pereche pentru `f` este `v`, deoarece numerele de ordine ale acestor litere sînt 5 și 21, de sumă 26, deci suma lor Vigenère este `a`. Așadar, dacă există chei care încep cu litera `v`, restrîngem căutarea la acelea și continuăm cu cea de-a doua literă a lui s. Dacă nu, atunci examinăm literele `w`, `x`, `y`, `z`, `a`, `b`, …, pînă cînd găsim chei care încep cu una dintre aceste litere.

De aceea, algoritmul pentru acest caz particular inserează toate cheile într-un trie. Apoi, pentru fiecare șir s, pornește din rădăcina trie-ului și coboară nivel cu nivel, urmînd la nivelul i litera

care minimizează perechea cu $s[i]$.

Încercare cu un singur trie

Cum tratăm restricția de divizibilitate? Prima mea încercare, care nu duce nicăieri, a fost să țin în fiecare nod din trie un vector cu lungimile posibile ale cheilor care se continuă în acel subarbore. Suma lungimilor acestor vectori este egală cu suma lungimilor cheilor, deci acceptabilă. Pentru a procesa un șir s, este necesar să adăugăm următoarea regulă la coborîrea prin trie. La fiecare nivel i, vom urma litera care formează perechea minimă cu $s[i]$ și care duce spre un nod al cărui vector include un divizor al lui $|s|$.

Această soluție este ineficientă și greoi de codat. Pe fiecare nivel al trie-ului putem avea $\mathcal{O}(\sqrt{k})$ chei de lungimi diferite, ceea ce ne duce la complexitatea $\mathcal{O}(S\sqrt{k})$, unde $S \leq 400\,000$ este suma lungimilor șirurilor.

Soluția cu trie-uri multiple

În loc de aceasta, putem să generăm cîte un trie pentru fiecare lungime distinctă a cheilor. Pentru exemplul din enunț, vom crea 4 trie-uri pentru lungimile de șiruri 3, 4, 6 și 8. Atunci, pentru o interogare s, vom face cîte o coborîre în trie pentru fiecare divizor d al lui $|s|$ (desigur, numai dacă chiar există chei de lungimea d).

În teorie, aceasta poate duce la multe coborîri atunci cînd lungimile șirurilor sînt numere extrem compuse, dar să nu uităm că suma acestor lungimi nu depășește 400 000. De exemplu, putem avea un șir de lungime $|s| = 166\,320 = 2^4 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11$, care are 160 de divizori. Dar nu putem avea mai mult de două astfel de șiruri.

Există multe detalii de implementare care merită discutate.

Pot exista circa $\sqrt{2 \cdot 400\,000} < 1\,000$ lungimi de chei distincte, deci poate fi nevoie să ținem 1 000 de trie-uri cu cel mult 400 000 de noduri în total. Cum procedăm? Putem recurge la alocarea dinamică a nodurilor, dar eu am preferat, din nou, să aloc un vector global de noduri din care fiecare trie ia un nod cînd are nevoie.

Pentru maparea efectivă a lungimilor cheilor la trie-uri, am folosit pur și simplu un `unordered_map`.

Ca să fac un experiment, la această implementare nu am mai prealocat vectori de 26 de pointeri la fii, ci i-am creat doar la nevoie. Fiecare pointer este o pereche constînd din nodul-destinație și din litera de pe muchie.

Pentru un șir s dat, este nevoie să construim efectiv criptările pentru fiecare divizor al lui $|s|$, ca să le putem compara și să alegem cheia optimă. Eu am ales să fac acest lucru chiar pe parcursul coborîrii în trie. De exemplu, dacă $|s| = 15$ și examinăm trie-ul cheilor de lungime 3, atunci:

- Litera aleasă la nivelul 0 trebuie adunată la $s[0]$, $s[3]$, $s[6]$, $s[9]$ și $s[12]$.
- Litera aleasă la nivelul 1 trebuie adunată la $s[1]$, $s[4]$, $s[7]$, $s[10]$ și $s[13]$.
- Litera aleasă la nivelul 2 trebuie adunată la $s[2]$, $s[5]$, $s[8]$, $s[11]$ și $s[14]$.

Partea XI

Probleme diverse

Capitolul 43

Probleme diverse

Acest capitol include probleme care fie necesită doar cunoștințe de bază (engl. *core*, numite în supa culturală și probleme ad-hoc), fie necesită cunoștințe specifice, dar pentru care cursul încă nu are un capitol dedicat.

43.1 Problema Liars (Baraj ONI 2025)

enunț • sursă

43.1.1 Generalități

Problema este una tipică de recurență pe arbore[1], dar acest curs nu are un capitol dedicat subiectului. Poate într-o zi cu soare. Problema necesită multă atenție la implementare.

Să începem cu numărarea configurațiilor posibile. În problemele de recurență pe arbore, de obicei judecăm recursiv. Informația pe care o dorim să o calculăm într-un nod u decurge din combinarea informațiilor similare calculate în fiecare dintre fiii lui u. În plus, **încercăm să decuplăm subarborele lui u de restul arborelui**. Pentru aceasta, luăm în calcul toate posibilitățile pentru u sau (în alte probleme) pentru părintele lui u. Nu știm care dintre posibilități ne va trebui în final, dar astfel ne asigurăm că problema nu „se revarsă" în restul arborelui.

43.1.2 Numărarea configurațiilor

În cazul de față, vom considera pe rînd că u este sincer, apoi mincinos. Să considerăm cazul în care u este sincer și fie r numărul de vecini mincinoși ai lui u (citit de la intrare). **În principiu** dorim să calculăm recursiv, pentru fiecare fiu v al lui u, numărul de configurații valide pentru

[1]Simt nevoia unei digresiuni despre termenul *programare dinamică*. Olimpicii tind să denumească orice recurență programare dinamică (și, pentru maximum de efect, să-și denumească variabilele aferente `dp`). Dar această denumire este incorectă în cazul arborilor. Pentru a putea încadra o soluție ca programare dinamică, ea are nevoie de două trăsături: substructura optimă și suprapunerea subproblemelor. În cazul recurențelor pe arbore, prin definiție **nu** există subprobleme suprapuse, deoarece în acest caz subproblemele unui nod sînt fiii nodului, deci prin definiție sînt disjuncte.

v și subarborele său, apoi să contorizăm numărul de moduri în care putem alege r fii mincinoși pentru u. Doar că ne mai lipsește o informație: valoarea de adevăr a părintelui lui u, fie el p. Dacă p este mincinos, atunci lui u îi mai trebuie doar $r-1$ fii mincinoși.

Astfel ajungem la mulțimea finală de informații necesare în fiecare nod u, respectiv numărul de configurații valide pentru subarborele lui u în patru cazuri:

1. u este sincer, iar p este sincer;
2. u este sincer, iar p este mincinos;
3. u este mincinos, iar p este sincer;
4. u este mincinos, iar p este mincinos.

Să considerăm acum că avem aceste 4 cantități calculate în toți fiii lui u. Dacă u este sincer, atunci, după cum am spus, ne interesează în cîte moduri putem asigna r sau $r-1$ fii mincinoși (în funcție de valoarea lui p. Dacă u este mincinos, atunci dimpotrivă ne interesează să asignăm orice număr de fii mincinoși în afară de r, respectiv $r-1$. Această din urmă cantitate este mai simplu de calculat dacă aflăm numărul total de configurații în subarborele lui u (indiferent de numărul de fii mincinoși) din care scădem cantitatea care l-ar face pe u să fie sincer.

Cum combinăm informațiile din fii? Limitele problemei ne permit complexitatea $\mathcal{O}(d_u^2)$ în fiecare nod, unde cu d_u am notat numărul de fii ai lui u. Aceasta ne permite o logică rezonabil de simplă (la acest nivel). Pentru nodul curent u, și pentru o valoare fixată pentru u, să calculăm numărul de configurații în care printre fiii lui u există $0, 1, 2, d_u$ mincinoși.

Aceasta este o subproblemă de programare dinamică (reală, în acest caz). Fie $C(i, j)$ numărul de moduri de a obține j mincinoși folosind primii i fii. Inițial, $C(0, 0) = 1$, căci cu primii 0 fii putem obține (doar) 0 mincinoși într-un singur mod. Apoi, $C(i, j)$ provine

- din $C(i-1, j)$ dacă asignăm al i-lea fiu ca fiind sincer;
- din $C(i-1, j-1)$ dacă asignăm al j-lea fiu ca fiind mincinos.

43.1.3 Găsirea unei configurații

Odată numărate configurațiile în această manieră *bottom up*, putem găsi una dintre ele într-o manieră *top down*. Ce valoare să punem în rădăcină (nodul 1)? Prin convenție, să asignăm rădăcina ca sinceră dacă ea raportează cel puțin o configurație validă în care ea este sinceră, altfel o asignăm ca mincinoasă. Din nou, efectul acestei asignări este că decuplăm subarborii rădăcinii.

O dată fixată valoarea unui nod u, îi putem asigna și fiii. Acest proces este elementar, dar necesită un pic de cod, în două faze.

În primă fază, asignăm toți fiii lui u care sînt impuși. Dacă un fiu v nu are nicio asignare în care el să fie sincer, iar părintele său u să aibă valoarea fixată, atunci obligatoriu v trebuie să fie mincinos. Similar decidem dacă v trebuie să fie sincer. Dintre restul fiilor, care pot lua orice valoare, știm cîți trebuie să fie sinceri și cîți mincinoși, în funcție de numărul de vecini mincinoși declarați de u și de valoarea acestuia de adevăr. Asignăm și restul de fii ai lui u respectînd aceste

cantități. Procedăm astfel pînă în frunze.

43.2 Problema Pavare infinită (Lot 2025)

enunț • sursă • coloană sonoră pentru codare

Editorialul prezintă o soluție tehnică:

- Încercăm toate modurile de a alipi două piese.
- Pentru o alipire dată, a treia, a patra piesă etc. trebuie așezate în linie cu primele două.
- Ia naștere o bandă infinită de piese, care are două contururi de cele două părți.
- Pentru a putea pava planul, trebuie să alipim un număr infinit de asemenea benzi. Pentru aceasta este necesar ca cele două contururi să poată fi suprapuse, adică să fie periodice, cu perioade egale, iar o perioadă să fie permutare circulară a celeilalte.

Iată însă și o soluție bazată pe... matematică recreativă. În copilărie am citit diverse cărți de amuzamente matematice (Gheorghe Păun, mai tîrziu Martin Gardner). Am văzut la un moment dat ceva despre pavări (engl. *tilings*) și știam că există un criteriu simplu legat de contur, dar nu îl mai țineam minte. Morala? Nu știi de unde sare iepurele! Citiți (și rețineți) cît de mult puteți din sfere care n-au legătură cu interesul vostru imediat.

Criteriul este remarcabil de simplu și se numește Criteriul de translație. Implementarea necesită doi pași principali.

43.2.1 Găsirea conturului

Acest pas sună (și este) de clasa a 9-a, dar este migălos. Eu văd două variante de implementare și, ca robotul lui Asimov, am pierdut vreo oră cumpănind pe care s-o implementez.

1. Fie, pornind din colțul stînga-sus al celui cel mai din stînga-sus pătrat, urmăm conturul.
2. Fie colectăm toate muchiile, pe care apoi le legăm cap la cap.

Pînă la urmă, am ales varianta a doua. Ulterior am constatat că prima era cu vreo 30 de linii mai scurtă. 😐 Am obținut două șiruri în baza 4 care descriu conturul în cele două sensuri. Să denumim aceste șiruri X și Y. Ele au o lungime egală, fie ea L, unde $L \leq 2n + 2$.

43.2.2 Verificarea criteriului

Verificarea se reduce la *hashing* și la comparări de șiruri. De aceea calculăm valori hash ale tuturor prefixelor (tratăm elementele șirurilor ca pe cifre în baza 4 și operăm modulo un număr mare prim). Prin diferență putem afla valoarea hash a oricărui subșir.

Acum căutăm un semiperimetru al conturului, așadar o jumătate din X, care să aibă structura ABC, iar jumătatea complementară din Y să aibă structura CBA. Încercăm fiecare rotație posibilă cu $0, 1, \ldots L/2-1$ poziții. Pentru o rotație cu k poziții, fie x subșirul $X[k \ldots k+L/2-1]$ și fie y semiperimetrul complementar din Y.

Încercăm fiecare pereche de lungimi pentru A și C. Mai exact, pentru fiecare lungime $|A|$ și fiecare lungime $|C|$,

- dacă prefixul de lungime $|A|$ din x coincide cu sufixul de lungime $|A|$ din y și
- dacă sufixul de lungime $|C|$ din x coincide cu prefixul de lungime $|C|$ din y și
- dacă porțiunile B rămase după eliminarea sufixelor și prefixelor sînt egale,
- atunci putem pava planul.

43.2.3 Detalii de implementare

Algoritmul este $\mathcal{O}(n^3)$, dar în practică el se mișcă fulger, deoarece numerele de prefixe din x care sînt sufixe în y și viceversa sînt minuscule. Iterăm doar prin acelea și obținem un algoritm mai degrabă $\mathcal{O}(n^2)$.

Am dublat șirul Y pentru că urmează să facem căutări din X în Y, căutări care este posibil să se înfășoare peste finalul lui Y.

43.3 Problema Triunghi (Lot 2024)

enunț • sursă

Problema este relativ directă și totuși înșelătoare: în concurs au existat doar 5 scoruri de 100 și unul de 78!

43.3.1 Soluția prin înjumătățire

Dat fiind că minimul nu are inversă, nu cred să existe soluții bazate pe diferențe de arii. O soluție bazată pe compunerea de subprobleme este însă mult mai facilă. În esență, un triunghi de mărime k se descompune în două triunghiuri de mărime $k/2$ și un pătrat de mărime $k/2$. Este nevoie de atenție la cazurile pare și impare. De exemplu:

- Un triunghi de mărime 20 se compune din triunghiuri și pătrate de mărime 10.
- Un triunghi de mărime 21 se compune din triunghiuri de mărime 10 și un pătrat de mărime 11.

Mai important,

- Un pătrat de mărime 20 se compune din patru pătrate de mărime 10.
- Un pătrat de mărime 21 nu se descompune frumos în pătrate mai mici.

Pentru acest ultim caz, editorialul propune să folosim patru pătrate de latură 10, o linie, o coloană și un element suplimentar. Așadar, este nevoie să calculăm patru elemente: triunghiuri, pătrate, linii și coloane. Soluția mea este posibil mai simplă:

- Un pătrat de mărime 21 se descompune în două pătrate de mărime 11 și două pătrate de mărime 10, care cauzează exact o celulă-duplicat.

La prima vedere, fiecare din aceste valori înjumătățite (10 și 11) poate continua să se ramifice independent. De fapt, numărul variantelor nu crește dincolo de 2. Iată un exemplu pentru $k = 183$. Pentru a afla informațiile de pe o linie este suficient să avem informațiile din linia următoare.

mărimea triunghiului	mărimea pătratului 1	mărimea pătratului 2
183	—	—
91	92	—
45	46	—
22	23	—
11	11	12
5	5	6
2	2	3
1	1	2

Tabela 43.1: Toate mărimile de triunghiuri și pătrate necesare pentru $k = 183$.

Așadar, la fiecare nivel al recurenței calculăm o mărime de triunghi și cel mult două mărimi de pătrate. Logica recurenței se complică ușor, în special dacă dorim să calculăm informațiile *in-place*.

43.3.2 Detalii de implementare

Este ciudat că triunghiul este orientat către dreapta-sus. La descompunere, linia scade, iar coloana crește. Am implementat și această soluție, dar mai naturală mi se pare cea în care atît linia, cît și coloana cresc. Pentru aceasta, am citit și am afișat matricea oglindită (cu prima linie ultima).

Limita pe Kilonova este destul de strînsă. Soluția mea cu citire/scriere rapidă necesită 360 ms, dar cea cu citire normală necesită 820 ms. La concurs, amintiți-vă să faceți întotdeauna un calcul sumar pentru mărimea fișierului de intrare și să accelerați citirea dacă ajută. În acest caz, 4 milioane de numere × 6 cifre cu spații = 28 MB. Și fișierele de ieșire ajungeau la 23 MB.

Nu avem memorie pentru $\mathcal{O}(n \log n)$ matrice, deci este vital să reținem două niveluri. Se poate și cu un singur nivel, dacă suprascriem informațiile în ordinea corectă.

43.3.3 Misterioasa soluție în $\mathcal{O}(n^2)$

Editorialul menționează o soluție pătratică. Ideea de bază este să împărțim matricea în pătrate de latură $k/2$ și să precalculăm informații. Eu nu sînt convins că funcționează.

43.4 Problema Subiectul I („Nervi”) (Lot 2024)

enunț • surse

43.4.1 Soluție în $\mathcal{O}(n^3)$

O soluție în $\mathcal{O}(n^3)$ primește 54 de puncte și este relativ evidentă: extindem fiecare spirală, pornind pe rînd din fiecare centru (l, c) în oricare dintre cele patru direcții și în cele două sensuri (orar sau antiorar). Maximul acestor opt posibilități ne arată raza maximă a unei spirale care pornește din (l, c). Răspunsul la problemă este suma acestor maxime pentru toate punctele de pornire (toate celulele matricei).

Trebuie să acordăm atenție spiralelor de latură pară. Dacă la cele impare centrul cade într-o celulă, la cele impare centrul care la intersecția a patru celule. Este important să numărăm aceste spirale exact o dată.

Pentru a obține complexitatea de $\mathcal{O}(n)$ per punct de pornire, trebuie să putem extinde spirala cu încă o latură (sau, de fapt, cu un colțar format din două laturi) în $\mathcal{O}(1)$. Putem face acest lucru precalculînd, pentru fiecare punct și fiecare dintre cele patru direcții, distanța pînă la primul element care nu este ordonat. De exemplu:

$$d[l][c][dreapta] = \begin{cases} 1 & \text{dacă} \quad a[l][c] < a[l][c+1] \\ 1 + d[l][c+1][dreapta] & \text{dacă} \quad a[l][c] \geq a[l][c+1] \end{cases}$$

Pentru a simplifica implementarea, putem să evităm să scriem cod similar pentru cele patru direcții. În loc de aceasta, putem scrie cod pentru o singură direcție și orientare, iar pe celelalte le obținem rotind și oglindind matricea.

43.4.2 Soluție în $\mathcal{O}(n^2)$

Pentru 100 de puncte este nevoie să găsim următoarea recurență. Raportat la centru, matricea se compune din patru triunghiuri dreptunghice. Pentru triunghiul care se extinde spre nord, dorim să răspundem la întrebarea: ce latură maximă poate avea acest triunghi astfel încît, pe fiecare linie orizontală, elementele să fie descrescătoare? Similar și pentru celelalte trei triunghiuri. Latura maximă a unei spirale impare este minimul dintre răspunsurile pentru cele patru triunghiuri.

Putem calcula în $\mathcal{O}(n^2)$ răspunsurile pentru toate triunghiurile care merg spre nord, observînd că putem obține triunghiul cu vîrful în (l, c) din două triunghiuri cu vîrfurile în $(l-1, c-1)$ și $(l-1, c+1)$.

Din nou, este nevoie să tratăm cu atenție spiralele de latură pară.

43.5 Problema Subiectul III („Single crossing”) (Lot 2024)

enunț • surse

Editorialul include o soluție elegantă, bazată pe inversiuni și pe metrici, dar care mă eludează. Iată însă o rezolvare care folosește tehnici elementare (sortare + testare).

43.5.1 Compararea permutărilor

Soluția are doi pași. Întîi construim un candidat de soluție, care are proprietatea că fie el este *single-crossing*, fie nu există soluție. Apoi verificăm dacă acest candidat este *single-crossing*.

Definim următorul comparator util pentru sortare. Despre două permutări nu putem spune foarte multe. În schimb, date fiind trei permutări X, Y și Z, să fixăm ordinea primelor două. Raportat la acestea există cel mult o poziție corectă pentru Z, din cele trei variante: stînga (ZXY), centru (XZY) și dreapta (XYZ). Cred că există multe metode de a o găsi. Iată una dintre ele.

Căutăm prima diferență între X și Y (prima poziție i unde $X_i \neq Y_i$). Fie aceste valori a și b. Atunci a apare înainte de b în X, iar b apare înainte de a în Y. Pentru concizie, notăm aceste proprietăți cu $a <_X b$ și $b <_Y a$. Verificăm cum apar a și b în Z. Iau naștere două cazuri.

1. Dacă $a <_Z b$, atunci ordinea corectă poate fi doar ZXY sau XZY, căci XYZ cauzează o dublă trecere $ab - ba - ab$. Căutăm prima diferență între X și Z, fie $c <_X d$ și $d <_Z c$. Căutăm c și d în Y. Iau naștere două subcazuri:
   (a) Dacă $c <_Y d$, atunci XZY devine incorectă (dublă trecere $cd - dc - cd$), deci singura ordonare este ZXY (sau niciuna);
   (b) Dacă $d <_Y c$, atunci ZXY devine incorectă, deci ordonarea este XZY (sau niciuna).
2. Similar pentru $b <_Z a$, unde avem de decis între XZY și XYZ.

43.5.2 Sortare

Pe baza acestui comparator, putem implementa o sortare. Quicksort are nevoie de un mic ajutor, căci la pasul inițial nu poate partiționa vectorul relativ la un singur pivot. De aceea, alegem două permutări la întîmplare dintre cele n, X și Y, iar relativ la acestea le partiționăm pe restul în trei grupe. Apoi, pentru fiecare grupă rulăm quicksortul obișnuit. La fiecare apel, quicksort alege un pivot și partiționează intervalul curent în stînga și în dreapta pivotului, folosind ca al treilea element una dintre permutările de la capătul intervalului, a cărei existență este garantată.

Complexitatea sortării este $O(nm \log n)$, pentru că sortăm n elemente, iar costul unei comparații este $O(m)$. La finalul sortării obținem un candidat. Prin logica partiționării acest candidat este singurul care respectă toate tripletele comparate. Dar este nevoie să ne asigurăm că respectă și toate celelalte triplete, deci că este într-adevăr *single-crossing*.

Putem face chiar și insert sort să meargă, cu observația că dacă implementăm comparatorul naiv, în $\mathcal{O}(m)$, atunci insert sort va rula în $\mathcal{O}(n^2 \cdot m)$, prea lent. Dar dacă implementăm comparatorul în $\mathcal{O}(\log m)$, de exemplu dacă aflăm prin căutare binară poziția primei diferențe între două permutări, atunci insert sort ar trebui să ia 100 de puncte.

43.5.3 Verificare

Să presupunem că prima permutare este $(1, 2, \ldots, m)$ (în practică, iterăm prin valorile care ne interesează în ordinea în care apar ele în prima permutare). Procedăm inductiv. Dacă primele k permutări sînt *single-crossing*, dorim să verificăm dacă cea de-a $k + 1$-a cauzează vreo dublă

trecere. Cu alte cuvinte, vrem să vedem dacă vreo pereche de valori $x < y$, care erau inversate (y apărea înaintea lui x în P_k), devin acum ordonate (x apare înaintea lui y în P_{k+1}). Procesăm valorile de la 1 la m în ordine crescătoare. Dacă valoarea x apare pe poziția i în P_k și pe poziția j în P_{k+1}, dorim să știm: am văzut anterior vreo valoare pe o poziție $i' > i$ în P_k și $j' < j$ în P_{k+1}? Putem afla asta cu un singur AIB, care notează valoarea i la poziția j. Atunci interogarea devine: dă-mi maximul pe pozițiile $[1 \dots j-1]$. Așadar, folosim un AIB cu modificare pe poziție și maxim pe prefix.

43.6 Problema Gurger Brill (Lot 2024)

enunț • sursă

Editorialul prezintă o soluție „de stă mîța în coadă" care folosește $12 + 3 \log n$ interogări. Iată și alte soluții, mai pămîntene, care merg pînă la $4 \log n$ interogări și obțin 92 de puncte. În concurs, punctajul maxim a fost de 40 de puncte...

43.6.1 Soluție în $\mathcal{O}(n^2)$ interogări

Pentru o primă persoană arbitrar aleasă, X, facem interogări cu X pe pozițiile $n, n-1, \dots$, pînă cînd răspunsul devine „da" (X a ajuns înaintea ambilor săi vecini). Atunci ultima persoană pe care a sărit-o X în permutare, să-i spunem Y, este vecină cu X. Astfel facem cel mult n interogări pentru a afla o primă pereche de vecini.

Apoi, vom afla celălalt vecin al lui Y. Vom începe cu Y pe penultima poziție și X pe ultima și vom face interogări succesive, deplasîndu-i concomitent pe Y și pe X spre stînga pînă cînd răspunsul pentru Y devine „da". Ultima persoană peste care am sărit este celălalt vecin al lui Y, fie el Z. Continuăm cu Z pe penultima poziție și Y pe ultima.

În acest fel facem de n ori cîte $\mathcal{O}(n)$ interogări pentru a afla întreaga permutare circulară.

43.6.2 Soluție în $\mathcal{O}(n \log n)$ interogări

Procedăm similar soluției în $\mathcal{O}(n^2)$, dar la fiecare pas căutăm binar, nu liniar, poziția în care răspunsul devine „da".

43.6.3 Soluție în $4\sqrt{n}$ interogări

Pentru ultimele două soluții, ca regulă generală, pentru fiecare interogare P facem și o interogare pentru permutarea răsturnată. Astfel, pentru o persoană X iau naștere trei cazuri:

- X ajunge devreme în prima interogare: ambii vecini ai lui X se află la dreapta lui X în P.
- X ajunge devreme în a doua interogare: ambii vecini ai lui X se află la stînga lui X în P.
- X ajunge tîrziu în ambele interogări: X are cîte un vecin de fiecare parte în P.

Împărțim persoanele în $r = \sqrt{n}$ blocuri de cîte r persoane. Cu alte cuvinte, le grupăm după cîtul împărțirii la r. Facem $2r$ interogări, rotind permutarea cu r elemente după fiecare pereche de interogări. Analizînd rezultatele, putem afla, pentru fiecare persoană, blocurile în care se află cei doi vecini ai săi.

Acum, regrupăm persoanele după restul împărțirii la r și facem $2r$ interogări similare. Astfel aflăm, pentru fiecare persoană, cîturile și resturile vecinilor la împărțirea prin r.

De exemplu, pentru $n = 100$ de persoane numerotate de la 0 la 99, după primele 20 de interogări aflăm cifrele zecilor pentru cei doi vecini ai fiecărei persoane, iar după încă 20 de interogări aflăm cifrele unităților.

Aceste date încă nu ne oferă permutarea completă. Dacă vecinii lui X au cifrele zecilor 1 și 2, iar cifrele unităților 3 și 4, atunci vecinii lui X ar putea fi 13 și 24 sau 14 și 23. Vom discuta remediul pentru acest neajuns în secțiunea următoare.

43.6.4 Soluție în $4 \log n$ interogări

Pentru fiecare bit semnificativ b, unde $0 \leq b < \lceil \log n \rceil$, vom folosi 4 întrebări pentru a afla informații despre bitul b al ambilor vecini ai tuturor persoanelor.

Pentru fiecare bit b, partiționăm numerele după bitul b și facem două interogări. Apoi răsturnăm cele două intervale și facem încă două interogări. Iată un exemplu pentru $n = 16$ și bitul 0. Am numerotat persoanele de la 0 la $n - 1$ pentru claritate.

```
P = 0 2 4 6 8 10 12 14   1 3 5 7 9 11 13 15   (și răsturnata)
Q = 14 12 10 8 6 4 2 0   15 13 11 9 7 5 3 1   (și răsturnata)
```

Analizînd cazurile posibile putem deduce informații despre bitul b în toți vecinii. Iată, de exemplu, posibilitățile pentru $X = 6$ în funcție de numărul de vecini în stînga pentru interogările de mai sus:

răspuns la P	răspuns la Q	paritatea vecinilor
0	0	doi vecini impari
0	1	un vecin par și unul impar
0	2	doi vecini pari
1	0	un vecin par și unul impar
1	1	doi vecini pari
2	0	doi vecini pari

La final nu aflăm vecinii exacți, ci distribuția biților în cei doi vecini (adică dacă pe fiecare poziție avem 0, 1 sau 2 biți de 1). De exemplu, dacă pe toate pozițiile avem cîte un 1 și un 0, vecinii pot fi 0 și 15 sau 3 și 12 sau 7 și 8 etc. De aceea, poate fi nevoie să alegem o persoană de start A și să îi testăm, pe rînd, toți vecinii posibili B care respectă configurațiile pe biți. Vom prefera o persoană A pentru care numărul de biți variabili să fie cît mai mic. Odată fixat B, putem calcula restul mesei în $\mathcal{O}(n)$, căci cunoscînd A și B, C este impus, apoi din B și C, D este impus, etc.

Pe teste aleatorii, acest algoritm găsește soluția. Ea poate fi pedepsită cu teste adversariale în care toate persoanele au mulți biți variabili între cei doi vecini. Atunci algoritmul găsește, cel mai probabil, o soluție greșită. Există două remedii:

- Identificarea deterministă a unui vecin al lui A, cu încă $\log n$ interogări (de exemplu, o căutare binară în care A variază, iar restul permutării rămîne nemodificat).
- Indirectarea tuturor întrebărilor și răspunsurilor printr-o permutare aleasă aleatoriu. Sursa anexată implementează acest remediu.

43.7 Problema Takara (Baraj ONI 2019)

enunț • surse

Problema necesită „doar" cozi monotone (deque-uri), pentru care acest curs nu are un capitol dedicat. Totuși, raționamentele din spate sînt dificile.

43.7.1 Formula de recurență

Recurența decurge relativ ușor din enunț. Fie $D_{i,j}$ costul pentru a găsi comoara în intervalul închis $[i, j]$. Atunci trebuie să existe undeva o „spargere", o primă interogare la o poziție k. Odată ce plătim costul C_k vom reduce problema fie la $[i, k-1]$, fie la $[k+1, j]$, și trebuie să presupunem scenariul cel mai defavorabil. Astfel rezultă formula de recurență:

$$D_{i,j} = \min_{i \le k \le j} \{C_k + \max(D_{i,k-1}, D_{k+1,j})\}$$

Prima sursă implementează fix acest lucru, în $\mathcal{O}(n^3)$, fără alte zorzoane. Dacă aș fi la concurs, aș trimite din principiu această sursă, ca să verific că primesc doar TLE, nu și alte erori. Ea cere doar 9 linii în plus față de scheletul programului, deci este un preț mic.

43.7.2 Eliminarea funcției max

Avem intuiția că trebuie să îl calculăm pe $D_{i,j}$ din $D_{i,j-1}$ și din $D_{i+1,j}$. Dar nu putem ataca frontal această idee. Între perechile $(D_{i,k-1}, D_{k+1,j})$ și $(D_{i,k-1}, D_{k+1,j+1})$, toate elementele din dreapta se modifică, ceea ce afectează toate operațiile de maxim. De altfel, în problemele de programare dinamică am observat adesea că formulele care implică maxime, minime sau valori absolute nu sînt utile ca atare, căci nu putem face calcule (sume etc.) pe astfel de valori. Este mai util să enumerăm intervalele pe care aceste funcții iau diverse valori.

Este nevoie să înțelegem mai bine natura valorilor din matrice. Un sfat practic este: tipăriți matricea! Cine știe ce amănunt interesant observați. În acest caz, observăm că matricea D crește spre dreapta pe linie și în sus pe coloană. Era și de așteptat, deoarece costul de a găsi comoara în intervalul $[i, j]$ trebuie să fie mai mic sau egal cu costurile pentru $[i-1, j]$ și $[i, j+1]$, care îl includ.

Acum, să privim mai atent formula de recurență. Observăm că valorile de care depinde $D_{i,j}$ aparțin liniei i sau coloanei j. Ele sînt comparate două cîte două pentru a alege maximul, ca în figura 43.1. Fiecare comparație implică două celule de aceeași culoare, una de pe linia i și una de pe linia j.

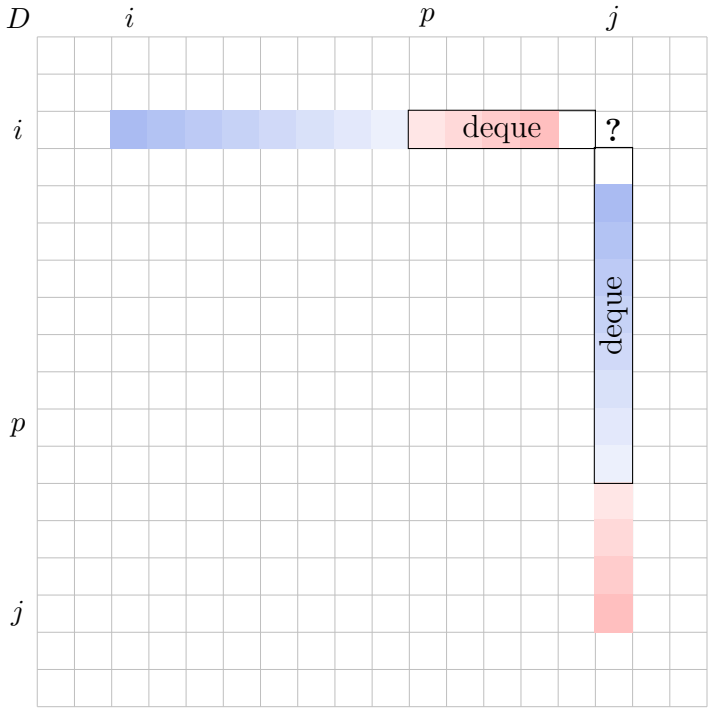


Figura 43.1: Matricea D și zonele de interes pentru calculul lui $D_{i,j}$.

Deoarece valorile D cresc la dreapta și în sus, înseamnă că pentru valori mici ale lui k maximul va fi dat de $D_{k+1,j}$, ceea ce este natural, întrucît intervalul $[i, k-1]$ este mai mic decît intervalul $[k+1, j]$. Cînd k crește, valorile $D_{i,k-1}$ cresc, iar valorile $D_{k+1,j}$ scad. Deci va exista o ultimă poziție, să-i spunem p, unde $D_{i,p-1} < D_{p+1,j}$.

Ce deducem de aici? Că valoarea lui $D_{i,j}$ este minimul tuturor valorilor din zona roșie de pe linia i și din zona albastră de pe coloana j. În limbaj matematic,

$$D_{i,j} = \min \Big\{ \min_{i \le k \le p} \{C_k + D_{k+1,j}\}, \min_{p < k \le j} \{C_k + D_{i,k-1}\} \Big\}$$

43.7.3 Reducerea la $\mathcal{O}(n^2)$

Acum **aproape** putem să gestionăm corect trecerea de la $D_{i,j}$ la $D_{i,j+1}$. Ce se schimbă în zona roșie de pe linia i? Mai facem o observație privind natura matricei D: valoarea lui p nu poate să scadă.

Într-adevăr, să nu uităm cine este p: este punctul din intervalul $[i, j]$ unde, dacă facem spargerea, partea stîngă devine mai costisitoare decît partea dreaptă. Cu siguranță, dacă j crește, partea dreaptă devine mai costisitoare, iar p poate rămîne constant sau poate avansa spre dreapta.

Recunoaștem nevoia clasică de a menține minimul într-o fereastră glisantă. Înseamnă că putem reprezenta cantitățile din zona roșie a liniei i cu o coadă dublă (deque). Dar cum procedăm cu cantitățile de pe coloana j? Acestea se schimbă radical cînd avansăm la coloana $j + 1$. Am putea să parcurgem matricea pe coloane, dar atunci dăm peste problema inversă: la urcarea pe o coloană toate elementele de pe linie se schimbă radical.

Soluția este elegantă: menținem cîte un deque **pe fiecare coloană**, plus încă un deque pe linia curentă i. Deque-ul pe linie avansează spre dreapta, iar cele pe coloane avansează în sus. Calculăm $D_{i,j}$ folosind deque-urile liniei i și coloanei j, apoi „punem la păstrare" deque-ul coloanei j. Pentru $D_{i+1,j}$ folosim deque-ul coloanei $j + 1$. Cu deque-ul coloanei j ne vom reîntîlni cînd va veni momentul să calculăm $D_{i-1,j}$.

O primă variantă a codului meu încapsula deque-urile într-o structură în care menținea explicit valoarea lui p. De exemplu, codul pentru deque-ul pe linie era:

```
struct row_queue {
  std::deque<int> q;
  int i, p;

  void init(int i) {
    this->i = p = i;
    q.clear();
    q.push_back(i);
  }

  int cost(int j) {
    return d[i][j - 1] + c[j];
  }

  int first() {
    return cost(q.front());
  }

  void advance(int j) {
    while (d[i][p - 1] < d[p + 1][j]) {
      p++;
    }
    while (q.front() < p) {
      q.pop_front();
    }
    int z = cost(j);
    while (!q.empty() && (z < cost(q.back()))) {
      q.pop_back();
    }
```

```
    q.push_back(j);
  }
};
```

În realitate, nu avem nevoie să stocăm explicit valoarea lui p. Pentru a ști dacă trebuie să eliminăm un element de la începutul deque-ului, de pe o poziție k, putem să testăm direct dacă $D_{i,k-1} < D_{k+1,j}$, cu alte cuvinte dacă elementul se află în zona albastră din figura 43.1. Am învățat această optimizare din sursa lui Andrei C.

43.8 Problema Tornade (Baraj ONI 2025)

enunț • sursă

Vă recomand să începeți prin a citi editorialul. Conform dictonului „pantofarul n-are pantofi", site-urile oficiale ale olimpiadei nu au aceste editoriale, dar îi mulțumesc comunității RoAlgo care face efortul de conservare. De asemenea, puteți citi o sursă, chiar bine scrisă, care implementează algoritmul în $\mathcal{O}(n \log n)$.

Iată însă și o soluție elementară, care folosește doar liste înlănțuite (pe steroizi, ce-i drept) pentru a simula toate operațiile în $\mathcal{O}(n)$. Să considerăm un vector inițial și două operații min:

```
înainte:                  1 1 2 2 3 3 2 2 1 1 2 2 3 3 2 2 1 1
după prima operație:       1 1 2 2 3 2 2 1 1 1 2 2 3 2 2 1 1
după a doua operație:        1 1 2 2 2 2 1 1 1 1 2 2 2 2 1 1
```

Să urmărim *seriile*, adică grupele maximale de valori egale. Este destul de evident că:

- Minimele locale (seria de 1) se lungesc cu 1.
- Minimele locale de la capetele vectorului își păstrează lungimea.
- Maximele locale (seriile de 3) se scurtează cu 1, inclusiv la capetele vectorului.
- Seriile de valori intermediare își mențin lungimea.

Dacă o serie cu maxime locale ajunge la lungime 0, ea dispare din vector. Seriile vecine ei se alipesc sau, dacă au valori egale, chiar se contopesc într-o singură serie. Este posibil ca în urma alipirii sau contopirii, aceste serii să devină maxime sau minime locale.

Dacă reprezentăm vectorul ca pe o **listă globală** de serii, pare că avem șanse să gestionăm toate operațiile în $\mathcal{O}(n)$. Dar trebuie să clarificăm:

- cum creștem / scădem lungimile tuturor optimelor locale în $\mathcal{O}(1)$;
- cum identificăm toate seriile care ajung la lungime 0 în $\mathcal{O}(1)$;
- cum luăm în evidență noile serii care devin optime locale în urma ștergerilor.

Vom stoca, pe lîngă lista globală, încă $2n$ liste. Pentru fiecare lungime l vom stoca:

- o listă cu toate seriile care sînt minime locale și au lungime l;
- o listă cu toate seriile care sînt maxime locale și au lungime l.

Ca să clarific: Fiecare `struct` care reprezintă o serie are, pentru dubla înlănțuire, patru pointeri: doi pentru lista globală și doi pentru lista cu optimele de aceeași lungime cu a sa.

Acum, cînd toate seriile de minime locale cresc cu o unitate, nu dorim să trecem prin $\mathcal{O}(n)$ serii și să le mutăm pe fiecare de la lungimea l la $l + 1$! Ce-i de făcut? Soluția este să ținem un contor global, `global_min_cnt`, o cantitate de adăugat la toate listele de minime locale pentru a afla lungimea reală. Dacă consultăm o serie și pe ea scrie lungimea l, lungimea ei reală este `l + global_min_cnt`. Atunci putem crește simultan lungimea tuturor seriilor minime locale în $\mathcal{O}(1)$: `global_min_cnt++`.

Un subcaz aici este că minimele locale de la capetele vectorului nu cresc. Dar noi le-am crescut implicit cînd am crescut `global_min_cnt`. Dacă există, aceste minime de la capete trebuie scoase din lista lor pe lungimi și mutate în lista pentru lungimea cu 1 mai mică. Astfel le „ținem pe loc" la aceeași lungime.

Au mai rămas de tratat maximele locale, care pierd 1 la lungime. Desigur, vom proceda la fel: folosim o variabilă globală pe care o decrementăm ca să obținem timp constant: `global_max_cnt--`. Mai ales, iterăm prin lista de maxime locale care prin decrementare au ajuns la lungime 0. Toate aceste elemente dispar. Iar dispariția lor trebuie tratată cu atenție pentru că trebuie să ne gîndim ce se întîmplă cu vecinii lor. Există nenumărate cazuri:

- Unul dintre vecini poate să nu existe.
- Vecinul era optim local și continuă să fie optim local.
- Vecinul nu era optim local, dar devine optim local.
- Vecinul era optim local și încetează să mai fie.
- Vecinii se unesc între ei, căci au aceeași valoare.

Putem trata simplu și unitar toate aceste cazuri:

- Scoatem ambele noduri din orice listă pe lungimi.
- Le clasificăm ca elemente intermediare (cu această ocazie le calculăm lungimea reală).
- Le unificăm dacă este cazul.
- Le reclasificăm corect conform noilor lor vecini.

Sursa – nu vă mint – n-a fost ușor de scris. M-am chinuit mai mult decît era necesar la acest ultim pas, al ștergerii seriilor de lungime 0. Dar, după cum am mai spus, la ONI și chiar și la Baraj este important să putem exprima în cod niște idei rezonabil de grele.

43.9 Problema As de Spade (Lot 2024)

enunț • surse

Editorialul, la care mi-am adus și eu o contribuție, include două soluții. Prima dintre ele folosește căutarea binară, înlocuind interogările din enunț cu interogări de tipul: cîte numere de lungime l încap înainte de poziția K? Cea de-a doua soluție, pe care o vom studia aici, procedează mai

„băbește”, compunînd cifră cu cifră, în baza 10, răspunsul la întrebarea „ce număr acoperă poziția K”?

Pentru a înțelege mai bine structura lui G, putem să judecăm problema doar cu ochii minții. Dar cred că folosim mai bine timpul dacă scriem rapid un *pretty printer*, un programel auxiliar care varsă un set de date.

Să pornim de la un exemplu, un subșir al lui G pentru $A = 1$, $B = 999\,999$:

număr	poziție în G
252526	4890865
252525	4890871
2525	4890877
25	4890881
252524	4890883
252523	4890889
25252	4890895
252522	4890900
252521	4890906

Facem observația relativ evidentă că toate numerele de aceeași lungime apar în G în ordine descrescătoare. Așadar, G este o interclasare a șirurilor de numere de lungimi diferite. Mai exact, printre numerele de lungime maximă apar prefixe mai scurte ale acestor numere. Atunci, să definim o **zonă** ca fiind un număr împreună cu toate prefixele numărului care sunt adiacente cu acesta în G. De exemplu, zona lui 252525 include prefixele 2525 și 25 și are lungime totală 12. În schimb, prefixul 25252 nu este în această zonă, ci în zona lui 252522. Zona lui 252521 conține doar numărul însuși, având lungime 6.

Putem deduce că un prefix se află în zona numărului de lungime maximă obținut prin completarea ciclică a prefixului până la lungimea maximă. De exemplu, 5287 face parte din zona lui 528752.

Cu această informație, pentru o interogare K putem afla zona de la poziția K. Completăm rezultatul cifră cu cifră. Întâi ne întrebăm: care este lungimea totală a zonelor care încep cu 9? Dar cu 8? Dar cu 7? Scădem aceste valori din K cât timp se poate. Dacă am stabilit, de exemplu, că prima cifră a zonei este 5, ne întrebăm: care este lungimea totală a zonelor care încep cu 59? Dar cu 58? Dar cu 57? Etc.

Fie $L(P, N)$ lungimea zonelor care încep cu prefixul P și nu depășesc valoarea N. Atunci răspunsul la întrebările de mai sus va fi $L(P, B) - L(P, A - 1)$, pentru prefixe P succesiv mai lungi. Pentru a-l afla pe $L(P, N)$ este nevoie de:

- Suma lungimilor numerelor care încep cu P, au între $|P|$ și $|N|$ cifre și nu depășesc N. Aceasta necesită doar aritmetică naivă.
- Suma lungimilor prefixelor lui P care sunt în zona lui P. Putem face acest lucru în $\mathcal{O}(|P|)$ dacă răspundem în $\mathcal{O}(1)$ la întrebarea: este Q în zona lui P? Vom da un exemplu aici: $Q = 523$ (3 cifre) este în zona lui $P = 5235235235$ (10 cifre) deoarece, prin repetare ciclică a lui Q până la 10 cifre, îl obținem exact pe P. Echivalent, primele $10 - 3 = 7$ cifre ale lui

P coincid cu ultimele 7 cifre, iar acest test îl putem face în $\mathcal{O}(1)$ dacă avem precalculate puterile lui 10.

Ultimul pas necesar, odată găsită zona, este să izolăm cifra dorită din zonă. Am abordat naiv acest pas, sortând toate numerele din zonă. Este dificil să anticipăm ordinea numerelor din zonă. De exemplu, pentru numărul 894848948, prefixul 89484894 îl precedă în G, dar prefixul 89484 vine după el.

Complexitatea totală este $\mathcal{O}(RD^2)$ per interogare, unde R este baza de numerație (10), iar D este lungimea maximă (18).

43.10 Problema Scurtcircuit (Lot 2025)

enunț • sursă

Problema necesită cîteva cunoștințe de nișă despre sumatoare. Eu aveam doar bazele, dar am reușit să redescopăr o bună parte, pînă la 90p. 🙂

Sigur, am putem proiecta sumatorul băbește. Pe ultima coloană adunăm doi biți, din care rezultă un rezultat și un bit de transport pentru penultima coloană. Rezultatul îl adunăm cu al treilea bit și rezultă încă un transport pentru penultima coloană etc. Dificultatea problemei constă în faptul că ține cont de adîncimea circuitului, adică de numărul maxim de porți înlănțuite astfel încît intrarea uneia să depindă de ieșirea precedentei. Această adîncime trebuie să fie oarecum logaritmică pentru punctaj maxim (vezi coloana D_{lim_5} din enunț). Sumatorul naiv va ajunge la adîncime $\mathcal{O}(nk)$, mult prea rea.

43.10.1 Compresorul 3:2

Să observăm că un sumator pentru două numere nu adună doar doi cîte doi biți. Pe fiecare poziție, sumatorul are nevoie de un circuit elementar cu **trei** intrări: biții de pe acea poziție și transportul de la poziția anterioară. Acest circuit se numește sumator integral (engl. *full adder*), spre deosebire de un sumator fără transport, care se numește semisumator (engl. *half adder*).

Rezultatul adunării a 3 biți va fi între 0 și 3 (cine a spus că nu învățăm și matematică la acest cerc??). Așadar, la ieșire ne ajung doar doi biți: rezultatul pe aceeași poziție și transportul pentru următoarea poziție. De aceea, sumatoarele integrale se mai numesc și compresoare 3:2.

Merită să petrecem puțin timp ca să proiectăm un compresor 3:2 cît mai eficient (conform problemei), căci este cărămida de bază a soluției. Adîncimea este crucială; numărul de celule suplimentare este doar cireașa de pe tort. Iată un sumator integral care folosește 3 celule suplimentare și are adîncime 3. Codul de mai jos adună biții din celulele x, y și z și pune suma în s și transportul (*carry*) în c. Folosim formulele:

- $s = x \oplus y \oplus z$
- $c = x \cdot y + y \cdot z + z \cdot x$ sau, ca să refolosim celulele, $x \cdot y + (x \oplus y) \cdot z$

```
void compress_32(int x, int y, int z, int* s, int* c) {
int x_xor_y = new_cell(OP_XOR, x, y);
*s = new_cell(OP_XOR, x_xor_y, z);
int x_and_y = new_cell(OP_AND, x, y);
int x_xor_y_and_z = new_cell(OP_AND, x_xor_y, z);
*c = new_cell(OP_OR, x_and_y, x_xor_y_and_z);
}
```

43.10.2 Reducerea de la k numere la două numere

Noi putem folosi compresorul 3:2 și la modul general, adunînd orice 3 biți vrem noi. Să privim numerele ca pe o matrice de $n \cdot k$ biți. Pe fiecare coloană c, luăm grupe de cîte trei biți și îi înlocuim cu doi: unul pe coloana c și unul pe coloana $c+1$. Procedăm astfel pe toate coloanele și obținem o nouă colecție de biți, toți calculabili la adîncime 3, căci așa funcționează sumatorul (biții rămași ca rest modulo 3 sînt la adîncime 0, dar ne interesează maximul).

Repetăm acest procedeu, reducînd la fiecare trecere numărul de biți la circa 2/3. Așadar, vor fi necesare circa $\log_{3/2}(n/2)$ iterații pentru a reduce n biți la 2 biți, iar fiecare iterație crește adîncimea cu 3. Mai există niște rotunjiri în sus datorită biților rămași modulo 3, dar ne încadrăm bine în limitele problemei. Ca să dăm un exemplu la întîmplare, pentru $n = 500$ și $k = 200$ ne trebuie $\log_{3/2} 250 < 14$ iterații ca să reducem 500 de numere la două numere, adică adîncime 42 din totalul de 65 permis pentru punctaj maxim.

43.10.3 Generarea și propagarea transportului

Am redus așadar problema la două numere de $k + \log_2 n$ biți (vom spune k pentru simplitate). Nu ne permitem să adunăm numerele naiv, de la dreapta la stînga, căci fiecare transport ar depinde de anteriorul, ceea ce ar duce la o adîncime $\mathcal{O}(k)$. Este nevoie să paralelizăm efortul, adică să scriem sumatoare progresiv mai mari pentru subsecvențe ale numerelor. Dar sumatoarele au nevoie, la intrare, de transport. Cum putem ști, pe distanțe mari, dacă există transport? Pentru aceasta, să introducem noțiunea de *generare-propagare* (GP).

Să considerăm o coloană i cu biții a și b. La adunare,

- Coloana **generează** transport dacă $a = b = 1$.
- Coloana **propagă** transportul primit de la coloana $i-1$ către coloana $i+1$ dacă $a \oplus b = 1$.

Să definim așadar cantitățile booleene g și p care indică, respectiv, dacă coloana i generează și propagă transport. Acum, să considerăm două zone mai mari de coloane, de lățimi arbitrare, alipite. Să spunem că le-am calculat caracteristicile g_1 și p_1 (pentru zona mai puțin semnificativă), respectiv g_2 și p_2 (pentru zona mai semnificativă). Ce putem spune despre reuniunea zonelor?

- Reuniunea generează transport dacă zona 2 generează transport **sau** zona 1 generează transport, iar zona 2 îl propagă. Așadar, $g = g_2 + g_1 \cdot p_2$ (operatorii $+$ și $\cdot$ sînt booleeni).
- Reuniunea propagă transportul dacă și zona 1, și zona 2 îl propagă. Așadar, $p = p_1 \cdot p_2$.

Putem exprima aceste formule ca pe un operator:

$$(g_1, p_1) \circ (g_2, p_2) = (g_2 + g_1 \cdot p_2, p_1 \cdot p_2)$$

Implementarea acestui operator este în funcția `combine()`, care are adîncime 2:

```
struct gen_prop {
  int g, p;
};

gen_prop combine(gen_prop a, gen_prop b) {
  gen_prop res;
  int a_gen_b_prop = new_cell(OP_AND, a.g, b.p);
  res.g = new_cell(OP_OR, b.g, a_gen_b_prop);
  res.p = new_cell(OP_AND, a.p, b.p);
  return res;
}
```

43.10.4 Adunarea a două numere

În sfîrșit putem ataca adunarea finală. Să definim G_i și P_i = generarea, respectiv propagarea transportului în zona de coloane $0 \ldots i$. Dacă reușim să le calculăm, atunci suma celor două numere este ușor de calculat. Transportul la intrare pe orice coloană i va fi G_{i-1}, iar suma (bitul de rezultat care ne interesează) va fi $a_i \oplus b_i \oplus G_{i-1} = P_i \oplus G_{i-1}$. Foarte convenabil, toate aceste valori pot fi calculate în paralel (adică la adîncime 1).

Iar, pentru a calcula G_i și P_i, vom construi o tabelă rară `GP`, unde `GP[p][i]` reține valorile parțiale `G[i-2^p+1...i]` și `P[i-2^p+1...i]`. Calculăm fiecare celulă combinînd două jumătăți, ca la RMQ. Ca să ne facem viața mai ușoară, definim și valorile `GP[p][i]` pentru care $i < 2^p$, cu convenția că în acele cazuri ne referim la tot intervalul $[0, i]$. Atunci pe ultima linie a tabelei `GP`, fie ea linia h (cu $2^h \geq k$) toate intervalele vor începe de la 0, deci linia h conține fix valorile G_i și P_i care ne interesează.

Calculul tabelei `GP` crește adîncimea circuitului cu $2 \log_2 k$ niveluri.

Mai rămîne doar să calculăm rezultatul, în adîncime 1, așa cum am spus.

Acest circuit se numește sumator Kogge-Stone. În prima mea încercare am folosit un alt sumator, Brent-Kung, dar acesta necesită o adîncime mai mare, $4 \log_2 k$ și depășea limita pentru punctaj maxim.

43.11 Problema Lungcircuit (Lot 2025)

enunț • surse

Problema Lungcircuit îmi aparține. Inițial căutam o problemă pentru lotul de juniori, bazată pe jocul Hungarian Rings. A rezultat o problemă prea grea pentru juniori. În seara dinaintea unei probe, comisia de seniori mi-a cerut problema ca să o dea a doua zi, pentru că problema avea un mare avantaj: era la cheie (teste, subtaskuri, soluții pentru toate subtaskurile, chiar și editorial). Uneori gradul de pregătire bate calitatea problemei. 🥲

Problema are o soluție „tractor" și o soluție elegantă, care este mai mult de idee. Editorialul include și soluții pentru subtaskuri.

43.11.1 Soluția „tractor"

Introducem doi termeni pentru a ușura descrierea. Numim pozițiile comune **noduri**. Primul nod este la poziția 0, iar ultimul nod este la poziția $(k-1)d$. Numim **lanțuri exterioare** pozițiile $(k-1)d+1 \dots n-1$ pe fiecare circuit.

Pentru subtaskurile mici, putem menține circuitele complete la fiecare mișcare. Pentru o soluție în $\mathcal{O}((q+n)\log k)$, să schimbăm paradigma. Să urmărim traseul fiecărei bile și să aflăm poziția ei finală după cele q mutări. Vom accelera simularea procesînd rapid porțiuni de circuit. Pornind de la mutarea 0, fiecare bilă va trece prin următoarele etape (sau o submulțime a lor):

1. Avansează pînă la următorul nod.
2. Avansează pînă la ultimul nod.
3. Avansează pînă la primul nod (parcurgînd lanțul exterior).
4. Găsește ultima mutare cînd bila mai revine în primul nod.
5. Avansează un număr întreg de noduri (cel mult $k-1$).
6. Avansează pozițiile rămase pînă la epuizarea operațiilor.

Pentru (1), (3) și (6), știind că bila se află pe circuitul c la mutarea t și că dorim să avanseze p pași, trebuie să putem răspunde la întrebarea „ce indice are a p-a mutare de tip c, începînd cu mutarea t"? Putem răspunde la aceste întrebări cu o precalculare ușoară, colectînd în doi vectori pozițiile literelor **R** și **A** în șirul de mutări.

Pentru (2) și (5), să remarcăm că bila pornește dintr-un nod, deci prima mișcare dictează pe ce drum pornește bila. Pentru a avansa un interval în $\mathcal{O}(1)$, dacă prima mișcare este c la momentul t, trebuie să găsim rapid a d-a apariție a lui c, începînd cu mutarea t, în șirul de mutări. Și această informație poate fi precalculată.

Noi dorim să avansăm eficient $\mathcal{O}(k)$ intervale, deci peste informația de mai sus construim o tabelă rară (*sparse table*). Astfel putem avansa pînă la ultimul nod în $\mathcal{O}(\log k)$.

Pentru (4), dorim să răspundem la întrebări de forma: „dacă o bilă se află în nodul 0 la momentul t, cînd va reveni ea în nodul 0?". Putem refolosi exact codul anterior. Dacă răspunsul există și este t', atunci putem completa de la sfîrșitul spre începutul vectorului de mutări momentul **ultimei** reveniri în nodul 0: el va fi același pentru t și pentru t'.

43.11.2 Soluție cu liste

O soluție în $\mathcal{O}(n+q)$, semnificativ mai simplu de codat, pornește de la următoarea observație. Bilele din noduri (în exemplul dat, 0, 3, 6, 9 și 12) au o traiectorie similară. Dacă prima mutare este **R** și bila 0 intră pe circuitul roșu, la fel vor face și celelalte. Cînd 0 revine într-un nod, la fel și celelalte. Similar sînt unite și bilele 1, 4, 7, 10 și 13.

De aceea, putem să grupăm aceste bile și să le mutăm pe toate cu efort mic. Concret, reprezentăm un circuit ca:

- d liste, prima de k bile, iar restul de cîte $k-1$ bile. Numim această structură **rețea**, căci elementele listelor sînt intercalate. Prima listă conține nodurile și este comună celor două circuite.
- Lanțul exterior (inițial 16-31, respectiv 42-57).

Ce se întîmplă la o mutare **R** (cazul **A** este simetric)?

- Lanțul exterior capătă o bilă la început, și anume ultimul nod (15).
- Lanțul exterior pierde o bilă de la final (31), care ajunge la începutul ultimei liste din rețea, care devine 31, 2, 5, 8, 11, 14.
- Permutăm circular listele rețelei. Ultima listă devine prima. Astfel, bila 31, care acum stă în nodul 0, devine în mod corect prima bilă din prima listă.
- Notificăm circuitul albastru că prima sa listă nu mai este $0, 3, \ldots$, ci este $31, 2, \ldots$

Cu această reprezentare putem simula toți pașii de mai sus, și deci toate mutările, în $\mathcal{O}(1)$ fiecare. Putem reprezenta atît listele din rețea, cît și lista de liste (rețeaua însăși), ca pe buffere circulare, ca să le putem permuta circular în $\mathcal{O}(1)$.

Unele implementări se simplifică considerabil dacă stocăm ultimul nod separat de rețea. Atunci toate listele din rețea au lungime egală, $k-1$.

Implementarea cu `deque`-uri este chiar mai scurtă, dar este foarte ineficientă cînd d este mare, deoarece creează o mulțime de `deque`-uri mici.

43.12 Problema Cal-cule (Lot 2025)

enunț • surse

43.12.1 Enunțul

Să clarificăm mai întîi enunțul problemei (uneori este necesar).

- Există un șir secret de $n \leq 1\,000$ elemente.
- La intrare primim n și un alt număr k.
- Trebuie să aflăm histograma (distribuția frecvențelor) din șirul secret.
- Putem face interogări despre histograma oricărui subșir, specificînd indicii elementelor din subșirul dorit.

- **Dar** nu primim ca răspuns histograma, ceea ce ar fi prea simplu. În schimb, comisia trece histograma printr-o funcție pe care tot noi o implementăm, care reduce o histogramă la o valoare între 0 și $k - 1$.

Concret,

- Trebuie să implementăm funcția `descuie_grajdul`.
- Din `descuie_grajdul` putem apela funcția comisiei, `intreaba_cal`, trimițîndu-i indicii din subșirul dorit.
- `intreaba_cal` calculează histograma subșirului, iar cu acea histogramă apelează funcția noastră, `dreseaza_cal`.
- Funcția `dreseaza_cal` trebuie să reducă orice vector la o valoare între 0 și $k - 1$.

Restricțiile includ un tabel de valori posibile pentru k. Vedem că, cu cît k este mai mare, cu atît limita pentru numărul de interogări scade (firește).

43.12.2 O soluție în $\mathcal{O}(n \log n)$

Prima mea idee a fost să aflu efectiv vectorul. Desigur, nu putem afla exact valorile, dar putem afla clasele de echivalență (pe ce poziții apar valori egale). De exemplu, este relevant că $v[0] = v[2] = v[7]$, dar nu este relevantă valoarea efectivă.

Să considerăm că am aflat clasele pe pozițiile $0, 1, \ldots, i - 1$ și dorim să aflăm clasa poziției i, adică dorim să aflăm cu ce poziție anterioară este egal $v[i]$ sau dacă $v[i]$ este o valoare încă nemaiîntîlnită. Putem căuta binar această informație astfel. Să colectăm indicii primei apariții a fiecărei valori distincte. De exemplu, dacă $i = 9$ și dacă știm că

- $v[0] = v[2] = v[7]$
- $v[1] = v[3] = v[4] = v[6]$
- $v[5] = v[8]$

atunci vom colecta indicii 0, 1 și 5. Subliniem că elementele $v[0]$, $v[1]$ și $v[5]$ sînt distincte. Cu alte cuvinte, vectorul de frecvențe pentru subșirul $(v[0], v[1], v[5])$ va fi (1, 1, 1). Acum, dacă adăugăm elementul $v[9]$ la acest subșir apar două posibilități:

1. Fie vectorul de frecvențe rezultat este (1, 1, 1, 1), caz în care $v[9]$ este o valoare nouă, diferită de $v[0 \ldots 8]$.
2. Fie vectorul de frecvențe rezultat este (1, 1, 2), caz în care $v[9]$ este egal cu unul dintre elementele anterioare.

Astfel, putem căuta binar cel mai scurt prefix al subșirului format din indicii primelor apariții care, împreună cu poziția $v[i]$, produce o frecvență de 2. Pentru aceasta, vom face `dreseaza_cal` să returneze 0 sau 1 după cum vectorul de frecvențe se termină sau nu cu o frecvență 2.

Această soluție ajunge la 8 000 de interogări, iar principalul său neajuns este că `dreseaza_cal` returnează doar două valori posibile, indiferent de valoarea lui k. Cu alte cuvinte, soluția nu încearcă să partiționeze spațiul de posibilități suficient de rapid.

43.12.3 Soluția optimă

Eu nu văd niciun mod de a împinge soluția anterioară mai departe. Prin natura ei, căutarea binară utilizează doar două valori din cele k, pentru că pune întrebări de tip da/nu.

În loc de asta, ne mulțumim să baleiem vectorul și să aflăm la fiecare pas histograma (distribuția frecvențelor). De exemplu, dacă primele 5 elemente sînt distincte, vom avea 5 valori cu frecvența 1. Dacă al 6-lea element este egal cu unul anterior, nu insistăm să aflăm **cu care**. Ne interesează doar noua histogramă: 4 frecvențe de 1 și o frecvență de 2.

Un indiciu valoros este că, pentru $k = 1\,000$, avem doar 999 de interogări. Cu alte cuvinte, pentru fiecare poziție de la 1 la $n-1$ trebuie să facem exact o interogare. Atunci, haideți să presupunem că am pus întrebări pentru elementele $0, 1, \ldots, i-1$ și să gîndim soluția prin prisma elementului i. Cînd îl procesăm, exact o frecvență va crește cu exact o unitate față de prefixul anterior. Dacă elementul este la prima apariție, un 0 devine 1. Altfel un 1 devine 2 sau un 2 devine 3 etc. Cum decidem? Am vrea să facem o interogare astfel încît toate cele k răspunsuri posibile să fie relevante. De exemplu, răspunsul 0 să indice că o frecvență 0 devine 1, răspunsul 1 să indice că o frecvență 1 devine 2 etc.

Ideal, am putea dresa calul să raporteze numărul de perechi egale în prefixul interogat. De exemplu, șirul $(7, 8, 8, 9, 9, 9)$ conține 4 perechi (perechea 8-8 și 3 perechi 9-9). Dacă extindem șirul la $(7, 8, 8, 9, 9, 9, 8)$, el va conține 6 perechi, cu 2 mai multe decît anteriorul, ceea ce arată că o frecvență a crescut de la 2 la 3. Așadar, de acum încolo cînd spunem „răspuns” ne referim la perechile formate de elementul curent, calculate ca diferența dintre răspunsul dat de cal și numărul de perechi cunoscute din interogarea pe care noi am format-o.

Partea bună este că nu avem nevoie să știm șirul ca să aflăm numărul de perechi, ci doar frecvențele. Deci `dreseaza_cal` doar însumează $f \cdot (f-1)/2$ pentru fiecare frecvență f. Dar trebuie clarificate multe aspecte.

Cazul cu puține valori distincte

Întîi, suma frecvențelor va depăși k. Cînd $k = n$, putem opera modulo k, deoarece la orice pas creșterea numărului de perechi nu va depăși $n-1$. Dar, cînd k este mic, nu putem diferenția răspunsul. De exemplu, dacă $k = 4$ și creșterea este 1, atunci s-ar putea ca o frecvență de 1 să fi devenit 2 sau una de 5 să fi devenit 6 etc. Putem reduce confuzia dacă numărul de elemente distincte este cel mult $k-1$. De exemplu, să considerăm următorul șir cu $k = 4$, frecvențele 1, 5, 9 și elementul deocamdată necunoscut x:

```
8 13 13 13 13 13 20 20 20 20 20 20 20 20 20 x
```

Vom construi interogarea astfel:

- Elementul cu frecvența 1 îl luăm o dată.
- Elementul cu frecvența 5 îl luăm de două ori.
- Elementul cu frecvența 9 îl luăm de trei ori.
- Îl adăugăm și pe x.

Rezultă interogarea 8 13 13 20 20 20 x (reamintesc că vom trimite poziții, nu valori, căci acesta este protocolul). Interpretăm răspunsul astfel:

- Dacă este 0, x este un element nou întîlnit.
- Dacă este 1, x este 8.
- Dacă este 2, x este 13.
- Dacă este 3, x este 20.

Așadar, deși frecvențele sînt mari, putem ști exact ce fel de element este x întrucît avem mai puțin de k elemente distincte.

Cazul cu multe valori distincte

Dacă numărul de elemente distincte este cel puțin k, vom folosi mai multe interogări, dar cum? Vom partiționa lista de frecvențe în k grupe cît mai egale. Apoi vom construi o interogare astfel încît, în funcție de răspuns, să putem restrînge căutarea la una dintre grupe. De exemplu, fie $k = 5$ și următoarea listă de frecvențe (inclusiv 0):

```
0 1 2 5 8 10 12 15 20 23 25 28 33
```

Vom partiționa elementele în grupele [0 1 2], [5 8 10], [12 15 20], [23 25] și [28 33]. Vom construi interogarea astfel:

- Elementele cu frecvențe 0, 1, 2 nu le luăm.
- Elementele cu frecvențe 5, 8, 10 le luăm o dată.
- Elementele cu frecvență 12, 15, 20 le luăm de 2 ori.
- Elementele cu frecvență 23, 25 le luăm de 3 ori.
- Elementele cu frecvență 28, 33 le luăm de 4 ori.
- Adăugăm și poziția curentă.

Acum, dacă răspunsul este 4, am redus posibilitățile la două: o frecvență de 28 a devenit 29 sau o frecvență de 33 a devenit 34. Facem a doua (și ultima) interogare pentru a diferenția aceste cazuri:

- Elementele cu frecvență 28 nu le luăm.
- Elementele cu frecvență 33 le luăm o dată.

Acum, dacă răspunsul este 0, atunci explicația este că un element de frecvență 28 a crescut la 29. Altfel răspunsul este 1, ceea ce înseamnă că un element de frecvență 33 a crescut la 34.

Localizarea elementelor cu o frecvență cerută

Mai este un ultim aspect, de care eu nu m-am prins pînă nu am citit soluția aceasta. Ca să adăugăm la interogare, de exemplu, elementele cu frecvențele 28 și 33 de 4 ori, trebuie să cunoaștem pozițiile aparițiilor acelor elemente. Dar cum? Din nou, nu este practic să încercăm să construim lista de poziții care conțin elemente egale.

Soluția este să notăm, pe fiecare poziție procesată, concluzia interogărilor: frecvența la care a ajuns acel element. Dacă un element a ajuns la frecvență 28, el a ajuns cîndva anterior și la frecvențele 1, 2, …, 27. Deci, pentru prima interogare de mai sus, putem proceda astfel:

- Tipărim toate pozițiile pe care am ajuns la frecvență 5.
- Tipărim toate pozițiile pe care am ajuns la frecvență 12.
- Tipărim toate pozițiile pe care am ajuns la frecvență 23.
- Tipărim toate pozițiile pe care am ajuns la frecvență 28.

Atunci obținem ce dorim, adică:

- Pozițiile elementelor cu frecvență 0, 1 sau 2 nu vor fi tipărite.
- Pozițiile elementelor cu frecvență 5, 8 sau 10 vor fi tipărite o singură dată, respectiv la momentul la care ating frecvența 5.
- …
- Pozițiile elementelor cu frecvență 28 sau 33 vor fi tipărite de 4 ori, respectiv la momentele la care ating frecvențele 5, 12, 23 și 28.

Detaliu de implementare

Evaluatorul ne impune să nu folosim memorie globală, ceea ce are sens, deoarece cu memorie globală funcțiile `descuie_grajdul` și `dreseaza_cal` ar putea comunica direct între ele, șuntînd funcția `intreaba_cal` și făcînd problema banală. Din păcate, evaluatorul este prea zelos. El interzice complet declararea de `struct`-uri sau clase (mă întreb de ce!). De aceea, implementarea anexată este lizibilă, dar nu se va compila. Am rescris sursa pentru a obține 100p, dar o consider ilizibilă (cel puțin după standardele mele).

43.13 Problema 6 de Pentagrame (Lot 2024)

enunț • sursă

De această problemă nu sînt mîndru. Am scos-o la capăt, dar este lungă și lentă. Soluția oficială folosește doar un arbore de intervale. Eu folosesc și un AIB modificat.

43.13.1 Observații teoretice

Fie A, B, C și D cadranele de sus-stînga, sus-dreapta, jos-stînga și respectiv jos-dreapta. Să considerăm un x fixat și să vedem cum evoluează partiționările pe măsură ce y crește. Cînd $y = 1$, în mod evident minimul va fi 0 și se va afla în C sau în D, iar maximul va fi pozitiv și se va afla în A sau în B. Cînd $y = n - 1$ situația se va inversa. Așadar, trebuie să existe:

- o primă coordonată y_1 unde minimul urcă (trece din C/D în A/B);
- o primă coordonată y_2 unde maximul coboară (trece din A/B în C/D).

Mai mult decît atît, odată ce minimul urcă, el va rămîne sus, căci A și B continuă să scadă, pe cînd C și D continuă să crească. Similar pentru maxim.

Mai este o observație-cheie care mie mi-a scăpat pînă după încheierea taberei de la Cluj. 😵 Să spunem că la coordonata y_1 minimul urcă în cadranul A. Atunci pe intervalul dintre y_1 și y_2 minimul va rămîne în A, iar maximul va rămîne în B pînă cînd maximul va coborî. Cele două nu pot face schimb de locuri, căci A continuă să scadă și ar trebui ca B să scadă și mai mult. Or, în clipa în care B nu mai este maxim, înseamnă că maximul a coborît în C/D.

Așadar iau naștere patru cazuri, tratate similar:

1. $y_1 \leq y_2$, adică minimul urcă primul.
   (a) Minimul este în A, iar maximul este în B.
   (b) Minimul este în B, iar maximul este în A.
2. $y_1 > y_2$, adică maximul coboară primul.
   (a) Minimul este în C, iar maximul este în D.
   (b) Minimul este în D, iar maximul este în C.

Nu trebuie să uităm despre ce se întîmplă în afara intervalului $[y_1, y_2]$. Aceste cazuri sînt mai ușoare. Pentru $y < y_1$, costul partiționării crește pe măsură ce y scade, căci A și C se dezechilibrează, la fel și B și D. Similar, pentru $y > y_2$, costul partiționării crește pe măsură ce y crește. Așadar, putem testa costul partiționării doar în punctele $y_1 - 1$ și $y_2 + 1$. Rămîne partea grea a problemei, testarea între y_1 și y_2.

43.13.2 Structuri de date

Pentru a afla costul partiționării într-un punct arbitrar, eu am folosit două AIB-uri pentru valorile punctelor înainte, respectiv după x. Cu aceeași structură și cu o căutare binară specifică AIB-ului putem căuta pozițiile unde minimul urcă, respectiv unde maximul coboară.

Să considerăm cazul 2.2 din cele patru de mai sus. Care este costul minim al unei partiționări? Avînd în vedere că maximul este în C și minimul în D, este suficient să menținem valoarea $C - D$ la fiecare y și să aflăm minimul din $C - D$ pe intervalul $[y_1, y_2]$. Sună a arbore de intervale! Haideți să vedem și cum facem actualizarea.

Vă reamintesc că C și D sînt sume parțiale ale greutăților pentru coordonate $\leq y$, la stînga și respectiv la dreapta lui x. Cînd x avansează cu 1, toate punctele care au coordonata egală cu noul x trec din D în C. Pentru un punct $P(x_p, y_p, val_p)$, diferența între C și D crește cu $2 \cdot val_p$, iar această creștere afectează cadranele C și D pentru toate partiționările la coordonate $\geq y_p$.

Așadar, avem într-adevăr nevoie de un arbore de intervale cu adăugare pe interval și minim pe interval. Pentru cazul cu minimul în C și maximul în D, toate valorile din aint vor fi negative, deci vom avea nevoie de „cea mai puțin negativă” valoare, adică de maxim. Iar pentru minimul și maximul în A și B avem nevoie tot de un arbore de intervale cu sumă pe interval, doar că acum o operație la coordonata y_p va afecta coordonatele $\leq y_p$.

Ca detaliu de implementare, am implementat arborele de intervale și iterativ. Spre surprinderea mea, sursa este de două ori mai lentă!

Postfață

Sper că parcurgerea acestui volum a fost și pentru voi o plimbare fascinantă prin lumea algoritmilor și a structurilor de date, după cum și pentru mine a fost o imensă plăcere să îl scriu.

Dar orizonturile sînt mai largi de atît. De aceea, aș vrea să vă las cîteva gînduri de final. De fapt unul singur.

Noi nu facem haine, înghețată, case sau bărci. Toate aceste lucruri sînt bune, dar influența lor asupra lumii în ansamblu este limitată. Noi scriem software; iar software-ul are potențialul să schimbe modul în care lumea gîndește și se comportă, cu efecte pe ani sau pe generații.

Iată cîteva exemple de efecte ale informatizării societății, cu bune și cu rele.

Planeta este mai mică. Comunicarea oriunde pe glob este facilă. Cînd eram eu student în Boston, sunam acasă cu $2/minut. Astăzi este gratis. Dar ușurința accesului depinde și de cooperarea furnizorilor (ISPs). Furnizorii nu vor ca Internetul să fie considerat o *utilitate publică*, la fel ca apa sau curentul. Ei vor să aibă monopolul asupra infrastructurii, dar să fie liberi să practice orice tarife doresc, cum ar fi tarife diferențiate pentru fiecare aplicație. De exemplu, în 2025 a picat net neutrality, încercarea democraților din SUA de a reglementa furnizorii ca pe utilități. Vom vedea care vor fi efectele

Avem standarde și protocoale pentru tot. Vă dați seama ce coșmar ar fi dacă fiecare site ar avea propriul său protocol? Dacă am avea nevoie de programe diferite ca să vizităm Wikipedia sau YouTube? Iar standardele (fie ele HTTP, HTML, XML, PDF, ODF...) există pentru că oamenii au militat pentru asta. Rezultatul este un internet descentralizat și (în general) universal accesibil. Necazul este că această deschidere se năruie în prezent. De exemplu, Facebook a prosperat la începuturi pentru că le-a oferit primilor săi utilizatori interfațarea cu Myspace, o fostă rețea socială. La rîndul său Facebook, acum că este într-o poziție dominantă, face eforturi considerabile ca să nu le permită altor nou-veniți să se integreze cu Facebook. Adobe încearcă de ani buni să monopolizeze formatul PDF, să încarce protocolul cu extensii nelibere care pot fi citite doar cu Adobe Reader (și nu sub GNU/Linux).

Avem calculatoare interoperabile (în general). Dacă descărcăm distribuția oficială a sistemului nostru de operare favorit, o putem instala pe orice nou calculator. Pericolul provine de la treacherous computing (numit impropriu și trusted computing). Acesta promite să ne ferească calculatorul de software neautorizat. Dar subtextul este „neautorizat de producător”. Sistemul TPM (treacherous platform module) face calculatorul să asculte de producător în detrimentul

utilizatorului.

Avem software liber pentru majoritatea nevoilor informatice. Aceasta deși toată lumea se întreabă, de 50 de ani, „de unde cîștigă oamenii ăștia bani?”. Întrebarea este însă prost pusă. Oamenii scriu software și aleg să-l publice sub o licență liberă pentru o mulțime de motive care nu țin de bani. Oamenii fac fapte bune pentru că e în natura umană. Sau pentru că au primit ceva gratuit la rîndul lor și se simt datori. Sau pentru că vor să-și cîștige nemurirea. Sau ca să cîștige inima unei fete / unui băiat. Dar motivul pentru care avem software liber pentru majoritatea nevoilor este că diverși oameni cu spirit de meștereală au dezasamblat drivere binare, au făcut *reverse engineering* la formate și la protocoale și au publicat versiuni libere ale acelorași unelte. Pericolul, astăzi, provine din DRM (*digital restrictions management*, numit impropriu și *digital rights management*). DRM nu este o unealtă informatică. Este o unealtă legală care incriminează încercarea de a ocoli anumite restricții digitale.

Avem, în era digitală, acces la mai multe opere artistice și tehnice decît orice altă generație. Pericolul vine din durata tot mai lungă a restricțiilor de copyright și de patente. În plus, în cazul software-ului, există pericolul licențelor draconice. Mulți dintre noi le acceptăm pe necitite, dar mesajul lor este dur: ne fac să promitem să fim niște prieteni răi, să nu împărțim acel produs cu nimeni. Un gest firesc acum 40 de ani (să-i împrumuți o carte sau o dischetă *floppy* prietenului tău) a fost îngreunat tehnic și stigmatizat din punct de vedere moral, transformîndu-i în criminali pe cei care știu să fenteze cătușele digitale. Voi sînteți generația digitală, care poate copia terabytes întregi cu o apăsare de taste, și pe care totuși marii deținători de copyright încearcă să vă convingă, prin îndoctrinare zilnică, că copierea este un act rușinos.

Sîntem generația cea mai avansată științific. Pe mine acest lucru mă umple de respect față de lungul lanț al oamenilor de știință care au trăit și au murit pentru ca noi să putem cunoaște universul în măsura în care îl cunoaștem astăzi. Dar, în același timp, Internetul de astăzi constă dintr-un *whitelist* – o mînă de site-uri cu reputație din care ne putem informa corect, – și o hazna de site-uri dezinformatoare și antiștiințifice.

* * *

De ce am enumerat aceste lucruri bune și rele? Pentru că voi sînteți cu toții elevi de excepție și veți avea porți deschise la toate companiile notabile. Vă încurajez ca, oriunde vă veți angaja, să nu renunțați la filtrul moral. Să vă puneți, la fiecare nou proiect, întrebarea: acest feature pe care mi-a cerut compania să-l implementez este un pas în direcția bună, sau un pas în direcția greșită?

Dacă angajatorul vostru vă cere să lucrați la o aplicație care neagă încălzirea globală sau teoria evoluției; sau care identifică persoane care au discutat pe Internet despre anumite cuvinte-cheie ca să-i predea autorităților; sau care verifică dacă piesele de schimb folosite pentru un telefon sînt originale și refuză să le folosească în caz contrar; sau care programează un automobil să iasă în stradă și să sune din claxon imediat ce proprietarul a omis să plătească rata, pentru ca recuperatorul să poată găsi mai ușor automobilul; în toate aceste cazuri sper să luați în calcul să vă căutați de lucru altundeva.

Vă promit că nu veți fi muritori de foame dacă vă veți autoimpune niște standarde de etică. Veți fi, poate, nevoiți să acceptați o ofertă cu 10% mai mică. Dar o conștiință curată merită deranjul. 🙂

Partea XII

Anexă: Cod-sursă

Anexa 1

Principii - Bune practici în programare

1.1 Generator de teste

înapoi

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

const int MAX_T = 100'000;

bool used[MAX_T + 1];
int n, q, max_p, max_t;

void read_command_line_args(int argc, char** argv) {
  if (argc != 5) {
    fprintf(stderr, "Usage: gen <n> <q> <max_p> <max_t>\n");
    exit(1);
  }

  n = atoi(argv[1]);
  q = atoi(argv[2]);
  max_p = atoi(argv[3]);
  max_t = atoi(argv[4]);
}

void init_rng() {
  struct timeval time;
  gettimeofday(&time, NULL);
  int seed = time.tv_sec * 1'000'000 + time.tv_usec;
  srand(seed);
}

int rand2(int min, int max) {
  return min + rand() % (max - min + 1);
```

```
}

void generate_p() {
  printf("%d %d\n", n, q);
  for (int i = 0; i < n; i++) {
    printf("%d ", rand2(1, max_p));
  }
  printf("\n");
}

void generate_t() {
  for (int i = 0; i < n; i++) {
    int t;
    do {
      t = rand2(1, max_t);
    } while (used[t]);
    printf("%d ", t);
    used[t] = true;
  }
  printf("\n");
}

void generate_queries() {
  for (int i = 0; i < q; i++) {
    int l = rand2(1, n);
    int r = rand2(1, n);
    if (l > r) {
      int tmp = l;
      l = r;
      r = tmp;
    }
    printf("%d %d\n", l, r);
  }
}

int main(int argc, char** argv) {
  read_command_line_args(argc, argv);
  init_rng();

  generate_p();
  generate_t();
  generate_queries();

  return 0;
}
```

Anexa 2

Arbori de intervale

2.1 Problema Xenia and Bit Operations (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1 << 17;

struct segment_tree {
  int v[2 * MAX_N];
  int n;

  void init(int n) {
    this->n = n;
  }

  void set(int pos, int val) {
    v[pos + n] = val;
  }

  void build() {
    bool is_or = true;
    for (int i = n - 1; i; i--) {
      int l = 2 * i, r = 2 * i + 1;
      v[i] = is_or ? (v[l] | v[r]) : (v[l] ^ v[r]);
      if ((i & (i - 1)) == 0) {
        is_or = !is_or;
      }
    }
  }

  int update(int pos, int val) {
    pos += n;
    v[pos] = val;
```

```
    bool is_or = true;
    for (pos /= 2; pos; pos /= 2) {
      int l = 2 * pos, r = 2 * pos + 1;
      v[pos] = is_or ? (v[l] | v[r]) : (v[l] ^ v[r]);
      is_or = !is_or;
    }
    return v[1];
  }
};

segment_tree st;
int num_queries;

void read_array() {
  int n;
  scanf("%d %d", &n, &num_queries);
  n = 1 << n;
  st.init(n);

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    st.set(i, x);
  }

  st.build();
}

void process_queries() {
  while (num_queries--) {
    int pos, val;
    scanf("%d %d", &pos, &val);
    int root = st.update(pos - 1, val);
    printf("%d\n", root);
  }
}

int main() {
  read_array();
  process_queries();

  return 0;
}
```

2.2 Problema Distinct Characters Queries (Codeforces)

❮ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>

const int MAX_N = 1 << 17;
const int T_UPDATE = 1;

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

struct segment_tree {
  int v[2 * MAX_N];
  int n;

  void init(char* s) {
    int len = strlen(s);
    n = next_power_of_2(len);

    for (int i = 0; i < len; i++) {
      v[i + n] = 1 << (s[i] - 'a');
    }

    build();
  }

  void build() {
    for (int i = n - 1; i; i--) {
      v[i] = v[2 * i] | v[2 * i + 1];
    }
  }

  void update(int pos, char val) {
    pos += n;
    v[pos] = 1 << (val - 'a');

    for (pos /= 2; pos; pos /= 2) {
      v[pos] = v[2 * pos] | v[2 * pos + 1];
    }
  }

  int popcount_query(int l, int r) {
    int mask = 0;

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        mask |= v[l++];
```

```
      }
      l >>= 1;

      if (!(r & 1)) {
        mask |= v[r--];
      }
      r >>= 1;
    }

    return __builtin_popcount(mask);
  }
};

segment_tree st;

void read_string() {
  char s[MAX_N + 1];
  scanf("%s", s);
  st.init(s);
}

void process_ops() {
  int num_ops, type;
  scanf("%d", &num_ops);

  while (num_ops--) {
    scanf("%d", &type);

    if (type == T_UPDATE) {
      int pos;
      char val;
      scanf("%d %c", &pos, &val);
      st.update(pos - 1, val);
    } else {
      int l, r;
      scanf("%d %d", &l, &r);
      int num_distinct = st.popcount_query(l - 1, r - 1);
      printf("%d\n", num_distinct);
    }
  }
}

int main() {
  read_string();
  process_ops();

  return 0;
}
```

2.3 Problema K-query (SPOJ)

înapoi

Sursă cu arbori de intervale.

```
// Complexitate: O(n log n)
#include <algorithm>
#include <stdio.h>

const int MAX_N = 32'768;
const int MAX_Q = 200'000;

int next_power_of_2(int n) {
  while (n & (n - 1)) {
    n += (n & -n);
  }

  return n;
}

struct segment_tree {
  short v[2 * MAX_N];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void set(int pos) {
    pos += n;
    while (pos) {
      v[pos]++;
      pos /= 2;
    }
  }

  short range_count(int l, int r) {
    int sum = 0;

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        sum += v[l++];
      }
      l >>= 1;

      if (!(r & 1)) {
```

```
        sum += v[r--];
      }
      r >>= 1;
    }

    return sum;
  }
};

struct elem {
  int val;
  short pos;
};

struct query {
  short left, right;
  int val;
  int orig_pos;
};

elem v[MAX_N];
query q[MAX_Q];
// Mai curat ar fi să punem răspunsurile în struct. Dar atunci ar trebui să
// sortăm interogările încă o dată la final.
short answer[MAX_Q];
segment_tree st;
int n, num_queries;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i].val);
    v[i].pos = i + 1;
  }

  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%hd %hd %d", &q[i].left, &q[i].right, &q[i].val);
    q[i].orig_pos = i;
  }
}

void sort_array_by_val() {
  std::sort(v, v + n, [](elem a, elem b) {
    return a.val > b.val;
  });
}

void sort_queries_by_val() {
  std::sort(q, q + num_queries, [](query& a, query& b) {
```

```
    return a.val > b.val;
  });
}

void process_queries() {
  st.init(n);

  int j = 0;
  for (int i = 0; i < num_queries; i++) {
    while ((j < n) && (v[j].val > q[i].val)) {
      st.set(v[j++].pos);
    }
    answer[q[i].orig_pos] = st.range_count(q[i].left, q[i].right);
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", answer[i]);
  }
}

int main() {
  read_data();
  sort_array_by_val();
  sort_queries_by_val();
  process_queries();
  write_answers();

  return 0;
}
```

Sursă cu arbori indexați binar.

```
// Complexitate: O(n log n)
#include <algorithm>
#include <stdio.h>

const int MAX_N = 30'000;
const int MAX_Q = 200'000;

struct fenwick_tree {
  short v[MAX_N + 1];
  int n;

  void init(int n) {
    this->n = n;
  }
```

```
  void set(int pos) {
    do {
      v[pos]++;
      pos += pos & -pos;
    } while (pos <= n);
  }

  short prefix_count(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
    }
    return sum;
  }

  short range_count(int l, int r) {
    return prefix_count(r) - prefix_count(l - 1);
  }
};

struct elem {
  int val;
  short pos;
};

struct query {
  short left, right;
  int val;
  int orig_pos;
};

elem v[MAX_N];
query q[MAX_Q];
// Mai curat ar fi să punem răspunsurile în struct. Dar atunci ar trebui să
// sortăm interogările încă o dată la final.
short answer[MAX_Q];
fenwick_tree fen;
int n, num_queries;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i].val);
    v[i].pos = i + 1;
  }

  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%hd %hd %d", &q[i].left, &q[i].right, &q[i].val);
```

```
    q[i].orig_pos = i;
  }
}

void sort_array_by_val() {
  std::sort(v, v + n, [](elem a, elem b) {
    return a.val > b.val;
  });
}

void sort_queries_by_val() {
  std::sort(q, q + num_queries, [](query& a, query& b) {
    return a.val > b.val;
  });
}

void process_queries() {
  fen.init(n);

  int j = 0;
  for (int i = 0; i < num_queries; i++) {
    while ((j < n) && (v[j].val > q[i].val)) {
      fen.set(v[j++].pos);
    }
    answer[q[i].orig_pos] = fen.range_count(q[i].left, q[i].right);
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", answer[i]);
  }
}

int main() {
  read_data();
  sort_array_by_val();
  sort_queries_by_val();
  process_queries();
  write_answers();

  return 0;
}
```

2.4 Problema Sereja and Brackets (Codeforces)

‹ înapoi • versiune online

```cpp
// Complexitate: O(n + q log n).
#include <stdio.h>
#include <string.h>

const int MAX_N = 1 << 20;

int next_power_of_2(int n) {
  while (n & (n - 1)) {
    n += (n & -n);
  }

  return n;
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

struct node {
  int matched, open, closed;

  node combine(node& other) {
    int new_matches = min(open, other.closed);
    return {
      .matched = matched + other.matched + 2 * new_matches,
      .open = open + other.open - new_matches,
      .closed = closed + other.closed - new_matches,
    };
  }
};

struct segment_tree {
  node v[2 * MAX_N];
  int n;

  void init(char* s) {
    int len = strlen(s);
    this->n = next_power_of_2(len);

    for (int i = 0; i < len; i++) {
      v[i + this->n] = {
        .matched = 0,
        .open = (s[i] == '('),
        .closed = (s[i] == ')'),
      };
    }

    build();
  }
```

```
  void build() {
    for (int i = n - 1; i; i--) {
      v[i] = v[2 * i].combine(v[2 * i + 1]);
    }
  }

  int query(int l, int r) {
    node left = { 0, 0, 0 };
    node right = { 0, 0, 0 };

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        left = left.combine(v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        right = v[r--].combine(right);
      }
      r >>= 1;
    }

    node answer = left.combine(right);
    return answer.matched;
  }
};

segment_tree st;

void read_string() {
  char s[MAX_N + 1];
  scanf("%s", s);
  st.init(s);
}

void process_ops() {
  int num_ops;
  scanf("%d", &num_ops);

  while (num_ops--) {
    int l, r;
    scanf("%d %d", &l, &r);
    printf("%d\n", st.query(l - 1, r - 1));
  }
}
```

```
int main() {
  read_string();
  process_ops();

  return 0;
}
```

2.5 Problema Copying Data (Codeforces)

înapoi • versiune online

```
// Complexitate: O(q log n)
#include <stdio.h>

const int MAX_N = 1 << 17;
const int T_COPY = 1;

struct change {
  int time;
  int shift;
};

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segment_tree {
  change v[2 * MAX_N];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  change query(int pos) {
    change latest = { 0, 0 };
    for (pos += n; pos; pos >>= 1) {
      if (v[pos].time > latest.time) {
        latest = v[pos];
      }
    }
    return latest;
  }

  void update(int l, int r, change c) {
    l += n;
    r += n;
```

```
    while (l <= r)  {
      if (l & 1) {
        v[l++] = c;
      }
      l >>= 1;

      if (!(r & 1)) {
        v[r--] = c;
      }
      r >>= 1;
    }
  }
};

segment_tree st;
int a[MAX_N], b[MAX_N];
int n, num_ops;

void read_arrays() {
  scanf("%d %d", &n, &num_ops);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
  for (int i = 0; i < n; i++) {
    scanf("%d", &b[i]);
  }
}

void process_ops() {
  st.init(n);
  int type, x, y, k;
  for (int op = 1; op <= num_ops; op++) {
    scanf("%d", &type);
    if (type == T_COPY) {
      scanf("%d %d %d", &x, &y, &k);
      x--;
      y--;
      change c = { .time = op, .shift = x - y };
      st.update(y, y + k - 1, c);
    } else {
      scanf("%d", &x);
      x--;
      change latest = st.query(x);
      int val = latest.time ? a[x + latest.shift] : b[x];
      printf("%d\n", val);
    }
  }
}

int main() {
```

```
  read_arrays();
  process_ops();

  return 0;
}
```

2.6 Problema PHF (FMI No Stress 2013)

înapoi • versiune online

```
#include <stdio.h>

typedef unsigned char byte;

const int MAX_N = 1'000'000;
const int MAX_SEGTREE_NODES = 1 << 21;

const int IDENTITY = 3;
const byte CROSS_TABLE[4][3] = {
  { 0, 1, 0 }, // P
  { 1, 1, 2 }, // H
  { 0, 2, 2 }, // F
  { 0, 1, 2 }, // identitate
};

char FROM_CHAR[27] = ".....2.1.......0..........";
char TO_CHAR[4] = "PHF";

byte from_char(char c) {
  return FROM_CHAR[c - 'A'] - '0';
}

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segment_tree_node {
  byte f[3];

  void make_leaf(byte c) {
    for (int i = 0; i < 3; i++) {
      f[i] = CROSS_TABLE[c][i];
    }
  }

  segment_tree_node compose(segment_tree_node other) {
    return {
      other.f[f[0]],
```

```
      other.f[f[1]],
      other.f[f[2]],
    };
  }
};

struct segment_tree {
  segment_tree_node v[MAX_SEGTREE_NODES];
  int n;

  void init(char* s, int _n) {
    n = next_power_of_2(_n);
    for (int i = 0; i < _n; i++) {
      v[n + i].make_leaf(s[i]);
    }

    for (int i = _n; i < n; i++) {
      v[n + i].make_leaf(IDENTITY);
    }

    for (int i = n - 1; i; i--) {
      v[i] = v[2 * i].compose(v[2 * i + 1]);
    }
  }

  void update(int pos, byte b) {
    pos += n;
    v[pos].make_leaf(b);
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = v[2 * pos].compose(v[2 * pos + 1]);
    }
  }

  char get_value(byte input) {
    return TO_CHAR[v[1].f[input]];
  }
};

char s[MAX_N + 1];
segment_tree st;
int n, num_queries;

void read_string() {
  scanf("%d %d ", &n, &num_queries);
  for (int i = 0; i < n; i++) {
    s[i] = from_char(getchar());
  }
}

void process_updates() {
```

```
  int pos;

  while (num_queries--) {
    scanf("%d ", &pos);
    pos--;
    s[pos] = from_char(getchar());
    if (pos) {
      st.update(pos - 1, s[pos]);
    }
    putchar(st.get_value(s[0]));
  }
}

int main() {
  read_string();
  st.init(s + 1, n - 1);
  putchar(st.get_value(s[0]));
  process_updates();
  putchar('\n');

  return 0;
}
```

2.7 Problema Points (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <set>
#include <stdio.h>

const int MAX_N = 1 << 18;
const int INFINITY = 2'000'000'000;
const int T_ADD = 1;
const int T_REMOVE = 2;
const int T_FIND = 3;

struct query {
  char type;
  int x, y;
};

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

int max(int x, int y) {
  return (x > y) ? x : y;
```

```
}

struct max_segment_tree {
  int v[2 * MAX_N];
  int n;

  void init(int n) {
    n++; // santinelă infinită la final
    n = next_power_of_2(n);
    this->n = n;

    for (int i = 1; i < 2 * n; i++) {
      v[i] = -1; // fără puncte
    }

    set(n - 1, INFINITY);
  }

  void set(int pos, int val) {
    pos += n;
    if (v[pos] != val) { // optimizare de viteză (nu ajută)
      v[pos] = val;
      for (pos /= 2; pos; pos /= 2) {
        v[pos] = max(v[2 * pos], v[2 * pos + 1]);
      }
    }
  }

  // Returnează prima poziție după pos unde valoarea depășește val.
  int find_first_after(int pos, int val) {
    pos += n;
    pos++;

    // Urcă pînă cînd găsim un maxim mai mare decît val.
    while (v[pos] <= val) {
      if (pos & 1) {
        pos++;
      } else {
        pos >>= 1;
      }
    }

    // Coboară spre valoarea dorită, mergînd la stînga oricînd putem.
    while (pos < n) {
      pos = (v[2 * pos] > val) ? (2 * pos) : (2 * pos + 1);
    }

    return pos - n;
  }
```

```
};

query q[MAX_N];
int pos[MAX_N]; // pentru normalizare
int orig_x[MAX_N];
max_segment_tree segtree;
std::set<int> whys[MAX_N];
int n;

void read_queries() {
  char s[10];
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%s %d %d", s, &q[i].x, &q[i].y);
    switch (s[0]) {
      case 'a': q[i].type = T_ADD; break;
      case 'r': q[i].type = T_REMOVE; break;
      default: q[i].type = T_FIND;
    }
  }
}

void normalize_x() {
  for (int i = 0; i < n; i++) {
    pos[i] = i;
  }

  std::sort(pos, pos + n, [](int a, int b) {
    return q[a].x < q[b].x;
  });

  int ptr = 0;
  orig_x[ptr] = q[pos[0]].x;
  for (int i = 0; i < n; i++) {
    if (q[pos[i]].x != orig_x[ptr]) {
      orig_x[++ptr] = q[pos[i]].x;
    }
    q[pos[i]].x = ptr;
  }
}

void add_query(int x, int y) {
  whys[x].insert(y);
  segtree.set(x, *whys[x].rbegin());
}

void remove_query(int x, int y) {
  whys[x].erase(y);
  if (whys[x].empty()) {
    segtree.set(x, -1);
```

```
  } else {
    segtree.set(x, *whys[x].rbegin());
  }
}

void find_query(int x, int y) {
  int first = segtree.find_first_after(x, y);
  if (first < n) {
    int first_above = *whys[first].upper_bound(y);
    printf("%d %d\n", orig_x[first], first_above);
  } else {
    printf("-1\n");
  }
}

void process_queries() {
  segtree.init(n);
  for (int i = 0; i < n; i++) {
    switch (q[i].type) {
      case T_ADD: add_query(q[i].x, q[i].y); break;
      case T_REMOVE: remove_query(q[i].x, q[i].y); break;
      default: find_query(q[i].x, q[i].y);
    }
  }
}

int main() {
  read_queries();
  normalize_x();
  process_queries();

  return 0;
}
```

2.8 Problema Medwalk (Lot 2025)

‹ înapoi • versiune online

```
// Complexitate: O((N + Q) log N log MAX_VAL).
//
// v2: Nu actualiza aint-ul de minime dacă valoarea nu s-a schimbat.
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_VALUE = 300'000;
const int MAX_POS_SEGTREE_NODES = 1 << 18;
```

```
const int MAX_VAL_SEGTREE_NODES = 1 << 20;
const int INF = 1'000'000;
const int OP_UPDATE = 1;

typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > set;

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct pair {
  int v[2];

  void set(int pos, int val) {
    v[pos] = val;
  }

  int get_min() {
    return min(v[0], v[1]);
  }

  int get_max() {
    return max(v[0], v[1]);
  }
};

pair mat[MAX_N];
int n, num_queries;

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

// Admite actualizări punctuale și interogări de minim pe interval.
struct min_segment_tree {
  int v[MAX_POS_SEGTREE_NODES];
  int n;

  void init(int size) {
    n = next_power_of_2(size);
```

```
  }

  void update(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = min(v[2 * pos], v[2 * pos + 1]);
    }
  }

  int range_min(int l, int r) {
    l += n;
    r += n;
    int result = INF;

    while (l <= r)  {
      if (l & 1) {
        result = min(result, v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = min(result, v[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

// Kudos https://stackoverflow.com/a/22075025/6022817
//
// Arbore de intervale pe valori. Fiecare nod care subîntinde valorile [x, y)
// reține un set cu pozițiile pe care apar acele valori.
struct val_segment_tree {
  set s[MAX_VAL_SEGTREE_NODES];
  int n;

  void init(int size) {
    n = next_power_of_2(size);
  }

  void insert(int pos, int val) {
    for (val += n; val; val /= 2) {
      s[val].insert(pos);
    }
  }

  void erase(int pos, int val) {
```

```
    for (val += n; val; val /= 2) {
      s[val].erase(pos);
    }
  }

  // Cîte poziții din [l, r] sînt ocupate de valori subîntinse de acest nod?
  int count_positions_between(int node, int l, int r) {
    return s[node].order_of_key(r + 1) - s[node].order_of_key(l);
  }

  // Contract: k este 0-based, iar [l, r] este interval închis.
  int kth_element(int k, int l, int r) {
    int node = 1;

    while (node < n) {
      int on_left = count_positions_between(2 * node, l, r);
      if (on_left > k) {
        node = 2 * node;
      } else {
        k -= on_left;
        node = 2 * node + 1;
      }
    }

    return node - n;
  }
};

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int r = 0; r < 2; r++) {
    for (int i = 0; i < n; i++) {
      int x;
      scanf("%d", &x);
      mat[i].set(r, x);
    }
  }
}

min_segment_tree maxima;
val_segment_tree occur;

void build_segment_trees() {
  maxima.init(n);
  for (int i = 0; i < n; i++) {
    maxima.update(i, mat[i].get_max());
  }

  occur.init(MAX_VALUE + 1);
  for (int i = 0; i < n; i++) {
```

```
    occur.insert(i, mat[i].get_min());
  }
}

void update(int row, int col, int val) {
  int old_min = mat[col].get_min();
  mat[col].set(row, val);
  int new_min = mat[col].get_min();

  maxima.update(col, mat[col].get_max());

  if (new_min != old_min) {
    occur.erase(col, old_min);
    occur.insert(col, new_min);
  }
}

int query(int left, int right) {
  int len = right - left + 2;
  int median_pos = (len - 1) / 2;
  int median = occur.kth_element(median_pos, left, right);
  int best_max = maxima.range_min(left, right);

  if (best_max >= median) {
    return median;
  } else {
    // best_max trebuie inserat înainte de median_pos. Răspunsul poate fi la
    // poziția median_pos - 1 sau poate fi chiar best_max.
    int prev = occur.kth_element(median_pos - 1, left, right);
    return max(prev, best_max);
  }
}

void process_queries() {
  while (num_queries--) {
    int type;
    scanf("%d", &type);
    if (type == OP_UPDATE) {
      int r, c, x;
      scanf("%d %d %d", &r, &c, &x);
      r--;
      c--;
      update(r, c, x);
    } else {
      int left, right;
      scanf("%d %d", &left, &right);
      left--;
      right--;
      printf("%d\n", query(left, right));
    }
```

```
  }
}

int main() {
  read_data();
  build_segment_trees();
  process_queries();

  return 0;
}
```

Anexa 3

Arbori de intervale cu propagare *lazy*

3.1 Problema Polynomial Queries (CSES)

înapoi

Sursă cu AINT iterativ.

```
// Complexitate: O(q log n)
//
// Metodă: menține un arbore de intervale care admite sume pe interval și
// adăugarea unei progresii pe interval.
#include <stdio.h>

const int MAX_SEGTREE_NODES = 1 << 19;
const int T_UPDATE = 1;

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

long long progression_sum(long long first, long long step, int len) {
  return first * len + step * (len - 1) * len / 2;
}

// Invarianți:
//
// 1. Valorile reale ale nodurilor subîntinse sînt valorile lor v respective
//    plus o progresie aritmetică definită prin first și step.
// 2. Valoarea v a unui nod nu include progresia nodului.
struct segment_tree_node {
  long long s;
  long long first, step;

  long long get_value(int size) {
    return s + progression_sum(first, step, size);
```

```
  }
};


struct segment_tree {
  segment_tree_node v[MAX_SEGTREE_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void set(int pos, int val) {
    v[pos + n].s = val;
  }


  void build() {
    for (int i = n - 1; i; i--) {
      v[i].s = v[2 * i].s + v[2 * i + 1].s;
    }
  }

  void push(int t, int size) {
    int l = 2 * t, r = 2 * t + 1;

    v[l].first += v[t].first;
    v[l].step += v[t].step;

    long long first_right = v[t].first + v[t].step * size / 2;
    v[r].first += first_right;
    v[r].step += v[t].step;

    v[t].s += progression_sum(v[t].first, v[t].step, size);
    v[t].first = 0;
    v[t].step = 0;
  }

  void push_path(int node, int size) {
    if (node) {
      push_path(node / 2, size * 2);
      push(node, size);
    }
  }

  void pull(int t, int size) {
    int l = 2 * t, r = 2 * t + 1;
    v[t].s = v[l].get_value(size / 2) + v[r].get_value(size / 2);
  }
```

```
void pull_path(int node) {
  for (int x = node / 2, size = 2; x; x /= 2, size <<= 1) {
    pull(x, size);
  }
}

void add_progression(int l, int r) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r, size = 1;
  push_path(l / 2, 2);
  push_path(r / 2, 2);

  while (l <= r)  {
    // Prima frunză subîntinsă de nodul x este x * size.
    if (l & 1) {
      v[l].first += l * size - orig_l + 1;
      v[l].step++;
      l++;
    }
    l >>= 1;

    if (!(r & 1)) {
      v[r].first += r * size - orig_l + 1;
      v[r].step++;
      r--;
    }
    r >>= 1;
    size <<= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

long long range_sum(int l, int r) {
  long long sum = 0;
  int size = 1;

  l += n;
  r += n;
  push_path(l / 2, 2);
  push_path(r / 2, 2);

  while (l <= r)  {
    if (l & 1) {
      sum += v[l++].get_value(size);
    }
    l >>= 1;
```

```
      if (!(r & 1)) {
        sum += v[r--].get_value(size);
      }
      r >>= 1;
      size <<= 1;
    }

    return sum;
  }

};

segment_tree st;
int n, num_ops;

void read_array() {
  scanf("%d %d", &n, &num_ops);
  st.init(n);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    st.set(i, x);
  }
  st.build();
}

void process_ops() {
  while (num_ops--) {
    int type, l, r;
    scanf("%d %d %d", &type, &l, &r);
    l--; r--;
    if (type == T_UPDATE) {
      st.add_progression(l, r);
    } else {
      printf("%lld\n", st.range_sum(l, r));
    }
  }
}

int main() {
  read_array();
  process_ops();

  return 0;
}
```

Sursă cu AINT recursiv.

```
// Complexitate: O(q log n)
```

```
//
// Metodă: menține un arbore de intervale care admite sume pe interval și
// adăugarea unei progresii pe interval.
#include <stdio.h>

const int MAX_N = 1 << 18;
const int T_UPDATE = 1;

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

long long progression_sum(long long first, long long step, int len) {
  return first * len + step * (len - 1) * len / 2;
}

// Invarianți:
//
// 1. Valorile reale ale nodurilor subîntinse sînt valorile lor v respective
//    plus o progresie aritmetică definită prin first și step.
// 2. v[k] include first[k] și step[k], dar nu și valorile de la strămoși.
// 3. Toate intervalele sînt [închis, deschis).
struct segment_tree {
  long long v[2 * MAX_N];
  long long first[2 * MAX_N];
  long long step[2 * MAX_N];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void set(int pos, int val) {
    v[pos + n] = val;
  }

  void build() {
    for (int i = n - 1; i; i--) {
      v[i] = v[2 * i] + v[2 * i + 1];
    }
  }
```

```
void push(int t, int len) {
  first[2 * t] += first[t];
  step[2 * t] += step[t];
  v[2 * t] += progression_sum(first[t], step[t], len / 2);

  int right_first = first[t] + step[t] * len / 2;
  first[2 * t + 1] += right_first;
  step[2 * t + 1] += step[t];
  v[2 * t + 1] += progression_sum(right_first, step[t], len / 2);

  first[t] = 0;
  step[t] = 0;
}

void pull(int t) {
  v[t] = v[2 * t] + v[2 * t + 1];
}

// pos1 = poziția unde scriem 1
void add_progression_helper(int t, int pl, int pr, int l, int r, int pos1) {
  if (l >= r) {
    return;
  } else if ((l == pl) && (r == pr)) {
    int value_at_l = l - pos1 + 1;
    first[t] += value_at_l;
    step[t]++;
    v[t] += progression_sum(value_at_l, 1, r - l);
  } else {
    push(t, pr - pl);
    int mid = (pl + pr) >> 1;
    add_progression_helper(2 * t, pl, mid, l, min(r, mid), pos1);
    add_progression_helper(2 * t + 1, mid, pr, ::max(l, mid), r, pos1);
    pull(t);
  }
}

void add_progression(int left, int right) {
  add_progression_helper(1, 0, n, left, right, left);
}

long long range_sum_helper(int t, int pl, int pr, int l, int r) {
  if (l >= r) {
    return 0;
  } else if ((l == pl) && (r == pr)) {
    return v[t];
  } else {
    push(t, pr - pl);
    int mid = (pl + pr) >> 1;
    return range_sum_helper(2 * t, pl, mid, l, min(r, mid)) +
      range_sum_helper(2 * t + 1, mid, pr, ::max(l, mid), r);
```

```
    }
  }

  long long range_sum(int left, int right) {
    return range_sum_helper(1, 0, n, left, right);
  }
};

segment_tree s;
int n, num_ops;

void read_array() {
  scanf("%d %d", &n, &num_ops);
  s.init(n);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    s.set(i, x);
  }
  s.build();
}

void process_ops() {
  while (num_ops--) {
    int type, l, r;
    scanf("%d %d %d", &type, &l, &r);
    if (type == T_UPDATE) {
      s.add_progression(l - 1, r);
    } else {
      printf("%lld\n", s.range_sum(l - 1, r));
    }
  }
}

int main() {
  read_array();
  process_ops();

  return 0;
}
```

3.2 Problema Nezzar and Binary String (Codeforces)

‹ înapoi • versiune online

```
// Complexitate: O(q log n).
#include <stdio.h>
```

```
const int MAX_N = 256 * 1024;
const int MAX_OPS = 200'000;
const int ST_CLEAN = 2;

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

// Arbore de intervale cu valori 0/1, atribuire pe interval și sumă pe
// interval.
//
// Contract: state[x] poate fi:
// * 0/1 pentru a indica o valoare de atribuit pe toți descendenții lui x, sau
// * ST_CLEAN pentru a arăta că am apelat deja push.
//
// sum[node] ține cont și de state[node].
struct segment_tree {
  int sum[2 * MAX_N];
  int state[2 * MAX_N];
  int n;

  void init(char* s, int len) {
    n = next_power_of_2(len);
    for (int i = 0; i < len; i++) {
      sum[n + i] = s[i] - '0';
    }
    for (int i = len; i < n; i++) {
      sum[n + i] = 0;
    }
    for (int i = 1; i < 2 * n; i++) {
      state[i] = ST_CLEAN;
    }

    build();
  }

  void build () {
    for (int i = n - 1; i; i--) {
      sum[i] = sum[2 * i] + sum[2 * i + 1];
    }
  }

  void push(int x) {
    if (state[x] != ST_CLEAN) {
      state[2 * x] = state[2 * x + 1] = state[x];
      sum[2 * x] = sum[2 * x + 1] = sum[x] / 2;
      state[x] = ST_CLEAN;
    }
  }
```

```
void push_all() {
  for (int i = 1; i < n; i++) {
    push(i);
  }
}

void push_path(int x) {
  int bits = __builtin_popcount(n - 1);
  for (int b = bits; b; b--) {
    push(x >> b);
  }
}

void pull_path(int x) {
  for (x /= 2; x; x /= 2) {
    if (state[x] == ST_CLEAN) {
      sum[x] = sum[2 * x] + sum[2 * x + 1];
    }
  }
}

void range_set(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;
  int size = 1;

  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      sum[l] = size * val;
      state[l++] = val;
    }
    l >>= 1;

    if (!(r & 1)) {
      sum[r] = size * val;
      state[r--] = val;
    }
    r >>= 1;
    size <<= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

int range_sum(int l, int r) {
```

```
    int result = 0;

    l += n;
    r += n;
    push_path(l);
    push_path(r);

    while (l <= r)  {
      if (l & 1) {
        result += sum[l++];
      }
      l >>= 1;

      if (!(r & 1)) {
        result += sum[r--];
      }
      r >>= 1;
    }

    return result;
  }

  bool equals(char* s, int len) {
    push_all();

    int i = 0;
    while ((i < len) && (s[i] - '0' == sum[n + i])) {
      i++;
    }

    return (i == len);
  }
};

struct operation {
  int l, r;
};

segment_tree st;
char start[MAX_N + 1], finish[MAX_N + 1];
operation op[MAX_OPS];
int len, num_ops;

void read_data() {
  scanf("%d %d %s %s", &len, &num_ops, start, finish);
  for (int i = 0; i < num_ops; i++) {
    scanf("%d %d\n", &op[i].l, &op[i].r);
    op[i].l--;
    op[i].r--;
  }
```

```
}

bool process_op(operation op) {
  int num_ones = st.range_sum(op.l, op.r);
  int num_zeroes = (op.r - op.l + 1) - num_ones;
  if (num_ones == num_zeroes) {
    return false;
  } else {
    int majority = (num_ones > num_zeroes) ? 1 : 0;
    st.range_set(op.l, op.r, majority);
    return true;
  }
}

void process_test() {
  read_data();
  st.init(finish, len);
  int i = num_ops - 1;
  while ((i >= 0) && process_op(op[i])) {
    i--;
  }

  bool success = (i < 0) && st.equals(start, len);
  printf("%s\n", success ? "YES" : "NO");
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    process_test();
  }

  return 0;
}
```

3.3 Problema Simple (infO(1)Cup 2019)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 200'000;
const int MAX_SEGTREE_NODES = 1 << 19;
const long long INF = 1LL << 60;
const int OP_ADD = 0;

long long min(long long x, long long y) {
```

```
  return (x < y) ? x :  y;
}

long long max(long long x, long long y) {
  return (x > y) ? x :  y;
}

// Un arbore de intervale cu propagare lazy. Fiecare nod reține
// * minimul par,  maximul par, minimul impar și maximul impar;
// * o cantitate delta de adăugat pe tot subarborele.
//
// Contract: Nodul curent și-a aplicat deja delta sie însuși.
struct node {
  long long e, E, o, O;
  long long delta;

  void empty() {
    // Astfel operațiile de minim/maxim funcționează fără cazuri particulare.
    // 2x pentru că ne lăsăm loc să creștem/scădem.
    e = o = 2 * INF;
    E = O = -2 * INF;
    delta = 0;
  }

  void set(int val) {
    empty();
    if (val % 2) {
      o = O = val;
    } else {
      e = E = val;
    }
  }

  void swap() {
    long long tmp = e; e = o; o = tmp;
    tmp = E; E = O; O = tmp;
  }

  void push(node& a, node& b) {
    a.add(delta);
    b.add(delta);
    delta = 0;
  }

  void pull(node a, node b) {
    // Dacă nodul este *dirty*, atunci el știe mai bine situația curentă. Fiii
    // săi au informație perimată.
    if (!delta) {
      e = min(a.e, b.e);
      E = max(a.E, b.E);
```

```
      o = min(a.o, b.o);
      O = max(a.O, b.O);
    }
  }

  void add(long long val) {
    delta += val;
    if (val % 2) {
      // Exemplu: [9,15] și [6,30] +5 ⇒ [11,35] și [14,20].
      swap();
    }
    e += val;
    E += val;
    o += val;
    O += val;
  }
};

struct segment_tree {
  node v[MAX_SEGTREE_NODES];
  int n, bits;

  void init(int _n) {
    bits = 32 - __builtin_clz(_n - 1);
    n = 1 << bits;

    for (int i = n + _n; i < 2 * n; i++) {
      // Necesar deoarece altfel nodurile de la _n la n rămîn pe zero.
      v[i].empty();
    }
  }

  void raw_set(int pos, int val) {
    v[pos + n].set(val);
  }

  void build() {
    for (int i = n - 1; i; i--) {
      v[i].pull(v[2 * i], v[2 * i + 1]);
    }
  }

  void push_path(int pos) {
    for (int b = bits - 1; b; b--) {
      int t = pos >> b;
      v[t].push(v[2 * t], v[2 * t + 1]);
    }
  }

  void pull_path(int pos) {
```

```
  for (pos /= 2; pos; pos /= 2) {
    v[pos].pull(v[2 * pos], v[2 * pos + 1]);
  }
}

void range_add(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;
  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      v[l++].add(val);
    }
    l >>= 1;

    if (!(r & 1)) {
      v[r--].add(val);
    }
    r >>= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

node range_query(int l, int r) {
  node accumulator;
  accumulator.empty();

  l += n;
  r += n;
  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      accumulator.pull(accumulator, v[l++]);
    }
    l >>= 1;

    if (!(r & 1)) {
      accumulator.pull(accumulator, v[r--]);
    }
    r >>= 1;
  }

  return accumulator;
```

```
  }
};

segment_tree st;
int n;

void read_array_into_segtree() {
  scanf("%d", &n);
  st.init(n);

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    st.raw_set(i, x);
  }

  st.build();
}

void process_ops() {
  int num_ops, type, l, r, val;

  scanf("%d", &num_ops);
  while (num_ops--) {
    scanf("%d %d %d", &type, &l, &r);
    l--;
    r--;
    if (type == OP_ADD) {
      scanf("%d", &val);
      st.range_add(l, r, val);
    } else {
      node nd = st.range_query(l, r);
      long long e = (nd.e > INF) ? -1 : nd.e;
      long long O = (nd.O < -INF) ? -1 : nd.O;
      printf("%lld %lld\n", e, O);
    }
  }
}

int main() {
  read_array_into_segtree();
  process_ops();
  return 0;
}
```

3.4 Problema Balama (Baraj ONI 2024)

înapoi

Sursă cu AINT iterativ (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 200'000;
const int MAX_SEGTREE_NODES = 1 << 19;

struct hinge {
  int r, pos;
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

struct segment_tree_node {
  int m, delta;
};

struct segment_tree {
  segment_tree_node v[MAX_SEGTREE_NODES];
  int n, bits;

  void init(int _n) {
    n = next_power_of_2(_n);
    bits = 31 - __builtin_clz(n);
  }

  void push_path(int node) {
    for (int b = bits; b; b--) {
      int t = node >> b;
      v[2 * t].delta += v[t].delta;
      v[2 * t].m += v[t].delta;
      v[2 * t + 1].delta += v[t].delta;
      v[2 * t + 1].m += v[t].delta;
      v[t].delta = 0;
    }
  }

  void pull_path(int node) {
```

```
    for (int t = node / 2; t; t /= 2) {
      v[t].m = v[t].delta + max(v[2 * t].m, v[2 * t + 1].m);
    }
  }

  int range_max_and_inc(int l, int r) {
    l += n;
    r += n;
    push_path(l);
    push_path(r);

    int result = 0;
    int orig_l = l, orig_r = r;

    while (l <= r)  {
      if (l & 1) {
        result = max(result, v[l].m);
        v[l].m++;
        v[l].delta++;
        l++;
      }
      l >>= 1;

      if (!(r & 1)) {
        result = max(result, v[r].m);
        v[r].m++;
        v[r].delta++;
        r--;
      }
      r >>= 1;
    }

    pull_path(orig_l);
    pull_path(orig_r);

    return result;
  }
};

hinge h[MAX_N];
int sol[MAX_N];
segment_tree st;
int n, k;

void read_data() {
  FILE* f = fopen("balama.in", "r");
  fscanf(f, "%d %d", &n, &k);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &h[i].r);
    h[i].pos = i;
```

```
  }
  fclose(f);
}

void sort_by_r() {
  std::sort(h, h + n, [](hinge a, hinge b) {
    return a.r > b.r;
  });
}

void scan_hinges() {
  int next_to_fill = 0;
  int i = 0;

  st.init(n);

  while (next_to_fill < k) {
    int left = max(h[i].pos - k + 1, 0);
    int right = min(h[i].pos, n - k);
    int m = st.range_max_and_inc(left, right);
    if (m == next_to_fill) {
      sol[next_to_fill++] = h[i].r;
    }
    i++;
  }
}

void write_answers() {
  FILE* f = fopen("balama.out", "w");
  for (int i = k - 1; i >= 0; i--) {
    fprintf(f, "%d ", sol[i]);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_data();
  sort_by_r();
  scan_hinges();
  write_answers();

  return 0;
}
```

Sursă cu AINT recursiv (versiune online).

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_N = 1 << 18;

struct hinge {
  int r, pos;
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int next_power_of_2(int n) {
  while (n & (n - 1)) {
    n += (n & -n);
  }

  return n;
}

// Arbore de segmente cu operațiile:
//
// 1. update: inc(left, right) -- incrementează pozițiile [left, right]
// 2. query: max(left, right) -- returnează maximul din [left, right]
//
// Invariant:
//
// * lazy_sum este valoarea de adăugat pe fiecare nod din subarbore
// * m este maximul real din subarbore, inclusiv lazy_sum
struct segment_tree {
  int m[2 * MAX_N];
  int lazy_sum[2 * MAX_N];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void push(int t) {
    lazy_sum[2 * t] += lazy_sum[t];
    m[2 * t] += lazy_sum[t];
    lazy_sum[2 * t + 1] += lazy_sum[t];
    m[2 * t + 1] += lazy_sum[t];
    lazy_sum[t] = 0;
  }

  void pull(int t) {
```

```
    m[t] = ::max(m[2 * t], m[2 * t + 1]);
  }

  void inc_helper(int t, int pl, int pr, int l, int r) {
    if (l >= r) {
      return;
    } else if ((l == pl) && (r == pr)) {
      lazy_sum[t]++;
      m[t]++;
    } else {
      push(t);
      int mid = (pl + pr) >> 1;
      inc_helper(2 * t, pl, mid, l, min(r, mid));
      inc_helper(2 * t + 1, mid, pr, ::max(l, mid), r);
      pull(t);
    }
  }

  void inc(int left, int right) {
    inc_helper(1, 0, n, left, right + 1);
  }

  int max_helper(int t, int pl, int pr, int l, int r) {
    if (l >= r) {
      return 0;
    } else if ((l == pl) && (r == pr)) {
      return m[t];
    } else {
      push(t);
      int mid = (pl + pr) >> 1;
      return ::max(max_helper(2 * t, pl, mid, l, min(r, mid)),
                   max_helper(2 * t + 1, mid, pr, ::max(l, mid), r));
    }
  }

  int max(int left, int right) {
    return max_helper(1, 0, n, left, right + 1);
  }
};

hinge h[MAX_N];
int sol[MAX_N];
segment_tree st;
int n, k;

void read_data() {
  FILE* f = fopen("balama.in", "r");
  fscanf(f, "%d %d", &n, &k);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &h[i].r);
```

```
    h[i].pos = i;
  }
  fclose(f);
}

void sort_by_r() {
  std::sort(h, h + n, [](hinge a, hinge b) {
    return a.r > b.r;
  });
}

void scan_hinges() {
  int next_to_fill = 0;
  int i = 0;

  st.init(n);

  while (next_to_fill < k) {
    int left = max(h[i].pos - k + 1, 0);
    int right = min(h[i].pos, n - k);
    int m = st.max(left, right);
    st.inc(left, right);
    if (m == next_to_fill) {
      sol[next_to_fill++] = h[i].r;
    }
    i++;
  }
}

void write_answers() {
  FILE* f = fopen("balama.out", "w");
  for (int i = k - 1; i >= 0; i--) {
    fprintf(f, "%d ", sol[i]);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_data();
  sort_by_r();
  scan_hinges();
  write_answers();

  return 0;
}
```

3.5 Problema A Simple Task (Codeforces)

‹ înapoi

Sursă cu AINT iterativ (versiune online).

```
#include <stdio.h>

const int MAX_LEN = 1 << 17;
const int SIGMA = 26;

const int ST_DESC = 0;
const int ST_ASC = 1;
const int ST_CLEAN = 2;

// Parametrii necesari pentru a itera crescător / descrescător.
const int LOOP_START[2] = { SIGMA - 1, 0 };
const int LOOP_END[2]   = {        -1, SIGMA };
const int LOOP_STEP[2]  = {        -1, +1 };

int min(int x, int y) {
  return (x < y) ? x : y;
}

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct node {
  int f[SIGMA];
  char state;

  int get_first_nonzero() {
    int i = 0;
    while (!f[i]) {
      i++;
    }
    return i;
  }

  void add(node& a, node& b) {
    for (int i = 0; i < SIGMA; i++) {
      f[i] = a.f[i] + b.f[i];
    }
  }

  void accumulate(node& src) {
    for (int i = 0; i < SIGMA; i++) {
      f[i] += src.f[i];
    }
```

```
  }

  // Sparge this în a și b. Îl lasă intact pe this.
  void split(node& a, node& b, int a_size, int dir) {
    for (int i = LOOP_START[dir]; i != LOOP_END[dir]; i += LOOP_STEP[dir]) {
      int to_a = min(a_size, f[i]);
      a.f[i] = to_a;
      b.f[i] = f[i] - to_a;
      a_size -= to_a;
    }
    a.state = b.state = dir;
    state = ST_CLEAN;
  }

  // Distribuie quantity litere din this în dest.
  void distribute(node& dest, int quantity, int dir, int new_state) {
    for (int i = LOOP_START[dir]; i != LOOP_END[dir]; i += LOOP_STEP[dir]) {
      int to_dest = min(quantity, f[i]);
      dest.f[i] = to_dest;
      f[i] -= to_dest;
      quantity -= to_dest;
    }
    dest.state = new_state;
  }
};

struct segment_tree {
  node v[2 * MAX_LEN], acc;
  int n, bits;

  void init(char* s, int len) {
    n = next_power_of_2(len);
    bits = 31 - __builtin_clz(n);

    for (int i = 0; i < len; i++) {
      int c = s[i] - 'a';
      for (int t = n + i; t; t /= 2) {
        v[t].f[c]++;
      }
    }

    for (int i = 1; i < 2 * n; i++) {
      v[i].state = ST_CLEAN;
    }
  }

  void push(int pos, int size) {
    if (v[pos].state != ST_CLEAN) {
      int l = 2 * pos, r = 2 * pos + 1;
      v[pos].split(v[l], v[r], size / 2, v[pos].state);
```

```
  }
}

void push_path(int node) {
  for (int b = bits, size = n; b; b--, size >>= 1) {
    push(node >> b, size);
  }
}

void push_all(int node, int size) {
  if (node < n) {
    push(node, size);
    push_all(2 * node, size / 2);
    push_all(2 * node + 1, size / 2);
  }
}

void pull_path(int pos) {
  for (pos /= 2; pos; pos /= 2) {
    // Nu trage frecvențele literelor din fii în noduri dirty. Pot fi
    // învechite.
    if (v[pos].state == ST_CLEAN) {
      v[pos].add(v[2 * pos], v[2 * pos + 1]);
    }
  }
}

void collect(int l, int r) {
  l += n;
  r += n;
  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      acc.accumulate(v[l++]);
    }
    l >>= 1;

    if (!(r & 1)) {
      acc.accumulate(v[r--]);
    }
    r >>= 1;
  }
}

void distribute(int l, int r, int dir) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;
```

```
    int size = 1;

    while (l <= r)  {
      if (l & 1) {
        acc.distribute(v[l++], size, dir, dir);
      }
      l >>= 1;

      if (!(r & 1)) {
        acc.distribute(v[r--], size, 1 - dir, dir);
      }
      r >>= 1;
      size <<= 1;
    }

    pull_path(orig_l);
    pull_path(orig_r);
  }

  void sort(int l, int r, int dir) {
    collect(l, r);
    distribute(l, r, dir);
  }

  void export_(char* dest, int len) {
    push_all(1, n);

    for (int i = 0; i < len; i++) {
      dest[i] = 'a' + v[n + i].get_first_nonzero();
    }
  }
};

char s[MAX_LEN + 2];
segment_tree st;
int len, num_ops;

void read_string() {
  scanf("%d %d %s", &len, &num_ops, s);
}

void process_ops() {
  int left, right, dir;

  while (num_ops--) {
    scanf("%d %d %d", &left, &right, &dir);
    st.sort(left - 1, right - 1, dir);
  }
}
```

```
void write_answer() {
  printf("%s\n", s);
}

int main() {
  read_string();
  st.init(s, len);
  process_ops();
  st.export_(s, len);
  write_answer();

  return 0;
}
```

Sursă cu AINT recursiv (versiune online).

```
#include <stdio.h>

const int MAX_LEN = 1 << 17;
const int SIGMA = 26;

const int ST_DESC = 0;
const int ST_ASC = 1;
const int ST_CLEAN = 2;

// Parametrii necesari pentru a itera crescător / descrescător.
const int LOOP_START[2] = { SIGMA - 1, 0 };
const int LOOP_END[2]   = {        -1, SIGMA };
const int LOOP_STEP[2]  = {        -1, +1 };

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct node {
  int f[SIGMA];
  unsigned char state;

  int get_first_nonzero() {
    int i = 0;
    while (!f[i]) {
```

```
      i++;
    }
    return i;
  }

  void add(node& a, node& b) {
    for (int i = 0; i < SIGMA; i++) {
      f[i] = a.f[i] + b.f[i];
    }
  }

  void accumulate(node& src) {
    for (int i = 0; i < SIGMA; i++) {
      f[i] += src.f[i];
    }
  }

  // Sparge this în a și b. Îl lasă intact pe this.
  void split(node& a, node& b, int a_size) {
    for (int i = LOOP_START[state]; i != LOOP_END[state]; i += LOOP_STEP[state]) {
      int to_a = min(a_size, f[i]);
      a.f[i] = to_a;
      b.f[i] = f[i] - to_a;
      a_size -= to_a;
    }
    a.state = b.state = state;
  }

  // Distribuie quantity litere din this în dest.
  void distribute(node& dest, int quantity) {
    for (int i = LOOP_START[state]; i != LOOP_END[state]; i += LOOP_STEP[state]) {
      int to_dest = min(quantity, f[i]);
      dest.f[i] = to_dest;
      f[i] -= to_dest;
      quantity -= to_dest;
    }
    dest.state = state;
  }
};

struct segment_tree {
  node v[2 * MAX_LEN], acc;
  int orig_len, n;

  void init(char* s, int len) {
    orig_len = len;
    n = next_power_of_2(len);

    for (int i = 0; i < len; i++) {
      int c = s[i] - 'a';
```

```
    v[n + i].f[c] = 1;
  }

  for (int i = n - 1; i; i--) {
    v[i].add(v[2 * i], v[2 * i + 1]);
  }

  for (int i = 1; i < 2 * n; i++) {
    v[i].state = ST_CLEAN;
  }
}

void push(int pos, int size) {
  if (v[pos].state != ST_CLEAN) {
    int l = 2 * pos, r = 2 * pos + 1;
    v[pos].split(v[l], v[r], size / 2);
    v[pos].state = ST_CLEAN;
  }
}

void push_all() {
  for (int start = 1; start < n; start *= 2) {
    int size = n / start;
    for (int i = start; i < 2 * start; i++) {
      push(i, size);
    }
  }
}

void pull(int pos) {
  v[pos].add(v[2 * pos], v[2 * pos + 1]);
}

void collect(int node, int pl, int pr, int l, int r) {
  if ((l == pl) && (r == pr)) {
    acc.accumulate(v[node]);
  } else if (l < r) {
    push(node, pr - pl);
    int mid = (pl + pr) >> 1;
    collect(2 * node, pl, mid, l, min(r, mid));
    collect(2 * node + 1, mid, pr, max(l, mid), r);
  }
}

void distribute(int node, int pl, int pr, int l, int r) {
  if ((l == pl) && (r == pr)) {
    acc.distribute(v[node], pr - pl);
  } else if (l < r) {
    int mid = (pl + pr) >> 1;
    distribute(2 * node, pl, mid, l, min(r, mid));
```

```
      distribute(2 * node + 1, mid, pr, max(l, mid), r);
      pull(node);
    }
  }

  void sort(int l, int r, int dir) {
    collect(1, 0, n, l, r);
    acc.state = dir;
    distribute(1, 0, n, l, r);
  }

  void export_(char* dest) {
    push_all();

    for (int i = 0; i < orig_len; i++) {
      dest[i] = 'a' + v[n + i].get_first_nonzero();
    }
  }
};

char s[MAX_LEN + 2];
segment_tree st;
int len, num_ops;

void read_string() {
  scanf("%d %d %s", &len, &num_ops, s);
}

void process_ops() {
  int left, right, dir;

  while (num_ops--) {
    scanf("%d %d %d", &left, &right, &dir);
    st.sort(left - 1, right, dir); // 0-based, [închis, deschis)
  }
}

void write_answer() {
  printf("%s\n", s);
}

int main() {
  read_string();
  st.init(s, len);
  process_ops();
  st.export_(s);
  write_answer();

  return 0;
```

```
}
```

3.6 Problema Periodic RMQ Problem (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_SPARSE_TABLE_LAYERS = 17;

const int MAX_OPS = 128 * 1024;
const int MAX_ENDPOINTS = 2 * MAX_OPS;
const int MAX_COMPRESSED_RANGES = 2 * MAX_ENDPOINTS;
const int T_UPDATE = 1;
const int T_QUERY = 2;

const int INFINITY = 2'000'000'000;

struct operation {
  int type;
  int l, r;
  int val;
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

int log2(int n) {
  return 31 - __builtin_clz(n);
}

struct base_array {
  int v[MAX_SPARSE_TABLE_LAYERS][MAX_N];
  int n;

  void build_rmq() {
    for (int i = 1, size = 2; size <= n; i++, size *= 2) {
      for (int j = 0; j + size <= n; j++) {
        v[i][j] = min(v[i - 1][j], v[i - 1][j + size / 2]);
      }
    }
```

```
  }

  int rmq(int l, int r) {
    int row = log2(r - l + 1); // De ex. rîndul 3 pentru lungimile 8-15.
    return min(v[row][l],
               v[row][r - (1 << row) + 1]);
  }

  int periodic_rmq(int l, int r) {
    if (r - l + 1 >= n) {
      return rmq(0, n - 1);
    } else {
      l %= n;
      r %= n;
      if (l <= r) {
        return rmq(l, r);
      } else {
        return min(rmq(l, n - 1), rmq(0, r));
      }
    }
  }
};

// Un arbore de intervale cu atribuire pe interval și RMQ pe interval.
//
// Contract: m[x] este minimul nodurilor subîntinse de x. Cînd dirty[x] este
// true, atunci m[x] trebuie atribuit pe tot intervalul.
struct segment_tree {
  int m[2 * MAX_COMPRESSED_RANGES];
  bool dirty[2 * MAX_COMPRESSED_RANGES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int i = this->n; i < 2 * this->n; i++) {
      m[i] = INFINITY;
    }
  }

  void raw_set(int pos, int val) {
    m[pos + n] = val;
  }

  void build() {
    for (int i = n - 1; i; i--) {
      m[i] = min(m[2 * i], m[2 * i + 1]);
    }
  }

  void push(int x) {
```

```
  if (dirty[x]) {
    dirty[2 * x] = dirty[2 * x + 1] = true;
    m[2 * x] = m[2 * x + 1] = m[x];
    dirty[x] = false;
  }
}

void push_path(int x) {
  int bits = __builtin_popcount(n - 1);
  for (int b = bits; b; b--) {
    push(x >> b);
  }
}

void pull_path(int x) {
  for (x /= 2; x; x /= 2) {
    if (!dirty[x]) {
      m[x] = min(m[2 * x], m[2 * x + 1]);
    }
  }
}

void range_set(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;

  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      m[l] = val;
      dirty[l++] = true;
    }
    l >>= 1;

    if (!(r & 1)) {
      m[r] = val;
      dirty[r--] = true;
    }
    r >>= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

int rmq(int l, int r) {
  int result = INFINITY;
```

```
    l += n;
    r += n;
    push_path(l);
    push_path(r);

    while (l <= r)  {
      if (l & 1) {
        result = min(result, m[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = min(result, m[r--]);
      }
      r >>= 1;
    }

    return result;
  }

};

base_array base;
operation op[MAX_OPS];
int coord[MAX_COMPRESSED_RANGES];
segment_tree st;
int num_ops, num_ranges;

void read_data() {
  int ignored;
  scanf("%d %d", &base.n, &ignored);
  for (int i = 0; i < base.n; i++) {
    scanf("%d", &base.v[0][i]);
  }
  scanf("%d", &num_ops);
  for (int i = 0; i < num_ops; i++) {
    scanf("%d %d %d", &op[i].type, &op[i].l, &op[i].r);
    op[i].l--;
    op[i].r--;
    if (op[i].type == T_UPDATE) {
      scanf("%d", &op[i].val);
    }
  }
}

void remove_duplicates() {
  int size = 1;
  for (int i = 1; i < num_ranges; i++) {
    if (coord[i] != coord[i - 1]) {
```

```
      coord[size++] = coord[i];
    }
  }
  num_ranges = size;
}

int bin_search(int val) {
  int l = 0, r = num_ranges;
  while (coord[l] != val) {
    int mid = (l + r) / 2;
    if (coord[mid] > val) {
      r = mid;
    } else {
      l = mid;
    }
  }
  return l;
}

void normalize_endpoints() {
  for (int i = 0; i < num_ops; i++) {
    coord[num_ranges++] = op[i].l;
    coord[num_ranges++] = op[i].l + 1;
    coord[num_ranges++] = op[i].r;
    coord[num_ranges++] = op[i].r + 1;
  }

  std::sort(coord, coord + num_ranges);
  remove_duplicates();

  for (int i = 0; i < num_ops; i++) {
    op[i].l = bin_search(op[i].l);
    op[i].r = bin_search(op[i].r);
  }
}

void build_compressed_segtree() {
  st.init(num_ranges - 1);

  for (int i = 0; i < num_ranges - 1; i++) {
    int rmq = base.periodic_rmq(coord[i], coord[i + 1] - 1);
    st.raw_set(i, rmq);
  }

  st.build();
}

void process_ops() {
  for (int i = 0; i < num_ops; i++) {
    if (op[i].type == T_UPDATE) {
```

```
      st.range_set(op[i].l, op[i].r, op[i].val);
    } else {
      int m = st.rmq(op[i].l, op[i].r);
      printf("%d\n", m);
    }
  }
}

int main() {
  read_data();
  normalize_endpoints();
  base.build_rmq();
  build_compressed_segtree();
  process_ops();

  return 0;
}
```

3.7 Problema Circuit (Lot 2025)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 300'000;
const int MAX_SEGTREE_NODES = 1 << 20;
const int MOD = 1'000'000'007;
const int HALF = (MOD + 1) / 2; // Pentru că (HALF * 2) % MOD = 1.

struct elem {
  int val;
  int pos;
};

elem v[MAX_N];
int pow2[MAX_N + 1], inv_pow2[MAX_N + 1];
int n;

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segtree_node {
  int sum;    // suma din intervalul subîntins
  int lazy;   // numărul de dublări de aplicat
  // Invariant: pentru nodul curent, sum include lazy.
};
```

```
// Arbore de segmente cu dublare pe prefix și sumă pe prefix.
struct segment_tree {
  segtree_node v[MAX_SEGTREE_NODES];
  int n, bits;

  void init(int size) {
    n = next_power_of_2(size);
    bits = __builtin_popcount(n - 1);

    for (int pos = n; pos < n + size; pos++) {
      v[pos].sum = 1;
      v[pos].lazy = 0;
    }

    for (int pos = n - 1; pos; pos--) {
      v[pos].sum = v[2 * pos].sum + v[2 * pos + 1].sum;
      v[pos].lazy = 0;
    }
  }

  void push(int pos) {
    int l = v[pos].lazy;
    if (l) {
      v[2 * pos].sum = (long long)v[2 * pos].sum * pow2[l] % MOD;
      v[2 * pos].lazy += l;
      v[2 * pos + 1].sum = (long long)v[2 * pos + 1].sum * pow2[l] % MOD;
      v[2 * pos + 1].lazy += l;
      v[pos].lazy = 0;
    }
  }

  void push_path(int pos) {
    for (int b = bits; b; b--) {
      push(pos >> b);
    }
  }

  void pull_path(int pos) {
    for (pos /= 2; pos; pos /= 2) {
      if (!v[pos].lazy) {
        v[pos].sum = (v[2 * pos].sum + v[2 * pos + 1].sum) % MOD;
      }
    }
  }

  void double_prefix(int r) {
    r += n;
    int orig_r = r;
```

```
    push_path(r);

    while (r)  {
      if (!(r & 1)) {
        v[r].sum = 2 * v[r].sum % MOD;
        v[r--].lazy++;
      }
      r >>= 1;
    }

    pull_path(orig_r);
  }

  int prefix_sum(int r) {
    long long result = 0;

    r += n;
    push_path(r);

    int quot = v[r].lazy;

    while (r)  {
      if (!(r & 1)) {
        result += v[r--].sum;
      }
      r >>= 1;
    }

    return result % MOD * inv_pow2[quot] % MOD;
  }
};

segment_tree pref_st, suf_st;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i].val);
    v[i].pos = i;
  }
}

void sort_by_value() {
  std::sort(v, v + n, [](elem a, elem b) {
    return (a.val < b.val);
  });
}

void compute_powers() {
  pow2[0] = inv_pow2[0] = 1;
```

```
  for (int i = 1; i <= n; i++) {
    pow2[i] = 2 * pow2[i - 1] % MOD;
    inv_pow2[i] = (long long)HALF * inv_pow2[i - 1] % MOD;
  }
}

int count_contributions() {
  __int128 sum = 0;

  pref_st.init(n);
  suf_st.init(n);

  for (int i = 0; i < n; i++) {
    int p = v[i].pos;
    int pref_sum = pref_st.prefix_sum(p);
    int suf_sum = suf_st.prefix_sum(n - 1 - p);
    sum += (__int128)pref_sum * suf_sum * v[i].val;
    pref_st.double_prefix(p);
    suf_st.double_prefix(n - 1 - p);
  }
  return sum % MOD;
}

int main() {
  read_data();
  sort_by_value();
  compute_powers();
  int answer = count_contributions();
  printf("%d\n", answer);

  return 0;
}
```

3.8 Problema Babel (Baraj ONI 2025)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 200'000;
const int MAX_POW_2 = 1 << 18;
const int MOD = 666'013;
const int NONE = 3;

const int mod3[6] = { 0, 1, 2, 0, 1, 2 };
int pow4[MAX_N / 2];

int next_power_of_2(int x) {
```

```
  return 1 << (32 - __builtin_clz(x - 1));
}

// Un aint cu operațiile de atribuire pe interval și evaluare punctuală.
// Nodurile în care se descompune o atribuire reține timpul și valoarea
// atribuirii.
struct r_segment_tree_node {
  int time;
  int val;
};

struct r_segment_tree {
  r_segment_tree_node v[2 * MAX_POW_2];
  int n, time;

  void init(int _n) {
    n = next_power_of_2(_n);
    time = 0;
  }

  void set(int pos, int val) {
    v[pos + n] = { 0, val };
  }

  void range_set(int l, int r, int val) {
    l += n;
    r += n;
    time++;

    while (l <= r)  {
      if (l & 1) {
        v[l++] = { time, val };
      }
      l >>= 1;

      if (!(r & 1)) {
        v[r--] = { time, val };
      }
      r >>= 1;
    }
  }

  // Returnează valoarea cu timestamp-ul cel mai mare dintre toți strămoșii.
  int get(int pos) {
    pos += n;
    int result = v[pos].val;
    int max_time = v[pos].time;
    for (pos /= 2; pos; pos /= 2) {
      if (v[pos].time > max_time) {
        result = v[pos].val;
```

```
        max_time = v[pos].time;
      }
    }
    return result;
  }
};

struct segment_tree_node {
  int sum[3]; // suma lui 4^i pentru pozițiile i pe care valoarea este 0/1/2
  int sum_all; // suma puterilor lui 4 subîntinse
  unsigned char lazy_val;
  unsigned char lazy_incs; // modulo 3

  bool is_lazy() {
    return (lazy_val != NONE) || lazy_incs;
  }

  // Atribuie tuturor frunzelor subîntinse valoarea val.
  void set(int val) {
    sum[0] = sum[1] = sum[2] = 0;
    sum[val] = sum_all;
    lazy_val = val;
    lazy_incs = 0;
  }

  // Valoarea lui f[] pe această poziție este indicată de cantitatea nenulă
  // din sum[].
  int get() {
    return sum[0] ? 0 : (sum[1] ? 1 : 2);
  }

  // Crește toate frunzele subîntinse cu val (modulo 3).
  void inc(int val) {
    if (lazy_val != NONE) {
      set(mod3[lazy_val + val]);
    } else {
      rotate(val);
      lazy_incs = mod3[lazy_incs + val];
    }
  }

  void rotate(int cnt) {
    if (cnt == 1) {
      int tmp = sum[2];
      sum[2] = sum[1];
      sum[1] = sum[0];
      sum[0] = tmp;
    } else if (cnt == 2) {
      int tmp = sum[0];
      sum[0] = sum[1];
```

```
      sum[1] = sum[2];
      sum[2] = tmp;
    }
  }

  void push(segment_tree_node& a, segment_tree_node& b) {
    if (lazy_val != NONE) {
      a.set(lazy_val);
      b.set(lazy_val);
      lazy_val = NONE;
    } else if (lazy_incs) {
      a.inc(lazy_incs);
      b.inc(lazy_incs);
      lazy_incs = 0;
    }
  }

  void pull(segment_tree_node& a, segment_tree_node& b) {
    for (int i = 0; i < 3; i++) {
      sum[i] = (a.sum[i] + b.sum[i]) % MOD;
    }
  }
};

struct segment_tree {
  segment_tree_node v[MAX_POW_2]; // fără dublare, căci are mărime n/2
  int n, bits;

  void init(int _n) {
    n = next_power_of_2(_n);
    bits = __builtin_popcount(n - 1);
  }

  void raw_set(int pos, int val) {
    int p = pos + n;
    v[p].sum_all = pow4[pos];
    v[p].set(val);
  }

  void build() {
    for (int i = n - 1; i; i--) {
      v[i].pull(v[2 * i], v[2 * i + 1]);
      // sum_all nu se schimbă după construcție
      v[i].sum_all = (v[2 * i].sum_all + v[2 * i + 1].sum_all) % MOD;
      v[i].lazy_val = NONE;
      v[i].lazy_incs = 0;
    }
  }

  void push_path(int pos) {
```

```
  for (int b = bits; b; b--) {
    int ancestor = pos >> b;
    v[ancestor].push(v[2 * ancestor], v[2 * ancestor + 1]);
  }
}

void pull_path(int pos) {
  for (pos /= 2; pos; pos /= 2) {
    if (!v[pos].is_lazy()) {
      v[pos].pull(v[2 * pos], v[2 * pos + 1]);
    }
  }
}

void range_set(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;

  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      v[l++].set(val);
    }
    l >>= 1;

    if (!(r & 1)) {
      v[r--].set(val);
    }
    r >>= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

void prefix_inc(int r, int val) {
  r += n;
  int orig_r = r;

  push_path(r);

  // ultimul nod acoperit de [0,r] va avea indicele putere a lui 2.
  while (r & (r - 1)) {
    if (!(r & 1)) {
      v[r--].inc(val);
    }
    r >>= 1;
```

```
    }
    v[r].inc(val);

    pull_path(orig_r);
  }

  int get(int pos) {
    pos += n;
    push_path(pos);
    return v[pos].get();
  }
};

r_segment_tree r;
segment_tree f[2];
FILE *fin, *fout;
int n, num_ops;

void read_disks() {
  fscanf(fin, "%d %d", &n, &num_ops);
  r.init(n);

  for (int i = 0; i < 3; i++) {
    int num_disks, size;
    fscanf(fin, "%d", &num_disks);
    while (num_disks--) {
      fscanf(fin, "%d", &size);
      size--;
      r.set(size, i);
    }
  }
}

void compute_pow4() {
  pow4[0] = 1;
  for (int i = 1; i <= n / 2; i++) {
    pow4[i] = 4 * pow4[i - 1] % MOD;
  }
}

void init_f() {
  f[0].init((n + 1) / 2);
  f[1].init(n / 2);

  int expected = r.get(n - 1);
  for (int i = n - 1; i >= 0; i--) {
    int rod = r.get(i);
    int diff = mod3[3 + rod - expected];
    f[i % 2].raw_set(i / 2, diff);
```

```
    if (rod != expected) {
      expected = 3 - rod - expected;
    }
  }

  f[0].build();
  f[1].build();
}

int compute_cost() {
  // suma pe pozițiile unde f ≠ 0, adică f ∈ {1, 2}
  return (f[0].v[1].sum[1] +
          f[0].v[1].sum[2] +
          2 * f[1].v[1].sum[1] +
          2 * f[1].v[1].sum[2]) % MOD;
}

// Returnează noul f[pos] când r[pos] se schimbă.
int get_new_f(int pos, int new_r) {
  if (pos == n - 1) {
    return 0; // Discul n-1 este întotdeauna pe tija corectă.
  } else {
    int old_r = r.get(pos);
    int old_f = f[pos % 2].get(pos / 2);
    int e = mod3[3 + old_r - old_f];
    return mod3[3 + new_r - e];
  }
}

// Returnează variația lui f[pos-1] când r[pos] se schimbă.
int get_delta_f(int pos, int new_r) {
  if (!pos) {
    return 0;
  }

  int f_pos = f[pos % 2].get(pos / 2);
  int r_pos = new_r;
  int e_pos = mod3[3 + r_pos - f_pos];

  int f_prev = f[(pos - 1) % 2].get((pos - 1) / 2);
  int r_prev = r.get(pos - 1);
  int e_prev = mod3[3 + r_prev - f_prev];

  int new_e = (r_pos == e_pos) ? e_pos : (3 - e_pos - r_pos);

  if (new_e == e_prev) {
    return 0;
  } else {
    int new_f = mod3[3 + r_prev - new_e];
    return mod3[3 + new_f - f_prev];
```

```
  }
}

void update_rod(int from, int to, int new_rod) {
  // Calculăm cele două valori pentru alternanța pe pozițiile from...to.
  // Un pic de magie ca să calculăm intervalele din f[0] și în f[1].
  int new_f_to = get_new_f(to, new_rod);

  f[to % 2].range_set((from + 1 - to % 2) / 2, to / 2, new_f_to);
  if (from < to) {
    int other_f = mod3[3 - new_f_to];
    f[1 - to % 2].range_set((from + to % 2) / 2, (to - 1) / 2, other_f);
  }

  // Corectăm vectorul r.
  r.range_set(from, to, new_rod);

  // Ne uităm dacă e[from-1] s-a modificat. Dacă nu, așteptările pentru
  // discurile 1...from-1 nu se modifică. Dacă da, ele variază cu ±1.
  int delta = get_delta_f(from, new_rod);
  if (delta) {
    f[(from - 1) % 2].prefix_inc((from - 1) / 2, delta);
    if (from >= 2) {
      f[1 - (from - 1) % 2].prefix_inc(from / 2 - 1, 3 - delta);
    }
  }
}

void process_ops() {
  fprintf(fout, "%d\n", compute_cost());

  while (num_ops--) {
    int from, to, new_rod;
    fscanf(fin, "%d %d %d", &from, &to, &new_rod);
    from--;
    new_rod--;
    to--;
    update_rod(from, to, new_rod);
    fprintf(fout, "%d\n", compute_cost());
  }
}

int main() {
  fin = fopen("babel.in", "r");
  fout = fopen("babel.out", "w");

  read_disks();
  compute_pow4();
  init_f();
  process_ops();
```

```
  fclose(fin);
  fclose(fout);

  return 0;
}
```

Anexa 4

Ștergerea din structuri de date care nu admit ștergeri

4.1 Problema Addition on Segments (Codeforces)

‹ înapoi • versiune online

```
#include <bitset>
#include <stdio.h>
#include <vector>

const int MAX_N = 10'000;
const int MAX_SEGTREE = 32'768;

typedef std::bitset<MAX_N + 1> bitset;

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segment_tree {
  std::vector<short> exes[MAX_SEGTREE];
  bitset accumulator; // OR-ul tuturor bitseturilor pentru toate intervalele
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void add_x(int l, int r, int x) {
    l += n;
    r += n;

    while (l <= r)  {
```

```
      if (l & 1) {
        exes[l++].push_back(x);
      }
      l >>= 1;

      if (!(r & 1)) {
        exes[r--].push_back(x);
      }
      r >>= 1;
    }
  }

  void dfs(int pos, bitset current) {
    for (int x: exes[pos]) {
      current |= current << x;
    }
    if (pos >= n) {
      accumulator |= current;
    } else {
      dfs(2 * pos, current);
      dfs(2 * pos + 1, current);
    }
  }

  bitset count_obtainable_sums() {
    accumulator.reset();
    bitset current(1);
    dfs(1, current);
    return accumulator;
  }
};

segment_tree st;
int n;

void read_intervals_into_segtree() {
  int num_intervals;
  scanf("%d %d", &n, &num_intervals);
  st.init(n);

  while (num_intervals--) {
    int l, r, x;
    scanf("%d %d %d", &l, &r, &x);
    st.add_x(l - 1, r - 1, x);
  }
}

void write_answer(bitset& answer) {
  int set_bits_in_1_n = (answer >> 1).count() - (answer >> (n + 1)).count();
  printf("%d\n", set_bits_in_1_n);
```

```
  for (int i = 1; i <= n; i++) {
    if (answer[i]) {
      printf("%d ", i);
    }
  }
  printf("\n");
}

int main() {
  read_intervals_into_segtree();
  bitset answer = st.count_obtainable_sums();
  write_answer(answer);

  return 0;
}
```

4.2 Problema Extending a Set of Points (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <unordered_map>
#include <vector>

const int MAX_OPS = 300'000;
const int MAX_COORD = 300'000;
const int MAX_SEGTREE_NODES = 1 << 20;

struct point {
  int x, y;

  bool operator==(const point &other) const {
    return (x == other.x) && (y == other.y);
  }
};

struct point_hash {
  std::size_t operator()(const point& p) const {
    return p.x * MAX_COORD + p.y;
  }
};

struct change {
  int u, v;
  int par_u;
  int size_x_v, size_y_v;
  long long total_area;
};
```

```
struct disjoint_set_forest {
  int par[2 * MAX_COORD + 1];
  int size_x[2 * MAX_COORD + 1], size_y[2 * MAX_COORD + 1];
  long long total_area;

  // Stiva de modificări făcute.
  change st[MAX_COORD];
  int ss;

  void init() {
    // Valorile inițiale pentru size_x și size_y elimină cazurile particulare.
    // Acum formula reuniunii (k·l, k'·l') ⇒ (k+k')·(l+l') este valabilă și
    // prima dată când întîlnim un anume x sau y.
    for (int i = 0; i < MAX_COORD; i++) {
      par[i] = i;
      size_x[i] = 0;
      size_y[i] = 1;
    }
    for (int i = 0; i < MAX_COORD; i++) {
      par[i + MAX_COORD] = i + MAX_COORD;
      size_x[i + MAX_COORD] = 1;
      size_y[i + MAX_COORD] = 0;
    }
    total_area = 0;
  }

  void push_change(int u, int v) {
    st[ss++] = {
      .u = u,
      .v = v,
      .par_u = par[u],
      .size_x_v = size_x[v],
      .size_y_v = size_y[v],
      .total_area = total_area,
    };
  }

  int find(int u) {
    return (par[u] == u) ? u : find(par[u]);
  }

  void unite(int u, int v) {
    if (u != v) {
      par[u] = v;
      total_area -= (long long)size_x[u] * size_y[u];
      total_area -= (long long)size_x[v] * size_y[v];
      size_x[v] += size_x[u];
      size_y[v] += size_y[u];
      total_area += (long long)size_x[v] * size_y[v];
```

```
    }
  }

  void process_point(point p) {
    int u = find(p.x);
    int v = find(p.y + MAX_COORD);
    if ((long long)size_x[u] * size_y[u] >
        (long long)size_x[v] * size_y[v]) {
      int tmp = u; u = v; v = tmp;
    }

    push_change(u, v);
    unite(u, v);
  }

  void rollback() {
    change& c = st[--ss];
    par[c.u] = c.par_u;
    size_x[c.v] = c.size_x_v;
    size_y[c.v] = c.size_y_v;
    total_area = c.total_area;
  }
};

long long sol[MAX_OPS];

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segment_tree {
  std::vector<point> points[MAX_SEGTREE_NODES];
  disjoint_set_forest grids;
  int n, orig_n;

  void init(int _n) {
    orig_n = _n;
    n = next_power_of_2(_n);
    grids.init();
  }

  void add_point_on_range(point p, int l, int r) {
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        points[l++].push_back(p);
      }
      l >>= 1;
```

```
      if (!(r & 1)) {
        points[r--].push_back(p);
      }
      r >>= 1;
    }
  }

  void dfs(int node) {
    for (point p: points[node]) {
      grids.process_point(p);
    }

    if (node >= n) {
      if (node - n < orig_n) {
        sol[node - n] = grids.total_area;
      }
    } else {
      dfs(2 * node);
      dfs(2 * node + 1);
    }

    for (unsigned i = 0; i < points[node].size(); i++) {
      grids.rollback();
    }
  }
};

segment_tree st;
int n;

void read_intervals_into_segtree() {
  scanf("%d", &n);
  st.init(n);
  std::unordered_map<point, int, point_hash> map;

  for (int time = 0; time < n; time++) {
    point p;
    scanf("%d %d", &p.x, &p.y);
    p.x--;
    p.y--;
    auto it = map.find(p);
    if (it == map.end()) {
      map[p] = time;
    } else {
      st.add_point_on_range(p, it->second, time - 1);
      map.erase(p);
    }
  }
```

```
  for (auto key_val: map) {
    st.add_point_on_range(key_val.first, key_val.second, n - 1);
  }
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    printf("%lld%c", sol[i], (i == n - 1) ? '\n' : ' ');
  }
}

int main() {
  read_intervals_into_segtree();
  st.dfs(1);
  write_answers();

  return 0;
}
```

Anexa 5

Arbori indexați binar

5.1 Problema The Permutation Game Again (SPOJ)

înapoi

```
#include <stdio.h>

const int MAX_N = 200'000;
const int MOD = 1'000'000'007;

struct fenwick_tree {
  int v[MAX_N + 1];
  int n;

  void init(int n) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
      v[i] = 0;
    }
  }

  void check(int pos) {
    do {
      v[pos]++;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
```

```
  }
};

fenwick_tree fen;

void solve_test() {
  int n, x;
  scanf("%d", &n);
  fen.init(n);
  long long rank = 0;

  for (int place = n; place; place--) {
    scanf("%d", &x);
    int not_seen_before = x - 1 - fen.prefix_sum(x);
    rank = (rank * place + not_seen_before) % MOD;
    fen.check(x);
  }

  rank++; // Noi calculăm rangul începînd cu 0.

  printf("%lld\n", rank);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

5.2 Problema Multiset (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000'000;

struct fenwick_tree {
  int v[MAX_N + 1];
  int n, max_p2;

  void init(int n) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));
```

```
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  // Returnează poziția unde suma parțială atinge sau depășește sum.
  int bin_search(int sum) {
    int pos = 0;

    for (int interval = max_p2; interval; interval >>= 1) {
      if ((pos + interval <= n) && (v[pos + interval] < sum)) {
        sum -= v[pos + interval];
        pos += interval;
      }
    }

    return pos + 1;
  }

  void remove_kth_element(int k) {
    int pos = bin_search(k);
    add(pos, -1);
  }

  int get_smallest() {
    int pos = bin_search(1);
    return (pos <= n) ? pos : 0;
  }
};

fenwick_tree fen;
int n, num_ops;

void read_initial_multiset() {
  scanf("%d %d", &n, &num_ops);
  fen.init(n);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    fen.add(x, 1);
  }
}

void process_queries() {
  while (num_ops--) {
    int x;
```

```
    scanf("%d", &x);
    if (x > 0) {
      fen.add(x, 1);
    } else {
      fen.remove_kth_element(-x);
    }
  }
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_initial_multiset();
  process_queries();
  int answer = fen.get_smallest();
  write_answer(answer);

  return 0;
}
```

5.3 Problema Hanoi Factory (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;

struct ring {
  int in, out, h;
};

long long max(long long x, long long y) {
  return (x > y) ? x : y;
}

struct max_fenwick_tree {
  long long* v;
  int n;

  void init(long long* v, int n) {
    this->v = v;
    this->n = n;
    for (int i = 0; i <= n; i++) {
      v[i] = 0;
```

```
    }
  }

  void improve(int pos, long long val) {
    do {
      v[pos] = max(v[pos], val);
      pos += pos & -pos;
    } while (pos <= n);
  }

  long long prefix_max(int pos) {
    long long m = 0;
    while (pos) {
      m = max(m, v[pos]);
      pos &= pos - 1;
    }
    return m;
  }
};

ring r[MAX_N];
// folosit și pentru arborele fenwick, și pentru normalizarea mărimilor
long long v[2 * MAX_N + 1];
max_fenwick_tree fen;
int n;

void read_rings() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &r[i].in, &r[i].out, &r[i].h);
  }
}

void sort_rings() {
  std::sort(r, r + n, [](ring& a, ring& b) {
    return (a.out > b.out) || ((a.out == b.out) && (a.in > b.in));
  });
}

int bin_search(int val) {
  int l = 0, r = 2 * n;
  while (v[l] != val) { // valoarea există garantat în v
    int m = (l + r) / 2;
    if (v[m] > val) {
      r = m;
    } else {
      l = m;
    }
  }
  return l;
```

```
}

void normalize_diameters() {
  for (int i = 0; i < n; i++) {
    v[2 * i] = r[i].in;
    v[2 * i + 1] = r[i].out;
  }
  std::sort(v, v + 2 * n);

  for (int i = 0; i < n; i++) {
    r[i].in = 1 + bin_search(r[i].in);
    r[i].out = 1 + bin_search(r[i].out);
  }
}

long long solve_recurrence() {
  fen.init(v, 2 * n);

  long long max_height = 0;
  for (int i = 0; i < n; i++) {
    long long best = r[i].h + fen.prefix_max(r[i].out - 1);
    fen.improve(r[i].in, best);
    max_height = max(max_height, best);
  }

  return max_height;
}

void write_answer(long long answer) {
  printf("%lld\n", answer);
}

int main() {
  read_rings();
  sort_rings();
  normalize_diameters();
  long long answer = solve_recurrence();
  write_answer(answer);

  return 0;
}
```

5.4 Problema Subsequences (Codeforces)

‹ înapoi • versiune online

```
// Folosim un singur vector pentru cnt, înlocuind cîte un element pe rînd.
#include <stdio.h>
```

```
const int MAX_N = 100'000;

struct fenwick_tree {
  long long v[MAX_N + 1];
  int n;

  void init(int n) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
      v[i] = 0;
    }
  }

  void add(int pos, long long val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  long long prefix_sum(int pos) {
    long long s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }
};

fenwick_tree fen;
int a[MAX_N];
long long cnt[MAX_N];
int n, k;

void read_data() {
  scanf("%d %d", &n, &k);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
}

void iterate_lengths() {
  for (int i = 0; i < n; i++) {
    cnt[i] = 1;
  }

  while (k--) {
    fen.init(n);
```

```
    for (int i = 0; i < n; i++) {
      long long old_cnt = cnt[i];
      cnt[i] = fen.prefix_sum(a[i] - 1);
      fen.add(a[i], old_cnt);
    }
  }
}

long long array_sum(long long* v, int n) {
  long long sum = 0;
  for (int i = 0; i < n; i++) {
    sum += v[i];
  }
  return sum;
}

void write_answer(long long answer) {
  printf("%lld\n", answer);
}

int main() {
  read_data();
  iterate_lengths();
  long long total = array_sum(cnt, n);
  write_answer(total);

  return 0;
}
```

5.5 Problema D-query (SPOJ)

‹ înapoi

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 30000;
const int MAX_QUERIES = 200000;
const int MAX_VAL = 1000000;

struct query {
  short l, r;
  int orig_index;
  short answer;
};

struct fenwick_tree {
```

```
  short v[MAX_N + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  short prefix_sum(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
    }
    return sum;
  }

  short range_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
  }
};

int a[MAX_N + 1];
query q[MAX_QUERIES];
short prev_occur[MAX_VAL + 1];
fenwick_tree fen;
int n, num_queries;

void read_data() {
  scanf("%d", &n);
  for (int i = 1; i <= n; i++) {
    scanf("%d", &a[i]);
  }
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%hd %hd", &q[i].l, &q[i].r);
    q[i].orig_index = i;
  }
}

void sort_queries_by_right_end() {
  std::sort(q, q + num_queries, [](query& a, query& b) {
    return a.r < b.r;
  });
```

```
}

void update_last_occurrence(int pos) {
  if (prev_occur[a[pos]]) {
    fen.add(prev_occur[a[pos]], -1);
  }
  prev_occur[a[pos]] = pos;
  fen.add(pos, +1);
}

int answer_queries_ending_at(int right, int q_index) {
  while ((q_index < num_queries) && (q[q_index].r == right)) {
    q[q_index].answer = fen.range_sum(q[q_index].l, q[q_index].r);
    q_index++;
  }

  return q_index;
}

void scan_array() {
  fen.init(n);
  int q_index = 0;

  for (int i = 1; i <= n; i++) {
    update_last_occurrence(i);
    q_index = answer_queries_ending_at(i, q_index);
  }
}

void sort_queries_by_orig_index() {
  std::sort(q, q + num_queries, [](query& a, query& b) {
    return a.orig_index < b.orig_index;
  });
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", q[i].answer);
  }
}

int main() {
  read_data();
  sort_queries_by_right_end();
  scan_array();
  sort_queries_by_orig_index();
  write_answers();

  return 0;
```

```
}
```

5.6 Problema Magic Board (CodeChef)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_SIZE = 500'000;
const int MAX_TIME = 500'000;
const int MAX_WORD_LENGTH = 8;
enum op_type { OP_ROW_SET, OP_COL_SET, OP_ROW_QUERY, OP_COL_QUERY };

struct operation {
  op_type type;
  int index, value;
};

struct fenwick_tree {
  int v[MAX_TIME + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void set(int pos) {
    add(pos, +1);
  }

  void unset(int pos) {
    add(pos, -1);
  }

  int count(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
```

```
  }
};

struct line_info {
  int time[MAX_SIZE + 1];
  bool value[MAX_SIZE + 1];
  fenwick_tree change[2];
  int size, num_ops;

  void init(int size, int num_ops) {
    this->size = size;
    this->num_ops = num_ops;
    change[0].init(num_ops);
    change[1].init(num_ops);
  }

  void update(int index, int now, int new_value) {
    int old_value = value[index];
    int old_time = time[index];

    if (old_time) {
      change[old_value].unset(old_time);
    }

    change[new_value].set(now);
    time[index] = now;
    value[index] = new_value;
  }

  int count_zeroes(int index, int now, line_info& other) {
    int last_reset = time[index];
    if (!last_reset) {
      last_reset = num_ops;
    }
    int last_value = value[index];
    int changes = other.change[1 - last_value].count(last_reset);

    return (last_value == 0)
      ? (size - changes)
      : changes;
  }

  void query(int index, int now, line_info& other) {
    int num_zeroes = count_zeroes(index, now, other);
    printf("%d\n", num_zeroes);
  }

};

line_info rows, cols;
```

```
operation read_op() {
  char word[MAX_WORD_LENGTH + 1];
  operation op;

  scanf("%s", word);
  if (word[3] == 'Q') {
    op.type = (word[0] == 'R') ? OP_ROW_QUERY : OP_COL_QUERY;
    scanf("%d", &op.index);
  } else {
    op.type = (word[0] == 'R') ? OP_ROW_SET : OP_COL_SET;
    scanf("%d %d", &op.index, &op.value);
  }

  return op;
}

void process_op(operation op, int time) {
  if (op.type == OP_ROW_SET) {
    rows.update(op.index, time, op.value);
  } else if (op.type == OP_COL_SET) {
    cols.update(op.index, time, op.value);
  } else if (op.type == OP_ROW_QUERY) {
    rows.query(op.index, time, cols);
  } else { // OP_COL_QUERY
    cols.query(op.index, time, rows);
  }
}

void process_ops() {
  int size, num_ops;
  scanf("%d %d", &size, &num_ops);
  rows.init(size, num_ops);
  cols.init(size, num_ops);

  // Inversăm direcţia timpului astfel încît operaţiile din AIB-uri să fie pe
  // prefix, nu pe sufix.
  for (int time = num_ops; time; time--) {
    operation op = read_op();
    process_op(op, time);
  }
}

int main() {
  process_ops();

  return 0;
}
```

5.7 Problema Little Artem and Time Machine (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_QUERIES = 100'000;
const int T_INSERT = 1;
const int T_ERASE = 2;
const int NONE = -1;

struct fenwick_tree {
  int v[MAX_QUERIES + 1];
  int n;

  void init(int _n) {
    n = _n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }
};

struct query {
  int index;
  int type;
  int time;
  int val;
};

query q[MAX_QUERIES];
int answer[MAX_QUERIES];
fenwick_tree fen;
int n;
```

```
void read_queries() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &q[i].type, &q[i].time, &q[i].val);
    q[i].index = i + 1; // 1-based datorită indexării AIB-ului
  }
}

void sort_by_value_then_time() {
  std::sort(q, q + n, [](query& a, query& b) {
    return (a.val < b.val) ||
      ((a.val == b.val) && (a.time < b.time));
  });
}

void process_batch(int start, int end) {
  for (int i = start; i < end; i++) {
    switch (q[i].type) {
      case T_INSERT:
        fen.add(q[i].index, +1);
        break;
      case T_ERASE:
        fen.add(q[i].index, -1);
        break;
      default:
        answer[q[i].index - 1] = fen.prefix_sum(q[i].index);
    }
  }
}

void cleanup_batch(int start, int end) {
  for (int i = start; i < end; i++) {
    switch (q[i].type) {
      case T_INSERT:
        fen.add(q[i].index, -1);
        break;
      case T_ERASE:
        fen.add(q[i].index, +1);
        break;
      default:
        ; // nimic
    }
  }
}

void process_queries() {
  for (int i = 0; i < n; i++) {
    answer[i] = NONE;
  }
  fen.init(n);
```

```
  int start = 0;
  while (start < n) {
    int end = start + 1;
    while ((end < n) && (q[end].val == q[start].val)) {
      end++;
    }
    process_batch(start, end);
    cleanup_batch(start, end);
    start = end;
  }
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    if (answer[i] != NONE) {
      printf("%d\n", answer[i]);
    }
  }
}

int main() {
  read_queries();
  sort_by_value_then_time();
  process_queries();
  write_answers();

  return 0;
}
```

5.8 Problema Ball (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_LADIES = 500'000;

struct lady {
  int x, y, z;
};

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct suffix_max_fenwick_tree {
```

```
  int v[MAX_LADIES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void update(int pos, int val) {
    pos = n + 1 - pos;
    do {
      v[pos] = max(v[pos], val);
      pos += pos & -pos;
    } while (pos <= n);
  }

  int suffix_max(int pos) {
    pos = n + 1 - pos;
    int result = 0;
    while (pos) {
      result = max(result, v[pos]);
      pos &= pos - 1;
    }
    return result;
  }
};

lady l[MAX_LADIES];
suffix_max_fenwick_tree fen;
int pos[MAX_LADIES]; // folosit pentru normalizare
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &l[i].x);
  }
  for (int i = 0; i < n; i++) {
    scanf("%d", &l[i].y);
  }
  for (int i = 0; i < n; i++) {
    scanf("%d", &l[i].z);
  }
}

void normalize_x() {
  for (int i = 0; i < n; i++) {
    pos[i] = i;
  }

  std::sort(pos, pos + n, [](int a, int b) {
```

```
    return l[a].x < l[b].x;
  });

  int old_x = l[pos[0]].x, new_x = 1;
  for (int i = 0; i < n; i++) {
    if (l[pos[i]].x != old_x) {
      old_x = l[pos[i]].x;
      new_x++;
    }
    l[pos[i]].x = new_x;
  }
}

void sort_ladies_by_z_desc_x_asc() {
  std::sort(l, l + n, [](lady& a, lady& b) {
    return (a.z > b.z) || ((a.z == b.z) && (a.x < b.x));
  });
}

int count_self_murderers() {
  fen.init(n);

  int result = 0;
  for (int i = 0; i < n; i++) {
    int prev_max_y = fen.suffix_max(l[i].x + 1);
    result += (prev_max_y > l[i].y);
    fen.update(l[i].x, l[i].y);
  }

  return result;
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  normalize_x();
  sort_ladies_by_z_desc_x_asc();
  int answer = count_self_murderers();
  write_answer(answer);

  return 0;
}
```

Anexa 6

Structuri 2D offline

6.1 Problema Festival (ONI 2025, clasele 11-12)

‹ înapoi • versiune online

```cpp
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_N = 100'000;
const int MAX_SEGTREE_NODES = 1 << 18;
const int INFINITY = 2'000'000'000;

struct festival {
  int x, y, sat; // timpul este y
};

struct normalizer {
  int v[MAX_N];
  int n;

  void add(int val) {
    v[n++] = val;
  }

  void sort() {
    std::sort(v, v + n);
    n = std::unique(v, v + n) - v;
  }

  int get_first_gte(int val) {
    return std::lower_bound(v, v + n, val) - v;
  }

  int get_last_lte(int val) {
```

```
    return std::upper_bound(v, v + n, val) - v - 1;
  }

  // Syntactic sugar pentru cînd val există garantat în v.
  int get(int val) {
    return get_first_gte(val);
  }
};

// Nodurile din AINT sînt AIB-uri.
struct segment_tree_node {
  std::vector<int> ys;
  std::vector<long long> values;

  void reserve_y(int y) {
    ys.push_back(y);
  }

  void build() {
    ys.push_back(-INFINITY); // pentru indexarea de la 1
    std::sort(ys.begin(), ys.end());
    auto last = std::unique(ys.begin(), ys.end());
    ys.erase(last, ys.end());
    values.resize(ys.size());
  }

  // Atenție, nu este garantat că y există în ys. Caută ultimul ys <= y.
  int find_y(int y) {
    return std::upper_bound(ys.begin(), ys.end(), y) - ys.begin() - 1;
  }

  void maximize_val(int y, long long val) {
    unsigned pos = find_y(y);
    do {
      values[pos] = std::max(values[pos], val);
      pos += pos & -pos;
    } while (pos < values.size());
  }

  long long prefix_max(int y) {
    unsigned pos = find_y(y);
    long long m = 0;
    while (pos) {
      m = std::max(m, values[pos]);
      pos &= pos - 1;
    }
    return m;
  }
};
```

```
int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct segment_tree {
  segment_tree_node v[MAX_SEGTREE_NODES];
  int n;

  void init(int size) {
    n = next_power_of_2(size);
  }

  void reserve_point(int pos, int y) {
    for (pos += n; pos; pos /= 2) {
      v[pos].reserve_y(y);
    }
  }

  void build() {
    for (int pos = 1; pos < 2 * n; pos++) {
      v[pos].build();
    }
  }

  void maximize_val(int pos, int y, long long val) {
    for (pos += n; pos; pos /= 2) {
      v[pos].maximize_val(y, val);
    }
  }

  long long rectangle_max(int l, int r, int max_y) {
    long long result = 0;
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        result = std::max(result, v[l++].prefix_max(max_y));
      }
      l >>= 1;

      if (!(r & 1)) {
        result = std::max(result, v[r--].prefix_max(max_y));
      }
      r >>= 1;
    }

    return result;
  }
};
```

```
festival f[MAX_N];
normalizer norm;
segment_tree left_st, right_st;
long long answer;
int n, d;

void read_data() {
  FILE* fin = fopen("festival.in", "r");
  fscanf(fin, "%d %d", &n, &d);
  for (int i = 0; i < n; i++) {
    fscanf(fin, "%d %d %d", &f[i].y, &f[i].x, &f[i].sat);
  }
  fclose(fin);
}

void normalize_x() {
  for (int i = 0; i < n; i++) {
    norm.add(f[i].x);
  }
  norm.sort();
}

void sort_festivals_by_y() {
  std::sort(f, f + n, [](festival a, festival b) {
    return a.y < b.y;
  });
}

void dry_run() {
  left_st.init(norm.n);
  right_st.init(norm.n);

  for (int i = 0; i < n; i++) {
    int nx = norm.get(f[i].x);
    left_st.reserve_point(nx, f[i].y - f[i].x);
    right_st.reserve_point(nx, f[i].y + f[i].x);
  }

  left_st.build();
  right_st.build();
}

void process_festivals() {
  for (int i = 0; i < n; i++) {
    int nx = norm.get(f[i].x);
    int l_d = norm.get_first_gte(f[i].x - d);
    int r_d = norm.get_last_lte(f[i].x + d);
    long long l_score = left_st.rectangle_max(l_d, nx, f[i].y - f[i].x);
    long long r_score = right_st.rectangle_max(nx, r_d, f[i].y + f[i].x);
```

```
    long long score = f[i].sat + std::max(l_score, r_score);
    answer = std::max(answer, score);
    left_st.maximize_val(nx, f[i].y - f[i].x, score);
    right_st.maximize_val(nx, f[i].y + f[i].x, score);
  }
}

void write_answer() {
  FILE* f = fopen("festival.out", "w");
  fprintf(f, "%lld\n", answer);
}

int main() {
  read_data();
  normalize_x();
  sort_festivals_by_y();
  dry_run();
  process_festivals();
  write_answer();

  return 0;
}
```

6.2 Problema Medwalk revizitată (Lot 2025)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_N = 100'000;
const int MAX_QUERIES = 100'000;
const int MAX_VALUE = 300'000;
const int MAX_SEGTREE_NODES = 1 << 18;
const int INF = 1'000'000;
const int OP_UPDATE = 1;

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct matrix {
  int v[2][MAX_N + 1];
```

```
  void set(int row, int col, int val) {
    v[row][col] = val;
  }

  int get_min(int col) {
    return min(v[0][col], v[1][col]);
  }

  int get_max(int col) {
    return max(v[0][col], v[1][col]);
  }
};

struct query {
  int type;
  // Syntactic sugar ca să putem folosi <row, col, val> sau <left, right>, nu
  // <x, y, z>.
  union { int row; int left; };
  union { int col; int right; };
  int val;
};

matrix mat;
query q[MAX_QUERIES];
int n, num_queries;

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

// Admite actualizări punctuale și interogări de minim pe interval.
struct min_segment_tree {
  int v[MAX_SEGTREE_NODES];
  int n;

  void init(int size) {
    n = next_power_of_2(size);
  }

  void update(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = min(v[2 * pos], v[2 * pos + 1]);
    }
  }

  int range_min(int l, int r) {
    l += n;
```

```
    r += n;
    int result = INF;

    while (l <= r)  {
      if (l & 1) {
        result = min(result, v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = min(result, v[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

// Kudos https://usaco.guide/plat/2DRQ?lang=cpp#offline-2d-bit și
// https://kilonova.ro/submissions/752782
//
// Arbore Fenwick 2D offline.
struct fenwick_2d {
  // Valorile distincte pe fiecare coloană.
  std::vector<int> col_rows[MAX_VALUE + 1];

  // Arborele 1D pe fiecare coloană, peste valorile existente.
  std::vector<int> col_fen[MAX_VALUE + 1];

  int n, max_p2;

  void init(int n) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));
    for (int i = 0; i <= n; i++) {
      col_rows[i].push_back(0);
    }
  }

  // Notează faptul că rîndul row va fi folosit cel puțin o dată pe coloana
  // col.
  void reserve(int row, int col) {
    do {
      col_rows[col].push_back(row);
      col += col & -col;
    } while (col <= n);
  }

  void build() {
```

```
  for (int col = 1; col <= n; col++) {
    std::vector<int>& v = col_rows[col]; // syntactic sugar
    std::sort(v.begin(), v.end());
    v.erase(std::unique(v.begin(), v.end()), v.end());
    col_fen[col].resize(v.size() + 1);
  }
}

int leftmost_gte(int row, int col) {
  std::vector<int>& v = col_rows[col];
              return std::lower_bound(v.begin(), v.end(), row) - v.begin();
}

// Adaugă delta (bifează / debifează) pentru rîndul row și toți succesorii
// săi în fiecare AIB 1D, atît pe coloana col cît și pe toate coloanele
// succesoare în AIB-ul 2D.
void add(int row, int col, int delta) {
  do {
    int pos = leftmost_gte(row, col);
    do {
      col_fen[col][pos] += delta;
      pos += pos & -pos;
    } while (pos <= (int)col_fen[col].size());
    col += col & -col;
  } while (col <= n);
}

int prefix_sum(int pos, int col) {
  int s = 0;
  while (pos) {
    s += col_fen[col][pos];
    pos &= pos - 1;
  }
  return s;
}

int count_less_than(int val, int col) {
  // Reminder: AIB-ul 1D este normalizat. Află pe ce rînd se află val.
  int pos = leftmost_gte(val, col);
  return prefix_sum(pos - 1, col);
}

int count_in_range(int col, int l, int r) {
  return count_less_than(r + 1, col) - count_less_than(l, col);
}

// Contract: k este 0-based, iar [l, r] este un interval închis de valori
// (rînduri).
int kth_element(int k, int l, int r) {
  int col = 0;
```

```
    for (int interval = max_p2; interval; interval >>= 1) {
      if (col + interval <= n) {
        int cnt = count_in_range(col + interval, l, r);
        if (cnt <= k) {
          k -= cnt;
          col += interval;
        }
      }
    }

    return col + 1;
  }
};

min_segment_tree maxima;
fenwick_2d minima;

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int r = 0; r < 2; r++) {
    for (int c = 1; c <= n; c++) {
      int x;
      scanf("%d", &x);
      mat.set(r, c, x);
    }
  }

  for (int i = 0; i < num_queries; i++) {
    query& z = q[i];
    scanf("%d", &z.type);
    if (z.type == OP_UPDATE) {
      scanf("%d %d %d", &z.row, &z.col, &z.val);
      z.row--;
    } else {
      scanf("%d %d", &z.left, &z.right);
    }
  }
}

void build_maxima_segtree() {
  maxima.init(n + 1);
  for (int i = 1; i <= n; i++) {
    maxima.update(i, mat.get_max(i));
  }
}

void simulate() {
  // Rezervă minimele inițiale.
  for (int c = 1; c <= n; c++) {
```

```
    minima.reserve(c, mat.get_min(c));
  }

  // Rezervă minimele care iau naștere prin actualizări.
  matrix mat_copy = mat;
  for (int i = 0; i < num_queries; i++) {
    if (q[i].type == OP_UPDATE) {
      mat_copy.set(q[i].row, q[i].col, q[i].val);
      minima.reserve(q[i].col, mat_copy.get_min(q[i].col));
    }
  }
}

void build_fenwick() {
  minima.init(MAX_VALUE);
  simulate();
  minima.build();

  for (int c = 1; c <= n; c++) {
    minima.add(c, mat.get_min(c), +1);
  }
}

void update(int row, int col, int val) {
  int old_min = mat.get_min(col);
  mat.set(row, col, val);
  int new_min = mat.get_min(col);

  maxima.update(col, mat.get_max(col));

  if (new_min != old_min) {
    minima.add(col, old_min, -1);
    minima.add(col, new_min, +1);
  }
}

int query(int left, int right) {
  int len = right - left + 2;
  int median_pos = (len - 1) / 2;
  int median = minima.kth_element(median_pos, left, right);
  int best_max = maxima.range_min(left, right);

  if (best_max >= median) {
    return median;
  } else {
    // best_max trebuie inserat înainte de median_pos. Răspunsul poate fi la
    // poziția median_pos - 1 sau poate fi chiar best_max.
    int prev = minima.kth_element(median_pos - 1, left, right);
    return max(prev, best_max);
  }
```

```
}

void process_queries() {
  for (int i = 0; i < num_queries; i++) {
    if (q[i].type == OP_UPDATE) {
      update(q[i].row, q[i].col, q[i].val);
    } else {
      printf("%d\n", query(q[i].left, q[i].right));
    }
  }
}

int main() {
  read_data();
  build_maxima_segtree();
  build_fenwick();
  process_queries();

  return 0;
}
```

Anexa 7

Descompunere în radical

7.1 Problema Mexitate (ONI 2018 clasa a 9-a)

înapoi

Sursă naivă (versiune online).

```
#include <stdio.h>

const int MAX_ELEMS = 400'000;
const int MOD = 1'000'000'007;

// Costul calculării funcției mex() este mare. Versiunea 2 va folosi o
// structură mai eficientă pentru frequency_tracker.
struct frequency_tracker {
  int f[MAX_ELEMS + 2]; // rows * cols + 1 în cel mai rău caz

  void add(int x) {
    f[x]++;
  }

  void remove(int x) {
    f[x]--;
  }

  int mex() {
    int x = 1;
    while (f[x]) {
      x++;
    }
    return x;
  }
};

int mat[MAX_ELEMS];
```

```
frequency_tracker ft;
int rows, cols, k, l;

void swap(int& x, int& y) {
  int tmp = x;
  x = y;
  y = tmp;
}

void read_right_matrix(FILE* f) {
  for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
      fscanf(f, "%d", &mat[r * cols + c]);
    }
  }
}

void read_transposed_matrix(FILE* f) {
  for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
      fscanf(f, "%d", &mat[c * rows + r]);
    }
  }
}

void read_data() {
  FILE* f = fopen("mexitate.in", "r");
  fscanf(f, "%d %d %d %d", &rows, &cols, &k, &l);
  if (k <= l) {
    read_right_matrix(f);
  } else {
    read_transposed_matrix(f);
    swap(rows, cols);
    swap(k, l);
  }
  fclose(f);
}

int get(int r, int c) {
  return mat[r * cols + c];
}

int north_west_corner() {
  for (int r = 0; r < k; r++) {
    for (int c = 0; c < l; c++) {
      ft.add(get(r, c));
    }
  }
  return ft.mex();
}
```

```
void move_right(int r, int c) {
  for (int i = r; i < r + k; i++) {
    ft.remove(get(i, c));
    ft.add(get(i, c + l));
  }
}

void move_left(int r, int c) {
  for (int i = r; i < r + k; i++) {
    ft.remove(get(i, c + l - 1));
    ft.add(get(i, c - 1));
  }
}

void move_down(int r, int c) {
  for (int j = c; j < c + l; j++) {
    ft.remove(get(r, j));
    ft.add(get(r + k, j));
  }
}


int left_right_snake() {
  north_west_corner();
  long long result = ft.mex();
  int r = 0, c = 0;
  int final_r = rows - k;
  int final_c = (final_r % 2) ? 0 : (cols - l);

  while ((r != final_r) || (c != final_c)) {
    if ((r % 2 == 0) && (c < cols - l)) {
      move_right(r, c++);
    } else if ((r % 2 == 1) && (c > 0)) {
      move_left(r, c--);
    } else {
      move_down(r++, c);
    }
    result = result * ft.mex() % MOD;
  }

  return result;
}

void write_answer(int answer) {
  FILE* f = fopen("mexitate.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}
```

```
int main() {
  read_data();
  int answer = left_right_snake();
  write_answer(answer);

  return 0;
}
```

Sursă eficientă (versiune online).

```
#include <stdio.h>

const int MAX_ELEMS = 400'000;
const int BLOCK_SIZE = 400;
const int MAX_BLOCKS = (MAX_ELEMS - 1) / BLOCK_SIZE + 1;
const int MOD = 1'000'000'007;

struct frequency_tracker {
  int f[MAX_ELEMS + 2]; // rows * cols + 1 în cel mai rău caz
  int block_nonzero[MAX_BLOCKS];

  void init() {
    add(0); // Ca să nu-l returnăm niciodată.
  }

  void add(int x) {
    if (++f[x] == 1) {
      block_nonzero[x / BLOCK_SIZE]++;
    }
  }

  void remove(int x) {
    if (--f[x] == 0) {
      block_nonzero[x / BLOCK_SIZE]--;
    }
  }

  int mex() {
    int b = 0;
    while (block_nonzero[b] == BLOCK_SIZE) {
      b++;
    }
    int x = b * BLOCK_SIZE;
    while (f[x]) {
      x++;
    }
    return x;
  }
};
```

```
int mat[MAX_ELEMS];
frequency_tracker ft;
int rows, cols, k, l;

void swap(int& x, int& y) {
  int tmp = x;
  x = y;
  y = tmp;
}

void read_right_matrix(FILE* f) {
  for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
      fscanf(f, "%d", &mat[r * cols + c]);
    }
  }
}

void read_transposed_matrix(FILE* f) {
  for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
      fscanf(f, "%d", &mat[c * rows + r]);
    }
  }
}

void read_data() {
  FILE* f = fopen("mexitate.in", "r");
  fscanf(f, "%d %d %d %d", &rows, &cols, &k, &l);
  if (k <= l) {
    read_right_matrix(f);
  } else {
    read_transposed_matrix(f);
    swap(rows, cols);
    swap(k, l);
  }
  fclose(f);
}

int get(int r, int c) {
  return mat[r * cols + c];
}

int north_west_corner() {
  for (int r = 0; r < k; r++) {
    for (int c = 0; c < l; c++) {
      ft.add(get(r, c));
    }
  }
```

```
  return ft.mex();
}

void move_right(int r, int c) {
  for (int i = r; i < r + k; i++) {
    ft.remove(get(i, c));
    ft.add(get(i, c + l));
  }
}

void move_left(int r, int c) {
  for (int i = r; i < r + k; i++) {
    ft.remove(get(i, c + l - 1));
    ft.add(get(i, c - 1));
  }
}

void move_down(int r, int c) {
  for (int j = c; j < c + l; j++) {
    ft.remove(get(r, j));
    ft.add(get(r + k, j));
  }
}


int left_right_snake() {
  ft.init();
  north_west_corner();
  long long result = ft.mex();
  int r = 0, c = 0;
  int final_r = rows - k;
  int final_c = (final_r % 2) ? 0 : (cols - l);

  while ((r != final_r) || (c != final_c)) {
    if ((r % 2 == 0) && (c < cols - l)) {
      move_right(r, c++);
    } else if ((r % 2 == 1) && (c > 0)) {
      move_left(r, c--);
    } else {
      move_down(r++, c);
    }
    result = result * ft.mex() % MOD;
  }

  return result;
}

void write_answer(int answer) {
  FILE* f = fopen("mexitate.out", "w");
  fprintf(f, "%d\n", answer);
```

```
  fclose(f);
}

int main() {
  read_data();
  int answer = left_right_snake();
  write_answer(answer);

  return 0;
}
```

7.2 Problema Give Away (SPOJ)

înapoi

Sursă cu multiseturi PBDS.

```
// Complexitate: O(Q sqrt N log N).
//
// Metodă: Descompunere în radical. Pe fiecare bloc reține vectorul naiv și un
// set de valori pentru căutări în timp logaritmic. Avem nevoie de multisets
// pentru că pot exista duplicate și avem nevoie de PBDS pentru funcția
// order_of_key.
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <stdio.h>

const int MAX_N = 500000;
const int BLOCK_SIZE = 10000;
const int MAX_BLOCKS = (MAX_N - 1) / BLOCK_SIZE + 1;
const int T_QUERY = 0;

// Multiseturile PBDS sînt obscene, dar par să meargă. Ștergerile necesită cod
// extra. Vezi https://stackoverflow.com/q/59731946/6022817
typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less_equal<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > ordered_set;

// Toate intervalele sînt [închis, deschis).
struct block {
  int v[BLOCK_SIZE];
  ordered_set s;

  void set(int pos, int val) {
```

```
    if (v[pos]) {
      int rank = s.order_of_key(v[pos]);
      ordered_set::iterator it = s.find_by_order(rank);
      s.erase(it);
    }
    v[pos] = val;
    s.insert(v[pos]);
  }

  int partial_count(int l, int r, int val) {
    int cnt = 0;
    while (l < r) {
      cnt += (v[l++] >= val);
    }
    return cnt;
  }

  int prefix_count(int end, int val) {
    return partial_count(0, end, val);
  }

  int suffix_count(int start, int val) {
    return partial_count(start, BLOCK_SIZE, val);
  }

  int whole_count(int val) {
    return s.size() - s.order_of_key(val);
  }
};

block b[MAX_BLOCKS];
int n;

void read_array() {
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    b[i / BLOCK_SIZE].set(i % BLOCK_SIZE, x);
  }
}

int process_query(int l, int r, int val) {
  int bl = l / BLOCK_SIZE, offset_l = l % BLOCK_SIZE;
  int br = r / BLOCK_SIZE, offset_r = r % BLOCK_SIZE;
  if (bl == br) {
    return b[bl].partial_count(offset_l, offset_r, val);
  } else {
    int cnt = b[bl].suffix_count(offset_l, val)
```

```
      + b[br].prefix_count(offset_r, val);
    for (int i = bl + 1; i < br; i++) {
      cnt += b[i].whole_count(val);
    }
    return cnt;
  }
}

void process_update(int pos, int val) {
  b[pos / BLOCK_SIZE].set(pos % BLOCK_SIZE, val);
}

void process_ops() {
  int num_ops, type, pos1, pos2, val;
  scanf("%d", &num_ops);

  while (num_ops--) {
    scanf("%d", &type);
    if (type == T_QUERY) {
      scanf("%d %d %d", &pos1, &pos2, &val);
      int count = process_query(pos1 - 1, pos2, val);
      printf("%d\n", count);
    } else {
      scanf("%d %d", &pos1, &val);
      process_update(pos1 - 1, val);
    }
  }
}

int main() {
  read_array();
  process_ops();

  return 0;
}
```

Sursă elementară.

```
// Complexitate: O(Q sqrt N log N).
//
// Metodă: Descompunere în radical. Pe fiecare bloc reține vectorul naiv și un
// vector sortat pentru căutări în timp logaritmic.
#include <algorithm>
#include <stdio.h>

const int MAX_N = 500'000;
const int BLOCK_SIZE = 3'000;
const int MAX_BLOCKS = (MAX_N - 1) / BLOCK_SIZE + 1;
const int T_QUERY = 0;
```

```
// Toate intervalele sînt [închis, deschis).
struct block {
  int v[BLOCK_SIZE];
  int s[BLOCK_SIZE];
  int size;

  void push(int val) {
    v[size] = s[size] = val;
    size++;
  }

  void sort() {
    std::sort(s, s + size);
  }

  // Returnează cea mai din stînga poziție a unui element >= val.
  int bin_search(int val) {
    if (val < s[0]) {
      return 0;
    } else if (val > s[size - 1]) {
      return size;
    }
    int l = -1, r = size - 1; // (l, r]

    while (r - l > 1) {
      int mid = (l + r) >> 1;
      if (s[mid] < val) {
        l = mid;
      } else {
        r = mid;
      }
    }

    return r;
  }

  void migrate_left(int pos) {
    int save = s[pos];
    while (pos && (s[pos - 1] > save)) {
      s[pos] = s[pos - 1];
      pos--;
    }
    s[pos] = save;
  }

  void migrate_right(int pos) {
    int save = s[pos];
    while ((pos < size - 1) && (s[pos + 1] < save)) {
      s[pos] = s[pos + 1];
```

```
      pos++;
    }
    s[pos] = save;
  }

  void set(int pos, int val) {
    int sorted_pos = bin_search(v[pos]);
    s[sorted_pos] = val;
    migrate_left(sorted_pos);
    migrate_right(sorted_pos);
    v[pos] = val;
  }

  int partial_count(int l, int r, int val) {
    int cnt = 0;
    while (l < r) {
      cnt += (v[l++] >= val);
    }
    return cnt;
  }

  int prefix_count(int end, int val) {
    return partial_count(0, end, val);
  }

  int suffix_count(int start, int val) {
    return partial_count(start, BLOCK_SIZE, val);
  }

  int whole_count(int val) {
    return size - bin_search(val);
  }
};

block b[MAX_BLOCKS];
int n;

void read_array() {
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    b[i / BLOCK_SIZE].push(x);
  }

  int end_block = (n - 1) / BLOCK_SIZE + 1;
  for (int i = 0; i < end_block; i++) {
    b[i].sort();
  }
```

```
}

int process_query(int l, int r, int val) {
  int bl = l / BLOCK_SIZE, offset_l = l % BLOCK_SIZE;
  int br = r / BLOCK_SIZE, offset_r = r % BLOCK_SIZE;
  if (bl == br) {
    return b[bl].partial_count(offset_l, offset_r, val);
  } else {
    int cnt = b[bl].suffix_count(offset_l, val)
      + b[br].prefix_count(offset_r, val);
    for (int i = bl + 1; i < br; i++) {
      cnt += b[i].whole_count(val);
    }
    return cnt;
  }
}

void process_update(int pos, int val) {
  b[pos / BLOCK_SIZE].set(pos % BLOCK_SIZE, val);
}

void process_ops() {
  int num_ops, type, pos1, pos2, val;
  scanf("%d", &num_ops);

  while (num_ops--) {
    scanf("%d", &type);
    if (type == T_QUERY) {
      scanf("%d %d %d", &pos1, &pos2, &val);
      int count = process_query(pos1 - 1, pos2, val);
      printf("%d\n", count);
    } else {
      scanf("%d %d", &pos1, &val);
      process_update(pos1 - 1, val);
    }
  }
}

int main() {
  read_array();
  process_ops();

  return 0;
}
```

7.3 Problema Holes (Codeforces)

❮ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 100'000;
// Preferăm blocuri puțin mai mari deoarece actualizările au *cache locality*,
// pe cînd interogările sar mai mult.
const int BUCKET_SIZE = 1'000;
const int OP_UPDATE = 0;

struct hole {
  int power;
  int last;  // ultima destinație din același bloc
  int jumps; // numărul de salturi pînă la last
};

hole h[MAX_N + 1];
int n, num_queries;

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int i = 0; i < n; i++) {
    scanf("%d", &h[i].power);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

// Actualizează găurile de la începutul blocului pînă la pos inclusiv.
void update_bucket_of(int pos) {
  int start = pos / BUCKET_SIZE * BUCKET_SIZE;
  int end = min(start + BUCKET_SIZE, n);
  for (int i = pos; i >= start; i--) {
    int dest = i + h[i].power;
    if (dest < end) {
      h[i].last = h[dest].last;
      h[i].jumps = h[dest].jumps + 1;
    } else {
      h[i].last = i;
      h[i].jumps = 0;
    }
  }
}

void init_buckets() {
  for (int start = 0; start < n; start += BUCKET_SIZE) {
    int end = min(start + BUCKET_SIZE - 1, n - 1);
    update_bucket_of(end);
  }
```

```
}

void query(int pos, int* last, int* count) {
  *count = 0;

  do {
    *count += h[pos].jumps + 1;
    *last = h[pos].last;
    pos = *last + h[*last].power;
  } while (pos < n);
}

void process_queries() {
  while (num_queries--) {
    int type, a;
    scanf("%d %d", &type, &a);
    a--;

    if (type == OP_UPDATE) {
      int power;
      scanf("%d", &power);
      h[a].power = power;
      update_bucket_of(a);
    } else {
      int last, num_jumps;
      query(a, &last, &num_jumps);
      printf("%d %d\n", 1 + last, num_jumps);
    }
  }
}

int main() {
  read_data();
  init_buckets();
  process_queries();

  return 0;
}
```

7.4 Problema Piezișă (Baraj ONI 2022)

‹ înapoi

Sursă forță brută (versiune online).

```
#include <algorithm>
#include <stdio.h>
```

```cpp
#define MAX_N 500000
#define INFINITY (MAX_N + 1)
#define NONE -1

int v[MAX_N + 1]; // partial xor's
int pos[MAX_N + 1];
int first[MAX_N + 2];
int n, distinct;

int max(int x, int y) {
  return (x > y) ? x : y;
}

// Returnează cea mai din dreapta apariție a lui val pe poziția p sau înainte,
// sau NONE dacă val nu apare pe poziția p sau înainte.
int rightmost(int val, int p) {
  int l = first[val], r = first[val + 1]; // [l, r)
  while (r - l > 1) {
    int m = (l + r) >> 1;
    if (pos[m] > p) {
      r = m;
    } else {
      l = m;
    }
  }

  // Caz special: p < orice poziție unde apare val. Căutarea binară normală ar
  // returna pos[l].
  return (pos[l] <= p) ? pos[l] : NONE;
}

int main() {
  FILE* fin = fopen("piezisa.in", "r");
  FILE* fout = fopen("piezisa.out", "w");
  fscanf(fin, "%d", &n);

  // Citește datele și calculează xor-uri parțiale.
  v[0] = 0;
  for (int i = 1; i <= n; i++) {
    int x;
    fscanf(fin, "%d", &x);
    v[i] = v[i - 1] ^ x;
  }

  // Sortează pozițiile după valoarea xor parțială, apoi după poziție.
  for (int i = 0; i <= n; i++) {
    pos[i] = i;
  }
  std::sort(pos, pos + n + 1, [](int a, int b) {
    return (v[a] < v[b]) || ((v[a] == v[b]) && (a < b));
```

```
  });

  // Renumerotează valorile începînd cu n; colectează pozițiile.
  int from = -1;
  distinct = 0;
  for (int i = 0; i <= n; i++) {
    if (v[pos[i]] != from) {
      from = v[pos[i]];
      first[distinct++] = i;
    }
    v[pos[i]] = distinct - 1;
  }
  first[distinct] = n + 1;
  // Acum pos[first[i]...first[i+1]] conține o listă ordonată cu pozițiile
  // aparițiilor valorii i.

  int num_queries;
  fscanf(fin, "%d", &num_queries);
  while (num_queries--) {
    int l, r;
    fscanf(fin, "%d %d", &l, &r);
    r++;

    int end = r, best = INFINITY;
    // cît timp avem loc să avansăm și sperăm să îmbunătățim soluția existentă
    while ((end <= n) && (end - l < best)) {
      int start = rightmost(v[end], l);
      if ((start != NONE) && (end - start < best)) {
        best = end - start;
      }
      end++;
    }

    fprintf(fout, "%d\n", (best == INFINITY) ? NONE : best);
  }

  fclose(fin);
  fclose(fout);

  return 0;
}
```

Sursă cu metoda 1 (versiune online).

```
#include <algorithm>
#include <math.h>
#include <stdio.h>

const int MAX_BUCKETS = 354;
```

```
const int MAX_BUCKET_SIZE = 1416;
const int MAX_N = MAX_BUCKETS * MAX_BUCKET_SIZE;
const int MAX_Q = 500000;
const int INF = MAX_N + 1;
const int NONE = -1;

struct query {
  int l, r, orig_index;
};

int v[MAX_N + 1]; // xor-uri parțiale
int pos[MAX_N + 1];
int ptr[MAX_N + 1];
int last[MAX_N + 1];
int best[MAX_BUCKETS];

query q[MAX_Q];
// Logic ar trebui stocat în query.answer, dar astfel evităm o sortare.
int answer[MAX_Q];

int n, num_queries;
int bs, nb;

void read_array() {
  scanf("%d", &n);

  v[0] = 0;
  for (int i = 1; i <= n; i++) {
    scanf("%d", &v[i]);
    v[i] ^= v[i - 1];
  }
}

void read_queries() {
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].l, &q[i].r);
    q[i].r++;
    q[i].orig_index = i;
  }
}

void sort_queries() {
  std::sort(q, q + num_queries, [](query a, query b) {
    return (a.l < b.l);
  });
}

void sort_positions() {
  for (int i = 0; i <= n; i++) {
```

```
    pos[i] = i;
  }
  std::sort(pos, pos + n + 1, [](int a, int b) {
    return (v[a] < v[b]) || ((v[a] == v[b]) && (a > b));
  });
}

void preprocess_values() {
  nb = 0.5 * sqrt(n + 1); // determinat experimental
  bs = n / nb + 1;

  sort_positions();

  // renumerotează valorile de la 0; creează pointeri
  int prev = -1, distinct = 0;
  for (int i = 0; i <= n; i++) {
    if (v[pos[i]] == prev) {
      v[pos[i]] = distinct - 1;
      bool same_bucket = (pos[i] / bs == pos[i - 1] / bs);
      ptr[pos[i]] = same_bucket
        ? ptr[pos[i - 1]]
        : pos[i - 1];
    } else {
      // începi o serie nouă de la sfîrșitul vectorului
      prev = v[pos[i]];
      v[pos[i]] = distinct++;
      ptr[pos[i]] = NONE;
    }
  }
}

inline int min(int x, int y) {
  return (x < y) ? x : y;
}

int answer_query(int l, int r) {
  int result = INF;

  // procedează incremental pînă la blocul următor
  int b = r / bs + 1, boundary = min(b * bs, n + 1);
  while (r < boundary) {
    result = min(result, r - last[v[r]]);
    r++;
  }

  // procedează bloc cu bloc restul vectorului
  while (b < nb) {
    result = min(result, best[b++]);
  }
```

```
  return (result == INF) ? NONE : result;
}

void scan() {
  for (int i = 0; i <= n; i++) {
    last[i] = -INF;
  }
  for (int i = 0; i < nb; i++) {
    best[i] = INF;
  }

  int qi = 0;
  for (int l = 0; l <= n; l++) {
    // actualizează-l pe last
    last[v[l]] = l;

    // actualizează-l pe best
    for (int i = ptr[l]; i != NONE; i = ptr[i]) {
      int b = i / bs;
      best[b] = min(best[b], i - l);
    }

    // răspunde la interogările care încep la l
    while (qi < num_queries && q[qi].l == l) {
      answer[q[qi].orig_index] = answer_query(q[qi].l, q[qi].r);
      qi++;
    }
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", answer[i]);
  }
}

int main() {
  read_array();
  read_queries();
  sort_queries();
  preprocess_values();
  scan();
  write_answers();

  return 0;
}
```

Sursă cu metoda 2 (versiune online).

```cpp
// Rescrisă după https://infoarena.ro/job_detail/3032904
//
// Complexitate: O((N + Q) sqrt(N)).
#include <algorithm>
#include <stdio.h>

const int MAX_N = 500'000;
const int BLOCK_SIZE = 700;
const int NUM_BLOCKS = MAX_N / BLOCK_SIZE + 1;
const int MAX_Q = 500'000;
const int INFINITY = 1'000'000;
const int NONE = -1;

struct query {
  int l, r;
  int orig_index;
  int block;
};

int v[MAX_N + 1];
query q[MAX_Q];
int left[MAX_N + 1], right[MAX_N + 1];
int* pos = left; // folosit inițial pentru normalizare;

// Logic ar trebui stocat în query.answer, dar astfel evităm o sortare.
int answer[MAX_Q];

int n, num_queries, max_value;

void read_array() {
  scanf("%d", &n);

  v[0] = 0;
  for (int i = 1; i <= n; i++) {
    scanf("%d", &v[i]);
    v[i] ^= v[i - 1];
  }
}

void read_queries() {
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].l, &q[i].r);
    q[i].l++;
    q[i].r++; // inteval închis, indexat de la 1
    q[i].orig_index = i;
    q[i].block = q[i].l / BLOCK_SIZE;
  }
}
```

```
void normalize_array() {
  for (int i = 1; i <= n; i++) {
    pos[i] = i;
  }
  std::sort(pos + 1, pos + n + 1, [](int a, int b) {
    return v[a] < v[b];
  });

  int prev = NONE;
  max_value = NONE;
  for (int i = 0; i <= n; i++) {
    if (v[pos[i]] != prev) {
      max_value++;
    }
    prev = v[pos[i]];
    v[pos[i]] = max_value;
  }
}

void sort_queries_by_left_block() {
  std::sort(q, q + num_queries, [](query a, query b) {
    return (a.block < b.block) ||
      ((a.block == b.block) && (a.r > b.r));
  });
}

void init_left() {
  for (int i = 0; i <= max_value; i++) {
    left[i] = -INFINITY;
  }
}

void init_right() {
  for (int i = 0; i <= max_value; i++) {
    right[i] = +INFINITY;
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void answer_query(query q, int closest_left_right) {
  int best = closest_left_right;
  for (int i = q.block * BLOCK_SIZE; i < q.l; i++) {
    best = min(best, right[v[i]] - i);
  }

  answer[q.orig_index] = (best > n) ? NONE : best;
```

```
}

int answer_queries_in_block(int block, int q_index) {
  init_right();
  int ptr = n + 1;
  int closest_left_right = INFINITY;
  while ((q_index < num_queries) && (q[q_index].block == block)) {
    while (ptr > q[q_index].r) {
      --ptr;
      right[v[ptr]] = ptr;
      int dist = ptr - left[v[ptr]];
      closest_left_right = min(closest_left_right, dist);
    }
    answer_query(q[q_index], closest_left_right);
    q_index++;
  }

  return q_index;
}

void update_left(int block) {
  int start = block * BLOCK_SIZE;
  int end = min(start + BLOCK_SIZE - 1, n);
  for (int i = start; i <= end; i++) {
    left[v[i]] = i;
  }
}

void scan_blocks() {
  init_left();
  int q_index = 0;

  int last_block = n / BLOCK_SIZE;
  for (int b = 0; b <= last_block; b++) {
    q_index = answer_queries_in_block(b, q_index);
    update_left(b);
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", answer[i]);
  }
}

int main() {
  read_array();
  read_queries();
  normalize_array();
  sort_queries_by_left_block();
```

```
  scan_blocks();

  write_answers();

  return 0;
}
```

7.5 Problema Serega and Fun (Codeforces)

‹ înapoi

Sursă cu `deque` (versiune online).

```
#include <deque>
#include <stdio.h>
#include <unordered_map>

const int MAX_N = 100'000;
const int BUCKET_SIZE = 2'000;
const int MAX_BUCKETS = (MAX_N - 1) / BUCKET_SIZE + 1;
const int TYPE_ROTATE = 1;
const int TYPE_COUNT = 2;

struct freq {
  std::unordered_map<int, short> map;

  void add(int val) {
    map[val]++;
  }

  void remove(int val) {
    auto it = map.find(val);
    if (it->second == 1) {
      map.erase(it);
    } else {
      it->second--;
    }
  }

  int count(int val) {
    auto it = map.find(val);
    return (it == map.end()) ? 0 : it->second;
  }
};

struct bucket {
  freq f;
```

```
std::deque<int> deq;

void push(int val) {
  deq.push_back(val);
  f.add(val);
}

void set(int pos, int val) {
  f.remove(deq[pos]);
  deq[pos] = val;
  f.add(val);
}

int shift(int val) {
  deq.push_front(val);
  f.add(val);
  int last = deq.back();
  deq.pop_back();
  f.remove(last);
  return last;
}

void rotate(int l, int r) {
  int val = deq[r];
  deq.erase(deq.begin() + r);
  deq.insert(deq.begin() + l, val);
}

int remove(int pos) {
  int val = deq[pos];
  f.remove(val);
  deq.erase(deq.begin() + pos);
  return val;
}

int remove_last() {
  return remove(BUCKET_SIZE - 1);
}

void insert(int pos, int val) {
  deq.insert(deq.begin() + pos, val);
  f.add(val);
}

void insert_first(int val) {
  insert(0, val);
}

int partial_count(int l, int r, int k) {
  int result = 0;
```

```
    while (l <= r) {
      result += (deq[l++] == k);
    }
    return result;
  }

  int prefix_count(int pos, int k) {
    return partial_count(0, pos, k);
  }

  int suffix_count(int pos, int k) {
    return partial_count(pos, BUCKET_SIZE - 1, k);
  }

};

bucket buck[MAX_BUCKETS];
int n, num_ops;
int last_answer;

void read_data() {
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    buck[i / BUCKET_SIZE].push(x);
  }

  scanf("%d", &num_ops);
}

int bucket_count(int l, int r, int k) {
  int result = 0;
  while (l < r) {
    result += buck[l++].f.count(k);
  }
  return result;
}

void process_count_op(int l, int r, int k) {
  int bl = l / BUCKET_SIZE, offset_l = l % BUCKET_SIZE;
  int br = r / BUCKET_SIZE, offset_r = r % BUCKET_SIZE;
  if (bl == br) {
    last_answer = buck[bl].partial_count(offset_l, offset_r, k);
  } else {
    last_answer =
      buck[bl].suffix_count(offset_l, k) +
      bucket_count(bl + 1, br, k) +
      buck[br].prefix_count(offset_r, k);
```

```
  }

  printf("%d\n", last_answer);
}

void process_rotate_op(int l, int r) {
  int bl = l / BUCKET_SIZE, offset_l = l % BUCKET_SIZE;
  int br = r / BUCKET_SIZE, offset_r = r % BUCKET_SIZE;
  if (bl == br) {
    buck[bl].rotate(offset_l, offset_r);
  } else {
    int from_left = buck[bl].remove_last();
    for (int b = bl + 1; b < br; b++) {
      from_left = buck[b].shift(from_left);
    }
    int to_right = buck[br].remove(offset_r);
    buck[br].insert_first(from_left);
    buck[bl].insert(offset_l, to_right);
  }
}

int transform(int x) {
  return last_answer
    ? (x + last_answer - 1) % n + 1
    : x;
}

void process_ops() {
  while (num_ops--) {
    int type, l, r, k;
    scanf("%d %d %d", &type, &l, &r);
    l = transform(l) - 1;
    r = transform(r) - 1;
    if (l > r) {
      int tmp = l; l = r; r = tmp;
    }

    if (type == TYPE_COUNT) {
      scanf("%d", &k);
      k = transform(k);
      process_count_op(l, r, k);
    } else {
      process_rotate_op(l, r);
    }
  }
}

int main() {
  read_data();
  process_ops();
```

```
  return 0;
}
```

Sursă cu descompunere după poziții (versiune online).

```
#include <math.h>
#include <stdio.h>

const int MAX_BUCKETS = 160;
const int MAX_BUCKET_SIZE = 634;
const int MAX_N = MAX_BUCKETS * MAX_BUCKET_SIZE;
const int TYPE_ROTATE = 1;
const int TYPE_COUNT = 2;

typedef struct {
  short freq[MAX_N];
  int circ[MAX_BUCKET_SIZE];
  int start;
} bucket;

bucket buck[MAX_BUCKETS];
int modulo[2 * MAX_BUCKET_SIZE];
int bs; // bucket size
int n, numOps;
int lastAnswer;

void initBuckets() {
  bs = 2 * sqrt(n);

  for (int i = 0; i < 2 * bs; i++) {
    modulo[i] = i % bs;
  }
}

void bucketSetInitial(int pos, int val) {
  buck[pos / bs].circ[pos % bs] = val;
  buck[pos / bs].freq[val]++;
}

void readInputData() {
  scanf("%d", &n);
  initBuckets();

  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    bucketSetInitial(i, x);
  }
```

```
  scanf("%d", &numOps);
}

int realPos(int b, int pos) {
  return modulo[pos + buck[b].start];
}

int next(int pos) {
  return modulo[pos + 1];
}

int prev(int pos) {
  return modulo[pos + bs - 1];
}

void bucketSet(int b, int pos, int val) {
  bucket& g = buck[b];
  int realP = realPos(b, pos);
  int oldVal = g.circ[realP];
  g.circ[realP] = val;
  g.freq[oldVal]--;
  g.freq[val]++;
}

int partialCount(int b, int l, int r, int k) {
  int realL = realPos(b, l);
  int realR = realPos(b, r);

  // Numără-l separat pe realR ca să îl putem folosi ca terminator de buclă.
  int result = (buck[b].circ[realR] == k);

  while (realL != realR) {
    result += (buck[b].circ[realL] == k);
    realL = next(realL);
  }
  return result;
}

int bucketCount(int l, int r, int k) {
  int result = 0;
  while (l < r) {
    result += buck[l++].freq[k];
  }
  return result;
}

void processCountOp(int l, int r, int k) {
  int bl = l / bs, br = r / bs;
  int lpos = l - bl * bs, rpos = r - br * bs;
```

```
  if (bl == br) {
    lastAnswer = partialCount(bl, lpos, rpos, k);
  } else {
    lastAnswer =
      partialCount(bl, lpos, bs - 1, k) +
      partialCount(br, 0, rpos, k) +
      bucketCount(bl + 1, br, k);
  }

  printf("%d\n", lastAnswer);
}

int partialRotate(int b, int l, int r) {
  int *v = buck[b].circ;
  int realL = realPos(b, l);
  int realR = realPos(b, r);
  int prevR = prev(realR);
  int save = v[realR];

  while (realR != realL) {
    v[realR] = v[prevR];
    realR = prevR;
    prevR = prev(realR);
  }

  v[realL] = save;
  return save;
}

int bucketRotate(int b) {
  bucket& g = buck[b];
  g.start = prev(g.start);
  return g.circ[g.start];
}

void processRotateOp(int l, int r) {
  int bl = l / bs, br = r / bs;
  int lpos = l - bl * bs, rpos = r - br * bs;
  if (bl == br) {
    partialRotate(bl, lpos, rpos);
  } else {
    int fromLeft = partialRotate(bl, lpos, bs - 1);
    for (int b = bl + 1; b < br; b++) {
      int toRight = bucketRotate(b);
      bucketSet(b, 0, fromLeft);
      fromLeft = toRight;
    }
    int toRight = partialRotate(br, 0, rpos);
    bucketSet(br, 0, fromLeft);
    bucketSet(bl, lpos, toRight);
```

```
  }
}

int transform(int x) {
  return lastAnswer
    ? (x + lastAnswer - 1) % n + 1
    : x;
}

void processOps() {
  while (numOps--) {
    int type, l, r, k;
    scanf("%d %d %d", &type, &l, &r);
    l = transform(l) - 1;
    r = transform(r) - 1;
    if (l > r) {
      int tmp = l; l = r; r = tmp;
    }

    if (type == TYPE_COUNT) {
      scanf("%d", &k);
      k = transform(k);
      processCountOp(l, r, k);
    } else {
      processRotateOp(l, r);
    }
  }
}

int main() {
  readInputData();
  processOps();

  return 0;
}
```

Sursă cu descompunere după operații (versiune online).

```
#include <math.h>
#include <stdio.h>

const int MAX_BUCKETS = 160;
const int MAX_BUCKET_SIZE = 634;
const int OVERFLOW_FACTOR = 2;
const int MAX_N = MAX_BUCKETS * MAX_BUCKET_SIZE;
const int TYPE_ROTATE = 1;
const int TYPE_COUNT = 2;

struct bucket {
```

```
short freq[MAX_N];
int data[OVERFLOW_FACTOR * MAX_BUCKET_SIZE];
int size;

void append(int x) {
  data[size++] = x;
  freq[x]++;
}

// Returnează numărul de elemente vărsate. Golește data, size și freq.
int empty(int* dest) {
  for (int i = 0; i < size; i++) {
    dest[i] = data[i];
    freq[data[i]]--;
  }
  int old_size = size;
  size = 0;

  return old_size;
}

int partial_count(int l, int r, int val) {
  int cnt = 0;
  while (l <= r) {
    cnt += (data[l++] == val);
  }

  return cnt;
}

int prefix_count(int pos, int val) {
  return partial_count(0, pos, val);
}

int suffix_count(int pos, int val) {
  return partial_count(pos, size - 1, val);
}

void insert(int pos, int val) {
  freq[val]++;
  size++;
  for (int i = size - 1; i > pos; i--) {
    data[i] = data[i - 1];
  }
  data[pos] = val;
}

int remove(int pos) {
  int result = data[pos];
  freq[result]--;
```

```
    for (int i = pos; i < size - 1; i++) {
      data[i] = data[i + 1];
    }
    size--;

    return result;
  }
};

bucket buck[MAX_BUCKETS];
int naive[MAX_N];
int bs, nb; // mărimea blocului, numărul de blocuri
int n;
int last_answer;

// Stochează informații despre blocul în care se află un indice real.
struct bucket_info {
  int b;      // numărul blocului
  int start;  // indicele primului element din bloc
  int offset; // offset-ul indicelui dat față de start

  void find_next_bucket(int index) {
    while (start + buck[b].size <= index) {
      start += buck[b++].size;
    }
    offset = index - start;
  }
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

void distribute_buckets() {
  // Evită O(n) împărțiri deoarece vom rula acest cod de O(sqrt(n)) ori.
  for (int b = 0; b < nb; b++) {
    int start = b * bs;
    int end = min(start + bs, n);
    for (int i = start; i < end; i++) {
      buck[b].append(naive[i]);
    }
  }
}

void collect_buckets() {
  int ptr = 0;
  for (int i = 0; i < nb; i++) {
    ptr += buck[i].empty(&naive[ptr]);
  }
}
```

```
void read_data() {
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    scanf("%d", &naive[i]);
  }
}

void init_buckets() {
  bs = 2 * sqrt(n);
  nb = (n - 1) / bs + 1;
  distribute_buckets();
}

int cross_bucket_count(int l, int r, int k) {
  int result = 0;
  while (l < r) {
    result += buck[l++].freq[k];
  }
  return result;
}

// [l, r] inclusiv
void process_count_op(int l, int r, int k) {
  bucket_info bl = { 0, 0, 0 };
  bl.find_next_bucket(l);
  bucket_info br = bl;
  br.find_next_bucket(r);

  if (bl.b == br.b) {
    last_answer = buck[bl.b].partial_count(bl.offset, br.offset, k);
  } else {
    last_answer =
      buck[bl.b].suffix_count(bl.offset, k) +
      buck[br.b].prefix_count(br.offset, k) +
      cross_bucket_count(bl.b + 1, br.b, k);
  }

  printf("%d\n", last_answer);
}

void process_rotate_op(int l, int r) {
  bucket_info bl = { 0, 0, 0 };
  bl.find_next_bucket(l);
  bucket_info br = bl;
  br.find_next_bucket(r);

  int x = buck[br.b].remove(br.offset);
  buck[bl.b].insert(bl.offset, x);
```

```
  if (buck[bl.b].size == OVERFLOW_FACTOR * bs) {
    collect_buckets();
    distribute_buckets();
  }
}

int transform(int x) {
  return last_answer
    ? (x + last_answer - 1) % n + 1
    : x;
}

void process_ops() {
  int num_ops;
  scanf("%d", &num_ops);

  while (num_ops--) {
    int type, l, r, k;
    scanf("%d %d %d", &type, &l, &r);
    l = transform(l) - 1;
    r = transform(r) - 1;
    if (l > r) {
      int tmp = l; l = r; r = tmp;
    }

    if (type == TYPE_COUNT) {
      scanf("%d", &k);
      k = transform(k);
      process_count_op(l, r, k);
    } else {
      process_rotate_op(l, r);
    }
  }
}

int main() {
  read_data();
  init_buckets();
  process_ops();

  return 0;
}
```

7.6 Problema Rev (Lot 2008)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <stdlib.h>

const int MAX_N = 100'000;
const int OP_BLOCK_SIZE = 150;

// Direcția este de la stînga la dreapta sau de la dreapta la stînga după cum
// start   end.
struct block {
  int start, end;

  int len() {
    return 1 + abs(end - start);
  }

  int dir() {
    return (start < end) ? +1 : -1;
  }

  int get_index_at(int pos) {
    return start + dir() * pos;
  }

  void reverse() {
    int tmp = start;
    start = end;
    end = tmp;
  }
};

struct block_list {
  block v[2 * OP_BLOCK_SIZE + 1];
  int cnt;

  // Cînd defragmentăm structura, citim dintr-un vector și scriem în celălalt.
  int base_array[2][MAX_N + 1];
  int row;

  void init(int n) {
    for (int i = 0; i < n; i++) {
      base_array[0][i] = i + 1;
    }
    row = 0;
    v[0] = { 0, n - 1 };
    cnt = 1;
  }

  void defrag() {
    int ptr = 0;
```

```
  for (int i = 0; i < cnt; i++) {
    int dir = v[i].dir();
    for (int pos = v[i].start; pos != v[i].end + dir; pos += dir) {
      base_array[1 - row][ptr++] = base_array[row][pos];
    }
  }
  row = 1 - row;
  v[0] = { 0, ptr - 1 };
  cnt = 1;
}

void get_block_and_real_index(int pos, int& b, int& ind) {
  b = 0;
  while (v[b].len() <= pos) {
    pos -= v[b++].len();
  }

  ind = v[b].get_index_at(pos);
}

int get_element_at(int pos) {
  int b, ind;
  get_block_and_real_index(pos, b, ind);
  return base_array[row][ind];
}

void split_block(int b, int last_left) {
  for (int i = cnt - 1; i >= b; i--) {
    v[i + 1] = v[i];
  }
  v[b + 1].start = last_left + v[b].dir();
  v[b].end = last_left;
  cnt++;
}

void reverse_blocks(int l, int r) {
  for (int i = l; i <= r; i++) {
    v[i].reverse();
  }

  while (l < r) {
    block tmp = v[l];
    v[l] = v[r];
    v[r] = tmp;
    l++;
    r--;
  }
}

void reverse(int l, int r) {
```

```
    int bl, indl, br, indr;
    get_block_and_real_index(l, bl, indl);
    get_block_and_real_index(r, br, indr);

    if (indr != v[br].end) {
      split_block(br, indr);
    }
    if (indl != v[bl].start) {
      split_block(bl, indl - v[bl].dir());
      // Inserarea unui nou bloc împinge bl și br la dreapta.
      bl++;
      br++;
    }
    reverse_blocks(bl, br);
  }
};

block_list blist;
FILE *fin, *fout;

void init_perm() {
  int n;
  fscanf(fin, "%d", &n);
  blist.init(n);
}

void process_ops() {
  int num_ops, l, r, pos;
  fscanf(fin, "%d", &num_ops);

  while (num_ops--) {
    fscanf(fin, "%d %d %d", &l, &r, &pos);
    blist.reverse(l - 1, r - 1);
    if (num_ops % OP_BLOCK_SIZE == 0) {
      blist.defrag();
    }
    int val = blist.get_element_at(pos - 1);
    fprintf(fout, "%d\n", val);
  }
}

int main() {
  fin = fopen("rev.in", "r");
  fout = fopen("rev.out", "w");

  init_perm();
  process_ops();

  fclose(fin);
  fclose(fout);
```

```
  return 0;
}
```

7.7 Problema Time to Raid Cowavans (Codeforces)

înapoi • versiune online

```
// Complexitate: O((q + n) sqrt n).
#include <algorithm>
#include <stdio.h>

const int MAX_N = 300'000;
const int MAX_Q = 300'000;
const int PREPROCESS_LIMIT = 547;

struct query {
  int id, first, step;
};

int w[MAX_N + 1];
long long prep[MAX_N + 1];
query q[MAX_Q];
long long answer[MAX_Q];
int n, num_queries;

void read_data() {
  scanf("%d", &n);
  for (int i = 1; i <= n; i++) {
    scanf("%d", &w[i]);
  }
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].first, &q[i].step);
    q[i].id = i;
  }
}

void sort_queries() {
  std::sort(q, q + num_queries, [](query a, query b) {
    return a.step < b.step;
  });
}

long long naive_prog_sum(int first, int step) {
  long long sum = 0;
  for (int i = first; i <= n; i += step) {
    sum += w[i];
```

```
  }
  return sum;
}

void preprocess(int step) {
  for (int i = n; (i > n - step) && (i >= 1); i--) {
    prep[i] = w[i];
  }
  for (int i = n - step; i >= 1; i--) {
    prep[i] = w[i] + prep[i + step];
  }
}

void process_queries() {
  sort_queries();
  for (int i = 0; i < num_queries; i++) {
    if (q[i].step > PREPROCESS_LIMIT) {
      answer[q[i].id] = naive_prog_sum(q[i].first, q[i].step);
    } else {
      if (!i || (q[i].step != q[i - 1].step)) {
        preprocess(q[i].step);
      }
      answer[q[i].id] = prep[q[i].first];
    }
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%lld\n", answer[i]);
  }
}

int main() {
  read_data();
  process_queries();
  write_answers();

  return 0;
}
```

7.8 Problema Sandor (Baraj ONI 2025)

‹ înapoi • versiune online

```
// Complexitate O(N sqrt N), provenită din 3 for-uri imbricate a cîte
// O(sqrt N) iterații.
#include <stdio.h>
```

```
const int MAX_N = 400'000;
const int MAX_WEIGHT = 800'000;

struct run {
  int val, cnt;
};

run r[MAX_N];
// jump[w] este obiectul maxim care nu depășește greutatea w. Mai exact,
// jump[w] este cel mai mic i a.î r[i].val ≤ w
int jump[MAX_WEIGHT + 1];
long long sol[MAX_WEIGHT + 1];
int task, n, num_runs, capacity;

int min(int x, int y) {
  return (x < y) ? x : y;
}

void read_data() {
  FILE* f = fopen("sandor.in", "r");
  int x;

  fscanf(f, "%d %d %d", &task, &n, &capacity);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &x);
    if (num_runs && (x == r[num_runs - 1].val)) {
      r[num_runs - 1].cnt++;
    } else {
      r[num_runs++] = { x, 1 };
    }
  }

  fclose(f);
}

void compute_jumps() {
  int i = 0;
  for (int w = MAX_WEIGHT; w >= 0; w--) {
    while ((i < num_runs) && (r[i].val > w)) {
      i++;
    }
    jump[w] = i;
  }
}

// Rulează algoritmul lui Sandor pentru capacitatea dată. Returnează greutatea
// acumulată.
int do_the_sandor(int start, int capacity) {
  int c = capacity;
```

```
  int i = start;

  while (i < num_runs) {
    int taken = min(c / r[i].val, r[i].cnt);
    c -= taken * r[i].val;
    // Dacă din c = 100 am luat 2 * r[i].val = 20, fiindcă atîtea erau, nu are
    // sens să continui de la 80. Următorul număr poate fi doar 9 sau mai mic.
    i = jump[min(c, r[i].val - 1)];
  }

  return capacity - c;
}

// Presupunînd că am tăiat deja două obiecte înainte de start și am obținut
// greutatea weight_so_far, rulează Sandor simplu pe restul vectorului și
// adaugă rezultatul la soluție.
void cut_0(int start, int weight_so_far, long long multiplier) {
  int s = do_the_sandor(start, capacity - weight_so_far);
  sol[weight_so_far + s] += multiplier;
}

// Presupunînd că am tăiat deja un obiect înainte de start și am obținut
// greutatea weight_so_far, și că de la poziția start începînd mai am num_num
// obiecte, încearcă să elimini cîte un obiect în toate modurile posibile.
//
// Observăm că, dacă tăiem alt obiect decît cele pe care le-ar pune Sandor în
// rucsac, vom obține aceeași sumă ca și pe vectorul nemodificat.
void cut_1(int start, int weight_so_far, int num_num, long long multiplier) {
  int c = capacity - weight_so_far;
  int i = start;

  while (i < num_runs) {
    int taken = min(c / r[i].val, r[i].cnt);
    if (taken == r[i].cnt) {
      num_num -= taken;
      int all_but_one = (taken - 1) * r[i].val;
      cut_0(i + 1, weight_so_far + all_but_one, multiplier * taken);
    }
    weight_so_far += taken * r[i].val;
    c -= taken * r[i].val;
    i = jump[min(c, r[i].val - 1)];
  }

  // Toate numerele pe care nu le-am tăiat explicit merg pe soluția originală,
  // care duce la sumă weight_so_far.
  sol[weight_so_far] += multiplier * num_num;
}

// Pentru bucla exterioară este important să obținem tot complexitate
// O(sqrt N). De aceea, considerăm simultan fiecare grupă de obiecte
```

```
// consecutive NEincluse în rucsac.
void cut_2() {
  long long multiplier = 0;
  int weight_so_far = 0;
  int c = capacity;
  int num_right = n;

  for (int i = 0; i < num_runs; i++) {
    if (r[i].val > c) {
      multiplier += r[i].cnt;
    } else {
      int cnt = r[i].cnt;
      // Taie două obiecte dinainte de i. Rămîn 0 de tăiat.
      cut_0(i, weight_so_far, multiplier * (multiplier - 1) / 2);

      // Taie un obiect dinainte de i. Rămîne unul de tăiat.
      cut_1(i, weight_so_far, num_right, multiplier);

      // Taie două obiecte din grupa i. În rest, pune în rucsac ce se poate.
      if (r[i].cnt >= 2) {
        int taken = min(c / r[i].val, r[i].cnt - 2);
        cut_0(i + 1, weight_so_far + taken * r[i].val, cnt * (cnt - 1) / 2);
      }

      // Taie un obiect din grupa i. În rest, pune în rucsac ce se poate.
      int taken = min(c / r[i].val, r[i].cnt - 1);
      cut_1(i + 1, weight_so_far + taken * r[i].val, num_right - r[i].cnt, cnt);

      // Nu tăia nimic, bagă în rucsac.
      taken = min(c / r[i].val, r[i].cnt);
      c -= taken * r[i].val;
      weight_so_far += taken * r[i].val;

      multiplier = 0;
    }

    num_right -= r[i].cnt;
  }

  // Nu mai putem băga nimic în rucsac, dar din ultimele @multiplier obiecte
  // trebuie să tăiem două.
  sol[weight_so_far] += multiplier * (multiplier - 1) / 2;
}

void write_solution() {
  FILE* f = fopen("sandor.out", "w");
  for (int i = 0; i <= capacity; i++) {
    fprintf(f, "%lld ", sol[i]);
  }
  fprintf(f, "\n");
```

```
  fclose(f);
}

int main() {
  read_data();
  compute_jumps();

  if (task == 1) {
    cut_1(0, 0, n, 1);
  } else {
    cut_2();
  }

  write_solution();

  return 0;
}
```

7.9 Problema Puzzle-bila (Lot 2025)

‹ înapoi • versiune online

```
// Complexitate: O(n·m·sqrt(m) + n·m·log(m)).
#include <stdio.h>

const int MAX_COLS = 50'000;
const int MAX_LOG = 16;
const int MAX_DISTINCT_LENGTHS = 320;
const int INFTY = 2'000'000;
const int NONE = -1;

int min(int x, int y) {
  return (x < y) ? x : y;
}

int log2(int x) {
  return 31 - __builtin_clz(x);
}

struct sparse_table {
  int v[MAX_LOG][MAX_COLS];
  int n;

  void build(int* src, int n, bool with_shifts) {
    for (int i = 0; i < n; i++) {
      v[0][i] = with_shifts ? (src[i] - i) : src[i];
    }
    for (int p = 1; (1 << p) <= n; p++) {
```

```
      for (int i = 0; i <= n - (1 << p); i++) {
        v[p][i] = min(v[p - 1][i], v[p - 1][i + (1 << (p - 1))]);
      }
    }
  }

  int range_min(int l, int r) {
    l = (l < 0) ? 0 : l;
    if (l > r) {
      return INFTY;
    }
    int row = log2(r - l + 1);
    return  min(v[row][l], v[row][r - (1 << row) + 1]);
  }
};

sparse_table st, st_shift;
bool row[MAX_COLS];
int prev1[MAX_COLS];
int distinct_len[MAX_DISTINCT_LENGTHS], num_lengths;
int dp[MAX_COLS];
int end[MAX_COLS + 1]; // coloana pe care se termină ultima fereastră de lățime l
int num_rows, num_cols;

void read_size() {
  scanf("%d %d ", &num_rows, &num_cols);
}

void read_row() {
  for (int c = 0; c < num_cols; c++) {
    row[c] = getchar() - '0';
  }
  getchar();
}

void init_all() {
  dp[0] = 0;
  for (int c = 1; c < num_cols; c++) {
    dp[c] = INFTY;
  }

  for (int l = 0; l <= num_cols; l++) {
    end[l] = NONE;
  }
}

void compute_prev1() {
  prev1[0] = row[0] ? 0 : -1;
  for (int c = 1; c < num_cols; c++) {
    prev1[c] = row[c] ? c : prev1[c - 1];
```

```
  }
}

void add_length(int len, int end_col) {
  if (len && (end[len] == NONE)) {
    distinct_len[num_lengths++] = len;
  }
  end[len] = end_col;
}

void reset_distinct_lengths() {
  while (num_lengths) {
    end[distinct_len[--num_lengths]] = NONE;
  }
}

// Adu ferestre de zerouri din stînga, deja închise, aliniindu-le cu col.
void slide_windows_right(int col) {
  dp[col] = INFTY;
  for (int i = 0; i < num_lengths; i++) {
    int len = distinct_len[i];
    int cost = st.range_min(col - len + 1, col) + (col - end[len]);
    dp[col] = min(dp[col], cost);
  }
}

// Adu ferestre de zerouri din dreapta, deja închise, aliniindu-le cu col.
void slide_windows_left(int col) {
  for (int i = 0; i < num_lengths; i++) {
    int len = distinct_len[i];
    int cost = st_shift.range_min(col - len + 1, col) + end[len] - len + 1;
    dp[col] = min(dp[col], cost);
  }
}

void slide_current_window(int col, int r_len) {
  if (!row[col]) {
    int l = 1 + prev1[col];
    int r = col + r_len - 1;
    int len = r - l + 1;
    dp[col] = min(dp[col], st.range_min(l, col));
    int cost_r = st_shift.range_min(col - len + 1, l - 1) + l;
    dp[col] = min(dp[col], cost_r);
  }
}

void scan_left_to_right() {
  reset_distinct_lengths();

  int cur_len = 0;
```

```
  for (int c = 0; c < num_cols; c++) {
    if (row[c]) {
      add_length(cur_len, c - 1);
      cur_len = 0;
    } else {
      cur_len++;
    }
    slide_windows_right(c);
  }
}

void scan_right_to_left() {
  reset_distinct_lengths();

  int cur_len = 0;
  for (int c = num_cols - 1; c >= 0; c--) {
    if (row[c]) {
      add_length(cur_len, c + cur_len);
      cur_len = 0;
    } else {
      cur_len++;
    }
    slide_windows_left(c);
    slide_current_window(c, cur_len);
  }
}

void process_rows() {
  init_all();
  for (int r = 0; r < num_rows; r++) {
    read_row();
    compute_prev1();
    st.build(dp, num_cols, false);
    st_shift.build(dp, num_cols, true);
    scan_left_to_right();
    scan_right_to_left();
  }
}

void write_result() {
  int res = dp[num_cols - 1];
  printf("%d\n", (res == INFTY) ? NONE : res);
}

int main() {
  read_size();
  process_rows();
  write_result();

  return 0;
```

```
}
```

Anexa 8

Algoritmul lui Mo

8.1 Problema Powerful Array (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 200'000;
const int BLOCK_SIZE = 300;
const int MAX_QUERIES = 200'000;
const int MAX_VAL = 1'000'000;

typedef unsigned long long u64;

struct query {
  int l, r;
  int orig_index;
};

int v[MAX_N];
int f[MAX_VAL + 1];
u64 power;
query q[MAX_QUERIES];
u64 answer[MAX_QUERIES]; // separat de q[] ca să evităm o sortare la final
int n, num_queries;

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
  }
  for (int i = 0; i < num_queries; i++) {
    int l, r;
    scanf("%d %d", &l, &r);
```

```
    q[i].l = l - 1;
    q[i].r = r - 1;
    q[i].orig_index = i;
  }
}

void sort_queries_by_left_block() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
      return a.r < b.r;
    }
  });
}

void add(int pos) {
  f[v[pos]]++;
  power += (u64)v[pos] * (2 * f[v[pos]] - 1);
}

void remove(int pos) {
  f[v[pos]]--;
  power -= (u64)v[pos] * (2 * f[v[pos]] + 1);
}

void mo() {
  int l = 0, r = -1; // vid
  for (int i = 0; i < num_queries; i++) {
    while (l > q[i].l) {
      add(--l);
    }
    while (r < q[i].r) {
      add(++r);
    }
    while (l < q[i].l) {
      remove(l++);
    }
    while (r > q[i].r) {
      remove(r--);
    }
    answer[q[i].orig_index] = power;
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
```

```
    printf("%llu\n", answer[i]);
  }
}

int main() {
  read_data();
  sort_queries_by_left_block();
  mo();

  write_answers();

  return 0;
}
```

8.2 Problema Most Frequent Value (SPOJ)

înapoi

```
// Complexitate: O((n + q) sqrt n).
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;
const int BLOCK_SIZE = 316;
const int MAX_QUERIES = 100'000;
const int MAX_VAL = 100'000;

struct query {
  int l, r;
  int orig_index;
};

int v[MAX_N];
int f[MAX_VAL + 1];
int num_having_f[MAX_N + 1], max_f;
query q[MAX_QUERIES];
int answer[MAX_QUERIES];
int n, num_queries;

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
  }
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].l, &q[i].r);
    q[i].orig_index = i;
  }
```

```
}

void sort_queries_by_left_block() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
      return a.r < b.r;
    }
  });
}

void add(int pos) {
  int x = ++f[v[pos]];
  num_having_f[x - 1]--;
  num_having_f[x]++;
  if (x > max_f) {
    max_f = x;
  }
}

void remove(int pos) {
  int x = --f[v[pos]];
  num_having_f[x + 1]--;
  num_having_f[x]++;
  if (num_having_f[max_f] == 0) {
    max_f--;
  }
}

void mo() {
  num_having_f[0] = n;
  int l = 0, r = -1; // vid
  for (int i = 0; i < num_queries; i++) {
    while (l > q[i].l) {
      add(--l);
    }
    while (r < q[i].r) {
      add(++r);
    }
    while (l < q[i].l) {
      remove(l++);
    }
    while (r > q[i].r) {
      remove(r--);
    }
    answer[q[i].orig_index] = max_f;
```

```
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", answer[i]);
  }
}

int main() {
  read_data();
  sort_queries_by_left_block();
  mo();

  write_answers();

  return 0;
}
```

8.3 Problema RangeMode (Infoarena Cup 2013)

‹ înapoi • versiune online

```
// Complexitate: O((n + q) sqrt n).
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100000;
const int BLOCK_SIZE = 316;
const int MAX_QUERIES = 100000;
const int MAX_VAL = 100000;

struct query {
  int l, r;
  int orig_index;
};

int v[MAX_N];
int f[MAX_VAL + 1]; // frecvențele elementelor
int right_ptr;      // poziția ultimului element adăugat în f
int right_best;     // răspunsul considerînd strict dreapta blocului curent
query q[MAX_QUERIES];
int answer[MAX_QUERIES];
int n, num_queries;

void read_data() {
  FILE* f = fopen("rangemode.in", "r");
  fscanf(f, "%d %d", &n, &num_queries);
```

```
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &v[i]);
  }
  for (int i = 0; i < num_queries; i++) {
    int l, r;
    fscanf(f, "%d %d", &l, &r);
    q[i].l = l - 1;
    q[i].r = r - 1;
    q[i].orig_index = i;
  }
  fclose(f);
}

void sort_queries_by_left_block() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    return (x < y) || ((x == y) && (a.r < b.r));
  });
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void clear_f() {
  for (int i = 0; i <= MAX_VAL; i++) {
    f[i] = 0;
  }
}

void optimize(int& best, int candidate) {
  if ((f[candidate] > f[best]) ||
      ((f[candidate] == f[best]) && (candidate < best))) {
    best = candidate;
  }
}

void extend_right(int until) {
  while (right_ptr < until) {
    int val = v[++right_ptr];
    f[val]++;
    optimize(right_best, val);
  }
}

int add_current_block(int l, int r) {
  int best = right_best;
  for (int i = l; i <= r; i++) {
    f[v[i]]++;
    optimize(best, v[i]);
```

```
  }
  return best;
}

void remove_current_block(int l, int r) {
  for (int i = l; i <= r; i++) {
    f[v[i]]--;
  }
}

void process_query(query q, int last_element_in_block) {
  extend_right(q.r);

  // Interogarea nu trece neapărat în blocul următor. Poate fi complet
  // cuprinsă în blocul curent.
  int right = min(q.r,last_element_in_block);
  int best = add_current_block(q.l, right);
  answer[q.orig_index] = best;
  remove_current_block(q.l, right);
}

void scan_blocks() {
  int qi = 0;
  for (int start = 0; start < n; start += BLOCK_SIZE) {
    int end = start + BLOCK_SIZE;
    right_ptr = end - 1;
    right_best = 0;
    while ((qi < num_queries) && (q[qi].l < end)) {
      process_query(q[qi++], end - 1);
    }
    clear_f();
  }
}

void write_answers() {
  FILE* f = fopen("rangemode.out", "w");
  for (int i = 0; i < num_queries; i++) {
    fprintf(f, "%d\n", answer[i]);
  }
  fclose(f);
}

int main() {
  read_data();
  sort_queries_by_left_block();
  scan_blocks();

  write_answers();

  return 0;
```

```
}
```

8.4 Problema Kiloforces (Concurs RoAlgo #8)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_PROBLEMS = 100'000;
const int MAX_QUERIES = 100'000;
const int MAX_PENALTY = 100'000;
const int BLOCK_SIZE = 400;

struct problem {
  int score;
  int penalty;
};

struct query {
  int l, r;
  int id;
};

struct fen_data {
  long long sum;
  int cnt;

  fen_data operator +=(fen_data& other) {
    sum += other.sum;
    cnt += other.cnt;
    return *this;
  }
};

struct fenwick_tree {
  fen_data v[MAX_PENALTY + 1];

  fen_data prefix_sum(int pos) {
    fen_data d = { 0, 0 };
    while (pos) {
      d += v[pos];
      pos &= pos - 1;
    }
    return d;
  }

  void add(int pos, fen_data val) {
```

```
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= MAX_PENALTY);
  }

  void check(int pos) {
    add(pos, { .sum = pos, .cnt = 1 });
  }

  void uncheck(int pos) {
    add(pos, { .sum = -pos, .cnt = -1 });
  }
};

struct mo_data {
  fenwick_tree fen;
  long long score;
  int width;

  void include(problem p) {
    fen_data d = fen.prefix_sum(p.penalty);
    score += p.score;
    score -= d.sum;
    score -= (long long)p.penalty * (width - d.cnt);
    width++;
    fen.check(p.penalty);
  }

  void exclude(problem p) {
    fen.uncheck(p.penalty);
    fen_data d = fen.prefix_sum(p.penalty);
    score -= p.score;
    score += d.sum;
    score += (long long)p.penalty * (width - d.cnt - 1);
    width--;
  }
};

problem p[MAX_PROBLEMS];
query q[MAX_QUERIES];
long long answer[MAX_QUERIES];
mo_data mod;
int n, num_queries;

void read_data() {
  scanf("%d %d", &n, &num_queries);
  for (int i = 1; i <= n; i++) {
    scanf("%d", &p[i].score);
  }
```

```
  for (int i = 1; i <= n; i++) {
    scanf("%d", &p[i].penalty);
  }
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].l, &q[i].r);
    q[i].id = i;
  }
}

void sort_queries_in_mo_order() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
      return a.r < b.r;
    }
  });
}

void mo() {
  int l = 1, r = 0; // gol

  for (int i = 0; i < num_queries; i++) {
    while (l > q[i].l) {
      mod.include(p[--l]);
    }
    while (r < q[i].r) {
      mod.include(p[++r]);
    }
    while (l < q[i].l) {
      mod.exclude(p[l++]);
    }
    while (r > q[i].r) {
      mod.exclude(p[r--]);
    }

    answer[q[i].id] = mod.score;
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%lld\n", answer[i]);
  }
}

int main() {
```

```
  read_data();
  sort_queries_in_mo_order();
  mo();

  write_answers();

  return 0;
}
```

8.5 Problema Machine Learning (Codeforces)

înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <unordered_map>

const int MAX_N = 100'000;
const int BLOCK_SIZE = 2'154; // n^{2/3}
const int MAX_OPS = 100'000;

const int T_QUERY = 1;
const int T_UPDATE = 2;

struct query {
  int l, r, time, answer;
};

struct update {
  int pos, old_val, val, time;
};

int a[MAX_N];
query q[MAX_OPS];
update u[MAX_OPS];
int tmp[MAX_N + MAX_OPS];
int n, num_ops, num_queries, num_updates;

// Date specifice pentru Mo.
int f[MAX_N + MAX_OPS];
int num_having_f[MAX_N + MAX_OPS];

// Renumerotează valorile din vector și actualizările.
struct normalizer {
  std::unordered_map<int, int> map;

  int normalize(int x) {
    auto it = map.find(x);
```

```
    if (it == map.end()) {
      int result = 1 + map.size();
      map[x] = result;
      return result;
    } else {
      return it->second;
    }
  }
};

normalizer norm;

void read_array() {
  scanf("%d %d", &n, &num_ops);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    a[i] = norm.normalize(x);
  }
}

void read_ops() {
  for (int i = 0; i < n; i++) {
    tmp[i] = a[i];
  }

  u[num_updates++] = { 0, 0, 0, 0 }; // santinelă stînga (t = 0)
  for (int t = 1; t <= num_ops; t++) {
    int type;
    scanf("%d", &type);
    if (type == T_QUERY) {
      int l, r;
      scanf("%d %d", &l, &r);
      q[num_queries++] = { l - 1, r - 1, t };
    } else {
      int pos, val;
      scanf("%d %d", &pos, &val);
      pos--;
      val = norm.normalize(val);
      u[num_updates++] = { pos, tmp[pos], val, t };
      tmp[pos] = val;
    }
  }
  u[num_updates++] = { 0, 0, 0, n + num_ops }; // santinelă dreapta (t = ∞)
}

void sort_queries() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
```

```
      return (x < y);
    }

    x = a.r / BLOCK_SIZE, y = b.r / BLOCK_SIZE;
    if (x != y) {
      return (x < y);
    }

    return a.time < b.time;
  });
}

void change_frequency(int val, int delta) {
  num_having_f[f[val]]--;
  f[val] += delta;
  num_having_f[f[val]]++;
}

void add(int pos) {
  change_frequency(a[pos], +1);
}

void remove(int pos) {
  change_frequency(a[pos], -1);
}

void change_value(int l, int r, int pos, int val) {
  if ((pos >= l) && (pos <= r)) {
    change_frequency(a[pos], -1);
    change_frequency(val, +1);
  }
  a[pos] = val;
}

int get_mex() {
  int mex = 1;
  while (num_having_f[mex]) {
    mex++;
  }
  return mex;
}

void mo() {
  num_having_f[0] = n + num_updates; // ceva infinit
  int l = 0, r = -1, u_index = 1;
  for (int i = 0; i < num_queries; i++) {
    while (l > q[i].l) {
      add(--l);
    }
    while (r < q[i].r) {
```

```
      add(++r);
    }
    while (l < q[i].l) {
      remove(l++);
    }
    while (r > q[i].r) {
      remove(r--);
    }
    while (u[u_index].time < q[i].time) {
      change_value(l, r, u[u_index].pos, u[u_index].val);
      u_index++;
    }
    while (u[u_index - 1].time > q[i].time) {
      u_index--;
      change_value(l, r, u[u_index].pos, u[u_index].old_val);
    }
    q[i].answer = get_mex();
  }
}

void sort_queries_by_time() {
  std::sort(q, q + num_queries, [](query a, query b) {
    return a.time < b.time;
  });
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", q[i].answer);
  }
}

int main() {
  read_array();
  read_ops();
  sort_queries();
  mo();
  sort_queries_by_time();
  write_answers();

  return 0;
}
```

Anexa 9

Skip lists

9.1 Implementare de *skip lists* augmentate

înapoi

```
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

const int N = 500'000;
const int MAX_LEVELS = 21;
const int INF = 2'000'000'000;

struct node {
  int val;
  int height;
  int next[MAX_LEVELS];
  int dist[MAX_LEVELS];
};

struct skip_list {
  node a[N + 2];
  int prev[MAX_LEVELS], prev_ord[MAX_LEVELS];
  int size; // următoarea celulă nealocată

  // Folosește celulele 0 și 1 pentru santinele: turnuri de MAX_LEVELS
  // pointeri pentru -∞ și +∞. Toți pointerii de la -∞ pointează la +∞, la
  // distanță 1.
  void init() {
    size = 2;
    a[0].val = -INF;
    a[1].val = +INF;
    a[0].height = a[1].height = MAX_LEVELS;
    for (int l = 0; l < MAX_LEVELS; l++) {
```

```
    a[0].next[l] = 1;
    a[0].dist[l] = 1;
  }
}

// Alege o înălțime de turn din distribuția geometrică cu p = 0,5.
// Rezultatul returnat este în [1, MAX_LEVELS).
int get_height() {
  int r = rand() | (1 << (MAX_LEVELS - 2));
  return 1 + __builtin_ctz(r);
}

void insert(int val) {
  // Atribuie valoarea și creează turnul de pointeri și de distanțe.
  a[size].val = val;
  int h = get_height();
  a[size].height = h;

  int pos = 0, order = 0;

  for (int l = MAX_LEVELS - 1; l >= 0; l--) {
    // Avansează pe orizontală pe nivelul l, doar peste valori strict mai
    // mici. Calculează și indicele inserării.
    while (a[a[pos].next[l]].val < val) {
      order += a[pos].dist[l];
      pos = a[pos].next[l];
    }

    // Interpune celula size între pos și următoarea celulă.
    a[pos].dist[l]++; // Include și nodul însuși.
    if (l < h) {
      a[size].next[l] = a[pos].next[l];
      a[pos].next[l] = size;
      prev[l] = pos;
      prev_ord[l] = order;
    }
  }

  // Acum că ne știm propriul indice, actualizează distanțele.
  order++;
  for (int l = 0; l < h; l++) {
    a[size].dist[l] = a[prev[l]].dist[l] - (order - prev_ord[l]);
    a[prev[l]].dist[l] = order - prev_ord[l];
  }

  size++;
}

void erase(int val) {
  int pos = 0;
```

```
  for (int l = MAX_LEVELS - 1; l >= 0; l--) {
    while (a[a[pos].next[l]].val < val) {
      pos = a[pos].next[l];
    }

    a[pos].dist[l]--;
    if (a[a[pos].next[l]].val == val) {
      a[pos].dist[l] += a[a[pos].next[l]].dist[l];
      a[pos].next[l] = a[a[pos].next[l]].next[l];
    }
  }

  size--;
}

// Avansează pe fiecare nivel, doar peste valori strict mai mici.
bool contains(int val) {
  int pos = 0;

  for (int l = MAX_LEVELS - 1; l >= 0; l--) {
    while (a[a[pos].next[l]].val < val) {
      pos = a[pos].next[l];
    }
  }

  // Următoarea celulă ar putea fi cea căutată.
  return (a[a[pos].next[0]].val == val);
}

// Presupune că valoarea există și este unică.
int order_of(int val) {
  int pos = 0, order = 0;

  // Cod identic cu cel de la inserare.
  for (int l = MAX_LEVELS - 1; l >= 0; l--) {
    while (a[a[pos].next[l]].val < val) {
      order += a[pos].dist[l];
      pos = a[pos].next[l];
    }
  }

  return order;
}

int kth_element(int k) {
  int pos = 0;
  k++; // Sari peste santinela -∞.

  for (int l = MAX_LEVELS - 1; l >= 0; l--) {
```

```
      // Suma distanțelor nu trebuie să depășească k.
      while (a[pos].dist[l] <= k) {
        k -= a[pos].dist[l];
        pos = a[pos].next[l];
      }
    }

    return a[pos].val;
  }
};

skip_list sl;
int perm[N];

void gen_perm() {
  // Amestecă mulțimea {0, 1, ..., N - 1}.
  for (int i = 0; i < N; i++) {
    int j = rand() % (i + 1);
    perm[i] = perm[j];
    perm[j] = i;
  }
}

int main() {
  srand(time(NULL));
  gen_perm();

  // Exemplu de folosire și test de corectitudine.
  sl.init();

  for (int i = 0; i < N; i++) {
    sl.insert(perm[i]);
    int j = rand() % N;
    assert(sl.contains(perm[j]) == (j <= i));
  }

  for (int i = 0; i < N; i++) {
    assert(sl.order_of(i) == i);
    assert(sl.kth_element(i) == i);
  }

  for (int i = 0; i < N; i++) {
    sl.erase(perm[i]);
    int j = rand() % N;
    assert(sl.contains(perm[j]) == (j > i));
  }

  return 0;
}
```

Anexa 10

Treaps

10.1 Implementare de treap augmentat

înapoi

Toate implementările sînt compatibile ca specificații cu implementările de *skip lists*, de aceea am omis funcția `main()` cu exemplele de folosire.

Implementare cu rotații

```
struct node {
  int key, pri;
  int cnt;
  int l, r;
};

struct treap {
  node v[N + 1];
  int n, root;

  void init() {
    n = 1; // Marchează nodul 0 ca folosit, întrucît 0 înseamnă NULL.
    root = 0;
  }

  int search(int key) {
    int t = root;
    while (t && (key != v[t].key)) {
      t = (key < v[t].key) ? v[t].l : v[t].r;
    }
    return t;
  }

  bool contains(int key) {
```

```
  return search(key) != 0;
}

int order_of(int key) {
  int result = 0;
  int t = root;
  while (t) {
    if (key > v[t].key) {
      result += 1 + v[v[t].l].cnt;
      t = v[t].r;
    } else {
      t = v[t].l;
    }
  }

  return result;
}

int kth_element(int k) {
  int t = root;
  while (k != v[v[t].l].cnt) {
    if (k < v[v[t].l].cnt) {
      t = v[t].l;
    } else {
      k = k - 1 - v[v[t].l].cnt;
      t = v[t].r;
    }
  }
  return v[t].key;
}

void update_cnt(int t) {
  if (t) {
    v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
  }
}

// Rotește t spre dreapta și returnează noua rădăcină.
int rotate_right(int t) {
  int x = v[t].l;
  v[t].l = v[x].r;
  v[x].r = t;
  update_cnt(t);
  update_cnt(x);
  return x;
}

// Rotește t spre stînga și returnează noua rădăcină.
int rotate_left(int t) {
  int x = v[t].r;
```

```
  v[t].r = v[x].l;
  v[x].l = t;
  update_cnt(t);
  update_cnt(x);
  return x;
}

// Inserează cheia key în subarborele lui t și scrie în t noua rădăcină.
void insert(int& t, int key) {
  if (!t) {
    v[n] = { .key = key, .pri = rand(), .cnt = 1, .l = 0, .r = 0 };
    t = n++;
  } else if (key < v[t].key) {
    // Inserează în stînga. Contăm pe fiul stîng să se reorganizeze intern.
    // Rotește spre dreapta dacă t și fiul stîng au prioritățile inversate.
    insert(v[t].l, key);
    if (v[v[t].l].pri > v[t].pri) {
      t = rotate_right(t);
    }
  } else {
    insert(v[t].r, key);
    if (v[v[t].r].pri > v[t].pri) {
      t = rotate_left(t);
    }
  }
  update_cnt(t);
}

void insert(int key) {
  insert(root, key);
}

// Șterge cheia key din subarborele lui t și scrie în t noua rădăcină. Cheia
// poate să nu existe în treap.
void erase(int& t, int key) {
  if (!t) {
    return; // Nimic de făcut, cheia nu există în treap.
  } else if (key < v[t].key) {
    erase(v[t].l, key);
  } else if (key > v[t].key) {
    erase(v[t].r, key);
  } else if (!v[t].l && !v[t].r) {
    t = 0; // Am împins cheia pînă într-o frunză.
  } else if (!v[t].l || (v[t].r && (v[v[t].r].pri > v[v[t].l].pri))) {
    // Împinge elementul de șters cu un nivel mai jos printr-o rotație la
    // stînga sau la dreapta, în funcție de prioritățile fiilor.
    t = rotate_left(t);
    erase(v[t].l, key);
  } else {
    t = rotate_right(t);
```

```
      erase(v[t].r, key);
    }
    update_cnt(t);
  }

  void erase(int key) {
    erase(root, key);
  }
};
```

Implementare cu split și merge

```
struct node {
  int key, pri;
  int cnt;
  int l, r;
};

struct treap {
  node v[N + 1];
  int n, root;

  void init() {
    n = 1; // Marchează nodul 0 ca folosit, întrucît 0 înseamnă NULL.
    root = 0;
  }

  int search(int key) {
    int t = root;
    while (t && (key != v[t].key)) {
      t = (key < v[t].key) ? v[t].l : v[t].r;
    }
    return t;
  }

  bool contains(int key) {
    return search(key) != 0;
  }

  int order_of(int key) {
    int result = 0;
    int t = root;
    while (t) {
      if (key > v[t].key) {
        result += 1 + v[v[t].l].cnt;
        t = v[t].r;
      } else {
        t = v[t].l;
```

```
    }
  }

  return result;
}

int kth_element(int k) {
  int t = root;
  while (k != v[v[t].l].cnt) {
    if (k < v[v[t].l].cnt) {
      t = v[t].l;
    } else {
      k = k - 1 - v[v[t].l].cnt;
      t = v[t].r;
    }
  }
  return v[t].key;
}

void update_cnt(int t) {
  if (t) {
    v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
  }
}

// Sparge subarborele lui t în valori ≤ key și > key și pune cele două
// rădăcini în l și în r.
void split(int t, int key, int& l, int& r) {
  if (!t) {
    l = r = 0;
  } else if (v[t].key <= key) {
    split(v[t].r, key, v[t].r, r);
    l = t;
    update_cnt(t);
  } else {
    split(v[t].l, key, l, v[t].l);
    r = t;
    update_cnt(t);
  }
}

// Unifică subarborii lui l și r și pune rădăcina rezultată în t.
void merge(int& t, int l, int r) {
  if (!l) {
    t = r;
  } else if (!r) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    merge(v[l].r, v[l].r, r);
    t = l;
```

```
    } else {
      merge(v[r].l, l, v[r].l);
      t = r;
    }
    update_cnt(t);
  }

  // Inserează elementul în subarborele lui t și scrie în t noua rădăcină.
  void insert(int& t, int elem) {
    if (!t) {
      t = elem;
    } else if (v[elem].pri > v[t].pri) {
      split(t, v[elem].key, v[elem].l, v[elem].r);
      t = elem;
    } else {
      insert(v[t].key <= v[elem].key ? v[t].r : v[t].l, elem);
    }
    update_cnt(t);
  }

  void insert(int key) {
    v[n] = { .key = key, .pri = rand(), .cnt = 1, .l = 0, .r = 0 };
    insert(root, n++);
  }

  // Șterge cheia key din subarborele lui t și scrie în t noua rădăcină.
  void erase(int& t, int key) {
    if (v[t].key == key) {
      merge(t, v[t].l, v[t].r);
    } else {
      erase(key < v[t].key ? v[t].l : v[t].r, key);
    }
    update_cnt(t);
  }

  void erase(int key) {
    erase(root, key);
  }
};
```

Implementare cu split și merge și alocare dinamică

```
struct treap {
  int key, pri, cnt;
  treap *l, *r;

  treap() {
  }
```

```
treap(int key) {
  this->key = key;
  pri = rand() >> 1;
  cnt = 1;
  l = r = NULL;
}

void init() {
  // Pentru compatibilitate cu structurile alocate static, folosim funcția
  // init(), nu constructorul, pentru inițializarea rădăcinii. Rădăcina va
  // exista mereu și va avea cheie minimă și prioritate maximă.
  key = 0;
  pri = RAND_MAX;
  cnt = 0;
  l = r = NULL;
}

treap* search(int key) {
  treap* t = this->r;
  while (t && key != t->key) {
    t = (key < t->key) ? t->l : t->r;
  }
  return t;
}

bool contains(int key) {
  return search(key) != NULL;
}

int count(treap* t) {
  return t ? t->cnt : 0;
}

int order_of(int key) {
  int result = 0;
  treap* t = this->r;
  while (t) {
    if (key > t->key) {
      result += 1 + count(t->l);
      t = t->r;
    } else {
      t = t->l;
    }
  }

  return result;
}

int kth_element(int k) {
```

```
  treap* t = this->r;
  int c;
  while (k != (c = count(t->l))) {
    if (k < c) {
      t = t->l;
    } else {
      k -= c + 1;
      t = t->r;
    }
  }
  return t->key;
}

void update_cnt(treap* t) {
  if (t) {
    t->cnt = 1 + count(t->l) + count(t->r);
  }
}

// Sparge subarborele lui t în valori ≤ key și > key și pune cele două
// rădăcini în l și în r.
void split(treap* t, int key, treap** l, treap** r) {
  if (!t) {
    *l = *r = NULL;
  } else if (t->key <= key) {
    split(t->r, key, &t->r, r);
    *l = t;
    update_cnt(t);
  } else {
    split(t->l, key, l, &t->l);
    *r = t;
    update_cnt(t);
  }
}

// Unifică subarborii lui l și r și pune rădăcina rezultată în t.
void merge(treap** t, treap* l, treap* r) {
  if (!l) {
    *t = r;
  } else if (!r) {
    *t = l;
  } else if (l->pri > r->pri) {
    merge(&l->r, l->r, r);
    *t = l;
  } else {
    merge(&r->l, l, r->l);
    *t = r;
  }
  update_cnt(*t);
}
```

```
  // Inserează cheia key în subarborele lui t și scrie în t noua rădăcină.
  void insert(treap** t, treap* elem) {
    if (!*t) {
      *t = elem;
    } else if (elem->pri > (*t)->pri) {
      split(*t, elem->key, &elem->l, &elem->r);
      *t = elem;
    } else {
      insert((*t)->key <= elem->key ? &(*t)->r : &(*t)->l, elem);
    }
    update_cnt(*t);
  }

  void insert(int key) {
    treap* elem = new treap(key);
    insert(&this->r, elem);
  }

  // Șterge cheia key din subarborele lui t și scrie în t noua rădăcină.
  void erase(treap** t, int key) {
    if ((*t)->key == key) {
      merge(t, (*t)->l, (*t)->r);
    } else {
      erase((key < (*t)->key) ? &(*t)->l : &(*t)->r, key);
    }
    update_cnt(*t);
  }

  void erase(int key) {
    erase(&this->r, key);
  }
};
```

Implementare de treap implicit

```
// Treap implicit implementat cu split și merge.
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <vector>

const int N = 1'000;
const int MAX_VAL = 1'000'000;

struct node {
  int val, pri;
```

```
  int cnt;
  int l, r;
};

struct treap {
  node v[10 * N + 1]; // Nu facem garbage collection.
  int n, root;

  void init() {
    n = 1; // Marchează nodul 0 ca folosit, întrucît 0 înseamnă NULL.
    root = 0;
  }

  void update_cnt(int t) {
    if (t) {
      v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
    }
  }

  // Sparge subarborele lui t în primele pos valori și restul și pune cele
  // două rădăcini în l și în r.
  void split(int t, int pos, int& l, int& r) {
    if (!t) {
      l = r = 0;
    } else if (pos <= v[v[t].l].cnt) {
      split(v[t].l, pos, l, v[t].l);
      r = t;
      update_cnt(t);
    } else {
      split(v[t].r, pos - v[v[t].l].cnt - 1, v[t].r, r);
      l = t;
      update_cnt(t);
    }
  }

  // Unifică subarborii lui l și r și pune rădăcina rezultată în t.
  void merge(int& t, int l, int r) {
    if (!l) {
      t = r;
    } else if (!r) {
      t = l;
    } else if (v[l].pri > v[r].pri) {
      merge(v[l].r, v[l].r, r);
      t = l;
    } else {
      merge(v[r].l, l, v[r].l);
      t = r;
    }
    update_cnt(t);
  }
```

```
// Inserează elementul elem la poziția pos și scrie în t noua rădăcină.
void insert(int& t, int pos, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, pos, v[elem].l, v[elem].r);
    t = elem;
  } else if (pos <= v[v[t].l].cnt) {
    insert(v[t].l, pos, elem);
  } else {
    insert(v[t].r, pos - v[v[t].l].cnt - 1, elem);
  }
  update_cnt(t);
}

void insert(int pos, int val) {
  v[n] = { .val = val, .pri = rand(), .cnt = 1, .l = 0, .r = 0 };
  insert(root, pos, n++);
}

// Șterge poziția pos din subarborele lui t și scrie în t noua rădăcină.
void erase(int& t, int pos) {
  if (pos == v[v[t].l].cnt) {
    merge(t, v[t].l, v[t].r);
  } else if (pos < v[v[t].l].cnt) {
    erase(v[t].l, pos);
  } else {
    erase(v[t].r, pos - 1 - v[v[t].l].cnt);
  }
  update_cnt(t);
}

void erase(int pos) {
  erase(root, pos);
}

// Returnează al pos-lea element (practic o rescriere a lui kth_element).
int get_ptr(int pos) {
  int t = root;

  while (pos != v[v[t].l].cnt) {
    if (pos < v[v[t].l].cnt) {
      t = v[t].l;
    } else {
      pos -= v[v[t].l].cnt + 1;
      t = v[t].r;
    }
  }
```

```
    return t;
  }

  int get(int pos) {
    int t = get_ptr(pos);
    return v[t].val;
  }

  void set(int pos, int val) {
    int t = get_ptr(pos);
    v[t].val = val;
  }
};

treap t;
std::vector<int> v;
int values[N];

void compare() {
  for (unsigned i = 0; i < v.size(); i++) {
    assert(v[i] == t.get(i));
  }
}

void insert_op() {
  if (v.size() < N) {
    int pos = rand() % (v.size() + 1);
    int val = rand() % (MAX_VAL + 1);
    v.insert(v.begin() + pos, val);
    t.insert(pos, val);
    compare();
  }
}

void erase_op() {
  if (v.size()) {
    int pos = rand() % v.size();
    v.erase(v.begin() + pos);
    t.erase(pos);
    compare();
  }
}

void set_op() {
  if (v.size()) {
    int pos = rand() % v.size();
    int val = rand() % (MAX_VAL + 1);
    v[pos] = val;
    t.set(pos, val);
    compare();
```

```
  }
}

int main() {
  srand(time(NULL));

  t.init();

  // Umple-l până la 50%.
  for (int i = 0; i < N / 2; i++) {
    insert_op();
  }

  // Fă diverse operații.
  for (int i = 0; i < N * 10; i++) {
    switch (rand() % 3) {
      case 0: insert_op(); break;
      case 1: erase_op(); break;
      case 2: set_op(); break;
    }
  }

  // Golește-l.
  while (v.size()) {
    erase_op();
  }

  return 0;
}
```

10.2 Problema Cut and Paste (CSES)

‹ înapoi

```
#include <stdio.h>
#include <stdlib.h>

const int MAX_N = 200'000;

struct node {
  char c;
  int pri;
  int cnt;
  int l, r;
};

struct treap {
  node v[MAX_N + 1];
```

```
int root;

void update_cnt(int t) {
  if (t) {
    v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
  }
}

void split(int t, int pos, int& l, int& r) {
  if (!t) {
    l = r = 0;
  } else if (pos <= v[v[t].l].cnt) {
    split(v[t].l, pos, l, v[t].l);
    r = t;
    update_cnt(t);
  } else {
    split(v[t].r, pos - v[v[t].l].cnt - 1, v[t].r, r);
    l = t;
    update_cnt(t);
  }
}

void merge(int& t, int l, int r) {
  if (!l) {
    t = r;
  } else if (!r) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    merge(v[l].r, v[l].r, r);
    t = l;
  } else {
    merge(v[r].l, l, v[r].l);
    t = r;
  }
  update_cnt(t);
}

void insert(int& t, int pos, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, pos, v[elem].l, v[elem].r);
    t = elem;
  } else if (pos <= v[v[t].l].cnt) {
    insert(v[t].l, pos, elem);
  } else {
    insert(v[t].r, pos - v[v[t].l].cnt - 1, elem);
  }
  update_cnt(t);
}
```

```
  void move_to_end(int a, int b) {
    int x, y, z;
    split(root, a, x, y);
    split(y, b - a + 1, y, z);
    merge(x, x, z);
    merge(root, x, y);
  }

  void read(int len) {
    root = 0;
    for (int i = 1; i <= len; i++) {
      v[i] = { .c = (char)getchar(), .pri = rand(), .cnt = 1, .l = 0, .r = 0 };
      insert(root, i, i);
    }
  }

  void write(int t) {
    if (t) {
      write(v[t].l);
      putchar(v[t].c);
      write(v[t].r);
    }
  }

  void write() {
    write(root);
    putchar('\n');
  }
};

treap t;

int main() {
  int n, q, a, b;
  scanf("%d %d ", &n, &q);
  t.read(n);

  while (q--) {
    scanf("%d %d", &a, &b);
    t.move_to_end(a - 1, b - 1);
  }

  t.write();

  return 0;
}
```

10.3 Problema Reversals and Sums (CSES)

înapoi

```
#include <stdio.h>
#include <stdlib.h>

const int MAX_N = 200'000;
const int T_REVERSE = 1;

// Contract:
//
// În orice nod u, subtree_sum este suma subarborelui cu rădăcina în u. Ea
// este menținută în timp real, ca și cnt (folosit pentru treapul implicit).
// Cîmpul boolean flip indică că subarborele curent trebuie răsturnat și este
// menținut lazy. Un subarbore trebuie realmente răsturnat dacă xor-ul biților
// flip din toți strămoșii, inclusiv el însuși, este 1.
struct node {
  int val, pri;
  int cnt;
  bool flip;
  long long subtree_sum;
  int l, r;
};

struct treap {
  node v[MAX_N + 1];
  int root;

  void update_node(int t) {
    if (t) {
      v[t].cnt = 1 + v[v[t].l].cnt + v[v[t].r].cnt;
      v[t].subtree_sum = v[t].val + v[v[t].l].subtree_sum + v[v[t].r].subtree_sum;
    }
  }

  void push(int t) {
    if (v[t].flip) {
      v[t].flip = false;
      if (v[t].l) {
        v[v[t].l].flip ^= 1;
      }
      if (v[t].r) {
        v[v[t].r].flip ^= 1;
      }
      int aux = v[t].l;
      v[t].l = v[t].r;
      v[t].r = aux;
    }
```

```
}

void split(int t, int pos, int& l, int& r) {
  if (!t) {
    l = r = 0;
    return;
  }

  // Cînd coborîm pe fiul stîng sau drept, este important să știm care este
  // care.
  push(t);
  if (pos <= v[v[t].l].cnt) {
    split(v[t].l, pos, l, v[t].l);
    r = t;
  } else {
    split(v[t].r, pos - v[v[t].l].cnt - 1, v[t].r, r);
    l = t;
  }
  update_node(t);
}

void merge(int& t, int l, int r) {
  if (!l) {
    t = r;
  } else if (!r) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    // Nodul l devine rădăcină. În fiul drept al lui l vom unifica fiul său
    // drept curent cu nodul r. Deci este important ca l să știe foarte bine
    // cine este fiul său drept.
    push(l);
    merge(v[l].r, v[l].r, r);
    t = l;
  } else {
    push(r);
    merge(v[r].l, l, v[r].l);
    t = r;
  }
  update_node(t);
}

// Prin natura problemei, aici nu este important să facem push, deoarece
// încă nu avem informații lazy.
void insert(int& t, int pos, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, pos, v[elem].l, v[elem].r);
    t = elem;
  } else if (pos <= v[v[t].l].cnt) {
```

```
      insert(v[t].l, pos, elem);
    } else {
      insert(v[t].r, pos - v[v[t].l].cnt - 1, elem);
    }
    update_node(t);
  }

  void reverse(int a, int b) {
    int x, y, z;
    split(root, a, x, y);
    split(y, b - a + 1, y, z);
    v[y].flip = true;
    merge(x, x, y);
    merge(root, x, z);
  }

  long long range_sum(int a, int b) {
    int x, y, z;
    split(root, a, x, y);
    split(y, b - a + 1, y, z);
    // Rezultatul nu este influențat de biții flip din subarbore.
    long long result = v[y].subtree_sum;
    merge(x, x, y);
    merge(root, x, z);
    return result;
  }

  void read(int n) {
    root = 0;
    for (int i = 1; i <= n; i++) {
      int val;
      scanf("%d", &val);
      v[i] = { .val = val, .pri = rand(), .l = 0, .r = 0 };
      insert(root, i, i);
    }
  }
};

treap t;

int main() {
  int n, num_ops;
  scanf("%d %d", &n, &num_ops);
  t.read(n);

  while (num_ops--) {
    int type, a, b;
    scanf("%d %d %d", &type, &a, &b);
    a--;
    b--;
```

```
    if (type == T_REVERSE) {
      t.reverse(a, b);
    } else {
      printf("%lld\n", t.range_sum(a, b));
    }
  }

  return 0;
}
```

10.4 Problema T-Shirts (Codeforces)

înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

const int MAX_TSHIRTS = 200'000;
const int MAX_CUSTOMERS = 200'000;

int answer[MAX_CUSTOMERS];

struct tshirt {
  int cost;
  int quality;
};

// Contract:
//
// Valoarea reală a unui nod este bugetul minus suma valorilor lazy_budget din
// toți strămoșii, inclusiv nodul însuși. Numărul de tricouri cumpărate de un
// nod este suma valorilor lazy_bought din toți strămoșii inclusiv el însuși.
struct node {
  int pri;
  int budget, bought;
  int lazy_budget, lazy_bought;
  int orig_id;
  int l, r;
};

struct treap {
  node v[MAX_CUSTOMERS + 1];
  int n, root;

  node make_node(int budget, int orig_id) {
    int r = ((rand() & 0x7fff) << 15) | (rand() & 0x7fff);
```

```
    return {
      .pri = r,
      .budget = budget,
      .bought = 0,
      .lazy_budget = 0,
      .lazy_bought = 0,
      .orig_id = orig_id,
      .l = 0,
      .r = 0,
    };
  }

  void init() {
    n = 1;
    root = 0;
  }

  void push(int t) {
    if (v[t].l) {
      v[v[t].l].lazy_budget += v[t].lazy_budget;
      v[v[t].l].lazy_bought += v[t].lazy_bought;
    }
    if (v[t].r) {
      v[v[t].r].lazy_budget += v[t].lazy_budget;
      v[v[t].r].lazy_bought += v[t].lazy_bought;
    }
    v[t].budget += v[t].lazy_budget;
    v[t].bought += v[t].lazy_bought;
    v[t].lazy_budget = 0;
    v[t].lazy_bought = 0;
  }

  void split(int t, int budget, int& l, int& r) {
    if (!t) {
      l = r = 0;
      return;
    }

    push(t);
    if (v[t].budget < budget) {
      split(v[t].r, budget, v[t].r, r);
      l = t;
    } else {
      split(v[t].l, budget, l, v[t].l);
      r = t;
    }
  }

  void merge(int& t, int l, int r) {
    if (!l) {
```

```
    t = r;
  } else if (!r) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    merge(v[l].r, v[l].r, r);
    t = l;
  } else {
    merge(v[r].l, l, v[r].l);
    t = r;
  }
}

void _insert(int& t, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, v[elem].budget, v[elem].l, v[elem].r);
    t = elem;
  } else {
    push(t);
    _insert(v[elem].budget <= v[t].budget ? v[t].l : v[t].r, elem);
    // Vezi https://codeforces.com/blog/entry/92340#comment-814503 pentru
    // motivul operatorului <=.
  }
}

void insert(int budget, int orig_id) {
  v[n] = make_node(budget, orig_id);
  _insert(root, n++);
}

void insert_all(int& dest, int src) {
  if (src) {
    push(src);
    insert_all(dest, v[src].l);
    insert_all(dest, v[src].r);
    v[src].l = v[src].r = 0; // decuplează nodul de treapul prezent
    _insert(dest, src);
  }
}

void purchase(int cost) {
  int no_buy, buy, poor, rich;
  split(root, cost, no_buy, buy);
  v[buy].lazy_bought++;
  v[buy].lazy_budget -= cost;
  split(buy, cost, poor, rich);
  insert_all(no_buy, poor);
  merge(root, no_buy, rich);
}
```

```
  void _collect_answers(int t) {
    if (t) {
      push(t);
      answer[v[t].orig_id] = v[t].bought;
      _collect_answers(v[t].l);
      _collect_answers(v[t].r);
    }
  }

  void collect_answers() {
    _collect_answers(root);
  }
};

treap t;
tshirt tee[MAX_TSHIRTS];
int num_tshirts, num_customers;

void init_rng() {
  struct timeval time;
  gettimeofday(&time, NULL);
  int seed = time.tv_sec * 1'000'000 + time.tv_usec;
  srand(seed);
}

void read_data() {
  scanf("%d", &num_tshirts);
  for (int i = 0; i < num_tshirts; i++) {
    scanf("%d %d", &tee[i].cost, &tee[i].quality);
  }

  t.init();
  scanf("%d", &num_customers);
  for (int i = 0; i < num_customers; i++) {
    int budget;
    scanf("%d", &budget);
    t.insert(budget, i);
  }
}

void sort_tshirts() {
  std::sort(tee, tee + num_tshirts, [](tshirt a, tshirt b) {
    return (a.quality > b.quality) ||
      ((a.quality == b.quality) && (a.cost < b.cost));
  });
}

void iterate_thirts() {
  for (int i = 0; i < num_tshirts; i++) {
```

```
    t.purchase(tee[i].cost);
  }
}

void write_answers() {
  for (int i = 0; i < num_customers; i++) {
    printf("%d ", answer[i]);
  }
  printf("\n");
}

int main() {
  init_rng();
  read_data();
  sort_tshirts();
  iterate_thirts();
  t.collect_answers();
  write_answers();

  return 0;
}
```

10.5 Problema Clasament (Lot 2016)

înapoi

Soluție cu treap (versiune online).

```
// Complexitate: O((n + q) log n).
#include <map>
#include <stdio.h>
#include <stdlib.h>

typedef unsigned long long u64;

const int MAX_N = 50'000;
const int BASE = 60'133;

struct student {
  int score;
  int time;

  friend bool operator< (student a, student b) {
    return (a.score > b.score) ||
      ((a.score == b.score) & (a.time < b.time));
  }
};
```

```
student s[MAX_N];
u64 power[MAX_N + 1];
int n, num_ops;

struct node {
  int id, pri;
  int cnt;
  u64 hash_code;
  int l, r;
};

struct treap {
  node v[MAX_N + 1];
  int n, root;

  void init(int _n) {
    n = 1; // Începem de la 1 deoarece 0 înseamnă NULL.

    for (int i = 0; i < _n; i++) {
      v[n] = { .id = i, .pri = rand(), .l = 0, .r = 0 };
      insert(root, n++);
    }
  }

  void update_node(int t) {
    if (t) {
      node &l = v[v[t].l];
      node &r = v[v[t].r]; // syntactic sugar
      v[t].cnt = 1 + l.cnt + r.cnt;
      v[t].hash_code =
        l.hash_code * power[1 + r.cnt] +
        v[t].id * power[r.cnt] +
        r.hash_code;
    }
  }

  // Sparge subarborele lui t în studenți ≤ id și > id. Pune rădăcinile în l
  // și r.
  void split(int t, int id, int& l, int& r) {
    if (!t) {
      l = r = 0;
    } else if (s[id] < s[v[t].id]) {
      split(v[t].l, id, l, v[t].l);
      r = t;
      update_node(t);
    } else {
      split(v[t].r, id, v[t].r, r);
      l = t;
      update_node(t);
    }
```

```
}

// Unifică subarborii lui l și r și pune rădăcina rezultată în t.
void merge(int& t, int l, int r) {
  if (!l) {
    t = r;
  } else if (!r) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    merge(v[l].r, v[l].r, r);
    t = l;
  } else {
    merge(v[r].l, l, v[r].l);
    t = r;
  }
  update_node(t);
}

// Inserează nodul elemn în subarborele lui t și pune noua rădăcină în t.
void insert(int& t, int elem) {
  if (!t) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, v[elem].id, v[elem].l, v[elem].r);
    t = elem;
  } else {
    insert(s[v[elem].id] < s[v[t].id] ? v[t].l : v[t].r, elem);
  }
  update_node(t);
}

// Șterge studentul id din subarborele lui t și pune noua rădăcină în t.
// Returnează indicele nodului șters.
int erase(int& t, int id) {
  int result;

  if (v[t].id == id) {
    result = t;
    merge(t, v[t].l, v[t].r);
  } else {
    result = erase(s[id] < s[v[t].id] ? v[t].l : v[t].r, id);
  }

  update_node(t);
  return result;
}

void update_score(int id, int score, int time) {
  int z = erase(root, id);
  s[id] = { score, time };
```

```
    v[z].l = v[z].r = 0;
    insert(root, z);
  }

  u64 get_hash_code() {
    return v[root].hash_code;
  }
};

treap t;
FILE *fin, *fout;

void read_scores() {
  fscanf(fin, "%d %d", &n, &num_ops);
  for (int i = 0; i < n; i++) {
    fscanf(fin, "%d", &s[i].score);
  }
}

void compute_powers() {
  power[0] = 1;
  for (int i = 1; i <= n; i++) {
    power[i] = power[i - 1] * BASE;
  }
}

void process_ops() {
  t.init(n);

  std::map<u64, int> seen;
  seen[t.get_hash_code()] = 1;

  for (int time = 1; time <= num_ops; time++) {
    int i, score;
    fscanf(fin, "%d %d", &i, &score);
    i--;
    t.update_score(i, score, time);
    fprintf(fout, "%d\n", seen[t.get_hash_code()]);
    seen[t.get_hash_code()]++;
  }
}

int main() {
  fin = fopen("clasament.in", "r");
  fout = fopen("clasament.out", "w");

  read_scores();
  compute_powers();
  process_ops();
```

```
  fclose(fin);
  fclose(fout);

  return 0;
}
```

Soluție cu descompunere în radical (versiune online).

```
// Complexitate: O((q + n) sqrt n).
#include <algorithm>
#include <map>
#include <stdio.h>

typedef unsigned short u16;
typedef unsigned long long u64;

const int MAX_N = 50'000;
const int BASE = 60'133;
const int BLOCK_SIZE = 223;

struct student {
  int score;
  int time;

  friend bool operator< (student a, student b) {
    return (a.score > b.score) ||
      ((a.score == b.score) & (a.time < b.time));
  }
};

student s[MAX_N];
int n, num_ops;

struct block {
  u16 v[2 * BLOCK_SIZE];
  int size;
  u64 hash_code; // valoarea numerică în baza n, modulo 2^64

  void compute_hash_code() {
    hash_code = 0;
    for (int i = 0; i < size; i++) {
      hash_code = hash_code * BASE + v[i];
    }
  }

  void init_from_sorted_array(u16* a, int len) {
    size = len;
    for (int i = 0; i < len; i++) {
      v[i] = a[i];
```

```
    }
    compute_hash_code();
  }

  // Presupune că blocul nu este gol.
  int last() {
    return v[size - 1];
  }

  void erase_student(int x) {
    int i = 0;
    while (v[i] != x) {
      i++;
    }
    while (i < size - 1) {
      v[i] = v[i + 1];
      i++;
    }
    size--;
    compute_hash_code();
  }

  void insert_student(int x) {
    int i = size++;
    while ((i > 0) && (s[x] < s[v[i - 1]])) {
      v[i] = v[i - 1];
      i--;
    }
    v[i] = x;
    compute_hash_code();
  }
};

struct sqrt_decomposition {
  block b[MAX_N / BLOCK_SIZE + 1];
  int n;
  int nb; // numărul de blocuri
  u64 power[2 * BLOCK_SIZE + 1];
  bool needs_rebalancing;
  u16 ord[MAX_N]; // pentru ordinea inițială și pentru redistribuiri

  // Sortăm scorurile inițiale ca să facem inițializarea în O(n log n). Am putea
  // și să inserăm valorile în structură, dar ne-ar costa O(n sqrt n).
  void sort_scores() {
    for (int i = 0; i < n; i++) {
      ord[i] = i;
    }
    std::sort(ord, ord + n, [](int a, int b) {
      return s[a] < s[b];
    });
```

```
}

void compute_powers() {
  power[0] = 1;
  for (int i = 1; i <= 2 * BLOCK_SIZE; i++) {
    power[i] = power[i - 1] * BASE;
  }
}

void collect_scores() {
  int k = 0;
  for (int i = 0; i < nb; i++) {
    for (int j = 0; j < b[i].size; j++) {
      ord[k++] = b[i].v[j];
    }
  }
}

void distribute_scores() {
  nb = 0;
  int start = 0;
  while (start < n) {
    int len = std::min(BLOCK_SIZE, n - start);
    b[nb++].init_from_sorted_array(ord + start, len);
    start += BLOCK_SIZE;
  }
}

void init(int n) {
  this->n = n;
  compute_powers();
  sort_scores();
  distribute_scores();
}

void erase_student(int x) {
  int i = 0;
  while (s[b[i].last()] < s[x]) {
    i++;
  }
  b[i].erase_student(x);
  needs_rebalancing |= (b[i].size == 1);
}

void insert_student(int x) {
  int i = 0;
  while ((i < nb - 1) && (s[b[i].last()] < s[x])) {
    i++;
  }
  b[i].insert_student(x);
```

```
    needs_rebalancing |= (b[i].size == 2 * BLOCK_SIZE);
  }

  void rebalance() {
    collect_scores();
    distribute_scores();
    needs_rebalancing = false;
  }

  void update_score(int x, int score, int time) {
    erase_student(x);
    s[x] = { score, time };
    insert_student(x);
    if (needs_rebalancing) {
      rebalance();
    }
  }

  u64 get_hash_code() {
    u64 result = 0;
    for (int i = 0; i < nb; i++) {
      result = result * power[b[i].size] + b[i].hash_code;
    }
    return result;
  }
};

sqrt_decomposition decomp;
FILE *fin, *fout;

void read_scores() {
  fscanf(fin, "%d %d", &n, &num_ops);
  for (int i = 0; i < n; i++) {
    fscanf(fin, "%d", &s[i].score);
  }
}

void process_ops() {
  decomp.init(n);

  std::map<u64, int> seen;
  seen[decomp.get_hash_code()] = 1;

  for (int time = 1; time <= num_ops; time++) {
    int i, score;
    fscanf(fin, "%d %d", &i, &score);
    i--;
    decomp.update_score(i, score, time);
    u64 h = decomp.get_hash_code();
    fprintf(fout, "%d\n", seen[h]);
```

```
    seen[h]++;
  }
}

int main() {
  fin = fopen("clasament.in", "r");
  fout = fopen("clasament.out", "w");

  read_scores();
  process_ops();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

10.6 Problema Roata Norocului (Lot 2024)

‹ înapoi • versiune online

```
// Complexitate: O(n log n).
#include <stdio.h>
#include <stdlib.h>

typedef long long i64;

const int MAX_NODES = 200'000;
const int MAX_P2 = 1 << 18;
const int INF = 1'000'000'001;
const int NIL = -1;

const int TYPE_UPDATE = 1;

int min(int x, int y) {
  return (x < y) ? x : y;
}

struct treap_node {
  int key, pri;
  i64 sum;
  int cnt;
  int l, r;
};

treap_node treap_buf[2 * MAX_NODES];
int treap_ptr;
```

```
struct treap {
  treap_node *v;
  int n;

  treap_node make_node(int key, int pri) {
    return {
      .key = key,
      .pri = pri,
      .sum = key,
      .cnt = 1,
      .l = NIL,
      .r = NIL,
    };
  }

  void init(int size) {
    v = treap_buf + treap_ptr;
    treap_ptr += size + 1;
    v[0] = make_node(INF, INF);
    n = 1;
  }

  int get_count(int t) {
    return (t == NIL) ? 0 : v[t].cnt;
  }

  i64 get_sum(int t) {
    return (t == NIL) ? 0 : v[t].sum;
  }

  void update_sum_cnt(int t) {
    v[t].sum = get_sum(v[t].l) + get_sum(v[t].r) + v[t].key;
    v[t].cnt = get_count(v[t].l) + get_count(v[t].r) + 1;
  }

  void split(int t, int key, int& l, int& r) {
    if (t == NIL) {
      l = r = NIL;
    } else if (v[t].key <= key) {
      split(v[t].r, key, v[t].r, r);
      l = t;
      update_sum_cnt(t);
    } else {
      split(v[t].l, key, l, v[t].l);
      r = t;
      update_sum_cnt(t);
    }
  }

  void insert(int& t, int elem) {
```

```
  if (t == NIL) {
    t = elem;
  } else if (v[elem].pri > v[t].pri) {
    split(t, v[elem].key, v[elem].l, v[elem].r);
    t = elem;
  } else {
    insert(v[t].key <= v[elem].key ? v[t].r : v[t].l, elem);
  }
  update_sum_cnt(t);
}

void insert(int key) {
  v[n++] = make_node(key, rand());
  insert(v[0].l, n - 1);
}

void merge(int& t, int l, int r) {
  if (l == NIL) {
    t = r;
  } else if (r == NIL) {
    t = l;
  } else if (v[l].pri > v[r].pri) {
    merge(v[l].r, v[l].r, r);
    t = l;
  } else {
    merge(v[r].l, l, v[r].l);
    t = r;
  }
  if (t != NIL) {
    update_sum_cnt(t);
  }
}

// Returnează nodul șters.
int erase(int& t, int key) {
  int result;
  if (v[t].key == key) {
    result = t;
    merge(t, v[t].l, v[t].r);
  } else {
    result = erase(key < v[t].key ? v[t].l : v[t].r, key);
  }
  if (t != NIL) {
    update_sum_cnt(t);
  }
  return result;
}

void change(int old_key, int new_key) {
  int t = erase(v[0].l, old_key);
```

```
    v[t] = make_node(new_key, rand());
    insert(v[0].l, t);
  }

  void totals_up_to(int key, int& cnt, i64& sum) {
    cnt = sum = 0;

    int t = 0;
    while (t != NIL) {
      if (key > v[t].key) {
        cnt += get_count(v[t].l) + 1;
        sum += get_sum(v[t].l) + v[t].key;
        t = v[t].r;
      } else {
        t = v[t].l;
      }
    }
  }

  // Returnează suma contribuțiilor pe muchiile spre fii cînd valoarea nodului
  // se schimbă.
  i64 get_contrib_change(int old_val, int new_val) {
    int min_val = min(old_val, new_val);
    int max_val = old_val + new_val - min_val;

    int l, r;
    i64 sum_l, sum_center;
    int size = n - 1;

    totals_up_to(min_val, l, sum_l);
    totals_up_to(max_val, r, sum_center);
    sum_center -= sum_l;

    i64 delta =
      sum_center                    // contribuții noi pe [l, r)
      + (i64)(size - r) * max_val   // contribuții noi pe [r, end)
      - (i64)(size - l) * min_val; // contribuții vechi pe [l, end)

    return (new_val > old_val) ? delta : -delta;
  }
};

struct node {
  int adj;
  int val;
  int parent;
  int num_children;
  int tin, tout;
  treap trp;
};
```

```
struct edge {
  int v, next;
};

struct segment_tree_node {
  i64 sum;     // suma valorilor nodurilor din subarbore
  i64 contrib; // suma contribuțiilor nodurilor din subarbore
  int min_val; // valoarea minimă a unui nod din subarbore

  void include(segment_tree_node& other) {
    sum += other.sum;
    contrib += other.contrib;
    min_val = min(min_val, other.min_val);
  }
};

struct segment_tree {
  segment_tree_node v[2 * MAX_P2];
  int n;

  void init(int n) {
    while (n & (n - 1)) {
      n += (n & -n);
    }
    this->n = n;
    for (int i = n; i <= 2 * n; i++) {
      v[i].min_val = INF;
    }
  }

  void set_value(int pos, int val) {
    pos += n;
    int delta = val - v[pos].sum;
    v[pos].sum = v[pos].min_val = val;

    for (pos >>= 1; pos; pos >>= 1) {
      v[pos].sum += delta;
      v[pos].min_val = min(v[2 * pos].min_val, v[2 * pos + 1].min_val);
    }
  }

  void add_contrib(int pos, i64 delta) {
    for (pos += n; pos; pos >>= 1) {
      v[pos].contrib += delta;
    }
  }

  // [l, r] interval închis
  i64 get_collapse_cost(int l, int r) {
```

```
    segment_tree_node acc = { 0, 0, INF };
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        acc.include(v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        acc.include(v[r--]);
      }
      r >>= 1;
    }

    return acc.sum - acc.min_val - acc.contrib;
  }
};

node a[MAX_NODES + 1];
edge e[MAX_NODES];
segment_tree st;
int n, tact;

void add_edge(int u, int parent) {
  static int pos = 1;
  a[u].parent = parent;
  a[parent].num_children++;
  e[pos] = { u, a[parent].adj };
  a[parent].adj = pos++;
}

void read_tree() {
  scanf("%d", &n);
  for (int u = 1; u <= n; u++) {
    scanf("%d", &a[u].val);
  }
  for (int u = 2; u <= n; u++) {
    int p;
    scanf("%d", &p);
    add_edge(u, p);
  }
};

void linearize_dfs(int u) {
  a[u].tin = tact++;

  for (int p = a[u].adj; p; p = e[p].next) {
    linearize_dfs(e[p].v);
```

```
  }

  a[u].tout = tact - 1; // Vrem ca [tin, tout] să fie închis.
}

void build_segment_tree() {
  st.init(n + 1);

  for (int u = 1; u <= n; u++) {
    st.set_value(a[u].tin, a[u].val);

    i64 contrib = 0;
    a[u].trp.init(a[u].num_children);
    for (int p = a[u].adj; p; p = e[p].next) {
      int v = e[p].v;
      a[u].trp.insert(a[v].val);
      contrib += min(a[u].val, a[v].val);
    }
    st.add_contrib(a[u].tin, contrib);
  }
}

void update(int u, int val) {
  // modificarea contribuției în u
  i64 delta = a[u].trp.get_contrib_change(a[u].val, val);
  st.add_contrib(a[u].tin, delta);

  // modificarea contribuției în părintele lui u
  int p = a[u].parent;
  if (p) {
    a[p].trp.change(a[u].val, val);
    delta = min(a[p].val, val) - min(a[p].val, a[u].val);
    st.add_contrib(a[p].tin, delta);
  }

  // modificarea valorii
  a[u].val = val;
  st.set_value(a[u].tin, a[u].val);
}

i64 query(int u) {
  return st.get_collapse_cost(a[u].tin, a[u].tout);
}

void process_ops() {
  int num_ops, type, u, val;

  scanf("%d", &num_ops);
  while (num_ops--) {
    scanf("%d %d", &type, &u);
```

```
    if (type == TYPE_UPDATE) {
      scanf("%d", &val);
      update(u, val);
    } else {
      printf("%lld\n", query(u));
    }
  }
}

int main() {
  read_tree();
  linearize_dfs(1);
  build_segment_tree();
  process_ops();

  return 0;
}
```

Anexa 11

Păduri de mulțimi disjuncte

11.1 Problema Restructuring Company (Codeforces)

‹ înapoi • versiune online

```
// Complexitate: O((n + q) log* n).
#include <stdio.h>

const int MAX_N = 200'000;
const int T_MERGE = 1;
const int T_MERGE_RANGE = 2;
const int T_QUERY = 3;

struct disjoint_set_forest {
  int p[MAX_N + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  // Unește v de u. Fără union by rank.
  void unite(int u, int v) {
    p[find(v)] = find(u);
  }

  bool are_united(int u, int v) {
    return find(u) == find(v);
```

```
  }
};

disjoint_set_forest dsf;
int end[MAX_N + 1];

void merge_range(int x, int y) {
  while (x < y) {
    dsf.unite(x, y);
    int old_end = end[x];
    end[x] = y;
    x = old_end + 1;
  }
}

int main() {
  int n, q;
  scanf("%d %d", &n, &q);

  dsf.init(n);
  for (int i = 1; i <= n; i++) {
    end[i] = i;
  }

  while (q--) {
    int type, x, y;
    scanf("%d %d %d", &type, &x, &y);
    if (type == T_MERGE) {
      dsf.unite(x, y);
    } else if (type == T_MERGE_RANGE) {
      merge_range(x, y);
    } else { // T_QUERY
      printf(dsf.are_united(x,  y) ? "YES\n" : "NO\n");
    }
  }

  return 0;
}
```

11.2 Problema Trampoline (infO(1) Cup 2020)

înapoi

Sursă cu *binary lifting* (versiune online).

```
// Complexitate: O((n + q) log n).
#include <algorithm>
#include <map>
```

```cpp
#include <stdio.h>

const int MAX_TRAMPOLINES = 200'000;
const int MAX_QUERIES = 200'000;
const int MAX_LOG = 18;

struct trampoline {
  int r, c;

  // cîte rînduri putem coborî de la această trambulină
  int len;

  // parent[i] = trambulina cu 2^i rînduri sub noi, spre dreapta
  int parent[MAX_LOG + 1];
};

struct query {
  int r1, c1, r2, c2;
  int orig_index;
};

trampoline t[MAX_TRAMPOLINES + 1];
std::map<int,int> active_t;
query q[MAX_QUERIES];
bool answer[MAX_QUERIES];
int n, num_queries;

void read_data() {
  int r, c; // ignorate
  scanf("%d %d %d\n", &r, &c, &n);

  for (int i = 1; i <= n; i++) {
    scanf("%d %d", &t[i].r, &t[i].c);
  }

  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d %d %d", &q[i].r1, &q[i].c1, &q[i].r2, &q[i].c2);
    q[i].orig_index = i;
  }
}

// Sortează după col descrescător, apoi după row descrescător.
void sort_trampolines() {
  std::sort(t + 1, t + n + 1, [](trampoline a, trampoline b) {
    return (a.c > b.c) || ((a.c == b.c) && (a.r > b.r));
  });
}

// Idem.
```

```
void sort_queries() {
  std::sort(q, q + num_queries, [](query a, query b) {
    return (a.c1 > b.c1) || ((a.c1 == b.c1) && (a.r1 > b.r1));
  });
}

void process_trampoline(int i) {
  auto it = active_t.find(t[i].r + 1);
  if (it == active_t.end()) {
    // Nu există altă trambulină verde la care să sărim. Încă putem coborî un rînd folosind
    // această trambulină.
    t[i].len = 1;
  } else {
    int j = it->second;
    t[i].len = 1 + t[j].len;
    t[i].parent[0] = j;
    for (int k = 1; k <= MAX_LOG; k++) {
      t[i].parent[k] = t[t[i].parent[k - 1]].parent[k - 1];
    }
  }
  active_t[t[i].r] = i;
}

// Călătorește delta rînduri în jos de la t[i].
int descend(int delta, int i) {
  while (delta) {
    int bit = __builtin_ctz(delta);
    i = t[i].parent[bit];
    delta &= delta - 1;
  }
  return i;
}

bool process_query(query& q) {
  if (q.r1 == q.r2) {
    // Putem merge la dreapta, dar nu și la stînga, pe rîndul curent.
    return (q.c1 <= q.c2);
  }

  if (q.r1 > q.r2) {
    // Nu putem urca.
    return false;
  }

  auto it = active_t.find(q.r1);
  if (it == active_t.end()) {
    return false; // Nu există trambulină pe rîndul nostru.
  }

  int i = it->second;
```

```
  int to_descend = q.r2 - q.r1;
  if (t[i].len < to_descend) {
    // Există un șir de trambuline, dar nu coboară suficient.
    return false;
  }

  // Coboară doar până la rîndul r2-1. Ne pasă de coloana acelei trambuline,
  // pentru că ne va permite să coborîm la r2.
  int j = descend(to_descend - 1, i);
  return (t[j].c <= q.c2); // Am ajuns pe rîndul r2 înainte sau după c2?
}

void scan() {
  int j = 0; // indicele interogării
  for (int i = 1; i <= n; i++) {
    process_trampoline(i);
    // Acum putem răspunde la toate interogările care încep la dreapta sau sub
    // t[i + 1].
    while ((j < num_queries) &&
           ((i == n) ||
            (q[j].c1 > t[i + 1].c) ||
            ((q[j].c1 == t[i + 1].c) && (q[j].r1 > t[i + 1].r)))) {
      answer[q[j].orig_index] = process_query(q[j]);
      j++;
    }
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf(answer[i] ? "Yes\n" : "No\n");
  }
}

int main() {
  read_data();
  sort_trampolines();
  sort_queries();
  scan();
  write_answers();

  return 0;
}
```

Sursă cu mulțimi disjuncte (versiune online).

```
// Complexitate: O(n log n) datorită sortării, O(n log* n) în rest.
//
// v3: Citire rapidă. Problema este acum dominată de std::sort, care ocupă cam
```

```
// 65% din timpul de rulare.
#include <algorithm>
#include <ctype.h>
#include <stdio.h>

const int MAX_QUERIES = 200'000;
const int MAX_EVENTS = 600'000; // trambuline + 2 * interogări
const int T_QUERY_START = 1;
const int T_QUERY_END = 2;
const int T_TRAMPOLINE = 3;
const int NONE = -1;

struct event {
  int r, c;
  int index; // relevant doar pentru interogări
  unsigned char type;
};

// O structură de mulțimi disjuncte pentru indici de interogări. Reține și
// rîndul curent normalizat pentru fiecare indice.
struct disjoint_set_forest {
  int p[MAX_QUERIES];
  int row[MAX_QUERIES]; // relevant doar pentru rădăcini
  int n;

  void init(int n) {
    for (int u = 0; u < n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  int get_row(int u) {
    return row[find(u)];
  }

  // Unifică v cu u întotdeauna.
  void unite(int u, int v) {
    p[find(v)] = find(u);
  }
};

event e[MAX_EVENTS];
int orig_r[MAX_EVENTS];
bool answer[MAX_QUERIES];
```

```
disjoint_set_forest dsf;

// at_row[i] = indicile mulțimii de pe rîndul normalizat i
int at_row[MAX_EVENTS];

int n; // numărul de evenimente
int num_queries;

struct fast_input {
  static const int BUF_SIZE = 4096;
  char buf[BUF_SIZE + 1];
  int pos = BUF_SIZE;

  char get_char() {
    if (pos == BUF_SIZE) {
      int read = fread(buf, 1, BUF_SIZE, stdin);
      buf[read] = '\0';
      pos = 0;
    }
    return buf[pos++];
  }

  int read_int() {
    int result = 0;
    char c;

    do {
      c = get_char();
    } while (isspace(c));

    do {
      result = 10 * result + c - '0';
      c = get_char();
    } while (isdigit(c));

    return result;
  }
};

void read_data() {
  fast_input f;

  f.read_int();
  f.read_int(); // ignoră r și c
  n = f.read_int();

  for (int i = 0; i < n; i++) {
    e[i] = { f.read_int(), f.read_int(), 0, T_TRAMPOLINE };
  }
```

```
  num_queries = f.read_int();
  for (int i = 0; i < num_queries; i++) {
    int r1 = f.read_int();
    int c1 = f.read_int();
    int r2 = f.read_int();
    int c2 = f.read_int();
    // Ignoră interogările care se termină înainte să înceapă. Vezi
    // comentariul pentru process_query_end().
    if ((r2 >= r1) && (c2 >= c1)) {
      e[n++] = { r1, c1, i, T_QUERY_START };
      e[n++] = { r2, c2, i, T_QUERY_END };
    }
  }
}

void normalize_rows() {
  std::sort(e, e + n, [](event a, event b) {
    return (a.r < b.r);
  });

  int ptr = -1;

  for (int i = 0; i < n; i++) {
    if ((ptr < 0) || (e[i].r != orig_r[ptr])) {
      orig_r[++ptr] = e[i].r;
    }
    e[i].r = ptr;
  }
}

// Sortează crescător după coloană, apoi crescător după rînd, apoi după tipul
// evenimentului (pentru că se poate să începem o interogare, apoi să coborîm
// imediat).
void sort_events() {
  std::sort(e, e + n, [](event a, event b) {
    return
      (a.c < b.c) ||
      ((a.c == b.c) && (a.r < b.r)) ||
      ((a.c == b.c) && (a.r == b.r) && (a.type < b.type));
  });
}

void process_query_start(event& e) {
  if (at_row[e.r] == NONE) {
    // e este singura interogare de pe acest rînd.
    dsf.row[e.index] = e.r;
    at_row[e.r] = e.index;
  } else {
    dsf.unite(at_row[e.r], e.index);
  }
```

```
}

// Interogarea este fezabilă dacă a reușit să călătorească la e.r *sau mai
// jos* (pentru că ne-am fi putut opri mai devreme dacă doream). Totuși,
// aceasta nu distinge cazul în care r2 > r1 încă de la început, care nu este
// fezabil. Pe acela îl filtrăm la intrare.
void process_query_end(event& e) {
  // Care sînt coordonatele prezente?
  int row = dsf.get_row(e.index);
  answer[e.index] = (row >= e.r);
}

// Toate interogările de pe acest rînd coboară la următorul, dar numai dacă
// coordonatele originale diferă prin exact 1.
//
// Notă: Aceasta înseamnă că niciodată nu coborîm la un rînd unde nu există
// nici trambuline nici interogări. Acel rînd nici nu există printre
// coordonatele normalizate. Aceasta este corect; nu avem niciun motiv să
// facem asta.
void process_trampoline(event& e) {
  if ((at_row[e.r] != NONE) && (orig_r[e.r + 1] == orig_r[e.r] + 1)) {
    if (at_row[e.r + 1] == NONE) {
      // Nu există interogări pe rîndul e.r + 1. Mută mulțimea acolo.
      dsf.row[at_row[e.r]]++;
      at_row[e.r + 1] = at_row[e.r];
    } else {
      // Unifică mulțimile.
      dsf.unite(at_row[e.r + 1], at_row[e.r]);
    }
    // Nu mai avem interogări pe acest rînd (pot apărea altele mai tîrziu).
    at_row[e.r] = NONE;
  }
}

void scan() {
  // Inițializează linia de baleiere.
  for (int i = 0; i < n; i++) {
    at_row[i] = NONE;
  }
  dsf.init(num_queries);

  for (int i = 0; i < n; i++) {
    switch (e[i].type) {
      case T_QUERY_START:
        process_query_start(e[i]);
        break;
      case T_QUERY_END:
        process_query_end(e[i]);
        break;
      case T_TRAMPOLINE:
```

```
        process_trampoline(e[i]);
        break;
    }
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf(answer[i] ? "Yes\n" : "No\n");
  }
}

int main() {
  read_data();
  normalize_rows();
  sort_events();
  scan();
  write_answers();

  return 0;
}
```

Anexa 12

Operații pe biți

12.1 Problema Fill the Bag (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_BITS = 29;
const int IMPOSSIBLE = -1;

int c[MAX_BITS + 1];
long long bag_space, to_spare;

int log2(int x) {
  return 31 - __builtin_clz(x);
}

long long min(long long x, long long y) {
  return (x < y) ? x : y;
}

void read_data() {
  int num_boxes, box;
  scanf("%lld %d", &bag_space, &num_boxes);
  to_spare = -bag_space;

  while (num_boxes--) {
    scanf("%d", &box);
    to_spare += box;
    c[log2(box)]++;
  }
}

int get_min_cuts() {
  int cuts = 0;
```

```
  for (int i = MAX_BITS; i >= 0; i--) {
    // Preferă cutiile mai mari pentru a evita tăieturile.
    int size = 1 << i;
    int to_take = min(c[i], bag_space / size);
    c[i] -= to_take;
    bag_space -= (long long)to_take * size;

    // Taie o cutie dacă există și dacă nu ne permitem să o sărim. Mută
    // jumătățile cu cutiile mai mici.
    if (i && c[i] && (to_spare < (long long)c[i] * size)) {
      cuts++;
      c[i]--;
      c[i - 1] += 2;
    }

    to_spare -= (long long)c[i] * size;
    c[i] = 0; // curățenie
  }

  return bag_space ? IMPOSSIBLE : cuts;
}

void process_test() {
  read_data();
  printf("%d\n", get_min_cuts());
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    process_test();
  }

  return 0;
}
```

12.2 Problema Compatible Numbers (Codeforces)

‹ înapoi

Sursă cu un întreg per mască (versiune online).

```
// Inspirată de https://codeforces.com/contest/165/submission/290884750
#include <stdio.h>

const int MAX_N = 1'000'000;
```

```
const int MAX_BITS = 22;
const int MAX_VALUE = 1 << MAX_BITS;

int a[MAX_N];
int fit[1 << MAX_BITS];
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
    fit[a[i]] = a[i];
  }
}

void compute_fits() {
  // Propagă măștile în ordinea bitului, apoi în ordinea măștii.
  for (int bit = 0; bit < MAX_BITS; bit++) {
    for (int mask = 1; mask < MAX_VALUE; mask++) {
      if ((mask & (1 << bit)) && !fit[mask]) {
        fit[mask] = fit[mask ^ (1 << bit)];
      }
    }
  }
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    int neg = (MAX_VALUE - 1) ^ a[i];
    int answer = fit[neg] ? fit[neg] : -1;
    printf("%d ", answer);
  }
  printf("\n");
}

int main() {
  read_data();
  compute_fits();
  write_answers();

  return 0;
}
```

Sursă cu un singur bit per mască (versiune online).

```
// Ca și versiunea anterioară, dar stochează un singur bit și recuperează
// răspunsurile mai târziu.
#include <stdio.h>
```

```
const int MAX_N = 1'000'000;
const int MAX_BITS = 22;
const int MAX_VALUE = 1 << MAX_BITS;

struct bitset {
  unsigned v[MAX_VALUE / 32];

  void set(int pos) {
    v[pos >> 5] |= 1 << (pos & 31);
  }

  bool get(int pos) {
    return v[pos >> 5] >> (pos & 31) & 1;
  }
};

int a[MAX_N];
bitset fit;
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
    fit.set(a[i]);
  }
}

void compute_fits() {
  for (int bit = 0; bit < MAX_BITS; bit++) {
    for (int mask = 1; mask < MAX_VALUE; mask++) {
      if ((mask & (1 << bit)) && fit.get(mask ^ (1 << bit))) {
        fit.set(mask);
      }
    }
  }
}

// Găsește un număr de la intrare care încape în mask.
int recover_number(int mask) {
  int copy = mask;
  while (copy) {
    int singleBit = copy & -copy;
    int submask = mask ^ singleBit;
    if (fit.get(submask)) {
      // Am găsit un bit de eliminat. Resetează procesul.
      mask = copy = submask;
    } else {
      copy ^= singleBit;
    }
```

```
  }
  return mask;
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    int neg = (MAX_VALUE - 1) ^ a[i];
    int answer = fit.get(neg)
      ? recover_number(neg)
      : -1;
    printf("%d ", answer);
  }
  printf("\n");
}

int main() {
  read_data();
  compute_fits();
  write_answers();

  return 0;
}
```

12.3 Problema Yet Another Substring Reverse (Codeforces)

❮ înapoi • versiune online

```
#include <stdio.h>
#include <ctype.h>

const int MAX_LEN = 1'000'000;
const int SIGMA = 20;

char s[MAX_LEN + 1];
bool obtainable[1 << SIGMA];
char submask_pop[1 << SIGMA];
int len;

void read_data() {
  char c;
  len = 0;
  while (isalpha((c = getchar()))) {
    s[len++] = c - 'a';
  }
}

void build_obtainable_masks() {
  for (int i = 0; i < len; i++) {
```

```
    int mask = 0;
    int j = i;
    while ((j < len) && !(mask & (1 << s[j]))) {
      mask |= 1 << s[j++];
      obtainable[mask] = true;
    }
  }
}

void improve(char& x, int y) {
  if (y > x) {
    x = y;
  }
}

void build_submask_populations() {
  for (int mask = 1; mask < (1 << SIGMA); mask++) {
    if (obtainable[mask]) {
      submask_pop[mask] = __builtin_popcount(mask);
    } else {
      for (int bit = 0; bit < SIGMA; bit++) {
        if (mask & (1 << bit)) {
          int s = submask_pop[mask ^ (1 << bit)];
          improve(submask_pop[mask], s);
        }
      }
    }
  }
}

void find_pair() {
  char ans = 0;

  for (int mask = 1; mask < (1 << SIGMA); mask++) {
    if (obtainable[mask]) {
      int complement = (1 << SIGMA) - 1 - mask;
      improve(ans, __builtin_popcount(mask) + submask_pop[complement]);
    }
  }

  printf("%d\n", ans);
}

int main() {
  read_data();
  build_obtainable_masks();
  build_submask_populations();
  find_pair();

  return 0;
```

```
}
```

12.4 Problema Cases (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1 << 18;
const int MAX_SIGMA = 18;

char s[MAX_N + 1];
bool bad[1 << MAX_SIGMA];
int f[MAX_SIGMA]; // frecvențe în fereastra glisantă
int n, sigma, max_word_length;

void read_data() {
  scanf("%d %d %d ", &n, &sigma, &max_word_length);
  for (int i = 0; i < n; i++) {
    s[i] = getchar() - 'A';
  }
}

void reset() {
  for (int i = 0; i < sigma; i++) {
    f[i] = 0;
  }

  for (int i = 0; i < (1 << sigma); i++) {
    bad[i] = false;
  }
}

int complement(int mask) {
  return ((1 << sigma) - 1) ^ mask;
}

void include(int& mask, int letter) {
  if (++f[letter] == 1) {
    mask ^= (1 << letter);
  }
}

void exclude(int& mask, int letter) {
  if (--f[letter] == 0) {
    mask ^= (1 << letter);
  }
}
```

```
// Glisează o fereastră de lungime max_word_length. Menține masca ferestrei
// curente. Complementul ferestrei este garantat rău, întrucît fereastra
// (max_word_length caractere consecutive) nu include cazuri.
void init_bad_masks() {
  int mask = 0;

  for (int i = 0; i < max_word_length - 1; i++) {
    include(mask, s[i]);
  }

  for (int i = max_word_length - 1; i < n; i++) {
    include(mask, s[i]);
    bad[complement(mask)] = true;
    exclude(mask, s[i - max_word_length + 1]);
  }

  int end_mask = 1 << s[n - 1];
  bad[complement(end_mask)] = true;
}

// Dacă o mască este rea, atunci toate submăștile ei sînt rele.
void propagate_bad_masks() {
  for (int mask = (1 << sigma) - 1; mask; mask--) {
    if (bad[mask]) {
      for (int bit = 0; bit < sigma; bit++) {
        if (mask & (1 << bit)) {
          bad[mask ^ (1 << bit)] = true;
        }
      }
    }
  }
}

int get_min_pop_good_mask() {
  int min = sigma;

  for (int mask = 0; mask < (1 << sigma); mask++) {
    if (!bad[mask]) {
      int pop = __builtin_popcount(mask);
      if (pop < min) {
        min = pop;
      }
    }
  }

  return min;
}

int main() {
```

```
  int num_tests;

  scanf("%d", &num_tests);
  while(num_tests--) {
    read_data();
    reset();
    init_bad_masks();
    propagate_bad_masks();
    int answer = get_min_pop_good_mask();
    printf("%d\n", answer);
  }

  return 0;
}
```

12.5 Problema The Minimum Number of Variables (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 23;
const int NONE = -1;

struct bitset {
  unsigned v[(1 << MAX_N) / 32];

  void set(int pos) {
    v[pos >> 5] |= 1 << (pos & 31);
  }

  bool get(int pos) {
    return v[pos >> 5] >> (pos & 31) & 1;
  }
};

// poss[mask] = true dacă și numai dacă putem procesa primele i variabile
// a.î. variabilele curente să fie cele dictate de mască. Notă: i este dat de
// bitul cel mai semnificativ al măștii.
bitset poss;

// sum[mask] = true dacă și numai dacă variabilele dictate de mască pot fi
// folosite ca să obținem a i+1-a variabilă.
bitset sum;

int a[MAX_N], n;
```

```
void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int log2(int x) {
  return 31 - __builtin_clz(x);
}

// Presupune că submăștile au fost deja actualizate.
bool can_obtain_sum(unsigned mask, int target) {
  int lsb = log2(mask & -mask);
  if (sum.get(mask ^ (1 << lsb))) {
    return true;
  }

  while (mask) {
    int bit = log2(mask);
    mask ^= (1 << bit);
    if (a[lsb] + a[bit] == target) {
      return true;
    }
  }

  return false;
}

// Menține măștile pe bit-1 biți care pot genera a[bit].
void update_sum(int bit) {
  for (unsigned mask = (1u << (bit - 1)); mask < (1u << bit); mask++) {
    if (can_obtain_sum(mask, a[bit])) {
      sum.set(mask);
    }
  }
}

void set_all_continuations(unsigned mask, int bit) {
  poss.set(mask ^ (1 << bit)); // folosește o variabilă nouă
  unsigned cp = mask;
  while (cp) {
    unsigned lsb = cp & -cp;
    poss.set(mask ^ (1 << bit) ^ lsb); // înlocuiește o variabilă existentă
    cp ^= lsb;
```

```
  }
}

void update_poss(int bit) {
  for (unsigned mask = (1u << (bit - 1)); mask < (1u << bit); mask++) {
    if (poss.get(mask) && sum.get(mask)) {
      set_all_continuations(mask, bit);
    }
  }
}

void visit_all_bits() {
  // putem procesa obiectul 0 (mask 000...001 = 1)
  poss.set(1);

  for (int bit = 1; bit < n; bit++) {
    update_sum(bit);
    update_poss(bit);
  }
}

// Returnează populația minimă printre măștile care au bitul n-1 setat.
int get_min_pop_mask() {
  int min_pop = n + 1;
  for (unsigned mask = 1 << (n - 1); mask < (1u << n); mask++) {
    if (poss.get(mask)) {
      min_pop = min(min_pop, __builtin_popcount(mask));
    }
  }
  return (min_pop == n + 1) ? NONE : min_pop;
}

int main() {
  read_data();
  visit_all_bits();
  int answer = get_min_pop_mask();
  printf("%d\n", answer);

  return 0;
}
```

12.6 Problema Math Test (Codeforces)

înapoi

Sursă varianta 1 (versiune online).

```
// Inspirată de https://codeforces.com/contest/165/submission/290884750
```

```
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MAX_BITS = 22;
const int MAX_VALUE = 1 << MAX_BITS;

int a[MAX_N];
int fit[1 << MAX_BITS];
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
    fit[a[i]] = a[i];
  }
}

void compute_fits() {
  // Propagă măștile în ordinea bitului, apoi în ordinea măștii.
  for (int bit = 0; bit < MAX_BITS; bit++) {
    for (int mask = 1; mask < MAX_VALUE; mask++) {
      if ((mask & (1 << bit)) && !fit[mask]) {
        fit[mask] = fit[mask ^ (1 << bit)];
      }
    }
  }
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    int neg = (MAX_VALUE - 1) ^ a[i];
    int answer = fit[neg] ? fit[neg] : -1;
    printf("%d ", answer);
  }
  printf("\n");
}

int main() {
  read_data();
  compute_fits();
  write_answers();

  return 0;
}
```

Sursă varianta 2 (versiune online).

```
// Ca și versiunea anterioară, dar stochează un singur bit și recuperează
```

```
// răspunsurile mai târziu.
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MAX_BITS = 22;
const int MAX_VALUE = 1 << MAX_BITS;

struct bitset {
  unsigned v[MAX_VALUE / 32];

  void set(int pos) {
    v[pos >> 5] |= 1 << (pos & 31);
  }

  bool get(int pos) {
    return v[pos >> 5] >> (pos & 31) & 1;
  }
};

int a[MAX_N];
bitset fit;
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
    fit.set(a[i]);
  }
}

void compute_fits() {
  for (int bit = 0; bit < MAX_BITS; bit++) {
    for (int mask = 1; mask < MAX_VALUE; mask++) {
      if ((mask & (1 << bit)) && fit.get(mask ^ (1 << bit))) {
        fit.set(mask);
      }
    }
  }
}

// Găsește un număr de la intrare care încape în mask.
int recover_number(int mask) {
  int copy = mask;
  while (copy) {
    int singleBit = copy & -copy;
    int submask = mask ^ singleBit;
    if (fit.get(submask)) {
      // Am găsit un bit de eliminat. Resetează procesul.
      mask = copy = submask;
```

```
    } else {
      copy ^= singleBit;
    }
  }
  return mask;
}

void write_answers() {
  for (int i = 0; i < n; i++) {
    int neg = (MAX_VALUE - 1) ^ a[i];
    int answer = fit.get(neg)
      ? recover_number(neg)
      : -1;
    printf("%d ", answer);
  }
  printf("\n");
}

int main() {
  read_data();
  compute_fits();
  write_answers();

  return 0;
}
```

12.7 Implementare de bitset

‹ înapoi

```
// Pregătește un vector de <poziție, valoare>. La fiecare trecere, shiftează
// pozițiile cu 1. Notează valorile anterioare. La final algoritmului, compară
// abordarea naivă cu abordarea compactată.
#include <bitset>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

const int ARRAY_SIZE = 1'000'000;
const int NUM_CHANGES = 100'000;
const int NUM_PASSES = 10'000;

struct change {
  int pos;
  bool val, prev_val;
};

struct packed_32 {
```

```
  unsigned v[ARRAY_SIZE / 32];

  void set_branching(int pos, bool val) {
    unsigned& c = v[pos >> 5];
    unsigned mask = 1 << (pos & 31);

    if (val) {
      c |= mask;
    } else {
      c &= ~mask;
    }
  }

  void set(int pos, bool val) {
    unsigned& c = v[pos >> 5];
    int shift = pos & 31;

    // Prima jumătate curăță bitul, iar a doua îl scrie.
    c = (c & ~(1 << shift)) | (val << shift);
  }

  bool get(int pos) {
    return (v[pos >> 5] >> (pos & 31)) & 1;
  }
};

struct packed_8 {
  unsigned char v[ARRAY_SIZE / 8];

  void set(int pos, bool val) {
    unsigned char& c = v[pos >> 3];
    int shift = pos & 7;

    // Prima jumătate curăță bitul, iar a doua îl scrie.
    c = (c & ~(1 << shift)) | (val << shift);
  }

  bool get(int pos) {
    return (v[pos >> 3] >> (pos & 7)) & 1;
  }
};

change c[NUM_CHANGES];
bool v[ARRAY_SIZE];
std::bitset<ARRAY_SIZE> bs;
packed_32 p32;
packed_8 p8;
long long t0;

void init_rng() {
```

```
  struct timeval time;
  gettimeofday(&time, NULL);
  int seed = time.tv_sec * 1'000'000 + time.tv_usec;
  srand(seed);
}

void mark_time() {
  timeval tv;

  gettimeofday (&tv, NULL);
  t0 = 1000LL * tv.tv_sec + tv.tv_usec / 1000;
}

void report_time(const char* msg) {
  timeval tv;

  gettimeofday (&tv, NULL);
  long long t = 1000LL * tv.tv_sec + tv.tv_usec / 1000;

  int millis = t - t0;
  int ops = NUM_CHANGES * NUM_PASSES * 2; // am măsurat citiri + scrieri

  printf("%s: %lld ms for %d ops (%d Mops/sec)\n", msg, t - t0,
         ops, ops / millis / 1000);
}

void gen_locations() {
  for (int i = 0; i < NUM_CHANGES; i++) {
    c[i].pos = rand() % (ARRAY_SIZE - NUM_PASSES);
    c[i].val = rand() % 2;
  }
}

void run_p32_branching() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = p32.get(pos);
      p32.set_branching(pos, c[i].val);
    }
  }
}

void run_p32() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = p32.get(pos);
      p32.set(pos, c[i].val);
    }
```

```
  }
}

void run_p8() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = p8.get(pos);
      p8.set(pos, c[i].val);
    }
  }
}

void run_bitset() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = bs[pos];
      bs[pos] = c[i].val;
    }
  }
}

void run_naive() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = v[pos];
      v[pos] = c[i].val;
    }
  }
}

void verify_p32() {
  for (int i = 0; i < ARRAY_SIZE; i++) {
    if (v[i] != p32.get(i)) {
      printf("Error: position %d, naive is %d, p32 is %d.\n",
             i, v[i], p32.get(i));
      exit(1);
    }
  }
}

void verify_p8() {
  for (int i = 0; i < ARRAY_SIZE; i++) {
    if (v[i] != p8.get(i)) {
      printf("Error: position %d, naive is %d, p8 is %d.\n",
             i, v[i], p8.get(i));
      exit(1);
    }
```

```
  }
}

void verify_bitset() {
  for (int i = 0; i < ARRAY_SIZE; i++) {
    if (v[i] != bs[i]) {
      printf("Error: position %d, naive is %d, bitset is %d.\n",
             i, v[i], (int)bs[i]);
      exit(1);
    }
  }
}

int main(void) {
  init_rng();
  gen_locations();

  mark_time();
  run_naive();
  report_time("naive");

  mark_time();
  run_p32_branching();
  report_time("packed array, 32 bit, branching");
  verify_p32();

  mark_time();
  run_p32();
  report_time("packed array, 32 bit");
  verify_p32();

  mark_time();
  run_p8();
  report_time("packed array, 8 bit");
  verify_p8();

  mark_time();
  run_bitset();
  report_time("STL bitset");
  verify_bitset();

  return 0;
}
```

12.8 Implementare de variabile pe doi biți

‹ înapoi

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

const int ARRAY_SIZE = 1'000'000;
const int NUM_CHANGES = 100'000;
const int NUM_PASSES = 10'000;

struct change {
  int pos;
  char val, prev_val;
};

struct packed_array {
  unsigned v[ARRAY_SIZE / 16];

  void set(int pos, int val) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    v[word] = (v[word] & ~(3 << offset)) | (val << offset);
  }

  int get(int pos) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    return (v[word] >> offset) & 3;
  }
};

change c[NUM_CHANGES];
char v[ARRAY_SIZE];
packed_array packed;
long long t0;

void init_rng() {
  struct timeval time;
  gettimeofday(&time, NULL);
  int seed = time.tv_sec * 1'000'000 + time.tv_usec;
  srand(seed);
}

void mark_time() {
  timeval tv;

  gettimeofday (&tv, NULL);
  t0 = 1000LL * tv.tv_sec + tv.tv_usec / 1000;
}

void report_time(const char* msg) {
```

```
  timeval tv;

  gettimeofday (&tv, NULL);
  long long t = 1000LL * tv.tv_sec + tv.tv_usec / 1000;

  int millis = t - t0;
  int ops = NUM_CHANGES * NUM_PASSES * 2; // am măsurat citiri + scrieri

  printf("%s: %lld ms for %d ops (%d Mops/sec)\n", msg, t - t0,
         ops, ops / millis / 1000);
}

void gen_locations() {
  for (int i = 0; i < NUM_CHANGES; i++) {
    c[i].pos = rand() % (ARRAY_SIZE - NUM_PASSES);
    c[i].val = rand() % 4;
  }
}

void run_packed() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = packed.get(pos);
      packed.set(pos, c[i].val);
    }
  }
}

void run_naive() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int i = 0; i < NUM_CHANGES; i++) {
      int pos = c[i].pos + pass;
      c[i].prev_val = v[pos];
      v[pos] = c[i].val;
    }
  }
}

void verify_packed() {
  for (int i = 0; i < ARRAY_SIZE; i++) {
    if (v[i] != packed.get(i)) {
      printf("Error: position %d, naive is %d, packed is %d.\n",
             i, v[i], packed.get(i));
      exit(1);
    }
  }
}

int main(void) {
```

```
  init_rng();
  gen_locations();

  mark_time();
  run_naive();
  report_time("naive");

  mark_time();
  run_packed();
  report_time("packed array, 32 bit");
  verify_packed();

  return 0;
}
```

12.9 Problema DNA Evolution (Codeforces)

‹ înapoi

Sursă varianta 1 (versiune online).

```
#include <stdio.h>
#include <string.h>

const int MAX_LEN = 100'000;
const int BLOCK_SIZE = 500;
const int MAX_BLOCKS = (MAX_LEN - 1) / BLOCK_SIZE + 1;
const int LATIN_SIGMA = 26;
const int SIGMA = 4;
const int MAX_Q_LEN = 10;
const int OP_UPDATE = 1;
const int OP_QUERY = 2;

char s[MAX_LEN + 2];

struct block_info {
  int start;
  short cnt[SIGMA][MAX_Q_LEN * (MAX_Q_LEN + 1) / 2];

  void init(int start, int end) {
    this->start = start;
    for (int pos = start; pos < end; pos++) {
      update(s[pos], pos, +1);
    }
  }

  void update(int c, int pos, int val) {
    for (int mod = 1; mod <= MAX_Q_LEN; mod++) {
```

```
      int gauss = mod * (mod - 1) / 2;
      cnt[c][gauss + (pos - start) % mod] += val;
    }
  }

  int query(char* q, int qlen, int rem) {
    int result = 0;
    for (int i = 0; i < qlen; i++) {
      // Numără aparițiile lui q[rem] la poziții egale cu i modulo qlen.
      int pos = qlen * (qlen - 1) / 2 + i;
      result += cnt[(int)q[rem]][pos];
      rem = (rem + 1) % qlen;
    }
    return result;
  }
};

char letter_map[LATIN_SIGMA];
block_info block[MAX_BLOCKS];
int n, num_blocks;

void precompute_letter_map() {
  letter_map['A' - 'A'] = 0;
  letter_map['C' - 'A'] = 1;
  letter_map['G' - 'A'] = 2;
  letter_map['T' - 'A'] = 3;
}

void translate_string(char* s) {
  for (char* c = s; *c; c++) {
    *c = letter_map[*c - 'A'];
  }
}

void read_string() {
  scanf("%s", s);
  n = strlen(s);
  translate_string(s);
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void init_blocks() {
  num_blocks = (n - 1) / BLOCK_SIZE + 1;
  for (int b = 0; b < num_blocks; b++) {
    int start = b * BLOCK_SIZE;
    int end = min(start + BLOCK_SIZE, n);
    block[b].init(start, end);
```

```
  }
}

void update(int pos, int c) {
  int b = pos / BLOCK_SIZE;
  block[b].update(s[pos], pos, -1);
  s[pos] = c;
  block[b].update(s[pos], pos, +1);
}

int naive_count(int l, int r, char* q, int qlen, int rem) {
  int result = 0;
  while (l < r) {
    result += (s[l] == q[rem]);
    rem = (rem + 1) % qlen;
    l++;
  }
  return result;
}

int block_sum(int bl, int br, char* q, int qlen, int rem) {
  int result = 0;
  while (bl < br) {
    result += block[bl].query(q, qlen, rem);
    rem = (rem + BLOCK_SIZE) % qlen;
    bl++;
  }
  return result;
}

int query(int l, int r, char* q, int qlen) { // [l inclusiv, r exclusiv)
  int bl = l / BLOCK_SIZE;
  int br = r / BLOCK_SIZE;

  if (bl == br) {
    return naive_count(l, r, q, qlen, 0);
  } else {
    int rem_l = ((bl + 1) * BLOCK_SIZE - l) % qlen;
    int rem_r = (br * BLOCK_SIZE - l) % qlen;
    return naive_count(l, (bl + 1) * BLOCK_SIZE, q, qlen, 0)
      + naive_count(br * BLOCK_SIZE, r, q, qlen, rem_r)
      + block_sum(bl + 1, br, q, qlen, rem_l);
  }
}

void process_ops() {
  int num_ops, type;
  scanf("%d", &num_ops);
  while (num_ops--) {
    scanf("%d", &type);
```

```
    if (type == OP_UPDATE) {
      int x;
      char c;
      scanf("%d %c", &x, &c);
      update(x - 1, letter_map[c - 'A']);
    } else {
      int l, r, qlen;
      char q[MAX_Q_LEN + 1];
      scanf("%d %d %s", &l, &r, q);
      qlen = strlen(q);
      translate_string(q);
      int answer = query(l - 1, r, q, qlen);
      printf("%d\n", answer);
    }
  }
}

int main() {
  precompute_letter_map();
  read_string();
  init_blocks();
  process_ops();

  return 0;
}
```

Sursă varianta 2 (versiune online).

```
#include <stdio.h>
#include <string.h>

const int MAX_LEN = 100'000;
const int BLOCK_SIZE = 1'112;
const int MAX_BLOCKS = (MAX_LEN - 1) / BLOCK_SIZE + 1;
const int LATIN_SIGMA = 26;
const int SIGMA = 4;
const int MAX_Q_LEN = 10;
const int OP_UPDATE = 1;
const int OP_QUERY = 2;

// Un vector de doi biți pe element.
struct packed_array {
  unsigned v[MAX_LEN / 16];

  void set(int pos, int val) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    v[word] = (v[word] & ~(3 << offset)) | (val << offset);
  }
```

```
  int get(int pos) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    return (v[word] >> offset) & 3;
  }
};

packed_array s;

struct block_info {
  int start;
  short cnt[SIGMA][MAX_Q_LEN * (MAX_Q_LEN + 1) / 2];

  void init(int start, int end) {
    this->start = start;
    for (int pos = start; pos < end; pos++) {
      update(s.get(pos), pos, +1);
    }
  }

  void update(int c, int pos, int val) {
    for (int mod = 1; mod <= MAX_Q_LEN; mod++) {
      int gauss = mod * (mod - 1) / 2;
      cnt[c][gauss + (pos - start) % mod] += val;
    }
  }

  int query(char* q, int qlen, int rem) {
    int result = 0;
    for (int i = 0; i < qlen; i++) {
      // Numără aparițiile lui q[rem] la poziții egale cu i modulo qlen.
      int pos = qlen * (qlen - 1) / 2 + i;
      result += cnt[(int)q[rem]][pos];
      rem = (rem + 1) % qlen;
    }
    return result;
  }
};

char letter_map[LATIN_SIGMA];
block_info block[MAX_BLOCKS];
int n, num_blocks;

void precompute_letter_map() {
  letter_map['A' - 'A'] = 0;
  letter_map['C' - 'A'] = 1;
  letter_map['G' - 'A'] = 2;
  letter_map['T' - 'A'] = 3;
}
```

```
int translate_char(char c) {
  return letter_map[c - 'A'];
}

void translate_string(char* s) {
  for (char* c = s; *c; c++) {
    *c = translate_char(*c);
  }
}

void read_string() {
  char c;
  while ((c = getchar()) != '\n') {
    s.set(n++, translate_char(c));
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void init_blocks() {
  num_blocks = (n - 1) / BLOCK_SIZE + 1;
  for (int b = 0; b < num_blocks; b++) {
    int start = b * BLOCK_SIZE;
    int end = min(start + BLOCK_SIZE, n);
    block[b].init(start, end);
  }
}

void update(int pos, int c) {
  int b = pos / BLOCK_SIZE;
  block[b].update(s.get(pos), pos, -1);
  s.set(pos, c);
  block[b].update(c, pos, +1);
}

int naive_count(int l, int r, char* q, int qlen, int rem) {
  int result = 0;
  while (l < r) {
    result += (s.get(l) == q[rem]);
    rem = (rem + 1) % qlen;
    l++;
  }
  return result;
}

int block_sum(int bl, int br, char* q, int qlen, int rem) {
  int result = 0;
```

```
  while (bl < br) {
    result += block[bl].query(q, qlen, rem);
    rem = (rem + BLOCK_SIZE) % qlen;
    bl++;
  }
  return result;
}

int query(int l, int r, char* q, int qlen) { // [l inclusiv, r exclusiv)
  int bl = l / BLOCK_SIZE;
  int br = r / BLOCK_SIZE;

  if (bl == br) {
    return naive_count(l, r, q, qlen, 0);
  } else {
    int rem_l = ((bl + 1) * BLOCK_SIZE - l) % qlen;
    int rem_r = (br * BLOCK_SIZE - l) % qlen;
    return naive_count(l, (bl + 1) * BLOCK_SIZE, q, qlen, 0)
      + naive_count(br * BLOCK_SIZE, r, q, qlen, rem_r)
      + block_sum(bl + 1, br, q, qlen, rem_l);
  }
}

void process_ops() {
  int num_ops, type;
  scanf("%d", &num_ops);
  while (num_ops--) {
    scanf("%d", &type);
    if (type == OP_UPDATE) {
      int x;
      char c;
      scanf("%d %c", &x, &c);
      update(x - 1, letter_map[c - 'A']);
    } else {
      int l, r, qlen;
      char q[MAX_Q_LEN + 1];
      scanf("%d %d %s", &l, &r, q);
      qlen = strlen(q);
      translate_string(q);
      int answer = query(l - 1, r, q, qlen);
      printf("%d\n", answer);
    }
  }
}

int main() {
  precompute_letter_map();
  read_string();
  init_blocks();
  process_ops();
```

```
  return 0;
}
```

Sursă varianta 3 (versiune online).

```
#include <stdio.h>
#include <string.h>

const int MAX_LEN = 100'000;
const int BLOCK_SIZE = 1'112;
const int MAX_BLOCKS = (MAX_LEN - 1) / BLOCK_SIZE + 1;
const int LATIN_SIGMA = 26;
const int SIGMA = 4;
const int MAX_Q_LEN = 10;
const int OP_UPDATE = 1;
const int OP_QUERY = 2;

// Un vector de doi biți pe element.
struct packed_array {
  unsigned v[MAX_LEN / 16];

  void set(int pos, int val) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    v[word] = (v[word] & ~(3 << offset)) | (val << offset);
  }

  int get(int pos) {
    int word = pos / 16;
    int offset = pos % 16 * 2;
    return (v[word] >> offset) & 3;
  }
};

packed_array s;
char next[MAX_Q_LEN][MAX_Q_LEN + 1];
char next_bl[MAX_Q_LEN][MAX_Q_LEN + 1];
char letter_map[LATIN_SIGMA];
int n;

struct block_info {
  int start;
  short cnt[SIGMA][MAX_Q_LEN * (MAX_Q_LEN + 1) / 2];

  void init(int start, int end) {
    this->start = start;
    for (int pos = start; pos < end; pos++) {
      update(s.get(pos), pos, +1);
```

```
    }
  }

  void update(int c, int pos, int val) {
    for (int mod = 1; mod <= MAX_Q_LEN; mod++) {
      int gauss = mod * (mod - 1) / 2;
      cnt[c][gauss + (pos - start) % mod] += val;
    }
  }

  int query(char* q, int qlen, int rem) {
    int gauss = qlen * (qlen - 1) / 2;
    int result = 0;
    for (int i = 0; i < qlen; i++) {
      // Numără aparițiile lui q[rem] la poziții egale cu i modulo qlen.
      result += cnt[(int)q[rem]][gauss + i];
      rem = next[rem][qlen];
    }
    return result;
  }
};

block_info block[MAX_BLOCKS];

void precompute_letter_map() {
  letter_map['A' - 'A'] = 0;
  letter_map['C' - 'A'] = 1;
  letter_map['G' - 'A'] = 2;
  letter_map['T' - 'A'] = 3;
}

int translate_char(char c) {
  return letter_map[c - 'A'];
}

void translate_string(char* s) {
  for (char* c = s; *c; c++) {
    *c = translate_char(*c);
  }
}

void read_string() {
  char c;
  while ((c = getchar()) != '\n') {
    s.set(n++, translate_char(c));
  }
}

void precompute_next() {
  for (int mod = 1; mod <= MAX_Q_LEN; mod++) {
```

```
    for (int i = 0; i < mod; i++) {
      next[i][mod] = (i + 1) % mod;
      next_bl[i][mod] = (i + BLOCK_SIZE) % mod;
    }
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void init_blocks() {
  int num_blocks = (n - 1) / BLOCK_SIZE + 1;
  for (int b = 0; b < num_blocks; b++) {
    int start = b * BLOCK_SIZE;
    int end = min(start + BLOCK_SIZE, n);
    block[b].init(start, end);
  }
}

void update(int pos, int c) {
  int b = pos / BLOCK_SIZE;
  block[b].update(s.get(pos), pos, -1);
  s.set(pos, c);
  block[b].update(c, pos, +1);
}

int naive_count(int l, int r, char* q, int qlen, int& rem) {
  int result = 0;
  while (l < r) {
    result += (s.get(l) == q[rem]);
    rem = next[rem][qlen];
    l++;
  }
  return result;
}

int block_sum(int bl, int br, char* q, int qlen, int& rem) {
  int result = 0;
  while (bl < br) {
    result += block[bl].query(q, qlen, rem);
    rem = next_bl[rem][qlen];
    bl++;
  }
  return result;
}

int query(int l, int r, char* q, int qlen) { // [l inclusiv, r exclusiv)
  int bl = l / BLOCK_SIZE;
  int br = r / BLOCK_SIZE;
```

```
  int rem = 0;

  if (bl == br) {
    return naive_count(l, r, q, qlen, rem);
  } else {
    int result = naive_count(l, (bl + 1) * BLOCK_SIZE, q, qlen, rem);
    result += block_sum(bl + 1, br, q, qlen, rem);
    result += naive_count(br * BLOCK_SIZE, r, q, qlen, rem);
    return result;
  }
}

void process_ops() {
  int num_ops, type;
  scanf("%d", &num_ops);
  while (num_ops--) {
    scanf("%d", &type);
    if (type == OP_UPDATE) {
      int x;
      char c;
      scanf("%d %c", &x, &c);
      update(x - 1, translate_char(c));
    } else {
      int l, r, qlen;
      char q[MAX_Q_LEN + 1];
      scanf("%d %d %s", &l, &r, q);
      qlen = strlen(q);
      translate_string(q);
      int answer = query(l - 1, r, q, qlen);
      printf("%d\n", answer);
    }
  }
}

int main() {
  precompute_letter_map();
  precompute_next();
  read_string();
  init_blocks();
  process_ops();

  return 0;
}
```

12.10 Problema Crisp String (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_LEN = 100'000;
const int MAX_SIGMA = 17;
const int NONE = -1;

struct bitset {
  int n, num_words;
  unsigned v[(1 << MAX_SIGMA) / 32];

  void init(int n) {
    this->n = n;
    num_words = (n - 1) / 32 + 1;
  }

  void clear() {
    for (int i = 0; i < num_words; i++) {
      v[i] = 0;
    }
  }

  bool get(int pos) {
    return (v[pos >> 5] >> (pos & 31)) & 1;
  }

  void set(int pos) {
    v[pos >> 5] |= 1 << (pos & 31);
  }

  bitset& operator|=(const bitset& other) {
    for (int i = 0; i < num_words; i++) {
      v[i] |= other.v[i];
    }
    return *this;
  }
};

char s[MAX_LEN + 1];
bool compat[MAX_SIGMA][MAX_SIGMA];
int freq[MAX_SIGMA];
bitset bad, global_bad;
int n, sigma;

void read_data() {
  scanf("%d %d %s", &n, &sigma, s);
  for (int i = 0; i < sigma; i++) {
    for (int j = 0; j < sigma; j++) {
      int x;
      scanf("%d", &x);
```

```
      compat[i][j] = x;
    }
  }
}

void translate_string() {
  for (int i = 0; i < n; i++) {
    s[i] -= 'a';
    freq[(int)s[i]]++;
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void include_submasks(int x, int y) {
  // Iterează doar prin măștile care au biții x și y nesetați.
  unsigned mask = ((1 << sigma) - 1) ^ ((1 << x) | (1 << y));

  bad.set(mask);
  for (unsigned s = mask; s; s = (s - 1) & mask) {
    // s iterează prin submăști în ordine descrescătoare. Noi dorim să iterăm
    // în ordine crescătoare.
    unsigned t = mask ^ s;
    if (!bad.get(t)) {
      for (int bit = 0; bit < sigma; bit++) {
        if ((t & (1 << bit)) && bad.get(t ^ (1 << bit))) {
          bad.set(t);
        }
      }
    }
  }
}

void gen_bad_masks_for_pair(int x, int y) {
  int last_seen = NONE;
  unsigned mask = 0;
  for (int i = 0; i < n; i++) {
    if ((s[i] == x) || (s[i] == y)) {
      if (last_seen == x + y - s[i]) { // cealaltă valoare
        bad.set(mask);
      }
      mask = 0;
      last_seen = s[i];
    } else {
      mask |= (1 << s[i]);
    }
  }
```

```
  include_submasks(x, y);
  global_bad |= bad;
  bad.clear();
}

void gen_bad_masks() {
  bad.init(1 << sigma);
  global_bad.init(1 << sigma);

  for (int x = 0; x < sigma; x++) {
    for (int y = x; y < sigma; y++) {
      if (freq[x] && freq[y] && !compat[x][y]) {
        gen_bad_masks_for_pair(x, y);
      }
    }
  }
}

// Marchează care rele toate măștile inaccesibile din 0 prin orice secvență de
// eliminări.
void gen_unreachable_masks() {
  for (unsigned mask = 1; mask < (1u << sigma); mask++) {
    if (!global_bad.get(mask)) {
      bool reachable = false;
      for (int bit = 0; bit < sigma; bit++) {
        reachable |=
          (mask & (1 << bit)) &&
          !global_bad.get(mask ^ (1 << bit));
      }
      if (!reachable) {
        global_bad.set(mask);
      }
    }
  }
}

int get_min_pop() {
  int result = n;
  for (unsigned mask = 0; mask < (1u << sigma); mask++) {
    if (!global_bad.get(mask)) {
      int pop = 0;
      for (int bit = 0; bit < sigma; bit++) {
        if (!(mask & (1 << bit))) {
          pop += freq[bit];
        }
      }
      result = min(result, pop);
    }
  }
  return result;
```

```
}

int main() {
  read_data();
  translate_string();
  gen_bad_masks();
  gen_unreachable_masks();
  int answer = get_min_pop();
  printf("%d\n", answer);

  return 0;
}
```

Anexa 13

Tehnica *Meet in the middle*

13.1 Problema Meet in the Middle (CSES)

înapoi

Sursă cu sortare naivă.

```cpp
// Complexitate: O(n · 2^(n/2)).
#include <algorithm>
#include <stdio.h>

const int MAX_N = 40;
const int SET_SIZE = 1 << (MAX_N / 2);
const int INFINITY = 2'000'000'000;

int v[MAX_N];
int n, target;

struct arr {
  int v[SET_SIZE + 2], size;

  void push(int val) {
    v[size++] = val;
  }

  void rec(int* src, int pos, int sum_so_far) {
    if (pos < 0) {
      push(sum_so_far);
      return;
    }
    rec(src, pos - 1, sum_so_far);
    if (sum_so_far + src[pos] <= target) {
      rec(src, pos - 1, sum_so_far + src[pos]);
    }
  }
```

```
  void gen_sums(int* src, int n) {
    push(-INFINITY);
    push(+INFINITY);
    rec(src, n - 1, 0);
    std::sort(v, v + size);
  }
};

arr a, b;

void read_data() {
  scanf("%d %d", &n, &target);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
  }
}

void generate_subset_sums() {
  int n1 = n / 2;
  int n2 = n - n1;
  a.gen_sums(v, n1);
  b.gen_sums(v + n1, n2);
}

long long merge_sets() {
  long long answer = 0;
  int i = 1, j = b.size - 1;

  while ((i < a.size) && (j > 0)) {
    if (a.v[i] + b.v[j] < target) {
      i++;
    } else if (a.v[i] + b.v[j] > target) {
      j--;
    } else {
      int starti = i, startj = j;
      while (a.v[i] == a.v[starti]) {
        i++;
      }
      while (b.v[j] == b.v[startj]) {
        j--;
      }
      answer += (long long)(i - starti) * (startj - j);
    }
  }

  return answer;
}

int main() {
```

```
  read_data();
  generate_subset_sums();
  long long answer = merge_sets();
  printf("%lld\n", answer);

  return 0;
}
```

Sursă cu interclasare.

```
// Complexitate: O(2^(n/2)).
#include <stdio.h>

const int MAX_N = 40;
const int SET_SIZE = 1 << (MAX_N / 2);
const int INFINITY = 2'000'000'000;

int v[MAX_N];
int sum1[SET_SIZE + 2], sum2[SET_SIZE + 2];
int n, n1, n2, size1, size2, target;

void read_data() {
  scanf("%d %d", &n, &target);
  for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
  }
}

// Interclasează v[1...n] și v[1...n] + x în v[1...2n].
int include_element(int* v, int n, int x) {
  int i = n, j = n;
  for (int k = 2 * n; k >= 1; k--) {
    if (v[i] > v[j] + x) {
      v[k] = v[i--];
    } else {
      v[k] = v[j--] + x;
    }
  }

  n *= 2;
  while (v[n] > target) {
    n--;
  }

  return n;
}

int gen_sums(int* v, int n, int* dest) {
  int size = 1;
```

```
  dest[0] = -INFINITY;
  dest[1] = 0;
  for (int i = 0; i < n; i++) {
    size = include_element(dest, size, v[i]);
  }

  return size;
}

void generate_subset_sums() {
  n1 = n / 2;
  n2 = n - n1;
  size1 = gen_sums(v, n1, sum1);
  size2 = gen_sums(v + n1, n2, sum2);
}

void merge_sets(int* a, int na, int* b, int nb) {
  long long answer = 0;

  a[na + 1] = INFINITY;
  b[0] = -INFINITY;
  int i = 1, j = nb;

  while ((i <= na) && (j > 0)) {
    if (a[i] + b[j] < target) {
      i++;
    } else if (a[i] + b[j] > target) {
      j--;
    } else {
      int starti = i, startj = j;
      while (a[i] == a[starti]) {
        i++;
      }
      while (b[j] == b[startj]) {
        j--;
      }
      answer += (long long)(i - starti) * (startj - j);
    }
  }

  printf("%lld\n", answer);
}

int main() {
  read_data();
  generate_subset_sums();
  merge_sets(sum1, size1, sum2, size2);

  return 0;
```

```
}
```

13.2 Problema Anya and Cubes (Codeforces)

‹ înapoi

Sursă simplă (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 25;
const int MAX_FACTORIAL = 18;   // 19! > 10^16
const int MAX_LEFT = 1'594'323; // 3^13
const int MAX_RIGHT = 531'441;  // 3^12

int a[MAX_N], n, num_stickers;
long long target_sum;
long long fact[MAX_FACTORIAL + 1];

struct info {
  long long sum;
  char stickers;

  bool before(info other) {
    return (sum < other.sum) ||
      ((sum == other.sum) & (stickers < other.stickers));
  }
};

info left[MAX_LEFT + 2], right[MAX_RIGHT + 2];
int num_left, num_right;

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_FACTORIAL; i++) {
    fact[i] = fact[i - 1] * i;
  }
}

void read_data() {
  scanf("%d %d %lld", &n, &num_stickers, &target_sum);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
}

struct subset_generator {
```

```
  int end;
  info* dest;
  int size;

  void rec(int k, long long sum, char stickers) {
    if (k == end) {
      dest[size++] = { sum, stickers };
      return;
    }

    rec(k + 1, sum, stickers);
    if (sum + a[k] <= target_sum) {
      rec(k + 1, sum + a[k], stickers);
    }
    if ((stickers < num_stickers) &&
        (a[k] <= MAX_FACTORIAL) &&
        (sum + fact[a[k]] <= target_sum)) {
      rec(k + 1, sum + fact[a[k]], stickers + 1);
    }
  }

  void run(int start, int end, info* dest, int& size) {
    this->end = end;
    this->dest = dest;
    this->size = 0;

    rec(start, 0, 0);

    size = this->size;
    // santinele infinite
    dest[size++] = { -(target_sum + 1), 0 };
    dest[size++] = { (target_sum + 1), 0 };

    std::sort(dest, dest + size, [](info a, info b) {
      return a.before(b);
    });
  }
};

subset_generator sub_gen;

// Capsule care izolează unele variabile.
struct half_merger {
  int l, r;
  long long result;

  void merge_equal_sums() {
    long long l_sum = left[l].sum;
    int r_start = r;
    while (right[r_start].sum == right[r].sum) {
```

```
      r_start--;
    }

    while (left[l].sum == l_sum) {
      if ((r > r_start) &&
          (left[l].stickers + right[r].stickers > num_stickers)) {
        r--;
      } else {
        result += r - r_start;
        l++;
      }
    }

    r = r_start;
  }

  long long run() {
    result = 0;
    l = 0;
    r = num_right - 1;

    while (l < num_left) {
      if (left[l].sum + right[r].sum < target_sum) {
        l++;
      } else if (left[l].sum + right[r].sum > target_sum) {
        r--;
      } else {
        merge_equal_sums();
      }
    }

    return result;
  }
};

half_merger merger;

int main() {
  precompute_factorials();
  read_data();
  sub_gen.run(n / 2, n, left, num_left);
  sub_gen.run(0, n / 2, right, num_right);
  long long ans = merger.run();
  printf("%lld\n", ans);

  return 0;
}
```

Sursă cu memorie compactată (versiune online).

```cpp
#include <algorithm>
#include <stdio.h>

const int MAX_N = 25;
const int MAX_FACTORIAL = 18;   // 19! > 10^16
const int MAX_LEFT = 1'594'323; // 3^13
const int MAX_RIGHT = 531'441;  // 3^12
const int BITS = 5;

int a[MAX_N], n, num_stickers;
long long target_sum;
long long fact[MAX_FACTORIAL + 1];

struct info {
  long long v;

  long long sum() {
    return v >> BITS;
  }

  int stickers() {
    return v & ((1 << BITS) - 1);
  }

  void set(long long sum, int stickers) {
    v = (sum << BITS) + stickers;
  }
};

info left[MAX_LEFT + 2], right[MAX_RIGHT + 2];
int num_left, num_right;

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_FACTORIAL; i++) {
    fact[i] = fact[i - 1] * i;
  }
}

void read_data() {
  scanf("%d %d %lld", &n, &num_stickers, &target_sum);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
}

struct subset_generator {
  int end;
  info* dest;
```

```
  int size;

  void rec(int k, long long sum, char stickers) {
    if (k == end) {
      dest[size++].set(sum, stickers);
      return;
    }

    rec(k + 1, sum, stickers);
    if (sum + a[k] <= target_sum) {
      rec(k + 1, sum + a[k], stickers);
    }
    if ((stickers < num_stickers) &&
        (a[k] <= MAX_FACTORIAL) &&
        (sum + fact[a[k]] <= target_sum)) {
      rec(k + 1, sum + fact[a[k]], stickers + 1);
    }
  }

  void run(int start, int end, info* dest, int& size) {
    this->end = end;
    this->dest = dest;
    this->size = 0;

    rec(start, 0, 0);

    size = this->size;
    // santinele infinite
    dest[size++].set(-(target_sum + 1), 0);
    dest[size++].set((target_sum + 1), 0);

    std::sort(dest, dest + size, [](info a, info b) {
      return a.v < b.v;
    });
  }
};

subset_generator sub_gen;

// Capsule care izolează unele variabile.
struct half_merger {
  int l, r;
  long long result;

  void merge_equal_sums() {
    long long l_sum = left[l].sum();
    int r_start = r;
    while (right[r_start].sum() == right[r].sum()) {
      r_start--;
    }
```

```
    while (left[l].sum() == l_sum) {
      if ((r > r_start) &&
          (left[l].stickers() + right[r].stickers() > num_stickers)) {
        r--;
      } else {
        result += r - r_start;
        l++;
      }
    }

    r = r_start;
  }

  long long run() {
    result = 0;
    l = 0;
    r = num_right - 1;

    while (l < num_left) {
      if (left[l].sum() + right[r].sum() < target_sum) {
        l++;
      } else if (left[l].sum() + right[r].sum() > target_sum) {
        r--;
      } else {
        merge_equal_sums();
      }
    }

    return result;
  }
};

half_merger merger;

int main() {
  precompute_factorials();
  read_data();
  sub_gen.run(n / 2, n, left, num_left);
  sub_gen.run(0, n / 2, right, num_right);
  long long ans = merger.run();
  printf("%lld\n", ans);

  return 0;
}
```

13.3 Problema Prime Gift (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

typedef unsigned long long u64;

const int MAX_PRIMES = 16;
const int LEFT_FRAC = 6;
const int MAX_NUM_UGLY = 1'000'000;
const u64 LIMIT = 1e18;

int primes[MAX_PRIMES];
int num_primes, mid;
u64 kth;
u64 ugly1[MAX_NUM_UGLY], ugly2[MAX_NUM_UGLY];
int num_ugly1, num_ugly2;

struct ugly_generator {
  int ptr, last;
  u64* v;

  void rec(int k, u64 prod) {
    if (k == last) {
      v[ptr++] = prod;
    } else {
      for (__int128 f = prod; f <= LIMIT; f *= primes[k]) {
        rec(k + 1, f);
      }
    }
  }

  void run(int first_prime, int last_prime, u64* dest, int& size) {
    ptr = 0;
    last = last_prime;
    v = dest;
    rec(first_prime, 1);
    std::sort(dest, dest + ptr);
    size = ptr;
  }
};

ugly_generator ugly_gen;

void read_data() {
  scanf("%d", &num_primes);
  for (int i = 0; i < num_primes; i++) {
```

```
    scanf("%d", &primes[i]);
  }
  scanf("%llu", &kth);
  mid = num_primes * LEFT_FRAC / MAX_PRIMES;
}

u64 count_ugly(u64 up_to) {
  u64 result = 0;
  int j = num_ugly2 - 1;
  for (int i = 0; i < num_ugly1; i++) {
    while ((__int128)ugly1[i] * ugly2[j] > up_to) {
      j--;
    }
    result += (j + 1);
  }
  return result;
}

u64 bin_search_kth_number() {
  u64 l = 0, r = LIMIT; // (l exclusiv, r inclusiv]
  while (r - l > 1) {
    u64 mid = (l + r) / 2;
    if (count_ugly(mid) < kth) {
      l = mid;
    } else {
      r = mid;
    }
  }
  return r;
}

int main() {
  read_data();
  ugly_gen.run(0, mid, ugly1, num_ugly1);
  ugly_gen.run(mid, num_primes, ugly2, num_ugly2);
  u64 ans = bin_search_kth_number();
  printf("%llu\n", ans);

  return 0;
}
```

13.4 Problema Lizard Era: Beginning (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_QUESTS = 25;
const int MAX_SUBSETS = 2'125'764; // 3^13 + 3^12
const long long INFINITY = 300'000'000;
const int POW3[] = {
  1, 3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049, 177147,
  531441, 1594323, 43046721,
};

const bool LEFT = false;
const bool RIGHT = true;

struct attitude {
  int a, b, c;
};

struct subset {
  int a, tritmask;
  long long fp; // fingerprint (amprentă)

  void set(int a, int b, int c, int tritmask, bool half) {
    this->a = a;
    this->tritmask = tritmask;

    // Amprenta este combinația lui (b-a) și (c-a), urmată de un bit care
    // indică jumătatea (stînga/dreapta). Pentru a grupa împreună submulțimile
    // care se potrivesc, negăm amprenta jumătății drepte.
    if (half == LEFT) {
      fp = (INFINITY * (b - a + INFINITY) + (c - a + INFINITY)) *  2;
    } else {
      fp = (INFINITY * (a - b + INFINITY) + (a - c + INFINITY)) *  2 + 1;
    }
  }
};

attitude att[MAX_QUESTS];
subset sub[MAX_SUBSETS];
int n, mid, num_subsets;

// O capsulă ca să nu facem variabilele from, to și half globale.
struct dfs_wrapper {
  int from, to;
  bool half;

  void run(int k, int tritmask, int a, int b, int c)  {
    if (k == to) {
      sub[num_subsets++].set(a, b, c, tritmask, half);
      return;
    }

    // a, b și respectiv c stau acasă
```

```
    run(k + 1, tritmask, a, b + att[k].b, c + att[k].c);
    run(k + 1, tritmask + POW3[k - from], a + att[k].a, b, c + att[k].c);
    run(k + 1, tritmask + 2 * POW3[k - from], a + att[k].a, b + att[k].b, c);
  }

  void master(int from, int to, int half) {
    this->from = from;
    this->to = to;
    this->half = half;

    run(from, 0, 0, 0, 0);
  }
};

dfs_wrapper dfs_wrap;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &att[i].a, &att[i].b, &att[i].c);
  }
  mid = n / 2;
}

void sort_subsets() {
  std::sort(sub, sub + num_subsets, [] (subset& p, subset& q) {
    if (p.fp != q.fp) {
      return p.fp < q.fp;
    } else if (p.fp & 1) {
      // jumătatea dreaptă
      return p.a > q.a;
    } else {
      // jumătatea stîngă
      return p.a < q.a;
    }
  });
}

long long find_match() {
  int best_a = -INFINITY;
  long long best_mask = -1;

  for (int i = 0; i < num_subsets - 1; i++) {
    if ((sub[i].fp != sub[i + 1].fp) &&
        (sub[i].fp / 2 == sub[i + 1].fp / 2) &&
        (sub[i].a + sub[i + 1].a > best_a)) {
      best_a = sub[i].a + sub[i + 1].a;
      best_mask = (long long)sub[i + 1].tritmask * POW3[mid] + sub[i].tritmask;
    }
  }
```

```
  return best_mask;
}

void print_solution(long long tritmask) {
  if (tritmask == -1) {
    printf("Impossible\n");
    return;
  }

  for (int i = 0; i < n; i++) {
    switch (tritmask % 3) {
      case 0:
        printf("MW\n");
        break;
      case 1:
        printf("LW\n");
        break;
      default:
        printf("LM\n");
    }
    tritmask /= 3;
  }
}

int main() {
  read_data();

  dfs_wrap.master(0, mid, LEFT);
  dfs_wrap.master(mid, n, RIGHT);
  sort_subsets();
  long long tritmask = find_match();
  print_solution(tritmask);

  return 0;
}
```

13.5 Problema Make Them Similar (Codeforces)

înapoi

Sursă simplă (versiune online).

```
#include <stdio.h>

const int MAX_N = 100;
const int BITS = 15;
const int MAX_TUPLES = 1 << (BITS + 1);
```

```
const bool LEFT = false;
const bool RIGHT = true;

int n;

struct tuple {
  char diff[MAX_N - 1];
  bool half;
  short mask;

  bool matches(const tuple& other) {
    if (half == other.half) {
      return false;
    }

    int i = 0;
    while ((i < n - 1) && (diff[i] == other.diff[i]))  {
      i++;
    }

    return (i == n - 1);
  }
};

int a[MAX_N], b[MAX_N];
tuple t[MAX_TUPLES];
int perm[MAX_TUPLES];
int num_tuples;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    a[i] = x >> BITS;
    b[i] = x & ((1 << BITS) - 1);
  }
}

void iterate_masks(int* v, bool half) {
  for (int mask = 0; mask < (1 << BITS); mask++) {
    int pc0 = __builtin_popcount(mask ^ v[0]);
    tuple& x = t[num_tuples];
    x.mask = mask;
    x.half = half;

    for (int i = 1; i < n; i++) {
      int pc = __builtin_popcount(mask ^ v[i]);
      x.diff[i - 1] = (half == LEFT) ? (pc - pc0) : (pc0 - pc);
    }
```

```
    num_tuples++;
  }
}

void swap(int i, int j) {
  int tmp = perm[i];
  perm[i] = perm[j];
  perm[j] = tmp;
}

// Sortează tuplurile t[first...last), care sînt egale pînă la poziția pos-1.
// Modifică perm[] în pasul de partiționare, dar nu modifică t[], căci ar fi
// costisitor.
void ternary_sort(int first, int last, int pos) {
  if (last - first <= 1) {
    return;
  }

  int i = first, left = first, right = last;
  char pivot = t[perm[first]].diff[pos];

  // Invariant de buclă:
  // * perm[first ... left)  a pos-a diferență este < pivot
  // * perm[left ... i)      a pos-a diferență este == pivot
  // * perm[i]               tuplul în curs de evaluare
  // * perm(i ... right)     tupluri încă neevaluate
  // * perm[right ... last)  a pos-a diferență este > pivot
  while (i < right) {
    char x = t[perm[i]].diff[pos];
    if (x < pivot) {
      swap(i++, left++);
    } else if (x > pivot)  {
      swap(i, --right);
    } else {
      i++;
    }
  }

  ternary_sort(first, left, pos);
  if (pos < n - 1) {
    ternary_sort(left, right, pos + 1);
  }
  ternary_sort(right, last, pos);
}

void sort_tuples() {
  for (int i = 0; i < num_tuples; i++) {
    perm[i] = i;
  }
```

```
  ternary_sort(0, num_tuples, 0);
}

void print_solution(int i, int j) {
  int high = (t[i].half == LEFT) ? t[i].mask : t[j].mask;
  int low = (t[i].half == LEFT) ? t[j].mask : t[i].mask;
  int x = (high << BITS) + low;
  printf("%d\n", x);
}

void find_match() {
  int i = 1;

  while ((i < num_tuples) && !t[perm[i - 1]].matches(t[perm[i]])) {
    i++;
  }

  if (i == num_tuples) {
    printf("-1\n");
  } else {
    print_solution(perm[i - 1], perm[i]);
  }
}

int main() {
  read_data();
  iterate_masks(a, LEFT);
  iterate_masks(b, RIGHT);
  sort_tuples();
  find_match();

  return 0;
}
```

Sursă cu economie de memorie (versiune online).

```
// În plus față de prima versiune, calculează diferențele la cerere pentru a
// economisi (multă) memorie. Nu mai avem nevoie de vectorul perm[] -- este
// ieftin să interschimbăm tupluri.
#include <stdio.h>

const int MAX_N = 100;
const int BITS = 15;
const int MAX_TUPLES = 1 << (BITS + 1);
const char LEFT = 0;
const char RIGHT = 1;

int a[2][MAX_N];
int n;
```

```
struct tuple {
  unsigned char half;
  unsigned short mask;

  int diff(int pos) {
    int delta =
      __builtin_popcount(mask ^ a[half][pos]) -
      __builtin_popcount(mask ^ a[half][0]);

    return (1 - 2 * half) * delta;
  }

  bool matches(tuple& other) {
    if (half == other.half) {
      return false;
    }

    int i = 1;
    while ((i < n) && (diff(i) == other.diff(i)))  {
      i++;
    }

    return (i == n);
  }
};

tuple t[MAX_TUPLES];
int num_tuples;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    int x;
    scanf("%d", &x);
    a[0][i] = x >> BITS;
    a[1][i] = x & ((1 << BITS) - 1);
  }
}

void swap(int i, int j) {
  tuple tmp = t[i];
  t[i] = t[j];
  t[j] = tmp;
}

// Sortează tuplurile t[first...last), care sînt egale pînă la poziția pos-1.
void ternary_sort(int first, int last, int pos) {
  if (last - first <= 1) {
    return;
```

```
  }

  int i = first, left = first, right = last;
  char pivot = t[first].diff(pos);

  // Invariant de buclă:
  // * t[first ... left)  a pos-a diferență este < pivot
  // * t[left ... i)      a pos-a diferență este == pivot
  // * t[i]               tuplul în curs de evaluare
  // * t(i ... right)     tupluri încă neevaluate
  // * t[right ... last)  a pos-a diferență este > pivot
  while (i < right) {
    char x = t[i].diff(pos);
    if (x < pivot) {
      swap(i++, left++);
    } else if (x > pivot)  {
      swap(i, --right);
    } else {
      i++;
    }
  }

  ternary_sort(first, left, pos);
  if (pos < n) {
    ternary_sort(left, right, pos + 1);
  }
  ternary_sort(right, last, pos);
}

void create_and_sort_tuples() {
  for (unsigned short mask = 0; mask < (1 << BITS); mask++) {
    t[num_tuples++] = { false, mask };
    t[num_tuples++] = { true, mask };
  }
  ternary_sort(0, num_tuples, 1);
}

void print_solution(int pos) {
  int sol = (t[pos].half == LEFT)
    ? ((t[pos].mask << BITS) | t[pos + 1].mask)
    : ((t[pos + 1].mask << BITS) | t[pos].mask);
  printf("%d\n", sol);
}

void find_match() {
  int i = 1;
  while ((i < num_tuples) && !t[i - 1].matches(t[i])) {
    i++;
  }
```

```
  if (i == num_tuples) {
    printf("-1\n");
  } else {
    print_solution(i - 1);
  }
}

int main() {
  read_data();
  create_and_sort_tuples();
  find_match();

  return 0;
}
```

13.6 Problema Light Switches (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

typedef unsigned long long u64;

const int MAX_BULBS = 1'000;
const int MAX_SWITCHES = 30;
const int FRAC_LEFT = 18;
const int HASH_BASE = 37;
const u64 HASH_MOD = 18'446'744'073'709'551'263ull;
const int HASH_MAP_SIZE = 1 << (FRAC_LEFT + 2); // 4x numărul de valori
const char NONE = -1;

u64 powers[MAX_BULBS];

// tabelă hash cu linear probing
struct hash_map {
  u64 key[HASH_MAP_SIZE];
  char val[HASH_MAP_SIZE];

  void init() {
    for (int i = 0; i < HASH_MAP_SIZE; i++) {
      val[i] = NONE;
    }
  }

  int find_pos(u64 k) {
    int pos = k % HASH_MAP_SIZE;
    while ((key[pos] != k) && (val[pos] != NONE)) {
```

```
      pos = (pos + 1) % HASH_MAP_SIZE;
    }
    return pos;
  }

  void minimize(u64 k, char v) {
    int pos = find_pos(k);
    if ((val[pos] == NONE) || (v < val[pos])) {
      key[pos] = k;
      val[pos] = v;
    }
  }

  char find(u64 k) {
    int pos = find_pos(k);
    return val[pos];
  }
};

struct state {
  u64 mask;
  char touched;
};

u64 sw[MAX_SWITCHES];
hash_map left_map;
state right_table[(1 << (MAX_SWITCHES - FRAC_LEFT)) + 1];
int n, s, num_days, left, right;

u64 read_bitset() {
  u64 mask = 0;
  int k, pos;
  scanf("%d", &k);
  while (k--) {
    scanf("%d", &pos);
    pos--;
    mask ^= powers[pos];
  }
  return mask;
}

void read_switches() {
  scanf("%d %d %d", &n, &s, &num_days);
  for (int i = 0; i < s; i++) {
    sw[i] = read_bitset();
  }
  left = s * FRAC_LEFT / MAX_SWITCHES;
  right = s - left;
}
```

```
void compute_powers() {
  powers[0] = 1;
  for (int i = 1; i < MAX_BULBS; i++) {
    powers[i] = (__int128)powers[i - 1] * HASH_BASE % HASH_MOD;
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void rec_left(int level, char touched, u64 mask) {
  if (level == left) {
    left_map.minimize(mask, touched);
  } else {
    rec_left(level + 1, touched, mask);
    rec_left(level + 1, touched + 1, mask ^ sw[level]);
  }
}

void create_left_hash_map() {
  left_map.init();
  rec_left(0, 0, 0);
}

void rec_right(int level, char touched, u64 mask) {
  static int pos = 0;

  if (level == s) {
    right_table[pos++] = { mask, touched };
  } else {
    rec_right(level + 1, touched, mask);
    rec_right(level + 1, touched + 1, mask ^ sw[level]);
  }
}

void create_right_table() {
  rec_right(left, 0, 0);
  std::sort(right_table, right_table + (1 << right), [](state a, state b) {
    return a.touched < b.touched;
  });
  right_table[1 << right] = { 0, MAX_SWITCHES + 1 }; // santinelă
}

void answer_queries() {
  while (num_days--) {
    u64 day = read_bitset();
    int min_touched = s + 1;

    int p = 0;
```

```
    while (right_table[p].touched < min_touched) {
      u64 desired = day ^ right_table[p].mask;
      char left_touched = left_map.find(desired);
      if (left_touched != NONE) {
        min_touched = min(min_touched, left_touched + right_table[p].touched);
      }
      p++;
    }
    printf("%d\n", (min_touched == s + 1) ? -1 : min_touched);
  }
}

int main() {
  compute_powers();
  read_switches();
  create_left_hash_map();
  create_right_table();
  answer_queries();

  return 0;
}
```

13.7 Problema Algebra Flash (Codeforces)

înapoi • versiune online

```
#include <stdio.h>

const int MAX_COLORS = 40;
const int INFINITY = 1'000'000'000;

typedef unsigned long long u64;

u64 rel[MAX_COLORS];
int cost[MAX_COLORS];
u64 rhs_mask;
int num_colors, mid;

void add_relation(int a, int b) {
  rel[a] |= (1ull << b);
  rel[b] |= (1ull << a);
}

// Adaugă relațiile <c[1],c[1]> și <c[n],c[n]> pentru a forța alegerea acestor
// două culori.
void read_data() {
  int n, prev, col;
  scanf("%d %d %d", &n, &num_colors, &prev);
```

```
  prev--;

  add_relation(prev, prev);
  for (int i = 1; i < n; i++) {
    scanf("%d", &col);
    col--;
    add_relation(prev, col);
    prev = col;
  }
  add_relation(prev, prev);

  for (int i = 0; i < num_colors; i++) {
    scanf("%d", &cost[i]);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int log2(u64 mask) {
  return 63 - __builtin_clzll(mask);
}

// Returnează o mască cu 1 de la msb la lsb inclusiv.
u64 bit_range(int msb, int lsb) {
  return ((1ull << (msb + 1)) - 1) ^ ((1ull << lsb) - 1);
}

void precompute_info() {
  mid = num_colors / 2;
  rhs_mask = bit_range(mid - 1, 0);
}

struct right_half {
  // Costul redus al unei măşti este costul minim necesar pentru a satisface
  // relaţiile între perechi de biţi 1 din mască.
  int red_cost[1 << (MAX_COLORS / 2)];

  int get_mask_cost(u64 mask) {
    int sum = 0;
    while (mask) {
      int b = log2(mask);
      sum += cost[b];
      mask ^= (1 << b);
    }
    return sum;
  }

  void compute_red_costs() {
```

```
    for (u64 mask = 1; mask < (1ull << mid); mask++) {
      int k = log2(mask);

      // Opțiunea 1: cumpără bitul k. Redu biții rămași.
      u64 remaining = mask ^ (1 << k);
      int c1 = cost[k] + red_cost[remaining];

      // Opțiunea 2: nu cumpăra bitul k. Neapărat cumpără-i toate relațiile.
      // Aceasta tratează și cazul în care k este în relație cu el însuși.
      u64 rel_mask = mask & rel[k];
      remaining = (mask ^ (1 << k)) & ~rel[k];
      int c2 = get_mask_cost(rel_mask) + red_cost[remaining];

      red_cost[mask] = min(c1, c2);
    }
  }

  int cheapest_expansion(u64 bought_mask) {
    u64 not_bought = rhs_mask ^ bought_mask;
    return get_mask_cost(bought_mask) + red_cost[not_bought];
  }
};

right_half right;

struct left_half {
  int min_cost;

  void rec(int k, int c, u64 left_mask, u64 right_mask) {
    if (k == num_colors) {
      int right_cost = right.cheapest_expansion(right_mask);
      min_cost = min(min_cost, c + right_cost);
    } else {
      // Trebuie să cumpărăm bitul k dacă are relații nesatisfăcute în
      // jumătatea stîngă unde k este bitul mai semnificativ.
      u64 unsat = rel[k] & bit_range(k, mid) & ~left_mask;
      if (!unsat) {
        // Putem să nu cumpărăm bitul k. Notează relațiile lui k în jumătatea
        // dreaptă.
        u64 rhs_relations = rel[k] & rhs_mask;
        rec(k + 1, c, left_mask, right_mask | rhs_relations);
      }
      rec(k + 1, c + cost[k], left_mask | (1ull << k), right_mask);
    }
  }

  void generate_all_valid() {
    min_cost = INFINITY;
    rec(mid, 0, 0, 0);
  }
```

```
};

left_half left;

int main() {
  read_data();
  precompute_info();
  right.compute_red_costs();
  left.generate_all_valid();
  printf("%d\n", left.min_cost);

  return 0;
}
```

13.8 Problema Helping Hiasat (Codeforces)

‹ înapoi • versiune online

```
#include <iostream>
#include <unordered_map>

const int MAX_M = 40;
const int T_CHANGE = 1;

// Normalizează numele.
struct normalizer {
  std::unordered_map<std::string, int> map;

  int normalize(std::string s) {
    auto it = map.find(s);
    if (it == map.end()) {
      int result = map.size();
      map[s] = result;
      return result;
    } else {
      return it->second;
    }
  }
};

long long exclude[MAX_M];
char max_submask[1 << (MAX_M / 2)];
int m, half;

int max(int x, int y) {
  return (x > y) ? x : y;
}
```

```
void exclude_pairs(long long mask) {
  for (int b = 0; b < m; b++) {
    if (mask & (1ll << b)) {
      exclude[b] |= mask ^ (1ll << b);
    }
  }
}

void read_data_and_create_exclusions() {
  int n, type;
  long long mask = 0;
  normalizer norm;

  std::ios::sync_with_stdio(false);
  std::cin.tie(nullptr);

  std::cin >> n >> m;
  half = m / 2;

  while (n--) {
    std::cin >> type;
    if (type == T_CHANGE) {
      exclude_pairs(mask);
      mask = 0;
    } else {
      std::string s;
      std::cin >> s;
      int id = norm.normalize(s);
      mask |= 1ll << id;
    }
  }

  exclude_pairs(mask);
}

// k = bitul curent
// mask = o mască pe primii k biți
void rec_left(int k, int mask) {
  if (k == half) {
    max_submask[mask] = __builtin_popcount(mask);
  } else {
    rec_left(k + 1, mask);
    if ((exclude[k] & mask) == 0) {
      rec_left(k + 1, mask | (1 << k));
    }
  }
}

void propagate_feasible_masks_left() {
  for (int mask = 0; mask < (1 << half); mask++) {
```

```
    if (!max_submask[mask]) { // optimizare de viteză
      for (int b = 0; b < half; b++) {
        if (mask & (1 << b)) {
          max_submask[mask] = max(max_submask[mask],
                                  max_submask[mask ^ (1 << b)]);
        }
      }
    }
  }
}

int max_pop = 0;

void rec_right(int k, long long mask, int excluded_left) {
  if (k == m) {
    int allowed_left = ((1 << half) - 1) ^ excluded_left;
    int pop = __builtin_popcountll(mask) + max_submask[allowed_left];
    max_pop = max(max_pop, pop);
  } else {
    rec_right(k + 1, mask, excluded_left);
    if ((exclude[k] & mask) == 0) {
      int excluded_left_by_k = exclude[k] & ((1 << half) - 1);
      rec_right(k + 1, mask | (1ll << k), excluded_left | excluded_left_by_k);
    }
  }
}

int main() {
  read_data_and_create_exclusions();
  rec_left(0, 0);
  propagate_feasible_masks_left();
  rec_right(half, 0, 0);
  std::cout << max_pop << '\n';

  return 0;
}
```

Anexa 14

Arbori - probleme esențiale

14.1 Generator simplu de arbori aleatorii

‹ înapoi

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

int n, k;

void usage() {
  fprintf(stderr, "Apel: ./generator <n> <k>\n");
  fprintf(stderr, "\n");
  fprintf(stderr, "Generează un arbore cu n noduri și k căi.\n");
  exit(1);
}

void parse_command_line_args(int argc, char** argv) {
  if (argc != 3) {
    usage();
  }

  n = atoi(argv[1]);
  k = atoi(argv[2]);
}

void init_rng() {
  struct timeval time;
  gettimeofday(&time, NULL);
  srand(time.tv_usec);
}

int rand2(int min, int max) {
  return min + rand() % (max - min + 1);
```

```
}

int main(int argc, char** argv) {
  parse_command_line_args(argc, argv);
  init_rng();

  printf("%d %d\n", n, k);
  for (int i = 2; i <= n; i++) {
    printf("%d %d\n", i, rand2(1, i - 1));
  }
  for (int i = 0; i < k; i++) {
    printf("%d %d\n", rand2(1, n), rand2(1, n));
  }

  return 0;
}
```

14.2 Generator avansat de arbori aleatorii

înapoi

```
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <unistd.h>

#define MAX_N 1'000'000
#define NIL -1

int v[MAX_N], p[MAX_N];

void usage() {
  fprintf(stderr, "Apel: gen <n> <chain_length> <dense_node> <op_type> [...]\n");
  fprintf(stderr, "  op_type = 0:\n");
  fprintf(stderr, "    fără valori sau actualizări\n");
  fprintf(stderr, "  op_type = 1 <num_ops> <max_value>:\n");
  fprintf(stderr, "    valori inițiale, actualizări pe nod, interogări pe nod\n");
  fprintf(stderr, "  op_type = 2 <num_ops>:\n");
  fprintf(stderr, "    fără valori sau actualizări, cu interogări pe cale\n");
  _exit(1);
}

void shuffle(int* v, int n) {
  for (int i = 0; i < n; i++) {
    int j = rand() % (i + 1);
    int tmp = v[i];
    v[i] = v[j];
```

```
    v[j] = tmp;
  }
}

void generate_tree(int n, int chain_length, int dense_node) {
  for (int i = 0; i < n; i++) {
    v[i] = i;
    p[i] = NIL;
  }

  shuffle(v, n);

  // Înlănțuie nodurile [0, chain_length).
  for (int i = 1; i < chain_length; i++) {
    p[v[i]] = v[i - 1];
  }

  // Conectează nodurile [chain_length, n - dense_node) la noduri aleatorii
  // dinaintea lor.
  for (int i = chain_length; i < n - dense_node; i++) {
    p[v[i]] = v[rand() % i];
  }

  // Conectează nodurile [n - dense_node, n) de un același părinte.
  int parent = rand() % (n - dense_node);
  for (int i = n - dense_node; i < n; i++) {
    p[v[i]] = v[parent];
  }

  // Nu tipări muchiile chiar în ordinea asta.
  shuffle(v, n);

  printf("%d\n", n);
  for (int i = 0; i < n; i++) {
    int x = v[i], y = p[v[i]];
    if (y != NIL) {
      // Interschimbă aleatoriu fiul și părintele.
      if (rand() % 2) {
        int tmp = x;
        x = y;
        y = tmp;
      }
      printf("%d %d\n", 1 + x, 1 + y);
    }
  }
}

void generate_ops(int n, int num_ops, int max_value) {
  // Valorile inițiale.
  for (int i = 0; i < n; i++) {
```

```
    v[i] = rand() % (max_value + 1);
    printf("%d ", v[i]);
  }
  printf("\n");

  printf("%d\n", num_ops);
  while (num_ops--) {
    int u = rand() % n;
    if (rand() % 2) {
      // actualizare -- asigură-te că nu depășim max_value
      int delta = rand() % (max_value + 1) - v[u];
      v[u] += delta;
      printf("1 %d %d\n", 1 + u, delta);
    } else {
      // interogare
      printf("2 %d\n", 1 + u);
    }
  }
}

void generate_lca_queries(int n, int num_ops) {
  // Generează perechi de noduri distincte.
  printf("%d\n", num_ops);
  while (num_ops--) {
    int u = rand() % n, v;
    do {
      v = rand() % n;
    } while (u == v);
    printf("%d %d\n", 1 + u, 1 + v);
  }
}

int main(int argc, char** argv) {
  if (argc < 5) {
    usage();
  }

  int n = atoi(argv[1]);
  int chain_length = atoi(argv[2]);
  int dense_node = atoi(argv[3]);
  int op_type = atoi(argv[4]);

  assert(chain_length + dense_node <= n);

  struct timeval time;
  gettimeofday(&time, NULL);
  srand(time.tv_usec);

  generate_tree(n, chain_length, dense_node);
```

```
  int num_ops, max_value;
  switch (op_type) {
    case 0:
      // Nimic altceva de făcut.
      break;

    case 1:
      if (argc != 7) {
        usage();
      }
      num_ops = atoi(argv[5]);
      max_value = atoi(argv[6]);
      generate_ops(n, num_ops, max_value);
      break;

    case 2:
      if (argc != 6) {
        usage();
      }
      num_ops = atoi(argv[5]);
      generate_lca_queries(n, num_ops);
      break;

    default:
      assert(false);
  }

  return 0;
}
```

14.3 Problema Subordinates (CSES)

înapoi

Sursă cu vectori STL.

```
#include <iostream>
#include <vector>

const int MAX_N = 200'000;

int s[MAX_N + 1]; // mărimea subarborelui, inclusiv nodul însuși
std::vector<int> c[MAX_N + 1]; // c[u] = fiii lui u
int n;

void read_data() {
  std::cin >> n;
  for (int u = 2; u <= n; u++) {
```

```
    int v;
    std::cin >> v;
    c[v].push_back(u);
  }
}

void depth_first_search(int u) {
  s[u] = 1;
  for (int v: c[u]) {
    depth_first_search(v);
    s[u] += s[v];
  }
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    std::cout << (s[u] - 1) << ' ';
  }
  std::cout << '\n';
}

int main() {
  read_data();
  depth_first_search(1);
  write_answers();

  return 0;
}
```

Sursă cu liste înlănțuite.

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;   // începutul listei de adiacență
  int size;  // mărimea subarborelui, inclusiv nodul însuși
};

cell list[MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_child(int u, int v) {
```

```
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  scanf("%d",  &n);
  for (int u = 2; u <= n; u++) {
    int par;
    scanf("%d", &par);
    add_child(par, u);
  }
}

void depth_first_search(int u) {
  nd[u].size = 1;
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    depth_first_search(v);
    nd[u].size += nd[v].size;
  }
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    printf("%d ", nd[u].size - 1);
  }
  printf("\n");
}

int main() {
  read_data();
  depth_first_search(1);
  write_answers();

  return 0;
}
```

14.4 Problema Tree Matching (CSES)

‹ înapoi

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
```

```
};

struct node {
  int adj;
  bool matched;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int max_match;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int n, u, v;

  scanf("%d",  &n);
  for (int i = 1; i < n; i++) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

void dfs(int u, int parent) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      dfs(v, u);
      if (!nd[u].matched && !nd[v].matched) {
        max_match++;
        nd[u].matched = nd[v].matched = true;
      }
    }
  }
}

void write_answer() {
  printf("%d\n", max_match);
}

int main() {
  read_data();
  dfs(1, 0);
  write_answer();
```

```
  return 0;
}
```

14.5 Problema Tree Diameter (CSES)

înapoi

Sursă cu două parcurgeri DFS.

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct rec {
  int node, dist;

  void improve(rec child) {
    if (child.dist + 1 > dist) {
      dist = child.dist + 1;
      node = child.node;
    }
  }
};

cell list[2 * MAX_NODES];
int adj[MAX_NODES + 1];

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, adj[u] };
  adj[u] = pos++;
}

void read_data() {
  int n, u, v;

  scanf("%d",  &n);
  for (int i = 1; i < n; i++) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}
```

```
rec dfs(int u, int parent) {
  rec result = { u, 0 }; // nodul însuși
  for (int ptr = adj[u]; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      result.improve(dfs(v, u));
    }
  }
  return result;
}

int main() {
  read_data();
  rec farthest = dfs(1, 0);
  rec farthest2 = dfs(farthest.node, 0);
  int diam = farthest2.dist;
  printf("%d\n", diam);

  return 0;
}
```

Sursă cu o singură parcurgere DFS.

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

cell list[2 * MAX_NODES];
int adj[MAX_NODES + 1];
int diam;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, adj[u] };
  adj[u] = pos++;
}

void read_data() {
  int n, u, v;

  scanf("%d",  &n);
  for (int i = 1; i < n; i++) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
```

```
  }
}

void maximize(int& x, int y) {
  x = (x > y) ? x : y;
}

// Returnează lungimea în muchii a căii celei mai lungi de la acest nod la
// orice frunză din subarbore.
int dfs(int u, int parent) {
  int dist = 0; // self
  for (int ptr = adj[u]; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      int l = 1 + dfs(v, u);
      maximize(diam, dist + l); // combină cu fiii anteriori
      maximize(dist, l);        // actualizează calea cea mai lungă
    }
  }
  return dist;
}

int main() {
  read_data();
  dfs(1, 0);
  printf("%d\n", diam);

  return 0;
}
```

14.6 Problema Tree Distances II (CSES)

înapoi

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;            // începutul listei de adiacență
  int size;           // mărimea subarborelui, inclusiv nodul însuși
  long long sum_dist; // suma distanțelor la toate celelalte noduri
};
```

```
cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  scanf("%d",  &n);
  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

void size_dfs(int u, int parent, int depth) {
  nd[1].sum_dist += depth;
  nd[u].size = 1;
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      size_dfs(v, u, depth + 1);
      nd[u].size += nd[v].size;
    }
  }
}

void reroot_dfs(int u, int parent) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      nd[v].sum_dist = nd[u].sum_dist + n - 2 * nd[v].size;
      reroot_dfs(v, u);
    }
  }
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    printf("%lld ", nd[u].sum_dist);
  }
  printf("\n");
}

int main() {
```

```
  read_data();
  size_dfs(1, 0, 0);
  reroot_dfs(1, 0);
  write_answers();

  return 0;
}
```

14.7 Problema White-Black Balanced Subtrees (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 4'000;
const int WHITE = 0;
const int BLACK = 1;

struct node {
  short parent;
  short num_children; // fii care încă nu ne-au dat raportul
  short cnt[2];
  unsigned char color;
};

node nd[MAX_NODES + 1];
int n, answer;

void read_data() {
  scanf("%d", &n);
  for (int i = 2; i <= n; i++) {
    scanf("%hd ", &nd[i].parent);
    nd[nd[i].parent].num_children++;
  }

  for (int i = 1; i <= n; i++) {
    nd[i].color = (getchar() == 'B') ? BLACK : WHITE;
  }
}

// Toți fiii lui u i-au dat raportul. Este momentul ca u să își numere propria
// contribuție, apoi să i-o raporteze părintelui.
void process(node& u) {
  u.cnt[u.color]++; // numără-te pe tine însuți
  answer += (u.cnt[WHITE] == u.cnt[BLACK]);

  nd[u.parent].cnt[WHITE] += u.cnt[WHITE];
  nd[u.parent].cnt[BLACK] += u.cnt[BLACK];
```

```
}

void traverse_tree() {
  for (int u = 1; u <= n; u++) {
    int s = u;
    while (s && !nd[s].num_children) {
      process(nd[s]);
      s = nd[s].parent;
      nd[s].num_children--;
    }
  }
}

void reset() {
  for (int i = 1; i <= n; i++) {
    nd[i] = { 0, 0, 0, 0, 0 };
  }
  answer = 0;
}

void solve_test() {
  read_data();
  traverse_tree();
  printf("%d\n", answer);
  reset();
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

14.8 Problema Blood Cousins (Codeforces)

‹ înapoi • versiune online

```
// Complexitate: O(n + q).
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;

struct cell {
```

```
  int v, next;
};

struct query_cell {
  int v, p, answer, next;
};

struct node {
  int adj;
  int qptr; // pointer în lista de interogări
};

cell list[MAX_NODES + 1];
query_cell q[MAX_QUERIES + 1];
node nd[MAX_NODES + 1];
int stack[MAX_NODES];
int n, num_queries;

void read_input_data() {
  int list_ptr = 1;

  scanf("%d", &n);

  // Pseudonodul 0 va avea toate rădăcinile drept fii.
  for (int u = 1; u <= n; u++) {
    int p;
    scanf("%d", &p);
    list[list_ptr] = { u, nd[p].adj };
    nd[p].adj = list_ptr++;
  }

  scanf("%d", &num_queries);
  for (int i = 1; i <= num_queries; i++) {
    scanf("%d %d", &q[i].v, &q[i].p);
  }
}

void distribute_queries() {
  for (int u = 1; u <= n; u++) {
    nd[u].qptr = 0;
  }
  for (int i = 1; i <= num_queries; i++) {
    if (q[i].v) { // poate fi 0 după primul DFS
      q[i].next = nd[q[i].v].qptr;
      nd[q[i].v].qptr = i;
    }
  }
}

void replace_query_nodes_dfs(int u, int depth) {
```

```
  stack[depth] = u;

  // Scanează toate interogările despre u și înlocuiește nodul cu al p-lea său
  // strămoș.
  for (int ptr = nd[u].qptr; ptr; ptr = q[ptr].next) {
    q[ptr].v = (depth > q[ptr].p)
      ? stack[depth - q[ptr].p]
      : 0;
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    replace_query_nodes_dfs(list[ptr].v, depth + 1);
  }

  stack[depth] = 0; // curățenie pentru al doilea DFS
}

void answer_queries_dfs(int u, int depth) {
  stack[depth]++;

  // Scanează toate interogările despre u și reține contorul inițial.
  for (int ptr = nd[u].qptr; ptr; ptr = q[ptr].next) {
    q[ptr].answer = stack[depth + q[ptr].p];
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    answer_queries_dfs(list[ptr].v, depth + 1);
  }

  // Rescanează interogările și reține diferența între valorile contorului.
  for (int ptr = nd[u].qptr; ptr; ptr = q[ptr].next) {
    q[ptr].answer = stack[depth + q[ptr].p] - q[ptr].answer - 1;
  }
}

void write_output_data() {
  for (int i = 1; i <= num_queries; i++) {
    printf("%d ", q[i].answer);
  }
  printf("\n");
}

int main() {
  read_input_data();
  distribute_queries();
  replace_query_nodes_dfs(0, 0);
  distribute_queries();
  answer_queries_dfs(0, 0);
  write_output_data();
```

```
  return 0;
}
```

Anexa 15

Arbori - liniarizare

15.1 Problema Tree Queries (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 200'000;
const int NIL = -1;

struct list {
  int v, next;
};

struct node {
  int parent;
  int ptr; // pointer în lista de adiacență
  int time_in, time_out;
};

list adj[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n, num_queries;

void add_neighbor(int u, int v, int pos) {
  adj[pos] = { v, nd[u].ptr };
  nd[u].ptr = pos;
}

void read_tree() {
  scanf("%d %d", &n, &num_queries);
  for (int u = 1; u <= n; u++) {
    nd[u].ptr = NIL;
  }
```

```
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v, 2 * i);
    add_neighbor(v, u, 2 * i + 1);
  }
}

void euler_tour(int u, int parent) {
  static int time = 0;

  nd[u].parent = parent;
  nd[u].time_in = ++time;
  for (int ptr = nd[u].ptr; ptr != NIL; ptr = adj[ptr].next) {
    int v = adj[ptr].v;
    if (v != parent) {
      euler_tour(v, u);
    }
  }
  nd[u].time_out = ++time;
}

bool is_ancestor(int u, int v) {
  return
    (nd[u].time_in <= nd[v].time_in) &&
    (nd[u].time_out >= nd[v].time_out);
}

void process_queries() {
  while (num_queries--) {
    int size, u, lowest;
    bool has_path = true;
    scanf("%d %d", &size, &lowest);
    lowest = nd[lowest].parent;

    while (--size) {
      scanf("%d", &u);
      u = nd[u].parent;
      if (is_ancestor(lowest, u)) {
        lowest = u;
      } else if (!is_ancestor(u, lowest)) {
        has_path = false;
      }
    }

    printf(has_path ? "YES\n" : "NO\n");
  }
}

int main() {
```

```
  read_tree();
  euler_tour(1, 1);
  process_queries();

  return 0;
}
```

15.2 Problema Subtree Queries (CSES)

‹ înapoi

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct list {
  int val, next;
};

struct node {
  int val;
  int start, finish;
  int ptr; // pointer în lista de adiacență
};

struct fenwick_tree {
  long long v[MAX_NODES + 1];
  int n;

  void build(int n) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
      int j = i + (i & -i);
      if (j <= n) {
        v[j] += v[i];
      }
    }
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  long long sum(int pos) {
    long long s = 0;
```

```
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  long long range_sum(int x, int y) {
    return sum(y) - sum(x - 1);
  }
};

list adj[2 * MAX_NODES];
node nd[MAX_NODES + 1];
fenwick_tree fen;
int n, num_queries;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  adj[ptr] = { v, nd[u].ptr };
  nd[u].ptr = ptr++;
}

void read_input_data() {
  scanf("%d %d", &n, &num_queries);
  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].val);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void flatten(int u) {
  static int time = 0;

  nd[u].start = ++time;

  for (int ptr = nd[u].ptr; ptr; ptr = adj[ptr].next) {
    int v = adj[ptr].val;
    if (!nd[v].start) {
      flatten(v);
    }
  }

  nd[u].finish = time;
```

```
}

void build_fenwick_tree() {
  for (int u = 1; u <= n; u++) {
    fen.v[nd[u].start] = nd[u].val;
  }

  fen.build(n);
}

void process_queries() {
  while (num_queries--) {
    int type, u;
    scanf("%d %d", &type, &u);
    if (type == 1) {
      int val;
      scanf("%d", &val);
      fen.add(nd[u].start, val - nd[u].val);
      nd[u].val = val;
    } else {
      printf("%lld\n", fen.range_sum(nd[u].start, nd[u].finish));
    }
  }
}

int main() {
  read_input_data();
  flatten(1);
  build_fenwick_tree();
  process_queries();

  return 0;
}
```

15.3 Problema Path Queries (CSES)

înapoi

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct list {
  int val, next;
};

struct node {
  int val;
```

```
  int time_in, time_out;
  int ptr; // pointer în lista de adiacență
};

struct fenwick_tree {
  long long v[2 * MAX_NODES + 1];
  int n;

  void build(int n) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
      int j = i + (i & -i);
      if (j <= n) {
        v[j] += v[i];
      }
    }
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  long long sum(int pos) {
    long long s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }
};

list adj[2 * MAX_NODES];
node nd[MAX_NODES + 1];
fenwick_tree fen;
int n, num_queries;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  adj[ptr] = { v, nd[u].ptr };
  nd[u].ptr = ptr++;
}

void read_input_data() {
  scanf("%d %d", &n, &num_queries);
  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].val);
```

```
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void flatten(int u) {
  static int time = 0;

  nd[u].time_in = ++time;

  for (int ptr = nd[u].ptr; ptr; ptr = adj[ptr].next) {
    int v = adj[ptr].val;
    if (!nd[v].time_in) {
      flatten(v);
    }
  }

  nd[u].time_out = ++time;
}

void build_fenwick_tree() {
  for (int u = 1; u <= n; u++) {
    fen.v[nd[u].time_in] = nd[u].val;
    fen.v[nd[u].time_out] = -nd[u].val;
  }

  fen.build(2 * n);
}

void process_queries() {
  while (num_queries--) {
    int type, u;
    scanf("%d %d", &type, &u);
    if (type == 1) {
      int val;
      scanf("%d", &val);
      int delta = val - nd[u].val;
      fen.add(nd[u].time_in, delta);
      fen.add(nd[u].time_out, -delta);
      nd[u].val = val;
    } else {
      printf("%lld\n", fen.sum(nd[u].time_in));
    }
  }
}
```

```
int main() {
  read_input_data();
  flatten(1);
  build_fenwick_tree();
  process_queries();

  return 0;
}
```

15.4 Problema New Year Tree (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 400'000;
const int MAX_NODES_ROUNDED = 1 << 19;
const int T_PAINT = 1;

typedef unsigned long long u64;

// Un arbore de segmente care admite operațiile:
//
// * range_set(int l, int r, int val)
//   Atribuie valoarea val pe [l, r]. Presupune că 1 ≤ val ≤ 60.
// * range_count_distinct(int l, int r)
//   Returnează numărul de valori distincte din [l, r].
//
// Contract: mask[x] este masca valorilor din subarborele lui x. Dacă dirty[x]
// este nonzero, atunci dirty[x] trebuie scris peste tot în subarborele lui x
// și mask[x] deja respectă dirty[x].
struct segment_tree {
  u64 mask[2 * MAX_NODES_ROUNDED];
  char dirty[2 * MAX_NODES_ROUNDED];
  int n, bits;

  int next_power_of_2(int n) {
    return 1 << (32 - __builtin_clz(n - 1));
  }

  void init(int size) {
    n = next_power_of_2(size);
    bits = __builtin_popcount(n - 1);
  }

  void raw_set(int pos, char val) {
    mask[pos + n] = 1ull << val;
```

```
}

void build() {
  for (int i = n - 1; i; i--) {
    mask[i] = mask[2 * i] | mask[2 * i + 1];
  }
}

void push(int x) {
  if (dirty[x]) {
    dirty[2 * x] = dirty[2 * x + 1] = dirty[x];
    mask[2 * x] = mask[2 * x + 1] = mask[x];
    dirty[x] = 0;
  }
}

void push_path(int x) {
  for (int b = bits; b; b--) {
    push(x >> b);
  }
}

void pull_path(int x) {
  for (x /= 2; x; x /= 2) {
    if (!dirty[x]) {
      mask[x] = mask[2 * x] | mask[2 * x + 1];
    }
  }
}

void range_set(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;

  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      mask[l] = 1ull << val;
      dirty[l++] = val;
    }
    l >>= 1;

    if (!(r & 1)) {
      mask[r] = 1ull << val;
      dirty[r--] = val;
    }
    r >>= 1;
```

```
    }

    pull_path(orig_l);
    pull_path(orig_r);
  }

  int range_count_distinct(int l, int r) {
    u64 result = 0;

    l += n;
    r += n;
    push_path(l);
    push_path(r);

    while (l <= r)  {
      if (l & 1) {
        result |= mask[l++];
      }
      l >>= 1;

      if (!(r & 1)) {
        result |= mask[r--];
      }
      r >>= 1;
    }

    return __builtin_popcountll(result);
  }

};

struct cell {
  int v, next;
};

struct node {
  int adj; // lista de adiacență
  int time_in, time_out;
  unsigned char color;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
segment_tree segtree;
int n, num_ops;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
```

```
}

void read_tree() {
  scanf("%d %d", &n, &num_ops);
  for (int u = 1; u <= n; u++) {
    scanf("%hhd", &nd[u].color);
  }
  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void flatten_tree(int u) {
  static int time = 0;

  nd[u].time_in = time++;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].time_in && (v != 1)) { // nu revizita rădăcina
      flatten_tree(v);
    }
  }

  nd[u].time_out = time - 1;
}

void make_segtree() {
  segtree.init(n);
  for (int u = 1; u <= n; u++) {
    segtree.raw_set(nd[u].time_in, nd[u].color);
  }
  segtree.build();
}

void process_ops() {
  while (num_ops--) {
    int type, u, color;
    scanf("%d %d", &type, &u);
    int l = nd[u].time_in, r = nd[u].time_out;

    if (type == T_PAINT) {
      scanf("%d", &color);
      segtree.range_set(l, r, color);
    } else {
      int colors = segtree.range_count_distinct(l, r);
      printf("%d\n", colors);
```

```
    }
  }
}

int main() {
  read_tree();
  flatten_tree(1);
  make_segtree();
  process_ops();

  return 0;
}
```

15.5 Problema Max Flow (USACO)

înapoi

Sursă cu vectori de diferențe pe arbore.

```
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_PATHS = 100'000;
const int NIL = -1;

struct list {
  int val, next;
};

struct node {
  int parent;
  int ptr; // pointer în lista de adiacență
  int path_ptr; // pointer în lista de căi
  int load;
};

struct path {
  int u, v, lca;
};

list adj[2 * MAX_NODES];
list path_list[2 * MAX_PATHS];
node nd[MAX_NODES + 1];
path p[MAX_PATHS];
int ds_parent[MAX_NODES + 1];
int n, num_paths;

/*******************************************************************************/
```

```
/*                  Implementare de mulţimi disjuncte.                    */
/************************************************************************/

void ds_init() {
  for (int u = 1; u <= n; u++) {
    ds_parent[u] = u;
  }
}

int ds_find(int u) {
  return (ds_parent[u] == u)
    ? u
    : (ds_parent[u] = ds_find(ds_parent[u]));
}

// Întotdeauna unifică v în u. Fără unificare după rang.
void ds_union(int u, int v) {
  ds_parent[ds_find(v)] = ds_find(u);
}

/************************************************************************/
/*      Implementarea algoritmului lui Tarjan pentru LCA offline.       */
/************************************************************************/

void offline_lca_dfs(int u, int parent) {
  nd[u].parent = parent;

  for (int ptr = nd[u].ptr; ptr != NIL; ptr = adj[ptr].next) {
    int v = adj[ptr].val;
    if (v != parent) {
      offline_lca_dfs(v, u);
      ds_union(u, v);
    }
  }

  for (int ptr = nd[u].path_ptr; ptr != NIL; ptr = path_list[ptr].next) {
    int i = path_list[ptr].val;
    int v = (p[i].u == u) ? p[i].v : p[i].u;
    if (nd[v].parent) {
      p[i].lca = ds_find(v);
    }
  }
}

void offline_lca() {
  ds_init();
  offline_lca_dfs(1, 1);
}

/************************************************************************/
```

```
/*                          Codul principal.                          */
/**********************************************************************/

void add_neighbor(int u, int v) {
  static int ptr = 0;
  adj[ptr] = { v, nd[u].ptr };
  nd[u].ptr = ptr++;
}

void add_path(int ind, int u) {
  static int ptr = 0;
  path_list[ptr] = { ind, nd[u].path_ptr };
  nd[u].path_ptr = ptr++;
}

void read_input_data() {
  FILE* f = fopen("maxflow.in", "r");
  fscanf(f, "%d %d", &n, &num_paths);
  for (int u = 1; u <= n; u++) {
    nd[u].ptr = nd[u].path_ptr = NIL;
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(f, "%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }

  for (int i = 0; i < num_paths; i++) {
    fscanf(f, "%d %d", &p[i].u, &p[i].v);
    add_path(i, p[i].u);
    add_path(i, p[i].v);
  }
  fclose(f);
}

void mark_differences() {
  for (int i = 0; i < num_paths; i++) {
    nd[p[i].u].load++;
    nd[p[i].v].load++;
    nd[p[i].lca].load--;
    if (p[i].lca != 1) {
      int lca_parent = nd[p[i].lca].parent;
      nd[lca_parent].load--;
    }
  }
}

void sum_differences(int u) {
```

```
  for (int ptr = nd[u].ptr; ptr != NIL; ptr = adj[ptr].next) {
    int v = adj[ptr].val;
    if (v != nd[u].parent) {
      sum_differences(v);
      nd[u].load += nd[v].load;
    }
  }
}

int get_max_load() {
  int result = 0;
  for (int u = 1; u <= n; u++) {
    if (nd[u].load > result) {
      result = nd[u].load;
    }
  }
  return result;
}

void write_answer(int answer) {
  FILE* f = fopen("maxflow.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}

int main() {
  read_input_data();
  offline_lca();
  mark_differences();
  sum_differences(1);
  int answer = get_max_load();
  write_answer(answer);

  return 0;
}
```

Sursă cu liniarizare + AIB.

```
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_PATHS = 100'000;

struct cell {
  int v, next;
};

// Stochează două liste de noduri în fiecare nod u: una pentru căile care
```

```
// încep în u și alta pentru căile care se termină în u.
struct node {
  int adj;
  int path_begin, path_end; // pointers to list of nodes
  int start, finish;
};

struct fenwick_tree {
  unsigned short v[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int count(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int range_count(int x, int y) {
    return count(y) - count(x - 1);
  }
};

cell list[2 * MAX_NODES + 2 * MAX_PATHS + 1];
node nd[MAX_NODES + 1];
fenwick_tree active_start, active_finish;
int n, num_paths;
FILE* fin;

void add_to_list(int& head, int u) {
  static int ptr = 1;
  list[ptr] = { u, head };
  head = ptr++;
}

void read_tree() {
  fscanf(fin, "%d %d", &n, &num_paths);
```

```
  for (int i = 1; i < n; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_to_list(nd[u].adj, v);
    add_to_list(nd[v].adj, u);
  }
}

void read_paths() {
  for (int i = 0; i < num_paths; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    if (nd[u].start > nd[v].start) {
      int tmp = u; u = v; v = tmp;
    }
    add_to_list(nd[u].path_begin, v);
    add_to_list(nd[v].path_end, u);
  }
}

void compute_ranges(int u) {
  static int time = 0;

  nd[u].start = ++time;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].start) {
      compute_ranges(v);
    }
  }

  nd[u].finish = time;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int process_paths_starting_at(int u) {
  int cnt = 0;

  for (int ptr = nd[u].path_begin; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    active_finish.add(nd[v].start, +1);
    cnt++;
  }

  active_start.add(nd[u].start, +cnt);
```

```
  return cnt;
}

void process_paths_ending_at(int v) {
  int cnt = 0;

  for (int ptr = nd[v].path_end; ptr; ptr = list[ptr].next) {
    int u = list[ptr].v;
    active_start.add(nd[u].start, -1);
    cnt++;
  }

  active_finish.add(nd[v].start, -cnt);
}

int max_load = 0;

void compute_load(int u) {
  // Căi care se termină în subarborele lui u.
  int load = active_finish.range_count(nd[u].start, nd[u].finish);

  // Căi care încep în u.
  load += process_paths_starting_at(u);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (nd[v].start > nd[u].start) {
      compute_load(v);
      // Căi care au început într-un descendent al lui u și se vor termina fie
      // în alt fiu al lui u, fie în afara subarborelui lui u.
      load += active_start.range_count(nd[v].start, nd[v].finish);
    }
  }

  process_paths_ending_at(u);
  max_load = max(max_load, load);
}

void write_answer() {
  FILE* f = fopen("maxflow.out", "w");
  fprintf(f, "%d\n", max_load);
  fclose(f);
}

int main() {
  fin = fopen("maxflow.in", "r");
  read_tree();
  compute_ranges(1);
  read_paths();
  fclose(fin);
```

```
  active_start.init(n);
  active_finish.init(n);

  compute_load(1);
  write_answer();

  return 0;
}
```

15.6 Problema Distinct Colors (CSES)

înapoi

```
#include <stdio.h>
#include <unordered_map>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int color, distinct;
  int adj; // lista de adiacență
};

// Un arbore Fenwick extins cu culori. Când colorăm un bit cu o culoare,
// arborele șterge automat apariția anterioară a culorii.
struct fenwick_tree {
  int v[MAX_NODES + 1];
  std::unordered_map<int, int> last_pos;
  int n, total;

  void init(int n) {
    this->n = n;
    total = 0;
  }

  void add(int pos, int val) {
    total += val;
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }
```

```
  void colorize(int pos, int color) {
    auto last = last_pos.find(color);
    if (last != last_pos.end()) {
      add(last->second, -1);
      last->second = pos;
    } else {
      last_pos[color] = pos;
    }
    add(pos, +1);
  }

  // count[pos..n] = total - count[1..pos-1]
  int suffix_count(int pos) {
    int s = total;
    pos--;
    while (pos) {
      s -= v[pos];
      pos &= pos - 1;
    }
    return s;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
fenwick_tree fen;
int n;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].color);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void dfs(int u) {
```

```
  static int time = 0;
  int entry_time = ++time;

  fen.colorize(time, nd[u].color);
  nd[u].color = 0; // previne revizitarea

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (nd[v].color) {
      dfs(v);
    }
  }

  nd[u].distinct = fen.suffix_count(entry_time);
}

void write_output_data() {
  for (int u = 1; u <= n; u++) {
    printf("%d ", nd[u].distinct);
  }
  printf("\n");
}

int main() {
  read_input_data();
  fen.init(n);
  dfs(1);
  write_output_data();

  return 0;
}
```

15.7 Problema Arb (Lot 2008)

‹ înapoi

Sursă cu liniarizare (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 250'000;
const int MAX_QUERIES = 500'000;

struct cell {
  int val, next;
};

struct node {
```

```
  int adj;
  int val;
  int t_in, t_out;
  int depth;
};

struct query {
  int u;
  int dist;
  int answer;
};

struct fenwick_tree {
  int v[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int range_sum(int x, int y) {
    if (y > n) {
      y = n;
    }
    return prefix_sum(y) - prefix_sum(x - 1);
  }
};

node nd[MAX_NODES + 1];
query q[MAX_QUERIES];
cell list[2 * MAX_NODES + MAX_QUERIES + 1];
int nodes_by_depth[MAX_NODES + 1];
int queries_by_depth[MAX_NODES + 1];
fenwick_tree fen;
```

```
int n, num_queries;
FILE* f;

void add_to_list(int& head, int u) {
  static int ptr = 1;
  list[ptr] = { u, head };
  head = ptr++;
}

void read_tree() {
  fscanf(f, "%d", &n);

  for (int u = 1; u <= n; u++) {
    fscanf(f, "%d", &nd[u].val);
  }

  for (int u = 2; u <= n; u++) {
    int p;
    fscanf(f, "%d", &p);
    add_to_list(nd[p].adj, u);
  }
}

void flatten(int u, int depth) {
  static int time = 0;
  nd[u].t_in = ++time;
  nd[u].depth = depth;
  add_to_list(nodes_by_depth[depth], u);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    flatten(list[ptr].val, depth + 1);
  }

  nd[u].t_out = time;
}

void read_queries() {
  fscanf(f, "%d", &num_queries);

  for (int i = 0; i < num_queries; i++) {
    fscanf(f, "%d %d", &q[i].u, &q[i].dist);
    int d = nd[q[i].u].depth + q[i].dist;
    add_to_list(queries_by_depth[d], i);
  }
}

void bfs() {
  fen.init(n);
  for (int d = 1; d <= n; d++) {
    for (int ptr = nodes_by_depth[d]; ptr; ptr = list[ptr].next) {
```

```
      int u = list[ptr].val;
      fen.add(nd[u].t_in, nd[u].val);
    }
    for (int ptr = queries_by_depth[d]; ptr; ptr = list[ptr].next) {
      int i = list[ptr].val;
      int u = q[i].u;
      q[i].answer = fen.range_sum(nd[u].t_in, nd[u].t_out);
    }
  }
}

void write_answers() {
  FILE *f = fopen("arb.out", "w");
  for (int i = 0; i < num_queries; i++) {
    fprintf(f, "%d\n", q[i].answer);
  }
  fclose(f);
}

int main() {
  f = fopen("arb.in", "r");

  read_tree();
  flatten(1, 1);
  read_queries();

  fclose(f);

  bfs();
  write_answers();

  return 0;
}
```

Sursă fără liniarizare (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 250'000;
const int MAX_QUERIES = 500'000;

struct cell {
  int val, next;
};

struct node {
  int adj, q_head;
  int val;
};
```

```
struct query {
  int dist;
  int answer;
};

struct fenwick_tree {
  int v[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int range_sum(int x, int y) {
    if (y > n) {
      y = n;
    }
    return prefix_sum(y) - prefix_sum(x - 1);
  }
};

cell list[MAX_NODES + MAX_QUERIES + 1];
node nd[MAX_NODES + 1];
query q[MAX_QUERIES];
fenwick_tree fen;
int n, num_queries;

void add_to_list(int& head, int u) {
  static int ptr = 1;
  list[ptr] = { u, head };
  head = ptr++;
}
```

```
void read_data() {
  int p;

  FILE* f = fopen("arb.in", "r");

  fscanf(f, "%d", &n);

  for (int u = 1; u <= n; u++) {
    fscanf(f, "%d", &nd[u].val);
  }

  for (int u = 2; u <= n; u++) {
    fscanf(f, "%d", &p);
    add_to_list(nd[p].adj, u);
  }

  fscanf(f, "%d", &num_queries);

  for (int i = 0; i < num_queries; i++) {
    fscanf(f, "%d %d", &p, &q[i].dist);
    add_to_list(nd[p].q_head, i);
  }

  fclose(f);
}

void dfs(int u, int depth) {

  for (int ptr = nd[u].q_head; ptr; ptr = list[ptr].next) {
    int i = list[ptr].val;
    q[i].answer -= fen.range_sum(depth, depth + q[i].dist);
  }

  fen.add(depth, nd[u].val);
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    dfs(list[ptr].val, depth + 1);
  }

  for (int ptr = nd[u].q_head; ptr; ptr = list[ptr].next) {
    int i = list[ptr].val;
    q[i].answer += fen.range_sum(depth, depth + q[i].dist);
  }
}

void write_answers() {
  FILE *f = fopen("arb.out", "w");
  for (int i = 0; i < num_queries; i++) {
    fprintf(f, "%d\n", q[i].answer);
  }
  fclose(f);
```

```
}

int main() {
  read_data();
  fen.init(n);
  dfs(1, 1);
  write_answers();
}
```

Sursă combinată (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 250'000;
const int MAX_QUERIES = 500'000;

struct cell {
  int val, next;
};

struct node {
  int adj, q_head;
  int subtree_val;
};

struct query {
  int dist;
  int answer;
};

cell list[MAX_NODES + MAX_QUERIES + 1];
node nd[MAX_NODES + 1];
query q[MAX_QUERIES];
int s[MAX_NODES];
int n, num_queries;

void add_to_list(int& head, int u) {
  static int ptr = 1;
  list[ptr] = { u, head };
  head = ptr++;
}

void read_data() {
  int p;

  FILE* f = fopen("arb.in", "r");

  fscanf(f, "%d", &n);
```

```
  for (int u = 1; u <= n; u++) {
    fscanf(f, "%d", &nd[u].subtree_val);
  }

  for (int u = 2; u <= n; u++) {
    fscanf(f, "%d", &p);
    add_to_list(nd[p].adj, u);
  }

  fscanf(f, "%d", &num_queries);

  for (int i = 0; i < num_queries; i++) {
    fscanf(f, "%d %d", &p, &q[i].dist);
    add_to_list(nd[p].q_head, i);
  }

  fclose(f);
}

void subtree_val_dfs(int u) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].val;
    subtree_val_dfs(v);
    nd[u].subtree_val += nd[v].subtree_val;
  }
}

void dfs(int u, int depth) {
  s[depth] += nd[u].subtree_val;

  for (int ptr = nd[u].q_head; ptr; ptr = list[ptr].next) {
    int i = list[ptr].val;
    q[i].answer = nd[u].subtree_val + s[depth + q[i].dist + 1];
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    dfs(list[ptr].val, depth + 1);
  }

  for (int ptr = nd[u].q_head; ptr; ptr = list[ptr].next) {
    int i = list[ptr].val;
    q[i].answer -= s[depth + q[i].dist + 1];
  }
}

void write_answers() {
  FILE *f = fopen("arb.out", "w");
  for (int i = 0; i < num_queries; i++) {
    fprintf(f, "%d\n", q[i].answer);
  }
```

```
  fclose(f);
}

int main() {
  read_data();
  subtree_val_dfs(1);
  dfs(1, 0);
  write_answers();

  return 0;
}
```

15.8 Problema Disconnect (Infoarena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_NODES_ROUNDED = 1 << 17;
const int T_REMOVE = 1;

// Un arbore de intervale cu operațiile:
//
// update(l, r, val): scrie val peste tot în [l, r].
// query(pos): returnează prima valoare găsită călătorind în sus de la pos.
//
// Presupune că actualizările sînt fie disjuncte, fie imbricate. Cu alte
// cuvinte, nu există suprapuneri parțiale. Actualizările mai înguste au
// prioritate în fața celor mai late.
struct segment_tree_node {
  int val;
  int priority; // cu cît mai mică, cu atît mai importantă

  void update(int new_val, int new_priority) {
    if (new_priority < priority) {
      val = new_val;
      priority = new_priority;
    }
  }
};

struct segment_tree {
  segment_tree_node v[2 * MAX_NODES_ROUNDED];
  int n;

  int next_power_of_2(int x) {
    return 1 << (32 - __builtin_clz(x - 1));
```

```
  }

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int i = 1; i < 2 * this->n; i++) {
      v[i].priority = n + 1; // cea mai neimportantă
    }
  }

  int query(int pos) {
    for (pos += n; pos; pos >>= 1) {
      if (v[pos].val) {
        return v[pos].val;
      }
    }
    return 0;
  }

  void update(int l, int r, int val) {
    int priority = r - l + 1;
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        v[l++].update(val, priority);
      }
      l >>= 1;

      if (!(r & 1)) {
        v[r--].update(val, priority);
      }
      r >>= 1;
    }
  }
};

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int start, finish;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
segment_tree segtree;
```

```
int n, num_ops;
FILE *fin, *fout;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  fscanf(fin, "%d %d", &n, &num_ops);
  for (int i = 1; i < n; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void flatten_tree(int u) {
  static int time = 0;

  nd[u].start = time++;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].start && (v != 1)) { // nu revizita rădăcina
      nd[v].parent = u;
      flatten_tree(v);
    }
  }

  nd[u].finish = time - 1;
}

void mark_node(int u) {
  segtree.update(nd[u].start, nd[u].finish, u);
}

bool bad_node(int u) {
  return (u < 1) || (u > n);
}

void remove_edge(int u, int v) {
  if (bad_node(u) || bad_node(v)) {
    return;
  }
  if (nd[u].parent == v) {
    mark_node(u);
  } else if (nd[v].parent == u) {
```

```
    mark_node(v);
  }
}

bool path_exists(int u, int v) {
  if (bad_node(u) || bad_node(v)) {
    return false;
  }

  u = segtree.query(nd[u].start);
  v = segtree.query(nd[v].start);

  return (u == v);
}

void process_ops() {
  int v = 0;

  while (num_ops--) {
    int type, x, y;
    fscanf(fin, "%d %d %d", &type, &x, &y);

    x ^= v;
    y ^= v;

    if (type == T_REMOVE) {
      remove_edge(x, y);
    } else {
      if (path_exists(x, y)) {
        fprintf(fout, "YES\n");
        v = x;
      } else {
        fprintf(fout, "NO\n");
        v = y;
      }
    }
  }
}

int main() {
  fin = fopen("disconnect.in", "r");
  fout = fopen("disconnect.out", "w");

  read_tree();
  flatten_tree(1);
  segtree.init(n);
  process_ops();

  fclose(fin);
  fclose(fout);
```

```
  return 0;
}
```

Anexa 16

Arbori - small-to-large

16.1 Problema Fixed-Length Paths I (CSES)

înapoi

```
#include <deque>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

typedef std::deque<int> deque;

std::vector<int> adj[MAX_NODES + 1];
int n, k;
long long answer;

void read_input_data() {
  scanf("%d %d", &n, &k);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
}

// Presupune că src.size() <= dest.size().
void merge_into(deque& src, deque&dest) {
  int min_i = std::max(k - (int)dest.size() + 1, 0);
  int max_i = std::min((int)src.size() - 1, k);
  for (int i = min_i; i <= max_i; i++) {
    answer += (long long)src[i] * dest[k - i];
  }
```

```
  for (int i = 0; i < (int)src.size(); i++) {
    dest[i] += src[i];
  }
}

deque dfs(int u, int parent) {
  deque result = { 1 }; // nodul însuși
  for (int v: adj[u]) {
    if (v != parent) {
      deque d = dfs(v, u);
      d.push_front(0);
      if (d.size() > result.size()) {
        // Întotdeauna folosește swap(), care schimbă pointeri. Niciodată nu
        // folosi atribuirea, care copiază date.
        result.swap(d);
      }
      merge_into(d, result);
    }
  }

  if ((int)result.size() > k) {
    // Nu are rost să ținem statistici peste distanța k.
    result.pop_back();
  }
  return result;
}

void write_answer() {
  printf("%lld\n", answer);
}

int main() {
  read_input_data();
  dfs(1, 0);
  write_answer();

  return 0;
}
```

16.2 Problema Distinct Colors (CSES) (din nou)

înapoi

```
#include <stdio.h>
#include <unordered_set>

const int MAX_NODES = 200'000;
```

```
typedef std::unordered_set<int> set;

struct cell {
  int v, next;
};

struct node {
  int color, distinct;
  int adj;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_data() {
  scanf("%d", &n);

  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].color);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void merge_into(set& src, set& dest) {
  dest.merge(src);
  src.clear();
}

set dfs(int u) {
  // marchează-l ca vizitat ca să prevenim recursivitatea infinită
  nd[u].distinct = 1;
  set result = { nd[u].color };
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].distinct) {
      set s = dfs(v);
      if (s.size() > result.size()) {
```

```
        s.swap(result);
      }
      merge_into(s, result);
    }
  }

  nd[u].distinct = result.size();
  return result;
}

void write_answer() {
  for (int u = 1; u <= n; u++) {
    printf("%d ", nd[u].distinct);
  }
  printf("\n");
}

int main() {
  read_data();
  dfs(1);
  write_answer();

  return 0;
}
```

16.3 Problema Lomsat Gelral (Codeforces)

înapoi

Sursă cu tehnica *small-to-large* (versiune online).

```
#include <stdio.h>
#include <unordered_map>

const int MAX_NODES = 100'000;

struct freq_info {
  std::unordered_map<int, int> map; // culori => frecvențe
  int max_f;                        // cea mai mare frecvență din map
  long long sum;                    // suma culorilor având max_f

  freq_info(int color) {
    map[color] = 1;
    max_f = 1;
    sum = color;
  }

  void swap(freq_info& other) {
```

```
    map.swap(other.map);
    max_f = other.max_f;
    sum = other.sum;
    // Tehnic, ar trebui să copiem și valorile noastre în other, dar nu este
    // necesar.
  }

  void absorb(freq_info& src) {
    if (src.map.size() > map.size()) {
      swap(src);
    }

    for (auto [color, f]: src.map) {
      // Salvează noua frecvență ca să nu o tot recăutăm în map.
      int new_f = (map[color] += f);
      if (new_f > max_f) {
        max_f = new_f;
        sum = color;
      } else if (new_f == max_f) {
        sum += color;
      }
    }

    src.map.clear();
  }
};

struct cell {
  int val, next;
};

struct node {
  int color;
  int adj;
  long long sum;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_input_data() {
  scanf("%d", &n);
```

```
  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].color);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

freq_info dfs(int u) {
  freq_info result(nd[u].color);
  nd[u].color = 0; // previne recursia infinită

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].val;
    if (nd[v].color) {
      freq_info f = dfs(v);
      result.absorb(f);
    }
  }

  nd[u].sum = result.sum;
  return result;
}

void write_output_data() {
  for (int u = 1; u <= n; u++) {
    printf("%lld ", nd[u].sum);
  }
  printf("\n");
}

int main() {
  read_input_data();
  dfs(1);
  write_output_data();

  return 0;
}
```

Sursă cu algoritmul lui Mo (versiune online).

```
#include <algorithm>
#include <math.h>
#include <stdio.h>
```

```
const int MAX_NODES = 100'000;

struct list {
  int val, next;
};

struct node {
  int color, orig_pos;
  int start, finish;
  int ptr; // lista de adiacență
};

struct accountant {
  int* color;
  int f[MAX_NODES + 1];       // frecvența fiecărei culori
  int c[MAX_NODES + 1];       // numărul de culori având fiecare frecvență
  long long s[MAX_NODES + 1]; // suma culorilor având fiecare frecvență
  int left, right, max_f;

  void init(int* color) {
    this->color = color;
    left = 1; right = 0; // gol
  }

  void change_frequency(int x, int delta) {
    c[f[x]]--;
    s[f[x]] -= x;
    f[x] += delta;
    c[f[x]]++;
    s[f[x]] += x;
  }

  void include(int x) {
    change_frequency(x, +1);
    if (f[x] > max_f) {
      max_f++;
    }
  }

  void exclude(int x) {
    change_frequency(x, -1);
    if (!c[max_f]) {
      max_f--;
    }
  }

  long long query(int l, int r) {
    while (right < r) {
      include(color[++right]);
    }
```

```
    while (right > r) {
      exclude(color[right--]);
    }
    while (left < l) {
      exclude(color[left++]);
    }
    while (left > l) {
      include(color[--left]);
    }
    return s[max_f];
  }
};

list adj[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int color[MAX_NODES + 1];
long long answer[MAX_NODES + 1];
accountant acc;
int n, block_size;

void add_neighbor(int u, int v) {
  static int ptr = 1;
  adj[ptr] = { v, nd[u].ptr };
  nd[u].ptr = ptr++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].color);
    nd[u].orig_pos = u;
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_neighbor(u, v);
    add_neighbor(v, u);
  }
}

void dfs(int u) {
  static int time = 0;

  nd[u].start = ++time;
  color[time] = nd[u].color;

  for (int ptr = nd[u].ptr; ptr; ptr = adj[ptr].next) {
    int v = adj[ptr].val;
```

```
    if (!nd[v].start) {
      dfs(v);
    }
  }

  nd[u].finish = time;
}

void sort_in_mo_order() {
  std::sort(nd + 1, nd + n + 1, [](node& u, node& v) {
    // TODO fewer divisions
    int bu = u.start / block_size;
    int bv = v.start / block_size;
    if (bu != bv) {
      return bu < bv;
    } else if (bu % 2) {
      return u.finish < v.finish;
    } else {
      return u.finish > v.finish;
    }
  });
}

void process_queries() {
  block_size = sqrt(n);
  sort_in_mo_order();
  acc.init(color);
  for (int u = 1; u <= n; u++) {
    answer[nd[u].orig_pos] = acc.query(nd[u].start, nd[u].finish);
  }
}

void write_output_data() {
  for (int u = 1; u <= n; u++) {
    printf("%lld ", answer[u]);
  }
  printf("\n");
}

int main() {
  read_input_data();
  dfs(1);

  // acum fiecare nod este și o interogare [start, finish]
  process_queries();
  write_output_data();

  return 0;
}
```

16.4 Problema Tokens on a Tree (CodeChef)

‹ înapoi

Sursă cu tehnica *small-to-large* și liste proprii (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 1'000'000;

struct cell {
  int f, prev, next;
};

cell list[MAX_NODES + 1];
int list_ptr;
long long total_moves;

void swap(int& x, int& y) {
  int tmp = x;
  x = y;
  y = tmp;
}

struct node {
  int parent, num_children;
  int head, tail, length;
  bool has_coin;

  void reset(bool has_coin) {
    this->has_coin = has_coin;
    num_children = head = tail = length = 0;
  }

  // Procesează un nod după ce toți fiii săi i-au transmis datele lor.
  void process(node& par) {
    prepend_self();
    add_or_move_coin();
    trim();
    spill_into(par);
  }

  void prepend_self() {
    list[list_ptr].f = 0;
    list[list_ptr].prev = 0;
    list[list_ptr].next = head;
    if (head) {
      list[head].prev = list_ptr;
    } else {
      tail = list_ptr;
```

```
    }
    head = list_ptr++;
    length++;
  }

  void add_or_move_coin() {
    if (has_coin) {
      list[head].f = 1;
    } else if (list[tail].f) {
      list[head].f = 1;
      list[tail].f--;
      total_moves += length - 1;
    }
  }

  void trim() {
    while ((tail != head) && !list[tail].f) {
      tail = list[tail].prev;
      list[tail].next = 0;
      length--;
    }
  }

  void spill_into(node& other) {
    if (length > other.length) {
      swap_lists(other);
    }

    for (int p = head, q = other.head;
         p;
         p = list[p].next, q = list[q].next)  {
      list[q].f += list[p].f;
    }
  }

  void swap_lists(node& other) {
    swap(head, other.head);
    swap(tail, other.tail);
    swap(length, other.length);
  }
};

node nd[MAX_NODES + 1];
int n;

void read_input_data() {
  list_ptr = 1;

  scanf("%d ", &n);
```

```
  for (int u = 1; u <= n; u++) {
    int c = getchar();
    nd[u].reset(c == '1');
  }

  for (int u = 2; u <= n; u++) {
    scanf("%d", &nd[u].parent);
    nd[nd[u].parent].num_children++;
  }
}

void traverse() {
  for (int u = 1; u <= n; u++) {
    int p = u;
    while (p && !nd[p].num_children) {
      nd[p].process(nd[nd[p].parent]);
      p = nd[p].parent;
      nd[p].num_children--;
    }
  }
}

void run_test() {
  read_input_data();
  total_moves = 0;
  traverse();
  printf("%lld\n", total_moves);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    run_test();
  }

  return 0;
}
```

Sursă cu tehnica *small-to-large* și liste STL (versiune online).

```
#include <list>
#include <stdio.h>

const int MAX_NODES = 1'000'000;

long long total_moves;

void swap(int& x, int& y) {
```

```
  int tmp = x;
  x = y;
  y = tmp;
}

struct node {
  int parent, num_children;
  std::list<int> f; // numărul de monede la adîncimile 0, 1, ...
  bool has_coin;

  void reset(bool has_coin) {
    this->has_coin = has_coin;
    f.clear();
  }

  // Procesează un nod după ce toți fiii săi i-au transmis datele lor.
  void process(node& par) {
    prepend_self();
    trim();
    spill_into(par);
  }

  void prepend_self() {
    if (has_coin) {
      f.push_front(1);
    } else if (!f.empty() && f.back()) {
      f.push_front(1);
      f.back()--;
      total_moves += f.size() - 1;
    }
    // altfel nu există monete în subarbore și nu adăugăm un element 0
  }

  void trim() {
    while (!f.empty() && !f.back()) {
      f.pop_back();
    }
  }

  void spill_into(node& other) {
    if (f.size() > other.f.size()) {
      f.swap(other.f);
    }

    for (auto it = f.begin(), other_it = other.f.begin();
         it != f.end();
         it++, other_it++) {
      *other_it += *it;
    }
```

```
    f.clear();
  }
};

node nd[MAX_NODES + 1];
int n;

void read_input_data() {
  scanf("%d ", &n);

  for (int u = 1; u <= n; u++) {
    int c = getchar();
    nd[u].reset(c == '1');
  }

  for (int u = 2; u <= n; u++) {
    scanf("%d", &nd[u].parent);
    nd[nd[u].parent].num_children++;
  }
}

void traverse() {
  for (int u = 1; u <= n; u++) {
    int p = u;
    while (p && !nd[p].num_children) {
      nd[p].process(nd[nd[p].parent]);
      p = nd[p].parent;
      nd[p].num_children--;
    }
  }
}

void run_test() {
  read_input_data();
  total_moves = 0;
  traverse();
  printf("%lld\n", total_moves);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    run_test();
  }

  return 0;
}
```

Sursă cu liniarizare și RMQ (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 1'000'000;
const int MAX_SEGTREE_NODES = 1 << 21;

struct cell {
  int v, next;
};

struct node {
  int adj;
  bool has_coin;
};

int max(int x, int y) {
  return (x > y) ? x : y;
}

int next_power_of_2(int n) {
  return 1 << (32 - __builtin_clz(n - 1));
}

struct segment_tree {
  int v[MAX_SEGTREE_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int i = 1; i < 2 * this->n; i++) {
      v[i] = 0;
    }
  }

  void set(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = max(v[2 * pos], v[2 * pos + 1]);
    }
  }

  void optimize(int pos, int& max, int& best_pos) {
    if (v[pos] > max) {
      max = v[pos];
      best_pos = pos;
    }
  }
```

```
  void descend_and_erase(int pos) {
    while (pos < n) {
      pos = (v[pos] == v[2 * pos])
        ? (2 * pos)
        : (2 * pos + 1);
    }
    set(pos - n, 0);
  }

  // Returnează maximul din [l, r] și îl înlocuiește cu 0.
  int erase_rmq(int l, int r) {
    int result = -1, pos = 0;

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        optimize(l++, result, pos);
      }
      l >>= 1;

      if (!(r & 1)) {
        optimize(r--, result, pos);
      }
      r >>= 1;
    }

    // Coboară din pos într-o frunză care are aceeași valoare și șterge
    // valoarea.
    descend_and_erase(pos);

    return result;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
segment_tree segtree;
int n, list_ptr, dfs_time;

void add_child(int u, int v) {
  list[list_ptr] = { v, nd[u].adj };
  nd[u].adj = list_ptr++;
}

void read_input_data() {
  scanf("%d ", &n);
```

```
  for (int u = 1; u <= n; u++) {
    int c = getchar();
    nd[u].has_coin = (c == '1');
    nd[u].adj = 0; // null
  }

  list_ptr = 1;
  for (int u = 2; u <= n; u++) {
    int p;
    scanf("%d", &p);
    add_child(p, u);
  }
}

long long dfs(int u, int depth) {

  long long result = 0;
  int start = dfs_time++;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    result += dfs(list[ptr].v, depth + 1);
  }

  if (nd[u].has_coin) {
    segtree.set(start, depth);
  } else {
    int lowest_coin = segtree.erase_rmq(start, dfs_time - 1);
    if (lowest_coin) {
      segtree.set(start, depth);
      result += lowest_coin - depth;
    }
  }

  return result;
}

void run_test() {
  read_input_data();
  segtree.init(n);
  dfs_time = 0;
  long long moves = dfs(1, 1);
  printf("%lld\n", moves);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    run_test();
  }
```

```
  return 0;
}
```

16.5 Problema Blood Cousins Return (Codeforces)

înapoi

Sursă cu vectori de mulțimi (versiune online).

```
// Complexitate: O(n log n).
//
// Fiecare nod calculează un set de nume distincte la fiecare adîncime în
// subarborele său.
#include <iostream>
#include <unordered_map>
#include <set>
#include <string>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;

struct query {
  int orig_index;
  int depth;
};

struct node {
  std::vector<int> adj;
  std::vector<query> queries;
  int parent;
  int name;
};

struct name_map {
  std::unordered_map<std::string, int> map;

  int insert_and_get_number(std::string s) {
    auto it = map.find(s);
    if (it == map.end()) {
      int result = map.size();
      map[s] = result;
      return result;
    } else {
      return it->second;
    }
  }
```

```
};

// Urmărește elementele distincte la fiecare adîncime. Adîncimea 0 (care
// conține doar nodul însuși) este ultimul element din vector.
struct dist {
  std::vector<std::set<int>> d;

  void absorb(dist other) {
    if (other.d.size() > d.size()) {
      d.swap(other.d);
    }
    int offset = d.size() - other.d.size();
    for (unsigned i = 0; i < other.d.size(); i++) {
      std::set<int>& us = d[i + offset];
      std::set<int>& them = other.d[i];
      if (them.size() > us.size()) {
        us.swap(them);
      }
      us.merge(them);
      them.clear();
    }
  }

  void prepend(int name) {
    d.push_back({name});
  }

  int count_distinct(unsigned depth) {
    if (depth < d.size()) {
      return d[d.size() - 1 - depth].size();
    } else {
      return 0;
    }
  }
};

node nd[MAX_NODES + 1]; // Vom folosi nodul 0 ca pe un strămoș comun fals.
int sol[MAX_QUERIES];
int n, q;

void read_tree() {
  name_map map;

  std::cin >> n;
  for (int u = 1; u <= n; u++) {
    std::string name;
    std::cin >> name >> nd[u].parent;
    nd[nd[u].parent].adj.push_back(u);
    nd[u].name = map.insert_and_get_number(name);
  }
```

```
}

void read_queries() {
  std::cin >> q;
  for (int i = 0; i < q; i++) {
    int u, depth;
    std::cin >> u >> depth;
    nd[u].queries.push_back({i, depth});
  }
}

dist dfs(int u) {
  dist result;
  for (int v: nd[u].adj) {
    result.absorb(dfs(v));
  }
  result.prepend(nd[u].name);

  for (query q: nd[u].queries) {
    sol[q.orig_index] = result.count_distinct(q.depth);
  }
  return result;
}

void write_answers() {
  for (int i = 0; i < q; i++) {
    std::cout << sol[i] << '\n';
  }
}

int main() {
  read_tree();
  read_queries();
  dfs(0);
  write_answers();

  return 0;
}
```

Sursă cu liniarizare (versiune online).

```
// 1. Mapează numele la numere ca de obicei.
// 2. Rulează un DFS pentru a calcula adîncimile și timpii de intrare/ieșire.
// 3. Rescrie fiecare interogare ca <u,d> ca „interoghează nodurile de nivel
//    depth[u] + d descoperite de DFS între timpii [t_i[u], t_o[u]]”.
// 4. Ordonează interogările după adîncime, apoi după t_o.
// 5. Calculează ordinea BFS. Fiecare interogare corespunde unui interval
//    contiguu în această ordonare.
// 6. Răspunde la intrerogări folosind un AIB ca în problema D-query.
```

```cpp
#include <algorithm>
#include <stdio.h>
#include <unordered_map>
#include <string>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;
const int MAX_NAME_LENGTH = 20;

struct node {
  std::vector<int> children;
  int name;
  int tin, tout;
  int depth;
};

struct name_map {
  std::unordered_map<std::string, int> map;

  int insert_and_get_number(char* s) {
    std::string str(s);
    auto it = map.find(str);
    if (it == map.end()) {
      int result = map.size();
      map[str] = result;
      return result;
    } else {
      return it->second;
    }
  }
};

struct query {
  int orig_index;
  int depth;
  int tin, tout;
};

struct fenwick_tree {
  int v[MAX_NODES + 1];
  int last[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int delta) {
    do {
```

```
      v[pos] += delta;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void set(int pos, int name) {
    if (last[name]) {
      add(last[name], -1);
    }
    add(pos, +1);
    last[name] = pos;
  }

  int prefix_sum(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
    }
    return sum;
  }

  int range_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
  }
};

node nd[MAX_NODES + 1];
int bfs[MAX_NODES + 1];
query q[MAX_QUERIES];
int sol[MAX_QUERIES];
fenwick_tree fen;
int n, num_queries;

void read_tree() {
  name_map map;
  char name[MAX_NAME_LENGTH + 1];
  int parent;

  scanf("%d", &n);
  for (int u = 1; u <= n; u++) {
    scanf("%s %d", name, &parent);
    nd[u].name = map.insert_and_get_number(name);
    nd[parent].children.push_back(u);
  }
}

void dfs(int u) {
  static int time = 0;
  nd[u].tin = time++;
```

```
  for (int v: nd[u].children) {
    nd[v].depth = 1 + nd[u].depth;
    dfs(v);
  }

  nd[u].tout = time - 1;
}

void read_queries() {
  scanf("%d", &num_queries);

  for (int i = 0; i < num_queries; i++) {
    int u, depth;
    scanf("%d %d", &u, &depth);
    q[i] = { i, nd[u].depth + depth, nd[u].tin, nd[u].tout };
  }
}

void sort_queries() {
  std::sort(q, q + num_queries, [](query& a, query& b) {
    return (a.depth < b.depth) ||
      ((a.depth == b.depth) && (a.tout < b.tout));
  });
}

void breadth_first_search() {
  int head = 0, tail = 0;
  bfs[tail++] = 0; // pune rădăcina în coadă

  while (head != tail) {
    int u = bfs[head++];
    for (int v: nd[u].children) {
      bfs[tail++] = v;
    }
  }
}

bool query_ends_at(query& q, int k) {
  if (k == n) {
    return true;
  }

  node& u = nd[bfs[k + 1]]; // următorul în BFS
  return (u.depth > q.depth) || ((u.depth == q.depth) && (u.tout > q.tout));
}

// Unde începe această interogare? Caută cel mai din stînga nod din BFS la
// adîncimea q.depth și cu timpul DFS cel puțin q.tin.
int bin_search(query& q, int end) {
```

```
  int l = 0, r = end; // (r, l]
  while (r - l > 1) {
    int mid = (r + l) / 2;
    int v = bfs[mid];
    if ((nd[v].depth < q.depth) ||
        (nd[v].tin < q.tin)) {
      l = mid;
    } else {
      r = mid;
    }
  }

  int v = bfs[r];
  return (nd[v].depth == q.depth) && (nd[v].tin >= q.tin)
    ? r
    : (end + 1);
}

void answer_queries_ending_at(int k) {
  static int i = 0;
  while ((i < num_queries) && query_ends_at(q[i], k)) {
    int start = bin_search(q[i], k);
    sol[q[i].orig_index] = fen.range_sum(start, k);
    i++;
  }
}

void answer_queries() {
  fen.init(n);

  for (int i = 1; i <= n; i++) {
    int v = bfs[i];
    fen.set(i, nd[v].name);
    answer_queries_ending_at(i);
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", sol[i]);
  }
}

int main() {
  read_tree();
  dfs(0);
  breadth_first_search();
  read_queries();
  sort_queries();
  breadth_first_search();
```

```
  answer_queries();
  write_answers();

  return 0;
}
```

16.6 Problema Tree and Queries (Codeforces)

înapoi

Sursă cu PBDS (versiune online).

```
// Small-to-large relativ brut. Fiecare nod stochează informații despre
// subarbore:
//
// 1. Un map de culoare => frecvență.
// 2. Un multiset doar cu frecvențele.
//
// Folosim STL cu larghețe pentru un cod cît mai lent.
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <iostream>
#include <map>
#include <vector>

// Multiseturile PBDS sînt obscene, dar par să meargă. Ștergerile necesită cod
// extra. Vezi https://stackoverflow.com/q/59731946/6022817
typedef __gnu_pbds::tree<
  int,
  __gnu_pbds::null_type,
  std::less_equal<int>,
  __gnu_pbds::rb_tree_tag,
  __gnu_pbds::tree_order_statistics_node_update
  > multiset;

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;

struct query {
  int orig_index;
  int min_freq;
};

struct node {
  int color;
  std::vector<int> adj;
  std::vector<query> q;
};
```

```
struct freq_info {
  std::map<int, int> f;
  multiset sorted_f;

  freq_info(int color) {
    f.insert({color, 1});
    sorted_f.insert(1);
  }

  void update_frequency(int color, int cnt) {
    int new_freq;
    auto it = f.find(color);
    if (it != f.end()) {
      int old_freq = it->second;
      int rank = sorted_f.order_of_key(old_freq);
      multiset::iterator to_erase = sorted_f.find_by_order(rank);
      sorted_f.erase(to_erase);
      it->second += cnt;
      new_freq = it->second;
    } else {
      f.insert({color, cnt});
      new_freq = cnt;
    }
    sorted_f.insert(new_freq);
  }

  void absorb(freq_info& other) {
    if (other.f.size() > f.size()) {
      f.swap(other.f);
      sorted_f.swap(other.sorted_f);
    }
    for (auto [color, cnt]: other.f) {
      update_frequency(color, cnt);
    }
    other.f.clear();
    other.sorted_f.clear();
  }

  int count_freq_gte(int min_freq) {
    return f.size() - sorted_f.order_of_key(min_freq);
  }
};

node nd[MAX_NODES + 1];
int sol[MAX_QUERIES];
int n, q;

void read_data() {
  std::cin >> n >> q;
```

```
  for (int u = 1; u <= n; u++) {
    std::cin >> nd[u].color;
  }
  for (int i = 1; i < n; i++) {
    int u, v;
    std::cin >> u >> v;
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
  for (int i = 0; i < q; i++) {
    int u, min_freq;
    std::cin >> u >> min_freq;
    nd[u].q.push_back({i, min_freq});
  }
}

freq_info dfs(int u) {
  freq_info result(nd[u].color);
  nd[u].color = 0; // previne revizitarea fără a necesita un argument în plus

  for (int v: nd[u].adj) {
    if (nd[v].color) {
      freq_info fi = dfs(v);
      result.absorb(fi);
    }
  }

  for (query q: nd[u].q) {
    sol[q.orig_index] = result.count_freq_gte(q.min_freq);
  }

  return result;
}

void write_solution() {
  for (int i = 0; i < q; i++) {
    std::cout << sol[i] << '\n';
  }
}

int main() {
  read_data();
  dfs(1);
  write_solution();

  return 0;
}
```

Sursă cu DFS exclusiv (versiune online).

```cpp
// Small-to-large cu DFS exclusiv. Rescris după
// https://codeforces.com/contest/375/submission/5508178
//
// Folosește o singură structură de date globală conținînd:
//
// 1. Frecvența fiecărei culori.
// 2. Frecvențele frecvențelor („cîte culori au frecvența f?”).
// 3. Un AIB peste (2) care permite sume pe sufix.
//
// Funcționarea DFS-ului îi garantează fiecărui nod un moment cînd structura
// conține doar informații despre subarborele acelui nod.
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_COLORS = 100'000;
const int MAX_QUERIES = 100'000;

struct fenwick_tree {
  int v[MAX_NODES + 1];
  int total;
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    if (pos) {
      total += val;
      do {
        v[pos] += val;
        pos += pos & -pos;
      } while (pos <= n);
    }
  }

  int sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int suffix_sum(int pos) {
    return (pos > n)
      ? 0
```

```
      : (total - sum(pos - 1));
  }
};

struct query {
  int orig_index;
  int min_freq;
};

struct node {
  int color;
  int heavy;
  std::vector<int> adj;
  std::vector<query> q;
};

node nd[MAX_NODES + 1];
int freq[MAX_COLORS + 1];
fenwick_tree fen;
int sol[MAX_QUERIES];
int n, q;

void read_data() {
  scanf("%d %d", &n, &q);
  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].color);
  }
  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
  for (int i = 0; i < q; i++) {
    int u, min_freq;
    scanf("%d %d", &u, &min_freq);
    nd[u].q.push_back({i, min_freq});
  }
}

// Calculează fiul heavy al fiecărui nod. Șterge părintele nodului din lista
// de adiacență. Returnează mărimea subarborelui.
int size_dfs(int u, int parent) {
  int size = 1, max_c_size = 0;

  unsigned i = 0;
  while (i < nd[u].adj.size()) {
    int v = nd[u].adj[i];
    if (v == parent) {
      nd[u].adj[i] = nd[u].adj.back();
```

```
      nd[u].adj.pop_back();
    } else {
      int c = size_dfs(v, u);
      size += c;
      if (!nd[u].heavy || (c > max_c_size)) {
        nd[u].heavy = v;
        max_c_size = c;
      }
      i++;
    }
  }

  return size;
}

void change_freq(int color, int delta) {
  fen.add(freq[color], -1);
  freq[color] += delta;
  fen.add(freq[color], +1);
}

void toggle_subtree(int u, int delta) {
  change_freq(nd[u].color, delta);
  for (int v: nd[u].adj) {
    toggle_subtree(v, delta);
  }
}

void dfs(int u) {
  for (int v: nd[u].adj) {
    if (v != nd[u].heavy) {
      dfs(v);
      toggle_subtree(v, -1);
    }
  }

  if (nd[u].heavy) {
    dfs(nd[u].heavy);
  }

  for (int v: nd[u].adj) {
    if (v != nd[u].heavy) {
      toggle_subtree(v, +1);
    }
  }

  change_freq(nd[u].color, +1);

  for (query q: nd[u].q) {
    sol[q.orig_index] = fen.suffix_sum(q.min_freq);
```

```
  }
}

void write_solution() {
  for (int i = 0; i < q; i++) {
    printf("%d\n", sol[i]);
  }
}

int main() {
  read_data();
  size_dfs(1, 0);
  fen.init(n);
  dfs(1);
  write_solution();

  return 0;
}
```

16.7 Problema Dominant Indices (Codeforces)

înapoi

Sursă cu *small-to-large* (versiune online).

```
// Abordare small-to-large.
#include <vector>
#include <stdio.h>

const int MAX_NODES = 1'000'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int answer;
};

// Stochează histograma unui subarbore, adică numerele de noduri la fiecare
// adîncime relativă la rădăcina subarborelui. Vectorul h este *răsturnat*:
// h[0] stochează numărul de noduri cele mai depărtate de rădăcină, iar
// h.back() stochează 1 pentru rădăcina însăși.
//
// Histograma stochează și indicele unde h este maximizat. Ca departajare,
// preferăm indicele mai apropiat de rădăcină.
struct histogram {
```

```
  std::vector<int> h;
  int max_pos;

  histogram(): h(), max_pos(0) {
  }

  int size() {
    return h.size();
  }

  void swap(histogram& other) {
    h.swap(other.h);
    int tmp = max_pos;
    max_pos = other.max_pos;
    other.max_pos = tmp;
  }

  void include(histogram& src) {
    int shift = h.size() - src.h.size();
    // Aliniază ultimele elemente ale lui this și src.
    for (unsigned i = 0, j = shift; i < src.h.size(); i++, j++) {
      h[j] += src.h[i];
      optimize(j);
    }
  }

  void optimize(int pos) {
    if ((h[pos] > h[max_pos]) ||
        ((h[pos] == h[max_pos]) && (pos > max_pos))) {
      max_pos = pos;
    }
  }

  void prepend_root() {
    h.push_back(1);
    optimize(h.size() - 1);
  }

  int get_real_max_pos() {
    return h.size() - 1 - max_pos;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
```

```
  nd[u].adj = ptr++;
}

void read_tree() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

histogram dfs(int u, int parent) {
  histogram result;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      histogram h = dfs(v, u);
      if (h.size() > result.size()) {
        result.swap(h);
      }
      result.include(h);
    }
  }

  result.prepend_root();
  nd[u].answer = result.get_real_max_pos();

  return result;
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    printf("%d\n", nd[u].answer);
  }
}

int main() {
  read_tree();
  dfs(1, 0);
  write_answers();

  return 0;
}
```

Sursă cu DFS exclusiv (versiune online).

```cpp
// Abordare cu DFS exclusiv.
#include <stdio.h>
#include <vector>

const int MAX_NODES = 1'000'000;

struct node {
  std::vector<int> adj;
  int depth;
  int heavy;
  int answer;
};

node nd[MAX_NODES + 1];
int pop[MAX_NODES], min_ind;
int n;

void read_tree() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

// Calculează pentru fiecare nod adîncimea și fiul heavy. Șterge părintele din
// lista de adiacență a fiecărui nod pentru a accelera DFS-urile viitoare.
// Returnează mărimea subarborelui.
int size_dfs(int u, int parent) {
  int size = 1, max_c_size = 0;

  unsigned i = 0;
  while (i < nd[u].adj.size()) {
    int v = nd[u].adj[i];
    if (v == parent) {
      nd[u].adj[i] = nd[u].adj.back();
      nd[u].adj.pop_back();
    } else {
      nd[v].depth = 1 + nd[u].depth;
      int c = size_dfs(v, u);
      size += c;
      if (!nd[u].heavy || (c > max_c_size)) {
        nd[u].heavy = v;
        max_c_size = c;
      }
      i++;
```

```
    }
  }

  return size;
}

void insert_node(int u) {
  int d = nd[u].depth;
  pop[d]++;
  if ((pop[d] > pop[min_ind]) ||
      ((pop[d] == pop[min_ind]) && (d < min_ind))) {
    min_ind = d;
  }
}

void insert_subtree(int u) {
  insert_node(u);
  for (int v: nd[u].adj) {
    insert_subtree(v);
  }
}

void delete_subtree(int u) {
  pop[nd[u].depth]--;
  for (int v: nd[u].adj) {
    delete_subtree(v);
  }
}

void dfs(int u) {
  for (int v: nd[u].adj) {
    if (v != nd[u].heavy) {
      int save_min_ind = min_ind;
      dfs(v);
      delete_subtree(v);
      min_ind = save_min_ind;
    }
  }

  if (nd[u].heavy) {
    dfs(nd[u].heavy);
  }

  for (int v: nd[u].adj) {
    if (v != nd[u].heavy) {
      insert_subtree(v);
    }
  }

  insert_node(u);
```

```
  nd[u].answer = min_ind - nd[u].depth;
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    printf("%d\n", nd[u].answer);
  }
}

int main() {
  read_tree();
  size_dfs(1, 0);
  dfs(1);
  write_answers();

  return 0;
}
```

Anexa 17

Arbori - cel mai apropiat strămoș comun

17.1 LCA cu descompunere în radical

înapoi

```
#include <math.h>
#include <stdio.h>

const int MAX_NODES = 500'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
  // Fiecare nod stochează doi pointeri la strămoși: unul la părinte, altul cu
  // sqrt(n) niveluri mai sus.
  int parent;
  int jump;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int st[MAX_NODES + 1];    // stivă DFS pentru construcția pointerilor
int n, jump_distance;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
```

```
}

void read_input_data() {
  scanf("%d", &n);
  jump_distance = sqrt(n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează părinții, adîncimile și jump pointers.
void dfs(int u) {
  st[nd[u].depth] = u;  // Stiva reține nodurile gri.
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = (nd[v].depth > jump_distance)
        ? st[nd[v].depth - jump_distance]
        : 0;
      dfs(v);
    }
  }
}

int sqrt_lca(int u, int v) {
  if (nd[v].depth > nd[u].depth) {
    int tmp = u; u = v; v = tmp;
  }

  // Întîi adu-le la același nivel.
  while (nd[u].depth - jump_distance >= nd[v].depth) {
    u = nd[u].jump;
  }
  while (nd[u].depth > nd[v].depth) {
    u = nd[u].parent;
  }

  // Urcă blocuri întregi.
  while ((nd[u].depth >= jump_distance) && (nd[u].jump != nd[v].jump)) {
    u = nd[u].jump;
    v = nd[v].jump;
  }

  // Urcă nivel cu nivel.
```

```
  while (u != v) {
    u = nd[u].parent;
    v = nd[v].parent;
  }
  return u;
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", sqrt_lca(u, v));
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

17.2 LCA cu binary lifting ($\log n$ pointeri per nod)

‹ înapoi

Implementare cu urcare în paralel.

```
#include <stdio.h>

const int MAX_NODES = 500'000;
const int MAX_LOG = 19;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
  int jump[MAX_LOG];
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
```

```
int n, log_2;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);
  log_2 = 31 - __builtin_clz(n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează adîncimile și jump pointers.
void dfs(int u, int parent) {
  nd[u].depth = 1 + nd[parent].depth;
  nd[u].jump[0] = parent;
  for (int i = 0; i < log_2; i++) {
    nd[u].jump[i + 1] = nd[nd[u].jump[i]].jump[i];
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      dfs(v, u);
    }
  }
}

int lca(int u, int v) {
  if (nd[v].depth > nd[u].depth) {
    int tmp = u; u = v; v = tmp;
  }

  // Adu u și v la aceeași adîncime. Compară d[u] și d[v] bit cu bit.
  int pow = 0;
  while (nd[u].depth > nd[v].depth) {
    if ((nd[u].depth & (1 << pow)) != (nd[v].depth & (1 << pow))) {
      u = nd[u].jump[pow];
    }
    pow++;
  }
```

```
  if (u == v) {
    return u;
  } else {
    for (int i = log_2; i >= 0; i--) {
      if (nd[u].jump[i] != nd[v].jump[i]) {
        u = nd[u].jump[i];
        v = nd[v].jump[i];
      }
    }

    return nd[u].jump[0];
  }
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", lca(u, v));
  }
}

int main() {
  read_input_data();
  dfs(1, 0);
  answer_queries();

  return 0;
}
```

Implementare cu test de strămoș.

```
#include <stdio.h>

const int MAX_NODES = 500'000;
const int MAX_LOG = 19;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int time_in, time_out;
  int jump[MAX_LOG];
};

cell list[2 * MAX_NODES];
```

```
node nd[MAX_NODES + 1];
int n, log_2;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);
  log_2 = 31 - __builtin_clz(n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează timpii de intrare în noduri și jump
// pointers.
void dfs(int u, int parent) {
  static int time = 0;
  nd[u].time_in = time++;

  nd[u].jump[0] = parent;
  for (int i = 0; i < log_2; i++) {
    nd[u].jump[i + 1] = nd[nd[u].jump[i]].jump[i];
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      dfs(v, u);
    }
  }

  nd[u].time_out = time++;
}

bool is_ancestor(int u, int v) {
  return
    (nd[u].time_in <= nd[v].time_in) &&
    (nd[u].time_out >= nd[v].time_out);
}

int lca(int u, int v) {
  if (is_ancestor(u, v)) {
```

```
    return u;
  }

  // Găsește cel mai de sus strămoș al lui u care *nu* este strămoș al lui v.
  for (int i = log_2; i >= 0; i--) {
    if (nd[u].jump[i] && !is_ancestor(nd[u].jump[i], v)) {
      u = nd[u].jump[i];
    }
  }

  // Acum părintele lui u este strămoș al lui v.
  return nd[u].jump[0];
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", lca(u, v));
  }
}

int main() {
  read_input_data();
  dfs(1, 0);
  answer_queries();

  return 0;
}
```

17.3 LCA cu binary lifting (2 pointeri per nod)

‹ înapoi

Implementare cu urcare în paralel.

```
#include <stdio.h>

const int MAX_NODES = 500'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
```

```
  int parent;
  int jump;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează părinții, adîncimile și jump pointers.
void dfs(int u) {

  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = equal ? u3 : u;
      dfs(v);
    }
  }
}

int two_ptr_lca(int u, int v) {
  if (nd[v].depth > nd[u].depth) {
    int tmp = u; u = v; v = tmp;
  }

  // Întîi adu-le la același nivel.
  while (nd[u].depth > nd[v].depth) {
    u = (nd[nd[u].jump].depth >= nd[v].depth) ? nd[u].jump : nd[u].parent;
```

```
  }

  while (u != v) {
    if (nd[u].jump != nd[v].jump) {
      u = nd[u].jump;
      v = nd[v].jump;
    } else {
      u = nd[u].parent;
      v = nd[v].parent;
    }
  }
  return u;
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", two_ptr_lca(u, v));
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

Implementare cu test de strămoș.

```
#include <stdio.h>

const int MAX_NODES = 500'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
  int time_in, time_out;
  int parent;
  int jump;
};
```

```
cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează părinții, adîncimile și jump pointers.
void dfs(int u) {
  static int time = 1;
  nd[u].time_in = time++;

  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = equal ? u3 : u;
      dfs(v);
    }
  }

  nd[u].time_out = time++;
}

bool is_ancestor(int u, int v) {
  return
    (nd[u].time_in <= nd[v].time_in) &&
    (nd[u].time_out >= nd[v].time_out);
}

int two_ptr_lca(int u, int v) {
```

```
  // Găsește cel mai jos strămoș al lui u care este și strămoș al lui v.
  while (!is_ancestor(u, v)) {
    if (nd[u].jump && !is_ancestor(nd[u].jump, v)) {
      u = nd[u].jump;
    } else {
      u = nd[u].parent;
    }
  }

  return u;
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", two_ptr_lca(u, v));
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

Implementare cu formula LSB și urcare în paralel.

```
#include <stdio.h>

const int MAX_NODES = 500'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
  // Fiecare nod stochează doi pointeri la strămoși: unul la părinte, altul cu
  // LSB(adîncime) niveluri mai sus, similar unui arbore Fenwick.
  int parent;
  int jump;
};
```

```
cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int st[MAX_NODES + 1];    // stivă DFS pentru construcția pointerilor
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează părinții, adâncimile și jump pointers.
void dfs(int u) {
  st[nd[u].depth] = u;  // Stiva reține nodurile gri.

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = st[nd[v].depth & (nd[v].depth - 1)];
      dfs(v);
    }
  }
}

int two_ptr_lca(int u, int v) {
  if (nd[v].depth > nd[u].depth) {
    int tmp = u; u = v; v = tmp;
  }

  // Întîi adu-le la același nivel.
  while (nd[u].depth > nd[v].depth) {
    u = (nd[nd[u].jump].depth >= nd[v].depth) ? nd[u].jump : nd[u].parent;
  }

  while (u != v) {
    if (nd[u].jump != nd[v].jump) {
      u = nd[u].jump;
```

```
      v = nd[v].jump;
    } else {
      u = nd[u].parent;
      v = nd[v].parent;
    }
  }
  return u;
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", two_ptr_lca(u, v));
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

17.4 LCA cu algoritmul lui Tarjan (offline)

‹ înapoi

Implementarea folosește STL, ca să ne putem concentra pe algoritm. Vă reamintesc însă că arborii scriși cu STL sînt de două ori mai lenți și consumă dublul memoriei.

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 500'000;
const int MAX_QUERIES = 500'000;

struct disjoint_set_forest {
  int p[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }
```

```
  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  // Întotdeauna îl leagă pe v de u. Fără *union by rank*.
  void unite(int u, int v) {
    p[find(v)] = find(u);
  }
};

struct node {
  std::vector<int> adj, queries;
  bool visited;
};

struct query {
  int u, v, lca;
};

node nd[MAX_NODES + 1];
query q[MAX_QUERIES];
disjoint_set_forest  dsf;
int n, num_queries;

void read_input_data() {
  scanf("%d", &n);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].u, &q[i].v);
    nd[q[i].u].queries.push_back(i);
    nd[q[i].v].queries.push_back(i);
  }
}

void offline_lca(int u) {
  nd[u].visited = true;

  for (int v: nd[u].adj) {
    if (!nd[v].visited) {
      offline_lca(v);
```

```
      dsf.unite(u, v);
    }
  }

  for (int i: nd[u].queries) {
    int v = (q[i].u == u) ? q[i].v : q[i].u;
    if (nd[v].visited) {
      q[i].lca = dsf.find(v);
    }
  }
}

void answer_queries() {
  for (int i = 0; i < num_queries; i++) {
    printf("%d\n", q[i].lca);
  }
}

int main() {
  read_input_data();
  dsf.init(n);
  offline_lca(1);
  answer_queries();

  return 0;
}
```

17.5 Problema Gold Transfer (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 300'001;
const int OP_ADD_NODE = 1;
const int OP_BUY_GOLD = 2;

struct node {
  int depth;
  int supply;
  int cost;
  int parent;
  int jump;
};

struct purchase {
  int amount;
  long long cost;
```

```
};

node nd[MAX_NODES];
int stack[MAX_NODES];
int num_queries;

int min(int x, int y) {
  return (x < y) ? x : y;
}

void create_node_zero() {
  scanf("%d %d %d", &num_queries, &nd[0].supply, &nd[0].cost);
  nd[0].depth = 1;
}

void read_node(int u) {
  scanf("%d %d %d", &nd[u].parent, &nd[u].supply, &nd[u].cost);
  int p = nd[u].parent, p2 = nd[p].jump, p3 = nd[p2].jump;
  bool equal = (nd[p2].depth - nd[p].depth == nd[p3].depth - nd[p2].depth);
  nd[u].depth = 1 + nd[p].depth;
  nd[u].jump = equal ? p3 : p;
}

void buy_from_node(int u, int& demand, purchase& p) {
  int bought = min(demand, nd[u].supply);
  demand -= bought;
  nd[u].supply -= bought;
  p.amount += bought;
  p.cost += (long long)bought * nd[u].cost;
}

purchase buy_from_path(int path_end, int demand) {
  purchase result = { 0, 0 };
  int ptr = 1;
  stack[0] = path_end;

  // Binary lifting după caz. Păstrează rezultatele pe stivă pentru a obține
  // coborîrea în O(1) amortizat.
  while (ptr && demand) {
    int u = stack[ptr - 1];
    if (u && nd[nd[u].jump].supply) {
      stack[ptr++] = nd[u].jump;
    } else if (u && nd[nd[u].parent].supply) {
      stack[ptr++] = nd[u].parent;
    } else {
      buy_from_node(u, demand, result);
      if (!nd[u].supply) {
        ptr--;
      }
    }
```

```
  }

  return result;
}

void buy_gold() {
  int u, demand;
  scanf("%d %d", &u, &demand);
  purchase p = buy_from_path(u, demand);
  printf("%d %lld\n", p.amount, p.cost);
  fflush(stdout);
}

void process_queries() {
  for (int q = 1; q <= num_queries; q++) {
    int type;
    scanf("%d", &type);
    if (type == OP_ADD_NODE) {
      read_node(q);
    } else {
      buy_gold();
    }
  }
}

int main() {
  create_node_zero();
  process_queries();

  return 0;
}
```

17.6 Problema A and B and Lecture Rooms (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth;
  int time_in, time_out;
```

```
  int parent;
  int jump;

  int subtree_size() {
    return time_out - time_in + 1;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Traversează arborele și calculează părinții, adîncimile și *jump pointers*.
void dfs(int u) {
  static int time = 1;
  nd[u].time_in = time++;

  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);
  int dest = equal ? u3 : u;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = dest;
      dfs(v);
    }
  }

  nd[u].time_out = time - 1;
}
```

```
bool is_ancestor(int u, int v) {
  return
    (nd[u].time_in <= nd[v].time_in) &&
    (nd[u].time_out >= nd[v].time_out);
}

int two_ptr_lca(int u, int v) {
  // Găsește cel mai jos strămoș al lui u care este și strămoș al lui v.
  while (!is_ancestor(u, v)) {
    if (nd[u].jump && !is_ancestor(nd[u].jump, v)) {
      u = nd[u].jump;
    } else {
      u = nd[u].parent;
    }
  }

  return u;
}

int get_ancestor_at_depth(int u, int d) {
  while (nd[u].depth > d) {
    u = (nd[nd[u].jump].depth >= d) ? nd[u].jump : nd[u].parent;
  }
  return u;
}

int query(int u, int v) {
  if ((nd[u].depth + nd[v].depth) % 2) {
    return 0;
  }

  if (u == v) {
    return n;
  }

  int l = two_ptr_lca(u, v);
  if (nd[u].depth == nd[v].depth) {
    int u2 = get_ancestor_at_depth(u, nd[l].depth + 1);
    int v2 = get_ancestor_at_depth(v, nd[l].depth + 1);
    return n - nd[u2].subtree_size() - nd[v2].subtree_size();
  }

  if (nd[u].depth < nd[v].depth) {
    int tmp = u; u = v; v = tmp;
  }

  int mid_level = nd[l].depth + (nd[u].depth - nd[v].depth) / 2;
  int u2 = get_ancestor_at_depth(u, mid_level + 1);
  int mid = nd[u2].parent;
```

```
  return nd[mid].subtree_size() - nd[u2].subtree_size();
}

void answer_queries() {
  int num_queries, u, v;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &u, &v);
    printf("%d\n", query(u, v));
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

17.7 Problema Company (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_SEGTREE_NODES = 256 * 1024;
const int INF = 1'000'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int jump;
  int depth;
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
```

```
}

struct segment_tree_node {
  int min1, min2, max1, max2;

  void set(int val) {
    min1 = max1 = val;
    min2 = INF;
    max2 = -INF;
  }

  void combine(segment_tree_node& other) {
    if (min1 < other.min1) {
      // păstrăm min1
      min2 = min(min2, other.min1);
    } else {
      min2 = min(min1, other.min2);
      min1 = other.min1;
    }
    if (max1 > other.max1) {
      // păstrăm max1
      max2 = max(max2, other.max1);
    } else {
      max2 = max(max1, other.max2);
      max1 = other.max1;
    }
  }
};

// Un arbore de intervale care stochează primul și al doilea minim și
// maxim. După construcția inițială, admite interogări pe interval.
struct segment_tree {
  segment_tree_node v[MAX_SEGTREE_NODES];
  int n;

  int next_power_of_2(int x) {
    return 1 << (32 - __builtin_clz(x - 1));
  }

  void init(int n) {
    this->n = next_power_of_2(n);
  }

  void set(int pos, int val) {
    v[n + pos].set(val);
  }

  void build() {
    for (int i = n - 1; i; i--) {
      v[i] = v[2 * i];
```

```
      v[i].combine(v[2 * i + 1]);
    }
  }

  segment_tree_node query(int l, int r) {
    segment_tree_node result = { +INF, +INF, -INF, -INF };

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        result.combine(v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result.combine(v[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

cell list[MAX_NODES];
node nd[MAX_NODES + 1];
int dfs_order[MAX_NODES];
segment_tree st;
int n, num_queries;

void add_child(int p, int c) {
  static int ptr = 1;
  list[ptr] = { c, nd[p].adj };
  nd[p].adj = ptr++;
}

void read_tree() {
  scanf("%d %d", &n, &num_queries);

  for (int u = 2; u <= n; u++) {
    scanf("%d", &nd[u].parent);
    add_child(nd[u].parent, u);
  }
}

void dfs(int u) {
  static int time = 0;
  dfs_order[time] = u;
```

```
  st.set(u, time++);

  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    nd[v].depth = 1 + nd[u].depth;
    nd[v].parent = u;
    nd[v].jump = equal ? u3 : u;
    dfs(v);
  }
}

int lca(int u, int v) {
  if (nd[v].depth > nd[u].depth) {
    int tmp = u; u = v; v = tmp;
  }

  // Întîi adu-le la același nivel.
  while (nd[u].depth > nd[v].depth) {
    u = (nd[nd[u].jump].depth >= nd[v].depth) ? nd[u].jump : nd[u].parent;
  }

  while (u != v) {
    if (nd[u].jump != nd[v].jump) {
      u = nd[u].jump;
      v = nd[v].jump;
    } else {
      u = nd[u].parent;
      v = nd[v].parent;
    }
  }
  return u;
}

void query(int l, int r, int& kick, int& depth) {
  segment_tree_node m = st.query(l, r);
  int u = dfs_order[m.min1];
  int v = dfs_order[m.min2];
  int w = dfs_order[m.max2];
  int x = dfs_order[m.max1];

  kick = x;
  depth = nd[lca(u, w)].depth;

  int cand = nd[lca(v, x)].depth;
  if (cand > depth) {
    kick = u;
    depth = cand;
```

```
  }
}

void process_queries() {
  while (num_queries--) {
    int l, r, kick, depth;
    scanf("%d %d", &l, &r);
    query(l, r, kick, depth);
    printf("%d %d\n", kick, depth);
  }
}

int main() {
  read_tree();
  st.init(n + 1);
  dfs(1);
  st.build();
  process_queries();

  return 0;
}
```

17.8 Problema Duff in the Army (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_LENGTH = 10;
const int SENTINEL = 100'001; // populația maximă + 1

int min(int x, int y) {
  return (x < y) ? x : y;
}

int rbuf[2 * MAX_LENGTH]; // pentru combinarea a două liste

// Un *roster* este o listă ordonată de ID-uri de persoane.
struct roster {
  int n;
  int v[MAX_LENGTH + 1];

  void clear() {
    n = 0;
  }

  void add(int id) {
```

```
    int k = n;
    while (k && (id < v[k - 1])) {
      v[k] = v[k - 1];
      k--;
    }
    v[k] = id;
    if (n < MAX_LENGTH) {
      n++;
    }
  }

  void copy_from(roster& other) {
    n = other.n;
    for (int i = 0; i <= n; i++) {
      v[i] = other.v[i];
    }
  }

  void merge_from(roster& other, int limit) {
    v[n] = other.v[other.n] = SENTINEL;
    n = min(n + other.n, limit);
    int i = 0, j = 0;

    for (int k = 0; k < n; k++) {
      if (v[i] < other.v[j]) {
        rbuf[k] = v[i++];
      } else {
        rbuf[k] = other.v[j++];
      }
    }

    for (int i = 0; i < n; i++) {
      v[i] = rbuf[i];
    }
  }

  void print() {
    printf("%d", n);
    for (int i = 0; i < n; i++) {
      printf(" %d", v[i]);
    }
    printf("\n");
  }
};

struct cell {
  int v, next;
};

struct node {
```

```
  int adj;
  int depth;
  int time_in, time_out;
  int parent;
  int jump;

  // populația proprie
  roster ids;

  // populația până la destinația saltului, exclusiv
  roster jids;

  void compute_jump();
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int num_queries, num_people;

void node::compute_jump() {
  jids.copy_from(ids);

  int u = parent, u2 = nd[u].jump, u3 = nd[u2].jump;
  if (u3 && (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth)) {
    jump = u3;
    jids.merge_from(nd[u].jids, MAX_LENGTH);
    jids.merge_from(nd[u2].jids, MAX_LENGTH);
  } else {
    jump = u;
  }
}

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  int n, u, v;
  scanf("%d %d %d", &n, &num_people, &num_queries);

  for (int i = 0; i < n - 1; i++) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }

  for (int id = 1; id <= num_people; id++) {
    scanf("%d", &u);
```

```
    nd[u].ids.add(id);
  }
}

// Traversează arborele și calculează părinții, adîncimile, *jump pointers* și
// listele subîntinse de toți pointerii.
void dfs(int u) {
  static int time = 1;
  nd[u].time_in = time++;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].depth) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].compute_jump();
      dfs(v);
    }
  }

  nd[u].time_out = time++;
}

bool is_ancestor(int u, int v) {
  return
    (nd[u].time_in <= nd[v].time_in) &&
    (nd[u].time_out >= nd[v].time_out);
}

int climb_until_ancestor(int u, int v, int limit, roster& dest) {
  while (!is_ancestor(u, v)) {
    if (nd[u].jump && !is_ancestor(nd[u].jump, v)) {
      dest.merge_from(nd[u].jids, limit);
      u = nd[u].jump;
    } else {
      dest.merge_from(nd[u].ids, limit);
      u = nd[u].parent;
    }
  }

  return u;
}

// Urcă cu u și v pînă la LCA și colectează listele de pe drumuri.
void two_ptr_lca(int u, int v, int limit, roster& dest) {
  dest.clear();
  u = climb_until_ancestor(u, v, limit, dest);
  v = climb_until_ancestor(v, u, limit, dest);
  dest.merge_from(nd[u].ids, limit);
}
```

```
void answer_queries() {
  int u, v, limit;
  roster r;
  while (num_queries--) {
    scanf("%d %d %d", &u, &v, &limit);
    two_ptr_lca(u, v, limit, r);
    r.print();
  }
}

int main() {
  read_input_data();
  nd[1].depth = 1;
  dfs(1);
  answer_queries();

  return 0;
}
```

Anexa 18

Arbori - algoritmul lui Mo pe arbore

18.1 Problema Dating (Codeforces)

înapoi

Sursă cu LCA implementat cu doi pointeri (versiune online).

```
#include <algorithm>
#include <stdio.h>
#include <unordered_map>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;
const int BLOCK_SIZE = 775; // de ordinul a sqrt(2n)

struct normalizer {
  std::unordered_map<int, int> map;

  int normalize(int x) {
    auto it = map.find(x);
    if (it == map.end()) {
      int result = 1 + map.size();
      map[x] = result;
      return result;
    } else {
      return it->second;
    }
  }
};

struct node {
  std::vector<int> adj; // listă de adiacență
  int tin, tout;        // timpi de intrare și ieșire din DFS
  int parent, jump;     // pointeri pentru LCA
```

```
  int depth;
  int fav;
  bool gender;
};

struct query {
  int l, r;
  int lca;
  int orig_index;
};

node nd[MAX_NODES + 1];
int euler[2 * MAX_NODES + 1];
query q[MAX_QUERIES];
unsigned answer[MAX_QUERIES];
int n, num_queries;

void read_tree() {
  scanf("%d ", &n);
  for (int u = 1; u <= n; u++) {
    nd[u].gender = (getchar() == '1');
    getchar();
  }

  normalizer norm;
  for (int u = 1; u <= n; u++) {
    int x;
    scanf("%d", &x);
    nd[u].fav = norm.normalize(x);
  }

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

void dfs(int u) {
  static int time = 0;
  nd[u].tin = ++time;
  euler[time] = u;

  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);

  for (auto v: nd[u].adj) {
    if (!nd[v].tin) {
      nd[v].depth = 1 + nd[u].depth;
```

```
      nd[v].parent = u;
      nd[v].jump = equal ? u3 : u;
      dfs(v);
    }
  }

  nd[u].tout = ++time;
  euler[time] = u;
}

bool is_ancestor(int u, int v) {
  return
    (nd[u].tin <= nd[v].tin) &&
    (nd[u].tout >= nd[v].tout);
}

int lca(int u, int v) {
  // Găsește cel mai jos strămoș al lui u care este și strămoș al lui v.
  while (!is_ancestor(u, v)) {
    if (nd[u].jump && !is_ancestor(nd[u].jump, v)) {
      u = nd[u].jump;
    } else {
      u = nd[u].parent;
    }
  }

  return u;
}

void read_queries() {
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    int l = lca(u, v);
    if (nd[u].tin > nd[v].tin) {
      std::swap(u, v);
    }
    if (l == u) {
      q[i] = { nd[u].tin, nd[v].tin, 0, i };
    } else {
      q[i] = { nd[u].tout, nd[v].tin, l, i };
    }
  }
}

void sort_queries_in_mo_order() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
```

```
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
      return a.r < b.r;
    }
  });
}

struct mo_tracker {
  bool on[MAX_NODES + 1];  // nodurile care apar exact o dată în interval
  int f[MAX_NODES + 1][2]; // frecvența numerelor favorite pentru fiecare gen
  int l, r;
  unsigned num_pairs;

  void init() {
    l = 1;
    r = 0;
  }

  void toggle(int pos) {
    int u = euler[pos];
    on[u] = !on[u];
    int sign = on[u] ? +1 : -1;
    f[nd[u].fav][nd[u].gender] += sign;
    num_pairs += sign * f[nd[u].fav][!nd[u].gender];
  }

  unsigned query(int target_l, int target_r, int extra_fav, bool extra_gender) {
    while (l > target_l) {
      toggle(--l);
    }
    while (r < target_r) {
      toggle(++r);
    }
    while (l < target_l) {
      toggle(l++);
    }
    while (r > target_r) {
      toggle(r--);
    }

    if (extra_fav) {
      return num_pairs + f[extra_fav][!extra_gender];
    } else {
      return num_pairs;
    }
  }
};
```

```
mo_tracker tracker;

void answer_queries() {
  tracker.init();
  for (int i = 0; i < num_queries; i++) {
    int extra_fav = nd[q[i].lca].fav;
    int extra_gender = nd[q[i].lca].gender;
    unsigned result = tracker.query(q[i].l, q[i].r, extra_fav, extra_gender);
    answer[q[i].orig_index] = result;
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%u\n", answer[i]);
  }
}

int main() {
  read_tree();
  nd[1].depth = 1;
  dfs(1);
  read_queries();
  sort_queries_in_mo_order();
  answer_queries();
  write_answers();

  return 0;
}
```

Sursă cu LCA implementat cu algoritmul lui Tarjan (versiune online).

```
#include <algorithm>
#include <stdio.h>
#include <unordered_map>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;
const int BLOCK_SIZE = 775; // de ordinul a sqrt(2n)

struct disjoint_set_forest {
  int p[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }
```

```
  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  // Întotdeauna îl alipește pe v la u. Fără *union by rank*.
  void unite(int u, int v) {
    p[find(v)] = find(u);
  }
};

struct normalizer {
  std::unordered_map<int, int> map;

  int normalize(int x) {
    auto it = map.find(x);
    if (it == map.end()) {
      int result = 1 + map.size();
      map[x] = result;
      return result;
    } else {
      return it->second;
    }
  }
};

struct cell {
  int val, next;
};

struct node {
  int adj;       // listă de adiacență
  int queries;   // listă de interogări pentru acest nod
  int tin, tout; // timpi de intrare și ieșire din DFS
  int fav;
  bool gender;
};

struct query {
  // Redenumește și refolosește câmpurile pentru a economisi memorie.
  union { int u; int l; };
  union { int v; int r; };
  int lca;
  int orig_index;
};

cell list[2 * MAX_NODES];
cell qlist[2 * MAX_QUERIES + 1];
node nd[MAX_NODES + 1];
```

```
int euler[2 * MAX_NODES + 1];
disjoint_set_forest dsf;
query q[MAX_QUERIES];
unsigned answer[MAX_QUERIES];
int n, num_queries;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void add_query(int ind, int u) {
  static int pos = 1;
  qlist[pos] = { ind, nd[u].queries };
  nd[u].queries = pos++;
}

void read_tree() {
  scanf("%d ", &n);
  for (int u = 1; u <= n; u++) {
    nd[u].gender = (getchar() == '1');
    getchar();
  }

  normalizer norm;
  for (int u = 1; u <= n; u++) {
    int x;
    scanf("%d", &x);
    nd[u].fav = norm.normalize(x);
  }

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

void read_queries() {
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    scanf("%d %d", &q[i].u, &q[i].v);
    add_query(i, q[i].u);
    add_query(i, q[i].v);
    q[i].orig_index = i;
  }
}
```

```
void process_lca_queries(int u) {
  for (int ptr = nd[u].queries; ptr; ptr = qlist[ptr].next) {
    int i = qlist[ptr].val;
    int v = (q[i].u == u) ? q[i].v : q[i].u;
    if (nd[v].tin) {
      q[i].lca = dsf.find(v);
    }
  }
}

void dfs(int u) {
  static int time = 0;
  nd[u].tin = ++time;
  euler[time] = u;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].val;
    if (!nd[v].tin) {
      dfs(v);
      dsf.unite(u, v);
    }
  }

  process_lca_queries(u);

  nd[u].tout = ++time;
  euler[time] = u;
}

// Acum că știm LCA-urile și timpii Euler, calculează intervalele.
void rewrite_queries() {
  for (int i = 0; i < num_queries; i++) {
    if (nd[q[i].u].tin > nd[q[i].v].tin) {
      int tmp = q[i].u;
      q[i].u = q[i].v;
      q[i].v = tmp;
    }
    if (q[i].lca == q[i].u) {
      q[i].l = nd[q[i].u].tin;
      q[i].r = nd[q[i].v].tin;
      q[i].lca = 0;
    } else {
      q[i].l = nd[q[i].u].tout;
      q[i].r = nd[q[i].v].tin;
    }
  }
}

void sort_queries_in_mo_order() {
  std::sort(q, q + num_queries, [](query a, query b) {
```

```
    int x = a.l / BLOCK_SIZE, y = b.l / BLOCK_SIZE;
    if (x != y) {
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
      return a.r < b.r;
    }
  });
}

struct mo_tracker {
  bool on[MAX_NODES + 1];  // nodurile care apar exact o dată în interval
  int f[MAX_NODES + 1][2]; // frecvența numerelor favorite pentru fiecare gen
  int l, r;
  unsigned num_pairs;

  void init() {
    l = 1;
    r = 0;
  }

  void toggle(int pos) {
    int u = euler[pos];
    on[u] = !on[u];
    int sign = on[u] ? +1 : -1;
    f[nd[u].fav][nd[u].gender] += sign;
    num_pairs += sign * f[nd[u].fav][!nd[u].gender];
  }

  unsigned query(int target_l, int target_r, int extra_fav, bool extra_gender) {
    while (l > target_l) {
      toggle(--l);
    }
    while (r < target_r) {
      toggle(++r);
    }
    while (l < target_l) {
      toggle(l++);
    }
    while (r > target_r) {
      toggle(r--);
    }

    if (extra_fav) {
      return num_pairs + f[extra_fav][!extra_gender];
    } else {
      return num_pairs;
    }
  }
```

```
};

mo_tracker tracker;

void answer_queries() {
  tracker.init();
  for (int i = 0; i < num_queries; i++) {
    int extra_fav = nd[q[i].lca].fav;
    int extra_gender = nd[q[i].lca].gender;
    unsigned result = tracker.query(q[i].l, q[i].r, extra_fav, extra_gender);
    answer[q[i].orig_index] = result;
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%u\n", answer[i]);
  }
}

int main() {
  read_tree();
  read_queries();
  dsf.init(n);
  dfs(1);
  rewrite_queries();
  sort_queries_in_mo_order();
  answer_queries();
  write_answers();

  return 0;
}
```

18.2 Problema Notdivisible (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_QUERIES = 100'000;

// determinate experimental
const int BLOCK_SIZE_MO = 1'300;
const int BLOCK_SIZE_MEX = 250;
```

```
// Al 100.001-lea număr prim este 1.299.721
// https://t5k.org/lists/small/100000.txt
const int MAX_VAL = 1'300'000;
const int NONE = -1;

struct prime_list {
  bool is_composite[MAX_VAL + 1];
  int p[MAX_NODES + 1];
  int n;

  void build() {
    for (int d = 2; d * d <= MAX_VAL; d++) {
      if (!is_composite[d]) {
        for (int i = d * d; i <= MAX_VAL; i += d) {
          is_composite[i] = 1;
        }
      }
    }

    for (int i = 2; n <= MAX_NODES; i++) {
      if (!is_composite[i]) {
        p[n++] = i;
      }
    }
  }

  int find(int val) {
    int pos = std::lower_bound(p, p + n, val) - p;
    return ((pos < n) && (p[pos] == val))
      ? pos
      : NONE;
  }

  int get(int k) {
    return p[k];
  }
};

prime_list primes;

struct query {
  int l, r;
  int extra_val;
  int orig_index;
};

struct node {
  std::vector<int> adj;
  int tin, tout;
  int parent, jump; // pointerii pentru LCA
```

```
  int depth;
  int val;
};

struct tree {
  node nd[MAX_NODES + 1];
  int euler[2 * MAX_NODES + 1];

  void read() {
    int n, u, v, val;

    scanf("%d", &n);
    for (int u = 1; u <= n; u++) {
      scanf("%d", &val);
      nd[u].val = primes.find(val);
    }

    for (int i = 1; i < n; i++) {
      scanf("%d %d", &u, &v);
      nd[u].adj.push_back(v);
      nd[v].adj.push_back(u);
    }
  }

  void flatten(int u) {
    static int time = 0;
    nd[u].tin = ++time;
    euler[time] = u;

    int u2 = nd[u].jump, u3 = nd[u2].jump;
    bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);

    for (auto v: nd[u].adj) {
      if (!nd[v].tin) {
        nd[v].depth = 1 + nd[u].depth;
        nd[v].parent = u;
        nd[v].jump = equal ? u3 : u;
        flatten(v);
      }
    }

    nd[u].tout = ++time;
    euler[time] = u;
  }

  bool is_ancestor(int u, int v) {
    return
      (nd[u].tin <= nd[v].tin) &&
      (nd[u].tout >= nd[v].tout);
  }
```

```
  int lca(int u, int v) {
    // Găsește cel mai jos strămoș al lui u care este și strămoș al lui v.
    while (!is_ancestor(u, v)) {
      if (nd[u].jump && !is_ancestor(nd[u].jump, v)) {
        u = nd[u].jump;
      } else {
        u = nd[u].parent;
      }
    }

    return u;
  }

  query make_query(int& u, int& v) {
    if (nd[u].tin > nd[v].tin) {
      std::swap(u, v);
    }
    int l = lca(u, v);
    if (l == u) {
      return { nd[u].tin, nd[v].tin, NONE };
    } else {
      return { nd[u].tout, nd[v].tin, nd[l].val };
    }
  }
};

tree t;
query q[MAX_QUERIES];
int answer[MAX_QUERIES];
int num_queries;

void read_queries() {
  scanf("%d", &num_queries);
  for (int i = 0; i < num_queries; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    q[i] = t.make_query(u, v);
    q[i].orig_index = i;
  }
}

void sort_queries_in_mo_order() {
  std::sort(q, q + num_queries, [](query a, query b) {
    int x = a.l / BLOCK_SIZE_MO, y = b.l / BLOCK_SIZE_MO;
    if (x != y) {
      return (x < y);
    } else if (x % 2) {
      return a.r > b.r;
    } else {
```

```
      return a.r < b.r;
    }
  });
}

struct mex_tracker {
  int f[MAX_NODES + 1];
  int bl_nonzero[(MAX_NODES + 1) / BLOCK_SIZE_MEX];

  void add(int x) {
    if (x != NONE) {
      if (++f[x] == 1) {
        bl_nonzero[x / BLOCK_SIZE_MEX]++;
      }
    }
  }

  void remove(int x) {
    if (x != NONE) {
      if (--f[x] == 0) {
        bl_nonzero[x / BLOCK_SIZE_MEX]--;
      }
    }
  }

  int get_mex() {
    int bl = 0;
    while (bl_nonzero[bl] == BLOCK_SIZE_MEX) {
      bl++;
    }

    int x = bl * BLOCK_SIZE_MEX;
    while (f[x]) {
      x++;
    }
    return x;
  }
};

struct mo_tracker {
  bool on[MAX_NODES + 1];  // nodurile care apar exact o dată
  mex_tracker mex;
  int l, r;

  void init() {
    l = 1;
    r = 0;
  }

  // smelly code: nu ar trebui să aibă acces la datele interne ale lui t
```

```
  void toggle(int pos) {
    int u = t.euler[pos];
    on[u] = !on[u];

    if (on[u]) {
      mex.add(t.nd[u].val);
    } else {
      mex.remove(t.nd[u].val);
    }
  }

  unsigned query(int target_l, int target_r, int extra_val) {
    while (l > target_l) {
      toggle(--l);
    }
    while (r < target_r) {
      toggle(++r);
    }
    while (l < target_l) {
      toggle(l++);
    }
    while (r > target_r) {
      toggle(r--);
    }

    mex.add(extra_val);
    int result = mex.get_mex();
    mex.remove(extra_val);
    return result;
  }
};

mo_tracker tracker;

void answer_queries() {
  tracker.init();
  for (int i = 0; i < num_queries; i++) {
    int mex = tracker.query(q[i].l, q[i].r, q[i].extra_val);
    answer[q[i].orig_index] = primes.get(mex);
  }
}

void write_answers() {
  for (int i = 0; i < num_queries; i++) {
    printf("%u\n", answer[i]);
  }
}

int main() {
  primes.build();
```

```
  t.read();
  t.flatten(1);
  read_queries();
  sort_queries_in_mo_order();
  answer_queries();
  write_answers();

  return 0;
}
```

Anexa 19

Arbori - descompunere *heavy-light*

19.1 Problema Heavy Path Decomposition (Infoarena)

înapoi

Versiunea 1.

```
#include <stdio.h>

const int MAX_NODES = 1 << 17;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int val;
  int depth;
  int parent;
  int heavy;  // fiul cu subarborele maxim
  int head;   // vîrful lanțului nostru
  int tin;    // momentul descoperirii în DFS
};

node nd[MAX_NODES + 1];

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

int max(int x, int y) {
  return (x > y) ? x : y;
}
```

```
struct segment_tree {
  int v[2 * MAX_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int u = 1; u <= n; u++) {
      v[nd[u].tin + this->n] = nd[u].val;
    }
    for (int pos = this->n - 1; pos; pos--) {
      v[pos] = max(v[2 * pos], v[2 * pos + 1]);
    }
  }

  void update(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = max(v[2 * pos], v[2 * pos + 1]);
    }
  }

  int rmq(int l, int r) {
    int result = -1;

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        result = max(result, v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = max(result, v[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

cell list[2 * MAX_NODES];
segment_tree segtree;
int n, num_queries;
FILE *fin, *fout;

void add_edge(int u, int v) {
```

```
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  fscanf(fin, "%d %d", &n, &num_queries);
  for (int u = 1; u <= n; u++) {
    fscanf(fin, "%d", &nd[u].val);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Returnează mărimea subarborelui lui u.
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      nd[v].depth = 1 + nd[u].depth;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].tin = time++;

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
```

```
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // începe un lanț nou
    }
  }
}

int query(int u, int v) {
  int result = 0;
  while (nd[u].head != nd[v].head) {
    // Interogare de prefix pe lanțul de jos.
    if (nd[nd[v].head].depth > nd[nd[u].head].depth) {
      int tmp = u; u = v; v = tmp;
    }
    result = max(result, segtree.rmq(nd[nd[u].head].tin, nd[u].tin));
    // Saltul la părintele capului de lanț ne plasează pe un lanț nou.
    u = nd[nd[u].head].parent;
  }

  // Ultima interogare are loc pe lanțul comun.
  if (nd[u].depth > nd[v].depth) {
    int tmp = u; u = v; v = tmp;
  }
  result = max(result, segtree.rmq(nd[u].tin, nd[v].tin));

  return result;
}

void process_queries() {
  while (num_queries--) {
    int t, x, y;
    fscanf(fin, "%d %d %d", &t, &x, &y);
    if (t) {
      fprintf(fout, "%d\n", query(x, y));
    } else {
      segtree.update(nd[x].tin, y);
    }
  }
}

int main() {
  fin = fopen("heavypath.in", "r");
  fout = fopen("heavypath.out", "w");

  read_data();
  heavy_dfs(1);
  decompose_dfs(1, 1);
```

```
  segtree.init(n);
  process_queries();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

Versiunea 2.

```
#include <stdio.h>

const int MAX_NODES = 1 << 17;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int val;
  int parent;
  int heavy;  // fiul cu subarborele maxim
  int head;   // vîrful lanțului nostru
  int tin;    // momentul descoperirii în DFS
};

node nd[MAX_NODES + 1];

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct segment_tree {
  int v[2 * MAX_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int u = 1; u <= n; u++) {
      v[nd[u].tin + this->n] = nd[u].val;
    }
    for (int pos = this->n - 1; pos; pos--) {
      v[pos] = max(v[2 * pos], v[2 * pos + 1]);
```

```
    }
  }

  void update(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos /= 2; pos; pos /= 2) {
      v[pos] = max(v[2 * pos], v[2 * pos + 1]);
    }
  }

  int rmq(int l, int r) {
    int result = -1;

    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        result = max(result, v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = max(result, v[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

cell list[2 * MAX_NODES];
segment_tree segtree;
int n, num_queries;
FILE *fin, *fout;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  fscanf(fin, "%d %d", &n, &num_queries);
  for (int u = 1; u <= n; u++) {
    fscanf(fin, "%d", &nd[u].val);
  }
```

```
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Returnează mărimea subarborelui lui u.
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].tin = time++;

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // începe un lanț nou
    }
  }
}

int query(int u, int v) {
  int result = 0;
```

```
  while (nd[u].head != nd[v].head) {
    // Interogare de prefix pe lanțul de jos.
    if (nd[v].tin > nd[u].tin) {
      int tmp = u; u = v; v = tmp;
    }
    result = max(result, segtree.rmq(nd[nd[u].head].tin, nd[u].tin));
    // Saltul la părintele capului de lanț ne plasează pe un lanț nou.
    u = nd[nd[u].head].parent;
  }

  // Ultima interogare are loc pe lanțul comun.
  if (nd[u].tin > nd[v].tin) {
    int tmp = u; u = v; v = tmp;
  }
  result = max(result, segtree.rmq(nd[u].tin, nd[v].tin));

  return result;
}

void process_queries() {
  while (num_queries--) {
    int t, x, y;
    fscanf(fin, "%d %d %d", &t, &x, &y);
    if (t) {
      fprintf(fout, "%d\n", query(x, y));
    } else {
      segtree.update(nd[x].tin, y);
    }
  }
}

int main() {
  fin = fopen("heavypath.in", "r");
  fout = fopen("heavypath.out", "w");

  read_data();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  segtree.init(n);
  process_queries();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

19.2 Problema Disruption (USACO)

‹ înapoi

Soluție cu descompunere *heavy-light*.

```
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_SEGTREE_NODES = 1 << 17;
const int INFINITY = 2'000'000'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;
  int head;
  int pos;
};

struct edge {
  int u, v;
};

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

// Un arbore de intervale care suportă operațiile:
//
// 1. set(l, r, m): atribuie v[i] = min(v[i], m) pe toate pozițiile l ≤ i ≤ r.
// 2. get(i): returnează v[i].
//
// În plus, toate operațiile get() vin după toate operațiile set().
struct segment_tree {
  int v[MAX_SEGTREE_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int i = 1; i < 2 * this->n; i++) {
      v[i] = INFINITY;
```

```
    }
  }

  void set(int l, int r, int val) {
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        v[l] = min(v[l], val);
        l++;
      }
      l >>= 1;

      if (!(r & 1)) {
        v[r] = min(v[r], val);
        r--;
      }
      r >>= 1;
    }
  }

  void push_all() {
    for (int i = 1; i < n; i++) {
      v[2 * i] = min(v[2 * i], v[i]);
      v[2 * i + 1] = min(v[2 * i + 1], v[i]);
    }
  }

  int get(int pos) {
    return v[pos + n];
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
edge e[MAX_NODES];
segment_tree segtree;
FILE* fin;
int n, num_extra_paths;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  fscanf(fin, "%d %d", &n, &num_extra_paths);
  for (int i = 0; i < n - 1; i++) {
```

```
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    e[i] = { u, v };
    add_edge(u, v);
    add_edge(v, u);
  }
}

int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].pos = time++;

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v);
    }
  }
}

void set_cost_on_path(int u, int v, int cost) {
  while (nd[u].head != nd[v].head) {
    if (nd[v].pos > nd[u].pos) {
```

```
      int tmp = u; u = v; v = tmp;
    }
    segtree.set(nd[nd[u].head].pos, nd[u].pos, cost);
    u = nd[nd[u].head].parent;
  }

  if (nd[u].pos > nd[v].pos) {
    int tmp = u; u = v; v = tmp;
  }

  // Nu scrie nimic pe nodul LCA însuși.
  segtree.set(nd[u].pos + 1, nd[v].pos, cost);
}

void read_extra_paths() {
  while (num_extra_paths--) {
    int u, v, cost;
    fscanf(fin, "%d %d %d\n", &u, &v, &cost);
    set_cost_on_path(u, v, cost);
  }
}

void compute_answers() {
  FILE* fout = fopen("disrupt.out", "w");

  segtree.push_all();
  for (int i = 0; i < n - 1; i++) {
    int child = (nd[e[i].u].parent == e[i].v) ? e[i].u : e[i].v;
    int cost = segtree.get(nd[child].pos);
    cost = (cost == INFINITY) ? -1 : cost;
    fprintf(fout, "%d\n", cost);
  }

  fclose(fout);
}

int main() {
  fin = fopen("disrupt.in", "r");
  read_tree();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  segtree.init(n);
  read_extra_paths();
  compute_answers();
  fclose(fin);

  return 0;
}
```

Soluție cu tehnica *small-to-large*.

```cpp
#include <algorithm>
#include <set>
#include <stdio.h>
#include <vector>

typedef unsigned short u16;
typedef std::set<u16> set;

const int MAX_NODES = 50'000;
const int MAX_REPL_PATHS = 50'000;

struct node {
  std::vector<u16> adj;
  std::vector<u16> repl_ids;
  u16 parent;
  int min_cost;
};

struct edge {
  u16 u, v;
};

struct repl_path {
  u16 u, v;
  int cost;
};

node nd[MAX_NODES + 1];
edge e[MAX_NODES];
repl_path repl[MAX_REPL_PATHS];
int n, num_repl_paths;

void read_data() {
  FILE* f = fopen("disrupt.in", "r");

  fscanf(f, "%d %d", &n, &num_repl_paths);
  for (int i = 0; i < n - 1; i++) {
    u16 u, v;
    fscanf(f, "%hd %hd", &u, &v);
    e[i] = { u, v };
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  for (int i = 0; i < num_repl_paths; i++) {
    fscanf(f, "%hd %hd %d\n", &repl[i].u, &repl[i].v, &repl[i].cost);
  }
```

```
  fclose(f);
}

void sort_repl_paths() {
  std::sort(repl, repl + num_repl_paths, [](repl_path& a, repl_path& b) {
    return a.cost < b.cost;
  });
}

void distribute_repl_paths() {
  for (int i = 0; i < num_repl_paths; i++) {
    nd[repl[i].u].repl_ids.push_back(i);
    nd[repl[i].v].repl_ids.push_back(i);
  }
}

void merge_sets(set& parent, set& child) {
  if (child.size() > parent.size()) {
    parent.swap(child);
  }
  for (u16 id: child) {
    std::pair<set::iterator, bool> p = parent.insert(id);
    if (!p.second) {
      // Inserarea a eșuat deoarece ID-ul este deja în set; șterge-l.
      parent.erase(p.first);
    }
  }
  child.clear();
}

set dfs(int u) {
  set s;
  for (u16 id: nd[u].repl_ids) {
    s.insert(id);
  }

  for (u16 v: nd[u].adj) {
    if (v != nd[u].parent) {
      nd[v].parent = u;
      set s2 = dfs(v);
      merge_sets(s, s2);
    }
  }

  if (s.empty()) {
    nd[u].min_cost = -1;
  } else {
    u16 first = *s.begin();
    nd[u].min_cost = repl[first].cost;
```

```
  }

  return s;
}

void compute_answers() {
  FILE* fout = fopen("disrupt.out", "w");

  for (int i = 0; i < n - 1; i++) {
    int child = (nd[e[i].u].parent == e[i].v) ? e[i].u : e[i].v;
    int cost = nd[child].min_cost;
    fprintf(fout, "%d\n", cost);
  }

  fclose(fout);
}

int main() {
  read_data();
  sort_repl_paths();
  distribute_repl_paths();
  dfs(1);
  compute_answers();

  return 0;
}
```

Soluție cu tehnica *small-to-large* cu DFS exclusiv.

```
#include <algorithm>
#include <stdio.h>
#include <vector>

typedef unsigned short u16;

const int MAX_NODES = 50'000;
const int MAX_REPL_PATHS = 50'000;

struct node {
  std::vector<u16> adj;
  std::vector<u16> repl_ids;
  u16 parent;
  u16 heavy;
  u16 min_id;
};

struct edge {
  u16 u, v;
};
```

```
struct repl_path {
  u16 u, v;
  int cost;
};

// Un arbore Fenwick de booleeni. Admite operația „cel mai mic bit setat”.
struct fenwick_tree {
  u16 v[MAX_NODES + 1];
  bool raw[MAX_NODES + 1];
  int n;
  int max_p2;

  void init(int n) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));
  }

  void add(int pos, int delta) {
    do {
      v[pos] += delta;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void toggle(int pos) {
    add(pos, raw[pos] ? -1 : +1);
    raw[pos] = !raw[pos];
  }

  // Returnează poziția primului bit setat sau n + 1 dacă arborele este gol.
  int get_first_set_bit() {
    int pos = 0;

    for (int interval = max_p2; interval; interval >>= 1) {
      if ((pos + interval <= n) && (v[pos + interval] == 0)) {
        pos += interval;
      }
    }

    return pos + 1;
  }
};

node nd[MAX_NODES + 1];
edge e[MAX_NODES];
repl_path repl[MAX_REPL_PATHS + 1];
fenwick_tree fen;
int n, num_repl;
```

```
void read_data() {
  FILE* f = fopen("disrupt.in", "r");

  fscanf(f, "%d %d", &n, &num_repl);
  for (int i = 0; i < n - 1; i++) {
    u16 u, v;
    fscanf(f, "%hd %hd", &u, &v);
    e[i] = { u, v };
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  for (int i = 1; i <= num_repl; i++) {
    fscanf(f, "%hd %hd %d\n", &repl[i].u, &repl[i].v, &repl[i].cost);
  }

  fclose(f);
}

void sort_repl_paths() {
  std::sort(repl + 1, repl + num_repl + 1, [](repl_path& a, repl_path& b) {
    return a.cost < b.cost;
  });
}

void distribute_repl_paths() {
  for (int i = 1; i <= num_repl; i++) {
    nd[repl[i].u].repl_ids.push_back(i);
    nd[repl[i].v].repl_ids.push_back(i);
  }
}

int size_dfs(int u) {
  int size = 1, max_c_size = 0;

  for (u16 v: nd[u].adj) {
    if (v != nd[u].parent) {
      nd[v].parent = u;
      int c = size_dfs(v);
      size += c;
      if (!nd[u].heavy || (c > max_c_size)) {
        nd[u].heavy = v;
        max_c_size = c;
      }
    }
  }

  return size;
}
```

```
void toggle_node(int u) {
  for (u16 id: nd[u].repl_ids) {
    fen.toggle(id);
  }
}

void toggle_subtree(int u) {
  toggle_node(u);
  for (int v: nd[u].adj) {
    if (v != nd[u].parent) {
      toggle_subtree(v);
    }
  }
}

void dfs(int u) {
  for (u16 v: nd[u].adj) {
    if ((v != nd[u].parent) && (v != nd[u].heavy)) {
      dfs(v);
      toggle_subtree(v);
    }
  }

  if (nd[u].heavy) {
    dfs(nd[u].heavy);
  }

  for (u16 v: nd[u].adj) {
    if ((v != nd[u].parent) && (v != nd[u].heavy)) {
      toggle_subtree(v);
    }
  }

  toggle_node(u);
  nd[u].min_id = fen.get_first_set_bit();
}

void compute_answers() {
  FILE* fout = fopen("disrupt.out", "w");

  for (int i = 0; i < n - 1; i++) {
    int child = (nd[e[i].u].parent == e[i].v) ? e[i].u : e[i].v;
    int id = nd[child].min_id;
    int cost = (id <= num_repl) ? repl[id].cost : -1;
    fprintf(fout, "%d\n", cost);
  }

  fclose(fout);
}
```

```
int main() {
  read_data();
  sort_repl_paths();
  distribute_repl_paths();
  fen.init(num_repl);
  size_dfs(1);
  dfs(1);
  compute_answers();

  return 0;
}
```

19.3 Problema Subway Lines (Codeforces)

❮ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int OP_UPDATE = 1;
const int OP_QUERY = 2;

struct node {
  std::vector<int> adj;
  int parent;
  int depth;
  int heavy;  // fiul heavy
  int head;   // începutul lanțului
  int lo, hi; // intervalul de înălțimi, relevant doar pentru capetele de lanț
  int op_id;  // ID-ul operației care a setat lo și hi
};

node nd[MAX_NODES + 1];
int n, num_queries;
int op_id; // contor global

void read_tree() {
  scanf("%d %d", &n, &num_queries);
  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}
```

```
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int v: nd[u].adj) {
    if (v != nd[u].parent) {
      nd[v].depth = nd[u].depth + 1;
      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  nd[u].head = head;

  for (int v: nd[u].adj) {
    if (v == nd[u].heavy) {
      decompose_dfs(v, head);
    } else if (v != nd[u].parent) {
      decompose_dfs(v, v); // la v începe un lanț nou
    }
  }
}

int chain_op(int h, int u, int v, int op) {
  if (op == OP_UPDATE) {
    nd[h].lo = nd[u].depth;
    nd[h].hi = nd[v].depth;
    nd[h].op_id = op_id;
  } else if ((nd[h].op_id == op_id) &&    // calea [a,b] a trecut pe aici
             (nd[u].depth <= nd[h].hi) && // intervalele se intersectează
             (nd[v].depth >= nd[h].lo)) {
    return
      std::min(nd[v].depth, nd[h].hi) -
      std::max(nd[u].depth, nd[h].lo) + 1;
  }

  return 0;
}

int visit_path(int u, int v, int op) {
  int result = 0;
```

```
  while (nd[u].head != nd[v].head) {
    if (nd[nd[v].head].depth > nd[nd[u].head].depth) {
      std::swap(u, v);
    }
    int h = nd[u].head;
    result += chain_op(h, h, u, op);
    u = nd[h].parent;
  }

  if (nd[u].depth > nd[v].depth) {
    std::swap(u, v);
  }
  result += chain_op(nd[u].head, u, v, op);

  return result;
}

void process_queries() {
  while (num_queries--) {
    op_id++;
    int a, b, c, d;
    scanf("%d %d %d %d", &a, &b, &c, &d);
    visit_path(a, b, OP_UPDATE);
    printf("%d\n", visit_path(c, d, OP_QUERY));
  }
}

int main() {
  read_tree();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  process_queries();

  return 0;
}
```

19.4 Problema Rafaela (Lot 2014)

‹ înapoi

Soluție cu HLD (versiune online).

```
// Complexitate: O(q log^2 n).
// Metodă: descompunere heavy-light.
#include <stdio.h>

const int MAX_NODES = 200'000;
```

```
const int MAX_SEGTREE_NODES = 1 << 19;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;      // fiul cu cel mai mare subarbore
  int head;       // începutul lanțului nostru
  int tin, tout;  // momentele intrării și ieșirii din DFS
  int init_pop;   // populația inițială
};

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct segtree_node {
  int m;    // maximul din intervalul subîntins
  int lazy; // sumă de adăugat la tot intervalul
  // Invariant: pentru nodul curent, m include lazy
};

// Arbore de segmente cu adăugare pe interval și maxim pe interval.
struct segment_tree {
  segtree_node v[MAX_SEGTREE_NODES];
  int n, bits;

  void init(int size) {
    n = next_power_of_2(size);
    bits = __builtin_popcount(n - 1);
  }

  void raw_set(int pos, int val) {
    v[pos + n].m = val;
  }

  void build() {
    for (int pos = n - 1; pos; pos--) {
      v[pos].m = max(v[2 * pos].m, v[2 * pos + 1].m);
    }
  }

  void push(int pos) {
```

```
  if (v[pos].lazy) {
    v[2 * pos].m += v[pos].lazy;
    v[2 * pos].lazy += v[pos].lazy;
    v[2 * pos + 1].m += v[pos].lazy;
    v[2 * pos + 1].lazy += v[pos].lazy;
    v[pos].lazy = 0;
  }
}

void push_path(int pos) {
  for (int b = bits; b; b--) {
    push(pos >> b);
  }
}

void pull_path(int pos) {
  for (pos /= 2; pos; pos /= 2) {
    if (!v[pos].lazy) {
      v[pos].m = max(v[2 * pos].m, v[2 * pos + 1].m);
    }
  }
}

void range_add(int l, int r, int val) {
  l += n;
  r += n;
  int orig_l = l, orig_r = r;

  push_path(l);
  push_path(r);

  while (l <= r)  {
    if (l & 1) {
      v[l].m += val;
      v[l++].lazy += val;
    }
    l >>= 1;

    if (!(r & 1)) {
      v[r].m += val;
      v[r--].lazy += val;
    }
    r >>= 1;
  }

  pull_path(orig_l);
  pull_path(orig_r);
}

int range_max(int l, int r) {
```

```
    int result = 0;

    l += n;
    r += n;
    push_path(l);
    push_path(r);

    while (l <= r)  {
      if (l & 1) {
        result = max(result, v[l++].m);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = max(result, v[r--].m);
      }
      r >>= 1;
    }

    return result;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
segment_tree segtree;
FILE *fin, *fout;
int n, num_queries, total_pop;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  fscanf(fin, "%d %d", &n, &num_queries);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }

  for (int u = 1; u <= n; u++) {
    fscanf(fin, "%d", &nd[u].init_pop);
    total_pop += nd[u].init_pop;
  }
}
```

```
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      nd[u].init_pop += nd[v].init_pop;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].tin = time++;
  segtree.raw_set(nd[u].tin, nd[u].init_pop);

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // la v începe un lanț nou
    }
  }

  nd[u].tout = time - 1;
}

void update(int u, int delta) {
  total_pop += delta;
  while (u) {
    int h = nd[u].head;
    segtree.range_add(nd[h].tin, nd[u].tin, delta);
    u = nd[h].parent;
  }
```

```
}

int query(int u) {
  int size_of_u = segtree.range_max(nd[u].tin, nd[u].tin);
  int from_parent = total_pop - size_of_u;
  int from_any_child = segtree.range_max(nd[u].tin + 1, nd[u].tout);
  return max(from_parent, from_any_child);
}

void process_ops() {
  char type;
  int u, delta;

  while (num_queries--) {
    fscanf(fin, " %c", &type);
    if (type == 'U') {
      fscanf(fin, "%d %d ", &delta, &u);
      update(u, delta);
    } else {
      fscanf(fin, "%d ", &u);
      fprintf(fout, "%d\n", query(u));
    }
  }
}

int main() {
  fin = fopen("rafaela.in", "r");
  fout = fopen("rafaela.out", "w");

  read_tree();
  segtree.init(n);
  heavy_dfs(1);
  decompose_dfs(1, 1);
  segtree.build();
  process_ops();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

Soluție cu HLD și `multiset` (versiune online).

```
// Complexitate: O(q log^2 n).
// Metodă: descompunere heavy-light + multiset.
#include <set>
#include <stdio.h>
```

```
const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;      // fiul cu cel mai mare subarbore
  int head;       // începutul lanțului nostru
  int pos;        // momentul descoperirii în dfs
  int init_pop;   // populația inițială
  std::multiset<int> set;
};

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct fenwick_tree {
  int v[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void range_add(int l, int r, int val) {
    add(l, val);
    add(r + 1, -val);
  }

  void point_add(int pos, int val) {
    range_add(pos, pos, val);
  }

  int get(int pos) {
    long long s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
```

```
    return s;
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
fenwick_tree fen;
FILE *fin, *fout;
int n, num_queries, total_pop;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  fscanf(fin, "%d %d", &n, &num_queries);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }

  for (int u = 1; u <= n; u++) {
    fscanf(fin, "%d", &nd[u].init_pop);
    total_pop += nd[u].init_pop;
  }
}

int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      nd[u].init_pop += nd[v].init_pop;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }
```

```
  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 1;

  nd[u].head = head;
  nd[u].pos = time++;
  fen.point_add(nd[u].pos, nd[u].init_pop);

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // la v începe un lanț nou
    }
  }
}

void update_parent_set(int h, int delta) {
  int p = nd[h].parent;
  std::multiset<int>& s = nd[p].set; // syntactic sugar
  int old = fen.get(nd[h].pos);
  auto it = s.find(old);
  if (it != s.end()) {
    s.erase(it);
  }
  s.insert(old + delta);
}

void update(int u, int delta) {
  total_pop += delta;
  while (u) {
    int h = nd[u].head;
    update_parent_set(h, delta);
    fen.range_add(nd[h].pos, nd[u].pos, delta);
    u = nd[h].parent;
  }
}

int query(int u) {
  int size_of_u = fen.get(nd[u].pos);
  int max_flow = total_pop - size_of_u;

  if (nd[u].heavy) {
    int from_heavy_child = fen.get(nd[nd[u].heavy].pos);
    max_flow = max(max_flow, from_heavy_child);
```

```
  }

  if (!nd[u].set.empty()) {
    max_flow = max(max_flow, *nd[u].set.rbegin());
  }

  return max_flow;
}

void process_ops() {
  char type;
  int u, delta;

  while (num_queries--) {
    fscanf(fin, " %c", &type);
    if (type == 'U') {
      fscanf(fin, "%d %d ", &delta, &u);
      update(u, delta);
    } else {
      fscanf(fin, "%d ", &u);
      fprintf(fout, "%d\n", query(u));
    }
  }
}

int main() {
  fin = fopen("rafaela.in", "r");
  fout = fopen("rafaela.out", "w");

  read_tree();
  fen.init(n);
  heavy_dfs(1);
  decompose_dfs(1, 1);
  process_ops();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

Soluție cu descompunere în radical după interogări (versiune online).

```
// Metodă: descompunere în radical după interogări.
#include <algorithm>
#include <stdio.h>
#include <unordered_set>
#include <vector>
```

```
const int MAX_NODES = 200'000;
const int MAX_QUERIES = 200'000;
const int BLOCK_SIZE = 3'000;

struct query {
  int id;             // ordinea de la intrare
  int outside_delta;  // modificări în afara subarborelui
  int child_delta;    // modificări la fiul curent
  int max_touched;    // maximul dintre fiii anteriori

  query(int id) {
    this->id = id;
    outside_delta = 0;
    child_delta = 0;
    max_touched = 0;
  }
};

struct node {
  std::vector<int> adj;
  std::vector<query> q; // interogări (din blocul curent)
  int parent;
  int tin, tout;        // timpii de intrare/ieșire din DFS
  int pop, spop;        // populația din nod, respectiv din subarbore
};

struct update {
  int id, u, delta;
};

node nd[MAX_NODES + 1];
update updates[BLOCK_SIZE];
int answer[MAX_QUERIES + 1];
std::unordered_set<int> nodes_with_queries;
int n, num_ops, num_updates;
FILE* fin;

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

void read_data() {
  fscanf(fin, "%d %d", &n, &num_ops);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
```

```
    fscanf(fin, "%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  for (int u = 1; u <= n; u++) {
    fscanf(fin, "%d", &nd[u].pop);
  }
}

// Calculează timpii DFS și părinții
void initial_dfs(int u) {
  static int time = 0;
  nd[u].tin = time++;

  for (int v: nd[u].adj) {
    if (v != nd[u].parent) {
      nd[v].parent = u;
      initial_dfs(v);
    }
  }

  nd[u].tout = time - 1;
}

void read_ops(int start, int end) {
  char type;
  int u, delta;

  num_updates = 0;
  for (int id = start; id < end; id++) {
    fscanf(fin, " %c", &type);
    if (type == 'U') {
      fscanf(fin, "%d %d ", &delta, &u);
      updates[num_updates++] = { id, u, delta };
      // vor intra în vigoare în următorul bloc
      nd[u].pop += delta;
    } else {
      fscanf(fin, "%d ", &u);
      nd[u].q.push_back(query(id));
      nodes_with_queries.insert(u);
    }
  }
}

void sort_updates_by_dfs_time() {
  std::sort(updates, updates + num_updates, [](update& a, update& b) {
    return nd[a.u].tin < nd[b.u].tin;
  });
}
```

```
void dfs(int u) {
  nd[u].spop = nd[u].pop;

  for (int v: nd[u].adj) {
    if (v != nd[u].parent) {
      dfs(v);
      nd[u].spop += nd[v].spop;
    }
  }
}

// Notifică-l pe u că există actualizarea upd în afara subarborelui său.
void process_outside_update(int u, update& upd) {
  for (query& q: nd[u].q) {
    if (upd.id < q.id) {
      q.outside_delta += upd.delta;
    }
  }
}

// Notifică-l pe u că există actualizarea upd în fiul său curent.
void process_child_update(int u, update& upd) {
  for (query& q: nd[u].q) {
    if (upd.id < q.id) {
      q.child_delta += upd.delta;
    }
  }
}

int merge_init(int u) {
  int i = 0;

  while ((i < num_updates) && (nd[updates[i].u].tin < nd[u].tin)) {
    process_outside_update(u, updates[i++]);
  }

  while ((i < num_updates) && (updates[i].u == u)) {
    // Doar le sărim. Populația din u nu influențează interogările din u.
    i++;
  }

  return i;
}

int merge_advance(int u, int v, int i) {
  while ((i < num_updates) && (nd[updates[i].u].tout <= nd[v].tout)) {
    process_child_update(u, updates[i++]);
  }
```

```
  for (query& q: nd[u].q) {
    q.max_touched = max(q.max_touched, nd[v].spop + q.child_delta);
    q.child_delta = 0;
  }

  return i;
}

void merge_finish(int u, int i) {
  while (i < num_updates) {
    process_outside_update(u, updates[i++]);
  }
}

void answer_queries(int u, int max_untouched) {
  for (query& q: nd[u].q) {
    // fluxul pe muchia dinspre părinte
    answer[q.id] = nd[1].spop + q.outside_delta - nd[u].spop;

    // maximul fiilor afectați de actualizări
    answer[q.id] = max(answer[q.id], q.max_touched);

    // maximul fiilor neafectați de actualizări
    answer[q.id] = max(answer[q.id], max_untouched);
  }

  nd[u].q.clear();
}

void process_queries(int u) {
  int max_untouched = 0;
  int i = merge_init(u);

  for (int v: nd[u].adj) {
    if (v != nd[u].parent) {
      int old_i = i;
      i = merge_advance(u, v, i);

      if (i == old_i) {
        max_untouched = max(max_untouched, nd[v].spop);
      }
    }
  }

  merge_finish(u, i);
  answer_queries(u, max_untouched);
}

void process_nodes_with_queries() {
  for (int u: nodes_with_queries) {
```

```
    process_queries(u);
  }
  nodes_with_queries.clear();
}

void process_ops() {
  for (int start = 0; start < num_ops; start += BLOCK_SIZE) {
    int end = min(start + BLOCK_SIZE, num_ops);
    dfs(1);
    read_ops(start, end);
    sort_updates_by_dfs_time();
    process_nodes_with_queries();
  }
}

void write_answers() {
  FILE* f = fopen("rafaela.out", "w");

  for (int i = 0; i < num_ops; i++) {
    if (answer[i]) {
      fprintf(f, "%d\n", answer[i]);
    }
  }

  fclose(f);
}

int main() {
  fin = fopen("rafaela.in", "r");

  read_data();
  initial_dfs(1);
  process_ops();

  fclose(fin);

  write_answers();

  return 0;
}
```

19.5 Problema Doi arbori (Lot 2025)

‹ înapoi

Soluție cu descompunere *heavy-light* (versiune online).

```
// Complexitate: O(q log^2 n).
```

```cpp
// Metodă: Descompunere heavy-light.
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_SEGTREE_NODES = 1 << 19;
const int INFINITY = 3 * MAX_NODES;
const int OP_TOGGLE = 1;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;       // fiul cu subarborele maxim
  int head;        // începutul lanțului nostru
  int light_start; // începutul restului subarborelui, după lanțul greu
  int d;           // adîncimea
  int tin, tout;   // timpii DFS
};

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

struct segment_tree {
  int v[MAX_SEGTREE_NODES];
  int n;

  void init(int n) {
    this->n = next_power_of_2(n);
    for (int i = 1; i < 2 * this->n; i++) {
      v[i] = INFINITY;
    }
  }

  void set(int pos, int val) {
    pos += n;
    v[pos] = val;
    for (pos >>= 1; pos; pos >>= 1) {
      v[pos] = min(v[2 * pos], v[2 * pos + 1]);
    }
  }

  int get(int pos) {
```

```
    return v[pos + n];
  }

  int get_min(int l, int r) {
    int result = INFINITY;
    l += n;
    r += n;

    while (l <= r)  {
      if (l & 1) {
        result = min(result, v[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = min(result, v[r--]);
      }
      r >>= 1;
    }

    return result;
  }

  int get_min_plus(int l, int r, int delta) {
    int x = get_min(l, r);
    return (x == INFINITY) ? INFINITY : (x + delta);
  }
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
segment_tree seg, seg_diff, seg_2diff;
int n, num_ops;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  scanf("%d %d", &n, &num_ops);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}
```

```
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      nd[v].d = 1 + nd[u].d;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].tin = time++;

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }
  nd[u].light_start = time;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // începi un lanț nou
    }
  }

  nd[u].tout = time - 1;
}

void init_segment_trees() {
  seg.init(n);
  seg_diff.init(n);
  seg_2diff.init(n);
}

void toggle(int u) {
```

```
  int was_off = seg.get(nd[u].tin) == INFINITY;
  int val = was_off ? nd[u].d : INFINITY;
  seg.set(nd[u].tin, val);

  while (u) {
    seg_diff.set(nd[u].tin, (val == INFINITY) ? INFINITY : (val - nd[u].d));
    seg_2diff.set(nd[u].tin, (val == INFINITY) ? INFINITY : (val - 2 * nd[u].d));
    u = nd[nd[u].head].parent;
    if (u) {
      val = seg.get_min(nd[u].light_start, nd[u].tout);
    }
  }
}

// Găsește distanța minimă până la o frunză urcînd din u.
int up_query(int u) {
  int leaf_dist = INFINITY;
  int orig_u = u;

  while (u) {
    int h = nd[u].head;
    int on_path_h_u = seg_2diff.get_min_plus(nd[h].tin, nd[u].tin, nd[orig_u].d);
    int on_path_below_u = seg.get_min_plus(nd[u].tin, nd[u].tout, nd[orig_u].d - 2 * nd[u].d);
    leaf_dist = min(leaf_dist, on_path_h_u);
    leaf_dist = min(leaf_dist, on_path_below_u);
    u = nd[h].parent;
  }

  return leaf_dist;
}

// Găsește distanța în jos până la o frunză de la orice nod de pe lanțul greu
// u-v.
int path_score(int u, int v) {
  int on_path_u_v = seg_diff.get_min(nd[u].tin, nd[v].tin);
  int in_subtree_of_v = seg.get_min_plus(nd[v].tin, nd[v].tout, -nd[v].d);
  return min(on_path_u_v, in_subtree_of_v);
}

int query(int u, int v) {
  int orig_u = u, orig_v = v;

  int leaf_dist = INFINITY;
  // Urcă-l pe u sau v cît timp sînt pe lanțuri diferite.
  while (nd[u].head != nd[v].head) {
    if (nd[v].tin > nd[u].tin) {
      int tmp = u; u = v; v = tmp;
    }
    int h = nd[u].head;
    leaf_dist = min(leaf_dist, path_score(h, u));
```

```
    u = nd[h].parent;
  }

  // O ultimă operație când u și v se află pe același lanț.
  if (nd[u].tin > nd[v].tin) {
    int tmp = u; u = v; v = tmp;
  }
  leaf_dist = min(leaf_dist, path_score(u, v));

  leaf_dist = min(leaf_dist, up_query(u));

  if (leaf_dist == INFINITY) {
    return -1;
  } else {
    int dist_uv = nd[orig_u].d + nd[orig_v].d - 2 * nd[u].d;
    return dist_uv + 2 * leaf_dist;
  }
}

void process_ops() {
  int type, u, v;

  while (num_ops--) {
    scanf("%d %d", &type, &u);
    if (type == OP_TOGGLE) {
      toggle(u);
    } else {
      scanf("%d", &v);
      printf("%d\n", query(u, v));
    }
  }
}

int main() {
  read_tree();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  init_segment_trees();
  process_ops();

  return 0;
}
```

Soluție parțială cu descompunere în radical (versiune online).

```
// Complexitate: O((n + q) sqrt q).
// Metodă: Descompunere în radical după interogări.
#include <stdio.h>
```

```
const int MAX_NODES = 200'000;
const int MAX_OPS = 200'000;
const int MAX_LOG = 19;
const int BLOCK_SIZE = 2'000; // determinat experimental
const int INFINITY = 2 * MAX_NODES;

const int OP_TOGGLE = 1;
const int OP_QUERY = 2;

struct cell {
  int v, next;
};

cell list[2 * MAX_NODES];

struct node {
  int adj;
  int depth;

  // binary lifting cu doi pointeri per nod
  int parent, jump;

  int dist; // distanța pînă la cea mai apropiată frunză safe

  // distanța pînă la cea mai apropiată frunză safe de la orice nod subîntins
  // de jump pointer (excluzînd nodul destinație)
  int jdist;

  bool active;
  bool safe;
};

struct operation {
  int type, u, v;
};

// O simplă coadă.
struct queue {
  int v[MAX_NODES];
  int head, tail;

  void init() {
    head = tail = 0;
  }

  void enqueue(int u) {
    v[tail++] = u;
  }

  int dequeue() {
```

```
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

// O colecție care menține valori distincte între 1 și MAX_NODES și admite
// adăugări și ștergeri în O(1) și iterarea prin valori în O(nr. de valori).
struct tracker {
  int size;
  int t[BLOCK_SIZE];
  int pos[MAX_NODES + 1]; // BLOCK_SIZE = absent

  void init(int max_val) {
    for (int val = 1; val <= max_val; val++) {
      pos[val] = BLOCK_SIZE;
    }
    size = 0;
  }

  void clear() {
    for (int i = 0; i < size; i++) {
      pos[t[i]] = BLOCK_SIZE;
    }
    size = 0;
  }

  void add(int val) {
    pos[val] = size;
    t[size++] = val;
  }

  void remove(int val) {
    int p = pos[val];
    t[p] = t[size - 1]; // move last element in place of val
    pos[t[p]] = p;
    pos[val] = BLOCK_SIZE;
    size--;
  }

  void toggle(int val) {
    if (pos[val] == BLOCK_SIZE) {
      add(val);
    } else {
      remove(val);
    }
  }
};
```

```
node nd[MAX_NODES + 1];
operation op[MAX_OPS];
tracker unsafe;
queue q;
int n, num_ops;

int log(int x) {
  return 31 - __builtin_clz(x);
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

// Liniarizare cu RMQ, cu care putem răspunde la interogări de LCA în O(1).
struct lca_info {
  int d[MAX_LOG][2 * MAX_NODES];
  int tout[MAX_NODES + 1];
  int n; // lungimea liniarizării

  void append(int u) {
    tout[u] = n;
    d[0][n++] = u;
  }

  void build_rmq_table() {
    for (int l = 1; (1 << l) <= n; l++) {
      for (int i = 0; i <= n - (1 << l); i++) {
        int r = i + (1 << (l - 1));
        d[l][i] = (nd[d[l - 1][i]].depth < nd[d[l - 1][r]].depth)
          ? d[l - 1][i]
          : d[l - 1][r];
      }
    }
  }

  int lca(int u, int v) {
    int left = min(tout[u], tout[v]);
    int right = tout[u] + tout[v] - left;
    int bits = log(right - left + 1);
    int x = d[bits][left];
    int y = d[bits][right - (1 << bits) + 1];
    return (nd[x].depth < nd[y].depth) ? x : y;
  }

  int dist(int u, int v) {
    int l = lca(u, v);
    return nd[u].depth + nd[v].depth - 2 * nd[l].depth;
  }
```

```
};

lca_info l;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_data() {
  scanf("%d %d", &n, &num_ops);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }

  for (int i = 0; i < num_ops; i++) {
    scanf("%d %d", &op[i].type, &op[i].u);
    if (op[i].type == OP_QUERY) {
      scanf("%d", &op[i].v);
    }
  }
}

// Calculează părinți, adîncimi, jump pointers și liniarizarea.
void initial_dfs(int u) {
  int u2 = nd[u].jump, u3 = nd[u2].jump;
  bool equal = (nd[u2].depth - nd[u].depth == nd[u3].depth - nd[u2].depth);
  l.append(u);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {
      nd[v].depth = 1 + nd[u].depth;
      nd[v].parent = u;
      nd[v].jump = equal ? u3 : u;
      initial_dfs(v);
      l.append(u);
    }
  }
}

// Colecteaza un bloc cu tel mult BLOCK_SIZE operații toggle(). Marchează ca
// safe frunzele care sînt active acum și care nu vor fi modificate în bloc.
// Inițializează trackerul cu frunzele unsafe, dar care încep ca active.
// Returnează finalul blocului.
```

```
int classify_leaves(int start) {
  int num_toggles = 0;
  int end = start;

  unsafe.clear();
  for (int u = 1; u <= n; u++) {
    nd[u].safe = nd[u].active;
  }

  while ((end < num_ops) && (num_toggles < BLOCK_SIZE)) {
    if (op[end].type == OP_TOGGLE) {
      int u = op[end].u;
      if (nd[u].safe) {
        // Frunza este unsafe, dar începe ca activă.
        unsafe.add(u);
        nd[u].safe = false;
      }
      num_toggles++;
    }
    end++;
  }

  return end;
}

// Recalculează câmpul dist pentru toate nodurile.
void bfs_from_safe_leaves() {
  q.init();

  for (int u = 1; u <= n; u++) {
    if (nd[u].safe) {
      nd[u].dist = 0;
      q.enqueue(u);
    } else {
      nd[u].dist = INFINITY;
    }
  }

  while (!q.is_empty()) {
    int u = q.dequeue();
    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      if (nd[v].dist == INFINITY) {
        nd[v].dist = nd[u].dist + 1;
        q.enqueue(v);
      }
    }
  }
}
```

```
// Recalculează cîmpul jdist pentru toate nodurile.
void jdist_dfs(int u) {
  nd[u].jdist = nd[u].dist;

  int p = nd[u].parent;
  if (nd[u].jump != p) {
    int p2 = nd[p].jump;
    nd[u].jdist = min(nd[u].jdist, nd[p].jdist);
    nd[u].jdist = min(nd[u].jdist, nd[p2].jdist);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != p) {
      jdist_dfs(v);
    }
  }
}

// Returnează finalul blocului.
int preprocess_block(int start) {
  int end = classify_leaves(start);
  bfs_from_safe_leaves();
  jdist_dfs(1);

  return end;
}

// Returnează distanța minimă la o frunză safe de la orice nod de pe calea [u,
// ancestor].
int upwards_path_min(int u, int ancestor) {
  int result = nd[ancestor].dist;

  while (u != ancestor) {
    if (nd[u].jump && (nd[nd[u].jump].depth >= nd[ancestor].depth)) {
      result = min(result, nd[u].jdist);
      u = nd[u].jump;
    } else {
      result = min(result, nd[u].dist);
      u = nd[u].parent;
    }
  }

  return result;
}

int query(int u, int v) {
  // Rezultatul vizitării unei frunze safe.
  int w = l.lca(u, v);
  int leaf_dist = min(upwards_path_min(u, w), upwards_path_min(v, w));
```

```
  int dist_uv = nd[u].depth + nd[v].depth - 2 * nd[w].depth;
  int result = 2 * leaf_dist + dist_uv;

  // Rezultatul vizitării unei frunze unsafe.
  for (int i = 0; i < unsafe.size; i++) {
    int leaf = unsafe.t[i];
    result = min(result, l.dist(u, leaf) + l.dist(leaf, v));
  }

  return (result < INFINITY) ? result : -1;
}

void toggle(int u) {
  unsafe.toggle(u);
  nd[u].active = !nd[u].active;
}

void process_ops() {
  int block_end = 0;

  for (int i = 0; i < num_ops; i++) {
    if (i == block_end) {
      block_end = preprocess_block(i);
    }
    if (op[i].type == OP_TOGGLE) {
      toggle(op[i].u);
    } else {
      printf("%d\n", query(op[i].u, op[i].v));
    }
  }
}

int main() {
  read_data();
  initial_dfs(1);
  l.build_rmq_table();
  unsafe.init(n);
  process_ops();

  return 0;
}
```

19.6 Problema Query on a Tree VI (CodeChef)

‹ înapoi • versiune online

```
#include <stdio.h>
```

```
const int MAX_NODES = 100'000;
const int BLACK = 0;
const int WHITE = 1;
const int T_QUERY = 0;
const int T_TOGGLE = 1;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;  // fiul cu cel mai mare subarbore
  int head;   // începutul lanțului nostru
  int pos;    // timpul de descoperire în DFS / poziția în liniarizare
  int size;   // mărimea subarborelui (indiferent de culoare)
  bool color;
};

// Un AIB cu adăugare pe interval și interogare punctuală.
struct range_add_fenwick_tree {
  int v[MAX_NODES + 2];
  int n;

  void init(int n) {
    this->n = n;
  }

  void add(int pos, int delta) {
    do {
      v[pos] += delta;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void range_add(int l, int r, int delta) {
    add(l, delta);
    add(r + 1, -delta);
  }

  void point_add(int pos, int delta) {
    range_add(pos, pos, delta);
  }

  int point_query(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
```

```
    }
    return sum;
  }
};

// Un AIB de booleeni cu suport pentru căutări binare.
struct bool_fenwick_tree {
  int v[MAX_NODES + 1];
  int n;
  int max_p2;

  void init(int n, bool val) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));

    for (int i = 1; i <= n; i++) {
      v[i] += val;
      int j = i + (i & -i);
      if (j <= n) {
        v[j] += v[i];
      }
    }
  }

  void add(int pos, int delta) {
    do {
      v[pos] += delta;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void set(int pos) {
    add(pos, +1);
  }

  void clear(int pos) {
    add(pos, -1);
  }

  int prefix_sum(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
    }
    return sum;
  }

  int last_occurrence_of_partial_sum(int sum) {
    int pos = 0;
```

```
    for (int interval = max_p2; interval; interval >>= 1) {
      if ((pos + interval <= n) && (v[pos + interval] < sum)) {
        sum -= v[pos + interval];
        pos += interval;
      }
    }

    return pos;
  }

  // Returnează poziția ultimului 1 pe o poziție ≤ pos.
  int last_one_before(int pos) {
    int s = prefix_sum(pos);
    if (s) {
      return 1 + last_occurrence_of_partial_sum(s);
    } else {
      return 0;
    }
  }
};

cell list[2 * MAX_NODES];
range_add_fenwick_tree size[2];
bool_fenwick_tree color[2];
node nd[MAX_NODES + 1];
int hld_order[MAX_NODES + 1];
int n;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  scanf("%d", &n);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

void init_fenwick_trees() {
  size[BLACK].init(n);
  size[WHITE].init(n);
  color[BLACK].init(n, 1);
  color[WHITE].init(n, 0);
```

```
}

void heavy_dfs(int u) {
  nd[u].size = 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      heavy_dfs(v);
      nd[u].size += nd[v].size;
      if (!nd[u].heavy || (nd[v].size > nd[nd[u].heavy].size)) {
        nd[u].heavy = v;
      }

    }
  }
}

void decompose_dfs(int u, int head) {
  static int time = 1;

  nd[u].head = head;
  size[BLACK].point_add(time, nd[u].size);
  size[WHITE].point_add(time, 1);
  hld_order[time] = u;
  nd[u].pos = time++;

  if (nd[u].heavy) {
    decompose_dfs(nd[u].heavy, head);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && v != nd[u].heavy) {
      decompose_dfs(v, v); // începe un lanț nou
    }
  }
}

// Returnează cel mai depărtat strămoș w pe același lanț greu cu u astfel
// încît (1) toate nodurile de la w la u inclusiv au aceeași culoare și (2)
// părintele lui w are o culoare diferită. Notă: părintele lui w poate fi pe
// lanțul următor. Returnează 0 dacă w nu există.
int get_top_same_color(int u) {
  int start = nd[nd[u].head].pos;
  int pos_1 = color[!nd[u].color].last_one_before(nd[u].pos);
  if (pos_1 >= start) {
    // Există un nod de culoare opusă pe lanț.
```

```
    return hld_order[pos_1 + 1];
  }

  int next_path = nd[nd[u].head].parent;
  if (nd[next_path].color != nd[u].color) {
    // Părintele capului de lanț are culoarea opusă.
    return nd[u].head;
  }

  return 0;
}

int query(int u) {
  nd[0].color = !nd[u].color; // santinelă

  int t;
  while (u && (t = get_top_same_color(u)) == 0) {
    u = nd[nd[u].head].parent;
  }

  return size[nd[t].color].point_query(nd[t].pos);
}

void path_update(int u, int delta1, int delta2) {
  int c = nd[u].color;
  size[!c].point_add(nd[u].pos, delta1);

  // Toți strămoșii cîștigă/pierd cîtă vreme au culoarea părintelui.
  int t;
  while (u && (t = get_top_same_color(u)) == 0) {
    size[c].range_add(nd[nd[u].head].pos, nd[u].pos, delta2);
    u = nd[nd[u].head].parent;
  }

  if (u) {
    size[c].range_add(nd[t].pos, nd[u].pos, delta2);

    // Și primul strămoș de culoarea opusă cîștigă/pierde.
    t = nd[t].parent;
    if (t) {
      size[c].point_add(nd[t].pos, delta2);
    }
  }
}

void toggle(int u) {
  int old = nd[u].color;
  nd[u].color = !old;

  color[old].clear(nd[u].pos);
```

```
  color[!old].set(nd[u].pos);

  int p = nd[u].parent;
  if (p) {
    int lose = size[old].point_query(nd[u].pos);
    int gain = size[!old].point_query(nd[u].pos);
    if (nd[u].color == nd[p].color) {
      path_update(p, -lose, gain);
    } else {
      path_update(p, gain, -lose);
    }
  }
}

void process_ops() {
  int num_queries, type, u;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &type, &u);
    if (type == T_QUERY) {
      printf("%d\n", query(u));
    } else {
      toggle(u);
    }
  }
}

int main() {
  read_tree();
  init_fenwick_trees();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  process_ops();

  return 0;
}
```

19.7 Problema Adă caii (Lot 2025)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <unordered_set>

const int MAX_NODES = 100'000;
const int MAX_PATH = 1'800; // determinat experimental

typedef std::unordered_set<int> hash_set;
```

```
// Un buffer circular de n elemente care operează într-o zonă de memorie
// alocată extern.
struct circ_buf {
  int* v;
  int n;
  int offset; // Conținutul real al vectorului este v[offset]...v[offset-1].
  int num_nonzero;

  void init(int* _v, int _n) {
    v = _v;
    n = _n;
    offset = 0;
    num_nonzero = 0;

    for (int i = 0; i < n; i++) {
      v[i] = 0;
    }
  }

  bool is_empty() {
    return num_nonzero == 0;
  }

  int get(int pos) {
    return v[(pos + offset) % n];
  }

  void set(int pos, int val) {
    pos = (pos + offset) % n;
    num_nonzero -= (v[pos] != 0);
    v[pos] = val;
    num_nonzero += (v[pos] != 0);
  }

  void add(int pos, int delta) {
    set(pos, get(pos) + delta);
  }

  // Shiftează (conceptual) toate elementele spre stînga. Inserează un zero la
  // coadă. Returnează elementul care a ieșit din buffer.
  int shift_left() {
    int leftmost = v[offset];
    num_nonzero -= (leftmost != 0);
    v[offset] = 0;
    offset = (offset + 1) % n;
    return leftmost;
  }

  int shift_right() {
```

```
    offset = (offset + n - 1) % n;
    int rightmost = v[offset];
    num_nonzero -= (rightmost != 0);
    v[offset] = 0;
    return rightmost;
  }

  // Mută naiv elementele din stînga lui pos cu 1 mai la dreapta. Mută
  // eficient elementele din dreapta lui pos cu 1 mai la stînga. Returnează
  // valoarea inițială de pe poziția pos.
  int attract_left(int pos) {
    if (pos == 0) {
      return shift_left();
    }

    int extracted = get(pos);

    add(pos + 1, get(pos - 1));
    for (int i = pos; i >= 2; i--) {
      set(i, get(i - 2));
    }
    set(1, 0);
    shift_left();

    return extracted;
  }

  int attract_right(int pos) {
    if (pos == n - 1) {
      return shift_right();
    }

    int extracted = get(pos);

    add(pos - 1, get(pos + 1));
    for (int i = pos; i + 2 < n; i++) {
      set(i, get(i + 2));
    }
    set(n - 2, 0);
    shift_right();

    return extracted;
  }

  int attract(int pos) {
    return (2 * pos < n)
      ? attract_left(pos)
      : attract_right(pos);
  }
};
```

```
struct cell {
  int v, next;
};

struct node {
  int adj;
  int parent;
  int heavy;   // fiul cu cel mai mare subarbore
  int head;    // capătul lanțului nostru
  int pos;     // momentul descoperirii în dfs
  int tmp_pop; // populația din nod, doar temporar pînă construim HLD

  circ_buf pop; // populația pe lanț, folosită doar în capetele de lanț
};

// Nu putem muta instantaneu caii de pe un lanț L1 pe altul L2, căci dacă încă
// nu l-am procesat pe L2, vom muta ulterior caii încă odată. Deci ținem o
// listă de postprocesări de făcut după ce procesăm toate lanțurile.
struct postprocess {
  int node, pop;
};

cell list[2 * MAX_NODES];
int circ_buf_space[MAX_NODES];
node nd[MAX_NODES + 1];
hash_set nonempty_paths; // Doar prin aceste lanțuri vom itera.
postprocess postproc[MAX_NODES];
int n, num_queries, num_postproc;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  scanf("%d %d", &n, &num_queries);

  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].tmp_pop);
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}
```

```
int heavy_dfs(int u) {
  int my_size = 1, max_child_size = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent) {

      nd[v].parent = u;
      int child_size = heavy_dfs(v);
      my_size += child_size;
      if (child_size > max_child_size) {
        max_child_size = child_size;
        nd[u].heavy = v;
      }

    }
  }

  return my_size;
}

// Descompunere heavy-light cu o mică modificare: impunem o limită de
// MAX_PATH pe lungimea oricărui lanț.
void decompose_dfs(int u, int head) {
  static int time = 0;

  nd[u].head = head;
  nd[u].pos = time++;
  int path_size = nd[u].pos - nd[head].pos + 1;
  bool can_fit_heavy = (path_size < MAX_PATH);

  if (nd[u].heavy && can_fit_heavy) {
    decompose_dfs(nd[u].heavy, head);
  } else {
    // Aici se termină lanțul head-u.
    int* start_pos = circ_buf_space + nd[head].pos;
    nd[head].pop.init(start_pos, path_size);
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != nd[u].parent && ((v != nd[u].heavy) || !can_fit_heavy)) {
      decompose_dfs(v, v); // la v începe un lanț nou
    }
  }
}

void populate_circular_buffers() {
  for (int u = 1; u <= n; u++) {
```

```
    int h = nd[u].head;
    int pos = nd[u].pos - nd[h].pos;
    nd[h].pop.add(pos, nd[u].tmp_pop);
  }

  for (int u = 1; u <= n; u++) {
    if ((nd[u].head == u) && !nd[u].pop.is_empty()) {
      nonempty_paths.insert(u);
    }
  }
}

void migrate_paths_to_root(int u, hash_set& paths_to_root) {
  int prev_h = 0;
  while (u) {
    int h = nd[u].head;
    if (!nd[h].pop.is_empty()) {
      int pos = nd[u].pos - nd[h].pos;
      int extracted = nd[h].pop.attract(pos);
      if (prev_h && extracted) {
        // Caii din centrul migrației coboară la începutul următorului lanț.
        postproc[num_postproc++] = { prev_h, extracted };
      } else {
        // Caii din centrul migrației nu pleacă nicăieri.
        nd[h].pop.add(pos, extracted);
      }
    }

    paths_to_root.insert(h);
    u = nd[h].parent;
    prev_h = h;
  }
}

// Migrează caii de pe celelalte lanțuri nevide. Evită lanțurile deja migrate
// în migrate_paths_to_root.
void migrate_other_paths(int u, hash_set paths_to_root) {
  for (int h: nonempty_paths) {
    if (!paths_to_root.contains(h)) {
      int extracted = nd[h].pop.shift_left(); // spre rădăcină
      if (extracted) {
        postproc[num_postproc++] = { nd[h].parent, extracted };
      }
    }
  }
}

void post_migration() {
  // Ștergem din set în timp ce iterăm prin set.
  for (auto it = nonempty_paths.begin(); it != nonempty_paths.end(); ) {
```

```
    if (nd[*it].pop.is_empty()) {
      it = nonempty_paths.erase(it);
    } else {
      it++;
    }
  }

  while (num_postproc) {
    postprocess& p = postproc[--num_postproc];
    int h = nd[p.node].head;
    int pos = nd[p.node].pos - nd[h].pos;
    nd[h].pop.add(pos, p.pop);
    nonempty_paths.insert(h);
  }
}

void migrate(int u) {
  hash_set paths_to_root;
  migrate_paths_to_root(u, paths_to_root);
  migrate_other_paths(u, paths_to_root);
  post_migration();
}

int query(int u) {
  int h = nd[u].head;
  int pos = nd[u].pos - nd[h].pos;
  return nd[h].pop.get(pos);
}

void process_ops() {
  char type1, type2;
  int u;

  while (num_queries--) {
    scanf(" %c%c %d", &type1, &type2, &u);
    if (type1 == 'C') {
      migrate(u);
    } else {
      int claimed_pop;
      scanf("%d", &claimed_pop);
      printf("%d\n", claimed_pop == query(u));
    }
  }
}

int main() {
  read_tree();
  heavy_dfs(1);
  decompose_dfs(1, 1);
  populate_circular_buffers();
```

```
  process_ops();

  return 0;
}
```

Anexa 20

Arbori - descompunere în centroizi

20.1 Problema Finding a Centroid (CSES)

înapoi

Implementare recursivă și cu STL.

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

struct node {
  std::vector<int> adj;
  int size;
};

node nd[MAX_NODES + 1];
int n;

void read_input_data() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

void size_dfs(int u) {
  nd[u].size = 1;

  for (int v: nd[u].adj) {
```

```
    if (!nd[v].size) {
      size_dfs(v);
      nd[u].size += nd[v].size;
    }
  }
}

int find_centroid(int u) {
  for (int v: nd[u].adj) {
    if ((nd[v].size < nd[u].size) && (nd[v].size > n / 2)) {
      return find_centroid(v);
    }
  }

  return u;
}

int main() {
  read_input_data();
  size_dfs(1);
  int c = find_centroid(1);
  printf("%d\n", c);

  return 0;
}
```

Implementare iterativă și cu liste proprii.

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int size;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_child(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
```

```
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_child(u, v);
    add_child(v, u);
  }
}

void size_dfs(int u) {
  nd[u].size = 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].size) {
      size_dfs(v);
      nd[u].size += nd[v].size;
    }
  }
}

int get_heavy_child(int u) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((nd[v].size < nd[u].size) && (nd[v].size > n / 2)) {
      return v;
    }
  }
  return 0;
}

void find_centroid() {
  int u = 1, child;

  while ((child = get_heavy_child(u)) != 0) {
    u = child;
  }

  printf("%d\n", u);
}

int main() {
  read_input_data();
  size_dfs(1);
  find_centroid();
```

```
  return 0;
}
```

20.2 Problema Mystery Tree (CodeChef)

înapoi

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

struct node {
  std::vector<int> adj;
  int size;
  bool dead;
};

node nd[MAX_NODES + 1];
int n;

void read_tree() {
  scanf("%d", &n);

  // șterge listele de adiacență
  for (int u = 1; u <= n; u++) {
    nd[u].adj.clear();
    nd[u].dead = false;
  }

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int v: nd[u].adj) {
    if ((v != parent) && !nd[v].dead) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}
```

```
int find_centroid(int u, int limit) {
  for (int v: nd[u].adj) {
    if ((nd[v].size < nd[u].size) && (nd[v].size > limit)) {
      return find_centroid(v, limit);
    }
  }

  return u;
}

void solve(int u) {
  int prev = 0;

  while (u != -1) {
    size_dfs(u, 0);
    u = find_centroid(u, nd[u].size / 2);

    printf("1 %d\n", u);
    fflush(stdout);

    nd[u].dead = true;
    prev = u;
    scanf("%d", &u);
  }

  printf("2 %d\n", prev);
}

int main() {
  int num_tests, check = 1;
  scanf("%d", &num_tests);

  while (num_tests-- && (check == 1)) {
    read_tree();
    solve(1);
    scanf("%d", &check);
  }

  return 0;
}
```

20.3 Problema Ciel the Commander (Codeforces)

‹ înapoi

Soluție cu descompunere în centroizi (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int size;
  bool dead;
  char rank; // răspunsul
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_child(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_child(u, v);
    add_child(v, u);
  }
}

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((v != parent) && !nd[v].dead) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}

int find_centroid(int u, int limit) {
```

```
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((nd[v].size < nd[u].size) && !nd[v].dead && (nd[v].size > limit)) {
      return find_centroid(v, limit);
    }
  }

  return u;
}

void decompose(int u, int rank) {
  size_dfs(u, 0);
  u = find_centroid(u, nd[u].size / 2);

  // Aceasta este problema pe care o rezolvăm. Restul codului este șablonul
  // pentru descompunerea în centroizi.
  nd[u].rank = rank;

  nd[u].dead = true;
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].dead) {
      decompose(v, rank + 1);
    }
  }
}

void write_ranks() {
  for (int u = 1; u <= n; u++) {
    putchar(nd[u].rank);
    putchar((u == n) ? '\n' : ' ');
  }
}

int main() {
  read_input_data();
  decompose(1, 'A');
  write_ranks();

  return 0;
}
```

Soluție elementară (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 200'000;

struct cell {
```

```
  int v, next;
};

struct node {
  int adj;
  char rank; // răspunsul
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
int n;

void add_child(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  scanf("%d", &n);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_child(u, v);
    add_child(v, u);
  }
}

// Returnează indicele celui mai din dreapta bit care respectă regulile de
// combinare.
int get_valid_bit(unsigned once, unsigned twice) {
  // Ia cel mai mare rang care apare de două ori.
  int msb = twice
    ? (31 - __builtin_clz(twice))
    : -1;

  // Fiecare bit în stînga lui msb este OK.
  unsigned legal_twice = ~((1 << (msb + 1)) - 1);

  // Fiecare bit văzut o singură dată nu este OK.
  unsigned legal_once = ~once;

  // Trebuie să respectăm ambele criterii.
  unsigned both = legal_once & legal_twice;

  // Returnează indicele celui mai din dreapta bit.
  return __builtin_ctz(both);
}
```

```
// Returnează o mască pe 26 de biți a rangurilor vizibile (MSB este A, LSB
// este Z).
unsigned dfs(int u, int parent) {
  unsigned once = 0, twice = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      unsigned vis_mask = dfs(v, u);
      // Noduri văzute o dată înainte și a doua oară în subarborele lui v.
      twice |= vis_mask & once;
      once |= vis_mask;
    }
  }

  int bit = get_valid_bit(once, twice);
  nd[u].rank = 'Z' - bit;

  // Acest rang este vizibil și maschează toate rangurile mai mici.
  return (1 << bit) | (once & ~((1 << bit) - 1));
}

void write_ranks() {
  for (int u = 1; u <= n; u++) {
    putchar(nd[u].rank);
    putchar((u == n) ? '\n' : ' ');
  }
}

int main() {
  read_input_data();
  dfs(1, 0);
  write_ranks();

  return 0;
}
```

20.4 Problema Fixed-Length Paths I (CSES) (din nou)

‹ înapoi

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

struct node {
  std::vector<int> adj;
```

```
  int size;
  bool dead;
};

node nd[MAX_NODES + 1];
int freq[MAX_NODES];
int length;
long long answer;

void read_input_data() {
  int n;
  scanf("%d %d", &n, &length);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

/**
 * Aceasta este problema pe care o rezolvăm efectiv.
 **/

int max(int x, int y) {
  return (x > y) ? x : y;
}

int max_depth;

// Numără căile de lungime k pornind din u, mergînd în sus pînă la centroid și
// în jos în alt subarbore.
void count_dfs(int u, int parent, int depth) {
  max_depth = max(max_depth, depth);
  answer += freq[length - depth];

  for (int v: nd[u].adj) {
    if ((!nd[v].dead) && (v != parent) && (depth < length)) {
      count_dfs(v, u, depth + 1);
    }
  }
}

// Marchează adîncimile fiecărui nod.
void mark_dfs(int u, int parent, int depth) {
  freq[depth]++;

  for (int v: nd[u].adj) {
    if ((!nd[v].dead) && (v != parent) && (depth < length)) {
```

```
      mark_dfs(v, u, depth + 1);
    }
  }
}

// Numără căile de lungime k care trec prin u.
void count_paths_through(int u) {
  freq[0] = 1; // nodul u însuși
  max_depth = 0;

  for (int v: nd[u].adj) {
    if (!nd[v].dead) {
      count_dfs(v, u, 1);
      mark_dfs(v, u, 1);
    }
  }

  for (int d = 0; d <= max_depth; d++) {
    freq[d] = 0;
  }
}

/**
 * Restul codului (decompose(), size_dfs() și find_centroid()) este șablon. Îl
 * folosim deoarece ne permite să rulăm un DFS din fiecare centroid și să
 * obținem timp O(n log n).
 **/

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int v: nd[u].adj) {
    if ((v != parent) && !nd[v].dead) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (int v: nd[u].adj) {
    if ((nd[v].size < nd[u].size) && (nd[v].size > limit)) {
      return find_centroid(v, limit);
    }
  }

  return u;
}

void decompose(int u) {
```

```
  size_dfs(u, 0);
  u = find_centroid(u, nd[u].size / 2);

  // Aceasta este ce încercăm să rezolvăm.
  count_paths_through(u);

  nd[u].dead = true;
  for (int v: nd[u].adj) {
    if (!nd[v].dead) {
      decompose(v);
    }
  }
}

void write_answer() {
  printf("%lld\n", answer);
}

int main() {
  read_input_data();
  decompose(1);
  write_answer();

  return 0;
}
```

20.5 Problema Xenia and Tree (Codeforces)

înapoi

Soluție cu descompunere în radical după operații (versiune online).

```
// Complexitate: O((n + q) sqrt q).
// Metodă: Descompunere în radical după operații.
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_LOG = 18;
const int BLOCK_SIZE = 1'000; // determinat experimental
const int INFINITY = MAX_NODES;

const int OP_PAINT = 1;

struct cell {
  int v, next;
};

cell list[2 * MAX_NODES];
```

```
struct node {
  int adj;
  int depth;
  int dist; // distanța până la cel mai apropiat nod roșu
  bool red;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void init() {
    head = tail = 0;
  }

  void enqueue(int u) {
    v[tail++] = u;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

node nd[MAX_NODES + 1];
int fresh[BLOCK_SIZE];
queue q;
int n, num_ops, num_fresh;

int log(int x) {
  return 31 - __builtin_clz(x);
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

// O liniarizare DFS cu repetiție și informații pentru interogări de RMQ și
// LCA în O(1).
struct lca_info {
  int d[MAX_LOG][2 * MAX_NODES];
  int tout[MAX_NODES + 1];
  int n; // lungimea liniarizării

  void append(int u) {
```

```
    tout[u] = n;
    d[0][n++] = u;
  }

  void build_rmq_table() {
    for (int l = 1; (1 << l) <= n; l++) {
      for (int i = 0; i <= n - (1 << l); i++) {
        int r = i + (1 << (l - 1));
        d[l][i] = (nd[d[l - 1][i]].depth < nd[d[l - 1][r]].depth)
          ? d[l - 1][i]
          : d[l - 1][r];
      }
    }
  }

  int lca(int u, int v) {
    int left = min(tout[u], tout[v]);
    int right = tout[u] + tout[v] - left;
    int bits = log(right - left + 1);
    int x = d[bits][left];
    int y = d[bits][right - (1 << bits) + 1];
    return (nd[x].depth < nd[y].depth) ? x : y;
  }

  int dist(int u, int v) {
    int l = lca(u, v);
    return nd[u].depth + nd[v].depth - 2 * nd[l].depth;
  }
};

lca_info l;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree() {
  scanf("%d %d", &n, &num_ops);

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

// Calculează adîncimile și liniarizarea.
```

```
void initial_dfs(int u, int parent) {
  l.append(u);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      nd[v].depth = 1 + nd[u].depth;
      initial_dfs(v, u);
      l.append(u);
    }
  }
}

void bfs_from_red_nodes() {
  q.init();

  for (int u = 1; u <= n; u++) {
    if (nd[u].red) {
      nd[u].dist = 0;
      q.enqueue(u);
    } else {
      nd[u].dist = INFINITY;
    }
  }

  while (!q.is_empty()) {
    int u = q.dequeue();
    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      if (nd[v].dist == INFINITY) {
        nd[v].dist = nd[u].dist + 1;
        q.enqueue(v);
      }
    }
  }
}

void preprocess_block() {
  bfs_from_red_nodes();
  num_fresh = 0;
}

void paint(int u) {
  nd[u].red = true;
  fresh[num_fresh++] = u;
}

int query(int u) {
  int result = nd[u].dist;
  for (int i = 0; i < num_fresh; i++) {
```

```
    result = min(result, l.dist(u, fresh[i]));
  }

  return result;
}

void process_ops() {
  nd[1].red = true;

  for (int i = 0; i < num_ops; i++) {
    if (i % BLOCK_SIZE == 0) {
      preprocess_block();
    }
    int type, u;
    scanf("%d %d", &type, &u);
    if (type == OP_PAINT) {
      paint(u);
    } else {
      printf("%d\n", query(u));
    }
  }
}

int main() {
  read_tree();
  initial_dfs(1, 0);
  l.build_rmq_table();
  process_ops();

  return 0;
}
```

Soluție cu descompunere în centroizi (versiune online).

```
// Complexitate: O((n + q) log n).
// Metodă: Descompunere în centroizi.
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_LOG = 17;

struct node {
  std::vector<int> adj;
  int size;
  int level;
  int closest_red;
  int cparent; // părintele în arborele de centroizi
  int cdist[MAX_LOG]; // distanța până la părintele centroid de nivel k
```

```
  bool dead;
};

node nd[MAX_NODES + 1];
int freq[MAX_NODES];
int n, num_queries;

void read_input_data() {
  scanf("%d %d", &n, &num_queries);

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }
}

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int v: nd[u].adj) {
    if ((v != parent) && !nd[v].dead) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}

void dist_dfs(int u, int parent, int level) {
  for (int v: nd[u].adj) {
    if ((v != parent) && !nd[v].dead) {
      nd[v].cdist[level] = 1 + nd[u].cdist[level];
      dist_dfs(v, u, level);
    }
  }
}

int find_centroid(int u, int limit) {
  for (int v: nd[u].adj) {
    if ((nd[v].size < nd[u].size) && (nd[v].size > limit)) {
      return find_centroid(v, limit);
    }
  }

  return u;
}

void decompose(int u, int parent, int level) {
  size_dfs(u, 0);
```

```
  u = find_centroid(u, nd[u].size / 2);

  nd[u].dead = true;
  nd[u].level = level;
  nd[u].cparent = parent;

  dist_dfs(u, 0, level);

  for (int v: nd[u].adj) {
    if (!nd[v].dead) {
      decompose(v, u, level + 1);
    }
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void paint_red(int u) {
  int center = u;
  for (int l = nd[u].level; l >= 0; l--) {
    nd[center].closest_red = min(nd[center].closest_red, nd[u].cdist[l]);
    center = nd[center].cparent;
  }
}

int find_closest_red(int u) {
  int result = n; // infinit

  int center = u;
  for (int l = nd[u].level; l >= 0; l--) {
    int dist = nd[u].cdist[l] + nd[center].closest_red;
    result = min(result, dist);
    center = nd[center].cparent;
  }
  return result;
}

void init_distances() {
  for (int u = 1; u <= n; u++) {
    nd[u].closest_red = n; // infinit
  }
  paint_red(1);
}

void process_ops() {
  while (num_queries--) {
    int type, u;
    scanf("%d %d", &type, &u);
```

```
    if (type == 1) {
      paint_red(u);
    } else {
      printf("%d\n", find_closest_red(u));
    }
  }
}

int main() {
  read_input_data();
  decompose(1, 0, 0);
  init_distances();
  process_ops();

  return 0;
}
```

20.6 Problema Flareon (Lot 2017)

‹ înapoi • versiune online

```
#include "flareon.h"
#include <vector>

const int MAX_NODES = 200'000;

struct node {
  std::vector<int> adj;
  std::vector<int> pow; // puterile flăcărilor cu originea aici
  int size;
};

static node nd[MAX_NODES + 1];
static long long ans[MAX_NODES + 1];

struct plume_tracker {
  // Date din punctul de vedere al centroidului, pe măsură ce flăcările urcă
  // în subarbore și ajung la centroid.
  //
  // Frecvențele flăcărilor indexate după puterea rămasă. Nu ținem evidența
  // puterilor mai mari decît n, deoarece acelea nu se vor stinge niciodată.
  int freq[MAX_NODES];
  // Numărul de flăcări.
  int cnt;
  // Puterea totală.
  long long sum;

  // Mărimea subarborelui curent. Ne trebuie ca să ne încadrăm în efort
```

```
// O(mărimea_subarborelui), nu O(n).
int sts;

void init(int sts) {
  this->sts = sts;
  sum = cnt = 0;
  for (int i = 0; i < sts; i++) {
    freq[i] = 0;
  }
}

void add_plume(int power, int depth, int sign) {
  if (power > depth) {
    if (power - depth < sts)  {
      freq[power - depth] += sign;
    }
    cnt += sign;
    sum += sign * (power - depth);
  }
  // Altfel nu face nimic: flacăra nu va ajunge la centroid.
}

// Colectează flăcările din nodul u și notează contribuția lor la centroid.
// Cînd sign = +1 includem contribuțiile, iar cînd sign = -1, le excludem.
void collect(int u, int parent, int depth, int sign) {
  for (int p: nd[u].pow) {
    add_plume(p, depth, sign);
  }

  for (int v: nd[u].adj) {
    if (v != parent) {
      collect(v, u, depth + 1, sign);
    }
  }
}

void distribute(int u, int parent, int depth, int cnt, long long sum) {
  // Stinge unele flăcări și redu puterile celorlalte cu cîte 1.
  sum -= cnt;
  cnt -= freq[depth];
  ans[u] += sum;

  for (int v: nd[u].adj) {
    if (v != parent) {
      distribute(v, u, depth + 1, cnt, sum);
    }
  }
}

void process_plumes_through(int u) {
```

```
    init(nd[u].size);
    collect(u, 0, 0, +1);
    ans[u] +=  sum;

    for (int v: nd[u].adj) {
      // Cînd numărăm efectele flăcărilor într-un subarbore, mai întîi
      // excludem flăcările care provin tocmai din acel subarbore.
      collect(v, u, 1, -1);
      distribute(v, u, 1, cnt, sum);
      collect(v, u, 1, +1);
    }
  }
};

plume_tracker pt;

/**
 * Restul codului (decompose(), size_dfs() și find_centroid()) este șablon. Îl
 * folosim deoarece ne permite să rulăm un DFS din fiecare centroid și să
 * obținem timp O(n log n).
 **/

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int v: nd[u].adj) {
    if (v != parent) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (int v: nd[u].adj) {
    if ((nd[v].size < nd[u].size) && (nd[v].size > limit)) {
      return find_centroid(v, limit);
    }
  }

  return u;
}

void delete_edge(int u, int v) {
  int i = 0;
  while (nd[u].adj[i] != v) {
    i++;
  }
  nd[u].adj[i] = nd[u].adj.back();
  nd[u].adj.pop_back();
```

```
}

void delete_node(int u) {
  for (int v: nd[u].adj) {
    delete_edge(v, u);
  }
}

void decompose(int u) {
  size_dfs(u, 0);
  u = find_centroid(u, nd[u].size / 2);

  // Aceasta este problema pe care încercăm să o rezolvăm.
  pt.process_plumes_through(u);

  delete_node(u);
  for (int v: nd[u].adj) {
    decompose(v);
  }
}

void solve(int n, int m, int* v, int* pos, int* power) {
  for (int u = 2; u <= n; u++) {
    nd[u].adj.push_back(v[u - 2]);
    nd[v[u - 2]].adj.push_back(u);
  }

  for (int i = 0; i < m; i++) {
    nd[pos[i]].pow.push_back(power[i]);
  }

  decompose(1);
  answer(ans + 1); // shiftează vectorul cu 1
}
```

20.7 Problema Digit Tree (Codeforces)

înapoi

Soluție cu descompunere în centroizi și `std::map` (versiune online).

```
// Method: Centroid decomposition.
// Complexity: O(n log^2 n).
#include <map>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
```

```
struct edge {
  int v, digit;
};

struct node {
  std::vector<edge> adj;
  int size;
  bool dead;
};

node nd[MAX_NODES];
int pow10[MAX_NODES + 1];
int inv_pow10[MAX_NODES + 1];
int n, mod;
long long num_pairs;

void read_tree() {
  scanf("%d %d", &n, &mod);

  for (int i = 1; i < n; i++) {
    int u, v, digit;
    scanf("%d %d %d", &u, &v, &digit);
    nd[u].adj.push_back({v, digit % mod});
    nd[v].adj.push_back({u, digit % mod});
  }
}

void extended_euclid(int a, int b, int& x, int& y) {
  if (!b) {
    x = 1;
    y = 0;
  } else {
    int xp, yp;
    extended_euclid(b, a % b, xp, yp);
    x = yp;
    y = xp - (a / b) * yp;
  }
}

int inverse(int a, int mod) {
  int x, y;
  extended_euclid(a, mod, x, y);
  return (x + mod) % mod;
}

void precompute_pow10() {
  pow10[0] = 1;
  for (int i = 1; i <= n; i++) {
    pow10[i] = 10ll * pow10[i - 1] % mod;
```

```
  }

  long long inv = inverse(10, mod);
  inv_pow10[0] = 1;
  for (int i = 1; i <= n; i++) {
    inv_pow10[i] = inv * inv_pow10[i - 1] % mod;
  }
}

struct pair_counter {
  std::map<int, int> freq;

  void head_dfs(int u, int parent, int depth, int rem, int sign) {
    freq[rem] += sign;

    for (edge e: nd[u].adj) {
      if ((e.v != parent) && !nd[e.v].dead) {
        int new_rem = ((long long)pow10[depth] * e.digit + rem) % mod;
        head_dfs(e.v, u, depth + 1, new_rem, sign);
      }
    }
  }

  void tail_dfs(int u, int parent, int depth, int rem) {
    int h = (long long)inv_pow10[depth] * (mod - rem) % mod;

    // Numărul de moduri de a prefixa această coadă cu un început.
    num_pairs += freq[h];

    // Are coada curentă un rest egal cu zero?
    num_pairs += (rem == 0);

    for (edge e: nd[u].adj) {
      if ((e.v != parent) && !nd[e.v].dead) {
        int new_rem = ((long long)rem * 10 + e.digit) % mod;
        tail_dfs(e.v, u, depth + 1, new_rem);
      }
    }
  }

  void count_pairs_through(int u) {
    freq.clear();

    for (edge e: nd[u].adj) {
      if (!nd[e.v].dead) {
        head_dfs(e.v, u, 1, e.digit, +1);
      }
    }

    for (edge e: nd[u].adj) {
```

```
      if (!nd[e.v].dead) {
        head_dfs(e.v, u, 1, e.digit, -1);
        tail_dfs(e.v, u, 1, e.digit);
        head_dfs(e.v, u, 1, e.digit, +1);
      }
    }

    // Numărul de căi de început care au un rest 0.
    num_pairs += freq[0];
  }
};

pair_counter pc;

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (edge e: nd[u].adj) {
    if ((e.v != parent) && !nd[e.v].dead) {
      size_dfs(e.v, u);
      nd[u].size += nd[e.v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (edge e: nd[u].adj) {
    if ((nd[e.v].size < nd[u].size) && !nd[e.v].dead && (nd[e.v].size > limit)) {
      return find_centroid(e.v, limit);
    }
  }

  return u;
}

void decompose(int u) {
  size_dfs(u, -1);
  u = find_centroid(u, nd[u].size / 2);

  pc.count_pairs_through(u);

  nd[u].dead = true;
  for (edge e: nd[u].adj) {
    if (!nd[e.v].dead) {
      decompose(e.v);
    }
  }
}

void write_answer() {
```

```
  printf("%lld\n", num_pairs);
}

int main() {
  read_tree();
  precompute_pow10();
  decompose(0);
  write_answer();

  return 0;
}
```

Soluție cu descompunere în centroizi și căutări binare (versiune online).

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;

struct edge {
  int v, digit;
};

struct node {
  std::vector<edge> adj;
  int size;
};

// Stochează informații despre frecvența unui rest modulo M. Putem interoga
// aceste informații pentru un fiu dat al rădăcinii (0-based) sau pentru toți
// fiii.
struct rem_info {
  struct elem {
    int child, rem, freq;

    bool operator ==(elem other) {
      return (child == other.child) && (rem == other.rem);
    }

    bool operator <(elem other) {
      return (child < other.child) ||
        ((child == other.child) && (rem < other.rem));
    }

    bool operator >(elem other) {
      return (child > other.child) ||
        ((child == other.child) && (rem > other.rem));
    }
```

```
};

elem c[MAX_NODES], t[MAX_NODES];
int nc, nt;

void init() {
  nc = nt = 0;
}

void insert(int child, int rem) {
  c[nc++] = { child, rem, 1 };
  t[nt++] = { 0, rem, 1 };
}

int merge_duplicates(elem* v, int n) {
  if (n) {
    int j = 1;
    for (int i = 1; i < n; i++) {
      if (v[i] == v[j - 1]) {
        v[j - 1].freq++;
      } else {
        v[j++] = v[i];
      }
    }
    n = j;
  }
  return n;
}

void sort() {
  std::sort(c, c + nc);
  std::sort(t, t + nt);

  nc = merge_duplicates(c, nc);
  nt = merge_duplicates(t, nt);
}

int count_rem_0() {
  return (nt && (t[0].rem == 0))
    ? t[0].freq
    : 0;
}

int find(elem* v, int n, int child, int rem) {
  elem el = { child, rem, 0 };
  int pos = std::lower_bound(v, v + n, el) - v;
  return ((pos < n) && (v[pos] == el)) ? v[pos].freq : 0;
}

int count_not_in_child(int child, int rem) {
```

```
    return find(t, nt, 0, rem) - find(c, nc, child, rem);
  }
};

node nd[MAX_NODES];
int pow10[MAX_NODES + 1];
int inv_pow10[MAX_NODES + 1];
rem_info r;
int n, mod;
long long num_pairs;

void read_tree() {
  scanf("%d %d", &n, &mod);

  for (int i = 1; i < n; i++) {
    int u, v, digit;
    scanf("%d %d %d", &u, &v, &digit);
    nd[u].adj.push_back({v, digit});
    nd[v].adj.push_back({u, digit});
  }
}

void extended_euclid(int a, int b, int& x, int& y) {
  if (!b) {
    x = 1;
    y = 0;
  } else {
    int xp, yp;
    extended_euclid(b, a % b, xp, yp);
    x = yp;
    y = xp - (a / b) * yp;
  }
}

int inverse(int a, int mod) {
  int x, y;
  extended_euclid(a, mod, x, y);
  return (x + mod) % mod;
}

void precompute_pow10() {
  pow10[0] = 1;
  for (int i = 1; i <= n; i++) {
    pow10[i] = 10ll * pow10[i - 1] % mod;
  }

  long long inv = inverse(10, mod);
  inv_pow10[0] = 1;
  for (int i = 1; i <= n; i++) {
    inv_pow10[i] = inv * inv_pow10[i - 1] % mod;
```

```
  }
}

struct pair_counter {
  int child;

  void head_dfs(int u, int parent, int depth, int rem) {
    r.insert(child, rem);

    for (edge e: nd[u].adj) {
      if (e.v != parent) {
        int new_rem = ((long long)pow10[depth] * e.digit + rem) % mod;
        head_dfs(e.v, u, depth + 1, new_rem);
      }
    }
  }

  void tail_dfs(int u, int parent, int depth, int rem) {
    int h = (long long)inv_pow10[depth] * (mod - rem) % mod;

    // Numărul de moduri de a prefixa această coadă cu un început.
    num_pairs += r.count_not_in_child(child, h);

    // Are coada curentă un rest egal cu zero?
    num_pairs += (rem == 0);

    for (edge e: nd[u].adj) {
      if (e.v != parent) {
        int new_rem = ((long long)rem * 10 + e.digit) % mod;
        tail_dfs(e.v, u, depth + 1, new_rem);
      }
    }
  }

  void count_pairs_through(int u) {
    r.init();

    for (edge e: nd[u].adj) {
      child = e.v;
      head_dfs(e.v, u, 1, e.digit % mod);
    }

    r.sort();

    for (edge e: nd[u].adj) {
      child = e.v;
      tail_dfs(e.v, u, 1, e.digit % mod);
    }

    // Numărul de căi de început care au un rest 0.
```

```
    num_pairs += r.count_rem_0();
  }
};

pair_counter pc;

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (edge e: nd[u].adj) {
    if (e.v != parent) {
      size_dfs(e.v, u);
      nd[u].size += nd[e.v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (edge e: nd[u].adj) {
    if ((nd[e.v].size < nd[u].size) && (nd[e.v].size > limit)) {
      return find_centroid(e.v, limit);
    }
  }

  return u;
}

void delete_edge(int u, int v) {
  int i = 0;
  while (nd[u].adj[i].v != v) {
    i++;
  }
  nd[u].adj[i] = nd[u].adj.back();
  nd[u].adj.pop_back();
}

void delete_node(int u) {
  for (edge e: nd[u].adj) {
    delete_edge(e.v, u);
  }
}

void decompose(int u) {
  size_dfs(u, -1);
  u = find_centroid(u, nd[u].size / 2);

  pc.count_pairs_through(u);

  delete_node(u);
  for (edge e: nd[u].adj) {
```

```
    decompose(e.v);
  }
}

void write_answer() {
  printf("%lld\n", num_pairs);
}

int main() {
  read_tree();
  precompute_pow10();
  decompose(0);
  write_answer();

  return 0;
}
```

20.8 Problema Cucuruz (Concurs RoAlgo #7)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MOD = 1'000'000'007;

struct stats {
  int cnt, sum;

  stats operator -(stats other) {
    return { cnt - other.cnt, (sum + MOD - other.sum) % MOD };
  }

  stats operator +=(stats other) {
    cnt += other.cnt;
    sum = (sum + other.sum) % MOD;
    return *this;
  }
};

struct cell {
  int v, next;
};

struct node {
  int adj;
  int size;
  int val;
```

```
  bool dead;
};

cell list[2 * MAX_NODES];
node nd[MAX_NODES + 1];
// prev[d] = numărul și suma tuturor căilor centroid-u pentru toate nodurile u
// de adîncime d din fiii văzuți anterior
stats prev[MAX_NODES], sum_prev;
int p;
int answer;

void add_child(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_input_data() {
  int n;
  scanf("%d %d", &n, &p);

  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].val);
  }

  for (int i = 1; i < n; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_child(u, v);
    add_child(v, u);
  }
}

// Combină calea curentă centroid-u, de greutate sum_r, cu statisticile l
// despre căile de adîncime cel mult p - depth din fiii anteriori.
void count_paths(int u, int parent, int depth, stats l, int sum_r) {
  sum_r = (sum_r + nd[u].val) % MOD;
  answer = (answer + l.sum + (long long)l.cnt * sum_r) % MOD;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((!nd[v].dead) && (v != parent) && (depth < p)) {
      count_paths(v, u, depth + 1, l - prev[p - depth], sum_r);
    }
  }
}

// Adaugă calea curentă centroid-u, de greutate sum, la statistici.
void add_paths(int u, int parent, int depth, int sum) {
  sum = (sum + nd[u].val) % MOD;
```

```
  if (depth < p) {
    prev[depth] += {1, sum};
    sum_prev += {1, sum};
  }

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((!nd[v].dead) && (v != parent) && (depth < p)) {
      add_paths(v, u, depth + 1, sum);
    }
  }
}

void clear_prev(int depth) {
  for (int d = 0; (d < depth) && (d < p); d++) {
    prev[d] = {0, 0};
  }
  sum_prev = {0, 0};
}

void process_paths_through(int u) {
  prev[0] = sum_prev = {1, nd[u].val};

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].dead) {
      count_paths(v, u, 1, sum_prev, 0);
      add_paths(v, u, 1, nd[u].val);
    }
  }

  clear_prev(nd[u].size);
}

void size_dfs(int u, int parent) {
  nd[u].size = 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((v != parent) && !nd[v].dead) {
      size_dfs(v, u);
      nd[u].size += nd[v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if ((nd[v].size < nd[u].size) && (nd[v].size > limit)) {
```

```
      return find_centroid(v, limit);
    }
  }

  return u;
}

void decompose(int u) {
  size_dfs(u, 0);
  u = find_centroid(u, nd[u].size / 2);

  process_paths_through(u);

  nd[u].dead = true;
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (!nd[v].dead) {
      decompose(v);
    }
  }
}

int main() {
  read_input_data();
  decompose(1);
  printf("%d\n", answer);

  return 0;
}
```

20.9 Problema LaNaic (Lot 2024)

‹ înapoi • versiune online

```
// Complexitate: O(n log n).
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

struct edge {
  int v, color;
};

struct node {
  std::vector<edge> adj;
  int size;
```

```
  int height;
};

int max(int x, int y) {
  return (x > y) ? x : y;
}

int k, m, longest_path;

struct cost {
  // Pentru o lungime fixată, vom prefera lanțurile care necesită numărul
  // minim de schimbări, iar la egalitate pe cele care încep cu un prefix cît
  // mai scurt. Acest criteriu este optim. Nu avem niciun motiv să preferăm un
  // lanț cu mai multe schimbări și prefix mai scurt. Preferăm să facem acea
  // schimbare suplimentară doar dacă este nevoie, la combinarea a două
  // lanțuri.
  int changes, prefix;

  void optimize(cost other) {
    if ((other.changes < changes) ||
        ((other.changes == changes) && (other.prefix < prefix))) {
      *this = other;
    }
  }
};

struct costs {
  // v[len] = Costul minim pentru a ajunge la lungimea len.
  std::vector<cost> v;

  // Lungimea curentă a vectorului de costuri. Artificiu necesar deoarece
  // trebuie să ne încadrăm în O(max_depth) pentru fiecare subarbore, nu avem
  // voie O(n). Din același motiv nu putem inițializa vectorul în O(n).
  int max_len;

  costs() {
    v.reserve(MAX_NODES);
  }

  void swap(costs& other) {
    v.swap(other.v);
    std::swap(max_len, other.max_len);
  }

  void reset() {
    max_len = 0;
    v[0] = { 0, 0 };
  }

  void optimize(int len, cost c) {
```

```
  if (len > max_len) {
    v[len] = c;
    max_len = len;
  } else {
    v[len].optimize(c);
  }
}

// Combină doi vectori de costuri care pornesc pe muchii de culori opuse.
// Folosind tehnica celor doi pointeri, caută lungimea maximă dintre două
// elemente al căror cost însumat este cel mult m.
void combine_opposite(costs& other) {
  int i = 0, j = other.max_len;
  while ((i <= max_len) && (j >= 0)) {
    if (v[i].changes + other.v[j].changes <= m) {
      longest_path = max(longest_path, i + j);
      i++;
    } else {
      j--;
    }
  }
}

// Față de combine_opposite(), aici adăugăm 1 la cost dacă suma prefixelor
// depășește k.
void combine_same(costs& other) {
  int i = 0, j = other.max_len;
  while ((i <= max_len) && (j >= 0)) {
    int sum_costs =
      v[i].changes +
      other.v[j].changes +
      (v[i].prefix + other.v[j].prefix > k);

    if (sum_costs <= m) {
      longest_path = max(longest_path, i + j);
      i++;
    } else {
      j--;
    }
  }
}

void absorb(costs& other) {
  // Fiii sînt sortați după înălțimea absolută, NU după înălțimea fezabilă
  // cu limita de modificări.
  if (other.max_len > max_len) {
    swap(other);
  }
  for (int i = 0; i <= other.max_len; i++) {
    v[i].optimize(other.v[i]);
```

```
    }
  }
};

// c[0/1] = Costurile optime pentru fiii explorați anterior care pornesc pe
// muchii 0, respectiv 1. cur_cost = costurile în construcție pentru fiul
// curent.
costs c[2], cur_cost;

// Combină fiul curent, care pornește pe culoarea cur_color, cu informațiile
// existente din c[0] și c[1]. Apoi absoarbe informațiile de aceeași culoare
// cu fiul curent.
void combine_and_absorb_costs(int cur_color) {
  c[cur_color].combine_same(cur_cost);
  c[1 - cur_color].combine_opposite(cur_cost);
  cur_cost.absorb(c[cur_color]);
  cur_cost.swap(c[cur_color]);
  cur_cost.reset();
}

struct tree {
  node a[MAX_NODES + 1];
  int n;

  // Informații despre prefixul căii curente în DFS-ul din centroidul
  // curent. Declarate în afara recursivității ca să reducem numărul de
  // parametri ai lui cost_dfs().
  int prefix_len, prefix_color;

  void add_edge(int u, int v, int color) {
    a[u].adj.push_back({v, color});
    a[v].adj.push_back({u, color});
  }

  // Calculează înălțimile nodurilor (raportate la frunze).
  void height_dfs(int u, int parent) {
    a[u].height = 0;
    for (edge e: a[u].adj) {
      if (e.v != parent) {
        height_dfs(e.v, u);
        a[u].height = max(a[u].height, 1 + a[e.v].height);
      }
    }
  }

  // Notează faptul că din fiul curent există o cale de lungime length care
  // face changes schimbări. Încorporează și informația despre prefixul căii.
  void collect_length(int length, int changes) {
    changes += prefix_len / (k + 1);
    if (changes <= m) {
```

```
    int rem = prefix_len % (k + 1);
    cur_cost.optimize(length + prefix_len, { changes, rem });
  }
}

// Vizitează nodul u. Pînă la el am făcut changes schimbări. Calea curentă
// se termină cu un sufix de lungime suf_len și culoare suf_color.
void cost_dfs(int u, int parent, int depth, int changes,
              int suf_len, int suf_color) {
  collect_length(depth, changes);

  for (edge e: a[u].adj) {
    if (e.v != parent) {
      if (e.color != suf_color) {
        // Schimbă culoarea: resetează sufixul.
        cost_dfs(e.v, u, depth + 1, changes, 1, e.color);
      } else if (suf_len + 1 <= k) {
        // Aceeași culoare. Mai putem extinde sufixul curent.
        cost_dfs(e.v, u, depth + 1, changes, suf_len + 1, e.color);
      } else if (changes < m) {
        // Aceeași culoare. Este nevoie de o schimbare.
        cost_dfs(e.v, u, depth + 1, changes + 1, 0, e.color);
      }
    }
  }
}

// Cîtă vreme sîntem pe culoarea de început a fiului curent, notează
// lungimea căii. Pentru fiecare nod din subarbore ne interesează lungimea
// prefixului de aceeași culoare cu care începe calea spre acel nod. Pot
// exista schimbări și pe acest lanț, dar le vom contabiliza la final.
void same_color_dfs(int u, int parent, int depth) {
  prefix_len = depth;
  collect_length(0, 0); // Nu avem nimic decît prefixul

  for (edge e: a[u].adj) {
    if (e.v != parent) {
      if (e.color == prefix_color) {
        same_color_dfs(e.v, u, depth + 1);
      } else {
        // Se schimbă culoarea; lansează DFS-ul normal, care contabilizează
        // schimbările.
        prefix_len = depth;
        cost_dfs(e.v, u, 1, 0, 1, e.color);
      }
    }
  }
}

// Compară toate căile care trec prin u și actualizează maximul global.
```

```
void find_longest_path_through(int u) {
  // Procesează fiii în ordinea crescătoare a înălțimilor. Acest pas este
  // necesar ca să rulăm în O(mărimea componentei) per centroid. Mai jos,
  // combinarea fiului curent cu cei precedenți durează O(înălțimea maximă
  // de până acum), echivalent cu O(mărimea fiului curent) datorită
  // sortării.
  //
  // Facem sortarea naiv. Sortarea prin numărare este mai lentă.
  height_dfs(u, 0);
  std::sort(a[u].adj.begin(), a[u].adj.end(), [&](edge e, edge f) {
    return a[e.v].height < a[f.v].height;
  });

  c[0].reset();
  c[1].reset();

  for (edge e: a[u].adj) {
    // Notează culoarea muchiei pe care începe fiul curent.
    prefix_color = e.color;
    // Generează vectorul de costuri pentru fiul curent.
    same_color_dfs(e.v, u, 1);
    combine_and_absorb_costs(e.color);
  }
}

/*********************************************/
/* Cod standard de descompunere în centroizi. */
/*********************************************/

// Calculează mărimea subarborelui fiecărui nod.
void size_dfs(int u, int parent) {
  a[u].size = 1;

  for (edge e: a[u].adj) {
    if (e.v != parent) {
      size_dfs(e.v, u);
      a[u].size += a[e.v].size;
    }
  }
}

int find_centroid(int u, int limit) {
  for (edge e: a[u].adj) {
    if ((a[e.v].size < a[u].size) && (a[e.v].size > limit)) {
      return find_centroid(e.v, limit);
    }
  }

  return u;
}
```

```
  // Încercare de optimizare: ca să nu tot testăm cîmpul .dead, ştergem
  // efectiv nodul din listele de adiacenţă ale vecinilor.
  void delete_edge(int u, int v) {
    int i = 0;
    while (a[u].adj[i].v != v) {
      i++;
    }
    a[u].adj[i] = a[u].adj.back();
    a[u].adj.pop_back();
  }

  void delete_node(int u) {
    for (edge e: a[u].adj) {
      delete_edge(e.v, u);
    }
  }

  // Descompune recursiv arborele în centroizi.
  void decompose(int u) {
    size_dfs(u, 0);
    u = find_centroid(u, a[u].size / 2);

    // Aici este efortul propriu-zis pentru centroidul curent.
    find_longest_path_through(u);

    delete_node(u);
    for (edge e: a[u].adj) {
      decompose(e.v);
    }
  }
};

tree t;

void read_data() {
  scanf("%d %d %d", &t.n, &k, &m);
  for (int i = 0; i < t.n - 1; i++) {
    int u, v, color;
    scanf("%d %d %d", &u, &v, &color);
    t.add_edge(u, v, color);
  }
}

int main() {
  read_data();
  t.decompose(1);
  printf("%d\n", longest_path);

  return 0;
```

```
}
```

Anexa 21

Elemente de teoria numerelor

21.1 Suma unei progresii aritmetice

înapoi

```cpp
// Date fiind a și n, calculează 1 + a + a^2 + ... + a^{n-1} modulo MOD.
#include <assert.h>
#include <stdio.h>

const int MAX_A = 100;
const int MAX_N = 100;
const int MOD = 1'000'000'007;

int prog_sum_naive(int a, int n) {
  long long sum = 0;
  long long factor = 1;
  for (int p = 0; p < n; p++) {
    sum += factor;
    factor = factor * a % MOD;
  }
  sum %= MOD;
  return sum;
}

int prog_sum_iterative(int a, int n) {
  int num_bits = 32 - __builtin_clz(n);
  long long sum = 0;
  long long ak = 1;

  for (int i = 0; i < num_bits; i++) {
    // Contract: fie k numărul format din primii i biți ai lui n.
    // Atunci la această linie ak = a^k și sum = 1 + a + ... + a^{k-1}.
    // Din vechiul ak (deci a^{k/2}) calculăm noul sum.
    sum = (1 + ak) * sum % MOD;
    ak = ak * ak % MOD;
```

```
    int bit = (n >> (num_bits - i - 1)) & 1;
    if (bit) {
      sum = (1 + a * sum) % MOD;
      ak = ak * a % MOD;
    }
  }

  return sum;
}

// La fel ca prog_sum_iterative(), dar schimbă direcția lui i pentru cod puțin
// mai scurt.
int prog_sum_iterative_2(int a, int n) {
  int num_bits = 32 - __builtin_clz(n);
  long long sum = 0;
  long long ak = 1;

  for (int i = num_bits - 1; i >= 0; i--) {
    sum = (1 + ak) * sum % MOD;
    ak = ak * ak % MOD;

    if ((n >> i) & 1) {
      sum = (1 + a * sum) % MOD;
      ak = ak * a % MOD;
    }
  }

  return sum;
}

int bin_exp(int b, int e) {
  int result = 1;

  while (e) {
    if (e & 1) {
      result = (long long)result * b % MOD;
    }
    b = (long long)b * b % MOD;
    e >>= 1;
  }

  return result;
}

int prog_sum_recursive_slow(int a, int n) {
  if (n == 1) {
    return 1;
  }
```

```
  if (n & 1) {
    int rec = prog_sum_recursive_slow(a, n - 1);
    return (1 + (long long) a * rec) % MOD;
  } else {
    long long mult = 1 + bin_exp(a, n / 2);
    return mult * prog_sum_recursive_slow(a, n / 2) % MOD;
  }
}

struct prog {
  long long term; // stochează a^n pentru un n oarecare
  long long sum;  // stochează 1 + a + ... + a^{n-1} pentru același n
};

prog prog_sum_recursive(int a, int n) {
  if (n == 1) {
    return { .term = a, .sum = 1 };
  }

  if (n & 1) {
    prog p = prog_sum_recursive(a, n - 1);
    return {
      p.term * a % MOD,
      (1 + a * p.sum) % MOD,
    };
  } else {
    prog p = prog_sum_recursive(a, n / 2);
    return {
      p.term * p.term % MOD,
      (1 + p.term) * p.sum % MOD,
    };
  }
}

int main() {
  for (int a = 1; a <= MAX_A; a++) {
    for (int n = 1; n <= MAX_N; n++) {
      int naive = prog_sum_naive(a, n);
      assert(naive == prog_sum_iterative(a, n));
      assert(naive == prog_sum_iterative_2(a, n));
      assert(naive == prog_sum_recursive_slow(a, n));
      assert(naive == prog_sum_recursive(a, n).sum);
    }
  }

  return 0;
}
```

21.2 Problema Exponentiation (CSES)

înapoi

```c
#include <stdio.h>

const int MOD = 1'000'000'007;

int bin_exp(int b, int e) {
  int result = 1;

  while (e) {
    if (e & 1) {
      result = (long long)result * b % MOD;
    }
    b = (long long)b * b % MOD;
    e >>= 1;
  }

  return result;
}

int main() {
  int n, a, b;
  scanf("%d", &n);
  while (n--) {
    scanf("%d %d", &a, &b);
    printf("%d\n", bin_exp(a, b));
  }
}
```

21.3 Problema Exponentiation II (CSES)

înapoi

```c
#include <stdio.h>

const int MOD = 1'000'000'007;

int bin_exp(int b, int e, int mod) {
  int result = 1;

  while (e) {
    if (e & 1) {
      result = (long long)result * b % mod;
    }
    b = (long long)b * b % mod;
    e >>= 1;
```

```
  }

  return result;
}

int main() {
  int n, a, b, c;
  scanf("%d", &n);
  while (n--) {
    scanf("%d %d %d", &a, &b, &c);
    int e = bin_exp(b, c, MOD - 1);
    int answer = bin_exp(a, e, MOD);
    printf("%d\n", answer);
  }
}
```

21.4 Problema Eugene and Big Number (CodeChef)

‹ înapoi • versiune online

```
#include <stdio.h>

// get_decimal_mask(123) = 1000
long long get_decimal_mask(int a) {
  long long d = 1;
  while (a) {
    d *= 10;
    a /= 10;
  }
  return d;
}

int prog_sum(long long a, long long n, int mod) {
  int num_bits = 64 - __builtin_clzll(n);
  long long sum = 0;
  long long ak = 1;

  for (int i = 0; i < num_bits; i++) {
    sum = (1 + ak) * sum % mod;
    ak = ak * ak % mod;

    int bit = (n >> (num_bits - i - 1)) & 1;
    if (bit) {
      sum = (1 + a * sum) % mod;
      ak = ak * a % mod;
    }
  }
```

```
  return sum;
}

int process_test(int a, long long n, int mod) {
  long long d = get_decimal_mask(a);
  int psum = prog_sum(d, n, mod);
  return (long long)a * psum % mod;
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    int a, mod;
    long long n;
    scanf("%d %lld %d", &a, &n, &mod);
    printf("%d\n", process_test(a, n, mod));
  }

  return 0;
}
```

21.5 Problema Counting Divisors (CSES)

înapoi

```
#include <stdio.h>

const int MAX_VAL = 1'000'000;
unsigned char num_div[MAX_VAL + 1];

void sieve() {
  for (int div = 1; div <= MAX_VAL; div++) {
    for (int mult = div; mult <= MAX_VAL; mult += div) {
      num_div[mult]++;
    }
  }
}

void process_queries() {
  int num_tests, x;

  scanf("%d", &num_tests);
  while (num_tests--) {
    scanf("%d", &x);
    printf("%d\n", num_div[x]);
  }
}
```

```
int main() {
  sieve();
  process_queries();

  return 0;
}
```

21.6 Problema Common Divisors (CSES)

înapoi

```
#include <stdio.h>

const int MAX_VAL = 1'000'000;

int freq[MAX_VAL + 1];

void read_data() {
  int n, x;

  scanf("%d", &n);
  while (n--) {
    scanf("%d", &x);
    freq[x]++;
  }
}

int count_multiples(int div) {
  int result = 0;

  for (int mult = div; mult <= MAX_VAL; mult += div) {
    result += freq[mult];
  }

  return result;
}

int get_highest_gcd() {
  int div = MAX_VAL;
  while (count_multiples(div) < 2) {
    div--;
  }
  return div;
}

int main() {
```

```
  read_data();
  int answer = get_highest_gcd();
  printf("%d\n", answer);

  return 0;
}
```

21.7 Problema Sum of Divisors (CSES)

înapoi

```
#include <math.h>
#include <stdio.h>

const int MOD = 1'000'000'007;
const int HALF = (1 + MOD) / 2;

int main() {
  long long n;
  scanf("%lld", &n);
  int root = sqrt(n);

  // Pînă la sqrt(n)
  long long result = 0;
  for (int d = 1; n / d > root; d++) {
    // printf("Ia divizorul %d de %lld ori\n", d, n / d);
    result = (result + n / d * d) % MOD;
  }

  // Dincolo de sqrt(n)
  for (int k = root; k >= 1; k--) {
    long long min_d = n / (k + 1) + 1;
    long long max_d = n / k;

    long long gauss = (min_d + max_d) % MOD *
      ((max_d - min_d + 1) % MOD) % MOD *
      HALF % MOD;

    result = (result + gauss * k) % MOD;
    // printf("Ia divizorii de la %lld la %lld de cîte %d ori\n", min_d, max_d, k);
  }

  printf("%lld\n", result);
  return 0;
}
```

21.8 Problema Prime Multiples (CSES)

‹ înapoi

Sursă cu pinex implementat iterativ.

```
#include <stdio.h>
#include <stdlib.h>

const int MAX_DIV = 20;

long long n, d[MAX_DIV];
int num_div;

void read_data() {
  scanf("%lld %d", &n, &num_div);
  for (int i = 0; i < num_div; i++) {
    scanf("%lld", &d[i]);
  }
}

long long make_product(long long mask) {
  long long prod = -1;
  bool fits = true;

  for (int bit = 0; bit < num_div; bit++) {
    if (mask & (1 << bit)) {
      fits &= ((__int128)llabs(prod) * d[bit] <= n);
      prod *= -d[bit];
    }
  }

  return fits ? prod : (n + 1);
}

long long pinex() {
  long long result = 0;

  for (int mask = 1; mask < (1 << num_div); mask++) {
    long long prod = make_product(mask);
    result += n / prod;
  }

  return result;
}

void write_output(long long result) {
  printf("%lld\n", result);
}
```

```
int main() {
  read_data();
  long long result = pinex();
  write_output(result);

  return 0;
}
```

Sursă cu pinex implementat recursiv.

```
#include <stdio.h>
#include <stdlib.h>

const int MAX_DIV = 20;

long long n, d[MAX_DIV];
int num_div;

void read_data() {
  scanf("%lld %d", &n, &num_div);
  for (int i = 0; i < num_div; i++) {
    scanf("%lld", &d[i]);
  }
}

long long pinex(int k, long long product_so_far) {
  if (k == num_div) {
    return n / product_so_far;
  }

  long long result = pinex(k + 1, product_so_far);
  if ((__int128)llabs(product_so_far) * d[k] <= n) {
    result += pinex(k + 1, -product_so_far * d[k]);
  }
  return result;
}

void write_output(long long result) {
  printf("%lld\n", result);
}

int main() {
  read_data();
  long long result = n - pinex(0, 1);
  write_output(result);

  return 0;
}
```

21.9 Problema List of Integers (Codeforces)

‹ înapoi

Sursă cu factorizare completă (versiune online).

```
#include <stdio.h>

const int MAX_N = 1'000'000;
// 2 * 3 * 5 * 7 * 11 * 13 * 17 * 19 > MAX_N
const int MAX_PRIME_FACTORS = 7;
// Răspunsul pentru 1'000'000 510'510 1'000'000 este 6,5 M
const int INFINITY = 7'000'000;

int spf[MAX_N + 1];  // cel mai mic factor prim, e.g. spf[225] = 3
int rest[MAX_N + 1]; // restul după eliminarea factorilor egali, e.g. rest[225] = 25

void sieve() {
  for (int i = 2; i <= MAX_N; i++) {
    if (!spf[i]) {
      for (int j = 1, k = i; k <= MAX_N; j++, k += i) {
        if (!spf[k]) {
          spf[k] = i;
          rest[k] = j;
          while (rest[k] % i == 0) {
            rest[k] /= i;
          }
        }
      }
    }
  }
}

struct worker {
  int factors[MAX_PRIME_FACTORS];
  int num_factors;

  void factorize(int p) {
    num_factors = 0;
    while (p > 1) {
      factors[num_factors++] = spf[p];
      p = rest[p];
    }
  }

  int pinex(int up_to, int level, int product_so_far) {
    if (level == num_factors) {
      return up_to / product_so_far;
    }
```

```
    return
      pinex(up_to, level + 1, product_so_far) +
      pinex(up_to, level + 1, -product_so_far * factors[level]);
  }

  int count_coprimes(int up_to) {
    return pinex(up_to, 0, 1);
  }

  int find_kth_coprime(int k) {
    int l = 0, r = INFINITY; // (l exclusiv, r inclusiv]
    while (r - l > 1) {
      int mid = (l + r) / 2;
      if (count_coprimes(mid) < k) {
        l = mid;
      } else {
        r = mid;
      }
    }
    return r;
  }
};

void solve_test() {
  int x, p, k;
  scanf("%d %d %d", &x, &p, &k);

  worker w;
  w.factorize(p);
  int up_to_x = w.count_coprimes(x);
  int answer = w.find_kth_coprime(k + up_to_x);
  printf("%d\n", answer);
}

int main() {
  sieve();

  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

Sursă cu extragere de divizori (versiune online).

```
#include <stdio.h>

const int MAX_N = 1'000'000;
// 2 * 3 * 5 * 7 * 11 * 13 * 17 * 19 > MAX_N
const int MAX_PRIME_FACTORS = 7;
// Răspunsul pentru 1'000'000 510'510 1'000'000 este 6,5 M
const int INFINITY = 7'000'000;

int spf[MAX_N + 1];  // cel mai mic factor prim, e.g. spf[225] = 3
int rest[MAX_N + 1]; // restul după eliminarea factorilor egali, e.g. rest[225] = 25

void sieve() {
  for (int i = 2; i <= MAX_N; i++) {
    if (!spf[i]) {
      for (int j = 1, k = i; k <= MAX_N; j++, k += i) {
        if (!spf[k]) {
          spf[k] = i;
          rest[k] = j;
          while (rest[k] % i == 0) {
            rest[k] /= i;
          }
        }
      }
    }
  }
}

int pinex(int up_to, int remaining, int product_so_far) {
  if (remaining == 1) {
    return up_to / product_so_far;
  }

  return
      pinex(up_to, rest[remaining], product_so_far) +
      pinex(up_to, rest[remaining], -product_so_far * spf[remaining]);
}

int count_coprimes(int p, int up_to) {
  return pinex(up_to, p, 1);
}

int find_kth_coprime(int p, int k) {
  int l = 0, r = INFINITY; // (l exclusiv, r inclusiv]
  while (r - l > 1) {
    int mid = (l + r) / 2;
    if (count_coprimes(p, mid) < k) {
      l = mid;
    } else {
      r = mid;
```

```
    }
  }
  return r;
}

void solve_test() {
  int x, p, k;
  scanf("%d %d %d", &x, &p, &k);

  int up_to_x = count_coprimes(p, x);
  int answer = find_kth_coprime(p, k + up_to_x);
  printf("%d\n", answer);
}

int main() {
  sieve();

  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

21.10 Problema Small GCD (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_VAL = 100'000;

typedef unsigned long long u64;

int freq_gte[MAX_VAL + 2];
u64 triplets_having_f[MAX_VAL + 1];

void reset() {
  for (int i = 0; i <= MAX_VAL; i++) {
    freq_gte[i] = triplets_having_f[i] = 0;
  }
}

void read_frequencies() {
  int n, val;
```

```
  scanf("%d", &n);
  while (n--) {
    scanf("%d", &val);
    freq_gte[val]++;
  }
}

void partial_sum_frequencies() {
  for (int i = MAX_VAL - 1; i >= 0; i--) {
    freq_gte[i] += freq_gte[i + 1];
  }
}

void compute_triplets_having_f() {
  for (int x = MAX_VAL; x; x--) {
    u64& cnt = triplets_having_f[x];
    int left = 0;
    for (int b = x; b <= MAX_VAL; b += x) { // acesta este b din enunț
      int fb = freq_gte[b] - freq_gte[b + 1];
      int right = freq_gte[b + 1];
      cnt -= triplets_having_f[b];
      cnt += (u64)fb * (fb - 1) / 2 * (left + right);
      cnt += (u64)left * right * fb;
      cnt += (u64)fb * (fb - 1) * (fb - 2) / 6;
      left += fb;
    }
  }
}

u64 compute_sum_f() {
  u64 sum = 0;
  for (int i = 1; i <= MAX_VAL; i++) {
    sum += i * triplets_having_f[i];
  }
  return sum;
}

void solve_test() {
  reset();
  read_frequencies();
  partial_sum_frequencies();
  compute_triplets_having_f();
  u64 res = compute_sum_f();
  printf("%llu\n", res);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
```

```
    solve_test();
  }
  return 0;
}
```

21.11 Problema Secvxor (Baraj ONI 2023)

‹ înapoi

Sursă simplă (versiune online).

```
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_VAL = 1'000'000;

// Pentru fiecare x, spf[x] reține cel mai mic factor prim al lui x, iar
// rest[x] reține câtul după eliminarea tuturor factorilor minimi. Pentru a
// economisi memorie și a face spf[] să încapă pe short, prin convenție spf[x]
// = rest[x] = 0 când x este prim.
short spf[MAX_VAL + 1];
int rest[MAX_VAL + 1];

bool multiples[MAX_VAL + 1];
bool right[MAX_N];
int v[MAX_N], n;

void read_input_data() {
  FILE* f = fopen("secvxor.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &v[i]);
  }
  fclose(f);
}

void sieve() {
  for (int p = 2; p * p <= MAX_VAL; p++) {
    if (!spf[p]) {
      spf[p] = p;
      rest[p] = 1;
      for (int k = p, m = k * p; m <= MAX_VAL; k++, m += p) {
        if (!spf[m]) {
          spf[m] = p;
          rest[m] = (spf[k] == p) ? rest[k] : k;
        }
      }
    }
```

```
  }
}

void clear_multiples() {
  for (int i = 1; i <= MAX_VAL; i++) {
    multiples[i] = 0;
  }
}

// Vizitează toți divizorii liberi de pătrate ai lui val. Numără valorile care
// au divizori comuni cu val. Actualizează multiples[] după numărare.
bool count(int val, int prod) {
  if (val == 1) {
    int result = multiples[prod];
    if (prod > 1) {
      multiples[prod] ^= 1;
    }
    return result;
  } else {
    int s = spf[val] ? spf[val] : val;
    int r = rest[val] ? rest[val] : 1;
    return count(r, prod) ^ count(r, prod * s);
  }
}

void count_shared_factors_right() {
  for (int i = n - 1; i >= 0; i--) {
    right[i] = count(v[i], 1);
  }
}

int count_shared_factors_left() {
  int answer = 0;
  bool overlaps = false, left = false;

  for (int i = 0; i < n; i++) {
    overlaps ^= right[i] ^ left;
    answer ^= overlaps ? v[i] : 0;
    left = count(v[i], 1);
  }

  return answer;
}

void write_answer(int answer) {
  FILE* f = fopen("secvxor.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}
```

```
int main() {
  read_input_data();
  sieve();
  count_shared_factors_right();
  clear_multiples();
  int answer = count_shared_factors_left();
  write_answer(answer);

  return 0;
}
```

Sursă optimizată (versiune online).

```
#include <assert.h>
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_VAL = 1'000'000;

// Pentru fiecare x, spf[x] reține cel mai mic factor prim al lui x, iar
// rest[x] reține câtul după eliminarea tuturor factorilor minimi. Pentru a
// economisi memorie și a face spf[] să încapă pe short, prin convenție spf[x]
// = rest[x] = 0 când x este prim.
short spf[MAX_VAL + 1];
int rest[MAX_VAL + 1];

bool mult[MAX_VAL + 1];
bool freq[MAX_VAL + 1];
bool gmult[MAX_VAL + 1];
int v[MAX_N], n;

void read_input_data() {
  FILE* f = fopen("secvxor.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &v[i]);
    freq[v[i]] ^= 1;
  }
  fclose(f);
}

void sieve() {
  for (int p = 2; p * p <= MAX_VAL; p++) {
    if (!spf[p]) {
      spf[p] = p;
      rest[p] = 1;
      for (int k = p, m = k * p; m <= MAX_VAL; k++, m += p) {
        if (!spf[m]) {
          spf[m] = p;
```

```
          rest[m] = (spf[k] == p) ? rest[k] : k;
        }
      }
    }
  }
}

void compute_gmult() {
  for (int d = 2; d <= MAX_VAL; d++) {
    for (int m = d; m <= MAX_VAL; m += d) {
      gmult[d] ^= freq[m];
    }
  }
}

// Vizitează toți divizorii liberi de pătrate ai lui val. Numără valorile la
// stînga și la dreapta lui val care au divizori comuni cu val. Actualizează
// mult[] după numărare.
void count(int val, int prod, bool& left, bool& right) {
  if (val == 1) {
    left ^= mult[prod];
    if (prod > 1) {
      mult[prod] ^= 1;
    }
    right ^= gmult[prod] ^ mult[prod];
  } else {
    int s = spf[val] ? spf[val] : val;
    int r = rest[val] ? rest[val] : 1;
    count(r, prod, left, right);
    count(r, prod * s, left, right);
  }
}

int count_sequences() {
  int answer = 0;
  bool overlaps = false, prev_left = false;

  for (int i = 0; i < n; i++) {
    bool left = false, right = false;
    count(v[i], 1, left, right);
    overlaps ^= prev_left ^ right;
    answer ^= overlaps ? v[i] : 0;
    prev_left = left;
  }

  return answer;
}

void write_answer(int answer) {
  FILE* f = fopen("secvxor.out", "w");
```

```
  fprintf(f, "%d\n", answer);
  fclose(f);
}

int main() {
  read_input_data();
  sieve();
  compute_gmult();
  int answer = count_sequences();
  write_answer(answer);

  return 0;
}
```

21.12 Problema Afacere (Lot 2009)

‹ înapoi

Sursă simplă (versiune online).

```
// Complexitate: O(2^n * l + num_ext * l).
#include <stdio.h>

const int MAX_WORDS = 20;
const int MAX_LEN = 100;
const int MAX_EXTENSIONS = 100;
const int SIGMA = 26;
const int MOD = 1'000'000'007;

struct pie_data {
  char s[MAX_LEN + 1];
  int num_question_marks;

  void init(int len) {
    for (int i = 0; i < len; i++) {
      s[i] = '?';
    }
    num_question_marks = len;
  }

  // Adaugă un wildcard la setul curent de restricții.
  bool combine(pie_data src, char* w) {
    num_question_marks = src.num_question_marks;

    bool compatible = true;
    int i = 0;

    while (w[i] && compatible) {
```

```
      if (w[i] == '?') {
        // Nimic nu se schimbă. Dacă s[i] era '?' rămîne '?', dacă era impus
        // rămîne impus.
        s[i] = src.s[i];
      } else if (src.s[i] == '?') {
        // Pierdem o libertate pe coloana i: aveam '?', dar a venit o literă.
        s[i] = w[i];
        num_question_marks--;
      } else {
        s[i] = w[i];
        compatible = (src.s[i] == w[i]);
      }
      i++;
    }
    return compatible;
  }

};

char s[MAX_WORDS][MAX_LEN + 1];
int ext[MAX_EXTENSIONS];
int coefs[MAX_LEN + 1];
pie_data pd; // datele inițiale pentru pinex
int num_words, len, num_ext;

void read_data() {
  FILE* f = fopen("afacere.in", "r");
  fscanf(f, "%d %d %d", &num_words, &len, &num_ext);
  for (int i = 0; i < num_words; i++) {
    fscanf(f, "%s", s[i]);
  }
  for (int i = 0; i < num_ext; i++) {
    fscanf(f, "%d", &ext[i]);
  }
  fclose(f);
}

void pie_coefs(int k, int sign, pie_data& pd) {
  if (k == num_words) {
    coefs[pd.num_question_marks] += sign;
  } else {
    pie_coefs(k + 1, sign, pd);
    pie_data pd2;
    if (pd2.combine(pd, s[k])) {
      pie_coefs(k + 1, -sign, pd2);
    }
  }
}

int evaluate(int sigma) {
```

```
  long long poly_val = 0;
  long long sigma_pow = 1;
  for (int i = 0; i <= len; i++) {
    poly_val = (poly_val + sigma_pow * coefs[i]) % MOD;
    sigma_pow = sigma_pow * sigma % MOD;
  }
  return poly_val;
}

void evaluate_and_write() {
  FILE* f = fopen("afacere.out", "w");
  for (int i = 0; i < num_ext; i++) {
    fprintf(f, "%d\n", evaluate(SIGMA + ext[i]));
  }
  fclose(f);
}

int main() {
  read_data();
  pd.init(len);
  pie_coefs(0, -1, pd);
  coefs[len]++; // Compensează pentru submulțimea vidă.
  evaluate_and_write();

  return 0;
}
```

Sursă optimizată (versiune online).

```
// Complexitate: O(n^2 * l + 2^n + num_ext * l).
#include <stdio.h>

const int MAX_WORDS = 20;
const int MAX_LEN = 100;
const int MAX_EXTENSIONS = 100;
const int SIGMA = 26;
const int MOD = 1'000'000'007;

typedef unsigned __int128 u128;

struct wildcard {
  char s[MAX_LEN + 1];
  int compat;  // mască de compatibilitate cu celelalte wildcards
  u128 qmask;  // mască cu pozițiile caracterelor '?'
};

int ext[MAX_EXTENSIONS];
int coefs[MAX_LEN + 1];
wildcard w[MAX_WORDS];
```

```
int num_words, len, num_ext;

void read_data() {
  FILE* f = fopen("afacere.in", "r");
  fscanf(f, "%d %d %d", &num_words, &len, &num_ext);
  for (int i = 0; i < num_words; i++) {
    fscanf(f, "%s", w[i].s);
  }
  for (int i = 0; i < num_ext; i++) {
    fscanf(f, "%d", &ext[i]);
  }
  fclose(f);
}

void precompute_qmask() {
  for (int i = 0; i < num_words; i++) {
    for (int j = 0; j < len; j++) {
      if (w[i].s[j] == '?') {
        w[i].qmask |= (u128)1 << j;
      }
    }
  }
}

bool compatible_symbols(char a, char b) {
  return (a == '?') || (b == '?') || (a == b);
}

void precompute_compat() {
  for (int i = 0; i < num_words - 1; i++) {
    for (int j = i + 1; j < num_words; j++) {
      int k = 0;
      while ((k < len) && compatible_symbols(w[i].s[k], w[j].s[k])) {
        k++;
      }

      if (k == len) {
        w[i].compat |= 1 << j;
        w[j].compat |= 1 << i;
      }
    }
  }
}

int popcount(u128 x) {
  return
    __builtin_popcountll(x & 0xffffffffffffffffull) +
    __builtin_popcountll(x >> 64);
}
```

```
void pie_coefs(int k, int sign, int wmask, u128 qmask) {
  if (k == num_words) {
    coefs[popcount(qmask)] += sign;
  } else {
    pie_coefs(k + 1, sign, wmask, qmask);
    if ((wmask & w[k].compat) == wmask) {
      pie_coefs(k + 1, -sign, wmask | (1 << k), qmask & w[k].qmask);
    }
  }
}

int evaluate(int sigma) {
  long long poly_val = 0;
  long long sigma_pow = 1;
  for (int i = 0; i <= len; i++) {
    poly_val = (poly_val + sigma_pow * coefs[i]) % MOD;
    sigma_pow = sigma_pow * sigma % MOD;
  }
  return poly_val;
}

void evaluate_and_write() {
  FILE* f = fopen("afacere.out", "w");
  for (int i = 0; i < num_ext; i++) {
    fprintf(f, "%d\n", evaluate(SIGMA + ext[i]));
  }
  fclose(f);
}

int main() {
  read_data();
  precompute_qmask();
  precompute_compat();
  pie_coefs(0, -1, 0, ((u128)1 << len) - 1);
  coefs[len]++; // Compensează pentru submulțimea vidă.
  evaluate_and_write();

  return 0;
}
```

21.13 Problema Playing with GCD (SPOJ)

‹ înapoi

```
#include <stdio.h>

const int MAX_N = 100'000;
```

```
int phi[MAX_N + 1];
long long answer[MAX_N + 1];

void compute_all_phi() {
  for (int i = 0; i <= MAX_N; i++) {
    phi[i] = i;
  }

  for (int i = 2; i <= MAX_N; i++) {
    if (phi[i] == i) {
      for (int j = i; j <= MAX_N; j += i) {
        phi[j] -= phi[j] / i;
      }
    }
  }
}

void compute_all_answers() {
  for (int i = 1; i <= MAX_N; i++) {
    answer[i] = answer[i - 1] + (i - phi[i]);
  }
}

void serve_answers() {
  int num_queries, n;
  scanf("%d", &num_queries);
  for (int q = 1; q <= num_queries; q++) {
    scanf("%d", &n);
    printf("Case %d: %lld\n", q, answer[n]);
  }
}

int main() {
  compute_all_phi();
  compute_all_answers();
  serve_answers();

  return 0;
}
```

21.14 Problema Power Tower (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_MODULI = 60; // 2 * log_2{MAX_VAL}
```

```
int w[MAX_N + 1];
int mod[MAX_MODULI];
int n, last_mod;

void read_array() {
  scanf("%d %d", &n, &mod[0]);
  for (int i = 1; i <= n; i++) {
    scanf("%d", &w[i]);
  }
}

int euler_totient(int n) {
  int phi = n;
  for (int d = 2; d * d <= n; d++) {
    if (n % d == 0) {
      phi = phi / d * (d - 1);
      while (n % d == 0) {
        n /= d;
      }
    }
  }
  if (n > 1) {
    phi = phi / n * (n - 1);
  }
  return phi;
}

void compute_moduli() {
  last_mod = 0;
  while (mod[last_mod] != 1) {
    ++last_mod;
    mod[last_mod] = euler_totient(mod[last_mod - 1]);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int dirty_mod(long long x, int m) {
  return (x > m) ? (m + x % m) : x;
}

int bin_exp(int b, int e, int mod) {
  long long result = 1;

  while (e) {
    if (e & 1) {
      result = dirty_mod(result * b, mod);
```

```
    }
    b = dirty_mod((long long)b * b, mod);
    e >>= 1;
  }

  return result;
}

int query(int l, int r) {
  int end = min(l + last_mod - 1, r - 1);
  int res = w[end + 1];

  for (int i = end; i >= l; i--) {
    int reduced = dirty_mod(res, mod[i - l + 1]);
    res = bin_exp(w[i], reduced, mod[i - l]);
  }
  return res % mod[0];
}

void process_queries() {
  int num_queries, l, r;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &l, &r);
    printf("%d\n", query(l, r));
  }
}

int main() {
  read_array();
  compute_moduli();
  process_queries();

  return 0;
}
```

21.15 Problema GCD Extreme (SPOJ)

‹ înapoi

```
// Complexitate: O(n log n).
#include <stdio.h>

const int MAX_N = 1'000'000;

int phi[MAX_N + 1];
long long answer[MAX_N + 1];
```

```
void compute_all_phi() {
  for (int i = 0; i <= MAX_N; i++) {
    phi[i] = i;
  }

  for (int i = 2; i <= MAX_N; i++) {
    if (phi[i] == i) {
      for (int j = i; j <= MAX_N; j += i) {
        phi[j] -= phi[j] / i;
      }
    }
  }
}

void compute_all_answers() {
  for (int gcd = 1; gcd <= MAX_N; gcd++) {
    for (int k = 2; k * gcd <= MAX_N; k++) {
      answer[k * gcd] += gcd * phi[k];
    }
  }

  for (int col = 2; col <= MAX_N; col++) {
    answer[col] += answer[col - 1];
  }
}

void serve_answers() {
  int n;
  while (scanf("%d", &n) && n) {
    printf("%lld\n", answer[n]);
  }
}

int main() {
  compute_all_phi();
  compute_all_answers();
  serve_answers();

  return 0;
}
```

21.16 Problema Counting Coprime Pairs (CSES)

‹ înapoi

Sursă cu pinex.

```
#include <stdio.h>
```

```
const int MAX_VAL = 1'000'000;
const int MAX_FACTORS = 7;

// spf[x] = cel mai mic divizor al lui x
int spf[MAX_VAL + 1];

// num_mult[x] = numărul de multipli de x întîlniți pînă acum
int num_mult[MAX_VAL + 1];

// factorizarea numărului în curs de procesare
int factors[MAX_FACTORS], num_factors;

void compute_smallest_factors() {
  spf[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (!spf[i])  {
      for (int j = i; j <= MAX_VAL; j += i) {
        if (!spf[j]) {
          spf[j] = i;
        }
      }
    }
  }
}

void factorize(int x) {
  num_factors = 0;
  while (x != 1) {
    if (!num_factors || factors[num_factors - 1] != spf[x]) {
      factors[num_factors++] = spf[x];
    }
    x /= spf[x];
  }
}

// Dată fiind factorizarea unui număr liber de pătrate, numără cîte perechi
// coprime formează cu numere anterioare. Folosește pinex. De exemplu, dacă
// numărul este 14, el va fi coprim cu
//
// * toate numerele anterioare,
// * minus multiplii de 2 anteriori,
// * minus multiplii de 7 anteriori,
// * plus multiplii de 14 anteriori.
//
// Incrementează contoarele pentru fiecare dintre divizorii numărului.
long long pinex(int k, int sign, int prod_so_far) {
  if (k == num_factors) {
    return sign * num_mult[prod_so_far]++; // numără, apoi adaugă.
  }
```

```
  return
    pinex(k + 1, sign, prod_so_far) +
    pinex(k + 1, -sign, prod_so_far * factors[k]);
}

long long count_coprime_pairs() {
  long long result = 0;
  int n, x;
  scanf("%d", &n);

  while (n--) {
    scanf("%d", &x);
    factorize(x);
    result += pinex(0, +1, 1);
  }

  return result;
}

void write_answer(long long answer) {
  printf("%lld\n", answer);
}

int main() {
  compute_smallest_factors();
  long long answer = count_coprime_pairs();
  write_answer(answer);

  return 0;
}
```

Sursă cu Funcția Möbius (metoda 2).

```
#include <stdio.h>

const int MAX_VAL = 1'000'000;

// spf[x] = cel mai mic divizor al lui x
int spf[MAX_VAL + 1];
int mobius[MAX_VAL + 1];
int freq[MAX_VAL + 1];

void read_data() {
  int n, x;
  scanf("%d", &n);
  while (n--) {
    scanf("%d", &x);
    freq[x]++;
```

```
  }
}

void compute_smallest_factors() {
  spf[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (!spf[i])  {
      for (int j = i; j <= MAX_VAL; j += i) {
        if (!spf[j]) {
          spf[j] = i;
        }
      }
    }
  }
}

void compute_mobius() {
  mobius[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (spf[i] == spf[i / spf[i]]) {
      mobius[i] = 0;
    } else {
      mobius[i] = -mobius[i / spf[i]];
    }
  }
}

int count_multiples(int d) {
  int result = 0;
  for (int i = d; i <= MAX_VAL; i += d) {
    result += freq[i];
  }
  return result;
}

long long count_coprime_pairs() {
  long long result = 0;

  for (int d = 1; d <= MAX_VAL; d++) {
    if (mobius[d]) { // optimizare de viteză
      long long num_mult = count_multiples(d);
      result += mobius[d] * num_mult * (num_mult - 1) / 2;
    }
  }

  return result;
}

void write_answer(long long answer) {
  printf("%lld\n", answer);
```

```
}

int main() {
  read_data();
  compute_smallest_factors();
  compute_mobius();
  long long answer = count_coprime_pairs();
  write_answer(answer);

  return 0;
}
```

21.17 Problema Coprime Sequences (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_VAL = 100'000;
const int MOD = 1'000'000'007;
const int MAX_ROOT = 512;
const int ROOT_BITS = 9;

int spf[MAX_VAL + 1];
int mobius[MAX_VAL + 1];
int freq[MAX_VAL + 1];

// pow2: 2^0, 2^1, 2^2, ..., 2^511
// pow2_large: 2^0, 2^512, 2^1024, 2^1536, ...
int pow2[MAX_ROOT], pow2_large[MAX_ROOT];

void read_data() {
  int n, x;
  scanf("%d", &n);
  while (n--) {
    scanf("%d", &x);
    freq[x]++;
  }
}

void precompute_powers_of_2() {
  pow2[0] = 1;
  for (int i = 1; i < MAX_ROOT; i++) {
    pow2[i] = pow2[i - 1] * 2 % MOD;
  }

  pow2_large[0] = 1;
  pow2_large[1] = (2 * pow2[MAX_ROOT - 1]) % MOD;
```

```
  for (int i = 2; i < MAX_ROOT; i++) {
    pow2_large[i] = (long long)pow2_large[i - 1] * pow2_large[1] % MOD;
  }
}

int power_of_2(int e) {
  long long large = pow2_large[e >> ROOT_BITS];
  long long small = pow2[e & (MAX_ROOT - 1)];
  return large * small % MOD;
}

void compute_smallest_factors() {
  spf[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (!spf[i])  {
      for (int j = i; j <= MAX_VAL; j += i) {
        if (!spf[j]) {
          spf[j] = i;
        }
      }
    }
  }
}

void compute_mobius() {
  mobius[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (spf[i] == spf[i / spf[i]]) {
      mobius[i] = 0;
    } else {
      mobius[i] = -mobius[i / spf[i]];
    }
  }
}

int count_multiples(int d) {
  int result = 0;
  for (int i = d; i <= MAX_VAL; i += d) {
    result += freq[i];
  }
  return result;
}

int count_coprime_subsets() {
  long long result = 0;
  for (int d = 1; d <= MAX_VAL; d++) {
    int num_mult = count_multiples(d);
    result += MOD + mobius[d] * (power_of_2(num_mult) - 1);
  }
  return result % MOD;
```

```
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  precompute_powers_of_2();
  compute_smallest_factors();
  compute_mobius();
  int answer = count_coprime_subsets();
  write_answer(answer);

  return 0;
}
```

21.18 Problema Coprime Triples (CodeChef)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_VAL = 1'000'000;

int factor[MAX_VAL + 1];
char u[MAX_VAL + 1];

// multiples[x] = cîte numere de la intrare se divid cu x
int multiples[MAX_VAL + 1];

void compute_smallest_factors() {
  factor[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (!factor[i])  {
      for (int j = i; j <= MAX_VAL; j += i) {
        if (!factor[j]) {
          factor[j] = i;
        }
      }
    }
  }
}

void compute_mobius() {
  u[1] = 1;
  for (int i = 2; i <= MAX_VAL; i++) {
    if (factor[i] == factor[i / factor[i]]) {
```

```
      u[i] = 0;
    } else {
      u[i] = -u[i / factor[i]];
    }
  }
}

void read_data() {
  int n, x;

  scanf("%d", &n);
  while (n--) {
    scanf("%d", &x);
    multiples[x]++;
  }
}

void compute_multiples() {
  for (int d = 1; d <= MAX_VAL; d++) {
    for (int m = 2 * d; m <= MAX_VAL; m += d) {
      multiples[d] += multiples[m];
    }
  }
}

void count_coprime_triplets() {
  long long answer = 0;
  for (int gcd = 1; gcd <= MAX_VAL; gcd++) {
    if (u[gcd]) {
      int m = multiples[gcd];
      long long comb = (long long)m * (m - 1) * (m - 2) / 6;
      answer += (long long)u[gcd] * comb;
    }
  }
  printf("%lld\n", answer);
}

int main() {
  compute_smallest_factors();
  compute_mobius();
  read_data();
  compute_multiples();
  count_coprime_triplets();

  return 0;
}
```

21.19 Problema Coprime Arrays (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_VAL = 2'000'000;
const int MOD = 1'000'000'007;

int mu[MAX_VAL + 1];
int pow_n[MAX_VAL + 1];
long long changes[MAX_VAL + 1];
int n, k;

void read_data() {
  scanf("%d %d", &n, &k);
}

int bin_exp(int b, int e) {
  long long result = 1;

  while (e) {
    if (e & 1) {
      result = result * b % MOD;
    }
    b = (long long)b * b % MOD;
    e >>= 1;
  }

  return result;
}

void compute_powers() {
  for (int i = 1; i <= k; i++) {
    pow_n[i] = bin_exp(i, n);
  }
}

void compute_mobius(int n) {
  for (int i = 1; i <= n; i++) {
    mu[i] = 1;
  }

  for (int i = 2; i * i <= n; i++) {
    if (mu[i] == 1) {
      for (int j = i; j <= n; j += i) {
        mu[j] *= -i;
      }
      for (int j = i * i; j <= n; j += i * i) {
```

```
        mu[j] = 0;
      }
    }
  }

  for (int i = 2; i <= n; i++) {
    if (mu[i] == i) {
      mu[i] = 1;
    } else if (mu[i] == -i) {
      mu[i] = -1;
    } else if (mu[i] < 0) {
      mu[i] = 1;
    } else if (mu[i] > 0) {
      mu[i] = -1;
    }
  }
}

void compute_changes() {
  for (int div = 1; div <= k; div++) {
    for (int quot = 1, mult = div; mult <= k; quot++, mult += div) {
      changes[mult] += (pow_n[quot] - pow_n[quot - 1]) * mu[div];
    }
  }
}

void count_arrays() {
  long long cnt = 0;
  long long answer = 0;
  for (int i = 1; i <= k; i++) {
    cnt = (cnt + changes[i]) % MOD;
    if (cnt < 0) {
      cnt = MOD + cnt;
    }

    answer += cnt ^ i;
  }
  printf("%lld\n", answer % MOD);
}

int main() {
  read_data();
  compute_powers();
  compute_mobius(k);
  compute_changes();
  count_arrays();

  return 0;
}
```

21.20 Problema Mocha and Stars (Codeforces)

‹ înapoi • versiune online

```cpp
#include <stdio.h>

const int MAX_N = 50;
const int MAX_VAL = 100'000;
const int MOD = 998'244'353;

struct knapsack {
  int v[2][MAX_VAL + 1];
  int row;
  int max_val;

  void clear_row(int r) {
    for (int i = 0; i <= max_val; i++) {
      v[r][i] = 0;
    }
  }

  void partial_sums(int r) {
    for (int i = 1; i <= max_val; i++) {
      v[r][i] = (v[r][i] + v[r][i - 1]) % MOD;
    }
  }

  void init(int _max_val) {
    row = 0;
    max_val = _max_val;
    clear_row(0);
    v[0][0] = 1;
  }

  void add_range(int lo, int hi) {
    clear_row(1 - row);

    // Construiește vectorul de diferențe.
    for (int i = 0; i <= max_val - lo; i++) {
      v[1 - row][i + lo] = (v[1 - row][i + lo] + v[row][i]) % MOD;
      if (i + hi + 1 <= max_val) {
        v[1 - row][i + hi + 1] = (v[1 - row][i + hi + 1] + MOD - v[row][i]) % MOD;
      }
    }

    partial_sums(1 - row);
    row = 1 - row;
  }
```

```
  int sum_all() {
    long long sum = 0;
    for (int i = 0; i <= max_val; i++) {
      sum += v[row][i];
    }
    return sum % MOD;
  }
};

int lo[MAX_N], hi[MAX_N];
int mu[MAX_VAL + 1];
knapsack kn;
int n, m;

void read_data() {
  scanf("%d %d", &n, &m);
  for (int i = 0; i < n; i++) {
    scanf("%d %d", &lo[i], &hi[i]);
  }
}

void compute_mobius() {
  mu[1] = 1;
  for (int i = 1; i <= m; i++) {
    if (mu[i]) {
      for (int j = 2 * i; j <= m; j += i) {
        mu[j] -= mu[i];
      }
    }
  }
}

int count_partitions(int d) {
  kn.init(m / d);
  for (int i = 0; i < n; i++) {
    kn.add_range((lo[i] - 1) / d + 1, hi[i] / d);
  }
  return kn.sum_all();
}

int sum_over_divisors() {
  long long answer = 0;

  for (int d = 1; d <= m; d++) {
    if (mu[d]) {
      answer += (long long)mu[d] * count_partitions(d);
    }
  }

  answer %= MOD;
```

```
  if (answer < 0) {
    answer += MOD;
  }

  return answer % MOD;
}

int main() {
  read_data();
  compute_mobius();
  int answer = sum_over_divisors();
  printf("%d\n", answer);

  return 0;
}
```

21.21 Problema Pastile (Baraj ONI 2021)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_VAL = 200'000;
const int MOD = 1'000'000'007;

char mobius[MAX_VAL + 1];

// lte[x] = numărul de nuanțe având ≤ x pastile
int lte[MAX_VAL + 1], min_val, max_val;

int min(int x, int y) {
  return (x < y) ? x :  y;
}

int max(int x, int y) {
  return (x > y) ? x :  y;
}

void read_frequencies() {
  int n, x;
  min_val = MAX_VAL + 1;
  max_val = 0;

  scanf("%d", &n);
  while (n--) {
    scanf("%d", &x);
    lte[x]++;
    min_val = min(min_val, x);
```

```
    max_val = max(max_val, x);
  }
}

void partial_sum_frequencies() {
  for (int i = min_val + 1; i <= max_val; i++) {
    lte[i] += lte[i - 1];
  }
}

// Lentă, dar nediferențiabilă până la MAX_VAL.
void compute_mobius(int n) {
  mobius[1] = 1;
  for (int i = 1; i <= n; i++) {
    if (mobius[i]) {
      for (int j = 2 * i; j <= n; j += i) {
        mobius[j] -= mobius[i];
      }
    }
  }
}

int mod_pow(long long b, int e) {
  long long result = 1;

  while (e) {
    if (e & 1) {
      result = result * b % MOD;
    }
    b = b * b % MOD;
    e >>= 1;
  }

  return result;
}

long long count_combos_having_gcd(int gcd) {
  long long prod = 1;

  // Frecvențele de la 2 × gcd până la 3 × gcd - 1 oferă 2 variante.
  // Frecvențele de la 3 × gcd până la 4 × gcd - 1 oferă 3 variante etc.
  for (int c = 2; c * gcd <= max_val; c++) {
    int start = c * gcd;
    int end = min(start + gcd - 1, max_val);
    int shades = lte[end] - lte[start - 1];
    prod = prod * mod_pow(c, shades) % MOD;
  }

  return prod;
}
```

```
int count_coprime_combos() {
  long long result = 0;
  for (int gcd = 1; gcd <= min_val; gcd++) {
    if (mobius[gcd]) {
      result += count_combos_having_gcd(gcd) * mobius[gcd] + MOD;
    }
  }
  return result % MOD;
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_frequencies();
  partial_sum_frequencies();
  compute_mobius(min_val);
  int answer = count_coprime_combos();
  write_answer(answer);

  return 0;
}
```

21.22 Problema Beautiful Numbers (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MOD = 1'000'000'007;

int fact[MAX_N + 1];
bool is_favorite[10];

void extended_euclid(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
```

```
}

int inverse(int x) {
  // Rezolvă x * y + k * MOD = 1.
  int y, k, d;
  extended_euclid(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

void precompute(int n) {
  fact[0] = 1;
  for (int i = 1; i <= n; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }
}

bool is_beautiful(int x) {
  while (x) {
    if (!is_favorite[x % 10]) {
      return false;
    }
    x /= 10;
  }
  return true;
}

__int128 comb(int n, int k) {
  // n! / [ k! * (n-k)! ]
  int denom = (long long)fact[k] * fact[n - k] % MOD;
  return (__int128)fact[n] * inverse(denom);
}

int iterate_a_and_b(int n, int a, int b) {
  is_favorite[a] = is_favorite[b] = true;
  __int128 result = 0;

  for (int num_a = n, num_b = 0; num_a >= 0; num_a--, num_b++) {
    int sum = a * num_a + b * num_b;
    if (is_beautiful(sum)) {
      // Dacă suma este frumoasă, atunci orice combinație de num_a a-uri și
      // num_b b-uri va fi frumoasă.
      result += comb(n, num_a);
    }
  }

  return result % MOD;
}

int main() {
  int a, b, n;
```

```
  scanf("%d %d %d", &a, &b, &n);
  precompute(n);
  int result = iterate_a_and_b(n, a, b);
  printf("%d\n", result);

  return 0;
}
```

Anexa 22

Teorema chinezească a resturilor

22.1 Problema Koxia and Number Theory (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100;
const int NUM_PRIMES = 15; // până la MAX_N / 2
const int PRIMES[NUM_PRIMES] = {
  2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,
};

int n;
long long a[MAX_N + 1];
int cnt[MAX_N];

void read_array() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%lld", &a[i]);
  }
}

bool has_duplicates() {
  std::sort(a, a + n);
  a[n] = a[n - 1]; // santinelă
  int i = 1;
  while (a[i] != a[i - 1]) {
    i++;
  }
  return (i < n);
}
```

```
bool some_remainder_has_frequency_under_2_mod(int p) {
  for (int i = 0; i < p; i++) {
    cnt[i] = 0;
  }

  int num_good_remainders = p;
  int i = 0;
  while ((i < n) && num_good_remainders) {
    int rem = a[i] % p;
    if (++cnt[rem] == 2) {
      num_good_remainders--;
    }
    i++;
  }

  return num_good_remainders;
}

bool check_all_remainders() {
  int p = 0;

  while ((p < NUM_PRIMES) &&
         some_remainder_has_frequency_under_2_mod(PRIMES[p])) {
    p++;
  }

  return (p == NUM_PRIMES);
}

void solve_test() {
  read_array();
  if (!has_duplicates() && check_all_remainders()) {
    printf("YES\n");
  } else {
    printf("NO\n");
  }
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

22.2 Problema Remainders Game (Codeforces)

‹ înapoi • versiune online

```
// Reimplementată după https://codeforces.com/contest/687/submission/18789803
#include <stdio.h>

int gcd(int a, int b) {
  while (b) {
    int tmp = a;
    a = b;
    b = tmp % b;
  }
  return a;
}

int lcm(int a, int b) {
  return a / gcd(a, b) * b;
}

int main() {
  int n, k, c;
  int minimax = 1;

  scanf("%d %d", &n, &k);
  while (n--) {
    scanf("%d", &c);
    minimax = lcm(minimax, gcd(k, c));
  }

  printf((minimax == k) ? "Yes\n" : "No\n");

  return 0;
}
```

22.3 Problema Decipher the String (Codeforces)

‹ înapoi

Sursă cu baza 26 (versiune online).

```
#include <stdio.h>
#include <string.h>

const int SIGMA = 26;
const int MAX_LENGTH = 10'000;

typedef char str[MAX_LENGTH + 1];
```

```
str orig, t, query, ans[3];
int n;

void read_data() {
  scanf("%s", t);
  n = strlen(t);
}

void build_query(int change_every) {
  int val = -1;
  for (int i = 0; i < n; i++) {
    if (i % change_every == 0) {
      val = (val + 1) % SIGMA;
    }
    query[i] = val + 'a';
  }
}

void build_and_send_query(int change_every, char* ans) {
  build_query(change_every);
  printf("? %s\n", query);
  fflush(stdout);
  scanf(" %s", ans);
}

void restore_orig() {
  for (int i = 0; i < n; i++) {
    int c0 = ans[0][i] - 'a';
    int c1 = ans[1][i] - 'a';
    int c2 = ans[2][i] - 'a';
    int pos = c0 + c1 * SIGMA + c2 * SIGMA * SIGMA;
    orig[pos] = t[i];
  }
}

void write_answer() {
  printf("! %s\n", orig);
}

int main() {
  read_data();
  build_and_send_query(1, ans[0]);
  build_and_send_query(SIGMA, ans[1]);
  build_and_send_query(SIGMA * SIGMA, ans[2]);
  restore_orig();
  write_answer();

  return 0;
```

```
}
```

Sursă cu CRT (versiune online).

```
#include <stdio.h>
#include <string.h>

const int SIGMA = 26;
const int MAX_LENGTH = 10'000;
const int NUM_MODULI = 3;
const int MOD[NUM_MODULI] = { 18, 23, 25 }; // de ce nu?
const int BIG_MOD = 18 * 23 * 25;

typedef char str[MAX_LENGTH + 1];

str orig, t, query, ans[NUM_MODULI];
int n;
int coef[NUM_MODULI], rem[NUM_MODULI];

struct euclid {
  int d, x, y;
};

euclid extended_euclid(int a, int b) {
  euclid e;
  e.x = 1;
  e.y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = e.x - q * xp; e.x = tmp;
    tmp = yp; yp = e.y - q * yp; e.y = tmp;
  }
  e.d = a;
  return e;
}

int inverse(int a, int mod) {
  euclid e = extended_euclid(a, mod);
  return (e.x >= 0) ? e.x : (e.x + mod);
}

// Precalculează partea comună a CRT (coeficientul fiecărui modul).
void precompute() {
  for (int i = 0; i < NUM_MODULI; i++) {
    int prod_exclude = BIG_MOD / MOD[i];
    int inv = inverse(prod_exclude % MOD[i], MOD[i]);
    coef[i] = (long long)prod_exclude * inv % BIG_MOD;
```

```
  }
}

int solve_crt() {
  long long result = 0;
  for (int i = 0; i < NUM_MODULI; i++) {
    result += (long long)rem[i] * coef[i];
  }
  return result % BIG_MOD;
}

void read_data() {
  scanf("%s", t);
  n = strlen(t);
}

void build_query(int modulo) {
  for (int i = 0; i < n; i++) {
    query[i] = i % modulo + 'a';
  }
}

void send_queries() {
  for (int i = 0; i < NUM_MODULI; i++) {
    build_query(MOD[i]);
    printf("? %s\n", query);
    fflush(stdout);
    scanf(" %s", ans[i]);
  }
}

void restore_orig() {
  for (int i = 0; i < n; i++) {
    for (int m = 0; m < NUM_MODULI; m++) {
      rem[m] = ans[m][i] - 'a';
    }
    int pos = solve_crt();
    orig[pos] = t[i];
  }
}

void write_answer() {
  printf("! %s\n", orig);
}

int main() {
  precompute();
  read_data();
  send_queries();
  restore_orig();
```

```
  write_answer();

  return 0;
}
```

22.4 Problema Two Chandeliers (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <numeric>
#include <stdio.h>

const int MAX_N = 500'000;
const int MAX_VAL = 2 * MAX_N;
const int NONE = -1;

struct chandelier {
  int n;
  int c[MAX_N];
};

chandelier p, q;
int pos_in_q[MAX_VAL + 1];
long long overlap[MAX_N];
int num_overlaps;
long long patience;

struct euclid {
  int d, x, y;
};

long long lcm(int a, int b) {
  return (long long)a * b / std::gcd(a, b);
}

euclid extended_euclid(int a, int b) {
  euclid e;
  e.x = 1;
  e.y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = e.x - q * xp; e.x = tmp;
    tmp = yp; yp = e.y - q * yp; e.y = tmp;
  }
  e.d = a;
```

```
  return e;
}

int inverse(int a, int mod) {
  euclid e = extended_euclid(a, mod);
  return (e.x >= 0) ? e.x : (e.x + mod);
}

long long solve_crt(int a, int b) {
  int g = std::gcd(p.n, q.n);
  if ((b - a) % g) {
    return NONE;
  }

  int red_m = p.n / g;
  int red_n = q.n / g;
  int inv = inverse(red_m, red_n);
  long long k = (long long)(b - a) / g * inv;
  long long l = lcm(p.n, q.n);
  long long x = ((k + l)* p.n + a) % l; // tratează și k negativ
  return x;
}

void clear_pos_in_q() {
  for (int val = 0; val <= MAX_VAL; val++) {
    pos_in_q[val] = NONE;
  }
}

void read_data() {
  scanf("%d %d %lld", &p.n, &q.n, &patience);
  patience--;

  for (int i = 0; i < p.n; i++) {
    scanf("%d", &p.c[i]);
  }

  for (int i = 0; i < q.n; i++) {
    scanf("%d", &q.c[i]);
    pos_in_q[q.c[i]] = i;
  }
}

int collect_overlaps() {
  int cnt = 0;
  for (int i = 0; i < p.n; i++) {
    int j = pos_in_q[p.c[i]];
    if (j != NONE) {
      long long day = solve_crt(i, j);
      if (day != NONE) {
```

```
        overlap[num_overlaps++] = day;
      }
    }
  }
  return cnt;
}

long long count_days_last_period(long long patience) {
  if (!num_overlaps) {
    return patience;
  }

  std::sort(overlap, overlap + num_overlaps);

  int i = 0;
  while ((i < num_overlaps) && (patience >= overlap[i] - i)) {
    i++;
  }
  // Aici, i este prima zi de suprapunere la care nu ajungem.

  return patience + i;
}

long long find_kth_patient_day() {
  collect_overlaps();

  long long period = lcm(p.n, q.n);
  long long anger_per_period = period - num_overlaps;
  long long num_periods = patience / anger_per_period;
  patience %= anger_per_period;

  long long days_last_period = count_days_last_period(patience);

  return num_periods * period + days_last_period + 1;
}

void write_answer(long long answer) {
  printf("%lld\n", answer);
}

int main() {
  clear_pos_in_q();
  read_data();
  long long answer = find_kth_patient_day();
  write_answer(answer);

  return 0;
}
```

22.5 Problema GCD Guess (Codeforces)

‹ înapoi

Sursă cu descompunere pe biți (versiune online).

```
#include <stdio.h>

const int NUM_BITS = 30;

unsigned ask(unsigned a, unsigned b) {
  printf("? %u %u\n", a, b);
  fflush(stdout);

  int response;
  scanf("%d", &response);
  return response;
}

bool kth_bit_is_1(int k, int so_far) {
  int gcd = ask((1 << k) - so_far,
                (3 << k) - so_far);
  return gcd % (2 << k) == 0;
}

void solve_test_case() {
  unsigned answer = 0;

  for (int k = 0; k < NUM_BITS; k++) {
    bool bit = kth_bit_is_1(k, answer);
    answer += (bit << k);
  }

  printf("! %u\n", answer);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test_case();
  }

  return 0;
}
```

Sursă cu CRT (versiune online).

```
// Metoda descrisă în https://codeforces.com/blog/entry/101663.
```

```
#include <stdio.h>

// 8 moduli coprimi între ei si produsul lor.
const int NUM_MODULI = 8;
const int MOD[NUM_MODULI] = { 16, 9, 7, 11, 13, 17, 19, 23 };
const int BIG_MOD = 1'070'845'776;

int coef[NUM_MODULI];
int rem[NUM_MODULI];

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

// Folosește algoritmul lui Euclid extins, nu algoritmul lui Fermat, deoarece
// unii dintre moduli sînt compuși.
int inverse(int x, int mod) {
  int y, k, d;
  extended_euclid_iterative(x, mod, d, y, k);
  return (y >= 0) ? y : (y + mod);
}

// Precalculează partea comună a CRT (coeficientul fiecărui modul).
void precompute() {
  for (int i = 0; i < NUM_MODULI; i++) {
    int prod_exclude = BIG_MOD / MOD[i];
    int inv = inverse(prod_exclude, MOD[i]);
    coef[i] = (long long)prod_exclude * inv % BIG_MOD;
  }
}

unsigned ask(unsigned a, unsigned b) {
  printf("? %u %u\n", a, b);
  fflush(stdout);

  int response;
  scanf("%d", &response);
  return response;
}

int solve_crt() {
```

```
  long long result = 0;
  for (int i = 0; i < NUM_MODULI; i++) {
    result += (long long)rem[i] * coef[i];
  }
  return result % BIG_MOD;
}

void solve_test_case() {
  for (int a = 23; a >= 1; a--) {
    int gcd = ask(a, a + BIG_MOD);
    for (int i = 0; i < NUM_MODULI; i++) {
      if (gcd % MOD[i] == 0) {
        // Poate fi negativ pentru valori mari ale lui a, dar vom găsi unul
        // pozitiv mai tîrziu.
        rem[i] = MOD[i] - a;
        // printf("Am aflat că x ≡ %d mod %d\n", rem[i], MOD[i]);
      }
    }
  }

  int solution = solve_crt();
  printf("! %u\n", solution);
}

int main() {
  precompute();

  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test_case();
  }

  return 0;
}
```

Sursă cu CRT optimizată(versiune online).

```
// Implementarea ideii din https://codeforces.com/contest/1665/submission/236496990
#include <stdio.h>

const int NUM_MODULI = 4;

struct congruence {
  int base;
  int exponent;
  int modulus;      // sinonim pentru base^exponent
  int max_exponent; // scopul final
  int rem;          // candidatul curent pentru a satisface x mod base^exponent
```

```
  congruence(int base, int max_exponent) {
    this->base = base;
    this->max_exponent = max_exponent;
  }

  void reset() {
    exponent = 1;
    modulus = base;
    // Testează candidații în ordine descrescătoare ca să evităm cazuri-limită
    // cu 0.
    rem = base - 1;
  }

  void advance(int gcd) {
    // Am reușit să satisfacem x mod base^exponent?
    bool found = false;
    if (gcd % modulus == 0) {
      found = true;
    } else if (rem < 2 * modulus / base) {
      // Prin eliminare, următorul candidat este satisfăcător.
      rem -= modulus / base;
      found = true;
    }

    if (found) {
      // Am învățat că rem este satisfăcător modulo modulus.
      exponent++;
      if (!is_solved()) {
        modulus *= base;
        // Ia-o de la început.
        rem += (base - 1) * modulus / base;
      }
    } else {
      rem -= modulus / base;
    }
  }

  bool is_solved() {
    return exponent > max_exponent;
  }

  // Pregătește congruența pentru CRT-ul final: x ≡ rem % modulus.
  void flip() {
    exponent = 0; // Marchează-l ca nerezolvat.
    rem = (modulus - rem) % modulus;
  }
};

congruence c[NUM_MODULI] = {
```

```
  congruence(2, 11),
  congruence(3, 6),
  congruence(5, 2),
  congruence(7, 2),
};

unsigned ask(unsigned a, unsigned b) {
  printf("? %u %u\n", a, b);
  fflush(stdout);

  int response;
  scanf("%d", &response);
  return response;
}

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x, int mod) {
  int y, k, d;
  extended_euclid_iterative(x, mod, d, y, k);
  return (y >= 0) ? y : (y + mod);
}

bool finished() {
  bool all_solved = true;
  for (int i = 0; i < NUM_MODULI; i++) {
    all_solved &= c[i].is_solved();
  }
  return all_solved;
}

void solve_crt(unsigned& solution, unsigned& big_mod) {
  big_mod = 1;
  for (int i = 0; i < NUM_MODULI; i++) {
    if (!c[i].is_solved()) {
      big_mod *= c[i].modulus;
    }
  }
```

```
  solution = 0;
  for (int i = 0; i < NUM_MODULI; i++) {
    if (!c[i].is_solved()) {
      int prod_exclude = big_mod / c[i].modulus;
      int inv = inverse(prod_exclude, c[i].modulus);
      long long all = (long long)c[i].rem * prod_exclude % big_mod * inv;
      solution = (all + solution) % big_mod;
    }
  }
}

void learn_from_question() {
  unsigned common_rem, big_mod;
  solve_crt(common_rem, big_mod);
  int gcd = ask(common_rem, common_rem + big_mod);

  for (int i = 0; i < NUM_MODULI; i++) {
    if (!c[i].is_solved()) {
      c[i].advance(gcd);
    }
  }
}

void solve_test_case() {
  for (int i = 0; i < NUM_MODULI; i++) {
    c[i].reset();
  }

  while (!finished()) {
    learn_from_question();
  }

  for (int i = 0; i < NUM_MODULI; i++) {
    c[i].flip();
  }

  unsigned x, big_mod;
  solve_crt(x, big_mod);
  printf("! %u\n", x);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test_case();
  }

  return 0;
```

```
}
```

Anexa 23

Elemente de bază în combinatorică

23.1 Problema Binomial Coefficients (CSES)

înapoi

```cpp
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MOD = 1'000'000'007;

int fact[MAX_N + 1], inv_fact[MAX_N + 1];

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}
```

```
void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_N; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }

  inv_fact[MAX_N] = inverse(fact[MAX_N]);
  for (int i = MAX_N - 1; i >= 0; i--) {
    inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
  }
}

void process_queries() {
  int num_tests, n, k;
  scanf("%d", &num_tests);
  while (num_tests--) {
    scanf("%d %d", &n, &k);
    printf("%d\n", comb(n, k));
  }
}

int main() {
  precompute_factorials();
  process_queries();

  return 0;
}
```

23.2 Problema Creating Strings II (CSES)

‹ înapoi

```
#include <ctype.h>
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MOD = 1'000'000'007;
const int SIGMA = 26;

int fact[MAX_N + 1], inv_fact[MAX_N + 1];
int freq[SIGMA];
int n;

void read_string() {
  int c;
  while (islower(c = getchar())) {
    freq[c - 'a']++;
    n++;
```

```
  }
}

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= n; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }

  inv_fact[n] = inverse(fact[n]);
  for (int i = n - 1; i >= 0; i--) {
    inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
  }
}

void count_permutations() {
  long long answer = fact[n];
  for (int c = 0; c < SIGMA; c++) {
    answer = answer * inv_fact[freq[c]] % MOD;
  }
  printf("%lld\n", answer);
}

int main() {
  read_string();
  precompute_factorials();
  count_permutations();

  return 0;
}
```

23.3 Problema Distributing Apples (CSES)

înapoi

```cpp
#include <stdio.h>

const int MAX_VAL = 2'000'000;
const int MOD = 1'000'000'007;

int fact[MAX_VAL + 1], inv_fact[MAX_VAL + 1];

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_VAL; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }

  inv_fact[MAX_VAL] = inverse(fact[MAX_VAL]);
  for (int i = MAX_VAL - 1; i >= 0; i--) {
    inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
  }
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}

void process_query() {
  int n, k;
  scanf("%d %d", &n, &k);
```

```
  printf("%d\n", comb(n + k - 1, k));
}

int main() {
  precompute_factorials();
  process_query();

  return 0;
}
```

23.4 Problema Christmas Party (CSES)

înapoi

Sursă cu principiul includerii și excluderii, varianta 1.

```
#include <stdio.h>

const int MOD = 1'000'000'007;

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int count_derangements(int n) {
  long long fact = 1;
  for (int i = 1; i <= n; i++) {
    fact = fact * i % MOD;
  }

  long long inv_fact = inverse(fact);
  int sign = (n % 2) ? -1 : 1;
  long long sum = 0;
```

```
  for (int i = n; i >= 0; i--) {
    // Invarianți:
    //   sign = (-1)^i
    //   inv_fact = 1/i!
    sum = (sum + fact * sign * inv_fact) % MOD;
    sign = -sign;
    inv_fact = inv_fact * i % MOD;
  }

  return sum;
}

int main() {
  int n;
  scanf("%d", &n);
  printf("%d\n", count_derangements(n));

  return 0;
}
```

Sursă cu principiul includerii și excluderii, varianta 2.

```
#include <stdio.h>

const int MOD = 1'000'000'007;

int count_derangements(int n) {
  long long n_fact_over_i_fact = 1;
  int sign = (n % 2) ? -1 : 1;
  long long sum = 0;

  for (int i = n; i >= 0; i--) {
    sum = (sum + sign * n_fact_over_i_fact) % MOD;
    sign = -sign;
    n_fact_over_i_fact = n_fact_over_i_fact * i % MOD;
  }

  return (sum + MOD) % MOD;
}

int main() {
  int n;
  scanf("%d", &n);
  printf("%d\n", count_derangements(n));

  return 0;
}
```

Sursă cu formulă de recurență.

```
#include <stdio.h>

const int MOD = 1'000'000'007;

int count_derangements(int n) {
  if (n == 0) {
    return 1;
  } else if (n == 1) {
    return 0;
  }

  int a, b = 0, c = 1; // termenii 1 și 2
  for (int i = 3; i <= n; i++) {
    a = b;
    b = c;
    c = (long long)(i - 1) * (a + b) % MOD;
  }
  return c;
}

int main() {
  int n;
  scanf("%d", &n);
  printf("%d\n", count_derangements(n));

  return 0;
}
```

23.5 Problema Bracket Sequences I (CSES)

înapoi

```
//   comb(2n, n) / (n + 1) =
//   (2n)! / (n! * n! * (n + 1)) =
//   (n + 2) * (n + 3) * ... * (2n) / n!
#include <stdio.h>

const int MOD = 1'000'000'007;

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
```

```
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int catalan(int n) {
  long long fact = 1;
  for (int i = 1; i <= n; i++) {
    fact = fact * i % MOD;
  }

  long long result = inverse(fact);
  for (int i = n + 2; i <= 2 * n; i++) {
    result = result * i % MOD;
  }

  return result;
}

int num_strings_of_length(int n) {
  return (n % 2)
    ? 0
    : catalan(n / 2);
}

int main() {
  int n;
  scanf("%d", &n);
  printf("%d\n", num_strings_of_length(n));

  return 0;
}
```

23.6 Problema Bracket Sequences II (CSES)

‹ înapoi

```
// (k + 1) / (n + 1) * comb(2n - k, n)
#include <ctype.h>
#include <stdio.h>
```

```
const int MOD = 1'000'000'007;
int n, k;
bool feasible;

void read_string() {
  scanf("%d ", &n);
  int excess = 0;
  int c;

  while ((excess >= 0) && ispunct(c = getchar())) {
    k++;
    excess += (c == '(') ? +1 : -1;
  }

  feasible = (n % 2 == 0) && (excess >= 0);

  // Redu problema la lungime n, începînd cu k paranteze deschise.
  n = (n - k + excess) / 2;
  k = excess;
}

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int extended_catalan() {
  if (!feasible) {
    return 0;
  }

  long long fact = 1;
  for (int i = 1; i <= n + 1; i++) {
    fact = fact * i % MOD;
  }
  long long result = (long long)(k + 1) * inverse(fact) % MOD;
```

```
  for (int i = n - k + 1; i <= 2 * n - k; i++) {
    result = result * i % MOD;
  }

  return result;
}

int main() {
  read_string();
  printf("%d\n", extended_catalan());

  return 0;
}
```

23.7 Problema Counting Necklaces (CSES)

‹ înapoi

```
#include <stdio.h>

const int MAX_LEN = 1'000'000;
const int MOD = 1'000'000'007;

int p[MAX_LEN + 1];

void precompute_powers(int base, int up_to) {
  p[0] = 1;
  for (int i = 1; i <= up_to; i++) {
    p[i] = (long long)p[i - 1] * base % MOD;
  }
}

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
```

```
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int count_necklaces(int len, int colors) {
  long long sum = 0;
  for (int i = 0; i < len; i++) {
    int d, x, y;
    extended_euclid_iterative(i, len, d, x, y);
    sum += p[d];
  }

  return sum % MOD * inverse(len) % MOD;
}

int main() {
  int len, colors;
  scanf("%d %d", &len, &colors);
  precompute_powers(colors, len);
  printf("%d\n", count_necklaces(len, colors));

  return 0;
}
```

23.8 Problema Counting Grids (CSES)

înapoi

```
#include <stdio.h>

const int MOD = 1'000'000'007;
const int QUARTER = 250'000'002;

long long pow2(long long e) {
  long long result = 1;
  long long b = 2;

  while (e) {
    if (e & 1) {
      result = result * b % MOD;
    }
    b = b * b % MOD;
    e >>= 1;
  }

  return result;
}
```

```
int count_grids(long long n) {
  long long sum;
  if (n % 2 == 0) {
    sum =
      pow2(n * n) +          // nicio rotație
      2 * pow2(n * n / 4) +  // rotație cu 90/270°
      pow2(n * n / 2);       // rotație cu 180°
  } else {
    sum =
      pow2(n * n) +
      2 * pow2((n / 2) * (n / 2) + (n / 2) + 1) +
      pow2((n /2) * n + (n / 2) + 1);
  }

  return sum * QUARTER % MOD;
}

int main() {
  int n;
  scanf("%d", &n);
  printf("%d\n", count_grids(n));

  return 0;
}
```

23.9 Problema Number of Permutations (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 300'000;
const int MOD = 998'244'353;

struct pair {
  int a, b;
};

typedef bool (*cmpFunc)(pair, pair);
typedef bool (*equalsFunc)(pair, pair);

pair v[MAX_N + 1];
int fact[MAX_N + 1];
int n;

bool cmpA(pair p, pair q) {
  return p.a < q.a;
```

```
}

bool cmpB(pair p, pair q) {
  return p.b < q.b;
}

bool cmpBoth(pair p, pair q) {
  return (p.a < q.a) || ((p.a == q.a) && (p.b < q.b));
}

bool equalsA(pair p, pair q) {
  return p.a == q.a;
}

bool equalsB(pair p, pair q) {
  return p.b == q.b;
}

bool equalsBoth(pair p, pair q) {
  return (p.a == q.a) && (p.b == q.b);
}

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d", &v[i].a, &v[i].b);
  }
  v[n].a = v[n].b = n + 1; // santinele infinite
}

void compute_fact() {
  fact[0] = 1;
  for (int i = 1; i <= n; i++) {
    fact[i] = (long long)i * fact[i - 1] % MOD;
  }
}

int count(cmpFunc cmp, equalsFunc equals) {
  std::sort(v, v + n, cmp);

  long long result = 1;
  int run = 1;
  for (int i = 1; i <= n; i++) {
    if (equals(v[i], v[i - 1])) {
      run++;
    } else {
      result = result * fact[run] % MOD;
      run = 1;
    }
  }
```

```
  return result;
}

bool is_sorted_by_b() {
  for (int i = 1; i < n; i++) {
    if (v[i].b < v[i - 1].b) {
      return false;
    }
  }
  return true;
}

// Dintre cele n! permutări, scade-le pe cele sortate după a sau după b și
// adună-le la loc pe cele sortate după a și după b.
int inclusion_exclusion() {
  long long result = (long long)fact[n]
    + (MOD - count(cmpA, equalsA))
    + (MOD - count(cmpB, equalsB));

  int both = count(cmpBoth, equalsBoth);
  if (is_sorted_by_b()) {
    result += both;
  }
  return result % MOD;
}

int main() {
  read_data();
  compute_fact();
  int result = inclusion_exclusion();
  printf("%d\n", result);

  return 0;
}
```

23.10 Problema Counting Factorizations (Codeforces)

‹ înapoi • versiune online

```
// Complexitate: O(n^2).
#include <stdio.h>

const int MAX_N = 2'022;
const int MAX_VAL = 1'000'000;
const int MOD = 998'244'353;

short freq[MAX_VAL + 1];
```

```
short pfreq[2 * MAX_N]; // frecvența numerelor prime
int fact[2 * MAX_N + 1], inv_fact[2 * MAX_N + 1];
int d[2 * MAX_N + 1];   // spațiu pentru programarea dinamică
int n;
int num_primes;         // distincte
long long comp_contrib; // contribuția de la numere compuse

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < 2 * n; i++) {
    int x;
    scanf("%d", &x);
    freq[x]++;
  }
}

int bin_exp(int b, int e) {
  long long result = 1;

  while (e) {
    if (e & 1) {
      result = result * b % MOD;
    }
    b = (long long)b * b % MOD;
    e >>= 1;
  }

  return result;
}

void compute_fact() {
  fact[0] = 1;
  for (int i = 1; i <= 2 * n; i++) {
    fact[i] = (long long)i * fact[i - 1] % MOD;
  }

  inv_fact[2 * n] = bin_exp(fact[2 * n], MOD - 2);
  for (int i = 2 * n - 1; i >= 0; i--) {
    inv_fact[i] = (long long)(i + 1) * inv_fact[i + 1] % MOD;
  }
}

bool is_prime(int x) {
  for (int d = 2; d * d <= x; d++) {
    if (x % d == 0) {
      return false;
    }
  }

  return (x != 1);
```

```
}

void collect_primes() {
  comp_contrib = fact[n];
  for (int i = 1; i <= MAX_VAL; i++) {
    if (freq[i]) {
      if (is_prime(i)) {
        pfreq[num_primes++] = freq[i];
      } else {
        comp_contrib = comp_contrib * inv_fact[freq[i]] % MOD;
      }
    }
  }
}

// d[i][j] = Suma contribuțiilor dacă alegem j prime până la poziția i.
void build_dp() {
  d[0] = 1;

  for (int i = 0; i < num_primes; i++) {
    for (int j = i + 1; j > 0; j--) {
      int no_take = (long long)inv_fact[pfreq[i]] * d[j] % MOD;
      int take = (long long)inv_fact[pfreq[i] - 1] * d[j - 1] % MOD;
      d[j] = (take + no_take) % MOD;
    }

    d[0] = (long long)inv_fact[pfreq[i]] * d[0] % MOD;
  }
}

void wrap_it_up() {
  int answer = comp_contrib * d[n] % MOD;
  printf("%d\n", answer);
}

int main() {
  read_data();
  compute_fact();
  collect_primes();
  build_dp();
  wrap_it_up();

  return 0;
}
```

23.11 Problema Monocarp and the Set (Codeforces)

înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 300'000;
const int MOD = 998'244'353;

char s[MAX_N];
int n, num_ops;
long long answer;

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

void read_data() {
  scanf("%d %d %s", &n, &num_ops, s);
}

void compute_first_answer() {
  answer = 1;
  for (int i = 1; i < n - 1; i++) {
    if (s[i] == '?') {
      answer = answer * i % MOD;
    }
  }
}

void update_string() {
  int pos;
  char c;
  scanf("%d %c", &pos, &c);
  pos--;

  if (s[pos] != c) { // optimizare de viteză
    if ((pos > 0) && (s[pos] == '?')) {
```

```
      answer = answer * inverse(pos) % MOD;
    }
    s[pos] = c;
    if ((pos > 0) && (s[pos] == '?')) {
      answer = answer * pos % MOD;
    }
  }
}

void write_answer() {
  int real_answer = (s[0] == '?') ? 0 : answer;
  printf("%d\n", real_answer);
}

int main() {
  read_data();
  compute_first_answer();
  write_answer();

  while (num_ops--) {
    update_string();
    write_answer();
  }

  return 0;
}
```

23.12 Problema Devu and Flowers (Codeforces)

‹ înapoi

Sursă cu calcularea inverselor (versiune online).

```
#include <stdio.h>

const int MAX_BOXES = 20;
const int MOD = 1'000'000'007;

long long cap[MAX_BOXES], num_objects;
int num_boxes, denominator;

// Calculează x^{-1} % MOD ca x^{MOD-2} % MOD.
int inverse(int x) {
  long long result = 1;
  int e = MOD - 2;

  while (e) {
    if (e & 1) {
```

```
      result = result * x % MOD;
    }
    x = (long long)x * x % MOD;
    e >>= 1;
  }

  return result;
}

void precompute_denominator() {
  denominator = 1;
  for (int i = 1; i < num_boxes; i++) {
    denominator = (long long)denominator * inverse(i) % MOD;
  }
}

// Returnează combinări(n, num_boxes - 1).
int comb(long long n) {
  n %= MOD; // altfel result * n poate depăși
  n += MOD; // altfel n + 1 - i poate trece pe negativ

  long long result = 1;
  for (int i = 1; i < num_boxes; i++) {
    result = result * (n + 1 - i) % MOD;
  }
  return result * denominator % MOD;
}

void read_data() {
  scanf("%d %lld", &num_boxes, &num_objects);
  for (int i = 0; i < num_boxes; i++) {
    scanf("%lld", &cap[i]);
  }
}

int pie(int box, long long num_objects, int sign) {
  if (box == num_boxes) {
    int c = comb(num_objects + num_boxes - 1);
    return (sign == 1) ? c : (MOD - c);
  }

  int sum = pie(box + 1, num_objects, sign);
  if (num_objects > cap[box]) {
    sum += pie(box + 1, num_objects - cap[box] - 1, -sign);
  }

  return sum % MOD;
}
// Mmmmm, pie.
```

```
void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  precompute_denominator();
  int answer = pie(0, num_objects, +1);
  write_answer(answer);

  return 0;
}
```

Sursă cu inverse precalculate (versiune online).

```
// v3: Hardcodăm 1/(num_boxes-1)!, doar ca distracție.
#include <stdio.h>

const int MAX_BOXES = 20;
const int MOD = 1'000'000'007;

// DENOM[i] = 1 / (i-1)!
int DENOM[MAX_BOXES + 1] = {
  0, 1, 1, 500000004, 166666668, 41666667, 808333339, 301388891, 900198419,
  487524805, 831947206, 283194722, 571199524, 380933296, 490841026, 320774361,
  821384963, 738836565, 514049213, 639669405, 402087866,
};

long long cap[MAX_BOXES], num_objects;
int num_boxes;

// Returnează combinări(n, num_boxes - 1).
int comb(long long n) {
  n %= MOD; // altfel result * n poate depăși
  n += MOD; // altfel n + 1 - i poate trece pe negativ

  long long result = DENOM[num_boxes];
  for (int i = 1; i < num_boxes; i++) {
    result = result * (n + 1 - i) % MOD;
  }
  return result;
}

void read_data() {
  scanf("%d %lld", &num_boxes, &num_objects);
  for (int i = 0; i < num_boxes; i++) {
    scanf("%lld", &cap[i]);
  }
}
```

```
int pie(int box, long long num_objects, int sign) {
  if (box == num_boxes) {
    int c = comb(num_objects + num_boxes - 1);
    return (sign == 1) ? c : (MOD - c);
  }

  int sum = pie(box + 1, num_objects, sign);
  if (num_objects > cap[box]) {
    sum += pie(box + 1, num_objects - cap[box] - 1, -sign);
  }

  return sum % MOD;
}
// Mmmmm, pie.

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  int answer = pie(0, num_objects, +1);
  write_answer(answer);

  return 0;
}
```

23.13 Problema Lengthening Sticks (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

int min(int x, int y) {
  return (x < y) ? x : y;
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

long long count_all(int a, int b, int c, int l) {
  long long result = 0;
  int start = max(0, b + c - a); // Mai jos ne asigurăm că h ≥ 0.
  for (int x = start; x <= l; x++) {
    long long h = min(l - x, a + x - b - c);
    result += (h + 1) * (h + 2) / 2;
```

```
  }

  return result;
}

int main() {
  int a, b, c, l;
  scanf("%d %d %d %d\n", &a, &b, &c, &l);

  long long result =
    (long long)(l + 3) * (l + 2) * (l + 1) / 6
    - count_all(a, b, c, l)
    - count_all(b, c, a, l)
    - count_all(c, a, b, l);

  printf("%lld\n", result);

  return 0;
}
```

23.14 Problema Shuffle (Codeforces)

înapoi

Sursă în $\mathcal{O}(n^2)$ (versiune online).

```
#include <stdio.h>

const int MAX_N = 5'000;
const int MOD = 998'244'353;

int fact[MAX_N + 1], inv_fact[MAX_N + 1];
char s[MAX_N + 1];
int n, k, total_ones;

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
  }
  d = a;
}
```

```
int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= n; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }

  inv_fact[n] = inverse(fact[n]);
  for (int i = n - 1; i >= 0; i--) {
    inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
  }
}

void read_data() {
  scanf("%d %d ", &n, &k);
  for (int i = 0; i < n; i++) {
    s[i] = getchar() - '0';
    total_ones += (s[i] == 1);
  }
}

int count_shuffles(int l, int r, int num_ones) {
  if (num_ones > k) {
    return 0;
  }

  int num_zeroes = r - l + 1 - num_ones;

  int outer_zeroes = (s[l] == 0) + (s[r] == 0);
  int outer_ones = 2 - outer_zeroes;

  int inner_zeroes = num_zeroes - outer_ones;
  int inner_ones = num_ones - outer_zeroes;

  return ((inner_zeroes < 0) || (inner_ones < 0))
    ? 0
    : comb(inner_zeroes + inner_ones, inner_ones);
}

int try_all_pairs() {
  long long result = 0;
```

```
  if (total_ones >= k) {
    for (int l = 0; l < n; l++) {
      int num_ones = (s[l] == 1);
      for (int r = l + 1; r < n; r++) {
        num_ones += (s[r] == 1);
        result += count_shuffles(l, r, num_ones);
      }
    }
  }

  result++; // neschimbat

  return result % MOD;
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  precompute_factorials();
  int answer = try_all_pairs();
  write_answer(answer);

  return 0;
}
```

Sursă în $\mathcal{O}(n)$ (versiune online).

```
#include <stdio.h>

const int MAX_N = 5'000;
const int MOD = 998'244'353;

int fact[MAX_N + 1], inv_fact[MAX_N + 1];
char s[MAX_N + 1];
int n, k, total_ones;

void extended_euclid_iterative(int a, int b, int& d, int& x, int& y) {
  x = 1;
  y = 0;
  int xp = 0, yp = 1;
  while (b) {
    int q = a / b;
    int tmp = b; b = a - q * b; a = tmp;
    tmp = xp; xp = x - q * xp; x = tmp;
    tmp = yp; yp = y - q * yp; y = tmp;
```

```
  }
  d = a;
}

int inverse(int x) {
  int y, k, d;
  extended_euclid_iterative(x, MOD, d, y, k);
  return (y >= 0) ? y : (y + MOD);
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= n; i++) {
    fact[i] = (long long)fact[i - 1] * i % MOD;
  }

  inv_fact[n] = inverse(fact[n]);
  for (int i = n - 1; i >= 0; i--) {
    inv_fact[i] = (long long)inv_fact[i + 1] * (i + 1) % MOD;
  }
}

void read_data() {
  scanf("%d %d ", &n, &k);
  for (int i = 0; i < n; i++) {
    s[i] = getchar() - '0';
    total_ones += (s[i] == 1);
  }
}

void extend_to_k_ones(int& r, int& ones) {
  while ((r < n - 1) && (ones < k)) {
    r++;
    ones += (s[r] == 1);
  }

  // Consumă și zerourile.
  while ((r < n - 1) && (s[r + 1] == 0)) {
    r++;
  }
}

void contract(int& l, int& ones) {
  ones -= (s[l] == 1);
}
```

```
int count_shuffles(int l, int r, int ones) {
  int zeroes = (r - l + 1) - ones;
  if (!ones || !zeroes) {
    // Fereastra are toate valorile egale. Nu putem schimba capătul stîng.
    return 0;
  }

  ones -= (s[l] == 0);
  zeroes -= (s[l] == 1);
  return comb(zeroes + ones, ones);
}

int try_all_left_ends() {
  long long result = 1; // neschimbat

  int r = -1, ones = 0;
  for (int l = 0; l < n; l++) {
    extend_to_k_ones(r, ones);
    result += count_shuffles(l, r, ones);
    contract(l, ones);
  }

  return result % MOD;
}

void write_answer(int answer) {
  printf("%d\n", answer);
}

int main() {
  read_data();
  precompute_factorials();
  int answer = ((k == 0) || (total_ones < k))
    ? 1
    : try_all_left_ends();
  write_answer(answer);

  return 0;
}
```

23.15 Problema Manhattan (Lot 2017)

‹ înapoi

Sursă cu agregare pe zone (versiune online).

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_COMB = 200'000;
const int MOD = 666'013;

struct zone {
  int x1, y1, x2, y2;

  void translate(int dx) {
    x1 += dx;
    x2 += dx;
  }

  void mirror_diag() {
    std::swap(x1, y1);
    std::swap(x2, y2);
  }
};

zone a, b;
int fact[MAX_COMB + 1], inv_fact[MAX_COMB + 1];

void read_zones() {
  FILE* f = fopen("manhattan.in", "r");
  fscanf(f, "%d %d %d %d", &a.x1, &a.y1, &a.x2, &a.y2);
  fscanf(f, "%d %d %d %d", &b.x1, &b.y1, &b.x2, &b.y2);
  fclose(f);
}

void normalize_zones() {
  // Asigură-te că există un x care separă zonele.
  if ((a.y1 > b.y2) || (b.y1 > a.y2)) {
    a.mirror_diag();
    b.mirror_diag();
  }

  // Adu zona A în stînga.
  if (a.x1 > b.x1) {
    std::swap(a, b);
  }

  // Adu latura stîngă a lui A pe axa Oy.
  b.translate(-a.x1);
  a.translate(-a.x1);

  // Acum B este complet la dreapta lui A.
}

// Calculează x^{-1} % MOD ca x^{MOD-2} % MOD.
int inverse(int x) {
  long long result = 1;
```

```
  int e = MOD - 2;

  while (e) {
    if (e & 1) {
      result = result * x % MOD;
    }
    x = (long long)x * x % MOD;
    e >>= 1;
  }

  return result;
}

void compute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_COMB; i++) {
    fact[i] = (long long)i * fact[i - 1] % MOD;
  }

  inv_fact[MAX_COMB] = inverse(fact[MAX_COMB]);
  for (int i = MAX_COMB - 1; i >= 0; i--) {
    inv_fact[i] = (long long)(i + 1) * inv_fact[i + 1] % MOD;
  }
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] * inv_fact[n - k] % MOD;
}

// Numără căile de la linia (0,0)-(w,0) la zona (B.x1, d1)-(B.x2, d2).
// Presupune că 0 ≤ d1 ≤ d2.
int count_line_to_zone_paths(int d1, int d2) {
  int x1 = b.x1, x2 = b.x2, w = a.x2;

  int result =
    + (comb(x2 + d2 + 3, x2 + 1) - comb(x2 + d1 + 2, x2 + 1))
    - (comb(x2 + d2 + 2 - w, x2 - w) - comb(x2 + d1 + 1 - w, x2 - w))
    - (comb(x1 + d2 + 2, x1) - comb(x1 + d1 + 1, x1))
    + (comb(x1 + d2 + 1 - w, x1 - 1 - w) - comb(x1 + d1 - w, x1 - 1 - w));

  return (result + 8 * MOD) % MOD;
}

// Numără drumurile între oricare linie din A și zona B, spărgînd zona B în
// două dacă linia din A o intersectează.
int count_paths() {
  long long result = 0;

  for (int h = a.y1; h <= a.y2; h++) {
    if (h <= b.y1) {
```

```
      result += count_line_to_zone_paths(b.y1 - h, b.y2 - h);
    } else if (h >= b.y2) {
      result += count_line_to_zone_paths(h - b.y2, h - b.y1);
    } else {
      result += count_line_to_zone_paths(0, b.y2 - h);
      result += count_line_to_zone_paths(1, h - b.y1);
    }
  }

  return result % MOD;
}

void write_answer(int answer) {
  FILE* f = fopen("manhattan.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}

int main() {
  read_zones();
  normalize_zones();
  compute_factorials();
  int answer = count_paths();
  write_answer(answer);

  return 0;
}
```

Sursă cu inverse precalculate (versiune online).

```
#include <algorithm>
#include <stdio.h>
#include <stdlib.h>

const int MAX_COMB = 200'000;
const int MOD = 666'013;

struct zone {
  int x1, y1, x2, y2;

  void translate(int dx) {
    x1 += dx;
    x2 += dx;
  }

  void mirror_diag() {
    std::swap(x1, y1);
    std::swap(x2, y2);
  }
```

```
};

zone a, b;
int fact[MAX_COMB + 1], inv_fact[MAX_COMB + 1];

void read_zones() {
  FILE* f = fopen("manhattan.in", "r");
  fscanf(f, "%d %d %d %d", &a.x1, &a.y1, &a.x2, &a.y2);
  fscanf(f, "%d %d %d %d", &b.x1, &b.y1, &b.x2, &b.y2);
  fclose(f);
}

void normalize_zones() {
  // Asigură-te că există un x care separă zonele.
  if ((a.y1 > b.y2) || (b.y1 > a.y2)) {
    a.mirror_diag();
    b.mirror_diag();
  }

  // Adu zona A în stînga.
  if (a.x1 > b.x1) {
    std::swap(a, b);
  }

  // Adu latura stîngă a lui A pe axa Oy.
  b.translate(-a.x1);
  a.translate(-a.x1);

  // Acum B este complet la dreapta lui A.
}

// Calculează x^{-1} % MOD ca x^{MOD-2} % MOD.
int inverse(int x) {
  long long result = 1;
  int e = MOD - 2;

  while (e) {
    if (e & 1) {
      result = result * x % MOD;
    }
    x = (long long)x * x % MOD;
    e >>= 1;
  }

  return result;
}

void compute_factorials() {
  fact[0] = 1;
  for (int i = 1; i <= MAX_COMB; i++) {
```

```
    fact[i] = (long long)i * fact[i - 1] % MOD;
  }

  inv_fact[MAX_COMB] = inverse(fact[MAX_COMB]);
  for (int i = MAX_COMB - 1; i >= 0; i--) {
    inv_fact[i] = (long long)(i + 1) * inv_fact[i + 1] % MOD;
  }
}

int comb(int n, int k) {
  return (long long)fact[n] * inv_fact[k] * inv_fact[n - k] % MOD;
}

// Numără drumurile de la linia (0,0)-(A.x2,0) la linia (A.x1,d)-(A.x2,d)
int count_paths_dist(int d) {
  return (
    + comb(b.x2 + d + 2, d + 2)
    - comb(b.x2 + d + 1 - a.x2, d + 2)
    - comb(b.x1 + d + 1, d + 2)
    + comb(b.x1 + d - a.x2, d + 2)
    + 2 * MOD) % MOD;
}

// Numără perechile de linii din A și B aflate la distanță d pe verticală.
int line_pairs_having_dist(int d) {
  int y1 = b.y1 - d, y2 = b.y2 - d;
  // Reuniunea intervalelor (A.y1,A.y2) și (y1,y2).
  return std::max(std::min(y2, a.y2) - std::max(y1, a.y1) + 1, 0);
}

// Numără drumurile între oricare linie din A și oricare linie din B, grupînd
// perechile de linii după distanța dintre ele.
int count_paths() {
  long long result = 0;

  for (int d = b.y1 - a.y2; d <= b.y2 - a.y1; d++) {
    result += (long long)line_pairs_having_dist(d) *
      count_paths_dist(abs(d)) % MOD;
  }

  return result % MOD;
}

void write_answer(int answer) {
  FILE* f = fopen("manhattan.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}

int main() {
```

```
  read_zones();
  normalize_zones();
  compute_factorials();
  int answer = count_paths();
  write_answer(answer);

  return 0;
}
```

Anexa 24

Combinatorică - rangul permutărilor și al combinărilor

24.1 Rangul permutărilor

înapoi

```
// Notă: _pdep_u32 necesită argumentul -march=native la compilare.
//
// Ranking și unranking de permutări prin mai multe metode. Nu sînt ieșite din
// comun. Ne interesează să fie ușor de codat.
#include <assert.h>
#include <immintrin.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

// Vom face ranking și unranking pe NUM_PASSES * NUM_PERMS permutări aleatorii
// de mărime n.
const int NUM_PERMS = 1'000;
const int NUM_PASSES = 10'000;
const int N = 20;
const int MAX_P2 = 16; // pentru fenwick_bin_search()

typedef unsigned long long u64;

typedef struct {
  u64 rank, witness;
  int v[N];
} permutation;

permutation p[NUM_PERMS], q[NUM_PERMS];
u64 fact[N];
char kth[1 << N][N]; // kth[x][k] = al k-lea bit zero în x (0-based)
```

```
void generate_permutations() {
  for (int i = 0; i < NUM_PERMS; i++) {
    for (int j = 0; j < N; j++) {
      int k = rand() % (j + 1);
      p[i].v[j] = p[i].v[k];
      p[i].v[k] = j;
    }
  }
}

void precompute_factorials() {
  fact[0] = 1;
  for (int i = 1; i < N; i++) {
    fact[i] = fact[i - 1] * i;
  }
}

void precompute_kth() {
  for (int i = 0; i < (1 << N); i++) {
    int cnt = 0;
    for (int k = 0; k < N; k++) {
      if ((i & (1 << k)) == 0) {
        kth[i][cnt++] = k;
      }
    }
    // Pozițiile de la cnt la N sînt irelevante pentru că nu există destui
    // biți setați. Fie le atribuim o valoare specială, fie ne asigurăm că
    // codul nu citește acele poziții.
  }
}

void verify_rank() {
  for (int i = 0; i < NUM_PERMS; i++) {
    assert(p[i].rank == p[i].witness);
  }
}

void verify_unrank() {
  for (int i = 0; i < NUM_PERMS; i++) {
    for (int j = 0; j < N; j++) {
      assert(p[i].v[j] == q[i].v[j]);
    }
  }
}

/*****************************************************************
 *                    Gestiunea timpului                         *
 *****************************************************************/
long long t0;
```

```
long long get_time() {
  struct timeval time;
  gettimeofday(&time, NULL);
  return 1000ll * time.tv_sec + time.tv_usec / 1000;
}

void mark_time() {
  t0 = get_time();
}

void report_time(const char* msg) {
  int millis = get_time() - t0;
  int ops = NUM_PASSES * NUM_PERMS;

  printf("%s: %d ms pentru %d operații (%d Mops/sec)\n",
         msg, millis, ops, ops / millis / 1000);
}

/*****************************************************************
 *                        Metoda naivă                           *
 *****************************************************************/

bool used[N];

void rank_naive() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      u64 rank = 0;
      for (int i = 0; i < N; i++) {
        // Numără elementele mai mici decît p[i].
        int cnt = 0;
        for (int j = 0; j < i; j++) {
          cnt += (p[k].v[j] < p[k].v[i]);
        }
        rank += fact[N - 1 - i] * (p[k].v[i] - cnt);
      }
      p[k].rank = p[k].witness = rank;
    }
  }
}

void unrank_naive() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      for (int i = 0; i < N; i++) {
        used[i] = false;
      }
      u64 rank = p[k].rank;
      for (int i = 0; i < N; i++) {
```

```
        int quot = rank / fact[N - 1 - i];
        rank %= fact[N - 1 - i];
        // Caută a quot-a valoare nefolosită.
        int j = 0;
        while (quot || used[j]) {
          quot -= !used[j++];
        }
        q[k].v[i] = j;
        used[j] = true;
      }
    }
  }
}

/*****************************************************************
 *                       Arbori Fenwick                          *
 *****************************************************************/

// Reminder: AIB-urile sînt indexate de la 1.
int fen[N + 1];

void fenwick_clear() {
  for (int i = 1; i <= N; i++) {
    fen[i] = 0;
  }
}

// Pune toate valorile pe 1 și apoi construiește AIB-ul.
// https://stackoverflow.com/a/31070683/6022817
void fenwick_set_1() {
  for (int i = 1; i <= N; i++) {
    fen[i] = 1;
  }
  for (int i = 1; i <= N; i++) {
    int j = i + (i & -i);
    if (j <= N) {
      fen[j] += fen[i];
    }
  }
}

void fenwick_add(int pos, int val) {
  do {
    fen[pos] += val;
    pos += pos & -pos;
  } while (pos <= N);
}

int fenwick_sum(int pos) {
  int s = 0;
```

```
  while (pos) {
    s += fen[pos];
    pos &= pos - 1;
  }
  return s;
}

// Returnează cea mai mare poziție pentru care fen[pos] < val.
int fenwick_bin_search(int val) {
  int pos = 0;

  for (int interval = MAX_P2; interval; interval >>= 1) {
    if ((pos + interval <= N) && (fen[pos + interval] < val)) {
      val -= fen[pos + interval];
      pos += interval;
    }
  }

  return pos;
}

void rank_fenwick() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      fenwick_clear();
      u64 rank = 0;
      for (int i = 0; i < N; i++) {
        // Numără elementele mai mici decît p[i] -- ca în versiunea naivă.
        int cnt = fenwick_sum(p[k].v[i] + 1);
        rank += fact[N - 1 - i] * (p[k].v[i] - cnt);
        fenwick_add(p[k].v[i] + 1, 1);
      }
      p[k].rank = rank;
    }
  }
}

void unrank_fenwick() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      u64 rank = p[k].rank;
      // Valorile 1 din AIB indică elemente încă folosibile.
      fenwick_set_1();
      for (int i = 0; i < N; i++) {
        int quot = rank / fact[N - 1 - i];
        rank %= fact[N - 1 - i];
        // Caută a quot-a valoare folosibilă.
        int j = fenwick_bin_search(quot + 1);
        q[k].v[i] = j;
        fenwick_add(j + 1, -1);
```

```
      }
    }
  }
}

/*****************************************************************
 *                           Popcount                            *
 *****************************************************************/

void rank_popcount() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      u64 rank = 0;
      int mask = 0;
      for (int i = 0; i < N; i++) {
        // Numără biții setați pe pozițiile [0...p[i]).
        int cnt = __builtin_popcount(mask & ((1 << p[k].v[i]) - 1));
        rank += fact[N - 1 - i] * (p[k].v[i] - cnt);
        mask |= (1 << p[k].v[i]);
      }
      p[k].rank = rank;
    }
  }
}

void unrank_popcount() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      u64 rank = p[k].rank;
      int mask = 0;
      for (int i = 0; i < N; i++) {
        int quot = rank / fact[N - 1 - i];
        rank %= fact[N - 1 - i];
        // Caută a quot-a valoare nefolosită.
        int j = kth[mask][quot];
        q[k].v[i] = j;
        mask |= (1 << j);
      }
    }
  }
}

void unrank_popcount2() {
  for (int pass = 0; pass < NUM_PASSES; pass++) {
    for (int k = 0; k < NUM_PERMS; k++) {
      u64 rank = p[k].rank;
      int mask = 0;
      for (int i = 0; i < N; i++) {
        int quot = rank / fact[N - 1 - i];
        rank %= fact[N - 1 - i];
```

```
        // Caută a quot-a valoare nefolosită.
        int zero_bit_mask = _pdep_u32(1 << quot, ~mask);
        int j = _tzcnt_u32(zero_bit_mask);

        q[k].v[i] = j;
        mask |= (1 << j);
      }
    }
  }
}

int main() {
  struct timeval time;
  gettimeofday(&time, NULL);
  srand(time.tv_usec);

  mark_time();
  precompute_factorials();
  precompute_kth();
  generate_permutations();
  rank_naive();
  report_time("inițializare");
  printf("--\n");

  mark_time();
  rank_naive();
  report_time("rank naiv");
  verify_rank();

  mark_time();
  rank_fenwick();
  report_time("rank cu AIB");
  verify_rank();

  mark_time();
  rank_popcount();
  report_time("rank cu popcount");
  verify_rank();
  printf("--\n");

  mark_time();
  unrank_naive();
  report_time("unrank naiv");
  verify_unrank();

  mark_time();
  unrank_fenwick();
  report_time("unrank cu AIB");
  verify_unrank();
```

```
  mark_time();
  unrank_popcount();
  report_time("unrank cu popcount și tabel");
  verify_unrank();

  mark_time();
  unrank_popcount2();
  report_time("unrank cu popcount și pdep");
  verify_unrank();

  return 0;
}
```

24.2 Rangul combinărilor

‹ înapoi

```
// Ranking și unranking de combinări în ordine colexicografică.
#include <algorithm>
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

// Vom face ranking și unranking TRIALS combinări aleatorii de N luate cîte K.
#define TRIALS 10'000'000
#define N 60
#define K 30
#define COLEX_UNRANK_STEP 3 // de ordinul a sqrt(N/K)

typedef unsigned long long u64;

typedef struct {
  u64 rank;
  int v[K];
} combination;

combination c[TRIALS];
bool seen[N]; // folosit pentru generarea combinăriilor aleatorii
u64 choose[N + 1][K + 1];

/*****************************************************************
 *                    Gestiunea timpului                         *
 *****************************************************************/
long long __time1;

long long get_time() {
```

```
  struct timeval time;
  gettimeofday(&time, NULL);
  return 1000ll * time.tv_sec + time.tv_usec / 1000;
}

void mark_time() {
  __time1 = get_time();
}

void report_time(const char* msg) {
  long long time2 = get_time();
  fprintf(stderr, "%s: %lld milisecunde\n", msg, time2 - __time1);
}

/*****************************************************************
 *   Ordine colexicografică: ranking în O(k), unranking în O(n)   *
 *****************************************************************/

u64 rank_colex(int* c) {
  u64 result = 0;
  for (int i = 0; i < K; i++) {
    result += choose[(int)c[i]][i + 1];
  }
  return result;
}

void unrank_colex(u64 rank, int* witness) {
  int n = N - 1;
  for (int k = K; k; k--) {
    while (choose[n][k] > rank) {
      n--;
    }
    rank -= choose[n][k];
    assert(n == witness[k - 1]);
  }
}

// ca și unrank_colex, dar cu o optimizare (sare mai mulți pași deodată)
void unrank_colex2(u64 rank, int* witness) {
  int n = N - 1;
  for (int k = K; k; k--) {
    while (n >= COLEX_UNRANK_STEP && choose[n - COLEX_UNRANK_STEP][k] > rank) {
      n -= COLEX_UNRANK_STEP;
    }
    while (choose[n][k] > rank) {
      n--;
    }
    rank -= choose[n][k];
    assert(n == witness[k - 1]);
  }
```

```
}

int main() {
  struct timeval time;
  gettimeofday(&time, NULL);
  srand(time.tv_usec);

  // precalculează combinările
  mark_time();
  for (int n = 0; n <= N; n++) {
    for (int k = 0; k <= K; k++) {
      if (k == 0) {
        choose[n][k] = 1;
      } else if (n == 0) {
        choose[n][k] = 0;
      } else {
        choose[n][k] = choose[n - 1][k] + choose[n - 1][k - 1];
      }
    }
  }
  printf("Benchmark C(%d,%d) = %.3e, pas unranking=%d, %dM experimente\n",
         N, K, (double)choose[N][K], COLEX_UNRANK_STEP, TRIALS / 1'000'000);
  report_time("Precalcularea combinărilor");

  // generează experimentele
  mark_time();
  for (int i = 0; i < TRIALS; i++) {
    for (int j = 0; j < K; j++) {
      int val;
      do {
        val = rand() % N;
      } while (seen[val]);
      seen[val] = true;
      c[i].v[j] = val;
    }
    std::sort(c[i].v, c[i].v + K);

    for (int j = 0; j < K; j++) {
      seen[(int)c[i].v[j]] = false;
    }
  }
  report_time("Generarea experimentelor");
  printf("--\n");

  // benchmarking
  mark_time();
  for (int i = 0; i < TRIALS; i++) {
    c[i].rank = rank_colex(c[i].v);
  }
  report_time("Ranking colexicografic");
```

```
  mark_time();
  for (int i = 0; i < TRIALS; i++) {
    unrank_colex(c[i].rank, c[i].v);
  }
  report_time("Unranking colexicografic");
  mark_time();

  for (int i = 0; i < TRIALS; i++) {
    unrank_colex2(c[i].rank, c[i].v);
  }
  report_time("Unranking colexicografic cu optimizare");
  return 0;
}
```

24.3 Problema Misha and Permutation Summation (Codeforces)

‹ înapoi • versiune online

```
// Metodă calculăm rangul lui P și Q în baza factorial. Adunăm rangurile și
// ignorăm ultimul transport (peste cifra a N-a). Calculăm permutarea pentru
// rangul-sumă.
#include <stdio.h>

const int MAX_N  = 200'000;

int r[MAX_N], n; // r[] acumulează rangurile lui P și Q

struct fenwick_tree {
  int v[MAX_N + 1];
  int n, max_p2;

  void init(int n) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));
    for (int i = 1; i <= n; i++) {
      v[i] = 0;
    }
  }

  void add(int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }
```

```
  int prefix_sum(int pos) {
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  int bin_search(int val) {
    int pos = 0;

    for (int interval = max_p2; interval; interval >>= 1) {
      if ((pos + interval <= n) && (v[pos + interval] < val)) {
        val -= v[pos + interval];
        pos += interval;
      }
    }

    return pos;
  }
};

fenwick_tree fen;

// Citește o permutare și îi adaugă rangul factorial în r[].
void read_and_rank() {
  fen.init(n);

  for (int i = n - 1; i >= 0; i--) {
    int x;
    scanf("%d", &x);
    x++;
    int cnt = fen.prefix_sum(x);
    r[i] += (x - 1) - cnt;
    fen.add(x, 1);
  }
}

void carry() {
  int t = 0;
  for (int i = 0; i < n; i++) {
    r[i] += t;
    t = (r[i] > i);
    r[i] -= t * (i + 1);
  }
}

// Face unrank la r[] și tipărește permutarea.
```

```
void unrank_and_write() {
  // Valorile de 1 din AIB indică elemente folosibile.
  // Avem deja toate valorile pe 1 ca urmare a lui read_and_rank().
  for (int i = n - 1; i >= 0; i--) {
    // Găsește cea de-a r[i]-a valoare nefolosită.
    int x = fen.bin_search(r[i] + 1);
    printf("%d ", x);
    fen.add(x + 1, -1);
  }
  printf("\n");
}

int main() {
  scanf("%d", &n);
  read_and_rank();
  read_and_rank();
  carry();
  unrank_and_write();

  return 0;
}
```

24.4 Problema Inversion Sort (SPOJ)

înapoi

```
// 1. Calculează rangurile permutărilor și le mapează la intervalul [0...10!).
// 2. Rulează BFS ca să calculeze distanțele.
// 3. Pentru o interogare, întîi remapeaz-o ca să pornească de la abcdefghij.
//
// v2: Generează mutările schimbînd cîte o singură pereche.
// v3: Folosește popcount-ul propriu.
#include <stdio.h>
#include <string.h>

const int LENGTH = 10;
const int NUM_PERMS = 3'628'800;
const int INFINITY = 0xff;
const char* START = "abcdefghij";
const int FACT[LENGTH] = { 1, 1, 2, 6, 24, 120, 720, 5'040, 40'320, 362'880 };

struct string {
  char s[LENGTH + 1];
  int rank;

  void set(const char* s, int rank) {
    strcpy(this->s, s);
    this->rank = rank;
```

```
  }

  void flip(int l, int r) {
    while (l < r) {
      swap(l++, r--);
    }
  }

  void swap(int i, int j) {
    int tmp = s[i]; s[i] = s[j]; s[j] = tmp;
  }
};

string q[NUM_PERMS];
unsigned char dist[NUM_PERMS];
int pop[1 << LENGTH];

void precompute() {
  for (int i = 1; i < (1 << LENGTH); i++) {
    pop[i] = pop[i >> 1] + (i & 1);
  }
}

int rank(const char* s) {
  int result = 0, mask = 0;

  for (int i = 0; i < LENGTH; i++) {
    int bit = 1 << (s[i] - 'a');
    int cnt = pop[mask & (bit - 1)];
    mask |= bit;
    result += cnt * FACT[i];
  }

  return result;
}

void bfs() {
  for (int i = 0; i < NUM_PERMS; i++) {
    dist[i] = INFINITY;
  }

  int head = 0, tail = 0;
  int rank_start = rank(START);
  q[tail++].set(START, rank_start);
  dist[rank_start] = 0;

  while (head != tail) {
    string str = q[head++];

    for (int l = 1; l <= 2; l++) {
```

```
      for (int i = 0; i + l < LENGTH; i++) {
        int x = i, y = i + l;
        do {
          str.swap(x--, y++);
          int r2 = rank(str.s);
          if (dist[r2] == INFINITY) {
            dist[r2] = 1 + dist[str.rank];
            q[tail++].set(str.s, r2);
          }

        } while (x >= 0 && y < LENGTH);
        str.flip(x + 1, y - 1);
      }
    }
  }
}

// Aplică aceeași transformare pe dest care ar transforma src în START.
void remap(char* src, char* dest) {
  for (int i = 0; i < LENGTH; i++) {
    int j = 0;
    while (src[j] != dest[i]) {
      j++;
    }
    dest[i] = START[j];
  }
}

void process_queries() {
  char src[LENGTH + 1], dest[LENGTH + 1];
  while ((scanf("%s %s", src, dest) == 2) && strcmp(src, "*")) {
    remap(src, dest);
    printf("%d\n", dist[rank(dest)]);
  }
}

int main() {
  precompute();
  bfs();
  process_queries();

  return 0;
}
```

24.5 Problema Four Chips (SPOJ)

‹ înapoi

```
#include <stdio.h>

typedef unsigned char byte;

const int SIZE = 70;
const int NUM_BOARDS = 916'895; // comb(SIZE, 4)
const byte INFINITY = 0xff;
const int NUM_MOVES = 20; // 4 piese, fiecare cu 2 deplasări și 3 salturi
const int JUMP_TARGET[4][3] = { {1, 2, 3}, {0, 2, 3}, {0, 1, 3}, {0, 1, 2} };

int comb[SIZE + 1][5];

struct board {
  byte p[4];
  int rank;

  board operator=(const board& other) {
    for (int i = 0; i < 4; i++) {
      p[i] = other.p[i];
    }
    rank = other.rank;
    return *this;
  }

  void compute_rank() {
    rank =
      (comb[SIZE][4] - comb[SIZE - p[0]][4]) +
      (comb[SIZE - 1 - p[0]][3] - comb[SIZE - p[1]][3]) +
      (comb[SIZE - 1 - p[1]][2] - comb[SIZE - p[2]][2]) +
      (p[3] - p[2] - 1);
  }

  void sort(int i) {
    while ((i > 0) && (p[i] < p[i - 1])) {
      int tmp = p[i]; p[i] = p[i - 1]; p[i - 1] = tmp;
      i--;
    }
    while ((i < 3) && (p[i] > p[i + 1])) {
      int tmp = p[i]; p[i] = p[i + 1]; p[i + 1] = tmp;
      i++;
    }
  }

  bool empty(int pos) {
    return (pos >= 0) && (pos < SIZE) && (p[0] != pos) &&
      (p[1] != pos) && (p[2] != pos) && (p[3] != pos);
  }

  bool make_slide_move(int i, int delta) {
```

```
    if (empty(p[i] + delta)) {
      p[i] += delta;
      // nu este nevoie de resortare
      return true;
    } else {
      return false;
    }
  }

  bool make_jump_move(int i, int j) {
    int x = 2 * p[j] - p[i];
    if (empty(x)) {
      p[i] = x;
      sort(i);
      return true;
    } else {
      return false;
    }
  }

  bool make_move(int index) {
    int i = index / 5, j;
    int type = index % 5;

    bool success;
    switch (type) {
      case 0:
      case 1:
      case 2:
        j = JUMP_TARGET[i][type];
        success = make_jump_move(i, j);
        break;

      case 3:
        success = make_slide_move(i, +1);
        break;

      case 4:
        success = make_slide_move(i, -1);
        break;
    }

    if (success) {
      compute_rank();
    }

    return success;
  }
};
```

```
const board START = { { 0, 1, 2, 3 }, 0 };

board q[NUM_BOARDS];
byte dist[NUM_BOARDS];

void precompute_combinations() {
  comb[0][0] = 1;
  for (int n = 1; n <= SIZE; n++) {
    comb[n][0] = 1;
    for (int k = 1; k <= 4; k++) {
      comb[n][k] = comb[n - 1][k] + comb[n - 1][k - 1];
    }
  }
}

void bfs() {
  for (int i = 0; i < NUM_BOARDS; i++) {
    dist[i] = INFINITY;
  }
  dist[0] = 0;

  int head = 0, tail = 0;
  q[tail++] = START;

  while (head != tail) {
    board& b = q[head++];
    board b2;

    for (int m = 0; m < NUM_MOVES; m++) {
      b2 = b;
      if (b2.make_move(m) && (dist[b2.rank] == INFINITY)) {
        dist[b2.rank] = 1 + dist[b.rank];
        q[tail++] = b2;
      }
    }
  }
}

void process_queries() {
  int num_tests;
  board b;

  scanf("%d", &num_tests);

  while (num_tests--) {
    scanf("%hhd %hhd %hhd %hhd", &b.p[0], &b.p[1], &b.p[2], &b.p[3]);
    b.p[0]--;
    b.p[1]--;
    b.p[2]--;
    b.p[3]--;
```

```
    b.compute_rank();
    printf("%d\n", dist[b.rank]);
  }
}

int main() {
  precompute_combinations();
  bfs();
  process_queries();

  return 0;
}
```

24.6 Problema Long Permutation (Codeforces)

înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 14;
const int T_RANGE_SUM = 1;
const long long FACT[MAX_N] = {
  1, 1, 2, 6, 24, 120, 720, 5'040, 40'320, 362'880, 3'628'800, 39'916'800,
  479'001'600, 6'227'020'800,
};

// O permutare normală a mulțimii [0...n-1].
struct permutation {
  int v[MAX_N], psum[MAX_N];
  int n;

  void init(int n) {
    this->n = n;
  }

  int get_kth_free(int mask, int k) {
    int pos = 0;
    while (k || (mask & (1 << pos))) {
      k -= !(mask & (1 << pos));
      pos++;
    }
    return pos;
  }

  void unrank(long long r) {
    int occupancy = 0;
    for (int i = 0; i < n; i++) {
      int d = r / FACT[n - 1 - i];
```

```
      d = get_kth_free(occupancy, d);
      v[i] = d;
      psum[i] = v[i] + (i ? psum[i - 1] : 0);
      occupancy |= 1 << d;
      r %= FACT[n - 1 - i];
    }
  }
};

// O permutare care este sortată pe prefix și permutată doar pe sufix.
struct permutation_with_cherry_on_top {
  permutation p;
  int n, head, tail;
  long long rank;

  void init(int n) {
    this->n = n;
    rank = 0;
    tail = (n < MAX_N) ? n : MAX_N;
    head = n - tail;
    p.init(tail);
    p.unrank(0);
  }

  void advance(int x) {
    rank += x;
    p.unrank(rank);
  }

  long long prefix_sum(int k) {
    if (k <= head) {
      // Sumă pe interval în progresia sortată.
      return (long long)k * (k + 1) / 2;
    } else {
      // Toată partea sortată...
      return (long long)head * (head + 1) / 2
        // ...plus suma cozii permutate...
        + p.psum[k - head - 1]
        // ...plus faptul că elementele cozii încep de fapt la head+1
        + (long long)(k - head) * (head + 1);
    }
  }

  long long range_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
  }
};

permutation_with_cherry_on_top pc;
```

```
int main() {
  int n, q, type, l, r, x;
  scanf("%d %d", &n, &q);
  pc.init(n);

  while (q--) {
    scanf("%d", &type);
    if (type == T_RANGE_SUM) {
      scanf("%d %d", &l, &r);
      printf("%lld\n", pc.range_sum(l, r));
    } else {
      scanf("%d", &x);
      pc.advance(x);
    }
  }

  return 0;
}
```

24.7 Problema Arbperm2 (NerdArena)

‹ înapoi • versiune online

```
// Complexitate: O(n log n).
#include <stdio.h>

const int MAX_N = 100'000;

struct fenwick_tree {
  int v[MAX_N + 1];
  int n, max_p2;

  void init(int n) {
    this->n = n;
    max_p2 = 1 << (31 - __builtin_clz(n));
    for (int i = 1; i <= n; i++) {
      v[i] = 0;
    }
  }

  void set(int pos) {
    do {
      v[pos]++;
      pos += pos & -pos;
    } while (pos <= n);
  }

  int prefix_sum(int pos) {
```

```
    int s = 0;
    while (pos) {
      s += v[pos];
      pos &= pos - 1;
    }
    return s;
  }

  // Returnează ultima poziție unde numărul de zerouri ≤ k, datorită faptului
  // că AIB-ul este 1-based, pe cînd logica programului este 0-based.
  int get_kth_zero(int k) {
    int pos = 0;

    for (int size = max_p2; size; size >>= 1) {
      if ((pos + size <= n) && (size - v[pos + size] <= k)) {
        k -= size - v[pos + size];
        pos += size;
      }
    }

    return pos;
  }
};

int p[MAX_N];       // permutarea originală
int ord[MAX_N + 1]; // ord[val] = ordinea lui val printre elemente < val
fenwick_tree fen;
int n, k;

void read_data() {
  FILE* f = fopen("arbperm2.in", "r");
  fscanf(f, "%d %d", &n, &k);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &p[i]);
  }
  fclose(f);
}

void init_ord() {
  fen.init(n);
  for (int i = 0; i < n; i++) {
    ord[p[i]] = fen.prefix_sum(p[i]);
    fen.set(p[i]);
  }
}

void advance_ord() {
  for (int i = n; i >= 1; i--) {
    int old_k = k;
    k = (ord[i] + k) / i;
```

```
    ord[i] = (ord[i] + old_k) % i;
  }
}

void rebuild_p() {
  fen.init(n);
  for (int i = n; i >= 1; i--) {
    int pos = fen.get_kth_zero(ord[i]);
    p[pos] = i;
    fen.set(pos + 1);
  }
}

void write_answer() {
  FILE* f = fopen("arbperm2.out", "w");
  for (int i = 0; i < n; i++) {
    fprintf(f, "%d ", p[i]);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_data();
  init_ord();
  advance_ord();
  rebuild_p();
  write_answer();

  return 0;
}
```

24.8 Problema Hipersimetrie (ONI 2019, clasele 11-12)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>
#include <vector>

const int MAX_K = 1'000'000;
const int MAX_LOG = 30;
const int MOD = 1'000'000'007;
const int ROOT_N = 32'768;

typedef unsigned long long u64;

char k[MAX_K + 1];
```

```
int n, len_k;

struct powers_of_2 {
  // p: 2^0, 2^1, 2^2, ..., 2^32767
  // q: 2^0, 2^32768, 2^65536, 2^98304, ... 2^(32768*32767)
  int p[ROOT_N], q[ROOT_N];

  void precompute() {
    p[0] = 1;
    for (int i = 1; i < ROOT_N; i++) {
      p[i] = p[i - 1] * 2 % MOD;
    }

    q[0] = 1;
    q[1] = (2 * p[ROOT_N - 1]) % MOD;
    for (int i = 2; i < ROOT_N; i++) {
      q[i] = (u64)q[i - 1] * q[1] % MOD;
    }
  }

  int get(u64 e) {
    e %= (MOD - 1); // Mica teoremă a lui Fermat.
    u64 large = q[e / ROOT_N];
    u64 small = p[e % ROOT_N];
    return small * large % MOD;
  }
};

powers_of_2 pow2;

struct bit_stream {
  std::vector<bool> col[MAX_LOG];
  int height[MAX_LOG];
  int num_cols;

  void collect_heights(int n) {
    num_cols = 0;
    while (n) {
      if (n & 1) {
        height[num_cols++] = (n + 1) / 2;
      }
      n /= 2;
    }
  }

  void distribute_bits() {
    int c = 0;
    for (int b = len_k - 1; b >= 0; b--) {
      col[c].push_back(k[b] - '0');
      height[c]--;
```

```
      if (height[c] < height[c + 1]) {
        c++;
      } else {
        c = 0;
      }
    }
  }

  void init(int n) {
    collect_heights(n);
    distribute_bits();
  }

  std::vector<bool> get_next_column() {
    static int pos = 0;
    return col[pos++];
  }
};

bit_stream bs;

int compute_cross(int size) {
  std::vector<bool> col = bs.get_next_column();
  u64 pos = (u64)(size / 2) * (n + 1);

  u64 result = 0;

  if (col.size() && col[0]) {
    result += pow2.get(pos);
  }

  for (int i = 1; i < (int)col.size(); i++) {
    if (col[i]) {
      result += (u64)pow2.get(pos - (u64)n * i)
        + pow2.get(pos + (u64)n * i)
        + pow2.get(pos - i)
        + pow2.get(pos + i);
    }
  }

  return result % MOD;
}

int compute_matrix(int size) {
  if (!size) {
    return 0;
  }

  u64 cross = (size & 1) ? compute_cross(size) : 0;
```

```
  int shift = (1 + size) / 2;
  u64 small = compute_matrix(size / 2);
  u64 mult = (u64)(pow2.get(shift) + 1) * (pow2.get((u64)shift * n) + 1) % MOD;
  u64 corners = small * mult % MOD;

  return (corners + cross) % MOD;
}

void read_data() {
  FILE* f = fopen("hipersimetrie.in", "r");
  fscanf(f, "%d %s", &n, k);
  fclose(f);

  len_k = strlen(k);
}

void decrement_k() {
  char* s = k + len_k - 1;
  while (*s == '0') {
    *s = '1';
    s--;
  }
  *s = '0';
}

void write_result(int result) {
  FILE* f = fopen("hipersimetrie.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_data();
  decrement_k();
  pow2.precompute();
  bs.init(n);

  int result = compute_matrix(n);
  write_result(result);

  return 0;
}
```

24.9 Problema Trim (Baraj ONI 2023)

‹ înapoi • versiune online

```
#include <algorithm>
```

```cpp
#include <stdio.h>

const int MAX_BITS = 100;
const int MAX_QUERIES = 100'000;

struct query {
  long long pos;
  int orig_ind;
};

query q[MAX_QUERIES];
bool ans[MAX_QUERIES];
long long choose[MAX_BITS + 1][MAX_BITS + 1];
int n, num_queries;

void read_input_data() {
  FILE* f = fopen("trim.in", "r");
  fscanf(f, "%d %d", &n, &num_queries);
  for (int i = 0; i < num_queries; i++) {
    fscanf(f, "%lld", &q[i].pos);
    q[i].pos--;
    q[i].orig_ind = i;
  }
  fclose(f);
}

void precompute_combinations() {
  choose[0][0] = 1;
  for (int i = 1; i <= n; i++) {
    choose[i][0] = 1;
    for (int j = 1; j <= i; j++) {
      choose[i][j] = choose[i - 1][j] + choose[i - 1][j - 1];
    }
  }
}

void sort_queries() {
  std::sort(q, q + num_queries, [] (query a, query b) {
    return a.pos < b.pos;
  });
}

// Consideră toate măștile de n biți cu k biți de 1, unde biții 0 și n-1 sînt
// 1. Există C(n-2, k-2) astfel de măști. Din combinarea #comb (0-based)
// returnează bitul #pos (0-based, numărînd de la MSB spre LSB).
bool find_bit(int n, int k, long long comb, int pos) {
  if (pos == 0 || pos == n - 1) {
    return true;
  }
```

```
  n -= 2;
  k -= 2;
  pos--;

  while (pos--) {
    if (comb >= choose[n - 1][k]) {
      comb -= choose[n - 1][k];
      k--;
    }
    n--;
  }

  return (comb >= choose[n - 1][k]);
}

struct bit_stream {
  int qind;          // indicele interogării curente
  long long emitted; // numărul de biți emiși până acum

  // Grupul curent. De exemplu, pentru n = 5, vom emite grupurile:
  //
  // * numere cu 1 bit de 1 și de lățime 1
  // * numere cu 2 biți de 1 și de lățimi 2, 3, 4, 5
  // * numere cu 3 biți de 1 și de lățimi 3, 4, 5
  // * numere cu 4 biți de 1 și de lățimi 4, 5
  // * numere cu 5 biți de 1 și de lățime 5
  int num_bits, width;

  // Procesează separat numerele pe 1 bit. Există n astfel de numere.
  void answer_width_1() {
    while (qind < num_queries && q[qind].pos < n) {
      ans[q[qind++].orig_ind] = true;
    }
    emitted = n;
  }

  void advance_group() {
    if (width == n) {
      width = ++num_bits;
    } else {
      width++;
    }
  }

  void answer_queries_for_group() {
    long long combinations = choose[width - 2][num_bits - 2];
    int repetitions = n + 1 - width;
    long long comb_size = combinations * repetitions * width;

    while ((qind < num_queries) && (q[qind].pos < emitted + comb_size)) {
```

```
      long long offset = q[qind].pos - emitted;
      long long combination = offset / (repetitions * width);
      offset %= width;
      ans[q[qind++].orig_ind] = find_bit(width, num_bits, combination, offset);
    }

    emitted += comb_size;
  }

  // Emite biții în grupuri și răspunde la întrebări în ordine crescătoare.
  void emit() {
    qind = 0;
    num_bits = 1;
    width = n;

    answer_width_1();

    while (qind < num_queries) {
      advance_group();
      answer_queries_for_group();
    }
  }
};

bit_stream bs;

void write_answers() {
  FILE* f = fopen("trim.out", "w");
  for (int i = 0; i < num_queries; i++) {
    fputc(ans[i] + '0', f);
  }
  fputc('\n', f);
  fclose(f);
}

int main() {
  read_input_data();
  precompute_combinations();
  sort_queries();
  bs.emit();
  write_answers();

  return 0;
}
```

24.10 Problema Bubblesort (Lot 2010)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>

const int MAX_N = 300;
const long long BASE = 1e16;
const int DIGIT_GROUP = 16;
const int MAX_LEN = 40; // 300! < 10^615 < 10^{16*40}

int min(int x, int y) {
  return (x < y) ? x : y;
}

struct big_number {
  int len;
  long long v[MAX_LEN + 1];

  void init() {
    len = 1;
    v[0] = 1;
  }

  void read(FILE* f) {
    char s[MAX_LEN * DIGIT_GROUP];
    fscanf(f, "%s", s);
    int slen = strlen(s);
    long long coef = 1;
    for (int i = 0; i < slen; i++) {
      int digit = s[slen - 1 - i] - '0';
      if (i % DIGIT_GROUP == 0) {
        coef = 1;
        len++;
      }
      v[len - 1] += digit * coef;
      coef *= 10;
    }
  }

  void mult(int x) {
    int i = 0;
    long long carry = 0;
    while ((i < len) || carry) {
      carry = v[i] * x + carry;
      v[i] = carry % BASE;
      carry /= BASE;
      i++;
    }
    len = i;
  }
```

```
  void shrink() {
    while (len && !v[len - 1]) {
      len--;
    }
  }

  void div(int x) {
    long long carry = 0;
    for (int i = len - 1; i >= 0; i--) {
      carry = carry * BASE + v[i];
      v[i] = carry / x;
      carry %= x;
    }

    shrink();
  }

  // this = a - b. Funcționează și dacă this == a (deci calculează a -= b).
  void diff(big_number& a, big_number& b) {
    long long borrow = 0;
    len = a.len;

    for (int i = b.len; i < a.len; i++) {
      b.v[i] = 0;
    }

    for (int i = 0; i < len; i++) {
      v[i] = a.v[i] - b.v[i] - borrow;
      borrow = (v[i] < 0);
      v[i] += BASE * borrow;
    }

    shrink();
  }

  bool less_than(big_number& other) {
    if (len != other.len) {
      return (len < other.len);
    }

    int i = len - 1;
    while ((i >= 0) && (v[i] == other.v[i])) {
      i--;
    }
    return ((i >= 0) && (v[i] < other.v[i]));
  }
};

struct perm_counter {
  int choices[MAX_N];
```

```
  big_number count;
  int saved_choice;
  int n, m;

  void init(int n, int m) {
    this->n = n;
    this->m = m;
    count.init();

    for (int i = 0; i < n; i++) {
      choices[i] = min(m, n - i);
      count.mult(choices[i]);
    }
  }

  void clear(int i) {
    saved_choice = choices[i];
    if (choices[i]) {
      count.div(choices[i]);
    }
    choices[i] = 1;
  }

  void restore(int i) {
    choices[i] = saved_choice - 1;
    count.mult(choices[i]);
  }
};

perm_counter a, b;
big_number k; // 1-based ca să nu mai implementez decrementarea pe big_number
bool used[MAX_N + 1];
int sol[MAX_N];
int n, m;

void read_data() {
  FILE* f = fopen("bubblesort.in", "r");
  fscanf(f, "%d %d ", &n, &m);
  k.read(f);
  fclose(f);
}

void unrank() {
  a.init(n, m);
  b.init(n, m - 1);

  for (int i = 0; i < n; i++) {
    int val = -1;
    bool found = false;
    do {
```

```
      val++;
      if (!used[val]) {
        a.clear(val);
        b.clear(val);
        big_number count;
        count.diff(a.count, b.count);
        if (count.less_than(k)) {
          k.diff(k, count);
          a.restore(val);
          b.restore(val);
        } else {
          found = true;
        }
      }
    } while (!found);

    sol[i] = val;
    used[val] = true;
  }
}

void write_perm() {
  FILE* f = fopen("bubblesort.out", "w");
  for (int i = 0; i < n; i++) {
    fprintf(f, "%d%c", sol[i] + 1, (i < n - 1) ? ' ' : '\n');
  }
  fclose(f);
}

int main() {
  read_data();
  unrank();
  write_perm();

  return 0;
}
```

24.11 Problema Valentine's Day (infO(1) Cup 2023)

‹ înapoi • versiune online

```
// Complexitate: O(n log n log* n).
#include <set>
#include <stdio.h>

const int MAX_N = 300'000;

int a[MAX_N], p[MAX_N];
```

```
int freq[MAX_N + 1];
int choices[MAX_N + 1];
int n, k;

// Menține o mulțime de numere și produsul lor, limitat la k.
struct factor_set {
  int cnt[MAX_N + 1]; // cnt[x] = numărul de factori egali cu x
  int pos[MAX_N + 1]; // poziția lui x în lst[]
  int lst[MAX_N + 1]; // listă de factori distincți
  int n;              // numărul de factori distincți (mărimea lui lst)
  int total;          // numărul de factori

  void init() {
    n = total = 0;
  }

  void insert(int f) {
    if (f > 1) {
      cnt[f]++;
      total++;
      if (cnt[f] == 1) {
        pos[f] = n;
        lst[n++] = f;
      }
    }
  }

  void erase(int f) {
    if (f > 1) {
      cnt[f]--;
      total--;
      if (!cnt[f]) {
        n--;
        pos[lst[n]] = pos[f];
        lst[pos[f]] = lst[n];
      }
    }
  }

  int slow_product() {
    long long p = 1;
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < cnt[lst[i]]; j++) {
        p *= lst[i];
        if (p >= k) {
          return k;
        }
      }
    }
    return p;
```

```
  }

  int product() {
    if (total >= 31) {
      // Nu strică și poate ajuta ocazional.
      return k;
    } else {
      return slow_product();
    }
  }
};

factor_set fs;

struct disjoint_set_forest {
  int p[MAX_N + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  // Unifică stînga cu dreapta. Se așteaptă ca v > u. Fără union by rank.
  void unite(int u, int v) {
    p[find(u)] = find(v);
  }
};

disjoint_set_forest hand; // valorile din mînă, de așezat în p

void read_data() {
  scanf("%d %d", &n, &k);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
    freq[a[i]]++;
  }
}

void init_choices() {
  fs.init();

  int constr = 0; // numărul de constrîngeri <= val
  for (int val = 1; val <= n; val++) {
```

```
    constr += freq[val];
    choices[val] = constr - (val - 1); // val - 1 elemente au ales deja
    fs.insert(choices[val]);
  }
}

int find_value_to_satisfy(int constr) {
  int val = hand.find(constr);

  fs.erase(choices[val]);

  // Încearcă să avansezi din *it cît timp nu lăsăm în urmă mai mult de k
  // permutări.
  int p;
  while ((p = fs.product()) < k) {
    // Luăm constrîngerea constr, care este ≤ *it, așadar *it pierde o
    // variantă chiar dacă mai tîrziu alegem o valoare > *it.
    k -= p;
    choices[val]--;
    fs.insert(choices[val]);
    val = hand.find(val + 1);
    fs.erase(choices[val]);
  }

  return val;
}

void fill_permutation() {
  hand.init(n);

  for (int i = 0; i < n; i++) {
    p[i] = find_value_to_satisfy(a[i]);
    hand.unite(p[i], p[i] + 1);
  }
}

void write_answer() {
  for (int i = 0; i < n; i++) {
    printf("%d ", p[i]);
  }
  printf("\n");
}

int main() {
  read_data();
  init_choices();
  fill_permutation();
  write_answer();

  return 0;
```

```
}
```

Anexa 25

Rezolvarea recurențelor liniare prin exponențiere rapidă

25.1 Problema Al k-lea termen Fibonacci (Infoarena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MOD = 666'013;

struct matrix {
  long long a, b, c, d;
};

int read() {
  int n;

  FILE* f = fopen("kfib.in", "r");
  fscanf(f, "%d", &n);
  fclose(f);

  return n;
}

void write(int x) {
  FILE* f = fopen("kfib.out", "w");
  fprintf(f, "%d\n", x);
  fclose(f);
}

matrix mult(matrix p, matrix q) {
  return {
    (p.a * q.a + p.b * q.c) % MOD,
```

```
    (p.a * q.b + p.b * q.d) % MOD,
    (p.c * q.a + p.d * q.c) % MOD,
    (p.c * q.b + p.d * q.d) % MOD,
  };
}

matrix mod_pow(matrix a, int n) {
  matrix result = { 1, 0, 0, 1 };

  while (n) {
    if (n & 1) {
      result = mult(result, a);
    }
    a = mult(a, a);
    n >>= 1;
  }

  return result;
}

int fib(int n) {
  matrix m = { 1, 1, 1, 0 };
  matrix m_n = mod_pow(m, n - 1);
  return m_n.a;
}

int main() {
  int n = read();
  write(fib(n));

  return 0;
}
```

25.2 Problema Iepuri (Infoarena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MOD = 666'013;
const int SIZE = 3;

struct matrix {
  long long v[SIZE][SIZE];
};

matrix mult(matrix a, matrix b) {
  matrix res;
```

```
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      res.v[r][c] = 0;
      for (int i = 0; i < SIZE; i++) {
        res.v[r][c] += a.v[r][i] * b.v[i][c];
      }
      res.v[r][c] %= MOD;
    }
  }
  return res;
}

matrix mod_pow(matrix a, int n) {
  matrix result = {{ {1, 0, 0}, {0, 1, 0}, {0, 0, 1} }};

  while (n) {
    if (n & 1) {
      result = mult(result, a);
    }
    a = mult(a, a);
    n >>= 1;
  }

  return result;
}

void solve_test(FILE* fin, FILE* fout) {
  int x, y, z, a, b, c, n;
  fscanf(fin, "%d %d %d %d %d %d %d", &x, &y, &z, &a, &b, &c, &n);

  matrix m = {{ {a, b, c}, {1, 0, 0}, {0, 1, 0} }};
  matrix p = mod_pow(m, n - 2);
  int result = (p.v[0][0] * z + p.v[0][1] * y + p.v[0][2] * x) % MOD;

  fprintf(fout, "%d\n", result);
}

int main() {
  FILE* fin = fopen("iepuri.in", "r");
  FILE* fout = fopen("iepuri.out", "w");

  int num_tests;
  fscanf(fin, "%d", &num_tests);
  while (num_tests--) {
    solve_test(fin, fout);
  }

  fclose(fin);
  fclose(fout);
```

```
  return 0;
}
```

25.3 Problema Recurenta2 (Infoarena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MOD = 666'013;
const int SIZE = 7;

typedef long long matrix[SIZE][SIZE];

int n, x1, x2, a, b, c;

void read_data() {
  FILE* f = fopen("recurenta2.in", "r");
  fscanf(f, "%d %d %d %d %d %d", &a, &b, &c, &x1, &x2, &n);
  fclose(f);
}

void mult(matrix a, matrix b, matrix dest) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      dest[r][c] = 0;
      for (int i = 0; i < SIZE; i++) {
        dest[r][c] += a[r][i] * b[i][c];
      }
      dest[r][c] %= MOD;
    }
  }
}

void copy(matrix src, matrix dest) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      dest[r][c] = src[r][c];
    }
  }
}

void identity(matrix a) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      a[r][c] = 0;
    }
    a[r][r] = 1;
```

```
  }
}

void mod_pow(matrix a, int n, matrix dest) {
  matrix aux;
  identity(dest);

  while (n) {
    if (n & 1) {
      mult(dest, a, aux);
      copy(aux, dest);
    }
    mult(a, a, aux);
    copy(aux, a);
    n >>= 1;
  }
}

int solve_recurrence() {
  // Fiecare generație stochează
  //
  //     (X_n, X_{n-1}, n X_n, (n-1) X_{n-1}, S_n, n, 1).
  //
  // Aceasta este matricea de recurență.
  matrix m = {
    { a,     b,  0,  0,  0,  0,  c },
    { 1,     0,  0,  0,  0,  0,  0 },
    { a, 2 * b,  a,  b,  0,  c,  c },
    { 0,     0,  1,  0,  0,  0,  0 },
    { a, 2 * b,  a,  b,  1,  c,  c },
    { 0,     0,  0,  0,  0,  1,  1 },
    { 0,     0,  0,  0,  0,  0,  1 },
  };

  // Acesta este vectorul pentru n = 2.
  long long v[SIZE] = { x2, x1, 2 * x2, x1, 2 * x2 + x1, 2, 1 };

  matrix p;
  mod_pow(m, n - 2, p);

  // Acum S_n este p[4] * v.
  long long result = 0;
  for (int i = 0; i < SIZE; i++) {
    result += p[4][i] * v[i];
  }

  return result % MOD;
}

void write_answer(int answer) {
```

```
  FILE* f = fopen("recurenta2.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}

int main() {
  read_data();
  int answer = solve_recurrence();
  write_answer(answer);

  return 0;
}
```

25.4 Problema Graph Paths I (CSES)

înapoi

```
#include <stdio.h>

const int MOD = 1'000'000'007;
const int MAX_NODES = 100;

typedef int matrix[MAX_NODES][MAX_NODES];

matrix adj, aux, p;
int n, k;

void read_graph() {
  int m, u, v;
  scanf("%d %d %d", &n, &m, &k);

  while (m--) {
    scanf("%d %d", &u, &v);
    u--;
    v--;
    adj[u][v]++;
  }
}

void identity(matrix a) {
  for (int r = 0; r < n; r++) {
    for (int c = 0; c < n; c++) {
      a[r][c] = 0;
    }
    a[r][r] = 1;
  }
}
```

```
void copy(matrix src, matrix dest) {
  for (int r = 0; r < n; r++) {
    for (int c = 0; c < n; c++) {
      dest[r][c] = src[r][c];
    }
  }
}

// a *= b
void mult(matrix a, matrix b) {
  for (int r = 0; r < n; r++) {
    for (int c = 0; c < n; c++) {
      __int128 x = 0;
      for (int i = 0; i < n; i++) {
        x += (long long)a[r][i] * b[i][c];
      }
      aux[r][c] = x % MOD;
    }
  }
  copy(aux, a);
}

void mod_pow(matrix a, int n, matrix dest) {
  identity(dest);

  while (n) {
    if (n & 1) {
      mult(dest, a);
    }
    mult(a, a);
    n >>= 1;
  }
}

int count_paths() {
  mod_pow(adj, k, p);
  return p[0][n - 1];
}

int main() {
  read_graph();
  printf("%d\n", count_paths());

  return 0;
}
```

25.5 Problema Hprob (Infoarena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int SIZE = 24;

typedef double matrix[SIZE][SIZE];

struct perm {
  unsigned char a, b, c, d;
};

const perm UNRANK[SIZE] = {
  { 0, 1, 2, 3}, { 0, 1, 3, 2}, { 0, 2, 1, 3},
  { 0, 2, 3, 1}, { 0, 3, 1, 2}, { 0, 3, 2, 1},
  { 1, 0, 2, 3}, { 1, 0, 3, 2}, { 1, 2, 0, 3},
  { 1, 2, 3, 0}, { 1, 3, 0, 2}, { 1, 3, 2, 0},
  { 2, 0, 1, 3}, { 2, 0, 3, 1}, { 2, 1, 0, 3},
  { 2, 1, 3, 0}, { 2, 3, 0, 1}, { 2, 3, 1, 0},
  { 3, 0, 1, 2}, { 3, 0, 2, 1}, { 3, 1, 0, 2},
  { 3, 1, 2, 0}, { 3, 2, 0, 1}, { 3, 2, 1, 0},
};

unsigned char rank[4][4][4][4];

matrix m, aux, dest;
double p1, p2, p3;
int n;

void make_rank() {
  for (int i = 0; i < SIZE; i++) {
    perm p = UNRANK[i];
    rank[p.a][p.b][p.c][p.d] = i;
  }
}

void read_data(FILE* fin) {
  fscanf(fin, "%d %lf %lf %lf", &n, &p1, &p2, &p3);
}

int rot(int x) {
  perm p = UNRANK[x];
  return rank[p.d][p.a][p.b][p.c];
}

int mirror(int x) {
  perm p = UNRANK[x];
```

```
  return rank[p.d][p.c][p.b][p.a];
}

int swap(int x) {
  perm p = UNRANK[x];
  return rank[p.a][p.c][p.b][p.d];
}

void make_matrix() {
  for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
      m[i][j] = 0;
    }
    m[i][rot(i)] = p1;
    m[i][mirror(i)] = p2;
    m[i][swap(i)] = p3;
    m[i][i] = 1.0 - p1 - p2 - p3;
  }
}

void identity(matrix a) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      a[r][c] = 0;
    }
    a[r][r] = 1;
  }
}

void copy(matrix src, matrix dest) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      dest[r][c] = src[r][c];
    }
  }
}

// a *= b
void mult(matrix a, matrix b) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      aux[r][c] = 0;
      for (int i = 0; i < SIZE; i++) {
        aux[r][c] += a[r][i] * b[i][c];
      }
    }
  }
  copy(aux, a);
}
```

```
void bin_exp(matrix a, int n, matrix dest) {
  identity(dest);

  while (n) {
    if (n & 1) {
      mult(dest, a);
    }
    mult(a, a);
    n >>= 1;
  }
}

int main() {
  make_rank();

  FILE* fin = fopen("hprob.in", "r");
  FILE* fout = fopen("hprob.out", "w");

  int num_tests;
  fscanf(fin, "%d", &num_tests);
  while (num_tests--) {
    read_data(fin);
    make_matrix();
    bin_exp(m, n, dest);
    fprintf(fout, "%.5lf\n", dest[0][0]);
  }

  fclose(fin);
  fclose(fout);

  return 0;
}
```

25.6 Problema Chef and Riffles (CodeChef)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 300'000;

typedef int permutation[MAX_N + 1];

permutation p, dest, aux;
int n, k;

void read_data() {
  scanf("%d %d", &n, &k);
```

```
}

void make_perm() {
  int pos = 1;
  for (int i = 1; i <= n; i += 2) {
    p[pos++] = i;
  }
  for (int i = 2; i <= n; i += 2) {
    p[pos++] = i;
  }
}

void identity(permutation p) {
  for (int i = 1; i <= n; i++) {
    p[i] = i;
  }
}

// p[i] <- p[q[i]]
void mult(permutation p, permutation q) {
  for (int i = 1; i <= n; i++) {
    aux[i] = p[q[i]];
  }
  for (int i = 1; i <= n; i++) {
    p[i] = aux[i];
  }
}

void bin_exp(permutation a, int k, permutation dest) {
  identity(dest);

  while (k) {
    if (k & 1) {
      mult(dest, a);
    }
    mult(a, a);
    k >>= 1;
  }
}

void write_permutation(permutation p) {
  for (int i = 1; i <= n; i++) {
    printf("%d ", p[i]);
  }
  printf("\n");
}

void solve_test() {
  read_data();
  make_perm();
```

```
  bin_exp(p, k, dest);
  write_permutation(dest);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

25.7 Problema Zid (ONI 2024, clasele 11-12)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX = 5'000;
const int MOD = 1'000'000'007;

int n, m, k;
int t[MAX + 1];
int col_sum[MAX + 1];
int diag_sum[MAX + 1];
int mat[MAX / 2];
int ans[MAX / 2];

void read_data() {
  FILE* f = fopen("zid.in", "r");
  fscanf(f, "%d %d %d", &n, &m, &k);
  fclose(f);
}

void compute_width_1_tower() {
  for (int even = 0; 2 * even <= k; even++) {
    t[0] = t[1] = col_sum[0] = col_sum[1] = (even == 0);
    for (int h = 2; h <= n; h++) {
      t[h] = (col_sum[h - 1] + diag_sum[h - 2]) % MOD;
      col_sum[h] = (t[h] + col_sum[h - 2]) % MOD;
    }

    for (int h = n; h >= 2; h--) {
      diag_sum[h] = (t[h] + diag_sum[h - 2]) % MOD;
    }
    diag_sum[0] = diag_sum[1] = t[0];
```

```
    mat[even] = t[n];
  }
}

void col_mult(int* dest, int* src) {
  for (int i = k / 2; i >= 0; i--) {
    __int128 sum = 0;
    for (int j = 0; j <= i; j++) {
      sum += (long long)src[j] * dest[i - j];
    }
    dest[i] = sum % MOD;
  }
}

void compute_width_m_wall() {
  ans[0] = 1;
  while (m) {
    if (m & 1) {
      col_mult(ans, mat);
    }
    col_mult(mat, mat);
    m >>= 1;
  }
}

void write_answer() {
  FILE* f = fopen("zid.out", "w");
  fprintf(f, "%d\n", (k & 1) ? 0 : ans[k / 2]);
  fclose(f);
}

int main() {
  read_data();
  compute_width_1_tower();
  compute_width_m_wall();
  write_answer();

  return 0;
}
```

25.8 Problema Stairs and Lines (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int HEIGHT = 7;
```

```
const int SIZE = 1 << HEIGHT;
const int MOD = 1'000'000'007;

typedef int matrix[SIZE][SIZE];

matrix t;   // matricea de tranziție
matrix acc; // acumulatorul
matrix aux;

void make_transition_matrix(int h) {
  // Forțează pereți verticali de la h la HEIGHT - 1.
  int start_mask = SIZE - (1 << h);

  for (int left = start_mask; left < SIZE; left++) {
    for (int right = start_mask; right < SIZE; right++) {
      t[left][right] = 0;
      // Forțează pereți orizontali la înălțimile 0 și h.
      for (int horiz = 1 + (1 << h); horiz < (1 << (h + 1)); horiz += 2) {
        if ((left & right & horiz & (horiz >> 1)) == 0) {
          t[left][right]++;
        }
      }
    }
  }
}

void identity(matrix a) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      a[r][c] = 0;
    }
    a[r][r] = 1;
  }
}

void copy(matrix src, matrix dest) {
  for (int r = 0; r < SIZE; r++) {
    for (int c = 0; c < SIZE; c++) {
      dest[r][c] = src[r][c];
    }
  }
}

// a *= b
void mult(matrix a, matrix b, int start) {
  for (int r = start; r < SIZE; r++) {
    for (int c = start; c < SIZE; c++) {
      __int128 x = 0;
      for (int i = 0; i < SIZE; i++) {
        x += (long long)a[r][i] * b[i][c];
```

```
      }
      aux[r][c] = x % MOD;
    }
  }
  copy(aux, a);
}

// Matricea a este goală pe primele start linii și coloane.
void bin_exp(matrix a, int n, int start) {
  while (n) {
    if (n & 1) {
      mult(acc, a, start);
    }
    mult(a, a, start);
    n >>= 1;
  }
}

int main() {
  int w;

  identity(acc);
  for (int h = 1; h <= HEIGHT; h++) {
    scanf("%d", &w);
    make_transition_matrix(h);
    bin_exp(t, w, SIZE - (1 << h));
  }

  printf("%d\n", acc[SIZE - 1][SIZE - 1]);
}
```

25.9 Problema Marfa (Lot 2014)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 19;
const int MAX_K = 4;
const int MOD = 40'099;

// Fiecare celulă codifică o combinație (l,r). De aceea mărimile sînt
// pătratice.
typedef int vector[MAX_K * MAX_K];
typedef int matrix[MAX_K * MAX_K][MAX_K * MAX_K];

int z, k, n;
int daily_demand[MAX_N + 1];
```

```
// Funcții utilitare. Sînt plicticoase. Dar tocmai asta este ideea.
// Programele noastre nu trebuie să fie ezoterice. Nu trebuie să existe
// obscuritate intenționată. Nu trebuie să existe surprize. Programul
// propriu-zis este destul de dificil fără să-l mai și poluăm cu aceste
// blocuri. Dacă vreți senzații tari, dați-vă în roller coaster.

void vec_zero(vector a) {
  for (int i = 0; i < k * k; i++) {
    a[i] = 0;
  }
}

int vec_sum(vector a) {
  int sum = 0;
  for (int i = 0; i < k * k; i++) {
    sum += a[i];
  }

  return sum % MOD;
}

void vec_copy(vector src, vector dest) {
  for (int i = 0; i < k * k; i++) {
    dest[i] = src[i];
  }
}

void mat_copy(matrix src, matrix dest) {
  for (int r = 0; r < k * k; r++) {
    vec_copy(src[r], dest[r]);
  }
}

void mat_identity(matrix a) {
  for (int r = 0; r < k * k; r++) {
    vec_zero(a[r]);
    a[r][r] = 1;
  }
}

// a *= b
void mat_mult(matrix a, matrix b) {
  static matrix aux;

  for (int r = 0; r < k * k; r++) {
    for (int c = 0; c < k * k; c++) {
      long long x = 0;
      for (int i = 0; i < k * k; i++) {
        x += a[r][i] * b[i][c];
```

```
      }
      aux[r][c] = x % MOD;
    }
  }
  mat_copy(aux, a);
}

// Îl distruge pe a.
void mat_power(matrix a, int n, matrix dest) {
  mat_identity(dest);
  while (n) {
    if (n & 1) {
      mat_mult(dest, a);
    }
    mat_mult(a, a);
    n >>= 1;
  }
}

void read_data() {
  FILE* f = fopen("marfa.in", "r");
  fscanf(f, "%d %d %d", &z, &k, &n);

  for (int i = 1; i <= n; i++) {
    fscanf(f, "%d", &daily_demand[i]);
  }
  fclose(f);
}

// Date fiind numerele de posibilități pentru fiecare (l < k, r < k),
// calculează noile numere după o zi cu cererea dată.
void advance_day(vector a, int demand) {
  static vector aux;
  vec_zero(aux);

  for (int l = 0; l < k; l++) {
    for (int r = 0; r < k; r++) {
      int q = a[l * k + r];
      if (demand == 0) {
        // resetează l și r la 0
        aux[0] += q;
      } else if (demand == 1) {
        // crește-l pe l și resetează-l pe r sau viceversa
        if (l < k - 1) {
          aux[(l + 1) * k] += q;
        }
        if (r < k - 1) {
          aux[r + 1] += q;
        }
      } else {
```

```
        // crește-le și pe l și pe r
        if ((l < k - 1) && (r < k - 1)) {
          aux[(l + 1) * k + (r + 1)] += q;
        }
      }
    }
  }

  vec_copy(aux, a);
}

void make_transition_matrices(matrix week, matrix rem) {
  vector a;

  for (int l = 0; l < k; l++) {
    for (int r = 0; r < k; r++) {
      vec_zero(a);
      a[l * k + r] = 1;

      for (int dow = 1; dow <= n; dow++) { // ziua din săptămînă
        advance_day(a, daily_demand[dow]);
        if (dow == z % n) {
          vec_copy(a, rem[l * k + r]);
        }
      }

      vec_copy(a, week[l * k + r]);
    }
  }
}

int count_ways() {
  matrix week, rem, all;

  make_transition_matrices(week, rem);
  mat_power(week, z / n, all);
  if (z % n) {
    mat_mult(all, rem);
  }

  // Începem în starea 0 și putem termina în orice stare.
  return vec_sum(all[0]);
}

void write_answer(int answer) {
  FILE* f = fopen("marfa.out", "w");
  fprintf(f, "%d\n", answer);
  fclose(f);
}
```

```
int main() {
  read_data();
  int answer = count_ways();
  write_answer(answer);

  return 0;
}
```

25.10 Problema Options (IIOT 2017-2018 runda internațională)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_ROWS = 100;
const int MOD = 666'013;

bool is_land[MAX_ROWS][MAX_ROWS];
int nr, nc, dest_r;
long long dest_c;

struct matrix {
  int a[MAX_ROWS][MAX_ROWS];

  void copy_from(matrix& other) {
    for (int r = 0; r < nr; r++) {
      for (int c = 0; c < nr; c++) {
        a[r][c] = other.a[r][c];
      }
    }
  }

  void identity() {
    for (int r = 0; r < nr; r++) {
      for (int c = 0; c < nr; c++) {
        a[r][c] = 0;
      }
      a[r][r] = 1;
    }
  }

  // self *= m. Nu modifică matricea m.
  void mult_self(matrix& m) {
    static matrix aux;

    for (int r = 0; r < nr; r++) {
```

```
      for (int c = 0; c < nr; c++) {
        long long x = 0;
        for (int i = 0; i < nr; i++) {
          x += (long long)a[r][i] * m.a[i][c];
        }
        aux.a[r][c] = x % MOD;
      }
    }
    this->copy_from(aux);
  }

  // Modifică matricea m.
  void power(matrix& m, long long n) {
    identity();
    while (n) {
      if (n & 1) {
        mult_self(m);
      }
      m.mult_self(m);
      n >>= 1;
    }
  }
};

matrix t;   // matricea de tranziție pentru perioade întregi
matrix rem; // matricea de tranziție pentru ultimele dest_c % nc zile

void read_data() {
  // Conceptual îmi este mai simplu să judec matricea de tranziție pentru
  // blocul de coloane [0, nc) astfel: t[r1][r2] = numărul de drumuri care
  // încep pe linia r1 *din stînga blocului* (deci coloana -1) și se termină
  // pe linia r2 de pe coloana nc-1. De aceea, la citire mult prima coloană
  // ultima.
  scanf("%d %d ", &nr, &nc);
  for (int r = 0; r < nr; r++) {
    is_land[r][nc - 1] = getchar() - '0';
    for (int c = 0; c < nc - 1; c++) {
      is_land[r][c] = getchar() - '0';
    }
    getchar();
  }
  scanf("%d %lld", &dest_r, &dest_c);
  dest_r--;    // pentru că lucrăm 0-based
  dest_c -= 2; // pentru că lucrăm 0-based și am rotit matricea
}

void compute_transition_matrices() {
  matrix new_t;

  // Matricea t înainte de a face vreun pas.
```

```
  t.identity();

  for (int c = 0; c < nc; c++) {
    for (int r1 = 0; r1 < nr; r1++) {
      for (int r2 = 0; r2 < nr; r2++) {
        if (is_land[r2][c]) {
          int up = (r2 > 0) ? t.a[r1][r2 - 1] : 0;
          int mid = t.a[r1][r2];
          int down = (r2 < nr - 1) ? t.a[r1][r2 + 1] : 0;
          new_t.a[r1][r2] = (up + mid + down) % MOD;
        } else {
          new_t.a[r1][r2] = 0;
        }
      }
    }
    t.copy_from(new_t);
    // Invariant: t este matricea după parcurgerea coloanelor [0...c].
    if (c == dest_c % nc) {
      rem.copy_from(t);
    }
  }
}

int count_ways() {
  matrix overall;
  overall.power(t, dest_c / nc);
  overall.mult_self(rem);

  return overall.a[0][dest_r];
}

int main() {
  read_data();
  compute_transition_matrices();
  int answer = count_ways();
  printf("%d\n", answer);

  return 0;
}
```

Anexa 26

Geometrie - Elemente de bază

26.1 Problema Cuiburi (Baraj ONI 2010)

‹ înapoi • versiune online

```
#include <algorithm>
#include <math.h>
#include <stdio.h>

const int MAX_NESTS = 2'000;
const int T_RECTANGLE = 0;
const int T_CIRCLE = 1;

bool closer(int x1, int y1, int x2, int y2, int dist) {
  return (x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1) <= dist * dist;
}

struct nest {
  double area;
  union { short x1; short x; };
  union { short y1; short y; };
  union { short x2; short r; };
  union { short y2; short unused; };
  unsigned char type;

  void read(FILE* f) {
    fscanf(f, "%hhu", &type);
    if (type == T_RECTANGLE) {
      fscanf(f, "%hd %hd %hd %hd", &x1, &y1, &x2, &y2);
    } else {
      fscanf(f, "%hd %hd %hd", &x, &y, &r);
    }

    compute_area();
  }
```

```
  void compute_area() {
    if (type == T_RECTANGLE) {
      area = ((int)x2 - x1) * ((int)y2 - y1);
    } else {
      area = M_PI * r * r;
    }
  }

  bool fits_in(nest& other) {
    if (type == T_RECTANGLE) {
      if (other.type == T_RECTANGLE) {
        return
          (x1 >= other.x1) && (y1 >= other.y1) &&
          (x2 <= other.x2) && (y2 <= other.y2);
      } else {
        return
          closer(x1, y1, other.x, other.y, other.r) &&
          closer(x1, y2, other.x, other.y, other.r) &&
          closer(x2, y1, other.x, other.y, other.r) &&
          closer(x2, y2, other.x, other.y, other.r);
      }
    } else {
      if (other.type == T_RECTANGLE) {
        return
          (x >= other.x1 + r) &&
          (x <= other.x2 - r) &&
          (y >= other.y1 + r) &&
          (y <= other.y2 - r);
      } else {
        return closer(x, y, other.x, other.y, other.r - r);
      }
    }
  }
};

nest a[MAX_NESTS];
short len[MAX_NESTS];
int n;

void read_nests() {
  FILE* f = fopen("cuiburi.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    a[i].read(f);
  }
  fclose(f);
}

void sort_nests() {
```

```
  std::sort(a, a + n, [](nest& p, nest& q) {
    return p.area < q.area;
  });
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

int find_longest_subsequence() {
  short max_len = 0;

  for (int i = 0; i < n; i++) {
    len[i] = 1;
    for (int j = 0; j < i; j++) {
      if (a[j].fits_in(a[i])) {
        len[i] = max(len[i], 1 + len[j]);
      }
    }

    max_len = max(max_len, len[i]);
  }

  return max_len;
}

void write_result(int result) {
  FILE* f = fopen("cuiburi.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_nests();
  sort_nests();
  int result = find_longest_subsequence();
  write_result(result);

  return 0;
}
```

26.2 Problema Ace (OJI 2017, clasa a 9-a)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_ROWS = 1'000;
```

```
short a[MAX_ROWS][MAX_ROWS];
int task, rows, cols, result;

void read_matrix() {
  FILE* f = fopen("ace.in", "r");
  fscanf(f, "%d %d %d", &task, &rows, &cols);

  for (int r = rows - 1; r >= 0; r--) {
    for (int c = cols - 1; c >= 0; c--) {
      fscanf(f, "%hd", &a[r][c]);
    }
  }

  fclose(f);
}

// Numără de la (dr,dc) și avansînd cu (dr,dc). Pentru distanța pe orizontală
// ar trebui să folosim numărul iterației, dar orice combinație liniară de r
// și c este bună. Deci folosim (r+c) ca să eliminăm o variabilă.
void count_direction(int dr, int dc) {
  int max_h = 0, max_g = 1;

  for (int r = dr, c = dc;
       r < rows && c < cols;
       r += dr, c += dc) {
    if (a[r][c] * max_g > max_h * (r + c)) {
      result++;
      max_h = a[r][c];
      max_g = r + c;
    }
    a[r][c] = 0;
  }
}

void count_visible() {
  if (task == 1) {
    count_direction(0, 1);
    count_direction(1, 0);
  } else {
    for (int dr = 0; dr < rows; dr++) {
      for (int dc = 0; dc < cols; dc++) {
        if (a[dr][dc]) {
          count_direction(dr, dc);
        }
      }
    }
  }
}
```

```
void write_result() {
  FILE* f = fopen("ace.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_matrix();
  count_visible();
  write_result();
  return 0;
}
```

26.3 Problema Baba Oarba (Lot 2016)

‹ înapoi • versiune online

```
#include "babaoarba.h"
#include <math.h>

const int NUM_JIGGLES = 25;
const int CLOSER = 1;
const int FURTHER = 0;
const int FOUND = -1;

void play() {
  double l = 0.0, r = 2 * M_PI;
  bool found = false;

  int iter = 0;
  do {
    double mid = (l + r) / 2;
    int answer = makeStep(mid + M_PI / 2);
    if (answer == CLOSER) {
      l = mid;
      // There and back again.
      makeStep(mid - M_PI / 2);
    } else if (answer == FURTHER) {
      r = mid;
      makeStep(mid - M_PI / 2);
    } else {
      found = true;
    }
  } while (!found && (++iter < NUM_JIGGLES));

  double angle = (l + r) / 2;

  while (!found) {
```

```
    found = (makeStep(angle) == FOUND);
  }
}
```

Un fișier `babaoarba.h`, util pentru testarea offline:

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

const int MAX_COORD = 70'000;

extern void play();

struct point {
  double x, y;

  double dist2() {
    return x * x + y * y;
  }

};

point tassadar;

void init_rng() {
  struct timeval time;
  gettimeofday(&time, NULL);
  int micros = time.tv_sec * 1'000'000 + time.tv_usec;
  srand(micros);
}

int makeStep(double angle) {
  printf("Got called with angle %.6lf\n", angle);
  point new_t = {
    tassadar.x + cos(angle),
    tassadar.y + sin(angle),
  };

  int result;
  if (new_t.dist2() <= 1) {
    result = -1;
  } else if (new_t.dist2() < tassadar.dist2()) {
    result = 1;
  } else {
    result = 0;
  }
```

```
  tassadar = new_t;
  printf("New coords (%.6lf,%.6lf), dist %.6lf, result %d\n",
         tassadar.x, tassadar.y, sqrt(tassadar.dist2()), result);

  return result;
}

int main() {
  init_rng();

  tassadar.x = rand() % MAX_COORD + 2;
  tassadar.y = rand() % MAX_COORD + 2;
  printf("Chose %.0lf %.0lf\n", tassadar.x, tassadar.y);
  play();

  return 0;
}
```

26.4 Problema Arhitect (OJI 2023, clasa a 10-a)

înapoi

Sursă cu perechi (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;

struct slope {
  int dx, dy;

  // Ne interesează doar pante în [0°, 90°). Vom reformula o pantă de 110° ca
  // pe o pantă de 20° pentru a le număra împreună.
  void set(int dx, int dy) {
    // Rotește cu 90° până când ajungem la x > 0 și y >= 0.
    while ((dx <= 0) || (dy < 0)) {
      int tmp = dy;
      dy = -dx;
      dx = tmp;
    }
    this->dx = dx;
    this->dy = dy;
  }

  bool operator ==(const slope& o) const {
    return (long long)dx * o.dy == (long long)dy * o.dx;
  }
```

```
  bool operator <(const slope& o) const {
    return (long long)dx * o.dy < (long long)dy * o.dx;
  }
};

slope s[MAX_N];
int n;

void read_slopes() {
  FILE* f = fopen("arhitect.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    int x1, y1, x2, y2;
    fscanf(f, "%d %d %d %d", &x1, &y1, &x2, &y2);
    s[i].set(x2 - x1, y2 - y1);
  }
  fclose(f);
}

int count_duplicates() {
  std::sort(s, s + n);
  int run = 1, max_run = 1;
  for (int i = 1; i < n; i++) {
    run = (s[i] == s[i - 1]) ? (run + 1) : 1;
    if (run > max_run) {
      max_run = run;
    }
  }
  return max_run;
}

void write_result(int result) {
  FILE* f = fopen("arhitect.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_slopes();
  int result = count_duplicates();
  write_result(result);
  return 0;
}
```

Sursă cu numere reale (versiune online).

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_N = 100'000;

double s[MAX_N];
int n;

// Ne interesează doar pante în [0°, 90°). Vom reformula o pantă de 110° ca pe
// o pantă de 20° pentru a le număra împreună.
double make_slope(int dx, int dy) {
  // Rotește cu 90° până când ajungem la x > 0 și y >= 0.
  while ((dx <= 0) || (dy < 0)) {
    int tmp = dy;
    dy = -dx;
    dx = tmp;
  }
  return (double)dy / dx;
}

void read_slopes() {
  FILE* f = fopen("arhitect.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    int x1, y1, x2, y2;
    fscanf(f, "%d %d %d %d", &x1, &y1, &x2, &y2);
    s[i] = make_slope(x2 - x1, y2 - y1);
  }
  fclose(f);
}

int count_duplicates() {
  std::sort(s, s + n);
  int run = 1, max_run = 1;
  for (int i = 1; i < n; i++) {
    run = (s[i] == s[i - 1]) ? (run + 1) : 1;
    if (run > max_run) {
      max_run = run;
    }
  }
  return max_run;
}

void write_result(int result) {
  FILE* f = fopen("arhitect.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_slopes();
  int result = count_duplicates();
```

```
  write_result(result);
  return 0;
}
```

26.5 Problema Elicoptere (OJI 2016, clasele 11-12)

înapoi • versiune online

```
#include <algorithm>
#include <math.h>
#include <stdio.h>

const int MAX_N = 100;
const int MAX_EDGES = MAX_N * (MAX_N - 1) / 2;

struct point {
  int x, y;
};

struct triangle {
  point a, b, c;
};

struct edge {
  int u, v;
  double d;
};

triangle t[MAX_N];
edge e[MAX_EDGES];
int comp[MAX_N];
int n, num_edges, k, task, num_selected_edges, num_connected_pairs;
double total_length;

void read_data() {
  FILE* f = fopen("elicoptere.in", "r");
  fscanf(f, "%d %d %d", &task, &n, &k);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d %d %d %d %d %d",
           &t[i].a.x, &t[i].a.y, &t[i].b.x, &t[i].b.y, &t[i].c.x, &t[i].c.y);
  }
  fclose(f);
}

double min(double a, double b) {
  return (a < b) ? a : b;
}
```

```
double min3(double a, double b, double c) {
  return min(min(a, b), c);
}

double dist_horiz(point p, point a, point b) {
  if ((a.y == p.y) && (p.y == b.y)) {
    // a, b, p coliniare pe orizontală
    return min(fabs(p.x - a.x), fabs(p.x - b.x));
  } else if ((long long)(p.y - a.y) * (p.y - b.y) <= 0) {
    // p.y se află între a.y și b.y
    double x = a.x + (double)(p.y - a.y) * (b.x - a.x) / (b.y - a.y);
    return fabs(p.x - x);
  } else {
    // orizontala din p *nu* întîlnește ab; returnează infinit
    return k + 1;
  }
}

double dist_point_segment(point p, point a, point b) {
  // Pentru a afla distanța pe verticală, oglindim punctele față de diagonală.
  return min(dist_horiz(p, a, b),
             dist_horiz({p.y, p.x}, {a.y, a.x}, {b.y, b.x}));
}

double dist_point_triangle(point p, triangle t) {
  return min3(dist_point_segment(p, t.a, t.b),
              dist_point_segment(p, t.b, t.c),
              dist_point_segment(p, t.c, t.a));
}

double dist_triangle_triangle(triangle& g, triangle& h) {
  return min(min3(dist_point_triangle(g.a, h),
                  dist_point_triangle(g.b, h),
                  dist_point_triangle(g.c, h)),
             min3(dist_point_triangle(h.a, g),
                  dist_point_triangle(h.b, g),
                  dist_point_triangle(h.c, g)));
}

void create_edges() {
  for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
      double d = dist_triangle_triangle(t[i], t[j]);
      if (d <= k) {
        e[num_edges++] = { i, j, d };
      }
    }
  }
}
```

```
void init_comp() {
  for (int i = 0; i < n; i++) {
    comp[i] = i;
  }
}

void unite(int u, int v) {
  for (int i = 0; i < n; i++) {
    if (comp[i] == u) {
      comp[i] = v;
    }
  }
}

void kruskal() {
  std::sort(e, e + num_edges, [](edge a, edge b) {
    return a.d < b.d;
  });

  init_comp();

  for (int i = 0; i < num_edges; i++) {
    if (comp[e[i].u] != comp[e[i].v]) {
      unite(comp[e[i].u], comp[e[i].v]);
      num_selected_edges++;
      total_length += e[i].d;
    }
  }
}

void write_answer() {
  FILE* f = fopen("elicoptere.out", "w");
  if (task == 1) {
    fprintf(f, "%d\n", num_selected_edges);
  } else if (task == 2) {
    fprintf(f, "%d\n", num_connected_pairs);
  } else {
    fprintf(f, "%0.6lf\n", total_length);
  }
  fclose(f);
}

void count_connected_pairs() {
  for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
      num_connected_pairs += (comp[i] == comp[j]);
    }
  }
}
```

```
int main() {
  read_data();
  create_edges();
  kruskal();
  count_connected_pairs();
  write_answer();
}
```

26.6 Problema Arpa and an Exam About Geometry (Codeforces)

‹ înapoi • versiune online

```
#include <iostream>

long long dist2(long long ax, long long ay, long long bx, long long by) {
  return ((ax - bx) * (ax - bx) + (ay - by) * (ay - by));
}

int main() {
  long long ax, ay, bx, by, cx, cy;
  std::cin >> ax >> ay >> bx >> by >> cx >> cy;

  bool noncollinear = ((cx - ax) * (by - ay) - (cy - ay) * (bx - ax) != 0);
  bool equidistant = dist2(ax, ay, bx, by) == dist2(bx, by, cx, cy);

  std::cout << ((noncollinear && equidistant) ? "Yes\n" : "No\n");

  return 0;
}
```

26.7 Problema Part (Baraj ONI 2006)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_TRIANGLES = 150;

struct point {
  int x, y;

  bool on_board(int a, int b) {
    return a * y + b * x <= a * b;
  }
```

```
};

// Ar sta mai bine pe triangle, dar aceasta ar crea o dependință circulară.
long long triangle_area(point p, point q, point r) {
  return (r.x - p.x) * (q.y - p.y) - (r.y - p.y) * (q.x - p.x);
}

struct segment {
  point p, q;

  // Exclude intersecțiile degenerate (pe drepte-suport confundate sau la
  // capetele segmentelor).
  bool intersects(segment other) {
    long long a1 = triangle_area(p, q, other.p);
    long long a2 = triangle_area(p, q, other.q);
    long long a3 = triangle_area(other.p, other.q, p);
    long long a4 = triangle_area(other.p, other.q, q);
    return (a1 * a2 < 0) && (a3 * a4 < 0);
  }
};

struct triangle {
  point p, q, r;

  void init(FILE* f) {
    fscanf(f, "%d %d %d %d %d %d", &p.x, &p.y, &q.x, &q.y, &r.x, &r.y);
    if (area() < 0) {
      point tmp = q;
      q = r;
      r = tmp;
    }
  }

  long long area() {
    return triangle_area(p, q, r);
  }

  bool on_board(int a, int b) {
    return
      p.on_board(a, b) &&
      q.on_board(a, b) &&
      r.on_board(a, b);
  }

  // Două scenarii:
  //
  // 1. două segmente se intersectează (nedegenerat), sau
  // 2. unul dintre triunghiuri are toate punctele în interiorul sau pe
  //    laturile celuilalt.
  bool overlaps(triangle& other) {
```

```
    return
      intersects_segment({other.p, other.q}) ||
      intersects_segment({other.q, other.r}) ||
      intersects_segment({other.r, other.p}) ||

      contains(other) ||
      other.contains(*this);
  }

  bool intersects_segment(segment s) {
    return
      segment(p, q).intersects(s) ||
      segment(q, r).intersects(s) ||
      segment(r, p).intersects(s);
  }

  bool contains(triangle& other) {
    return
      contains(other.p) &&
      contains(other.q) &&
      contains(other.r);
  }

  bool contains(point a) {
    return
      (triangle_area(p, q, a) >= 0) &&
      (triangle_area(q, r, a) >= 0) &&
      (triangle_area(r, p, a) >= 0);
  }
};

triangle t[MAX_TRIANGLES];
FILE *fin, *fout;

void process_test() {
  int a, b, n;
  fscanf(fin, "%d %d %d", &a, &b, &n);

  bool all_on_board = true;
  bool overlaps = false;
  long long total_area = 0;

  for (int i = 0; i < n; i++) {
    t[i].init(fin);
    total_area += t[i].area();
    all_on_board = all_on_board && t[i].on_board(a, b);
    for (int j = 0; j < i; j++) {
      overlaps = overlaps || t[i].overlaps(t[j]);
    }
  }
```

```
  bool same_area = (total_area == a * b);
  fprintf(fout, "%d\n", same_area && all_on_board && !overlaps);
}

int main() {
  fin = fopen("part.in", "r");
  fout = fopen("part.out", "w");

  int num_tests;
  fscanf(fin, "%d", &num_tests);
  while (num_tests--) {
    process_test();
  }

  fclose(fin);
  fclose(fout);

  return 0;
}
```

26.8 Problema Bear and Floodlight (Codeforces)

‹ înapoi • versiune online

```
#define _USE_MATH_DEFINES // https://codeforces.com/blog/entry/51835
#include <math.h>
#include <stdio.h>

const int MAX_N = 20;

double min(double x, double y) {
  return (x < y) ? x : y;
}

double max(double x, double y) {
  return (x > y) ? x : y;
}

struct floodlight {
  int xc, yc;
  double c, s;

  void setAngle(int angle) {
    double rad = M_PI * angle / 180;
    c = cos(rad);
    s = sin(rad);
  }
```

```
  // Extinde segmentul vizibil la dreapta de la x, folosind unghiul acestui
  // reflector.
  double extend(double x, int limit) {
    // Rotește (x, 0) cu <c,s> grade în jurul lui (xc, yc).
    double rotx = (x - xc) * c - (0 - yc)* s + xc;
    double roty = (x - xc) * s + (0 - yc)* c + yc;

    if (roty >= yc) {
      // Reflectorul bate la dreapta sau chiar mai sus, deci acoperă axa Ox
      // până la infinit (și dincolo!)
      return limit;
    }

    // Intersectează segmentul (xc, yc) - (rotx, roty) cu axa Ox.
    double newx = (xc * roty - yc * rotx) / (roty - yc);
    return min(newx, limit);
  }
};

floodlight f[MAX_N];
double bestx[1 << MAX_N];
int n, startx, endx;

void read_data() {
  scanf("%d %d %d", &n, &startx, &endx);
  for (int i = 0; i < n; i++) {
    int angle;
    scanf("%d %d %d", &f[i].xc, &f[i].yc, &angle);
    f[i].setAngle(angle);
  }
}

double dyn_prog() {
  bestx[0] = startx;
  for (int mask = 1; mask < (1 << n); mask++) {
    bestx[mask] = startx;
    for (int b = 0; b < n; b++) {
      if (mask & (1 << b)) {
        bestx[mask] = max(bestx[mask], f[b].extend(bestx[mask ^ (1 << b)], endx));
      }
    }
  }

  return bestx[(1 << n) - 1] - startx;
}

int main() {
  read_data();
  double result = dyn_prog();
```

```
  printf("%0.9lf\n", result);

  return 0;
}
```

26.9 Problema TrapEZZ (Moisil++ 2023, clasele 11-12)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <stdlib.h>

const int MAX_POINTS = 1'000;
const int MAX_SEGMENTS = MAX_POINTS * (MAX_POINTS - 1) / 2;

struct point {
  int x, y;
};

int gcd(int a, int b) {
  while (b) {
    int tmp = a % b;
    a = b;
    b = tmp;
  }
  return a;
}

struct line {
  int a, b, c;

  void normalize() {
    int g = gcd(abs(c), gcd(abs(a), abs(b)));
    if ((a < 0) || ((a == 0) & (b < 0))) {
      g = -g;
    }
    a /= g;
    b /= g;
    c /= g;
  }

  bool operator ==(line& other) {
    return (a == other.a) && (b == other.b) && (c == other.c);
  }
};

// Pentru segmente cu o mediatoare comună, teoretic trebuie să cunoaștem
```

```
// ecuațiile dreptelor. Dar nu ne trebuie ecuația completă -- știm deja că au
// aceleași valori a și b, deoarece sînt paralele. De aceea, stocăm doar
// valorile c, care denotă pozițiile segmentelor pe mediatoare.
struct segment {
  int len2; // lungimea la pătrat
  line bis; // mediatoarea
  int pos;  // poziția pe mediatoare

  void from_points(point p, point q) {
    len2 = (q.x - p.x) * (q.x - p.x) + (q.y - p.y) * (q.y - p.y);
    bis = {
      .a = q.x - p.x,
      .b = q.y - p.y,
      .c = (q.x * q.x - p.x * p.x + q.y * q.y - p.y * p.y) / 2,
    };
    bis.normalize();

    line l = {
      .a = q.y - p.y,
      .b = p.x - q.x,
      .c = p.y * (q.x - p.x) - p.x * (q.y - p.y),
    };
    l.normalize();
    pos = l.c;
  }
};

point p[MAX_POINTS];
segment s[MAX_SEGMENTS + 1];
int num_points, num_segments;

void read_data() {
  FILE* f = fopen("trapezz.in", "r");
  fscanf(f, "%d", &num_points);
  for (int i = 0; i < num_points; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
    p[i].x *= 2;
    p[i].y *= 2; // astfel ca toate mijloacele să aibă coordonate întregi
  }
  fclose(f);
}

void create_segments() {
  for (int i = 0; i < num_points - 1; i++) {
    for (int j = i + 1; j < num_points; j++) {
      s[num_segments++].from_points(p[i], p[j]);
    }
  }
}
```

```
void sort_by_bisectors() {
  std::sort(s, s + num_segments, [](segment& s, segment& t) {
    if (s.bis.a != t.bis.a) {
      return s.bis.a < t.bis.a;
    } else if (s.bis.b != t.bis.b) {
      return s.bis.b < t.bis.b;
    } else if (s.bis.c != t.bis.c) {
      return s.bis.c < t.bis.c;
    } else {
      return s.pos < t.pos;
    }
  });
}

void sort_by_length(int start, int end) {
  std::sort(s + start, s + end, [](segment& s, segment& t) {
    return s.len2 < t.len2;
  });
}

// TODO: Codul următoarelor două funcții este aproape duplicat.
int count_collinear_pairs(int start, int end) {
  int result = 0, i = start;
  while (i < end) {
    int j = i + 1;
    while ((j < end) && (s[i].pos == s[j].pos)) {
      j++;
    }
    result += (j - i) * (j - i - 1) / 2;
    i = j;
  }
  return result;
}

int count_same_length_pairs(int start, int end) {
  int result = 0, i = start;
  while (i < end) {
    int j = i + 1;
    while ((j < end) && (s[i].len2 == s[j].len2)) {
      j++;
    }
    result += (j - i) * (j - i - 1) / 2;
    i = j;
  }
  return result;
}

// Acest grup de segmente arată ca o frigăruie. Toate au o mediatoare comună
// și sînt sortate după poziția pe această mediatoare.
int count_bisector(int start, int end) {
```

```
  int result = (end - start) * (end - start - 1) / 2;
  result -= count_collinear_pairs(start, end);
  sort_by_length(start, end);
  result -= count_same_length_pairs(start, end);
  // Notă: nu este nevoie de pinex. Două segmente nu pot fi și coliniare, și
  // de aceeași lungime. Atunci ar fi confundate, or enunțul promite că
  // punctele sînt distincte.
  return result;
}

int count_trapezoids() {
  s[num_segments].bis = { 0, 0, 0 }; // santinelă

  int i = 0, result = 0;
  while (i < num_segments) {
    int j = i + 1;
    while (s[i].bis == s[j].bis) {
      j++;
    }
    result += count_bisector(i, j);
    i = j;
  }
  return result;
}

void write_result(int result) {
  FILE* f = fopen("trapezz.out", "w");
  fprintf(f, "%d\n", result);
  fclose(f);
}

int main() {
  read_data();
  create_segments();
  sort_by_bisectors();
  int result = count_trapezoids();
  write_result(result);

  return 0;
}
```

26.10 Problema Terenuri (Baraj ONI 2011)

‹ înapoi • versiune online

```
#include <map>
#include <math.h>
#include <stdio.h>
```

```
struct point {
  double x, y;

  void translate(point other) {
    x -= other.x;
    y -= other.y;
  }

  double dist2() {
    return x * x + y * y;
  }
};

typedef std::map<double, point>::iterator iter;

std::map<double, point> hull;
point center;

// True dacă și numai dacă ∠ABC virează spre dreapta.
bool reflex_angle(point a, point b, point c) {
  return (b.x - a.x) * (c.y - b.y) - (c.x - b.x) * (b.y - a.y) < 0;
}

// Treci la punctul anterior, circular.
iter prev(iter x) {
  return (x == hull.begin())
    ? std::prev(hull.end())
    : std::prev(x);
}

// Treci la punctul următor, circular.
iter next(iter x) {
  x = std::next(x);
  return (x == hull.end())
    ? hull.begin()
    : x;
}

// Returnează hull.end() dacă avem deja un punct pe aceeași rază, dar mai
// depărtat de centru. În acest caz nu avem nimic de făcut.
iter insert(double angle, point p) {
  iter it = hull.lower_bound(angle);
  bool exists = (it != hull.end()) && (it->first == angle);

  if (exists && (p.dist2() <= it->second.dist2())) {
    // Exista deja un punct mai depărtat de centru, pe aceeași rază.
    return hull.end();
  }
```

```
  if (exists) {
    // Exista deja un punct, dar mai apropiat de centru.
    it->second = p;
  } else {
    // Inserează la poziția deja căutată.
    it = hull.insert(it, {angle, p});
  }

  return it;
}

void update_hull(point p) {
  p.translate(center);
  double angle = atan2(p.y, p.x);

  iter it = insert(angle, p);
  if (it == hull.end()) {
    return;
  }

  // Curățenie printre punctele anterioare.
  iter b = prev(it), a = prev(b);
  while ((hull.size() > 3) && reflex_angle(a->second, b->second, p)) {
    hull.erase(b);
    b = a;
    a = prev(a);
  }

  // Curățenie printre punctele următoare.
  iter c = next(it), d = next(c);
  while ((hull.size() > 3) && reflex_angle(p, c->second, d->second)) {
    hull.erase(c);
    c = d;
    d = next(d);
  }

  // Curăță și punctul însuși.
  if (reflex_angle(b->second, p, c->second)) {
    hull.erase(it);
  }
}

int main() {
  FILE* fin = fopen("terenuri.in", "r");
  FILE* fout = fopen("terenuri.out", "w");
  int n, m;
  point p, q;

  fscanf(fin, "%d %d", &n, &m);
  fscanf(fin, "%lf %lf %lf %lf", &p.x, &p.y, &q.x, &q.y);
```

```
  center = { (p.x + q.x) / 2, (p.y + q.y) / 2 };

  update_hull(p);
  update_hull(q);

  for (int i = 2; i < m + n; i++) {
    fscanf(fin, "%lf %lf", &p.x, &p.y);
    update_hull(p);
    if (i >= n - 1) {
      fprintf(fout, "%lu\n", hull.size());
    }
  }

  fclose(fin);
  fclose(fout);
  return 0;
}
```

26.11 Problema Metin2 (Finala IIOT 2021-22)

‹ înapoi • versiune online

```
#include <algorithm>
#include <deque>
#include <math.h>
#include <stdio.h>

const int MAX_STONES = 100'000;

struct ipoint {
  int x, y;
};

struct dpoint {
  double x, y;
};

struct line {
  long long a, b, c;
  ipoint p; // un punct cunoscut a fi pe linie
  double angle;

  void from_points(ipoint p, ipoint q) {
    a = p.y - q.y;
    b = q.x - p.x;
    c = (long long)p.x * q.y - (long long)q.x * p.y;
    this->p = p;
    angle = atan2(q.y - p.y, q.x - p.x);
```

```
  }

  bool has_left(ipoint p) {
    return a * p.x + b * p.y + c > 0;
  }

  bool has_right(dpoint p) {
    return a * p.x + b * p.y + c < 0;
  }

  dpoint intersect(line h) {
    return {
      (double)(b * h.c - c * h.b) / (a * h.b - b * h.a),
      (double)(c * h.a - a * h.c) / (a * h.b - b * h.a),
    };
  }
};

line l[4 * MAX_STONES];
std::deque<int> dl; // indicii liniilor
std::deque<dpoint> poly;
int num_lines;

void read_data() {
  int n;
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    ipoint p, q, r, s;
    scanf("%d %d %d %d %d %d %d %d",
          &p.x, &p.y, &q.x, &q.y, &r.x, &r.y, &s.x, &s.y);
    l[num_lines++].from_points(p, q);
    l[num_lines++].from_points(q, r);
    l[num_lines++].from_points(r, s);
    l[num_lines++].from_points(s, p);
  }
}

void sort_lines() {
  std::sort(l, l + num_lines, [](line& l1, line& l2) {
    return
      (l1.angle < l2.angle) ||
      ((l1.angle == l2.angle) && l2.has_left(l1.p));
  });
}

void unique_lines() {
  int j = 1;
  for (int i = 1; i < num_lines; i++) {
    if (l[i].angle != l[i - 1].angle) {
```

```
      l[j++] = l[i];
    }
  }
  num_lines = j;
}

void build_intersection() {
  dl.push_front(0);
  for (int i = 1; i < num_lines; i++) {
    while (!poly.empty() && (l[i].has_right(poly.front()))) {
      dl.pop_front();
      poly.pop_front();
    }
    poly.push_front(l[i].intersect(l[dl.front()]));
    dl.push_front(i);
  }

  unsigned old_size;
  do {
    old_size = dl.size();
    if (l[dl.front()].has_right(poly.back())) {
      dl.pop_back();
      poly.pop_back();
    }

    if (l[dl.back()].has_right(poly.front())) {
      dl.pop_front();
      poly.pop_front();
    }
  } while (dl.size() != old_size);

  poly.push_front(l[dl.front()].intersect(l[dl.back()]));
}

long double compute_area() {
  poly.push_back(poly.front());
  long double area = 0;
  for (unsigned i = 0; i < poly.size() - 1; i++) {
    area += (poly[i + 1].x - poly[i].x) * (poly[i].y + poly[i + 1].y);
  }

  return area / 2;
}

int main() {
  read_data();
  sort_lines();
  unique_lines();
  build_intersection();
  long double area = compute_area();
```

```
  printf("%.9Lf\n", area);
  return 0;
}
```

Anexa 27

Geometrie - Algoritmi specifici

27.1 Problema Copaci (Infoarena)

‹ înapoi • versiune online

```cpp
// Teorema lui Pick: A = I + B/2 - 1, deci I = A + 1 - B/2. În practică:
// 2I = 2A + 1 - B, căci formula pentru arie calculează dublul ariei.
#include <stdio.h>
#include <stdlib.h>

struct point {
  long long x, y;
};

int gcd(point p, point q) {
  long long x = llabs(p.x - q.x);
  long long y = llabs(p.y - q.y);
  while (y) {
    int tmp = x % y;
    x = y;
    y = tmp;
  }
  return x;
}

long long vector(point p, point q) {
  return p.x * q.y - p.y * q.x;
}

int main() {
  point first, prev, p;
  long long boundary = 0, twice_area = 0;
  int n;

  FILE* f = fopen("copaci.in", "r");
```

```
  fscanf(f, "%d %lld %lld", &n, &first.x, &first.y);
  prev = first;

  for (int i = 1; i < n; i++) {
    fscanf(f, "%lld %lld", &p.x, &p.y);
    twice_area += vector(prev, p);
    boundary += gcd(prev, p);
    prev = p;
  }
  twice_area += vector(prev, first);
  boundary += gcd(prev, first);

  fclose(f);

  f = fopen("copaci.out", "w");
  fprintf(f, "%lld\n", (llabs(twice_area) + 2 - boundary) / 2);
  fclose(f);

  return 0;
}
```

27.2 Problema Emptri (Lot 2015)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000'000;

int phi[MAX_N + 1];

int read_n() {
  int n;
  FILE* f = fopen("emptri.in", "r");
  fscanf(f, "%d", &n);
  fclose(f);
  return n;
}

void compute_phi(int n) {
  for (int i = 1; i <= n; i++) {
    phi[i] = i;
  }
  for (int i = 2; i <= n; i++) {
    if (phi[i] == i) {
      for (int j = i; j <= n; j += i) {
        phi[j] -= phi[j] / i;
      }
```

```
    }
  }
}

long long sum_phi(int n) {
  long long sum = 0;
  for (int i = 2; i <= n; i++) {
    sum += phi[i];
  }
  return sum * 2 + 1;
}

void write_answer(long long answer) {
  FILE* f = fopen("emptri.out", "w");
  fprintf(f, "%lld\n", answer);
  fclose(f);
}

int main() {
  int n = read_n();
  compute_phi(n);
  long long answer = sum_phi(n);
  write_answer(answer);

  return 0;
}
```

27.3 Problema Înfășurătoare convexă (Infoarena)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 120'000;

struct point {
  double x, y;
};

point p[MAX_N];
int st[MAX_N], ss;  // stivă pentru Graham's scan
int n;

void read_data() {
  FILE *f = fopen("infasuratoare.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
```

```
    fscanf(f, "%lf %lf", &p[i].x, &p[i].y);
  }
  fclose(f);
}

void find_extreme() {
  int min = 0;
  for (int i = 1; i < n; i++) {
    if ((p[i].y < p[min].y) || (p[i].y == p[min].y && p[i].x < p[min].x)) {
      min = i;
    }
  }
  std::swap(p[0], p[min]);
}

double orientation(point a, point b, point c) {
  return (b.x - a.x) * (c.y - b.y) - (c.x - b.x) * (b.y - a.y);
}

double dist2(point a, point b) {
  return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}

void polar_sort() {
  std::sort(p + 1, p + n, [](point a, point b) {
    return orientation(p[0], a, b) > 0;
  });
}

void graham_scan() {
  ss = 0;
  for (int i = 0; i < n; i++) {
    while ((ss >= 2) &&
           (orientation(p[st[ss - 2]], p[st[ss - 1]], p[i]) < 0)) {
      ss--;
    }
    st[ss++] = i;
  }
}

void write_hull() {
  FILE* f = fopen("infasuratoare.out", "w");
  fprintf(f, "%d\n", ss);
  for (int i = 0; i < ss; i++) {
    fprintf(f, "%.6lf %.6lf\n", p[st[i]].x, p[st[i]].y);
  }
  fclose(f);
}

int main(void) {
```

```
  read_data();
  find_extreme();
  polar_sort();
  graham_scan();
  write_hull();

  return 0;
}
```

27.4 Problema Înfășurătoare convexă (NerdArena)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;

struct point {
  int x, y;
};

point p[MAX_N];
int st[MAX_N], ss;  // stivă cu indicii punctelor
int n;

void read_data() {
  FILE *f = fopen("infasuratoare.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
  }
  fclose(f);
}

void find_extreme() {
  int min = 0;
  for (int i = 1; i < n; i++) {
    if ((p[i].y < p[min].y) || (p[i].y == p[min].y && p[i].x < p[min].x)) {
      min = i;
    }
  }
  std::swap(p[0], p[min]);
}

long long orientation(point a, point b, point c) {
  return ((long long)(b.x - a.x)) * (c.y - b.y)
    - ((long long)(c.x - b.x)) * (b.y - a.y);
```

```
}

long long dist2(point a, point b) {
  return ((long long)(a.x - b.x)) * (a.x - b.x)
    + ((long long)(a.y - b.y)) * (a.y - b.y);
}

void polar_sort() {
  std::sort(p + 1, p + n, [] (point a, point b) {
    long long disc = orientation(p[0], a, b);
    if (disc == 0) {
      // a și b sînt coliniare cu p[0]; primul vrem să fie cel mai apropiat
      return dist2(a, p[0]) < dist2(b, p[0]);
    } else {
      return disc > 0;
    }
  });
}

void graham_scan() {
  ss = 0;
  for (int i = 0; i < n; i++) {
    while ((ss >= 2) &&
           (orientation(p[st[ss - 2]], p[st[ss - 1]], p[i]) <= 0)) {
      ss--;
    }
    st[ss++] = i;
  }
}

void write_hull() {
  FILE* f = fopen("infasuratoare.out", "w");
  fprintf(f, "%d\n", ss);
  for (int i = 0; i < ss; i++) {
    fprintf(f, "%d %d\n", p[st[i]].x, p[st[i]].y);
  }
  fclose(f);
}

int main(void) {
  read_data();
  find_extreme();
  polar_sort();
  graham_scan();
  write_hull();

  return 0;
}
```

27.5 Problema Testament (Lot 2011)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_HEIRS = 100;
const int MAX_POINTS = 100;
const int LEFT = +1;
const int RIGHT = -1;

struct point {
  int x, y;
  int orig;
};

int sgn(int x) {
  return (x > 0) - (x < 0);
}

int orientation(point a, point b, point c) {
  int det = (b.x - a.x) * (c.y - b.y) - (c.x - b.x) * (b.y - a.y);
  return sgn(det);
}

int hull[MAX_POINTS], interior[MAX_POINTS];
int num_hull, num_interior;

struct stack {
  point p[MAX_POINTS];
  int size, dir;

  void init(point q, int dir) {
    p[0] = q;
    size = 1;
    this->dir = dir;
  }

  void add(point q) {
    while ((size >= 2) &&
           (orientation(p[size - 2], p[size - 1], q) == -dir)) {
      size--;
      interior[num_interior++] = p[size].orig;
    }
    p[size++] = q;
  }
};
```

```
point p[MAX_HEIRS * MAX_POINTS];
stack low, high;
int num_heirs, num_points;

void read_data() {
  FILE* f = fopen("testament.in", "r");
  fscanf(f, "%d %d", &num_points, &num_heirs);
  for (int i = 0; i < num_heirs * num_points; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
    p[i].orig = i + 1;
  }
  fclose(f);
}

void sort_points() {
  std::sort(p, p + num_heirs * num_points, [](point a, point b) {
    return (a.x < b.x) ||
      ((a.x == b.x) && (a.y < b.y));
  });
}

void monotone_chains(point* p) {
  high.init(p[0], RIGHT);
  low.init(p[0], LEFT);
  for (int i = 1; i < num_points; i++) {
    if (orientation(p[0], p[num_points - 1], p[i]) == LEFT) {
      high.add(p[i]);
    } else {
      low.add(p[i]);
    }
  }
  // Ultimul punct este pe ambele lanțuri.
  high.add(p[num_points - 1]);
}

void collect_hull() {
  for (int i = 0; i < low.size - 1; i++) {
    hull[num_hull++] = low.p[i].orig;
  }
  for (int i = high.size - 1; i > 0; i--) {
    hull[num_hull++] = high.p[i].orig;
  }
}

void write_hull(FILE* f) {
  fprintf(f, "%d %d", num_hull, num_interior);
  for (int i = 0; i < num_hull; i++) {
    fprintf(f, " %d", hull[i]);
  }
  for (int i = 0; i < num_interior; i++) {
```

```
    fprintf(f, " %d", interior[i]);
  }
  fprintf(f, "\n");
}

void divide_points() {
  FILE* f = fopen("testament.out", "w");
  for (int i = 0; i < num_heirs; i++) {
    num_hull = num_interior = 0;
    monotone_chains(p + i * num_points);
    collect_hull();
    write_hull(f);
  }
  fclose(f);
}

int main() {
  read_data();
  sort_points();
  divide_points();

  return 0;
}
```

27.6 Problema Magic (JBOI 2023)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_WIZARDS = 200'000;

struct wizard {
  int x, e;
  int p, q;
};

wizard w[MAX_WIZARDS];
int st[MAX_WIZARDS], ss;
int n;

void read_input_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d", &w[i].x, &w[i].e);
  }
}
```

```
// Test standard de orientare trigonometrică.
bool is_reflex(int i, int j, int x, int e) {
  return
    (long long)(w[i].x - x) * (w[j].e - e) >
    (long long)(w[j].x - x) * (w[i].e - e);
}

// Elimină vrăjitorul anterior din stivă cîtă vreme vrăjitorul curent (A) este
// mai sus sau (B) face un unghi reflex cu cei doi vrăjitori anteriori.
void evict(int x, int e) {
  while ((ss && (e >= w[st[ss - 1]].e)) ||
         ((ss > 1) && is_reflex(st[ss - 2], st[ss - 1], x, e))) {
    ss--;
  }
}

// Vrăjitorul me are un nou mentor c. Vezi dacă este mai bun decît vechiul
// mentor.
void compare_mentors(int me, int c) {
  // Dacă q nu există, atunci evident alege p/q din c. Altfel compară vechea
  // și noua pantă.
  if (!w[me].q ||
      ((long long)w[c].e * w[me].q >
       (long long)w[me].p * (w[me].x - w[c].x))) {
    w[me].p = w[c].e;
    w[me].q = w[me].x - w[c].x;
  }
}

void iterate_wizards() {
  ss = 0; // mărimea stivei
  st[ss++] = 0;

  for (int i = 1; i < n; i++) {
    // Mai întîi privește de la nivelul solului și alege-ți un mentor.
    evict(w[i].x, 0);
    compare_mentors(i, st[ss - 1]);

    // Apoi privește mai de sus și continuă să elimini.
    evict(w[i].x, w[i].e);
    st[ss++] = i;
  }
}

// Oglindește coordonatele x. Astfel scăpăm de nevoia de a copia algoritmul cu
// stivă de la stînga la dreapta și de la dreapta la stînga.
void reverse_wizards() {
  int min_coord = w[0].x, max_coord = w[n - 1].x;

  for (int i = 0, j = n - 1; i < j; i++, j--) {
```

```
    wizard tmp = w[i];
    w[i] = w[j];
    w[j] = tmp;
  }

  for (int i = 0; i < n; i++) {
    w[i].x = min_coord + max_coord - w[i].x;
  }
}

void find_mentors() {
  iterate_wizards();
  reverse_wizards();
  iterate_wizards();
  reverse_wizards();
}

int gcd(int x, int y) {
  while (y) {
    int tmp = x;
    x = y;
    y = tmp % y;
  }
  return x;
}

void write_reduced_fractions() {
  for (int i = 0; i < n; i++) {
    int d = gcd(w[i].p, w[i].q);
    printf("%d %d\n", w[i].p / d, w[i].q / d);
  }
}

int main() {
  read_input_data();
  find_mentors();
  write_reduced_fractions();

  return 0;
}
```

27.7 Problema Triangular Queries (CodeChef)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_N = 300000;
const int MAX_Q = 200000;
const int MAX_COORD = 300000;

// Ordinea este importanta.  Punctul trebuie adăugat *după* ce procesăm orice
// bază care îl conține, ca sa nu îl scădem. În schimb, punctul trebuie
// adăugat *înainte* să procesăm orice vîrf de triunghi cu care se suprapune,
// ca să îl luăm în calcul.
const int T_BASE = 0;
const int T_POINT = 1;
const int T_TIP = 2;

struct point {
  int x, y;
};

struct triangle {
  int x, y, d;
  int answer;
};

struct event {
  int diag;
  int obj_id;
  unsigned char type;
};

struct fenwick {
  int v[MAX_COORD + 1];

  void inc(int pos) {
    do {
      v[pos]++;
      pos += pos & -pos;
    } while (pos <= MAX_COORD);
  }

  int partial_sum(int pos) {
    int sum = 0;
    while (pos) {
      sum += v[pos];
      pos &= pos - 1;
    }
    return sum;
  }
};

point p[MAX_N];
triangle t[MAX_Q];
event e[MAX_N + 2 * MAX_Q];
```

```
fenwick fx, fy;
int num_points, num_triangles, num_events;
int points_seen;

void read_data() {
  scanf("%d %d", &num_points, &num_triangles);
  for (int i = 0; i < num_points; i++) {
    scanf("%d %d", &p[i].x, &p[i].y);
  }
  for (int i = 0; i < num_triangles; i++) {
    scanf("%d %d %d", &t[i].x, &t[i].y, &t[i].d);
  }
}

void create_events() {
  for (int i = 0; i < num_points; i++) {
    e[num_events++] = {
      .diag = p[i].x + p[i].y,
      .obj_id = i,
      .type = T_POINT,
    };
  }
  for (int i = 0; i < num_triangles; i++) {
    e[num_events++] = {
      .diag = t[i].x + t[i].y + t[i].d,
      .obj_id = i,
      .type = T_BASE,
    };
    e[num_events++] = {
      .diag = t[i].x + t[i].y,
      .obj_id = i,
      .type = T_TIP,
    };
  }
}

void sort_events() {
  std::sort(e, e + num_events, [](event a, event b) {
    return (a.diag > b.diag) ||
      ((a.diag == b.diag) && (a.type < b.type));
  });
}

void base_event(int id) {
  t[id].answer = points_seen
    - fx.partial_sum(t[id].x - 1)
    - fy.partial_sum(t[id].y - 1);
}

void point_event(int id) {
```

```
  fx.inc(p[id].x);
  fy.inc(p[id].y);
  points_seen++;
}

void tip_event(int id) {
  int total = points_seen
    - fx.partial_sum(t[id].x - 1)
    - fy.partial_sum(t[id].y - 1);
  t[id].answer = total - t[id].answer;
}

void process_events() {
  for (int i = 0; i < num_events; i++) {
    switch (e[i].type) {
      case T_BASE: base_event(e[i].obj_id); break;
      case T_POINT: point_event(e[i].obj_id); break;
      case T_TIP: tip_event(e[i].obj_id); break;
    }
  }
}

void write_answers() {
  for (int i = 0; i < num_triangles; i++) {
    printf("%d\n", t[i].answer);
  }
}

int main() {
  read_data();
  create_events();
  sort_events();
  process_events();
  write_answers();

  return 0;
}
```

27.8 Problema Nuke (Lot 2010)

‹ înapoi • versiune online

```
#include <algorithm>
#include <map>
#include <stdio.h>

const int MAX_POINTS = 200'000;
const int MAX_CIRCLES = 100'000;
```

```
const int MAX_EVENTS = MAX_POINTS + 2 * MAX_CIRCLES;

const int T_BEGIN_CIRCLE = 0;
const int T_POINT = 1;
const int T_END_CIRCLE = 2;

struct point {
  int x, y;
};

struct circle {
  int x, y, r, pop;

  void update_pop(point p) {
    long long dx2 = (long long)(p.x - x) * (p.x - x);
    long long dy2 = (long long)(p.y - y) * (p.y - y);
    long long r2 = (long long) r * r;
    pop += (dx2 + dy2 <= r2);
  }
};

struct event {
  int x, type, obj_id;
};

point p[MAX_POINTS];
circle c[MAX_CIRCLES];
event evt[MAX_EVENTS];
std::map<int, int> circle_ys;
int n, m, num_evt;

void read_data() {
  FILE* f = fopen("nuke.in", "r");
  fscanf(f, "%d %d", &n, &m);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
  }
  for (int i = 0; i < m; i++) {
    fscanf(f, "%d %d %d", &c[i].x, &c[i].y, &c[i].r);
  }
  fclose(f);
}

void create_events() {
  for (int i = 0; i < n; i++) {
    evt[num_evt++] = { p[i].x, T_POINT, i };
  }
  for (int i = 0; i < m; i++) {
    evt[num_evt++] = { c[i].x - c[i].r, T_BEGIN_CIRCLE, i };
    evt[num_evt++] = { c[i].x + c[i].r, T_END_CIRCLE, i };
```

```
  }
}

void sort_events() {
  std::sort(evt, evt + num_evt, [](event& a, event& b) {
    return (a.x < b.x) ||
      ((a.x == b.x) && (a.type < b.type));
  });
}

void begin_circle(int id) {
  circle_ys[c[id].y] = id;
}

void end_circle(int id) {
  circle_ys.erase(c[id].y);
}

void process_point(point p) {
  auto it = circle_ys.lower_bound(p.y);
  if (it != circle_ys.end()) {
    c[it->second].update_pop(p);
  }
  if (it != circle_ys.begin()) {
    c[(--it)->second].update_pop(p);
  }
}

void sweep_line() {
  for (int i = 0; i < num_evt; i++) {
    int id = evt[i].obj_id;
    switch (evt[i].type) {
      case T_BEGIN_CIRCLE: begin_circle(id); break;
      case T_END_CIRCLE: end_circle(id); break;
      case T_POINT: process_point(p[id]); break;
    }
  }
}

void write_answers() {
  FILE* f = fopen("nuke.out", "w");
  for (int i = 0; i < m; i++) {
    fprintf(f, "%d\n", c[i].pop);
  }
  fclose(f);
}

int main() {
  read_data();
  create_events();
```

```
  sort_events();
  sweep_line();
  write_answers();

  return 0;
}
```

27.9 Problema Hill Walk (USACO Gold 2013)

înapoi

```
#include <algorithm>
#include <set>
#include <stdio.h>

const int MAX_N = 100'000;

// Returnează true dacă și numai dacă (x1,y1), (x2,y2) și (x3,y3) sînt în
// ordine trigonometrică.
long long orientation(int x1, int y1, int x2, int y2, int x3, int y3) {
  return ((long long)(x2 - x1)) * (y3 - y2)
    - ((long long)(x3 - x2)) * (y2 - y1);
}

struct segment {
  int x1, y1, x2, y2;
  int ind;

  // Returneaza true dacă și numai dacă acest segment este „sub” o.
  bool operator<(segment const& o) const {
    if (x2 < o.x2) {
      // Acest segment *nu* este sub o dacă (x2,y2) cade pe o, adică dacă
      // (o.x1,o.y1), (o.x2,o.y2) și (x2,y2) sînt în ordine
      // antitrigonometrică.
      return orientation(o.x1, o.y1, o.x2, o.y2, x2, y2) < 0;
                } else {
      // Acest segment *este* sub o dacă (o.x2,o.y2) cade pe acest segment,
      // adică dacă (x1,y1), (x2,y2) și (o.x2,o.y2) sînt în ordine
      // trigonometrică.
      return orientation(x1, y1, x2, y2, o.x2, o.y2) > 0;
                }
  }
};

struct event {
  int ind;
  int x, y;
};
```

```
segment seg[MAX_N];
event evt[2 * MAX_N];
std::set<segment> set;
int n, bessie, hill_count;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d %d %d", &seg[i].x1, &seg[i].y1, &seg[i].x2, &seg[i].y2);
    seg[i].ind = i;
  }
}

void create_and_sort_events() {
  for (int i = 0; i < n; i++) {
    evt[2 * i] = { i, seg[i].x1, seg[i].y1 };
    evt[2 * i + 1] = { i, seg[i].x2, seg[i].y2 };
  }

  std::sort(evt, evt + 2 * n, [](event& a, event& b) {
    return (a.x < b.x) || ((a.x == b.x) && (a.y < b.y));
  });
}

void process_end(event& e) {
  std::set<segment>::iterator it = set.find(seg[e.ind]);

  if (e.ind == bessie) {
    if (it == set.begin()) {
      bessie = n;
    } else {
      std::set<segment>::iterator prev = std::prev(it);
      bessie = prev->ind;
      hill_count++;
    }
  }

  set.erase(it);
}

void sweep_line() {
  bessie = 0;
  hill_count = 1;

  for (int i = 0; i < 2 * n; i++) {
    event& e = evt[i];
    segment& s = seg[e.ind];
    if (e.x == s.x1) {
      set.insert(s);
```

```
    } else {
      process_end(e);
    }
  }
}

int main() {
  read_data();
  create_and_sort_events();
  sweep_line();
  printf("%d\n", hill_count);

  return 0;
}
```

27.10 Problema Ydist (Lot 2014)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_Q = 100'000;
const int NONE = -1;

struct point {
  int x, y;
  int qindex; // sau NONE dacă acest punct este o bilă, nu o interogare
};

point p[MAX_N + MAX_Q];
int st[MAX_N], ss; // stiva de puncte-candidat
double ans[MAX_Q];
int n, q;

void read_data() {
  FILE* f = fopen("ydist.in", "r");
  fscanf(f, "%d %d", &n, &q);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
    p[i].qindex = NONE;
  }
  for (int i = 0; i < q; i++) {
    fscanf(f, "%d %d", &p[i + n].x, &p[i + n].y);
    p[i + n].qindex = i;
  }
  fclose(f);
```

```
}

void polar_sort() {
  // Procesează bilele înaintea interogărilor pe o rază. Vrem să luăm bilele
  // în calcul pentru interogări (cu distanța 0).
  std::sort(p, p + n + q, [](point a, point b) {
    long long disc = (long long)a.y * b.x - (long long)a.x * b.y;
    return (disc > 0) ||
      ((disc == 0) & (a.qindex == NONE));
  });
}

long long orientation(point& a, point& b, point& c) {
  return (long long)(b.x - a.x) * (c.y - b.y)
    - (long long)(c.x - b.x) * (b.y - a.y);
}

void process_point(int i) {
  // Elimină punctele la NE de p[i].
  while (ss &&
         ((p[st[ss - 1]].x >= p[i].x) ||
          (p[st[ss - 1]].y >= p[i].y))) {
    ss--;
  }

  // Elimină punctele care încalcă concavitatea stivei.
  while ((ss >= 2) &&
         (orientation(p[st[ss - 2]], p[st[ss - 1]], p[i]) < 0)) {
    ss--;
  }
  st[ss++] = i;
}

// Calculează distanța verticală de la p la raza (0,0)-r.
double dist_to_ray(point p, point r) {
  return p.y - (double)r.y * p.x / r.x;
}

void process_query(int i) {
  // Dacă ultimul punct din stivă este mai departe de rază decît penultimul,
  // el poate fi șters (pentru razele viitoare el va fi și mai departe).
  double d1 = dist_to_ray(p[st[ss - 1]], p[i]), d2;
  while ((ss >= 2) &&
         ((d2 = dist_to_ray(p[st[ss - 2]], p[i])) < d1)) {
    ss--;
    d1 = d2;
  }
  ans[p[i].qindex] = d1;
}
```

```
void scan_points() {
  for (int i = 0; i < n + q; i++) {
    if (p[i].qindex == NONE) {
      process_point(i);
    } else {
      process_query(i);
    }
  }
}

void write_answers() {
  FILE* f = fopen("ydist.out", "w");
  for (int i = 0; i < q; i++) {
    fprintf(f, "%.7f\n", ans[i]);
  }
  fclose(f);
}

int main() {
  read_data();
  polar_sort();
  scan_points();
  write_answers();

  return 0;
}
```

27.11 Problema Fossil in the Ice (CodeChef)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100000;

typedef struct {
  int x, y;
} point;

point p[MAX_N];
int st[MAX_N];  // stivă pentru Graham's scan
int n;

void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d", &p[i].x, &p[i].y);
```

```
  }
}

long long area(point a, point b, point c) {
  return
    (long long)(b.x - a.x) * (c.y - b.y) -
    (long long)(b.y - a.y) * (c.x - b.x);
}

long long dist2(point a, point b) {
  return
    (long long)(a.x - b.x) * (a.x - b.x) +
    (long long)(a.y - b.y) * (a.y - b.y);
}

void find_min_y() {
  int min = 0;
  for (int i = 1; i < n; i++) {
    if ((p[i].y < p[min].y) || (p[i].y == p[min].y && p[i].x < p[min].x)) {
      min = i;
    }
  }
  std::swap(p[0], p[min]);
}

void polar_sort() {
  std::sort(p + 1, p + n, [](point a, point b) {
    long long d = area(p[0], a, b);
    return (d > 0) ||
      ((d == 0) && (dist2(a, p[0]) < dist2(b, p[0])));
  });
}

void graham_scan() {
  int ss = 0;
  for (int i = 0; i < n; i++) {
    while ((ss >= 2) &&
           (area(p[st[ss - 2]], p[st[ss - 1]], p[i]) <= 0)) {
      ss--;
    }
    st[ss++] = i;
  }

  // Colectează punctele de pe înfășurătoare în ordine trigonometrică.
  for (int i = 0; i < ss; i++) {
    p[i] = p[st[i]];
  }
  n = ss;
}
```

```
long long max(long long a, long long b) {
  return (a > b) ? a : b;
}

int next(int k) {
  return (k == n - 1) ? 0 : (k + 1);
}

long long rotating_calipers() {
  int j = 0, j2 = 1;
  long long result = 0;

  for (int i = 0; i < n; i++) {
    // Mică optimizare: calculează d(i, j) o singură dată.
    long long d = dist2(p[i], p[j]), d2;
    while ((d2 = dist2(p[i], p[j2])) > d) {
      d = d2;
      j = j2;
      j2 = next(j2);
    }
    result = max(result, d);
    // Invariant: j este cel mai depărtat punct de i.
  }

  return result;
};

long long find_diameter() {
  if (n <= 1) {
    return 0;
  } else if (n == 2)  {
    return dist2(p[0], p[1]);
  } else {
    find_min_y();
    polar_sort();
    graham_scan();
    return rotating_calipers();
  }
}

int main(void) {
  int num_tests;

  scanf("%d", &num_tests);

  while (num_tests--) {
    read_data();
    long long diameter = find_diameter();
    printf("%lld\n", diameter);
  }
```

```
  return 0;
}
```

27.12 Problema Rubarba (Infoarena)

înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 100'000;

struct point {
  int x, y;

  point operator - (const point& other) {
    return { x - other.x, y - other.y };
  }
};

point p[MAX_N];
int st[MAX_N];  // stivă pentru Graham's scan
int n;

void read_data() {
  FILE* f = fopen("rubarba.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d %d", &p[i].x, &p[i].y);
  }
  fclose(f);
}

void find_min_y() {
  int min = 0;
  for (int i = 1; i < n; i++) {
    if ((p[i].y < p[min].y) || (p[i].y == p[min].y && p[i].x < p[min].x)) {
      min = i;
    }
  }
  std::swap(p[0], p[min]);
}

long long dot(point a, point b) {
  return (long long)a.x * b.x + (long long)a.y * b.y;
}
```

```
long long area(point a, point b, point c) {
  return
    (long long)(b.x - a.x) * (c.y - b.y) -
    (long long)(b.y - a.y) * (c.x - b.x);
}

long long dist2(point a, point b) {
  return dot(b - a, b - a);
}

void polar_sort() {
  std::sort(p + 1, p + n, [](point a, point b) {
    long long d = area(p[0], a, b);
    return (d > 0) ||
      ((d == 0) && (dist2(a, p[0]) < dist2(b, p[0])));
  });
}

void graham_scan() {
  int ss = 0;
  for (int i = 0; i < n; i++) {
    while ((ss >= 2) &&
           (area(p[st[ss - 2]], p[st[ss - 1]], p[i]) <= 0)) {
      ss--;
    }
    st[ss++] = i;
  }

  // Colectează punctele de pe înfășurătoare în ordine trigonometrică.
  for (int i = 0; i < ss; i++) {
    p[i] = p[st[i]];
  }
  n = ss;
}

int next(int k) {
  return (k == n - 1) ? 0 : (k + 1);
}

// Returnează aria triunghiului sprijinit pe b1b2 cu extremele r (dreapta), a
// (opus) și l (stînga).
//
// Înălțimea dreptunghiului este distanța de la a la b1b2, pe care o putem
// exprima ca 2 * aria(Δab1b2) / |b1b2|. Lățimea dreptunghiului este proiecția
// vectorului r - l pe dreapta b1b2. Aceasta prin definiție este (r - l) ·
// b1b2 / |b1b2| (produsul scalar). Aria dreptunghiului este deci:
//
// (r - l) · b1b2 * aria(Δab1b2) / |b1b2|^2
long double rectangle_area(int b1, int b2, int r, int a, int l) {
  return (long double)dot(p[b2] - p[b1], p[r] - p[l])
```

```
    / dist2(p[b1], p[b2])
    * area(p[b1], p[b2], p[a]);
}

void find_extremes(int b1, int& b2, int& r, int& a, int& l) {
  b2 = next(b1);
  point base = p[b2] - p[b1];

  while (dot(base, p[next(r)] - p[b1]) > dot(base, p[r] - p[b1])) {
    r = next(r);
  }

  a = r;
  while (area(p[b1], p[b2], p[next(a)]) > area(p[b1], p[b2], p[a])) {
    a = next(a);
  }

  l = a;
  while (dot(base, p[next(l)] - p[b1]) < dot(base, p[l] - p[b1])) {
    l = next(l);
  }
}

// Pentru o explicație vizuală a funționării algoritmului, vezi
// https://www.youtube.com/watch?v=OaGvH450jRM
long double rotating_calipers() {
  int j, r = 1, a, l;
  long double min_area = 0;

  for (int i = 0; i < n; i++) {
    find_extremes(i, j, r, a, l);
    long double area = rectangle_area(i, j, r, a, l);
    if (!i || (area < min_area)) {
      min_area = area;
    }
  }

  return min_area;
}

long double find_min_area() {
  find_min_y();
  polar_sort();
  graham_scan();

  return (n <= 2) ? 0 : rotating_calipers();
}

void write_result(long double result) {
  FILE* f = fopen("rubarba.out", "w");
```

```
  fprintf(f, "%.2Lf\n", result);
  fclose(f);
}

int main() {
  read_data();
  long double result = find_min_area();
  write_result(result);

  return 0;
}
```

Anexa 28

Grafuri - Arbori parțiali minimi

28.1 Problema Arbore parțial de cost minim (Infoarena)

înapoi

Sursă cu algoritmul lui Kruskal (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 400'000;

struct disjoint_set_forest {
  int p[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  void unite(int u, int v) {
    p[find(v)] = find(u);
  }

  bool are_connected(int u, int v) {
    return find(u) == find(v);
  }
```

```
};

struct edge {
  int u, v;
  short cost;
  bool chosen;
};

edge e[MAX_EDGES];
disjoint_set_forest dsf;
int n, m, mst_cost;

void read_data() {
  FILE* f = fopen("apm.in", "r");

  fscanf(f, "%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    fscanf(f, "%d %d %hd", &e[i].u, &e[i].v, &e[i].cost);
  }

  fclose(f);
}

void kruskal() {
  std::sort(e, e + m, [](edge& a, edge& b) {
    return a.cost < b.cost;
  });
  dsf.init(n);

  for (int i = 0; i < m; i++) {
    if (!dsf.are_connected(e[i].u, e[i].v)) {
      e[i].chosen = true;
      mst_cost += e[i].cost;
      dsf.unite(e[i].u, e[i].v);
    }
  }
}

void write_mst() {
  FILE* f = fopen("apm.out", "w");

  fprintf(f, "%d\n", mst_cost);
  fprintf(f, "%d\n", n - 1);
  for (int i = 0; i < m; i++) {
    if (e[i].chosen) {
      fprintf(f, "%d %d\n", e[i].u, e[i].v);
    }
  }

  fclose(f);
```

```
}

int main() {
  read_data();
  kruskal();
  write_mst();

  return 0;
}
```

Sursă cu algoritmul lui Prim (versiune online).

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 400'000;
const int INFINITY = 1'001; // mai mare decît orice cost

struct edge {
  int v;
  short cost;

  // Vezi https://stackoverflow.com/a/3141107/6022817 pentru "const"
  bool operator<(edge other) const {
    return cost > other.cost;
  }
};

struct node {
  std::vector<edge> adj;
  int parent;
  short dist;
  bool visited;
};

node nd[MAX_NODES + 1];
int n, mst_cost;

void read_data() {
  int m;
  FILE* f = fopen("apm.in", "r");

  fscanf(f, "%d %d", &n, &m);
  while (m--) {
    int u, v;
    short cost;
    fscanf(f, "%d %d %hd", &u, &v, &cost);
    nd[u].adj.push_back({v, cost});
```

```
    nd[v].adj.push_back({u, cost});
  }

  fclose(f);
}

void prim() {
  // Priority queue cu reinserare. Un nod poate trece prin coadă de mai multe
  // ori, iar prin coadă pot trece în total m noduri.
  std::priority_queue<edge> q;
  for (int u = 2; u <= n; u++) {
    nd[u].dist = INFINITY;
  }
  q.push({1, 0});

  while (!q.empty()) {
    int u = q.top().v, cost = q.top().cost;
    q.pop();

    if (!nd[u].visited) {
      nd[u].visited = true;
      mst_cost += cost;
      for (edge e: nd[u].adj) {
        if (!nd[e.v].visited && (e.cost < nd[e.v].dist)) {
          nd[e.v].dist = e.cost;
          nd[e.v].parent = u;
          q.push({e.v, e.cost});
        }
      }
    }
  }
}

void write_mst() {
  FILE* f = fopen("apm.out", "w");

  fprintf(f, "%d\n", mst_cost);
  fprintf(f, "%d\n", n - 1);
  for (int u = 2; u <= n; u++) {
    fprintf(f, "%d %d\n", nd[u].parent, u);
  }

  fclose(f);
}

int main() {
  read_data();
  prim();
  write_mst();
```

```
  return 0;
}
```

28.2 Problema Autobuze3 (AGM 2015)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 400'000;

struct disjoint_set_forest {
  int p[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  void unite(int u, int v) {
    p[find(v)] = find(u);
  }

  bool are_connected(int u, int v) {
    return find(u) == find(v);
  }
};

struct edge {
  int u, v;
  short cost;
};

struct node {
  std::vector<int> adj; // doar muchiile din APM
  std::vector<int> ppl; // ppl[0] este șoferul, care își conduce propriul autobuz
};
```

```
edge e[MAX_EDGES];
node nd[MAX_NODES + 1];
disjoint_set_forest dsf;
int n, m, mst_cost;
FILE *fin, *fout;

void read_data() {
  fscanf(fin, "%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    fscanf(fin, "%d %d %hd", &e[i].u, &e[i].v, &e[i].cost);
  }
}

void kruskal() {
  std::sort(e, e + m, [](edge& a, edge& b) {
    return a.cost < b.cost;
  });
  dsf.init(n);

  mst_cost = 0;
  for (int i = 0; i < m; i++) {
    if (!dsf.are_connected(e[i].u, e[i].v)) {
      mst_cost += e[i].cost;
      dsf.unite(e[i].u, e[i].v);
      nd[e[i].u].adj.push_back(e[i].v);
      nd[e[i].v].adj.push_back(e[i].u);
    }
  }
}

void merge(int u, int v) {
  // small to large
  if (nd[v].ppl.size() > nd[u].ppl.size()) {
    nd[u].ppl.swap(nd[v].ppl);
  }
  for (int p: nd[v].ppl) {
    nd[u].ppl.push_back(p);
    fprintf(fout, "Move %d %d %d\n", p, nd[v].ppl[0], nd[u].ppl[0]);
  }
  nd[v].ppl.clear();
}

void dfs(int u, int parent) {
  nd[u].ppl.push_back(u);

  for (int v: nd[u].adj) {
    if (v != parent) {
      dfs(v, u);
      fprintf(fout, "Drive %d %d %d\n", nd[v].ppl[0], v, u);
      merge(u, v);
```

```
    }
  }
}

void gen_ops() {
  fprintf(fout, "%d\n", mst_cost);
  dfs(1, 0);
  fprintf(fout, "Gata\n");
}

void clear() {
  for (int u = 1; u <= n; u++) {
    nd[u].adj.clear();
  }
  nd[1].ppl.clear();
}

int main() {
  fin = fopen("autobuze3.in", "r");
  fout = fopen("autobuze3.out", "w");

  int num_tests;
  fscanf(fin, "%d", &num_tests);
  while (num_tests--) {
    read_data();
    kruskal();
    gen_ops();
    clear();
  }

  fclose(fin);
  fclose(fout);

  return 0;
}
```

28.3 Problema Rusuoaica (FMI No Stress 9 Warmup)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 400'000;

struct edge {
  int u, v, c;
```

```
};

struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int n, num_trees;

  void init(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
    }
    num_trees = n;
  }

  int find(int u) {
    return (parent[u] == u)
      ? u
      : (parent[u] = find(parent[u]));
  }

  // Returnează true dacă chiar a unificat u cu v.
  bool merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      parent[u] = v;
      num_trees--;
      return true;
    } else {
      return false;
    }
  }

  int get_num_trees() {
    return num_trees;
  }
};

disjoint_set_forest dsf;
edge e[MAX_EDGES];
int n, m, build_cost, total_cost;

void read_and_filter_edges() {
  int real_m;
  FILE* f = fopen("rusuoaica.in", "r");

  fscanf(f, "%d %d %d", &n, &real_m, &build_cost);

  while (real_m--) {
    fscanf(f, "%d %d %d", &e[m].u, &e[m].v, &e[m].c);
```

```
    if (e[m].c <= build_cost) {
      m++;
    } else {
      total_cost -= e[m].c;
    }
  }

  fclose(f);
}

void sort_edges() {
  std::sort(e, e + m, [] (edge& a, edge& b) {
    return a.c < b.c;
  });
}

void kruskal() {
  dsf.init(n);
  sort_edges();

  for (int i = 0; i < m; i++) {
    if (dsf.merge(e[i].u, e[i].v)) {
      total_cost += e[i].c;
    } else {
      total_cost -= e[i].c;
    }
  }

  total_cost += build_cost * (dsf.get_num_trees() - 1);
}

void write_answer() {
  FILE* f = fopen("rusuoaica.out", "w");
  fprintf(f, "%d\n", total_cost);
  fclose(f);
}

int main() {
  read_and_filter_edges();
  kruskal();
  write_answer();

  return 0;
}
```

28.4 Problema MinOr Tree (Codeforces)

‹ înapoi • versiune online

```cpp
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;
const int MAX_BIT = 29;

struct edge {
  int u, v, weight;
};

struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int n, num_trees;

  void reset(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
    }
    num_trees = n;
  }

  int find(int u) {
    return (parent[u] == u)
      ? u
      : (parent[u] = find(parent[u]));
  }

  void merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      parent[u] = v;
      num_trees--;
    }
  }

  bool all_connected() {
    return num_trees == 1;
  }

};

edge e[MAX_EDGES];
disjoint_set_forest ds;
int num_nodes, num_edges, ority;

void read_graph() {
  ority = 0;
```

```
  scanf("%d %d", &num_nodes, &num_edges);
  for (int i = 0; i < num_edges; i++) {
    scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].weight);
    ority |= e[i].weight;
  }
}

bool is_connected(int mask) {
  ds.reset(num_nodes);
  int i = 0;
  while ((i < num_edges) && !ds.all_connected()) {
    if ((e[i].weight & mask) == e[i].weight) {
      ds.merge(e[i].u, e[i].v);
    }
    i++;
  }
  return ds.all_connected();
}

void find_ority()  {
  for (int b = MAX_BIT; b >= 0; b--) {
    if ((ority & (1 << b)) &&
        is_connected(ority ^ (1 << b))) {
      ority ^= 1 << b;
    }
  }
}

void write_ority() {
  printf("%d\n", ority);
}

void process_test() {
  read_graph();
  find_ority();
  write_ority();
}

int main() {
  int num_tests;

  scanf("%d", &num_tests);
  while (num_tests--) {
    process_test();
  }

  return 0;
}
```

28.5 Problema 0-1 MST (Codeforces)

‹ înapoi

Sursă cu algoritmul lui Kruskal (versiune online).

```cpp
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;

// O structură de mulțimi disjuncte augmentată cu trei informații:
// * numărul de mulțimi;
// * mărimea fiecărei mulțimi;
// * lista de rădăcini.
struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  int s[MAX_NODES + 1];
  int root_pos[MAX_NODES + 1]; // root_pos[u] = poziția lui u în roots[]
  int roots[MAX_NODES + 1];
  int n, num_trees;

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
      s[u] = 1;
      root_pos[u] = u;
      roots[u] = u;
    }
    num_trees = n;
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  void unite(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      s[u] += s[v];
      p[v] = u;
      // Înlocuiește-l pe v cu ultimul element din roots și actualizează
      // root_pos.
      int repl = roots[num_trees - 1];
      roots[root_pos[v]] = repl;
      root_pos[repl] = root_pos[v];
      num_trees--;
```

```
    }
  }

  void unite_incoming(int u, int* incoming) {
    u = find(u);
    int i = 0;
    while (i < num_trees) {
      if ((roots[i] == u) || (incoming[roots[i]] == s[roots[i]])) {
        i++;
      } else {
        unite(u, roots[i]);
      }
    }
  }
};

std::vector<int> unadj[MAX_NODES + 1];
int incoming[MAX_NODES + 1];
disjoint_set_forest dsf;
int n;

void read_data() {
  int m, u, v;
  scanf("%d %d", &n, &m);
  while (m--) {
    scanf("%d %d", &u, &v);
    unadj[u].push_back(v);
    unadj[v].push_back(u);
  }
}

void process_node(int u) {
  // Numără muchiile 1 care intră în fiecare componentă.
  for (int v: unadj[u]) {
    incoming[dsf.find(v)]++;
  }

  dsf.unite_incoming(u, incoming);

  // Curățenie.
  for (int v: unadj[u]) {
    incoming[dsf.find(v)] = 0;
  }
}

void connectivity() {
  dsf.init(n);
  for (int u = 1; u <= n; u++) {
    process_node(u);
  }
```

```
}

int main() {
  read_data();
  connectivity();
  printf("%d\n", dsf.num_trees - 1);

  return 0;
}
```

Sursă cu BFS (versiune online).

```
#include <unordered_set>
#include <queue>
#include <stdio.h>

const int MAX_NODES = 100'000;

std::unordered_set<int> adj[MAX_NODES + 1];
int unseen[MAX_NODES], num_unseen;
int n, answer;

void read_graph() {
  int m, u, v;
  scanf("%d %d", &n, &m);
  while (m--) {
    scanf("%d %d", &u, &v);
    adj[u].insert(v);
    adj[v].insert(u);
  }
}

void init_seen() {
  for (int u = 1; u <= n; u++) {
    unseen[num_unseen++] = u;
  }
}

void bfs(int u) {
  std::queue<int> q;
  q.push(u);
  while (!q.empty()) {
    int u = q.front();
    q.pop();

    int i = 0;
    while (i < num_unseen) {
      int v = unseen[i];
      if (adj[u].find(v) == adj[u].end()) {
```

```
        q.push(v);
        unseen[i] = unseen[--num_unseen];
      } else {
        i++;
      }
    }
  }
}

void bfs_wrapper() {
  int num_comps = 0;
  while (num_unseen) {
    num_comps++;
    int u = unseen[--num_unseen];
    bfs(u);
  }
  answer = num_comps - 1;
}

void write_answer() {
  printf("%d\n", answer);
}

int main() {
  read_graph();
  init_seen();
  bfs_wrapper();
  write_answer();

  return 0;
}
```

28.6 Problema Minimum Spanning Tree for Each Edge (Codeforces)

‹ înapoi • versiune online

```
// Adaptată după https://codeforces.com/contest/609/submission/129056851
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;

struct edge {
  int u, v;
  int weight;
```

```
  int orig;
};

struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  int size[MAX_NODES + 1];
  // costul necesar pentru a părăsi componenta prin părinte
  int w[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
      size[u] = 1;
    }
  }

  // Returnează costul maxim pe calea u-v dacă u și v sînt conectate. Altfel
  // unifică u cu v și returnează 0.
  //
  // Trebuie să ne asigurăm că, pentru a părăsi subarborele lui v ca să
  // ajungem la restul arborelui, weight este cea mai scumpă muchie. Ne bazăm
  // pe faptul că procesăm muchiile în ordine crescătoare a costului.
  //
  // Totuși, asta înseamnă că nu putem folosi compresia căii. Dar folosim
  // unirea după mărime ca să ne asigurăm că înălțimea componentelor nu
  // depășește log n.
  int try_unite(int u, int v, int weight) {
    int maxw = 0;

    while ((u != v) && (p[u] != u || p[v] != v)) {
      if ((p[u] == u) || ((p[v] != v) && (size[v] < size[u]))) {
        int tmp = u; u = v; v = tmp;
      }
      maxw = (w[u] > maxw) ? w[u] : maxw;
      u = p[u];
    }

    if (u == v) {
      return maxw;
    } else {
      unite(u, v, weight);
      return 0;
    }
  }

  void unite(int u, int v, int weight) {
    if (size[u] < size[v]) {
      int tmp = u; u = v; v = tmp;
    }
```

```
    size[u] += size[v];
    p[v] = u;
    w[v] = weight;
  }
};

edge e[MAX_EDGES];
disjoint_set_forest dsf;
int diff[MAX_EDGES]; // răspunsurile, relative la APM-ul pe care îl vom afla
int n, m;
long long mst_weight;

void read_data() {
  scanf("%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].weight);
    e[i].orig = i;
  }
}

void sort_edges_by_weight() {
  std::sort(e, e + m, [] (edge& a, edge& b) {
    return a.weight < b.weight;
  });
}

void kruskal() {
  dsf.init(n);
  for (int i = 0; i < m; i++) {
    edge& z = e[i]; // syntactic sugar
    int maxw = dsf.try_unite(z.u, z.v, z.weight);
    if (maxw) {
      // Ca să o introducem pe z în APM, vom scoate o muchie de cost maxw.
      diff[z.orig] = z.weight - maxw;
    } else {
      mst_weight += z.weight;
      diff[z.orig] = 0;
    }
  }
}

void write_answers() {
  for (int i = 0; i < m; i++) {
    printf("%lld\n", mst_weight + diff[i]);
  }
}

int main() {
  read_data();
  sort_edges_by_weight();
```

```
  kruskal();
  write_answers();

  return 0;
}
```

28.7 Problema Apm2 (Infoarena)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 10'000;
const int MAX_EDGES = 100'000;

struct edge {
  int u, v, c;
};

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int cost[MAX_NODES + 1]; // costul părăsirii subarborelui
  int size[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
      size[u] = 1;
    }
  }

  int find(int u) {
    while (parent[u] != u) {
      u = parent[u];
    }
    return u;
  }

  void merge(int u, int v, int c) {
    u = find(u);
    v = find(v);
```

```
    if (u != v) {
      if (size[u] > size[v]) {
        int tmp = u; u = v; v = tmp;
      }
      parent[u] = v;
      cost[u] = c;
      size[v] += size[u];
    }
  }

  int get_max_cost(int u, int v) {
    int result = 0;

    while (u != v) {
      if (size[u] < size[v]) {
        result = max(result, cost[u]);
        u = parent[u];
      } else {
        result = max(result, cost[v]);
        v = parent[v];
      }
    }

    return result;
  }
};

edge e[MAX_EDGES];
disjoint_set_forest dsf;
int num_nodes, num_edges, num_queries;
FILE* fin;

void read_graph() {
  fin = fopen("apm2.in", "r");
  fscanf(fin, "%d %d %d", &num_nodes, &num_edges, &num_queries);
  for (int i = 0; i < num_edges; i++) {
    fscanf(fin, "%d %d %d", &e[i].u, &e[i].v, &e[i].c);
  }
}

void sort_edges_by_cost() {
  std::sort(e, e + num_edges, [] (edge& a, edge& b) {
    return a.c < b.c;
  });
}

void kruskal() {
  sort_edges_by_cost();
  dsf.init(num_nodes);
  for (int i = 0; i < num_edges; i++) {
```

```
    dsf.merge(e[i].u, e[i].v, e[i].c);
  }
}

void answer_queries() {
  FILE* fout = fopen("apm2.out", "w");

  while (num_queries--) {
    int u, v;
    fscanf(fin, "%d %d", &u, &v);
    int max_cost = dsf.get_max_cost(u, v);
    fprintf(fout, "%d\n", max_cost - 1);
  }

  fclose(fin);
  fclose(fout);
}

int main() {
  read_graph();
  kruskal();
  answer_queries();

  return 0;
}
```

28.8 Problema Edges in MST (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 100'000;

const int T_SOME = 0; // răspunsul implicit, dacă nu este suprascris
const int T_NONE = 1;
const int T_ALL = 2;

struct edge {
  int u, v;
  int cost;
  int orig;
};

struct disjoint_set_forest {
```

```
  int p[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }

  void unite(int u, int v) {
    p[find(v)] = find(u);
  }

  bool are_united(int u, int v) {
    return find(u) == find(v);
  }
};

struct reduced_edge {
  int v, orig; // O muchie u-v care are indexul „orig” în datele de intrare.
};

struct node {
  std::vector<reduced_edge> adj;
  int time, low;
};

edge e[MAX_EDGES];
disjoint_set_forest dsf;
node nd[MAX_NODES + 1];
int roots[2 * MAX_NODES], num_roots; // Rădăcini pentru grupa Kruskal curentă.
int answer[MAX_EDGES];
int n, m;

void read_data() {
  scanf("%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].cost);
    e[i].orig = i;
  }
}

void sort_edges() {
  std::sort(e, e + m, [](edge& a, edge& b) {
    return a.cost < b.cost;
```

```
  });
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int time;

void bridge_dfs(int u, int orig) {
  nd[u].time = nd[u].low = ++time;

  for (reduced_edge r: nd[u].adj) {
    if (r.orig != orig) {
      int v = r.v;
      if (!nd[v].time) {
        bridge_dfs(v, r.orig);
        nd[u].low = min(nd[u].low, nd[v].low);
        if (nd[v].low > nd[u].time) {
          answer[r.orig] = T_ALL;
        }

      } else {
        if (nd[v].time < nd[u].time) {
          nd[u].low = min(nd[u].low, nd[v].time);
        }
      }
    }
  }
}

void bridge_dfs_driver() {
  time = 0;
  for (int i = 0; i < num_roots; i++) {
    if (!nd[roots[i]].time) {
      bridge_dfs(roots[i], -1);
    }
  }
}

void dfs_cleanup() {
  for (int i = 0; i < num_roots; i++) {
    int u = roots[i];
    nd[u].adj.clear();
    nd[u].time = nd[u].low = 0;
  }
  num_roots = 0;
}

// Este important ca acest bloc să ruleze în O(batch_size). Dacă buclele scapă
```

```
// în O(n), vor duce la O(n²) global. De aceea, colectează capetele muchiilor
// și rulează DFS doar din acele noduri.
void process_batch(int start, int end) {
  num_roots = 0;

  for (int i = start; i < end; i++) {
    int u = dsf.find(e[i].u), v = dsf.find(e[i].v);
    if (u == v) {
      answer[e[i].orig] = T_NONE;
    } else {
      nd[u].adj.push_back({v, e[i].orig});
      nd[v].adj.push_back({u, e[i].orig});
      roots[num_roots++] = u;
      roots[num_roots++] = v;
    }
  }

  bridge_dfs_driver();
  dfs_cleanup();

  // Acum aplică modificările.
  for (int i = start; i < end; i++) {
    dsf.unite(e[i].u, e[i].v);
  }
}

void kruskal() {
  sort_edges();
  dsf.init(n);

  int i = 0;
  while (i < m) {
    int j = i + 1;
    while ((j < m) && (e[j].cost == e[i].cost)) {
      j++;
    }
    process_batch(i, j);
    i = j;
  }
}

void write_answers() {
  for (int i = 0; i < m; i++) {
    switch (answer[i]) {
      case T_SOME: printf("at least one\n"); break;
      case T_NONE: printf("none\n"); break;
      case T_ALL:  printf("any\n"); break;
    }
  }
}
```

```
int main() {
  read_data();
  kruskal();
  write_answers();

  return 0;
}
```

28.9 Problema Envy (Codeforces)

înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 500'000;
const int MAX_EDGES = 1'000'000;
const int MAX_QUERIES = 500'000;
const int NONE = MAX_QUERIES + 1;

struct edge {
  int u, v, w;
  int q;
};

// Pădure de mulțimi disjuncte augmentată cu suport pentru undo. Reține
// modificările pe o stivă. Poate confirma modificările golind stiva.
struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  int u_stack[MAX_NODES + 1], p_stack[MAX_NODES + 1];
  int stack_size;

  void init(int n) {
    stack_size = 0;
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  void save(int u) {
    u_stack[stack_size] = u;
    p_stack[stack_size] = p[u];
    stack_size++;
  }

  int find(int u) {
    if (p[u] == u) {
```

```
      return u;
    } else {
      save(u);
      p[u] = find(p[u]);
      return p[u];
    }
  }

  // Returnează true dacă și numai dacă u și v erau în arbori diferiți.
  bool unite(int u, int v) {
    u = find(u);
    v = find(v);
    if (u == v) {
      return false;
    } else {
      save(u);
      p[u] = v;
      return true;
    }
  }

  void commit() {
    stack_size = 0;
  }

  void restore() {
    while (stack_size) {
      stack_size--;
      p[u_stack[stack_size]] = p_stack[stack_size];
    }
  }
};

edge e[MAX_EDGES];
disjoint_set_forest dsf;
bool answer[MAX_QUERIES];
int n, m, num_queries;

void read_data() {
  scanf("%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
    e[i].q = NONE;
  }

  scanf("%d", &num_queries);
  for (int q = 0; q < num_queries; q++) {
    int qsize;
    scanf("%d", &qsize);
    while (qsize--) {
```

```
      int ind;
      scanf("%d", &ind);
      ind--;
      e[m++] = { e[ind].u, e[ind].v, e[ind].w, q };
    }
    answer[q] = true;
  }
}

void sort_edges() {
  std::sort(e, e + m, [] (edge& a, edge& b) {
    return (a.w < b.w) ||
      ((a.w == b.w) && (a.q < b.q));
  });
}

int find_batch(int start) {
  int end = start;
  while ((end < m) && (e[start].w == e[end].w) && (e[start].q == e[end].q)) {
    end++;
  }
  return end;
}

void process_equal_weights(int start, int end) {
  int k = start;
  while ((k < end) && answer[e[k].q]) {
    answer[e[k].q] = dsf.unite(e[k].u, e[k].v);
    k++;
  }
  dsf.restore();
}

void process_edges() {
  dsf.init(n);
  int start = 0;
  while (start < m) {
    if (e[start].q == NONE) {
      dsf.unite(e[start].u, e[start].v);
      dsf.commit();
      start++;
    } else {
      int end = find_batch(start);
      process_equal_weights(start, end);
      start = end;
    }
  }
}

void write_answers() {
```

```
  for (int i = 0; i < num_queries; i++) {
    printf("%s\n", answer[i] ? "YES" : "NO");
  }
}

int main() {
  read_data();
  sort_edges();
  process_edges();
  write_answers();

  return 0;
}
```

28.10 Problema Gift (Codeforces)

înapoi

Sursă bazată pe Kruskal repetat (versiune online).

```
// Complexitate: O(mn log* m + m log m).
#include <algorithm>
#include <stdio.h>

const int MAX_NODES = 200;
const int MAX_EDGES = 50'000;
const long long INFINITY = 3'000'000'000'000'000'000LL;

struct edge {
  int u, v, g, s;
};

struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }
  }

  int find(int u) {
    return (p[u] == u)
      ? u
      : (p[u] = find(p[u]));
  }
```

```
  bool are_separate(edge e) {
    return find(e.u) != find(e.v);
  }

  void unite(edge e) {
    p[find(e.v)] = find(e.u);
  }
};

edge e[MAX_EDGES];
edge e_by_s[MAX_EDGES];
disjoint_set_forest dsf;
int n, m, m_by_s;
long long g, s;

void read_data() {
  scanf("%d %d %lld %lld", &n, &m, &g, &s);
  for (int i = 0; i < m; i++) {
    scanf("%d %d %d %d", &e[i].u, &e[i].v, &e[i].g, &e[i].s);
  }
}

void sort_edges_by_gold() {
  std::sort(e, e + m, [](edge& a, edge& b) {
    return a.g < b.g;
  });
}

void add_admissible_edge(edge e) {
  int i = m_by_s++;
  while ((i > 0) && (e_by_s[i - 1].s > e.s)) {
    e_by_s[i] = e_by_s[i - 1];
    i--;
  }
  e_by_s[i] = e;
}

void kruskal() {
  dsf.init(n);
  int new_m_by_s = 0;

  for (int i = 0; i < m_by_s; i++) {
    if (dsf.are_separate(e_by_s[i])) {
      dsf.unite(e_by_s[i]);
      e_by_s[new_m_by_s++] = e_by_s[i];
    }
  }

  m_by_s = new_m_by_s;
```

```
}

long long try_all_g() {
  long long min_cost = INFINITY;

  int i = 0;
  while (i < m) {
    int j = i;
    while ((j < m) && (e[j].g == e[i].g)) {
      add_admissible_edge(e[j++]);
    }

    kruskal();
    if (m_by_s == n - 1) {
      min_cost = std::min(min_cost, g * e[i].g + s * e_by_s[m_by_s - 1].s);
    }

    i = j;
  }

  return min_cost;
}

int main() {
  read_data();
  sort_edges_by_gold();
  long long answer = try_all_g();
  printf("%lld\n", (answer == INFINITY) ? -1 : answer);

  return 0;
}
```

Sursă cu actualizarea APM (versiune online).

```
// Complexitate: O(mn + m log m).
#include <algorithm>
#include <set>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200;
const int MAX_EDGES = 50'000;
const long long INFINITY = 3e18;

struct edge {
  int u, v, g, s;
};

struct mst_graph {
```

```
struct neighbor {
  int v, s;
};

struct node {
  std::vector<neighbor> adj;
};

node nd[MAX_NODES + 1];
std::multiset<int> silvers;
int n, num_comps;

void init(int _n) {
  n = num_comps = _n;
}

int get_max_s() {
  return *silvers.rbegin();
}

bool is_connected() {
  return num_comps == 1;
}

void remove_neighbor(int u, int v) {
  int i = 0;
  while (nd[u].adj[i].v != v) {
    i++;
  }
  nd[u].adj[i] = nd[u].adj.back();
  nd[u].adj.pop_back();
}

void connect(edge e) {
  nd[e.u].adj.push_back({e.v, e.s});
  nd[e.v].adj.push_back({e.u, e.s});
  silvers.insert(e.s);
  num_comps--;
}

void disconnect(edge e) {
  remove_neighbor(e.u, e.v);
  remove_neighbor(e.v, e.u);
  silvers.erase(silvers.find(e.s));
  num_comps++;
}

edge get_max_edge_dfs(int u, int parent, int target) {
  edge max = { .s = 0 };
  for (neighbor nb: nd[u].adj) {
```

```
      if (nb.v != parent) {
        if (nb.v == target) {
          return { .u = u, .v = nb.v, .s = nb.s};
        }
        edge sub = get_max_edge_dfs(nb.v, u, target);
        if (sub.s && (nb.s > sub.s)) {
          sub = { .u = u, .v = nb.v, .s = nb.s };
        }
        if (sub.s > max.s) {
          max = sub;
        }
      }
    }
    return max;
  }

  void include_edge(edge e) {
    edge max = get_max_edge_dfs(e.u, 0, e.v);
    if (!max.s) {
      // u și v nu au fost conectate încă
      connect(e);
    } else if (e.s < max.s) {
      // u și v sînt conectate, dar e este mai bună (mai ieftină în argint)
      disconnect(max);
      connect(e);
    }
  }
};

edge e[MAX_EDGES];
mst_graph mst;
int n, m;
long long g, s;

void read_data() {
  int m_with_self;
  scanf("%d %d %lld %lld", &n, &m_with_self, &g, &s);
  for (int i = 0; i < m_with_self; i++) {
    scanf("%d %d %d %d", &e[m].u, &e[m].v, &e[m].g, &e[m].s);
    if (e[m].u != e[m].v) {
      m++;
    }
  }
}

void sort_edges_by_gold() {
  std::sort(e, e + m, [](edge& a, edge& b) {
    return a.g < b.g;
  });
}
```

```
long long scan_g() {
  long long min_cost = INFINITY;
  mst.init(n);

  for (int i = 0; i < m; i++) {
    mst.include_edge(e[i]);
    if (mst.is_connected()) {
      min_cost = std::min(min_cost, g * e[i].g + s * mst.get_max_s());
    }
  }

  return min_cost;
}

int main() {
  read_data();
  sort_edges_by_gold();
  long long answer = scan_g();
  printf("%lld\n", (answer == INFINITY) ? -1 : answer);

  return 0;
}
```

28.11 Problema DFS Trees (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;
const int WHITE = 0;
const int BLACK = -1;

struct cell {
  int val, next;
};

struct node {
  int ptr, nptr; // liste de adiacență pentru muchii de arbore/nonarbore
  int state;     // 0 = alb; -1 = negru; d > 0 = gri la adîncime d
  int bad_count;
};

struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int n;
```

```
  void init(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
    }
  }

  int find(int u) {
    return (parent[u] == u)
      ? u
      : (parent[u] = find(parent[u]));
  }

  // Unifică v în u. Fără *union by rank*.
  // Returnează true dacă și numai dacă unificarea chiar a reușit.
  bool merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      parent[v] = u;
      return true;
    } else {
      return false;
    }
  }
};

cell list[2 * MAX_EDGES + 1];
node nd[MAX_NODES + 1];
int stack[MAX_NODES];
disjoint_set_forest dsf;
int n, list_pos = 1;

void add_tree_edge(int u, int v) {
  list[list_pos] = { v, nd[u].ptr };
  nd[u].ptr = list_pos++;
}

void add_non_tree_edge(int u, int v) {
  list[list_pos] = { v, nd[u].nptr };
  nd[u].nptr = list_pos++;
}

void read_graph_and_kruskal() {
  int num_edges, u, v;

  scanf("%d %d", &n, &num_edges);
  dsf.init(n);
  while (num_edges--) {
```

```
    scanf("%d %d", &u, &v);
    if (dsf.merge(u, v)) {
      add_tree_edge(u, v);
      add_tree_edge(v, u);
    } else {
      add_non_tree_edge(u, v);
      add_non_tree_edge(v, u);
    }
  }
}

// Nodul u este nodul DFS activ. Nodul v este fie gri, fie negru.
void mark_subtrees(int u, int v) {
  if (nd[v].state == BLACK) {
    nd[1].bad_count++;
    nd[v].bad_count--;
    nd[u].bad_count--;
  } else {
    int w = stack[nd[v].state + 1];
    nd[w].bad_count++;
    nd[u].bad_count--;
  }
}

void dfs_mark_counts(int u, int depth) {
  nd[u].state = depth;
  stack[depth] = u;

  // Marchează muchiile de nonarbore.
  for (int ptr = nd[u].nptr; ptr; ptr = list[ptr].next) {
    int v = list[ptr].val;
    if (nd[v].state != WHITE) {
      mark_subtrees(u, v);
    }
  }

  // Traversează muchiile de arbore.
  for (int ptr = nd[u].ptr; ptr; ptr = list[ptr].next) {
    int v = list[ptr].val;
    if (nd[v].state == WHITE) {
      dfs_mark_counts(v, depth + 1);
    }
  }

  nd[u].state = BLACK;

}

void dfs_propagate_counts(int u, int parent) {
  for (int ptr = nd[u].ptr; ptr; ptr = list[ptr].next) {
```

```
    int v = list[ptr].val;
    if (v != parent) {
      nd[v].bad_count += nd[u].bad_count;
      dfs_propagate_counts(v, u);
    }
  }
}

void write_answers() {
  for (int u = 1; u <= n; u++) {
    char answer = (nd[u].bad_count == 0) ? '1' : '0';
    putchar(answer);
  }
  putchar('\n');
}

int main() {
  read_graph_and_kruskal();

  dfs_mark_counts(1, 1);
  dfs_propagate_counts(1, 0);

  write_answers();

  return 0;
}
```

Anexa 29

Grafuri - Conectivitate

29.1 Problema Course Schedule (CSES)

‹ înapoi

Sursă cu BFS (algoritmul lui Kahn).

```cpp
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;

struct node {
  std::vector<int> adj;
  int in_degree;
};

std::vector<int> order;
node nd[MAX_NODES + 1];
int n;

void read_graph() {
  int num_edges, u, v;

  scanf("%d %d", &n, &num_edges);
  while (num_edges--) {
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].in_degree++;
  }
}

void bfs() {
  std::queue<int> q;
```

```
  for (int u = 1; u <= n; u++) {
    if (!nd[u].in_degree) {
      q.push(u);
    }
  }

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (auto v: nd[u].adj) {
      nd[v].in_degree--;
      if (!nd[v].in_degree) {
        q.push(v);
      }
    }
  }
}

void write_order() {
  if ((int)order.size() == n) {
    for (int u: order) {
      printf("%d ", u);
    }
    printf("\n");
  } else {
    printf("IMPOSSIBLE\n");
  }
}

int main() {
  read_graph();
  bfs();
  write_order();

  return 0;
}
```

Sursă cu DFS.

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int WHITE = 0;
const int GRAY = 1;
const int BLACK = 2;

struct node {
```

```
  std::vector<int> adj;
  char color;
};

node nd[MAX_NODES + 1];
std::vector<int> order;
bool has_cycle;
int n;

void read_graph() {
  int num_edges, u, v;

  scanf("%d %d", &n, &num_edges);
  while (num_edges--) {
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
  }
}

void dfs(int u) {
  has_cycle |= (nd[u].color == GRAY);

  if (nd[u].color == WHITE) {
    nd[u].color = GRAY;
    for (auto v: nd[u].adj) {
      dfs(v);
    }
    nd[u].color = BLACK;
    order.push_back(u);
  }
}

void dfs_driver() {
  for (int u = 1; u <= n; u++) {
    dfs(u);
  }
}

void write_order() {
  if (has_cycle) {
    printf("IMPOSSIBLE\n");
  } else {
    for (int i = n - 1; i >= 0; i--) {
      printf("%d ", order[i]);
    }
    printf("\n");
  }
}

int main() {
```

```
  read_graph();
  dfs_driver();
  write_order();

  return 0;
}
```

29.2 Problema Book (Codeforces)

‹ înapoi

Sursă cu BFS (algoritmul lui Kahn).

```
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int in_degree;
  int pass;
};

struct queue {
  int a[MAX_NODES];
  int head, tail;

  void init() {
    head = tail = 0;
  }

  void push(int u) {
    a[tail++] = u;
  }

  int pop() {
    return a[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};
```

```
cell list[MAX_EDGES + 1];
node nd[MAX_NODES + 1];
queue q;
int n, num_edges, num_learned;

void clear_graph() {
  num_edges = 0;
  num_learned = 0;
  for (int u = 1; u <= n; u++) {
    nd[u].adj = 0;
    nd[u].pass = 0;
  }
}

void add_edge(int u, int v) {
  list[++num_edges] = { v, nd[u].adj };
  nd[u].adj = num_edges;
}

void read_graph() {
  scanf("%d", &n);

  clear_graph();
  for (int u = 1; u <= n; u++) {
    scanf("%d", &nd[u].in_degree);
    for (int i = 0; i < nd[u].in_degree; i++) {
      int v;
      scanf("%d", &v);
      add_edge(v, u);
    }
  }
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

void update_passes(int u, int v) {
  int pass_v = (u < v) ? nd[u].pass : (nd[u].pass + 1);
  nd[v].pass = max(nd[v].pass, pass_v);
}

void bfs() {
  q.init();

  for (int u = 1; u <= n; u++) {
    if (!nd[u].in_degree) {
      nd[u].pass = 1;
      q.push(u);
```

```
    }
  }

  while (!q.is_empty()) {
    num_learned++;
    int u = q.pop();
    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      update_passes(u, v);
      nd[v].in_degree--;
      if (!nd[v].in_degree) {
        q.push(v);
      }
    }
  }
}

int get_max_passes() {
  if (num_learned < n) {
    return -1;
  }

  int m = 0;
  for (int u = 1; u <= n; u++) {
    m = max(m, nd[u].pass);
  }
  return m;
}

void solve_test() {
  read_graph();
  bfs();
  int answer = get_max_passes();
  printf("%d\n", answer);
}

int main() {

  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

Sursă cu DFS.

```cpp
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;

const int WHITE = 0;
const int GRAY = 1;
const int BLACK = 2;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int pass;
  char color;
};

cell list[MAX_EDGES + 1];
node nd[MAX_NODES + 1];
int n, m, max_pass;
bool has_cycle;

void clear_graph() {
  m = 0;
  max_pass = 1;
  has_cycle = false;
  for (int u = 1; u <= n; u++) {
    nd[u].adj = 0;
    nd[u].pass = 1;
    nd[u].color = WHITE;
  }
}

void add_edge(int u, int v) {
  list[++m] = { v, nd[u].adj };
  nd[u].adj = m;
}

void read_graph() {
  scanf("%d", &n);

  clear_graph();
  for (int u = 1; u <= n; u++) {
    int in_degree, v;
    scanf("%d", &in_degree);
    while (in_degree--) {
      scanf("%d", &v);
```

```
      add_edge(v, u);
    }
  }
}

void maximize(int& x, int y) {
  if (y > x) {
    x = y;
  }
}

void top_sort(int u) {
  has_cycle |= (nd[u].color == GRAY);
  if (nd[u].color != WHITE) {
    return;
  }

  nd[u].color = GRAY;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    top_sort(v);
    maximize(nd[u].pass, nd[v].pass + (u > v));
    maximize(max_pass, nd[u].pass);
  }

  nd[u].color = BLACK;
}

void solve_test() {
  read_graph();

  for (int u = 1; u <= n; u++) {
    top_sort(u);
  }
  printf("%d\n", has_cycle ? -1 : max_pass);
}

int main() {

  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

29.3 Problema Componente tare conexe (Infoarena)

înapoi

Sursă cu algoritmul lui Kosaraju și STL (versiune online).

```cpp
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;

struct node {
  std::vector<int> adj, adjt;
  bool mark;
};

node n[MAX_NODES + 1];
std::vector<int> order;
std::vector<std::vector<int>> comps;
int num_nodes;

void read_graph() {
  FILE* f = fopen("ctc.in", "r");
  int num_edges, u, v;
  fscanf(f, "%d %d", &num_nodes, &num_edges);
  for (int i = 1; i <= num_edges; i++) {
    fscanf(f, "%d %d", &u, &v);
    n[u].adj.push_back(v);
    n[v].adjt.push_back(u);
  }
  fclose(f);
}

void dfs(int u) {
  n[u].mark = true;
  for (auto v: n[u].adj) {
    if (!n[v].mark) {
      dfs(v);
    }
  }

  order.push_back(u);
}

void dfs_driver() {
  for (int u = 1; u <= num_nodes; u++) {
    if (!n[u].mark) {
      dfs(u);
    }
  }
```

```
}

void tdfs(int u) {
  n[u].mark = false;
  for (auto v: n[u].adjt) {
    if (n[v].mark) {
      tdfs(v);
    }
  }

  comps.back().push_back(u);
}

void transpose_dfs_driver() {
  for (int i = num_nodes - 1; i >= 0; i--) {
    int u = order[i];
    if (n[u].mark) {
      comps.push_back({});
      tdfs(u);
    }
  }
}

void write_components() {
  FILE* f = fopen("ctc.out", "w");
  fprintf(f, "%lu\n", comps.size());
  for (auto vec: comps)  {
    for (auto u: vec) {
      fprintf(f, "%d ", u);
    }
    fprintf(f, "\n");
  }
  fclose(f);
}

int main() {
  read_graph();
  dfs_driver();
  transpose_dfs_driver();
  write_components();

  return 0;
}
```

Sursă cu algoritmul lui Kosaraju și structuri proprii (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 100'000;
```

```
const int MAX_EDGES = 200'000;

struct edge {
  int v, next;
};

struct node {
  int adj;
  bool mark;
};

node n[MAX_NODES + 1];
edge e[MAX_EDGES + 1];
int order[MAX_NODES], order_size;
int comp[MAX_NODES + 1], comp_size;
int comp_start[MAX_NODES + 1], num_comps;
int num_nodes;

void read_graph() {
  FILE* f = fopen("ctc.in", "r");
  int num_edges, u, v;
  fscanf(f, "%d %d", &num_nodes, &num_edges);
  for (int i = 1; i <= num_edges; i++) {
    fscanf(f, "%d %d", &u, &v);
    e[i] = { v, n[u].adj };
    n[u].adj = i;
  }
  fclose(f);
}

void dfs(int u) {
  n[u].mark = true;
  for (int ptr = n[u].adj; ptr; ptr = e[ptr].next) {
    int v = e[ptr].v;
    if (!n[v].mark) {
      dfs(v);
    }
  }

  order[order_size++] = u;
}

void dfs_driver() {
  for (int u = 1; u <= num_nodes; u++) {
    if (!n[u].mark) {
      dfs(u);
    }
  }
}
```

```
// Transpune graful, reutilizînd vectorul de muchii.
void transpose() {
  int *old_adj = comp; // syntactic sugar
  for (int u = 1; u <= num_nodes; u++) {
    old_adj[u] = n[u].adj;
    n[u].adj = 0;
  }
  for (int u = 1; u <= num_nodes; u++) {
    int ptr = old_adj[u];
    while (ptr) {
      // mută e[ptr] din lista de adiacență a lui u în a lui v.
      int v = e[ptr].v;
      int next = e[ptr].next;
      e[ptr] = { u, n[v].adj };
      n[v].adj = ptr;
      ptr = next;
    }
  }
}

void tdfs(int u) {
  n[u].mark = false;
  for (int ptr = n[u].adj; ptr; ptr = e[ptr].next) {
    int v = e[ptr].v;
    if (n[v].mark) {
      tdfs(v);
    }
  }

  comp[comp_size++] = u;
}

void transpose_dfs_driver() {
  for (int i = num_nodes - 1; i >= 0; i--) {
    int u = order[i];
    if (n[u].mark) {
      comp_start[num_comps++] = comp_size;
      tdfs(u);
    }
  }
  comp_start[num_comps] = num_nodes;
}

void write_components() {
  FILE* f = fopen("ctc.out", "w");
  fprintf(f, "%d\n", num_comps);
  for (int i = 0; i < num_comps; i++) {
    for (int j = comp_start[i]; j < comp_start[i + 1]; j++) {
      fprintf(f, "%d ", comp[j]);
    }
```

```
    fprintf(f, "\n");
  }
  fclose(f);
}

int main() {
  read_graph();
  dfs_driver();
  transpose();
  transpose_dfs_driver();
  write_components();

  return 0;
}
```

Sursă cu algoritmul lui Tarjan și STL (versiune online).

```
#include <algorithm>
#include <stack>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;

struct node {
  std::vector<int> adj;
  int time_in;
  int low;
  bool on_stack;
};

node n[MAX_NODES + 1];
std::stack<int> st;  // stiva DFS
std::vector<std::vector<int>> comps;
int num_nodes, time;

void read_graph() {
  FILE* f = fopen("ctc.in", "r");
  int num_edges, u, v;
  fscanf(f, "%d %d", &num_nodes, &num_edges);
  for (int i = 1; i <= num_edges; i++) {
    fscanf(f, "%d %d", &u, &v);
    n[u].adj.push_back(v);
  }
  fclose(f);
}

void pop_scc(int last) {
  std::vector<int> c;
```

```
  int v;
  do {
    v = st.top();
    st.pop();
    c.push_back(v);
    n[v].on_stack = false;
  } while (v != last);
  comps.push_back(c);
}

void dfs(int u) {
  n[u].time_in = n[u].low = ++time;
  n[u].on_stack = true;
  st.push(u);

  for (auto v: n[u].adj) {
    if (!n[v].time_in) {
      // Traversează fiul alb și notează cît poate să urce.
      dfs(v);
      n[u].low = std::min(n[u].low, n[v].low);
    } else if (n[v].on_stack) {
      // Putem urca pînă la nivelul lui v
      n[u].low = std::min(n[u].low, n[v].time_in);
    }
  }

  // Dacă niciun nod din subarborele lui u nu poate urca mai sus decît u,
  // atunci stiva de la u pînă la vîrf este o CTC.
  if (n[u].low == n[u].time_in) {
    pop_scc(u);
  }
}

void dfs_driver() {
  for (int u = 1; u <= num_nodes; u++) {
    if (!n[u].time_in) {
      dfs(u);
    }
  }
}

void write_components() {
  FILE* f = fopen("ctc.out", "w");
  fprintf(f, "%lu\n", comps.size());
  for (auto vec: comps)  {
    for (auto u: vec) {
      fprintf(f, "%d ", u);
    }
    fprintf(f, "\n");
  }
```

```
  fclose(f);
}

int main() {
  read_graph();
  dfs_driver();
  write_components();

  return 0;
}
```

29.4 Problema Mr. Kitayuta's Technology (Codeforces)

înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 100'000;
const int GRAY = 1;
const int BLACK = 2;

struct edge {
  int v, next;
};

struct node {
  int adj;
  char color;
};

// O pădure de mulțimi disjuncte augmentată cu mărimea și nodurile fiecărei
// rădăcini.
struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int size[MAX_NODES + 1];
  int next[MAX_NODES + 1];
  int last[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
      size[u] = 1;
      next[u] = 0;
      last[u] = u;
    }
```

```
  }

  int find(int u) {
    return (parent[u] == u)
      ? u
      : (parent[u] = find(parent[u]));
  }

  void merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      parent[v] = u;
      size[u] += size[v];
      next[last[u]] = v;
      last[u] = last[v];
    }
  }
};

node nd[MAX_NODES + 1];
edge e[MAX_EDGES + 1];
disjoint_set_forest dsf;
int n;

void add_edge(int u, int v) {
  static int pos = 1;
  e[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_graph_and_wcc() {
  int num_edges, u, v;
  scanf("%d %d", &n, &num_edges);
  dsf.init(n);

  while (num_edges--) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    dsf.merge(u, v);
  }
}

bool find_cycle(int u) {
  if (nd[u].color == GRAY) {
    return true;
  } else if (nd[u].color == BLACK) {
    return false;
  }
```

```
  nd[u].color = GRAY;
  int ptr = nd[u].adj;
  while (ptr && !find_cycle(e[ptr].v)) {
    ptr = e[ptr].next;
  }
  nd[u].color = BLACK;

  return ptr;
}

int satisfy_wcc(int u) {
  int size = dsf.size[u];
  while (u && !find_cycle(u)) {
    u = dsf.next[u];
  }
  return u ? size : (size - 1);
}

void satisfy_all_wccs() {
  int answer = 0;
  for (int u = 1; u <= n; u++) {
    if (dsf.parent[u] == u) {
      answer += satisfy_wcc(u);
    }
  }
  printf("%d\n", answer);
}

int main() {
  read_graph_and_wcc();
  satisfy_all_wccs();

  return 0;
}
```

29.5 Problema Ralph and Mushrooms (Codeforces)

‹ înapoi • versiune online

```
#include <math.h>
#include <stdio.h>

const int MAX_NODES = 1'000'000;
const int MAX_EDGES = 1'000'000;

typedef unsigned long long u64;

struct edge {
```

```
  int v, mushrooms, next;
};

struct node {
  u64 inc_profit, exc_profit; // profitul cu/fără CTC-ul lui u
  int adj;
  int time_in;
  int low;
  bool on_stack;
};

node nd[MAX_NODES + 1];
edge e[MAX_EDGES + 1];
int st[MAX_NODES], ss;  // stiva DFS
int n, start_node;

void read_graph() {
  int num_edges, u;
  scanf("%d %d", &n, &num_edges);
  for (int i = 1; i <= num_edges; i++) {
    scanf("%d %d %d", &u, &e[i].v, &e[i].mushrooms);
    e[i].next = nd[u].adj;
    nd[u].adj = i;
  }
  scanf("%d", &start_node);
}

u64 max(u64 x, u64 y) {
  return (x > y) ? x : y;
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

u64 get_mushroom_profit(int m) {
  // Implementează https://oeis.org/A060432. Remarcăm că diferențele sînt 1,
  // 2, 2, 3, 3, 3, 4, 4, 4, 4 etc. De aceea, rezolvă t * (t + 1) / 2 ≤ m.
  // Ultimul termen cu un t întreg este o sumă de pătrate.
  int t = (sqrtl(8 * m + 1) - 1) * 0.5;
  u64 base = t * (t + 1) / 2;
  u64 sum = base * (2 * t + 1) / 3;
  sum += (u64)(m - base) * (t + 1);
  return sum;
}

void update_profits(int u, int v, int mushrooms) {
  if (nd[v].on_stack) {
    // muchie internă
    u64 p = get_mushroom_profit(mushrooms);
```

```
    nd[u].exc_profit = max(nd[u].exc_profit, nd[v].exc_profit);
    if (u == v) {
      nd[u].inc_profit += p;
    } else {
      nd[u].inc_profit += p + nd[v].inc_profit;
      nd[v].inc_profit = 0;
    }
  } else {
    // muchie externă
    nd[u].exc_profit = max(nd[u].exc_profit, nd[v].inc_profit + mushrooms);
  }
}

void pop_scc(int root) {
  u64 scc_profit = nd[root].inc_profit + nd[root].exc_profit;

  int v;
  do {
    v = st[--ss];
    nd[v].on_stack = false;
    nd[v].inc_profit = scc_profit;
  } while (v != root);
}

void tarjan(int u) {
  static int time = 0;

  nd[u].time_in = nd[u].low = ++time;
  nd[u].on_stack = true;
  st[ss++] = u;

  for (int ptr = nd[u].adj; ptr; ptr = e[ptr].next) {
    int v = e[ptr].v;

    if (!nd[v].time_in) {
      tarjan(v);
      nd[u].low = min(nd[u].low, nd[v].low);
    } else if (nd[v].on_stack) {
      nd[u].low = min(nd[u].low, nd[v].time_in);
    }

    update_profits(u, v, e[ptr].mushrooms);
  }

  if (nd[u].low == nd[u].time_in) {
    pop_scc(u);
  }
}

void write_answer() {
```

```
  printf("%llu\n", nd[start_node].inc_profit);
}

int main() {
  read_graph();
  tarjan(start_node);
  write_answer();

  return 0;
}
```

29.6 Problema Bertown Roads (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 300'000;

struct edge {
  int u, v;
};

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth, low;
};

node nd[MAX_NODES + 1];
cell list[2 * MAX_EDGES + 1];
edge e[MAX_EDGES];
int n, sol_edges;
bool has_bridges;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_graph() {
  int num_edges, u, v;
  scanf("%d %d", &n, &num_edges);
```

```
  while (num_edges--) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void dfs(int u, int parent) {
  nd[u].depth = nd[u].low = nd[parent].depth + 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;

    if (!nd[v].depth) {
      // muchie de arbore
      e[sol_edges++] = { u, v };
      dfs(v, u);
      nd[u].low = min(nd[u].low, nd[v].low);
      has_bridges |= (nd[v].low > nd[u].depth);

    } else if ((v != parent) && (nd[v].depth < nd[u].depth)) {
      // muchie înapoi
      e[sol_edges++] = { u, v };
      nd[u].low = min(nd[u].low, nd[v].depth);

    }
  }
}

void write_answer() {
  if (has_bridges) {
    printf("0\n");
  } else {
    for (int i = 0; i < sol_edges; i++) {
      printf("%d %d\n", e[i].u, e[i].v);
    }
  }
}

int main() {
  read_graph();
  dfs(1, 0);
  write_answer();

  return 0;
```

```
}
```

29.7 Problema Tourist Reform (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 400'000;
const int MAX_EDGES = 400'000;

struct edge {
  int u, v;
};

struct cell {
  int edge_index, next;
};

struct node {
  int adj;
  int depth, low;
};

node nd[MAX_NODES + 1];
cell list[2 * MAX_EDGES + 1];
edge e[MAX_EDGES + 1];
int n, m, stack_size, max_comp_size, max_comp_node;

void add_edge(int u, int edge_index) {
  static int pos = 1;
  list[pos] = { edge_index, nd[u].adj };
  nd[u].adj = pos++;
}

void read_graph() {
  scanf("%d %d", &n, &m);

  for (int i = 1; i <= m; i++) {
    scanf("%d %d", &e[i].u, &e[i].v);
    add_edge(e[i].u, i);
    add_edge(e[i].v, i);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}
```

```
void pop_bcc(int root, int root_stack_pos) {
  if (stack_size - root_stack_pos > max_comp_size) {
    max_comp_size = stack_size - root_stack_pos;
    max_comp_node = root;
  }
  stack_size = root_stack_pos;
}

void dfs(int u, int parent) {
  nd[u].depth = nd[u].low = nd[parent].depth + 1;
  int stack_pos = stack_size++;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    edge& f = e[list[ptr].edge_index];
    int v = (f.u == u) ? f.v : f.u;

    if (!nd[v].depth) {
      dfs(v, u);
      nd[u].low = min(nd[u].low, nd[v].low);
    } else if ((v != parent) && (nd[v].depth < nd[u].depth)) {
      nd[u].low = min(nd[u].low, nd[v].depth);
    }
  }

  if (nd[u].low > nd[parent].depth) {
    pop_bcc(u, stack_pos);
  }
}

void wipe_depths() {
  for (int u = 1; u <= n; u++) {
    nd[u].depth = 0;
  }
}

void orient_edges(int u, int parent) {
  nd[u].depth =  nd[parent].depth + 1;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    edge& f = e[list[ptr].edge_index];
    int v = (f.u == u) ? f.v : f.u;

    if (!nd[v].depth) {
      f = { v, u };
      orient_edges(v, u);
    } else if ((v != parent) && (nd[v].depth < nd[u].depth)) {
      f = { v, u };
    }
  }
```

```
}

void write_answer() {
  printf("%d\n", max_comp_size);
  for (int i = 1; i <= m; i++) {
    printf("%d %d\n", e[i].u, e[i].v);
  }
}

int main() {
  read_graph();
  dfs(1, 0);
  wipe_depths();
  orient_edges(max_comp_node, 0);
  write_answer();

  return 0;
}
```

29.8 Problema We Need More Bosses (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 300'000;
const int MAX_EDGES = 300'000;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int depth, low;
};

node nd[MAX_NODES + 1];
cell list[2 * MAX_EDGES + 1];
int n, diam;

void add_edge(int u, int v) {
  static int pos = 1;
  list[pos] = { v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
```

```
  int num_edges, u, v;
  scanf("%d %d", &n, &num_edges);

  while (num_edges--) {
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

void minimize(int& x, int y) {
  x = (y < x) ? y : x;
}

void maximize(int& x, int y) {
  x = (y > x) ? y : x;
}

// Returnează distanța maximă în punți de la u la orice frunză.
int dfs(int u, int parent) {
  nd[u].depth = nd[u].low = nd[parent].depth + 1;
  int dist = 0; // distanța maximă văzută în fiii anteriori

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;

    if (!nd[v].depth) {
      int dist2 = dfs(v, u); // distanța maximă văzută în v
      if (nd[v].low > nd[u].depth) {
        dist2++; // u-v este punte
      }
      minimize(nd[u].low, nd[v].low);
      maximize(diam, dist2 + dist);
      maximize(dist, dist2);
    } else if (v != parent) {
      minimize(nd[u].low, nd[v].depth);
    }
  }

  return dist;
}

void write_answer() {
  printf("%d\n", diam);
}

int main() {
  read_data();
  dfs(1, 0);
  write_answer();
```

```
  return 0;
}
```

29.9 Problema Forbidden Cities (CSES)

înapoi

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 200'000;
const int MAX_QUERIES = 100'000;

struct node {
  std::vector<int> adj;           // lista de adiacență
  std::vector<int> queries;       // interogări în care eu sînt capătul
  std::vector<int> notifications; // interogări în care eu sînt centrul
  int tin, tout;
  int low;
  bool notify_me;
};

struct query {
  int a, b, c;
  bool red_flag;
};

node nd[MAX_NODES + 1];
query q[MAX_QUERIES + 1];
int n, num_queries;

void read_data() {
  int num_edges, u, v;
  scanf("%d %d %d", &n, &num_edges, &num_queries);

  while (num_edges--) {
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  for (int i = 1; i <= num_queries; i++) {
    scanf("%d %d %d", &q[i].a, &q[i].b, &q[i].c);
    nd[q[i].a].queries.push_back(i);
    nd[q[i].b].queries.push_back(i);
  }
```

```
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool is_ancestor(int u, int v) {
  return (nd[v].tin >= nd[u].tin) && (nd[v].tout <= nd[u].tout);
}

void send_notifications(int u) {
  for (int ind: nd[u].queries) {
    int c = q[ind].c;
    if (nd[c].notify_me) {
      nd[c].notifications.push_back(ind);
    }
  }
}

void receive_notifications(int u, int child) {
  for (int ind: nd[u].notifications) {
    if (is_ancestor(child, q[ind].a) ^ is_ancestor(child, q[ind].b)) {
      q[ind].red_flag = true;
    }
  }
}

void clear_notifications(int u) {
  nd[u].notifications.clear();
}

void dfs(int u, int parent) {
  static int time = 0;
  nd[u].tin = nd[u].low = ++time;
  nd[u].notify_me = true;

  for (int v: nd[u].adj) {
    if (!nd[v].tin) {
      dfs(v, u);
      nd[u].low = min(nd[u].low, nd[v].low);

      if (nd[v].low >= nd[u].tin) {
        // v nu îl poate ocoli pe u: u este punct de articulație
        receive_notifications(u, v);
      }
      clear_notifications(u);

    } else if (v != parent) {
      nd[u].low = min(nd[u].low, nd[v].tin);
    }
```

```
  }

  send_notifications(u);
  nd[u].notify_me = false;
  nd[u].tout = ++time;
}

void write_answers() {
  for (int i = 1; i <= num_queries; i++) {
    q[i].red_flag |= (q[i].a == q[i].c) || (q[i].b == q[i].c);
    printf(q[i].red_flag ? "NO\n" : "YES\n");
  }
}

int main() {
  read_data();
  dfs(1, 0);
  write_answers();

  return 0;
}
```

29.10 Problema Case of Computer Network (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;

struct node {
  // vecinii
  std::vector<int> adj;
  // celelalte capete ale mesajelor care încep / se termină aici
  std::vector<int> pairs;
  // adîncimea mea și cea mai joasă adîncime accesibilă din subarborele meu
  int depth, low;
  // surse și destinații deschise în subarborele meu
  int num_src, num_dest;
};

struct disjoint_set_forest {
  int parent[MAX_NODES + 1];
  int n;

  void init(int n) {
    this->n = n;
```

```
    for (int u = 1; u <= n; u++) {
      parent[u] = u;
    }
  }

  int find(int u) {
    return (parent[u] == u)
      ? u
      : (parent[u] = find(parent[u]));
  }

  void merge(int u, int v) {
    parent[find(v)] = find(u);
  }
};

node nd[MAX_NODES + 1];
disjoint_set_forest ds;
int n;
bool red_flag;

void read_data() {
  int num_edges, num_queries, u, v;
  scanf("%d %d %d", &n, &num_edges, &num_queries);

  while (num_edges--) {
    scanf("%d %d", &u, &v);
    nd[u].adj.push_back(v);
    nd[v].adj.push_back(u);
  }

  for (int i = 1; i <= num_queries; i++) {
    scanf("%d %d", &u, &v);
    nd[u].num_src++;
    nd[v].num_dest++;
    nd[u].pairs.push_back(v);
    nd[v].pairs.push_back(u);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void endpoint_action(int u) {
  for (auto v: nd[u].pairs) {
    if (nd[v].depth) {
      int lca = ds.find(v);
      // Spune-i LCA-ului să închidă o pereche (sursă, destinație).
      nd[lca].num_src--;
```

```
      nd[lca].num_dest--;
    }
  }
}

void descendant_action(int u, int parent) {
  // Propagă numerele de capete la rădăcina 2ECC.
  nd[parent].num_src += nd[u].num_src;
  nd[parent].num_dest += nd[u].num_dest;
  nd[u].num_src = 0;
  nd[u].num_dest = 0;
}

void bridge_action(int u, int parent) {
  if (nd[u].low > nd[parent].depth) {
    if (nd[u].num_src && nd[u].num_dest) {
      red_flag = true;
    }
  }
}

void dfs(int u, int parent) {
  nd[u].depth = nd[u].low = nd[parent].depth + 1;

  endpoint_action(u);

  int num_parent_edges = 0;
  for (auto v: nd[u].adj) {
    num_parent_edges += (v == parent);

    if (!nd[v].depth) {
      dfs(v, u);
      ds.merge(u, v);
      nd[u].low = min(nd[u].low, nd[v].low);
      descendant_action(v, u);
    } else if ((v != parent) || ((v == parent) && (num_parent_edges > 1))) {
      nd[u].low = min(nd[u].low, nd[v].depth);
    }
  }

  bridge_action(u, parent);
}

void dfs_driver() {
  for (int u = 1; u <= n; u++) {
    if (!nd[u].depth) {
      dfs(u, 0);
      // Asigură-te că niciun mesaj nu circulă între două componente.
      if (nd[u].num_src || nd[u].num_dest) {
        red_flag = true;
```

```
      }
    }
  }
}

void write_answer() {
  printf(red_flag ? "No\n" : "Yes\n");
}

int main() {
  read_data();
  ds.init(n);
  dfs_driver();
  write_answer();

  return 0;
}

// If it's worth doing, it's worth doing well.
```

29.11 Problema Dog Trick Competition 2 (IIOT 2023-2024 runda 4)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <unordered_set>

const int MAX_NODES = 250'000;
const int MAX_EDGES = 250'000;
const int MAX_TRICKS = 250'000;
const int INFINITY = 2 * MAX_TRICKS;

struct edge_set {
  std::unordered_set<long long> set;

  void insert(int u, int v) {
    set.insert((long long)u * (MAX_NODES + 1) + v);
  }

  bool contains(int u, int v) {
    return set.contains((long long)u * (MAX_NODES + 1) + v);
  }
};

struct cell {
  int v, next;
```

```
};

struct node {
  int adj;       // lista de adiacență
  int first_pos; // prima apariție a acestui nod în t[]
  int comp;      // numărul CTC a acestui nod

  // câmpuri pentru algoritmul lui Tarjan pentru CTC
  int time_in;
  int low;
  bool on_stack;
};

// Un interval de indici din t[] pe care o CTC îi poate servi.
struct range {
  int l, r;

  void minimize(range other) {
    if (other.l < l) {
      *this = other;
    }
  }

  // this = intervalul subarborelui nodului curent
  // child = cel mai timpuriu interval al oricărei CTC descendente
  void combine(range child) {
    if (child.l < l) {
      *this = child;
    } else if (child.l == r + 1) {
      r = child.r;
    }
  }
};

node nd[MAX_NODES + 1];
cell list[MAX_EDGES + 1];
range scc[MAX_NODES + 1];
int st[MAX_NODES], ss; // stiva DFS
int t[MAX_TRICKS + 1]; // santinelă la t[num_tricks]
edge_set eset;
int num_comps;

void minimize(int& x, int y) {
  if (y < x) {
    x = y;
  }
}

void read_data() {
  int num_tricks, n, num_edges, u, v;
```

```
  scanf("%d %d", &num_tricks, &n);
  for (int u = 1; u <= n; u++) {
    nd[u].first_pos = INFINITY;
  }
  for (int i = 1; i <= num_tricks; i++) {
    scanf("%d", &t[i]);
    minimize(nd[t[i]].first_pos, i);
  }

  scanf("%d", &num_edges);
  for (int i = 1; i <= num_edges; i++) {
    scanf("%d %d", &u, &v);
    eset.insert(u, v);
    list[i] = { v, nd[u].adj };
    nd[u].adj = i;
  }
}

void process_scc(int last) {
  int v;
  num_comps++;
  range& c = scc[num_comps]; // syntactic sugar
  c.l = INFINITY;

  // Stiva conține nodurile din CTC. Ia prima apariție în t[].
  do {
    v = st[--ss];
    nd[v].comp = num_comps;
    nd[v].on_stack = false;
    minimize(c.l, nd[v].first_pos);
  } while (v != last);

  // Extinde intervalul la dreapta.
  c.r = c.l;
  while (nd[t[c.r + 1]].comp == num_comps) {
    c.r++;
  }
}

range dfs(int u) {
  static int time = 0;
  nd[u].time_in = nd[u].low = ++time;
  nd[u].on_stack = true;
  st[ss++] = u;

  range result = { INFINITY, INFINITY };

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
```

```
    if (!nd[v].time_in) {
      // Explorează un nod nou.
      result.minimize(dfs(v));
      minimize(nd[u].low, nd[v].low);
    } else if (nd[v].on_stack) {
      // Muchie înapoi.
      minimize(nd[u].low, nd[v].time_in);
    } else {
      // Muchie la o CTC explorată anterior. Acesta este singurul moment cînd
      // schimbăm rezultatul pentru CTC curentă. Operațiile return propagă
      // aceste valori pînă la rădăcina CTC.
      result.minimize(scc[nd[v].comp]);
    }
  }

  if (nd[u].low == nd[u].time_in) {
    // u este rădăcina unei CTC.
    process_scc(u);
    scc[num_comps].combine(result);
    return scc[num_comps];
  } else {
    return result;
  }
}

void compute_score() {
  // Nodul de start este t[1] și aparține lui scc[num_comps].
  int score = 2;
  for (int i = 2; i <= scc[num_comps].r; i++) {
    score += 1 + eset.contains(t[i - 1], t[i]);
  }
  printf("%d\n", score);
}

int main() {
  read_data();
  dfs(t[1]);
  compute_score();

  return 0;
}
```

29.12 Problema Regate (ONI 2022, clasele 11-12)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
```

```
const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;

struct edge {
  int v, cost, next;
};

struct bridge {
  int u, v, cost;
};

struct node {
  int adj;
  int comp; // componenta biconexă din care face parte
  int d;
  int low;
  long long cost; // inițial costul, apoi îl înlocuim cu răspunsul pentru nod
};

node nd[MAX_NODES + 1];
edge e[2 * MAX_EDGES + 1];
bridge b[MAX_NODES];
int st[MAX_NODES + 1], ss; // stiva pentru algoritmul lui Tarjan
int ord[MAX_NODES];
int n, comps;

// Invarianți:
// * size[u] este relevant doar pentru rădăcinile din DSF;
// * costul unei valori este suma din cost[] de la valoare la rădăcină.
struct disjoint_set_forest {
  long long cost[MAX_NODES];
  int p[MAX_NODES];
  int size[MAX_NODES];

  void init(int n) {
    for (int i = 0; i < n; i++) {
      p[i] = i;
    }
  }

  int find(int u) {
    // Compresia căii cu menținerea invariantului de cost.
    if (p[u] == u) {
      return u;
    } else {
      int r = find(p[u]);
      if (p[u] != r) {
        cost[u] += cost[p[u]];
        p[u] = r;
```

```
      }
      return r;
    }
  }

  int get_size(int u) {
    return size[find(u)];
  }

  long long get_cost(int u) {
    int r = find(u);
    // Acum fie r = u, fie r = p[u].
    return (r == u) ? cost[u] : (cost[u] + cost[r]);
  }

  inline void unite(int u, int v, long long bridge_cost) {
    u = find(u);
    v = find(v);
    cost[u] += bridge_cost * size[v];
    cost[v] += bridge_cost * size[u] - cost[u];
    p[v] = u;
    size[u] += size[v];
  }
};

disjoint_set_forest ds;

int min(int x, int y) {
  return (x < y) ? x : y;
}

// Scoate o componentă din stivă pînă la nodul root.
void pop_component(int root) {
  do {
    nd[st[--ss]].comp = comps;
    ds.size[comps]++;
  } while (st[ss] != root);
  comps++;
}

void find_bridges(int u, int depth) {
  nd[u].d = nd[u].low = depth;
  st[ss++] = u;

  for (int pos = nd[u].adj; pos; pos = e[pos].next) {
    int v = e[pos].v;
    if (!nd[v].d) {
      // Muchie înainte.
      find_bridges(v, depth + 1);
      nd[u].low = min(nd[u].low, nd[v].low);
```

```
      if (nd[v].low > nd[u].d) {
        // La v începe o nouă componentă. Notează componenta și puntea.
        b[comps] = { u, v, e[pos].cost };
        pop_component(v);
      }
    } else if (nd[v].d < nd[u].d - 1) {
      // Muchie înapoi.
      nd[u].low = min(nd[u].low, nd[v].d);
    }
  }
}

void add_edge(int u, int v, int cost) {
  static int ptr = 1;
  e[ptr] = { v, cost, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_data() {
  FILE* f = fopen("regate.in", "r");
  int m, u, v, c;

  fscanf(f, "%d %d", &n, &m);
  for (u = 1; u <= n; u++) {
    fscanf(f, "%lld", &nd[u].cost);
  }
  while (m--) {
    fscanf(f, "%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, c);
  }
  fclose(f);
}

void sort_bridges() {
  std::sort(b, b + comps - 1, [](bridge& a, bridge& b) {
    return a.cost > b.cost;
  });
}

void sort_nodes() {
  for (int i = 0; i < n; i++) {
    ord[i] = i + 1;
  }
  std::sort(ord, ord + n, [](int u, int v) {
    return nd[u].cost > nd[v].cost;
  });
}
```

```
void process_bridge(bridge b) {
  int c1 = nd[b.u].comp;
  int c2 = nd[b.v].comp;
  ds.unite(c1, c2, b.cost);
}

// Nodul u este mai ieftin decît toate punțile de pînă acum. Folosește acest
// cost pentru conectarea cu toți vecinii din componenta sa (unificată). La
// acest cost vom adăuga conectarea cu alte componente cu punți mai ieftine,
// dar acel cost deocamdată nu îl știm. De aceea, scădem costul prezent al
// componentei și îl vom readăuga la final. Prin diferență, obținem exact
// costul interconectării prin punți viitoare.
void process_node(int u) {
  int sz = ds.get_size(nd[u].comp);
  nd[u].cost = nd[u].cost * (sz - 1) - ds.get_cost(nd[u].comp);
}

void merge_bridges_and_nodes() {
  ds.init(comps);

  int i = 0, j = 0;
  while ((i < comps - 1) || (j < n)) {
    if ((j == n) || ((i < comps - 1) && (b[i].cost > nd[ord[j]].cost))) {
      process_bridge(b[i++]);
    } else {
      process_node(ord[j++]);
    }
  }
}

void write_data() {
  FILE* f = fopen("regate.out", "w");
  for (int u = 1; u <= n; u++) {
    fprintf(f, "%lld\n", ds.get_cost(nd[u].comp) + nd[u].cost);
  }
  fclose(f);
}

int main() {
  read_data();
  find_bridges(1, 1);
  pop_component(1); // DFS-ul lasă o componentă pe stivă.
  sort_bridges();
  sort_nodes();
  merge_bridges_and_nodes();
  write_data();

  return 0;
}
```

Anexa 30

Grafuri - Distanțe

30.1 Problema Bellman-Ford (Infoarena)

înapoi

Sursă cu algoritmul Bellman-Ford (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_EDGES = 250'000;
const int INFINITY = 100'000'000;

struct edge {
  unsigned short u, v;
  short cost;
};

edge e[MAX_EDGES];
int d[MAX_NODES + 1];
int n, m;
bool negative_cycle;

void read_data() {
  FILE* f = fopen("bellmanford.in", "r");
  fscanf(f, "%d %d", &n, &m);
  for (int i = 0; i < m; i++) {
    fscanf(f, "%hu %hu %hd", &e[i].u, &e[i].v, &e[i].cost);
  }
  fclose(f);
}

bool relax_edge(edge& e) {
  if (d[e.u] + e.cost < d[e.v]) {
    d[e.v] = d[e.u] + e.cost;
```

```
    return true;
  } else {
    return false;
  }
}

void bellman_ford() {
  for (int u = 1; u <= n; u++) {
    d[u] = INFINITY;
  }
  d[1] = 0;

  bool any_decreases = true;
  int stage = 0;
  while ((stage < n) && any_decreases) {
    any_decreases = false;
    for (int i = 0; i < m; i++) {
      any_decreases |= relax_edge(e[i]);
    }
    stage++;
  }

  negative_cycle = any_decreases;
}

void write_answer() {
  FILE* f = fopen("bellmanford.out", "w");
  if (negative_cycle) {
    fprintf(f, "Ciclu negativ!\n");
  } else {
    for (int u = 2; u <= n; u++) {
      fprintf(f, "%d ", d[u]);
    }
    fprintf(f, "\n");
  }
  fclose(f);
}

int main() {
  read_data();
  bellman_ford();
  write_answer();

  return 0;
}
```

Sursă cu algoritmul SPFA (versiune online).

```
#include <stdio.h>
```

```
const int MAX_NODES = 50'000;
const int MAX_EDGES = 250'000;
const int INFINITY = 100'000'000;

struct node {
  int adj;
  int d;
  unsigned short relax_count;
};

struct cell {
  unsigned short v;
  short cost;
  int next;
};

struct queue {
  unsigned short v[MAX_NODES + 1];
  bool in_queue[MAX_NODES + 1];
  int head, tail;

  bool is_empty() {
    return head == tail;
  }

  void push(int u) {
    if (!in_queue[u]) {
      in_queue[u] = true;
      v[tail++] = u;
      if (tail == MAX_NODES + 1) {
        tail = 0;
      }
    }
  }

  int pop() {
    int u = v[head++];
    in_queue[u] = false;
    if (head == MAX_NODES + 1) {
      head = 0;
    }
    return u;
  }
};

cell list[MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n;
```

```
bool negative_cycle;

void add_edge(unsigned short u, unsigned short v, short cost) {
  static int pos = 1;

  list[pos] = { v, cost, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int num_edges;
  FILE* f = fopen("bellmanford.in", "r");
  fscanf(f, "%d %d", &n, &num_edges);

  while (num_edges--) {
    int u, v, cost;
    fscanf(f, "%d %d %d", &u, &v, &cost);
    add_edge(u, v, cost);
  }
  fclose(f);
}

void relax_all_edges(int u) {
  for (int pos = nd[u].adj; pos; pos = list[pos].next) {
    int v = list[pos].v, cost = list[pos].cost;

    if (nd[u].d + cost < nd[v].d) {
      nd[v].d = nd[u].d + cost;
      q.push(v);
    }
  }

  nd[u].relax_count++;
  negative_cycle |= (nd[u].relax_count == n);
}

void bellman_ford() {
  for (int u = 1; u <= n; u++) {
    nd[u].d = INFINITY;
  }
  nd[1].d = 0;
  q.push(1);

  while (!q.is_empty() && !negative_cycle) {
    int u = q.pop();
    relax_all_edges(u);
  }
}

void write_answer() {
```

```
  FILE* f = fopen("bellmanford.out", "w");
  if (negative_cycle) {
    fprintf(f, "Ciclu negativ!\n");
  } else {
    for (int u = 2; u <= n; u++) {
      fprintf(f, "%d ", nd[u].d);
    }
    fprintf(f, "\n");
  }
  fclose(f);
}

int main() {
  read_data();
  bellman_ford();
  write_answer();

  return 0;
}
```

30.2 Problema Dijkstra (Infoarena)

‹ înapoi

Sursă cu heap propriu și `decrease-key` (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_EDGES = 250'000;
const int INFINITY = 1'000'000'000;

typedef unsigned short u16;

struct adj_list {
  unsigned short v, dist;
  int next;
};

struct node {
  int adj;
  int dist;
};

adj_list list[MAX_EDGES + 1];
node nd[MAX_NODES + 1];
int n;
```

```
struct heap {
  u16 v[MAX_NODES + 1];
  u16 pos[MAX_NODES + 1]; // pos[u] = 0  <==>  nodul u lipseşte din heap
  int size;

  bool is_empty() {
    return size == 0;
  }

  void decrease_key(int u) {
    if (!pos[u]) {
      v[++size] = u;
      pos[u] = size;
    }

    // Deplasează gaura în sus cît timp se poate.
    int x = pos[u];
    while ((x > 1) && (nd[v[x / 2]].dist > nd[u].dist)) {
      v[x] = v[x / 2];
      pos[v[x]] = x;
      x /= 2;
    }

    // Umple gaura cu nodul u.
    v[x] = u;
    pos[u] = x;
  }

  u16 remove_min() {
    int result = v[1];
    pos[result] = 0;

    int u = v[size--];

    if (size) {
      // Deplasează gaura în jos cît timp se poate.
      bool swapped = true;
      int x = 1;
      while ((2 * x <= size) && swapped) {
        int l = 2 * x, r = 2 * x + 1;
        int y = (r <= size) && (nd[v[r]].dist < nd[v[l]].dist)
          ? r : l;

        if (nd[v[y]].dist < nd[u].dist) {
          v[x] = v[y];
          pos[v[x]] = x;
          x = y;
        } else {
          swapped = false;
        }
```

```
      }

      // Umple gaura cu nodul u.
      v[x] = u;
      pos[u] = x;
    }

    return result;
  }

};

heap h;

void add_edge(u16 u, u16 v, u16 dist) {
  static int ptr = 0;

  list[++ptr] = { v, dist, nd[u].adj };
  nd[u].adj = ptr;
}

void read_graph() {
  int num_edges;

  FILE* f = fopen("dijkstra.in", "r");
  fscanf(f, "%d %d", &n, &num_edges);

  while (num_edges--) {
    int u, v, dist;
    fscanf(f, "%d %d %d\n", &u, &v, &dist);
    add_edge(u, v, dist);
  }
  fclose(f);
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }

  nd[1].dist = 0;
  h.decrease_key(1);

  while (!h.is_empty()) {
    int u = h.remove_min();
    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      int edge_dist = list[ptr].dist;
      if (nd[u].dist + edge_dist < nd[v].dist) {
        nd[v].dist = nd[u].dist + edge_dist;
```

```
        h.decrease_key(v);
      }
    }
  }
}

void write_distances() {
  FILE* f = fopen("dijkstra.out", "w");
  for (int u = 2; u <= n; u++) {
    if (nd[u].dist == INFINITY) {
      nd[u].dist = 0;
    }
    fprintf(f, "%d ", nd[u].dist);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_graph();
  dijkstra();
  write_distances();

  return 0;
}
```

Sursă cu heap propriu, fără `decrease-key` (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 50'000;
const int MAX_EDGES = 250'000;
const int INFINITY = 1'000'000'000;

typedef unsigned short u16;

struct adj_list {
  unsigned short v, dist;
  int next;
};

struct node {
  int adj;
  int dist;
};

struct heap_elt {
  u16 u;
  int dist;
```

```
};

struct heap {
  heap_elt v[MAX_EDGES + 1];
  int size;

  bool is_empty() {
    return size == 0;
  }

  void insert(u16 u, int dist) {
    // Deplasează gaura în sus cît timp se poate.
    int pos = ++size;
    while ((pos > 1) && (v[pos / 2].dist > dist)) {
      v[pos] = v[pos / 2];
      pos /= 2;
    }

    // Umple gaura cu nodul u.
    v[pos] = { u, dist };
  }

  heap_elt remove_min() {
    heap_elt result = v[1];
    heap_elt save = v[size--];

    if (size) {
      // Deplasează gaura în jos cît timp se poate.
      bool swapped = true;
      int x = 1;
      while ((2 * x <= size) && swapped) {
        int l = 2 * x, r = 2 * x + 1;
        int y = (r <= size) && (v[r].dist < v[l].dist)
          ? r : l;

        if (v[y].dist < save.dist) {
          v[x] = v[y];
          x = y;
        } else {
          swapped = false;
        }
      }

      // Umple gaura cu elementul save.
      v[x] = save;
    }

    return result;
  }
};
```

```
adj_list list[MAX_EDGES + 1];
node nd[MAX_NODES + 1];
heap h;
int n;

void add_edge(u16 u, u16 v, u16 dist) {
  static int ptr = 0;

  list[++ptr] = { v, dist, nd[u].adj };
  nd[u].adj = ptr;
}

void read_graph() {
  int num_edges;

  FILE* f = fopen("dijkstra.in", "r");
  fscanf(f, "%d %d", &n, &num_edges);

  while (num_edges--) {
    int u, v, dist;
    fscanf(f, "%d %d %d\n", &u, &v, &dist);
    add_edge(u, v, dist);
  }
  fclose(f);
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }

  nd[1].dist = 0;
  h.insert(1, 0);

  while (!h.is_empty()) {
    heap_elt top = h.remove_min();
    int u = top.u;

    if (top.dist == nd[u].dist) {
      for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
        int v = list[ptr].v;
        int edge_dist = list[ptr].dist;
        if (nd[u].dist + edge_dist < nd[v].dist) {
          nd[v].dist = nd[u].dist + edge_dist;
          h.insert(v, nd[v].dist);
        }
      }
    }
  }
```

```
}

void write_distances() {
  FILE* f = fopen("dijkstra.out", "w");
  for (int u = 2; u <= n; u++) {
    if (nd[u].dist == INFINITY) {
      nd[u].dist = 0;
    }
    fprintf(f, "%d ", nd[u].dist);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_graph();
  dijkstra();
  write_distances();

  return 0;
}
```

Sursă cu `priority_queue` din STL (versiune online).

```
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 50'000;
const int MAX_EDGES = 250'000;
const int INFINITY = 1'000'000'000;

typedef unsigned short u16;

struct edge {
  u16 v, dist;
};

struct node {
  std::vector<edge> adj;
  int dist;
};

struct pq_elt {
  u16 u;
  int dist;

  friend bool operator< (const pq_elt& a, const pq_elt& b) {
    return a.dist > b.dist;
```

```
  }
};

node nd[MAX_NODES + 1];
std::priority_queue<pq_elt> pq;
int n;

void read_graph() {
  int num_edges;

  FILE* f = fopen("dijkstra.in", "r");
  fscanf(f, "%d %d", &n, &num_edges);

  while (num_edges--) {
    u16 u, v, dist;
    fscanf(f, "%hd %hd %hd\n", &u, &v, &dist);
    nd[u].adj.push_back({ v, dist});
  }
  fclose(f);
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }

  nd[1].dist = 0;
  pq.push({1, 0});

  while (!pq.empty()) {
    pq_elt top = pq.top();
    pq.pop();
    int u = top.u;

    if (top.dist == nd[u].dist) {
      for (auto e: nd[u].adj) {
        u16 v = e.v;
        if (nd[u].dist + e.dist < nd[v].dist) {
          nd[v].dist = nd[u].dist + e.dist;
          pq.push({v, nd[v].dist});
        }
      }
    }
  }
}

void write_distances() {
  FILE* f = fopen("dijkstra.out", "w");
  for (int u = 2; u <= n; u++) {
    if (nd[u].dist == INFINITY) {
```

```
      nd[u].dist = 0;
    }
    fprintf(f, "%d ", nd[u].dist);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_graph();
  dijkstra();
  write_distances();

  return 0;
}
```

30.3 Problema Monsters (CSES)

înapoi

```
#include <stdio.h>
#include <queue>

const int MAX_SIZE = 1'000;
const int INFINITY = MAX_SIZE * MAX_SIZE + 1;

struct point {
  short r, c;
};

struct cell {
  int monster_dist, hero_dist;
  char dir;
  bool wall, edge;
};

std::queue<point> q;
cell a[MAX_SIZE + 2][MAX_SIZE + 2];
int rows, cols;
point hero;

void init_distances() {
  for (short r = 1; r <= rows; r++) {
    for (short c = 1; c <= cols; c++) {
      a[r][c].monster_dist = INFINITY;
      a[r][c].hero_dist = INFINITY;
    }
  }
```

```
}

void read_data() {
  scanf("%d %d ", &rows, &cols);

  init_distances();

  for (short r = 1; r <= rows; r++) {
    for (short c = 1; c <= cols; c++) {
      switch (getchar()) {
        case '#':
          a[r][c].wall = true;
          break;
        case 'A':
          hero = { r, c };
          break;
        case 'M':
          q.push({ r, c });
          a[r][c].monster_dist = 0;
          break;
      }
    }
    getchar();
  }
}

void pad_matrix() {
  for (int r = 0; r <= rows + 1; r++) {
    a[r][0].wall = a[r][cols + 1].wall = true;
  }
  for (int c = 0; c <= cols + 1; c++) {
    a[0][c].wall = a[rows + 1][c].wall = true;
  }
}

void mark_edges() {
  for (int r = 1; r <= rows; r++) {
    a[r][1].edge = a[r][cols].edge = true;
  }
  for (int c = 1; c <= cols; c++) {
    a[1][c].edge = a[rows][c].edge = true;
  }
}

void monster_move(short r, short c, int dist) {
  if (!a[r][c].wall && a[r][c].monster_dist == INFINITY) {
    a[r][c].monster_dist = dist;
    q.push({ r, c });
  }
}
```

```
void monster_bfs() {
  while (!q.empty()) {
    point p = q.front();
    q.pop();
    int d = a[p.r][p.c].monster_dist;

    monster_move(p.r, p.c - 1, d + 1);
    monster_move(p.r, p.c + 1, d + 1);
    monster_move(p.r - 1, p.c, d + 1);
    monster_move(p.r + 1, p.c, d + 1);
  }
}

void hero_move(short r, short c, int dist, char dir) {
  if (!a[r][c].wall &&
      (a[r][c].hero_dist == INFINITY) &&
      (dist < a[r][c].monster_dist)) {
    a[r][c].hero_dist = dist;
    a[r][c].dir = dir;
    q.push({ r, c });
  }
}

// Returnează ultimul punct explorat.
point hero_bfs() {
  a[hero.r][hero.c].hero_dist = 0;
  q.push(hero);
  point p = { 0, 0 };

  while (!q.empty() && !a[p.r][p.c].edge) {
    p = q.front();
    q.pop();
    int d = a[p.r][p.c].hero_dist;

    hero_move(p.r, p.c - 1, d + 1, 'L');
    hero_move(p.r, p.c + 1, d + 1, 'R');
    hero_move(p.r - 1, p.c, d + 1, 'U');
    hero_move(p.r + 1, p.c, d + 1, 'D');
  }

  return p;
}

void trace(int r, int c) {
  if (a[r][c].hero_dist) {
    switch (a[r][c].dir) {
      case 'L': trace(r, c + 1); break;
      case 'R': trace(r, c - 1); break;
      case 'U': trace(r + 1, c); break;
```

```
      case 'D': trace(r - 1, c); break;
    }
    putchar(a[r][c].dir);
  }
}

void solution(point dest) {
  if (a[dest.r][dest.c].edge) {
    printf("YES\n%d\n", a[dest.r][dest.c].hero_dist);
    trace(dest.r, dest.c);
    putchar('\n');
  } else {
    printf("NO\n");
  }
}

int main() {
  read_data();
  pad_matrix();
  mark_edges();
  monster_bfs();
  point dest = hero_bfs();
  solution(dest);

  return 0;
}
```

30.4 Problema Shortest Routes II (CSES)

‹ înapoi

```
#include <stdio.h>

const int MAX_NODES = 500;
const long long INFINITY = 1'000'000'000'000LL;

long long d[MAX_NODES + 1][MAX_NODES + 1];
int n, num_queries;

void init_distances() {
  for (int u = 1; u <= n; u++) {
    for (int v = 1; v <= n; v++) {
      d[u][v] = (u == v) ? 0 : INFINITY;
    }
  }
}

void minimize(long long& x, long long y) {
```

```
  if (y < x) {
    x = y;
  }
}

void read_data() {
  int m, u, v, cost;
  scanf("%d %d %d", &n, &m, &num_queries);

  init_distances();

  while (m--) {
    scanf("%d %d %d", &u, &v, &cost);
    // Atenție la muchiile multiple între aceleași noduri.
    minimize(d[u][v], cost);
    minimize(d[v][u], cost);
  }
}

void floyd_warshall() {
  for (int k = 1; k <= n; k++) {
    for (int u = 1; u <= n; u++) {
      for (int v = 1; v <= n; v++) {
        minimize(d[u][v], d[u][k] + d[k][v]);
      }
    }
  }
}

void process_queries() {
  while (num_queries--) {
    int u, v;
    scanf("%d %d", &u, &v);
    printf("%lld\n", (d[u][v] == INFINITY) ? -1 : d[u][v]);
  }
}

int main() {
  read_data();
  floyd_warshall();
  process_queries();

  return 0;
}
```

30.5 Problema High Score (CSES)

înapoi

```cpp
#include <stdio.h>
#include <vector>

const int MAX_NODES = 50'000;
const long long INFINITY = 5'000'000'000'000LL;

struct edge {
  int v;
  int cost;
};

struct node {
  std::vector<edge> adj;
  long long dist;
  bool infinite;
  bool can_reach_dest;
};

node nd[MAX_NODES];
int n;

void read_data() {
  int m, u, v, cost;
  scanf("%d %d", &n, &m);
  while (m--) {
    scanf("%d %d %dd", &u, &v, &cost);
    nd[u].adj.push_back({v, -cost});
  }
  nd[n].can_reach_dest = true;
}

bool relax_edge(int u, int v, int cost) {
  nd[u].can_reach_dest |= nd[v].can_reach_dest;
  if ((nd[u].dist != INFINITY) && (nd[u].dist + cost < nd[v].dist)) {
    nd[v].dist = nd[u].dist + cost;
    return true;
  } else {
    return false;
  }
}

void bellman_ford() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }
  nd[1].dist = 0;

  for (int stage = 1; stage <= n; stage++) {
    for (int u = 1; u <= n; u++) {
```

```
      for (edge e: nd[u].adj) {
        if (relax_edge(u, e.v, e.cost) && (stage == n)) {
          nd[e.v].infinite = true;
        }
      }
    }
  }
}

bool reachable_infinite() {
  for (int u = 1; u <= n; u++) {
    if (nd[u].infinite && nd[u].can_reach_dest) {
      return true;
    }
  }
  return false;
}

int main() {
  read_data();
  bellman_ford();
  int inf = reachable_infinite();
  printf("%lld\n", inf ? -1 : -nd[n].dist);

  return 0;
}
```

30.6 Problema Flight Discount (CSES)

‹ înapoi

```
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 200'000;
const long long INFINITY = 1e14;

struct edge {
  int v, dist;
};

struct node {
  std::vector<edge> adj;
  long long dist[2]; // costul minim pînă aici cu/fără reducere
};
```

```
struct pq_elt {
  long long dist;
  int u;
  bool used;

  friend bool operator< (const pq_elt a, const pq_elt b) {
    return a.dist > b.dist;
  }
};

node nd[MAX_NODES + 1];
std::priority_queue<pq_elt> pq;
int n;

void read_data() {
  int m, u, v, dist;

  scanf("%d %d", &n, &m);

  while (m--) {
    scanf("%d %d %d\n", &u, &v, &dist);
    nd[u].adj.push_back({ v, dist });
  }
}

void improve(int u, bool state_u, int v, bool state_v, int cost) {
  long long d = nd[u].dist[state_u] + cost;
  if (d < nd[v].dist[state_v]) {
    nd[v].dist[state_v] = d;
    pq.push({d, v, state_v});
  }
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist[0] = nd[u].dist[1] = INFINITY;
  }

  nd[1].dist[0] = 0;
  pq.push({0, 1, false});

  while (!pq.empty()) {
    pq_elt t = pq.top();
    pq.pop();

    // Dat fiind că inserăm nodurile de multiple ori, trebuie să verificăm
    // dacă distanța este cea curentă.
    if (t.dist == nd[t.u].dist[t.used]) {
      for (auto e: nd[t.u].adj) {
        if (!t.used) {
```

```
          improve(t.u, t.used, e.v, true, e.dist / 2);
        }

        improve(t.u, t.used, e.v, t.used, e.dist);
      }
    }
  }
}

int main() {
  read_data();
  dijkstra();
  printf("%lld\n", nd[n].dist[true]);

  return 0;
}
```

30.7 Problema Three States (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000;
const int MAX_DEQUE = 1 << 17;
const int NUM_STATES = 3;
const char OPEN = '.';
const char WALL = '#';
const int INFINITY = 100'000'000;

typedef int dist_matrix[MAX_N + 2][MAX_N + 2];

struct point {
  short r, c;
};

const int NUM_DIR = 4;
const point DELTA[NUM_DIR] = { { -1, 0 }, { 0, 1 }, { 1, 0 }, { 0, -1 } };

struct deque {
  point buf[MAX_DEQUE];
  int left, right, size;

  void init() {
    size = 0;
  }

  bool is_empty() {
```

```
    return !size;
  }

  void push_left(point p) {
    if (size) {
      left = (left + MAX_DEQUE - 1) % MAX_DEQUE;
    } else {
      left = right = 0;
    }
    buf[left] = p;
    size++;
  }

  void push_right(point p) {
    if (size) {
      right = (right + 1) % MAX_DEQUE;
    } else {
      left = right = 0;
    }
    buf[right] = p;
    size++;
  }

  point pop_left() {
    point result = buf[left];
    size--;
    left = (left + 1) % MAX_DEQUE;
    return result;
  }
};

char map[MAX_N + 2][MAX_N + 2];
dist_matrix dist[NUM_STATES];
deque q;
int rows, cols;

void read_data() {
  scanf("%d %d ", &rows, &cols);
  for (int r = 1; r <= rows; r++) {
    scanf("%s ", &map[r][1]);
  }
}

void pad_map() {
  for (int r = 1; r <= rows; r++) {
    map[r][0] = map[r][cols + 1] = WALL;
  }
  for (int c = 1; c <= cols; c++) {
    map[0][c] = map[rows + 1][c] = WALL;
  }
```

```
}

void init_dist(dist_matrix& dist) {
  for (int r = 1; r <= rows; r++) {
    for (int c = 1; c <= cols; c++) {
      dist[r][c] = INFINITY;
    }
  }
}

void bfs(int state, short start_row, short start_col) {
  init_dist(dist[state]);
  dist[state][start_row][start_col] = 0;

  q.init();
  q.push_left({start_row, start_col});

  while (!q.is_empty()) {
    point p = q.pop_left();

    for (int dir = 0; dir < NUM_DIR; dir++) {
      short r = p.r + DELTA[dir].r;
      short c = p.c + DELTA[dir].c;

      if (map[r][c] != WALL) {
        int cost = (map[r][c] == OPEN) ? 1 : 0;
        int my_dist = dist[state][p.r][p.c];
        if (my_dist + cost < dist[state][r][c]) {
          dist[state][r][c] = my_dist + cost;
          if (cost) {
            q.push_right({r, c});
          } else {
            q.push_left({r, c});
          }
        }
      }
    }
  }
}

void bfs_driver() {
  point start[NUM_STATES];
  for (short r = 1; r <= rows; r++) {
    for (short c = 1; c <= cols; c++) {
      if ((map[r][c] != WALL) && (map[r][c] != OPEN)) {
        start[map[r][c] - '1'] = { r, c };
      }
    }
  }
```

```
  for (int s = 0; s < NUM_STATES; s++) {
    bfs(s, start[s].r, start[s].c);
  }
}

int min(int x, int y) {
  return (x < y) ?  x : y;
}

int choose_center() {
  int min_sum = INFINITY;

  for (int r = 1; r <= rows; r++) {
    for (int c = 1; c <= cols; c++) {
      int sum = (map[r][c] == '.') ? (1 - NUM_STATES) : 0;
      for (int s = 0; s < NUM_STATES; s++) {
        sum += dist[s][r][c];
      }
      min_sum = min(min_sum, sum);
    }
  }

  return min_sum;
}

void write_answer(int sum_dist) {
  int answer = (sum_dist == INFINITY) ? -1 : sum_dist;
  printf("%d\n", answer);
}

int main() {
  read_data();
  pad_map();
  bfs_driver();
  int sum_dist = choose_center();
  write_answer(sum_dist);

  return 0;
}
```

30.8 Problema Graph and Graph (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <stdlib.h>
#include <queue>
#include <vector>
```

```
const int MAX_NODES = 1'000;
const int INFINITY = 1'000'000'001;
const int NONE = -1;

struct graph {
  std::vector<short> adj[MAX_NODES + 1];

  void clear(int n) {
    for (int u = 1; u <= n; u++) {
      adj[u].resize(0);
    }
  }

  void read_edges() {
    int m, u, v;
    scanf("%d", &m);
    while (m--) {
      scanf("%d %d", &u, &v);
      adj[u].push_back(v);
      adj[v].push_back(u);
    }
  }
};

struct pq_elt {
  short u1, u2;
  int dist;

  friend bool operator<(const pq_elt& a, const pq_elt& b) {
    return a.dist > b.dist;
  }
};

graph g1, g2;
int d[MAX_NODES + 1][MAX_NODES + 1];
int n;
short start1, start2;

void read_data() {
  scanf("%d %hd %hd", &n, &start1, &start2);
  g1.clear(n);
  g2.clear(n);
  g1.read_edges();
  g2.read_edges();
}

int dijkstra() {
  for (int u = 1; u <= n; u++) {
    for (int v = 1; v <= n; v++) {
```

```
      d[u][v] = INFINITY;
    }
  }
  int answer = NONE;

  std::priority_queue<pq_elt> pq;
  d[start1][start2] = 0;
  pq.push({start1, start2, 0});

  while (!pq.empty() && (answer == NONE)) {
    pq_elt e = pq.top();
    pq.pop();
    if (e.dist == d[e.u1][e.u2]) {
      for (short v1: g1.adj[e.u1]) {
        for (short v2: g2.adj[e.u2]) {
          if ((e.u1 == e.u2) && (v1 == v2)) {
            answer = e.dist;
          }
          if (e.dist + abs(v1 - v2) < d[v1][v2]) {
            d[v1][v2] = e.dist + abs(v1 - v2);
            pq.push({v1, v2, d[v1][v2]});
          }
        }
      }
    }
  }

  return answer;
}

void solve_test() {
  read_data();
  int answer = dijkstra();
  printf("%d\n", answer);
}

int main() {
  int num_tests;

  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

30.9 Problema Termite (Lot 2014)

înapoi

Sursă cu *small-to-large* (versiune online).

```cpp
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 300'000;
const int MAX_EDGES = 400'000;
const int MAX_QUERIES = 400'000;
const int INFINITY = 300'000'000; // costul maxim este 1'000

struct query {
  int a, b;
  int t;
  int disconnect_time;

  int get_delta_t() {
    return std::max(disconnect_time - t, 0);
  }
};

struct qlist {
  int id, next;
};

struct edge {
  int v, dist;

  friend bool operator< (const edge& a, const edge& b) {
    return a.dist > b.dist;
  }
};

struct elist {
  int v, cost, next;
};

struct node {
  int adj;
  int dist;
};

node nd[MAX_NODES + 1];
elist e[2 * MAX_EDGES + 1];
query q[MAX_QUERIES];
int ord[MAX_NODES]; // nodurile în ordinea dispariției
```

```
std::priority_queue<edge> pq;
int n, num_queries;

struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  qlist list[2 * MAX_QUERIES + 1]; // spațiu pentru celulele din liste
  int queries[MAX_NODES + 1]; // lista de interogări a fiecărui nod
  int qsize[MAX_NODES + 1];   // mărimea listei de interogări a fiecărui nod

  void add_query(int id, int u) {
    static int ptr = 1;
    list[ptr] = { id, queries[u] };
    queries[u] = ptr++;
    qsize[u]++;
  }

  void init(int n) {
    for (int u = 1; u <= n; u++) {
      p[u] = u;
    }

    for (int i = 0; i < num_queries; i++) {
      add_query(i, q[i].a);
      add_query(i, q[i].b);
    }
  }

  int find(int u) {
    return (p[u] == u) ? u : (p[u] = find(p[u]));
  }

  void move_queries(int u, int v, int time) {
    int ptr = queries[u];
    while (ptr) {
      int next = list[ptr].next; // Îl salvăm căci poate fi distrus mai jos.
      int id = list[ptr].id;
      if (q[id].disconnect_time == INFINITY) {
        // Calculează componenta celuilalt capăt al interogării.
        int compa = find(q[id].a), compb = find(q[id].b);
        int w = (u == compa) ? compb : compa;
        if (w == v) {
          // Acesta este momentul la care q[id] se (de)conectează.
          q[id].disconnect_time = time;
        } else {
          // Mută celula la lista lui v.
          list[ptr].next = queries[v];
          queries[v] = ptr;
          qsize[v]++;
        }
      }
```

```
      ptr = next;
    }
  }

  inline void unite(int u, int v, int time) {
    u = find(u);
    v = find(v);

    if (u != v) {
      if (qsize[u] > qsize[v]) {
        std::swap(u, v);
      }
      move_queries(u, v, time);
      p[u] = v;
    }
  }
};

disjoint_set_forest dsf;

void add_edge(int u, int v, int cost) {
  static int ptr = 1;
  e[ptr] = { v, cost, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_data_and_init_dist() {
  FILE* f = fopen("termite.in", "r");
  int m, k, u, v, c;

  fscanf(f, "%d %d %d %d", &n, &m, &k, &num_queries);

  for (u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }

  while (k--) {
    fscanf(f, "%d", &u);
    nd[u].dist = 0;
    pq.push({ u, 0 });
  }

  while (m--) {
    fscanf(f, "%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, c);
  }

  for (int i = 0; i < num_queries; i++) {
    fscanf(f, "%d %d %d", &q[i].a, &q[i].b, &q[i].t);
```

```
    q[i].disconnect_time = INFINITY;
  }

  fclose(f);
}

void dijkstra() {
  int num_seen = 0;

  while (!pq.empty()) {
    auto [u, dist] = pq.top();
    pq.pop();

    if (dist == nd[u].dist) {
      ord[num_seen++] = u;
      for (int ptr = nd[u].adj; ptr; ptr = e[ptr].next) {
        int v = e[ptr].v, cost = e[ptr].cost;
        if (nd[u].dist + cost < nd[v].dist) {
          nd[v].dist = nd[u].dist + cost;
          pq.push({v, nd[v].dist});
        }
      }
    }
  }
}

void reverse_connectivity() {
  // Caz particular: interogări cu a == b.
  for (int i = 0; i < num_queries; i++) {
    if (q[i].a == q[i].b) {
      q[i].disconnect_time = nd[q[i].a].dist;
    }
  }

  dsf.init(n);

  for (int i = n - 1; i >= 0; i--) {
    int u = ord[i];

    for (int ptr = nd[u].adj; ptr; ptr = e[ptr].next) {
      int v = e[ptr].v;
      if (nd[v].dist >= nd[u].dist) {
        dsf.unite(u, v, nd[u].dist);
      }
    }
  }
}

void write_answers() {
  FILE* f = fopen("termite.out", "w");
```

```
  for (int i = 0; i < num_queries; i++) {
    fprintf(f, "%d\n", q[i].get_delta_t());
  }
  fclose(f);
}

int main() {
  read_data_and_init_dist();
  dijkstra();
  reverse_connectivity();
  write_answers();

  return 0;
}
```

Sursă cu mulțimi disjuncte modificate (versiune online).

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 300'000;
const int MAX_EDGES = 400'000;
const int INFINITY = 300'000'000; // costul maxim este 1'000

struct edge {
  int v, dist;

  friend bool operator< (const edge& a, const edge& b) {
    return a.dist > b.dist;
  }
};

struct node {
  int adj;
  int dist;
};

struct cell {
  int v, cost, next;
};

struct disjoint_set_forest {
  int p[MAX_NODES + 1];
  int size[MAX_NODES + 1];
  // Momentul minim la care putem părăsi componenta prin părinte.
  int t[MAX_NODES + 1];

  void init(int n) {
    for (int u = 1; u <= n; u++) {
```

```
      p[u] = u;
      size[u] = 1;
    }
  }

  int find(int u) {
    while (p[u] != u) {
      u = p[u];
    }
    return u;
  }

  void unite(int u, int v, int time) {
    u = find(u);
    v = find(v);
    if (u != v) {
      if (size[u] > size[v]) {
        std::swap(u, v);
      }

      p[u] = v;
      size[v] += size[u];
      t[u] = time;
    }
  }

  int get_min_edge(int u, int v) {
    int result = INFINITY;
    while (u != v) {
      if (size[u] < size[v]) {
        result = std::min(result, t[u]);
        u = p[u];
      } else {
        result = std::min(result, t[v]);
        v = p[v];
      }
    }
    return result;
  }
};

node nd[MAX_NODES + 1];
cell list[2 * MAX_EDGES + 1];
int ord[MAX_NODES]; // nodurile în ordinea dispariției
std::priority_queue<edge> pq;
disjoint_set_forest dsf;
int n, num_queries;
FILE *fin, *fout;

void add_edge(int u, int v, int cost) {
```

```
  static int ptr = 1;
  list[ptr] = { v, cost, nd[u].adj };
  nd[u].adj = ptr++;
}

void read_tree_and_init_dist() {
  int m, k, u, v, c;

  fscanf(fin, "%d %d %d %d", &n, &m, &k, &num_queries);

  for (u = 1; u <= n; u++) {
    nd[u].dist = INFINITY;
  }

  while (k--) {
    fscanf(fin, "%d", &u);
    nd[u].dist = 0;
    pq.push({ u, 0 });
  }

  while (m--) {
    fscanf(fin, "%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, c);
  }
}

void dijkstra() {
  int num_seen = 0;

  while (!pq.empty()) {
    auto [u, dist] = pq.top();
    pq.pop();

    if (dist == nd[u].dist) {
      ord[num_seen++] = u;
      for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
        int v = list[ptr].v;
        int cost = list[ptr].cost;
        if (nd[u].dist + cost < nd[v].dist) {
          nd[v].dist = nd[u].dist + cost;
          pq.push({v, nd[v].dist});
        }
      }
    }
  }
}

void reverse_connectivity() {
  dsf.init(n);
```

```
  for (int i = n - 1; i >= 0; i--) {
    int u = ord[i];

    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      if (nd[v].dist >= nd[u].dist) {
        dsf.unite(u, v, nd[u].dist);
      }
    }
  }
}

void process_queries() {
  int u, v, time;

  while (num_queries--) {
    fscanf(fin, "%d %d %d", &u, &v, &time);

    int answer = (u == v)
      ? nd[u].dist   // O, nu! Un caz particular!
      : dsf.get_min_edge(u, v);

    fprintf(fout, "%d\n", std::max(answer - time, 0));
  }
}

int main() {
  fin = fopen("termite.in", "r");
  fout = fopen("termite.out", "w");

  read_tree_and_init_dist();
  dijkstra();
  reverse_connectivity();
  process_queries();

  fclose(fin);
  fclose(fout);

  return 0;
}
```

30.10 Problema President and Roads (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>
```

```
const int MAX_NODES = 100'000;
const int MAX_EDGES = 100'000;
const long long INFINITY = 1LL << 62;
const int MOD = 1'000'003'751;

const int FORWARD = 0;
const int REVERSE = 1;

struct cell {
  int v;
  int cost;
  int next;
};

struct node {
  int adj[2];
  long long dist[2];

  // Numărul de moduri de a ajunge la acest nod mergînd înainte/înapoi.
  // Matematic, reprezentarea este incorectă. Numărul de moduri poate ușor să
  // depășească 64 de biți. De exemplu, formăm triunghiuri cu fiecare trei
  // noduri și le înlănțuim. Atunci numărul de moduri de a ajunge la
  // destinație este 2^(n/3). Operăm modulo un număr prim mare și ne rugăm la
  // Zeus.
  int ways[2];
};

struct heap_elt {
  int u;
  long long dist;
};

struct heap {
  heap_elt v[MAX_EDGES + 1];
  int size;

  bool is_empty() {
    return size == 0;
  }

  void insert(int u, long long dist) {
    int pos = ++size;
    while ((pos > 1) && (v[pos / 2].dist > dist)) {
      v[pos] = v[pos / 2];
      pos /= 2;
    }

    v[pos] = { u, dist };
  }
```

```
  heap_elt remove_min() {
    heap_elt result = v[1];
    heap_elt save = v[size--];

    if (size) {
      bool swapped = true;
      int x = 1;
      while ((2 * x <= size) && swapped) {
        int l = 2 * x, r = 2 * x + 1;
        int y = (r <= size) && (v[r].dist < v[l].dist)
          ? r : l;

        if (v[y].dist < save.dist) {
          v[x] = v[y];
          x = y;
        } else {
          swapped = false;
        }
      }

      v[x] = save;
    }

    return result;
  }
};

cell list[2 * MAX_EDGES + 1];
node nd[MAX_NODES + 1];
heap h;
int n, num_edges, source, sink;

void add_edge(int u, int v, int dist) {
  static int ptr = 0;

  list[++ptr] = { v, dist, nd[u].adj[FORWARD] };
  nd[u].adj[FORWARD] = ptr;

  list[++ptr] = { u, dist, nd[v].adj[REVERSE] };
  nd[v].adj[REVERSE] = ptr;

  // Notă: a k-a muchie ocupă pozițiile 2k-1 și 2k în list[].
}

void read_graph() {
  scanf("%d %d %d %d", &n, &num_edges, &source, &sink);

  for (int i = 0; i < num_edges; i++) {
    int u, v, dist;
    scanf("%d %d %d", &u, &v, &dist);
```

```
    add_edge(u, v, dist);
  }
}

void dijkstra(int source, int dir) {
  for (int u = 1; u <= n; u++) {
    nd[u].dist[dir] = INFINITY;
    nd[u].ways[dir] = 0;
  }

  nd[source].dist[dir] = 0;
  nd[source].ways[dir] = 1;
  h.insert(source, 0);

  while (!h.is_empty()) {
    heap_elt top = h.remove_min();
    int u = top.u;

    // Nu face efortul dacă distanța lui u s-a modificat între timp.
    if (top.dist == nd[u].dist[dir]) {
      for (int ptr = nd[u].adj[dir]; ptr; ptr = list[ptr].next) {
        int v = list[ptr].v;
        long long old_dist = nd[v].dist[dir];
        long long new_dist = top.dist + list[ptr].cost;

        if (new_dist < old_dist) {
          nd[v].dist[dir] = new_dist;
          nd[v].ways[dir] = nd[u].ways[dir];
          h.insert(v, new_dist);
        } else if (new_dist == old_dist) {
          nd[v].ways[dir] = (nd[v].ways[dir] + nd[u].ways[dir]) % MOD;
        }
      }
    }
  }
}

void compute_answers() {
  long long min_dist = nd[sink].dist[FORWARD];
  int min_ways = nd[sink].ways[FORWARD];

  for (int i = 1; i <= num_edges; i++) {
    int u = list[2 * i].v;
    int v = list[2 * i - 1].v;
    int cost = list[2 * i].cost;

    long long dist_me = nd[u].dist[FORWARD] + cost + nd[v].dist[REVERSE];
    int ways_me = (long long)nd[u].ways[FORWARD] * nd[v].ways[REVERSE] % MOD;

    bool is_bridge = (dist_me == min_dist) && (ways_me == min_ways);
```

```
    if (is_bridge) {
      printf("YES\n");
    } else {
      long long guarantee = min_dist - nd[u].dist[FORWARD] - nd[v].dist[REVERSE] - 1;
      if (guarantee >= 1) {
        printf("CAN %lld\n", cost - guarantee);
      } else {
        printf("NO\n");
      }
    }
  }
}

int main() {
  read_graph();
  dijkstra(source, FORWARD);
  dijkstra(sink, REVERSE);
  compute_answers();

  return 0;
}
```

30.11 Problema Nastya and Unexpected Guest (Codeforces)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_ISLANDS = 10'000;
const int MAX_GREEN = 1'000;
const int INFINITY = 1'000'000'000;

struct cell {
  short island, rem;
  int next;
};

int d[MAX_ISLANDS + 2][MAX_GREEN];
int x[MAX_ISLANDS + 2];

struct bucket_queue {
  cell list[MAX_ISLANDS * MAX_GREEN]; // buffer comun
  int list_ptr; // următorul element disponibil in bufferul comun

  int head[MAX_GREEN + 1]; // începuturi de bloc
  int pos; // blocul curent
```

```
  int n; // numărul de blocuri
  int size; // numărul de elemente în toate blocurile

  void init(int n) {
    this->n = n;
  }

  bool is_empty() {
    return !size;
  }

  void push(int steps_ahead, short island, short rem) {
    int p = (pos + steps_ahead) % n;
    list[++list_ptr] = { island, rem, head[p] };
    head[p] = list_ptr;
    size++;
  }

  cell pop() {
    while (!head[pos]) {
      pos = (pos + 1) % n;
    }

    size--;
    int h = head[pos];
    head[pos] = list[h].next;
    return list[h];
  }

};

bucket_queue bq;
int target, num_islands;
int green_duration, red_duration;

void read_data() {
  scanf("%d %d", &target, &num_islands);
  for (int i = 1; i <= num_islands; i++) {
    scanf("%d", &x[i]);
  }
  scanf("%d %d", &green_duration, &red_duration);
}

void preprocess_coords() {
  std::sort(x + 1, x + num_islands + 1);
  x[0] = x[1] - (green_duration + 1);
  x[num_islands + 1] = x[num_islands] + (green_duration + 1);
}

void init_distances() {
```

```
  for (int i = 1; i <= num_islands; i++) {
    for (int rem = 0; rem < green_duration; rem++) {
      d[i][rem] = INFINITY;
    }
  }
  d[1][0] = 0;
}

void try_hop(short src, short dest, short src_rem, int cost) {
  int dest_rem = src_rem + cost;
  if (dest_rem > green_duration) {
    return;
  } else if (dest_rem == green_duration) {
    dest_rem = 0;
  }

  if (d[src][src_rem] + cost < d[dest][dest_rem]) {
    d[dest][dest_rem] = d[src][src_rem] + cost;
    bq.push(cost, dest, dest_rem);
  }
}

void dial() {
  init_distances();
  bq.init(green_duration + 1);
  bq.push(0, 1, 0);

  while (!bq.is_empty()) {
    cell c = bq.pop();
    try_hop(c.island, c.island - 1, c.rem, x[c.island] - x[c.island - 1]);
    try_hop(c.island, c.island + 1, c.rem, x[c.island + 1] - x[c.island]);
  }

}

void write_min_distance() {
  int min = INFINITY;
  for (int rem = 0; rem < green_duration; rem++) {
    if (d[num_islands][rem] < min) {
      min = d[num_islands][rem];
    }
  }

  int answer = -1;
  if (min < INFINITY) {
    answer = min + (min  - 1) / green_duration * red_duration;
  }
  printf("%d\n", answer);
}
```

```
int main() {
  read_data();
  preprocess_coords();
  dial();
  write_min_distance();
  return 0;
}
```

30.12 Problema Minimum Path (Codeforces)

‹ înapoi

Sursă cu heap propriu (versiune online).

```
#include <stdio.h>

const int MAX_NODES = 200'000;
const int MAX_EDGES = 200'000;
const long long INFINITY = 1'000'000'000'000'000'000LL;

const int NEITHER = 0;
const int REMOVED = 1;
const int ADDED = 2;
const int BOTH = 3;
const int NUM_STATES = 4;

const int IMPOSSIBLE = -1;

const int FACTOR[NUM_STATES][NUM_STATES] = {
  {          1,          0,          2, 1 },
  { IMPOSSIBLE,          1, IMPOSSIBLE, 2 },
  { IMPOSSIBLE, IMPOSSIBLE,          1, 0 },
  { IMPOSSIBLE, IMPOSSIBLE, IMPOSSIBLE, 1 },
};

struct adj_list {
  int v, dist;
  int next;
};

struct node {
  long long dist[NUM_STATES];
  int adj;
};

struct heap_elt {
  long long dist;
  int u;
```

```
  unsigned char state;
};

struct heap {
  heap_elt v[9 * MAX_EDGES + 1]; // 9 combinații posibile în FACTOR
  int size;

  bool is_empty() {
    return size == 0;
  }

  void insert(int u, unsigned char state, long long dist) {
    int pos = ++size;
    while ((pos > 1) && (v[pos / 2].dist > dist)) {
      v[pos] = v[pos / 2];
      pos /= 2;
    }

    v[pos] = { dist, u, state };
  }

  heap_elt remove_min() {
    heap_elt result = v[1];
    heap_elt save = v[size--];

    if (size) {
      bool swapped = true;
      int x = 1;
      while ((2 * x <= size) && swapped) {
        int l = 2 * x, r = 2 * x + 1;
        int y = (r <= size) && (v[r].dist < v[l].dist)
          ? r : l;

        if (v[y].dist < save.dist) {
          v[x] = v[y];
          x = y;
        } else {
          swapped = false;
        }
      }

      v[x] = save;
    }

    return result;
  }
};

adj_list list[2 * MAX_EDGES + 1];
node nd[MAX_NODES + 1];
```

```
heap h;
int n;

void add_edge(int u, int v, int dist) {
  static int ptr = 0;

  list[++ptr] = { v, dist, nd[u].adj };
  nd[u].adj = ptr;
}

void read_graph() {
  int num_edges;

  scanf("%d %d", &n, &num_edges);

  while (num_edges--) {
    int u, v, dist;
    scanf("%d %d %d", &u, &v, &dist);
    add_edge(u, v, dist);
    add_edge(v, u, dist);
  }
}

void relax_all(int u, int su) {
  for (int sv = 0; sv < NUM_STATES; sv++) {
    int factor = FACTOR[su][sv];
    if (factor != IMPOSSIBLE) {
      for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
        int v = list[ptr].v;
        long long new_dist = nd[u].dist[su] + factor * list[ptr].dist;
        if (new_dist < nd[v].dist[sv]) {
          nd[v].dist[sv] = new_dist;
          h.insert(v, sv, new_dist);
        }
      }
    }
  }
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    for (int s = 0; s < NUM_STATES; s++) {
      nd[u].dist[s] = INFINITY;
    }
  }

  nd[1].dist[NEITHER] = 0;
  h.insert(1, NEITHER, 0);

  while (!h.is_empty()) {
```

```
    heap_elt top = h.remove_min();
    int u = top.u;
    int su = top.state;

    if (top.dist == nd[u].dist[su]) {
      relax_all(u, su);
    }
  }
}

void write_distances() {
  for (int u = 2; u <= n; u++) {
    printf("%lld ", nd[u].dist[BOTH]);
  }
  printf("\n");
}

int main() {
  read_graph();
  dijkstra();
  write_distances();

  return 0;
}
```

Sursă cu STL (versiune online).

```
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 200'000;
const long long INFINITY = 1e15;

const int NEITHER = 0;
const int REMOVED = 1;
const int ADDED = 2;
const int BOTH = 3;
const int NUM_STATES = 4;

const int IMPOSSIBLE = -1;

const int FACTOR[NUM_STATES][NUM_STATES] = {
  {          1,          0,          2, 1 },
  { IMPOSSIBLE,          1, IMPOSSIBLE, 2 },
  { IMPOSSIBLE, IMPOSSIBLE,          1, 0 },
  { IMPOSSIBLE, IMPOSSIBLE, IMPOSSIBLE, 1 },
};
```

```
struct edge {
  int v, dist;
};

struct node {
  long long dist[NUM_STATES];
  std::vector<edge> adj;
};

struct pq_elt {
  long long dist;
  int u;
  unsigned char state;

  friend bool operator< (const pq_elt& a, const pq_elt& b) {
    return a.dist > b.dist;
  }
};

node nd[MAX_NODES + 1];
std::priority_queue<pq_elt> pq;
int n;

void read_graph() {
  int num_edges;

  scanf("%d %d", &n, &num_edges);

  while (num_edges--) {
    int u, v, dist;
    scanf("%d %d %d", &u, &v, &dist);
    nd[u].adj.push_back({v, dist});
    nd[v].adj.push_back({u, dist});
  }
}

void relax_all(int u, int su) {
  for (unsigned char sv = 0; sv < NUM_STATES; sv++) {
    int factor = FACTOR[su][sv];
    if (factor != IMPOSSIBLE) {
      for (auto e: nd[u].adj) {
        int v = e.v;
        long long new_dist = nd[u].dist[su] + factor * e.dist;
        if (new_dist < nd[v].dist[sv]) {
          nd[v].dist[sv] = new_dist;
          pq.push({new_dist, v, sv});
        }
      }
    }
  }
```

```
}

void dijkstra() {
  for (int u = 1; u <= n; u++) {
    for (int s = 0; s < NUM_STATES; s++) {
      nd[u].dist[s] = INFINITY;
    }
  }

  nd[1].dist[NEITHER] = 0;
  pq.push({0, 1, NEITHER});

  while (!pq.empty()) {
    pq_elt top = pq.top();
    pq.pop();
    int u = top.u;
    int su = top.state;

    if (top.dist == nd[u].dist[su]) {
      relax_all(u, su);
    }
  }
}

void write_distances() {
  for (int u = 2; u <= n; u++) {
    printf("%lld ", nd[u].dist[BOTH]);
  }
  printf("\n");
}

int main() {
  read_graph();
  dijkstra();
  write_distances();

  return 0;
}
```

30.13 Problema Shortest Path (Codeforces)

‹ înapoi

Sursă cu `unordered_set` global de triplete interzise, cu memorie $\mathcal{O}(n^2)$ (versiune online).

```
#include <queue>
#include <stdio.h>
#include <unordered_set>
```

```cpp
#include <vector>

const int MAX_NODES = 3'000;

struct edge {
  short u, v;
};

struct triplet_set {
  std::unordered_set<long long> s;

  long long hash(int x, int y, int z) {
    return (long long)(x - 1) * MAX_NODES * MAX_NODES +
      (y - 1) * MAX_NODES +
      (z - 1);
  }

  void insert(int x, int y, int z) {
    s.insert(hash(x, y, z));
  }

  bool contains(int x, int y, int z) {
    return s.contains(hash(x, y, z));
  }
};

std::vector<short> adj[MAX_NODES + 1];
short d[MAX_NODES + 1][MAX_NODES + 1];
short prev[MAX_NODES + 1][MAX_NODES + 1];
triplet_set triplets;
int n;

void read_data() {
  int num_edges, num_triplets, u, v, w;
  scanf("%d %d %d\n", &n, &num_edges, &num_triplets);

  while (num_edges--) {
    scanf("%d %d", &u, &v);
    adj[u].push_back(v);
    adj[v].push_back(u);
  }

  while (num_triplets--) {
    scanf("%d %d %d", &u, &v, &w);
    triplets.insert(u, v, w);
  }
}

edge bfs() {
  std::queue<edge> q;
```

```
  edge last = { 0, 0 };

  q.push({1, 1});

  while (!q.empty() && (last.v != n)) {
    edge e = q.front();
    q.pop();

    for (short w: adj[e.v]) {
      if (!d[e.v][w] && !triplets.contains(e.u, e.v, w)) {
        d[e.v][w] = 1 + d[e.u][e.v];
        prev[e.v][w] = e.u;
        q.push({e.v, w});
        if (w == n) {
          last = {e.v, w};
        }
      }
    }
  }

  return last;
}

void recover_solution(int u, int v) {
  if (u > 1) {
    recover_solution(prev[u][v], u);
  }
  printf("%d ", u);
}

void write_answer(edge last) {
  if (last.v == n) {
    printf("%d\n", d[last.u][n]);
    recover_solution(last.u, n);
    printf("%d\n", n);
  } else {
    printf("-1\n");
  }
}

int main() {
  read_data();
  edge last = bfs();
  write_answer(last);

  return 0;
}
```

Sursă cu memorie $\mathcal{O}(m+n+k)$ (versiune online).

```cpp
#include <algorithm>
#include <queue>
#include <stdio.h>

const int MAX_NODES = 3'000;
const int MAX_EDGES = 20'000;
const int MAX_TRIPLETS = 100'000;

typedef unsigned short u16;

struct edge {
  u16 u, v;       // capete
  int bad;        // primul indice în t[] pentru <u,v,*>
  u16 d;          // distanța
  u16 prev;       // indicele muchiei precedente pe calea cea mai scurtă
};

struct triplet {
  u16 u, v, w;
};

edge e[2 * MAX_EDGES + 1];
// adj[u] = primul indice în e[] al muchiilor ce pornesc din u
u16 adj[MAX_NODES + 1];
triplet t[MAX_TRIPLETS + 1];
std::queue<int> q;
int n, m, num_triplets, last_edge;

void read_data() {
  scanf("%d %d %d\n", &n, &m, &num_triplets);

  for (int i = 0; i < m; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    e[2 * i + 1].u = u;
    e[2 * i + 1].v = v;
    e[2 * i + 2].u = v;
    e[2 * i + 2].v = u;
  }

  for (int i = 0; i < num_triplets; i++) {
    scanf("%hu %hu %hu", &t[i].u, &t[i].v, &t[i].w);
  }
}

void sort_data() {
  std::sort(e + 1, e + 2 * m + 1, [](edge& a, edge& b) {
    return (a.u < b.u) ||
      ((a.u == b.u) && (a.v < b.v));
```

```
  });

  std::sort(t, t + num_triplets, [](triplet& a, triplet& b) {
    return (a.u < b.u) ||
      ((a.u == b.u) && (a.v < b.v)) ||
      ((a.u == b.u) && (a.v == b.v) && (a.w < b.w));
  });
}

// Distribuie tripleții la muchii. Pentru fiecare muchie (u,v) află primul
// triplet <u,v,*>, dacă există.
void distribute_triplets() {
  int j = 0;
  t[num_triplets] = { (u16)(n + 1), (u16)(n + 1), (u16)(n + 1)}; // santinelă

  for (int i = 1; i <= 2 * m; i++) {
    while ((t[j].u < e[i].u) || ((t[j].u == e[i].u) && (t[j].v < e[i].v))) {
      j++;
    }
    e[i].bad = j; // posibil incorect, dar vom verifica mai tîrziu
  }
}

// Construiește listele de adiacență pentru fiecare nod.
void distribute_edges() {
  for (int i = 2 * m; i; i--) {
    adj[e[i].u] = i;
  }
}

void relax_good_successors(int ind) {
  edge& f = e[ind];

  // Interclasează continuările rele pentru f cu muchiile care pornesc din
  // f.v. Relaxează muchiile care nu sînt rele.
  int j = f.bad;
  for (unsigned i = adj[f.v]; e[i].u == f.v; i++) {
    int w = e[i].v;
    while ((t[j].u == f.u) && (t[j].v == f.v) && (t[j].w < w)) {
      j++;
    }

    if (!e[i].d && ((t[j].u != f.u) || (t[j].v != f.v) || (t[j].w != w))) {
      // Nu am găsit niciun triplet <f.u, f.v, w>
      e[i].d = f.d + 1;
      e[i].prev = ind;
      if (e[i].v == n) {
        last_edge = i;
      }
      q.push(i);
```

```
    }
  }
}

void bfs() {
  // Inițializează coada cu o muchie fictivă 1-1.
  e[0].u = e[0].v = 1;
  q.push(0);

  while (!q.empty() && !last_edge) {
    unsigned ind = q.front();
    q.pop();
    relax_good_successors(ind);
  }
}

void recover_solution(int ind) {
  if (ind) {
    recover_solution(e[ind].prev);
    printf("%d ", e[ind].u);
  }
}

void write_answer() {
  if (last_edge) {
    printf("%d\n", e[last_edge].d);
    recover_solution(last_edge);
    printf("%d\n", n);
  } else {
    printf("-1\n");
  }
}
int main() {
  read_data();
  sort_data();
  distribute_triplets();
  distribute_edges();
  bfs();
  write_answer();

  return 0;
}
```

Anexa 31

Grafuri - Drumuri euleriene

31.1 Problema Ciclu Eulerian (Infoarena)

înapoi

Sursă recursivă (versiune online).

```
// Implementare recursivă.
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 500'000;

struct edge {
  int v;
  bool active;
  int next;
};

struct node {
  int adj;
  bool even_degree;
};

node nd[MAX_NODES + 1];
edge e[2 * MAX_EDGES + 1];
int cycle[MAX_EDGES + 1];
int n, m, cycle_len, num_odd;

void add_edge(int u, int v) {
  static int ptr = 1;
  e[ptr] = { v, true, nd[u].adj };
  nd[u].adj = ptr++;
  nd[u].even_degree = !nd[u].even_degree;
  num_odd += nd[u].even_degree ? -1 : +1;
```

```
}

void read_graph() {
  FILE* f = fopen("ciclueuler.in", "r");
  fscanf(f, "%d %d", &n, &m);
  for (int i = 1; i <= m; i++) {
    int u, v;
    fscanf(f, "%d %d", &u, &v);
    // Muchia u-v va ocupa două poziții consecutive în e, 2k-1 și 2k.
    add_edge(u, v);
    add_edge(v, u);
  }
  fclose(f);
}

void delete_edge(int ptr) {
  int rev = ((ptr + 1) ^ 1) - 1;
  e[ptr].active = e[rev].active = false;
}

void dfs(int u) {
  // Nu merge implementat cu for. Să considerăm un graf cu n=2 și 500.000 de
  // muchii între nodurile 1 și 2. Atunci la fiecare nivel de recursivitate
  // fiecare nod va parcurge o listă de lungime O(m).
  while (nd[u].adj) {
    int ptr = nd[u].adj;
    nd[u].adj = e[ptr].next;
    if (e[ptr].active) {
      delete_edge(ptr);
      dfs(e[ptr].v);
    }
  }
  cycle[cycle_len++] = u;
}

void write_solution() {
  FILE* f = fopen("ciclueuler.out", "w");
  if (num_odd) {
    fprintf(f, "-1\n");
  } else {
    for (int i = 0; i < m; i++) {
      fprintf(f, "%d ", cycle[i]);
    }
    fprintf(f, "\n");
  }
  fclose(f);
}

int main() {
  read_graph();
```

```
  if (!num_odd) {
    dfs(1);
  }
  write_solution();

  return 0;
}
```

Sursă iterativă (versiune online).

```
// Implementare iterativă.
#include <stdio.h>

const int MAX_NODES = 100'000;
const int MAX_EDGES = 500'000;

struct edge {
  int v;
  bool active;
  int next;
};

struct node {
  int adj;
  bool even_degree;
};

node nd[MAX_NODES + 1];
edge e[2 * MAX_EDGES + 1];
int st[MAX_EDGES];
int cycle[MAX_EDGES + 1];
int n, m, cycle_len, num_odd;

void add_edge(int u, int v) {
  static int ptr = 1;
  e[ptr] = { v, true, nd[u].adj };
  nd[u].adj = ptr++;
  nd[u].even_degree = !nd[u].even_degree;
  num_odd += nd[u].even_degree ? -1 : +1;
}

void read_graph() {
  FILE* f = fopen("ciclueuler.in", "r");
  fscanf(f, "%d %d", &n, &m);
  for (int i = 1; i <= m; i++) {
    int u, v;
    fscanf(f, "%d %d", &u, &v);
    // Muchia u-v va ocupa două poziţii consecutive în e, 2k-1 şi 2k.
    add_edge(u, v);
```

```
    add_edge(v, u);
  }
  fclose(f);
}

int find_neighbor(int u) {
  // Sari peste muchiile șterse.
  while (nd[u].adj && !e[nd[u].adj].active) {
    nd[u].adj = e[nd[u].adj].next;
  }

  if (nd[u].adj) {
    int ptr = nd[u].adj;
    int rev = ((ptr + 1) ^ 1) - 1;
    e[rev].active = false;
    nd[u].adj = e[ptr].next;
    return e[ptr].v;
  } else {
    return 0;
  }
}

void iterative_dfs() {
  int ss = 0; // stack size
  st[ss++] = 1;

  while (ss) {
    int u = st[ss - 1];
    int v = find_neighbor(u);
    if (v) {
      st[ss++] = v;
    } else {
      cycle[cycle_len++] = u;
      ss--;
    }
  }
}

void write_solution() {
  FILE* f = fopen("ciclueuler.out", "w");
  if (num_odd) {
    fprintf(f, "-1\n");
  } else {
    for (int i = 0; i < m; i++) {
      fprintf(f, "%d ", cycle[i]);
    }
    fprintf(f, "\n");
  }
  fclose(f);
}
```

```
int main() {
  read_graph();
  if (!num_odd) {
    iterative_dfs();
  }
  write_solution();

  return 0;
}
```

31.2 Problema De Bruijn Sequence (CSES)

înapoi

Sursă recursivă.

```
#include <stdio.h>

const int MAX_NODES = 1 << 14;

int n, mask;
char state[MAX_NODES];

void read_data() {
  scanf("%d", &n);
  mask = (1 << (n - 1)) - 1;
}

void euler_tour(int u) {
  while (state[u] < 2) {
    int v = ((u << 1) + state[u]) & mask;
    state[u]++;
    euler_tour(v);
  }

  putchar('0' + (u & 1));
}

void euler_tour_wrapper() {
  if (n == 1) {
    printf("01\n");
    return;
  }

  euler_tour(0);

  for (int i = 0; i < n - 2; i++) {
```

```
    putchar('0');
  }
  putchar('\n');
}

int main() {
  read_data();
  euler_tour_wrapper();
  return 0;
}
```

Sursă iterativă.

```
#include <stdio.h>

const int MAX_NODES = 1 << 14;

int n, mask;
char state[MAX_NODES];
int st[2 * MAX_NODES];

void read_data() {
  scanf("%d", &n);
  mask = (1 << (n - 1)) - 1;
}

void euler_tour() {
  if (n == 1) {
    printf("01\n");
    return;
  }

  int size = 1;
  st[0] = 0;

  while (size) {
    int u = st[size - 1];
    if (state[u] == 2) {
      putchar('0' + (u & 1));
      size--;
    } else {
      st[size++] = ((u << 1) + state[u]) & mask;
      state[u]++;
    }
  }

  for (int i = 0; i < n - 2; i++) {
    putchar('0');
  }
```

```
  putchar('\n');
}

int main() {
  read_data();
  euler_tour();
  return 0;
}
```

31.3 Problema Tanya and Password (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_LEN = 200'000;
const int BITS = 6;
const int BASE = 1 << BITS;
const int MAX_NODES = BASE * BASE;
const int NUM_RANGES = 3;
const int RANGES[NUM_RANGES][2] = {{'0', '9'}, {'A', 'Z'}, {'a', 'z'}};

struct node {
  int adj[BASE]; // vector de adiacență
  int ptr;       // prima valoare nenulă din adj
  int in, out;   // gradele la intrare / ieșire
};

node nd[MAX_NODES];
int len;
unsigned char rank[128], unrank[BASE];
short sol[MAX_LEN + 2];
short st[MAX_LEN + 2];

void make_tables() {
  int d = 1;

  for (int r = 0; r < NUM_RANGES; r++) {
    for (int c = RANGES[r][0]; c <= RANGES[r][1]; c++) {
      rank[c] = d++;
      unrank[rank[c]] = c;
    }
  }
}

int make_node(int x, int y) {
  return x * BASE + y;
}
```

```
int upper(int u) {
  return u >> BITS;
}

int lower(int u) {
  return u & (BASE - 1);
}

void add_edge(int x, int y, int z) {
  int u = make_node(x, y);
  int v = make_node(y, z);
  nd[u].adj[z]++;
  nd[u].out++;
  nd[v].in++;
}

void read_graph() {
  scanf("%d ", &len);
  for (int i = 0; i < len; i++) {
    unsigned char a, b, c;
    scanf("%c%c%c ", &a, &b, &c);
    // Răsturnat, deoarece algoritmul generează circuitul răsturnat.
    add_edge(rank[c], rank[b], rank[a]);
  }
}

int get_start_node() {
  int num_zero = 0, num_one = 0;
  int start_node = 0;

  for (int u = 0; u < MAX_NODES; u++) {
    if (nd[u].out == nd[u].in) {
      num_zero++;
    }
    if (nd[u].out == nd[u].in + 1) {
      num_one++;
      start_node = u;
    }
    if (nd[u].out && !start_node) {
      start_node = u;
    }
  }

  if ((num_zero == MAX_NODES) ||
      ((num_zero == MAX_NODES - 2) && (num_one == 1))) {
    return start_node;
  } else {
    return 0;
  }
```

```
}

int find_neighbor(int u) {
  int& v = nd[u].ptr;
  while ((v < BASE) && !nd[u].adj[v]) {
    v++;
  }

  if (v < BASE) {
    nd[u].adj[v]--;
    return make_node(lower(u), v);
  } else {
    return 0;
  }
}

bool euler_tour() {
  int start_node = get_start_node();
  if (!start_node) {
    return false;
  }

  int st_size = 1, sol_size = 0;
  st[0] = start_node;

  while (st_size) {
    int u = st[st_size - 1];
    int v = find_neighbor(u);
    if (v) {
      st[st_size++] = v;
    } else {
      sol[sol_size++] = u;
      st_size--;
    }
  }

  return (sol_size == len + 1);
}

void write_solution() {
  printf("YES\n");
  for (int i = 0; i <= len; i++) {
    putchar(unrank[lower(sol[i])]);
  }
  putchar(unrank[upper(sol[len])]);
  putchar('\n');
}

int main() {
  make_tables();
```

```
  read_graph();
  bool feasible = euler_tour();

  if (feasible) {
    write_solution();
  } else {
    printf("NO\n");
  }

  return 0;
}
```

Anexa 32

Grafuri - Fluxuri

32.1 Problema Download Speed (CSES)

înapoi

Sursă cu metoda Ford-Fulkerson (DFS).

```
// Metodă: Ford Fulkerson + DFS.
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  bool seen;
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}
```

```
void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void reset_dfs() {
  for (int u = 1; u <= n; u++) {
    nd[u].seen = false;
  }
}

// Găsește un drum de creștere de la u la destinație și pompează cel mult
// max_pump unități pe el. Returnează fluxul pompat efectiv sau 0 dacă nu
// există niciun drum de creștere.
int dfs(int u, int max_pump) {
  nd[u].seen = true;
  if (u == n) {
    return max_pump;
  }

  int pumped = 0;
  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    int v = e[pos].v;
    int residual = e[pos].cap;
    if (!pumped && residual && !nd[v].seen) {
      pumped = dfs(v, min(max_pump, residual));
      if (pumped) {
        e[pos].cap -= pumped;
        e[pos ^ 1].cap += pumped;
      }
    }
  }

  return pumped;
}
```

```
long long ford_fulkerson() {
  long long flow = 0, augment;

  do {
    reset_dfs();
    augment = dfs(1, INFINITY);
    flow += augment;
  } while (augment);

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);

  return 0;
}
```

Sursă cu algoritmul Edmonds-Karp.

```
// Algoritmul Edmonds-Karp (Ford-Fulkerson + BFS).
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
```

```
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
```

```
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

long long ford_fulkerson() {
  long long flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);
```

```
  return 0;
}
```

Sursă cu algoritmul Edmonds-Karp cu drumuri de creștere multiple.

```
// Algoritmul Edmonds-Karp cu o încercare de optimizare. La fiecare iterație
// încercăm să găsim mai multe drumuri de creștere din același arbore BFS.
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
```

```
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}
```

```
int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

long long ford_fulkerson() {
  long long flow = 0;

  while (bfs_reachable()) {
    // Succesorii destinației sînt și predecesorii ei.
    for (int pos = nd[n].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if ((v == 1) || (nd[v].parent != NIL)) {
        // Conectează destinația la v în arborele BFS.
        nd[n].parent = v;
        nd[n].edge = pos ^ 1;
        int augment = path_minimum();
        if (augment) {
          pump_on_path(augment);
          flow += augment;
        }
      }
    }
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);

  return 0;
}
```

Sursă cu algoritmul lui Dinic.

```
// Algoritmul lui Dinic.
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short ptr;
  short dist;

  bool is_reachable() {
    return dist != NIL;
  }
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
```

```
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].dist = NIL;
  }
  nd[1].dist = 0;
  q.clear();
  q.enqueue(1);

  while (!q.is_empty()) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if (!nd[v].is_reachable() && e[pos].cap) {
        nd[v].dist = nd[u].dist + 1;
        q.enqueue(v);
      }
    }
  }

  return nd[n].is_reachable();
}
```

```
void reset_dfs_pointers() {
  for (int u = 1; u <= n; u++) {
    nd[u].ptr = nd[u].adj;
  }
}

int dfs(int u, int incoming_flow) {
  if (u == n) {
    return incoming_flow;
  }

  int result = 0;
  while (!result && (nd[u].ptr != NIL)) {
    int p = nd[u].ptr;
    int v = e[p].v, cap = e[p].cap;
    if (cap && (nd[v].dist == nd[u].dist + 1)) {
      result = dfs(v, min(cap, incoming_flow));
      e[p].cap -= result;
      e[p ^ 1].cap += result;
    }
    // Rămîi pe aceeași muchie dacă mai este folosibilă. Aceasta scade timpul
    // de rulare de 6 ori.
    if (!result) {
      nd[u].ptr = e[p].next;
    }
  }

  return result;
}

long long dinic() {
  long long flow = 0;

  while (bfs_reachable()) {
    reset_dfs_pointers();
    int augment;
    while ((augment = dfs(1, INFINITY))) {
      flow += augment;
    }
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = dinic();
  printf("%lld\n", max_flow);

  return 0;
```

```
}
```

32.2 Problema Police Chase (CSES)

‹ înapoi

Sursă cu algoritmul Edmonds-Karp.

```
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int NIL = -1;
const int INFINITY = 1'000'000;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[4 * MAX_EDGES];
```

```
node nd[MAX_NODES + 1];
queue q;
int n;

void add_edge(short u, short v) {
  static int pos = 0;

  e[pos] = { v, 1, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
```

```
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int ford_fulkerson() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void write_min_cut(int flow) {
  printf("%d\n", flow);

  for (int u = 1; u <= n; u++) {
    if ((u == 1) || (nd[u].parent != NIL)) {
      for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
        int v = e[pos].v;
        if ((nd[v].parent == NIL)) {
          printf("%d %d\n", u, v);
        }
      }
    }
  }
}

int main() {
  read_data();
```

```
  int max_flow = ford_fulkerson();
  write_min_cut(max_flow);

  return 0;
}
```

32.3 Problema Distinct Routes (CSES)

înapoi

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int NIL = -1;
const int INFINITY = 1'001;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
std::queue<short> q;
int n, num_edges;

void add_edge(short u, short v, short cap) {
  static int pos = 0;

  e[pos] = { v, cap, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  scanf("%d %d", &n, &num_edges);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  for (int i = 0; i < num_edges; i++) {
    int u, v;
```

```
    scanf("%d %d", &u, &v);
    add_edge(u, v, 1);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q = {};
  q.push(1);

  while (!q.empty() && (nd[n].parent == NIL)) {
    int u = q.front();
    q.pop();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.push(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
```

```
  }
}

int ford_fulkerson() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void clear_parent() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
}

void dfs(int u, int parent, int edge) {
  nd[u].parent = parent;
  nd[u].edge = edge;

  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    int v = e[pos].v;
    int rev = pos ^ 1;
    // Follow a regular edge if there is residual flow on the reverse edge.
    if ((nd[v].parent == NIL) &&
        !(pos & 1) &&
        e[rev].cap) {
      dfs(v, u, pos);
    }
  }
}

void print_path(int u, int len) {
  if (u == 1) {
    printf("%d\n1 ", len);
  } else {
    int edge = nd[u].edge;
    e[edge].cap++;
    e[edge ^ 1].cap--;
    print_path(nd[u].parent, len + 1);
    printf("%d ", u);
  }
}

void recover_paths() {
```

```
  do {
    clear_parent();
    dfs(1, NIL, NIL);
    if (nd[n].parent != NIL) {
      print_path(n, 1);
      printf("\n");
    }
  } while (nd[n].parent != NIL);
}

int main() {
  read_data();
  int max_flow = ford_fulkerson();
  printf("%d\n", max_flow);
  recover_paths();

  return 0;
}
```

32.4 Problema Soldier and Traveling (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_CITIES = 100;
const int MAX_ROADS = 200;
const int MAX_NODES = 2 * MAX_CITIES + 2;
const int MAX_EDGES = 3 * MAX_CITIES + 2 * MAX_ROADS;
const int INFINITY = 1'000;
const int NIL = -1;

struct cell {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent;
  short edge;

  bool is_reachable() {
    return (parent != NIL);
  }
};
```

```
struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

cell e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n, num_nodes, num_edges;
int source, sink;
int sum_source, sum_sink;

void add_edge(short u, short v, short c) {
  static int pos = 0;

  e[pos] = { v, c, nd[u].adj };
  nd[u].adj = pos++;
}

void add_edge_pair(short u, short v, short c) {
  add_edge(u, v, c);
  add_edge(v, u, 0);
}

void read_data() {
  int m, val;
  scanf("%d %d", &n, &m);
  num_nodes = sink = 2 * n + 2;
  source = 2 * n + 1;

  for (int u =  1; u <= num_nodes; u++) {
    nd[u].adj = NIL;
  }
```

```
  for (int u = 1; u <= n; u++) {
    scanf("%d", &val);
    add_edge_pair(source, u, val);
    sum_source += val;
  }

  for (int u = 1; u <= n; u++) {
    scanf("%d", &val);
    add_edge_pair(u + n, sink, val);
    sum_sink += val;
  }

  for (int u = 1; u <= n; u++) {
    add_edge_pair(u, u + n, INFINITY);
  }

  while (m--) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge_pair(u, v + n, INFINITY);
    add_edge_pair(v, u + n, INFINITY);
  }
}

bool bfs_reachable() {
  for (int u = 1; u <= num_nodes; u++) {
    nd[u].parent = NIL;
  }
  nd[source].parent = source;
  q.clear();
  q.enqueue(source);

  while (!q.is_empty() && !nd[sink].is_reachable()) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if (!nd[v].is_reachable() && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return nd[sink].is_reachable();
}

int min(int x, int y) {
  return (x < y) ? x : y;
}
```

```
int path_minimum() {
  int min_cap = INFINITY;
  int u = num_nodes;
  while (u != source) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = num_nodes;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void print_travel_matrix() {
  int dest[n + 1];
  for (int u = 1; u <= n; u++) {
    for (int v = 1; v <= n; v++) {
      dest[v] = 0;
    }
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v - n;
      int count = e[pos ^ 1].cap;
      dest[v] = count;
    }
    for (int v = 1; v <= n; v++) {
      printf("%d ", dest[v]);
    }
    printf("\n");
  }
}
```

```
int main() {
  read_data();

  if (sum_source != sum_sink) {
    printf("NO\n");
  } else {
    int max_flow = edmonds_karp();
    if (max_flow != sum_source) {
      printf("NO\n");
    } else {
      printf("YES\n");
      print_travel_matrix();
    }
  }

  return 0;
}
```

32.5 Problema Fox and Dinner (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 200;
const int MAX_NODES = MAX_N + 2;
const int MAX_EDGES = (MAX_N / 2) * (MAX_N / 2) + MAX_N;
const int MAX_VAL = 10'000;
const int NIL = -1;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_N + 2];
  int head, tail;

  void clear() {
    head = tail = 0;
  }
```

```
  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

short age[MAX_N + 1];
bool composite[2 * MAX_VAL + 1];
node nd[MAX_NODES];
edge e[2 * MAX_EDGES]; // include şi muchiile reziduale
queue q;
bool used[MAX_N + 1];
unsigned char cycle[MAX_N][MAX_N];
int cycle_len[MAX_N];
int num_cycles;
int n, num_odd, source, sink;
bool possible;

void read_data() {
  scanf("%d", &n);
  for (int i = 1; i <= n; i++) {
    scanf("%hd", &age[i]);
    num_odd += age[i] % 2;
  }
  possible = (2 * num_odd == n);
}

void sieve() {
  for (int i = 2; i <= 2 * MAX_VAL; i++) {
    if (!composite[i]) {
      for (int j = 2 * i; j <= 2 * MAX_VAL; j += i) {
        composite[j] = true;
      }
    }
  }
}

void add_edge(short u, short v, short cap) {
  static int ptr = 0;

  e[ptr] = { v, cap, nd[u].adj };
  nd[u].adj = ptr++;
```

```
  e[ptr] = { u, 0, nd[v].adj };
  nd[v].adj = ptr++;
}

void build_graph() {
  source = 0;
  sink = n + 1;

  for (int u = 0; u <= n + 1; u++) {
    nd[u].adj = NIL;
  }

  for (int i = 1; i <= n; i++) {
    if (age[i] % 2) {
      add_edge(i, sink, 2);
    } else {
      add_edge(source, i, 2);
    }
  }

  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      if ((age[j] % 2) && !composite[age[i] + age[j]]) {
        add_edge(i, j, 1);
      }
    }
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 0; u <= n + 1; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(source);

  while (!q.is_empty() && (nd[sink].parent == NIL)) {
    int u = q.dequeue();
    for (int ptr = nd[u].adj; ptr != NIL; ptr = e[ptr].next) {
      int v = e[ptr].v;
      if ((nd[v].parent == NIL) && e[ptr].cap) {
        nd[v].parent = u;
        nd[v].edge = ptr;
        q.enqueue(v);
      }
    }
```

```
  }

  return (nd[sink].parent != NIL);
}

int path_minimum() {
  int u = sink;
  int min_cap = e[nd[sink].edge].cap;

  while (u != source) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = sink;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

int find_neighbor(int u) {
  used[u] = true;

  for (int ptr = nd[u].adj; ptr != NIL; ptr = e[ptr].next) {
    int v = e[ptr].v;
    if (!used[v] &&
        (((age[u] % 2 == 0) && !e[ptr].cap) ||
         ((age[u] % 2 == 1) && e[ptr].cap))) {
      return v;
    }
  }

  return NIL;
```

```
}

void collect_cycle(int u) {
  int len = 0;

  cycle[num_cycles][len++] = u;
  while ((u = find_neighbor(u)) != NIL) {
    cycle[num_cycles][len++] = u;
  }

  cycle_len[num_cycles++] = len;
}

void collect_cycles() {
  used[source] = used[sink] = true;

  for (int i = 1; i <= n; i++) {
    if ((age[i] % 2 == 0) && !used[i]) {
      collect_cycle(i);
    }
  }
}

void write_cycles() {
  printf("%d\n", num_cycles);
  for (int i = 0; i < num_cycles; i++) {
    printf("%d", cycle_len[i]);
    for (int j = 0; j < cycle_len[i]; j++) {
      printf(" %d", cycle[i][j]);
    }
    printf("\n");
  }
}

int main() {
  read_data();

  if (possible) {
    sieve();
    build_graph();
    possible = (edmonds_karp() == n);
  }

  if (possible) {
    collect_cycles();
    write_cycles();
  } else {
    printf("Impossible\n");
  }
```

```
  return 0;
}
```

32.6 Problema Petya and Graph (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000;
const int MAX_M = 1'000;

const int MAX_NODES = 1 + MAX_M + MAX_N + 1;
const int MAX_EDGES = MAX_M + 2 * MAX_M + MAX_N;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  long long cap;
  short v, next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};
```

```
edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
long long total_profit;
int n;

void add_edge(short u, short v, long long cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void add_edge_pair(short u, short v, long long cap) {
  add_edge(u, v, cap);
  add_edge(v, u, 0); // muchia reziduală
}

// 1 = sursă
// 2...m+1 = muchii
// m+2...m+n+1 = noduri
// m+n+2 = destinație
void read_data() {
  int orig_n, orig_m;

  scanf("%d %d", &orig_n, &orig_m);
  n = orig_n + orig_m + 2;

  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }

  for (int u = orig_m + 2; u < n; u++) {
    int loss;
    scanf("%d", &loss);
    add_edge_pair(u, n, loss);
  }

  for (int e = 2; e <= orig_m + 1; e++) {
    int u, v, profit;
    scanf("%d %d %d", &u, &v, &profit);
    total_profit += profit;
    add_edge_pair(1, e, profit);
    add_edge_pair(e, u + orig_m + 1, INFINITY);
    add_edge_pair(e, v + orig_m + 1, INFINITY);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
```

```
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Ia în calcul doar capacități pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

long long edmonds_karp() {
  long long flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
```

```
    flow += augment;
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = edmonds_karp();
  printf("%lld\n", total_profit - max_flow);

  return 0;
}
```

32.7 Problema Asfalt (Lot 2011)

‹ înapoi • versiune online

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 5'000;
const int INFINITY = 1'000'000'000;
const int DIR_SOURCE = 0;
const int DIR_SINK = 1;
const int NIL = -1;

struct cell {
  int c;
  short v;
  short next;
  bool forward;
};

struct node {
  short adj;
  short parent;
  short edge;
  int dist[2];

  bool is_reachable() {
    return (parent != NIL);
  }
};

struct heap_elt {
  int dist;
```

```
  short u;

  friend bool operator< (const heap_elt& a, const heap_elt& b) {
    return a.dist > b.dist;
  }
};

cell e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
std::priority_queue<heap_elt> h;
int n, num_edges, source, sink;

void add_edge(short u, short v, int c) {
  static int pos = 0;

  e[pos] = { c, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  FILE *f = fopen("asfalt.in", "r");

  fscanf(f, "%d %d", &n, &num_edges);
  fscanf(f, "%d %d", &source, &sink);

  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }

  for (int i = 0; i < num_edges; i++) {
    int u, v, c;
    fscanf(f, "%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, c);
  }

  fclose(f);
}

void relax_all(int u, int dir) {
  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    short v = e[pos].v;
    int new_dist = nd[u].dist[dir] + e[pos].c;
    if (new_dist < nd[v].dist[dir]) {
      nd[v].dist[dir] = new_dist;
      h.push({new_dist, v});
    }
  }
}
```

```
void dijkstra(short start, int dir) {
  for (int u = 1; u <= n; u++) {
    nd[u].dist[dir] = INFINITY;
  }

  nd[start].dist[dir] = 0;
  h.push({0, start});

  while (!h.empty()) {
    heap_elt top = h.top();
    h.pop();

    if (top.dist == nd[top.u].dist[dir]) {
      relax_all(top.u, dir);
    }
  }
}

bool is_min_path_edge(int u, int edge_cost, int v) {
  return nd[u].dist[DIR_SOURCE] + edge_cost + nd[v].dist[DIR_SINK] ==
    nd[source].dist[DIR_SINK];
}

void convert_to_flow_network() {
  for (int u = 1; u <= n; u++) {
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v, c = e[pos].c;
      // Lăsăm unul dintre noduri să configureze capacităţile ambelor muchii.
      // Notăm care este muchia înainte.
      if (u < v) {
        e[pos].c = e[pos ^ 1].c = 0;
        if (is_min_path_edge(u, c, v)) {
          e[pos].c = 1;
          e[pos].forward = true;
        } else if (is_min_path_edge(v, c, u)) {
          e[pos ^ 1].c = 1;
          e[pos ^ 1].forward = true;
        }
      }
    }
  }
}

bool bfs_reachable() {
  std::queue<int> q;

  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  nd[source].parent = source;
```

```
  q.push(source);

  while (!q.empty() && (nd[sink].parent == NIL)) {
    int u = q.front();
    q.pop();

    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if ((nd[v].parent == NIL) && e[pos].c) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.push(v);
      }
    }
  }

  return nd[sink].is_reachable();
}

void pump_on_path() {
  int u = sink;
  while (u != source) {
    e[nd[u].edge].c--;
    e[nd[u].edge ^ 1].c++;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    pump_on_path();
    flow++;
  }

  return flow;
}

bool is_cross_cut_flow_edge(int u, int v, int pos) {
  return
    nd[u].is_reachable() &&
    !nd[v].is_reachable() &&
    e[pos].forward &&
    (e[pos ^ 1].c == 1);   // e[pos] este o muchie saturată de flux
}

void write_min_cut(int flow) {
  FILE* f = fopen("asfalt.out", "w");
  fprintf(f, "%d\n", flow);
```

```
  for (int u = 1; u <= n; u++) {
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if (is_cross_cut_flow_edge(u, v, pos)) {
        fprintf(f, "%d %d\n", u, v);
      }
    }
  }

  fclose(f);
}

int main() {
  read_data();
  dijkstra(source, DIR_SOURCE);
  dijkstra(sink, DIR_SINK);
  convert_to_flow_network();
  int max_flow = edmonds_karp();
  write_min_cut(max_flow);

  return 0;
}
```

32.8 Problema Echipe (Lot 2025)

‹ înapoi • versiune online

```
#include <algorithm>
#include <set>
#include <stdio.h>

typedef std::multiset<int> multiset;

const int MAX_N = 150'000;
const long long INF = 1'000'000'000'000'000ll;

// Structura grafului. A_ sînt clasele persoanelor, iar B_ sînt rolurile.
const int S = 0;
const int A_C = 1;
const int A_M = 2;
const int A_MC = 3;
const int A_I = 4;
const int A_IC = 5;
const int A_IM = 6;
const int A_IMC = 7;
const int B_C = 8;
const int B_M = 9;
```

```
const int B_I = 10;
const int B_IM = 11;
const int T = 12;
const int NIL = -1;
const int NUM_CLASSES = 8;
const int NUM_NODES = 13;
const int FIRST_A = 1;
const int LAST_A = 7;
const int FIRST_B = 8;
const int LAST_B = 11;

const int ADJ[NUM_NODES][NUM_NODES] = {
  /* S */     { A_C, A_M, A_MC, A_I, A_IC, A_IM, A_IMC, NIL },
  /* A_C */   { S, B_C, NIL },
  /* A_M */   { S, B_M, B_IM, NIL },
  /* A_MC */  { S, B_C, B_M, B_IM, NIL },
  /* A_I */   { S, B_I, B_IM, NIL },
  /* A_IC */  { S, B_C, B_I, B_IM, NIL },
  /* A_IM */  { S, B_M, B_I, B_IM, NIL },
  /* A_IMC */ { S, B_C, B_M, B_I, B_IM, NIL },
  /* B_C */   { A_C, A_MC, A_IC, A_IMC, T, NIL },
  /* B_M */   { A_M, A_MC, A_IM, A_IMC, T, NIL },
  /* B_I */   { A_I, A_IC, A_IM, A_IMC, T, NIL },
  /* B_IM */  { A_M, A_MC, A_I, A_IC, A_IM, A_IMC, T, NIL },
  /* T */     { B_C, B_M, B_I, B_IM, NIL },
};

struct person {
  int attr, risk;
};

// O rețea de flux. Ocupă circa 700 de octeți, deci o putem copia cu totul la
// nevoie.
struct network_flow {
  int cap[NUM_NODES][NUM_NODES];
  bool seen[NUM_NODES];
  int flow;

  void build_from_persons(person* p, int n) {
    for (int i = 0; i < n; i++) {
      cap[S][p[i].attr]++;
    }

    // capacități infinite de la A la B
    for (int u = FIRST_A; u <= LAST_A; u++) {
      for (int i = 0; ADJ[u][i] != NIL; i++) {
        int v = ADJ[u][i];
        if ((v >= FIRST_B) && (v <= LAST_B)) {
          cap[u][v] = n;
        }
```

```
    }
  }
}

void increase_sink_capacities() {
  for (int u = FIRST_B; u <= LAST_B; u++) {
    cap[u][T]++;
  }
}

void increase_source_capacity(int a) {
  cap[S][a]++;
}

void reset_dfs() {
  for (int u = 0; u < NUM_NODES; u++) {
    seen[u] = false;
  }
}

bool dfs(int u) {
  seen[u] = true;
  if (u == T) {
    return true;
  }

  for (int i = 0; ADJ[u][i] != NIL; i++) {
    int v = ADJ[u][i];
    if (cap[u][v] && !seen[v] && dfs(v)) {
      cap[u][v]--;
      cap[v][u]++;
      return true;
    }
  }

  return false;
}

void ford_fulkerson() {
  bool reachable;
  do {
    reset_dfs();
    reachable = dfs(S);
    flow += reachable;
  } while (reachable);
}

void remove_unit(int a_node) {
  if (cap[S][a_node]) {
    // Există persoane cu aceste atribute încă necuplate. Elimină pur și
```

```
      // simplu una dintre aceste persoane.
      cap[S][a_node]--;
    } else {
      // Toate persoanele cu aceste atribute sînt cuplate. Caută un b către
      // care se duce fluxul și elimină unitatea S->a->b->T.
      int b = FIRST_B;
      while (!cap[b][a_node]) {
        b++;
      }

      cap[a_node][S]--;
      cap[a_node][b]++;
      cap[b][a_node]--;
      cap[b][T]++;
      cap[T][b]--;
      flow--;
    }
  }
};

struct team_collection {
  multiset risks_in[NUM_CLASSES];
  multiset risks_out[NUM_CLASSES];
  long long total_in_risk; // menținut incremental

  void include(person p) {
    risks_in[p.attr].insert(p.risk);
    total_in_risk += p.risk;
  }

  void exclude(person p) {
    risks_out[p.attr].insert(p.risk);
  }

  // Înlocuiește un risc (care garantat există).
  void replace_value(int attr, int old_risk, int new_risk) {
    multiset& rin = risks_in[attr];
    multiset& rout = risks_out[attr];

    auto it = rin.find(old_risk);
    if (it != rin.end()) {
      rin.erase(it);
      rin.insert(new_risk);
      total_in_risk += new_risk - old_risk;
    } else {
      it = rout.find(old_risk);
      rout.erase(it);
      rout.insert(new_risk);
    }
  }
```

```
  // Asigură-te că nicio valoare din risks_in nu este mai mare decît nicio
  // valoare din risks_out.
  void resort_sets(int attr) {
    multiset& rin = risks_in[attr];
    multiset& rout = risks_out[attr];

    if (!rin.empty() && !rout.empty()) {
      int last_in = *rin.rbegin();
      int first_out = *rout.begin();
      if (last_in > first_out) {
        rin.erase(std::prev(rin.end()));
        rout.erase(rout.begin());
        rin.insert(first_out);
        rout.insert(last_in);
        total_in_risk += first_out - last_in;
      }
    }
  }

  // Mută un element de la excluse la incluse.
  void move_in(int attr) {
    auto it = risks_out[attr].begin();
    total_in_risk += *it;
    risks_in[attr].insert(*it);
    risks_out[attr].erase(it);
  }

  // Mută un element de la incluse la excluse.
  void move_out(int attr) {
    auto it = std::prev(risks_in[attr].end());
    total_in_risk -= *it;
    risks_out[attr].insert(*it);
    risks_in[attr].erase(it);
  }
};

network_flow g;
team_collection tc;
person p[MAX_N];
int ind[MAX_N]; // pentru sortarea după risc
int n, num_teams;

void read_people() {
  scanf("%d ", &n);
  for (int i = 0; i < n; i++) {
    p[i].attr = getchar() - '0';
    p[i].attr += 2 * (getchar() - '0');
    p[i].attr += 4 * (getchar() - '0');
    scanf("%d ", &p[i].risk);
```

```
  }
}

void find_num_teams() {
  network_flow orig_g;
  g.build_from_persons(p, n);
  do {
    orig_g = g;
    num_teams++;
    g.increase_sink_capacities();
    g.ford_fulkerson();
  } while (g.flow == 4 * num_teams);

  // Ultima creștere a modificat graful, apoi a eșuat.
  num_teams--;
  g = orig_g;
}

void sort_by_risk() {
  for (int i = 0; i < n; i++) {
    ind[i] = i;
  }
  std::sort(ind, ind + n, [](int a, int b) {
    return p[a].risk > p[b].risk;
  });
}

void find_min_cost_teams() {
  sort_by_risk();

  for (int i = 0; i < n; i++) {
    person& dude = p[ind[i]];
    network_flow orig_g = g;
    g.remove_unit(dude.attr);
    g.ford_fulkerson();
    if (g.flow == 4 * num_teams) {
      tc.exclude(dude);
    } else {
      tc.include(dude);
      g = orig_g;
    }
  }

  printf("%d %lld\n", num_teams, tc.total_in_risk);
}

// Încearcă să înlocuiască ultimul element inclus din del cu primul element
// exclus din ins. Operează pe graful gc, pe care îl inițializează cu o copie
// a lui g.
//
```

```
// Returnează noul risc, sau INF dacă înlocuirea este imposibilă sau neoptimă.
long long try_swap_pair(int del, int ins, network_flow* gc, long long best_so_far) {
  if (del == ins ||
      tc.risks_in[del].empty() ||
      tc.risks_out[ins].empty()) {
    return INF;
  }

  int del_risk = *tc.risks_in[del].rbegin();
  int ins_risk = *tc.risks_out[ins].begin();
  long long new_risk = tc.total_in_risk - del_risk + ins_risk;

  if (new_risk >= best_so_far) {
    return INF; // Este o schimbare în rău.
  }

  // Acum are sens să încercăm să refacem fluxul.
  *gc = g;
  gc->remove_unit(del);
  gc->increase_source_capacity(ins);
  gc->ford_fulkerson();

  return (gc->flow == 4 * num_teams) ? new_risk : INF;
}

// Încearcă pe rînd să schimbe fiecare pereche (del, ins) unde first_del ≤ del
// ≤ last_del și first_ins ≤ ins ≤ last_ins. Unul dintre aceste intervale are
// lungime 0.
void try_swap(int first_del, int last_del, int first_ins, int last_ins) {
  int step_del = (last_del > first_del);
  int step_ins = (last_ins > first_ins);

  network_flow best_g = g, gc = g;
  long long best_risk = tc.total_in_risk;
  int best_del = 0, best_ins = 0;

  for (int del = first_del, ins = first_ins;
       (del <= last_del) && (ins <= last_ins);
       del += step_del, ins += step_ins) {
    long long new_risk = try_swap_pair(del, ins, &gc, best_risk);
    if (new_risk < best_risk) {
      best_risk = new_risk;
      best_g = gc;
      best_del = del;
      best_ins = ins;
    }
  }

  if (best_risk < tc.total_in_risk) {
    g = best_g;
```

```
    tc.move_out(best_del);
    tc.move_in(best_ins);
  }
}

void try_swap_out(int attr) {
  try_swap(attr, attr, FIRST_A, LAST_A);
}

void try_swap_in(int attr) {
  try_swap(FIRST_A, LAST_A, attr, attr);
}

void replace(int attr, int old_risk, int new_risk) {
  tc.replace_value(attr, old_risk, new_risk);
  tc.resort_sets(attr);
  if (new_risk > old_risk) {
    try_swap_out(attr);
  } else if (new_risk < old_risk) {
    try_swap_in(attr);
  }
}

void process_queries() {
  int num_queries, pid, risk;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &pid, &risk);
    pid--;
    replace(p[pid].attr, p[pid].risk, risk);
    p[pid].risk = risk;
    printf("%lld\n", tc.total_in_risk);
  }
}

int main() {
  read_people();
  find_num_teams();
  find_min_cost_teams();
  process_queries();

  return 0;
}
```

Anexa 33

Grafuri - Cuplaje

33.1 Problema School Dance (CSES)

înapoi

Sursă cu algoritmul Hopcroft-Karp.

```
// Algoritmul Hopcroft-Karp.
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 500;
const int INFINITY = 32'767;

struct left_node {
  std::vector<short> adj;
  short dist;
  short pair;
};

struct right_node {
  short pair;
};

left_node l[MAX_NODES + 1];
right_node r[MAX_NODES + 1];
int num_l, num_r;

void read_data() {
  int m, u, v;

  scanf("%d %d %d", &num_l, &num_r, &m);
  for (int i = 1; i <= m; i++) {
    scanf("%d %d", &u, &v);
```

```
    l[u].adj.push_back(v);
  }
}

std::queue<short> put_unmatched_in_queue() {
  std::queue<short> q;
  for (int u = 1; u <= num_l; u++) {
    if (l[u].pair) {
      l[u].dist = INFINITY;
    } else {
      l[u].dist = 0;
      q.push(u);
    }
  }
  return q;
}

bool bfs() {
  std::queue<short> q = put_unmatched_in_queue();
  bool reachable = false;

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v: l[u].adj) {
      int p = r[v].pair;
      if (!p) {
        reachable = true;
      } else if (l[p].dist == INFINITY) {
        l[p].dist = l[u].dist + 1;
        q.push(p);
      }
    }
  }

  return reachable;
}

bool dfs(int u) {
  for (int v: l[u].adj) {
    int p = r[v].pair;
    if (!p || ((l[p].dist == l[u].dist + 1) && dfs(p))) {
      l[u].pair = v;
      r[v].pair = u;
      return true;
    }
  }

  l[u].dist = INFINITY;
  return false;
```

```
}

int hopcroft_karp() {
  int size = 0;
  while (bfs()) {
    for (int u = 1; u <= num_l; u++) {
      if (!l[u].pair && dfs(u)) {
        size++;
      }
    }
  }
  return size;
}

void recover_matching() {
  for (int u = 1; u <= num_l; u++) {
    if (l[u].pair) {
      printf("%d %d\n", u, l[u].pair);
    }
  }
}

int main() {
  read_data();
  printf("%d\n", hopcroft_karp());
  recover_matching();

  return 0;
}
```

Sursă cu algoritmul lui Kuhn.

```
// Algoritmul lui Kuhn.
#include <stdio.h>
#include <vector>

const int MAX_N = 500;

struct left_node {
  std::vector<int> adj;
  bool vis;
};

left_node l[MAX_N + 1];
int r[MAX_N + 1];
int num_l, num_r;

void read_data() {
  int num_edges, u, v;
```

```
  scanf("%d %d %d", &num_l, &num_r, &num_edges);
  while (num_edges--) {
    scanf("%d %d", &u, &v);
    l[u].adj.push_back(v);
  }
}

void reset_visited() {
  for (int u = 1; u <= num_l; u++) {
    l[u].vis = false;
  }
}

bool dfs(int u) {
  if (l[u].vis) {
    return false;
  }
  l[u].vis = true;

  for (int v: l[u].adj) {
    if (!r[v] || dfs(r[v])) {
      r[v] = u;
      return true;
    }
  }

  return false;
}

int kuhn() {
  int size = 0;
  for (int u = 1; u <= num_l; u++) {
    reset_visited();
    size += dfs(u);
  }
  return size;
}

void write_solution(int max_matching) {
  printf("%d\n", max_matching);
  for (int u = 1; u <= num_r; u++) {
    if (r[u]) {
      printf("%d %d\n", r[u], u);
    }
  }
}

int main() {
  read_data();
  int max_matching = kuhn();
```

```
  write_solution(max_matching);

  return 0;
}
```

33.2 Problema Revoluție (ONI 2009, clasele 11-12)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 127;

bool a[MAX_N + 1][MAX_N + 1];
bool vis[MAX_N + 1];
unsigned char rpair[MAX_N + 1], cpair[MAX_N + 1];
unsigned char sol[MAX_N + 1];
int n, num_moves;

void read_data() {
  FILE* f = fopen("revolutie.in", "r");
  fscanf(f, "%d ", &n);
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      a[r][c] = (fgetc(f) == '1');
      fgetc(f);
    }
  }
  fclose(f);
}

void reset_visited() {
  for (int row = 1; row <= n; row++) {
    vis[row] = false;
  }
}

bool dfs(int row) {
  if (vis[row]) {
    return false;
  }
  vis[row] = true;

  for (int col = 1; col <= n; col++) {
    if (a[row][col] && (!cpair[col] || dfs(cpair[col]))) {
      rpair[row] = col;
      cpair[col] = row;
      return true;
```

```
    }
  }

  return false;
}

int kuhn() {
  int size = 0;
  for (int row = 1; row <= n; row++) {
    reset_visited();
    size += dfs(row);
  }
  return size;
}

void collect_moves() {
  for (int col = 1; col <= n; col++) {
    if (cpair[col] != col) {
      num_moves++;
      int r = cpair[col];
      sol[col] = r;
      rpair[r] = rpair[col];
      cpair[rpair[r]] = r;
    }
  }
}

void recover_solution(int max_matching) {
  FILE* f = fopen("revolutie.out", "w");
  if (max_matching < n) {
    fprintf(f, "-1\n");
  } else {
    collect_moves();
    fprintf(f, "%d\n", num_moves);
    for (int c = 1; c <= n; c++) {
      if (sol[c]) {
        fprintf(f, "L %d %d\n", c, sol[c]);
      }
    }
  }
  fclose(f);
}

int main() {
  read_data();
  int max_matching = kuhn();
  recover_solution(max_matching);
  return 0;
}
```

33.3 Problema Valuable Paper (Codeforces)

‹ înapoi • versiune online

```cpp
// Metodă: Caută binar greutatea maximă necesară și rulează algoritmul
// Hopcroft-Karp pentru fiecare candidat.
#include <algorithm>
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_NODES = 10'000;
const int MAX_EDGES = 100'000;
const int INF_DIST = 32'767;
const int INF_WEIGHT = 1'000'000'001;

struct edge {
  short v;
  int weight;
};

struct left_node {
  std::vector<edge> adj;
  short dist;
  short pair;
};

struct right_node {
  short pair;
};

int weights[MAX_EDGES + 1];
left_node l[MAX_NODES + 1];
right_node r[MAX_NODES + 1];
int num_nodes, num_edges;
int max_weight;

void read_data() {
  short u, v;
  int weight;

  scanf("%d %d", &num_nodes, &num_edges);
  for (int i = 0; i < num_edges; i++) {
    scanf("%hd %hd  %d", &u, &v, &weight);
    l[u].adj.push_back({v, weight});
    weights[i] = weight;
  }
}
```

```
void sort_weights() {
  std::sort(weights, weights + num_edges);
  for (int u = 1; u <= num_nodes; u++) {
    std::sort(l[u].adj.begin(), l[u].adj.end(), [](edge a, edge b) {
      return a.weight < b.weight;
    });
    l[u].adj.push_back({0, INF_WEIGHT}); // santinelă
  }
}

std::queue<short> put_unmatched_in_queue() {
  std::queue<short> q;
  for (int u = 1; u <= num_nodes; u++) {
    if (l[u].pair) {
      l[u].dist = INF_DIST;
    } else {
      l[u].dist = 0;
      q.push(u);
    }
  }
  return q;
}

bool bfs() {
  std::queue<short> q = put_unmatched_in_queue();
  bool reachable = false;

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int i = 0; l[u].adj[i].weight <= max_weight; i++) {
      int v = l[u].adj[i].v;
      int p = r[v].pair;
      if (!p) {
        reachable = true;
      } else if (l[p].dist == INF_DIST) {
        l[p].dist = l[u].dist + 1;
        q.push(p);
      }
    }
  }

  return reachable;
}

bool dfs(int u) {
  for (int i = 0; l[u].adj[i].weight <= max_weight; i++) {
    int v = l[u].adj[i].v;
    int p = r[v].pair;
    if (!p || ((l[p].dist == l[u].dist + 1) && dfs(p))) {
```

```
      l[u].pair = v;
      r[v].pair = u;
      return true;
    }
  }

  l[u].dist = INF_DIST;
  return false;
}

bool hopcroft_karp() {
  for (int u = 1; u <= num_nodes; u++) {
    l[u].pair = r[u].pair = 0;
  }

  int size = 0;
  while (bfs()) {
    for (int u = 1; u <= num_nodes; u++) {
      if (!l[u].pair && dfs(u)) {
        size++;
      }
    }
  }
  return (size == num_nodes);
}

int bin_search_max_weight() {
  // lo = Număr de muchii cunoscut ca fiind imposibil.
  // hi = Număr de muchii cunoscut a fi posibil, cu o mențiune. Pornim cu hi =
  // num_edges, care înseamnă „chiar dacă folosim toate muchiile, tot nu găsim
  // un cuplaj maxim”. Subliniem că nu accesăm niciodată weights[num_edges].
  int lo = num_nodes - 2, hi = num_edges;
  while (hi - lo > 1) {
    int mid = (lo + hi) / 2;
    max_weight = weights[mid];
    if (hopcroft_karp()) {
      hi = mid;
    } else {
      lo = mid;
    }
  }

  return (hi == num_edges) ? -1 : weights[hi];
}

int main() {
  read_data();
  sort_weights();
  int result = bin_search_max_weight();
  printf("%d\n", result);
```

```
  return 0;
}
```

33.4 Problema Teroriști (Baraj ONI 2010)

înapoi • versiune online

```
// Metodă: Construim un graf cu sursă, destinație și două coloane de 26 de
// noduri și respectiv 26*27/2 noduri. Rulăm algoritmul lui Dinic.
#include <queue>
#include <stdio.h>
#include <vector>

const int MAX_WORD = 1'000'000;

const int SIGMA = 26;
const int NUM_DIGRAMS = SIGMA * (SIGMA + 1) / 2;
const int MAX_NODES = 1 + SIGMA + NUM_DIGRAMS + 1;
const int MAX_EDGES = SIGMA + SIGMA * SIGMA + NUM_DIGRAMS;
const int NONE = -1;
const int INFINITY = 1'000'000'001;

// Pentru fiecare digramă între 0 și 26*27/2-1, stochează frecvența sa și o
// listă de poziții pe care apare în fișierul de intrare.
struct digrams {
  int freq[NUM_DIGRAMS];
  std::vector<int> pos[NUM_DIGRAMS];
  short code[SIGMA][SIGMA];

  void init() {
    int k = 0;
    for (int i = 0; i < SIGMA; i++) {
      for (int j = i; j < SIGMA; j++) {
        code[i][j] = code[j][i] = k++;
      }
    }
  }

  void push(char c1, char c2, int line) {
    int k = code[c1 - 'a'][c2 - 'a'];
    freq[k]++;
    pos[k].push_back(line);
  }

  int pop(int code) {
    int result = pos[code].back();
    pos[code].pop_back();
```

```
    return result;
  }
};

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short ptr;
  short dist;

  bool is_reachable() {
    return dist != NONE;
  }
};

int min(int x, int y) {
  return (x < y) ? x : y;
}

struct graph {
  edge e[2 * MAX_EDGES];
  node n[MAX_NODES + 1];
  int num_nodes, source, sink;

  void add_edge(short u, short v, int cap) {
    static int pos = 0;

    e[pos] = { cap, v, n[u].adj };
    n[u].adj = pos++;
  }

  void add_edge_pair(short u, short v, int cap) {
    add_edge(u, v, cap);
    add_edge(v, u, 0);
  }

  void init(int* letter_freq, digrams dig) {
    num_nodes = SIGMA + NUM_DIGRAMS + 2;
    source = num_nodes - 2;
    sink = num_nodes - 1;

    for (int u = 0; u < num_nodes; u++) {
      n[u].adj = NONE;
    }

    for (int i = 0; i < SIGMA; i++) {
```

```
    add_edge_pair(source, i, letter_freq[i]);
    for (int j = 0; j < SIGMA; j++) {
      int code = dig.code[i][j];
      add_edge_pair(i, code + SIGMA, INFINITY);
    }
  }

  for (int i = 0; i < NUM_DIGRAMS; i++) {
    add_edge_pair(i + SIGMA, sink, dig.freq[i]);
  }
}

bool bfs_reachable() {
  for (int u = 0; u < num_nodes; u++) {
    n[u].dist = NONE;
  }
  n[source].dist = 0;

  std::queue<short> q;
  q.push(source);

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int pos = n[u].adj; pos != NONE; pos = e[pos].next) {
      int v = e[pos].v;
      if (!n[v].is_reachable() && e[pos].cap) {
        n[v].dist = n[u].dist + 1;
        q.push(v);
      }
    }
  }

  return n[sink].is_reachable();
}

void reset_dfs_pointers() {
  for (int u = 0; u < num_nodes; u++) {
    n[u].ptr = n[u].adj;
  }
}

int dfs(int u, int incoming_flow) {
  if (u == sink) {
    return incoming_flow;
  }

  int result = 0;
  while (!result && (n[u].ptr != NONE)) {
    int p = n[u].ptr;
```

```
      int v = e[p].v, cap = e[p].cap;
      if (cap && (n[v].dist == n[u].dist + 1)) {
        result = dfs(v, min(cap, incoming_flow));
        e[p].cap -= result;
        e[p ^ 1].cap += result;
      }
      // Rămîi pe aceeași muchie dacă o mai putem folosi.
      if (!result) {
        n[u].ptr = e[p].next;
      }
    }

    return result;
  }

  void dinic() {
    while (bfs_reachable()) {
      reset_dfs_pointers();
      while (dfs(source, INFINITY)) {
      }
    }
  }

  int get_digram(char c) {
    // Caută o muchie reziduală dinspre dreapta (dinspre digrame). Scoate o
    // unitate de flux de pe ea și returnează codul digramei.
    int u = c - 'a';
    short& p = n[u].ptr;
    while (!(p ^ 1) || !e[p ^ 1].cap) {
      p = e[p].next;
    }
    e[p ^ 1].cap--;
    return e[p].v - SIGMA;
  }
};

graph g;
char word[MAX_WORD + 1];
digrams dig;
int letter_freq[SIGMA];

void read_data() {
  int ignored, num_pairs;

  FILE* f = fopen("teroristi.in", "r");
  fscanf(f, "%d %d %s ", &ignored, &num_pairs, word);

  for (int line = 1; line <= num_pairs; line++) {
    char c1, c2;
    fscanf(f, "%c %c ", &c1, &c2);
```

```
    dig.push(c1, c2, line);
  }
  fclose(f);
}

void compute_letter_frequencies() {
  for (char* s = word; *s; s++) {
    letter_freq[*s - 'a']++;
  }
}

void recover_solution() {
  g.reset_dfs_pointers();
  FILE* f = fopen("teroristi.out", "w");
  for (char* s = word; *s; s++) {
    int code = g.get_digram(*s);
    int line = dig.pop(code);
    fprintf(f, "%d ", line);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  dig.init();
  read_data();
  compute_letter_frequencies();
  g.init(letter_freq, dig);
  g.dinic();
  recover_solution();

  return 0;
}
```

33.5 Problema Brevity Is the Soul of Wit (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>

const int SIGMA = 27;
const int MAX_LEFT = 200;
const int MAX_RIGHT = 531'441; // 27^4
const int MAX_EDGES = 17'901 * MAX_LEFT; // C(26,4)+C(26,3)+C(26,2)+C(26,1)
const int MAX_LENGTH = 10;

struct edge {
  int v, next;
```

```
};

struct left_node {
  int adj;
  int pair; // nenecesar pentru Kuhn, dar simplifică recuperarea soluției
  char word[MAX_LENGTH + 1];
  bool vis;
};

edge e[MAX_EDGES + 1];
left_node l[MAX_LEFT + 1];
unsigned char r[MAX_RIGHT + 1];
int n;

void read_data() {
  FILE* f = fopen("input.txt", "r");
  fscanf(f, "%d", &n);
  for (int i = 1; i <= n; i++) {
    fscanf(f, "%s", l[i].word);
  }
  fclose(f);
}

void add_edge(int u, int v) {
  static int ptr = 1;
  e[ptr] = { v, l[u].adj };
  l[u].adj = ptr++;
}

void build_graph() {
  for (int u = 1; u <= n; u++) {
    char* s = l[u].word;
    for (int a = 0; s[a] != '\0'; a++) {
      int code1 = s[a] - 'a' + 1;
      add_edge(u, code1);
      for (int b = a + 1; s[b] != '\0'; b++) {
        int code2 = code1 * SIGMA + s[b] - 'a' + 1;
        add_edge(u, code2);
        for (int c = b + 1; s[c] != '\0'; c++) {
          int code3 = code2 * SIGMA + s[c] - 'a' + 1;
          add_edge(u, code3);
          for (int d = c + 1; s[d] != '\0'; d++) {
            int code4 = code3 * SIGMA + s[d] - 'a' + 1;
            add_edge(u, code4);
          }
        }
      }
    }
  }
}
```

```
void reset_visited() {
  for (int u = 1; u <= n; u++) {
    l[u].vis = false;
  }
}

bool kuhn_dfs(int u) {
  if (l[u].vis) {
    return false;
  }
  l[u].vis = true;

  for (int ptr = l[u].adj; ptr; ptr = e[ptr].next) {
    int v = e[ptr].v;
    if (!r[v] || kuhn_dfs(r[v])) {
      l[u].pair = v;
      r[v] = u;
      return true;
    }
  }

  return false;
}

bool kuhn() {
  int u = 1;
  while ((u <= n) && kuhn_dfs(u)) {
    reset_visited();
    u++;
  }

  return (u > n);
}

void decode(int code, char* dst) {
  int len = 0;
  while (code) {
    dst[len++] = code % SIGMA + 'a' - 1;
    code /= SIGMA;
  }
  dst[len] = '\0';
  for (int i = 0, j = len - 1; i < j; i++, j--) {
    char tmp = dst[i];
    dst[i] = dst[j];
    dst[j] = tmp;
  }
}

void recover_solution(FILE* f) {
```

```
  char s[MAX_LENGTH + 1];
  for (int u = 1; u <= n; u++) {
    decode(l[u].pair, s);
    fprintf(f, "%s\n", s);
  }
}

int main() {
  read_data();
  build_graph();
  bool has_solution = kuhn();

  FILE* f = fopen("output.txt", "w");
  if (has_solution) {
    recover_solution(f);
  } else {
    fprintf(f, "-1\n");
  }
  fclose(f);

  return 0;
}
```

33.6 Problema Access Levels (Codeforces)

înapoi • versiune online

```
#include <stdio.h>

const int MAX_USERS = 500;
const int MAX_DOCUMENTS = 500;
const int NONE = -1;

int num_users, num_documents;

struct t_document {
  bool users[MAX_USERS];
  bool incl[MAX_DOCUMENTS];

  // Date pentru algoritmul lui Kuhn
  short match;    // cu ce document a fost împerecheat
  bool has_match; // true dacă și numai dacă alt document ne consideră pereche
  bool vis;       // bit de vizitare

  short group, level;

  bool is_included_in(t_document& other, bool tiebreaker) {
    bool equal = true;
```

```
    for (int u = 0; u < num_users; u++) {
      if (users[u] > other.users[u]) {
        return false;
      }
      equal &= (users[u] == other.users[u]);
    }
    return !equal || tiebreaker;
  }
};

struct t_user {
  int level[MAX_DOCUMENTS];
};

t_document doc[MAX_DOCUMENTS];
t_user user[MAX_USERS];

void read_data() {
  scanf("%d %d ", &num_users, &num_documents);
  for (int u = 0; u < num_users; u++) {
    for (int d = 0; d < num_documents; d++) {
      doc[d].users[u] = (getchar() == '1');
    }
    getchar();
  }
}

void compute_inclusions() {
  for (int i = 0; i < num_documents; i++) {
    for (int j = 0; j < num_documents; j++) {
      doc[i].incl[j] = doc[i].is_included_in(doc[j], (i < j));
    }
  }
}

void reset_visited() {
  for (int i = 0; i < num_documents; i++) {
    doc[i].vis = false;
  }
}

bool kuhn_dfs(int u) {
  if (doc[u].vis) {
    return false;
  }

  doc[u].vis = true;
  for (int v = 0; v < num_documents; v++) {
    if (doc[u].incl[v]) {
      if ((doc[v].match == NONE) || kuhn_dfs(doc[v].match)) {
```

```
        doc[v].match = u;
        return true;
      }
    }
  }

  return false;
}

void kuhn_matching() {
  for (int d = 0; d < num_documents; d++) {
    doc[d].match = NONE;
  }

  for (int d = 0; d < num_documents; d++) {
    reset_visited();
    kuhn_dfs(d);
  }
}

void build_group(int d, int group) {
  // Dă-le utilizatorilor nivelul 1, iar documentelor niveluri începînd cu 2.
  // Astfel, implicit nimeni nu poate vedea nimic.
  for (int u = 0; u < num_users; u++) {
    user[u].level[group] = 1;
  }

  int level = 2;
  while (d != NONE) {
    for (int u = 0; u < num_users; u++) {
      if (doc[d].users[u]) {
        // Oferă acces doar la nevoie.
        user[u].level[group] = level;
      }
    }
    doc[d].group = group;
    doc[d].level = level++;
    d = doc[d].match;
  }
}

int build_groups() {
  for (int d = 0; d < num_documents; d++) {
    if (doc[d].match != NONE) {
      doc[doc[d].match].has_match = true;
    }
  }

  int group = 0;
  for (int d = 0; d < num_documents; d++) {
```

```
    if (!doc[d].has_match) {
      build_group(d, group++);
    }
  }

  return group;
}

void write_solution(int num_groups) {
  printf("%d\n", num_groups);

  for (int d = 0; d < num_documents; d++) {
    printf("%d ", doc[d].group + 1);
  }
  printf("\n");

  for (int d = 0; d < num_documents; d++) {
    printf("%d ", doc[d].level);
  }
  printf("\n");

  for (int u = 0; u < num_users; u++) {
    for (int g = 0; g < num_groups; g++) {
      printf("%d ", user[u].level[g]);
    }
    printf("\n");
  }
}

int main() {
  read_data();
  compute_inclusions();
  kuhn_matching();
  int num_groups = build_groups();
  write_solution(num_groups);

  return 0;
}
```

33.7 Problema Pswap (Baraj ONI 2021)

‹ înapoi • versiune online

```
#include <ctype.h>
#include <stdio.h>
#include <vector>

const int MAX_N = 2'500;
```

```
const int MAX_M = 5'000;
const int HASH_BASE = 5'113;

struct fast_input {
  static const int BUF_SIZE = 1 << 20;
  char buf[BUF_SIZE + 1];
  int pos = BUF_SIZE;

  char get_char() {
    if (pos == BUF_SIZE) {
      int read = fread(buf, 1, BUF_SIZE, stdin);
      buf[read] = '\0';
      pos = 0;
    }
    return buf[pos++];
  }

  int read_int() {
    int result = 0;
    char c;
    do {
      c = get_char();
    } while (!isdigit(c));

    do {
      result = 10 * result + c - '0';
      c = get_char();
    } while (isdigit(c));

    return result;
  }
};

// Aceleași date pentru nodurile stîngi și drepte, deoarece deocamdată nu știm
// care este care.
struct node {
  std::vector<short> adj;
  short pair;
  bool is_left;
  bool vis;
};

short p[MAX_N + 1][MAX_M + 1];
short inv[MAX_N + 1][MAX_M + 1]; // permutările inverse
short diff[MAX_N + 1][MAX_N + 1]; // prima diferență
unsigned h[MAX_N + 1][MAX_M + 1];
int ord[MAX_N];
unsigned pow_b[MAX_M + 1];
node nd[MAX_N + 1];
int n, m;
```

```
void read_and_hash_data() {
  fast_input f;
  n = f.read_int();
  m = f.read_int();
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      p[i][j] = f.read_int();
      inv[i][p[i][j]] = j;
      h[i][j] = h[i][j - 1] * HASH_BASE + p[i][j];
    }
  }
}

void precompute_powers() {
  pow_b[0] = 1;
  for (int i = 1; i <= m; i++) {
    pow_b[i] = pow_b[i - 1] * HASH_BASE;
  }
}

// Returnează true dacă și numai dacă p[a] este pară.
bool is_even(int a) {
  bool result = true;

  // Numără lungimile ciclurilor. Adaugă m la fiecare element pentru a-l
  // marca.
  for (int i = 0; i < m; i++) {
    if (p[a][i] < m) {
      int j = i, len = 0;
      do {
        p[a][j] += m;
        j = p[a][j] - m;
        len++;
      } while (j != i);
      result ^= !(len & 1); // ciclurile pare schimbă paritatea
    }
  }

  // Refă p[a].
  for (int i = 0; i < m; i++) {
    p[a][i] -= m;
  }

  return result;
}

void compute_parities() {
  for (int i = 1; i <= n; i++) {
    nd[i].is_left = is_even(i);
```

```
  }
}

void swap(int i, int j) {
  int tmp = ord[i];
  ord[i] = ord[j];
  ord[j] = tmp;
}

void mark_differences(int first, int left, int right, int last, int pos) {
  for (int i = first; i < left; i++) {
    for (int j = left; j < last; j++) {
      diff[ord[i]][ord[j]] = pos;
      diff[ord[j]][ord[i]] = pos;
    }
  }
  for (int i = left; i < right; i++) {
    for (int j = right; j < last; j++) {
      diff[ord[i]][ord[j]] = pos;
      diff[ord[j]][ord[i]] = pos;
    }
  }
}

// Sortează permutările ord[first..last), care sînt egale pînă la poziția
// pos-1.
void ternary_sort(int first, int last, int pos) {
  if (last - first <= 1) {
    return;
  }

  int i = first, left = first, right = last;
  short pivot = p[ord[first]][pos];

  // Invariant de buclă:
  // * ord[first ... left)     poziția pos este < pivot
  // * ord[left ... i)         poziția pos este == pivot
  // * ord[i]                  permutarea în curs de evaluare
  // * ord(i ... right)        permutări încă neevaluate
  // * ord[right ... last)     poziția pos este > pivot
  while (i < right) {
    short x = p[ord[i]][pos];
    if (x < pivot) {
      swap(i++, left++);
    } else if (x > pivot)  {
      swap(i, --right);
    } else {
      i++;
    }
  }
```

```
  mark_differences(first, left, right, last, pos);

  ternary_sort(first, left, pos);
  if (pos < m) {
    ternary_sort(left, right, pos + 1);
  }
  ternary_sort(right, last, pos);
}

void compute_first_diff() {
  for (int i = 0; i < n; i++) {
    ord[i] = i + 1;
  }
  ternary_sort(0, n, 1);
}

unsigned range_hash(int i, int l, int r) {
  return h[i][r] - h[i][l - 1] * pow_b[r - l + 1];
}

// Returnează true dacă și numai dacă p[i][l..r] == p[j][l..r].
bool hash_equal(int i, int j, int l, int r) {
  return range_hash(i, l, r) == range_hash(j, l, r);
}

bool similar_perm(int i, int j) {
  if (nd[i].is_left == nd[j].is_left) {
    return false;
  }

  // Dacă p[i][first] = 4 și p[j][first] = 10, ne întrebăm unde apare 10 în
  // p[i]. Aceste două diferențe trebuie să fie singurele.
  int first = diff[i][j];
  int last = inv[i][p[j][first]];
  return hash_equal(i, j, first + 1, last - 1) &&
    hash_equal(i, j, last + 1, m);
}

void build_graph() {
  for (int i = 1; i < n; i++) {
    for (int j = i + 1; j <= n; j++) {
      if (similar_perm(i, j)) {
        nd[i].adj.push_back(j);
        nd[j].adj.push_back(i);
      }
    }
  }
}
```

```
void clear_vis() {
  for (int u = 1; u <= n; u++) {
    nd[u].vis = false;
  }
}

bool dfs(int u) {
  if (nd[u].vis) {
    return false;
  }
  nd[u].vis = true;

  for (int v: nd[u].adj) {
    if (!nd[v].pair || dfs(nd[v].pair)) {
      nd[v].pair = u;
      return true;
    }
  }
  return false;
}

int kuhn() {
  int size = 0;
  for (int u = 1; u <= n; u++) {
    if (nd[u].is_left) {
      clear_vis();
      size += dfs(u);
    }
  }
  return size;
}

int main() {
  read_and_hash_data();
  precompute_powers();
  compute_parities();
  compute_first_diff();
  build_graph();
  int max_matching = kuhn();
  printf("%d\n", n - max_matching);
}
```

Anexa 34

Grafuri - Fluxuri de cost minim

34.1 Problema Parcel Delivery (CSES)

înapoi

Sursă cu algoritmul lui Dijkstra pentru augmentare (DFS).

```
#include <algorithm>
#include <queue>
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int INF = 1'000'000'000;
const int NIL = -1;

struct edge {
  short cap, cost;
  short v, next;
};

struct node {
  short adj;
  int d;              // distanța de sursă
  short parent, edge; // părintele și muchia incidentă în SPFA
  bool in_queue;
};

node nd[MAX_NODES + 1];
edge e[2 * MAX_EDGES];
std::queue<int> q;
int n, desired_flow, total_cost;

void add_edge(short u, short v, short cap, short cost) {
  static int pos = 0;
```

```
  e[pos] = { cap, cost, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d %d", &n, &m, &desired_flow);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, cap, cost;
    scanf("%d %d %d %d", &u, &v, &cap, &cost);
    add_edge(u, v, cap, cost);
    add_edge(v, u, 0, -cost);
  }
}

void enqueue(int u) {
  if (!nd[u].in_queue) {
    nd[u].in_queue = true;
    q.push(u);
  }
}

int dequeue() {
  int u = q.front();
  q.pop();
  nd[u].in_queue = false;
  return u;
}

void spfa() {
  for (int u = 1; u <= n; u++) {
    nd[u].d = INF;
    nd[u].in_queue = false;
  }

  nd[1].d = 0;
  enqueue(1);

  while (!q.empty()) {
    int u = dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if (e[pos].cap && (nd[u].d + e[pos].cost < nd[v].d)) {
        nd[v].d = nd[u].d + e[pos].cost;
        nd[v].parent = u;
        nd[v].edge = pos;
```

```
        enqueue(v);
      }
    }
  }
}

int path_minimum() {
  short min_cap = desired_flow;
  int u = n;
  while (u != 1) {
    min_cap = std::min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void augment_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

void min_cost_max_flow() {
  while (desired_flow && (nd[n].d < INF)) {
    spfa();
    int flow = path_minimum();
    augment_path(flow);
    desired_flow -= flow;
    total_cost += flow * nd[n].d;
  }
}

int main() {
  read_data();
  min_cost_max_flow();
  printf("%d\n", (nd[n].d == INF) ? -1 : total_cost);

  return 0;
}
```

34.2 Problema Task Assignment (CSES)

‹ înapoi

Sursă cu algoritmul lui Dijkstra pentru augmentare (DFS).

```cpp
#include <queue>
#include <stdio.h>

const int MAX_N = 402; // 2n + 2
const int INF = 1'000'000;

struct node {
  int d;   // distanța de sursă
  short p; // părintele în SPFA
  bool in_queue;
};

node nd[MAX_N];
bool cap[MAX_N][MAX_N];
short cost[MAX_N][MAX_N];
std::queue<int> q;
int num_people, n, source, sink;
int total_cost;

void add_edge(int u, int v, int c) {
  cost[u][v] = c;
  cost[v][u] = -c;
  cap[u][v] = true;
  cap[v][u] = false;
}

void read_data_and_build_graph() {
  scanf("%d", &num_people);

  n = 2 * num_people + 2;
  source = 0;
  sink = n - 1;

  for (int u = 1; u <= num_people; u++) {
    add_edge(source, u, 0);
    add_edge(u + num_people, sink, 0);
    for (int v = num_people + 1; v <= 2 * num_people; v++) {
      int c;
      scanf("%d", &c);
      add_edge(u, v, c);
    }
  }
}

void relax_range(int u, int vstart, int vend) {
  for (int v = vstart; v <= vend; v++) {
    if (cap[u][v] && (nd[u].d + cost[u][v] < nd[v].d)) {
      nd[v].d = nd[u].d + cost[u][v];
      nd[v].p = u;
```

```
      if (!nd[v].in_queue) {
        nd[v].in_queue = true;
        q.push(v);
      }
    }
  }
}

void relax_neighbors(int u) {
  if (u == source) {
    relax_range(source, 1, num_people);
  } else if (u <= num_people) {
    relax_range(u, source, source);
    relax_range(u, num_people + 1, 2 * num_people);
  } else if (u <= 2 * num_people) {
    relax_range(u, 1, num_people);
    relax_range(u, sink, sink);
  } else { // u == sink
    relax_range(u, num_people + 1, 2 * num_people);
  }
}

void spfa() {
  for (int u = 1; u < n; u++) {
    nd[u] = { .d = INF, .in_queue = false };
  }

  nd[source] = { .d = 0, .in_queue = true };
  q.push(source);

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    nd[u].in_queue = false;
    relax_neighbors(u);
  }
}

void augment_path() {
  int u = sink;
  do {
    int p = nd[u].p;
    cap[p][u] = false;
    cap[u][p] = true;
    u = p;
  } while (u != source);
  total_cost += nd[sink].d;
}

void min_cost_max_flow() {
```

```
  for (int i = 0; i < num_people; i++) {
    spfa();
    augment_path();
  }
}

void write_assignment() {
  printf("%d\n", total_cost);
  for (int u = 1; u <= num_people; u++) {
    int v = num_people + 1;
    while (cap[u][v]) {
      v++;
    }
    printf("%d %d\n", u, v - num_people);
  }
}

int main() {
  read_data_and_build_graph();
  min_cost_max_flow();
  write_assignment();

  return 0;
}
```

34.3 Problema Match (IIOT 2019-2020 runda 3)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 252; // m + n + 2
const int MAX_EDGES = 1'400; // 2 × (m + 2m + 2n)
const int NIL = -1;

struct edge {
  short cap, cost;
  short v, next;
};

struct node {
  short adj;
  bool vis;
};

node nd[MAX_NODES + 1];
edge e[2 * MAX_EDGES];
int n, m, source, sink, max_flow, min_cost, epos;
```

```
void add_edge(short u, short v, short cap, short cost) {
  e[epos] = { cap, cost, v, nd[u].adj };
  nd[u].adj = epos++;
}

void add_edge_pair(short u, short v, short cap, short cost) {
  add_edge(u, v, cap, cost);
  add_edge(v, u, 0, -cost);
}

void read_data_and_make_graph() {
  epos = 0;
  scanf("%d %d", &n, &m);
  source = m + n;
  sink = m + n + 1;
  for (int u = 0; u < m + n + 2; u++) {
    nd[u].adj = NIL;
  }

  // Creează întîi muchiile de la jucători la destinație, ca să le aflăm ușor
  // ID-urile cînd le vom manipula capacitățile.
  for (int u = 0; u < n; u++) {
    add_edge_pair(m + u, sink, 0, 0); // vor fi folosite doar în FMCM
    add_edge_pair(m + u, sink, 0, 1); // capacitatea va crește în faza I
  }
  for (int i = 0; i < m; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge_pair(source, i, 1, 0);
    add_edge_pair(i, m + u - 1, 1, 0);
    add_edge_pair(i, m + v - 1, 1, 0);
  }
}

// Crește capacitățile pe muchiile jucător-destinație de cost 1.
void increase_sink_capacities() {
  for (int u = 0; u < n; u++) {
    e[4 * u + 2].cap++;
  }
}

void reset_dfs() {
  for (int u = 0; u < m + n + 2; u++) {
    nd[u].vis = false;
  }
}

// Caută un drum spre destinație. Dacă îl găsește, ajustează capacitățile.
bool dfs(int u, bool cost_zero_only) {
```

```
  nd[u].vis = true;
  if (u == sink) {
    return true;
  }

  bool found = false;
  int pos = nd[u].adj;
  while (!found && (pos != NIL)) {
    int v = e[pos].v;
    if (e[pos].cap && !nd[v].vis && (!cost_zero_only || !e[pos].cost)) {
      found = dfs(v, cost_zero_only);
      e[pos].cap -= found;
      e[pos ^ 1].cap += found;
    }
    pos = e[pos].next;
  }

  return found;
}

void ford_fulkerson() {
  bool augment;
  do {
    reset_dfs();
    augment = dfs(source, false);
    max_flow += augment;
  } while (augment);
}

// Capacitățile pe muchiile de la jucători la destinație semnifică punctajul
// maxim permis. Crește simultan aceste capacități cu cîte o unitate pînă cînd
// poate obține un flux de mărime cel puțin m.
int find_min_max_score() {
  int result = 0;
  max_flow = 0;

  do {
    result++;
    increase_sink_capacities();
    ford_fulkerson();
  } while (max_flow < m);

  return result;
}

// Crește capacitatea muchiilor jucător-destinație de cost 0 și o scade pe cea
// a muchiilor de cost 1.
void increase_cost_0_capacities() {
  for (int u = 0; u < n; u++) {
    // 4u, 4u+1 : muchia de cost 0 + reziduala
```

```
    // 4u+2, 4u+3: muchia de cost 1 + reziduala
    if (e[4 * u + 3].cap) {
      // Există capacitate reziduală pe muchia de cost 1, deci putem muta
      // această unitate de flux pe muchia de cost 0.
      e[4 * u + 3].cap--;
      e[4 * u + 1].cap++;
      min_cost--;
    } else {
      // Deja pompăm tot fluxul pe muchia de cost 0, deci doar reglăm
      // capacitățile.
      e[4 * u + 2].cap--;
      e[4 * u].cap++;
    }
  }
}

// Caută un ciclu de cost negativ care conține destinația. Dacă îl găsește,
// actualizează fluxul.
bool find_cycle(int u) {
  reset_dfs();
  return dfs(u, true);
}

void find_negative_cost_cycles() {
  int pos = nd[sink].adj;
  while (pos != NIL) {
    if ((e[pos].cost == -1) && e[pos].cap && find_cycle(e[pos].v)) {
      e[pos].cap--;
      e[pos ^ 1].cap++;
      min_cost--;
    } else {
      pos = e[pos].next;
    }
  }
}

// Crește simultan capacitățile pe muchiile jucător-sursă de cost 0 cît timp
// le poate satura.
int find_max_min_score(int max) {
  int x = 0;
  min_cost = m; // faza I a pompat flux doar pe muchii de cost 1
  do {
    x++;
    increase_cost_0_capacities();
    find_negative_cost_cycles();
  } while (min_cost == m - n * x);
  return x - 1;
}

void solve_test() {
```

```
  read_data_and_make_graph();
  int min_max_score = find_min_max_score();
  int max_min_score = find_max_min_score(min_max_score);
  printf("%d %d\n", min_max_score, max_min_score);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

34.4 Problema Hoață (Baraj ONI 2022)

înapoi

Implementare cu SPFA (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_ROOMS = 300;
const int MAX_WEIGHT = 300;
const int MAX_NODES = MAX_ROOMS * (MAX_WEIGHT + 1) + 1;
const int MAX_EDGES = MAX_NODES * 4; // două muchii per cameră + reziduale
const int INF = 1'000'000'000;

struct edge {
  int cap, cost;
  int v, next;
};

struct node {
  int adj;
  int d;            // distanța de la sursă
  int parent, edge; // părintele și muchia incidentă în SPFA
};

struct queue {
  int v[MAX_NODES];
  bool in_queue[MAX_NODES];
  int head, tail;

  bool is_empty() {
```

```
    return head == tail;
  }

  void push(int u) {
    if (!in_queue[u]) {
      in_queue[u] = true;
      v[tail++] = u;
      if (tail == MAX_NODES) {
        tail = 0;
      }
    }
  }

  int pop() {
    int u = v[head++];
    in_queue[u] = false;
    if (head == MAX_NODES) {
      head = 0;
    }
    return u;
  }
};

node nd[MAX_NODES];
edge e[MAX_EDGES + 2];
queue q;
int n, num_thieves, source, sink, total_cost;
int epos;

void add_edge(int u, int v, int cap, int cost) {
  e[epos] = { cap, cost, v, nd[u].adj };
  nd[u].adj = epos++;
}

void add_edge_pair(int u, int v, int cap, int cost) {
  add_edge(u, v, cap, cost);
  add_edge(v, u, 0, -cost);
}

void read_data_and_build_graph() {
  epos = 2; // lasă 0/1 pentru NIL
  total_cost = 0;

  int num_rooms, g, val, weight, alarm;
  scanf("%d %d %d", &num_rooms, &num_thieves, &g);
  n = num_rooms * (g + 1) + 1;
  source = 0;
  sink = n - 1;
  for (int u = 0; u < n; u++) {
    nd[u].adj = 0;
```

```
  }

  for (int r = 0; r < num_rooms; r++) {
    scanf("%d %d %d", &val, &weight, &alarm);
    for (int w = 0; w <= g; w++) {
      int u = r * (g + 1) + w;

      if (w + weight <= g) {
        add_edge_pair(u, u + weight, num_thieves, -val);
      }

      int v = (r < num_rooms - 1) ? (u + (g + 1)) : sink;
      add_edge_pair(u, v, alarm, 0);
    }
  }
}

void spfa() {
  for (int u = 0; u < n; u++) {
    nd[u].d = INF;
  }

  nd[source].d = 0;
  q.push(source);

  while (!q.is_empty()) {
    int u = q.pop();
    for (int pos = nd[u].adj; pos; pos = e[pos].next) {
      int v = e[pos].v;
      if (e[pos].cap && (nd[u].d + e[pos].cost < nd[v].d)) {
        nd[v].d = nd[u].d + e[pos].cost;
        nd[v].parent = u;
        nd[v].edge = pos;
        q.push(v);
      }
    }
  }
}

int path_minimum() {
  int min_cap = num_thieves;
  int u = sink;
  while (u != source) {
    min_cap = std::min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void augment_path(int flow) {
```

```
  int u = sink;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

void min_cost_max_flow() {
  do {
    spfa();
    int flow = path_minimum();
    augment_path(flow);
    total_cost += flow * nd[sink].d;
    num_thieves -= flow;
  } while (num_thieves && (nd[sink].d < INF));
}

void solve_test() {
  read_data_and_build_graph();
  min_cost_max_flow();
  printf("%d\n", (nd[sink].d == INF) ? -1 : -total_cost);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

Implementare cu algoritmii lui Johnson și Dijkstra (versiune online).

```
// Nu este nevoie de Bellman-Ford, întrucît graful original este un dag.
#include <algorithm>
#include <queue>
#include <stdio.h>

const int MAX_ROOMS = 300;
const int MAX_WEIGHT = 300;
const int MAX_NODES = MAX_ROOMS * (MAX_WEIGHT + 1) + 1;
const int MAX_EDGES = MAX_NODES * 4; // două muchii per cameră + reziduale
const int INF = 1'000'000'000;

struct edge {
  int cap, cost;
```

```
  int v, next;
};

struct node {
  int adj;
  int d;            // distanța de la sursă
  int parent, edge; // părintele și muchia incidentă în Dijkstra
};

typedef std::pair<int, int> pair;

node nd[MAX_NODES];
edge e[MAX_EDGES + 2];
std::priority_queue<pair, std::vector<pair>, std::greater<pair>> pq;
int n, num_thieves, source, sink, total_cost, sink_potential;
int epos;

void add_edge(int u, int v, int cap, int cost) {
  e[epos] = { cap, cost, v, nd[u].adj };
  nd[u].adj = epos++;
}

void add_edge_pair(int u, int v, int cap, int cost) {
  add_edge(u, v, cap, cost);
  add_edge(v, u, 0, -cost);
}

void read_data_and_build_graph() {
  epos = 2; // lasă 0/1 pentru NIL
  total_cost = 0;
  sink_potential = 0;

  int num_rooms, g, val, weight, alarm;
  scanf("%d %d %d", &num_rooms, &num_thieves, &g);
  n = num_rooms * (g + 1) + 1;
  source = 0;
  sink = n - 1;
  for (int u = 0; u < n; u++) {
    nd[u].adj = 0;
  }

  for (int r = 0; r < num_rooms; r++) {
    scanf("%d %d %d", &val, &weight, &alarm);
    for (int w = 0; w <= g; w++) {
      int u = r * (g + 1) + w;

      if (w + weight <= g) {
        add_edge_pair(u, u + weight, num_thieves, -val);
      }
```

```
      int v = (r < num_rooms - 1) ? (u + (g + 1)) : sink;
      add_edge_pair(u, v, alarm, 0);
    }
  }
}

void init_potentials() {
  for (int u = 0; u < n; u++) {
    nd[u].d = INF;
  }
  nd[source].d = 0;

  for (int u = 0; u < n; u++) {
    for (int pos = nd[u].adj; pos; pos = e[pos].next) {
      if (e[pos].cap) {
        int v = e[pos].v;
        nd[v].d = std::min(nd[v].d, nd[u].d + e[pos].cost);
      }
    }
  }
}

void reweight_edges() {
  for (int u = 0; u < n; u++) {
    if (nd[u].d < INF) {
      for (int pos = nd[u].adj; pos; pos = e[pos].next) {
        int v = e[pos].v;
        e[pos].cost += nd[u].d - nd[v].d;
      }
    }
  }

  sink_potential += nd[sink].d;
}

bool dijkstra() {
  for (int u = 0; u < n; u++) {
    nd[u].d = INF;
  }

  nd[source].d = 0;
  pq.push({0, source});

  while (!pq.empty()) {
    auto [dist, u] = pq.top();
    pq.pop();

    if (dist == nd[u].d) {
      for (int pos = nd[u].adj; pos; pos = e[pos].next) {
        // Relaxează muchia.
```

```
        int v = e[pos].v;
        if (e[pos].cap && (nd[u].d + e[pos].cost < nd[v].d)) {
          nd[v].d = nd[u].d + e[pos].cost;
          nd[v].parent = u;
          nd[v].edge = pos;
          pq.push({nd[v].d, v});
        }
      }
    }
  }

  return (nd[sink].d < INF);
}

int path_minimum() {
  int min_cap = num_thieves;
  int u = sink;
  while (u != source) {
    min_cap = std::min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void augment_path(int flow) {
  int u = sink;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

void min_cost_max_flow() {
  init_potentials();
  reweight_edges();

  while (num_thieves && dijkstra()) {
    reweight_edges();
    int flow = path_minimum();
    augment_path(flow);
    total_cost += flow * sink_potential;
    num_thieves -= flow;
  }
}

void solve_test() {
  read_data_and_build_graph();
  min_cost_max_flow();
  printf("%d\n", (nd[sink].d == INF) ? -1 : -total_cost);
```

```
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);
  while (num_tests--) {
    solve_test();
  }

  return 0;
}
```

34.5 Problema Lifts (IATI Shumen 2022)

‹ înapoi • versiune online

```
#include <algorithm>
#include <queue>
#include <stdio.h>

const int MAX_REQUESTS = 10'000;
const int MAX_NODES = 120'000;
const int MAX_EDGES = 800'000;
const long long INF = 1e9;
const long long INFL = 1e18;

struct request {
  int src, dest;
};

struct edge {
  int v, next;
  int cap;
  long long cost;
};

struct node {
  int adj;
  int parent, edge;  // părintele și muchia incidentă în Dijkstra
  long long d;       // distanța de la sursă
};

request req[MAX_REQUESTS];
int ord[MAX_REQUESTS]; // pentru sortări
node nd[MAX_NODES];
edge e[MAX_EDGES + 2];
int n, num_req, num_elev, source, sink;
long long sink_potential;
```

```
void read_data() {
  scanf("%d %d", &num_req, &num_elev);
  for (int i = 0; i < num_req; i++) {
    scanf("%d %d", &req[i].src, &req[i].dest);
  }
}

void add_edge(int u, int v, int cap, int cost) {
  static int pos = 2;

  e[pos] = { v, nd[u].adj, cap, cost };
  nd[u].adj = pos++;
}

void add_edge_pair(int u, int v, int cap, int cost) {
  add_edge(u, v, cap, cost);
  add_edge(v, u, 0, -cost);
}

void sort_ranges(int lo, int mid, int hi) {
  std::sort(ord + lo, ord + mid, [](int a, int b) {
    return (req[a].dest < req[b].dest);
  });
  std::sort(ord + mid, ord + hi, [](int a, int b) {
    return (req[a].src < req[b].src);
  });
}

// Creează noduri noi și leagă-le 1:1 cu intervalul [lo, hi). Înlănțuie-le
// crescător sau descrescător conform cu sort_asc.
int add_nodes(int lo, int hi, bool sort_asc) {
  int n0 = n;
  n += hi - lo;

  for (int i = 0; i < hi - lo; i++) {
    add_edge_pair(n0 + i, ord[lo + i], 1, 0);
  }
  for (int i = 0; i < hi - lo - 1; i++) {
    int from = n0 + i + !sort_asc;
    int to = n0 + i + sort_asc;
    int cost = req[ord[lo + i + 1]].src - req[ord[lo + i]].src;
    add_edge_pair(from, to, num_elev, cost);
  }

  return n0;
}

// Creează muchii de la toate nodurile din [lo, mid) la toate nodurile din
// [mid, hi). Pentru aceasta, interclasează listele ordonate de valori src și
```

```
// dest și folosește listele sortate de noduri care încep de la n_asc și
// n_desc.
void add_back_edges(int lo, int mid, int hi, int n_asc, int n_desc) {
  int j = mid;
  for (int i = lo; i < mid; i++) {
    while ((j < hi) && (req[ord[j]].src <= req[ord[i]].dest)) {
      j++;
      n_asc++;
      n_desc++;
    }
    if (j < hi) {
      int cost = req[ord[j]].src - req[ord[i]].dest;
      add_edge_pair(num_req + ord[i], n_asc, 1, cost);
    }
    if (j > mid) {
      int cost = req[ord[i]].dest - req[ord[j - 1]].src;
      add_edge_pair(num_req + ord[i], n_desc - 1, 1, cost);
    }
  }
}

// Grupează comenzile în perechi de intervale de lungime l. Pentru fiecare
// pereche, leagă toate comenzile din primul interval de toate comenzile din
// al doilea. Folosește 2·l noduri suplimentare pentru a crea două liste care
// parcurg comenzile crescător, respectiv descrescător după src.
void add_layer(int l) {
  for (int mid = l; mid < num_req; mid += 2 * l) {
    int lo = mid - l, hi = std::min(mid + l, num_req);
    if (l <= 12) {
      // Avem nevoie de C(l, 2) muchii, cea ce este mai ieftin decît să creăm
      // 2l noduri și 6l muchii.
      for (int i = lo; i < mid; i++) {
        for (int j = mid; j < hi; j++) {
          add_edge_pair(num_req + i, j, 1, std::abs(req[i].dest - req[j].src));
        }
      }
    } else {
      sort_ranges(lo, mid, hi);
      int n_asc = add_nodes(mid, hi, true);
      int n_desc = add_nodes(mid, hi, false);
      add_back_edges(lo, mid, hi, n_asc, n_desc);
    }
  }
}

// [0, num_req):            nodurile din stînga.
// [num_req, 2 num_req):    nodurile din dreapta.
// 2 num_req:               sursa
// 2 num_req + 1:           destinația
// apoi alte noduri alocate pentru structura pe niveluri
```

```
void build_graph() {
  source = 2 * num_req;
  sink = 2 * num_req + 1;
  n = 2 * num_req + 2;
  for (int i = 0; i < num_req; i++) {
    add_edge_pair(source, i, 1, 0);
    add_edge_pair(i, i + num_req, 1, -INF);
    add_edge_pair(i + num_req, sink, 1, 0);
  }

  for (int i = 0; i < num_req; i++) {
    ord[i] = i;
  }
  for (int l = 1; l < num_req; l <<= 1) {
    add_layer(l);
  }
}

// Sortarea topologică este suficientă (și, cred, necesară) pentru
// inițializarea potențialelor. Nu este nevoie de Bellman-Ford, întrucît
// graful este un dag.
void top_sort() {
  // Calculează gradele. Folosim cîmpul edge ca să nu declarăm un cîmp nou.
  for (int u = 0; u < n; u++) {
    nd[u].d = INFL;
    for (int pos = nd[u].adj; pos; pos = e[pos].next) {
      if (e[pos].cap) {
        nd[e[pos].v].edge++;
      }
    }
  }

  std::queue<int> q;
  for (int u = 0; u < n; u++) {
    if (!nd[u].edge) {
      nd[u].d = 0;
      q.push(u);
    }
  }

  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int pos = nd[u].adj; pos; pos = e[pos].next) {
      if (e[pos].cap) {
        int v = e[pos].v;
        nd[v].d = std::min(nd[v].d, nd[u].d + e[pos].cost);
        if (!--nd[v].edge) {
          q.push(v);
        }
```

```
      }
    }
  }
}

void reweight_edges() {
  for (int u = 0; u < n; u++) {
    for (int pos = nd[u].adj; pos; pos = e[pos].next) {
      int v = e[pos].v;
      e[pos].cost += nd[u].d - nd[v].d;
    }
  }

  sink_potential += nd[sink].d;
}

void dijkstra() {
  std::priority_queue<std::pair<long long, int>> pq;

  for (int u = 0; u < n; u++) {
    nd[u].d = INFL;
  }

  nd[source].d = 0;
  pq.push({0, source});

  while (!pq.empty()) {
    auto [dist, u] = pq.top();
    pq.pop();

    if (-dist == nd[u].d) { // pq este un max-heap
      for (int pos = nd[u].adj; pos; pos = e[pos].next) {
        int v = e[pos].v;
        if (e[pos].cap && (nd[u].d + e[pos].cost < nd[v].d)) {
          nd[v].d = nd[u].d + e[pos].cost;
          nd[v].parent = u;
          nd[v].edge = pos;
          pq.push({-nd[v].d, v});
        }
      }
    }
  }
}

void augment_path() {
  for (int u = sink; u != source; u = nd[u].parent) {
    e[nd[u].edge].cap--;
    e[nd[u].edge ^ 1].cap++;
  }
}
```

```
long long min_cost_max_flow() {
  long long total_cost = num_req * INF;

  top_sort();
  reweight_edges();

  while (num_elev--) {
    dijkstra();
    reweight_edges();
    augment_path();
    total_cost += sink_potential;
  }

  return total_cost;
}

int main() {
  read_data();
  build_graph();
  long long total_cost = min_cost_max_flow();
  printf("%lld\n", total_cost);

  return 0;
}
```

Anexa 35

Șiruri - Algoritmul Rabin-Karp

35.1 Problema Radio (Lot 2010)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MAX_M = 2'500;
const int BASE = 113;    // baza pentru hash
const int MOD = 1 << 20; // modulul pentru hash
const int MAX_COLLISIONS = 9; // determinat experimental
const int NONE = -1;

int n, m, l, k;
char s[MAX_N + 1], p[MAX_M + 1]; // șirul și cuvîntul curent
int min_match; // lungimea minimă garantată de căutat exact
int cur_word;    // operația curentă
long long base_pow; // BASE^{min_match - 1} % MOD, pentru Rabin-Karp
int cnt[MOD];       // numărul de apariții ale fiecărei valori hash
int where[MOD][MAX_COLLISIONS]; // pozițiile acestor apariții
short tried[MOD]; // cache cu pozițiile deja încercate pentru cuvîntul curent
FILE *fin, *fout;

//  a/b
int ceil(int a, int b) {
  return (a - 1) / b + 1;
}

void read_string() {
  fscanf(fin, "%d %d %d %d %s ", &n, &m, &l, &k, s);
  min_match = ceil(l - k, k + 1);
}

// Rabin-Karp (hash în fereastră glisantă) pe s.
```

```
void hash_string() {
  base_pow = 1;
  for (int i = 0; i < min_match - 1; i++) {
    base_pow = base_pow * BASE % MOD;
  }

  long long hash = 0;
  for (int i = 0; i < min_match - 1; i++) {
    hash = (hash * BASE + (s[i] - 'a')) % MOD;
  }
  for (int start = 0, end = min_match - 1; end < n; start++, end++) {
    hash = (hash * BASE + (s[end] - 'a')) % MOD;
    where[hash][cnt[hash]++] = start;
    hash = (hash + MOD - base_pow * (s[start] - 'a') % MOD) % MOD;
  }
}

// Numără naiv diferențele între s[start...] și p.
bool at_most_k_mismatches(int start) {
  if (tried[start] == cur_word) {
    return false;
  }
  tried[start] = cur_word;

  int num_diffs = 0;
  int i = start;
  while ((num_diffs <= k) && (i < start + l)) {
    num_diffs += (s[i] != p[i - start]);
    i++;
  }
  return (num_diffs <= k);
}

// Încearcă să suprapună p[pstart..pstart+min_match-1] peste s.
bool try_offset(int pstart, int hash) {
  for (int i = 0; i < cnt[hash]; i++) {
    int sstart = where[hash][i];
    // sstart trebuie să fie suficient de departe de marginile lui s pentru ca
    // p să poată fi suprapus corect (reminder: la sstart începe doar
    // fragmentul p[pstart...pstart-min_match+1]).
    if ((sstart >= pstart) &&
        (sstart <= n - l + pstart) &&
        at_most_k_mismatches(sstart - pstart)) {
      return true;
    }
  }

  return false;
}
```

```
// Rabin-Karp peste cuvîntul curent. Caută în s fiecare fereastră a lui p de
// mărime min_match.
bool query() {
  long long hash = 0;
  for (int i = 0; i < min_match - 1; i++) {
    hash = (hash * BASE + (p[i] - 'a')) % MOD;
  }

  bool found = false;
  int start = 0, end = min_match - 1;
  while (!found && (end < l)) {
    hash = (hash * BASE + (p[end] - 'a')) % MOD;
    found = try_offset(start, hash);
    hash = (hash + MOD - base_pow * (p[start] - 'a') % MOD) % MOD;
    start++;
    end++;
  }

  return found;
}

void process_queries() {
  for (cur_word = 1; cur_word <= m; cur_word++) {
    fscanf(fin, "%s ", p);
    fprintf(fout, "%d\n", query());
  }
}

int main() {
  fin = fopen("radio.in", "r");
  fout = fopen("radio.out", "w");

  read_string();
  hash_string();
  process_queries();

  fclose(fin);
  fclose(fout);
  return 0;
}
```

Anexa 36

Șiruri - Algoritmul Knuth-Morris-Pratt

36.1 Problema Bart (NerdArena)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 500'000;

char s[MAX_N + 1];
int pi[MAX_N];

int main(void) {
  FILE *f = fopen("bart.in", "r");
  fscanf(f, "%s", s);
  fclose(f);

  pi[0] = 0;
  int k = 0, n = 1;
  while (s[n]) {
    while ((k > 0) && (s[k] != s[n])) {
      k = pi[k - 1];
    }
    if (s[k] == s[n]) {
      k++;
    }
    pi[n++] = k;
  }

  s[n - pi[n - 1]] = '\0';

  f = fopen("bart.out", "w");
  fprintf(f, "%s\n", s);
  fclose(f);
```

```
}
```

36.2 Problema Cifru (Baraj ONI 2006)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 100'000;
const int MAX_M = 10'000;
const int NONE = -1;

short s[2 * MAX_N + 1], p[MAX_N];
int pos[MAX_M + 1];
int pi[MAX_N];
int prev_p[MAX_N];     // apariția anterioară a lui p[i] în p
int prev_s[2 * MAX_N]; // apariția anterioară a lui s[i] în s
int perm[MAX_M + 1];
int n, m, shift;

void read_data() {
  FILE* f = fopen("cifru.in", "r");

  fscanf(f, "%d %d", &n, &m);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%hd", &p[i]);
  }
  for (int i = 0; i < n; i++) {
    fscanf(f, "%hd", &s[i]);
    s[i + n] = s[i];
  }

  fclose(f);
}

void build_prev(short* p, int n, int* prev) {
  for (int i = 1; i <= m; i++) {
    pos[i] = NONE;
  }
  for (int i = 0; i < n; i++) {
    prev[i] = pos[p[i]];
    pos[p[i]] = i;
  }
}

bool matches(int k, int i, int* prev) {
  if (prev_p[k] == NONE) {
    return (prev[i] < i - k);
```

```
  } else {
    return (i - prev[i] == k - prev_p[k]);
  }
}

void build_pi() {
  pi[0] = 0;
  int k = 0; // starea curentă
  for (int i = 1; i < n; i++) {
    while ((k > 0) && !matches(k, i, prev_p)) {
      k = pi[k - 1];
    }
    if (matches(k, i, prev_p)) {
      k++;
    }
    pi[i] = k;
  }
}

void kmp() {
  int k = 0; // starea curentă
  int i = 0;
  while (k != n) {
    while ((k > 0) && !matches(k, i, prev_s)) {
      k = pi[k - 1];
    }
    if (matches(k, i, prev_s)) {
      k++;
    }
    i++;
  }

  shift = i - n;
}

void build_perm() {
  for (int i = 0; i < n; i++) {
    perm[p[i]] = s[(i + shift) % n];
  }
}

void write_answer() {
  FILE* f = fopen("cifru.out", "w");
  fprintf(f, "%d\n", shift);
  for (int i = 1; i <= m; i++) {
    fprintf(f, "%d%c", perm[i], (i == m) ? '\n' : ' ');
  }
  fclose(f);
}
```

```
int main() {
  read_data();
  build_prev(p, n, prev_p);
  build_prev(s, 2 * n, prev_s);
  build_pi();
  kmp();
  build_perm();
  write_answer();

  return 0;
}
```

36.3 Problema Anthem of Berland (Codeforces)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>

const int MAX_LENGTH = 10'000'000;
const int SQRT = 3'200;
const int SIGMA = 26;
const int NONE = -1;

char s[MAX_LENGTH + 1], p[SQRT];
int pi[SQRT];
int delta[SQRT][SIGMA];
int d[2][SQRT];
int ns, np, row;

void read_data() {
  scanf("%s %s", s, p);
  ns = strlen(s);
  np = strlen(p);
}

void build_pi() {
  int k = 0;
  for (int q = 1; q < np; q++) {
    while ((k > 0) && (p[k] != p[q])) {
      k = pi[k - 1];
    }
    if (p[k] == p[q]) {
      k++;
    }
    pi[q] = k;
  }
}
```

```
// Vezi Cormen ediția a 4-a, exercițiul 32.4-8.
void build_delta() {
  delta[0][p[0] - 'a'] = 1;

  for (int c = 'a'; c <= 'z'; c++) {
    for (int q = 1; q <= np; q++) {
      delta[q][c - 'a'] = ((q < np) && (p[q] == c))
        ? (q + 1)
        : delta[pi[q - 1]][c - 'a'];
    }
  }
}

void improve(int& x, int y) {
  x = (x > y) ? x : y;
}

void build_dp() {
  d[0][0] = 0;
  for (int j = 1; j <= np; j++) {
    d[0][j] = NONE;
  }
  for (int i = 0; i < ns; i++) {
    for (int j = 0; j <= np; j++) {
      d[1 - row][j] = NONE;
    }
    for (int j = 0; j <= np; j++) {
      if (d[row][j] != NONE) {
        int start = (s[i] == '?') ? 'a' : s[i];
        int end = (s[i] == '?') ? 'z' : s[i];
        for (int c = start; c <= end; c++) {
          int new_st = delta[j][c - 'a'];
          improve(d[1 - row][new_st], d[row][j] + (new_st == np));
        }
      }
    }
    row = 1 - row;
  }
}

void write_answer() {
  int max = 0;
  for (int i = 0; i <= np; i++) {
    improve(max, d[row][i]);
  }
  printf("%d\n", max);
}

int main() {
```

```
  read_data();
  if (np > ns) {
    printf("0\n"); // paranoia este bună
  } else {
    build_pi();
    build_delta();
    build_dp();
    write_answer();
  }

  return 0;
}
```

Anexa 37

Șiruri - Quicksort ternar

37.1 Problema Cuvinte (ONI 2021, clasele 11-12)

‹ înapoi • versiune online

```cpp
#include <algorithm>
#include <stdio.h>
#include <string.h>

const int MAX_LEN = 300'000;
const int MAX_N = 300'000;
const int MOD = 1'000'000'007;

char buf[MAX_LEN + MAX_N]; // lasă loc pentru terminatorii de șir
char* s[MAX_N];            // cuvintele de la intrare

int f[MAX_LEN + 1];     // frecvențele lungimilor cuvintelor citite
int g[MAX_LEN + 1];     // frecvențele lungimilor cuvintelor de adăugat
int cnt[MAX_LEN + 1];   // numărul de subșiruri distincte de fiecare lungime
int n, sigma, max_extra_len;

void read_data() {
  int ptr = 0, num_extra, len;
  scanf("%d %d %d", &n, &num_extra, &sigma);

  for (int i = 0; i < n; i++) {
    s[i] = buf + ptr;
    scanf("%s", s[i]);
    ptr += strlen(s[i]) + 1; // include \0
  }

  while (num_extra--) {
    scanf("%d", &len);
    g[len]++;
    max_extra_len = std::max(len, max_extra_len);
```

```
  }
}

void tsort(int left, int right, int pos) {
  // Optimizare care previne recursivitatea pentru restul șirului.
  if (right - left == 1) {
    while (s[left][pos] != '\0') {
      cnt[++pos]++;
    }
    f[pos]++;
  }

  if (right - left <= 1) {
    return;
  }

  int i = left, lt = left, gt = right;
  char pivot = s[left][pos];

  while ((i < gt) && s[i][pos]) {
    if (s[i][pos] < pivot) {
      std::swap(s[i++], s[lt++]);
    } else if (s[i][pos] > pivot)  {
      std::swap(s[i], s[--gt]);
    } else {
      i++;
    }
  }

  if (i < gt) {
    // Am găsit un șir terminat aici. Restul șirurilor sînt continuări ale
    // sale și nu mai contează în statistici, deci nu mai reapelăm.
    f[pos]++;
  } else {
    cnt[pos + 1]++; // pentru zona din mijloc
    tsort(left, lt, pos);
    tsort(lt, gt, pos + 1);
    tsort(gt, right, pos);
  }
}

int scan_lengths() {
  long long result = 1, sigma_l = 1, c = 0;

  for (int l = 1; l <= max_extra_len; l++) {
    c = (c + f[l - 1] + g[l - 1]) * sigma % MOD;
    sigma_l = sigma_l * sigma % MOD;
    long long legal = sigma_l - c - cnt[l] + MOD;
    for (int j = 0; j < g[l]; j++) {
      result = result * (legal - j) % MOD;
```

```
    }
  }

  return result;
}

int main() {
  read_data();
  tsort(0, n, 0);
  int result = scan_lengths();
  printf("%d\n", result);

  return 0;
}
```

37.2 Problema Sabin (Baraj ONI 2015)

‹ înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>

const int MAX_STRINGS = 40'000;
const int MAX_QUERIES = 20'000;
const int MAX_LEN = 15 * 30;
const int MAX_BIG_SHELF = 30'000 * 30;

struct string {
  char* p;
  short qindex;
};

struct query {
  short lo_prefix, hi_prefix;
  short answer;
};

char buf[MAX_STRINGS][MAX_LEN + 1];
string s[MAX_STRINGS];
query q[MAX_QUERIES + 1];
char big_shelf[MAX_BIG_SHELF + 1];
int n, k, p, num_queries;

// Interclasează cele K×P litere.
void read_string(int ind, FILE* f) {
  s[ind].p = buf[ind];
  for (int i = 0; i < k; i++) {
    for (int j = 0; j < p; j++) {
```

```
      s[ind].p[k * j + i] = fgetc(f);
    }
    fgetc(f);
  }
}

// Construiește șirul explicit pentru interogare și pune-l laolaltă cu
// șirurile inițiale.
void read_query(int ind, FILE* f) {
  int score;
  fscanf(f, "%d", &score);
  q[ind].lo_prefix = k * score;
  q[ind].hi_prefix = k * (score + 1);

  int pos = n + ind - 1;
  s[pos].p = buf[pos];
  s[pos].qindex = ind;
  for (int i = 0; i < k; i++) {
    int book;
    fscanf(f, "%d", &book);
    book--;
    for (int j = 0; j < p; j++) {
      s[pos].p[k * j + i] = big_shelf[p * book + j];
    }
  }
}

void read_data() {
  int m;
  FILE* f = fopen("sabin.in", "r");
  fscanf(f, "%d %d %d %d %d ", &n, &k, &m, &p, &num_queries);
  for (int i = 0; i < n; i++) {
    read_string(i, f);
  }
  for (int i = 0; i < m; i++) {
    fscanf(f, "%s ", big_shelf + i * p);
  }
  for (int i = 1; i <= num_queries; i++) {
    read_query(i, f);
  }
  fclose(f);
}

int count_strings(int first, int last) {
  int res = 0;
  for (int i = first; i < last; i++) {
    res += (s[i].qindex == 0);
  }
  return res;
}
```

```
void answer_query(int ind, int prefix_len, int num_strings) {
  if (ind) {
    if (q[ind].lo_prefix == prefix_len) {
      q[ind].answer = num_strings;
      q[ind].lo_prefix = -1; // nu mai întreba niciodată despre acest prefix
    } else if (q[ind].hi_prefix == prefix_len) {
      q[ind].answer -= num_strings;
      q[ind].hi_prefix = -1;
    }
  }
}

void tsort(int left, int right, int pos) {
  if (right - left <= 1) {
    return;
  }

  int num_strings = count_strings(left, right);
  int i = left, lt = left, gt = right;
  char pivot = s[left].p[pos];

  while (i < gt) {
    answer_query(s[i].qindex, pos, num_strings);
    if (s[i].p[pos] < pivot) {
      std::swap(s[i++], s[lt++]);
    } else if (s[i].p[pos] > pivot)  {
      std::swap(s[i], s[--gt]);
    } else {
      i++;
    }
  }

  tsort(left, lt, pos);
  if (s[lt].p[pos] != '\0') {
    tsort(lt, gt, pos + 1);
  }
  tsort(gt, right, pos);
}

void write_answers() {
  FILE* f = fopen("sabin.out", "w");
  for (int i = 1; i <= num_queries; i++) {
    fprintf(f, "%d\n", q[i].answer);
  }
  fclose(f);
}

int main() {
  read_data();
```

```
  tsort(0, n + num_queries, 0);
  write_answers();

  return 0;
}
```

Anexa 38

Șiruri de sufixe

38.1 Problema Carry Bit (IIOT 2023-2024 runda 3)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>

const int MAX_N = 200'000;
const int MAX_SEGTREE_NODES = 1 << 20;

char s[2 * MAX_N + 1]; // A și B reunite.
int n;

void read_strings() {
  scanf("%d %s ", &n, s);
  scanf("%s", s + n);
  for (int i = 0; i < n; i++) {
    s[n + i] ^= 1;
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int next_power_of_2(int x) {
  return 1 << (32 - __builtin_clz(x - 1));
}

struct min_segment_tree {
  int m[MAX_SEGTREE_NODES];
  int n;

  void init(int* v, int _n) {
```

```
    n = next_power_of_2(_n);
    for (int i = 0; i < _n; i++) {
      m[n + i] = v[i];
    }

    for (int i = n - 1; i; i--) {
      m[i] = min(m[2 * i], m[2 * i + 1]);
    }
  }

  int range_min(int l, int r) {
    l += n;
    r += n;
    int result = n;

    while (l <= r)  {
      if (l & 1) {
        result = min(result, m[l++]);
      }
      l >>= 1;

      if (!(r & 1)) {
        result = min(result, m[r--]);
      }
      r >>= 1;
    }

    return result;
  }
};

struct suffix_array {
  const char* s;
  int n, num_classes;

  int p[2 * MAX_N + 1];
  int c[2 * MAX_N + 1];
  int p2[2 * MAX_N + 1];
  int c2[2 * MAX_N + 1];

  int cnt[256];

  int lcp[2 * MAX_N + 1];
  min_segment_tree st;

  void init(char* s, int n) {
    this->s = s;
    this->n = n;
  }
```

```
int circ(int x, int n) {
  return (x >= n) ? (x - n) : x;
}

void build_single_letters() {
  memset(cnt, 0, sizeof(int) * 256);
  for (int i = 0; i < n; i++) {
    cnt[(int)s[i]]++;
  }
  for (int i = 1; i <= 'z'; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p[--cnt[(int)s[i]]] = i;
  }

  c[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
  }
  num_classes = 1 + c[p[n - 1]];
}

void build_prefix_len(int len) {
  memset(cnt, 0, sizeof(int) * num_classes);
  for (int i = 0; i < n; i++) {
    cnt[c[i]]++;
  }
  for (int i = 1; i < num_classes; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p2[i] = circ(p[i] - len + n, n);
  }
  for (int i = n - 1; i >= 0; i--) {
    p[--cnt[c[p2[i]]]] = p2[i];
  }

  c2[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    bool diff =  (c[p[i]] != c[p[i - 1]]) ||
      (c[circ(p[i] + len, n)] != c[circ(p[i - 1] + len, n)]);
    c2[p[i]] = c2[p[i - 1]] + diff;
  }
  num_classes = 1 + c2[p[n - 1]];
  memcpy(c, c2, sizeof(int) * n);
}
```

```
  void build() {
    build_single_letters();

    int len = 1;
    while (num_classes < n) {
      build_prefix_len(len);
      len <<= 1;
    }
  }

  void build_lcp() {
    int l = 0;
    for (int i = 0; i < n; i++) {
      int k = c[i];
      if (k) {
        int j = p[k - 1];
        while (s[i + l] == s[j + l]) {
          l++;
        }
        lcp[k] = l;

        if (l) {
          l--;
        }
      }
    }

    st.init(lcp, n + 1);
  }

  int get_lcp(int i, int j) {
    if (i > j) {
      int tmp = i; i = j; j = tmp;
    }
    return st.range_min(i + 1, j);
  }
};

suffix_array suf;

bool query(int x, int y, int l) {
  int p1 = suf.c[x];
  int p2 = suf.c[y + n];
  int pref = suf.get_lcp(p1, p2);

  return (pref >= l) || (s[x + pref] == '0');
}

void process_queries() {
  int q, x, y, l;
```

```
  scanf("%d", &q);
  while (q--) {
    scanf("%d %d %d", &x, &y, &l);
    printf("%d ", query(x, y, l));
  }
  printf("\n");
}

int main() {
  read_strings();
  suf.init(s, 2 * n + 1);
  suf.build();
  suf.build_lcp();
  process_queries();

  return 0;
}
```

38.2 Problema Periodic Words (IIOT 2023-2024 runda 1)

‹ înapoi

Soluție cu hashing (versiune online).

```
#include <stdio.h>

typedef unsigned long long u64;

const int MAX_N = 100'000;
const int BASE = 61; // modulo 2^64

char s[MAX_N + 2];
int spf[MAX_N + 1]; // spf[x] = cel mai mic factor prim al lui x
u64 bpow[MAX_N + 1];
u64 h[MAX_N + 1];
int n;

void read_and_hash() {
  scanf("%d %s", &n, s + 1);

  bpow[0] = 1;
  for (int i = 1; i <= n; i++) {
    h[i] = h[i - 1] * BASE + (s[i] - 'a');
    bpow[i] = bpow[i - 1] * BASE;
  }
}
```

```
void sieve() {
  spf[1] = 1;
  for (int i = 2; i <= n; i++) {
    if (!spf[i])  {
      for (int j = i; j <= n; j += i) {
        if (!spf[j]) {
          spf[j] = i;
        }
      }
    }
  }
}

u64 substr_hash(int l, int r) {
  return h[r] - h[l - 1] * bpow[r - l + 1];
}

bool matches(int l, int r, int per) {
  return substr_hash(l, r - per) == substr_hash(l + per, r);
}

int query(int l, int r) {
  int len = r - l + 1;
  bool found = false;

  while (!found && (len > 1)) {
    int x = spf[len];
    found = matches(l, r, (r - l + 1) / x);
    do {
      len /= x;
    } while (len % x == 0);
  }

  return found;
}

void process_queries() {
  int q, l, r;

  scanf("%d", &q);
  while (q--) {
    scanf("%d %d", &l, &r);
    l++;
    r++;
    printf(query(l, r) ? "YES\n" : "NO\n");
  }
}

int main() {
  read_and_hash();
```

```
  sieve();
  process_queries();

  return 0;
}
```

Soluție cu șiruri de sufixe (versiune online).

```
// Cînd ai un ciocan, totul în jurul tău pare un cui. Fără legătură cu asta,
// iată o soluție cu șiruri de sufixe + LCP + RMQ + chiuveta din bucătărie.
#include <algorithm>
#include <stdio.h>
#include <string.h>

const int MAX_N = 100'000;
const int MAX_LOG = 17;

struct suffix_array {
  const char* s;
  int n, num_classes;

  int p[MAX_N + 1];
  int c[MAX_N + 1];
  int p2[MAX_N + 1];
  int c2[MAX_N + 1];

  int cnt[256];

  int lcp[MAX_N + 1];

  void init(char* s, int n) {
    this->s = s;
    this->n = n;
  }

  int circ(int x, int n) {
    return (x >= n) ? (x - n) : x;
  }

  void build_single_letters() {
    memset(cnt, 0, sizeof(int) * 256);
    for (int i = 0; i < n; i++) {
      cnt[(int)s[i]]++;
    }
    for (int i = 1; i <= 'z'; i++) {
      cnt[i] += cnt[i - 1];
    }

    for (int i = 0; i < n; i++) {
```

```
    p[--cnt[(int)s[i]]] = i;
  }

  c[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
  }
  num_classes = 1 + c[p[n - 1]];
}

void build_prefix_len(int len) {
  memset(cnt, 0, sizeof(int) * num_classes);
  for (int i = 0; i < n; i++) {
    cnt[c[i]]++;
  }
  for (int i = 1; i < num_classes; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p2[i] = circ(p[i] - len + n, n);
  }
  for (int i = n - 1; i >= 0; i--) {
    p[--cnt[c[p2[i]]]] = p2[i];
  }

  c2[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    bool diff =  (c[p[i]] != c[p[i - 1]]) ||
      (c[circ(p[i] + len, n)] != c[circ(p[i - 1] + len, n)]);
    c2[p[i]] = c2[p[i - 1]] + diff;
  }
  num_classes = 1 + c2[p[n - 1]];
  memcpy(c, c2, sizeof(int) * n);
}

void build() {
  build_single_letters();

  int len = 1;
  while (num_classes < n) {
    build_prefix_len(len);
    len <<= 1;
  }

  // Permutarea inversă
  for (int i = 0; i < n; i++) {
    p2[p[i]] = i;
  }
}
```

```
  void build_lcp() {
    int l = 0;
    for (int i = 0; i < n; i++) {
      int k = c[i];
      if (k) {
        int j = p[k - 1];
        while (s[i + l] == s[j + l]) {
          l++;
        }
        lcp[k] = l;

        if (l) {
          l--;
        }
      }
    }
  }
};

int log2(int x) {
  return 31 - __builtin_clz(x);
}

struct sparse_table {
  int v[MAX_LOG][MAX_N + 1];

  void build(int* src, int n) {
    for (int i = 0; i < n; i++) {
      v[0][i] = src[i];
    }
    for (int p = 1; (1 << p) <= n; p++) {
      for (int i = 0; i <= n - (1 << p); i++) {
        v[p][i] = std::min(v[p - 1][i], v[p - 1][i + (1 << (p - 1))]);
      }
    }
  }

  int range_min(int l, int r) {
    int row = log2(r - l + 1);
    return std::min(v[row][l], v[row][r - (1 << row) + 1]);
  }
};

char s[MAX_N + 1];
int spf[MAX_N + 1]; // spf[x] = cel mai mic factor prim al lui x
suffix_array suf;
sparse_table st;
int n;
```

```
void read_string() {
  scanf("%d %s", &n, s);
}

void sieve() {
  spf[1] = 1;
  for (int i = 2; i <= n; i++) {
    if (!spf[i])  {
      for (int j = i; j <= n; j += i) {
        if (!spf[j]) {
          spf[j] = i;
        }
      }
    }
  }
}

void the_mother_of_all_preprocessing() {
  sieve();
  suf.init(s, n + 1);
  suf.build();
  suf.build_lcp();
  st.build(suf.lcp, n + 1);
}

// Este necesar ca sufixele de la l și l + per să aibă un prefix comun de cel
// puțin (r - l + 1) - per caractere.
bool matches(int l, int r, int per) {
  int x = suf.p2[l], y = suf.p2[l + per];
  if (x > y) {
    std::swap(x, y);
  }

  int lcp = st.range_min(x + 1, y);
  return lcp >= (r - l + 1) - per;
}

int query(int l, int r) {
  int len = r - l + 1;
  bool found = false;

  while (!found && (len > 1)) {
    int x = spf[len];
    found = matches(l, r, (r - l + 1) / x);
    do {
      len /= x;
    } while (len % x == 0);
  }

  return found;
```

```
}

void process_queries() {
  int q, l, r;

  scanf("%d", &q);
  while (q--) {
    scanf("%d %d", &l, &r);
    printf(query(l, r) ? "YES\n" : "NO\n");
  }
}

int main() {
  read_string();
  the_mother_of_all_preprocessing();
  process_queries();

  return 0;
}
```

38.3 Problema Phone Book (Finala IIOT 2022-2023)

‹ înapoi • versiune online

```
#include <stdio.h>
#include <string.h>

const int MAX_SUM_LEN = 200'000;

char concat[2 * MAX_SUM_LEN];
char* s[MAX_SUM_LEN];
int answer[MAX_SUM_LEN];
int num_s;
int c_len; // Lungimea șirurilor concatenate, inclusiv delimitatorii \0.

void read_data() {
  scanf("%d", &num_s);
  for (int i = 0; i < num_s; i++) {
    s[i] = concat + c_len;
    scanf(" %s", s[i]);
    c_len += strlen(s[i]) + 1;
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}
```

```
int max(int x, int y) {
  return (x > y) ? x : y;
}

struct suffix_array {
  const char* s;
  int n, num_classes;

  int p[2 * MAX_SUM_LEN];
  int c[2 * MAX_SUM_LEN];
  int p2[2 * MAX_SUM_LEN];
  int c2[2 * MAX_SUM_LEN];

  int cnt[256];

  int lcp[2 * MAX_SUM_LEN];

  void init(char* s, int n) {
    this->s = s;
    this->n = n;
  }

  int circ(int x, int n) {
    return (x >= n) ? (x - n) : x;
  }

  void build_single_letters() {
    memset(cnt, 0, sizeof(int) * 256);
    for (int i = 0; i < n; i++) {
      cnt[(int)s[i]]++;
    }
    for (int i = 1; i <= 'z'; i++) {
      cnt[i] += cnt[i - 1];
    }

    for (int i = 0; i < n; i++) {
      p[--cnt[(int)s[i]]] = i;
    }

    c[p[0]] = 0;
    for (int i = 1; i < n; i++) {
      c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
    }
    num_classes = 1 + c[p[n - 1]];
  }

  void build_prefix_len(int len) {
    memset(cnt, 0, sizeof(int) * num_classes);
    for (int i = 0; i < n; i++) {
      cnt[c[i]]++;
```

```
  }
  for (int i = 1; i < num_classes; i++) {
    cnt[i] += cnt[i - 1];
  }

  for (int i = 0; i < n; i++) {
    p2[i] = circ(p[i] - len + n, n);
  }
  for (int i = n - 1; i >= 0; i--) {
    p[--cnt[c[p2[i]]]] = p2[i];
  }

  c2[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    bool diff =  (c[p[i]] != c[p[i - 1]]) ||
      (c[circ(p[i] + len, n)] != c[circ(p[i - 1] + len, n)]);
    c2[p[i]] = c2[p[i - 1]] + diff;
  }
  num_classes = 1 + c2[p[n - 1]];
  memcpy(c, c2, sizeof(int) * n);
}

void build() {
  build_single_letters();

  int len = 1;
  while (num_classes < n) {
    build_prefix_len(len);
    len <<= 1;
  }
}

void build_lcp() {
  int l = 0;
  for (int i = 0; i < n; i++) {
    int k = c[i];
    if (k) {
      int j = p[k - 1];
      while (s[i + l] == s[j + l]) {
        l++;
      }
      lcp[k] = l;

      if (l) {
        l--;
      }
    }
  }
}
```

```
  // p2[] și c2[] nu ne mai trebuie. Înlocuiește-l pe p2 cu ID-ul șirului din
  // care provine sufixul. Înlocuiește-l pe c2 cu deplasarea în acel șir.
  void replace_by_string_id() {
    int id = 0, len = strlen(s);
    for (int i = 0; i < n; i++) {
      p2[c[i]] = id;
      c2[c[i]] = len--;
      if (s[i] == '\0') {
        id++;
        len = strlen(s + i + 1);
      }
    }
  }
};

suffix_array suf;
int min_right[2 * MAX_SUM_LEN];

// Toate sufixele din [start, end) provin din același șir. Dorim să selectăm
// un prefix diferit de sufixul de la start-1 (dacă există) și de cel la end
// (dacă există).
void process_batch(int start, int end) {
  int id = suf.p2[start];

  min_right[end] = suf.lcp[end];
  for (int i = end - 1; i > start; i--) {
    min_right[i] = min(suf.lcp[i], min_right[i + 1]);
  }

  int min_left = suf.lcp[start];
  for (int i = start; i < end; i++) {
    min_left = min(min_left, suf.lcp[i]);
    int lcp_start = (start > num_s) ? min_left : 0;
    int lcp_end = (end < c_len) ? min_right[i + 1] : 0;
    int to_take = 1 + max(lcp_start, lcp_end);
    if (to_take <= suf.c2[i]) {
      answer[id] = min(answer[id], to_take);
    }
  }
}

void compute_answers() {
  for (int i = 0; i < num_s; i++) {
    answer[i] = c_len; // infinit
  }
  int i = num_s; // Sari peste sufixele goale.
  while (i < c_len) {
    int j = i + 1;
    while ((j < c_len) && (suf.p2[j] == suf.p2[i])) {
      j++;
```

```
    }
    process_batch(i, j);
    i = j;
  }
}

void write_answers() {
  for (int i = 0; i < num_s; i++) {
    printf("%d\n", (answer[i] == c_len) ? -1 : answer[i]);
  }
}

int main() {
  read_data();
  suf.init(concat, c_len);
  suf.build();
  suf.build_lcp();
  suf.replace_by_string_id();
  compute_answers();
  write_answers();

  return 0;
}
```

Anexa 39

Arbori trie

39.1 Problema Enigma (Lot 2011)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 300'000;
const int MAX_M = 4'500;
const int MAX_S = 100;
const int SIGMA = 26;
const int MOD = 31'333;

// Un trie cu rădăcina 1. Nodul 0 este nefolosit.
struct trie {
  struct node {
    int child[SIGMA];
    int cnt;
  };

  node nd[MAX_M * MAX_S + 1];
  int size = 1;

  void insert(char* s) {
    int u = 1;
    for (int i = 0; s[i]; i++) {
      int c = s[i] - 'a';
      if (!nd[u].child[c]) {
        nd[u].child[c] = ++size;
      }
      u = nd[u].child[c];
      nd[u].cnt++;
    }
  }
};
```

```
char s[MAX_N + 2]; // indexat de la 1
int ways[MAX_N + 1];
trie t;
int n;

void read_and_build_trie() {
  FILE* f = fopen("enigma.in", "r");
  int num_words;
  char word[MAX_S + 1];

  fscanf(f, "%d %d %s", &n, &num_words, s + 1);
  while (num_words--) {
    fscanf(f, "%s", word);
    t.insert(word);
  }
  fclose(f);
}

void dyn_prog() {
  ways[0] = 1;
  for (int i = 0; i <= n; i++) {
    ways[i] %= MOD; // o singură dată, după ce am adunat toate contribuțiile
    int j = i + 1;
    int u = 1;
    while (s[j] && (u = t.nd[u].child[s[j] - 'a'])) {
      ways[j++] += ways[i] * t.nd[u].cnt;
    }
  }
}

void write_answer() {
  FILE* f = fopen("enigma.out", "w");
  fprintf(f, "%d\n", ways[n]);
  fclose(f);
}

int main() {
  read_and_build_trie();
  dyn_prog();
  write_answer();

  return 0;
}
```

39.2 Problema Maximum Xor Subarray (CSES)

înapoi

```cpp
#include <algorithm>
#include <stdio.h>

const int MAX_N = 200'000;
const int NUM_BITS = 30;

struct trie {
  int child[MAX_N * NUM_BITS + 1][2];
  int size = 1;

  void insert(int x) {
    int u = 1;
    for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
      int val = (x >> bit) & 1;
      if (!child[u][val]) {
        child[u][val] = ++size;
      }
      u = child[u][val];
    }
  }

  // Găsește xor-ul maxim între x și o valoare inserată anterior.
  int find_xor(int x) {
    int u = 1, pair = 0;
    for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
      int val = !((x >> bit) & 1);
      if (!child[u][val]) {
        val = !val;
      }
      u = child[u][val];
      pair = (pair << 1) + val;
    }
    return x ^ pair;
  }
};

trie t;

int main() {
  int answer = 0, partial_xor = 0, x, n;
  scanf("%d", &n);

  t.insert(0);
  while (n--) {
    scanf("%d", &x);
    partial_xor ^= x;
    answer = std::max(answer, t.find_xor(partial_xor));
    t.insert(partial_xor);
  }
```

```
  printf("%d\n", answer);

  return 0;
}
```

39.3 Problema Esoteric Bovines (IIOT 2023-2024 runda internațională)

înapoi

Sursă cu căutare binară (versiune online).

```
#include <stdio.h>

const int MAX_N = 200'000;
const int NUM_BITS = 60;

struct trie {
  int child[MAX_N * NUM_BITS + 1][2];
  int cnt[MAX_N * NUM_BITS + 1];
  int size;

  void reset_node(int u) {
    child[u][0] = child[u][1] = cnt[u] = 0;
  }

  void init() {
    size = 1;
    reset_node(1);
  }

  void insert(long long x) {
    int u = 1;
    for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
      int val = (x >> bit) & 1;
      if (!child[u][val]) {
        child[u][val] = ++size;
        reset_node(size);
      }
      u = child[u][val];
      cnt[u]++;
    }
  }

  // Numără valorile a_i care respectă a_i ^ b < x.
  int count_pairs(long long b, long long x) {
```

```
    int sum = 0;
    int u = 1, bit = NUM_BITS - 1;
    while (u && (bit >= 0)) {
      int bval = (b >> bit) & 1;
      int xval = (x >> bit) & 1;
      if (xval) {
        sum += cnt[child[u][bval]];
        u = child[u][!bval];
      } else {
        u = child[u][bval];
      }
      bit--;
    }
    return sum;
  }
};

trie t;
long long b[MAX_N];
int n;
long long k;

void read_data_and_build_trie() {
  t.init();
  scanf("%d %lld", &n, &k);
  for (int i = 0; i < n; i++) {
    long long x;
    scanf("%lld", &x);
    t.insert(x);
  }
  for (int i = 0; i < n; i++) {
    scanf("%lld", &b[i]);
  }
}

// Numără perechile (a_i, b_j) care respectă a_i ^ b_j < x.
long long count_pairs(long long x) {
  long long sum = 0;
  for (int i = 0; i < n; i++) {
    sum += t.count_pairs(b[i], x);
  }
  return sum;
}

long long bin_search() {
  long long lo = 0, hi = 1ll << NUM_BITS;
  while (hi - lo > 1) {
    long long mid = (lo + hi) / 2;
    if (count_pairs(mid) >= k) {
      hi = mid;
```

```
    } else {
      lo = mid;
    }
  }
  return lo;
}

void solve_test_case() {
  read_data_and_build_trie();
  long long ans = bin_search();
  printf("%lld\n", ans);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test_case();
  }

  return 0;
}
```

Sursă cu parcurgere în lățime (versiune online).

```
#include <stdio.h>

const int MAX_N = 200'000;
const int NUM_BITS = 60;

struct pair {
  long long val;
  int u;
};

struct trie {
  int child[MAX_N * NUM_BITS + 1][2];
  int cnt[MAX_N * NUM_BITS + 1];
  int size;

  pair p[MAX_N];
  int num_pairs;

  void reset_node(int u) {
    child[u][0] = child[u][1] = cnt[u] = 0;
  }

  void init() {
```

```
  size = 1;
  reset_node(1);
  num_pairs = 0;
}

void add_val_to_root(long long x) {
  p[num_pairs++] = { x, 1 };
}

void insert(long long x) {
  int u = 1;
  for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
    int val = (x >> bit) & 1;
    if (!child[u][val]) {
      child[u][val] = ++size;
      reset_node(size);
    }
    u = child[u][val];
    cnt[u]++;
  }
}

long long count_zeroes(int bit) {
  long long sum = 0;
  for (int i = 0; i < num_pairs; i++) {
    int u = p[i].u;
    int val = (p[i].val >> bit) & 1;
    sum += cnt[child[u][val]];
  }
  return sum;
}

void push_once(int bit, int target_val) {
  int j = 0;
  for (int i = 0; i < num_pairs; i++) {
    int u = p[i].u;
    int val = (p[i].val >> bit) & 1;
    int c = child[u][val ^ target_val];
    if (c) {
      p[j++] = { p[i].val, c };
    }
  }
  num_pairs = j;
}

long long push_to_bottom(long long k) {
  long long answer = 0;

  for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
    long long z = count_zeroes(bit);
```

```
      if (z < k) {
        k -= z;
        push_once(bit, 1);
        answer += 1ll << bit;
      } else {
        push_once(bit, 0);
      }
    }

    return answer;
  }
};

trie t;
int n;
long long k;

void read_data_and_build_trie() {
  long long x;

  t.init();
  scanf("%d %lld", &n, &k);
  for (int i = 0; i < n; i++) {
    scanf("%lld", &x);
    t.insert(x);
  }
  for (int i = 0; i < n; i++) {
    scanf("%lld", &x);
    t.add_val_to_root(x);
  }
}

void solve_test_case() {
  read_data_and_build_trie();
  long long ans = t.push_to_bottom(k);
  printf("%lld\n", ans);
}

int main() {
  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test_case();
  }

  return 0;
}
```

Sursă cu parcurgere în lățime, optimizată (versiune online).

```
// Idee inspirată de https://kilonova.ro/submissions/372990
#include <algorithm>
#include <stdio.h>

const int MAX_N = 200'000;
const int NUM_BITS = 60;

struct range_pair {
  int la, ra, lb, rb;
  int za, zb; // punctul din [la, ra) până unde avem zerouri
};

range_pair p[2][MAX_N];
int row, np[2];
long long a[MAX_N], b[MAX_N];
int n;
long long k;

void read_data() {
  scanf("%d %lld", &n, &k);
  for (int i = 0; i < n; i++) {
    scanf("%lld", &a[i]);
  }
  for (int i = 0; i < n; i++) {
    scanf("%lld", &b[i]);
  }
}

long long count_zeroes(int bit) {
  long long cnt = 0;
  for (int i = 0; i < np[row]; i++) {
    range_pair& q = p[row][i];
    q.za = q.la;
    while ((q.za < q.ra) && (((a[q.za] >> bit) & 1) == 0)) {
      q.za++;
    }
    q.zb = q.lb;
    while ((q.zb < q.rb) && (((b[q.zb] >> bit) & 1) == 0)) {
      q.zb++;
    }
    cnt += (long long)(q.za - q.la) * (q.zb - q.lb) +
      (long long)(q.ra - q.za) * (q.rb - q.zb);
  }
  return cnt;
}

void add_range(range_pair q) {
```

```
  if ((q.la < q.ra) && (q.lb < q.rb)) {
    p[1 - row][np[1 - row]++] = q;
  }
}

void push_once(int target_val) {
  np[1 - row] = 0;
  for (int i = 0; i < np[row]; i++) {
    range_pair q = p[row][i];
    if (target_val == 0) {
      add_range({q.la, q.za, q.lb, q.zb});
      add_range({q.za, q.ra, q.zb, q.rb});
    } else {
      add_range({q.la, q.za, q.zb, q.rb});
      add_range({q.za, q.ra, q.lb, q.zb});
    }
  }
}

long long push_ranges() {
  p[0][0] = { 0, n, 0, n };
  row = 0;
  np[0] = 1;

  long long answer = 0;

  for (int bit = NUM_BITS - 1; bit >= 0; bit--) {
    long long z = count_zeroes(bit);
    if (z < k) {
      k -= z;
      push_once(1);
      answer += 1ll << bit;
    } else {
      push_once(0);
    }
    row = 1 - row;
  }

  return answer;
}

void solve_test_case() {
  read_data();
  std::sort(a, a + n);
  std::sort(b, b + n);
  long long ans = push_ranges();
  printf("%lld\n", ans);
}

int main() {
```

```
  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    solve_test_case();
  }

  return 0;
}
```

39.4 Problema Cipher (IIOT 2022-2023 runda internațională)

înapoi • versiune online

```
#include <algorithm>
#include <stdio.h>
#include <string.h>
#include <vector>
#include <unordered_map>

const int MAX_LEN = 200'000;
const int MAX_ROOT = 1'000;
const int SIGMA = 26;
const int NONE = -1;

// Informații despre candidatul curent și optimul găsit.
char s[MAX_LEN + 1], cand[MAX_LEN + 1], best[MAX_LEN + 1];
int best_id;

int num_queries;

// Un pointer la fiu.
struct ptr {
  int u;       // fiul
  char letter; // litera de pe muchie (0...SIGMA-1, nu 'a'...'z')
};

struct trie_node {
  std::vector<ptr> children; // listă de fii
  int id; // șirul terminat aici (doar pentru frunze)
};

trie_node nd[MAX_LEN]; // spațiu comun pentru toate trie-urile
int nd_ptr;

int circ(int x) {
```

```
  return (x >= SIGMA) ? (x - SIGMA) : x;
}

struct trie {
  int root;
  int klen;

  int get_child(int u, char letter) {
    for (ptr p: nd[u].children) {
      if (p.letter == letter) {
        return p.u;
      }
    }
    nd[u].children.push_back({++nd_ptr, letter});
    return nd_ptr;
  }

  void insert(int id, char* s) {
    int u = root;
    for (int i = 0; s[i]; i++) {
      u = get_child(u, s[i] - 'a');
    }
    if (!nd[u].id) { // dintre două chei identice, preferă ID-ul minim
      nd[u].id = id;
    }
  }

  // Literele fiind ordonate, returnează prima literă începînd cu cea dorită.
  ptr find_continuation(int u, char letter) {
    char want = circ(SIGMA - letter);
    for (ptr p: nd[u].children) {
      if (p.letter >= want) {
        return p;
      }
    }
    return nd[u].children[0];
  }

  // Construiește în cand[] cel mai mic candidat și returnează ID-ul cheii cu
  // care l-a obținut.
  int build_cand(char* s, int slen) {
    int u = root;
    for (int i = 0; i < klen; i++) {
      ptr p = find_continuation(u, s[i] - 'a');
      for (int j = i; j < slen; j += klen) {
        // s[i] se repetă din klen în klen caractere
        cand[j] = circ(s[j] - 'a' + p.letter) + 'a';
      }
      u = p.u;
    }
```

```
    return nd[u].id;
  }
};

trie t[MAX_ROOT];
std::unordered_map<int, int> len_to_trie;

int get_trie_for_string(char* s) {
  static int num_tries = 0;

  int len = strlen(s);
  auto it = len_to_trie.find(len);
  if (it == len_to_trie.end()) {
    t[num_tries] = { .root = ++nd_ptr, .klen = len };
    len_to_trie[len] = num_tries;
    return num_tries++;
  } else {
    return it->second;
  }
}

void read_key_strings_into_trie() {
  int k;
  scanf("%d %d ", &num_queries, &k);
  for (int i = 1; i <= k; i++) {
    scanf("%s ", s);
    int trie_id = get_trie_for_string(s);
    t[trie_id].insert(i, s);
  }
}

void sort_children() {
  for (int u = 1; u <= nd_ptr; u++) {
    std::sort(nd[u].children.begin(), nd[u].children.end(), [](ptr a, ptr b) {
      return a.letter < b.letter;
    });
  }
}

void try_length(char* s, int slen, int klen) {
  auto it = len_to_trie.find(klen);
  if (it != len_to_trie.end()) {
    int tree_id = it->second;
    int cand_id = t[tree_id].build_cand(s, slen);
    int cmp = strcmp(cand, best);
    if ((best_id == NONE) ||
        (cmp < 0) ||
        ((cmp == 0) && (cand_id < best_id))) {
      strcpy(best, cand);
      best_id = cand_id;
```

```
    }
  }
}

void process_queries() {
  while (num_queries--) {
    scanf("%s ", s);
    int len = strlen(s);
    best_id = NONE;

    for (int d = 1; d * d <= len; d++) {
      if (len % d == 0) {
        try_length(s, len, d);
        try_length(s, len, len / d);
      }
    }
    printf("%d\n", best_id);
  }
}

int main() {
  read_key_strings_into_trie();
  sort_children();
  process_queries();

  return 0;
}
```

Anexa 40

Probleme diverse

40.1 Problema Liars (Baraj ONI 2025)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_NODES = 5'000;
const int MOD = 666'013;

const int ANY = -1;
const int LIAR = 0;
const int TRUTH_TELLER = 1;

// Combinații posibile pentru nodul curent × părintele
const int LIAR_P_LIAR = 0;
const int LIAR_P_TT = 1;
const int TT_P_LIAR = 2;
const int TT_P_TT = 3;

struct cell {
  int v, next;
};

struct node {
  int adj;
  int num_children;
  int resp;       // numărul de vecini mincinoși declarați de u
  int assignment; // valoarea asignată lui u în soluția construită
  int ways[4];    // În câte moduri putem fixa calitatea nodului și a părintelui?
};

node nd[MAX_NODES + 1];
cell list[2 * MAX_NODES];
int n;
```

```
// Vectorul pentru combinatorică. Ne ajunge unul singur, căci îl completăm în
// fiecare nod după ce procesăm fiii. a[i] = numărul de moduri de a avea i fii
// mincinoși. Calitatea părintelui este fixată în momentul apelului. Din
// fiecare fiu vom folosi cîmpurile ways[LIAR_P_LIAR] și ways[TT_P_LIAR]
// pentru a fixa părintele ca mincinos, respectiv ways[LIAR_P_TT] și
// ways[TT_P_TT] pentru a fixa părintele ca sincer.
//
// Îl îmbrăcăm într-un struct ca să oferim funcții getter pentru indici
// incorecți.
struct combo {
  long long a[MAX_NODES + 1];
  int curr_node;

  void merge_children(int u, int parent, int c_liar, int c_tt) {
    curr_node = u;

    // Înainte de a procesa fiii, avem un singur mod de a avea 0 mincinoși.
    a[0] = 1;
    int i = 0;

    for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
      int v = list[ptr].v;
      if (v != parent) {
        i++;
        // Toți fiii sînt mincinoși.
        a[i] = a[i - 1] * nd[v].ways[c_liar] % MOD;

        for (int j = i - 1; j; j--) {
          // Aveam deja j mincinoși și adăugăm un sincer...
          long long gain_tt = a[j] * nd[v].ways[c_tt];
          // ... sau aveam j - 1 mincinoși și adăugăm un mincinos.
          long long gain_liar = a[j - 1] * nd[v].ways[c_liar];
          a[j] = (gain_tt + gain_liar) % MOD;
        }

        // Toți fiii sînt sinceri.
        a[0] = a[0] * nd[v].ways[c_tt] % MOD;
      }
    }
  }

  int get_sum() {
    long long result = 0;
    for (int i = 0; i <= nd[curr_node].num_children; i++) {
      result += a[i];
    }
    return result % MOD;
  }
```

```
  int get(int k) {
    if ((k >= 0) && (k <= nd[curr_node].num_children)) {
      return a[k];
    } else {
      return 0;
    }
  }
};

combo c;

void add_edge(int u, int v) {
  static int ptr = 1;
  list[ptr] = { v, nd[u].adj };
  nd[u].adj = ptr++;
  nd[u].num_children++;
}

void read_data() {
  FILE* f = fopen("liars.in", "r");
  fscanf(f, "%d", &n);
  for (int u = 1; u <= n; u++) {
    fscanf(f, "%d", &nd[u].resp);
  }
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    fscanf(f, "%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
  fclose(f);

  for (int u = 2; u <= n; u++) {
    nd[u].num_children--; // toate nodurile în afară de 1 au un părinte
  }
}

void count_dfs(int u, int parent) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      count_dfs(v, u);
    }
  }

  int r = nd[u].resp;
  // Cazul I: u este sincer.
  c.merge_children(u, parent, LIAR_P_TT, TT_P_TT);

  // Dacă și părintele este sincer, cei r mincinoși se află printre
```

```
  // fii.
  nd[u].ways[TT_P_TT] = c.get(r);

  // Dacă părintele este mincinos, ceilalți r - 1 mincinoși se află
  // printre fii.
  nd[u].ways[TT_P_LIAR] = c.get(r - 1);

  // Cazul II: u este mincinos.
  c.merge_children(u, parent, LIAR_P_LIAR, TT_P_LIAR);
  int sum = c.get_sum();

  // Dacă părintele este sincer, u trebuie să aibă orice în afară de r
  // mincinoși printre fii.
  nd[u].ways[LIAR_P_TT] = (sum + MOD - c.get(r)) % MOD;

  // Dacă și părintele este mincinos, u trebuie să aibă orice în afară de
  // r - 1 mincinoși printre fii.
  nd[u].ways[LIAR_P_LIAR] = (sum + MOD - c.get(r - 1)) % MOD;
}

// Atribuie toți fiii impuși.
int assign_forced(int u, int parent) {
  int c_liar = (nd[u].assignment == LIAR) ? LIAR_P_LIAR : LIAR_P_TT;
  int c_tt = (nd[u].assignment == LIAR) ? TT_P_LIAR : TT_P_TT;
  int num_assigned = 0;

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      if (!nd[v].ways[c_liar]) {
        nd[v].assignment = TRUTH_TELLER;
      } else if (!nd[v].ways[c_tt]) {
        nd[v].assignment = LIAR;
        num_assigned++;
      } else {
        nd[v].assignment = ANY;
      }
    }
  }

  return num_assigned;
}

void assign_optional(int u, int parent, int liars_left) {
  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (nd[v].assignment == ANY) {
      if (liars_left) {
        nd[v].assignment = LIAR;
        liars_left--;
```

```
      } else {
        nd[v].assignment = TRUTH_TELLER;
      }
    }
  }
}

// Atribuie valori fiilor lui u. Nodul lui u are deja valoare. Bug: dacă
// numărul de posibilități este 0 modulo MOD, el va fi considerat incorect ca
// fiind 0.
void assign_dfs(int u, int parent) {
  int assigned_liars = assign_forced(u, parent);
  // Și părintele poate minți (ce familie de cacao).
  assigned_liars += parent && (nd[parent].assignment == LIAR);

  int liars_left;
  if (nd[u].assignment == TRUTH_TELLER) {
    liars_left = nd[u].resp - assigned_liars;
  } else if (assigned_liars == nd[u].resp) {
    liars_left = 1; // Nu ne putem opri acum, căci u spune adevărul.
  } else {
    liars_left = 0;
  }

  assign_optional(u, parent, liars_left);

  for (int ptr = nd[u].adj; ptr; ptr = list[ptr].next) {
    int v = list[ptr].v;
    if (v != parent) {
      assign_dfs(v, u);
    }
  }
}

void write_answer() {
  FILE* f = fopen("liars.out", "w");
  int ans = (nd[1].ways[LIAR_P_TT] + nd[1].ways[TT_P_TT]) % MOD;
  fprintf(f, "%d\n", ans);
  for (int u = 1; u <= n; u++) {
    fprintf(f, "%d ", nd[u].assignment);
  }
  fprintf(f, "\n");
  fclose(f);
}

int main() {
  read_data();
  count_dfs(1, 0);
  nd[1].assignment = nd[1].ways[TT_P_TT] ? TRUTH_TELLER : LIAR;
  assign_dfs(1, 0);
```

```
  write_answer();

  return 0;
}
```

40.2 Problema Pavare infinită (Lot 2025)

înapoi • versiune online

```
// Implementează criteriul de translație <https://minos.tessera.li/tiling>
#include <stdio.h>
#include <unordered_set>

const int MAX_SQUARES = 2'300;
const int MAX_EDGES = 2 * MAX_SQUARES + 2; // cazul denegerat, n×1
const int MOD = 999'999'667;
const int INV_4_MOD = MOD / 4 + 1;

// Implementez propria pereche pentru că „r” și „c” înseamnă „row” și
// „column”, pe cînd „first” și „second” nu înseamnă nimic.
struct pair {
  int r, c;

  bool operator==(const pair& other) const {
    return (r == other.r) && (c == other.c);
  }

  struct hash {
    std::size_t operator()(const pair& p) const {
      return p.r * MAX_SQUARES + p.c;
    }
  };
};

struct edge {
  pair start, end;

  bool touches(pair p) {
    return (start == p) || (end == p);
  }

  void swap(edge& other) {
    edge tmp = *this;
    *this = other;
    other = tmp;
  }

  void orient_from(pair src) {
```

```
    if (start != src) {
      pair tmp = start;
      start = end;
      end = tmp;
    }
  }

  int get_dir() {
    // Este important ca left ^ right = 1 și up ^ down = 1. Mai jos folosim
    // acest artificiu ca să parcurgem conturul în ambele sensuri fără cod
    // suplimentar.
    if (end.r == start.r - 1) {
      return 0;
    } else if (end.r == start.r + 1) {
      return 1;
    } else if (end.c == start.c - 1) {
      return 2;
    } else {
      return 3;
    }
  }
};

int pow4[2 * MAX_EDGES + 1], inv_pow4[2 * MAX_EDGES + 1];

// Un șir 1-based de valori în baza 4, cu hash-uri parțiale.
struct hashed_string {
  char s[MAX_EDGES];
  int h[MAX_EDGES];
  int len;

  void clear() {
    len = 0;
  }

  void push(int c) {
    s[len] = c;
    h[len] = len
      ? ((long long)pow4[len] * c + h[len - 1]) % MOD
      : c;
    len++;
  }

  // inclusiv
  int substring_hash(int lo, int hi) {
    int before = lo ? h[lo - 1] : 0;
    return (long long)(MOD + h[hi] - before) * inv_pow4[lo] % MOD;
  }
};
```

```
// datele de intrare
std::unordered_set<pair, pair::hash> squares;

// colecția de muchii pe care ulterior o vom sorta
edge e[MAX_EDGES];
int num_edges, half_perim;

// conturul rezultat
int dir[MAX_EDGES];

// parcurgerile în cele două sensuri
hashed_string s1, s2;

// lungimile prefixelor din primul contur care sînt sufixe în al doilea
int match_left[MAX_EDGES], num_match_left;

// lungimile sufixelor din primul contur care sînt prefixe în al doilea
int match_right[MAX_EDGES], num_match_right;

void precompute_pow4() {
  pow4[0] = inv_pow4[0] = 1;
  for (int i = 1; i <= 2 * MAX_EDGES; i++) {
    pow4[i] = 4ll * pow4[i - 1] % MOD;
    inv_pow4[i] = (long long)INV_4_MOD * inv_pow4[i - 1] % MOD;
  }
}

void read_data() {
  int num_squares;
  scanf("%d", &num_squares);

  squares.clear();
  while (num_squares--) {
    pair sq;
    scanf("%d %d", &sq.r, &sq.c);
    squares.insert(sq);
  }
}

// Extrage muchiile din pătratele de la intrare.
void collect_edges() {
  num_edges = 0;
  for (pair sq: squares) {
    if (!squares.contains({sq.r - 1, sq.c})) {
      e[num_edges++] = {{sq.r - 1, sq.c - 1}, {sq.r - 1, sq.c}};
    }
    if (!squares.contains({sq.r, sq.c + 1})) {
      e[num_edges++] = {{sq.r - 1, sq.c}, {sq.r, sq.c}};
    }
    if (!squares.contains({sq.r + 1, sq.c})) {
```

```
      e[num_edges++] = {{sq.r, sq.c - 1}, {sq.r, sq.c}};
    }
    if (!squares.contains({sq.r, sq.c - 1})) {
      e[num_edges++] = {{sq.r - 1, sq.c - 1}, {sq.r, sq.c -1}};
    }
  }
  half_perim = num_edges / 2;
}

// Ordonează muchiile. Pornind cu una la întîmplare, o caută pe următoarea
// care se potrivește cap la cap cu prima etc. O(n^2) pentru că motive.
// Notează direcțiile în dir[] (coordonatele absolute nu ne mai interesează
// ulterior).
void trace_contour() {
  dir[0] = e[0].get_dir();

  int i = 1;
  do {
    // Găsește o continuare pentru e[i - 1].
    int j = i;
    while (!e[j].touches(e[i - 1].end)) {
      j++;
    }
    e[i].swap(e[j]);
    e[i].orient_from(e[i - 1].end);
    dir[i] = e[i].get_dir();
    i++;
  } while (e[i - 1].end != e[0].start);

  // Caz netratat: dacă i < num_edges, atunci nu am înnodat toate muchiile. De
  // ce? Pentru că figura are găuri.
}

// Construiește șirurile în baza 4 cu hashuri parțiale. Al doilea este dublat,
// deoarece cu o singură copie jumătatea pe care o testăm ar putea fi
// înfășurată peste finalul șirului.
void build_strings() {
  s1.clear();
  for (int i = 0; i < num_edges; i++) {
    s1.push(dir[i]);
  }

  s2.clear();
  for (int times = 0; times < 2; times++) {
    for (int i = num_edges - 1; i >= 0; i--) {
      s2.push(dir[i] ^ 1);
    }
  }
}
```

```
// Colectează prefixele din primul contur care sînt sufixe în al doilea și
// viţăvercea.
void collect_matches(int start1, int start2) {
  num_match_left = num_match_right = 0;

  int end1 = start1 + half_perim - 1;
  int end2 = start2 + half_perim - 1;

  for (int i = 1; i < half_perim; i++) {
    int h1 = s1.substring_hash(start1, start1 + i - 1);
    int h2 = s2.substring_hash(end2 - i + 1, end2);
    if (h1 == h2) {
      match_left[num_match_left++] = i;
    }

    h1 = s1.substring_hash(end1 - i + 1, end1);
    h2 = s2.substring_hash(start2, start2 + i - 1);
    if (h1 == h2) {
      match_right[num_match_right++] = i;
    }
  }
}

// Pentru două jumătăţi care încep la start1 pe primul contur, respectiv la
// start2 pe al doilea, testează dacă eliminînd l caractere din stînga și r
// din dreapta din primul contur (și invers din al doilea), bucăţile care
// rămîn sînt egale.
bool test_middle(int start1, int start2, int l, int r) {
  if (l + r > half_perim) {
    return false;
  }

  int end1 = start1 + half_perim - 1;
  int end2 = start2 + half_perim - 1;

  int h1 = s1.substring_hash(start1 + l, end1 - r);
  int h2 = s2.substring_hash(start2 + r, end2 - l);

  return (h1 == h2);
}

// Testează toate posibilităţile de a elimina prefixe și sufixe colectate
// anterior. O(n^2), dar posibilităţile sînt foarte puţine.
bool halves_match(int start1, int start2) {
  collect_matches(start1, start2);

  for (int i = 0; i < num_match_left; i++) {
    for (int j = 0; j < num_match_right; j++) {
      int l = match_left[i];
      int r = match_right[j];
```

```
      if (test_middle(start1, start2, l, r)) {
        return true;
      }
    }
  }

  return false;
}

// Dance for me, dance for me, dance for me, oh-oh-oh.
bool translation_criterion() {
  collect_edges();
  trace_contour();
  build_strings();

  for (int start1 = 0; start1 < num_edges; start1++) {
    int start2 = num_edges - start1;
    if (halves_match(start1, start2)) {
      return true;
    }
  }

  return false;
}

int main() {
  precompute_pow4();

  int num_tests;
  scanf("%d", &num_tests);

  while (num_tests--) {
    read_data();
    int answer = translation_criterion();
    puts(answer ? "Yes" : "No");
  }

  return 0;
}
```

40.3 Problema Triunghi (Lot 2024)

‹ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 3000;
```

```
struct max_info {
  int val, cnt;
};

typedef max_info mat[MAX_N][MAX_N];

mat sq[2], tr;
int orig[MAX_N][MAX_N];
int nr, nc, k;

void read_data() {
  scanf("%d %d %d", &nr, &nc, &k);

  // Citește liniile în ordine inversă astfel încît triunghiurile să se
  // extindă în ordinea crescătoare a liniilor și coloanelor.
  for (int r = nr - 1; r >= 0; r--) {
    for (int c = 0; c < nc; c++) {
      scanf("%d", &orig[r][c]);
      tr[r][c] = sq[0][r][c] = { orig[r][c], 1};
    }
  }
}

bool odd(int x) {
  return (x & 1) == 1;
}

bool even(int x) {
  return x && ((x & 1) == 0);
}

int max(int x, int y) {
  return (x > y) ? x : y;
}

max_info max3(max_info a, max_info b, max_info c) {
  int m = max(max(a.val, b.val), c.val);

  return {
    m,
    a.cnt * (a.val == m) +
    b.cnt * (b.val == m) +
    c.cnt * (c.val == m)
  };
}

max_info max4(max_info a, max_info b, max_info c, max_info d) {
  int m = max(max(max(a.val, b.val), c.val), d.val);

  return {
```

```
    m,
    a.cnt * (a.val == m) +
    b.cnt * (b.val == m) +
    c.cnt * (c.val == m) +
    d.cnt * (d.val == m)
  };
}

void combine_triangles(int size_tr, mat sq, int size_sq) {
  for (int r = 0; r <= nr - (size_tr + size_sq); r++) {
    for (int c = 0; c <= nc - (size_tr + size_sq); c++) {
      tr[r][c] = max3(sq[r][c],
                      tr[r][c + size_sq],
                      tr[r + size_sq][c]);
    }
  }
}

void combine_equal_squares(mat src, int size, mat dest) {
  for (int r = 0; r <= nr - 2 * size; r++) {
    for (int c = 0; c <= nc - 2 * size; c++) {
      dest[r][c] = max4(src[r][c],
                        src[r][c + size],
                        src[r + size][c],
                        src[r + size][c + size]);
    }
  }
}

// Calculează 11x11 din 6x6 în colțurile din stînga-sus și dreapta-jos și 5x5
// în colțurile din dreapta-sus și stînga jos. Centrul este numărat dublu.
void combine_unequal_squares(mat sm, mat lg, int size, mat dest) {
  for (int r = 0; r <= nr - (2 * size + 1); r++) {
    for (int c = 0; c <= nc - (2 * size + 1); c++) {
      dest[r][c] = max4(lg[r][c],
                        sm[r][c + size + 1],
                        sm[r + size + 1][c],
                        lg[r + size][c + size]);
      if (orig[r + size][c + size] == dest[r][c].val) {
        dest[r][c].cnt--;
      }
    }
  }
}

// Calculează toate triunghiurile de mărime t și pătratele de mărimi s0 și s1.
// s0 și s1 pot fi 0. Dacă ambele sînt nonzero, atunci s1 = s0 + 1.
// Triunghiurile sînt formate din două triunghiuri mai mici și un pătrat.
// Pătratele de latură pară sînt formate din patru pătrate mai mici. Pătratele
// impare sînt formate din două perechi de pătrate mai mici (jumătatea
```

```
// rotunjită în sus și respectiv în jos).
//
// De exemplu, t=183 se sparge în următoarele straturi. Din fiecare strat mai
// mic îl aflăm pe următorul mai mare.
//
//   compute_all(183,  0,  0)
//   compute_all( 91, 92,  0)
//   compute_all( 45, 46,  0)
//   compute_all( 22, 23,  0)
//   compute_all( 11, 11, 12)
//   compute_all(  5,  5,  6)
//   compute_all(  2,  2,  3)
//   compute_all(  1,  1,  2)
void compute_all(int t, int s0, int s1) {
  if (t == 1) {
    if (s0 == 2) {
      combine_equal_squares(sq[0], 1, sq[0]);
    } else if (s1 == 2) {
      combine_equal_squares(sq[0], 1, sq[1]);
    }
    return;
  }

  int rec_t = t / 2;
  int rec_s0 = s0 ? (s0 / 2) : ((t + 1) / 2);
  int rec_s1 = s1 ? ((s1 + 1) / 2) : (odd(s0) ? (rec_s0 + 1) : 0);
  compute_all(rec_t, rec_s0, rec_s1);

  mat& sq_for_tri = (rec_t + rec_s0 == t) ? sq[0] : sq[1];
  combine_triangles(rec_t, sq_for_tri, t - rec_t);

  // De ex. combină pătrate de laturi 10 și 11 pentru a obține pătrate de
  // laturi 20 și 21. Deoarece suprascriem, trebuie să procesăm latura impară
  // prima.
  int odd_idx = odd(s0) ? 0 : (odd(s1) ? 1 : -1);
  if (odd_idx != -1) {
    combine_unequal_squares(sq[0], sq[1], rec_s0, sq[odd_idx]);
  }

  if (even(s0)) {
    combine_equal_squares(sq[0], rec_s0, sq[0]);
  } else if (even(s1)) {
    combine_equal_squares(sq[1], rec_s1, sq[1]);
  }
}

void write_results() {
  // Răstoarnă din nou liniile în vederea tipăririi.
  for (int r = nr - k; r >= 0; r--) {
    for (int c = 0; c <= nc - k; c++) {
```

```
      printf("%d ", tr[r][c].val);
    }
    putchar('\n');
  }
  for (int r = nr - k; r >= 0; r--) {
    for (int c = 0; c <= nc - k; c++) {
      printf("%d ", tr[r][c].cnt);
    }
    putchar('\n');
  }
}

int main() {
  read_data();
  compute_all(k, 0, 0);
  write_results();

  return 0;
}
```

40.4 Problema Subiectul I („Nervi”) (Lot 2024)

‹ înapoi

Sursă în $\mathcal{O}(n^3)$ (versiune online).

```
// Complexitate: O(n^3).
#include <stdio.h>

const int MAX_N = 1'000;
const int INFINITY = 2'000'000'000;
const int DIR = 4;
const int NORTH = 0;
const int EAST = 1;
const int SOUTH = 2;
const int WEST = 3;

int max(int x, int y) {
  return (x > y) ? x : y;
}

void swap(short& a, short& b) {
  int tmp = a; a = b; b = tmp;
}

struct cell {
  int val;
  short dist[DIR]; // distanța pînă la primul element prost ordonat
```

```
  // Raza maximă a unei matrice impare centrate în celulă, respectiv a unei
  // matrice pare centrate în colțul de sud-est al celulei. Le contorizăm
  // separat, căci la rotație cele pare migrează în celula vecină.
  short odd, even;

  void improve_score(int odd, int even) {
    this->odd = max(this->odd, odd);
    this->even = max(this->even, even);
  }

  void rotate() {
    int tmp = dist[NORTH];
    dist[NORTH] = dist[EAST];
    dist[EAST] = dist[SOUTH];
    dist[SOUTH] = dist[WEST];
    dist[WEST] = tmp;
  }

  void mirror() {
    swap(dist[NORTH], dist[WEST]);
    swap(dist[SOUTH], dist[EAST]);
  }
};

cell a[MAX_N + 2][MAX_N + 2];
int n;

void swap_cells(cell& a, cell& b) {
  cell tmp = a; a = b; b = tmp;
}

void read_data() {
  scanf("%d", &n);
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      scanf("%d", &a[r][c].val);
    }
  }
}

void pad_matrix() {
  for (int i = 0; i <= n + 1; i++) {
    a[i][0].val = a[0][i].val = INFINITY;
    a[i][n + 1].val = a[n + 1][i].val = INFINITY;
  }
}

void rotate_ccw() {
  for (int r = 1; r <= n / 2; r++) {
    for (int c = 1; c <= (n + 1) / 2; c++) {
```

```
      cell tmp = a[r][c];
      a[r][c] = a[c][n + 1 - r];
      a[c][n + 1 - r] = a[n + 1 - r][n + 1 - c];
      a[n + 1 - r][n + 1 - c] = a[n + 1 - c][r];
      a[n + 1 - c][r] = tmp;
    }
  }
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      a[r][c].rotate();
      a[r][c].even = a[r + 1][c].even;
    }
  }
}

void mirror() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c < r; c++) {
      swap_cells(a[r][c], a[c][r]);
    }
  }
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      a[r][c].mirror();
    }
  }
}

void compute_north_distance() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      cell& x = a[r][c];
      cell& y = a[r - 1][c];
      x.dist[NORTH] = (x.val >= y.val)
        ? (1 + y.dist[NORTH])
        : 1;
    }
  }
}

void compute_distances() {
  for (int i = 0; i < DIR; i++) {
    compute_north_distance();
    rotate_ccw();
  }
}

// Numără doar spiralele centrate în (r,c) care pornesc spre est, apoi spre
// sud. De restul se ocupă rotațiile și oglindirea.
void count_spirals_centered_at(int r, int c) {
```

```
  a[r][c].improve_score(1, 0); // YES, WE CAN.

  int r2 = r, c2 = c, side = 1, old_side;
  do {
    old_side = side;
    if (side % 2) {
      // extinde la sud și est
      if ((a[r2][c2].val >= a[r2][c2 + 1].val) &&
          (a[r2][c2 + 1].dist[SOUTH] >= side + 1) &&
          (a[r2 + side][c2 + 1].dist[WEST] >= side + 1)) {
        r2 += side;
        c2 -= (side - 1);
        side++;
      }
    } else {
      // extinde la nord și vest
      if ((a[r2][c2].val >= a[r2][c2 - 1].val) &&
          (a[r2][c2 - 1].dist[NORTH] >= side + 1) &&
          (a[r2 - side][c2 - 1].dist[EAST] >= side + 1)) {
        r2 -= side;
        c2 += (side - 1);
        side++;
      }
    }
  } while (side > old_side);

  a[r][c].improve_score((side + 1) / 2, side / 2);
}

void count_clockwise_spirals() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      count_spirals_centered_at(r, c);
    }
  }
}

void count_spirals() {
  for (int i = 0; i < 2; i++) {
    compute_distances();
    for (int j = 0; j < DIR; j++) {
      count_clockwise_spirals();
      rotate_ccw();
    }
    mirror();
  }
}

unsigned sum_spirals() {
  unsigned result = 0;
```

```
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      result += a[r][c].odd + a[r][c].even;
    }
  }
  return result;
}

int main() {
  read_data();
  pad_matrix();
  count_spirals();
  printf("%u\n", sum_spirals());

  return 0;
}
```

Sursă în $\mathcal{O}(n^2)$ (versiune online).

```
// Complexitate: O(n^2).
#include <stdio.h>

const int MAX_N = 1'000;
const int INFINITY = 2'000'000'000;
const int DIR = 4;
const int NORTH = 0;
const int EAST = 1;
const int SOUTH = 2;
const int WEST = 3;

int max(int x, int y) {
  return (x > y) ? x : y;
}

int min(int a, int b) {
  return (a < b) ? a : b;
}

int min6(int a, int b, int c, int d, int e, int f) {
  return min(a, min(b, min(c, min(d, min(e, f)))));
}

void swap(short& a, short& b) {
  int tmp = a; a = b; b = tmp;
}

struct cell {
  int val;
  short tri[DIR];
```

```
  // Raza maximă a unei matrice impare centrate în celulă, respectiv a unei
  // matrice pare centrate în colțul de sud-est al celulei.
  short odd, even;

  void improve_score(int odd, int even) {
    this->odd = max(this->odd, odd);
    this->even = max(this->even, even);
  }

  void rotate() {
    int tmp = tri[NORTH];
    tri[NORTH] = tri[EAST];
    tri[EAST] = tri[SOUTH];
    tri[SOUTH] = tri[WEST];
    tri[WEST] = tmp;
  }

  void mirror() {
    swap(tri[NORTH], tri[WEST]);
    swap(tri[SOUTH], tri[EAST]);
  }
};

cell a[MAX_N + 2][MAX_N + 2];
int n;

void swap_cells(cell& a, cell& b) {
  cell tmp = a; a = b; b = tmp;
}

void read_data() {
  scanf("%d", &n);
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      scanf("%d", &a[r][c].val);
    }
  }
}

void pad_matrix() {
  for (int i = 0; i <= n + 1; i++) {
    a[i][0].val = a[0][i].val = INFINITY;
    a[i][n + 1].val = a[n + 1][i].val = INFINITY;
  }
}

void rotate_ccw() {
  for (int r = 1; r <= n / 2; r++) {
    for (int c = 1; c <= (n + 1) / 2; c++) {
      cell tmp = a[r][c];
```

```
      a[r][c] = a[c][n + 1 - r];
      a[c][n + 1 - r] = a[n + 1 - r][n + 1 - c];
      a[n + 1 - r][n + 1 - c] = a[n + 1 - c][r];
      a[n + 1 - c][r] = tmp;
    }
  }
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      a[r][c].rotate();
      a[r][c].even = a[r + 1][c].even;
    }
  }
}

void mirror() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c < r; c++) {
      swap_cells(a[r][c], a[c][r]);
    }
  }
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      a[r][c].mirror();
    }
  }
}

bool decreasing(cell& x, cell& y, cell& z) {
  return (x.val >= y.val) && (y.val >= z.val);
}

void compute_north_triangle() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      cell& x = a[r - 1][c - 1];
      cell& y = a[r - 1][c];
      cell& z = a[r - 1][c + 1];
      a[r][c].tri[NORTH] = decreasing(x, y, z)
        ? (1 + min(x.tri[NORTH], z.tri[NORTH]))
        : 1;
    }
  }
}

void compute_triangles() {
  for (int i = 0; i < DIR; i++) {
    compute_north_triangle();
    if (i < DIR - 1) {
      rotate_ccw();
    }
```

```
  }
}

// Spirala impară se termină în colțul de nord-est. Încearcă să adaugi un
// colțar pe laturile de est, apoi sud.
bool can_extend_to_even_size(int r, int c, int radius) {
  return
    (a[r + 1][c + 1].val >= a[r + 1][c].val) &&
    (a[r][c + 1].tri[NORTH] >= radius) &&
    (a[r][c + 1].tri[EAST] >= radius) &&
    (a[r + 1][c + 1].tri[EAST] >= radius) &&
    (a[r + 1][c + 1].tri[SOUTH] >= radius) &&
    (a[r + 1][c].tri[SOUTH] >= radius);
}

// Numără doar spiralele centrate în (r,c) care pornesc spre est, apoi spre
// sud. De restul se ocupă rotațiile și oglindirea.
void count_spirals_centered_at(int r, int c) {
  if (!decreasing(a[r][c], a[r][c + 1], a[r + 1][c + 1])) {
    a[r][c].improve_score(1, 0);
    return;
  }
  int odd = min6(a[r][c].tri[NORTH],
                 a[r][c].tri[WEST],
                 a[r][c].tri[SOUTH],
                 a[r][c + 1].tri[NORTH] + 1,
                 a[r][c + 1].tri[EAST] + 1,
                 a[r + 1][c + 1].tri[EAST] + 1);
  int even = (odd - 1) + can_extend_to_even_size(r, c, odd);
  a[r][c].improve_score(odd, even);
}

void count_clockwise_spirals() {
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      count_spirals_centered_at(r, c);
    }
  }
}

void count_spirals() {
  for (int i = 0; i < 2; i++) {
    compute_triangles();
    for (int j = 0; j < DIR; j++) {
      count_clockwise_spirals();
      if (j < DIR - 1) {
        rotate_ccw();
      }
    }
    if (!i) {
```

```
      mirror();
    }
  }
}

unsigned sum_spirals() {
  unsigned result = 0;
  for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= n; c++) {
      result += a[r][c].odd + a[r][c].even;
    }
  }
  return result;
}

int main() {
  read_data();
  pad_matrix();
  count_spirals();
  printf("%u\n", sum_spirals());

  return 0;
}
```

40.5 Problema Subiectul III („Single crossing”) (Lot 2024)

‹ înapoi • versiune online

```
// Complexitate: O(mn log n).
#include <algorithm>
#include <stdio.h>
#include <vector>

const int MAX_MN = 1000000;
const int LEFT = 0;
const int CENTER = 1;
const int RIGHT = 2;

int n, m;

int max(int x, int y) {
  return (x > y) ? x : y;
}

struct max_fenwick {
  int v[MAX_MN + 1], n;

  void init(int n) {
```

```
    this->n = n;
    for (int i = 1; i <= n; i++) {
      v[i] = 0;
    }
  }

  void set(int pos, int val) {
    do {
      v[pos] = max(v[pos], val);
      pos += pos & -pos;
    } while (pos <= n);
  }

  int partial_max(int pos) {
    int m = 0;
    while (pos) {
      m = max(m, v[pos]);
      pos &= pos - 1;
    }
    return m;
  }
};

struct perm {
  int* el;
  int* inv; // inv[i] = poziția lui i în el
  std::vector<int> orig; // indicii originali ai permutărilor duplicate

  void compute_inv() {
    for (int i = 0; i < m; i++) {
      inv[el[i]] = i;
    }
  }

  bool before(int a, int b) {
    return inv[a] < inv[b];
  }

  int get_first_difference(perm other) {
    int i = 0;
    while ((i < m) && (el[i] == other.el[i])) {
      i++;
    }
    return i;
  }

  bool operator ==(perm other) {
    return get_first_difference(other) == m;
  }
};
```

```
int buf[MAX_MN], ibuf[MAX_MN];
perm p[MAX_MN];
max_fenwick fen;

void read_data() {
  scanf("%d %d", &n, &m);
  for (int i = 0; i < n; i++) {
    p[i].el = buf + i * m;
    p[i].inv = ibuf + i * m;
    p[i].orig = { i };
    for (int j = 0; j < m; j++) {
      scanf("%d", &p[i].el[j]);
      p[i].el[j]--;
    }
    p[i].compute_inv();
  }
}

void group_duplicates() {
  std::sort(p, p + n, [](perm x, perm y) {
    int pos = x.get_first_difference(y);
    return (pos < m) && (x.el[pos] < y.el[pos]);
  });

  int i = 0;
  for (int j = 0; j < n; j++) {
    if (i && (p[i - 1] == p[j])) {
      p[i - 1].orig.push_back(p[j].orig[0]);
    } else {
      p[i++] = p[j];
    }
  }

  n = i;

  for (int i = 0; i < n; i++) {
    std::sort(p[i].orig.begin(), p[i].orig.end());
  }
}

void get_discriminant(perm x, perm y, int& a, int& b) {
  int pos = x.get_first_difference(y);
  a = x.el[pos];
  b = y.el[pos];
}

int relative_to(perm x, perm y, perm z) {
  int a, b, c, d;
  get_discriminant(x, y, a, b);
```

```
  if (z.before(a, b)) {
    get_discriminant(x, z, c, d);
    return y.before(c, d) ? LEFT : CENTER;
  } else {
    get_discriminant(y, z, c, d);
    return x.before(c, d) ? RIGHT : CENTER;
  }
}

// Alege două permutări x și y și partiționează [0...n) în 3.
//
// Invariant:
// * p[0...x) sînt < x
// * p[x+1...i) sînt între x și y
// * p[i] este permutarea evaluată în prezent
// * p[i+1...y) sînt permutări încă neevaluate
// * p[y+1...n) sînt > y
void three_way_partition(int& x, int& y) {
  x = 0;
  y = n - 1;
  int i = 1;
  perm tmp;
  while (i < y) {
    switch (relative_to(p[x], p[y], p[i])) {
      case LEFT:
        tmp = p[i];
        p[i] = p[x + 1];
        p[x + 1] = p[x];
        p[x] = tmp;
        x++;
        break;
      case CENTER:
        // Las-o, las-o, las-o-așa
        i++;
        break;
      case RIGHT:
        tmp = p[i];
        p[i] = p[y - 1];
        p[y - 1] = p[y];
        p[y] = tmp;
        y--;
        break;
    }
  }
}

// Quicksort relativ la un pivot ales, x, și la una dintre margini.
//
// Invariant:
// * p[l...x) sînt < x
```

```
// * p[x] este pivotul
// * p[x+1] este permutarea evaluată în prezent
// * p[x+1...y) sînt permutări încă neevaluate
// * p[y...r) sînt > x
void quicksort(int l, int r) { // [l, r)
  if (r - l <= 1) {
    return;
  }

  int x = l, y = r;
  while (x + 1 < y) {
    bool go_left =
      ((l > 0) && (relative_to(p[l - 1], p[x], p[x + 1]) != RIGHT)) ||
      ((r < n) && (relative_to(p[x], p[r], p[x + 1]) == LEFT));
    perm tmp = p[x + 1];

    if (go_left) {
      p[x + 1] = p[x];
      p[x] = tmp;
      x++;
    } else {
      y--;
      p[x + 1] = p[y];
      p[y] = tmp;
    }
  }

  quicksort(l, x);
  quicksort(x + 1, r);
}

void quicksort_driver() {
  int x, y;
  three_way_partition(x, y);
  quicksort(0, x);
  quicksort(x + 1, y);
  quicksort(y + 1, n);
}

void reverse_if_needed() {
  if (p[0].orig[0] > p[n - 1].orig[0]) {
    for (int i = 0, j = n - 1; i < j; i++, j--) {
      perm tmp = p[i];
      p[i] = p[j];
      p[j] = tmp;
    }
  }
}

bool has_double_crossing(perm x, perm y) {
```

```
  fen.init(m);
  for (int pos = 0; pos < m; pos++) {
    int val = p[0].el[pos];
    int from = x.inv[val] + 1; // +1 pentru arborele Fenwick
    int to = y.inv[val] + 1;
    if (fen.partial_max(to) > from) {
      return true;
    }
    fen.set(to, from);
  }

  return false;
}

bool all_single_crossings() {
  for (int i = 1; i < n - 1; i++) {
    if (has_double_crossing(p[i], p[i + 1])) {
      return false;
    }
  }

  return true;
}

void write_solution() {
  for (int i = 0; i < n; i++) {
    for (int x: p[i].orig) {
      printf("%d ", x + 1);
    }
  }
  printf("\n");
}

void write_no_solution() {
  printf("-1\n");
}

int main() {
  int num_tests;

  scanf("%d", &num_tests);
  while (num_tests--) {
    read_data();
    group_duplicates();
    quicksort_driver();
    reverse_if_needed();
    if (all_single_crossings()) {
      write_solution();
    } else {
      write_no_solution();
```

```
    }
  }

  return 0;
}
```

40.6 Problema Gurger Brill (Lot 2024)

‹ înapoi • versiune online

```
// Complexitate: O(N log N) și exact 4  log N  întîlniri. Pentru a trece unele
// teste adversariale, trece totul printr-o permutare aleatorie.
#include <algorithm>
#include <random>
#include "gurgerbrill.h"

const int MAX_N = 100'000;

// Vezi comentariile pentru datele din BIT_CONFIG.
const int BIT_CONFIG[2][3][3] = {
  { { 3, 1, 0 }, { 1, 0, 0 }, { 0, 0, 0 } },
  { { 0, 0, 3 }, { 0, 3, 1 }, { 3, 1, 0 } },
};

std::vector<int> perm, question, resp, sol;
int hp[MAX_N], hp1[MAX_N + 1]; // permutare ascunsă și inversa ei
unsigned char left1[MAX_N], left2[MAX_N];
bool seen[MAX_N];
int masks[MAX_N][2];
int n;

void gen_hidden_perm() {
  std::mt19937 rng(0);
  for (int i = 0; i < n; i++) {
    hp[i] = i + 1;
  }
  std::shuffle(hp, hp + n, rng);
  for (int i = 0; i < n; i++) {
    hp1[hp[i]] = i;
  }
}

// Trece permutarea prin cea ascunsă. Apelează query(). Traduce
// răspunsul înapoi în notația noastră.
void wrap_meeting() {
  for (int i = 0; i < n; i++) {
    question[i] = hp[perm[i]];
  }
```

```
  resp = query(question);
  for (int i = 0; i < (int)resp.size(); i++) {
    resp[i] = hp1[resp[i]];
  }
}

// Separă valorile [0...n) după valoarea bitului b. Returnează numărul de
// valori din stînga.
int partition_by_bit(int b) {
  int left = 0, right = n;
  for (int i = 0; i < n; i++) {
    if (i & (1 << b)) {
      perm[--right] = i;
    } else {
      perm[left++] = i;
    }
  }

  return left;
}

// Face două interogări, pentru perm și pentru perm răsturnat. Scrie în dest
// numărul de vecini din stînga ai fiecărei persoane.
void two_way_query(unsigned char* dest) {
  for (int i = 0; i < n; i++) {
    dest[i] = 1;
  }
  wrap_meeting();
  for (int early_person: resp) {
    dest[early_person] = 0;
  }

  std::reverse(perm.begin(), perm.end());

  wrap_meeting();
  for (int early_person: resp) {
    dest[early_person] = 2;
  }

  std::reverse(perm.begin(), perm.end());

}

void deduce_kth_bit(int b) {
  for (int i = 0; i < n; i++) {
    int my_side = (i >> b) & 1;
    int mask = BIT_CONFIG[my_side][left1[i]][left2[i]];
    masks[i][0] |= (mask & 1) << b;
    masks[i][1] |= (mask >> 1) << b;
  }
```

```
}

int get_num_bits(int n) {
  return 32 - __builtin_clz(n - 1);
}

// Face cîte două perechi de interogări, pe baza cărora află biții vecinilor
// fiecărei persoane.
void build_neighbor_bits() {
  int num_bits = get_num_bits(n);
  for (int b = 0; b < num_bits; b++) {
    int sep = partition_by_bit(b);
    two_way_query(left1);
    std::reverse(perm.begin(), perm.begin() + sep);
    std::reverse(perm.begin() + sep, perm.end());
    two_way_query(left2);
    deduce_kth_bit(b);
  }
}

// Caută persoana cu numărul minim de biți variabili între cei doi vecini.
int find_most_constrained_person() {
  int min_pop = MAX_N, min_person = MAX_N;
  for (int i = 0; i < n; i++) {
    int x = masks[i][0] ^ masks[i][1];
    int pc = __builtin_popcount(x);
    if (pc < min_pop) {
      min_pop = pc;
      min_person = i;
    }
  }

  return min_person;
}

bool is_submask(int submask, int mask) {
  return (mask & submask) == submask;
}

// Returnează true dacă și numai dacă masca mask respectă cerințele pe biți
// pentru vecinii lui person.
bool fits_in(int mask, int person) {
  return
    is_submask(mask, masks[person][0] | masks[person][1]) &&
    is_submask(~mask, ~masks[person][0] | ~masks[person][1]);
}

// Dîndu-se un vecin al lui person (care îi respectă cerințele pe biți), îl
// returnează pe celălalt.
int other_neighbor(int person, int first_neighbor) {
```

```
  return masks[person][0] ^ masks[person][1] ^ first_neighbor;
}

// Returnează true dacă și numai dacă, începînd cu persoanele A și B, restul
// mesei poate fi completat.
bool complete_circle(int a, int b) {
  for (int i = 0; i < n; i++) {
    seen[i] = false;
  }
  seen[a] = true;
  sol[0] = a;

  int num_seen = 1;
  while ((b < n) && !seen[b]) {
    seen[b] = true;
    sol[num_seen++] = b;
    int tmp = other_neighbor(b, a);
    a = b;
    b = tmp;
  }

  return (num_seen == n);
}

// Caută primul vecin îl lui first astfel încît să putem completa masa.
void try_all_neighbors(int first) {
  sol.resize(n);
  int i = 0;
  while ((i == first) ||
         !fits_in(i, first) ||
         !complete_circle(first, i)) {
    i++;
  }
}

// Trece și soluția prin permutarea ascunsă.
void wrap_answer() {
  for (int i = 0; i < n; i++) {
    sol[i] = hp[sol[i]];
  }
}

std::vector<int> find_places(int N) {
  n = N;
  perm.resize(n);
  question.resize(n);
  gen_hidden_perm();
  build_neighbor_bits();
  int first = find_most_constrained_person();
  try_all_neighbors(first);
```

```
  wrap_answer();
  return sol;
}
```

40.7 Problema Takara (Baraj ONI 2019)

înapoi

Sursă în $\mathcal{O}(n^3)$ (versiune online).

```
#include <algorithm>
#include <stdio.h>

const int MAX_N = 5'000;

int n;
int c[MAX_N];
int d[MAX_N][MAX_N];

void read_data() {
  FILE *f = fopen("takara.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &c[i]);
  }
  fclose(f);
}

void compute_costs() {
  for (int i = n - 1; i >= 0; i--) {
    for (int j = i + 1; j < n; j++) {
      d[i][j] = std::min(c[i] + d[i + 1][j], c[j] + d[i][j - 1]);
      for (int k = i + 1; k < j; k++) {
        d[i][j] = std::min(d[i][j],
                           c[k] + std::max(d[i][k - 1], d[k + 1][j]));
      }
    }
  }
}

void write_result() {
  FILE* f = fopen("takara.out", "w");
  fprintf(f, "%d\n", d[0][n - 1]);
  fclose(f);
}

int main() {
  read_data();
```

```
  compute_costs();
  write_result();

  return 0;
}
```

Sursă în $\mathcal{O}(n^2)$ (versiune online).

```
#include <algorithm>
#include <deque>
#include <stdio.h>

const int MAX_N = 5'000;

int n;
int c[MAX_N];
int d[MAX_N][MAX_N];

struct row_queue {
  std::deque<int> q;
  int i;

  void init(int i) {
    this->i = i;
    q.clear();
    q.push_back(i);
  }

  int cost(int j) {
    return d[i][j - 1] + c[j];
  }

  int first() {
    return cost(q.front());
  }

  void advance(int j) {
    while (d[i][q.front() - 1] < d[q.front() + 1][j]) {
      q.pop_front();
    }
    int z = cost(j);
    while (!q.empty() && (z < cost(q.back()))) {
      q.pop_back();
    }
    q.push_back(j);
  }
};

struct col_queue {
```

```
  std::deque<int> q;
  int j;

  void init(int j) {
    this->j = j;
    q.push_back(j);
  }

  int cost(int i) {
    return c[i] + d[i + 1][j];
  }

  int first() {
    return cost(q.front());
  }

  void advance(int i) {
    while (d[i][q.front() - 1] > d[q.front() + 1][j]) {
      q.pop_front();
    }
    int z = cost(i);
    while (!q.empty() && (z < cost(q.back()))) {
      q.pop_back();
    }
    q.push_back(i);
  }
};

row_queue rq;
col_queue cq[MAX_N];

void read_data() {
  FILE *f = fopen("takara.in", "r");
  fscanf(f, "%d", &n);
  for (int i = 0; i < n; i++) {
    fscanf(f, "%d", &c[i]);
  }
  fclose(f);
}

void compute_costs() {
  for (int i = n - 1; i >= 0; i--) {
    rq.init(i);
    cq[i].init(i);

    for (int j = i + 1; j < n; j++) {
      rq.advance(j);
      cq[j].advance(i);
      d[i][j] = std::min(rq.first(), cq[j].first());
    }
```

```
  }
}

void write_result() {
  FILE* f = fopen("takara.out", "w");
  fprintf(f, "%d\n", d[0][n - 1]);
  fclose(f);
}

int main() {
  read_data();
  compute_costs();
  write_result();

  return 0;
}
```

40.8 Problema Tornade (Baraj ONI 2025)

‹ înapoi • versiune online

```
// Complexitate: O(n).
#include <stdio.h>

const int MAX_N = 1'000'000;
const int T_LOCAL_MIN = 0;
const int T_LOCAL_MAX = 1;
const int T_MIDDLE = 2;

// O serie de elemente egale. Contract:
//
// * Lungimea reală a unei serii este cnt + global_min_cnt pentru minime
//   locale, cnt + global_max_cnt pentru maxime locale și cnt simplu pentru
//   valori intermediare.
//
// * Un optim local aparține listei de minime sau de maxime corespunzătoare
//   valorii cnt. Valorile intermediare nu fac parte din nicio listă pe
//   lungimi.
struct run {
  int val, cnt;
  char type;
  int gprev, gnext; // pointeri în lista înlănțuită globală
  int sprev, snext; // pointeri în lista înlănțuită de optime locale (dacă există)
  int self;         // indicele propriu în r[], pentru înlănțuire

  void add_to_size_list(int& head);
  void classify();
  void remove_from_size_list();
```

```
  void declassify();
  void decrease_cnt();
  void remove_from_global_list();
  void merge_with_next();
};

run r[MAX_N + 1]; // indexat de la 1; 0 înseamnă nul
int global_min_cnt, global_max_cnt;
int ghead, gtail;
int min_head[MAX_N], max_head[MAX_N];
int n;

int mod(int x) {
  return (x < 0) ? (x + n) : x;
}

void run::add_to_size_list(int& head) {
  if (head) {
    r[head].sprev = self;
    snext = head;
  }
  head = self;
}

void run::classify() {
  if ((!gprev || (val < r[gprev].val)) &&
      (!gnext || (val < r[gnext].val))) {
    type = T_LOCAL_MIN;
    cnt -= global_min_cnt;
    this->add_to_size_list(min_head[mod(cnt)]);
  } else if ((!gprev || (val > r[gprev].val)) &&
             (!gnext || (val > r[gnext].val))) {
    type = T_LOCAL_MAX;
    cnt -= global_max_cnt;
    this->add_to_size_list(max_head[mod(cnt)]);
  } else {
    type = T_MIDDLE;
  }
}

void run::remove_from_size_list() {
  if (type == T_MIDDLE) {
    return;
  }

  int m = mod(cnt);
  int& head = (type == T_LOCAL_MIN) ? min_head[m] : max_head[m];

  if (sprev) {
    r[sprev].snext = snext;
```

```
  } else {
    head = snext;
  }

  if (snext) {
    r[snext].sprev = sprev;
  }

  sprev = snext = 0;
}

void run::declassify() {
  remove_from_size_list();
  if (type == T_LOCAL_MIN) {
    cnt += global_min_cnt;
  } else if (type == T_LOCAL_MAX) {
    cnt += global_max_cnt;
  }
  type = T_MIDDLE;
}

// Cînd cnt se schimbă, seria migrează la o nouă listă pe lungimi.
void run::decrease_cnt() {
  declassify();
  cnt--;
  classify();
}

void run::remove_from_global_list() {
  if (gprev) {
    r[gprev].gnext = gnext;
  } else {
    ghead = gnext;
  }

  if (gnext) {
    r[gnext].gprev = gprev;
  } else {
    gtail = gprev;
  }
}

void run::merge_with_next() {
  cnt += r[gnext].cnt;
  r[gnext].remove_from_global_list();
}

void add_run(int val, int cnt) {
  static int ptr = 1;
```

```
  gtail = ptr;
  r[ptr].val = val;
  r[ptr].cnt = cnt;
  r[ptr].self = ptr;
  if (ptr > 1) {
    r[ptr - 1].gnext = ptr;
    r[ptr].gprev = ptr - 1;
  }
  ptr++;
}

void read_array(FILE* f) {
  int prev, val, cnt = 1;

  fscanf(f, "%d %d", &n, &prev);
  ghead = 1;
  for (int i = 1; i < n; i++) {
    fscanf(f, "%d ", &val);
    if (val == prev) {
      cnt++;
    } else {
      add_run(prev, cnt);
      prev = val;
      cnt = 1;
    }
  }
  add_run(prev, cnt);
}

void init_local_optima() {
  for (int ptr = ghead; ptr; ptr = r[ptr].gnext) {
    r[ptr].classify();
  }
}

void delete_local_optimum(int i) {
  int prev = r[i].gprev;
  int next = r[i].gnext;

  r[i].remove_from_global_list();

  if (prev) {
    r[prev].declassify();
  }
  if (next) {
    r[next].declassify();
  }

  if (prev && next && (r[prev].val == r[next].val)) {
    r[prev].merge_with_next();
```

```
    r[prev].classify();
  } else {
    if (prev) {
      r[prev].classify();
    }
    if (next) {
      r[next].classify();
    }
  }
}

void delete_zero_size_list(int& head) {
  for (int ptr = head; ptr; ptr = r[ptr].snext) {
    delete_local_optimum(ptr);
  }
  head = 0;
}

void operation(int& best_cnt, int& worst_cnt, int type, int* head) {
  best_cnt++;

  if (r[ghead].type == type) {
    r[ghead].decrease_cnt();
  }
  if (r[gtail].type == type) {
    r[gtail].decrease_cnt();
  }

  worst_cnt--;
  int pos = mod(-worst_cnt);
  delete_zero_size_list(head[pos]);
}

// Procesează operații până când ajungem la o singură serie cu elemente
// egale. Poate fi nevoie sau nu să procesăm toate operațiile M și m. Astfel
// evităm cazul particular în care ultima serie rămasă este simultan maxim și
// minim local.
void build_pyramid(FILE* f) {
  while (r[ghead].gnext) {
    if (fgetc(f) == 'm') {
      operation(global_min_cnt, global_max_cnt, T_LOCAL_MIN, max_head);
    } else {
      operation(global_max_cnt, global_min_cnt, T_LOCAL_MAX, min_head);
    }
  }
}

int main() {
  FILE* fin = fopen("tornade.in", "r");
  read_array(fin);
```

```
  init_local_optima();
  build_pyramid(fin);
  fclose(fin);

  FILE* fout = fopen("tornade.out", "w");
  fprintf(fout, "%d\n", r[ghead].val);
  fclose(fout);

  return 0;
}
```

40.9 Problema As de Spade (Lot 2024)

‹ înapoi • versiune online

Un *pretty-printer*.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

int main() {
  long long a, b;
  std::cin >> a >> b;

  std::vector<std::string> v(b - a + 1);
  for (long long i = a; i <= b; i++) {
    v[i - a] = std::to_string(i);
  }

  std::cerr << "Sortez...\n";
  std::sort(v.begin(), v.end(), [](std::string& a, std:: string& b) {
    return a + b > b + a || (a + b == b + a && a > b);
  });

  int len = 0;
  for (int i = 0; i < (int)v.size(); i++) {
    std::cout << ':' << v[i] << " @ " << len << '\n';
    len += v[i].size();
  }
  std::cout << "\n";

  return 0;
}
```

Sursa propriu-zisă.

```cpp
// Complexitate: O(B D^2) per interogare, unde B=10, D=18.
#include <algorithm>
#include <stdio.h>

const int MAX_DIGITS = 19;

typedef unsigned long long u64;
typedef unsigned __int128 u128;

u64 p10[MAX_DIGITS];

struct number {
  u64 x;
  unsigned char len;

  number() { }

  number(u64 x) {
    this->x = x;

    len = 0;
    while (x) {
      len++;
      x /= 10;
    }
  }

  void append(int digit) {
    x = x * 10 + digit;
    len++;
  }

  void times10() {
    x *= 10;
    len++;
  }

  void divide10() {
    x /= 10;
    len--;
  }

  int get_kth_digit_from_left(int k) {
    return x / p10[len - 1 - k] % 10;
  }

  // Returnează true dacă și numai dacă aparținem zonei lui z. Exemplu: 523
  // este în zona lui 5235235235 pentru că, dacă îl extindem ciclic pe 523 la
  // 10 cifre, obținem 5235235235. Putem răspunde în O(1) dacă reformulăm
```

```
  // întrebarea: verifică dacă primele 7 cifre ale lui 5235235235 corespund cu
  // ultimele 7 cifre.
  bool in_zone_of(number z) {
    return z.x / p10[len] == z.x % p10[z.len - len];
  }

  // 44644 vine înaintea lui 446444, dar 442444 vine înaintea lui 44244.
  bool before(number other) {
    u128 me_first = (u128)x * p10[other.len] + other.x;
    u128 other_first = (u128)other.x * p10[len] + x;
    return (me_first > other_first) ||
      ((me_first == other_first) && (len > other.len));
  }
};

number a, b;

void compute_powers() {
  p10[0] = 1;
  for (int i = 1; i < MAX_DIGITS; i++) {
    p10[i] = p10[i - 1] * 10;
  }
}

u64 min(u64 x, u64 y) {
  return (x < y) ? x : y;
}

// Returnează lungimea totală a prefixelor lui p din zona lui p. De exemplu,
// pentru p=2323, returnează 2 (pentru numărul 23).
u64 len_prefix_in_prefix(number p, number b) {
  int count = 0;

  number prefix = p;
  do {
    prefix.divide10();
  } while (prefix.x > b.x);

  while (prefix.len) {
    if (prefix.in_zone_of(p)) {
      count += prefix.len;
    }
    prefix.divide10();
  }

  return count;
}

// Returnează suma lungimilor zonelor x ≤ b care încep cu prefix. Include și
// prefixele lui prefix din aceeași zonă.
```

```
u64 len_prefix_up_to(number prefix, number b) {
  u64 result = len_prefix_in_prefix(prefix, b);
  u64 p = 1;

  while (prefix.x <= b.x) {
    // prefix = 123, b = 523847 --> 1000 variante de lungime 6, dar
    // prefix = 523, b = 523847 -->  848 variante de lungime 6
    result += min(p, b.x - prefix.x + 1) * prefix.len;
    prefix.times10();
    p *= 10;
  }

  return result;
}

// Returnează suma lungimilor zonelor a ≤ x ≤ b care încep cu prefix.
u64 len_prefix(number prefix) {
  return len_prefix_up_to(prefix, b) - len_prefix_up_to(prefix, a);
}

// Returnează a k-a cifră a concatenării numerelor din v.
int get_digit_from_set(number* v, u64 k) {
  int i = 0;
  while (k >= v[i].len) {
    k -= v[i].len;
    i++;
  }

  return v[i].get_kth_digit_from_left(k);
}

// Dată fiind o zonă ca {232323, 2323, 23}, returnează a k-a cifră a
// concatenării. Ordinea poate fi complexă. De exemplu, pentru n=894848948,
// prefixul 89484894 îl precede pe n, dar prefixul 89484 îl urmează. De aceea,
// sortăm numerele naiv.
int get_digit_from_zone(number n, u64 k) {
  number v[MAX_DIGITS];
  number prefix = n;
  int num_numbers = 0;

  // Colectează prefixele.
  do {
    if (prefix.in_zone_of(n)) {
      v[num_numbers++] = prefix;
    }
    prefix.divide10();
  } while (prefix.x > a.x);

  std::sort(v, v + num_numbers, [](number x, number y) {
    return x.before(y);
```

```
  });

  return get_digit_from_set(v, k);
}

// Umple o zonă de b.len cifre. Procedează de la stînga la dreapta. Primele
// întrebări sînt: care este lungimea totală a zonelor care încep cu 9, 8, 7
// ș.a.m.d.? Aceasta identifică prima cifră. Dacă, de exemplu, prima cifră
// este 5, următoarele întrebări sînt: care este lungimea totală a zonelor
// care încep cu 59, 58, 57 ș.a.m.d.?
int get_digit_at(u64 pos) {
  number prefix(0);
  while (prefix.x * 10 <= b.x) {
    prefix.append(9);
    u64 count;
    while ((count = len_prefix(prefix)) <= pos) {
      pos -= count;
      prefix.x--;
    }
  }

  return get_digit_from_zone(prefix, pos);
}

int main() {
  compute_powers();

  int num_queries;
  u64 _a, _b;

  scanf("%llu %llu %d", &_a, &_b, &num_queries);
  _a--; // operăm pe (a, b]
  a = number(_a);
  b = number(_b);

  while (num_queries--) {
    u64 pos;
    scanf("%llu", &pos);
    putchar('0' + get_digit_at(pos));
  }
  putchar('\n');

  return 0;
}
```

40.10 Problema Scurtcircuit (Lot 2025)

‹ înapoi • versiune online

```cpp
#include <stdio.h>
#include <string.h>
#include <vector>

const int MAX_K = 100'000;
const int MAX_LOG = 17;
const int MAX_CELLS = 2'500'000;

int OP_AND = 0;
int OP_OR = 1;
int OP_XOR = 2;

const char OP_STRINGS[3][5] = { "0001", "0111", "0110" };

struct cell {
  int op, pos1, pos2;
};

// Indicii celulelor unde stocăm biții de generare / propagare pentru o zonă.
struct gen_prop {
  int g, p;
};

// O matrice de biți indexată după coloane. mat[col] este coloana de biți de
// pe poziția col (de valoare 2^col).
std::vector<int> mat[MAX_K + MAX_LOG];
std::vector<int> new_mat[MAX_K + MAX_LOG];

// GP[p][i] = valorile generate și propagate de biții i-2^p+1...i
gen_prop GP[MAX_LOG][MAX_K + MAX_LOG];

// cells reține suplimentare generate, fără cele n·k de la intrare.
cell cells[MAX_CELLS];
int n, k, orig_k, pos_zero, num_cells, expected_digits, last_gp_row;

void read_data() {
  scanf("%d %d", &n, &k);
}

void init_bit_matrix() {
  for (int r = 0; r < n; r++) {
    for (int c = 0; c < k; c++) {
      mat[c].push_back(r * k + c);
    }
  }
}

int new_cell(int op, int pos1, int pos2) {
  cells[num_cells] = { op, pos1, pos2 };
```

```
  return num_cells++ + n * orig_k;
}

int ceil_log2(int n) {
  return 32 - __builtin_clz(n - 1);
}

void init_cells() {
  orig_k = k;
  expected_digits = k + ceil_log2(n);
  pos_zero = new_cell(OP_XOR, 0, 0); // x^x = 0
}

// Calculează suma a 3 biți. Rezultatul va avea 2 biți. Cel mai semnificativ
// este c_out (transportul), cel mai puțin semnificativ este s. Are adîncime 3
// și folosește 3 celule auxiliare.
//
// s = x xor y xor z
// c_out = x·y + y·z + z·x sau, ca să refolosim celulele, x·y + (x xor y)·z
void compress_32(int x, int y, int z, int* s, int* c_out) {
  int x_xor_y = new_cell(OP_XOR, x, y);
  *s = new_cell(OP_XOR, x_xor_y, z);
  int x_and_y = new_cell(OP_AND, x, y);
  int x_xor_y_and_z = new_cell(OP_AND, x_xor_y, z);
  *c_out = new_cell(OP_OR, x_and_y, x_xor_y_and_z);
}

void compress_column(int c) {
  while (mat[c].size() >= 3) {
    int x = mat[c].back(); mat[c].pop_back();
    int y = mat[c].back(); mat[c].pop_back();
    int z = mat[c].back(); mat[c].pop_back();
    int s, c_out;
    compress_32(x, y, z, &s, &c_out);
    new_mat[c].push_back(s);
    new_mat[c + 1].push_back(c_out);
  }

  for (int x: mat[c]) {
    new_mat[c].push_back(x);
  }
  mat[c].clear();
}

void pad_columns_to_length_2() {
  for (int c = 0; c < k; c++) {
    if (mat[c].size() == 1) {
      mat[c].push_back(pos_zero);
    }
  }
```

```
}

// Comprimă cîte 3 biți folosind un compresor 3:2 pînă cînd rămînem cu 3 biți
// pe fiecare coloană.
void compress_to_two_numbers() {
  bool three_or_more;
  do {
    three_or_more = false;
    for (int c = 0; c < k; c++) {
      compress_column(c);
      three_or_more |= (new_mat[c].size() >= 3);
    }
    if (new_mat[k].size()) {
      k++;
    }
    for (int c = 0; c < k; c++) {
      mat[c].swap(new_mat[c]);
    }
  } while (three_or_more);

  pad_columns_to_length_2();
}

// (a.g, a.p) ∘ (b.g, b.p) = (b_g + a_g · b_p, a_p · b_p)
// În cuvinte: Intervalul a ∪ b generează transport dacă b generează transport
// sau dacă a generează transport, iar b îl propagă. Intervalul a ∪ b propagă
// transportul dacă și a și b îl propagă.
gen_prop combine(gen_prop a, gen_prop b) {
  gen_prop res;
  int a_gen_b_prop = new_cell(OP_AND, a.g, b.p);
  res.g = new_cell(OP_OR, b.g, a_gen_b_prop);
  res.p = new_cell(OP_AND, a.p, b.p);
  return res;
}

void up_sweep() {
  for (int i = 0; i < k; i++) {
    GP[0][i].g = new_cell(OP_AND, mat[i][0], mat[i][1]);
    GP[0][i].p = new_cell(OP_XOR, mat[i][0], mat[i][1]);
  }

  last_gp_row = 0;
  for (int p = 1, len = 1; len < k; p++, len <<= 1) {
    last_gp_row = p;
    for (int i = 0; i < len; i++) {
      GP[p][i] = GP[p - 1][i];
    }
    for (int i = len; i < k; i++) {
      GP[p][i] = combine(GP[p - 1][i - len], GP[p - 1][i]);
    }
```

```
  }
}

void compute_sum() {
  gen_prop* gp = GP[last_gp_row];

  new_cell(OP_XOR, GP[0][0].p, pos_zero);
  for (int i = 1; i < k; i++) {
    new_cell(OP_XOR, GP[0][i].p, gp[i - 1].g);
  }
  new_cell(OP_XOR, gp[k - 1].g, pos_zero);

  for (int i = k + 1; i < expected_digits; i++) {
    new_cell(OP_XOR, pos_zero, pos_zero);
  }
}

// Adună două numere cu un sumator Kogge-Stone.
void add_numbers() {
  up_sweep();
  compute_sum();
}

struct fast_output {
  static const int BUFFER_SIZE = 128 * 1024;
  char num[MAX_CELLS][8];
  char buf[BUFFER_SIZE];
  int ptr;

  void init(int max_val) {
    num[0][0] = '\0';
    for (int i = 1; i <= max_val; i++) {
      strcpy(num[i], num[i / 10]);
      int l = strlen(num[i]);
      num[i][l] = '0' + i % 10;
      num[i][l + 1] = '\0';
    }
    num[0][0] = '0';
    num[0][1] = '\0';

    ptr = 0;
  }

  void flush() {
    fwrite(buf, 1, ptr, stdout);
    ptr = 0;
  }

  void put_char(char c) {
    buf[ptr++] = c;
```

```
    if (ptr == BUFFER_SIZE) {
      flush();
    }
  }

  void put_int(int x) {
    put_string(num[x]);
  }

  void put_string(const char* s) {
    for (const char* c = s; *c; c++) {
      put_char(*c);
    }
  }
};

fast_output out;

void write_circuit() {
  out.init(num_cells + n * orig_k);

  out.put_int(num_cells);
  out.put_char('\n');
  for (int i = 0; i < num_cells; i++) {
    out.put_string(OP_STRINGS[cells[i].op]);
    out.put_char(' ');
    out.put_int(cells[i].pos1);
    out.put_char(' ');
    out.put_int(cells[i].pos2);
    out.put_char('\n');
  }

  out.flush();
}

int main() {
  read_data();
  init_bit_matrix();
  init_cells();
  compress_to_two_numbers();
  add_numbers();
  write_circuit();

  return 0;
}
```

40.11 Problema Lungcircuit (Lot 2025)

‹ înapoi

Sursă tractor (versiune online).

```cpp
// Complexitate: O((N + Q) log Q).
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MAX_Q = 1'000'000;
const int MAX_LOG = 20;
const bool LEFT = false;
const bool RIGHT = true;

// moment[i] reține informații despre situația înainte de procesarea mutării
// #i (0 ≤ i < q).
struct moment {
  // mutarea (LEFT/RIGHT)
  bool move;

  // A cîta mutare de tipul său (R/A) este aceasta, numărînd de la sfîrșit?
  // Dacă mutarea este R, garantează că occurrence[LEFT][ord] = acest moment.
  // Similar pentru A.
  int ord;

  // Momentul următoarei mutări opuse (R dacă mutarea curentă este A și
  // viceversa).
  int next_opp;

  // Considerînd bila aflată în nodul de sus, la ce moment va încheia ea
  // ultima tură completă și va reveni în nodul de sus? Stochează chiar
  // momentul curent dacă bila nu poate descrie nicio tură completă.
  int full_laps;
};

int n, k, d, q;
moment m[MAX_Q + 1];
int sol[2][MAX_N];

// pozițiile mutărilor R/A, descrescător de la q-1 la 0.
int occurrence[2][MAX_Q];

// st[i][t] = momentul la care ajungem dacă parcurgem 2^i intervale începînd
// de la momentul t.
int st[MAX_LOG][MAX_Q + 1];

int min(int x, int y) {
  return (x < y) ? x : y;
}
```

```
void read_data() {
  scanf("%d %d %d %d ", &n, &k, &d, &q);
  for (int i = 0; i < q; i++) {
    m[i].move = (getchar() == 'R') ? LEFT : RIGHT;
  }
}

void make_occurrences() {
  m[q].next_opp = q;
  int next[2] = { q, q };
  int ptr[2] = { 0, 0 };

  for (int i = q - 1; i >= 0; i--) {
    bool side = m[i].move;
    m[i].ord = ptr[side];
    m[i].next_opp = next[!side];
    occurrence[side][ptr[side]++] = i;
    next[side] = i;
  }
}

void make_sparse_table() {
  for (int t = 0; t < q; t++) {
    if (m[t].ord < d - 1) {
      st[0][t] = 0;
    } else {
      bool side = m[t].move;
      st[0][t] = 1 + occurrence[side][m[t].ord - d + 1];
    }
  }
  st[0][q] = 0;

  for (int i = 1; i < MAX_LOG; i++) {
    for (int t = 0; t < q; t++) {
      int mid = st[i - 1][t];
      st[i][t] = mid ? st[i - 1][mid] : 0;
    }
  }
}

// Informații despre o bilă aflată pe circuitul side la poziția pos după
// primele time mutări.
struct state {
  int time;
  bool side;
  int pos;

  bool is_node() {
    return (pos <= (k - 1) * d) && !(pos % d);
```

```
}

bool is_between_nodes() {
  return (pos < (k - 1) * d) && (pos % d);
}

// Face cel mult atîţia paşi pe o porţiune care poate începe dintr-un nod,
// dar nu conţine alte noduri.
void move_at_most(int steps) {
  if (is_node()) {
    // Nodurile sînt pe ambele circuite. Partea pe care o avem notată nu
    // este relevantă. Alege o parte în funcţie de mutarea dată.
    side = m[time].move;
  } else if (side != m[time].move) {
    time = m[time].next_opp; // Sari peste mutările circuitului opus.
  }

  if (time < q) {
    steps = min(steps, m[time].ord + 1);
    pos = (pos + steps) % n;
    int last_ord = m[time].ord - (steps - 1);
    time = 1 + occurrence[side][last_ord];
  }
}

void move_to_next_node() {
  if (is_between_nodes()) {
    move_at_most(d - pos % d);
  }
}

void move_to_last_node() {
  if (is_node()) {
    int intervals = (k - 1) - (pos / d);
    for (int i = MAX_LOG - 1; i >= 0; i--) {
      if (((1 << i) <= intervals) && st[i][time]) {
        time = st[i][time];
        pos += d * (1 << i);
        intervals -= (1 << i);
      }
    }
  }
}

void move_to_first_node() {
  if (pos >= (k - 1) * d) {
    move_at_most(n - pos);
  }
}
```

```
  void move_full_laps() {
    if (pos == 0) {
      time = m[time].full_laps;
    }
  }

  void mark_solution(int ball_number) {
    sol[side][pos] = ball_number;
    if (is_node()) {
      sol[!side][pos] = ball_number;
    }
  }

  void compute_trajectory(int ball_number) {
    move_to_next_node();
    move_to_last_node();
    move_to_first_node();
    move_full_laps();
    move_to_last_node();
    move_at_most(q); // Știm că restul drumului nu va mai întîlni noduri.
    mark_solution(ball_number);
  }
};

void compute_full_laps() {
  m[q].full_laps = q;

  for (int t = q - 1; t >= 0; t--) {
    state s = { t, LEFT, 0 };
    s.move_to_last_node();
    s.move_to_first_node();

    m[t].full_laps = (s.pos == 0) && (s.time > t)
      ? m[s.time].full_laps
      : t;      // Odată plecați din t, nu mai revenim niciodată.
  }
}

void compute_trajectories() {
  // În stînga, bilele [0, n) stau pe pozițiile [0, n).
  for (int i = 0; i < n; i++) {
    state s = { 0, LEFT, i };
    s.compute_trajectory(i);
  }

  // În dreapta, bilele [n, 2n-k) sar peste noduri.
  int cnt = n;
  for (int i = 0; i < n; i++) {
    state s = { 0, RIGHT, i };
    if (!s.is_node()) {
```

```
      s.compute_trajectory(cnt++);
    }
  }
}

void write_solution() {
  long long hash_left = 0, hash_right = 0;
  for (long long i = 0; i < n; i++) {
    hash_left += i * sol[LEFT][i];
    hash_right += i * sol[RIGHT][i];
  }

  printf("%lld\n", hash_left);
  printf("%lld\n", hash_right);
}

int main() {
  read_data();
  make_occurrences();
  make_sparse_table();
  compute_full_laps();
  compute_trajectories();
  write_solution();

  return 0;
}
```

Sursă cu liste (versiune online).

```
// Complexitate: O(N + Q).
#include <stdio.h>

const int MAX_N = 1'000'000;
const int MAX_Q = 1'000'000;
const int MAX_DEQUES = MAX_N / 2; // deoarece K >= 2

char q[MAX_Q + 1];
int n, k, d, num_outer, num_queries;

int buf[2 * MAX_N];
int buf_ptr; // primul element disponibil în buf[]

// Un vector circular de exact end-start întregi, alocat în buf[].
struct deque {
  int start;
  int end;
  int head; // poziția primului element

  // Inițializează vectorul cu progresia first + i*step (size termeni).
```

```
  void init_progression(int size, int first, int step) {
    start = head = buf_ptr;
    buf_ptr += size;
    end = buf_ptr;

    for (int i = 0; i < size; i++) {
      buf[start + i] = first + i * step;
    }
  }

  // Adaugă x la început. Șterge și returnează ultimul element.
  int shift(int x) {
    head = (head == start) ? (end - 1) : (head - 1);
    int result = buf[head];
    buf[head] = x;
    return result;
  }

  int get(int pos) {
    pos += head;
    if (pos >= end) {
      pos -= (end - start);
    }
    return buf[pos];
  }
};

struct circuit {
  deque mesh[MAX_DEQUES];
  int mesh_head; // indicele primei liste din rețea
  int last_node;
  deque outer;

  void rotate(circuit& other) {
    // Rotește rețeaua.
    mesh_head = mesh_head ? (mesh_head - 1) : (d - 1);

    // Mută ultimul nod la începutul lanțului exterior. Obține de la acesta
    // noul prim nod.
    int first_node = outer.shift(last_node);

    // Mută primul nod la începutul primei liste din rețea. Obține de la
    // aceasta noul ultim nod.
    last_node = mesh[mesh_head].shift(first_node);

    // Notifică celălalt circuit că prima sa listă este totuna cu prima
    // noastră listă.
    other.mesh[other.mesh_head] = mesh[mesh_head];
    other.last_node = last_node;
  }
```

```
  // Enumeră conținutul: listele din rețea interclasate, apoi lanțul exterior.
  long long hash_code() {
    long long result = 0, pos = 0;

    int h = mesh_head;
    for (int i = 0; i < k - 1; i++) {
      for (int j = 0; j < d; j++) {
        result += pos++ * mesh[h].get(i);
        h = (h + 1) % d;
      }
    }
    result += pos++ * last_node;

    for (int i = 0; i < num_outer; i++) {
      result += pos++ * outer.get(i);
    }

    return result;
  }
};

circuit a, b;

void make_circuits() {
  scanf("%d %d %d %d %s", &n, &k, &d, &num_queries, q);

  num_outer = n - (k - 1) * d - 1;

  for (int i = 0; i < d; i++) {
    a.mesh[i].init_progression(k - 1, i, d);
  }
  a.last_node = (k - 1) * d;
  a.outer.init_progression(num_outer, (k - 1) * d + 1, 1);

  b.mesh[0] = a.mesh[0];
  for (int i = 1; i < d; i++) {
    b.mesh[i].init_progression(k - 1, n + i - 1, d - 1);
  }
  b.last_node = a.last_node;
  b.outer.init_progression(num_outer, n + (k - 1) * (d - 1), 1);
}

void make_moves() {
  for (int i = 0; i < num_queries; i++) {
    if (q[i] == 'R') {
      a.rotate(b);
    } else {
      b.rotate(a);
    }
```

```
  }
}

void write_hash_codes() {
  printf("%lld\n", a.hash_code());
  printf("%lld\n", b.hash_code());
}

int main() {
  make_circuits();
  make_moves();
  write_hash_codes();

  return 0;
}
```

40.12 Problema Cal-cule (Lot 2025)

înapoi

Sursă lentă (versiune online).

```
// Soluție neoptimă. Folosește doar valorile 0 și 1 indiferent de k. Poate
// ajunge la 8.000 de interogări.
#include "calcule.h"

#include <algorithm>
#include <vector>

// Returnează 1 (true) dacă și numai dacă există elemente duplicate.
int dreseaza_cal(int n, int k, std::vector<int> frecvente_sortate) {
  return frecvente_sortate.back() == 2;
}

// Pune o întrebare despre pos[0...len] + indicele extra.
bool make_query(int n, std::vector<int> pos, int len, int extra) {
  std::vector<int> query = std::vector<int>(pos.begin(), pos.begin() + len + 1);
  query.push_back(extra);
  return intreaba_cal(query);
}

// Caută binar valoarea de la poziția ind printre primele apariții ale
// elementelor cunoscute.
int find_previous_occurrence_of(int ind, int n, std::vector<int>& pos) {
  int lo = -1, hi = pos.size(); // (lo, hi]
  while (hi - lo > 1) {
    int mid = (lo + hi) / 2;
    int has_duplicates = make_query(n, pos, mid, ind);
```

```
    if (has_duplicates) {
      hi = mid;
    } else {
      lo = mid;
    }
  }

  return hi;
}

void scan_input(int n, std::vector<int>& pos, std::vector<int>& freq) {
  // Află dacă valoarea input[i] este nouă sau dacă am întîlnit-o anterior, și
  // dacă da, pe ce poziție.
  for (int i = 1; i < n; i++) {
    int prev = find_previous_occurrence_of(i, n, pos);
    if (prev < (int)pos.size()) {
      freq[prev]++;
    } else {
      pos.push_back(i);
      freq.push_back(1);
    }
  }
}

std::vector<int> descuie_grajdul(int n, int k) {
  // Nu cunoaștem valorile din vector, așa că le vom boteza 0, 1, 2, ... în
  // ordinea primei lor apariții. De exemplu, vom boteza „0” primul element.
  std::vector<int> pos;  // poziția primei apariții
  std::vector<int> freq; // frecvența

  pos.push_back(0);
  freq.push_back(1);

  scan_input(n, pos, freq);

  std::sort(freq.begin(), freq.end());
  return freq;
}
```

Sursă eficientă (versiune online).

```
#include "calcule.h"

#include <vector>

typedef std::vector<int> vector;

const int MAX_N = 1'000;
```

```
// Returnează numărul de perechi egale, modulo k.
int dreseaza_cal(int n, int k, vector frecvente_sortate) {
  int s = 0;
  for (int f: frecvente_sortate) {
    s += f * (f - 1) / 2;
  }
  return s % k;
}

struct freq_info {
  int cnt;    // Câte elemente au această frecvență?
  vector pos; // La ce poziții a atins vreun element această frecvență?
};

struct range {
  int lo_f, hi_f;
};

// O structură care menține situația frecvențelor. Funcția extend() face
// interogările necesare pentru a încorpora încă un element.
struct guesser {
  int n, k;
  freq_info freq[MAX_N + 1];
  range rng[MAX_N];
  int lo_f, hi_f;     // frecvențele-candidat pe durata lui extend()
  int pairs_in_query; // perechi cunoscute printre pozițiile puse în interogare

  guesser(int n, int k) {
    this->n = n;
    this->k = k;
    init_freq();
  }

  void init_freq() {
    for (int i = 0; i < n; i++) {
      freq[i].cnt = 0;
    }
    freq[0].cnt = n - 1; // pozițiile nu ne trebuie
    freq[1].cnt = 1;
    freq[1].pos.push_back(0);
  }

  int count_distinct_freqs() {
    int cnt = 0;
    for (int f = lo_f; f <= hi_f; f++) {
      cnt += (freq[f].cnt > 0);
    }
    return cnt;
  }
```

```
void add_to_query(int f, int freq_in_query, vector& dest) {
  if (freq_in_query) {
    for (int p: freq[f].pos) {
      dest.push_back(p);
      pairs_in_query += freq_in_query - 1;
    }
  }
}

int find_next_nonzero_freq(int f) {
  do {
    f++;
  } while ((f <= n) && !freq[f].cnt);
  return f;
}

vector build_query(int curr_pos) {
  vector result;

  pairs_in_query = 0;
  int num_dist = count_distinct_freqs();
  int f = lo_f;
  for (int rem = 0; rem < k; rem++) {
    int batch_size = num_dist / k + (rem < num_dist % k);
    if (batch_size) {
      add_to_query(f, rem, result);
      rng[rem].lo_f = f;
      while (batch_size) {
        rng[rem].hi_f = f;
        f = find_next_nonzero_freq(f);
        batch_size--;
      }
    }
  }

  result.push_back(curr_pos);
  return result;
}

// Ținînd cont de răspunsul calului și de numărul cunoscut de perechi,
// restrînge intervalul de frecvențe-candidat.
void interpret_response(int resp) {
  int growth = (resp + k - (pairs_in_query % k)) % k;
  lo_f = rng[growth].lo_f;
  hi_f = rng[growth].hi_f;
}

void update_frequencies(int curr_pos) {
  freq[lo_f].cnt--;
  freq[lo_f + 1].cnt++;
```

```
    freq[lo_f + 1].pos.push_back(curr_pos);
  }

  void extend(int curr_pos) {
    lo_f = 0;
    hi_f = n;

    while (lo_f != hi_f) {
      vector query = build_query(curr_pos);
      int resp = intreaba_cal(query);
      interpret_response(resp);
    }

    update_frequencies(curr_pos);
  }

  vector get_sorted_frequencies() {
    vector result;
    for (int i = 1; i < n; i++) {
      for (int j = 0; j < freq[i].cnt; j++) {
        result.push_back(i);
      }
    }
    return result;
  }
};

vector descuie_grajdul(int n, int k) {
  guesser g(n, k);

  for (int i = 1; i < n; i++) {
    g.extend(i);
  }

  return g.get_sorted_frequencies();
}
```

40.13 Problema 6 de Pentagrame (Lot 2024)

‹ înapoi • versiune online

```
// Complexitate: O(n log n).
#include <algorithm>
#include <stdio.h>

typedef long long i64;

const int MAX_POINTS = 200'000;
```

```
const int MAX_P2 = 1 << 18;
const i64 INF = 1e18;

struct point {
  int x, y, val;
};

point p[MAX_POINTS];
// Greutățile totale pentru fiecare y la stînga și la dreapta x-ului curent.
i64 real_l[MAX_POINTS + 1], real_r[MAX_POINTS + 1];
int n;

i64 min(i64 x, i64 y) {
  return (x < y) ? x : y;
}

i64 max(i64 x, i64 y) {
  return (x > y) ? x : y;
}

i64 min4(i64 a, i64 b, i64 c, i64 d) {
  return min(min(a, b), min(c, d));
}

i64 max4(i64 a, i64 b, i64 c, i64 d) {
  return max(max(a, b), max(c, d));
}

// Stochează sumele celor patru cadrane pentru o coordonată y dată, pentru
// x-ul curent.
struct split {
  i64 tl, bl, tr, br; // top-left, bottom-left, top-right, bottom-right
  int y;

  void advance(i64 dl, i64 dr, int delta_y) {
    tl -= dl;
    bl += dl;
    tr -= dr;
    br += dr;
    y += delta_y;
  }

  void advance() {
    y++;
    tl -= real_l[y];
    bl += real_l[y];
    tr -= real_r[y];
    br += real_r[y];
  }
```

```
  i64 get_cost() {
    return max4(tl, tr, bl, br) - min4(tl, tr, bl, br);
  }
};

struct fenwick_tree {
  i64 l[MAX_POINTS + 1], r[MAX_POINTS + 1], total_l, total_r;
  int n, max_p2;

  void init(int n) {
    this->n = max_p2 = n;

    while (max_p2 & (max_p2 - 1)) {
      max_p2 &= max_p2 - 1;
    }
  }

  void add(i64* v, int pos, int val) {
    do {
      v[pos] += val;
      pos += pos & -pos;
    } while (pos <= n);
  }

  void add_right(int pos, int val) {
    add(r, pos, val);
    total_r += val;
  }

  void move_left(int pos, int val) {
    add(l, pos, val);
    total_l += val;
    add(r, pos, -val);
    total_r -= val;
  }

  // Căutare binară pentru o funcție treaptă.
  split find_overtake_pos(bool min_ascend) {
    split s = {
      .tl = total_l, .bl = 0,
      .tr = total_r, .br = 0,
      .y = 0,
    };

    for (int interval = max_p2; interval; interval >>= 1) {
      if (s.y + interval <= n) {
        i64 dl = l[s.y + interval], dr = r[s.y + interval];
        bool cond =
          (min_ascend && (min(s.bl + dl, s.br + dr) < min(s.tl - dl, s.tr - dr))) ||
          (!min_ascend && (max(s.tl - dl, s.tr - dr) > max(s.bl + dl, s.br + dr)));
```

```
        if (cond) {
          s.advance(dl, dr, interval);
        }
      }
    }

    return s;
  }

  // Returnează cel mai mare y a.î. minimul este bl sau br.
  split find_min_ascend_pos() {
    return find_overtake_pos(true);
  }

  // Returnează cel mai mare y a.î. maximul este tl sau tr.
  split find_max_descend_pos() {
    return find_overtake_pos(false);
  }
};

// Un arbore de intervale cu suport pentru operațiile:
//
// 1. Add(L, R, X): adaugă X la fiecare poziție din [L, R].
// 2. Min(L, R): returnează valoarea minimă din [L, R].
// 3. Max(L, R): returnează valoarea maximă din [L, R].
//
// Contract: lazy trebuie adăugat la mi și la ma pe întregul subarbore.
struct segment_tree {
  i64 mi[2 * MAX_P2], ma[2 * MAX_P2]; // I love my Meemaw.
  i64 lazy[2 * MAX_P2];
  int n;

  void init(int n) {
    while (n & (n - 1)) {
      n += (n & -n);
    }
    this->n = n;
  }

  void push(int node) {
    mi[node] += lazy[node];
    ma[node] += lazy[node];
    lazy[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }

  void pull(int node) {
    mi[node] = min(mi[2 * node] + lazy[2 * node],
                   mi[2 * node + 1] + lazy[2 * node + 1]);
```

```
    ma[node] = max(ma[2 * node] + lazy[2 * node],
                   ma[2 * node + 1] + lazy[2 * node + 1]);
  }

  // [L, R)
  void update(int node, int pl, int pr, int l, int r, int delta) {
    if (l >= r) {
      return;
    } else if ((l == pl) && (r == pr)) {
      lazy[node] += delta;
    } else {
      push(node);
      int mid = (pl + pr) >> 1;
      update(2 * node, pl, mid, l, min(r, mid), delta);
      update(2 * node + 1, mid, pr, max(l, mid), r, delta);
      pull(node);
    }
  }

  // [L, R]
  void update(int l, int r, int delta) {
    update(1, 0, n, l, r + 1, delta);
  }

  i64 query(int node, int pl, int pr, int l, int r, bool is_min) {
    if (l >= r) {
      return is_min ? INF : -INF;
    } else if ((l == pl) && (r == pr)) {
      return lazy[node] + (is_min ? mi[node] : ma[node]);
    } else {
      push(node);
      int mid = (pl + pr) >> 1;
      i64 left = query(2 * node, pl, mid, l, min(r, mid), is_min);
      i64 right = query(2 * node + 1, mid, pr, max(l, mid), r, is_min);
      return is_min ? min(left, right) : max(left, right);
    }
  }

  i64 min_query(int l, int r) {
    return query(1, 0, n, l, r + 1, true);
  }

  i64 max_query(int l, int r) {
    return query(1, 0, n, l, r + 1, false);
  }
};

fenwick_tree fen;
segment_tree st_top, st_bottom;
```

```
void read_data() {
  scanf("%d", &n);
  for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &p[i].x, &p[i].y, &p[i].val);
  }
}

void sort_by_x() {
  std::sort(p, p + n, [] (point a, point b) {
    return a.x < b.x;
  });
}

void find_and_print_best() {
  split s1 = fen.find_min_ascend_pos();
  split s2 = fen.find_max_descend_pos();
  i64 cost;

  if (s1.y < s2.y) {
    // Ambele extreme sus.
    s2.advance();
    cost = min(s1.get_cost(), s2.get_cost());
    s1.advance();
    if (s1.tl < s1.tr) {
      // TL este min, TR este max
      cost = min(cost, -st_top.max_query(s1.y, s2.y - 1));
    } else {
      // TL este max, TR este min
      cost = min(cost, st_top.min_query(s1.y, s2.y - 1));
    }
  } else {
    // Ambele extreme jos.
    s1.advance();
    cost = min(s2.get_cost(), s1.get_cost());
    s2.advance();
    if (s2.bl < s2.br) {
      // BL este min, BR este max
      cost = min(cost, -st_bottom.max_query(s2.y, s1.y - 1));
    } else {
      // BL este max, BR este min
      cost = min(cost, st_bottom.min_query(s2.y, s1.y - 1));
    }
  }

  printf("%lld\n", cost);
}

void scan_points() {
  // TODO: Le putem construi în O(n).
  fen.init(n);
```

```
  st_top.init(n + 1);
  st_bottom.init(n + 1);
  for (int i = 0; i < n; i++) {
    fen.add_right(p[i].y, p[i].val);
    real_r[p[i].y] += p[i].val;
    st_top.update(0, p[i].y - 1, -p[i].val);
    st_bottom.update(p[i].y, n, -p[i].val);
  }

  int i = 0;
  for (int x = 1; x < n; x++) {
    while ((i < n) && (p[i].x <= x)) {
      fen.move_left(p[i].y, p[i].val);
      real_r[p[i].y] -= p[i].val;
      real_l[p[i].y] += p[i].val;
      st_top.update(0, p[i].y - 1, 2 * p[i].val);
      st_bottom.update(p[i].y, n, 2 * p[i].val);
      i++;
    }
    find_and_print_best();
  }
}

int main() {
  read_data();
  sort_by_x();
  scan_points();

  return 0;
}
```